Java Source Code Compilation

Total Files: 833

---

1. `_1.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1. Two Sum
 *
 * Given an array of integers, return indices of the two numbers such that they add up to
a specific target.
 * You may assume that each input would have exactly one solution, and you may not use
the same element twice.
 *
 * Example:
 * Given nums = [2, 7, 11, 15], target = 9,
 * Because nums[0] + nums[1] = 2 + 7 = 9,
 * return [0, 1].
 */
public class _1 {

    public static class Solution1 {
        public int[] twoSum(int[] nums, int target) {
            Map<Integer, Integer> map = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(target - nums[i])) {
                    return new int[]{map.get(target - nums[i]), i};
                } else {
                    map.put(nums[i], i);
                }
            }
            return new int[]{-1, -1};
        }
    }

}
```

2. `_2.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 2. Add Two Numbers

 You are given two linked lists representing two non-negative numbers.
 The digits are stored in reverse order and each of their nodes contain a single digit.
 Add the two numbers and return it as a linked list.

Input: (2 -> 4 -> 3) + (5 -> 6 -> 4)
Output: 7 -> 0 -> 8
 */

public class _2 {
    public static class Solution1 {
        public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
            ListNode result = new ListNode(0);
            ListNode tmp = result;
            int sum = 0;
            while (l1 != null || l2 != null) {
                sum /= 10;
                if (l1 != null) {
                    sum += l1.val;
                    l1 = l1.next;
                }
                if (l2 != null) {
                    sum += l2.val;
                    l2 = l2.next;
                }
                tmp.next = new ListNode(sum % 10);
                tmp = tmp.next;
            }
            if (sum / 10 == 1) {
                tmp.next = new ListNode(1);//this means there's a carry, so we add
additional 1, e.g. [5] + [5] = [0, 1]
            }
            return result.val == 0 ? result.next : result;
        }
    }

}
```

3. `_3.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 3. Longest Substring Without Repeating Characters
 *
 * Given a string, find the length of the longest substring without repeating characters.

 Examples:

 Given "abcabcbb", the answer is "abc", which the length is 3.

 Given "bbbbb", the answer is "b", with the length of 1.

 Given "pwwkew", the answer is "wke", with the length of 3.

 Note that the answer must be a substring, "pwke" is a subsequence and not a substring.
 */

public class _3 {

    public static class Solution1 {
        public int lengthOfLongestSubstring(String s) {
            int result = 0;
            Map<Character, Integer> map = new HashMap();
            for (int i = 0, j = i; j < s.length(); ) {
                if (!map.containsKey(s.charAt(j)) || (map.containsKey(s.charAt(j)) &&
map.get(s.charAt(j)) == 0)) {
                    map.put(s.charAt(j), 1);
                    result = Math.max(j - i + 1, result);
                    j++;
                } else {
                    map.put(s.charAt(i), map.get(s.charAt(i)) - 1);
                    i++;
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * Sliding Window
         * O(n) time
         * O(min(m,n)) or O(k) space
         */
        public int lengthOfLongestSubstring(String s) {
```

```
        int n = s.length();
        Set<Character> set = new HashSet<>();
        int result = 0;
        int i = 0;
        int j = 0;
        while (i < n && j < n) {
            /**Try to extend the range i, j*/
            if (!set.contains(s.charAt(j))) {
                set.add(s.charAt(j++));
                result = Math.max(result, j - i);
            } else {
                set.remove(s.charAt(i++));
            }
        }
        return result;
    }
}

public static class Solution3 {
    /**
     * Sliding Window
     * O(n) time
     * O(n) space
     */
    public int lengthOfLongestSubstring(String s) {
        if (s.length() == 0) {
            return 0;
        }
        int max = 0;
        Map<Character, Integer> map = new HashMap<>();
        /**Try to extend the range (i, j)*/
        for (int i = 0, j = 0; i < s.length(); i++) {
            if (map.containsKey(s.charAt(i))) {
                j = Math.max(j, map.get(s.charAt(i)) + 1);
            }
            map.put(s.charAt(i), i);
            max = Math.max(max, i + 1 - j);
        }
        return max;
    }
}

public static class Solution4 {
    /**
     * Sliding Window Optimized
     * O(n) time
     * O(n) space
     */
    public int lengthOfLongestSubstring(String s) {
        if (s.length() == 0) {
            return 0;
        }
        int max = 0;
        int[] index = new int[128];
```

```
            /**Try to extend the range (i, j)*/
            for (int i = 0, j = 0; j < s.length(); j++) {
                i = Math.max(index[s.charAt(j)], i);
                max = Math.max(max, j - i + 1);
                index[s.charAt(j)] = j + 1;
            }
            return max;
        }
    }
}
```

4. `_4.java`

```
package com.fishercoder.solutions;

import static java.lang.Math.max;
import static java.lang.Math.min;

/**
 4. Median of Two Sorted Arrays

 There are two sorted arrays nums1 and nums2 of size m and n respectively.

 Find the median of the two sorted arrays. The overall run time complexity should be
O(log (m+n)).

 Example 1:
 nums1 = [1, 3]
 nums2 = [2]
 The median is 2.0

 Example 2:
 nums1 = [1, 2]
 nums2 = [3, 4]
 The median is (2 + 3)/2 = 2.5
 */
public class _4 {

    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/28602/concise-java-solution-based-
on-binary-search
         *
         * The key point of this problem is to ignore half part of A and B each step
recursively by comparing the median of remaining A and B:

         if (aMid < bMid) Keep [aRight + bLeft]
         else Keep [bRight + aLeft]

         As the following: time=O(log(m + n))
         */
        public double findMedianSortedArrays(int[] A, int[] B) {
            int m = A.length;
            int n = B.length;
            int l = (m + n + 1) / 2;
            int r = (m + n + 2) / 2;
            return (getkth(A, 0, B, 0, l) + getkth(A, 0, B, 0, r)) / 2.0;
        }

        public double getkth(int[] A, int aStart, int[] B, int bStart, int k) {
            if (aStart > A.length - 1) {
                return B[bStart + k - 1];
            }
            if (bStart > B.length - 1) {
                return A[aStart + k - 1];
```

```
            }
            if (k == 1) {
                return Math.min(A[aStart], B[bStart]);
            }

            int aMid = Integer.MAX_VALUE;
            int bMid = Integer.MAX_VALUE;
            if (aStart + k / 2 - 1 < A.length) {
                aMid = A[aStart + k / 2 - 1];
            }
            if (bStart + k / 2 - 1 < B.length) {
                bMid = B[bStart + k / 2 - 1];
            }

            if (aMid < bMid) {
                return getkth(A, aStart + k / 2, B, bStart, k - k / 2);// Check: aRight +
bLeft
            } else {
                return getkth(A, aStart, B, bStart + k / 2, k - k / 2);// Check: bRight +
aLeft
            }
        }
    }

    public static class Solution2 {
        /**
         * Reference: https://leetcode.com/discuss/28843/my-accepted-java-solution:
         * Basic Idea is very similar to K-selection. it's easier to understand if you
imagine this to be chopping off the last K elements from a total of len(A) + len(B)
elements,
         * where K = (len(A) + len(B))/2.
         * we want to remove K, but each time we can remove only at most K/2 elements,
         * because we can only be sure that these elements are not within the first
(len(A) + len(B)) -K elements.
         */
        public double findMedianSortedArrays(int[] nums1, int[] nums2) {
            int K = nums1.length + nums2.length;
            if (K % 2 == 0) {
                return (findMedianSortedArrays(nums1, nums2, (K - K / 2)) +
findMedianSortedArrays(nums1, nums2, (K - (K / 2 + 1)))) / 2;
            } else {
                return findMedianSortedArrays(nums1, nums2, K - (K / 2 + 1));
            }
        }

        // k is the number of elements to REMOVE, or "Chop off"
        public double findMedianSortedArrays(int[] A, int[] B, int K) {

            int lowA = 0;
            int lowB = 0;
            int highA = A.length;
            int highB = B.length;
            int midA;
            int midB;
```

```
            while (K > 0 && highA > 0 && highB > 0) {
                int chopA = max(1, min(K / 2, (highA) / 2));
                int chopB = max(1, min(K / 2, (highB) / 2));

                midA = highA - chopA;
                midB = highB - chopB;
                if (A[midA] < B[midB]) { // here A[0 .. midA] < B[midB], and we know that
B[0 .. midB-1] < B[midB], so B[midB..highB] can not possibly be within the first (len(A)
+ len(B) - K) elements, and can be safely removed.
                    highB = midB;
                    K = K - chopB;
                } else {
                    highA = midA;
                    K = K - chopA;
                }
            }

            if (highA == 0 && highB == 0) {
                return 0;
            }
            if (highA == 0) {
                return B[highB - 1 - K];
            }
            if (highB == 0) {
                return A[highA - 1 - K];
            }
            return max(A[highA - 1], B[highB - 1]);
        }
    }
}
```

5. `_5.java`

```
package com.fishercoder.solutions;

/**
 * 5. Longest Palindromic Substring
 *
 * Given a string s, find the longest palindromic substring in s. You may assume that the
maximum length of s is 1000.

 Example:
 Input: "babad"
 Output: "bab"
 Note: "aba" is also a valid answer.

 Example:
 Input: "cbbd"
 Output: "bb"
 */
public class _5 {

    public static class Solution1 {
        private int low;
        private int maxLen;

        public String longestPalindrome(String s) {
            int len = s.length();
            if (len < 2) {
                return s;
            }

            for (int i = 0; i < len - 1; i++) {
                extendPalindrome(s, i, i); // assume odd length, try to extend Palindrome
as possible
                extendPalindrome(s, i, i + 1); // assume even length.
            }
            return s.substring(low, low + maxLen);
        }

        private void extendPalindrome(String s, int left, int right) {
            while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right))
{
                left--;
                right++;
            }
            if (maxLen < right - left - 1) {
                low = left + 1;
                maxLen = right - left - 1;
            }
        }
    }
}
```

6. `_6.java`

```
package com.fishercoder.solutions;

/**
 * 6. ZigZag Conversion
 *
 * The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows
like this:
 * (you may want to display this pattern in a fixed font for better legibility)

 P   A   H   N
 A P L S I I G
 Y   I   R

 And then read line by line: "PAHNAPLSIIGYIR"
 Write the code that will take a string and make this conversion given a number of rows:

 string convert(string text, int nRows);
 convert("PAYPALISHIRING", 3) should return "PAHNAPLSIIGYIR".
 */

public class _6 {
        public static class Solution1 {
                public String convert(String s, int numRows) {
                        StringBuilder[] sb = new StringBuilder[numRows];
                        char[] c = s.toCharArray();
                        int len = s.length();
                        for (int i = 0; i < numRows; i++) {
                                sb[i] = new StringBuilder();//this is an important step
to initialize it
                        }
                        int i = 0;
                        while (i < len) {
                                for (int index = 0; index < numRows && i < len; index++)
{
                                        sb[index].append(c[i++]);// vertically down
                                }

                                for (int index = numRows - 2; index >= 1 && i < len;
index--) {
                                        /**Why it should start from numRows - 2? Think of
the example when numRows = 3
                                         the starting point of obliquely going up is 1,
which is numRows-2.*/
                                        sb[index].append(c[i++]);// obliquely up
                                }
                        }

                        for (i = 1; i < numRows; i++) {
                                sb[0].append(sb[i]);
                        }
                        return sb[0].toString();
```

```
                }
        }

}
```

7. `_7.java`

```
package com.fishercoder.solutions;

/**
 * 7. Reverse Integer

 Given a 32-bit signed integer, reverse digits of an integer.

 Example 1:
 Input: 123
 Output: 321

 Example 2:
 Input: -123
 Output: -321

 Example 3:
 Input: 120
 Output: 21

 Note:
 Assume we are dealing with an environment which could only store integers within the 32-
bit signed integer range: [−231,  231 − 1].
 For the purpose of this problem, assume that your function returns 0 when the reversed
integer overflows.
 */
public class _7 {

    public static class Solution1 {
        /**This solution is NOT meeting the problem requirement actually:
         * Assume we are dealing with an environment which could only store integers
within the 32-bit signed integer range: [−231,  231 − 1].*/
        public int reverse(int x) {
            long result = 0;
            while (x != 0) {
                result = result * 10 + x % 10;
                x /= 10;
                System.out.println("result = " + result);
                if (result > Integer.MAX_VALUE || result < Integer.MIN_VALUE) {
                    System.out.println("break out..");
                    return 0;
                }
            }
            return (int) result;
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/problems/reverse-integer/discuss/4060/My-
accepted-15-lines-of-code-for-Java*/
        public int reverse(int x) {
            int result = 0;
```

```
            while (x != 0) {
                int lastDigit = x % 10;
                int newResult = result * 10 + lastDigit;
                if (result != (newResult - lastDigit) / 10) {
                    return 0;
                }
                x /= 10;
                result = newResult;
            }
            return result;
        }
    }
}
```

8. `_8.java`

```
package com.fishercoder.solutions;

/**
 * 8. String to Integer (atoi)
 *
 * Implement atoi to convert a string to an integer.
 *
 * Hint: Carefully consider all possible input cases. If you want a challenge, please do
not see below and ask yourself what are the possible input cases.
 * Notes: It is intended for this problem to be specified vaguely (ie, no given input
specs). You are responsible to gather all the input requirements up front.
 *
 * Requirements for atoi:
 * The function first discards as many whitespace characters as necessary until the first
non-whitespace character is found.
 * Then, starting from this character, takes an optional initial plus or minus sign
followed by as many numerical digits as possible, and interprets them as a numerical
value.
 * The string can contain additional characters after those that form the integral
number, which are ignored and have no effect on the behavior of this function.
 * If the first sequence of non-whitespace characters in str is not a valid integral
number,
 * or if no such sequence exists because either str is empty or it contains only
whitespace characters, no conversion is performed.
 * If no valid conversion could be performed, a zero value is returned. If the correct
value is out of the range of representable values, INT_MAX (2147483647) or INT_MIN
(-2147483648) is returned.*/

public class _8 {

        public static class Solution1 {
                /**
                 * four corner cases:
                 * 1. discards all leading zeroes
                 * 2. sign of the number
                 * 3. overflow
                 * 4. invalid input
                 * */
                public int myAtoi(String str) {
                        int pointer = 0;
                        int result = 0;
                        while (pointer < str.length() &&
Character.isWhitespace(str.charAt(pointer))) {
                                pointer++;
                        }
                        if (pointer == str.length()) {
                                return 0;
                        }
                        boolean negativeFlag = (str.charAt(pointer) == '-');
                        if (str.charAt(pointer) == '+' || str.charAt(pointer) == '-') {
                                pointer++;
```

```
                        }
                        for (; pointer < str.length(); pointer++) {
                                if (str.charAt(pointer) > '9' || str.charAt(pointer) <
'0') {
                                        break;
                                } else {
                                        int digit = str.charAt(pointer) - '0';
                                        if (!negativeFlag && result > (Integer.MAX_VALUE
- digit) / 10) {
                                                return Integer.MAX_VALUE;
                                        } else if (negativeFlag && result <
(Integer.MIN_VALUE + digit) / 10) {
                                                return Integer.MIN_VALUE;
                                        }
                                        result = result * 10 + (negativeFlag ? -digit :
digit);
                                }
                        }
                        return result;
                }
        }

}
```

9. `_9.java`

```
package com.fishercoder.solutions;

/**
 * 9. Palindrome Number
 *
 * Determine whether an integer is a palindrome. Do this without extra space.
 *
 * Some hints:
 *
 * Could negative integers be palindromes? (ie, -1)
 *
 * If you are thinking of converting the integer to string, note the restriction of using
extra space.
 *
 * You could also try reversing an integer. However, if you have solved the problem
"Reverse Integer", you know that the reversed integer might overflow. How would you
handle such case?
 *
 * There is a more generic way of solving this problem.
 */
public class _9 {

    /**credit: https://discuss.leetcode.com/topic/8090/9-line-accepted-java-code-without-
the-need-of-handling-overflow
     * reversing only half and then compare if they're equal.*/
    public static class Solution1 {
        public boolean isPalindrome(int x) {
            if (x < 0) {
                return false;
            } else if (x == 0) {
                return true;
            } else if (x % 10 == 0) {
                return false;
            }
            int reversed = 0;
            while (x > reversed) {
                int digit = x % 10;
                reversed *= 10;
                reversed += digit;
                x /= 10;
            }
            return (x == reversed || x == reversed / 10);
        }
    }

}
```

10. `_10.java`

```
package com.fishercoder.solutions;

/**
 * 10. Regular Expression Matching
 *
 * Implement regular expression matching with support for '.' and '*'.
 *
 * '.' Matches any single character.
 * '*' Matches zero or more of the preceding element.
 *
 * The matching should cover the entire input string (not partial).
 *
 * The function prototype should be:
 * bool isMatch(const char *s, const char *p)
 *
 * Some examples:
 * isMatch("aa","a") → false
 * isMatch("aa","aa") → true
 * isMatch("aaa","aa") → false
 * isMatch("aa", "a*") → true
 * isMatch("aa", ".*") → true
 * isMatch("ab", ".*") → true
 * isMatch("aab", "c*a*b") → true
 */

public class _10 {

    public static class Solution1 {
        public boolean isMatch(String s, String p) {
            if (s == null || p == null) {
                return false;
            }
            boolean[][] dp = new boolean[s.length() + 1][p.length() + 1];
            dp[0][0] = true;
            for (int i = 0; i < p.length(); i++) { //here's the p's length, not s's
                if (p.charAt(i) == '*' && dp[0][i - 1]) {
                    dp[0][i + 1] = true; //here's y axis should be i+1
                }
            }
            for (int i = 0; i < s.length(); i++) {
                for (int j = 0; j < p.length(); j++) {
                    if (p.charAt(j) == '.' || p.charAt(j) == s.charAt(i)) {
                        dp[i + 1][j + 1] = dp[i][j];
                    }
                    if (p.charAt(j) == '*') {
                        if (p.charAt(j - 1) != s.charAt(i) && p.charAt(j - 1) != '.') {
                            dp[i + 1][j + 1] = dp[i + 1][j - 1];
                        } else {
                            dp[i + 1][j + 1] = (dp[i + 1][j] || dp[i][j + 1] || dp[i + 1]
[j - 1]);
                        }
```

```
                    }
                }
            }
            return dp[s.length()][p.length()];
        }
    }

}
```

11. `_11.java`

```
package com.fishercoder.solutions;

/**
 * 11. Container With Most Water
 *
 * Given n non-negative integers a1, a2, ..., an,
 * where each represents a point at coordinate (i, ai).
 * n vertical lines are drawn such that the two endpoints of line i is at (i, ai) and (i,
0).
 * Find two lines, which together with x-axis forms a container, such that the container
contains the most water.

 Note: You may not slant the container and n is at least 2.
 */
public class _11 {

    public static class Solution1 {
        public int maxArea(int[] height) {
            int max = 0;
            int i = 0;
            int j = height.length - 1;
            while (i < j) {
                max = Math.max(Math.min(height[i], height[j]) * (j - i), max);
                if (height[i] <= height[j]) {
                    // we need to find the shorter one,
                    // then calculate its area
                    i++;
                } else {
                    j--;
                }
            }
            return max;
        }
    }

}
```

12. `_12.java`

```
package com.fishercoder.solutions;

/**
 * 12. Integer to Roman
 *
 * Given an integer, convert it to a roman numeral.
 * Input is guaranteed to be within the range from 1 to 3999.
 */
public class _12 {

    public static class Solution1 {
        public String intToRoman(int num) {
            String[] M = new String[]{"", "M", "MM", "MMM"};
            String[] C = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
            String[] X = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
            String[] I = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
            return M[num / 1000] + C[(num % 1000) / 100] + X[(num % 100) / 10] + I[num %
10];
        }
    }

}
```

13. `_13.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 13. Roman to Integer
 *
 * Given a roman numeral, convert it to an integer.
 * Input is guaranteed to be within the range from 1 to 3999.
 * */

public class _13 {

    public static class Solution1 {
        public int romanToInt(String s) {
            Map<Character, Integer> map = new HashMap();
            map.put('I', 1);
            map.put('V', 5);
            map.put('X', 10);
            map.put('L', 50);
            map.put('C', 100);
            map.put('D', 500);
            map.put('M', 1000);

            int result = 0;
            for (int i = 0; i < s.length(); i++) {
                if (i > 0 && map.get(s.charAt(i)) > map.get(s.charAt(i - 1))) {
                    result += map.get(s.charAt(i)) - 2 * map.get(s.charAt(i - 1));
                } else {
                    result += map.get(s.charAt(i));
                }
            }
            return result;
        }
    }

}
```

14. `_14.java`

```
package com.fishercoder.solutions;

/**
 * 14. Longest Common Prefix
 *
 * Write a function to find the longest common prefix string amongst an array of strings.
 *
 * If there is no common prefix, return an empty string "".
 *
 * Example 1:
 * Input: ["flower","flow","flight"]
 * Output: "fl"
 *
 * Example 2:
 * Input: ["dog","racecar","car"]
 * Output: ""
 * Explanation: There is no common prefix among the input strings.
 *
 * Note:
 * All given inputs are in lowercase letters a-z.
 */

public class _14 {

    public static class Solution1 {
        //horizontal scan
        public String longestCommonPrefix(String[] strs) {
            if (strs.length == 0) {
                return "";
            }
            String prefix = strs[0];
            for (int i = 1; i < strs.length; i++) {
                while (strs[i].indexOf(prefix) != 0) {
                    prefix = prefix.substring(0, prefix.length() - 1);
                    if (prefix.isEmpty()) {
                        return "";
                    }
                }
            }
            return prefix;
        }
    }

    public static class Solution2 {
        //vertical scan
        public String longestCommonPrefix(String[] strs) {
            if (strs.length == 0) {
                return "";
            }
            for (int i = 0; i < strs[0].length(); i++) {
                char c = strs[0].charAt(i);
```

```
                for (int j = 1; j < strs.length; j++) {
                    if (i == strs[j].length() || strs[j].charAt(i) != c) {
                        return strs[0].substring(0, i);
                    }
                }
            }
            return strs[0];
        }
    }
}
```

15. `_15.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 15. 3Sum
 *
 * Given an array S of n integers, are there elements a, b, c in S such that a + b + c =
0?
 * Find all unique triplets in the array which gives the sum of zero.
 *
 * Note: The solution set must not contain duplicate triplets.

 For example, given array S = [-1, 0, 1, 2, -1, -4],

 A solution set is:
 [
 [-1, 0, 1],
 [-1, -1, 2]
 ]
 */

public class _15 {

    public static class Solution1 {
        public List<List<Integer>> threeSum(int[] nums) {
            Arrays.sort(nums);

            List<List<Integer>> result = new ArrayList<>();
            for (int i = 0; i < nums.length - 2; i++) {
                if (i >= 1 && nums[i] == nums[i - 1]) {
                    continue;
                }
                int left = i + 1;
                int right = nums.length - 1;
                while (left < right) {
                    int sum = nums[i] + nums[left] + nums[right];
                    if (sum == 0) {
                        result.add(Arrays.asList(nums[i], nums[left], nums[right]));

                        while (left < right && nums[left] == nums[left + 1]) {
                            left++;
                        }

                        while (left < right && nums[right] == nums[right - 1]) {
                            right--;
                        }
                        left++;
                        right--;
```

```
                    } else if (sum > 0) {
                        right--;
                    } else {
                        left++;
                    }
                }
            }
            return result;
        }
    }
}
```

16. `_16.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 16. 3Sum Closest
 *
 * Given an array S of n integers, find three integers in S such that the sum is closest
to a given number, target.
 * Return the sum of the three integers. You may assume that each input would have
exactly one solution.
 * For example, given array S = {-1 2 1 -4}, and target = 1.
 * The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).
 */

public class _16 {

    public static class Solution1 {
        public int threeSumClosest(int[] nums, int target) {
            Arrays.sort(nums);
            int len = nums.length;
            if (len < 3) {
                int sum = 0;
                for (int i : nums) {
                    sum += i;
                }
                return sum;
            }
            int sum = nums[0] + nums[1] + nums[2];
            for (int i = 0; i < len - 2; i++) {
                int left = i + 1;
                int right = len - 1;
                while (left < right) {
                    int thisSum = nums[i] + nums[left] + nums[right];
                    if (Math.abs(target - thisSum) < Math.abs(target - sum)) {
                        sum = thisSum;
                        if (sum == target) {
                            return sum;
                        }
                    } else if (target > thisSum) {
                        left++;
                    } else {
                        right--;
                    }
                }
            }
            return sum;
        }
    }

}
```

17. `_17.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 17. Letter Combinations of a Phone Number
 *
 * Given a digit string, return all possible letter combinations that the number could
represent.
 * A mapping of digit to letters (just like on the telephone buttons) is given below.

 Input:Digit string "23"
 Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"].

 Note:
 Although the above answer is in lexicographical order, your answer could be in any order
you want.
 */

public class _17 {

    public static class Solution1 {
        public List<String> letterCombinations(String digits) {
            List<String> result = new ArrayList();
            if (digits.length() == 0) {
                return result;
            }

            String[] digits2Letters = new String[]{"", "", "abc", "def", "ghi", "jkl",
"mno", "pqrs", "tuv", "wxyz"};

            result.add("");//this line is important, otherwise result is empty and Java
will default it to an empty String
            for (int i = 0; i < digits.length(); i++) {
                result = combine(digits2Letters[digits.charAt(i) - '0'], result);
            }

            return result;
        }

        List<String> combine(String letters, List<String> result) {
            List<String> newResult = new ArrayList();

            for (int i = 0; i < letters.length(); i++) {
                //the order of the two for loops doesn't matter, you could swap them and
it still works.
                for (String str : result) {
                    newResult.add(str + letters.charAt(i));
                }
            }
```

```
            return newResult;
        }
    }
}
```

18. `_18.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 18. 4Sum
 *
 * Given an array S of n integers, are there elements a, b, c, and d in S such that a + b
+ c + d = target?
 * Find all unique quadruplets in the array which gives the sum of target.

 Note: The solution set must not contain duplicate quadruplets.

 For example, given array S = [1, 0, -1, 0, -2, 2], and target = 0.

 A solution set is:
 [
 [-1,  0, 0, 1],
 [-2, -1, 1, 2],
 [-2,  0, 0, 2]
 ]*/

public class _18 {

    public static class Solution1 {
        public List<List<Integer>> fourSum(int[] nums, int target) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null || nums.length == 0) {
                return result;
            }
            Arrays.sort(nums);
            for (int i = 0; i < nums.length - 3; i++) {
                if (i > 0 && nums[i - 1] == nums[i]) {
                    continue;
                }
                for (int j = i + 1; j < nums.length - 2; j++) {
                    if (j > i + 1 && nums[j - 1] == nums[j]) {
                        continue;
                    }
                    int left = j + 1;
                    int right = nums.length - 1;
                    while (left < right) {
                        int sum = nums[i] + nums[j] + nums[left] + nums[right];
                        if (sum == target) {
                            result.add(Arrays.asList(nums[i], nums[j], nums[left],
nums[right]));

                            while (left + 1 < right && nums[left] == nums[left + 1]) {
                                left++;
                            }
```

```
                            while (right - 1 > left && nums[right] == nums[right - 1]) {
                                right--;
                            }
                            left++;
                            right--;
                        } else if (sum > target) {
                            right--;
                        } else {
                            left++;
                        }
                    }
                }
            }
            return result;
        }
    }
}
```

19. `_19.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;
import com.fishercoder.common.utils.CommonUtils;

/**19. Remove Nth Node From End of List
 *
 * Given a linked list, remove the nth node from the end of list and return its head.
 * For example, Given linked list: 1->2->3->4->5, and n = 2.
 * After removing the second node from the end, the linked list becomes 1->2->3->5.

Note:
Given n will always be valid.
Try to do this in one pass.
 */

public class _19 {

    public static class Solution1 {
        /**
         * Naive/most straightforward approach:
         * go through the list, find its total length, then go through the list a second
time:
         * this time, pause at the delta point, then assign its next.next pointer to
next.
         * This approach has to traverse the list twice, not one-pass.
         */
        public ListNode removeNthFromEnd(ListNode head, int n) {
            ListNode temp = head;
            int len = 0;
            while (temp != null) {
                temp = temp.next;
                len++;
            }
            if (n == len) {
                return head.next;
            }

            temp = head;
            int cut = len - n;
            while (cut-- > 1) {
                temp = temp.next;
            }
            if (temp.next != null) {
                temp.next = temp.next.next;
                return head;
            }
            return null;
        }
    }
```

```
    public static class Solution2 {
        public ListNode removeNthFromEnd(ListNode head, int n) {
            //this approach uses two pointers, fast moves first for n nodes, when fast
reaches n, then we start to move slow
            //then, when fast reaches null, slow reaches the point where the node should
be deleted.
            ListNode dummy = new ListNode(-1);
            dummy.next = head;
            ListNode slow = head;
            ListNode fast = head;
            int tempN = n;
            while (tempN-- > 0) {
                fast = fast.next;
            }

            if (fast == null) {
                if (n > 0) {
                    // this is for cases like this: [1,2] 2 or [1,2,3,4] 4, namely,
remove the head of
                    // the list and return the second node from the original list
                    dummy.next = dummy.next.next;
                }
                return dummy.next;
            }

            fast = fast.next;//we'll have to move fast pointer one node forward before
moving the two together, this way,
            //when fast reaches null, slow will be at the previous node to the node that
should be deleted, thus, we can change the next pointer easily

            while (fast != null) {
                fast = fast.next;
                slow = slow.next;
            }

            if (slow.next != null) {
                slow.next = slow.next.next;
            }
            return dummy.next;
        }
    }

    public static class Solution3 {
        //a more concise version using the same idea
        public ListNode removeNthFromEnd(ListNode head, int n) {
            ListNode dummy = new ListNode(-1);
            dummy.next = head;
            ListNode slow = dummy;
            ListNode fast = dummy;
            while (fast.next != null) {
                if (n <= 0) {
                    slow = slow.next;
                }
                fast = fast.next;
```

```
                n--;
            }
            if (slow.next != null) {
                slow.next = slow.next.next;
            }
            return dummy.next;
        }
    }
}
```

20. `_20.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 20. Valid Parentheses
 *
 * Given a string containing just the characters '(', ')', '{', '}', '[' and ']',
determine if the input string is valid.
 *
 * An input string is valid if:
 *
 * 1. Open brackets must be closed by the same type of brackets.
 * 2. Open brackets must be closed in the correct order.
 *
 * Note that an empty string is also considered valid.
 *
 * Example 1:
 * Input: "()"
 * Output: true
 *
 * Example 2:
 * Input: "()[]{}"
 * Output: true
 *
 * Example 3:
 * Input: "(]"
 * Output: false
 *
 * Example 4:
 * Input: "([)]"
 * Output: false
 *
 * Example 5:
 * Input: "{[]}"
 * Output: true
 * */
public class _20 {
    public static class Solution1 {
        public boolean isValid(String s) {
            Stack<Character> stack = new Stack<>();
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == '(' || s.charAt(i) == '{' || s.charAt(i) == '[') {
                    stack.push(s.charAt(i));
                } else {
                    if (stack.isEmpty()) {
                        return false;
                    } else {
                        if (stack.peek() == '(' && s.charAt(i) != ')') {
                            return false;
                        } else if (stack.peek() == '{' && s.charAt(i) != '}') {
```

```
                            return false;
                        } else if (stack.peek() == '[' && s.charAt(i) != ']') {
                            return false;
                        }
                        stack.pop();
                    }
                }
            }
            return stack.isEmpty();
        }
    }

    public static class Solution2 {
        /**
         * A more concise solution:
         * credit: https://leetcode.com/problems/valid-parentheses/discuss/9178/Short-
java-solution
         * */
        public boolean isValid(String s) {
            Stack<Character> stack = new Stack<>();
            for (char c : s.toCharArray()) {
                if (c == '(') {
                    stack.push(')');
                } else if (c == '{') {
                    stack.push('}');
                } else if (c == '[') {
                    stack.push(']');
                } else if (stack.isEmpty() || stack.pop() != c) {
                    return false;
                }
            }
            return stack.isEmpty();
        }
    }
}
```

21. `_21.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 21. Merge Two Sorted Lists
 *
 * Merge two sorted linked lists and return it as a new list.
 * The new list should be made by splicing together the nodes of the first two lists.
 *
 * Example:
 * Input: 1->2->4, 1->3->4
 * Output: 1->1->2->3->4->4
 *
 * */

public class _21 {
    public static class Solution1 {
        /**
         * recursive solution
         * Time: O(m+n)
         * Space: O(m+n)
         * */
        public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
            if (l1 == null) {
                return l2;
            }
            if (l2 == null) {
                return l1;
            }
            if (l1.val < l2.val) {
                l1.next = mergeTwoLists(l1.next, l2);
                return l1;
            } else {
                l2.next = mergeTwoLists(l1, l2.next);
                return l2;
            }
        }
    }

    public static class Solution2 {
        /**
         * iterative solution
         * Time: O(m+n)
         * Space: O(1)
         */
        public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
            ListNode pre = new ListNode(-1);
            ListNode curr = pre;

            while (l1 != null && l2 != null) {
```

```
                if (l1.val < l2.val) {
                    curr.next = l1;
                    l1 = l1.next;
                } else {
                    curr.next = l2;
                    l2 = l2.next;
                }
                curr = curr.next;
            }

            curr.next = l1 == null ? l2 : l1;
            return pre.next;
        }
    }

}
```

22. `_22.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 22. Generate Parentheses
 *
 * Given n pairs of parentheses, write a function to generate all combinations of well-
formed parentheses.
 *
 * For example, given n = 3, a solution set is:

 [
 "((()))",
 "(()())",
 "(())()",
 "()(())",
 "()()()"
 ]*/

public class _22 {
    public static class Solution1 {
        public List<String> generateParenthesis(int n) {
            List<String> result = new ArrayList();
            backtrack(result, "", 0, 0, n);
            return result;
        }

        void backtrack(List<String> result, String str, int left, int right, int max) {
            if (str.length() == max * 2) {
                result.add(str);
                return;
            }

            if (left < max) {
                backtrack(result, str + "(", left + 1, right, max);
            }

            if (right < left) {
                backtrack(result, str + ")", left, right + 1, max);
            }
        }
    }

    public static class Solution2 {
        public List<String> generateParenthesis(int n) {
            List<String> result = new ArrayList();
            if (n == 0) {
                return result;
            }
```

```
            helper(result, "", n, n);
            return result;
        }

        void helper(List<String> result, String par, int left, int right) {
            if (left > 0) {
                helper(result, par + "(", left - 1, right);
            }
            if (right > left) {
                helper(result, par + ")", left, right - 1);
            }
            if (right == 0) {
                result.add(par);
            }
        }
    }
}
```

23. `_23.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 23. Merge k Sorted Lists
 *
 * Merge k sorted linked lists and return it as one sorted list. Analyze and describe its
complexity.
 *
 * Example:
 *
 * Input:
 * [
 *   1->4->5,
 *   1->3->4,
 *   2->6
 * ]
 * Output: 1->1->2->3->4->4->5->6
 *
 * */

public class _23 {
    public static class Solution1 {
        public ListNode mergeKLists(ListNode[] lists) {
            PriorityQueue<ListNode> heap = new PriorityQueue((Comparator<ListNode>) (o1,
o2) -> o1.val - o2.val);

            for (ListNode node : lists) {
                if (node != null) {
                    heap.offer(node);
                }
            }

            ListNode pre = new ListNode(-1);
            ListNode temp = pre;
            while (!heap.isEmpty()) {
                ListNode curr = heap.poll();
                temp.next = new ListNode(curr.val);
                if (curr.next != null) {
                    heap.offer(curr.next);
                }
                temp = temp.next;
            }
            return pre.next;
        }
    }
```

```
}
```

24. `_24.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 24. Swap Nodes in Pairs
 *
 * Given a linked list, swap every two adjacent nodes and return its head.

 For example,
 Given 1->2->3->4, you should return the list as 2->1->4->3.

 Your algorithm should use only constant space. You may not modify the values in the
list, only nodes itself can be changed.

 */
public class _24 {
    public static class Solution1 {
        public ListNode swapPairs(ListNode head) {
            if (head == null || head.next == null) {
                return head;
            }
            ListNode second = head.next;
            ListNode third = second.next;
            second.next = head;
            head.next = swapPairs(third);
            return second;
        }
    }

}
```

25. `_25.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 25. Reverse Nodes in k-Group
 *
 * Given a linked list, reverse the nodes of a linked list k at a time and return its
modified list.
 * k is a positive integer and is less than or equal to the length of the linked list.
 * If the number of nodes is not a multiple of k then left-out nodes in the end should
remain as it is.
 * You may not alter the values in the nodes, only nodes itself may be changed.
 * Only constant memory is allowed.

 For example,
 Given this linked list: 1->2->3->4->5

 For k = 2, you should return: 2->1->4->3->5

 For k = 3, you should return: 3->2->1->4->5
 */

public class _25 {

    /**We use recursion to go all the way until the end: when the number of nodes are
smaller than k;
     * then we start to reverse each group of k nodes from the end towards the start.*/
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode curr = head;
        int count = 0;
        while (curr != null && count != k) {
            //find the k+1 node
            curr = curr.next;
            count++;
        }

        if (count == k) {
            /**after this below recursive call finishes, it'll return head;
             * then this returned "head" will become "curr", while the head
             * in its previous callstack is the real head after this call.
             * Setting up a break point will make all of this crystal clear.*/
            curr = reverseKGroup(curr, k);

            while (count-- > 0) {
                ListNode temp = head.next;
                head.next = curr;
                curr = head;
                head = temp;
            }
            head = curr;
```

```
        }
        return head;//we run out of nodes before we hit count == k, so we'll just
directly return head in this case as well
    }

}
```

26. `_26.java`

```
package com.fishercoder.solutions;

/**
 * 26. Remove Duplicates from Sorted Array
 *
 * Given a sorted array nums, remove the duplicates in-place such that each element
appear only once and return the new length.
 * Do not allocate extra space for another array, you must do this by modifying the input
array in-place with O(1) extra memory.
 *
 * Example 1:
 * Given nums = [1,1,2],
 * Your function should return length = 2, with the first two elements of nums being 1
and 2 respectively.
 * It doesn't matter what you leave beyond the returned length.
 *
 * Example 2:
 * Given nums = [0,0,1,1,1,2,2,3,3,4],
 * Your function should return length = 5, with the first five elements of nums being
modified to 0, 1, 2, 3, and 4 respectively.
 * It doesn't matter what values are set beyond the returned length.
 *
 * Clarification:
 * Confused why the returned value is an integer but your answer is an array?
 * Note that the input array is passed in by reference, which means modification to the
input array will be known to the caller as well.
 * Internally you can think of this:
 * // nums is passed in by reference. (i.e., without making a copy)
 * int len = removeDuplicates(nums);
 *
 * // any modification to nums in your function would be known by the caller.
 * // using the length returned by your function, it prints the first len elements.
 * for (int i = 0; i < len; i++) {
 *     print(nums[i]);
 * }
 * */

public class _26 {

  public static class Solution1 {
    /**Key: It doesn't matter what you leave beyond the returned length.*/
    public int removeDuplicates(int[] nums) {
      int i = 0;
      for (int j = 1; j < nums.length; j++) {
        if (nums[i] != nums[j]) {
          i++;
          nums[i] = nums[j];
        }
      }
      return i + 1;
    }
```

```
  }
}
```

27. `_27.java`

```
package com.fishercoder.solutions;

/**27. Remove Element

Given an array and a value, remove all instances of that value in place and return the new length.

Do not allocate extra space for another array, you must do this in place with constant memory.

The order of elements can be changed. It doesn't matter what you leave beyond the new length.

Example:
Given input array nums = [3,2,2,3], val = 3

Your function should return length = 2, with the first two elements of nums being 2.
 */

public class _27 {

  public static class Solution1 {
    public int removeElement(int[] nums, int val) {
      int i = 0;
      for (int j = 0; j < nums.length; j++) {
        if (nums[j] != val) {
          nums[i++] = nums[j];
        }
      }
      return i;
    }
  }
}
```

28. `_28.java`

```
package com.fishercoder.solutions;

/**
 * 28. Implement strStr()
 *
 * Implement strStr().
 *
 * Returns the index of the first occurrence of needle in haystack, or -1 if needle is
not part of haystack.
 */

public class _28 {

  public static class Solution1 {
    public int strStr(String haystack, String needle) {
      if (haystack == null || needle == null || haystack.length() < needle.length()) {
        return -1;
      }

      for (int i = 0; i <= haystack.length() - needle.length(); i++) {
        if (haystack.substring(i, i + needle.length()).equals(needle)) {
          return i;
        }
      }
      return -1;
    }
  }

}
```

29. `_29.java`

```
package com.fishercoder.solutions;

/**
 * 29. Divide Two Integers
 *
 * Divide two integers without using multiplication, division and mod operator.
 * If it is overflow, return MAX_INT.
 */
public class _29 {

  public static class Solution1 {
    public int divide(int dividend, int divisor) {
      if (divisor == 0 || (dividend == Integer.MIN_VALUE && divisor == -1)) {
        return Integer.MAX_VALUE;
      }
      if (dividend != Integer.MIN_VALUE
          && Math.abs(dividend) < Math.abs(divisor)) {
        return 0;
      }
      if (divisor == Integer.MIN_VALUE) {
        return (dividend == Integer.MIN_VALUE) ? 1 : 0;
      }

      boolean flag = (dividend < 0) ^ (divisor < 0);
      dividend = -Math.abs(dividend);
      divisor = -Math.abs(divisor);
      int[] num = new int[40];
      int[] multiple = new int[40];
      num[1] = divisor;
      multiple[1] = 1;

      for (int i = 2; i < 32 && num[i - 1] < 0; ++i) {
        num[i] = num[i - 1] << 1;
        multiple[i] = multiple[i - 1] << 1;
      }

      int result = 0;
      int index = 1;
      while (num[index] < 0) {
        ++index;
      }
      index -= 1;

      while (dividend <= divisor) {
        while (dividend <= num[index]) {
          result += multiple[index];
          dividend -= num[index];
        }
        --index;
      }
      return !flag ? result : -result;
```

```
    }
  }
}
```

30. `_30.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 30. Substring with Concatenation of All Words
 *
 * You are given a string, s, and a list of words, words, that are all of the same
length.
 * Find all starting indices of substring(s) in s that is a concatenation of each word in
words exactly once and without any intervening characters.

 For example, given:
 s: "barfoothefoobarman"
 words: ["foo", "bar"]

 You should return the indices: [0,9].
 (order does not matter).
 */
public class _30 {

  public static class Solution1 {
    /**TODO: this one is not AC'ed. fix this one.*/
    public List<Integer> findSubstring(String s, String[] words) {
      Map<String, Integer> map = new HashMap<>();
      for (String word : words) {
        map.put(word, 1);
      }
      List<Integer> result = new ArrayList<>();
      int startIndex = 0;
      int wordLen = words.length;
      for (int i = 0; i < s.length(); i++) {
        startIndex = i;
        Map<String, Integer> clone = new HashMap<>(map);
        int matchedWord = 0;
        for (int j = i + 1; j < s.length(); j++) {
          String word = s.substring(i, j);
          if (clone.containsKey(word) && clone.get(word) == 1) {
            clone.put(word, 0);
            i = j;
            matchedWord++;
          }
          if (matchedWord == wordLen) {
            boolean all = true;
            for (String key : clone.keySet()) {
              if (clone.get(key) != 0) {
                all = false;
                break;
```

```
            }
          }
          if (all) {
            result.add(startIndex);
          }
          matchedWord = 0;
        }
      }
    }
    return result;
  }
}
```

31. `_31.java`

```
package com.fishercoder.solutions;

/**
 * 31. Next Permutation

Implement next permutation, which rearranges numbers into the lexicographically next
greater permutation of numbers.

If such arrangement is not possible, it must rearrange it as the lowest possible order
(ie, sorted in ascending order).

The replacement must be in-place, do not allocate extra memory.

Here are some examples. Inputs are in the left-hand column and its corresponding outputs
are in the right-hand column.
1,2,3 → 1,3,2
3,2,1 → 1,2,3
1,1,5 → 1,5,1
 */

public class _31 {
  public static class Solution1 {
    /**
     * Leetcode has a very good article to illustrate this problem and with animation:
     * https://leetcode.com/articles/next-permutation/
     * 1. if the array is already in decrementing order, then there's no next larger
permutation possible.
     * 2. if not, start from the end of the array, find the first pair of numbers that
break the decrementing order
     * 3. then from that index going to the right again, find the element that is closest
bigger than this number, swap them
     * 4. reverse the right half of this array after this index
     */

    public void nextPermutation(int[] nums) {
      int i = nums.length - 2;
      while (i >= 0 && nums[i] >= nums[i + 1]) {
        i--;
      }
      if (i >= 0) {
        int j = nums.length - 1;
        while (j >= 0 && nums[i] >= nums[j]) {
          j--;
        }

        swap(nums, i, j);
      }

      reverse(nums, i + 1);
    }
```

```
    private void reverse(int[] nums, int start) {
      int end = nums.length - 1;
      while (start <= end) {
        int tmp = nums[start];
        nums[start++] = nums[end];
        nums[end--] = tmp;
      }
    }

    private void swap(int[] nums, int i, int j) {
      int tmp = nums[i];
      nums[i] = nums[j];
      nums[j] = tmp;
    }
  }
}
```

32. `_32.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 32. Longest Valid Parentheses
 *
 * Given a string containing just the characters '(' and ')', find the length of the
longest valid (well-formed) parentheses substring.
 * For "(()", the longest valid parentheses substring is "()", which has length = 2.
 * Another example is ")()())", where the longest valid parentheses substring is "()()",
which has length = 4.
 */
public class _32 {
  public static class Solution1 {
    public int longestValidParentheses(String s) {
      int result = 0;
      Stack<Integer> stack = new Stack();
      stack.push(-1);
      for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == '(') {
          stack.push(i);
        } else {
          stack.pop();
          if (stack.isEmpty()) {
            stack.push(i);
          } else {
            result = Math.max(result, i - stack.peek());
          }
        }
      }
      return result;
    }
  }
}
```

33. `_33.java`

```
package com.fishercoder.solutions;

/**
 * 33. Search in Rotated Sorted Array
 *
 * Suppose a sorted array is rotated at some pivot unknown to you beforehand.
 * (i.e., 0 1 2 4 5 6 7 might become 4 5 6 7 0 1 2).
 * You are given a target value to search. If found in the array return its index,
otherwise return -1.
 * You may assume no duplicate exists in the array.
 */
public class _33 {

    public static class Solution1 {

        public int search(int[] nums, int target) {
            if (nums == null || nums.length == 0) {
                return -1;
            }
            int minIdx = findMinIdx(nums);
            if (target == nums[minIdx]) {
                return minIdx;
            }
            int m = nums.length;
            int start = (target <= nums[m - 1]) ? minIdx : 0;
            int end = (target > nums[m - 1]) ? minIdx : m - 1;

            while (start <= end) {
                int mid = start + (end - start) / 2;
                if (nums[mid] == target) {
                    return mid;
                } else if (target > nums[mid]) {
                    start = mid + 1;
                } else {
                    end = mid - 1;
                }
            }
            return -1;
        }

        private int findMinIdx(int[] nums) {
            int start = 0;
            int end = nums.length - 1;

            while (start < end) {
                int mid = start + (end - start) / 2;
                if (nums[mid] > nums[end]) {
                    start = mid + 1;
                } else {
                    end = mid;
                }
```

```
            }
            return start;
        }
    }

    public static class Solution2 {
        public int search(int[] nums, int target) {
            if (nums == null || nums.length == 0) {
                return -1;
            }
            int lo = 0;
            int hi = nums.length - 1;
            while (lo < hi) {
                int mid = (lo + hi) / 2;
                if (nums[mid] == target) {
                    return mid;
                }

                if (nums[lo] <= nums[mid]) {
                    if (target >= nums[lo] && target < nums[mid]) {
                        hi = mid - 1;
                    } else {
                        lo = mid + 1;
                    }
                } else {
                    if (target > nums[mid] && target <= nums[hi]) {
                        lo = mid + 1;
                    } else {
                        hi = mid - 1;
                    }
                }
            }
            return nums[lo] == target ? lo : -1;
        }
    }
}
```

34. `_34.java`

```
package com.fishercoder.solutions;

/**
 * 34. Search for a Range
 *
 * Given an array of integers sorted in ascending order, find the starting and ending
position of a given target value.
 * Your algorithm's runtime complexity must be in the order of O(log n).
 * If the target is not found in the array, return [-1, -1].

 For example,
 Given [5, 7, 7, 8, 8, 10] and target value 8,
 return [3, 4].

 */
public class _34 {

    public int[] searchRange(int[] nums, int target) {
        int[] range = new int[2];
        range[0] = -1;
        range[1] = -1;
        if (nums == null || nums.length == 0) {
            return range;
        }
        int start = 0;
        int end = nums.length - 1;
        while (start + 1 < end) {
            int mid = start + (end - start) / 2;
            if (nums[mid] == target) {
                int left = mid;
                while (left - 1 >= 0 && nums[left] == nums[left - 1]) {
                    left--;
                }
                range[0] = left;
                int right = mid;
                while (right + 1 < nums.length && nums[right] == nums[right + 1]) {
                    right++;
                }
                range[1] = right;
                break;
            } else if (nums[mid] < target) {
                start = mid;
            } else {
                end = mid;
            }
        }

        if (nums[start] == target) {
            range[0] = start;
            while (start + 1 < nums.length && nums[start] == nums[start + 1]) {
                start++;
```

```
            }
            range[1] = start;
        } else if (nums[end] == target) {
            range[1] = end;
            while (end - 1 >= 0 && nums[end] == nums[end - 1]) {
                end--;
            }
            range[0] = end;
        }
        return range;
    }
}
```

35. `_35.java`

```
package com.fishercoder.solutions;

/**
 * 35. Search Insert Position
 *
 * Given a sorted array and a target value,
 * return the index if the target is found.
 * If not, return the index where it would be if it were inserted in order.

 You may assume no duplicates in the array.

 Here are few examples.
 [1,3,5,6], 5 → 2
 [1,3,5,6], 2 → 1
 [1,3,5,6], 7 → 4
 [1,3,5,6], 0 → 0
 */

public class _35 {

    public static class Solution1 {
        public int searchInsert(int[] nums, int target) {
            int len = nums.length;
            for (int i = 0; i < len; i++) {
                if (target <= nums[i]) {
                    return i;
                }
            }
            return len;
        }
    }

}
```

36. `_36.java`

```
package com.fishercoder.solutions;

/**
 * 36. Valid Sudoku
 *
 * Determine if a Sudoku is valid, according to: Sudoku Puzzles - The Rules.
 * The Sudoku board could be partially filled, where empty cells are filled with the
character '.'.
 *
 * A partially filled sudoku which is valid.

 Note:
 A valid Sudoku board (partially filled) is not necessarily solvable. Only the filled
cells need to be validated.*/
public class _36 {

  public static class Solution1 {
    public boolean isValidSudoku(char[][] board) {
      for (int i = 0; i < 9; i++) {
        if (!isValidRowOrColumn(board, i)) {
          return false;
        }
      }

      for (int j = 0; j < 9; j++) {
        if (!isValidCol(board, j)) {
          return false;
        }
      }

      for (int i = 0; i < 7; i = i + 3) {
        for (int j = 0; j < 7; j = j + 3) {
          if (!isValidSquare(board, i, j)) {
            return false;
          }
        }
      }
      return true;
    }

    boolean isValidRowOrColumn(char[][] board, int index) {
      int[] nums = new int[9];
      for (int i = 0; i < 9; i++) {
        nums[i] = 1;
      }
      for (int j = 0; j < 9; j++) {
        if (board[index][j] != '.') {
          nums[Character.getNumericValue(board[index][j]) - 1]--;
        }
      }
      for (int i : nums) {
```

```
        if (i < 0) {
          return false;
        }
      }
      return true;
    }

    boolean isValidCol(char[][] board, int col) {
      int[] nums = new int[9];
      for (int i = 0; i < 9; i++) {
        nums[i] = 1;
      }
      for (int i = 0; i < 9; i++) {
        if (board[i][col] != '.') {
          nums[Character.getNumericValue(board[i][col]) - 1]--;
        }
      }
      for (int i : nums) {
        if (i < 0) {
          return false;
        }
      }
      return true;
    }

    boolean isValidSquare(char[][] board, int row, int col) {
      int[] nums = new int[9];
      for (int i = 0; i < 9; i++) {
        nums[i] = 1;
      }
      for (int i = row; i < row + 3; i++) {
        for (int j = col; j < col + 3; j++) {
          if (board[i][j] != '.') {
            nums[Character.getNumericValue(board[i][j]) - 1]--;
          }
        }
      }
      for (int i : nums) {
        if (i < 0) {
          return false;
        }
      }
      return true;
    }
  }
}
```

37. `_37.java`

```
package com.fishercoder.solutions;

/**
 * 37. Sudoku Solver
 *
 * Determine if a 9x9 Sudoku board is valid. Only the filled cells need to be validated
according to the following rules:
 *
 * Each row must contain the digits 1-9 without repetition.
 * Each column must contain the digits 1-9 without repetition.
 * Each of the 9 3x3 sub-boxes of the grid must contain the digits 1-9 without
repetition.
 *
 * The Sudoku board could be partially filled, where empty cells are filled with the
character '.'.
 *
 * Example 1:
 *
 * Input:
 * [
 *   ["5","3",".",".","7",".",".",".","."],
 *   ["6",".",".","1","9","5",".",".","."],
 *   [".","9","8",".",".",".",".","6","."],
 *   ["8",".",".",".","6",".",".",".","3"],
 *   ["4",".",".","8",".","3",".",".","1"],
 *   ["7",".",".",".","2",".",".",".","6"],
 *   [".","6",".",".",".",".","2","8","."],
 *   [".",".",".","4","1","9",".",".","5"],
 *   [".",".",".",".","8",".",".","7","9"]
 * ]
 * Output: true
 * Example 2:
 *
 * Input:
 * [
 *   ["8","3",".",".","7",".",".",".","."],
 *   ["6",".",".","1","9","5",".",".","."],
 *   [".","9","8",".",".",".",".","6","."],
 *   ["8",".",".",".","6",".",".",".","3"],
 *   ["4",".",".","8",".","3",".",".","1"],
 *   ["7",".",".",".","2",".",".",".","6"],
 *   [".","6",".",".",".",".","2","8","."],
 *   [".",".",".","4","1","9",".",".","5"],
 *   [".",".",".",".","8",".",".","7","9"]
 * ]
 * Output: false
 * Explanation: Same as Example 1, except with the 5 in the top left corner being
 *     modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.
 * Note:
 *
 * A Sudoku board (partially filled) could be valid but is not necessarily solvable.
```

```
 * Only the filled cells need to be validated according to the mentioned rules.
 * The given board contain only digits 1-9 and the character '.'.
 * The given board size is always 9x9.
 *
 */
public class _37 {

    public static class Solution1 {
        public void solveSudoku(char[][] board) {
            if (board == null || board.length == 0) {
                return;
            }
            solve(board);
        }

        private boolean solve(char[][] board) {
            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    if (board[i][j] == '.') {
                        for (char c = '1'; c <= '9'; c++) {
                            //try 1 to 9
                            if (isValid(board, i, j, c)) {
                                board[i][j] = c;

                                if (solve(board)) {
                                    return true;
                                } else {
                                    board[i][j] = '.';//recover it to be '.'
                                }
                            }
                        }
                        return false;
                    }
                }
            }
            return true;
        }

        private boolean isValid(char[][] board, int row, int col, char c) {
            for (int i = 0; i < 9; i++) {
                if (board[i][col] != '.' && board[i][col] == c) {
                    return false;//check row
                }
                if (board[row][i] != '.' && board[row][i] == c) {
                    return false;//check column
                }
                if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] != '.' && board[3
* (row / 3) + i / 3][3 * (col / 3) + i % 3] == c) {
                    return false; //check 3*3 block
                }
            }
            return true;
        }
    }
```

```
}
```

38. `_38.java`

```
package com.fishercoder.solutions;

/**
 * 38. Count and Say
 *
 * The count-and-say sequence is the sequence of integers beginning as follows:
 1, 11, 21, 1211, 111221, ...

 1 is read off as "one 1" or 11.
 11 is read off as "two 1s" or 21.
 21 is read off as "one 2, then one 1" or 1211.
 Given an integer n, generate the nth sequence.

 Note: The sequence of integers will be represented as a string.*/

public class _38 {
  public static class Solution1 {
    public String countAndSay(int n) {
      StringBuilder curr = new StringBuilder("1");
      StringBuilder prev;
      int count;
      char say;
      for (int i = 1; i < n; i++) {
        prev = curr;
        curr = new StringBuilder();
        count = 1;
        say = prev.charAt(0);

        for (int j = 1, len = prev.length(); j < len; j++) {
          if (prev.charAt(j) != say) {
            curr.append(count).append(say);
            count = 1;
            say = prev.charAt(j);
          } else {
            count++;
          }
        }
        curr.append(count).append(say);
      }
      return curr.toString();
    }
  }

}
```

39. `_39.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 39. Combination Sum
 *
 * Given a set of candidate numbers (C) (without duplicates) and a target number (T),
 * find all unique combinations in C where the candidate numbers sums to T.

 The same repeated number may be chosen from C unlimited number of times.

 Note:
 All numbers (including target) will be positive integers.
 The solution set must not contain duplicate combinations.
 For example, given candidate set [2, 3, 6, 7] and target 7,
 A solution set is:
 [
 [7],
 [2, 2, 3]
 ]*/
public class _39 {

    public static class Solution1 {
        public List<List<Integer>> combinationSum(int[] candidates, int target) {
            List<List<Integer>> result = new ArrayList();
            Arrays.sort(candidates);
            backtracking(candidates, target, 0, new ArrayList(), result);
            return result;
        }

        void backtracking(int[] candidates, int target, int start, List<Integer> curr,
List<List<Integer>> result) {
            if (target > 0) {
                for (int i = start; i < candidates.length; i++) {
                    if (candidates[i] > target) {
                        continue;//pruning
                    }
                    curr.add(candidates[i]);
                    backtracking(candidates, target - candidates[i], i, curr, result);
                    curr.remove(curr.size() - 1);
                }
            } else if (target == 0) {
                result.add(new ArrayList(curr));
            }
        }
    }
}
```

40. `_40.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
/**
 * 40. Combination Sum II
 *
 * Given a collection of candidate numbers (C) and a target number (T), find all unique
combinations in C where the candidate numbers sums to T.
 * Each number in C may only be used once in the combination.

 Note:
 All numbers (including target) will be positive integers.
 The solution set must not contain duplicate combinations.
 For example, given candidate set [10, 1, 2, 7, 6, 1, 5] and target 8,
 A solution set is:
 [
 [1, 7],
 [1, 2, 5],
 [2, 6],
 [1, 1, 6]
 ]
 */
public class _40 {

    public static class Solution1 {
      public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList();
        Arrays.sort(candidates);
        backtracking(candidates, 0, result, target, new ArrayList());
        return result;
      }

      void backtracking(int[] candidates, int start, List<List<Integer>> result, int
target,
          List<Integer> curr) {
        if (target > 0) {
          for (int i = start; i < candidates.length; i++) {
            if (candidates[i] > target || (i > start && candidates[i - 1] ==
candidates[i])) {
              continue;
            }
            curr.add(candidates[i]);
            backtracking(candidates, i + 1, result, target - candidates[i], curr);
            curr.remove(curr.size() - 1);
          }
        } else if (target == 0) {
          result.add(new ArrayList(curr));
        }
      }
```

```
    }
}
```

41. `_41.java`

```
package com.fishercoder.solutions;

/**
 *41. First Missing Positive
 *
 *Given an unsorted integer array, find the first missing positive integer.

 For example,
 Given [1,2,0] return 3,
 and [3,4,-1,1] return 2.

 Your algorithm should run in O(n) time and uses constant space.
 */

public class _41 {

  public static class Solution1 {
    /**
     * Time: O(n) Space: O(1)
     *
     * Idea: put every number in its right position, e.g. put 5 in nums[4].
     */
    public int firstMissingPositive(int[] nums) {
      int i = 0;
      while (i < nums.length) {
        if (nums[i] > 0 && nums[i] != i + 1 && nums[i] - 1 < nums.length && nums[i] !=
nums[nums[i]
            - 1]) {
          swap(nums, i, nums[i] - 1);
        } else {
          i++;
        }
      }

      for (int j = 0; j < nums.length; j++) {
        if (nums[j] != j + 1) {
          return j + 1;
        }
      }

      return nums.length + 1;
    }

    void swap(int[] nums, int i, int j) {
      int temp = nums[i];
      nums[i] = nums[j];
      nums[j] = temp;
    }
  }
}
```

42. `_42.java`

```
package com.fishercoder.solutions;

/**
 * 42. Trapping Rain Water
 * Given n non-negative integers representing an elevation map where the width of each
bar is 1,
 * compute how much water it is able to trap after raining.

 For example,
 Given [0,1,0,2,1,0,1,3,2,1,2,1], return 6.

 The above elevation map is represented by array [0,1,0,2,1,0,1,3,2,1,2,1].
 In this case, 6 units of rain water (blue section) are being trapped.
 */

public class _42 {

    public static class Solution1 {
        /**
         * O(n) time and O(1) space, awesome!
         *
         * 1. first scan to find the max height index
         * 2. then scan from left up to max index and find all the water units up to the
max height
         * 3. then scan from right down to max index and find all the water units down to
the max height
         * 4. return the sum of those above two
         *
         * reference: https://discuss.leetcode.com/topic/22976/my-accepted-java-solution
         */
        public int trap(int[] height) {
            if (height == null || height.length <= 2) {
                return 0;
            }

            int max = height[0];
            int maxIndex = 0;
            for (int i = 0; i < height.length; i++) {
                if (height[i] > max) {
                    max = height[i];
                    maxIndex = i;
                }
            }

            int water = 0;

            int leftMax = height[0];
            for (int i = 0; i < maxIndex; i++) {
                if (height[i] > leftMax) {
                    leftMax = height[i];
                } else {
```

```
                    water += leftMax - height[i];
                }
            }

            int rightMax = height[height.length - 1];
            for (int i = height.length - 1; i > maxIndex; i--) {
                if (height[i] > rightMax) {
                    rightMax = height[i];
                } else {
                    water += rightMax - height[i];
                }
            }

            return water;
        }
    }
}
```

43. `_43.java`

```
package com.fishercoder.solutions;

/**
 * 43. Multiply Strings
 *
 * Given two non-negative integers num1 and num2 represented as strings, return the
product of num1 and num2.

 Note:

 The length of both num1 and num2 is < 110.
 Both num1 and num2 contains only digits 0-9.
 Both num1 and num2 does not contain any leading zero.
 You must not use any built-in BigInteger library or convert the inputs to integer
directly.

 */
public class _43 {

    /**Inspired by https://discuss.leetcode.com/topic/30508/easiest-java-solution-with-
graph-explanation
     * Basically, the rule we can find is that products of each two digits will land in
this position in the final product:
     * i+j and i+j+1*/
    public String multiply(String num1, String num2) {
        if (isZero(num1) || isZero(num2)) {
            return "0";
        }
        int[] a1 = new int[num1.length()];
        int[] a2 = new int[num2.length()];
        int[] product = new int[num1.length() + num2.length()];

        for (int i = a1.length - 1; i >= 0; i--) {
            for (int j = a2.length - 1; j >= 0; j--) {
                int thisProduct = Character.getNumericValue(num1.charAt(i)) *
Character.getNumericValue(num2.charAt(j));
                product[i + j + 1] += thisProduct % 10;
                if (product[i + j + 1] >= 10) {
                    product[i + j + 1] %= 10;
                    product[i + j]++;
                }
                product[i + j] += thisProduct / 10;
                if (product[i + j] >= 10) {
                    product[i + j] %= 10;
                    product[i + j - 1]++;
                }
            }
        }

        StringBuilder stringBuilder = new StringBuilder();
        for (int i = 0; i < product.length; i++) {
```

```
            if (i == 0 && product[i] == 0) {
                continue;
            }
            stringBuilder.append(product[i]);
        }
        return stringBuilder.toString();
    }

    private boolean isZero(String num) {
        for (char c : num.toCharArray()) {
            if (c != '0') {
                return false;
            }
        }
        return true;
    }

}
```

44. `_44.java`

```
package com.fishercoder.solutions;

/**
 * 44. Wildcard Matching
 * Implement wildcard pattern matching with support for '?' and '*'.

 '?' Matches any single character.
 '*' Matches any sequence of characters (including the empty sequence).

 The matching should cover the entire input string (not partial).

 The function prototype should be:
 bool isMatch(const char *s, const char *p)

 Some examples:
 isMatch("aa","a") → false
 isMatch("aa","aa") → true
 isMatch("aaa","aa") → false
 isMatch("aa", "*") → true
 isMatch("aa", "a*") → true
 isMatch("ab", "?*") → true
 isMatch("aab", "c*a*b") → false
 */
public class _44 {

  public static class Solution1 {
    public boolean isMatch(String s, String p) {
      boolean[][] match = new boolean[s.length() + 1][p.length() + 1];
      match[s.length()][p.length()] = true;
      for (int i = p.length() - 1; i >= 0; i--) {
        if (p.charAt(i) != '*') {
          break;
        } else {
          match[s.length()][i] = true;
        }
      }

      for (int i = s.length() - 1; i >= 0; i--) {
        for (int j = p.length() - 1; j >= 0; j--) {
          if (s.charAt(i) == p.charAt(j) || p.charAt(j) == '?') {
            match[i][j] = match[i + 1][j + 1];
          } else if (p.charAt(j) == '*') {
            match[i][j] = match[i + 1][j] || match[i][j + 1];
          } else {
            match[i][j] = false;
          }
        }
      }
      return match[0][0];
    }
```

```
  }
}
```

45. `_45.java`

```
package com.fishercoder.solutions;

/**
 * 45. Jump Game II
 *
 * Given an array of non-negative integers, you are initially positioned at the first
index of the array.
 * Each element in the array represents your maximum jump length at that position.
 * Your goal is to reach the last index in the minimum number of jumps.

 For example:
 Given array A = [2,3,1,1,4]

 The minimum number of jumps to reach the last index is 2. (Jump 1 step from index 0 to
1, then 3 steps to the last index.)

 Note:
 You can assume that you can always reach the last index.

 */
public class _45 {

  public static class Solution1 {
    public int jump(int[] nums) {
      int stepCount = 0;
      int lastJumpMax = 0;
      int currentJumpMax = 0;
      for (int i = 0; i < nums.length - 1; i++) {
        currentJumpMax = Math.max(currentJumpMax, i + nums[i]);
        if (i == lastJumpMax) {
          stepCount++;
          lastJumpMax = currentJumpMax;
        }
      }
      return stepCount;
    }
  }
}
```

46. `_46.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 46. Permutations
 *
 * Given a collection of distinct numbers, return all possible permutations.

 For example,
 [1,2,3] have the following permutations:
 [
 [1,2,3],
 [1,3,2],
 [2,1,3],
 [2,3,1],
 [3,1,2],
 [3,2,1]
 ]

 */

public class _46 {

    public static class Solution1 {
        public List<List<Integer>> permute(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            result.add(new ArrayList<>());
            return backtracking(result, nums, 0);
        }

        private List<List<Integer>> backtracking(List<List<Integer>> result, int[] nums,
int pos) {
            if (pos == nums.length) {
                return result;
            }
            List<List<Integer>> newResult = new ArrayList();
            for (List<Integer> eachList : result) {
                for (int i = 0; i <= eachList.size(); i++) {
                    //attn: i starts from 0
                    List<Integer> newList = new ArrayList(eachList);
                    newList.add(i, nums[pos]);
                    newResult.add(newList);
                }
            }
            result = newResult;
            return backtracking(result, nums, pos + 1);
        }
    }
```

```
}
```

47. `_47.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 47. Permutations II
 *
 * Given a collection of numbers that might contain duplicates, return all possible
unique permutations.

 For example,
 [1,1,2] have the following unique permutations:
 [
 [1,1,2],
 [1,2,1],
 [2,1,1]
 ]*/
public class _47 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/31445/really-easy-java-solution-
much-easier-than-the-solutions-with-very-high-vote
         */
        public List<List<Integer>> permuteUnique(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null || nums.length == 0) {
                return result;
            }
            boolean[] used = new boolean[nums.length];
            List<Integer> list = new ArrayList();
            Arrays.sort(nums);
            dfs(nums, used, list, result);
            return result;
        }


        private void dfs(int[] nums, boolean[] used, List<Integer> list,
List<List<Integer>> result) {
            if (list.size() == nums.length) {
                result.add(new ArrayList(list));
                return;
            }
            for (int i = 0; i < nums.length; i++) {
                if (used[i]) {
                    continue;
                }
                if (i > 0 && nums[i - 1] == nums[i] && !used[i - 1]) {
                    continue;
                }
```

```
                /**
                 * For this line, both !used[i-1] and used[i-1] will AC. It is because
the first one makes sure when
                 * duplicates are selected, the order is ascending (index from small to
large). However,
                 * the second one means the descending order.
                 */
                used[i] = true;
                list.add(nums[i]);
                dfs(nums, used, list, result);
                used[i] = false;
                list.remove(list.size() - 1);
            }
        }
    }
}
```

48. `_48.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 48. Rotate Image

 You are given an n x n 2D matrix representing an image.

 Rotate the image by 90 degrees (clockwise).

 Note:
 You have to rotate the image in-place, which means you have to modify the
 input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

 Example 1:
 Given input matrix =
 [
 [1,2,3],
 [4,5,6],
 [7,8,9]
 ],
 rotate the input matrix in-place such that it becomes:
 [
 [7,4,1],
 [8,5,2],
 [9,6,3]
 ]

 Example 2:
 Given input matrix =
 [
 [ 5, 1, 9,11],
 [ 2, 4, 8,10],
 [13, 3, 6, 7],
 [15,14,12,16]
 ],
 rotate the input matrix in-place such that it becomes:
 [
 [15,13, 2, 5],
 [14, 3, 4, 1],
 [12, 6, 8, 9],
 [16, 7,10,11]
 ]
 */
public class _48 {

    /**Note: this is an n*n matrix, in other words, it's a square, this makes it easier
as well.*/

    public static class Solution1 {
```

```
        //Time: O(n^2)
        //Space: O(1)
        public void rotate(int[][] matrix) {
            /**First swap the elements on the diagonal, then reverse each row:
             * 1, 2, 3                    1, 4, 7                      7, 4, 1
             * 4, 5, 6         becomes    2, 5, 8           becomes    8, 5, 2
             * 7, 8, 9                    3, 6, 9                      9, 6, 3
             **/
            int m = matrix.length;
            for (int i = 0; i < m; i++) {
                for (int j = i; j < m; j++) {
                    /**ATTN: j starts from i, so that the diagonal changes with itself,
results in no change.*/
                    int tmp = matrix[i][j];
                    matrix[i][j] = matrix[j][i];
                    matrix[j][i] = tmp;
                }
            }
            CommonUtils.print2DIntArray(matrix);

            /**then reverse*/
            for (int i = 0; i < m; i++) {
                int left = 0;
                int right = m - 1;
                while (left < right) {
                    int tmp = matrix[i][left];
                    matrix[i][left] = matrix[i][right];
                    matrix[i][right] = tmp;
                    left++;
                    right--;
                }
            }
        }
    }

    public static class Solution2 {
        /**First swap the rows bottom up, then swap the element on the diagonal:
         *
         * 1, 2, 3                          7, 8, 9                          7, 4, 1
         * 4, 5, 6           becomes        4, 5, 6           becomes        8, 5, 2
         * 7, 8, 9                          1, 2, 3                          9, 6, 3
         *
         * Time: O(n^2)
         * Space: O(1)
         */
        public void rotate(int[][] matrix) {
            int m = matrix.length;
            int n = matrix[0].length;
            int top = 0;
            int bottom = n - 1;
            while (top < bottom) {
                int[] tmp = matrix[top];
                matrix[top] = matrix[bottom];
                matrix[bottom] = tmp;
```

```
                top++;
                bottom--;
            }

            for (int i = 0; i < m; i++) {
                for (int j = i + 1; j < n; j++) {
                    int tmp = matrix[i][j];
                    matrix[i][j] = matrix[j][i];
                    matrix[j][i] = tmp;
                }
            }
        }
    }

    public static class Solution3 {
        public void rotate(int[][] matrix) {
            int n = matrix.length;
            for (int i = 0; i < n / 2; i++) {
                for (int j = i; j < n - i - 1; j++) {
                    //save the top
                    int top = matrix[i][j];

                    //move left to top
                    matrix[i][j] = matrix[n - 1 - j][i];

                    //move bottom to left
                    matrix[n - 1 - j][i] = matrix[n - i - 1][n - 1 - j];

                    //move right to bottom
                    matrix[n - i - 1][n - 1 - j] = matrix[j][n - i - 1];

                    //move top to right
                    matrix[j][n - i - 1] = top;
                }
            }
        }
    }

}
```

49. `_49.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 49. Group Anagrams
 *
 * Given an array of strings, group anagrams together.

 For example, given: ["eat", "tea", "tan", "ate", "nat", "bat"],
 Return:

 [
 ["ate", "eat","tea"],
 ["nat","tan"],
 ["bat"]
 ]

 Note: All inputs will be in lower-case.
 */

public class _49 {

  public static class Solution1 {
    public List<List<String>> groupAnagrams(String[] strs) {
      Map<String, List<String>> map = new HashMap<>();
      for (String word : strs) {
        char[] c = word.toCharArray();
        Arrays.sort(c);
        String key = new String(c);
        if (!map.containsKey(key)) {
          map.put(key, new ArrayList<>());
        }
        map.get(key).add(word);
      }
      return new ArrayList<>(map.values());
    }
  }
}
```

50. `_50.java`

```
package com.fishercoder.solutions;

/**
 * 50. Pow(x, n)
 *
 * Implement pow(x, n).

 Example 1:

 Input: 2.00000, 10
 Output: 1024.00000

 Example 2:

 Input: 2.10000, 3
 Output: 9.26100
 */
public class _50 {

  public static class Solution1 {
    /**
     * Time: O(logn)
     * Space: O(logn)
     */
    public double myPow(double x, int n) {
      long N = n;
      if (N < 0) {
        x = 1 / x;
        N = -N;
      }
      return fastPow(x, N);
    }

    private double fastPow(double x, long n) {
      if (n == 0) {
        return 1.0;
      }
      double half = fastPow(x, n / 2);
      if (n % 2 == 0) {
        return half * half;
      } else {
        return half * half * x;
      }
    }
  }

  public static class Solution2 {
    /**
     * Time: O(logn)
     * Space: O(1)
     */
```

```
    public double myPow(double x, int n) {
      long N = n;
      if (N < 0) {
        x = 1 / x;
        N = -N;
      }
      double answer = 1;
      double currentProduct = x;
      for (long i = N; i > 0; i /= 2) {
        if (i % 2 == 1) {
          answer = answer * currentProduct;
        }
        currentProduct *= currentProduct;
      }
      return answer;
    }
  }
}
```

51. `_51.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 51. N-Queens
 *
 * The n-queens puzzle is the problem of placing n queens on an n×n chessboard such that
no two queens attack each other.

 Given an integer n, return all distinct solutions to the n-queens puzzle.
 Each solution contains a distinct board configuration of the n-queens' placement, where
'Q' and '.' both indicate a queen and an empty space respectively.

 For example,
 There exist two distinct solutions to the 4-queens puzzle:

 [
 [".Q..",  // Solution 1
 "...Q",
 "Q...",
 "..Q."],

 ["..Q.",  // Solution 2
 "Q...",
 "...Q",
 ".Q.."]
 ]
 */
public class _51 {

  public static class Solution1 {

    public List<List<String>> solveNQueens(int n) {
      List<List<String>> result = new ArrayList<>();
      if (n <= 0) {
        return result;
      }
      search(n, new ArrayList<>(), result);
      return result;
    }

    private void search(int n, ArrayList<Integer> col, List<List<String>> result) {
      if (col.size() == n) {
        result.add(drawChessBoard(col));
        return;
      }

      for (int i = 0; i < n; i++) {
        if (!isValid(col, i)) {
```

```
          continue;
        }
        col.add(i);
        search(n, col, result);
        col.remove(col.size() - 1);
      }
    }

    private boolean isValid(ArrayList<Integer> col, int next) {
      int row = col.size();
      for (int i = 0; i < row; i++) {
        if (next == col.get(i)) {
          return false;
        }

        if (i - row == col.get(i) - next) {
          return false;
        }

        if (i - row == next - col.get(i)) {
          return false;
        }
      }
      return true;
    }

    private ArrayList<String> drawChessBoard(ArrayList<Integer> col) {
      ArrayList<String> chessBoard = new ArrayList<>();

      for (int i = 0; i < col.size(); i++) {
        String row = "";
        for (int j = 0; j < col.size(); j++) {
          if (col.get(j) == i) {
            row += "Q";
          } else {
            row += ".";
          }
        }
        chessBoard.add(row);
      }
      return chessBoard;
    }
  }
}
```

52. `_52.java`

```
package com.fishercoder.solutions;

/**
 * 52. N-Queens II
 *
 * Follow up for N-Queens problem.
 * Now, instead outputting board configurations, return the total number of distinct
solutions.
 */
public class _52 {

  public static class Solution1 {
    /**credit: https://discuss.leetcode.com/topic/29626/easiest-java-solution-1ms-98-22*/
    int count = 0;

    public int totalNQueens(int n) {
      boolean[] cols = new boolean[n];
      boolean[] diagnol = new boolean[2 * n];
      boolean[] antiDiagnol = new boolean[2 * n];
      backtracking(0, cols, diagnol, antiDiagnol, n);
      return count;
    }

    private void backtracking(int row, boolean[] cols, boolean[] diagnol, boolean[]
antiDiagnol,
        int n) {
      if (row == n) {
        count++;
      }
      for (int col = 0; col < n; col++) {
        int x = col - row + n;
        int y = col + row;
        if (cols[col] || diagnol[x] || antiDiagnol[y]) {
          continue;
        }
        cols[col] = true;
        diagnol[x] = true;
        antiDiagnol[y] = true;
        backtracking(row + 1, cols, diagnol, antiDiagnol, n);
        cols[col] = false;
        diagnol[x] = false;
        antiDiagnol[y] = false;
      }
    }
  }
}
```

53. `_53.java`

```
package com.fishercoder.solutions;

/**
 * 53. Maximum Subarray
 *
 * Find the contiguous subarray within an array (containing at least one number) which
has the largest sum.
 * For example, given the array [-2,1,-3,4,-1,2,1,-5,4], the contiguous subarray
[4,-1,2,1] has the largest sum = 6.
 */

public class _53 {

  public static class Solution1 {
    /**credit: https://discuss.leetcode.com/topic/5000/accepted-o-n-solution-in-java*/
    public int maxSubArray(int[] nums) {
      int maxSoFar = nums[0];
      int maxEndingHere = nums[0];
      for (int i = 1; i < nums.length; i++) {
        maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);
        maxSoFar = Math.max(maxEndingHere, maxSoFar);
      }
      return maxSoFar;
    }
  }
}
```

54. `_54.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 54. Spiral Matrix
 *
 * Given a matrix of m x n elements (m rows, n columns), return all elements of the
matrix in spiral order.

 For example,
 Given the following matrix:

 [
 [ 1, 2, 3 ],
 [ 4, 5, 6 ],
 [ 7, 8, 9 ]
 ]

 You should return [1,2,3,6,9,8,7,4,5].
 */
public class _54 {

  public static class Solution1 {
    /**credit: https://leetcode.com/problems/spiral-matrix/discuss/20599/Super-Simple-
and-Easy-to-Understand-Solution/185257*/
    public List<Integer> spiralOrder(int[][] matrix) {
      if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
        return new ArrayList<>();
      }
      int m = matrix.length;
      int n = matrix[0].length;
      List<Integer> result = new ArrayList();
      int left = 0;
      int right = n - 1;
      int top = 0;
      int bottom = m - 1;
      while (result.size() < m * n) {
        for (int j = left; j <= right && result.size() < m * n; j++) {
          result.add(matrix[top][j]);
        }
        top++;
        for (int i = top; i <= bottom && result.size() < m * n; i++) {
          result.add(matrix[i][right]);
        }
        right--;
        for (int j = right; j >= left && result.size() < m * n; j--) {
          result.add(matrix[bottom][j]);
        }
        bottom--;
```

```
        for (int i = bottom; i >= top && result.size() < m * n; i--) {
          result.add(matrix[i][left]);
        }
        left++;
      }
      return result;
    }
  }
}
```

55. `_55.java`

```
package com.fishercoder.solutions;

/**
 * 55. Jump Game
 *
 * Given an array of non-negative integers, you are initially positioned at the first
index of the array.
 * Each element in the array represents your maximum jump length at that position.
 * Determine if you are able to reach the last index.
 *
 * For example:
 * A = [2,3,1,1,4], return true.
 * A = [3,2,1,0,4], return false.*/
public class _55 {

    public static class Solution1 {
        public boolean canJump(int[] nums) {
            int farthest = nums[0];
            for (int i = 0; i < nums.length; i++) {
                if (i <= farthest && nums[i] + i > farthest) {
                    //i <= farthest is to make sure that this current i is within the
current range
                    // nums[i]+i > farthest is to make sure that it's necessary to update
farthest with current nums[i]+i
                    farthest = nums[i] + i;
                }
            }
            return farthest >= nums.length - 1;
        }
    }
}
```

56. `_56.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 56. Merge Intervals
 *
 * Given a collection of intervals, merge all overlapping intervals.

 For example,
 Given [1,3],[2,6],[8,10],[15,18],
 return [1,6],[8,10],[15,18].
 */
public class _56 {

  public static class Solution1 {
    public List<Interval> merge(List<Interval> intervals) {
        if (intervals.size() <= 1) {
            return intervals;
        }

        Collections.sort(intervals, (o1, o2) -> o1.start - o2.start);

        List<Interval> result = new ArrayList();
        for (int i = 0; i < intervals.size(); i++) {
            int start = intervals.get(i).start;
            int end = intervals.get(i).end;
            while (i < intervals.size() && end >= intervals.get(i).start) {
                end = Math.max(end, intervals.get(i).end);
                i++;
            }
            result.add(new Interval(start, end));
            i--;
        }
        return result;
    }
  }

}
```

57. `_57.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.ArrayList;
import java.util.List;

/**
 * 57. Insert Interval
 *
 * Given a set of non-overlapping intervals, insert a new interval into the intervals
(merge if necessary).

 You may assume that the intervals were initially sorted according to their start times.

 Example 1:
 Given intervals [1,3],[6,9], insert and merge [2,5] in as [1,5],[6,9].

 Example 2:
 Given [1,2],[3,5],[6,7],[8,10],[12,16], insert and merge [4,9] in as [1,2],[3,10],
[12,16].

 This is because the new interval [4,9] overlaps with [3,5],[6,7],[8,10].
 */
public class _57 {

  public static class Solution1 {
    public List<Interval> insert(List<Interval> intervals, Interval newInterval) {
      List<Interval> result = new ArrayList<>();
      int i = 0;
      // add all the intervals ending before newInterval starts
      while (i < intervals.size() && intervals.get(i).end < newInterval.start) {
        result.add(intervals.get(i++));
      }
      // merge all overlapping intervals to one considering newInterval
      while (i < intervals.size() && intervals.get(i).start <= newInterval.end) {
        newInterval = new Interval( // we could mutate newInterval here also
            Math.min(newInterval.start, intervals.get(i).start),
            Math.max(newInterval.end, intervals.get(i).end));
        i++;
      }
      result.add(newInterval);
      // add all the rest
      while (i < intervals.size()) {
        result.add(intervals.get(i++));
      }
      return result;
    }
  }
}
```

58. `_58.java`

```
package com.fishercoder.solutions;
/**
 * 58. Length of Last Word
 *
 * Given a string s consists of upper/lower-case alphabets and empty space characters '
', return the length of last word in the string.
 * If the last word does not exist, return 0.

 Note: A word is defined as a character sequence consists of non-space characters only.

 For example,
 Given s = "Hello World",
 return 5.

 */
public class _58 {

    public static class Solution1 {
        public int lengthOfLastWord(String s) {
            if (s == null || s.length() == 0) {
                return 0;
            }
            s = s.trim();
            int n = s.length() - 1;
            while (n >= 0 && s.charAt(n) != ' ') {
                n--;
            }
            return s.length() - n - 1;
        }
    }

}
```

59. `_59.java`

```
package com.fishercoder.solutions;

/**
 * 59. Spiral Matrix II
 *
 * Given an integer n, generate a square matrix filled with elements from 1 to n2 in
spiral order.

 For example,
 Given n = 3,

 You should return the following matrix:
 [
 [ 1, 2, 3 ],
 [ 8, 9, 4 ],
 [ 7, 6, 5 ]
 ]
 */
public class _59 {

  public static class Solution1 {
    /**credit: https://leetcode.com/problems/spiral-matrix-ii/discuss/22289/My-Super-
Simple-Solution.-Can-be-used-for-both-Spiral-Matrix-I-and-II/21907*/
    public int[][] generateMatrix(int n) {
      int[][] matrix = new int[n][n];
      if (n == 0) {
        return matrix;
      }
      int value = 1;
      int top = 0;
      int bottom = n - 1;
      int left = 0;
      int right = n - 1;
      while (left <= right && top <= bottom) {
        for (int j = left; j <= right; j++) {
          matrix[top][j] = value++;
        }
        top++;
        for (int i = top; i <= bottom; i++) {
          matrix[i][right] = value++;
        }
        right--;
        for (int j = right; j >= left; j--) {
          matrix[bottom][j] = value++;
        }
        bottom--;
        for (int i = bottom; i >= top; i--) {
          matrix[i][left] = value++;
        }
        left++;
      }
```

```
      return matrix;
    }
  }
}
```

60. `_60.java`

```
package com.fishercoder.solutions;

/**
 * 60. Permutation Sequence
 *
 * The set [1,2,3,…,n] contains a total of n! unique permutations.

 By listing and labeling all of the permutations in order,
 We get the following sequence (ie, for n = 3):

 "123"
 "132"
 "213"
 "231"
 "312"
 "321"
 Given n and k, return the kth permutation sequence.

 Note: Given n will be between 1 and 9 inclusive.
 */
public class _60 {

    public static class Solution1 {
        public String getPermutation(int n, int k) {
            int[] nums = new int[n + 1];
            int permcount = 1;
            for (int i = 0; i < n; i++) {
                nums[i] = i + 1; // put 1, 2, 3 ... n into nums[]
                permcount *= (i + 1);
            }

            k--;
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < n; i++) {
                permcount = permcount / (n - i);
                int idx = k / permcount;// the index that this position should
                // choose
                sb.append(nums[idx]);
                // left shift nums[] by one bit
                for (int j = idx; j < n - i; j++) {
                    nums[j] = nums[j + 1];
                }
                k %= permcount;
            }
            return sb.toString();
        }
    }
}
```

61. `_61.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 61. Rotate List
 *
 * Given a list, rotate_naive the list to the right by k places, where k is non-negative.

 For example:
 Given 1->2->3->4->5->NULL and k = 2,
 return 4->5->1->2->3->NULL.
 */
public class _61 {

  public static class Solution1 {
    //credit: https://discuss.leetcode.com/topic/26364/clean-java-solution-with-brief-
explanation
    //link the tail of the linked list to the head to form a circle, then count to find
the pint and cut it
    public ListNode rotateRight(ListNode head, int k) {
      if (head == null) {
        return head;
      }
      ListNode copyHead = head;
      int len = 1;
      while (copyHead.next != null) {
        copyHead = copyHead.next;
        len++;
      }
      copyHead.next = head;//link the tail and head to make it a circle
      for (int i = len - k % len; i > 1; i--) {
        head = head.next;
      }
      copyHead = head.next;
      head.next = null;//break the circle
      return copyHead;
    }
  }
}
```

62. `_62.java`

```
package com.fishercoder.solutions;

/**
 * 62. Unique Paths

A robot is located at the top-left corner of a m x n grid (marked 'Start' in the diagram below).
The robot can only move either down or right at any point in time. The robot is trying to reach
the bottom-right corner of the grid (marked 'Finish' in the diagram below).

How many possible unique paths are there?
 */
public class _62 {

  public static class Solution1 {
    /**
     * Another typical DP question, use a 2d array: the first row and the first column need to be
     * initialized to be 1 since there's only one way to reach every position in the first row and
     * the first column: either from left or top.
     */
    public int uniquePaths(int m, int n) {
      int[][] dp = new int[m][n];
      for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
          if (i == 0 || j == 0) {
            dp[i][j] = 1;
          } else {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
          }
        }
      }
      return dp[m - 1][n - 1];
    }
  }
}
```

63. `_63.java`

```
package com.fishercoder.solutions;

/**
 * 63. Unique Paths II

 Follow up for "Unique Paths":

Now consider if some obstacles are added to the grids. How many unique paths would there be?

An obstacle and empty space is marked as 1 and 0 respectively in the grid.

For example,
There is one obstacle in the middle of a 3x3 grid as illustrated below.

[
  [0,0,0],
  [0,1,0],
  [0,0,0]
]
The total number of unique paths is 2.

Note: m and n will be at most 100.*/
public class _63 {
  public static class Solution1 {
    /**
     * Idea: grid[i][j] has to be set to zero if obstacleGrid[i][j] == 1, otherwise, we can get
     * dp[i][j] from its top and left dp.
     */
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
      if (obstacleGrid == null || obstacleGrid.length == 0) {
        return 0;
      }

      int height = obstacleGrid.length;
      int width = obstacleGrid[0].length;
      int[][] dp = new int[height][width];
      dp[0][0] = obstacleGrid[0][0] == 1 ? 0 : 1;
      for (int i = 1; i < height; i++) {
        dp[i][0] = obstacleGrid[i][0] == 1 ? 0 : dp[i - 1][0];
      }
      for (int j = 1; j < width; j++) {
        dp[0][j] = obstacleGrid[0][j] == 1 ? 0 : dp[0][j - 1];
      }

      for (int i = 1; i < height; i++) {
        for (int j = 1; j < width; j++) {
          if (obstacleGrid[i][j] == 1) {
            dp[i][j] = 0;
          } else {
```

```
            int paths = 0;
            if (obstacleGrid[i - 1][j] == 0) {
              paths += dp[i - 1][j];
            }
            if (obstacleGrid[i][j - 1] == 0) {
              paths += dp[i][j - 1];
            }
            dp[i][j] = paths;
          }
        }
      }
      return dp[height - 1][width - 1];
    }
  }
}
```

64. `_64.java`

```
package com.fishercoder.solutions;

/**64. Minimum Path Sum

Given a m x n grid filled with non-negative numbers, find a path from top left to bottom
right which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.*/
public class _64 {
  public static class Solution1 {
    /**
     * Same idea as _70: have to initialize the first row and the first column and start
the for
     * loop from i==1 and j==1 for the rest of the matrix.
     */
    public int minPathSum(int[][] grid) {
      if (grid == null || grid.length == 0) {
        return 0;
      }

      int height = grid.length;
      int width = grid[0].length;
      int[][] dp = new int[height][width];
      dp[0][0] = grid[0][0];
      for (int i = 1; i < height; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
      }
      for (int j = 1; j < width; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
      }
      for (int i = 1; i < height; i++) {
        for (int j = 1; j < width; j++) {
          dp[i][j] = Math.min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j];
        }
      }
      return dp[height - 1][width - 1];
    }
  }
}
```

65. `_65.java`

```
package com.fishercoder.solutions;

/**
 * 65. Valid Number
 *
 * Validate if a given string is numeric.
 *
 * Some examples: "0" => true " 0.1 " => true "abc" => false "1 a" => false "2e10" =>
true
 *
 * Note: It is intended for the problem statement to be ambiguous. You should gather all
 * requirements up front before implementing one.
 */

public class _65 {
  /**credit: https://discuss.leetcode.com/topic/9490/clear-java-solution-with-ifs*/
  public static class Solution1 {
    public boolean isNumber(String s) {
      s = s.trim();

      boolean pointSeen = false;
      boolean eSeen = false;
      boolean numberSeen = false;
      boolean numberAfterE = true;
      for (int i = 0; i < s.length(); i++) {
        if ('0' <= s.charAt(i) && s.charAt(i) <= '9') {
          numberSeen = true;
          numberAfterE = true;
        } else if (s.charAt(i) == '.') {
          if (eSeen || pointSeen) {
            return false;
          }
          pointSeen = true;
        } else if (s.charAt(i) == 'e') {
          if (eSeen || !numberSeen) {
            return false;
          }
          numberAfterE = false;
          eSeen = true;
        } else if (s.charAt(i) == '-' || s.charAt(i) == '+') {
          if (i != 0 && s.charAt(i - 1) != 'e') {
            return false;
          }
        } else {
          return false;
        }
      }

      return numberSeen && numberAfterE;
    }
  }
```

```
  public static class Solution2 {
    /** credit: https://discuss.leetcode.com/topic/2973/java-solution-with-one-line */
    public boolean isNumber(String s) {
      return s.matches("(\\s*)[+-]?((\\.[0-9]+)|([0-9]+(\\.[0-9]*)?))(e[+-]?[0-9]+)?
(\\s*)");
    }
  }
}
```

66. `_66.java`

```
package com.fishercoder.solutions;

/**
 * 66. Plus One
 *
 * Given a non-empty array of digits representing a non-negative integer, plus one to the
integer.
 * The digits are stored such that the most significant digit is at the head of the list,
 * and each element in the array contain a single digit.
 *
 * You may assume the integer does not contain any leading zero, except the number 0
itself.
 *
 * Example 1:
 * Input: [1,2,3]
 * Output: [1,2,4]
 * Explanation: The array represents the integer 123.
 *
 * Example 2:
 * Input: [4,3,2,1]
 * Output: [4,3,2,2]
 * Explanation: The array represents the integer 4321.
 */
public class _66 {

  public static class Solution1 {
      /**credit: https://leetcode.com/problems/plus-one/discuss/24082/My-Simple-Java-
Solution*/
    public int[] plusOne(int[] digits) {
        int len = digits.length;
        for (int i = len - 1; i >= 0; i--) {
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        int[] newNumber = new int[len + 1];
        newNumber[0] = 1;
        return newNumber;
    }
  }
}
```

67. `_67.java`

```
package com.fishercoder.solutions;

/**
 * 67. Add Binary
 *
 * Given two binary strings, return their sum (also a binary string).
 * For example,
 * a = "11"
 * b = "1"
 * Return "100".
 */

public class _67 {
  public static class Solution1 {
    /**
     * credit: https://discuss.leetcode.com/topic/13698/short-ac-solution-in-java-with-explanation
     * 1. use StringBuilder.reverse() function! Nice!
     * 2. if a numeric number is represented/stored in String, how to get its value: use Character.getNumericValue(s.charAt(i))
     * 3. directly adding/subtracting chars will end up working with their ASCII numbers, e.g. chars[0] = 'a', chars[1] = 'b', then chars[0] + chars[1] will become 195.
     */
    public String addBinary(String a, String b) {
      int carry = 0;
      int i = a.length() - 1;
      int j = b.length() - 1;
      StringBuilder sb = new StringBuilder();
      while (i >= 0 || j >= 0) {
        int sum = carry;
        if (i >= 0) {
          sum += a.charAt(i--) - '0';
        }
        if (j >= 0) {
          sum += b.charAt(j--) - '0';
        }
        sb.append(sum % 2);
        carry = sum / 2;
      }
      if (carry != 0) {
        sb.append(carry);
      }
      return sb.reverse().toString();
    }
  }
}
```

68. `_68.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 68. Text Justification

 Given an array of words and a length L, format the text such that each line has exactly
L characters and is fully (left and right) justified.
 You should pack your words in a greedy approach; that is, pack as many words as you can
in each line.
 Pad extra spaces ' ' when necessary so that each line has exactly L characters.
 Extra spaces between words should be distributed as evenly as possible.
 If the number of spaces on a line do not divide evenly between words,
 the empty slots on the left will be assigned more spaces than the slots on the right.
 For the last line of text, it should be left justified and no extra space is inserted
between words.

 For example,
 words: ["This", "is", "an", "example", "of", "text", "justification."]
 L: 16.

 Return the formatted lines as:
 [
 "This    is    an",
 "example  of text",
 "justification.  "
 ]

 Note: Each word is guaranteed not to exceed L in length.

 Corner Cases:
 A line other than the last line might contain only one word. What should you do in this
case?
 In this case, that line should be left-justified.
 */
public class _68 {

  public static class Solution1 {
    public List<String> fullJustify(String[] words, int L) {
        ArrayList<String> result = new ArrayList();
        if (words == null || words.length == 0) {
            return result;
        }
        int count = 0;
        int last = 0;
        for (int i = 0; i < words.length; i++) {
            if (count + words[i].length() + (i - last) > L) {
                int spaceNum = 0;
                int extraNum = 0;
```

```
                if (i - last - 1 > 0) {
                    spaceNum = (L - count) / (i - last - 1);
                    extraNum = (L - count) % (i - last - 1);
                }
                StringBuilder sb = new StringBuilder();
                for (int j = last; j < i; j++) {
                    sb.append(words[j]);
                    if (j < i - 1) {
                        for (int k = 0; k < spaceNum; k++) {
                            sb.append(" ");
                        }
                        if (extraNum > 0) {
                            sb.append(" ");
                        }
                        extraNum--;
                    }
                }
                for (int j = sb.length(); j < L; j++) {
                    sb.append(" ");
                }
                result.add(sb.toString());
                count = 0;
                last = i;
            }
            count += words[i].length();
        }
        StringBuilder sb = new StringBuilder();
        for (int i = last; i < words.length; i++) {
            sb.append(words[i]);
            if (sb.length() < L) {
                sb.append(" ");
            }
        }
        for (int i = sb.length(); i < L; i++) {
            sb.append(" ");
        }
        result.add(sb.toString());
        return result;
    }
  }

}
```

69. `_69.java`

```
package com.fishercoder.solutions;

/**
 * 69. Sqrt(x)
 *
 * Implement int sqrt(int x).
 * Compute and return the square root of x.
 */

public class _69 {
    public static class Solution1 {
        /**A few key points:
         * 1. all variable use long type, otherwise overflow, just cast to int before
returning
         * 2. left start from 0, not 1
         * 3. right start from x/2 + 1, not from x*/
        public int mySqrt(int x) {
            long left = 0;
            long right = x / 2 + 1;
            while (left <= right) {
                long mid = left + (right - left) / 2;
                long result = mid * mid;
                if (result == (long) x) {
                    return (int) mid;
                } else if (result > x) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }
            return (int) right;
        }
    }
}
```

70. `_70.java`

```
package com.fishercoder.solutions;

/**
 * 70. Climbing Stairs

 You are climbing a stair case. It takes n steps to reach to the top.

 Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

 Note: Given n will be a positive integer.

 Example 1:

 Input: 2
 Output:  2
 Explanation:  There are two ways to climb to the top.

 1. 1 step + 1 step
 2. 2 steps

 Example 2:

 Input: 3
 Output:  3
 Explanation:  There are three ways to climb to the top.

 1. 1 step + 1 step + 1 step
 2. 1 step + 2 steps
 3. 2 steps + 1 step

 */

public class _70 {
  public static class Solution1 {
    //classical dp problem
    public int climbStairs(int n) {
      if (n < 1) {
        return 0;
      } else if (n < 4) {
        return n;
      } else {
        int[] dp = new int[n + 1];
        //the number of ways to reach step n could be calculated from n-1 and n-2
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
          dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
      }
```

```
    }
  }
}
```

71. `_71.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Deque;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Set;

/**
 * 71. Simplify Path

 Given an absolute path for a file (Unix-style), simplify it.

 For example,
 path = "/home/", => "/home"
 path = "/a/./b/../../c/", => "/c"

 Corner Cases:
 Did you consider the case where path = "/../"?
 In this case, you should return "/".
 Another corner case is the path might contain multiple slashes '/' together, such as
"/home//foo/".
 In this case, you should ignore redundant slashes and return "/home/foo".
 */
public class _71 {

  public static class Solution1 {
    public String simplifyPath(String path) {
      Deque<String> stack = new LinkedList<>();
      Set<String> skipSet = new HashSet<>(Arrays.asList("..", ".", ""));
      for (String dir : path.split("/")) {
        if (dir.equals("..") && !stack.isEmpty()) {
          stack.pop();
        } else if (!skipSet.contains(dir)) {
          stack.push(dir);
        }
      }
      String result = "";
      for (String dir : stack) {
        result = "/" + dir + result;
      }
      return result.isEmpty() ? "/" : result;
    }
  }
}
```

72. `_72.java`

```
package com.fishercoder.solutions;

/**
 * 72. Edit Distance
 *
 * Given two words word1 and word2,
 * find the minimum number of steps required to convert word1 to word2.
 * (each operation is counted as 1 step.)

 You have the following 3 operations permitted on a word:

 a) Insert a character
 b) Delete a character
 c) Replace a character
 */

public class _72 {

    public static class Solution1 {
        public int minDistance(String word1, String word2) {
            int m = word1.length();
            int n = word2.length();
            if (m == 0) {
                return n;
            }
            if (n == 0) {
                return m;
            }
            char[] str1 = word1.toCharArray();
            char[] str2 = word2.toCharArray();

            int[][] table = new int[m + 1][n + 1];
            for (int i = 0; i < m + 1; i++) {
                table[i][0] = i;
            }
            for (int j = 0; j < n + 1; j++) {
                table[0][j] = j;
            }
            for (int i = 1; i < m + 1; i++) {
                for (int j = 1; j < n + 1; j++) {
                    int cost = 0;
                    if (str1[i - 1] != str2[j - 1]) {
                        cost = 1;
                    }
                    table[i][j] = Math.min(Math.min(table[i - 1][j] + 1, table[i][j - 1]
+ 1),
                            table[i - 1][j - 1] + cost);
                }
            }
            return table[m][n];
        }
```

```
    }
}
```

73. `_73.java`

```
package com.fishercoder.solutions;

/**
 * 73. Set Matrix Zeroes
 *
 * Given a m x n matrix, if an element is 0, set its entire row and column to 0. Do it in
place.

 Follow up:

 Did you use extra space?
 A straight forward solution using O(mn) space is probably a bad idea.
 A simple improvement uses O(m + n) space, but still not the best solution.
 Could you devise a constant space solution?
 */
public class _73 {

    public static class Solution1 {
        /**
         * Space: O(m*n)
         */
        public void setZeroes(int[][] matrix) {
            if (matrix == null || matrix.length == 0) {
                return;
            }
            int height = matrix.length;
            int width = matrix[0].length;
            boolean[][] zero = new boolean[height][width];
            for (int i = 0; i < height; i++) {
                for (int j = 0; j < width; j++) {
                    if (matrix[i][j] == 0) {
                        zero[i][j] = true;
                    }
                }
            }
            for (int i = 0; i < height; i++) {
                for (int j = 0; j < width; j++) {
                    if (zero[i][j]) {
                        for (int k = 0; k < height; k++) {
                            matrix[k][j] = 0;
                        }
                        for (int k = 0; k < width; k++) {
                            matrix[i][k] = 0;
                        }
                    }
                }
            }
        }
    }

    public static class Solution2 {
```

```
    /**
     * Space: O(m+n)
     */
    public void setZeroes(int[][] matrix) {
        if (matrix == null || matrix.length == 0) {
            return;
        }
        int m = matrix.length;
        int n = matrix[0].length;
        boolean[] row = new boolean[m];
        boolean[] col = new boolean[n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    row[i] = true;
                    col[j] = true;
                }
            }
        }

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (row[i] && col[j]) {
                    for (int k = 0; k < m; k++) {
                        matrix[k][j] = 0;
                    }
                    for (int k = 0; k < n; k++) {
                        matrix[i][k] = 0;
                    }
                }
            }
        }
    }
}

public static class Solution3 {
    /**
     * Space: O(1)
     */
    public void setZeroes(int[][] matrix) {
        if (matrix == null || matrix.length == 0) {
            return;
        }
        int m = matrix.length;
        int n = matrix[0].length;
        boolean firstRow = false;
        boolean firstCol = false;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    if (i == 0) {
                        firstRow = true;
                    }
                    if (j == 0) {
```

```
                        firstCol = true;
                    }
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }

        if (firstRow) {
            for (int j = 0; j < n; j++) {
                matrix[0][j] = 0;
            }
        }

        if (firstCol) {
            for (int i = 0; i < m; i++) {
                matrix[i][0] = 0;
            }
        }
    }
}

public static class Solution4 {
    /**
     * Space: O(1)
     * credit: https://leetcode.com/problems/set-matrix-zeroes/discuss/26014/Any-
shorter-O(1)-space-solution
     */
    public void setZeroes(int[][] matrix) {
        int col0 = 1;
        int m = matrix.length;
        int n = matrix[0].length;
        /**the first iteration (first nested for loop) is to check from top row to
bottom row:
         * keep the first column state into variable col0;
         * then starting from the second column, check all the rest of the columns
and mark its top cell and its most-left cell if it
         * s a zero.*/
        for (int i = 0; i < m; i++) {
            if (matrix[i][0] == 0) {
                col0 = 0;
            }

            for (int j = 1; j < n; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
```

```
                        matrix[0][j] = 0;
                    }
                }
            }

            /**the second iteration (second nested for loop) is to check from bottom row
to the top row
             * from the right-most column to the second left-most column: as long as its
left-most column cell or its top row cell is zero, then set that cell to be zero
             * at last, check col0 variable, if it's zero, mark that row cell as zero*/
            for (int i = m - 1; i >= 0; i--) {
                for (int j = n - 1; j >= 1; j--) {
                    if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                        matrix[i][j] = 0;
                    }
                }
                if (col0 == 0) {
                    matrix[i][0] = 0;
                }
            }
        }
    }
}
```

74. `_74.java`

```
package com.fishercoder.solutions;

/**
 * 74. Search a 2D Matrix
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix.
 * This matrix has the following properties:

 Integers in each row are sorted from left to right.
 The first integer of each row is greater than the last integer of the previous row.
 For example,

 Consider the following matrix:

 [
 [1,   3,  5,  7],
 [10, 11, 16, 20],
 [23, 30, 34, 50]
 ]

 Given target = 3, return true.

 */
public class _74 {

    public static class Solution1 {
        public boolean searchMatrix(int[][] matrix, int target) {
            if (matrix == null || matrix.length == 0
                || matrix[0].length == 0
                || matrix[0][0] > target
                || matrix[matrix.length - 1][matrix[0].length - 1] < target) {
                return false;
            }
            int m = matrix.length;
            int n = matrix[0].length;
            int left = 0;
            int right = m * n - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;
                int row = mid / n;
                int col = mid % n;
                if (matrix[row][col] == target) {
                    return true;
                } else if (matrix[row][col] > target) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }
            return false;
        }
```

```
    }
}
```

75. `_75.java`

```
package com.fishercoder.solutions;

/**
 * 75. Sort Colors
 *
 * Given an array with n objects colored red, white or blue,
 * sort them so that objects of the same color are adjacent, with the colors in the order
red, white and blue.

 Here, we will use the integers 0, 1, and 2 to represent the color red, white, and blue
respectively.

 Note:
 You are not suppose to use the library's sort function for this problem.

 Follow up:

 A rather straight forward solution is a two-pass algorithm using counting sort.
 First, iterate the array counting number of 0's, 1's, and 2's,
 then overwrite array with total number of 0's, then 1's and followed by 2's.
 Could you come up with an one-pass algorithm using only constant space?
 */

public class _75 {

  public static class Solution1 {
    public void sortColors(int[] nums) {
      int zero = 0;
      int two = nums.length - 1;
      for (int i = 0; i <= two; ) {
        if (nums[i] == 0 && i > zero) {
          swap(nums, i, zero++);
        } else if (nums[i] == 2 && i < two) {
          swap(nums, i, two--);
        } else {
          i++;
        }
      }
    }

    void swap(int[] nums, int m, int n) {
      int temp = nums[m];
      nums[m] = nums[n];
      nums[n] = temp;
    }
  }
}
```

76. `_76.java`

```
package com.fishercoder.solutions;

/**
 * 76. Minimum Window Substring
 *
 * Given a string S and a string T, find the minimum window in S which will contain all
the characters in T in complexity O(n).

 For example,
 S = "ADOBECODEBANC"
 T = "ABC"

 Minimum window is "BANC".

 Note:

 If there is no such window in S that covers all characters in T, return the empty string
"".
 If there are multiple such windows, you are guaranteed that there will always be only
one unique minimum window in S.
 */

public class _76 {

  public static class Solution1 {
    public String minWindow(String s, String t) {
      int[] counts = new int[256];
      for (char c : t.toCharArray()) {
        counts[c]++;
      }

      int start = 0;
      int end = 0;
      int minStart = 0;
      int minLen = Integer.MAX_VALUE;
      int counter = t.length();
      while (end < s.length()) {
        if (counts[s.charAt(end)] > 0) {
          counter--;
        }

        counts[s.charAt(end)]--;
        end++;

        while (counter == 0) {
          if (end - start < minLen) {
            minStart = start;
            minLen = end - start;
          }
          counts[s.charAt(start)]++;
          if (counts[s.charAt(start)] > 0) {
```

```
            counter++;
          }
          start++;
        }
      }

      if (minLen == Integer.MAX_VALUE) {
        return "";
      }
      return s.substring(minStart, minStart + minLen);
    }
  }
}
```

77. `_77.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 77. Combinations
 *
 * Given two integers n and k, return all possible combinations of k numbers out of 1 ...
n.

 For example,
 If n = 4 and k = 2, a solution is:

 [
 [2,4],
 [3,4],
 [2,3],
 [1,2],
 [1,3],
 [1,4],
 ]
 */

public class _77 {

    public static class Solution1 {
        public List<List<Integer>> combine(int n, int k) {
            List<List<Integer>> result = new ArrayList();
            int[] nums = new int[n];
            for (int i = 0; i < n; i++) {
                nums[i] = i + 1;
            }
            backtracking(k, 0, nums, new ArrayList(), result);
            return result;
        }

        void backtracking(int k, int start, int[] nums, List<Integer> curr,
List<List<Integer>> result) {
            if (curr.size() == k) {
                result.add(new ArrayList(curr));
            } else if (curr.size() < k) {
                for (int i = start; i < nums.length; i++) {
                    curr.add(nums[i]);
                    backtracking(k, i + 1, nums, curr, result);
                    curr.remove(curr.size() - 1);
                }
            }
        }
    }
}
```

78. `_78.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 78. Subsets
 *
 * Given a set of distinct integers, nums, return all possible subsets.
 * Note: The solution set must not contain duplicate subsets.

 For example,
 If nums = [1,2,3], a solution is:
 [
 [3],
 [1],
 [2],
 [1,2,3],
 [1,3],
 [2,3],
 [1,2],
 []
 ]
 */

public class _78 {

    public static class Solution1 {
        public List<List<Integer>> subsets(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null) {
                return result;
            }
            result.add(new ArrayList());
            for (int i = 0; i < nums.length; i++) {
                List<List<Integer>> temp = new ArrayList();
                //you'll have to create a new one here, otherwise, it'll throw
ConcurrentModificationException.
                for (List<Integer> list : result) {
                    List<Integer> newList = new ArrayList(list);
                    newList.add(nums[i]);
                    temp.add(newList);
                }
                result.addAll(temp);
            }
            return result;
        }
    }

    public static class Solution2 {
        /**This is the most straightforward solution and easy to follow.*/
```

```
        public List<List<Integer>> subsets(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            backtracking(result, new ArrayList(), nums, 0);
            return result;
        }

        void backtracking(List<List<Integer>> result, List<Integer> list, int[] nums, int
start) {
            result.add(new ArrayList(list));
            for (int i = start; i < nums.length; i++) {
                list.add(nums[i]);
                backtracking(result, list, nums, i + 1);
                list.remove(list.size() - 1);
            }
        }
    }

    public static class Solution3 {
        /**
         * This is just a slight modification of Solution2, pay close to attention to
notice the difference between them.
         */
        public List<List<Integer>> subsets(int[] nums) {
            List<List<Integer>> result = new ArrayList<>();
            List<Integer> list = new ArrayList<>();
            result.add(list);
            backtracking(result, list, nums, 0);
            return result;
        }

        private void backtracking(List<List<Integer>> result, List<Integer> list, int[]
nums, int start) {
            for (int i = start; i < nums.length; i++) {
                list.add(nums[i]);
                result.add(new ArrayList<>(list));
                backtracking(result, list, nums, i + 1);
                list.remove(list.size() - 1);
            }
        }
    }
}
```

79. `_79.java`

```
package com.fishercoder.solutions;

/**
 * 79. Word Search
 *
 * Given a 2D board and a word, find if the word exists in the grid.
 * The word can be constructed from letters of sequentially adjacent cell,
 * where "adjacent" cells are those horizontally or vertically neighboring.
 * The same letter cell may not be used more than once.

 For example,
 Given board =
 [
 ['A','B','C','E'],
 ['S','F','C','S'],
 ['A','D','E','E']
 ]

 word = "ABCCED", -> returns true,
 word = "SEE", -> returns true,
 word = "ABCB", -> returns false.
 */

public class _79 {

    public static class Solution1 {

      public boolean exist(char[][] board, String word) {
        int m = board.length;
        int n = board[0].length;
        boolean[][] visited = new boolean[m][n];
        for (int i = 0; i < m; i++) {
          for (int j = 0; j < n; j++) {
            if (dfs(board, visited, i, j, word, 0)) {
              return true;
            }
          }
        }
        return false;
      }

      final int[] dirs = new int[] {0, 1, 0, -1, 0};

      boolean dfs(char[][] board, boolean[][] visited, int row, int col, String word, int
index) {
        if (index >= word.length() || word.charAt(index) != board[row][col]) {
          return false;
        } else if (index == word.length() - 1 && word.charAt(index) == board[row][col]) {
          visited[row][col] = true;
          return true;
        }
```

```
        visited[row][col] = true;//set it to true for this case
        boolean result = false;
        for (int i = 0; i < 4; i++) {
          int nextRow = row + dirs[i];
          int nextCol = col + dirs[i + 1];
          if (nextRow < 0 || nextRow >= board.length || nextCol < 0 || nextCol >=
board[0].length || visited[nextRow][nextCol]) {
            continue;
          }
          result = dfs(board, visited, nextRow, nextCol, word, index + 1);
          if (result) {
            return result;
          } else {
            visited[nextRow][nextCol] = false;//set it back to false if this road doesn't
work to allow it for other paths, this is backtracking!!!
          }
        }
        return result;
      }
    }

    public static class Solution2 {
        //credit: https://discuss.leetcode.com/topic/21142/my-java-solution

        boolean[][] visited;

        public boolean exist(char[][] board, String word) {
            int m = board.length;
            int n = board[0].length;
            visited = new boolean[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (word.charAt(0) == board[i][j] && search(board, word, i, j, 0)) {
                        return true;
                    }
                }
            }
            return false;
        }

        boolean search(char[][] board, String word, int i, int j, int pos) {
            if (pos == word.length()) {
                return true;
            }
            if (i < 0 || j < 0 || i >= board.length || j >= board[0].length ||
word.charAt(pos) != board[i][j] || visited[i][j]) {
                return false;
            }
            visited[i][j] = true;
            if (search(board, word, i + 1, j, pos + 1)
                || search(board, word, i - 1, j, pos + 1)
                || search(board, word, i, j + 1, pos + 1)
                || search(board, word, i, j - 1, pos + 1)) {
                return true;
```

```
            }

            visited[i][j] = false;
            return false;
        }
    }
}
```

80. `_80.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 80. Remove Duplicates from Sorted Array II
 *
 * Follow up for "Remove Duplicates":
 What if duplicates are allowed at most twice?

 For example,
 Given sorted array nums = [1,1,1,2,2,3],

 Your function should return length = 5,
 with the first five elements of nums being 1, 1, 2, 2 and 3. It doesn't matter what you
leave beyond the new length.
 */
public class _80 {

  public static class Solution1 {
    public int removeDuplicates(int[] nums) {
      int counter = 0;
      int len = nums.length;
      if (len == 0) {
        return 0;
      }
      if (len == 1) {
        return 1;
      }
      if (len == 2) {
        return 2;
      }

      List<Integer> a = new ArrayList();
      a.add(nums[0]);
      a.add(nums[1]);
      for (int i = 2; i < len; i++) {
        if (nums[i] != nums[i - 1]) {
          a.add(nums[i]);
        } else if (nums[i] != nums[i - 2]) {
          a.add(nums[i]);
        }
      }

      counter = a.size();
      for (int i = 0; i < counter; i++) {
        nums[i] = a.get(i);
      }
      return counter;
    }
```

```
  }

}
```

81. `_81.java`

```
package com.fishercoder.solutions;

/**
 * 81. Search in Rotated Sorted Array II
 *
 * Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.
 *
 * (i.e., [0,0,1,2,2,5,6] might become [2,5,6,0,0,1,2]).
 *
 * You are given a target value to search. If found in the array return true, otherwise
return false.
 *
 * Example 1:
 *
 * Input: nums = [2,5,6,0,0,1,2], target = 0
 * Output: true
 * Example 2:
 *
 * Input: nums = [2,5,6,0,0,1,2], target = 3
 * Output: false
 * Follow up:
 *
 * This is a follow up problem to Search in Rotated Sorted Array, where nums may contain
duplicates.
 * Would this affect the run-time complexity? How and why?
 */
public class _81 {

  public static class Solution1 {
    public boolean search(int[] nums, int target) {
      int start = 0;
      int end = nums.length - 1;

      //check each num so we will check start == end
      //We always get a sorted part and a half part
      //we can check sorted part to decide where to go next
      while (start <= end) {
        int mid = start + (end - start) / 2;
        if (nums[mid] == target) {
          return true;
        }

        //if left part is sorted
        if (nums[start] < nums[mid]) {
          if (target < nums[start] || target > nums[mid]) {
            //target is in rotated part
            start = mid + 1;
          } else {
            end = mid - 1;
          }
```

```
        } else if (nums[start] > nums[mid]) {
          //right part is rotated

          //target is in rotated part
          if (target < nums[mid] || target > nums[end]) {
            end = mid - 1;
          } else {
            start = mid + 1;
          }
        } else {
          //duplicates, we know nums[mid] != target, so nums[start] != target
          //based on current information, we can only move left pointer to skip one cell
          //thus in the worst case, we would have target: 2, and array like 11111111,
then
          //the running time would be O(n)
          start++;
        }
      }
      return false;
    }
  }
}
```

82. `_82.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 82. Remove Duplicates from Sorted List II
 *
 *  Given a sorted linked list, delete all nodes that have duplicate numbers, leaving
only distinct numbers from the original list.

 For example,
 Given 1->2->3->3->4->4->5, return 1->2->5.
 Given 1->1->1->2->3, return 2->3.

 */
public class _82 {
  public static class Solution1 {
    public ListNode deleteDuplicates(ListNode head) {
      if (head == null) {
        return head;
      }
      ListNode fakeHead = new ListNode(-1);
      fakeHead.next = head;
      ListNode pre = fakeHead;
      ListNode curr = head;
      while (curr != null) {
        while (curr.next != null && curr.val == curr.next.val) {
          curr = curr.next;
        }
        if (pre.next == curr) {
          pre = pre.next;
        } else {
          pre.next = curr.next;
        }
        curr = curr.next;
      }
      return fakeHead.next;
    }
  }

}
```

83. `_83.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 83. Remove Duplicates from Sorted List
 *
 * Given a sorted linked list, delete all duplicates such that each element appear only
once.

 For example,
 Given 1->1->2, return 1->2.
 Given 1->1->2->3->3, return 1->2->3.
 */
public class _83 {
  public static class Solution1 {
    public ListNode deleteDuplicates(ListNode head) {
      ListNode ret = new ListNode(-1);
      ret.next = head;
      while (head != null) {
        while (head.next != null && head.next.val == head.val) {
          head.next = head.next.next;
        }
        head = head.next;
      }
      return ret.next;
    }
  }

  public static class Solution2 {
    public ListNode deleteDuplicates(ListNode head) {
      ListNode curr = head;
      while (curr != null && curr.next != null) {
        if (curr.val == curr.next.val) {
          curr.next = curr.next.next;
        } else {
          curr = curr.next;
        }
      }
      return head;
    }
  }
}
```

84. `_84.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 84. Largest Rectangle in Histogram
 *
 * Given n non-negative integers representing the histogram's bar height where
 * the width of each bar is 1, find the area of largest rectangle in the histogram.

 Above is a histogram where width of each bar is 1, given height = [2,1,5,6,2,3].
 The largest rectangle is shown in the shaded area, which has area = 10 unit.

 For example,
 Given heights = [2,1,5,6,2,3],
 return 10.
 */
public class _84 {

  public static class Solution1 {

    /**
     * credit: https://leetcode.com/articles/largest-rectangle-histogram/#approach-5-
using-stack-accepted
     * and https://discuss.leetcode.com/topic/7599/o-n-stack-based-java-solution
     */
    public int largestRectangleArea(int[] heights) {
      int len = heights.length;
      Stack<Integer> s = new Stack<>();
      int maxArea = 0;
      for (int i = 0; i <= len; i++) {
        int h = (i == len ? 0 : heights[i]);
        if (s.isEmpty() || h >= heights[s.peek()]) {
          s.push(i);
        } else {
          int tp = s.pop();
          maxArea = Math.max(maxArea, heights[tp] * (s.isEmpty() ? i : i - 1 -
s.peek()));
          i--;
        }
      }
      return maxArea;
    }
  }

}
```

85. `_85.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 85. Maximal Rectangle
 *
 * Given a 2D binary matrix filled with 0's and 1's, find the largest rectangle
containing only 1's and return its area.

 For example, given the following matrix:

 1 0 1 0 0
 1 0 1 1 1
 1 1 1 1 1
 1 0 0 1 0
 Return 6.

 */
public class _85 {
  public static class Solution1 {
    public int maximalRectangle(char[][] matrix) {
      if (matrix.length == 0) {
        return 0;
      }
      int m = matrix.length;
      int n = matrix[0].length;
      int[] left = new int[n];
      int[] right = new int[n];
      int[] height = new int[n];
      Arrays.fill(left, 0);
      Arrays.fill(right, n);
      Arrays.fill(height, 0);
      int maxA = 0;
      for (int i = 0; i < m; i++) {
        int currLeft = 0;
        int currRight = n;

        //compute height, this can be achieved from either side
        for (int j = 0; j < n; j++) {
          if (matrix[i][j] == '1') {
            height[j]++;
          } else {
            height[j] = 0;
          }
        }

        //compute left, from left to right
        for (int j = 0; j < n; j++) {
          if (matrix[i][j] == '1') {
            left[j] = Math.max(left[j], currLeft);
```

```
          } else {
            left[j] = 0;
            currLeft = j + 1;
          }
        }

        //compute right, from right to left
        for (int j = n - 1; j >= 0; j--) {
          if (matrix[i][j] == '1') {
            right[j] = Math.min(right[j], currRight);
          } else {
            right[j] = n;
            currRight = j;
          }
        }

        //compute rectangle area, this can be achieved from either side
        for (int j = 0; j < n; j++) {
          maxA = Math.max(maxA, (right[j] - left[j]) * height[j]);
        }
      }
      return maxA;
    }
  }
}
```

86. `_86.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 86. Partition List
 *
 * Given a linked list and a value x, partition it such that all nodes less than x come
before nodes greater than or equal to x.

 You should preserve the original relative order of the nodes in each of the two
partitions.

 For example,
 Given 1->4->3->2->5->2 and x = 3,
 return 1->2->2->4->3->5.
 */
public class _86 {

  public static class Solution1 {
    public ListNode partition(ListNode head, int x) {
      if (head == null || head.next == null) {
        return head;
      }
      ListNode left = new ListNode(0);
      ListNode right = new ListNode(0);
      ListNode less = left;
      ListNode greater = right;
      while (head != null) {
        if (head.val < x) {
          less.next = head;
          less = less.next;
        } else {
          greater.next = head;
          greater = greater.next;
        }
        head = head.next;
      }
      greater.next = null;
      less.next = right.next;
      return left.next;
    }
  }
}
```

87. `_87.java`

```
package com.fishercoder.solutions;

/**
 * 87. Scramble String
 *
 * Given a string s1, we may represent it as a binary tree by partitioning it to two non-
empty substrings recursively.

 Below is one possible representation of s1 = "great":

     great
    /    \
   gr    eat
  / \    /  \
 g   r  e   at
            / \
           a   t
 To scramble the string, we may choose any non-leaf node and swap its two children.

 For example, if we choose the node "gr" and swap its two children, it produces a
scrambled string "rgeat".

     rgeat
    /    \
   rg    eat
  / \    /  \
 r   g  e   at
            / \
           a   t
 We say that "rgeat" is a scrambled string of "great".

 Similarly, if we continue to swap the children of nodes "eat" and "at", it produces a
scrambled string "rgtae".

      rgtae
     /    \
   rg    tae
  / \    /  \
 r   g  ta  e
       / \
      t   a
 We say that "rgtae" is a scrambled string of "great".

 Given two strings s1 and s2 of the same length, determine if s2 is a scrambled string of
s1.
 */
public class _87 {

    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/19158/accepted-java-solution */
        public boolean isScramble(String s1, String s2) {
```

```
          if (s1.equals(s2)) {
            return true;
          }
          if (s1.length() != s2.length()) {
            return false;
          }

          int[] letters = new int[26];
          for (int i = 0; i < s1.length(); i++) {
            letters[s1.charAt(i) - 'a']++;
            letters[s2.charAt(i) - 'a']--;
          }

          for (int i : letters) {
            if (i != 0) {
              return false;
            }
          }

          for (int i = 1; i < s1.length(); i++) {
            if (isScramble(s1.substring(0, i), s2.substring(0, i)) && isScramble(
                s1.substring(i), s2.substring(i))) {
              return true;
            }
            if (isScramble(s1.substring(0, i), s2.substring(s2.length() - i)) &&
isScramble(
                s1.substring(i), s2.substring(0, s2.length() - i))) {
              return true;
            }
          }
          return false;
        }
    }
}
```

88. `_88.java`

```
package com.fishercoder.solutions;

/**
 * 88. Merge Sorted Array
 *
 * Given two sorted integer arrays nums1 and nums2, merge nums2 into nums1 as one sorted
array.
 *
 * Note: You may assume that nums1 has enough space (size that is greater or equal to m +
n) to hold
 * additional elements from nums2. The number of elements initialized in nums1 and nums2
are m and n
 * respectively.
 */

public class _88 {

  public static class Solution1 {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
      int i = m - 1;
      int j = n - 1;
      int k = m + n - 1;
      while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j]) {
          nums1[k--] = nums1[i--];
        } else {
          nums1[k--] = nums2[j--];
        }
      }
      while (j >= 0) {
        nums1[k--] = nums2[j--];
      }
    }
  }
}
```

89. `_89.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 89. Gray Code
 *
 * The gray code is a binary numeral system where two successive values differ in only
one bit.
 * Given a non-negative integer n representing the total number of bits in the code,
 * print the sequence of gray code.
 * A gray code sequence must begin with 0.

 For example, given n = 2, return [0,1,3,2]. Its gray code sequence is:

 00 - 0
 01 - 1
 11 - 3
 10 - 2

 Note:

 For a given n, a gray code sequence is not uniquely defined.

 For example, [0,2,3,1] is also a valid gray code sequence according to the above
definition.

 For now, the judge is able to judge based on one instance of gray code sequence. Sorry
about that.
 */

public class _89 {

  public static class Solution1 {
    public List<Integer> grayCode(int n) {
      List<Integer> result = new ArrayList();
      for (int i = 0; i < (1 << n); i++) {
        result.add(i ^ (i >> 1));
      }
      return result;
    }
  }

  public static class Solution2 {
    public List<Integer> grayCode(int n) {
      List<Integer> result = new ArrayList();
      for (int i = 0; i < Math.pow(2, n); i++) {
        result.add(i ^ (i >> 1));
      }
      return result;
```

```
    }
  }

  public static void main(String... args) {
    System.out.println("------------------------------------------------------------------
------------------------");
    System.out.println("How to understand i << n? It means n to the power of two, see
below. So we have an equivalent solution, which is solution2.");
    System.out.println("1 << 2: " + (1 << 2));
    System.out.println("1 << 3: " + (1 << 3));
    System.out.println("1 << 4: " + (1 << 4));
    System.out.println("1 << 5: " + (1 << 5));
    System.out.println("1 << 6: " + (1 << 6));
    System.out.println("------------------------------------------------------------------
------------------------");
    System.out.println("How to understand i >> 1? It means to shift the number i to the
right by 1 bit, see below");
    System.out.println("0 >> 1: " + (0 >> 1));
    System.out.println("1 >> 1: " + (1 >> 1));
    System.out.println("2 >> 1: " + (2 >> 1));
    System.out.println("3 >> 1: " + (3 >> 1));
    System.out.println("4 >> 1: " + (4 >> 1));
    System.out.println("5 >> 1: " + (5 >> 1));
    System.out.println("6 >> 1: " + (6 >> 1));
  }

}
```

90. `_90.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 90. Subsets II
 *
 *Given a collection of integers that might contain duplicates, nums, return all possible
subsets.
 Note: The solution set must not contain duplicate subsets.

 For example,
 If nums = [1,2,2], a solution is:
 [
 [2],
 [1],
 [1,2,2],
 [2,2],
 [1,2],
 []
 ]
 */
public class _90 {

    public static class Solution1 {
        public List<List<Integer>> subsetsWithDup(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            List<Integer> empty = new ArrayList();
            result.add(empty);
            if (nums == null) {
                return result;
            }
            Arrays.sort(nums);
            for (int i = 0; i < nums.length; i++) {
                Set<List<Integer>> temp = new HashSet();
                for (List<Integer> list : result) {
                    List<Integer> newList = new ArrayList(list);
                    newList.add(nums[i]);
                    temp.add(newList);
                }
                result.addAll(temp);
            }
            Set<List<Integer>> resultSet = new HashSet();
            resultSet.addAll(result);
            result.clear();
            result.addAll(resultSet);
            return result;
```

```
        }
    }

    public static class Solution2 {
        public List<List<Integer>> subsetsWithDup(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            Arrays.sort(nums);
            backtrack(nums, 0, new ArrayList(), result);
            return result;
        }

        void backtrack(int[] nums, int start, List<Integer> curr, List<List<Integer>>
result) {
            result.add(new ArrayList(curr));
            for (int i = start; i < nums.length; i++) {
                if (i > start && nums[i] == nums[i - 1]) {
                    continue;
                }
                curr.add(nums[i]);
                backtrack(nums, i + 1, curr, result);
                curr.remove(curr.size() - 1);
            }
        }
    }

    public static class Solution3 {
        public List<List<Integer>> subsetsWithDup(int[] nums) {
            List<List<Integer>> result = new ArrayList<>();
            List<Integer> list = new ArrayList<>();
            result.add(list);
            Arrays.sort(nums);
            backtracking(nums, 0, result, list);
            return result;
        }

        private void backtracking(int[] nums, int start, List<List<Integer>> result,
List<Integer> list) {
            for (int i = start; i < nums.length; i++) {
                if (i > start && nums[i] == nums[i - 1]) {
                    continue;
                }
                list.add(nums[i]);
                result.add(new ArrayList<>(list));
                backtracking(nums, i + 1, result, list);
                list.remove(list.size() - 1);
            }
        }
    }
}
```

91. `_91.java`

```
package com.fishercoder.solutions;

/**
 * 91. Decode Ways
 *
 * A message containing letters from A-Z is being encoded to numbers using the following
mapping:

 'A' -> 1
 'B' -> 2
 ...
 'Z' -> 26
 Given an encoded message containing digits, determine the total number of ways to decode
it.

 For example,
 Given encoded message "12", it could be decoded as "AB" (1 2) or "L" (12).

 The number of ways decoding "12" is 2.
 */
public class _91 {
    /**Credit: https://discuss.leetcode.com/topic/35840/java-clean-dp-solution-with-
explanation
     * I used a dp array of size n + 1 to save subproblem solutions.
     * dp[0] means an empty string will have one way to decode,
     * dp[1] means the way to decode a string of size 1.
     *
     * I then check one digit and two digit combination and save the results along the
way.
     * In the end, dp[n] will be the end result.*/

    public static class Solution1 {
      public int numDecodings(String s) {
        if (s == null || s.length() == 0) {
          return 0;
        }
        int[] dp = new int[s.length() + 1];
        dp[0] = 1;
        dp[1] = (s.charAt(0) != '0') ? 1 : 0;
        for (int i = 2; i <= s.length(); i++) {
          int first = Integer.valueOf(s.substring(i - 1, i));
          int second = Integer.valueOf(s.substring(i - 2, i));
          if (first > 0 && first <= 9) {
            dp[i] += dp[i - 1];
          }
          if (second >= 10 && second <= 26) {
            dp[i] += dp[i - 2];
          }
        }
        return dp[s.length()];
      }
```

```
    }
}
```

92. `_92.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 92. Reverse Linked List II
 *
 * Reverse a linked list from position m to n. Do it in-place and in one-pass.
 *
 * For example: Given 1->2->3->4->5->NULL, m = 2 and n = 4,
 *
 * return 1->4->3->2->5->NULL.
 *
 * Note: Given m, n satisfy the following condition: 1 ≤ m ≤ n ≤ length of list.
 */
public class _92 {

  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/8976/simple-java-solution-with-clear-
explanation */
    public ListNode reverseBetween(ListNode head, int m, int n) {
      // use four nodes, pre, start, then, dummy
      // just reverse the nodes along the way
      ListNode dummy = new ListNode(-1);
      dummy.next = head;
      ListNode pre = dummy;
      for (int i = 0; i < m - 1; i++) {
        pre = pre.next;
      }

      ListNode start = pre.next;// start is the node prior to reversing, in the given
example,
      // start is node with value 1
      ListNode then = start.next;// then is the node that we'll start to reverse, in the
given
      // example, it's 2

      for (int i = 0; i < n - m; i++) {
        // pay special attention to this for loop, it's assigning then.next to
start.next, it
        // didn't initialize a new node
        // this does exactly what I desired to do, but I just didn't figure out how to
implement
        // it, thumbs up to the OP!
        start.next = then.next;
        then.next = pre.next;
        pre.next = then;
        then = start.next;
      }

      return dummy.next;
```

```
    }
  }
}
```

93. `_93.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 93. Restore IP Addresses
 *
 * Given a string containing only digits, restore it by returning all possible valid IP
address
 * combinations.
 *
 * For example: Given "25525511135",
 *
 * return ["255.255.11.135", "255.255.111.35"]. (Order does not matter)
 */
public class _93 {

  public static class Solution1 {
    public List<String> restoreIpAddresses(String s) {
      List<String> allValidIpAddresses = new ArrayList<>();
      if (s == null || s.length() > 12 || s.length() < 4) {
        return allValidIpAddresses;
      }
      backtracking(s, new ArrayList<>(), allValidIpAddresses, 0);
      return allValidIpAddresses;
    }

    private void backtracking(String s, ArrayList<String> bytes, List<String> result, int
pos) {
      if (bytes.size() == 4) {
        if (pos != s.length()) {
          return;
        }
        StringBuilder stringBuilder = new StringBuilder();
        for (int i = 0; i < 4; i++) {
          stringBuilder.append(bytes.get(i));
          stringBuilder.append(".");
        }
        stringBuilder.setLength(stringBuilder.length() - 1);
        result.add(stringBuilder.toString());
        return;
      }

      for (int i = pos; i < pos + 4 && i < s.length(); i++) {
        String oneByte = s.substring(pos, i + 1);
        if (!isValid(oneByte)) {
          continue;
        }
        bytes.add(oneByte);
        backtracking(s, bytes, result, i + 1);
```

```
        bytes.remove(bytes.size() - 1);
      }
    }

    private boolean isValid(String oneByte) {
      if (oneByte.charAt(0) == '0') {
        return oneByte.equals("0");
      }
      int num = Integer.valueOf(oneByte);
      return (num >= 0 && num < 256);
    }
  }
}
```

94. `_94.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

/**
 * 94. Binary Tree Inorder Traversal
 *
 * Given a binary tree, return the inorder traversal of its nodes' values.

 For example:
 Given binary tree [1,null,2,3],
 1
  \
   2
  /
 3
 return [1,3,2].

 Note: Recursive solution is trivial, could you do it iteratively?*/

public class _94 {

    public static class Solution1 {
        public List<Integer> inorderTraversal(TreeNode root) {
            return inorder(root, new ArrayList());
        }

        List<Integer> inorder(TreeNode root, List<Integer> result) {
            if (root == null) {
                return result;
            }
            inorder(root.left, result);
            result.add(root.val);
            return inorder(root.right, result);
        }
    }

    public static class Solution2 {
        //iterative approach
        public List<Integer> inorderTraversal(TreeNode root) {
            List<Integer> result = new ArrayList();
            Stack<TreeNode> stack = new Stack();
            while (root != null || !stack.isEmpty()) {
                while (root != null) {
                    stack.push(root);
                    root = root.left;
                }
```

```
                root = stack.pop();
                result.add(root.val);
                root = root.right;
            }
            return result;
        }
    }
}
```

95. `_95.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 95. Unique Binary Search Trees II
 *
 * Given an integer n, generate all structurally unique BST's (binary search trees) that
store values 1...n.

 For example,
 Given n = 3, your program should return all 5 unique BST's shown below.

 1         3     3      2      1
  \       /     /      / \      \
   3     2     1      1   3      2
  /     /       \                 \
 2     1         2                 3
 */
public class _95 {

        public static class Solution1 {
                public List<TreeNode> generateTrees(int n) {
                        List<TreeNode> result = new ArrayList();
                        if (n == 0) {
                                return result;
                        }
                        return generateTrees(1, n);
                }

                private List<TreeNode> generateTrees(int start, int end) {
                        List<TreeNode> result = new ArrayList();
                        if (start > end) {
                                result.add(null);
                                return result;
                        }
                        if (start == end) {
                                result.add(new TreeNode(start));
                                return result;
                        }

                        for (int i = start; i <= end; i++) {
                                List<TreeNode> leftList = generateTrees(start, i - 1);
                                List<TreeNode> rightList = generateTrees(i + 1, end);
                                for (TreeNode left : leftList) {
                                        for (TreeNode right : rightList) {
                                                TreeNode root = new TreeNode(i);
                                                root.left = left;
```

```
                                                root.right = right;
                                                result.add(root);
                                            }
                                        }
                                    }
                                    return result;
                                }
                            }
```

96. `_96.java`

```
package com.fishercoder.solutions;

/**
 * 96. Unique Binary Search Trees
 *
 * Given n, how many structurally unique BST's (binary search trees) that store values
1...n?

 For example,
 Given n = 3, there are a total of 5 unique BST's.

 1         3     3      2      1
 \       /     /      / \      \
 3      2     1      1   3      2
 /     /       \                 \
 2    1         2                 3

 */
public class _96 {

    public static class Solution1 {
        public int numTrees(int n) {
            int[] G = new int[n + 1];
            G[0] = G[1] = 1;

            for (int i = 2; i <= n; ++i) {
                for (int j = 1; j <= i; ++j) {
                    int temp = G[j - 1] * G[i - j];
                    G[i] = G[i] + temp;
                }
            }
            return G[n];
        }
    }
}
```

97. `_97.java`

```
package com.fishercoder.solutions;

/**
 * 97. Interleaving String
 *
 * Given s1, s2, s3, find whether s3 is formed by the interleaving of s1 and s2.
 * For example,
 * Given:
 * s1 = "aabcc",
 * s2 = "dbbca",
 * When s3 = "aadbbcbcac", return true.
 * When s3 = "aadbbbaccc", return false.
 */
public class _97 {
  public static class Solution1 {
    public boolean isInterleave(String s1, String s2, String s3) {
      int m = s1.length();
      int n = s2.length();
      if (m + n != s3.length()) {
        return false;
      }

      boolean[][] dp = new boolean[m + 1][n + 1];

      dp[0][0] = true;

      for (int i = 0; i < m; i++) {
        if (s1.charAt(i) == s3.charAt(i)) {
          dp[i + 1][0] = true;
        } else {
          //if one char fails, that means it breaks, the rest of the chars won't matter
any more.
          //Mian and I found one missing test case on Lintcode: ["b", "aabccc",
"aabbbcb"]
          //if we don't break, here, Lintcode could still accept this code, but Leetcode
fails it.
          break;
        }
      }

      for (int j = 0; j < n; j++) {
        if (s2.charAt(j) == s3.charAt(j)) {
          dp[0][j + 1] = true;
        } else {
          break;
        }
      }

      for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
          int k = i + j - 1;
```

```
          dp[i][j] = (s1.charAt(i - 1) == s3.charAt(k) && dp[i - 1][j])
              || (s2.charAt(j - 1) == s3.charAt(k) && dp[i][j - 1]);
        }
      }

      return dp[m][n];
    }
  }
}
```

98. `_98.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 98. Validate Binary Search Tree
 *
 * Given a binary tree, determine if it is a valid binary search tree (BST).

 Assume a BST is defined as follows:

 The left subtree of a node contains only nodes with keys less than the node's key.
 The right subtree of a node contains only nodes with keys greater than the node's key.
 Both the left and right subtrees must also be binary search trees.

 Example 1:
   2
  / \
 1   3
 Binary tree [2,1,3], return true.

 Example 2:
   1
  / \
 2   3
 Binary tree [1,2,3], return false.
 */
public class _98 {

    public static class Solution1 {

        public boolean isValidBST(TreeNode root) {
            return valid(root, null, null);
        }

        boolean valid(TreeNode root, Integer min, Integer max) {
            if (root == null) {
                return true;
            }
            if ((min != null && root.val <= min) || (max != null && root.val >= max)) {
                return false;
            }
            return valid(root.left, min, root.val) && valid(root.right, root.val, max);
        }
    }


    public static class Solution2 {
        public boolean isValidBST(TreeNode root) {
            if (root == null) {
                return true;
```

```
            }
            return dfs(root.left, Long.MIN_VALUE, root.val) && dfs(root.right, root.val,
Long.MAX_VALUE);
        }

        private boolean dfs(TreeNode root, long minValue, long maxValue) {
            if (root == null) {
                return true;
            }
            if (root != null && (root.val <= minValue || root.val >= maxValue)) {
                return false;
            }
            boolean leftResult = true;
            boolean rightResult = true;
            if (root.left != null) {
                leftResult = dfs(root.left, minValue, root.val);
            }
            if (root.right != null) {
                rightResult = dfs(root.right, root.val, maxValue);
            }
            return leftResult && rightResult;
        }
    }

}
```

99. `_99.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
/**
 * 99. Recover Binary Search Tree
 *
 * Two elements of a binary search tree (BST) are swapped by mistake.
 * Recover the tree without changing its structure.

 Example 1:

 Input: [1,3,null,null,2]

    1
   /
  3
  \
  2

 Output: [3,1,null,null,2]

   3
  /
 1
  \
  2
 Example 2:

 Input: [3,1,4,null,null,2]

    3
   / \
  1   4
 /
2

 Output: [2,1,4,null,null,3]

    2
   / \
  1   4
 /
3

 Follow up:
 A solution using O(n) space is pretty straight forward.
 Could you devise a constant space solution?
 */

public class _99 {
  public static class Solution1 {
```

```
    TreeNode firstElement = null;
    TreeNode secondElement = null;

    TreeNode prevElement = new TreeNode(Integer.MIN_VALUE);

    public void recoverTree(TreeNode root) {
      traverseTree(root);

      //swap the two elements
      int temp = firstElement.val;
      firstElement.val = secondElement.val;
      secondElement.val = temp;
    }

    private void traverseTree(TreeNode root) {
      if (root == null) {
        return;
      }

      traverseTree(root.left);

      //prevElement means the one previous to the current root, refer to in-order
traversal, previous element must be smaller than the current root
      //if it's bigger, then we find the first element, thus we store it in the variable
called firstElement
      if (firstElement == null && prevElement.val >= root.val) {
        firstElement = prevElement;
      }

      if (firstElement != null && prevElement.val >= root.val) {
        secondElement = root;
      }

      //this is the last step in the "do some business logic", so we'll always to have
update the previous node to be the current root before it traverses the right subtree
      //since the current root will be the new previous node for the right subtree.
      prevElement = root;

      traverseTree(root.right);
    }

  }
}
```

100. `_100.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 100. Same Tree
 *
 * Given two binary trees, write a function to check if they are equal or not. Two binary
trees are
 * considered equal if they are structurally identical and the nodes have the same value.
 *
 * Example 1:
 *
 * Input:     1         1
 *           / \       / \
 *          2   3     2   3
 *
 *         [1,2,3],   [1,2,3]
 *
 * Output: true
 *
 * Example 2:
 *
 * Input:     1         1
 *           /           \
 *          2             2
 *
 *         [1,2],     [1,null,2]
 *
 * Output: false
 *
 * Example 3:
 *
 * Input:     1         1
 *           / \       / \
 *          2   1     1   2
 *
 *         [1,2,1],   [1,1,2]
 *
 * Output: false
 */

public class _100 {

  public static class Solution1 {
    public boolean isSameTree(TreeNode p, TreeNode q) {
      if (p == null || q == null) {
        return p == q;
      }
      return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right,
q.right);
```

```
    }
  }
}
```

101. `_101.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**101. Symmetric Tree

Given a binary tree, check whether it is a mirror of itself (ie, symmetric around its
center).

For example, this binary tree [1,2,2,3,4,4,3] is symmetric:

    1
   / \
  2   2
 / \ / \
3  4 4  3

But the following [1,2,2,null,3,null,3] is not:

    1
   / \
  2   2
   \   \
   3    3

Note:
Bonus points if you could solve it both recursively and iteratively.
 */

public class _101 {
        public static class Solution1 {
                public boolean isSymmetric(TreeNode root) {
                        if (root == null) {
                                return true;
                        }
                        return isSymmetric(root.left, root.right);
                }

                private boolean isSymmetric(TreeNode left, TreeNode right) {
                        if (left == null || right == null) {
                                return left == right;
                        }
                        if (left.val != right.val) {
                                return false;
                        }
                        return isSymmetric(left.left, right.right) &&
isSymmetric(left.right, right.left);
                }
        }
}
```

102. `_102.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 102. Binary Tree Level Order Traversal

Given a binary tree, return the level order traversal of its nodes' values. (ie, from
left to right, level by level).

For example:
Given binary tree [3,9,20,null,null,15,7],

    3
   / \
  9  20
    /  \
   15   7

return its level order traversal as:

[
  [3],
  [9,20],
  [15,7]
]
*/
public class _102 {

  public static class Solution1 {
    public List<List<Integer>> levelOrder(TreeNode root) {
      List<List<Integer>> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      Queue<TreeNode> queue = new LinkedList();
      queue.offer(root);
      while (!queue.isEmpty()) {
        List<Integer> thisLevel = new ArrayList();
        int size = queue.size();
        for (int i = 0; i < size; i++) {
          TreeNode curr = queue.poll();
          thisLevel.add(curr.val);
          if (curr.left != null) {
            queue.offer(curr.left);
          }
```

```
          if (curr.right != null) {
            queue.offer(curr.right);
          }
        }
        result.add(thisLevel);
      }
      return result;
    }
  }
}
```

103. `_103.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 103. Binary Tree Zigzag Level Order Traversal
 *
Given a binary tree, return the zigzag level order traversal of its nodes' values.
 (ie, from left to right, then right to left for the next level and alternate between).

For example:
Given binary tree [3,9,20,null,null,15,7],
    3
   / \
  9  20
    /  \
   15   7
return its zigzag level order traversal as:
[
  [3],
  [20,9],
  [15,7]
]
*/
public class _103 {
    public static class Solution1 {
        public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
            Queue<TreeNode> q = new LinkedList();
            List<List<Integer>> levels = new ArrayList();
            if (root == null) {
                return levels;
            }
            q.offer(root);
            boolean forward = true;
            while (!q.isEmpty()) {
                int size = q.size();
                List<Integer> level = new ArrayList();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = q.poll();
                    level.add(curr.val);
                    if (curr.left != null) {
                        q.offer(curr.left);
                    }
                    if (curr.right != null) {
                        q.offer(curr.right);
```

```
                    }
                }
                if (forward) {
                    forward = false;
                    levels.add(level);
                } else {
                    Collections.reverse(level);
                    levels.add(level);
                    forward = true;
                }
            }
            return levels;
        }
    }
}
```

104. `_104.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 104. Maximum Depth of Binary Tree
 *
 * Given a binary tree, find its maximum depth.
 * The maximum depth is the number of nodes along the longest path from the root node
down to the farthest leaf node.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 * Given binary tree [3,9,20,null,null,15,7],
 *
 *     3
 *    / \
 *   9  20
 *     /  \
 *    15  7
 *
 * return its depth = 3.
 */
public class _104 {

  public static class Solution1 {
    public int maxDepth(TreeNode root) {
      if (root == null) {
        return 0;
      }
      return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
    }
  }

}
```

105. `_105.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 105. Construct Binary Tree from Preorder and Inorder Traversal
 * Given preorder and inorder traversal of a tree, construct the binary tree.

 Note:
 You may assume that duplicates do not exist in the tree.
 */
public class _105 {

        public static class Solution1 {
                /**
                 * credit: https://discuss.leetcode.com/topic/29838/5ms-java-clean-
solution-with-caching use
                 * HashMap as the cache so that accessing inorder index becomes O(1) time
Note: The first
                 * element of preorder array is the root!
                 */
                public TreeNode buildTree(int[] preorder, int[] inorder) {
                        Map<Integer, Integer> inorderMap = new HashMap();
                        for (int i = 0; i < inorder.length; i++) {
                                inorderMap.put(inorder[i], i);
                        }

                        /**At the beginning, both start from 0 to nums.length-1*/
                        return buildTree(preorder, 0, preorder.length - 1, inorderMap, 0,
inorder.length - 1);
                }

                private TreeNode buildTree(int[] preorder, int preStart, int preEnd,
                                Map<Integer, Integer> inorderMap, int inStart, int inEnd)
{
                        if (preStart > preEnd || inStart > inEnd) {
                                return null;
                        }

                        TreeNode root = new TreeNode(preorder[preStart]);
                        int inRoot = inorderMap.get(preorder[preStart]);
                        int numsLeft = inRoot - inStart;

                        /**It's easy to understand and remember:
                         * for the indices of inorder array:
                         * root.left should be inStart and inRoot-1 as new start and end
indices
                         * root.right should be inRoot+1 and inEnd as new start and end
```

```
indices
                         *
                         * since inRoot is being used already in this recursion call,
that's why we use inRoot-1 and inRoot+1
                         * this part is the same for both Leetcode 105 and Leetcode
106.*/
                        root.left = buildTree(preorder, preStart + 1, preStart +
numsLeft, inorderMap, inStart, inRoot - 1);
                        root.right = buildTree(preorder, preStart + numsLeft + 1, preEnd,
inorderMap, inRoot + 1, inEnd);
                        return root;
                }
        }
}
```

106. `_106.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 106. Construct Binary Tree from Inorder and Postorder Traversal
 *
 * Given inorder and postorder traversal of a tree, construct the binary tree.

 Note:
 You may assume that duplicates do not exist in the tree.
 */
public class _106 {
    public static class Solution1 {

        /**
         * https://discuss.leetcode.com/topic/3296/my-recursive-java-code-with-o-n-time-
and-o-n-space
         *
         * Note: the last element of postorder array is the root!
         *
         * The idea is to take the last element in postorder as the root; find the
position of the root
         * in the inorder array; then locate the range for left sub-tree and right sub-
tree and do
         * recursion, use a hashmap to record the index of root in the inorder array.
         */
        public TreeNode buildTree(int[] inorder, int[] postorder) {
            if (inorder == null || postorder == null || inorder.length !=
postorder.length) {
                return null;
            }
            HashMap<Integer, Integer> inorderMap = new HashMap<>();
            for (int i = 0; i < inorder.length; i++) {
                inorderMap.put(inorder[i], i);
            }
            /**At the beginning, both start from 0 to nums.length-1*/
            return buildTreeRecursively(inorderMap, 0, inorder.length - 1, postorder, 0,
                postorder.length - 1);
        }

        private TreeNode buildTreeRecursively(Map<Integer, Integer> inorderMap, int
inorderStart,
            int inorderEnd, int[] postorder, int postorderStart, int postorderEnd) {
            if (postorderStart > postorderEnd || inorderStart > inorderEnd) {
                return null;
            }
            TreeNode root = new TreeNode(postorder[postorderEnd]);
```

```
            int inRoot = inorderMap.get(postorder[postorderEnd]);
            int numsLeft = inRoot - inorderStart;

            /**It's easy to understand and remember:
             * for the indices of inorder array:
             * inStart and inRoot-1 as new start and end indices
             * inRoot+1 and inEnd as new start and end indices
             *
             * this is easy to understand and remember: since inRoot is already been used
in this recursion call, so we're going to use inRoot-1 and inRoot+1 for next recursion
call
             *
             * for the indices of postorder array:
             * postorderStart and postorderStart+numsLeft-1 should be the new start and
end indices
             * postorderStart+numsLeft and postorderEnd-1 should be the new start and end
indices
             *
             * this is also easy to understand and remember:
             * since the last one in postorder is the root and we have used it in this
recursion call already, so the end is definitely postorderEnd-1;
             * then the postorderEnd for root.left is contiguous to the postorderStart of
root.right, :)*/
            root.left = buildTreeRecursively(inorderMap, inorderStart, inRoot - 1,
postorder, postorderStart, postorderStart + numsLeft - 1);
            root.right = buildTreeRecursively(inorderMap, inRoot + 1, inorderEnd,
postorder, postorderStart + numsLeft, postorderEnd - 1);
            return root;
        }
    }
}
```

107. `_107.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**107. Binary Tree Level Order Traversal II

Given a binary tree, return the bottom-up level order traversal of its nodes' values.
(ie, from left to right, level by level from leaf to root).

For example:
Given binary tree [3,9,20,null,null,15,7],

    3
   / \
  9  20
    /  \
   15   7

return its bottom-up level order traversal as:

[
  [15,7],
  [9,20],
  [3]
]
*/

public class _107 {
        public static class Solution1 {
                public List<List<Integer>> levelOrder(TreeNode root) {
                        List<List<Integer>> result = new ArrayList();
                        if (root == null) {
                                return result;
                        }

                        Queue<TreeNode> q = new LinkedList();
                        q.offer(root);
                        while (!q.isEmpty()) {
                                List<Integer> thisLevel = new ArrayList<Integer>();
                                int qSize = q.size();
                                for (int i = 0; i < qSize; i++) {
                                        TreeNode curr = q.poll();
                                        thisLevel.add(curr.val);
                                        if (curr.left != null) {
                                                q.offer(curr.left);
```

```
                                        }
                                        if (curr.right != null) {
                                                q.offer(curr.right);
                                        }
                                }
                                result.add(thisLevel);
                        }
                        Collections.reverse(result);
                        return result;
                }
        }
}
```

108. `_108.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 108. Convert Sorted Array to Binary Search Tree
 *
 * Given an array where elements are sorted in ascending order, convert it to a height
balanced BST.
 */
public class _108 {

  public static class Solution1 {
    public TreeNode sortedArrayToBST(int[] num) {
      return rec(num, 0, num.length - 1);
    }

    public TreeNode rec(int[] num, int low, int high) {
      if (low > high) {
        return null;
      }
      int mid = low + (high - low) / 2;
      TreeNode root = new TreeNode(num[mid]);
      root.left = rec(num, low, mid - 1);
      root.right = rec(num, mid + 1, high);
      return root;
    }
  }
}
```

109. `_109.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;
import com.fishercoder.common.classes.TreeNode;

/**
 * 109. Convert Sorted List to Binary Search Tree
 *
 * Given a singly linked list where elements are sorted in ascending order, convert it to
a height balanced BST.
 */
public class _109 {

    public static class Solution1 {

        public TreeNode sortedListToBST(ListNode head) {
            return toBstRecursively(head, null);
        }

        public TreeNode toBstRecursively(ListNode start, ListNode end) {
            if (start == end) {
                return null;
            } else {
                ListNode mid = start;
                ListNode fast = start;
                while (fast != end && fast.next != end) {
                    mid = mid.next;
                    fast = fast.next.next;
                }

                TreeNode root = new TreeNode(mid.val);
                root.left = toBstRecursively(start, mid);
                root.right = toBstRecursively(mid.next, end);
                return root;
            }
        }
    }

}
```

110. `_110.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 110. Balanced Binary Tree
 *
 * Given a binary tree, determine if it is height-balanced.
 * For this problem, a height-balanced binary tree is defined as a binary tree in which
 * the depth of the two subtrees of every node never differ by more than 1.

 Example 1:
 Given the following tree [3,9,20,null,null,15,7]:

    3
   / \
  9  20
  /  \
 15  7

 Return true.

 Example 2:
 Given the following tree [1,2,2,3,3,null,null,4,4]:

     1
    / \
   2   2
  / \
 3   3
/ \
4 4

 Return false.
 */

public class _110 {

    public static class Solution1 {
        //recursively get the height of each subtree of each node, compare their
difference, if greater than 1, then return false
        //although this is working, but it's not efficient, since it repeatedly computes
the heights of each node every time
        //Its time complexity is O(n^2).
        public boolean isBalanced(TreeNode root) {
            if (root == null) {
                return true;
            }
            if (Math.abs(getH(root.left) - getH(root.right)) > 1) {
                return false;
            } else {
```

```
                return isBalanced(root.left) && isBalanced(root.right);
            }
        }

        private int getH(TreeNode root) {
            if (root == null) {
                return 0;//base case
            }
            int leftH = getH(root.left);
            int rightH = getH(root.right);
            return Math.max(leftH, rightH) + 1;
        }
    }

    public static class Solution2 {

        public boolean isBalanced(TreeNode root) {
            return getH(root) != -1;
        }

        private int getH(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int leftH = getH(root.left);
            if (leftH == -1) {
                return -1;
            }
            int rightH = getH(root.right);
            if (rightH == -1) {
                return -1;
            }
            if (Math.abs(leftH - rightH) > 1) {
                return -1;
            }
            return Math.max(leftH, rightH) + 1;
        }
    }

}
```

111. `_111.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 111. Minimum Depth of Binary Tree
 *
 * Given a binary tree, find its minimum depth.
 * The minimum depth is the number of nodes along the shortest path from the root node
down to the nearest leaf node.
 * */

public class _111 {
    public static class Solution1 {
      /**DFS*/
      public int minDepth(TreeNode root) {
        if (root == null) {
          return 0;
        }
        int left = minDepth(root.left);
        int right = minDepth(root.right);
        if (left == 0) {
          return right + 1;
        }
        if (right == 0) {
          return left + 1;
        }
        return Math.min(left, right) + 1;
      }
    }

  public static class Solution2 {
    /**BFS*/
    public int minDepth(TreeNode root) {
      if (root == null) {
        return 0;
      }
      Queue<TreeNode> q = new LinkedList();
      q.offer(root);
      int level = 0;
      while (!q.isEmpty()) {
        level++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
          TreeNode curr = q.poll();
          if (curr.left != null) {
            q.offer(curr.left);
          }
```

```
          if (curr.right != null) {
            q.offer(curr.right);
          }
          if (curr.left == null && curr.right == null) {
            return level;
          }
        }
      }
      return level;
    }
  }
}
```

112. `_112.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 112. Path Sum

Given a binary tree and a sum, determine if the tree has a root-to-leaf path such that
adding up all the values along the path equals the given sum.

For example:
Given the below binary tree and sum = 22,

              5
             / \
            4   8
           /   / \
          11  13  4
         /  \      \
        7    2      1

 return true, as there exist a root-to-leaf path 5->4->11->2 which sum is 22.*/
public class _112 {
  public static class Solution1 {
    public boolean hasPathSum(TreeNode root, int sum) {
      if (root == null) {
        return false;
      }
      if (root.val == sum && root.left == null && root.right == null) {
        return true;
      }
      return hasPathSum(root.left, sum - root.val) || hasPathSum(root.right, sum -
root.val);
    }
  }
}
```

113. `_113.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 113. Path Sum II

 Given a binary tree and a sum, find all root-to-leaf paths where each path's sum equals
the given sum.

For example:
Given the below binary tree and sum = 22,
              5
             / \
            4   8
           /   / \
          11  13  4
         /  \    / \
        7    2  5   1

 return
[
   [5,4,11,2],
   [5,8,4,5]
]
*/
public class _113 {

  public static class Solution1 {
    public List<List<Integer>> pathSum(TreeNode root, int sum) {
      List<List<Integer>> allPaths = new ArrayList();
      if (root == null) {
        return allPaths;
      }
      dfs(root, new ArrayList(), allPaths, sum);
      return allPaths;
    }

    private void dfs(TreeNode root, List<Integer> path, List<List<Integer>> allPaths, int
sum) {
      path.add(root.val);
      if (root.left != null) {
        dfs(root.left, path, allPaths, sum - root.val);
      }
      if (root.right != null) {
        dfs(root.right, path, allPaths, sum - root.val);
      }
      if (root.left == null && root.right == null) {
```

```
        /**Check if sum equals root.val, not sum equals zero!*/
        if (sum == root.val) {
          allPaths.add(new ArrayList(path));
        }
      }
      path.remove(path.size() - 1);
    }
  }
}
```

114. `_114.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 114. Flatten Binary Tree to Linked List
 *
 * Given a binary tree, flatten it to a linked list in-place.

 For example,
 Given

   1
  / \
 2  5
/ \  \
3 4  6

 The flattened tree should look like:
  1
   \
   2
    \
    3
     \
     4
      \
      5
       \
       6

 Hints:
 If you notice carefully in the flattened tree, each node's right child points to the
next node of a pre-order traversal.
 */
public class _114 {

    public static class Solution1 {
        public void flatten(TreeNode root) {
            while (root != null) {
                if (root.left != null) {
                    TreeNode previousNode = root.left;
                    while (previousNode.right != null) {
                        previousNode = previousNode.right;
                    }
                    previousNode.right = root.right;
                    root.right = root.left;
                    root.left = null;
                }
                root = root.right;
            }
```

```
        }
    }
}
```

115. `_115.java`

```
package com.fishercoder.solutions;

/**
 * 115. Distinct Subsequences
 *
 * Given a string S and a string T, count the number of distinct subsequences of S which
equals T. A
 * subsequence of a string is a new string which is formed from the original string by
deleting some
 * (can be none) of the characters without disturbing the relative positions of the
remaining
 * characters. (ie, "ACE" is a subsequence of "ABCDE" while "AEC" is not).
 *
 * Here is an example: S = "rabbbit", T = "rabbit" Return 3.
 */

public class _115 {
  public static class Solution1 {
    public int numDistinct(String s, String t) {
      int m = s.length();
      int n = t.length();
      int[][] dp = new int[m + 1][n + 1];

      char[] schar = s.toCharArray();
      char[] tchar = t.toCharArray();

      for (int i = 0; i <= m; i++) {
        dp[i][0] = 1;
      }

      for (int j = 1; j <= n; j++) {
        dp[0][j] = 0;
      }

      for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
          if (schar[i - 1] == tchar[j - 1]) {
            dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1];
          } else {
            dp[i][j] = dp[i - 1][j];
          }
        }
      }
      return dp[m][n];
    }
  }
}
```

116. `_116.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeLinkNode;

/**
 * 116. Populating Next Right Pointers in Each Node

 Given a binary tree

 struct TreeLinkNode {
     TreeLinkNode *left;
     TreeLinkNode *right;
     TreeLinkNode *next;
 }
 Populate each next pointer to point to its next right node. If there is no next right
node, the next pointer should be set to NULL.

 Initially, all next pointers are set to NULL.

 Note:

 You may only use constant extra space.
 You may assume that it is a perfect binary tree (ie, all leaves are at the same level,
and every parent has two children).
 For example,
 Given the following perfect binary tree,
      1
    /  \
   2    3
  / \  / \
 4  5  6  7

 After calling your function, the tree should look like:
      1 -> NULL
    /  \
   2 -> 3 -> NULL
  / \  / \
 4->5->6->7 -> NULL
 */

public class _116 {
  public static class Solution1 {
    /**
     * credit: https://discuss.leetcode.com/topic/1106/o-1-space-o-n-complexity-
iterative-solution
     * based on level order traversal
     */
    public void connect(TreeLinkNode root) {

      TreeLinkNode head = null; //head of the next level
      TreeLinkNode prev = null; //the leading node on the next level
```

```
      TreeLinkNode curr = root;  //current node of current level

      while (curr != null) {
        while (curr != null) { //iterate on the current level
          //left child
          if (curr.left != null) {
            if (prev != null) {
              prev.next = curr.left;
            } else {
              head = curr.left;
            }
            prev = curr.left;
          }
          //right child
          if (curr.right != null) {
            if (prev != null) {
              prev.next = curr.right;
            } else {
              head = curr.right;
            }
            prev = curr.right;
          }
          //move to next node
          curr = curr.next;
        }
        //move to next level
        curr = head;
        head = null;
        prev = null;
      }
    }
  }
}
```

117. `_117.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeLinkNode;

/**
 * 117. Populating Next Right Pointers in Each Node II
 *
 * Follow up for problem "Populating Next Right Pointers in Each Node".
 * What if the given tree could be any binary tree? Would your previous solution still
work?

 Note:

 You may only use constant extra space.
 For example,
 Given the following binary tree,
      1
    /  \
   2    3
  / \    \
 4   5    7
 After calling your function, the tree should look like:
      1 -> NULL
    /  \
   2 -> 3 -> NULL
  / \    \
 4-> 5 -> 7 -> NULL */

public class _117 {
    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/1106/o-1-space-o-n-complexity-
iterative-solution
        O(1) space, based on level order traversal*/
        public void connect(TreeLinkNode root) {

            TreeLinkNode head = null; //head of the next level
            TreeLinkNode prev = null; //the leading node on the next level
            TreeLinkNode cur = root;  //current node of current level

            while (cur != null) {

                while (cur != null) { //iterate on the current level
                    //left child
                    if (cur.left != null) {
                        if (prev != null) {
                            prev.next = cur.left;
                        } else {
                            head = cur.left;
                        }
                        prev = cur.left;
                    }
```

```
                    //right child
                    if (cur.right != null) {
                        if (prev != null) {
                            prev.next = cur.right;
                        } else {
                            head = cur.right;
                        }
                        prev = cur.right;
                    }
                    //move to next node
                    cur = cur.next;
                }

                //move to next level
                cur = head;
                head = null;
                prev = null;
            }
        }
    }
}
```

118. `_118.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 118. Pascal's Triangle
 * Given numRows, generate the first numRows of Pascal's triangle.

 For example, given numRows = 5,
 Return

 [
     [1],
    [1,1],
   [1,2,1],
  [1,3,3,1],
 [1,4,6,4,1]
 ]

 */
public class _118 {

  public static class Solution1 {
    /**fill out values from left to right*/
    public List<List<Integer>> generate(int numRows) {
      List<List<Integer>> result = new ArrayList();
      List<Integer> row = new ArrayList();
      for (int i = 0; i < numRows; i++) {
        row.add(0, 1);
        for (int j = 1; j < row.size() - 1; j++) {
          row.set(j, row.get(j) + row.get(j + 1));
        }
        result.add(new ArrayList(row));
      }
      return result;
    }
  }

  public static class Solution2 {
    /**fill out values from right to left
     * credit: https://leetcode.com/problems/pascals-triangle/discuss/38141/My-concise-
solution-in-Java/36127*/
    public List<List<Integer>> generate(int numRows) {
      List<List<Integer>> result = new ArrayList();
      List<Integer> row = new ArrayList();
      for (int i = 0; i < numRows; i++) {
        for (int j = row.size() - 1; j >= 1; j--) {
          row.set(j, row.get(j) + row.get(j - 1));
        }
        row.add(1);
```

```
        result.add(new ArrayList<>(row));
      }
      return result;
    }
  }
}
```

119. `_119.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 119. Pascal's Triangle II
 * Given an index k, return the kth row of the Pascal's triangle.

 For example, given k = 3,
 Return [1,3,3,1].

 Note:
 Could you optimize your algorithm to use only O(k) extra space?
 */

public class _119 {

  public static class Solution1 {
    public List<Integer> getRow(int rowIndex) {
      if (rowIndex < 0) {
        return new ArrayList();
      }
      List<List<Integer>> result = new ArrayList();
      List<Integer> row = new ArrayList();
      row.add(1);
      result.add(row);
      for (int i = 1; i <= rowIndex; i++) {
        List<Integer> newRow = new ArrayList();
        newRow.add(1);
        List<Integer> lastRow = result.get(i - 1);
        for (int j = 1; j < lastRow.size(); j++) {
          newRow.add(lastRow.get(j - 1) + lastRow.get(j));
        }
        newRow.add(1);
        result.add(newRow);
      }
      return result.get(result.size() - 1);
    }
  }

  public static class Solution2 {
    /** O(k) space */
    public List<Integer> getRow(int rowIndex) {
      List<Integer> row = new ArrayList<>();
      for (int i = 0; i <= rowIndex; i++) {
        row.add(0, 1);
        for (int j = 1; j < row.size() - 1; j++) {
          row.set(j, row.get(j) + row.get(j + 1));
        }
      }
```

```
      return row;
    }
  }
}
```

120. `_120.java`

```
package com.fishercoder.solutions;

import java.util.List;

/**
 * 120. Triangle

  Given a triangle, find the minimum path sum from top to bottom. Each step you may move
to adjacent numbers on the row below.

 For example, given the following triangle
 [
 [2],
 [3,4],
 [6,5,7],
 [4,1,8,3]
 ]
 The minimum path sum from top to bottom is 11 (i.e., 2 + 3 + 5 + 1 = 11).

 Note:
 Bonus point if you are able to do this using only O(n) extra space, where n is the total
number of rows in the triangle.*/

public class _120 {
  public static class Solution1 {
    public int minimumTotal(List<List<Integer>> triangle) {
      int n = triangle.size();
      List<Integer> cache = triangle.get(n - 1);

      for (int layer = n - 2; layer >= 0; layer--) {
        //for each layer
        for (int i = 0; i <= layer; i++) {
          //check its very node
          int value = Math.min(cache.get(i), cache.get(i + 1)) +
triangle.get(layer).get(i);
          cache.set(i, value);
        }
      }
      return cache.get(0);
    }
  }
}
```

121. `_121.java`

```
package com.fishercoder.solutions;

/**
 * 121. Best Time to Buy and Sell Stock
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 *
 * If you were only permitted to complete at most one transaction (ie, buy one and sell
one share of the stock),
 * design an algorithm to find the maximum profit.
 *
 * Example 1:
 * Input: [7, 1, 5, 3, 6, 4]
 * Output: 5
 *
 * max. difference = 6-1 = 5 (not 7-1 = 6, as selling price needs to be larger than
buying price)
 *
 * Example 2:
 * Input: [7, 6, 4, 3, 1]
 * Output: 0
 *
 * In this case, no transaction is done, i.e. max profit = 0.
 */

public class _121 {

    public static class Solution1 {
        /**
         * The key here is that you'll have to buy first, before you can sell. That
means, if the lower
         * price comes after a higher price, their combination won't work! Since you
cannot sell first
         * before you buy it.
         */
        public int maxProfit(int[] prices) {
            if (prices == null || prices.length == 0) {
                return 0;
            }
            int minBuy = prices[0];
            int maxProfit = 0;
            for (int i = 1; i < prices.length; i++) {
                if (prices[i] < minBuy) {
                    minBuy = prices[i];
                } else {
                    maxProfit = Math.max(maxProfit, prices[i] - minBuy);
                }
            }
            return maxProfit;
        }
```

```
    }
}
```

122. `_122.java`

```
package com.fishercoder.solutions;

/**
 * 122. Best Time to Buy and Sell Stock II
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 * Design an algorithm to find the maximum profit. You may complete as many transactions
as you like
 * (ie, buy one and sell one share of the stock multiple times).
 * However, you may not engage in multiple transactions at the same time (ie, you must
sell the stock before you buy again).
 * */

public class _122 {
    public static class Solution1 {
        //peak and valley approach
        public int maxProfit(int[] prices) {
            int pro = 0;
            int i = 0;
            while (i < prices.length - 1) {
                while (i < prices.length - 1 && prices[i] >= prices[i + 1]) {
                    i++;
                }
                int valley = prices[i];
                while (i < prices.length - 1 && prices[i] <= prices[i + 1]) {
                    i++;
                }
                int peak = prices[i];
                pro += peak - valley;
            }
            return pro;
        }
    }

    public static class Solution2 {
        //simple one pass approach: the above solution could be simplied as below
        public int maxProfit(int[] prices) {
            int pro = 0;
            for (int i = 0; i < prices.length - 1; i++) {
                if (prices[i + 1] > prices[i]) {
                    pro += prices[i + 1] - prices[i];
                }
            }
            return pro;
        }
    }
}
```

123. `_123.java`

```
package com.fishercoder.solutions;

/**
 * 123. Best Time to Buy and Sell Stock III
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 * Design an algorithm to find the maximum profit. You may complete at most two
transactions.

Note:
You may not engage in multiple transactions at the same time (ie, you must sell the stock
before you buy again).
 */

public class _123 {

    public static class Solution1 {
        //this is a very clear solution and very highly upvoted in Discuss, but not
extensible to K solution.
        public int maxProfit(int[] prices) {
            int buy1 = Integer.MIN_VALUE;
            int buy2 = Integer.MIN_VALUE;
            int sell1 = 0;
            int sell2 = 0;
            for (int i = 0; i < prices.length; i++) {
                buy1 = Math.max(buy1, -prices[i]);
                sell1 = Math.max(sell1, buy1 + prices[i]);
                buy2 = Math.max(buy2, sell1 - prices[i]);
                sell2 = Math.max(sell2, buy2 + prices[i]);
            }
            return sell2;
        }
    }
}
```

124. `_124.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 124. Binary Tree Maximum Path Sum

 Given a binary tree, find the maximum path sum.
 For this problem, a path is defined as any sequence of nodes from some starting node to
any node
 in the tree along the parent-child connections.

 The path must contain at least one node and does not need to go through the root.

 For example:
 Given the below binary tree,

   1
  / \
 2   3

 Return 6.
 */
public class _124 {

  public static class Solution1 {
    int max = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
      dfs(root);
      return max;
    }

    private int dfs(TreeNode root) {
      if (root == null) {
        return 0;
      }

      int left = Math.max(dfs(root.left), 0);
      int right = Math.max(dfs(root.right), 0);

      max = Math.max(max, root.val + left + right);

      return root.val + Math.max(left, right);
    }
  }

  public static class Solution2 {
```

```
    /**
     * This one uses a map to cache, but surprisingly, it's 10% slower than all
submissions compared
     * with solution1
     */
    int max = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
      Map<TreeNode, Integer> map = new HashMap<>();
      dfs(root, map);
      return max;
    }

    private int dfs(TreeNode root, Map<TreeNode, Integer> map) {
      if (root == null) {
        return 0;
      }
      if (map.containsKey(root)) {
        return map.get(root);
      }
      int left = Math.max(0, dfs(root.left, map));
      int right = Math.max(0, dfs(root.right, map));
      max = Math.max(max, root.val + left + right);
      int pathSum = root.val + Math.max(left, right);
      map.put(root, pathSum);
      return pathSum;
    }
  }
}
```

125. `_125.java`

```
package com.fishercoder.solutions;

/**
 * 125. Valid Palindrome

 Given a string, determine if it is a palindrome, considering only alphanumeric
characters and ignoring cases.

 For example,
 "A man, a plan, a canal: Panama" is a palindrome.
 "race a car" is not a palindrome.

 Note:
 Have you consider that the string might be empty? This is a good question to ask during
an interview.
 For the purpose of this problem, we define empty string as valid palindrome.
 */

public class _125 {

  public static class Solution1 {
    public boolean isPalindrome(String s) {
      int i = 0;
      int j = s.length() - 1;
      char[] chars = s.toCharArray();
      while (i < j) {
        while (i < j && !Character.isLetterOrDigit(chars[i])) {
          i++;
        }
        while (i < j && !Character.isLetterOrDigit(chars[j])) {
          j--;
        }
        if (Character.toLowerCase(chars[i]) != Character.toLowerCase(chars[j])) {
          return false;
        }
        i++;
        j--;
      }
      return true;
    }
  }
}
```

126. `_126.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;

/**
 * 126. Word Ladder II

 Given two words (beginWord and endWord), and a dictionary's word list,
 find all shortest transformation sequence(s) from beginWord to endWord, such that:

 Only one letter can be changed at a time
 Each transformed word must exist in the word list. Note that beginWord is not a
transformed word.

 For example,
 Given:
 beginWord = "hit"
 endWord = "cog"
 wordList = ["hot","dot","dog","lot","log","cog"]

 Return
 [
 ["hit","hot","dot","dog","cog"],
 ["hit","hot","lot","log","cog"]
 ]

 Note:
 Return an empty list if there is no such transformation sequence.
 All words have the same length.
 All words contain only lowercase alphabetic characters.
 You may assume no duplicates in the word list.
 You may assume beginWord and endWord are non-empty and are not the same.
 */

public class _126 {

    public static class Solution1 {
        /** Reference: https://discuss.leetcode.com/topic/2857/share-two-similar-java-
solution-that-accpted-by-oj */

        Map<String, List<String>> map;
        List<List<String>> results;

        public List<List<String>> findLadders(String start, String end, List<String>
dict) {
```

```
            results = new ArrayList<>();
            if (dict.size() == 0) {
                return results;
            }

            int min = Integer.MAX_VALUE;

            Queue<String> queue = new ArrayDeque<>();
            queue.add(start);

            map = new HashMap<>();

            Map<String, Integer> ladder = new HashMap<>();
            for (String string : dict) {
                ladder.put(string, Integer.MAX_VALUE);
            }
            ladder.put(start, 0);

            dict.add(end);
            //BFS: Dijisktra search
            while (!queue.isEmpty()) {

                String word = queue.poll();

                int step = ladder.get(word)
                    + 1;//'step' indicates how many steps are needed to travel to one
word.

                if (step > min) {
                    break;
                }

                for (int i = 0; i < word.length(); i++) {
                    StringBuilder builder = new StringBuilder(word);
                    for (char ch = 'a'; ch <= 'z'; ch++) {
                        builder.setCharAt(i, ch);
                        String newWord = builder.toString();
                        if (ladder.containsKey(newWord)) {

                            if (step > ladder.get(newWord)) {
                                //Check if it is the shortest path to one word.
                                continue;
                            } else if (step < ladder.get(newWord)) {
                                queue.add(newWord);
                                ladder.put(newWord, step);
                            } else {
                                // It is a KEY line. If one word already appeared in one
ladder,

                                // Do not insert the same word inside the queue twice.
Otherwise it gets TLE.
                            }
                            if (map.containsKey(newWord)) {
                                //Build adjacent Graph
                                map.get(newWord).add(word);
```

```
                            } else {
                                List<String> list = new LinkedList();
                                list.add(word);
                                map.put(newWord, list);
                                //It is possible to write three lines in one:
                                //map.put(new_word,new LinkedList<String>
(Arrays.asList(new String[]{word})));
                                //Which one is better?
                            }

                            if (newWord.equals(end)) {
                                min = step;
                            }
                        }
                        //End if dict contains new_word
                    }
                    //End:Iteration from 'a' to 'z'
                }
                //End:Iteration from the first to the last
            }
            //End While

            //BackTracking
            LinkedList<String> result = new LinkedList<>();
            backTrace(end, start, result);

            return results;
        }

        private void backTrace(String word, String start, List<String> list) {
            if (word.equals(start)) {
                list.add(0, start);
                results.add(new ArrayList<>(list));
                list.remove(0);
                return;
            }
            list.add(0, word);
            if (map.get(word) != null) {
                for (String s : map.get(word)) {
                    backTrace(s, start, list);
                }
            }
            list.remove(0);
        }
    }
}
```

127. `_127.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 127. Word Ladder
 *
 *  Given two words (beginWord and endWord), and a dictionary's word list,
 *  find the length of shortest transformation sequence from beginWord to endWord, such
that:
 *  Only one letter can be changed at a time.
 *  Each transformed word must exist in the word list. Note that beginWord is not a
transformed word.

 For example,

 Given:
 beginWord = "hit"
 endWord = "cog"
 wordList = ["hot","dot","dog","lot","log","cog"]

 As one shortest transformation is "hit" -> "hot" -> "dot" -> "dog" -> "cog", return its
length 5.

 Note:

 Return 0 if there is no such transformation sequence.
 All words have the same length.
 All words contain only lowercase alphabetic characters.
 You may assume no duplicates in the word list.
 You may assume beginWord and endWord are non-empty and are not the same.
 */

public class _127 {
    public static class Solution1 {

        public int ladderLength(String beginWord, String endWord, List<String> wordList)
{
            Set<String> beginSet = new HashSet<>();
            Set<String> endSet = new HashSet<>();
            Set<String> visited = new HashSet<>();
            Set<String> dict = new HashSet<>(wordList);
            int len = 1;

            beginSet.add(beginWord);

            if (dict.contains(endWord)) {
                endSet.add(endWord);
            }
```

```
            while (!beginSet.isEmpty() && !endSet.isEmpty()) {
                Set<String> nextBeginSet = new HashSet<>();
                for (String word : beginSet) {
                    char[] chars = word.toCharArray();
                    for (int i = 0; i < chars.length; i++) {
                        for (char c = 'a'; c <= 'z'; c++) {
                            char old = chars[i];
                            chars[i] = c;
                            String newWord = new String(chars);
                            if (endSet.contains(newWord)) {
                                return len + 1;
                            }

                            if (!visited.contains(newWord) && dict.contains(newWord)) {
                                visited.add(newWord);
                                nextBeginSet.add(newWord);
                            }
                            chars[i] = old;
                        }
                    }
                }

                beginSet = nextBeginSet;
                len++;
            }
            return 0;
        }
    }
}
```

128. `_128.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 128. Longest Consecutive Sequence

 Given an unsorted array of integers, find the length of the longest consecutive elements
sequence.

 For example,
 Given [100, 4, 200, 1, 3, 2],
 The longest consecutive elements sequence is [1, 2, 3, 4]. Return its length: 4.

 Your algorithm should run in O(n) complexity.
 */
public class _128 {
  public static class Solution1 {
    public int longestConsecutive(int[] nums) {
      Map<Integer, Integer> map = new HashMap();
      //<value, index>
      UnionFind uf = new UnionFind(nums);
      for (int i = 0; i < nums.length; i++) {
        if (map.containsKey(nums[i])) {
          continue;
        }
        map.put(nums[i], i);
        if (map.containsKey(nums[i] - 1)) {
          uf.union(i, map.get(nums[i] - 1));
          //note: we want to union this index and nums[i]-1's root index which we can get
from the map
        }
        if (map.containsKey(nums[i] + 1)) {
          uf.union(i, map.get(nums[i] + 1));
        }
      }
      return uf.maxUnion();
    }

    class UnionFind {
      int[] ids;

      public UnionFind(int[] nums) {
        ids = new int[nums.length];
        for (int i = 0; i < nums.length; i++) {
          ids[i] = i;
        }
      }
```

```
      public void union(int i, int j) {
        int x = find(ids, i);
        int y = find(ids, j);
        ids[x] = y;
      }

      public int find(int[] ids, int i) {
        while (i != ids[i]) {
          ids[i] = ids[ids[i]];
          i = ids[i];
        }
        return i;
      }

      public boolean connected(int i, int j) {
        return find(ids, i) == find(ids, j);
      }

      public int maxUnion() {
        //this is O(n)
        int max = 0;
        int[] count = new int[ids.length];
        for (int i = 0; i < ids.length; i++) {
          count[find(ids, i)]++;
          max = max < count[find(ids, i)] ? count[find(ids, i)] : max;
        }
        return max;
      }
    }
  }

  public static class Solution2 {
    //inspired by this solution: https://discuss.leetcode.com/topic/25493/simple-fast-
java-solution-using-set
    public int longestConsecutive(int[] nums) {
      if (nums == null || nums.length == 0) {
        return 0;
      }

      Set<Integer> set = new HashSet();
      for (int i : nums) {
        set.add(i);
      }
      int max = 1;

      for (int num : nums) {
        if (set.remove(num)) {
          int val = num;
          int count = 1;
          while (set.remove(val - 1)) {
            val--;//we find all numbers that are smaller than num and remove them from
the set
          }
```

```
          count += num - val;

          val = num;
          while (set.remove(val + 1)) {
            val++;//then we find all numbers that are bigger than num and also remove
them from the set
          }
          count += val - num;

          max = Math.max(max, count);
        }
      }
      return max;
    }
  }
}
```

129. `_129.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 129. Sum Root to Leaf Numbers
 *
 * Given a binary tree containing digits from 0-9 only, each root-to-leaf path could
represent a number.

 An example is the root-to-leaf path 1->2->3 which represents the number 123.

 Find the total sum of all root-to-leaf numbers.

 For example,

 1
 / \
 2   3
 The root-to-leaf path 1->2 represents the number 12.
 The root-to-leaf path 1->3 represents the number 13.

 Return the sum = 12 + 13 = 25.

 */
public class _129 {
  public static class Solution1 {
    public int sumNumbers(TreeNode root) {
      if (root == null) {
        return 0;
      }
      List<Integer> allNumbers = new ArrayList();
      dfs(root, new StringBuilder(), allNumbers);
      int sum = 0;
      for (int i : allNumbers) {
        sum += i;
      }
      return sum;
    }

    private void dfs(TreeNode root, StringBuilder sb, List<Integer> allNumbers) {
      sb.append(root.val);
      if (root.left != null) {
        dfs(root.left, sb, allNumbers);
      }
      if (root.right != null) {
        dfs(root.right, sb, allNumbers);
      }
```

```
      if (root.left == null && root.right == null) {
        allNumbers.add(Integer.parseInt(sb.toString()));
      }
      sb.deleteCharAt(sb.length() - 1);
    }
  }

  public static class Solution2 {
    public int sumNumbers(TreeNode root) {
      return dfs(root, 0);
    }

    private int dfs(TreeNode root, int sum) {
      if (root == null) {
        return 0;
      }
      if (root.left == null && root.right == null) {
        return sum * 10 + root.val;
      }
      return dfs(root.left, sum * 10 + root.val) + dfs(root.right, sum * 10 + root.val);
    }
  }

}
```

130. `_130.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 130. Surrounded Regions
 *
 * Given a 2D board containing 'X' and 'O' (the letter O), capture all regions surrounded
by 'X'.

 A region is captured by flipping all 'O's into 'X's in that surrounded region.

 For example,
 X X X X
 X O O X
 X X O X
 X O X X

 After running your function, the board should be:

 X X X X
 X X X X
 X X X X
 X O X X
 */
public class _130 {

    public static class Solution1 {

        /**
         * I won't call this problem hard, it's just confusing, you'll definitely want to
clarify what
         * the problem means before coding. This problem eactually means: any grid that
is 'O' but on
         * the four edges, will never be marked to 'X'; furthermore, any grid that is 'O'
and that is
         * connected with the above type of 'O' will never be marked to 'X' as well; only
all other
         * nodes that has any one direct neighbor that is an 'X' will be marked to 'X'.
         */

        int[] dirs = new int[] {0, 1, 0, -1, 0};

        public void solve(char[][] board) {
            if (board == null || board.length == 0 || board[0].length == 0) {
                return;
            }
            int m = board.length;
            int n = board[0].length;
            Queue<int[]> queue = new LinkedList();
```

```
            //check first row and last row and mark all those '0' on these two rows to be
'+' to let them be different from other 'O',
            //at the same time, we put them into the queue to get ready for a BFS to mark
all those adjacent 'O' nodes to '+' as well
            for (int j = 0; j < n; j++) {
                if (board[0][j] == 'O') {
                    board[0][j] = '+';
                    queue.offer(new int[] {0, j});
                }
                if (board[m - 1][j] == 'O') {
                    board[m - 1][j] = '+';
                    queue.offer(new int[] {m - 1, j});
                }
            }

            //check first column and last column too
            for (int i = 0; i < m; i++) {
                if (board[i][0] == 'O') {
                    board[i][0] = '+';
                    queue.offer(new int[] {i, 0});
                }
                if (board[i][n - 1] == 'O') {
                    board[i][n - 1] = '+';
                    queue.offer(new int[] {i, n - 1});
                }
            }

            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                for (int i = 0; i < 4; i++) {
                    int x = curr[0] + dirs[i];
                    int y = curr[1] + dirs[i + 1];
                    if (x >= 0 && x < m && y >= 0 && y < n && board[x][y] == 'O') {
                        board[x][y] = '+';
                        queue.offer(new int[] {x, y});
                    }
                }
            }

            //now we can safely mark all other 'O' to 'X', also remember to put those '+'
back to 'O'
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (board[i][j] == 'O') {
                        board[i][j] = 'X';
                    } else if (board[i][j] == '+') {
                        board[i][j] = 'O';
                    }
                }
            }
        }
    }
}
```

131. `_131.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 131. Palindrome Partitioning

 Given a string s, partition s such that every substring of the partition is a
palindrome.

 Return all possible palindrome partitioning of s.

 For example, given s = "aab",
 Return

 [
 ["aa","b"],
 ["a","a","b"]
 ]

 */
public class _131 {

  public static class Solution1 {
    public List<List<String>> partition(String s) {
      List<List<String>> result = new ArrayList();
      int n = s.length();
      boolean[][] dp = new boolean[n][n];
      for (int i = 0; i < n; i++) {
        for (int j = 0; j <= i; j++) {
          if (s.charAt(j) == s.charAt(i) && (j + 1 >= i - 1 || dp[j + 1][i - 1])) {
            // j+1 >= i-1 means j and i are adjance to each other or only one char apart
from each other
            //dp[j+1][i-1] means its inner substring is a palindrome, so as long as
s.charAt(j) == s.charAt(i), then dp[j][i] must be a palindrome.
            dp[j][i] = true;
          }
        }
      }

      for (boolean[] list : dp) {
        for (boolean b : list) {
          System.out.print(b + ", ");
        }
        System.out.println();
      }
      System.out.println();

      backtracking(s, 0, dp, new ArrayList(), result);
```

```
      return result;
    }

    void backtracking(String s, int start, boolean[][] dp, List<String> temp,
        List<List<String>> result) {
      if (start == s.length()) {
        List<String> newTemp = new ArrayList(temp);
        result.add(newTemp);
      }
      for (int i = start; i < s.length(); i++) {
        if (dp[start][i]) {
          temp.add(s.substring(start, i + 1));
          backtracking(s, i + 1, dp, temp, result);
          temp.remove(temp.size() - 1);
        }
      }
    }
  }
}
```

132. `_132.java`

```
package com.fishercoder.solutions;

/**
 * 132. Palindrome Partitioning II

 Given a string s, partition s such that every substring of the partition is a
palindrome.

 Return the minimum cuts needed for a palindrome partitioning of s.

 For example, given s = "aab",
 Return 1 since the palindrome partitioning ["aa","b"] could be produced using 1 cut.

 */
public class _132 {

  /**This solution is cooler than Jiuzhang:
https://discuss.leetcode.com/topic/32575/easiest-java-dp-solution-97-36*/

    public static class Solution1 {
      //cut[i] stands for the minimum number of cut needed to cut [0, i] into palindromes
      //we initiazlie cut[i] with its max possible value which is i, this is because a
single char is naturally a palindrome, so, we'll cut this string into all single-char
substrings, which is the max cuts needed

      //dp[j][i] == true stands for s.substring(j,i) is a palindrome
      public int minCut(String s) {
        int n = s.length();
        char[] c = s.toCharArray();
        boolean[][] dp = new boolean[n][n];
        int[] cut = new int[n];

        for (int i = 0; i < n; i++) {
          cut[i] = i;
          for (int j = 0; j <= i; j++) {
            if (c[i] == c[j] && (j + 1 > i - 1 || dp[j + 1][i - 1])) {
              dp[j][i] = true;
              if (j == 0) {
                cut[i] = 0;
              } else {
                cut[i] = (cut[i] < cut[j - 1] + 1) ? cut[i] : cut[j - 1] + 1;
              }
            }
          }
        }

        return cut[n - 1];
      }
    }
}
```

133. `_133.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.UndirectedGraphNode;

import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;

/**
 * 133. Clone Graph

 Clone an undirected graph.
 Each node in the graph contains a label and a list of its neighbors.


 OJ's undirected graph serialization:
 Nodes are labeled uniquely.

 We use # as a separator for each node, and , as a separator for node label and each
neighbor of the node.
 As an example, consider the serialized graph {0,1,2#1,2#2,2}.

 The graph has a total of three nodes, and therefore contains three parts as separated by
#.

 First node is labeled as 0. Connect node 0 to both nodes 1 and 2.
 Second node is labeled as 1. Connect node 1 to node 2.
 Third node is labeled as 2. Connect node 2 to node 2 (itself), thus forming a self-
cycle.
 Visually, the graph looks like the following:

    1
   / \
  /   \
 0 --- 2
      / \
      \_/

 */
public class _133 {

  public static class Solution1 {
    public UndirectedGraphNode cloneGraph(UndirectedGraphNode node) {
      if (node == null) {
        return node;
      }

      Map<Integer, UndirectedGraphNode> map = new HashMap();
      Queue<UndirectedGraphNode> queue = new LinkedList();
      UndirectedGraphNode root = new UndirectedGraphNode(node.label);
```

```
      map.put(root.label, root);
      queue.offer(node);
      //remember to offer the original input node into the queue which contains all the
information
      while (!queue.isEmpty()) {
        UndirectedGraphNode curr = queue.poll();
        for (UndirectedGraphNode eachNode : curr.neighbors) {
          if (!map.containsKey(eachNode.label)) {
            map.put(eachNode.label, new UndirectedGraphNode(eachNode.label));
            queue.offer(eachNode);
          }
          map.get(curr.label).neighbors.add(map.get(eachNode.label));
        }
      }
      return root;
    }
  }
}
```

134. `_134.java`

```
package com.fishercoder.solutions;

/**
 * 134. Gas Station
 *
 * There are N gas stations along a circular route, where the amount of gas at station i
is gas[i].
 You have a car with an unlimited gas tank and it costs cost[i] of gas to travel from
station i to its next station (i+1).
 You begin the journey with an empty tank at one of the gas stations.

 Return the starting gas station's index if you can travel around the circuit once,
otherwise return -1.

 Note:
 The solution is guaranteed to be unique.
 */
public class _134 {

  public static class Solution1 {
    /** Credit: https://discuss.leetcode.com/topic/5088/my-ac-is-o-1-space-o-n-running-
time-solution-does-anybody-have-posted-this-solution */
    public int canCompleteCircuit(int[] gas, int[] cost) {
      int start = gas.length - 1;
      int end = 0;
      int sum = gas[start] - cost[start];
      while (start > end) {
        if (sum >= 0) {
          sum += gas[end] - cost[end];
          end++;
        } else {
          start--;
          sum += gas[start] - cost[start];
        }
      }
      return sum >= 0 ? start : -1;
    }
  }

}
```

135. `_135.java`

```
package com.fishercoder.solutions;

/**
 * 135. Candy

 There are N children standing in a line. Each child is assigned a rating value.

 You are giving candies to these children subjected to the following requirements:

 Each child must have at least one candy.
 Children with a higher rating get more candies than their neighbors.
 What is the minimum candies you must give?
 */
public class _135 {

  public static class Solution1 {
    public int candy(int[] ratings) {
      int[] candy = new int[ratings.length];
      for (int i = 0; i < ratings.length; i++) {
        candy[i] = 1;
      }

      for (int i = 0; i < ratings.length - 1; i++) {
        if (ratings[i] < ratings[i + 1]) {
          candy[i + 1] = candy[i] + 1;
        }
      }

      for (int i = ratings.length - 1; i > 0; i--) {
        if (ratings[i] < ratings[i - 1]) {
          candy[i - 1] = Math.max(candy[i - 1], candy[i] + 1);
        }
      }

      int sum = 0;
      for (int i : candy) {
        sum += i;
      }

      return sum;
    }
  }
}
```

136. `_136.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;

/**136. Single Number

 Given a non-empty array of integers, every element appears twice except for one. Find
that single one.

 Note:
 Your algorithm should have a linear runtime complexity. Could you implement it without
using extra memory?

 Example 1:
 Input: [2,2,1]
 Output: 1

 Example 2:
 Input: [4,1,2,1,2]
 Output: 4
*/

public class _136 {

  public static class Solution1 {
    /**
     * Approach 1: use set, since this problem explicitly states that every element
appears twice
     * and only one appears once so, we could safely remove the ones that are already in
the set,
     * O(n) time and O(n) space. HashTable approach works similarly like this one, but it
could be
     * more easily extend to follow-up questions.
     */
    public int singleNumber(int[] nums) {
      Set<Integer> set = new HashSet();
      for (int i : nums) {
        if (!set.add(i)) {
          set.remove(i);
        }
      }
      return set.iterator().next();
    }
  }

  public static class Solution2 {
    /**
     * Approach 2: bit manipulation, use exclusive or ^ to solve this problem: we're
using the trick
```

```
     * here: every number ^ itself will become zero, so, the only remaining element will
be the one
     * that appeared only once.
     */
    public int singleNumber(int[] nums) {
      int res = 0;
      for (int i : nums) {
        res ^= i;
      }
      return res;
    }
  }
}
```

137. `_137.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 137. Single Number II

Given an array of integers, every element appears three times except for one. Find that
single one.

Note:
Your algorithm should have a linear runtime complexity. Could you implement it without
using extra memory?
*/
public class _137 {

  public static class Solution1 {
    public int singleNumber(int[] nums) {
      Map<Integer, Integer> map = new HashMap();
      for (int i : nums) {
        map.put(i, map.getOrDefault(i, 0) + 1);
      }
      for (int key : map.keySet()) {
        if (map.get(key) != 3) {
          return key;
        }
      }
      return 0;
    }
  }

  public static class Solution2 {
    /** Credit: https://discuss.leetcode.com/topic/11877/detailed-explanation-and-
generalization-of-the-bitwise-operation-method-for-single-numbers/2 */
    public int singleNumber(int[] nums) {
      int counter1 = 0;
      int counter2 = 0;
      int mask = 0;
      for (int num : nums) {
        counter2 ^= counter1 & num;
        counter1 ^= num;
        mask = ~(counter1 & counter2);
        counter1 &= mask;
        counter2 &= mask;
      }
      return counter1;
    }
  }
}
```

138. `_138.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 138. Copy List with Random Pointer
 *
 * A linked list is given such that each node contains an additional random
 * pointer which could point to any node in the list or null.
 * Return a deep copy of the list.
 * */

public class _138 {
  public static class Solution1 {
    public Node copyRandomList(Node head) {
      /**Key is the original nodes, value is the new nodes we're deep copying to.*/
      Map<Node, Node> map = new HashMap();
      Node node = head;

      //loop for the first time: copy the node themselves with only labels
      while (node != null) {
        map.put(node, new Node(node.val));
        node = node.next;
      }

      //loop for the second time: copy random and next pointers
      node = head;
      while (node != null) {
        map.get(node).next = map.get(node.next);
        map.get(node).random = map.get(node.random);
        node = node.next;
      }

      return map.get(head);
    }

    // Definition for singly-linked list with a random pointer.
    class Node {
      int val;

      Node next;
      Node random;

      Node(int x) {
        this.val = x;
      }
    }
  }
}
```

139. `_139.java`

```
package com.fishercoder.solutions;

import java.util.List;

/**
 * 139. Word Break
 *
 * Given a non-empty string s and a dictionary wordDict containing a list of non-empty
words,
 * determine if s can be segmented into a space-separated sequence of one or more
dictionary words.
 * You may assume the dictionary does not contain duplicate words.

 For example, given
 s = "leetcode",
 dict = ["leet", "code"].

 Return true because "leetcode" can be segmented as "leet code".

 UPDATE (2017/1/4):
 The wordDict parameter had been changed to a list of strings (instead of a set of
strings).
 Please reload the code definition to get the latest changes.
 */

public class _139 {

    public static class Solution1 {
        /** this beats 70.46% submission. */
        public boolean wordBreak(String s, List<String> wordDict) {
            int n = s.length();
            boolean[] dp = new boolean[n + 1];
            dp[0] = true;
            for (int i = 1; i <= n; i++) {
                for (int j = 0; j < i; j++) {
                    if (dp[j] && wordDict.contains(s.substring(j, i))) {
                        dp[i] = true;
                        break;
                    }
                }
            }
            return dp[n];
        }
    }

    public static class Solution2 {
        /**
         * Added pruning.
         * this beats 89.91% submissions.
         */
        public boolean wordBreak(String s, List<String> wordDict) {
```

```
            int maxLen = Integer.MIN_VALUE;
            for (String word : wordDict) {
                maxLen = (word.length() > maxLen) ? word.length() : maxLen;
            }

            int n = s.length();
            boolean[] dp = new boolean[n + 1];
            dp[0] = true;
            for (int i = 1; i <= n; i++) {
                for (int j = 0; j < i; j++) {
                    if ((i - j) > maxLen) {
                        continue;
                    }
                    if (dp[j] && wordDict.contains(s.substring(j, i))) {
                        dp[i] = true;
                        break;
                    }
                }
            }
            return dp[n];
        }
    }

    public static class Solution3 {
        /**
         * Added pruning, plus start from the end to check.
         * This beats 95.20% submissions.
         */
        public boolean wordBreak(String s, List<String> wordDict) {
            int maxLen = Integer.MIN_VALUE;
            for (String word : wordDict) {
                maxLen = (word.length() > maxLen) ? word.length() : maxLen;
            }

            int n = s.length();
            boolean[] dp = new boolean[n + 1];
            dp[0] = true;
            for (int i = 1; i <= n; i++) {
                for (int lastWordLength = 1; lastWordLength <= i && lastWordLength <=
maxLen;
                        lastWordLength++) {
                    if (!dp[i - lastWordLength]) {
                        continue;
                    }
                    String sub = s.substring(i - lastWordLength, i);
                    if (wordDict.contains(sub)) {
                        dp[i] = true;
                        break;
                    }
                }
            }
            return dp[n];
        }
    }
```

```
}
```

140. `_140.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

/**
 * 140. Word Break II

 Given a non-empty string s and a dictionary wordDict containing a list of non-empty
words,
 add spaces in s to construct a sentence where each word is a valid dictionary word.
 Return all such possible sentences.

 Note:

 The same word in the dictionary may be reused multiple times in the segmentation.
 You may assume the dictionary does not contain duplicate words.

 Example 1:

 Input:
 s = "catsanddog"
 wordDict = ["cat", "cats", "and", "sand", "dog"]
 Output:
 [
 "cats and dog",
 "cat sand dog"
 ]

 Example 2:

 Input:
 s = "pineapplepenapple"
 wordDict = ["apple", "pen", "applepen", "pine", "pineapple"]
 Output:
 [
 "pine apple pen apple",
 "pineapple pen apple",
 "pine applepen apple"
 ]
 Explanation: Note that you are allowed to reuse a dictionary word.

 Example 3:

 Input:
 s = "catsandog"
 wordDict = ["cats", "dog", "sand", "and", "cat"]
 Output:
 []
 */
```

```
public class _140 {
  public static class Solution1 {
    public List<String> wordBreak(String s, List<String> wordDict) {
      return dfs(s, wordDict, new HashMap<>());
    }

    List<String> dfs(String s, List<String> wordDict, HashMap<String, List<String>> map)
{
      if (map.containsKey(s)) {
        return map.get(s);
      }

      ArrayList<String> result = new ArrayList<>();
      if (s.length() == 0) {
        result.add("");
        return result;
      }

      for (String word : wordDict) {
        if (s.startsWith(word)) {
          List<String> subList = dfs(s.substring(word.length()), wordDict, map);
          for (String sub : subList) {
            result.add(word + (sub.length() == 0 ? "" : " ") + sub);
          }
        }
      }
      map.put(s, result);
      return result;
    }
  }
}
```

141. `_141.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.HashSet;
import java.util.Set;

/**
 * 141. Linked List Cycle

Given a linked list, determine if it has a cycle in it.

Follow up:
Can you solve it without using extra space?
 */
public class _141 {

        public static class Solution1 {
                public boolean hasCycle(ListNode head) {
                        Set<ListNode> set = new HashSet();
                        while (head != null) {
                                if (!set.add(head)) {
                                        return true;
                                }
                                head = head.next;
                        }
                        return false;
                }
        }

        public static class Solution2 {
                public boolean hasCycle(ListNode head) {
                        ListNode slow = head;
                        ListNode fast = head;
                        while (fast != null && fast.next != null) {
                                fast = fast.next.next;
                                slow = slow.next;
                                if (fast == slow) {
                                        return true;
                                }
                        }
                        return false;
                }
        }
}
```

142. `_142.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 142. Linked List Cycle II

 Given a linked list, return the node where the cycle begins. If there is no cycle,
return null.

 Note: Do not modify the linked list.

 Follow up:
 Can you solve it without using extra space?
 */
public class _142 {

  public static class Solution1 {
    public ListNode detectCycle(ListNode head) {
      ListNode slow = head;
      ListNode fast = head;
      while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
          ListNode slow2 = head;
          while (slow2 != slow) {
            slow = slow.next;
            slow2 = slow2.next;
          }
          return slow;
        }
      }
      return null;
    }
  }
}
```

143. `_143.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 143. Reorder List
 *
 * Given a singly linked list L: L0→L1→…→Ln-1→Ln,
 reorder it to: L0→Ln→L1→Ln-1→L2→Ln-2→…

 You must do this in-place without altering the nodes' values.

 For example,
 Given {1,2,3,4}, reorder it to {1,4,2,3}.
 */
public class _143 {

    public static class Solution1 {
        public void reorderList(ListNode head) {
            if (head == null || head.next == null) {
                return;
            }
            /* first we use two pointers to separate this list into two parts */
            ListNode slowNode = head;
            ListNode fastNode = head;
            while (fastNode.next != null) {
                fastNode = fastNode.next;
                if (fastNode.next != null) {
                    fastNode = fastNode.next;
                } else {
                    break;
                }
                slowNode = slowNode.next;
            }
            // two sublist heads
            ListNode head1 = head;
            ListNode head2 = slowNode.next;
            // detach the two sublists;
            slowNode.next = null;

            // reverse the second sublist
            ListNode cur = head2;
            ListNode post = cur.next;
            cur.next = null;
            while (post != null) {
                ListNode temp = post.next;
                post.next = cur;
                cur = post;
                post = temp;
            }
            head2 = cur;// the new head of the reversed sublist
```

```
            // merge the two sublists as required
            ListNode p = head1;
            ListNode q = head2;
            while (q != null) {
                ListNode temp1 = p.next;
                ListNode temp2 = q.next;
                p.next = q;
                q.next = temp1;
                p = temp1;
                q = temp2;
            }
        }
    }
}
```

144. `_144.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;

/**
 * 144. Binary Tree Preorder Traversal

 Given a binary tree, return the preorder traversal of its nodes' values.

 For example:
 Given binary tree {1,#,2,3},
 1
  \
   2
  /
 3
 return [1,2,3].

 Note: Recursive solution is trivial, could you do it iteratively?*/

public class _144 {
  public static class Solution1 {
    public List<Integer> preorderTraversal(TreeNode root) {
      List<Integer> list = new ArrayList();
      if (root == null) {
        return list;
      }
      Deque<TreeNode> stack = new ArrayDeque<>();
      stack.push(root);
      while (!stack.isEmpty()) {
        TreeNode curr = stack.pop();
        list.add(curr.val);
        /**We push right nodes onto the stack first, since they'll be popped out later
than
         * the left nodes, to meet the preorder: root -> left -> right. */
        if (curr.right != null) {
          stack.push(curr.right);
        }
        if (curr.left != null) {
          stack.push(curr.left);
        }
      }
      return list;
    }
  }
```

```
  public static class Solution2 {
    public List<Integer> preorderTraversal(TreeNode root) {
      List<Integer> list = new ArrayList();
      return pre(root, list);
    }

    List<Integer> pre(TreeNode root, List<Integer> list) {
      if (root == null) {
        return list;
      }
      list.add(root.val);
      pre(root.left, list);
      pre(root.right, list);
      return list;
    }
  }
}
```

145. `_145.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Stack;

/**
 * 145. Binary Tree Postorder Traversal

 Given a binary tree, return the postorder traversal of its nodes' values.

 For example:
 Given binary tree {1,#,2,3},
 1
  \
   2
  /
 3
 return [3,2,1].

 Note: Recursive solution is trivial, could you do it iteratively?*/

public class _145 {
  public static class Solution1 {
    /**
     * A tricky one: Modify the code for pre-order traversal
     * so that it becomes root->right->left,
     * and then reverse the result to get left->right->root.
     */
    public static List<Integer> postorderTraversal(TreeNode root) {
      List<Integer> result = new ArrayList();
      if (root == null) {
        return result;
      }
      Stack<TreeNode> stack = new Stack();
      stack.push(root);
      while (!stack.isEmpty()) {
        root = stack.pop();
        result.add(root.val);
        if (root.left != null) {
          stack.push(root.left);
        }
        if (root.right != null) {
          stack.push(root.right);
        }
      }
      Collections.reverse(result);
      return result;
```

```
    }
  }

  public static class Solution2 {
    public List<Integer> postorderTraversal(TreeNode root) {
      List<Integer> result = new ArrayList();
      return post(root, result);
    }

    List<Integer> post(TreeNode root, List<Integer> result) {
      if (root == null) {
        return result;
      }
      post(root.left, result);
      post(root.right, result);
      result.add(root.val);
      return result;
    }
  }
}
```

146. `_146.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * 146. LRU Cache
 *
 * Design and implement a data structure for Least Recently Used (LRU) cache. It should
support the following operations: get and put.

 get(key) - Get the value (will always be positive) of the key if the key exists in the
cache, otherwise return -1.
 put(key, value) - Set or insert the value if the key is not already present.
 When the cache reached its capacity, it should invalidate the least recently used item
before inserting a new item.

 Follow up:
 Could you do both operations in O(1) time complexity?

 Example:

 LRUCache cache = new LRUCache(2);//capacity

        cache.put(1, 1);
        cache.put(2, 2);
        cache.get(1);       // returns 1
        cache.put(3, 3);    // evicts key 2
        cache.get(2);       // returns -1 (not found)
        cache.put(4, 4);    // evicts key 1
        cache.get(1);       // returns -1 (not found)
        cache.get(3);       // returns 3
        cache.get(4);       // returns 4
        */

public class _146 {

    public class Solution1 {
        public class LRUCache {
            /**
             * The shortest implementation is to use LinkedHashMap:
             * specify a size of the LinkedHashMap;
             * override the removeEldestEntry method when its size exceeds max size:
             *
https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashMap.html#removeEldestEntry-
java.util.Map.Entry-
             * in the constructor, set the last boolean variable to be true: it means the
ordering mode,
             * if we set it to be true, it means in access order, false, means it's in
insertion order:
```

```
             *
https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashMap.html#LinkedHashMap-int-
float-boolean-
             */

            private Map<Integer, Integer> cache;
            private final int max;

            public LRUCache(int capacity) {
                max = capacity;
                cache = new LinkedHashMap<Integer, Integer>(capacity, 1.0f, true) {
                    public boolean removeEldestEntry(Map.Entry eldest) {
                        return cache.size() > max;
                    }
                };
            }

            public int get(int key) {
                return cache.getOrDefault(key, -1);
            }

            public void set(int key, int value) {
                cache.put(key, value);
            }
        }
    }

    public class Solution2 {
        public class LRUCache {
            /**
             * The more verbose solution is to write a doubly linked list plus a map.
             */
            private class Node {
                int key;
                int value;

                LRUCache.Node prev;
                LRUCache.Node next;

                Node(int k, int v) {
                    this.key = k;
                    this.value = v;
                }

                Node() {
                    this.key = 0;
                    this.value = 0;
                }
            }

            private int capacity;
            private int count;
            private LRUCache.Node head;
            private LRUCache.Node tail;
```

```
            private Map<Integer, LRUCache.Node> map;
            // ATTN: the value should be Node type! This is the whole point of having a
class called Node!

            public LRUCache(int capacity) {
                this.capacity = capacity;
                this.count = 0;// we need a count to keep track of the number of elements
in the cache so
                // that we know when to evict the LRU one from the cache
                this.map = new HashMap();
                head = new LRUCache.Node();
                tail = new LRUCache.Node();
                head.next = tail;
                tail.prev = head;
            }

            public int get(int key) {
                LRUCache.Node node = map.get(key);
                // HashMap allows value to be null, this is superior than HashTable!
                if (node == null) {
                    return -1;
                } else {

                    /**Do two operations: this makes the process more clear:
                     * remove the old node first, and then
                     * just add the node again.
                     * This will guarantee that this node will be at the latest position:
                     * the most recently used position.*/
                    remove(node);
                    add(node);

                    return node.value;
                }
            }

            public void set(int key, int value) {
                LRUCache.Node node = map.get(key);
                if (node == null) {
                    node = new LRUCache.Node(key, value);
                    map.put(key, node);
                    add(node);
                    count++;

                    if (count > capacity) {
                        /** ATTN: It's tail.prev, not tail, because tail is always an
invalid node, it
                         doesn't contain anything, it's always the tail.prev that is the
last node in the
                         cache*/
                        LRUCache.Node toDelete = tail.prev;
                        map.remove(toDelete.key);
                        remove(toDelete);
                        count--;
                    }
```

```
                } else {
                    remove(node);
                    node.value = value;
                    add(node);
                }
            }

            private void remove(LRUCache.Node node) {
                LRUCache.Node next = node.next;
                LRUCache.Node prev = node.prev;
                prev.next = next;
                next.prev = prev;
            }

            private void add(LRUCache.Node node) {
                // ATTN: we'll always add the node into the first position: head.next!!!!
                LRUCache.Node next = head.next;
                head.next = node;
                node.next = next;
                node.prev = head;
                next.prev = node;
            }
        }
    }
}
```

147. `_147.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 147. Insertion Sort List
 *
 * Sort a linked list using insertion sort.
 *
 * Algorithm of Insertion Sort:
 *
 *     Insertion sort iterates, consuming one input element each repetition, and growing
a sorted output list.
 *     At each iteration, insertion sort removes one element from the input data, finds
the location it belongs within the sorted list, and inserts it there.
 *     It repeats until no input elements remain.
 *
 *
 * Example 1:
 *
 * Input: 4->2->1->3
 * Output: 1->2->3->4
 *
 * Example 2:
 *
 * Input: -1->5->3->4->0
 * Output: -1->0->3->4->5
 */
public class _147 {

  public static class Solution1 {
    public ListNode insertionSortList(ListNode head) {
      ListNode temp = head;
      List<Integer> list = new ArrayList<>();
      while (temp != null) {
        list.add(temp.val);
        temp = temp.next;
      }
      Integer[] nums = list.toArray(new Integer[list.size()]);
      for (int i = 1; i < list.size(); i++) {
        for (int j = i - 1; j >= 0; j--) {
          if (nums[j] > nums[j + 1]) {
            int tempNum = nums[j];
            nums[j] = nums[j + 1];
            nums[j + 1] = tempNum;
          }
        }
      }
```

```
      ListNode newHead = head;
      for (int i = 0; i < nums.length; i++) {
        newHead.val = nums[i];
        newHead = newHead.next;
      }
      return head;
    }
  }
}
```

148. `_148.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 148. Sort List
 *
 * Sort a linked list in O(n log n) time using constant space complexity.
 */
public class _148 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/18100/java-merge-sort-solution
         * But this is not using constant space.
         */
        public ListNode sortList(ListNode head) {
            if (head == null || head.next == null) {
                return head;
            }

            //Step 1: split the list into halves
            ListNode prev = null;
            ListNode slow = head;
            ListNode fast = head;
            while (fast != null && fast.next != null) {
                prev = slow;
                fast = fast.next.next;
                slow = slow.next;
            }
            prev.next = null;

            //step 2: sort each half
            ListNode l1 = sortList(head);
            ListNode l2 = sortList(slow);

            //step 3: merge the two halves
            return merge(l1, l2);
        }

        private ListNode merge(ListNode l1, ListNode l2) {
            ListNode result = new ListNode(0);
            ListNode tmp = result;

            while (l1 != null && l2 != null) {
                if (l1.val < l2.val) {
                    tmp.next = l1;
                    l1 = l1.next;
                } else {
                    tmp.next = l2;
                    l2 = l2.next;
```

```
                }
                tmp = tmp.next;
            }

            if (l1 != null) {
                tmp.next = l1;
            }
            if (l2 != null) {
                tmp.next = l2;
            }
            return result.next;
        }
    }
}
```

149. `_149.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Point;
import java.util.HashMap;
import java.util.Map;

/**
 * 149. Max Points on a Line
 *
 * Given n points on a 2D plane, find the maximum number of points that lie on the same
straight line.
 *
 * Example 1:
 * Input: [[1,1],[2,2],[3,3]]
 * Output: 3
 * Explanation:
 * ^
 * |
 * |        o
 * |     o
 * |  o
 * +------------->
 * 0  1  2  3  4
 *
 * Example 2:
 * Input: [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
 * Output: 4
 * Explanation:
 * ^
 * |
 * |  o
 * |     o        o
 * |        o
 * |  o        o
 * +------------------->
 * 0  1  2  3  4  5  6
 *
 */
public class _149 {

  public static class Solution1 {
    /** credit: https://leetcode.com/problems/max-points-on-a-line/discuss/47113/A-java-
solution-with-notes */
    public int maxPoints(Point[] points) {
      if (points == null) {
        return 0;
      }
      if (points.length <= 2) {
        return points.length;
      }
```

```
      Map<Integer, Map<Integer, Integer>> map = new HashMap<>();
      int result = 0;
      for (int i = 0; i < points.length; i++) {
        map.clear();
        int overlap = 0;
        int max = 0;
        for (int j = i + 1; j < points.length; j++) {
          int x = points[j].x - points[i].x;
          int y = points[j].y - points[i].y;
          if (x == 0 && y == 0) {
            overlap++;
            continue;
          }
          int gcd = generateGCD(x, y);
          if (gcd != 0) {
            x /= gcd;
            y /= gcd;
          }

          if (map.containsKey(x)) {
            if (map.get(x).containsKey(y)) {
              map.get(x).put(y, map.get(x).get(y) + 1);
            } else {
              map.get(x).put(y, 1);
            }
          } else {
            Map<Integer, Integer> m = new HashMap<>();
            m.put(y, 1);
            map.put(x, m);
          }
          max = Math.max(max, map.get(x).get(y));
        }
        result = Math.max(result, max + overlap + 1);
      }
      return result;
    }

    private int generateGCD(int a, int b) {
      if (b == 0) {
        return a;
      } else {
        return generateGCD(b, a % b);
      }
    }
  }
}
```

150. `_150.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;
import java.util.Stack;

/**
 * 150. Evaluate Reverse Polish Notation
 *
 * Evaluate the value of an arithmetic expression in Reverse Polish Notation.
 *
 * Valid operators are +, -, *, /. Each operand may be an integer or another expression.
 *
 * Note:
 *
 *     Division between two integers should truncate toward zero.
 *     The given RPN expression is always valid. That means the expression would always
evaluate to a result and there won't be any divide by zero operation.
 *
 * Example 1:
 *
 * Input: ["2", "1", "+", "3", "*"]
 * Output: 9
 * Explanation: ((2 + 1) * 3) = 9
 *
 * Example 2:
 *
 * Input: ["4", "13", "5", "/", "+"]
 * Output: 6
 * Explanation: (4 + (13 / 5)) = 6
 *
 * Example 3:
 *
 * Input: ["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"]
 * Output: 22
 * Explanation:
 *   ((10 * (6 / ((9 + 3) * -11))) + 17) + 5
 * = ((10 * (6 / (12 * -11))) + 17) + 5
 * = ((10 * (6 / -132)) + 17) + 5
 * = ((10 * 0) + 17) + 5
 * = (0 + 17) + 5
 * = 17 + 5
 * = 22
 */
public class _150 {

    public static class Solution1 {
        public int evalRPN(String[] tokens) {
            Stack<String> stack = new Stack<>();
            Set<String> op = new HashSet();
            op.add("+");
```

```
            op.add("-");
            op.add("*");
            op.add("/");

            int exp = 0;
            String operand1 = "";
            String operand2 = "";
            for (int i = 0; i < tokens.length; ) {
                while ((i < tokens.length) && (!op.contains(tokens[i]))) {
                    stack.push(tokens[i]);
                    i++;
                }
                if (i == tokens.length) {
                    if (!stack.isEmpty()) {
                        return Integer.parseInt(stack.pop());
                    }
                } else if (op.contains(tokens[i])) {
                    if (!stack.isEmpty()) {
                        operand2 = stack.pop();
                    }
                    if (!stack.isEmpty() && !op.contains(stack.peek())) {
                        operand1 = stack.pop();
                    }
                    if (tokens[i].equals("+")) {
                        exp = Integer.parseInt(operand1) + Integer.parseInt(operand2);
                    } else if (tokens[i].equals("-")) {
                        exp = Integer.parseInt(operand1) - Integer.parseInt(operand2);
                    } else if (tokens[i].equals("*")) {
                        exp = Integer.parseInt(operand1) * Integer.parseInt(operand2);
                    } else if (tokens[i].equals("/")) {
                        exp = Integer.parseInt(operand1) / Integer.parseInt(operand2);
                    } else {
                        exp = Integer.parseInt(operand2);
                    }
                    stack.push(String.valueOf(exp));
                    i++;
                }
            }
            return Integer.parseInt(stack.pop());
        }
    }
}
```

151. `_151.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 151. Reverse Words in a String

  Given an input string, reverse the string word by word.
  For example,
  Given s = "the sky is blue",
  return "blue is sky the".

 Clarification:

 What constitutes a word?
 A sequence of non-space characters constitutes a word.
 Could the input string contain leading or trailing spaces?
 Yes. However, your reversed string should not contain leading or trailing spaces.
 How about multiple spaces between two words?
 Reduce them to a single space in the reversed string.
 */

public class _151 {
  public static class Solution1 {
    public String reverseWords(String s) {
      s.trim();
      if (s == null || s.length() == 0) {
        return "";
      }
      String[] words = s.split(" ");
      if (words == null || words.length == 0) {
        return "";
      }
      Deque<String> stack = new ArrayDeque<>();
      for (String word : words) {
        if (!word.equals("")) {
          stack.offer(word);
        }
      }
      StringBuilder stringBuilder = new StringBuilder();
      while (!stack.isEmpty()) {
        stringBuilder.append(stack.pollLast()).append(" ");
      }
      return stringBuilder.substring(0, stringBuilder.length() - 1).toString();
    }
  }
}
```

152. `_152.java`

```
package com.fishercoder.solutions;

/**
 * 152. Maximum Product Subarray

 Find the contiguous subarray within an array (containing at least one number) which has the largest product.

 For example, given the array [2,3,-2,4],
 the contiguous subarray [2,3] has the largest product = 6.
 */
public class _152 {
  public static class Solution1 {
    public int maxProduct(int[] nums) {
      int pos = nums[0];
      int neg = nums[0];
      int max = nums[0];
      for (int i = 1; i < nums.length; i++) {
        int temp = pos;
        pos = Math.max(nums[i], nums[i] * ((nums[i] >= 0) ? pos : neg));
        neg = Math.min(nums[i], nums[i] * ((nums[i] >= 0) ? neg : temp));
        max = Math.max(max, pos);
      }
      return max;
    }
  }
}
```

153. `_153.java`

```
package com.fishercoder.solutions;

/**
 * 153. Find Minimum in Rotated Sorted Array

  Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.

 (i.e.,  [0,1,2,4,5,6,7] might become  [4,5,6,7,0,1,2]).

 Find the minimum element.

 You may assume no duplicate exists in the array.

 Example 1:
 Input: [3,4,5,1,2]
 Output: 1

 Example 2:
 Input: [4,5,6,7,0,1,2]
 Output: 0

 */
public class _153 {
  public static class Solution1 {
    public int findMin(int[] nums) {
      int left = 0;
      int right = nums.length - 1;
      if (nums[left] < nums[right]) {
        return nums[left];
      }
      int min = nums[0];
      while (left + 1 < right) {
        int mid = left + (right - left) / 2;
        min = Math.min(min, nums[mid]);
        if (nums[mid] > nums[left]) {
          min = Math.min(nums[left], min);
          left = mid + 1;
        } else if (nums[mid] < nums[left]) {
          right = mid - 1;
        }
      }
      min = Math.min(min, Math.min(nums[left], nums[right]));
      return min;
    }
  }
}
```

154. `_154.java`

```
package com.fishercoder.solutions;

/**
 * 154. Find Minimum in Rotated Sorted Array II

 Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.

 (i.e.,  [0,1,2,4,5,6,7] might become  [4,5,6,7,0,1,2]).

 Find the minimum element.

 The array may contain duplicates.

 Example 1:
 Input: [1,3,5]
 Output: 1

 Example 2:
 Input: [2,2,2,0,1]
 Output: 0

 Note:
 This is a follow up problem to Find Minimum in Rotated Sorted Array.
 Would allow duplicates affect the run-time complexity? How and why?

 */
public class _154 {
  public static class Solution1 {
    public int findMin(int[] nums) {
      int left = 0;
      int right = nums.length - 1;
      if (nums[left] < nums[right]) {
        return nums[left];
      }
      int min = nums[0];
      while (left + 1 < right) {
        int mid = left + (right - left) / 2;
        min = Math.min(min, nums[mid]);
        if (nums[mid] > nums[left]) {
          min = Math.min(nums[left], min);
          left = mid + 1;
        } else if (nums[mid] < nums[left]) {
          right = mid - 1;
        } else {
          left++;
        }
      }
      min = Math.min(min, Math.min(nums[left], nums[right]));
      return min;
    }
```

```
  }
}
```

155. `_155.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 155. Min Stack
 * Design a stack that supports push, pop, top, and retrieving the minimum element in
constant time.

 * push(x) -- Push element x onto stack.
 * pop() -- Removes the element on top of the stack.
 * top() -- Get the top element.
 * getMin() -- Retrieve the minimum element in the stack.

 * Example:
 * MinStack minStack = new MinStack();
 * minStack.push(-2);
 * minStack.push(0);
 * minStack.push(-3);
 * minStack.getMin();   --> Returns -3.
 * minStack.pop();
 * minStack.top();      --> Returns 0.
 * minStack.getMin();   --> Returns -2.
 */

public class _155 {

    public static class Solution1 {
        class MinStack {
            private Stack<Integer> stack;
            private int min;

            /**
             * initialize your data structure here.
             */
            public MinStack() {
                stack = new Stack();
                min = Integer.MAX_VALUE;
            }

            public void push(int x) {
                /**All the trick happens here, we push the second minimum number onto the
stack before we push the newer one,
                 * this way, when popping, we could always get the next minimum one in
constant time.*/
                if (x <= min) {
                    stack.push(min);
                    min = x;
                }
                stack.push(x);
            }
```

```
            public void pop() {
                /**if the value on the top of the stack happens to be the current
minimum, we'll pop twice and change
                 * the current min value to be the last min value */
                if (min == stack.pop()) {
                    min = stack.pop();
                }
            }

            public int top() {
                return stack.peek();
            }

            public int getMin() {
                return min;
            }
        }
    }
}
```

156. `_156.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 156. Binary Tree Upside Down
 *
 * Given a binary tree where all the right nodes are either leaf nodes with a sibling
 * (a left node that shares the same parent node) or empty,
 * flip it upside down and turn it into a tree where the original right nodes turned into
left leaf nodes. Return the new root.

 For example:
 Given a binary tree {1,2,3,4,5},
    1
   / \
  2  3
 / \
4   5

 return the root of the binary tree [4,5,2,#,#,3,1].
    4
   / \
  5  2
    / \
   3  1

 confused what "{1,#,2,3}" means? > read more on how binary tree is serialized on OJ.
 */
public class _156 {

    public static class Solution1 {
        public TreeNode upsideDownBinaryTree(TreeNode root) {
            if (root == null || root.left == null && root.right == null) {
                return root;
            }
            TreeNode newRoot = upsideDownBinaryTree(root.left);
            root.left.left = root.right;
            root.left.right = root;
            root.left = null;
            root.right = null;
            return newRoot;
        }
    }
}
```

157. `_157.java`

```
package com.fishercoder.solutions;

/**
 * 157. Read N Characters Given Read4
 *
 * The API: int read4(char *buf) reads 4 characters at a time from a file.
 *
 * The return value is the actual number of characters read. For example, it returns 3 if
there is only 3 characters left in the file.
 *
 * By using the read4 API, implement the function int read(char *buf, int n) that reads n
characters from the file.
 *
 * Example 1:
 *
 * Input: buf = "abc", n = 4
 * Output: "abc"
 * Explanation: The actual number of characters read is 3, which is "abc".
 *
 * Example 2:
 *
 * Input: buf = "abcde", n = 5
 * Output: "abcde"
 *
 * Note:
 * The read function will only be called once for each test case.
 *
 */
public class _157 {
    public static class Solution1 {
        public int read(char[] buf, int n) {
            int index = 0;
            int next = 0;
            char[] buffer = new char[4];
            while (index < n && (next = read4(buffer)) != 0) {
                for (int i = 0; i < next && index < n; index++, i++) {
                    buf[index] = buffer[i];
                }
            }
            return index;
        }

        private int read4(char[] buffer) {
            //this is a fake method to make Eclipse happy
            return 0;
        }
    }
}
```

158. `_158.java`

```
package com.fishercoder.solutions;

/**
 158. Read N Characters Given Read4 II - Call multiple times
 The API: int read4(char *buf) reads 4 characters at a time from a file.

 The return value is the actual number of characters read. For example, it returns 3 if
there is only 3 characters left in the file.

 By using the read4 API, implement the function int read(char *buf, int n) that reads n
characters from the file.

 Note:
 The read function may be called multiple times.

 Example 1:

 Given buf = "abc"
 read("abc", 1) // returns "a"
 read("abc", 2); // returns "bc"
 read("abc", 1); // returns ""

 Example 2:

 Given buf = "abc"
 read("abc", 4) // returns "abc"
 read("abc", 1); // returns ""
 */
public class _158 {

  public static class Solution1 {
    /**
     * @param buf Destination buffer
     * @param n   Maximum number of characters to read
     * @return The number of characters read
     */
    private int buffPtr = 0;
    private int buffCnt = 0;
    private char[] buff = new char[4];

    public int read(char[] buf, int n) {
      int ptr = 0;
      while (ptr < n) {
        if (buffPtr == 0) {
          buffCnt = read4(buff);
        }
        if (buffCnt == 0) {
          break;
        }
        while (ptr < n && buffPtr < buffCnt) {
          buf[ptr++] = buff[buffPtr++];
```

```
        }
        if (buffPtr >= buffCnt) {
          buffPtr = 0;
        }
      }
      return ptr;
    }

    //This is a fake method to make IDE happy.
    private int read4(char[] buff) {
      return 1;
    }
  }
}
```

159. `_159.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 159. Longest Substring with At Most Two Distinct Characters

 Given a string s , find the length of the longest substring t  that contains at most 2
distinct characters.

 Example 1:

 Input: "eceba"
 Output: 3
 Explanation: t is "ece" which its length is 3.

 Example 2:

 Input: "ccaabbb"
 Output: 5
 Explanation: t is "aabbb" which its length is 5.
 */
public class _159 {
  public static class Solution1 {
    public int lengthOfLongestSubstringTwoDistinct(String s) {
      if (s.length() < 1) {
        return 0;
      }
      Map<Character, Integer> index = new HashMap<>();
      int lo = 0;
      int hi = 0;
      int maxLength = 0;
      while (hi < s.length()) {
        if (index.size() <= 2) {
          char c = s.charAt(hi);
          index.put(c, hi);
          hi++;
        }
        if (index.size() > 2) {
          int leftMost = s.length();
          for (int i : index.values()) {
            leftMost = Math.min(leftMost, i);
          }
          char c = s.charAt(leftMost);
          index.remove(c);
          lo = leftMost + 1;
        }
        maxLength = Math.max(maxLength, hi - lo);
      }
      return maxLength;
```

```
    }
  }
}
```

160. `_160.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.HashSet;
import java.util.Set;

/**
 * 160. Intersection of Two Linked Lists
 Write a program to find the node at which the intersection of two singly linked lists
begins.


 For example, the following two linked lists:

 A:          a1 → a2
                    ↘
                    c1 → c2 → c3
                    ↗
 B:     b1 → b2 → b3
 begin to intersect at node c1.

 Example 1:
 A:             4 → 1
                    ↘
                    8 → 4 → 5
                    ↗
 B:          5 → 0 → 1


 Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB =
3
 Output: Reference of the node with value = 8
 Input Explanation: The intersected node's value is 8 (note that this must not be 0 if
the two lists intersect).
 From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as
[5,0,1,8,4,5].
 There are 2 nodes before the intersected node in A; There are 3 nodes before the
intersected node in B.


 Example 2:
 A:         0 -> 9 → 1
                    ↘
                    2 → 4
                    ↗
 B:                3

 Input: intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
 Output: Reference of the node with value = 2
 Input Explanation: The intersected node's value is 2 (note that this must not be 0 if
```

```
the two lists intersect).
 From the head of A, it reads as [0,9,1,2,4]. From the head of B, it reads as [3,2,4].
 There are 3 nodes before the intersected node in A; There are 1 node before the
intersected node in B.


 Example 3:
 A:          2 → 6 -> 4

 B:          1 -> 5

 Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
 Output: null
 Input Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads
as [1,5].
 Since the two lists do not intersect, intersectVal must be 0, while skipA and skipB can
be arbitrary values.
 Explanation: The two lists do not intersect, so return null.


 Notes:

 If the two linked lists have no intersection at all, return null.
 The linked lists must retain their original structure after the function returns.
 You may assume there are no cycles anywhere in the entire linked structure.
 Your code should preferably run in O(n) time and use only O(1) memory.
 */

public class _160 {

    public static class Solution1 {

        public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
            int lenA = findLen(headA);
            int lenB = findLen(headB);
            /**align headA and headB to the same starting point and then move together
until we find the intersection point*/
            while (lenA < lenB) {
                headB = headB.next;
                lenB--;
            }

            while (lenB < lenA) {
                headA = headA.next;
                lenA--;
            }

            while (headA != headB) {
                headA = headA.next;
                headB = headB.next;
            }

            return headA;
        }
```

```
        private int findLen(ListNode head) {
            int len = 0;
            while (head != null) {
                head = head.next;
                len++;
            }
            return len;
        }

    }

    public static class Solution2 {
        /**
         * Most optimal solution:
         *
         * O(m+n) time
         * O(1) space
         * credit: https://discuss.leetcode.com/topic/28067/java-solution-without-
knowing-the-difference-in-len*/
        public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
            if (headA == null || headB == null) {
                return null;
            }

            ListNode a = headA;
            ListNode b = headB;

            /**if a and b have different lengths, then it will stop the loop after second
iteration*/
            while (a != b) {
                /**for the first iteration, it'll just reset the pointer to the head of
another linkedlist*/
                a = a == null ? headB : a.next;
                b = b == null ? headA : b.next;
            }
            return a;
        }
    }

    public static class Solution3 {
        /**
         * O(m+n) time
         * O(Math.max(m, n)) space
         * */
        public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
            Set<ListNode> set = new HashSet<>();
            while (headA != null) {
                set.add(headA);
                headA = headA.next;
            }

            while (headB != null) {
                if (set.contains(headB)) {
```

```
                    return headB;
                }
                headB = headB.next;
            }
            return null;
        }
    }
}
```

161. `_161.java`

```
package com.fishercoder.solutions;

/**
 * 161. One Edit Distance
 *
 * Given two strings s and t, determine if they are both one edit distance apart.
 *
 * Note:
 *
 * There are 3 possiblities to satisify one edit distance apart:
 *
 *     Insert a character into s to get t
 *     Delete a character from s to get t
 *     Replace a character of s to get t
 *
 * Example 1:
 * Input: s = "ab", t = "acb"
 * Output: true
 * Explanation: We can insert 'c' into s to get t.
 *
 * Example 2:
 * Input: s = "cab", t = "ad"
 * Output: false
 * Explanation: We cannot get t from s by only one step.
 *
 * Example 3:
 * Input: s = "1203", t = "1213"
 * Output: true
 * Explanation: We can replace '0' with '1' to get t.
 */
public class _161 {
  public static class Solution1 {
    public boolean isOneEditDistance(String s, String t) {
      char[] schar = s.toCharArray();
      char[] tchar = t.toCharArray();

      if (Math.abs(s.length() - t.length()) == 1) {
        char[] longer = (s.length() > t.length()) ? schar : tchar;
        char[] shorter = (longer == schar) ? tchar : schar;

        int diffCnt = 0;
        int i = 0;
        int j = 0;
        for (; i < shorter.length && j < longer.length; ) {
          if (longer[j] != shorter[i]) {
            diffCnt++;
            j++;
          } else {
            i++;
            j++;
          }
```

```
        }
        return diffCnt == 1
            || diffCnt
            == 0;//it could be the last char of the longer is the different one, in that
case, diffCnt remains to be zero
      } else if (s.length() == t.length()) {
        int diffCnt = 0;
        for (int i = 0; i < s.length(); i++) {
          if (schar[i] != tchar[i]) {
            diffCnt++;
          }
          if (diffCnt > 1) {
            return false;
          }
        }
        return diffCnt == 1;
      }
      return false;
    }
  }
}
```

162. `_162.java`

```
package com.fishercoder.solutions;

/**
 * 162. Find Peak Element

 A peak element is an element that is greater than its neighbors.

 Given an input array nums, where nums[i] ≠ nums[i+1], find a peak element and return its
index.

 The array may contain multiple peaks, in that case return the index to any one of the
peaks is fine.

 You may imagine that nums[-1] = nums[n] = -∞.

 Example 1:
 Input: nums = [1,2,3,1]
 Output: 2
 Explanation: 3 is a peak element and your function should return the index number 2.

 Example 2:
 Input: nums = [1,2,1,3,5,6,4]
 Output: 1 or 5
 Explanation: Your function can return either index number 1 where the peak element is 2,
 or index number 5 where the peak element is 6.

 Note:
 Your solution should be in logarithmic complexity.
 */

public class _162 {

  public static class Solution1 {
    /**
     * credit: https://discuss.leetcode.com/topic/29329/java-solution-and-explanation-
using-invariants
     *
     * Basically, we need to keep this invariant:
     * nums[left] > nums[left-1], then we could return left as the result
     * or nums[right] > nums[right+1], then we could return right as the result
     *
     * Time: O(Ologn)
     */
    public int findPeakElement(int[] nums) {
      if (nums == null || nums.length == 0) {
        return 0;
      }
      int left = 0;
      int right = nums.length - 1;
      while (left + 1 < right) {
        int mid = left + (right - left) / 2;
```

```
        if (nums[mid] < nums[mid + 1]) {
          left = mid;
        } else {
          right = mid;
        }
      }
      return (left == nums.length - 1 || nums[left] > nums[left + 1]) ? left : right;
    }
  }

  public static class Solution2 {
    /**
     * My original O(n) solution.
     */
    public int findPeakElement(int[] nums) {
      if (nums == null || nums.length == 0) {
        return 0;
      }
      int n = nums.length;
      int result = 0;
      for (int i = 0; i < n; i++) {
        if (i == 0 && n > 1 && nums[i] > nums[i + 1]) {
          result = i;
          break;
        } else if (i == n - 1 && i > 0 && nums[i] > nums[i - 1]) {
          result = i;
          break;
        } else if (i > 0 && i < n - 1 && nums[i] > nums[i - 1] && nums[i] > nums[i + 1])
{
          result = i;
          break;
        }
      }
      return result;
    }
  }
}
```

163. `_163.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 163. Missing Ranges
 *
 * Given a sorted integer array where the range of elements are in the inclusive range
[lower, upper], return its missing ranges.
 * For example, given [0, 1, 3, 50, 75], lower = 0 and upper = 99, return ["2", "4->49",
"51->74", "76->99"].
 */
public class _163 {
    public static class Solution1 {
        public List<String> findMissingRanges(int[] nums, int lower, int upper) {
            List<String> result = new ArrayList<>();
            long low = (long) lower - 1;
            long up = 0;
            for (int i = 0; i <= nums.length; i++) {
                if (i == nums.length) {
                    up = (long) upper + 1;
                } else {
                    up = nums[i];
                }
                if (up == low + 2) {
                    result.add(low + 1 + "");
                } else if (up > low + 2) {
                    result.add((low + 1) + "->" + (up - 1));
                }
                low = up;
            }
            return result;
        }
    }
}
```

164. `_164.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 164. Maximum Gap
 *
 * Given an unsorted array, find the maximum difference between the successive elements
in its sorted form.
 * Return 0 if the array contains less than 2 elements.
 *
 * Example 1:
 * Input: [3,6,9,1]
 * Output: 3
 * Explanation: The sorted form of the array is [1,3,6,9], either
 *              (3,6) or (6,9) has the maximum difference 3.
 *
 * Example 2:
 * Input: [10]
 * Output: 0
 * Explanation: The array contains less than 2 elements, therefore return 0.
 *
 * Note:
 *  You may assume all elements in the array are non-negative integers and fit in the 32-
bit signed integer range.
 *  Try to solve it in linear time/space.
 */
public class _164 {
  public static class Solution1 {
    /** brute force solution */
    public int maximumGap(int[] nums) {
      if (nums.length < 2) {
        return 0;
      }

      Arrays.sort(nums);
      int max = Integer.MIN_VALUE;
      for (int i = 1; i < nums.length; ) {
        while (i < nums.length && nums[i] == nums[i - 1]) {
          i++;
        }
        if (i == nums.length) {
          i--;
          max = (nums[i] - nums[i - 1] > max) ? nums[i] - nums[i - 1] : max;
          break;
        } else {
          max = (nums[i] - nums[i - 1] > max) ? nums[i] - nums[i - 1] : max;
        }
        if (nums[i] != nums[i - 1]) {
          i++;
        }
```

```
      }
      return max;
    }
  }
}
```

165. `_165.java`

```
package com.fishercoder.solutions;
/**
 * 165. Compare Version Numbers

 Compare two version numbers version1 and version2.
 If version1 > version2 return 1, if version1 < version2 return -1, otherwise return 0.

 You may assume that the version strings are non-empty and contain only digits and the .
character.
 The . character does not represent a decimal point and is used to separate number
sequences.
 For instance, 2.5 is not "two and a half" or "half way to version three", it is the
fifth second-level revision of the second first-level revision.

 Here is an example of version numbers ordering:

 0.1 < 1.1 < 1.2 < 13.37
 */

public class _165 {
  public static class Solution1 {
    public int compareVersion(String version1, String version2) {
      String[] v1s = version1.split(
          "\\.");//escaping it is very important! Otherwise, it's not going to work as
expected!
      String[] v2s = version2.split("\\.");
      int len = (v1s.length < v2s.length) ? v2s.length : v1s.length;
      for (int i = 0; i < len; i++) {
        if (v1s.length == i) {
          while (i < len) {
            if (Integer.parseInt(v2s[i]) > 0) {
              return -1;
            }
            i++;
          }
        } else if (v2s.length == i) {
          while (i < len) {
            if (Integer.parseInt(v1s[i]) > 0) {
              return 1;
            }
            i++;
          }
        } else {
          if (Integer.parseInt(v1s[i]) > Integer.parseInt(v2s[i])) {
            return 1;
          } else if (Integer.parseInt(v2s[i]) > Integer.parseInt(v1s[i])) {
            return -1;
          }
        }
      }
      return 0;
```

```
    }
  }
}
```

166. `_166.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 166. Fraction to Recurring Decimal
 *
 * Given two integers representing the numerator and denominator of a fraction, return
the fraction in string format.

 If the fractional part is repeating, enclose the repeating part in parentheses.

 For example,

 Given numerator = 1, denominator = 2, return "0.5".
 Given numerator = 2, denominator = 1, return "2".
 Given numerator = 2, denominator = 3, return "0.(6)".

 */
public class _166 {
  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/33311/simple-and-short-solution-in-
java */
    public String fractionToDecimal(int numerator, int denominator) {
      String sign =
          (numerator >= 0 && denominator >= 0) || (numerator < 0 && denominator < 0) ? ""
: "-";
      if (numerator == 0) {
        return "0";
      }
      long num = Math.abs((long) numerator);
      long deno = Math.abs((long) denominator);
      StringBuilder stringBuilder = new StringBuilder();
      stringBuilder.append(sign);
      long integral = Math.abs(num / deno);
      stringBuilder.append(integral);
      if (numerator % denominator == 0) {
        return stringBuilder.toString();
      } else {
        stringBuilder.append(".");
      }
      long remainder = num % deno;

      Map<Long, Integer> map = new HashMap<>();
      while (!map.containsKey(remainder)) {
        map.put(remainder, stringBuilder.length());
        long n = remainder * 10 / deno;
        remainder = remainder * 10 % deno;
        if (remainder != 0 || (remainder == 0 && !map.containsKey(remainder))) {
          stringBuilder.append(n);
```

```
        }
      }
      if (remainder != 0) {
        stringBuilder.insert(map.get(remainder), "(");
        stringBuilder.append(")");
      }
      return stringBuilder.toString();
    }
  }
}
```

167. `_167.java`

```
package com.fishercoder.solutions;

/**
 * 167. Two Sum II - Input array is sorted

 Given an array of integers that is already sorted in ascending order, find two numbers such that they add up to a specific target number.

 The function twoSum should return indices of the two numbers such that they add up to the target, where index1 must be less than index2.

 Note:

 Your returned answers (both index1 and index2) are not zero-based.
 You may assume that each input would have exactly one solution and you may not use the same element twice.

 Example:

 Input: numbers = [2,7,11,15], target = 9
 Output: [1,2]
 Explanation: The sum of 2 and 7 is 9. Therefore index1 = 1, index2 = 2.

 */

public class _167 {
    public static class Solution1 {
        public int[] twoSum(int[] numbers, int target) {
            int left = 0;
            int right = numbers.length - 1;
            while (left < right) {
                long sum = numbers[left] + numbers[right];
                if (sum > target) {
                    right--;
                } else if (sum < target) {
                    left++;
                } else {
                    return new int[]{left + 1, right + 1};
                }
            }
            return new int[]{-1, -1};
        }
    }
}
```

168. `_168.java`

```
package com.fishercoder.solutions;
/**
 *
 * 168. Excel Sheet Column Title

 Given a positive integer, return its corresponding column title as appear in an Excel
sheet.

 For example:

 1 -> A
 2 -> B
 3 -> C
 ...
 26 -> Z
 27 -> AA
 28 -> AB
 ...

 Example 1:

 Input: 1
 Output: "A"

 Example 2:

 Input: 28
 Output: "AB"

 Example 3:

 Input: 701
 Output: "ZY"

 */
public class _168 {
  public static class Solution1 {
    public String convertToTitle(int n) {
      /**Get the right most digit first, move to the left, e.g. when n = 28, we get 'B'
first, then we get 'A'.*/
      StringBuilder sb = new StringBuilder();
      while (n != 0) {
        int temp = (n - 1) % 26;
        sb.append((char) (temp + 65));
        n = (n - 1) / 26;
      }
      return sb.reverse().toString();
    }
  }
}
```

169. `_169.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 169. Majority Element

 Given an array of size n, find the majority element. The majority element is the element
that appears more than ⌊ n/2 ⌋ times.

 You may assume that the array is non-empty and the majority element always exist in the
array.

 Example 1:

 Input: [3,2,3]
 Output: 3

 Example 2:

 Input: [2,2,1,1,1,2,2]
 Output: 2
 */
public class _169 {
  public static class Solution1 {
    /**Moore Voting Algorithm*/
    public int majorityElement(int[] nums) {
      int count = 1;
      int majority = nums[0];
      for (int i = 1; i < nums.length; i++) {
        if (count == 0) {
          count++;
          majority = nums[i];
        } else if (nums[i] == majority) {
          count++;
        } else {
          count--;
        }
      }
      return majority;
    }
  }

  public static class Solution2 {
    public int majorityElement(int[] nums) {
      Map<Integer, Integer> map = new HashMap();
      for (int i : nums) {
        map.put(i, map.getOrDefault(i, 0) + 1);
        if (map.get(i) > nums.length / 2) {
```

```
          return i;
        }
      }
      return -1;
    }
  }

  public static class Solution3 {
    //This is O(nlogn) time.
    public int majorityElement(int[] nums) {
      Arrays.sort(nums);
      return nums[nums.length / 2];
    }
  }

  public static class Solution4 {
    //bit manipulation
    public int majorityElement(int[] nums) {
      int[] bit = new int[32];//because an integer is 32 bits, so we use an array of 32
long
      for (int num : nums) {
        for (int i = 0; i < 32; i++) {
          if ((num >> (31 - i) & 1) == 1) {
            bit[i]++;//this is to compute each number's ones frequency
          }
        }
      }
      int res = 0;
      //this below for loop is to construct the majority element: since every bit of this
element would have appeared more than n/2 times
      for (int i = 0; i < 32; i++) {
        bit[i] = bit[i] > nums.length / 2 ? 1
            : 0;//we get rid of those that bits that are not part of the majority number
        res += bit[i] * (1 << (31 - i));
      }
      return res;
    }
  }
}
```

170. `_170.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 170. Two Sum III - Data structure design
 *
 * Design and implement a TwoSum class. It should support the following operations: add
and find.
 *
 * add - Add the number to an internal data structure.
 * find - Find if there exists any pair of numbers which sum is equal to the value.
 *
 * Example 1:
 * add(1); add(3); add(5);
 * find(4) -> true
 * find(7) -> false
 *
 * Example 2:
 * add(3); add(1); add(2);
 * find(3) -> true
 * find(6) -> false
 */

public class _170 {
  public static class Solution1 {
    private Map<Integer, Integer> map = new HashMap();
    private List<Integer> list = new ArrayList();

    // Add the number to an internal data structure.
    public void add(int number) {
      list.add(number);
      map.put(number, map.getOrDefault(number, 0) + 1);
    }

    // Find if there exists any pair of numbers which sum is equal to the value.
    public boolean find(int value) {
      for (int i = 0; i < list.size(); i++) {
        int val1 = list.get(i);
        int val2 = value - val1;
        if (map.containsKey(val2)) {
          if (val1 == val2) {
            if (map.get(val2) > 1) {
              return true;
            }
          } else {
            return true;
          }
```

```
        }
      }
      return false;
    }
  }
}
```

171. `_171.java`

```
package com.fishercoder.solutions;

/**
 * 170. Two Sum III - Data structure design
 *
 * Given a column title as appear in an Excel sheet, return its corresponding column
number.

 For example:

 A -> 1
 B -> 2
 C -> 3
 ...
 Z -> 26
 AA -> 27
 AB -> 28

 */
public class _171 {

  public static class Solution1 {
    public int titleToNumber(String s) {
      char[] c = s.toCharArray();
      int result = 0;
      for (int i = s.length() - 1; i >= 0; i--) {
        result +=
            (c[i] - 64) * ((int) Math.pow(26, s.length() - i - 1));//The ASCII value of A
is 65
      }
      return result;
    }
  }
}
```

172. `_172.java`

```
package com.fishercoder.solutions;
/**
 * 172. Factorial Trailing Zeroes

 Given an integer n, return the number of trailing zeroes in n!.

 Example 1:

 Input: 3
 Output: 0
 Explanation: 3! = 6, no trailing zero.

 Example 2:

 Input: 5
 Output: 1
 Explanation: 5! = 120, one trailing zero.

 Note: Your solution should be in logarithmic time complexity.

 */
public class _172 {

  public static class Solution1 {
    public int trailingZeroes(int n) {
      int result = 0;
      while (n > 4) {
        n /= 5;
        result += n;
      }
      return result;
    }
  }
}
```

173. `_173.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

/**
 * 173. Binary Search Tree Iterator
 *
 * Implement an iterator over a binary search tree (BST). Your iterator will be
initialized with the root node of a BST.
 *
 * Calling next() will return the next smallest number in the BST.
 *
 * Note: next() and hasNext() should run in average O(1) time and uses O(h) memory, where
h is the height of the tree.
 */
public class _173 {

  public static class Solution1 {

    public static class BSTIterator {
      private Queue<Integer> queue;

      public BSTIterator(TreeNode root) {
        queue = new LinkedList<>();
        if (root != null) {
          dfs(root, queue);
        }
      }

      private void dfs(TreeNode root, Queue<Integer> q) {
        if (root.left != null) {
          dfs(root.left, q);
        }
        q.offer(root.val);
        if (root.right != null) {
          dfs(root.right, q);
        }
      }

      public boolean hasNext() {
        return !queue.isEmpty();
      }

      public int next() {
        return queue.poll();
      }
    }
```

```
  }

  public static class Solution2 {
    public static class BSTIterator {
      /**
       * This is a super cool/clever idea: use a stack to store all the current left
nodes of the BST, when pop(), we
       * push all its right nodes into the stack if there are any.
       * This way, we use only O(h) memory for this iterator, this is a huge saving when
the tree is huge
       * since h could be much smaller than n. Cheers!
       */

      private Stack<TreeNode> stack;

      public BSTIterator(TreeNode root) {
        stack = new Stack();
        pushToStack(root, stack);
      }

      private void pushToStack(TreeNode root, Stack<TreeNode> stack) {
        while (root != null) {
          stack.push(root);
          root = root.left;
        }
      }

      public boolean hasNext() {
        return !stack.isEmpty();
      }

      public int next() {
        TreeNode curr = stack.pop();
        pushToStack(curr.right, stack);
        return curr.val;
      }
    }
  }
}
```

174. `_174.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 174. Dungeon Game

 The demons had captured the princess (P) and imprisoned her in the bottom-right corner
of a dungeon. The dungeon consists of M x N rooms laid out in a 2D grid. Our valiant
knight (K) was initially positioned in the top-left room and must fight his way through
the dungeon to rescue the princess.

 The knight has an initial health point represented by a positive integer. If at any
point his health point drops to 0 or below, he dies immediately.

 Some of the rooms are guarded by demons, so the knight loses health (negative integers)
upon entering these rooms; other rooms are either empty (0's) or contain magic orbs that
increase the knight's health (positive integers).

 In order to reach the princess as quickly as possible, the knight decides to move only
rightward or downward in each step.

 Write a function to determine the knight's minimum initial health so that he is able to
rescue the princess.

 For example, given the dungeon below, the initial health of the knight must be at least
7 if he follows the optimal path RIGHT-> RIGHT -> DOWN -> DOWN.
 -2 (K)         -3      3
 -5     -10     1
 10     30      -5 (P)

 Note:

 The knight's health has no upper bound.
 Any room can contain threats or power-ups, even the first room the knight enters and the
bottom-right room where the princess is imprisoned.

 */
public class _174 {

  public static class Solution1 {
    /** This problem should fill the dp matrix from bottom right. */
    public int calculateMinimumHP(int[][] dungeon) {
      if (dungeon == null || dungeon.length == 0) {
        return 0;
      }

      int height = dungeon.length;
      int width = dungeon[0].length;
      int[][] dp = new int[height][width];
      dp[height - 1][width - 1] =
```

```
          (dungeon[height - 1][width - 1] > 0) ? 1 : 1 - dungeon[height - 1][width - 1];

      //fill the last column
      for (int i = height - 2; i >= 0; i--) {
        int temp = dp[i + 1][width - 1] - dungeon[i][width - 1];
        dp[i][width - 1] = Math.max(1, temp);
      }

      //fill the last row
      for (int j = width - 2; j >= 0; j--) {
        int temp = dp[height - 1][j + 1] - dungeon[height - 1][j];
        dp[height - 1][j] = Math.max(temp, 1);
      }

      for (int i = height - 2; i >= 0; i--) {
        for (int j = width - 2; j >= 0; j--) {
          int down = Math.max(1, dp[i + 1][j] - dungeon[i][j]);
          int right = Math.max(1, dp[i][j + 1] - dungeon[i][j]);
          dp[i][j] = Math.min(down, right);
        }
      }

      CommonUtils.printMatrix(dp);
      return dp[0][0];
    }
  }
}
```

175. `_179.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 179. Largest Number

 Given a list of non negative integers, arrange them such that they form the largest
number.

 Example 1:

 Input: [10,2]
 Output: "210"

 Example 2:

 Input: [3,30,34,5,9]
 Output: "9534330"

 Note: The result may be very large, so you need to return a string instead of an
integer.
 */
public class _179 {

  public static class Solution1 {
    public String largestNumber(int[] num) {
      if (num.length == 0) {
        return "";
      }
      if (num.length == 1) {
        return Integer.toString(num[0]);
      }
      String[] str = new String[num.length];
      for (int i = 0; i < num.length; i++) {
        str[i] = Integer.toString(num[i]);
      }
      Arrays.sort(str, new StringComparator());
      StringBuilder sb = new StringBuilder("");
      for (int i = num.length - 1; i >= 0; i--) {
        sb.append(str[i]);
      }
      if (sb.charAt(0) == '0') {
        return "0";
      }
      return sb.toString();
    }

    class StringComparator implements Comparator<String> {
      public int compare(String s1, String s2) {
```

```
        if (s1.length() == 0 && s2.length() == 0) {
          return 0;
        }
        if (s2.length() == 0) {
          return 1;
        }
        if (s1.length() == 0) {
          return -1;
        }
        for (int i = 0; i < s1.length() && i < s2.length(); i++) {
          if (s1.charAt(i) > s2.charAt(i)) {
            return 1;
          } else if (s1.charAt(i) < s2.charAt(i)) {
            return -1;
          }
        }
        if (s1.length() == s2.length()) {
          return 0;
        }
        if (s1.length() > s2.length()) {
          if (s1.charAt(0) < s1.charAt(s2.length())) {
            return 1;
          } else if (s1.charAt(0) > s1.charAt(s2.length())) {
            return -1;
          } else {
            return compare(s1.substring(s2.length()), s2);
          }
        } else {
          if (s2.charAt(0) < s2.charAt(s1.length())) {
            return -1;
          } else if (s2.charAt(0) > s2.charAt(s1.length())) {
            return 1;
          } else {
            return compare(s1, s2.substring(s1.length()));
          }
        }
      }
    }
  }
}
```

176. `_186.java`

```
package com.fishercoder.solutions;

/**
 * 186. Reverse Words in a String II

 Given an input string, reverse the string word by word. A word is defined as a sequence
of non-space characters.

 The input string does not contain leading or trailing spaces and the words are always
separated by a single space.

 For example,
 Given s = "the sky is blue",
 return "blue is sky the".

 Could you do it in-place without allocating extra space?
 */
public class _186 {
  public static class Solution1 {
    public void reverseWords(char[] s) {
      // Three steps to reverse
      // 1, reverse the whole sentence
      reverse(s, 0, s.length - 1);
      // 2, reverse each word
      int start = 0;
      for (int i = 0; i < s.length; i++) {
        if (s[i] == ' ') {
          reverse(s, start, i - 1);
          start = i + 1;
        }
      }
      // 3, reverse the last word, if there is only one word this will solve the corner
case
      reverse(s, start, s.length - 1);
    }

    private void reverse(char[] s, int start, int end) {
      while (start < end) {
        char temp = s[start];
        s[start++] = s[end];
        s[end--] = temp;
      }
    }
  }

}
```

177. `_187.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 187. Repeated DNA Sequences
 *
 * All DNA is composed of a series of nucleotides abbreviated as A, C, G, and T,
 * for example: "ACGAATTCCG". When studying DNA, it is sometimes useful to identify
repeated sequences within the DNA.
 * Write a function to find all the 10-letter-long sequences (substrings) that occur more
than once in a DNA molecule.

 For example,

 Given s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT",

 Return:
 ["AAAAACCCCC", "CCCCCAAAAA"].
 */
public class _187 {
    public static class Solution1 {
        public List<String> findRepeatedDnaSequences(String s) {
            Map<String, Integer> map = new HashMap();
            for (int i = 0; i < s.length() - 9; i++) {
                String sequence = s.substring(i, i + 10);
                map.put(sequence, map.getOrDefault(sequence, 0) + 1);
            }
            List<String> repeatedSequences = new ArrayList<>();
            for (Map.Entry<String, Integer> entry : map.entrySet()) {
                if (entry.getValue() > 1) {
                    repeatedSequences.add(entry.getKey());
                }
            }
            return repeatedSequences;
        }
    }
}
```

178. `_188.java`

```
package com.fishercoder.solutions;

/**

 188. Best Time to Buy and Sell Stock IV

 Say you have an array for which the ith element is the price of a given stock on day i.

 Design an algorithm to find the maximum profit. You may complete at most k transactions.

 Note:
 You may not engage in multiple transactions at the same time (ie, you must sell the
stock before you buy again).

 Example 1:
 Input: [2,4,1], k = 2
 Output: 2
 Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = 4-2 = 2.

 Example 2:
 Input: [3,2,6,5,0,3], k = 2
 Output: 7
 Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4.
 Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

 */
public class _188 {
  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/8984/a-concise-dp-solution-in-java */
    public int maxProfit(int k, int[] prices) {
      int len = prices.length;
      if (k >= len / 2) {
        return quickSolve(prices);
      }

      int[][] t = new int[k + 1][len];
      for (int i = 1; i <= k; i++) {
        int tmpMax = -prices[0];
        for (int j = 1; j < len; j++) {
          t[i][j] = Math.max(t[i][j - 1], prices[j] + tmpMax);
          tmpMax = Math.max(tmpMax, t[i - 1][j - 1] - prices[j]);
        }
      }
      return t[k][len - 1];
    }

    private int quickSolve(int[] prices) {
      int len = prices.length;
      int profit = 0;
      for (int i = 1; i < len; i++) {
        // as long as there is a price gap, we gain a profit.
```

```
        if (prices[i] > prices[i - 1]) {
          profit += prices[i] - prices[i - 1];
        }
      }
      return profit;
    }
  }
}
```

179. `_189.java`

```
package com.fishercoder.solutions;

/**
 * 189. Rotate Array

 Given an array, rotate the array to the right by k steps, where k is non-negative.

 Example 1:
 Input: [1,2,3,4,5,6,7] and k = 3
 Output: [5,6,7,1,2,3,4]
 Explanation:
 rotate 1 steps to the right: [7,1,2,3,4,5,6]
 rotate 2 steps to the right: [6,7,1,2,3,4,5]
 rotate 3 steps to the right: [5,6,7,1,2,3,4]

 Example 2:
 Input: [-1,-100,3,99] and k = 2
 Output: [3,99,-1,-100]
 Explanation:
 rotate 1 steps to the right: [99,-1,-100,3]
 rotate 2 steps to the right: [3,99,-1,-100]

 Note:
 Try to come up as many solutions as you can, there are at least 3 different ways to
solve this problem.
 Could you do it in-place with O(1) extra space?
 * */

public class _189 {

  public static class Solution1 {
    /**
     * O(n*k) time
     * O(1) space
     */
    public void rotate(int[] nums, int k) {
      int tmp;
      for (int i = 0; i < k; i++) {
        tmp = nums[nums.length - 1];
        for (int j = nums.length - 1; j > 0; j--) {
          nums[j] = nums[j - 1];
        }
        nums[0] = tmp;
      }
    }
  }

  public static class Solution2 {
    /**
     * using an extra array of the same size to copy it
     * O(n) time
```

```
     * O(n) space
     */
    public void rotate(int[] nums, int k) {
      int len = nums.length;
      int[] tmp = new int[len];
      for (int i = 0; i < len; i++) {
        tmp[(i + k) % len] = nums[i];
      }
      for (int i = 0; i < len; i++) {
        nums[i] = tmp[i];
      }
    }
  }

  public static class Solution3 {
    /**
     * reverse three times
     * O(n) time
     * O(1) space
     */
    public void rotate(int[] nums, int k) {
      int len = nums.length;
      k %= len;
      reverse(nums, 0, len - 1);
      reverse(nums, 0, k - 1);
      reverse(nums, k, len - 1);
    }

    private void reverse(int[] nums, int start, int end) {
      while (start < end) {
        int tmp = nums[start];
        nums[start] = nums[end];
        nums[end] = tmp;
        start++;
        end--;
      }
    }
  }
}
```

180. `_190.java`

```
package com.fishercoder.solutions;

/**
 * 190. Reverse Bits
 * Reverse bits of a given 32 bits unsigned integer.

 Example:
 Input: 43261596
 Output: 964176192

 Explanation: 43261596 represented in binary as 00000010100101000001111010011100,
 return 964176192 represented in binary as 00111001011110000010100101000000.

 Follow up:
 If this function is called many times, how would you optimize it?
 */

public class _190 {
    /**delimiting the binary string into 4 bits array will make it easier to
see/visualize:
     * original binary format:
     * 0000,0010,1001,0100,0001,1110,1001,1100,
     * after reversing, becomes:
     * 0011,1001,0111,1000,0010,1001,0100,0000
     * The most right side digit shifted to the most left side, the 2nd right side digit
shifted to the 2nd left side, so forth..*/

    /**This post: http://stackoverflow.com/questions/2811319/difference-between-and
     * gives a good explanation between logical right shift: ">>>" and arithmetic right
shift: ">>".
     * Basically, ">>" preserves the most left bit and treats it as the sign for this
number,
     * e.g. -2 represented in 8 bits is 11111110, thus -2 >> 1 will become 11111111, i.e.
-1
     * notice its sign bit (the most left one bit) is preserved
     * However, logical right shift ">>>" doesn't care about the first bit on the most
left,
     * it simply shifts every bit to the right.
     * e.g. -2 >>> 1 would become 11111111111111111111111111111111, i.e. 2147483647*/

 public static class Solution1 {
      // you need treat n as an unsigned value
      public int reverseBits(int n) {
        int res = 0;
        for (int i = 0; i < 32; i++) {
          res += n & 1;//get the most right bit each time
          n >>>= 1;//do UN-signed right shift by 1 each time
          if (i < 31) {
            res <<=
                1;//shift this number to the left by 1 each time, so that eventually,
this number is reversed
```

```
          }
        }
        return res;
      }
    }
}
```

181. `_191.java`

```
package com.fishercoder.solutions;

/**
 * 191. Number of 1 Bits
 *
 * Write a function that takes an unsigned integer and returns the number of ’1' bits it has (also known as the Hamming weight).
 * For example, the 32-bit integer ’11' has binary representation 00000000000000000000000000001011, so the function should return 3.*/

public class _191 {

    public static class Solution1 {
        /**Doing bitwise AND operation between n and n-1 will always flip the least significant 1 bit in n to zero
         example run for the above editorial solution: input 5, n will be 5&4 and becomes 4,
         then in the next run, n will become 4&3 which is 0, thus exit the while loop.*/
        public int hammingWeight(int n) {
            int bits = 0;
            while (n != 0) {
                bits++;
                n &= (n - 1);
            }
            return bits;
        }
    }

    public static class Solution2 {
        public int hammingWeight(int n) {
            int bits = 0;
            int mask = 1;
            for (int i = 0; i < 32; i++) {
                if ((n & mask) != 0) {
                    bits++;
                }
                mask <<= 1;
            }
            return bits;
        }
    }

    public static class Solution3 {
        public int hammingWeight(int n) {
            int bits = 0;
            for (int i = 0; i < 32; i++) {
                if ((n & 1) == 1) {
                    bits++;
                }
                if (n == 0) {
                    return bits;
```

```
                }
                /**must use unsigned right shift operator since the problem says this is
an unsigned value*/
                n >>>= 1;
            }
            return bits;
        }
    }

    public static class Solution4 {
        public int hammingWeight(int n) {
            int bits = 0;
            for (int i = 0; i < 32; i++) {
                bits += n & 1;
                n >>>= 1;
            }
            return bits;
        }
    }
}
```

182. `_198.java`

```
package com.fishercoder.solutions;

/**198. House Robber

 You are a professional robber planning to rob houses along a street.
 Each house has a certain amount of money stashed,
 the only constraint stopping you from robbing each of them is that adjacent houses have
security
 system connected and it will automatically contact the police if two adjacent houses
were broken into on the same night.
 Given a list of non-negative integers representing the amount of money of each house,
 determine the maximum amount of money you can rob tonight without alerting the police.

 Example 1:
 Input: [1,2,3,1]
 Output: 4
 Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
 Total amount you can rob = 1 + 3 = 4.

 Example 2:
 Input: [2,7,9,3,1]
 Output: 12
 Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money =
1).
 Total amount you can rob = 2 + 9 + 1 = 12.
 */
public class _198 {

    public static class Solution1 {
        public int rob(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            if (nums.length == 1) {
                return nums[0];
            }
            if (nums.length == 2) {
                return Math.max(nums[0], nums[1]);
            }
            int[] dp = new int[nums.length];
            dp[0] = nums[0];
            dp[1] = Math.max(nums[0], nums[1]);
            for (int i = 2; i < nums.length; i++) {
                dp[i] = Math.max(dp[i - 2] + nums[i], dp[i - 1]);
            }
            return dp[nums.length - 1];
        }
    }
}
```

183. `_199.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**199. Binary Tree Right Side View

Given a binary tree, imagine yourself standing on the right side of it, return the values
of the nodes you can see ordered from top to bottom.

For example:
Given the following binary tree,

   1            <---
 /   \
2     3         <---
 \     \
  5     4       <---

You should return [1, 3, 4]. */

public class _199 {

        public static class Solution1 {
                /**credit: https://leetcode.com/problems/binary-tree-right-side-
view/discuss/56012/My-simple-accepted-solution(JAVA)*/
                public List<Integer> rightSideView(TreeNode root) {
                        List<Integer> result = new ArrayList<>();
                        rightView(root, result, 0);
                        return result;
                }

                void rightView(TreeNode curr, List<Integer> result, int currDepth) {
                        if (curr == null) {
                                return;
                        }
                        if (currDepth == result.size()) {
                                result.add(curr.val);
                        }
                        rightView(curr.right, result, currDepth + 1);
                        rightView(curr.left, result, currDepth + 1);
                }
        }

        public static class Solution2 {
                /**BFS the tree*/
                public List<Integer> rightSideView(TreeNode root) {
```

```
                        List<Integer> result = new ArrayList<>();
                        if (root == null) {
                                return result;
                        }
                        Queue<TreeNode> q = new LinkedList<>();
                        q.offer(root);
                        while (!q.isEmpty()) {
                                int size = q.size();
                                for (int i = 0; i < size; i++) {
                                        TreeNode curr = q.poll();
                                        if (i == size - 1) {
                                                result.add(curr.val);
                                        }
                                        if (curr.left != null) {
                                                q.offer(curr.left);
                                        }
                                        if (curr.right != null) {
                                                q.offer(curr.right);
                                        }
                                }
                        }
                        return result;
                }
        }

}
```

184. `_200.java`

```
package com.fishercoder.solutions;

/**
 * 200. Number of Islands
 *
 * Given a 2d grid map of '1's (land) and '0's (water),
 * count the number of islands.
 * An island is surrounded by water and is formed by connecting adjacent lands
horizontally or vertically.
 * You may assume all four edges of the grid are all surrounded by water.

 Example 1:
 11110
 11010
 11000
 00000
 Answer: 1

 Example 2:
 11000
 11000
 00100
 00011
 Answer: 3

 */
public class _200 {

    public static class Solution1 {

        public int numIslands(char[][] grid) {
            if (grid == null || grid.length == 0) {
                return 0;
            }
            int count = 0;
            int m = grid.length;
            int n = grid[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == '1') {
                        count++;
                        dfs(grid, i, j, m, n);
                    }
                }
            }
            return count;
        }

        void dfs(char[][] grid, int i, int j, int m, int n) {
            if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == '0') {
                return;
```

```
            }
            grid[i][j] = '0';
            dfs(grid, i + 1, j, m, n);
            dfs(grid, i, j + 1, m, n);
            dfs(grid, i - 1, j, m, n);
            dfs(grid, i, j - 1, m, n);
        }
    }

    public static class Solution2 {

        class UnionFind {
            int count;
            int m;
            int n;
            int[] ids;

            public UnionFind(char[][] grid) {
                m = grid.length;
                n = grid[0].length;
                ids = new int[m * n];
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        if (grid[i][j] == '1') {
                            count++;
                            ids[i * n + j] = i * n + j;
                        }
                    }
                }
            }

            public void union(int i, int j) {
                int x = find(ids, i);
                int y = find(ids, j);
                if (x != y) {
                    /**note: this is when x != y, only in this case, we should union
these two nodes, which makes sense naturally.*/
                    count--;
                    ids[x] = y;//ids[y] = x; //also works
                }
            }

            public int find(int[] ids, int i) {
                if (ids[i] == i) {
                    return i;
                }
                return find(ids, ids[i]);
            }
        }

        public int numIslands(char[][] grid) {
            if (grid == null || grid.length == 0 || grid[0].length == 0) {
                return 0;
            }
```

```
            int[] dirs = new int[]{0, 1, 0, -1, 0};
            UnionFind uf = new UnionFind(grid);
            int m = grid.length;
            int n = grid[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == '1') {
                        for (int k = 0; k < 4; k++) {
                            int x = i + dirs[k];
                            int y = j + dirs[k + 1];
                            if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] == '1')
{
                                int id1 = i * n + j;
                                int id2 = x * n + y;
                                uf.union(id1, id2);
                            }
                        }
                    }
                }
            }
            return uf.count;
        }
    }
}
```

185. `_201.java`

```
package com.fishercoder.solutions;

/**
 * 201. Bitwise AND of Numbers Range
 *
 * Given a range [m, n] where 0 <= m <= n <= 2147483647, return the bitwise AND of all
numbers in this range, inclusive.
 *
 * For example, given the range [5, 7], you should return 4.
 */
public class _201 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/28538/java-python-easy-solution-
with-explanation
         * Bitwise AND operation within range actually turns out to be doing some
operations with just these two boundary numbers: m and n
         * e.g. 5 and 7, in binary, they are 101 and 111, the result for [5,7] is 5&6&7
which is 101&110&111
         * this we can understand it to be shifting the digits of m and n from left to
right until they become the same, then we pad that number with zeroes on the right side
         */
        public int rangeBitwiseAnd(int m, int n) {
            int i = 0;
            while (m != n) {
                m >>= 1;
                n >>= 1;
                i++;
            }
            return (n << i);
        }
    }
}
```

186. `_202.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 202. Happy Number
 *
 * Write an algorithm to determine if a number is "happy".
 * A happy number is a number defined by the following process:
 * Starting with any positive integer,
 * replace the number by the sum of the squares of its digits,
 * and repeat the process until the number equals 1 (where it will stay),
 * or it loops endlessly in a cycle which does not include 1.
 * Those numbers for which this process ends in 1 are happy numbers.
 *
 * Example: 19 is a happy number
 *
 * 12 + 92 = 82
 * 82 + 22 = 68
 * 62 + 82 = 100
 * 12 + 02 + 02 = 1
 */
public class _202 {
    public static class Solution1 {
        public boolean isHappy(int n) {
            if (n == 1) {
                return true;
            }
            Set<Integer> set = new HashSet();
            while (n != 1) {
                String str = String.valueOf(n);
                n = 0;
                for (int i = 0; i < str.length(); i++) {
                    int temp = Character.getNumericValue(str.charAt(i));
                    n += temp * temp;
                }
                if (n == 1) {
                    return true;
                }
                if (!set.add(n)) {
                    return false;
                }
            }
            return false;
        }
    }
}
```

187. `_203.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 203. Remove Linked List Elements
 *
 * Remove all elements from a linked list of integers that have value val.
 *
 * Example
 * Given: 1 --> 2 --> 6 --> 3 --> 4 --> 5 --> 6, val = 6
 * Return: 1 --> 2 --> 3 --> 4 --> 5
 */
public class _203 {
    public static class Solution1 {
        /**
         * This is a very good question to test your understanding of
pointers/memory/addresses, although it's marked as EASY.
         * All the three nodes: dummy, curr and prev are indispensable.
         *
         * 1. Eventually, we should return dummy.next as the final result.
         * 2. we assign head to curr, dummy to prev
         * 3. and then we use prev and curr to traverse through the list and do the work
         * 4. each time, we only move one node forward, so we don't need another while
loop inside the while loop
         * 5. KEY: if(curr.val == val), then curr is the node we want to remove, so,
we'll assign curr.next to prev.next, thus, prev won't have that node
         * else, we'll keep moving prev forward, so, just do prev = prev.next
         * but, for both cases, we'll also move curr forward, so we put curr = curr.next
in the outside.
         */
        public ListNode removeElements(ListNode head, int val) {
            ListNode dummy = new ListNode(-1);
            dummy.next = head;
            ListNode curr = head;
            ListNode prev = dummy;
            while (curr != null) {
                if (curr.val == val) {
                    prev.next = curr.next;
                } else {
                    prev = prev.next;
                }
                curr = curr.next;
            }
            return dummy.next;
        }
    }
}
```

188. `_204.java`

```
package com.fishercoder.solutions;

/**
 * 204. Count Primes
 *
 * Description:

 Count the number of prime numbers less than a non-negative number, n.

 Hint:

 Let's start with a isPrime function. To determine if a number is prime, we need to check if it is not divisible by any number less than n.
 The runtime complexity of isPrime function would be O(n) and hence counting the total prime numbers up to n would be O(n2). Could we do better?

 As we know the number must not be divisible by any number > n / 2,
 we can immediately cut the total iterations half by dividing only up to n / 2. Could we still do better?

 Let's write down all of 12's factors:

 2 × 6 = 12
 3 × 4 = 12
 4 × 3 = 12
 6 × 2 = 12
 As you can see, calculations of 4 × 3 and 6 × 2 are not necessary. Therefore, we only need to consider factors up to √n because,
 if n is divisible by some number p, then n = p × q and since p ≤ q, we could derive that p ≤ √n.

 Our total runtime has now improved to O(n1.5), which is slightly better. Is there a faster approach?

 public int countPrimes(int n) {
     int count = 0;
     for (int i = 1; i < n; i++) {
        if (isPrime(i)) count++;
     }
 return count;
 }

 private boolean isPrime(int num) {
     if (num <= 1) return false;
     // Loop's ending condition is i * i <= num instead of i <= sqrt(num)
     // to avoid repeatedly calling an expensive function sqrt().
     for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return false;
     }
     return true;
 }
```

The Sieve of Eratosthenes is one of the most efficient ways to find all prime numbers up to n.
But don't let that name scare you, I promise that the concept is surprisingly simple.

Sieve of Eratosthenes: algorithm steps for primes below 121. "Sieve of Eratosthenes Animation" by SKopp is licensed under CC BY 2.0.
We start off with a table of n numbers. Let's look at the first number, 2. We know all multiples of 2 must not be primes, so we mark them off as non-primes. Then we look at the next number, 3. Similarly, all multiples of 3 such as 3 × 2 = 6, 3 × 3 = 9, ... must not be primes, so we mark them off as well. Now we look at the next number, 4, which was already marked off.
What does this tell you? Should you mark off all multiples of 4 as well?
4 is not a prime because it is divisible by 2, which means all multiples of 4 must also be divisible by 2 and were already marked off.
So we can skip 4 immediately and go to the next number, 5.
Now, all multiples of 5 such as 5 × 2 = 10, 5 × 3 = 15, 5 × 4 = 20, 5 × 5 = 25, ... can be marked off.
There is a slight optimization here, we do not need to start from 5 × 2 = 10. Where should we start marking off?

In fact, we can mark off multiples of 5 starting at 5 × 5 = 25, because 5 × 2 = 10 was already marked off by multiple of 2,
similarly 5 × 3 = 15 was already marked off by multiple of 3.
Therefore, if the current number is p, we can always mark off multiples of p starting at p2, then in increments of p: p2 + p, p2 + 2p, ...
Now what should be the terminating loop condition?

It is easy to say that the terminating loop condition is p < n, which is certainly correct but not efficient. Do you still remember Hint #3?
Yes, the terminating loop condition can be p < √n, as all non-primes ≥ √n must have already been marked off. When the loop terminates,
all the numbers in the table that are non-marked are prime.

The Sieve of Eratosthenes uses an extra O(n) memory and its runtime complexity is O(n log log n).
For the more mathematically inclined readers, you can read more about its algorithm complexity on Wikipedia.

```
public int countPrimes(int n) {
    boolean[] isPrime = new boolean[n];
    for (int i = 2; i < n; i++) {
       isPrime[i] = true;
   }

    // Loop's ending condition is i * i < n instead of i < sqrt(n)
    // to avoid repeatedly calling an expensive function sqrt().

    for (int i = 2; i * i < n; i++) {
        if (!isPrime[i]) continue;
        for (int j = i * i; j < n; j += i) {
           isPrime[j] = false;
        }
   }
```

```
     int count = 0;
     for (int i = 2; i < n; i++) {
        if (isPrime[i]) count++;
     }

    return count;
 }
 */
public class _204 {

    public static class Solution1 {
        public int countPrimes(int n) {
            boolean[] notPrime = new boolean[n];
            int count = 0;
            for (int i = 2; i < n; i++) {
                if (!notPrime[i]) {
                    count++;
                    for (int j = 2; i * j < n; j++) {
                        notPrime[i * j] = true;
                    }
                }
            }
            return count;
        }
    }
}
```

189. `_205.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
/**
 * 205. Isomorphic Strings
 *
 * Given two strings s and t, determine if they are isomorphic.

 Two strings are isomorphic if the characters in s can be replaced to get t.

 All occurrences of a character must be replaced with another character while preserving
the order of characters. No two characters may map to the same character but a character
may map to itself.

 For example,
 Given "egg", "add", return true.

 Given "foo", "bar", return false.

 Given "paper", "title", return true.

 Note:
 You may assume both s and t have the same length.*/
public class _205 {
    /**space should be O(1) since it only has alphabetic letters which are capped at
52.*/

    public static class Solution1 {
        public boolean isIsomorphic(String s, String t) {
            if (s == null || s.length() == 0) {
                return (t == null || t.length() == 0);
            }
            if (t == null || t.length() == 0) {
                return (s == null || s.length() == 0);
            }
            char[] schar = s.toCharArray();
            char[] tchar = t.toCharArray();
            Map<Character, Character> map = new HashMap();
            if (s.length() != t.length()) {
                return false;
            }
            for (int i = 0; i < s.length(); i++) {
                if (map.containsKey(schar[i])) {
                    if (map.get(schar[i]) != tchar[i]) {
                        return false;
                    }
                } else {
                    if (map.containsValue(tchar[i])) {
                        return false;//this line is necessary for this case: ("ab", "aa")
                    }
```

```
                    map.put(schar[i], tchar[i]);
                }
            }
            return true;
        }
    }
}
```

190. `_206.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 206. Reverse Linked List
 *
 * Reverse a singly linked list.
 *
 * Example:
 * Input: 1->2->3->4->5->NULL
 * Output: 5->4->3->2->1->NULL
 *
 * Follow up:
 * A linked list can be reversed either iteratively or recursively. Could you implement
both?
 * */
public class _206 {

    public static class Solution1 {
        public ListNode reverseList(ListNode head) {
            ListNode prev = null;
            ListNode curr = head;
            while (curr != null) {
                ListNode next = curr.next;
                curr.next = prev;
                prev = curr;
                curr = next;
            }
            return prev;
        }
    }

    public static class Solution2 {
        public ListNode reverseList(ListNode head) {
            return reverse(head, null);
        }

        ListNode reverse(ListNode head, ListNode newHead) {
            if (head == null) {
                return newHead;
            }
            ListNode next = head.next;
            head.next = newHead;
            newHead = head;
            head = next;
            return reverse(head, newHead);
        }
    }

}
```

191. `_207.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;

/**
 * 207. Course Schedule
 *
 * There are a total of n courses you have to take, labeled from 0 to n - 1.
 Some courses may have prerequisites, for example to take course 0 you have to first take
course 1,
 which is expressed as a pair: [0,1]
 Given the total number of courses and a list of prerequisite pairs, is it possible for
you to finish all courses?

 For example:
 2, [[1,0]]
 There are a total of 2 courses to take.
 To take course 1 you should have finished course 0. So it is possible.

 2, [[1,0],[0,1]]
 There are a total of 2 courses to take.
 To take course 1 you should have finished course 0,
 and to take course 0 you should also have finished course 1. So it is impossible.

 Note:
 The input prerequisites is a graph represented by a list of edges, not adjacency
matrices. Read more about how a graph is represented.
 You may assume that there are no duplicate edges in the input prerequisites.
 click to show more hints.

 Hints:
 This problem is equivalent to finding if a cycle exists in a directed graph. If a cycle
exists, no topological ordering exists and therefore it will be impossible to take all
courses.
 Topological Sort via DFS - A great video tutorial (21 minutes) on Coursera explaining
the basic concepts of Topological Sort.
 Topological sort could also be done via BFS.
 */
public class _207 {

    public static class Solution1 {
        public boolean canFinish(int numCourses, int[][] prerequisites) {
            int[] indegree = new int[numCourses];
            for (int[] prereq : prerequisites) {
                indegree[prereq[0]]++;
            }
            Set<Integer> zeroDegree = new HashSet();
            for (int i = 0; i < numCourses; i++) {
                if (indegree[i] == 0) {
```

```
                    zeroDegree.add(i);
                }
            }
            if (zeroDegree.isEmpty()) {
                return false;
            }

            while (!zeroDegree.isEmpty()) {
                Iterator<Integer> it = zeroDegree.iterator();
                int course = it.next();
                zeroDegree.remove(course);
                for (int[] prereq : prerequisites) {
                    if (prereq[1] == course) {
                        indegree[prereq[0]]--;
                        if (indegree[prereq[0]] == 0) {
                            zeroDegree.add(prereq[0]);
                        }
                    }
                }
            }

            for (int i : indegree) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

192. `_208.java`

```
package com.fishercoder.solutions;

/**
 * 208. Implement Trie (Prefix Tree)
 *
 * Implement a trie with insert, search, and startsWith methods.

 Note:
 You may assume that all inputs are consist of lowercase letters a-z.
 */

public class _208 {
    public static class Solution1 {
        class TrieNode {

            char val;
            boolean isWord;
            TrieNode[] children = new TrieNode[26];

            // Initialize your data structure here.
            public TrieNode() {
            }

            public TrieNode(char c) {
                this.val = c;
            }
        }

        public class Trie {
            private TrieNode root;

            public Trie() {
                root = new TrieNode();
                root.val = ' ';//initialize root to be an empty char, this is a common
practice as how Wiki defines Trie data structure as well
            }

            // Inserts a word into the trie.
            public void insert(String word) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                    if (node.children[word.charAt(i) - 'a'] == null) {
                        node.children[word.charAt(i) - 'a'] = new
TrieNode(word.charAt(i));
                    }
                    node = node.children[word.charAt(i) - 'a'];
                }
                node.isWord = true;
            }

            // Returns if the word is in the trie.
```

```
            public boolean search(String word) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                    if (node.children[word.charAt(i) - 'a'] == null) {
                        return false;
                    }
                    node = node.children[word.charAt(i) - 'a'];
                }
                return node.isWord;
            }

            // Returns if there is any word in the trie
            // that starts with the given prefix.
            public boolean startsWith(String prefix) {
                TrieNode node = root;
                for (int i = 0; i < prefix.length(); i++) {
                    if (node.children[prefix.charAt(i) - 'a'] == null) {
                        return false;
                    }
                    node = node.children[prefix.charAt(i) - 'a'];
                }
                return true;
            }
        }

        // Your Trie object will be instantiated and called as such:
        // Trie trie = new Trie();
        // trie.insert("somestring");
        // trie.search("key");
    }
}
```

193. `_209.java`

```
package com.fishercoder.solutions;

/**
 * 209. Minimum Size Subarray Sum
 *
 * Given an array of n positive integers and a positive integer s,
 * find the minimal length of a contiguous subarray of which the sum ≥ s.
 * If there isn't one, return 0 instead.

 For example, given the array [2,3,1,2,4,3] and s = 7,
 the subarray [4,3] has the minimal length under the problem constraint.

 click to show more practice.

 More practice:
 If you have figured out the O(n) solution,
 try coding another solution of which the time complexity is O(n log n).
 */
public class _209 {

    public static class Solution1 {
        public int minSubArrayLen(int s, int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            int i = 0;
            int j = 0;
            int min = Integer.MAX_VALUE;
            int sum = 0;
            while (j < nums.length) {
                sum += nums[j++];

                while (sum >= s) {
                    min = Math.min(min, j - i);
                    sum -= nums[i++];
                }
            }
            return min == Integer.MAX_VALUE ? 0 : min;
        }
    }
}
```

194. `_210.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 210. Course Schedule II
 *
 * There are a total of n courses you have to take, labeled from 0 to n - 1.
 Some courses may have prerequisites, for example to take course 0 you have to first take course 1,
 which is expressed as a pair: [0,1]
 Given the total number of courses and a list of prerequisite pairs,
 return the ordering of courses you should take to finish all courses.

 There may be multiple correct orders,
 you just need to return one of them.
 If it is impossible to finish all courses, return an empty array.

 For example:

 2, [[1,0]]
 There are a total of 2 courses to take.
 To take course 1 you should have finished course 0. So the correct course order is [0,1]

 4, [[1,0],[2,0],[3,1],[3,2]]
 There are a total of 4 courses to take.
 To take course 3 you should have finished both courses 1 and 2.
 Both courses 1 and 2 should be taken after you finished course 0.
 So one correct course order is [0,1,2,3]. Another correct ordering is[0,2,1,3].

 Note:
 The input prerequisites is a graph represented by a list of edges, not adjacency matrices. Read more about how a graph is represented.
 You may assume that there are no duplicate edges in the input prerequisites.
 click to show more hints.

 Hints:
 This problem is equivalent to finding the topological order in a directed graph. If a cycle exists, no topological ordering exists and therefore it will be impossible to take all courses.
 Topological Sort via DFS - A great video tutorial (21 minutes) on Coursera explaining the basic concepts of Topological Sort.
 Topological sort could also be done via BFS.
 */
public class _210 {

    public static class Solution1 {
```

```
        public int[] findOrder(int numCourses, int[][] prerequisites) {
            int[] inDegree = new int[numCourses];
            for (int[] course : prerequisites) {
                inDegree[course[0]]++;
            }

            Set<Integer> zeroDegree = new HashSet();
            Queue<Integer> queue = new LinkedList();
            for (int i = 0; i < numCourses; i++) {
                if (inDegree[i] == 0) {
                    zeroDegree.add(i);
                    queue.offer(i);
                }
            }

            if (zeroDegree.isEmpty()) {
                return new int[0];
            }

            while (!zeroDegree.isEmpty()) {
                Iterator<Integer> it = zeroDegree.iterator();
                int course = it.next();
                zeroDegree.remove(course);
                for (int[] pre : prerequisites) {
                    if (course == pre[1]) {
                        inDegree[pre[0]]--;
                        if (inDegree[pre[0]] == 0) {
                            zeroDegree.add(pre[0]);
                            queue.offer(pre[0]);
                        }
                    }
                }
            }

            for (int i = 0; i < numCourses; i++) {
                if (inDegree[i] != 0) {
                    return new int[0];
                }
            }

            int[] result = new int[queue.size()];
            int i = 0;
            while (!queue.isEmpty()) {
                result[i++] = queue.poll();
            }
            return result;
        }
    }
}
```

195. `_211.java`

```
package com.fishercoder.solutions;

/**
 * 211. Add and Search Word - Data structure design
 *
 * Design a data structure that supports the following two operations:

 void addWord(word)
 bool search(word)
 search(word) can search a literal word or a regular expression string containing only
letters a-z or .. A . means it can represent any one letter.

 For example:

 addWord("bad")
 addWord("dad")
 addWord("mad")
 search("pad") -> false
 search("bad") -> true
 search(".ad") -> true
 search("b..") -> true
 Note:
 You may assume that all words are consist of lowercase letters a-z.

 You should be familiar with how a Trie works. If not, please work on this problem:
Implement Trie (Prefix Tree) first.
 */
public class _211 {
  public static class Solution1 {
    public static class WordDictionary {
      WordNode root;

      /** Initialize your data structure here. */
      public WordDictionary() {
        root = new WordNode();
      }

      public void addWord(String word) {
        char[] chars = word.toCharArray();
        addWord(chars, 0, root);
      }

      private void addWord(char[] chars, int index, WordNode parent) {
        char c = chars[index];
        int idx = c - 'a';
        WordNode node = parent.children[idx];
        if (node == null) {
          node = new WordNode();
          parent.children[idx] = node;
        }
        if (chars.length == index + 1) {
```

```
          node.isLeaf = true;
          return;
        }
        addWord(chars, ++index, node);
      }

      public boolean search(String word) {
        return search(word.toCharArray(), 0, root);
      }

      /** This is also a beautifully designed recursive function. */
      private boolean search(char[] chars, int index, WordNode parent) {
        if (index == chars.length) {
          if (parent.isLeaf) {
            return true;
          }
          return false;
        }
        WordNode[] childNodes = parent.children;
        char c = chars[index];
        if (c == '.') {
          for (int i = 0; i < childNodes.length; i++) {
            WordNode n = childNodes[i];
            if (n != null) {
              boolean b = search(chars, index + 1, n);
              if (b) {
                return true;
              }
            }
          }
          return false;
        }
        WordNode node = childNodes[c - 'a'];
        if (node == null) {
          return false;
        }
        return search(chars, ++index, node);
      }

      /** This is a cool/standard design for a Trie node class. */
      private class WordNode {
        boolean isLeaf;
        WordNode[] children = new WordNode[26];
      }
    }

    /**
     * Your WordDictionary object will be instantiated and called as such:
     * WordDictionary obj = new WordDictionary();
     * obj.addWord(word);
     * boolean param_2 = obj.search(word);
     */
  }
}
```

196. `_212.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 212. Word Search II
 * Given a 2D board and a list of words from the dictionary, find all words in the board.

 Each word must be constructed from letters of sequentially adjacent cell, where
"adjacent" cells are those horizontally or vertically neighboring. The same letter cell
may not be used more than once in a word.

 For example,
 Given words = ["oath","pea","eat","rain"] and board =

 [
 ['o','a','a','n'],
 ['e','t','a','e'],
 ['i','h','k','r'],
 ['i','f','l','v']
 ]
 Return ["eat","oath"].
 Note:
 You may assume that all inputs are consist of lowercase letters a-z.

 You would need to optimize your backtracking to pass the larger test. Could you stop
backtracking earlier?

 If the current candidate does not exist in all words' prefix, you could stop
backtracking immediately.
 What kind of data structure could answer such query efficiently?
 Does a hash table work? Why or why not? How about a Trie?
 If you would like to learn how to implement a basic trie, please work on this problem:
Implement Trie (Prefix Tree) first.*/
public class _212 {

    public static class Solution1 {
        public List<String> findWords(char[][] board, String[] words) {
            TrieNode root = buildTrie(words);
            List<String> result = new ArrayList();
            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    dfs(root, board, i, j, result);
                }
            }
            return result;
        }

        private void dfs(TrieNode root, char[][] board, int i, int j, List<String>
result) {
```

```
            char c = board[i][j];

            if (c == '#' || root.children[c - 'a'] == null) {
                return;
            }

            if (root.children[c - 'a'].word != null) {
                result.add(root.children[c - 'a'].word);
                root.children[c - 'a'].word = null;//de-duplicate
            }
            board[i][j] = '#';//mark it as visited to avoid cycles
            if (i > 0) {
                dfs(root.children[c - 'a'], board, i - 1, j, result);
            }
            if (j > 0) {
                dfs(root.children[c - 'a'], board, i, j - 1, result);
            }
            if (i + 1 < board.length) {
                dfs(root.children[c - 'a'], board, i + 1, j, result);
            }
            if (j + 1 < board[0].length) {
                dfs(root.children[c - 'a'], board, i, j + 1, result);
            }

            board[i][j] = c;
        }

        private TrieNode root;

        class TrieNode {
            String word;
            TrieNode[] children = new TrieNode[26];
        }

        private TrieNode buildTrie(String[] words) {
            TrieNode root = new TrieNode();
            for (String word : words) {
                char[] chars = word.toCharArray();
                TrieNode temp = root;
                for (char c : chars) {
                    if (temp.children[c - 'a'] == null) {
                        temp.children[c - 'a'] = new TrieNode();
                    }
                    temp = temp.children[c - 'a'];
                }
                temp.word = word;
            }
            return root;
        }
    }
}
```

197. `_213.java`

```
package com.fishercoder.solutions;
/**
 * 213. House Robber II

 Note: This is an extension of House Robber.

 After robbing those houses on that street,
 the thief has found himself a new place for his thievery
 so that he will not get too much attention.
 This time, all houses at this place are arranged in a circle.
 That means the first house is the neighbor of the last one.
 Meanwhile, the security system for these houses remain the same as for those in the
previous street.

Given a list of non-negative integers representing the amount of money of each house,
 determine the maximum amount of money you can rob tonight without alerting the police.
*/
public class _213 {
    public static class Solution1 {
        /**
         * Another dp problem:
         * separate them into two cases:
         * 1. rob from house 1 to n-1, get max1
         * 2. rob from house 2 to n, get max2 take the max from the above two max
         */
        public int rob(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            if (nums.length == 1) {
                return nums[0];
            }
            if (nums.length == 2) {
                return Math.max(nums[0], nums[1]);
            }
            int[] dp = new int[nums.length - 1];

            //rob 1 to n-1
            dp[0] = nums[0];
            dp[1] = Math.max(nums[0], nums[1]);
            for (int i = 2; i < nums.length - 1; i++) {
                dp[i] = Math.max(dp[i - 2] + nums[i], dp[i - 1]);
            }
            int prevMax = dp[nums.length - 2];

            //rob 2 to n
            dp = new int[nums.length - 1];
            dp[0] = nums[1];
            dp[1] = Math.max(nums[1], nums[2]);
            for (int i = 3; i < nums.length; i++) {
                dp[i - 1] = Math.max(dp[i - 3] + nums[i], dp[i - 2]);
```

```
            }
            return Math.max(prevMax, dp[nums.length - 2]);
        }
    }
}
```

198. `_214.java`

```
package com.fishercoder.solutions;

/**
 214. Shortest Palindrome

 Given a string S, you are allowed to convert it to a palindrome
 by adding characters in front of it.
 Find and return the shortest palindrome you can find by performing this transformation.

 For example:

 Given "aacecaaa", return "aaacecaaa".

 Given "abcd", return "dcbabcd".
 */
public class _214 {

    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/27261/clean-kmp-solution-with-
super-detailed-explanation*/
        /**
         * TODO: read it explanation and understand KMP completely.
         */
        public String shortestPalindrome(String s) {
            String temp = s + "#" + new StringBuilder(s).reverse().toString();
            int[] table = getTable(temp);
            //get the maximum palin part in s starts from 0
            return new StringBuilder(s.substring(table[table.length -
1])).reverse().toString() + s;
        }

        public int[] getTable(String s) {
            //get lookup table
            int[] table = new int[s.length()];

            //pointer that points to matched char in prefix part
            int index = 0;
            //skip index 0, we will not match a string with itself
            for (int i = 1; i < s.length(); i++) {
                if (s.charAt(index) == s.charAt(i)) {
                    //we can extend match in prefix and postfix
                    table[i] = table[i - 1] + 1;
                    index++;
                } else {
                    //match failed, we try to match a shorter substring

                    //by assigning index to table[i-1], we will shorten the match string
length, and jump to the
                    //prefix part that we used to match postfix ended at i - 1
                    index = table[i - 1];
```

```
                    while (index > 0 && s.charAt(index) != s.charAt(i)) {
                        //we will try to shorten the match string length until we revert
to the beginning of match (index 1)
                        index = table[index - 1];
                    }

                    //when we are here may either found a match char or we reach the
boundary and still no luck
                    //so we need check char match
                    if (s.charAt(index) == s.charAt(i)) {
                        //if match, then extend one char
                        index++;
                    }
                    table[i] = index;
                }
            }
            return table;
        }
    }
}
```

199. `_215.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Collections;
import java.util.PriorityQueue;

/**
 * 215. Kth Largest Element in an Array
 *
 * Find the kth largest element in an unsorted array. Note that it is the kth largest
element in the sorted order, not the kth distinct element.

 For example,
 Given [3,2,1,5,6,4] and k = 2, return 5.

 Note:
 You may assume k is always valid, 1 ≤ k ≤ array's length.

 */
public class _215 {

    public static class Solution1 {
        public int findKthLargest(int[] nums, int k) {
            Arrays.sort(nums);
            return nums[nums.length - k];
        }
    }

    public static class Solution2 {
        public int findKthLargest(int[] nums, int k) {
            PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
            for (int i : nums) {
                maxHeap.offer(i);
            }
            while (k-- > 1) {
                maxHeap.poll();
            }
            return maxHeap.poll();
        }
    }

    public static class Solution3 {
        /**Quick Select algorithm
         * Time: O(n) in average, O(n^2) in worst case
         *
         * Reference: https://discuss.leetcode.com/topic/14611/java-quick-select*/
        public int findKthLargest(int[] nums, int k) {
            int start = 0;
            int end = nums.length - 1;
            int index = nums.length - k;
```

```
            while (start < end) {
                int pivot = partition(nums, start, end);
                if (pivot < index) {
                    start = pivot + 1;
                } else if (pivot > index) {
                    end = pivot - 1;
                } else {
                    return nums[pivot];
                }
            }
            return nums[start];
        }

        int partition(int[] nums, int start, int end) {
            int pivot = start;
            while (start <= end) {
                while (start <= end && nums[start] <= nums[pivot]) {
                    start++;
                }
                while (start <= end && nums[end] > nums[pivot]) {
                    end--;
                }
                if (start > end) {
                    break;
                }
                swap(nums, start, end);
            }
            swap(nums, end, pivot);
            return end;
        }

        void swap(int[] nums, int i, int j) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }
    }
}
```

200. `_216.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 216. Combination Sum III
 *
 * Find all possible combinations of k numbers that add up to a number n,
 * given that only numbers from 1 to 9 can be used and each combination should be a
unique set of numbers.

 Example 1:
 Input: k = 3, n = 7
 Output: [[1,2,4]]

 Example 2:
 Input: k = 3, n = 9
 Output: [[1,2,6], [1,3,5], [2,3,4]]
 */
public class _216 {

    public static class Solution1 {
        public List<List<Integer>> combinationSum3(int k, int n) {
            List<List<Integer>> result = new ArrayList();
            int[] nums = new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9};
            backtracking(k, n, nums, 0, new ArrayList(), result);
            return result;
        }

        void backtracking(int k, int n, int[] nums, int start, List<Integer> curr,
List<List<Integer>> result) {
            if (n > 0) {
                for (int i = start; i < nums.length; i++) {
                    curr.add(nums[i]);
                    backtracking(k, n - nums[i], nums, i + 1, curr, result);
                    curr.remove(curr.size() - 1);
                }
            } else if (n == 0 && curr.size() == k) {
                result.add(new ArrayList(curr));
            }
        }
    }
}
```

201. `_217.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 217. Contains Duplicate
 *
 * Given an array of integers, find if the array contains any
 * duplicates. Your function should return true if any value appears at least twice in
the array,
 * and it should return false if every element is distinct.
 */
public class _217 {
    public static class Solution1 {
        public boolean containsDuplicate(int[] nums) {
            if (nums == null || nums.length == 0) {
                return false;
            }
            Set<Integer> set = new HashSet();
            for (int i : nums) {
                if (!set.add(i)) {
                    return true;
                }
            }
            return false;
        }
    }

}
```

202. `_218.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.TreeMap;
/**
 * 218. The Skyline Problem
 *
 * A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance.
 * Now suppose you are given the locations and height of all the buildings as shown on a cityscape photo (Figure A),
 * write a program to output the skyline formed by these buildings collectively (Figure B).
 *
 * The geometric information of each building is represented by a triplet of integers [Li, Ri, Hi],
 * where Li and Ri are the x coordinates of the left and right edge of the ith building, respectively,
 * and Hi is its height. It is guaranteed that 0 ≤ Li, Ri ≤ INT_MAX, 0 < Hi ≤ INT_MAX, and Ri - Li > 0.
 * You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.
 *
 * For instance, the dimensions of all buildings in Figure A are recorded as: [ [2 9 10], [3 7 15], [5 12 12], [15 20 10], [19 24 8] ] .
 * The output is a list of "key points" (red dots in Figure B) in the format of [ [x1,y1], [x2, y2], [x3, y3], ... ]
 * that uniquely defines a skyline.
 * A key point is the left endpoint of a horizontal line segment.
 * Note that the last key point, where the rightmost building ends,
 * is merely used to mark the termination of the skyline, and always has zero height.
 * Also, the ground in between any two adjacent buildings should be considered part of the skyline contour.

 For instance, the skyline in Figure B should be represented as:[ [2 10], [3 15], [7 12], [12 0], [15 10], [20 8], [24, 0] ].

 Notes:

 The number of buildings in any input list is guaranteed to be in the range [0, 10000].
 The input list is already sorted in ascending order by the left x position Li.
 The output list must be sorted by the x position.
 There must be no consecutive horizontal lines of equal height in the output skyline. For instance, [...[2 3], [4 5], [7 5], [11 5], [12 7]...] is not acceptable; the three lines of height 5 should be merged into one in the final output as such: [...[2 3], [4 5], [12 7], ...]*/

/**This video is super clear and helpful: https://www.youtube.com/watch?v=GSBLe8cKu0s
```

```
 Algorithm:
First observation: all the points in the final result come from the four angles that each building has
Scan through the horizontal lines
Use a PriorityQueue to hold each building, and make the PriorityQueue to sort on the height of the buildings
whenever we encounter the start of a building, we push it into the PriorityQueue,
whenever we finished scanning that building, we remove it from the PriorityQueue
Also, in the scan process, we'll keep updating the maxHeight in the PriorityQueue if we find a new maxHeight which means the building will be overshadowed by the new higher one

Three edge cases (see the graph illustration in the above video at 12'18"):
when two buildings have the same start point, the one with higher height shows up in the final result
when two buildings have the same end point, the one with higher height shows up in the final result
when the start point of one building is is also the end point of another building, the one with higher height shows up in the final result

 We use TreeMap over a normal PriorityQueue:
For the sake of efficiency (better time complexity), we'll use TreeMap which supports O(logn) for remove() operation,
 this is the reason we choose TreeMap over a normal PriorityQueue in Java (PriorityQueue supports add() and getMaxVal() in both O(logn) time, however, for remove(), it does NOT.)
But TreeMap in Java supports all the three operations in O(logn) time.*/

public class _218 {

    public static class Solution1 {

        class BuildingPoint implements Comparable<BuildingPoint> {
            int x;
            boolean isStart;
            int h;

            public BuildingPoint(int x, boolean isStart, int h) {
                this.x = x;
                this.h = h;
                this.isStart = isStart;
            }

            @Override
            public int compareTo(BuildingPoint o) {
                if (this.x != o.x) {
                    return this.x - o.x;
                } else {
                    if (this.isStart && o.isStart) {
                        return o.h - this.h;
                    } else if (this.isStart && !o.isStart) {
                        return -this.h - o.h;
                    } else if (!this.isStart && !o.isStart) {
                        return this.h - o.h;
                    } else {
                        return this.h + o.h;
```

```
                }
            }
        }
    }

    public List<int[]> getSkyline(int[][] buildings) {
        BuildingPoint[] bps = new BuildingPoint[buildings.length * 2];
        int index = 0;
        for (int[] building : buildings) {
            BuildingPoint bp1 = new BuildingPoint(building[0], true, building[2]);
            BuildingPoint bp2 = new BuildingPoint(building[1], false, building[2]);
            bps[index++] = bp1;
            bps[index++] = bp2;
        }

        //this is one key step:
        Arrays.sort(bps);

        List<int[]> result = new ArrayList();
        TreeMap<Integer, Integer> treeMap = new TreeMap();
        treeMap.put(0, 1);
        int prevMaxH = 0;
        for (BuildingPoint bp : bps) {
            //if it's a starting point, we'll add it into the final result
            if (bp.isStart) {
                if (treeMap.containsKey(bp.h)) {
                    treeMap.put(bp.h, treeMap.get(bp.h) + 1);
                } else {
                    treeMap.put(bp.h, 1);
                }
            } else if (!bp.isStart) {
                //if it's an ending point, we'll decrement/remove this entry
                if (treeMap.containsKey(bp.h) && treeMap.get(bp.h) > 1) {
                    treeMap.put(bp.h, treeMap.get(bp.h) - 1);
                } else {
                    treeMap.remove(bp.h);
                }
            }

            int currMaxH = treeMap.lastKey();
            if (currMaxH != prevMaxH) {
                result.add(new int[]{bp.x, currMaxH});
                prevMaxH = currMaxH;
            }

        }

        return result;
    }
}
}
```

203. `_219.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 219. Contains Duplicate II
 Given an array of integers and an integer k,
 find out whether there are two distinct indices i and j in the array
 such that nums[i] = nums[j] and the absolute difference between i and j is at most k.

 Example 1:
 Input: nums = [1,2,3,1], k = 3
 Output: true

 Example 2:
 Input: nums = [1,0,1,1], k = 1
 Output: true

 Example 3:
 Input: nums = [1,2,3,1,2,3], k = 2
 Output: false
 */
public class _219 {

    public static class Solution1 {
        public boolean containsNearbyDuplicate(int[] nums, int k) {
            Map<Integer, Integer> map = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(nums[i])) {
                    if (i - map.get(nums[i]) <= k) {
                        return true;
                    } else {
                        map.put(nums[i], i);
                    }
                } else {
                    map.put(nums[i], i);
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        public boolean containsNearbyDuplicate(int[] nums, int k) {
            Set<Integer> set = new HashSet<>();
            for (int i = 0; i < nums.length; i++) {
                if (!set.add(nums[i])) {
                    return true;
```

```
                }
                if (set.size() > k) {
                    set.remove(nums[i - k]);
                }
            }
            return false;
        }
    }
}
```

204. `_220.java`

```
package com.fishercoder.solutions;

import java.util.TreeSet;

/**
 * 220. Contains Duplicate III
 *
 * Given an array of integers, find out whether there are two
 * distinct indices i and j in the array such that the difference between nums[i] and
nums[j] is at
 * most t and the difference between i and j is at most k.
 */
public class _220 {

    public static class Solution1 {
        /**
         * TreeSet: per Java doc, is a NavigableSet implementation based on a TreeMap.
The elements are ordered
         * using their natural ordering, or by a Comparator provided at set creation
time, depending on
         * which constructor is used. This implementation provides guaranteed log(n) time
cost for the
         * basic operations (add, remove and contains).
         */

        /**
         * TreeSet turns out to be a super handy data structure for this problem. It
implements Set
         * interface and keeps elements in sorted order, plus it has two very handy APIs:
         * https://docs.oracle.com/javase/7/docs/api/java/util/TreeSet.html#ceiling(E):
Returns the
         * least element in this set greater than or equal to the given element, or null
if there is no
         * such element.
         * https://docs.oracle.com/javase/7/docs/api/java/util/TreeSet.html#floor(E):
Returns the
         * greatest element in this set less than or equal to the given element, or null
if there is no
         * such element.
         *
         * Idea: loop through this array, keep adding each element into the TreeSet, also
keep an eye on
         * the size of the TreeSet, if it's greater than the required distance of k, then
we remove the
         * left-most or oldest one from the set. For each element, we get the current
floor and the
         * current ceiling and see if it works, if it does, then we return true
immediately, otherwise,
         * we continue. Return false when we finished the loop.
         */
```

```
        public boolean containsNearbyAlmostDuplicate(int[] nums, int k, int t) {
            /**case to Long to avoid Integer overflow.*/
            TreeSet<Long> set = new TreeSet<>();
            for (int i = 0; i < nums.length; ++i) {
                Long s = set.ceiling((long) nums[i]);
                if (s != null && s - nums[i] <= t) {
                    return true;
                }

                Long g = set.floor((long) nums[i]);
                if (g != null && nums[i] - g <= t) {
                    return true;
                }

                set.add((long) nums[i]);
                if (set.size() > k) {
                    set.remove((long) nums[i - k]);
                }
            }
            return false;
        }
    }
}
```

205. `_221.java`

```
package com.fishercoder.solutions;

/**
 * 221. Maximal Square
 *
 * Given a 2D binary matrix filled with 0's and 1's, find the largest square containing
only 1's and return its area.

 For example, given the following matrix:

 1 0 1 0 0
 1 0 1 1 1
 1 1 1 1 1
 1 0 0 1 0
 Return 4.
 */
public class _221 {

    public static class Solution1 {
        /**
         * The idea is pretty straightforward: use a 2d dp table to store the
intermediate results
         */
        public int maximalSquare(char[][] matrix) {
            if (matrix == null || matrix.length == 0) {
                return 0;
            }
            int m = matrix.length;
            int n = matrix[0].length;
            int max = Integer.MIN_VALUE;
            int[][] dp = new int[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (i == 0 || j == 0) {
                        dp[i][j] = (matrix[i][j] == '1') ? 1 : 0;
                    } else {
                        if (matrix[i][j] == '0') {
                            dp[i][j] = 0;
                        } else {
                            dp[i][j] = Math.min(dp[i - 1][j], Math.min(dp[i][j - 1], dp[i
- 1][j - 1])) + 1;
                        }
                    }
                    max = (max < dp[i][j]) ? dp[i][j] : max;
                }
            }
            return max * max;
        }
    }
}
```

206. `_222.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 222. Count Complete Tree Nodes
 * Given a complete binary tree, count the number of nodes.
 * Definition of a complete binary tree from Wikipedia:
 * In a complete binary tree every level,
 * except possibly the last, is completely filled,
 * and all nodes in the last level are as far left as possible.
 * It can have between 1 and 2h nodes inclusive at the last level h.
 */
public class _222 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/21317/accepted-easy-understand-
java-solution/2
         */
        public int countNodes(TreeNode root) {
            int leftH = getLeftHeight(root);
            int rightH = getRightHeight(root);
            if (leftH == rightH) {
                return (1 << leftH) - 1;
            } else {
                return 1 + countNodes(root.left) + countNodes(root.right);
            }
        }

        private int getRightHeight(TreeNode root) {
            int height = 0;
            while (root != null) {
                root = root.right;
                height++;
            }
            return height;
        }

        private int getLeftHeight(TreeNode root) {
            int height = 0;
            while (root != null) {
                root = root.left;
                height++;
            }
            return height;
        }
    }

}
```

207. `_223.java`

```
package com.fishercoder.solutions;

/**
 * 223. Rectangle Area
 *
 * Find the total area covered by two rectilinear rectangles in a 2D plane.
 * Each rectangle is defined by its bottom left corner and top right corner as shown in
the figure.
 * Rectangle Area

 Example:

 Input: A = -3, B = 0, C = 3, D = 4, E = 0, F = -1, G = 9, H = 2
 Output: 45

 Note: Assume that the total area is never beyond the maximum possible value of int.*/
public class _223 {

    public static class Solution1 {
        public int computeArea(int A, int B, int C, int D, int E, int F, int G, int H) {
            int areaA = (C - A) * (D - B);
            int areaB = (G - E) * (H - F);

            int top = Math.min(D, H);
            int bottom = Math.max(B, F);
            int left = Math.max(A, E);
            int right = Math.min(C, G);

            int overlap = 0;
            if (top > bottom && right > left) {
                overlap = (top - bottom) * (right - left);
            }
            return areaA + areaB - overlap;
        }
    }
}
```

208. `_224.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

/**
 * 224. Basic Calculator
 *
 * Implement a basic calculator to evaluate a simple expression string.
 * The expression string may contain open ( and closing parentheses ), the plus + or
minus sign -, non-negative integers and empty spaces .
 * You may assume that the given expression is always valid.

 Some examples:
 "1 + 1" = 2
 " 2-1 + 2 " = 3
 "(1+(4+5+2)-3)+(6+8)" = 23
 Note: Do not use the eval built-in library function.
 */
public class _224 {

    public static class Solution1 {

        public int calculate(String s) {
            if (s == null || s.isEmpty()) {
                return 0;
            }

            s = s.replaceAll("\\s", "");
            char[] chars = s.toCharArray();
            List<String> filteredStr = new ArrayList();
            for (int i = 0; i < chars.length; ) {
                StringBuilder sb = new StringBuilder();
                while (i < chars.length && Character.isDigit(chars[i])) {
                    sb.append(chars[i]);
                    i++;
                }
                if (i == chars.length) {
                    if (sb.toString().length() != 0) {
                        filteredStr.add(sb.toString());
                    }
                } else {
                    if (sb.toString().length() != 0) {
                        filteredStr.add(sb.toString());
                    }
                    if (chars[i] == '+' || chars[i] == '-' || chars[i] == '(' || chars[i]
== ')') {
                        filteredStr.add(String.valueOf(chars[i]));
                    }
                    i++;
```

```
            }
        }

        for (String str : filteredStr) {
            System.out.print(str);
        }

        Stack<String> stack1 = new Stack();
        Stack<String> stack2 = new Stack();
        for (int i = 0; i < filteredStr.size(); ) {
            while (i < filteredStr.size() && !filteredStr.get(i).equals(")")) {
                stack1.push(filteredStr.get(i));
                i++;
            }
            if (i != filteredStr.size()) {
                while (!stack1.isEmpty() && !stack1.peek().equals("(")) {
                    stack2.push(stack1.pop());
                }
                stack1.pop();
                int exp = 0;
                while (!stack2.isEmpty()) {
                    if (stack2.size() == 1) {
                        stack1.push(stack2.pop());
                        break;
                    }
                    int operand1 = Integer.parseInt(stack2.pop());
                    String operator = stack2.pop();
                    int operand2 = Integer.parseInt(stack2.pop());
                    if (operator.equals("+")) {
                        exp = operand1 + operand2;
                    } else if (operator.equals("-")) {
                        exp = operand1 - operand2;
                    }
                    stack2.push(String.valueOf(exp));
                }
                i++;
            }
        }

        if (stack1.size() == 1) {
            return Integer.parseInt(stack1.pop());
        }

        while (!stack1.isEmpty()) {
            stack2.push(stack1.pop());
        }
        while (!stack2.isEmpty()) {
            if (stack2.size() == 1) {
                stack1.push(stack2.pop());
                break;
            }
            int exp = 0;
            int operand1 = Integer.parseInt(stack2.pop());
            String operator = stack2.pop();
```

```
                int operand2 = Integer.parseInt(stack2.pop());
                if (operator.equals("+")) {
                    exp = operand1 + operand2;
                } else if (operator.equals("-")) {
                    exp = operand1 - operand2;
                }
                stack2.push(String.valueOf(exp));
            }
            return Integer.parseInt(stack1.pop());
        }
    }

}
```

209. `_225.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;
/**
 * 225. Implement Stack using Queues
 *
 * Implement the following operations of a stack using queues.

 push(x) -- Push element x onto stack.
 pop() -- Removes the element on top of the stack.
 top() -- Get the top element.
 empty() -- Return whether the stack is empty.

 Notes:
 You must use only standard operations of a queue -- which means only push to back,
peek/pop from front, size, and is empty operations are valid.
 Depending on your language, queue may not be supported natively. You may simulate a
queue by using a list or deque (double-ended queue), as long as you use only standard
operations of a queue.
 You may assume that all operations are valid (for example, no pop or top operations will
be called on an empty stack).
 Update (2015-06-11):
 The class name of the Java function had been updated to MyStack instead of Stack.*/

public class _225 {

    public static class Solution1 {
        class MyStack {

            Queue<Integer> q = new LinkedList();

            // Push element x onto stack.
            public void push(int x) {
                q.offer(x);
                for (int i = 1; i < q.size(); i++) {
                    q.offer(q.remove());
                }
            }

            // Removes the element on top of the stack.
            public void pop() {
                q.poll();
            }

            // Get the top element.
            public int top() {
                return q.peek();
            }

            // Return whether the stack is empty.
```

```
            public boolean empty() {
                return q.isEmpty();
            }
        }
    }
}
```

210. `_226.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**226. Invert Binary Tree

Invert a binary tree.

     4
   /   \
  2     7
 / \   / \
1   3 6   9

to

     4
   /   \
  7     2
 / \   / \
9   6 3   1

Trivia:
This problem was inspired by this original tweet by Max Howell:
Google: 90% of our engineers use the software you wrote (Homebrew),
 but you can t invert a binary tree on a whiteboard so fuck off.
 */
public class _226 {

        public static class Solution1 {
                public TreeNode invertTree(TreeNode root) {
                        if (root == null) {
                                return root;
                        }
                        Queue<TreeNode> q = new LinkedList();
                        q.offer(root);
                        while (!q.isEmpty()) {
                                TreeNode curr = q.poll();
                                TreeNode temp = curr.left;
                                curr.left = curr.right;
                                curr.right = temp;
                                if (curr.left != null) {
                                        q.offer(curr.left);
                                }
                                if (curr.right != null) {
                                        q.offer(curr.right);
                                }
                        }
```

```
                        return root;
                }
        }

        public static class Solution2 {
                public TreeNode invertTree(TreeNode root) {
                        if (root == null) {
                                return root;
                        }
                        TreeNode temp = root.left;
                        root.left = root.right;
                        root.right = temp;
                        invertTree(root.left);
                        invertTree(root.right);
                        return root;
                }
        }

        public static class Solution3 {
                //more concise version
                public TreeNode invertTree(TreeNode root) {
                        if (root == null) {
                                return root;
                        }
                        TreeNode temp = root.left;
                        root.left = invertTree(root.right);
                        root.right = invertTree(temp);
                        return root;
                }
        }
}
```

211. `_227.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 227. Basic Calculator II
 *
 * Implement a basic calculator to evaluate a simple expression string.

 The expression string contains only non-negative integers, +, -, *, / operators and
empty spaces . The integer division should truncate toward zero.

 You may assume that the given expression is always valid.

 Some examples:
 "3+2*2" = 7
 " 3/2 " = 1
 " 3+5 / 2 " = 5
 Note: Do not use the eval built-in library function.
 */
public class _227 {

    public static class Solution1 {
        public int calculate(String s) {
            if (s == null || s.length() == 0) {
                return 0;
            }
            int len = s.length();
            Deque<Integer> stack = new ArrayDeque<>();
            int num = 0;
            char sign = '+';
            for (int i = 0; i < len; i++) {
                if (Character.isDigit(s.charAt(i))) {
                    num = num * 10 + s.charAt(i) - '0';
                }
                if ((!Character.isDigit(s.charAt(i))) && ' ' != s.charAt(i) || i == len -
1) {
                    if (sign == '+') {
                        stack.addLast(num);
                    } else if (sign == '-') {
                        stack.addLast(-num);
                    } else if (sign == '/') {
                        stack.addLast(stack.pollLast() / num);
                    } else if (sign == '*') {
                        stack.addLast(stack.pollLast() * num);
                    }
                    sign = s.charAt(i);
                    num = 0;
                }
            }
```

```
            int result = 0;
            while (!stack.isEmpty()) {
                result += stack.poll();
            }
            return result;
        }
    }
}
```

212. `_228.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 228. Summary Ranges
 *
 * Given a sorted integer array without duplicates, return the summary of its ranges.
 *
 * For example, given [0,1,2,4,5,7], return ["0->2","4->5","7"].
 */
public class _228 {

    public static class Solution1 {
        public List<String> summaryRanges(int[] nums) {
            List<String> result = new ArrayList<>();
            for (int i = 0; i < nums.length; i++) {
                String start = String.valueOf(nums[i]);
                int tmpI = i;
                while ((i + 1) < nums.length && (nums[i] + 1) == nums[i + 1]) {
                    i++;
                }
                if (tmpI == i) {
                    result.add(start);
                } else {
                    result.add(start + "->" + String.valueOf(nums[i]));
                }
            }
            return result;
        }
    }

}
```

213. `_229.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 229. Majority Element II
 *
 * Given an integer array of size n, find all elements that appear more than ⌊ n/3 ⌋
times.
 * The algorithm should run in linear time and in O(1) space.

 Hint:
 How many majority elements could it possibly have?
 Do you have a better hint? Suggest it!
 */
public class _229 {

    public static class Solution1 {
        public List<Integer> majorityElement(int[] nums) {
            Map<Integer, Integer> counterMap = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                if (counterMap.containsKey(nums[i])) {
                    counterMap.put(nums[i], counterMap.get(nums[i]) + 1);
                } else {
                    counterMap.put(nums[i], 1);
                }
            }
            int size = nums.length;
            List<Integer> result = new ArrayList<>();
            for (Integer i : counterMap.keySet()) {
                int threshold = size / 3;
                if (counterMap.get(i) > threshold) {
                    result.add(i);
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**Moore Voting algorithm*/
        public List<Integer> majorityElement(int[] nums) {
            List<Integer> result = new ArrayList<>();
            if (nums == null || nums.length == 0) {
                return result;
            }
            int count1 = 0;
            int count2 = 0;
```

```
            int candidate1 = 0;
            int candidate2 = 1;
            for (int num : nums) {
                if (num == candidate1) {
                    count1++;
                } else if (num == candidate2) {
                    count2++;
                } else if (count1 == 0) {
                    candidate1 = num;
                    count1 = 1;
                } else if (count2 == 0) {
                    candidate2 = num;
                    count2 = 1;
                } else {
                    count1--;
                    count2--;
                }
            }
            count1 = 0;
            count2 = 0;
            for (int num : nums) {
                if (num == candidate1) {
                    count1 += 2;
                } else {
                    count1--;
                }
                if (num == candidate2) {
                    count2 += 2;
                } else {
                    count2--;
                }
            }
            if (count1 > 0) {
                result.add(candidate1);
            }
            if (count2 > 0) {
                result.add(candidate2);
            }
            return result;
        }
    }

}
```

214. `_230.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 230. Kth Smallest Element in a BST
 *
 * Given a binary search tree, write a function kthSmallest to find the kth smallest
element in it.

 Note:
 You may assume k is always valid, 1 ? k ? BST's total elements.

 Follow up:
 What if the BST is modified (insert/delete operations) often and you need to find the
kth smallest frequently? How would you optimize the kthSmallest routine?

 */
public class _230 {

    public static class Solution1 {
        public int kthSmallest(TreeNode root, int k) {
            List<Integer> inorderList = new ArrayList<>();
            inorder(root, inorderList);
            return inorderList.get(k - 1);
        }

        private void inorder(TreeNode root, List<Integer> inorderList) {
            if (root == null) {
                return;
            }
            if (root.left != null) {
                inorder(root.left, inorderList);
            }
            inorderList.add(root.val);
            if (root.right != null) {
                inorder(root.right, inorderList);
            }
            return;
        }
    }

    public static class Solution2 {
        /**
         * Inorder traversal gives the natural ordering of a BST, no need to sort.
         */
        int count = 0;
        int result = Integer.MIN_VALUE;
```

```
        public int kthSmallest(TreeNode root, int k) {
            inorder(root, k);
            return result;
        }

        private void inorder(TreeNode root, int k) {
            if (root == null) {
                return;
            }
            inorder(root.left, k);
            count++;
            if (count == k) {
                result = root.val;
                return;
            }
            inorder(root.right, k);
        }
    }

}
```

215. `_231.java`

```
package com.fishercoder.solutions;

/**
 * 231. Power of Two
 *
 * Given an integer, write a function to determine if it is a power of two.
 */

public class _231 {
    public static class Solution1 {
        public boolean isPowerOfTwo(int n) {
            //after writing out the binary representation of some numbers: 1,2,4,8,16,32,
you can easily figure out that
            //every number that is power of two has only one bit that is 1
            //then we can apply that cool trick that we learned from {@link easy._191}:
n&(n-1) which will clear the least significant bit in n to zero
            return n > 0 && (n & (n - 1)) == 0;
        }
    }
}
```

216. `_232.java`

```
package com.fishercoder.solutions;

import java.util.Stack;
/**
 * 232. Implement Queue using Stacks
 *
 * Implement the following operations of a queue using stacks.

 push(x) -- Push element x to the back of queue.
 pop() -- Removes the element from in front of queue.
 peek() -- Get the front element.
 empty() -- Return whether the queue is empty.

 Notes:
 You must use only standard operations of a stack -- which means only push to top,
peek/pop from top, size, and is empty operations are valid.
 Depending on your language, stack may not be supported natively. You may simulate a
stack by using a list or deque (double-ended queue), as long as you use only standard
operations of a stack.
 You may assume that all operations are valid (for example, no pop or peek operations
will be called on an empty queue).
 */
public class _232 {

    public static class Solution1 {
        class MyQueue {

            Stack<Integer> input = new Stack();
            Stack<Integer> output = new Stack();

            // Push element x to the back of queue.
            public void push(int x) {
                input.push(x);
            }

            // Removes the element from in front of queue.
            public int pop() {
                peek();
                return output.pop();
            }

            // Get the front element.
            public int peek() {
                if (output.isEmpty()) {
                    while (!input.isEmpty()) {
                        output.push(input.pop());
                    }
                }
                return output.peek();
            }
```

```
            // Return whether the queue is empty.
            public boolean empty() {
                return input.isEmpty() && output.isEmpty();
            }
        }
    }
}
```

217. `_233.java`

```
package com.fishercoder.solutions;

/**
 * 233. Number of Digit One
 *
 * Given an integer n, count the total number of digit 1 appearing in all non-negative
integers less than or equal to n.

 For example:
 Given n = 13,
 Return 6, because digit 1 occurred in the following numbers: 1, 10, 11, 12, 13.

 Hint: Beware of overflow.
 */
public class _233 {
    public static class Solution1 {
        public int countDigitOne(int n) {
            int count = 0;
            for (long k = 1; k <= n; k *= 10) {
                long r = n / k;
                long m = n % k;
                // sum up the count of ones on every place k
                count += (r + 8) / 10 * k + (r % 10 == 1 ? m + 1 : 0);
            }
            return count;
        }
    }
}
```

218. `_234.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 234. Palindrome Linked List
 *
 * Given a singly linked list, determine if it is a palindrome.

 Follow up:
 Could you do it in O(n) time and O(1) space?
 */

public class _234 {
    public static class Solution1 {
        /**O(n) time
         * O(1) space
         * */
        public boolean isPalindrome(ListNode head) {
            if (head == null) {
                return true;
            }

            ListNode slow = head;
            ListNode fast = head;
            while (fast.next != null && fast.next.next != null) {
                fast = fast.next.next;
                slow = slow.next;
            }

            ListNode secondHalfHead = reverse(slow.next);
            ListNode firstHalfHead = head;
            while (firstHalfHead != null && secondHalfHead != null) {
                if (firstHalfHead.val != secondHalfHead.val) {
                    return false;
                }
                firstHalfHead = firstHalfHead.next;
                secondHalfHead = secondHalfHead.next;
            }
            return true;
        }

        private ListNode reverse(ListNode head) {
            ListNode newHead = null;
            while (head != null) {
                ListNode next = head.next;
                head.next = newHead;
                newHead = head;
```

```
                head = next;
            }
            return newHead;
        }
    }

    public static class Solution2 {
        /**O(n) time
         * O(n) space
         * */
        public boolean isPalindrome(ListNode head) {
            int len = 0;
            ListNode fast = head;
            ListNode slow = head;
            List<Integer> firstHalf = new ArrayList<>();
            while (fast != null && fast.next != null) {
                fast = fast.next.next;
                firstHalf.add(slow.val);
                slow = slow.next;
                len += 2;
            }
            if (fast != null) {
                len++;
            }
            if (len % 2 != 0) {
                slow = slow.next;
            }
            int i = firstHalf.size() - 1;
            while (slow != null) {
                if (firstHalf.get(i--) != slow.val) {
                    return false;
                }
                slow = slow.next;
            }
            return true;
        }
    }

}
```

219. `_235.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 235. Lowest Common Ancestor of a Binary Search Tree
 *
 * Given a binary search tree (BST), find the lowest common ancestor (LCA) of two given
nodes in the BST.
 * According to the definition of LCA on Wikipedia:
 * “The lowest common ancestor is defined between two nodes v and w as the lowest node in
T that has both v and w as descendants
 * (where we allow a node to be a descendant of itself).”

          _______6______
         /              \
      ___2__          ___8__
     /      \        /      \
     0      4        7       9
   /  \
  3   5

 For example, the lowest common ancestor (LCA) of nodes 2 and 8 is 6.
 Another example is LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself
according to the LCA definition.

 */
public class _235 {

    public static class Solution1 {
        public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
            if (root == null || p == root || q == root) {
                return root;
            }
            if ((root.val - p.val) * (root.val - q.val) > 0) {
                if (root.val - p.val > 0) {
                    return lowestCommonAncestor(root.left, p, q);
                }
                return lowestCommonAncestor(root.right, p, q);
            }
            return root;
        }
    }

}
```

220. `_236.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 236. Lowest Common Ancestor of a Binary Tree
 *
 * Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the
tree.
 * According to the definition of LCA on Wikipedia:
 * “The lowest common ancestor is defined between two nodes v and w as the lowest node in
T
 * that has both v and w as descendants (where we allow a node to be a descendant of
itself).”

           _______3______
          /              \
      ___5__          ___1__
    /      \        /      \
   6      _2       0       8
  / \
 7   4

 For example, the lowest common ancestor (LCA) of nodes 5 and 1 is 3.
 Another example is LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself
according to the LCA definition.*/

public class _236 {
    public static class Solution1 {

        public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
            if (root == null || root == p || root == q) {
                return root;
            }
            TreeNode left = lowestCommonAncestor(root.left, p, q);
            TreeNode right = lowestCommonAncestor(root.right, p, q);
            if (left != null && right != null) {
                return root;
            }
            return left != null ? left : right;
        }
    }

}
```

221. `_237.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 237. Delete Node in a Linked List
 *
 * Write a function to delete a node (except the tail) in a singly linked list, given
only access to that node.
 *
 * Given linked list -- head = [4,5,1,9], which looks like following:
 *
 * Example 1:
 * Input: head = [4,5,1,9], node = 5
 * Output: [4,1,9]
 * Explanation: You are given the second node with value 5, the linked list should become
4 -> 1 -> 9 after calling your function.
 *
 * Example 2:
 * Input: head = [4,5,1,9], node = 1
 * Output: [4,5,9]
 * Explanation: You are given the third node with value 1, the linked list should become
4 -> 5 -> 9 after calling your function.
 *
 * Note:
 * The linked list will have at least two elements.
 * All of the nodes' values will be unique.
 * The given node will not be the tail and it will always be a valid node of the linked
list.
 * Do not return anything from your function.
*/
public class _237 {

    public static class Solution1 {
        public void deleteNode(ListNode node) {
            node.val = node.next.val;
            node.next = node.next.next;
        }
    }
}
```

222. `_238.java`

```
package com.fishercoder.solutions;

/**
 * 238. Product of Array Except Self
 *
 * Given an array of n integers where n > 1, nums,
 * return an array output such that output[i] is equal to the product of all the elements
of nums except nums[i].
 * Solve it without division and in O(n).
 * For example, given [1,2,3,4], return [24,12,8,6].

 Follow up:
 Could you solve it with constant space complexity?
 (Note: The output array does not count as extra space for the purpose of space
complexity analysis.)
 */

public class _238 {

    public static class Solution1 {
        /**
         * Very straightforward idea: iterate through the array twice:
         * first time: get res[i] = res[i-1]*nums[i-1]
         * second time: have a variable called right, which means all the numbers product
to its right, then do
         * res[i] *= right;
         * right *= nums[i];
         * that's it.
         *
         * This could be very well illustrated with this example: [1,2,3,4]
         */
        public int[] productExceptSelf(int[] nums) {
            int n = nums.length;
            int[] result = new int[n];
            result[0] = 1;
            for (int i = 1; i < n; i++) {
                result[i] = result[i - 1] * nums[i - 1];
            }
            int right = 1;
            for (int i = n - 1; i >= 0; i--) {
                result[i] *= right;
                right *= nums[i];
            }
            return result;
        }
    }
}
```

223. `_239.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 239. Sliding Window Maximum
 *
 * Given an array nums, there is a sliding window of size k which is moving from the very
left of the array to the very right.
 * You can only see the k numbers in the window. Each time the sliding window moves right
by one position.

 For example:

 Given nums = [1,3,-1,-3,5,3,6,7], and k = 3.

 Window position                Max
 ---------------               -----
 [1  3  -1] -3  5  3  6  7      3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
 Therefore, return the max sliding window as [3,3,5,5,6,7].

 Note:
 You may assume k is always valid, ie: 1 ≤ k ≤ input array's size for non-empty array.

 Follow up:
 Could you solve it in linear time?

 Hint:
 How about using a data structure such as deque (double-ended queue)?
 The queue size need not be the same as the window’s size.
 Remove redundant elements and the queue should store only elements that need to be
considered.
 */
public class _239 {

    public static class Solution1 {
        public int[] maxSlidingWindow(int[] nums, int k) {
            if (nums == null || nums.length == 0 || k == 0) {
                return new int[0];
            }
            PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> b - a);
            int[] res = new int[nums.length - k + 1];
            for (int i = 0; i < nums.length; i++) {
                if (i < k) {
                    heap.offer(nums[i]);
                    if (i == k - 1) {
```

```
                    res[0] = heap.peek();
                }
            } else {
                heap.remove(nums[i - k]);
                heap.offer(nums[i]);
                res[i - k + 1] = heap.peek();
            }
        }
        return res;
    }
}
```

224. `_240.java`

```
package com.fishercoder.solutions;

/**
 * 240. Search a 2D Matrix II
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix.
 * This matrix has the following properties:

 Integers in each row are sorted in ascending from left to right.
 Integers in each column are sorted in ascending from top to bottom.
 For example,

 Consider the following matrix:

 [
 [1,   4,  7, 11, 15],
 [2,   5,  8, 12, 19],
 [3,   6,  9, 16, 22],
 [10, 13, 14, 17, 24],
 [18, 21, 23, 26, 30]
 ]
 Given target = 5, return true.

 Given target = 20, return false.
 */
public class _240 {

    public static class Solution1 {
        public boolean searchMatrix(int[][] matrix, int target) {
            if (matrix == null || matrix.length == 0) {
                return false;
            }
            int m = matrix.length;
            int n = matrix[0].length;
            int x = 0;
            int y = n - 1;
            while (x < m && y >= 0) {
                if (target == matrix[x][y]) {
                    return true;
                } else if (target > matrix[x][y]) {
                    x++;
                } else {
                    y--;
                }
            }
            return false;
        }
    }

}
```

225. `_241.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.List;

/**
 * 241. Different Ways to Add Parentheses
 *
 * Given a string of numbers and operators, return all possible results from computing
all the different possible ways to group numbers and operators. The valid operators are
+, - and *.


 Example 1
 Input: "2-1-1".

 ((2-1)-1) = 0
 (2-(1-1)) = 2
 Output: [0, 2]


 Example 2
 Input: "2*3-4*5"

 (2*(3-(4*5))) = -34
 ((2*3)-(4*5)) = -14
 ((2*(3-4))*5) = -10
 (2*((3-4)*5)) = -10
 (((2*3)-4)*5) = 10
 Output: [-34, -14, -10, -10, 10]

 */
public class _241 {
    public static class Solution1 {
        /**Time:  O(n * 4^n / n^(3/2)) ~= n * Catalan numbers = n * (C(2n, n) - C(2n, n -
1)),
         due to the size of the results is Catalan numbers,
         and every way of evaluation is the length of the string,
         so the time complexity is at most n * Catalan numbers.
         Space: O(n * 4^n / n^(3/2)), the cache size of lookup is at most n * Catalan
numbers.*/

        /**
         * Credit: https://discuss.leetcode.com/topic/19901/a-recursive-java-solution-
284-ms
         */
        public List<Integer> diffWaysToCompute(String input) {
            List<Integer> answer = new LinkedList<>();
            int len = input.length();
            for (int i = 0; i < len; i++) {
                if (input.charAt(i) == '+'
```

```
                    || input.charAt(i) == '-'
                    || input.charAt(i) == '*') {
                String part1 = input.substring(0, i);
                String part2 = input.substring(i + 1);
                List<Integer> answer1 = diffWaysToCompute(part1);
                List<Integer> answer2 = diffWaysToCompute(part2);
                for (int a1 : answer1) {
                    for (int a2 : answer2) {
                        int result = 0;
                        switch (input.charAt(i)) {
                            case '+':
                                result = a1 + a2;
                                break;
                            case '-':
                                result = a1 - a2;
                                break;
                            case '*':
                                result = a1 * a2;
                                break;
                            default:
                                break;
                        }
                        answer.add(result);
                    }
                }
            }
        }
        if (answer.size() == 0) {
            answer.add(Integer.valueOf(input));
        }
        return answer;
    }
}
}
```

226. `_242.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 242. Valid Anagram
 * Given two strings s and t, write a function to determine if t is an anagram of s.

 For example,
 s = "anagram", t = "nagaram", return true.
 s = "rat", t = "car", return false.

 Note:
 You may assume the string contains only lowercase alphabets.

 Follow up:
 What if the inputs contain unicode characters? How would you adapt your solution to such
case?
 */

public class _242 {

    public static class Solution1 {
        public boolean isAnagram(String s, String t) {
            char[] schar = s.toCharArray();
            char[] tchar = t.toCharArray();
            Arrays.sort(schar);
            Arrays.sort(tchar);
            return new String(schar).equals(new String(tchar));
        }
    }

    public static class Solution2 {
        public boolean isAnagram(String s, String t) {
            if (s == null || t == null || s.length() != t.length()) {
                return false;
            }
            int[] counts = new int[26];
            for (int i = 0; i < s.length(); i++) {
                counts[s.charAt(i) - 'a']++;
                counts[t.charAt(i) - 'a']--;
            }
            for (int i : counts) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

227. `_243.java`

```
package com.fishercoder.solutions;

/**
 * 243. Shortest Word Distance
 *
 * Given a list of words and two words word1 and word2, return the shortest distance
between these two words in the list.

 For example,
 Assume that words = ["practice", "makes", "perfect", "coding", "makes"].

 Given word1 = “coding”, word2 = “practice”, return 3.
 Given word1 = "makes", word2 = "coding", return 1.

 Note:
 You may assume that word1 does not equal to word2, and word1 and word2 are both in the
list.*/
public class _243 {
    public static class Solution1 {
        public int shortestDistance(String[] words, String word1, String word2) {
            int p = -1;
            int q = -1;
            int min = Integer.MAX_VALUE;
            for (int i = 0; i < words.length; i++) {
                if (words[i].equals(word1)) {
                    p = i;
                }
                if (words[i].equals(word2)) {
                    q = i;
                }
                if (p != -1 && q != -1) {
                    min = Math.min(min, Math.abs(p - q));
                }
            }
            return min;

        }
    }
}
```

228. `_244.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 244. Shortest Word Distance II
 *
 * This is a follow up of Shortest Word Distance. The only difference is now you are
given the list of words and your method will be called repeatedly many times with
different parameters. How would you optimize it?
 * Design a class which receives a list of words in the constructor, and implements a
method that takes two words word1 and word2 and return the shortest distance between
these two words in the list.

 For example,
 Assume that words = ["practice", "makes", "perfect", "coding", "makes"].

 Given word1 = “coding”, word2 = “practice”, return 3.
 Given word1 = "makes", word2 = "coding", return 1.

 Note:
 You may assume that word1 does not equal to word2, and word1 and word2 are both in the
list.
 */
public class _244 {
    public static class Solution1 {
        class WordDistance {

            private Map<String, List<Integer>> map;

            public WordDistance(String[] words) {
                map = new HashMap<>();
                for (int i = 0; i < words.length; i++) {
                    String w = words[i];
                    if (map.containsKey(w)) {
                        map.get(w).add(i);
                    } else {
                        List<Integer> list = new ArrayList<>();
                        list.add(i);
                        map.put(w, list);
                    }
                }
            }

            public int shortest(String word1, String word2) {
                List<Integer> list1 = map.get(word1);
                List<Integer> list2 = map.get(word2);
                int result = Integer.MAX_VALUE;
```

```
                for (int i = 0, j = 0; i < list1.size() && j < list2.size(); ) {
                    int index1 = list1.get(i);
                    int index2 = list2.get(j);
                    if (index1 < index2) {
                        result = Math.min(result, index2 - index1);
                        i++;
                    } else {
                        result = Math.min(result, index1 - index2);
                        j++;
                    }
                }
                return result;
            }
        }
    }
}
```

229. `_245.java`

```
package com.fishercoder.solutions;

/**
 * 245. Shortest Word Distance III
 *
 * This is a follow up of Shortest Word Distance. The only difference is now word1 could
be the same as word2.
 * Given a list of words and two words word1 and word2, return the shortest distance
between these two words in the list.
 * word1 and word2 may be the same and they represent two individual words in the list.

 For example,
 Assume that words = ["practice", "makes", "perfect", "coding", "makes"].

 Given word1 = “makes”, word2 = “coding”, return 1.
 Given word1 = "makes", word2 = "makes", return 3.

 Note:
 You may assume word1 and word2 are both in the list.
 */
public class _245 {

    public static class Solution1 {
        public int shortestWordDistance(String[] words, String word1, String word2) {
            int p1 = -1;
            int p2 = -1;
            int min = Integer.MAX_VALUE;
            for (int i = 0; i < words.length; i++) {
                if (words[i].equals(word1)) {
                    if (word1.equals(word2)) {
                        if (p1 != -1 && i - p1 < min) {
                            min = i - p1;
                        }
                        p1 = i;
                    } else {
                        p1 = i;
                        if (p2 != -1 && p1 - p2 < min) {
                            min = p1 - p2;
                        }
                    }
                } else if (words[i].equals(word2)) {
                    p2 = i;
                    if (p1 != -1 && p2 - p1 < min) {
                        min = p2 - p1;
                    }
                }
            }
            return min;
        }
    }
}
```

230. `_246.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 246. Strobogrammatic Number
 *
 * A strobogrammatic number is a number that looks the same when rotated 180 degrees
(looked at upside down).
 * Write a function to determine if a number is strobogrammatic. The number is
represented as a string.
 * For example, the numbers "69", "88", and "818" are all strobogrammatic.
 */

public class _246 {

    public static class Solution1 {
        public boolean isStrobogrammatic(String num) {
            int i = 0;
            int j = num.length() - 1;
            Map<Character, Character> map = new HashMap();
            map.put('8', '8');
            map.put('1', '1');
            map.put('0', '0');
            if (j == 0) {
                return map.containsKey(num.charAt(i));
            }

            map.put('9', '6');
            map.put('6', '9');
            while (i < j) {
                if (!map.containsKey(num.charAt(i)) || !map.containsKey(num.charAt(j))) {
                    return false;
                }
                if (map.get(num.charAt(i)) != num.charAt(j)) {
                    return false;
                }
                i++;
                j--;
            }
            return map.containsKey(num.charAt(i));
        }
    }


    public static class Solution2 {
        public boolean isStrobogrammatic(String num) {
            Set<Character> set = new HashSet();
```

```
            set.add('0');
            set.add('1');
            set.add('6');
            set.add('8');
            set.add('9');
            char[] nums = num.toCharArray();
            int i = 0;
            int j = num.length() - 1;
            while (i <= j) {
                if (!set.contains(nums[i]) || !set.contains(nums[j])) {
                    return false;
                }
                if (nums[i] == '6' && nums[j] != '9') {
                    return false;
                } else if (nums[i] == '9' && nums[j] != '6') {
                    return false;
                } else if (nums[i] == '1' && nums[j] != '1') {
                    return false;
                } else if (nums[i] == '8' && nums[j] != '8') {
                    return false;
                } else if (nums[i] == '0' && nums[j] != '0') {
                    return false;
                } else {
                    i++;
                    j--;
                }
            }
            return true;
        }
    }
}
```

231. `_247.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class _247 {
    public static class Solution1 {
        public List<String> findStrobogrammatic(int n) {
            return recursiveHelper(n, n);
        }

        private List<String> recursiveHelper(int n, int m) {
            if (n == 0) {
                return new ArrayList<>(Arrays.asList(""));
            }
            if (n == 1) {
                return new ArrayList<>(Arrays.asList("0", "1", "8"));
            }

            List<String> list = recursiveHelper(n - 2, m);
            List<String> res = new ArrayList<>();

            for (int i = 0; i < list.size(); i++) {
                String s = list.get(i);
                if (n != m) {
                    res.add("0" + s + "0");
                }
                res.add("1" + s + "1");
                res.add("6" + s + "9");
                res.add("8" + s + "8");
                res.add("9" + s + "6");
            }
            return res;
        }
    }
}
```

232. `_248.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

public class _248 {

    public static class Solution1 {
        /**Credit: https://discuss.leetcode.com/topic/31386/concise-java-solution
         *
         Construct char arrays from low.length() to high.length()
         Add stro pairs from outside
         When left > right, add eligible count
         */

        private static final char[][] pairs = {{'0', '0'}, {'1', '1'}, {'6', '9'}, {'8',
'8'}, {'9', '6'}};

        public int strobogrammaticInRange(String low, String high) {
            int[] count = {0};
            for (int len = low.length(); len <= high.length(); len++) {
                char[] c = new char[len];
                dfs(low, high, c, 0, len - 1, count);
            }
            return count[0];
        }

        public void dfs(String low, String high , char[] c, int left, int right, int[]
count) {
            if (left > right) {
                String s = new String(c);
                if ((s.length() == low.length() && s.compareTo(low) < 0)
                        || (s.length() == high.length() && s.compareTo(high) > 0)) {
                    return;
                }
                count[0]++;
                return;
            }
            for (char[] p : pairs) {
                c[left] = p[0];
                c[right] = p[1];
                if (c.length != 1 && c[0] == '0') {
                    continue;
                }
                if (left == right && p[0] != p[1]) {
                    continue;
                }
                dfs(low, high, c, left + 1, right - 1, count);
            }
        }
    }
```

```
    public static class Solution2 {
        Map<Character, Character> map = new HashMap<>();

        {
            map.put('1', '1');
            map.put('8', '8');
            map.put('6', '9');
            map.put('9', '6');
            map.put('0', '0');
        }

        String low = "";
        String high = "";

        public int strobogrammaticInRange(String low, String high) {
            this.low = low;
            this.high = high;
            int result = 0;
            for (int n = low.length(); n <= high.length(); n++) {
                int[] count = new int[1];
                strobogrammaticInRange(new char[n], count, 0, n - 1);
                result += count[0];
            }
            return result;
        }

        private void strobogrammaticInRange(char[] arr, int[] count, int lo, int hi) {
            if (lo > hi) {
                String s = new String(arr);
                if ((arr[0] != '0' || arr.length == 1) && compare(low, s) && compare(s,
high)) {
                    count[0]++;
                }
                return;
            }
            for (Character c : map.keySet()) {
                arr[lo] = c;
                arr[hi] = map.get(c);
                if ((lo == hi && c == map.get(c)) || lo < hi) {
                    strobogrammaticInRange(arr, count, lo + 1, hi - 1);
                }
            }
        }

        private boolean compare(String a, String b) {
            if (a.length() != b.length()) {
                return a.length() < b.length();
            }
            int i = 0;
            while (i < a.length() && a.charAt(i) == b.charAt(i)) {
                i++;
            }
            return i == a.length() ? true : a.charAt(i) <= b.charAt(i);
```

```
        }
    }
}
```

233. `_249.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class _249 {

    public static class Solution1 {
        public List<List<String>> groupStrings(String[] strings) {

            List<List<String>> result = new ArrayList<List<String>>();
            Map<String, List<String>> map = new HashMap<String, List<String>>();

            for (String word : strings) {
                String key = "";
                int offset = word.charAt(0) - 'a';
                for (int i = 1; i < word.length(); i++) {
                    key += (word.charAt(i) - offset + 26) % 26;
                }

                if (!map.containsKey(key)) {
                    map.put(key, new ArrayList<>());
                }
                map.get(key).add(word);
            }

            for (List<String> list : map.values()) {
                Collections.sort(list);
                result.add(list);
            }

            return result;
        }
    }

}
```

234. `_250.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

public class _250 {

    public static class Solution1 {
        public int countUnivalSubtrees(TreeNode root) {
            int[] count = new int[1];
            helper(root, count);
            return count[0];
        }

        private boolean helper(TreeNode node, int[] count) {
            if (node == null) {
                return true;
            }
            boolean left = helper(node.left, count);
            boolean right = helper(node.right, count);
            if (left && right) {
                if (node.left != null && node.val != node.left.val) {
                    return false;
                }
                if (node.right != null && node.val != node.right.val) {
                    return false;
                }
                count[0]++;
                return true;
            }
            return false;
        }
    }
}
```

235. `_251.java`

```
package com.fishercoder.solutions;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

public class _251 {

    public static class Solution1 {
        class Vector2D implements Iterator<Integer> {
            private Queue<Integer> cache;
            private List<List<Integer>> vec2d;

            public Vector2D(List<List<Integer>> vec2d) {
                this.vec2d = vec2d;
                this.cache = new LinkedList<Integer>();
                if (vec2d != null && vec2d.size() > 0) {
                    for (List<Integer> list : vec2d) {
                        for (int i : list) {
                            cache.offer(i);
                        }
                    }
                }
            }

            @Override
            public Integer next() {
                return cache.poll();
            }

            @Override
            public boolean hasNext() {
                return !cache.isEmpty();
            }
        }
    }
}
```

236. `_252.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class _252 {
    public static class Solution1 {
        public boolean canAttendMeetings(Interval[] intervals) {

            List<Interval> list = new ArrayList();
            for (Interval interval : intervals) {
                list.add(interval);
            }

            Collections.sort(list, (o1, o2) -> {
                if (o1.start > o2.start) {
                    return 1;
                } else {
                    return -1;
                }
            });

            for (int i = 0; i < list.size() - 1; i++) {
                if (list.get(i).end > list.get(i + 1).start) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

237. `_253.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.Arrays;
import java.util.PriorityQueue;

public class _253 {
    public static class Solution1 {

        public int minMeetingRooms(Interval[] intervals) {
            if (intervals == null || intervals.length == 0) {
                return 0;
            }

            // Sort the intervals by start time
            Arrays.sort(intervals, (a, b) -> a.start - b.start);

            // Use a min heap to track the minimum end time of merged intervals
            PriorityQueue<Interval> heap = new PriorityQueue<>(intervals.length, (a, b) -
> a.end - b.end);

            // start with the first meeting, put it to a meeting room
            heap.offer(intervals[0]);

            for (int i = 1; i < intervals.length; i++) {
                // get the meeting room that finishes earliest
                Interval interval = heap.poll();

                if (intervals[i].start >= interval.end) {
                    // if the current meeting starts right after
                    // there's no need for a new room, merge the interval
                    interval.end = intervals[i].end;
                } else {
                    // otherwise, this meeting needs a new room
                    heap.offer(intervals[i]);
                }

                // don't forget to put the meeting room back
                heap.offer(interval);
            }

            return heap.size();
        }
    }
}
```

238. `_254.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

public class _254 {

    public static class Solution1 {
        public List<List<Integer>> getFactors(int n) {
            List<List<Integer>> result = new ArrayList<>();
            helper(result, new ArrayList<>(), n, 2);
            return result;
        }

        public void helper(List<List<Integer>> result, List<Integer> item, int n, int
start) {
            if (n <= 1) {
                if (item.size() > 1) {
                    result.add(new ArrayList<>(item));
                }
                return;
            }

            for (int i = start; i <= n; ++i) {
                if (n % i == 0) {
                    item.add(i);
                    helper(result, item, n / i, i);
                    item.remove(item.size() - 1);
                }
            }
        }
    }
}
```

239. `_255.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

public class _255 {

    public static class Solution1 {
        public boolean verifyPreorder(int[] preorder) {
            int low = Integer.MIN_VALUE;
            Stack<Integer> stack = new Stack();
            for (int p : preorder) {
                if (p < low) {
                    return false;
                }
                while (!stack.empty() && p > stack.peek()) {
                    low = stack.pop();
                }
                stack.push(p);
            }
            return true;
        }
    }

}
```

240. `_256.java`

```
package com.fishercoder.solutions;

public class _256 {

    public static class Solution1 {
        public int minCost(int[][] costs) {
            if (costs == null || costs.length == 0) {
                return 0;
            }
            for (int i = 1; i < costs.length; i++) {
                costs[i][0] += Math.min(costs[i - 1][1], costs[i - 1][2]);
                costs[i][1] += Math.min(costs[i - 1][0], costs[i - 1][2]);
                costs[i][2] += Math.min(costs[i - 1][1], costs[i - 1][0]);
            }
            int n = costs.length - 1;
            return Math.min(Math.min(costs[n][0], costs[n][1]), costs[n][2]);
        }
    }
}
```

241. `_257.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

public class _257 {
    public static class Solution1 {
        //a very typical/good question to test your recursion/dfs understanding.
        public List<String> binaryTreePaths_more_concise(TreeNode root) {
            List<String> paths = new ArrayList<>();
            if (root == null) {
                return paths;
            }
            dfs(root, paths, "");
            return paths;
        }

        private void dfs(TreeNode root, List<String> paths, String path) {
            if (root.left == null && root.right == null) {
                paths.add(path + root.val);
                return;
            }
            path += root.val + "->";
            if (root.left != null) {
                dfs(root.left, paths, path);
            }
            if (root.right != null) {
                dfs(root.right, paths, path);
            }
        }
    }

    public static class Solution2 {
        public List<String> binaryTreePaths(TreeNode root) {
            List<String> paths = new ArrayList<>();
            dfs(root, paths, new StringBuilder());
            return paths;
        }

        private void dfs(TreeNode root, List<String> paths, StringBuilder sb) {
            if (root == null) {
                return;
            }
            if (root.left == null && root.right == null) {
                sb.append(root.val);
                paths.add(sb.toString());
                return;
            }
            sb.append(root.val + "->");
```

```
            String curr = sb.toString();
            if (root.left != null) {
                dfs(root.left, paths, sb);
            }
            sb.setLength(0);
            sb.append(curr);
            if (root.right != null) {
                dfs(root.right, paths, sb);
            }
        }
    }
}
```

242. `_258.java`

```
package com.fishercoder.solutions;

public class _258 {

    public static class Solution1 {
        //only three cases as the code shows
        public int addDigits(int num) {
            if (num == 0) {
                return 0;
            }
            if (num % 9 == 0) {
                return 9;
            }
            return num % 9;
        }
    }
}
```

243. `_259.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

public class _259 {

    public static class Solution1 {
        /**
         * Basically, very similar to 3Sum, but the key is that you'll have to add result
by (right-left), not just increment result by 1!
         */
        public int threeSumSmaller(int[] nums, int target) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            int result = 0;
            Arrays.sort(nums);
            for (int i = 0; i < nums.length - 2; i++) {
                int left = i + 1;
                int right = nums.length - 1;
                while (left < right) {
                    int sum = nums[i] + nums[left] + nums[right];
                    if (sum < target) {
                        result += right - left;//this line is key!
                        left++;
                    } else {
                        right--;
                    }
                }
            }
            return result;
        }
    }
}
```

244. `_260.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 260. Single Number III
 *
 * Given an array of numbers nums,
 * in which exactly two elements appear only once and all the other elements appear
exactly twice.
 * Find the two elements that appear only once.

For example:

Given nums = [1, 2, 1, 3, 2, 5], return [3, 5].

Note:
The order of the result is not important. So in the above example, [5, 3] is also
correct.
Your algorithm should run in linear runtime complexity. Could you implement it using only
constant space complexity?
*/

public class _260 {

    public static class Solution1 {
        public int[] singleNumber(int[] nums) {
            Map<Integer, Integer> map = new HashMap();
            for (int i : nums) {
                map.put(i, map.getOrDefault(i, 0) + 1);
            }

            int[] res = new int[2];
            int index = 0;
            for (int key : map.keySet()) {
                if (map.get(key) == 1) {
                    res[index++] = key;
                }
                if (index == 2) {
                    break;
                }
            }
            return res;
        }
    }

    public static class Solution2 {
        /**Credit: https://discuss.leetcode.com/topic/21605/accepted-c-java-o-n-time-o-1-
space-easy-solution-with-detail-explanations/2
         *
```

```
         * some more explanation about this algorithm:
         * two's complement: one number's two's complement number is computed as below:
         * reverse all bits of this number and then add one:
         * e.g. decimal number 2, in binary format: 0010 (4 bits)
         * reversing every single bit becomes 1101,
         * then add 1 to it, it becomes 1110
         *
         * so
         * num &= -num, in this case, 2 &= -2 becomes 2
         * */
        public int[] singleNumber(int[] nums) {
            int diff = 0;
            for (int num : nums) {
                diff ^= num;
            }

            //get least significant set bit
            diff &= -diff;

            int[] result = new int[2];
            for (int num : nums) {
                if ((num & diff) == 0) {
                    result[0] ^= num;
                } else {
                    result[1] ^= num;
                }
            }
            return result;
        }
    }
}
```

245. `_261.java`

```
package com.fishercoder.solutions;

/**
 *261. Graph Valid Tree
 *
 * Given n nodes labeled from 0 to n - 1 and a list of undirected edges
 * (each edge is a pair of nodes),
 * write a function to check whether these edges make up a valid tree.

 For example:

 Given n = 5 and edges = [[0, 1], [0, 2], [0, 3], [1, 4]], return true.

 Given n = 5 and edges = [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]], return false.

 Hint:

 Given n = 5 and edges = [[0, 1], [1, 2], [3, 4]], what should your return? Is this case
a valid tree?
 According to the definition of tree on Wikipedia:
 “a tree is an undirected graph in which any two vertices are connected by exactly one
path.
 In other words, any connected graph without simple cycles is a tree.”

 Note: you can assume that no duplicate edges will appear in edges.
 Since all edges are undirected, [0, 1] is the same as [1, 0] and thus will not appear
together in edges.
 */
public class _261 {

    public static class Solution1 {
        public boolean validTree(int n, int[][] edges) {
            int[] nums = new int[n];
            for (int i = 0; i < n; i++) {
                nums[i] = i;
            }

            for (int i = 0; i < edges.length; i++) {
                int x = find(nums, edges[i][0]);
                int y = find(nums, edges[i][1]);

                if (x == y) {
                    return false;
                }

                //union
                nums[x] = y;
            }

            return edges.length == n - 1;
        }
```

```
        int find(int[] nums, int i) {
            if (nums[i] == i) {
                return i;
            }
            return find(nums, nums[i]);
        }
    }
}
```

246. `_263.java`

```
package com.fishercoder.solutions;

public class _263 {

    public static class Solution1 {
        public boolean isUgly(int num) {
            if (num == 0) {
                return false;
            }
            int[] divisors = new int[]{5, 3, 2};
            for (int divisor : divisors) {
                while (num % divisor == 0) {
                    num /= divisor;
                }
            }
            return num == 1;
        }
    }

}
```

247. `_264.java`

```
package com.fishercoder.solutions;

/**
 * 264. Ugly Number II
 *
 * Write a program to find the n-th ugly number.

 Ugly numbers are positive numbers whose prime factors only include 2, 3, 5. For example,
1, 2, 3, 4, 5, 6, 8, 9, 10, 12 is the sequence of the first 10 ugly numbers.

 Note that 1 is typically treated as an ugly number, and n does not exceed 1690.
 */
public class _264 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/21791/o-n-java-solution
         */
        public int nthUglyNumber(int n) {
            int[] ugly = new int[n];
            ugly[0] = 1;
            int index2 = 0;
            int index3 = 0;
            int index5 = 0;
            int factor2 = 2;
            int factor3 = 3;
            int factor5 = 5;
            for (int i = 1; i < n; i++) {
                int min = Math.min(Math.min(factor2, factor3), factor5);
                ugly[i] = min;
                if (factor2 == min) {
                    factor2 = 2 * ugly[++index2];
                }
                if (factor3 == min) {
                    factor3 = 3 * ugly[++index3];
                }
                if (factor5 == min) {
                    factor5 = 5 * ugly[++index5];
                }
            }
            return ugly[n - 1];
        }
    }
}
```

248. `_265.java`

```
package com.fishercoder.solutions;

/**
 * 265. Paint House II
 *
 * There are a row of n houses, each house can be painted with one of the k colors.
 * The cost of painting each house with a certain color is different.
 * You have to paint all the houses such that no two adjacent houses have the same color.
 * The cost of painting each house with a certain color is represented by a n x k cost
matrix.
 *
 * For example, costs[0][0] is the cost of painting house 0 with color 0;
 * costs[1][2] is the cost of painting house 1 with color 2,
 * and so on...
 *
 * Find the minimum cost to paint all houses.

 Note:
 All costs are positive integers.

 Follow up:
 Could you solve it in O(nk) runtime?
 */
public class _265 {

        public static class Solution1 {
                public int minCostII(int[][] costs) {
                        if (costs == null || costs.length == 0) {
                                return 0;
                        }

                        int n = costs.length;
                        int k = costs[0].length;
                        // min1 is the index of the 1st-smallest cost till previous house
                        // min2 is the index of the 2nd-smallest cost till previous house
                        int min1 = -1;
                        int min2 = -1;

                        for (int i = 0; i < n; i++) {
                                int last1 = min1;
                                int last2 = min2;
                                min1 = -1;
                                min2 = -1;

                                for (int j = 0; j < k; j++) {
                                        if (j != last1) {
                                                // current color j is different to last
min1
                                                costs[i][j] += last1 < 0 ? 0 : costs[i -
1][last1];
                                        } else {
```

```
                                                costs[i][j] += last2 < 0 ? 0 : costs[i -
1][last2];
                                        }

                                        // find the indices of 1st and 2nd smallest cost
of painting current house i
                                        if (min1 < 0 || costs[i][j] < costs[i][min1]) {
                                                min2 = min1;
                                                min1 = j;
                                        } else if (min2 < 0 || costs[i][j] < costs[i]
[min2]) {
                                                min2 = j;
                                        }
                                }
                        }
                        return costs[n - 1][min1];
                }
        }

}
```

249. `_266.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

public class _266 {

    public static class Solution1 {
        public boolean canPermutePalindrome(String s) {

            char[] chars = s.toCharArray();
            Map<Character, Integer> map = new HashMap<>();
            for (char c : chars) {
                if (!map.containsKey(c)) {
                    map.put(c, 1);
                } else {
                    map.put(c, map.get(c) + 1);
                }
            }
            int evenCount = 0;
            for (Map.Entry<Character, Integer> e : map.entrySet()) {
                if (e.getValue() % 2 != 0) {
                    evenCount++;
                }
                if (evenCount > 1) {
                    return false;
                }
            }
            return true;
        }
    }

}
```

250. `_267.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class _267 {

    public static class Solution1 {
        public List<String> generatePalindromes(String s) {
            int odd = 0;
            String mid = "";
            List<String> res = new ArrayList();
            List<Character> list = new ArrayList();
            Map<Character, Integer> map = new HashMap();

            // step 1. build character count map and count odds
            for (int i = 0; i < s.length(); i++) {
                char c = s.charAt(i);
                map.put(c, map.containsKey(c) ? map.get(c) + 1 : 1);
                odd += map.get(c) % 2 != 0 ? 1 : -1;
            }

            // cannot form any palindromic string
            if (odd > 1) {
                return res;
            }

            // step 2. add half count of each character to list
            for (Map.Entry<Character, Integer> entry : map.entrySet()) {
                char key = entry.getKey();
                int val = entry.getValue();

                if (val % 2 != 0) {
                    mid += key;
                }

                for (int i = 0; i < val / 2; i++) {
                    list.add(key);
                }
            }

            // step 3. generate all the permutations
            getPerm(list, mid, new boolean[list.size()], new StringBuilder(), res);

            return res;
        }

        // generate all unique permutation from list
        void getPerm(List<Character> list, String mid, boolean[] used, StringBuilder sb,
```

```
                    List<String> res) {
            if (sb.length() == list.size()) {
                // form the palindromic string
                res.add(sb.toString() + mid + sb.reverse().toString());
                sb.reverse();
                return;
            }

            for (int i = 0; i < list.size(); i++) {
                // avoid duplication
                if (i > 0 && list.get(i) == list.get(i - 1) && !used[i - 1]) {
                    continue;
                }

                if (!used[i]) {
                    used[i] = true;
                    sb.append(list.get(i));
                    // recursion
                    getPerm(list, mid, used, sb, res);
                    // backtracking
                    used[i] = false;
                    sb.deleteCharAt(sb.length() - 1);
                }
            }
        }
    }
}
```

251. `_268.java`

```
package com.fishercoder.solutions;

public class _268 {

    public static class Solution1 {
        /**
         * we could take advantage of the array indices
         * then a number xor with itself is zero, so after we xor the entire array with
all of its indices, the missing number will show up.
         */
        public int missingNumber(int[] nums) {
            int xor = 0;
            int i = 0;
            for (; i < nums.length; i++) {
                xor = xor ^ i ^ nums[i];
            }
            return xor ^ i;
        }
    }

}
```

252. `_269.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

/**
 * 269. Alien Dictionary
 *
 * There is a new alien language which uses the latin alphabet.
 * However, the order among letters are unknown to you.
 * You receive a list of words from the dictionary, where words are sorted
lexicographically by the rules of this new language.
 * Derive the order of letters in this language.

 For example,
 Given the following words in dictionary,

 [
 "wrt",
 "wrf",
 "er",
 "ett",
 "rftt"
 ]
 The correct order is: "wertf".

 Note:
 You may assume all letters are in lowercase.
 If the order is invalid, return an empty string.
 There may be multiple valid order of letters, return any one of them is fine.
 */
public class _269 {
    public static class Solution1 {

        /**
         * reference: https://discuss.leetcode.com/topic/28308/java-ac-solution-using-bfs
         */
        public String alienOrder(String[] words) {
            Map<Character, Set<Character>> map = new HashMap();
            Map<Character, Integer> degree = new HashMap<>();
            String result = "";
            if (words == null || words.length == 0) {
                return result;
            }
            for (String s : words) {
                for (char c : s.toCharArray()) {
                    degree.put(c, 0);//keeps overwriting it, the purpose is to create one
```

```
entry
                    //for each letter in the degree map
                }
            }
            for (int i = 0; i < words.length - 1; i++) {
                String cur = words[i];
                String next = words[i + 1];
                int length = Math.min(cur.length(), next.length());
                for (int j = 0; j < length; j++) {
                    char c1 = cur.charAt(j);
                    char c2 = next.charAt(j);
                    if (c1 != c2) {
                        Set<Character> set = new HashSet<>();
                        if (map.containsKey(c1)) {
                            set = map.get(c1);
                        }
                        if (!set.contains(c2)) {
                            set.add(c2);
                            map.put(c1, set);
                            degree.put(c2, degree.get(c2) + 1);
                        }
                        break;
                    }
                }
            }
            Queue<Character> queue = new LinkedList<>();
            for (char c : degree.keySet()) {
                if (degree.get(c) == 0) {
                    queue.add(c);
                }
            }
            while (!queue.isEmpty()) {
                char c = queue.remove();
                result += c;
                if (map.containsKey(c)) {
                    for (char c2 : map.get(c)) {
                        degree.put(c2, degree.get(c2) - 1);
                        if (degree.get(c2) == 0) {
                            queue.add(c2);
                        }
                    }
                }
            }
            if (result.length() != degree.size()) {
                return "";
            }
            return result;
        }
    }

}
```

253. `_270.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

public class _270 {

    public static class Solution1 {
        //A general tree solution, this finished in 1 ms
        public int closestValue(TreeNode root, double target) {
            if (root == null) {
                return 0;
            }
            double delta = Double.MAX_VALUE;
            return dfs(root, target, delta, root.val);
        }

        private int dfs(TreeNode root, double target, double delta, int closestVal) {
            if (Math.abs(root.val - target) < delta) {
                closestVal = root.val;
                delta = Math.abs(root.val - target);
            }
            int leftVal = closestVal;
            if (root.left != null) {
                leftVal = dfs(root.left, target, delta, closestVal);
            }
            int rightVal = closestVal;
            if (root.right != null) {
                rightVal = dfs(root.right, target, delta, closestVal);
            }
            return (Math.abs(leftVal - target) > Math.abs(rightVal - target)) ? rightVal
: leftVal;
        }
    }

    public static class Solution2 {
        // BST solution
        // we can tailor the solution to use the BST feature: left subtrees are always
smaller than the root the right subtrees
        //this finished in 0 ms
        public int closestValue(TreeNode root, double target) {
            if (root == null) {
                return 0;
            }
            return dfs(root, target, root.val);
        }

        private int dfs(TreeNode root, double target, int minVal) {
            if (root == null) {
                return minVal;
            }
            if (Math.abs(root.val - target) < Math.abs(minVal - target)) {
```

```
                minVal = root.val;
            }
            if (target < root.val) {
                minVal = dfs(root.left, target, minVal);
            } else {
                minVal = dfs(root.right, target, minVal);
            }
            return minVal;
        }
    }

    public static class Solution3 {
        //a more concise solution
        public int closestValue(TreeNode root, double target) {
            if (root == null) {
                return 0;
            }
            return dfs(root, target, root.val);
        }

        private int dfs(TreeNode root, double target, int minVal) {
            if (root == null) {
                return minVal;
            }
            if (Math.abs(root.val - target) < Math.abs(minVal - target)) {
                minVal = root.val;
            }
            minVal = dfs(root.left, target, minVal);
            minVal = dfs(root.right, target, minVal);
            return minVal;
        }
    }

    public static class Solution4 {
        //BST iterative solution
        public int closestValue(TreeNode root, double target) {
            long minVal = Long.MAX_VALUE;
            while (root != null) {
                if (Math.abs(root.val - target) < Math.abs(minVal - target)) {
                    minVal = root.val;
                }
                if (target < root.val) {
                    root = root.left;
                } else {
                    root = root.right;
                }
            }
            return minVal == Long.MAX_VALUE ? 0 : (int) minVal;
        }
    }
}
```

254. `_271.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 271. Encode and Decode Strings
 *
 * Design an algorithm to encode a list of strings to a string.
 * The encoded string is then sent over the network and is decoded back to the original
list of strings.

 Machine 1 (sender) has the function:

 string encode(vector<string> strs) {
 // ... your code
 return encoded_string;
 }
 Machine 2 (receiver) has the function:
 vector<string> decode(string s) {
 //... your code
 return strs;
 }
 So Machine 1 does:

 string encoded_string = encode(strs);
 and Machine 2 does:

 vector<string> strs2 = decode(encoded_string);
 strs2 in Machine 2 should be the same as strs in Machine 1.

 Implement the encode and decode methods.

 Note:
 The string may contain any possible characters out of 256 valid ascii characters. Your
algorithm should be generalized enough to work on any possible characters.
 Do not use class member/global/static variables to store states. Your encode and decode
algorithms should be stateless.
 Do not rely on any library method such as eval or serialize methods. You should
implement your own encode/decode algorithm.
 */
public class _271 {
    public static class Solution1 {
        // Encodes a list of strings to a single string.
        public String encode(List<String> strs) {
            StringBuilder sb = new StringBuilder();
            for (String s : strs) {
                sb.append(s.length()).append('/').append(s);
            }
            return sb.toString();
        }
```

```
        // Decodes a single string to a list of strings.
        public List<String> decode(String s) {
            List<String> result = new ArrayList<>();
            int i = 0;
            while (i < s.length()) {
                int slash = s.indexOf('/', i);
                int size = Integer.valueOf(s.substring(i, slash));
                result.add(s.substring(slash + 1, slash + 1 + size));
                i = slash + size + 1;
            }
            return result;
        }
    }
}
```

255. `_272.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

/**
 * 272. Closest Binary Search Tree Value II
 *
 * Given a non-empty binary search tree and a target value, find k values in the BST that
are closest to the target.

 Note:
 Given target value is a floating point.
 You may assume k is always valid, that is: k ≤ total nodes.
 You are guaranteed to have only one unique set of k values in the BST that are closest
to the target.
 Follow up:
 Assume that the BST is balanced, could you solve it in less than O(n) runtime (where n =
total nodes)?

 Hint:

 Consider implement these two helper functions:
 getPredecessor(N), which returns the next smaller node to N.
 getSuccessor(N), which returns the next larger node to N.
 Try to assume that each node has a parent pointer, it makes the problem much easier.
 Without parent pointer we just need to keep track of the path from the root to the
current node using a stack.
 You would need two stacks to track the path in finding predecessor and successor node
separately.

 */
public class _272 {

    public static class Solution1 {
        public List<Integer> closestKValues(TreeNode root, double target, int k) {
            List<Integer> res = new ArrayList();

            Stack<Integer> s1 = new Stack(); // predecessors
            Stack<Integer> s2 = new Stack(); // successors

            inorder(root, target, false, s1);
            inorder(root, target, true, s2);

            while (k-- > 0) {
                if (s1.isEmpty()) {
                    res.add(s2.pop());
                } else if (s2.isEmpty()) {
```

```
                    res.add(s1.pop());
                } else if (Math.abs(s1.peek() - target) < Math.abs(s2.peek() - target)) {
                    res.add(s1.pop());
                } else {
                    res.add(s2.pop());
                }
            }

            return res;
        }

        // inorder traversal
        void inorder(TreeNode root, double target, boolean reverse, Stack<Integer> stack)
{
            if (root == null) {
                return;
            }

            inorder(reverse ? root.right : root.left, target, reverse, stack);
            // early terminate, no need to traverse the whole tree
            if ((reverse && root.val <= target) || (!reverse && root.val > target)) {
                return;
            }
            // track the value of current node
            stack.push(root.val);
            inorder(reverse ? root.left : root.right, target, reverse, stack);
        }
    }
}
```

256. `_273.java`

```
package com.fishercoder.solutions;

/**
 * 273. Integer to English Words
 *
 * Convert a non-negative integer to its english words representation. Given input is
guaranteed to be less than 231 - 1.

 For example,
 123 -> "One Hundred Twenty Three"
 12345 -> "Twelve Thousand Three Hundred Forty Five"
 1234567 -> "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"

 Hint:
 Did you see a pattern in dividing the number into chunk of words? For example, 123 and
123000.
 Group the number by thousands (3 digits). You can write a helper function that takes a
number less than 1000 and convert just that chunk to words.
 There are many edge cases. What are some good test cases? Does your code work with input
such as 0? Or 1000010? (middle chunk is zero and should not be printed out)
 */
public class _273 {

    public static class Solution1 {
        private String[] belowTen = new String[]{"Zero", "One", "Two", "Three", "Four",
"Five", "Six", "Seven", "Eight", "Nine"};
        private String[] belowTwenty = new String[]{"Ten", "Eleven", "Twelve",
"Thirteen", "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"};
        private String[] belowHundred = new String[]{"Twenty", "Thirty", "Forty",
"Fifty", "Sixty", "Seventy", "Eighty", "Ninety"};
        private String[] overThousand = new String[]{"Thousand", "Million", "Billion"};

        public String numberToWords(int num) {
            String result;
            if (num == 0) {
                return belowTen[num];
            }

            result = hundredHelper(num % 1000);
            num = num / 1000;
            int i = 0;
            while (i < 3 && num > 0) {
                if (num % 1000 > 0) {
                    result = hundredHelper(num % 1000) + overThousand[i] + " " + result;
                }
                num = num / 1000;
                i++;
            }

            return result.trim();
        }
```

```
        private String hundredHelper(int num) {
            String nstr = "";
            if (num >= 100) {
                nstr = belowTen[num / 100] + " Hundred ";
            }
            num = num % 100;
            if (num >= 20) {
                if (num % 10 != 0) {
                    nstr = nstr + belowHundred[num / 10 - 2] + " " + belowTen[num % 10] +
" ";
                } else {
                    nstr = nstr + belowHundred[num / 10 - 2] + " ";
                }
            } else if (num >= 10) {
                nstr = nstr + belowTwenty[num % 10] + " ";
            } else if (num > 0) {
                nstr = nstr + belowTen[num] + " ";
            }
            return nstr;
        }
    }
}
```

257. `_274.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 274. H-Index
 *
 * Given an array of citations (each citation is a non-negative integer) of a researcher, write a function to compute the researcher's h-index.
 * According to the definition of h-index on Wikipedia: "A scientist has index h if h of his/her N papers have at least h citations each, and the other N − h papers have no more than h citations each."
 * For example, given citations = [3, 0, 6, 1, 5], which means the researcher has 5 papers in total and each of them had received 3, 0, 6, 1, 5 citations respectively.
 * Since the researcher has 3 papers with at least 3 citations each and the remaining two with no more than 3 citations each, his h-index is 3.

 Note: If there are several possible values for h, the maximum one is taken as the h-index.

 Hint:

 An easy approach is to sort the array first.
 What are the possible values of h-index?
 A faster approach is to use extra space.
 */
public class _274 {
    public static class Solution1 {
        public int hIndex(int[] citations) {
            if (citations == null || citations.length == 0) {
                return 0;
            }

            Arrays.sort(citations);
            for (int i = 0; i < citations.length; i++) {
                if (citations[i] >= citations.length - i) {
                    return citations.length - i;
                }
            }
            return 0;
        }
    }
}
```

258. `_275.java`

```
package com.fishercoder.solutions;

/**
 * 275. H-Index II
 *
 * Follow up for H-Index: What if the citations array is sorted in ascending order?
 * Could you optimize your algorithm?
 */
public class _275 {
    public static class Solution1 {
        public int hIndex(int[] citations) {
            int left = 0;
            int len = citations.length;
            int right = len - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;
                if (citations[mid] >= (len - mid)) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }
            return len - left;
        }
    }
}
```

259. `_276.java`

```
package com.fishercoder.solutions;

/**
 * 276. Paint Fence

 There is a fence with n posts, each post can be painted with one of the k colors.

 You have to paint all the posts such that no more than two adjacent fence posts have the same color.

 Return the total number of ways you can paint the fence.

 Note:
 n and k are non-negative integers.*/

public class _276 {
    public static class Solution1 {
        public int numWays(int n, int k) {
            if (n == 0) {
                return 0;
            } else if (n == 1) {
                return k;
            }
            int sameColorCnt = k;
            int diffColorCnt = k * (k - 1);
            for (int i = 2; i < n; i++) {
                int temp = diffColorCnt;
                diffColorCnt = (diffColorCnt + sameColorCnt) * (k - 1);
                sameColorCnt = temp;
            }
            return sameColorCnt + diffColorCnt;
        }
    }
}
```

260. `_277.java`

```
package com.fishercoder.solutions;

/**
 * 277. Find the Celebrity
 *
 * Suppose you are at a party with n people (labeled from 0 to n - 1) and among them, there may exist one celebrity.
 * The definition of a celebrity is that all the other n - 1 people know him/her but he/she does not know any of them.
 * Now you want to find out who the celebrity is or verify that there is not one.
 * The only thing you are allowed to do is to ask questions like: "Hi, A. Do you know B?" to get information of whether A knows B.
 * You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).
 * You are given a helper function bool knows(a, b) which tells you whether A knows B. Implement a function int findCelebrity(n), your function should minimize the number of calls to knows.
 *
 * Note: There will be exactly one celebrity if he/she is in the party. Return the celebrity's label if there is a celebrity in the party. If there is no celebrity, return -1.
 */
public class _277 {

    public static class Solution1 {
        public int findCelebrity(int n) {
            int candidate = 0;
            for (int i = 1; i < n; i++) {
                if (knows(candidate, i)) {
                    candidate = i;
                }
            }
            for (int i = 0; i < n; i++) {
                if (i != candidate && (knows(candidate, i) || !knows(i, candidate))) {
                    return -1;
                }
            }
            return candidate;
        }

        //this is a mock-up method to make IDE happy.s
        private boolean knows(int i, int candidate) {
            return false;
        }
    }
}
```

261. `_278.java`

```
package com.fishercoder.solutions;

/**
 * 278. First Bad Version
 *
 * You are a product manager and currently leading a team to develop a new product.
 * Unfortunately, the latest version of your product fails the quality check.
 * Since each version is developed based on the previous version, all the versions after
a bad version are also bad.
 * Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one,
 * which causes all the following ones to be bad.
 *
 * You are given an API bool isBadVersion(version) which will return whether version is
bad.
 * Implement a function to find the first bad version. You should minimize the number of
calls to the API.*/
public class _278 {

        public static class Solution1 {
                public int firstBadVersion(int n) {
                        int left = 1;
                        int right = n;
                        if (isBadVersion(left)) {
                                return left;
                        }

                        while (left + 1 < right) {
                                int mid = left + (right - left) / 2;
                                if (isBadVersion(mid)) {
                                        right = mid;
                                } else {
                                        left = mid;
                                }
                        }

                        if (isBadVersion(left)) {
                                return left;
                        }
                        return right;
                }

                private boolean isBadVersion(int left) {
                        //this is a fake method to make Eclipse happy
                        return false;
                }
        }
}
```

262. `_279.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 279. Perfect Squares
 *
 * Given a positive integer n, find the least number of perfect square numbers (for
example, 1, 4, 9, 16, ...) which sum to n.
 * For example, given n = 12, return 3 because 12 = 4 + 4 + 4; given n = 13, return 2
because 13 = 4 + 9.
 */
public class _279 {
    public static class Solution1 {
        public int numSquares(int n) {
            int result = n;
            int num = 2;
            while (num * num <= n) {
                int temp1 = n / (num * num);
                int temp2 = n % (num * num);
                result = Math.min(result, temp1 + numSquares(temp2));
                num++;
            }
            return result;
        }
    }

    public static class Solution2 {
        //DP solution
        public int numSquares(int n) {
            int[] dp = new int[n + 1];
            Arrays.fill(dp, Integer.MAX_VALUE);
            dp[0] = 0;
            dp[1] = 1;
            for (int i = 1; i <= n; i++) {
                int min = Integer.MAX_VALUE;
                int j = 1;
                while (i - j * j >= 0) {
                    min = Math.min(min, dp[i - j * j] + 1);
                    j++;
                }
                dp[i] = min;
            }
            return dp[n];
        }
    }

}
```

263. `_280.java`

```
package com.fishercoder.solutions;

/**
 * Given an unsorted array nums, reorder it in-place such that nums[0] <= nums[1] >=
nums[2] <= nums[3]....
For example, given nums = [3, 5, 2, 1, 6, 4], one possible answer is [1, 6, 2, 5, 3, 4]*/
public class _280 {
    public static class Solution1 {
        public void wiggleSort(int[] nums) {
            for (int i = 1; i < nums.length; i++) {
                if ((i % 2 == 0 && nums[i] > nums[i - 1]) || (i % 2 == 1 && nums[i] <
nums[i - 1])) {
                    swap(nums, i);
                }
            }
        }

        void swap(int[] nums, int i) {
            int temp = nums[i - 1];
            nums[i - 1] = nums[i];
            nums[i] = temp;
        }
    }
}
```

264. `_281.java`

```
package com.fishercoder.solutions;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 281. Zigzag Iterator
 *
 * Given two 1d vectors, implement an iterator to return their elements alternately.

 For example, given two 1d vectors:

 v1 = [1, 2]
 v2 = [3, 4, 5, 6]
 By calling next repeatedly until hasNext returns false, the order of elements returned
by next should be: [1, 3, 2, 4, 5, 6].

 Follow up: What if you are given k 1d vectors? How well can your code be extended to
such cases?

 Clarification for the follow up question - Update (2015-09-18):
 The "Zigzag" order is not clearly defined and is ambiguous for k > 2 cases.
 If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic". For example,
given the following input:

 [1,2,3]
 [4,5,6,7]
 [8,9]
 It should return [1,4,8,2,5,9,3,6,7].
 */
public class _281 {

    public static class Solution1 {
        public static class ZigzagIterator {
            private Iterator<Integer> i;
            private Iterator<Integer> j;
            private Iterator<Integer> tmp;

            public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
                i = v2.iterator();
                j = v1.iterator();
            }

            public int next() {
                if (j.hasNext()) {
                    tmp = j;
                    j = i;
                    i = tmp;
                }
```

```
                return i.next();
            }

            public boolean hasNext() {
                return i.hasNext() || j.hasNext();
            }
        }
    }

    public static class Solution2 {
        public static class ZigzagIterator {

            Queue<Iterator<Integer>> queue;

            public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
                queue = new LinkedList<>();
                if (v1 != null && !v1.isEmpty()) {
                    Iterator<Integer> iterator1 = v1.iterator();
                    queue.offer(iterator1);
                }
                if (v2 != null && !v2.isEmpty()) {
                    Iterator<Integer> iterator2 = v2.iterator();
                    queue.offer(iterator2);
                }
            }

            public boolean hasNext() {
                return !queue.isEmpty();
            }

            public int next() {
                Iterator<Integer> iterator = queue.poll();
                int next = iterator.next();
                if (iterator.hasNext()) {
                    queue.offer(iterator);
                }
                return next;
            }
        }
    }
}
```

265. `_282.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 282. Expression Add Operators
 *
 * Given a string that contains only digits 0-9 and a target value,
 * return all possibilities to add binary operators (not unary) +, -, or * between the
digits
 * so they evaluate to the target value.

 Examples:
 "123", 6 -> ["1+2+3", "1*2*3"]
 "232", 8 -> ["2*3+2", "2+3*2"]
 "105", 5 -> ["1*0+5","10-5"]
 "00", 0 -> ["0+0", "0-0", "0*0"]
 "3456237490", 9191 -> []
 */
public class _282 {

    public static class Solution1 {
        public List<String> addOperators(String num, int target) {
            List<String> res = new ArrayList<String>();
            StringBuilder sb = new StringBuilder();
            dfs(res, sb, num, 0, target, 0, 0);
            return res;

        }

        private void dfs(List<String> res, StringBuilder sb, String num, int pos, int
target, long prev, long multi) {
            if (pos == num.length()) {
                if (target == prev) {
                    res.add(sb.toString());
                }
                return;
            }
            for (int i = pos; i < num.length(); i++) {
                if (num.charAt(pos) == '0' && i != pos) {
                    break;
                }
                long curr = Long.parseLong(num.substring(pos, i + 1));
                int len = sb.length();
                if (pos == 0) {
                    dfs(res, sb.append(curr), num, i + 1, target, curr, curr);
                    sb.setLength(len);
                } else {
                    dfs(res, sb.append("+").append(curr), num, i + 1, target, prev +
curr, curr);
```

```
                    sb.setLength(len);

                    dfs(res, sb.append("-").append(curr), num, i + 1, target, prev -
curr, -curr);
                    sb.setLength(len);

                    dfs(res, sb.append("*").append(curr), num, i + 1, target, prev -
multi + multi * curr, multi * curr);
                    sb.setLength(len);
                }
            }
        }
    }

}
```

266. `_283.java`

```
package com.fishercoder.solutions;

/**
 * 283. Move Zeroes
 *
 * Given an array nums, write a function to move all 0's to the end of it while
maintaining the relative order of the non-zero elements.
 *
 * For example, given nums = [0, 1, 0, 3, 12], after calling your function, nums should
be [1, 3, 12, 0, 0].
 *
 * Note:
 * You must do this in-place without making a copy of the array.
 * Minimize the total number of operations.*/
public class _283 {
    public static class Solution1 {
        public void moveZeroes(int[] nums) {
            //keep the last non-zero index and keep overwriting it, then append zeroes to
fill the end
            int j = 0;
            int i = 0;
            for (; j < nums.length; j++) {
                if (nums[j] != 0) {
                    nums[i++] = nums[j];
                }
            }
            for (; i < nums.length; i++) {
                nums[i] = 0;
            }
        }
    }

    public static class Solution2 {
        public void moveZeroes(int[] nums) {
            //this solutoin is the most optimal since it minimizes the number of
operations
            //the idea is to swap the non-zero element to the first zero number position
            for (int i = 0, j = 0; i < nums.length && j < nums.length; j++) {
                if (nums[j] != 0) {
                    int temp = nums[i];
                    nums[i++] = nums[j];
                    nums[j] = temp;
                }
            }
        }
    }

    //then I came up with this solution and got it AC'ed! Cheers!
    //basically, find the next non-zero number and swap it with the current zero number
    //Apparently it's not the most optimal, since this is basically an O(n^2) solution,
then I turned to Editorial solutions
```

```
    public static class Solution3 {
        public void moveZeroes(int[] nums) {
            for (int i = 0; i < nums.length - 1; i++) {
                if (nums[i] == 0) {
                    int j = i + 1;
                    while (j < nums.length && nums[j] == 0) {
                        j++;
                    }
                    if (j >= nums.length) {
                        return;
                    } else {
                        int temp = nums[j];
                        nums[j] = nums[i];
                        nums[i] = temp;
                    }
                }
            }
        }
    }

}
```

267. `_284.java`

```
package com.fishercoder.solutions;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

/**
 * 284. Peeking Iterator
 *
 * Given an Iterator class interface with methods: next() and hasNext(),
 * design and implement a PeekingIterator that support
 * the peek() operation -- it essentially peek() at the element that will be returned by
the next call to next().

 Here is an example. Assume that the iterator is initialized to the beginning of the
queue: [1, 2, 3].

 Call next() gets you 1, the first element in the queue.

 Now you call peek() and it returns 2, the next element. Calling next() after that still
return 2.

 You call next() the final time and it returns 3, the last element. Calling hasNext()
after that should return false.

 Follow up: How would you extend your design to be generic and work with all types, not
just integer?
 */
public class _284 {
    public static class Solution1 {
        public static class PeekingIterator implements Iterator<Integer> {

            private Queue<Integer> queue;

            public PeekingIterator(Iterator<Integer> iterator) {
                // initialize any member here.
                queue = new LinkedList<>();
                while (iterator.hasNext()) {
                    queue.add(iterator.next());
                }
            }

            // Returns the next element in the iteration without advancing the iterator.
            public Integer peek() {
                return queue.peek();
            }

            // hasNext() and next() should behave the same as in the Iterator interface.
            // Override them if needed.
            @Override
            public Integer next() {
```

```
                return queue.poll();
            }

            @Override
            public boolean hasNext() {
                return !queue.isEmpty();
            }
        }
    }
}
```

268. `_285.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.Iterator;
import java.util.Map;
import java.util.TreeMap;

/**285. Inorder Successor in BST

Given a binary search tree and a node in it, find the in-order successor of that node in
the BST.

Note: If the given node has no in-order successor in the tree, return null. */
public class _285 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/25698/java-python-solution-o-h-
time-and-o-1-space-iterative
         * The inorder traversal of a BST is the nodes in ascending order.
         * To find a successor, you just need to find the smallest one that is larger
than the given value since there are no duplicate values in a BST.
         * It's just like the binary search in a sorted list.
         * <p>
         * The time complexity should be O(h) where h is the depth of the result node.
         * succ is a pointer that keeps the possible successor.
         * Whenever you go left the current root is the new possible successor, otherwise
the it remains the same.
         * <p>
         * Only in a balanced BST O(h) = O(log n). In the worst case h can be as large as
n.
         */
        public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
            TreeNode successor = null;
            while (root != null) {
                if (p.val < root.val) {
                    successor = root;
                    root = root.left;
                } else {
                    root = root.right;
                }
            }
            return successor;
        }
    }

    public static class Solution2 {
        public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
            TreeMap<Integer, TreeNode> map = new TreeMap<>();
            inorderTraversal(root, map);
```

```
            Iterator<Map.Entry<Integer, TreeNode>> iterator = map.entrySet().iterator();
            while (iterator.hasNext()) {
                Map.Entry<Integer, TreeNode> entry = iterator.next();
                if (entry.getValue() == p) {
                    if (iterator.hasNext()) {
                        return iterator.next().getValue();
                    } else {
                        return null;
                    }
                }
            }
            return null;
        }

        private void inorderTraversal(TreeNode root, TreeMap<Integer, TreeNode> map) {
            if (root == null) {
                return;
            }
            inorderTraversal(root.left, map);
            map.put(root.val, root);
            inorderTraversal(root.right, map);
            return;
        }
    }

}
```

269. `_286.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * You are given a m x n 2D grid initialized with these three possible values.

 -1 - A wall or an obstacle.
 0 - A gate.
 INF - Infinity means an empty room. We use the value 231 - 1 = 2147483647 to represent
INF as you may assume that the distance to a gate is less than 2147483647.
 Fill each empty room with the distance to its nearest gate. If it is impossible to reach
a gate, it should be filled with INF.

 For example, given the 2D grid:
 INF  -1  0  INF
 INF INF INF  -1
 INF  -1 INF  -1
 0  -1 INF INF

 After running your function, the 2D grid should be:
 3  -1   0   1
 2   2   1  -1
 1  -1   2  -1
 0  -1   3   4

 */
public class _286 {
    public static class Solution1 {

        int[] dirs = new int[]{0, 1, 0, -1, 0};

        public void wallsAndGates(int[][] rooms) {
            if (rooms == null || rooms.length == 0 || rooms[0].length == 0) {
                return;
            }
            int m = rooms.length;
            int n = rooms[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (rooms[i][j] == 0) {
                        bfs(rooms, i, j, m, n);
                    }
                }
            }
        }

        void bfs(int[][] rooms, int i, int j, int m, int n) {
            for (int k = 0; k < 4; k++) {
                int x = dirs[k] + i;
```

```
                int y = dirs[k + 1] + j;
                if (x >= 0 && y >= 0 && x < m && y < n && rooms[x][y] > rooms[i][j] + 1)
{
                    rooms[x][y] = rooms[i][j] + 1;
                    bfs(rooms, x, y, m, n);
                }
            }
        }

    }

    public static class Solution2 {

        //push all gates into the queue first, and then put all its neighbours into the
queue with one distance to the gate, then continue to push the rest of the nodes into the
queue, and put all their neighbours into the queue with the nodes' value plus one until
the queue is empty
        int[] dirs = new int[]{0, 1, 0, -1, 0};

        public void wallsAndGates(int[][] rooms) {
            if (rooms == null || rooms.length == 0 || rooms[0].length == 0) {
                return;
            }
            int m = rooms.length;
            int n = rooms[0].length;
            Queue<int[]> queue = new LinkedList();
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (rooms[i][j] == 0) {
                        queue.offer(new int[]{i, j});
                    }
                }
            }

            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                for (int k = 0; k < 4; k++) {
                    int x = curr[0] + dirs[k];
                    int y = curr[1] + dirs[k + 1];
                    if (x >= 0 && x < m && y >= 0 && y < n && rooms[x][y] ==
Integer.MAX_VALUE) {
                        rooms[x][y] = rooms[curr[0]][curr[1]] + 1;
                        queue.offer(new int[]{x, y});
                    }
                }
            }
        }

    }
}
```

270. `_287.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 287. Find the Duplicate Number
 *
 * Given an array nums containing n + 1 integers where each integer is between 1 and n
(inclusive),
 * prove that at least one duplicate number must exist. Assume that there is only one
duplicate number, find the duplicate one.
 *
 Note:
 You must not modify the array (assume the array is read only).
 You must use only constant, O(1) extra space.
 Your runtime complexity should be less than O(n2).
 There is only one duplicate number in the array, but it could be repeated more than
once.

 */
public class _287 {

    public static class Solution1 {
        /**no-brainer, used O(n) space*/
        public int findDuplicate(int[] nums) {
            Set<Integer> set = new HashSet<>();
            int dup = 0;
            for (int i = 0; i < nums.length; i++) {
                if (!set.add(nums[i])) {
                    dup = nums[i];
                    break;
                }
            }
            return dup;
        }
    }

    public static class Solution2 {
        /** O(1) space */
        public int findDuplicate(int[] nums) {
            int slow = 0;
            int fast = 0;
            int finder = 0;
            while (true) {
                slow = nums[slow];
                fast = nums[nums[fast]];

                if (slow == fast) {
                    break;
                }
```

```
            }

            while (true) {
                slow = nums[slow];
                finder = nums[finder];
                if (slow == finder) {
                    return slow;
                }
            }
        }
    }
}
```

271. `_288.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;

/**An abbreviation of a word follows the form <first letter><number><last letter>. Below
are some examples of word abbreviations:

 a) it                      --> it    (no abbreviation)

 1
 b) d|o|g                   --> d1g

 1    1  1
 1---5----0----5--8
 c) i|nternationalizatio|n  --> i18n

 1
 1---5----0
 d) l|ocalizatio|n          --> l10n
 Assume you have a dictionary and given a word, find whether its abbreviation is unique
in the dictionary. A word's abbreviation is unique if no other word from the dictionary
has the same abbreviation.

 Example:
 Given dictionary = [ "deer", "door", "cake", "card" ]

 isUnique("dear") ->
 false

 isUnique("cart") ->
 true

 isUnique("cane") ->
 false

 isUnique("make") ->
 true
 */
public class _288 {
    public static class Solution1 {

        public class ValidWordAbbr {
            private Map<String, String> dict;

            public ValidWordAbbr(String[] dictionary) {
                dict = new HashMap();
                for (String word : dictionary) {
```

```
                    String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                    if (dict.containsKey(key) && !dict.get(key).equals(word)) {
                        dict.put(key, "");
                    } else {
                        dict.put(key, word);
                    }
                }
            }

            public boolean isUnique(String word) {
                String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                if (!dict.containsKey(key)) {
                    return true;
                } else {
                    return dict.get(key) != "" && dict.get(key).equals(word);
                }
            }
        }
    }

    public static class Solution2 {
        public class ValidWordAbbr {

            private Map<String, Set<String>> dict;

            public ValidWordAbbr(String[] dictionary) {
                dict = new HashMap();
                for (String word : dictionary) {
                    String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                    if (dict.containsKey(key)) {
                        Set<String> set = dict.get(key);
                        set.add(word);
                        dict.put(key, set);
                    } else {
                        Set<String> set = new HashSet();
                        set.add(word);
                        dict.put(key, set);
                    }
                }
            }

            public boolean isUnique(String word) {
                String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                if (!dict.containsKey(key)) {
                    return true;
                } else {
                    Set<String> set = dict.get(key);
                    if (set.size() != 1) {
                        return false;
                    }
```

```
                    Iterator<String> it = set.iterator();
                    return it.next().equals(word);
                }
            }
        }
    }
}
```

272. `_289.java`

```
package com.fishercoder.solutions;

/**
 * 289. Game of Life
 *
 * According to the Wikipedia's article: "The Game of Life, also known simply as Life,
 * is a cellular automaton devised by the British mathematician John Horton Conway in
1970."

 Given a board with m by n cells, each cell has an initial state live (1) or dead (0).
 Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the
 following four rules (taken from the above Wikipedia article):

 Any live cell with fewer than two live neighbors dies, as if caused by under-population.
 Any live cell with two or three live neighbors lives on to the next generation.
 Any live cell with more than three live neighbors dies, as if by over-population..
 Any dead cell with exactly three live neighbors becomes a live cell, as if by
reproduction.
 Write a function to compute the next state (after one update) of the board given its
current state.

 Follow up:
 Could you solve it in-place? Remember that the board needs to be updated at the same
time:
 You cannot update some cells first and then use their updated values to update other
cells.
 In this question, we represent the board using a 2D array.
 In principle, the board is infinite, which would cause problems when the active area
encroaches the
 border of the array. How would you address these problems?*/

public class _289 {
    public static class Solution1 {
        public void gameOfLife(int[][] board) {
            int height = board.length;
            int width = board[0].length;
            int[][] next = new int[height][width];
            int[][] directions = {{-1, -1}, {-1, 0}, {-1, 1}, {0, 1}, {1, 1}, {1, 0}, {1,
-1}, {0, -1}};

            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    int liveCellsCount = 0;
                    //count all its live cells

                    for (int[] dir : directions) {
                        int x = i + dir[0];
                        int y = j + dir[1];
                        if (x >= 0 && y >= 0 && x < height && y < width && board[x][y] ==
1) {
                            liveCellsCount++;
```

```
                    }
                }

                if (board[i][j] == 1) {
                    if (liveCellsCount <= 3 && liveCellsCount >= 2) {
                        next[i][j] = 1;
                    }
                } else if (board[i][j] == 0) {
                    if (liveCellsCount == 3) {
                        next[i][j] = 1;
                    }
                }
            }
        }

        for (int i = 0; i < board.length; i++) {
            for (int j = 0; j < board[0].length; j++) {
                board[i][j] = next[i][j];
            }
        }
    }
}
```

273. `_290.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 290. Word Pattern
 *
 * Given a pattern and a string str, find if str follows the same pattern.
 * Here follow means a full match, such that there is a bijection between a letter in
pattern and a non-empty word in str.

 Examples:
 pattern = "abba", str = "dog cat cat dog" should return true.
 pattern = "abba", str = "dog cat cat fish" should return false.
 pattern = "aaaa", str = "dog cat cat dog" should return false.
 pattern = "abba", str = "dog dog dog dog" should return false.

 Notes:
 You may assume pattern contains only lowercase letters, and str contains lowercase
letters separated by a single space.
 */

public class _290 {

    public static class Solution1 {
        public boolean wordPattern(String pattern, String str) {
            String[] words = str.split(" ");
            char[] patterns = pattern.toCharArray();
            Map<Character, String> map = new HashMap();
            if (patterns.length != words.length) {
                return false;
            }
            for (int i = 0; i < patterns.length; i++) {
                if (map.containsKey(patterns[i])) {
                    if (!map.get(patterns[i]).equals(words[i])) {
                        return false;
                    }
                } else {
                    if (map.containsValue(words[i])) {
                        return false;//this is for this case: "abba", "dog dog dog dog"
                    }
                    map.put(patterns[i], words[i]);
                }
            }
            return true;
        }
    }

}
```

274. `_291.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 291. Word Pattern II
 *
 * Given a pattern and a string str, find if str follows the same pattern.
 * Here follow means a full match, such that there is a bijection between a letter in
pattern and a non-empty substring in str.

 Examples:
 pattern = "abab", str = "redblueredblue" should return true.
 pattern = "aaaa", str = "asdasdasdasd" should return true.
 pattern = "aabb", str = "xyzabcxzyabc" should return false.

 Notes:
 You may assume both pattern and str contains only lowercase letters.
 */
public class _291 {

    public static class Solution1 {
        /**
         * We can try recursively:
         * say pattern is "abab", str is "redblueredblue"
         * first we try if "a" matches with "r", "b" matches with "e", we find it's not,
so we try to see if "b" matches "ed", and so on ...
         * then eventually, we find this pattern:
         * "a" matches "red"
         * "b" matches "blue"
         * then we'll just finish the str check based on this pattern
         * */
        public boolean wordPatternMatch(String pattern, String str) {
            Map<Character, String> map = new HashMap();
            Set<String> set = new HashSet();
            return isMatch(str, 0, pattern, 0, map, set);
        }

        private boolean isMatch(String str, int i, String pattern, int j, Map<Character,
String> map, Set<String> set) {
            //base case
            if (i == str.length() && j == pattern.length()) {
                return true;
            }
            if (i == str.length() || j == pattern.length()) {
                return false;
            }
```

```
            char c = pattern.charAt(j);

            if (map.containsKey(c)) {
                String s = map.get(c);

                //check to see if we can use s to match str.substring(i, i + s.length())
                if (!str.startsWith(s, i)) {
                    return false;
                }

                //if it's match, great, then let's check the rest
                return isMatch(str, i + s.length(), pattern, j + 1, map, set);
            }

            for (int k = i; k < str.length(); k++) {
                String p = str.substring(i, k + 1);

                if (set.contains(p)) {
                    continue;
                }

                map.put(c, p);
                set.add(p);

                //continue to match the rest
                if (isMatch(str, k + 1, pattern, j + 1, map, set)) {
                    return true;
                }

                //backtracking
                map.remove(c);
                set.remove(p);
            }

            //we've tried everything, but still no luck
            return false;
        }
    }
}
```

275. `_292.java`

```
package com.fishercoder.solutions;

/**You are playing the following Nim Game with your friend:
 * There is a heap of stones on the table, each time one of you take turns to remove 1 to 3 stones.
 * The one who removes the last stone will be the winner. You will take the first turn to remove the stones.
 * Both of you are very clever and have optimal strategies for the game.
 * Write a function to determine whether you can win the game given the number of stones in the heap.
 * For example, if there are 4 stones in the heap, then you will never win the game: no matter 1, 2, or 3 stones you remove,
 * the last stone will always be removed by your friend.

 Hint:
 If there are 5 stones in the heap, could you figure out a way to remove the stones such that you will always be the winner?*/

public class _292 {

    public static class Solution1 {
        /**
         * 1. If there are only 1 or 2 or 3 stones, you could always win by taking 1 or 2 or 3 stones;
         * 2. If there are 4 stones, you could never win because no matter you tak 1 or 2 or 3 stones, you could never take the 4th one;
         * 3. If there are 5 or 6 or 7 stones, you could always win because no matter how your opponent works, you'll always get the last one;
         * 4. Then we could deduce that as long as the number is not divisible by 4, you could always win.
         */

        public boolean canWinNim(int n) {
            return n % 4 != 0;
        }
    }
}
```

276. `_293.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
/**
 * 293. Flip Game
 *
 * You are playing the following Flip Game with your friend: Given a string that contains only
 * these two characters: + and -, you and your friend take turns to flip two consecutive "++" into "--".
 * The game ends when a person can no longer make a move and therefore the other person will be the winner.
 Write a function to compute all possible states of the string after one valid move.
 For example, given s = "++++", after one move, it may become one of the following states:

 [
 "--++",
 "+--+",
 "++--"
 ]
 If there is no valid move, return an empty list [].*/
public class _293 {

    public static class Solutoin1 {
        public List<String> generatePossibleNextMoves(String s) {
            List<String> result = new ArrayList<>();
            for (int i = 1; i < s.length(); i++) {
                if (s.charAt(i) == '+' && s.charAt(i - 1) == '+') {
                    result.add(s.substring(0, i - 1) + "--" + s.substring(i + 1));
                }
            }
            return result;
        }
    }
}
```

277. `_294.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * You are playing the following Flip Game with your friend:
 * Given a string that contains only these two characters: + and -,
 * you and your friend take turns to flip two consecutive "++" into "--".
 * The game ends when a person can no longer make a move and therefore the other person
will be the winner.

 Write a function to determine if the starting player can guarantee a win.

 For example, given s = "++++", return true. The starting player can guarantee a win by
flipping the middle "++" to become "+--+".

 Follow up:
 Derive your algorithm's runtime complexity.
 */
public class _294 {

    public static class Solution1 {
        public boolean canWin(String s) {
            List<String> res = new ArrayList<>();
            char[] charArray = s.toCharArray();
            for (int i = 0; i < s.length() - 1; i++) {
                if (charArray[i] == '+' && charArray[i + 1] == '+') {
                    //change these two bits to '-'
                    charArray[i] = '-';
                    charArray[i + 1] = '-';
                    res.add(String.valueOf(charArray));
                    //change these two bits back to '+' for its next move
                    charArray[i] = '+';
                    charArray[i + 1] = '+';
                }
            }
            /**The above part is the same of Flip Game I.
             * The only added part is the following piece of logic (so-called
backtracking.)*/
            for (String str : res) {
                if (!canWin(str)) {
                    return true;
                }
            }
            return false;
        }
    }

}
```

278. `_295.java`

```
package com.fishercoder.solutions;

import java.util.Collections;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 295. Find Median from Data Stream
 *
 * Median is the middle value in an ordered integer list.
 * If the size of the list is even, there is no middle value.
 * So the median is the mean of the two middle value.

 Examples:
 [2,3,4] , the median is 3

 [2,3], the median is (2 + 3) / 2 = 2.5

 Design a data structure that supports the following two operations:

 void addNum(int num) - Add a integer number from the data stream to the data structure.
 double findMedian() - Return the median of all elements so far.
 For example:

 addNum(1)
 addNum(2)
 findMedian() -> 1.5
 addNum(3)
 findMedian() -> 2
 */
public class _295 {
    /**A few key points for both following solutions:
     *
     * 1. always keep one queue one element more than the other if the number is odd,
offer into that one
     * first, then poll from that queue and offer into the other queue, then check
whether that queue is smaller
     * in size than the other, if so, poll one from the other queue and offer it into
this queue
     *
     * 2. only need to check whether this bigger queue size is greater than the other
queue when returning.*/

    public static class Solution1 {
        public static class MedianFinder {
            private Queue<Long> large;
            private Queue<Long> small;

            public MedianFinder() {
                large = new PriorityQueue<>();
                small = new PriorityQueue<>(Collections.reverseOrder());
```

```
            }

            // Adds a number into the data structure.
            public void addNum(int num) {
                large.offer((long) num);
                small.offer(large.poll());
                if (large.size() < small.size()) {
                    large.offer(small.poll());
                }
            }

            // Returns the median of current data stream
            public double findMedian() {
                if (large.size() > small.size()) {
                    return large.peek();
                }
                return (large.peek() + small.peek()) / 2.0;
            }
        }
    }

    public static class Solution2 {
        public static class MedianFinder {
            /**
             * credit: https://discuss.leetcode.com/topic/27521/short-simple-java-c-
python-o-log-n-o-1
             * The idea is for sure to use two heaps, one is max heap, one is min heap,
we always let the max heap be one element
             * bigger than min heap if the total number of elements is not even.
             * we could always get the median in O(1) time.
             * 1. use Long type to avoid overflow
             * 2. negate the numbers for small heap to save the effort for writing a
reverse comparator, brilliant!
             */

            private Queue<Long> large;
            private Queue<Long> small;

            /**
             * initialize your data structure here.
             */
            public MedianFinder() {
                large = new PriorityQueue<>();
                small = new PriorityQueue<>();
            }

            // Adds a number into the data structure.
            public void addNum(int num) {
                large.offer((long) num);
                small.offer(-large.poll());
                if (large.size() < small.size()) {
                    large.offer(-small.poll());
                }
            }
```

```
            // Returns the median of current data stream
            public double findMedian() {
                if (large.size() > small.size()) {
                    return large.peek();
                }
                return (large.peek() - small.peek()) / 2.0;
            }

        }
    }
}
```

279. `_296.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 296: Best Meeting Point
 *
 * A group of two or more people wants to meet and minimize the total travel distance.
 * You are given a 2D grid of values 0 or 1, where each 1 marks the home of someone in
the group.
 * The distance is calculated using Manhattan Distance, where distance(p1, p2) = |p2.x -
p1.x| + |p2.y - p1.y|.

 For example, given three people living at (0,0), (0,4), and (2,2):

 1 - 0 - 0 - 0 - 1
 |   |   |   |   |
 0 - 0 - 0 - 0 - 0
 |   |   |   |   |
 0 - 0 - 1 - 0 - 0
 The point (0,2) is an ideal meeting point, as the total travel distance of 2+2+2=6 is
minimal. So return 6.
 */
public class _296 {
        public static class Solution1 {
                public int minTotalDistance(int[][] grid) {
                        int m = grid.length;
                        int n = grid[0].length;

                        List<Integer> I = new ArrayList(m);
                        List<Integer> J = new ArrayList(n);

                        for (int i = 0; i < m; i++) {
                                for (int j = 0; j < n; j++) {
                                        if (grid[i][j] == 1) {
                                                I.add(i);
                                                J.add(j);
                                        }
                                }
                        }

                        return getMin(I) + getMin(J);
                }

                private int getMin(List<Integer> list) {
                        int ret = 0;

                        Collections.sort(list);
```

```
                        int i = 0;
                        int j = list.size() - 1;
                        while (i < j) {
                                ret += list.get(j--) - list.get(i++);
                        }

                        return ret;
                }
        }
}
```

280. `_297.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 297. Serialize and Deserialize Binary Tree
 *
 * Serialization is the process of converting a data structure or object into a sequence
of bits
 * so that it can be stored in a file or memory buffer,
 * or transmitted across a network connection link to be reconstructed later in the same
or another computer environment.
 * Design an algorithm to serialize and deserialize a binary tree.
 * There is no restriction on how your serialization/deserialization algorithm should
work.
 * You just need to ensure that a binary tree can be serialized to a string and this
string can
 * be deserialized to the original tree structure.

 For example, you may serialize the following tree

     1
    / \
   2   3
  / \
 4   5

 as "[1,2,3,null,null,4,5]",
 just the same as how LeetCode OJ serializes a binary tree.
 You do not necessarily need to follow this format, so please be creative and come up
with different approaches yourself.

 Note: Do not use class member/global/static variables to store states. Your serialize
and deserialize algorithms should be stateless.
 */
public class _297 {

    public static class Solution1 {
        /**
         * The idea is very straightforward:
         * use "#" as the terminator, do BFS, level order traversal to store all nodes
values into a StringBuilder.
         * When deserializing, also use a queue: pop the root into the queue first, then
use a for loop to construct each node,
         * then eventually just return the root.
         */

        // Encodes a tree to a single string.
```

```
        public String serialize(TreeNode root) {
            if (root == null) {
                return "";
            }

            StringBuilder sb = new StringBuilder();
            Queue<TreeNode> queue = new LinkedList();
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (curr == null) {
                        sb.append("# ");
                        continue;
                    }
                    sb.append(curr.val);
                    sb.append(" ");
                    queue.offer(curr.left);
                    queue.offer(curr.right);
                }
            }
            return sb.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.isEmpty()) {
                return null;
            }

            String[] nodes = data.split(" ");
            TreeNode root = new TreeNode(Integer.valueOf(nodes[0]));
            Queue<TreeNode> queue = new LinkedList();
            queue.offer(root);
            for (int i = 1; i < nodes.length; i++) {
                TreeNode curr = queue.poll();
                if (!nodes[i].equals("#")) {
                    curr.left = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.left);
                }
                if (!nodes[++i].equals("#")) {
                    curr.right = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.right);
                }
            }
            return root;
        }
    }
}
```

281. `_298.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * Given a binary tree, find the length of the longest consecutive sequence path.

 The path refers to any sequence of nodes from some starting node to any node in the tree
along the parent-child connections.
 The longest consecutive path need to be from parent to child (cannot be the reverse).

 For example,
   1
   \
   3
  / \
 2  4
     \
     5
 Longest consecutive sequence path is 3-4-5, so return 3.

   2
    \
    3
   /
  2
 /
1
 Longest consecutive sequence path is 2-3,not3-2-1, so return 2.
 */
public class _298 {

    public static class Solution1 {
        private int max = 1;

        public int longestConsecutive(TreeNode root) {
            if (root == null) {
                return 0;
            }
            dfs(root, 0, root.val);
            return max;
        }

        private void dfs(TreeNode root, int curr, int target) {
            if (root == null) {
                return;
            }
            if (root.val == target) {
                curr++;
            } else {
                curr = 1;
```

```
            }
            max = Math.max(max, curr);
            dfs(root.left, curr, root.val + 1);
            dfs(root.right, curr, root.val + 1);
        }
    }
}
```

282. `_299.java`

```
package com.fishercoder.solutions;

/**
 * 299. Bulls and Cows
 *
 * You are playing the following Bulls and Cows game with your friend:
 * You write down a number and ask your friend to guess what the number is.
 * Each time your friend makes a guess, you provide a hint that indicates how many digits
in said guess match your secret number exactly in
 * both digit and position (called "bulls") and how many digits match the secret number
but locate in the wrong position (called "cows").
 * Your friend will use successive guesses and hints to eventually derive the secret
number.

 For example:

 Secret number:  "1807"
 Friend's guess: "7810"
 Hint: 1 bull and 3 cows. (The bull is 8, the cows are 0, 1 and 7.)
 Write a function to return a hint according to the secret number and friend's guess, use
A to indicate the bulls and B to indicate the cows. In the above example, your function
should return "1A3B".

 Please note that both secret number and friend's guess may contain duplicate digits, for
example:

 Secret number:  "1123"
 Friend's guess: "0111"
 In this case, the 1st 1 in friend's guess is a bull, the 2nd or 3rd 1 is a cow, and your
function should return "1A1B".
 You may assume that the secret number and your friend's guess only contain digits, and
their lengths are always equal.*/
public class _299 {
    public static class Solution1 {
        public String getHint(String secret, String guess) {
            int[] secretCows = new int[10];
            int[] guessCows = new int[10];
            int bulls = 0;
            for (int i = 0; i < secret.length(); i++) {
                if (guess.charAt(i) == secret.charAt(i)) {
                    bulls++;
                } else {
                    secretCows[Character.getNumericValue(secret.charAt(i))]++;
                    guessCows[Character.getNumericValue(guess.charAt(i))]++;
                }
            }
            int cows = 0;
            for (int i = 0; i < 11; i++) {
                cows += Math.min(secretCows[i], guessCows[i]);
            }
            return bulls + "A" + cows + "B";
```

```
        }
    }
}
```

283. `_300.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 300. Longest Increasing Subsequence
 *
 * Given an unsorted array of integers, find the length of longest increasing
subsequence.
 For example,
 Given [10, 9, 2, 5, 3, 7, 101, 18],
 The longest increasing subsequence is [2, 3, 7, 101], therefore the length is 4.
 Note that there may be more than one LIS combination, it is only necessary for you to
return the length.

 Your algorithm should run in O(n2) complexity.

 Follow up: Could you improve it to O(nlogn) time complexity?
 */
public class _300 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/28719/short-java-solution-using-dp-
o-n-log-n
         * The idea is that as you iterate the sequence,
         * you keep track of the minimum value a subsequence of given length might end
with,
         * for all so far possible subsequence lengths.
         * So dp[i] is the minimum value a subsequence of length i+1 might end with.
         * Having this info, for each new number we iterate to,
         * we can determine the longest subsequence where it can be appended using binary
search.
         * The final answer is the length of the longest subsequence we found so far.
         */
        public int lengthOfLIS(int[] nums) {
            int[] dp = new int[nums.length];
            int len = 0;
            for (int x : nums) {
                /**Java Doc of this binarySearch API:
                 * @return index of the search key, if it is contained in the array
                 *         within the specified range;
                 *         otherwise, <tt>(-(<i>insertion point</i>) - 1)</tt>.  The
                 *         <i>insertion point</i> is defined as the point at which the
                 *         key would be inserted into the array: the index of the first
                 *         element in the range greater than the key,
                 *         or <tt>toIndex</tt> if all
                 *         elements in the range are less than the specified key.  Note
                 *         that this guarantees that the return value will be &gt;= 0 if
                 *         and only if the key is found.*/
```

```
                int index = Arrays.binarySearch(dp, 0, len, x);
                if (index < 0) {
                    index = -(index + 1);
                }
                dp[index] = x;
                if (index == len) {
                    len++;
                }
            }
            return len;
        }
    }
}
```

284. `_301.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.Set;

/**
 * 301. Remove Invalid Parentheses
 *
 * Remove the minimum number of invalid parentheses in order to make the input string
valid. Return all possible results.

 Note: The input string may contain letters other than the parentheses ( and ).

 Examples:
 "()())()" -> ["()()()", "(())()"]
 "(a)())()" -> ["(a)()()", "(a())()"]
 ")(" -> [""]
 */
public class _301 {

        public static class Solution1 {

                public List<String> removeInvalidParentheses(String s) {
                        List<String> result = new ArrayList<>();
                        if (s == null) {
                                return result;
                        }

                        Set<String> visited = new HashSet();
                        Queue<String> q = new LinkedList();

                        q.offer(s);
                        visited.add(s);

                        boolean found = false;

                        while (!q.isEmpty()) {
                                String curr = q.poll();
                                if (isValid(curr)) {
                                        found = true;
                                        result.add(curr);
                                }

                                if (found) {
                                        continue;//this means if the initial input is
already a valid one, we'll just directly return it and there's actually only one valid
result
```

```
                                }

                                for (int i = 0; i < curr.length(); i++) {
                                        if (curr.charAt(i) != '(' && curr.charAt(i) !=
')') {
                                                continue;//this is to rule out those non-
parentheses characters
                                        }

                                        String next = curr.substring(0, i) +
curr.substring(i + 1);
                                        if (!visited.contains(next)) {
                                                q.offer(next);
                                                visited.add(next);
                                        }
                                }

                        }
                        return result;
                }

                private boolean isValid(String str) {
                        char[] chars = str.toCharArray();
                        int count = 0;
                        for (int i = 0; i < chars.length; i++) {
                                char c = chars[i];
                                if (c == '(') {
                                        count++;
                                }
                                if (c == ')') {
                                        count--;
                                        if (count == -1) {
                                                return false;
                                        }
                                }
                        }
                        return count == 0;
                }
        }
}
```

285. `_302.java`

```
package com.fishercoder.solutions;

/**
 * 302. Smallest Rectangle Enclosing Black Pixels
 *
 * An image is represented by a binary matrix with 0 as a white pixel and 1 as a black
pixel.
 * The black pixels are connected, i.e., there is only one black region.
 * Pixels are connected horizontally and vertically.
 * Given the location (x, y) of one of the black pixels, return the area of the smallest
(axis-aligned) rectangle that encloses all black pixels.

 For example, given the following image:

 [
 "0010",
 "0110",
 "0100"
 ]
 and x = 0, y = 2,
 Return 6.
 */
public class _302 {
    public static class Solution1 {
        private char[][] image;

        public int minArea(char[][] iImage, int x, int y) {
            image = iImage;
            int m = image.length;
            int n = image[0].length;
            int left = searchColumns(0, y, 0, m, true);
            int right = searchColumns(y + 1, n, 0, m, false);
            int top = searchRows(0, x, left, right, true);
            int bottom = searchRows(x + 1, m, left, right, false);
            return (right - left) * (bottom - top);
        }

        private int searchColumns(int i, int j, int top, int bottom, boolean opt) {
            while (i != j) {
                int k = top;
                int mid = (i + j) / 2;
                while (k < bottom && image[k][mid] == '0') {
                    ++k;
                }
                if (k < bottom == opt) {
                    j = mid;
                } else {
                    i = mid + 1;
                }
            }
            return i;
```

```
        }

        private int searchRows(int i, int j, int left, int right, boolean opt) {
            while (i != j) {
                int k = left;
                int mid = (i + j) / 2;
                while (k < right && image[mid][k] == '0') {
                    ++k;
                }
                if (k < right == opt) {
                    j = mid;
                } else {
                    i = mid + 1;
                }
            }
            return i;
        }
    }
}
```

286. `_303.java`

```
package com.fishercoder.solutions;

/**
 * 303. Range Sum Query - Immutable
 * Given an integer array nums, find the sum of the elements between indices i and j (i ≤ j), inclusive.

 Example:
 Given nums = [-2, 0, 3, -5, 2, -1]

 sumRange(0, 2) -> 1
 sumRange(2, 5) -> -1
 sumRange(0, 5) -> -3

 Note:
 You may assume that the array does not change.
 There are many calls to sumRange function.
 */
public class _303 {
    public static class NumArray {
        int[] sums;

        public NumArray(int[] nums) {
            sums = new int[nums.length];
            for (int i = 0; i < nums.length; i++) {
                if (i == 0) {
                    sums[i] = nums[i];
                } else {
                    sums[i] = sums[i - 1] + nums[i];
                }
            }
        }

        public int sumRange(int i, int j) {
            if (i == 0) {
                return sums[j];
            }
            return sums[j] - sums[i - 1];
        }
    }
}

// Your NumArray object will be instantiated and called as such:
// NumArray numArray = new NumArray(nums);
// numArray.sumRange(0, 1);
// numArray.sumRange(1, 2);
```

287. `_304.java`

```
package com.fishercoder.solutions;

/**
 * 304. Range Sum Query 2D - Immutable
 *
 * Given a 2D matrix matrix, find the sum of the elements inside the rectangle defined by
its upper left corner (row1, col1) and lower right corner (row2, col2).

 Range Sum Query 2D
 The above rectangle (with the red border) is defined by (row1, col1) = (2, 1) and (row2,
col2) = (4, 3), which contains sum = 8.

 Example:
 Given matrix = [
 [3, 0, 1, 4, 2],
 [5, 6, 3, 2, 1],
 [1, 2, 0, 1, 5],
 [4, 1, 0, 1, 7],
 [1, 0, 3, 0, 5]
 ]

 sumRegion(2, 1, 4, 3) -> 8
 sumRegion(1, 1, 2, 2) -> 11
 sumRegion(1, 2, 2, 4) -> 12
 Note:
 You may assume that the matrix does not change.
 There are many calls to sumRegion function.
 You may assume that row1 ≤ row2 and col1 ≤ col2.
 */
public class _304 {

    public static class Solution1 {
        public class NumMatrix {

            public NumMatrix(int[][] matrix) {
                if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
                    return;
                }

                /**The dimensions of this tot matrix is actually 1 bigger than the given
matrix, cool!*/
                tot = new int[matrix.length + 1][matrix[0].length + 1];
                for (int i = 0; i < matrix.length; i++) {
                    for (int j = 0; j < matrix[0].length; j++) {
                        tot[i + 1][j + 1] =
                            matrix[i][j] + tot[i + 1][j] + tot[i][j + 1] - tot[i][j];
                    }
                }
            }

            public int sumRegion(int row1, int col1, int row2, int col2) {
```

```
                return tot[row2 + 1][col2 + 1] - tot[row2 + 1][col1] - tot[row1][col2 +
1]
                    + tot[row1][col1];
            }

            int[][] tot;
        }
    }
/**
 * Your NumMatrix object will be instantiated and called as such:
 * NumMatrix obj = new NumMatrix(matrix);
 * int param_1 = obj.sumRegion(row1,col1,row2,col2);
 */
}
```

288. `_305.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 305. Number of Islands II
 *
 * A 2d grid map of m rows and n columns is initially filled with water.
 * We may perform an addLand operation which turns the water at position (row, col) into
a land.
 * Given a list of positions to operate, count the number of islands after each addLand
operation.
 * An island is surrounded by water and is formed by connecting adjacent lands
horizontally or vertically.
 * You may assume all four edges of the grid are all surrounded by water.

 Example:

 Given m = 3, n = 3, positions = [[0,0], [0,1], [1,2], [2,1]].
 Initially, the 2d grid grid is filled with water. (Assume 0 represents water and 1
represents land).

 0 0 0
 0 0 0
 0 0 0
 Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land.

 1 0 0
 0 0 0   Number of islands = 1
 0 0 0
 Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land.

 1 1 0
 0 0 0   Number of islands = 1
 0 0 0
 Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land.

 1 1 0
 0 0 1   Number of islands = 2
 0 0 0
 Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land.

 1 1 0
 0 0 1   Number of islands = 3
 0 1 0
 We return the result as an array: [1, 1, 2, 3]

 Challenge:

 Can you do it in time complexity O(k log mn), where k is the length of the positions?
```

```
 */
public class _305 {
    public static class Solution1 {

        public int find(int[] father, int id) {
            int tf = father[id];
            while (tf != father[tf]) {
                tf = father[tf];
            }
            int cur = id;
            int tmp;
            while (father[cur] != tf) {
                tmp = father[cur];
                father[cur] = tf;
                cur = tmp;
            }
            return tf;
        }

        public void union(int[] father, int[] sz, int id1, int id2) {
            int tf1 = find(father, id1);
            int tf2 = find(father, id2);
            if (tf1 != tf2) {
                if (sz[tf1] > sz[tf2]) {
                    father[tf2] = tf1;
                    sz[tf1] += sz[tf2];
                } else {
                    father[tf1] = tf2;
                    sz[tf2] += sz[tf1];
                }
            }
        }

        public List<Integer> numIslands2(int m, int n, int[][] positions) {
            if (m == 0 || n == 0) {
                return new ArrayList<>();
            }
            ArrayList<Integer> res = new ArrayList();
            int[] father = new int[m * n];
            for (int i = 0; i < father.length; i++) {
                father[i] = -1;
            }
            int[] sz = new int[m * n];
            int[] dr = {0, 0, -1, 1};
            int[] dc = {-1, 1, 0, 0};
            int r;
            int c;
            int nr;
            int nc;
            int count = 0;
            for (int i = 0; i < positions.length; i++) {
                r = positions[i][0];
                c = positions[i][1];
                count++;
```

```
                father[r * n + c] = r * n + c;
                sz[r * n + c] = 1;
                for (int j = 0; j < 4; j++) {
                    nr = r + dr[j];
                    nc = c + dc[j];
                    if (nr >= 0 && nr < m && nc >= 0 && nc < n && father[nr * n + nc] !=
-1) {
                        if (find(father, r * n + c) != find(father, nr * n + nc)) {
                            count--;
                            union(father, sz, r * n + c, nr * n + nc);
                        }
                    }
                }
                res.add(count);
            }
            return res;
        }
    }
}
```

289. `_306.java`

```
package com.fishercoder.solutions;

/**
 * 306. Additive Number
 *
 * Additive number is a string whose digits can form additive sequence.

 A valid additive sequence should contain at least three numbers.
 Except for the first two numbers, each subsequent number in the sequence must be the sum of the preceding two.

 For example:
 "112358" is an additive number because the digits can form an additive sequence: 1, 1, 2, 3, 5, 8.

 1 + 1 = 2, 1 + 2 = 3, 2 + 3 = 5, 3 + 5 = 8
 "199100199" is also an additive number, the additive sequence is: 1, 99, 100, 199.
 1 + 99 = 100, 99 + 100 = 199
 Note: Numbers in the additive sequence cannot have leading zeros, so sequence 1, 2, 03 or 1, 02, 3 is invalid.

 Given a string containing only digits '0'-'9', write a function to determine if it's an additive number.

 Follow up:
 How would you handle overflow for very large input integers?
 */
public class _306 {
    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/29856/java-recursive-and-iterative-solutions/2 */
        public boolean isAdditiveNumber(String num) {
            int n = num.length();
            for (int i = 1; i <= n / 2; ++i) {
                for (int j = 1; Math.max(j, i) <= n - i - j; ++j) {
                    if (isValid(i, j, num)) {
                        return true;
                    }
                }
            }
            return false;
        }

        private boolean isValid(int i, int j, String num) {
            if (num.charAt(0) == '0' && i > 1) {
                return false;
            }
            if (num.charAt(i) == '0' && j > 1) {
                return false;
            }
            String sum;
```

```
            Long x1 = Long.parseLong(num.substring(0, i));
            Long x2 = Long.parseLong(num.substring(i, i + j));
            for (int start = i + j; start != num.length(); start += sum.length()) {
                x2 = x2 + x1;
                x1 = x2 - x1;
                sum = x2.toString();
                if (!num.startsWith(sum, start)) {
                    return false;
                }
            }
            return true;
        }
    }

}
```

290. `_307.java`

```
package com.fishercoder.solutions;

/**
 * 307. Range Sum Query - Mutable
 *
 * Given an integer array nums, find the sum of the elements between indices i and j (i ≤ j), inclusive.

 The update(i, val) function modifies nums by updating the element at index i to val.
 Example:
 Given nums = [1, 3, 5]

 sumRange(0, 2) -> 9
 update(1, 2)
 sumRange(0, 2) -> 8
 Note:
 The array is only modifiable by the update function.
 You may assume the number of calls to update and sumRange function is distributed evenly.
 */
public class _307 {

        public static class Solution1 {
                class NumArray {
                        class SegmentTreeNode {
                                SegmentTreeNode left;
                                SegmentTreeNode right;
                                int start;
                                int end;
                                int sum;

                                public SegmentTreeNode(int start, int end) {
                                        this.start = start;
                                        this.end = end;
                                        this.left = null;
                                        this.right = null;
                                        this.sum = 0;
                                }
                        }

                        private SegmentTreeNode root = null;

                        public NumArray(int[] nums) {
                                root = buildSegmentTree(nums, 0, nums.length - 1);
                        }

                        SegmentTreeNode buildSegmentTree(int[] nums, int start, int end) {

                                if (start > end) {
                                        return null;
                                } else {
```

```
                                        SegmentTreeNode root = new SegmentTreeNode(start,
end);
                                        if (start == end) {
                                                root.sum = nums[start];
                                        } else {
                                                int mid = start + (end - start) / 2;
                                                root.left = buildSegmentTree(nums, start,
mid);
                                                root.right = buildSegmentTree(nums, mid +
1, end);
                                                root.sum = root.left.sum +
root.right.sum;
                                        }
                                        return root;
                                }
                        }

                        void update(int i, int val) {
                                update(root, i, val);
                        }

                        void update(SegmentTreeNode root, int pos, int val) {
                                if (root.start == root.end) {
                                        root.sum = val;
                                } else {
                                        int mid = root.start + (root.end - root.start) /
2;
                                        if (pos <= mid) {
                                                update(root.left, pos, val);
                                        } else {
                                                update(root.right, pos, val);
                                        }
                                        root.sum = root.left.sum + root.right.sum;
                                }
                        }

                        public int sumRange(int i, int j) {
                                return sumRange(root, i, j);
                        }

                        int sumRange(SegmentTreeNode root, int start, int end) {
                                if (root.end == end && root.start == start) {
                                        return root.sum;
                                } else {
                                        int mid = root.start + (root.end - root.start) /
2;
                                        if (end <= mid) {
                                                return sumRange(root.left, start, end);
                                        } else if (start >= mid + 1) {
                                                return sumRange(root.right, start, end);
                                        } else {
                                                return sumRange(root.right, mid + 1, end)
+ sumRange(root.left, start, mid);
                                        }
```

```
                                }
                            }
                        }
                    }
}
```

291. `_308.java`

```
package com.fishercoder.solutions;

/**
 * 308. Range Sum Query 2D - Mutable
 *
 * Given a 2D matrix matrix,
 * find the sum of the elements inside the rectangle defined by its upper left corner
(row1, col1) and lower right corner (row2, col2).

 Range Sum Query 2D
 The above rectangle (with the red border) is defined by (row1, col1) = (2, 1) and (row2,
col2) = (4, 3), which contains sum = 8.

 Example:
 Given matrix = [
 [3, 0, 1, 4, 2],
 [5, 6, 3, 2, 1],
 [1, 2, 0, 1, 5],
 [4, 1, 0, 1, 7],
 [1, 0, 3, 0, 5]
 ]

 sumRegion(2, 1, 4, 3) -> 8
 update(3, 2, 2)
 sumRegion(2, 1, 4, 3) -> 10
 Note:
 The matrix is only modifiable by the update function.
 You may assume the number of calls to update and sumRegion function is distributed
evenly.
 You may assume that row1 ≤ row2 and col1 ≤ col2.
 */
public class _308 {
    public static class Solution1 {
        public class NumMatrix {
            int[][] nums;
            int[][] tree;
            int height;
            int width;

            public NumMatrix(int[][] matrix) {
                if (matrix.length == 0 || matrix[0].length == 0) {
                    return;
                }
                height = matrix.length;
                width = matrix[0].length;
                this.nums = new int[height][width];
                this.tree = new int[height + 1][width + 1];
                for (int i = 0; i < height; i++) {
                    for (int j = 0; j < width; j++) {
                        update(i, j, matrix[i][j]);
                    }
```

```
                }
            }

            public void update(int rowIndex, int colIndex, int newVal) {
                if (height == 0 || width == 0) {
                    return;
                }
                int delta = newVal - nums[rowIndex][colIndex];
                nums[rowIndex][colIndex] = newVal;
                for (int i = rowIndex + 1; i <= height; i += i & (-i)) {
                    for (int j = colIndex + 1; j <= width; j += j & (-j)) {
                        tree[i][j] += delta;//just use its previous value plus delta is
good
                    }
                }
            }

            public int sumRegion(int row1, int col1, int row2, int col2) {
                if (height == 0 || width == 0) {
                    return 0;
                }
                return sum(row2 + 1, col2 + 1) + sum(row1, col1) - sum(row1, col2 + 1) -
sum(
                        row2 + 1, col1);
            }

            private int sum(int row, int col) {
                int sum = 0;
                for (int i = row; i > 0; i -= i & (-i)) {
                    for (int j = col; j > 0; j -= j & (-j)) {
                        sum += tree[i][j];
                    }
                }
                return sum;
            }
        }

        /**
         * Your NumMatrix object will be instantiated and called as such:
         * NumMatrix obj = new NumMatrix(matrix);
         * obj.update(row,col,val);
         * int param_2 = obj.sumRegion(row1,col1,row2,col2);
         */
    }
}
```

292. `_309.java`

```
package com.fishercoder.solutions;

/**
 * 309. Best Time to Buy and Sell Stock with Cooldown
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.

 Design an algorithm to find the maximum profit. You may complete as many transactions as
you like (ie, buy one and sell one share of the stock multiple times) with the following
restrictions:

 You may not engage in multiple transactions at the same time (ie, you must sell the
stock before you buy again).
 After you sell your stock, you cannot buy stock on next day. (ie, cooldown 1 day)
 Example:

 prices = [1, 2, 3, 0, 2]
 maxProfit = 3
 transactions = [buy, sell, cooldown, buy, sell]
 */
public class _309 {
    public static class Solution1 {
        /**
         * The series of problems are typical dp. The key for dp is to find the variables
to
         * represent the states and deduce the transition function.
         *
         * Of course one may come up with a O(1) space solution directly, but I think it
is better
         * to be generous when you think and be greedy when you implement.
         *
         * The natural states for this problem is the 3 possible transactions : buy,
sell, rest.
         * Here rest means no transaction on that day (aka cooldown).
         *
         * Then the transaction sequences can end with any of these three states.
         *
         * For each of them we make an array, buy[n], sell[n] and rest[n].
         *
         * buy[i] means before day i what is the maxProfit for any sequence end with buy.
         *
         * sell[i] means before day i what is the maxProfit for any sequence end with
sell.
         *
         * rest[i] means before day i what is the maxProfit for any sequence end with
rest.
         *
         * Then we want to deduce the transition functions for buy sell and rest. By
definition we
         * have:
```

```
         *
         * buy[i] = max(rest[i-1]-price, buy[i-1])
         * sell[i] = max(buy[i-1]+price, sell[i-1])
         * rest[i] = max(sell[i-1], buy[i-1], rest[i-1])
         *
         * Where price is the price of day i. All of these are very straightforward. They
simply represents :
         *
         * (1) We have to `rest` before we `buy` and
         * (2) we have to `buy` before we `sell`
         * One tricky point is how do you make sure you sell before you buy, since from
the equations it seems that [buy, rest, buy] is entirely possible.
         *
         * Well, the answer lies within the fact that buy[i] <= rest[i] which means
rest[i] =
         * max(sell[i-1], rest[i-1]). That made sure [buy, rest, buy] is never occurred.
         *
         * A further observation is that and rest[i] <= sell[i] is also true therefore
         *
         * rest[i] = sell[i-1] Substitute this in to buy[i] we now have 2 functions
instead of 3:
         *
         * buy[i] = max(sell[i-2]-price, buy[i-1]) sell[i] = max(buy[i-1]+price, sell[i-
1]) This is
         * better than 3, but
         *
         * we can do even better
         *
         * Since states of day i relies only on i-1 and i-2 we can reduce the O(n) space
to O(1).
         * And here we are at our final solution:
         */
        public int maxProfit(int[] prices) {
            int sell = 0;
            int prevSell = 0;
            int buy = Integer.MIN_VALUE;
            int prevBuy;
            for (int price : prices) {
                prevBuy = buy;
                buy = Math.max(prevSell - price, prevBuy);
                prevSell = sell;
                sell = Math.max(prevBuy + price, prevSell);
            }
            return sell;
        }
    }

    public static class Solution2 {
        /**Surprisingly, this solution is even much faster than the one above provided by
the author.*/
        /**
         * Here I share my no brainer weapon when it comes to this kind of problems.
         *
         * 1. Define States
```

```
         *
         * To represent the decision at index i:
         *
         * buy[i]: Max profit till index i. The series of transaction is ending with a
buy.
         * sell[i]: Max profit till index i. The series of transaction is ending with a
sell.
         *
         * 2. Define Recursion
         *
         * buy[i]: To make a decision whether to buy at i, we either take a rest, by just
using the
         * old decision at i - 1, or sell at/before i - 2, then buy at i, We cannot sell
at i - 1,
         * then buy at i, because of cooldown.
         * sell[i]: To make a decision whether to sell at i, we either take a rest, by
just using the old decision at i - 1,
         * or buy at/before i - 1, then sell at i.
         *
         * So we get the following formula:
         *
         * buy[i] = Math.max(buy[i - 1], sell[i - 2] - prices[i]);
         * sell[i] = Math.max(sell[i - 1], buy[i - 1] + prices[i]);
         *
         * 3. Optimize to O(1) Space
         *
         * DP solution only depending on i - 1 and i - 2 can be optimized using O(1)
space.
         *
         * Let b2, b1, b0 represent buy[i - 2], buy[i - 1], buy[i]
         * Let s2, s1, s0 represent sell[i - 2], sell[i - 1], sell[i]
         *
         * Then arrays turn into Fibonacci like recursion:
         *
         * b0 = Math.max(b1, s2 - prices[i]);
         * s0 = Math.max(s1, b1 + prices[i]);
         *
         * 4. Write Code in 5 Minutes
         *
         * First we define the initial states at i = 0:
         *
         * We can buy. The max profit at i = 0 ending with a buy is -prices[0].
         * We cannot sell. The max profit at i = 0 ending with a sell is 0.
         */
        public int maxProfit(int[] prices) {
            if (prices == null || prices.length <= 1) {
                return 0;
            }

            int b0 = -prices[0];
            int b1 = b0;
            int b2 = b0;
            int s0 = 0;
            int s1 = s0;
```

```
            int s2 = s0;

            for (int i = 1; i < prices.length; i++) {
                b0 = Math.max(b1, s2 - prices[i]);
                s0 = Math.max(s1, b1 + prices[i]);
                b2 = b1;
                b1 = b0;
                s2 = s1;
                s1 = s0;
            }
            return s0;
        }
    }
}
```

293. `_310.java`

```
package com.fishercoder.solutions;


import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 310. Minimum Height Trees
 *
 * For a undirected graph with tree characteristics, we can choose any node as the root.
 * The result graph is then a rooted tree. Among all possible rooted trees, those with
minimum height are called minimum height trees (MHTs).
 * Given such a graph, write a function to find all the MHTs and return a list of their
root labels.

 Format
 The graph contains n nodes which are labeled from 0 to n - 1. You will be given the
number n and a list of undirected edges (each edge is a pair of labels).

 You can assume that no duplicate edges will appear in edges. Since all edges are
undirected, [0, 1] is the same as [1, 0] and thus will not appear together in edges.

 Example 1:

 Given n = 4, edges = [[1, 0], [1, 2], [1, 3]]

   0
   |
   1
  / \
 2   3
 return [1]

 Example 2:

 Given n = 6, edges = [[0, 3], [1, 3], [2, 3], [4, 3], [5, 4]]

 0  1  2
  \ | /
    3
    |
    4
    |
    5
 return [3, 4]

 Hint:
```

```
 How many MHTs can a graph have at most?
 Note:

 (1) According to the definition of tree on Wikipedia: “a tree is an undirected graph in
which any two vertices are connected by exactly one path. In other words, any connected
graph without simple cycles is a tree.”

 (2) The height of a rooted tree is the number of edges on the longest downward path
between the root and a leaf.
 */
public class _310 {

    public static class Solution1 {
        public List<Integer> findMinHeightTrees(int n, int[][] edges) {
            if (n == 1) {
                return Collections.singletonList(0);
            }

            List<Set<Integer>> adj = new ArrayList<>(n);
            for (int i = 0; i < n; ++i) {
                adj.add(new HashSet<>());
            }
            for (int[] edge : edges) {
                adj.get(edge[0]).add(edge[1]);
                adj.get(edge[1]).add(edge[0]);
            }

            List<Integer> leaves = new ArrayList<>();
            for (int i = 0; i < n; ++i) {
                if (adj.get(i).size() == 1) {
                    leaves.add(i);
                }
            }

            while (n > 2) {
                n -= leaves.size();
                List<Integer> newLeaves = new ArrayList<>();
                for (int i : leaves) {
                    int j = adj.get(i).iterator().next();
                    adj.get(j).remove(i);
                    if (adj.get(j).size() == 1) {
                        newLeaves.add(j);
                    }
                }
                leaves = newLeaves;
            }
            return leaves;
        }
    }
}
```

294. `_311.java`

```
package com.fishercoder.solutions;

/**
 * 311. Sparse Matrix Multiplication
 *
 * Given two sparse matrices A and B, return the result of AB.

 You may assume that A's column number is equal to B's row number.

 Example:

 A = [
 [ 1, 0, 0],
 [-1, 0, 3]
 ]

 B = [
 [ 7, 0, 0 ],
 [ 0, 0, 0 ],
 [ 0, 0, 1 ]
 ]

      |  1 0 0 |   | 7 0 0 |   |  7 0 0 |
 AB = | -1 0 3 | x | 0 0 0 | = | -7 0 3 |
                   | 0 0 1 |

 */
public class _311 {

    public static class Solution1 {
        public int[][] multiply(int[][] A, int[][] B) {
            int m = A.length;
            int n = A[0].length;
            int p = B[0].length;
            int[][] C = new int[m][p];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (A[i][j] != 0) {
                        for (int k = 0; k < p; k++) {
                            if (B[j][k] != 0) {
                                C[i][k] += A[i][j] * B[j][k];
                            }
                        }
                    }
                }
            }
            return C;
        }
    }
}
```

295. `_312.java`

```
package com.fishercoder.solutions;

/**
 * 312. Burst Balloons
 *
 * Given n balloons, indexed from 0 to n-1.
 * Each balloon is painted with a number on it represented by array nums.
 * You are asked to burst all the balloons.
 * If the you burst balloon i you will get nums[left] * nums[i] * nums[right] coins.
 * Here left and right are adjacent indices of i. After the burst, the left and right
then becomes adjacent.

 Find the maximum coins you can collect by bursting the balloons wisely.

 Note:
 (1) You may imagine nums[-1] = nums[n] = 1. They are not real therefore you can not
burst them.
 (2) 0 ≤ n ≤ 500, 0 ≤ nums[i] ≤ 100

 Example:

 Given [3, 1, 5, 8]

 Return 167

 nums = [3,1,5,8] --> [3,5,8] -->   [3,8]   -->  [8]  --> []
 coins =  3*1*5      +  3*5*8    +  1*3*8      + 1*8*1   = 167
 */
public class _312 {

    public static class Solution1 {
        public int maxCoins(int[] iNums) {
            int[] nums = new int[iNums.length + 2];
            int n = 1;
            for (int x : iNums) {
                if (x > 0) {
                    nums[n++] = x;
                }
            }
            nums[0] = nums[n++] = 1;

            int[][] dp = new int[n][n];
            for (int k = 2; k < n; ++k) {
                for (int left = 0; left < n - k; ++left) {
                    int right = left + k;
                    for (int i = left + 1; i < right; ++i) {
                        dp[left][right] = Math.max(dp[left][right],
                            nums[left] * nums[i] * nums[right] + dp[left][i] + dp[i]
[right]);
                    }
                }
```

```
            }
            return dp[0][n - 1];
        }
    }
}
```

296. `_313.java`

```
package com.fishercoder.solutions;

/**
 * 313. Super Ugly Number
 *
 * Write a program to find the nth super ugly number.
 * Super ugly numbers are positive numbers whose all prime factors
 * are in the given prime list primes of size k.
 * For example, [1, 2, 4, 7, 8, 13, 14, 16, 19, 26, 28, 32]
 * is the sequence of the first 12 super ugly numbers given primes = [2, 7, 13, 19] of
size 4.
 *
 * Note:
 * (1) 1 is a super ugly number for any given primes.
 * (2) The given numbers in primes are in ascending order.
 * (3) 0 < k ≤ 100, 0 < n ≤ 106, 0 < primes[i] < 1000.
 * (4) The nth super ugly number is guaranteed to fit in a 32-bit signed integer.
 */
public class _313 {

    public static class Solution1 {
        public int nthSuperUglyNumber(int n, int[] primes) {
            int[] ret = new int[n];
            ret[0] = 1;

            int[] indexes = new int[primes.length];

            for (int i = 1; i < n; i++) {
                ret[i] = Integer.MAX_VALUE;

                for (int j = 0; j < primes.length; j++) {
                    ret[i] = Math.min(ret[i], primes[j] * ret[indexes[j]]);
                }

                for (int j = 0; j < indexes.length; j++) {
                    if (ret[i] == primes[j] * ret[indexes[j]]) {
                        indexes[j]++;
                    }
                }
            }
            return ret[n - 1];
        }
    }

}
```

297. `_314.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.TreeMap;

/**
 * 314. Binary Tree Vertical Order Traversal
 *
 * Given a binary tree, return the vertical order traversal of its nodes' values. (ie,
from top to bottom, column by column).

 If two nodes are in the same row and column, the order should be from left to right.

 Examples:

 Given binary tree [3,9,20,null,null,15,7],
    3
   /\
  /  \
 9  20
    /\
   /  \
  15   7
 return its vertical order traversal as:
 [
 [9],
 [3,15],
 [20],
 [7]
 ]

 Given binary tree [3,9,8,4,0,1,7],
    3
   /\
  /  \
 9   8
/\   /\
/  \/  \
4  01   7
 return its vertical order traversal as:
 [
 [4],
 [9],
 [3,0,1],
 [8],
```

```
 [7]
 ]

 Given binary tree [3,9,8,4,0,1,7,null,null,null,2,5] (0's right child is 2 and 1's left
child is 5),
      3
     /\
    /  \
   9   8
  /\  /\
 /  \/  \
4  01   7
   /\
  /  \
 5   2
 return its vertical order traversal as:
 [
 [4],
 [9,5],
 [3,0,1],
 [8,2],
 [7]
 ]
 */
public class _314 {
    public static class Solution1 {
        public List<List<Integer>> verticalOrder(TreeNode root) {
            List<List<Integer>> result = new ArrayList();
            if (root == null) {
                return result;
            }
            Queue<TreeNode> bfsQ = new LinkedList();
            Queue<Integer> indexQ = new LinkedList();
            TreeMap<Integer, List<Integer>> map = new TreeMap();
            bfsQ.offer(root);
            indexQ.offer(
                0);//we set the root as index 0, left will be negative, right will be
positive
            while (!bfsQ.isEmpty()) {
                int qSize = bfsQ.size();
                for (int i = 0; i < qSize; i++) {
                    TreeNode curr = bfsQ.poll();
                    int index = indexQ.poll();
                    if (map.containsKey(index)) {
                        map.get(index).add(curr.val);
                    } else if (!map.containsKey(index)) {
                        List<Integer> list = new ArrayList();
                        list.add(curr.val);
                        map.put(index, list);
                    }
                    if (curr.left != null) {
                        bfsQ.offer(curr.left);
                        indexQ.offer(index - 1);
                    }
```

```
                    if (curr.right != null) {
                        bfsQ.offer(curr.right);
                        indexQ.offer(index + 1);
                    }
                }
            }
            for (int i : map.keySet()) {
                result.add(map.get(i));
            }
            return result;
        }
    }

    public static class Solution2 {
        public List<List<Integer>> verticalOrder(TreeNode root) {
            List<List<Integer>> result = new ArrayList();
            if (root == null) {
                return result;
            }
            Queue<TreeNode> bfsQ = new LinkedList();
            Queue<Integer> indexQ = new LinkedList();
            HashMap<Integer, List<Integer>> map = new HashMap();
            bfsQ.offer(root);
            indexQ.offer(
                0);//we set the root as index 0, left will be negative, right will be
positive
            int min = 0;
            int max = 0;
            while (!bfsQ.isEmpty()) {
                int qSize = bfsQ.size();
                for (int i = 0; i < qSize; i++) {
                    TreeNode curr = bfsQ.poll();
                    int index = indexQ.poll();
                    if (map.containsKey(index)) {
                        map.get(index).add(curr.val);
                    } else if (!map.containsKey(index)) {
                        List<Integer> list = new ArrayList();
                        list.add(curr.val);
                        map.put(index, list);
                    }
                    if (curr.left != null) {
                        bfsQ.offer(curr.left);
                        indexQ.offer(index - 1);
                        min = Math.min(min, index - 1);
                    }
                    if (curr.right != null) {
                        bfsQ.offer(curr.right);
                        indexQ.offer(index + 1);
                        max = Math.max(max, index + 1);
                    }
                }
            }
            for (int i = min; i <= max; i++) {
                result.add(map.get(i));
```

```
            }
            return result;
        }
    }

}
```

298. `_315.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.List;

/**
 * 315. Count of Smaller Numbers After Self
 *
 * You are given an integer array nums and you have to return a new counts array.
 * The counts array has the property where counts[i] is the number of smaller elements to
the right of nums[i].

 Example:

 Given nums = [5, 2, 6, 1]

 To the right of 5 there are 2 smaller elements (2 and 1).
 To the right of 2 there is only 1 smaller element (1).
 To the right of 6 there is 1 smaller element (1).
 To the right of 1 there is 0 smaller element.
 Return the array [2, 1, 1, 0].

 */
public class _315 {

    public static class Solution1 {
        class Node {
            int val;
            int sum;
            int dup = 1;
            Node left;
            Node right;

            public Node(int v, int s) {
                this.val = v;
                this.sum = s;
            }
        }

        public List<Integer> countSmaller(int[] nums) {
            Integer[] ans = new Integer[nums.length];
            Node root = null;
            for (int i = nums.length - 1; i >= 0; i--) {
                root = insertNode(nums[i], root, i, 0, ans);
            }
            return Arrays.asList(ans);
        }

        Node insertNode(int val, Node node, int i, int prevSum, Integer[] ans) {
            if (node == null) {
                node = new Node(val, 0);
```

```
                ans[i] = prevSum;
            } else if (val == node.val) {
                node.dup += 1;
                ans[i] = prevSum + node.sum;
            } else if (val > node.val) {
                node.right = insertNode(val, node.right, i, prevSum + node.sum +
node.dup, ans);
            } else {
                node.sum += 1;
                node.left = insertNode(val, node.left, i, prevSum, ans);
            }

            return node;
        }
    }
}
```

299. `_316.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 316. Remove Duplicate Letters
 *
 * Given a string which contains only lowercase letters,
 * remove duplicate letters so that every letter appear once and only once.
 * You must make sure your result is the smallest in lexicographical order among all
possible results.

 Example:
 Given "bcabc"
 Return "abc"

 Given "cbacdcbc"
 Return "acdb"
 */
public class _316 {
    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/32259/java-solution-using-stack-
with-comments/2 */
        public String removeDuplicateLetters(String s) {
            int[] res = new int[26]; //will contain number of occurences of character
(i+'a')
            boolean[] visited =
                new boolean[26]; //will contain if character (i+'a') is present in
current result Stack
            char[] ch = s.toCharArray();
            for (char c : ch) {  //count number of occurences of character
                res[c - 'a']++;
            }
            Deque<Character> st = new ArrayDeque<>(); // answer stack
            int index;
            for (char c : ch) {
                index = c - 'a';
                res[index]--;   //decrement number of characters remaining in the string
to be analysed
                if (visited[index]) {
                    //if character is already present in stack, dont bother
                    continue;
                }
                //if current character is smaller than last character in stack which
occurs later in the string again
                //it can be removed and  added later e.g stack = bc remaining string abc
then a can pop b and then c
                while (!st.isEmpty() && c < st.peek() && res[st.peek() - 'a'] != 0) {
                    visited[st.pop() - 'a'] = false;
                }
```

```
                st.push(c); //add current character and mark it as visited
                visited[index] = true;
            }

            StringBuilder sb = new StringBuilder();
            //pop character from stack and build answer string from back
            while (!st.isEmpty()) {
                sb.insert(0, st.pop());
            }
            return sb.toString();
        }
    }

    public static class Solution2 {
        /**
         * Credit: https://discuss.leetcode.com/topic/31404/a-short-o-n-recursive-greedy-
solution
         */
        public String removeDuplicateLetters(String s) {
            int[] count = new int[26];
            int pos = 0; // the position for the smallest s[i]
            for (int i = 0; i < s.length(); i++) {
                count[s.charAt(i) - 'a']++;
            }
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) < s.charAt(pos)) {
                    pos = i;
                }
                count[s.charAt(i) - 'a']--;
                if (count[s.charAt(i) - 'a'] == 0) {
                    break;
                }
            }
            return s.length() == 0 ? "" : s.charAt(pos) + removeDuplicateLetters(
                s.substring(pos + 1).replaceAll("" + s.charAt(pos), ""));
        }
    }
}
```

300. `_317.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 317. Shortest Distance from All Buildings
 *
 * You want to build a house on an empty land which reaches all buildings in the shortest amount of distance.
 * You can only move up, down, left and right. You are given a 2D grid of values 0, 1 or 2, where:

 Each 0 marks an empty land which you can pass by freely.
 Each 1 marks a building which you cannot pass through.
 Each 2 marks an obstacle which you cannot pass through.

 For example, given three buildings at (0,0), (0,4), (2,2), and an obstacle at (0,2):

 1 - 0 - 2 - 0 - 1
 |   |   |   |   |
 0 - 0 - 0 - 0 - 0
 |   |   |   |   |
 0 - 0 - 1 - 0 - 0

 The point (1,2) is an ideal empty land to build a house, as the total travel distance of 3+3+1=7 is minimal. So return 7.

 Note:
 There will be at least one building. If it is not possible to build such house according to the above rules, return -1.
 */

public class _317 {
    public static class Solution1 {
        public int shortestDistance(int[][] grid) {
            int m = grid.length;
            if (m == 0) {
                return -1;
            }
            int n = grid[0].length;
            int[][] reach = new int[m][n];
            int[][] distance = new int[m][n];
            int[] shift = new int[] {0, 1, 0, -1,
                0};//how these five elements is ordered is important since it denotes the neighbor of the current node
            int numBuilding = 0;

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
```

```
                    numBuilding++;
                    int dist = 1;
                    boolean[][] visited = new boolean[m][n];

                    Queue<int[]> q = new LinkedList<int[]>();
                    q.offer(new int[] {i, j});
                    while (!q.isEmpty()) {
                        int size = q.size();
                        for (int l = 0; l < size; l++) {
                            int[] current = q.poll();
                            for (int k = 0; k < 4; k++) {
                                int nextRow = current[0] + shift[k];
                                int nextCol = current[1] + shift[k + 1];
                                if (nextRow >= 0
                                    && nextRow < m
                                    && nextCol >= 0
                                    && nextCol < n
                                    && !visited[nextRow][nextCol]
                                    && grid[nextRow][nextCol] == 0) {
                                    distance[nextRow][nextCol] += dist;
                                    visited[nextRow][nextCol] = true;
                                    reach[nextRow][nextCol]++;
                                    q.offer(new int[] {nextRow, nextCol});
                                }
                            }
                        }
                        dist++;
                    }
                }
            }
        }

        int result = Integer.MAX_VALUE;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 0 && reach[i][j] == numBuilding && distance[i][j] <
result) {
                    result = distance[i][j];
                }
            }
        }
        return result == Integer.MAX_VALUE ? -1 : result;
    }
}

}
```

301. `_318.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**318. Maximum Product of Word Lengths
 *
Given a string array words, find the maximum value of length(word[i]) * length(word[j])
where the two words do not share common letters. You may assume that each word will
contain only lower case letters. If no such two words exist, return 0.

Example 1:
Given ["abcw", "baz", "foo", "bar", "xtfn", "abcdef"]
Return 16
The two words can be "abcw", "xtfn".

Example 2:
Given ["a", "ab", "abc", "d", "cd", "bcd", "abcd"]
Return 4
The two words can be "ab", "cd".

Example 3:
Given ["a", "aa", "aaa", "aaaa"]
Return 0
No such pair of words.*/
public class _318 {
    public static class Solution1 {
        //Inspired by this awesome post: https://discuss.leetcode.com/topic/35539/java-
easy-version-to-understand
        //Idea: this question states that all words consisted of lower case (total only
26 unique chars),
        //this is a big hint that we could use integer (total 32 bits) to represent each
char
        //values[i] means how many unique characters this string words[i] has
        public int maxProduct(String[] words) {
            if (words == null || words.length == 0) {
                return 0;
            }
            int len = words.length;
            int[] values = new int[len];
            for (int i = 0; i < words.length; i++) {
                String word = words[i];
                for (int j = 0; j < words[i].length(); j++) {
                    values[i] |= 1 << (word.charAt(j)
                        - 'a');//the reason for left shift by this number "word.charAt(j)
-'a'" is for 'a', otherwise 'a' - 'a' will be zero and 'a' will be missed out.
                }
            }
            int maxProduct = 0;
            for (int i = 0; i < words.length; i++) {
```

```
                for (int j = 0; j < words.length; j++) {
                    //check if values[i] AND values[j] equals to zero, this means they
share NO common chars
                    if ((values[i] & values[j]) == 0
                        && words[i].length() * words[j].length() > maxProduct) {
                        maxProduct = words[i].length() * words[j].length();
                    }
                }
            }
            return maxProduct;
        }
    }

    public static void main(String... strings) {
        _318 test = new _318();
        String[] words = new String[]{"abcw", "baz", "foo", "bar", "xtfn", "abcdef"};

        //The following is to understand what does left shift by 1 mean:
        //the tricky part is to understand how it's written for me:
        // "x << y" means left shift x by y bits
        //left shift is equivalent to multiplication of powers of 2, so "4 << 1" equals
to " 4 * 2^1"
        //similarly, "4 << 3" equals to "4 * 2^3" which equals "4 * 8"
        String sample = "f";
        int bits = 0;
        int shiftLeftByHowMany = 0;
        int shiftLeftResult = 0;
        for (int j = 0; j < sample.length(); j++) {
            shiftLeftByHowMany = sample.charAt(j) - 'a';
            shiftLeftResult = 1 << shiftLeftByHowMany;
            bits |= 1 << (sample.charAt(j) - 'a');
            //this means shift left 1 by "sample.charAt(j) -'a'" bits
            System.out.println("nonShiftLeft = " + shiftLeftByHowMany + "\tnonShiftLeft
binary form is: " + Integer.toBinaryString(shiftLeftByHowMany)
                    + "\nshiftLeft = " + shiftLeftResult + "\tshiftLeft binary form is: "
+ Integer.toBinaryString(shiftLeftResult)
                    + "\nbits = " + bits + "\tbits binary form is: " +
Integer.toBinaryString(bits));
            System.out.println(shiftLeftResult == (1 * Math.pow(2, shiftLeftByHowMany)));
        }

        //similarly, right shift is written like this: "x >> y", means shift x by y bits
        //4 >> 3 equals 4 * 2^3, see below:
        System.out.println(4 * 8 == (4 * Math.pow(2, 3)));
    }
}
```

302. `_319.java`

```
package com.fishercoder.solutions;

/**
 * 319. Bulb Switcher
 *
 * There are n bulbs that are initially off. You first turn on all the bulbs.
 * Then, you turn off every second bulb. On the third round,
 * you toggle every third bulb (turning on if it's off or turning off if it's on).
 * For the ith round, you toggle every i bulb. For the nth round, you only
 * toggle the last bulb. Find how many bulbs are on after n rounds.

 Example:

 Given n = 3.

 At first, the three bulbs are [off, off, off].
 After first round, the three bulbs are [on, on, on].
 After second round, the three bulbs are [on, off, on].
 After third round, the three bulbs are [on, off, off].

 So you should return 1, because there is only one bulb is on.
 */
public class _319 {

    public static class Solution1 {
        public int bulbSwitch(int n) {
            if (n < 2) {
                return n;
            }
            return (int) Math.sqrt(n);
        }
    }

}
```

303. `_320.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 320. Generalized Abbreviation
 *
 * Write a function to generate the generalized abbreviations of a word.

 Example:

 Given word = "word", return the following list (order does not matter):
 ["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1",
"1o2", "2r1", "3d", "w3", "4"]
 */
public class _320 {

    public static class Solution1 {
        public List<String> generateAbbreviations(String word) {
            List<String> result = new ArrayList<>();
            backtrack(word, result, 0, "", 0);
            return result;
        }

        private void backtrack(String word, List<String> result, int position, String
current,
            int count) {
            if (position == word.length()) {
                if (count > 0) {
                    current += count;
                }
                result.add(current);
            } else {
                backtrack(word, result, position + 1, current, count + 1);
                backtrack(word, result, position + 1,
                    current + (count > 0 ? count : "") + word.charAt(position), 0);
            }
        }
    }

}
```

304. `_321.java`

```
package com.fishercoder.solutions;

/**
 * 321. Create Maximum Number
 *
 * Given two arrays of length m and n with digits 0-9 representing two numbers.
 * Create the maximum number of length k <= m + n from digits of the two.
 * The relative order of the digits from the same array must be preserved.
 * Return an array of the k digits. You should try to optimize your time and space
complexity.

 Example 1:
 nums1 = [3, 4, 6, 5]
 nums2 = [9, 1, 2, 5, 8, 3]
 k = 5
 return [9, 8, 6, 5, 3]

 Example 2:
 nums1 = [6, 7]
 nums2 = [6, 0, 4]
 k = 5
 return [6, 7, 6, 0, 4]

 Example 3:
 nums1 = [3, 9]
 nums2 = [8, 9]
 k = 3
 return [9, 8, 9]
 */
public class _321 {
    public static class Solution1 {
        public int[] maxNumber(int[] nums1, int[] nums2, int k) {
            int n = nums1.length;
            int m = nums2.length;
            int[] ans = new int[k];
            for (int i = Math.max(0, k - m); i <= k && i <= n; ++i) {
                //what is this and why?
                int[] candidate = merge(maxArray(nums1, i), maxArray(nums2, k - i), k);
                if (greater(candidate, 0, ans, 0)) {
                    ans = candidate;
                }
            }
            return ans;
        }

        private boolean greater(int[] nums1, int i, int[] nums2, int j) {
            while (i < nums1.length && j < nums2.length && nums1[i] == nums2[j]) {
                i++;
                j++;
            }
            return j == nums2.length || (i < nums1.length && nums1[i] > nums2[j]);
```

```
        }

        private int[] merge(int[] nums1, int[] nums2, int k) {
            int[] ans = new int[k];
            for (int i = 0, j = 0, r = 0; r < k; r++) {
                ans[r] = greater(nums1, i, nums2, j) ? nums1[i++] : nums2[j++];
            }
            return ans;
        }

        private int[] maxArray(int[] nums, int k) {
            int n = nums.length;
            int[] ans = new int[k];
            for (int i = 0, j = 0; i < n; i++) {
                while (n - i + j > k && j > 0 && ans[j - 1] < nums[i]) {
                    j--;
                }
                if (j < k) {
                    ans[j++] = nums[i];
                }
            }
            return ans;
        }
    }
}
```

305. `_322.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 322. Coin Change
 *
 * You are given coins of different denominations and a total amount of money amount.
 * Write a function to compute the fewest number of coins that you need to make up that
amount.
 * If that amount of money cannot be made up by any combination of the coins, return -1.

 Example 1:
 coins = [1, 2, 5], amount = 11
 return 3 (11 = 5 + 5 + 1)

 Example 2:
 coins = [2], amount = 3
 return -1.

 Note:
 You may assume that you have an infinite number of each kind of coin.
 */
public class _322 {

    public static class Solution1 {
        public int coinChange(int[] coins, int amount) {
            if (amount < 1) {
                return 0;
            }
            int[] count = new int[amount];
            int result = helper(coins, amount, count);
            return result;
        }

        //remaining means the remaining coins after the last step;
        //count[remaining] means the minimum number of coins to sum up to remaining
        private int helper(int[] coins, int remaining, int[] count) {
            if (remaining < 0) {
                return -1;//not valid case, thus, per problem description, we should
return -1
            }
            if (remaining == 0) {
                return 0;//completed, this is also a base case for this recursive
function
            }
            if (count[remaining - 1] != 0) {
                return count[remaining - 1];//already computed, so reuse it.
            }
            int min = Integer.MAX_VALUE;
            for (int coin : coins) {
```

```
                int res = helper(coins, remaining - coin, count);
                if (res >= 0 && res < min) {
                    min = 1 + res;
                }
            }
            return count[remaining - 1] = (min == Integer.MAX_VALUE) ? -1 : min;
        }
    }

    public static class Solution2 {
        //dp solution
        public int coinChange(int[] coins, int amount) {
            int max = amount + 1;
            int[] dp = new int[max];
            Arrays.fill(dp, max);// initial the dp array with amount + 1 which is not
valid case.
            dp[0] = 0;//initial amount 0 = 0;
            for (int i = 1; i <= amount; i++) {
                for (int j = 0; j < coins.length; j++) {
                    if (coins[j] <= i) {
                        //the dp[coins[j]] will ba a valid case, then if dp[i - coins[j]]
is  valid
                        dp[i] = Math.min(dp[i], dp[i - coins[j]] + 1);
                        // then we update dp[i], otherwise dp[i] = max;
                    }
                }
            }
            return dp[amount] > amount ? -1 : dp[amount];
        }
    }

}
```

306. `_323.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 323. Number of Connected Components in an Undirected Graph
 *
 *  Given n nodes labeled from 0 to n - 1 and a list of undirected edges (each edge is a
pair of nodes), write a function to find the number of connected components in an
undirected graph.

 Example 1:

 0          3
 |          |
 1 --- 2    4

 Given n = 5 and edges = [[0, 1], [1, 2], [3, 4]], return 2.

 Example 2:

 0           4
 |           |
 1 --- 2 --- 3

 Given n = 5 and edges = [[0, 1], [1, 2], [2, 3], [3, 4]], return 1.

 Note:
 You can assume that no duplicate edges will appear in edges. Since all edges are
undirected, [0, 1] is the same as [1, 0] and thus will not appear together in edges.
 */
public class _323 {
    public static class Solution1 {

        public int countComponents(int n, int[][] edges) {
            if (n <= 1) {
                return n;
            }

            List<List<Integer>> adList = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                adList.add(new ArrayList<>());
            }

            for (int[] edge : edges) {
                adList.get(edge[0]).add(edge[1]);
                adList.get(edge[1]).add(edge[0]);
            }
```

```
            for (List<Integer> list : adList) {
                for (int i : list) {
                    System.out.print(i + ", ");
                }
                System.out.println();
            }

            boolean[] visited = new boolean[n];
            int count = 0;
            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    count++;
                    Queue<Integer> q = new LinkedList<>();
                    q.offer(i);
                    while (!q.isEmpty()) {
                        int index = q.poll();
                        visited[index] = true;
                        for (int j : adList.get(index)) {
                            if (!visited[j]) {
                                q.offer(j);
                            }
                        }
                    }
                }
            }

            return count;
        }
    }

}
```

307. `_324.java`

```
package com.fishercoder.solutions;

import java.util.Collections;
import java.util.PriorityQueue;

/**
 * 324. Wiggle Sort II
 *
 * Given an unsorted array nums, reorder it such that nums[0] < nums[1] > nums[2] <
nums[3]....

 Example:
 (1) Given nums = [1, 5, 1, 1, 6, 4], one possible answer is [1, 4, 1, 5, 1, 6].
 (2) Given nums = [1, 3, 2, 2, 3, 1], one possible answer is [2, 3, 1, 3, 1, 2].

 Note:
 You may assume all input has valid answer.

 Follow Up:
 Can you do it in O(n) time and/or in-place with O(1) extra space?
 */
public class _324 {

    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/41464/step-by-step-explanation-of-
index-mapping-in-java */
        public void wiggleSort(int[] nums) {
            int median = findKthLargest(nums, (nums.length + 1) / 2);
            int n = nums.length;

            int left = 0;
            int i = 0;
            int right = n - 1;

            while (i <= right) {

                if (nums[newIndex(i, n)] > median) {
                    swap(nums, newIndex(left++, n), newIndex(i++, n));
                } else if (nums[newIndex(i, n)] < median) {
                    swap(nums, newIndex(right--, n), newIndex(i, n));
                } else {
                    i++;
                }
            }
        }

        private int findKthLargest(int[] nums, int k) {
            PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
            for (int i : nums) {
                maxHeap.offer(i);
```

```
            }
            while (k-- > 1) {
                maxHeap.poll();
            }
            return maxHeap.poll();
        }

        private void swap(int[] nums, int i, int j) {
            int tmp = nums[i];
            nums[i] = nums[j];
            nums[j] = tmp;
        }

        private int newIndex(int index, int n) {
            return (1 + 2 * index) % (n | 1);
        }
    }
}
```

308. `_325.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 325. Maximum Size Subarray Sum Equals k
 *
 * Given an array nums and a target value k, find the maximum length of a subarray that
sums to k. If there isn't one, return 0 instead.
 *
 * Note:
 * The sum of the entire nums array is guaranteed to fit within the 32-bit signed integer
range.
 *
 * Example 1:
 * Given nums = [1, -1, 5, -2, 3], k = 3,
 * return 4. (because the subarray [1, -1, 5, -2] sums to 3 and is the longest)
 *
 * Example 2:
 * Given nums = [-2, -1, 2, 1], k = 1,
 * return 2. (because the subarray [-1, 2] sums to 1 and is the longest)
 *
 * Follow Up:
 * Can you do it in O(n) time?*/

public class _325 {

    public static class Solution1 {
        public int maxSubArrayLen(int[] nums, int k) {
            Map<Integer, Integer> map = new HashMap();
            int sum = 0;
            int max = 0;
            for (int i = 0; i < nums.length; i++) {
                sum += nums[i];
                if (sum == k) {
                    max = i + 1;
                } else if (map.containsKey(sum - k)) {
                    max = Math.max(max, i - map.get(sum - k));
                }
                if (!map.containsKey(sum)) {
                    map.put(sum, i);
                }
            }
            return max;
        }
    }

}
```

309. `_326.java`

```
package com.fishercoder.solutions;

/**
 * 326. Power of Three
 *
 * Given an integer, write a function to determine if it is a power of three.
 *
 * Follow up:
 * Could you do it without using any loop / recursion?
*/

public class _326 {
    public static class Solution1 {
        //regular method that has a loop
        public boolean isPowerOfThree(int n) {
            if (n < 3 && n != 1) {
                return false;
            }
            while (n != 1) {
                if (n % 3 != 0) {
                    return false;
                }
                n /= 3;
            }
            return true;
        }
    }

    public static class Solution2 {
        //find the max possible integer that is a power of 3, then do modulor with this
number
        public boolean isPowerOfThree(int n) {
            return (n > 0 && 1162261467 % n == 0);
        }
    }

    public static class Solution3 {
        //Editorial solution: it's pretty elegant to use base conversion which can be
easily extended to any radix k
        //Idea: for a number in base 10, if it's power of 10, then it must be in this
format: 10, 100, 1000... with a leading one and all trailing zeros
        //similarly, if a number is power of 3, then in its base 3 format, it must be in
this format as well: 10, 100, 1000, 1000...
        //some Java built-in function could help us along the way:
        public boolean isPowerOfThree(int n) {
            return Integer.toString(n, 3).matches("^10*$");
        }
    }

}
```

310. `_327.java`

```
package com.fishercoder.solutions;

/**
 * 327. Count of Range Sum
 *
 * Given an integer array nums, return the number of range sums that lie in [lower,
upper] inclusive.
 * Range sum S(i, j) is defined as the sum of the elements in nums between indices i and
j (i ≤ j), inclusive.

 Note:
 A naive algorithm of O(n2) is trivial. You MUST do better than that.

 Example:
 Given nums = [-2, 5, -1], lower = -2, upper = 2, Return 3.
 The three ranges are : [0, 0], [2, 2], [0, 2] and their respective sums are: -2, -1, 2.
 */
public class _327 {

    public static class Solution1 {
        /**
         * Time: O(n^2)
         * This results in TLE on Leetcode by the last test case.
         */
        public int countRangeSum(int[] nums, int lower, int upper) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            long[] sums = new long[nums.length];
            sums[0] = nums[0];
            for (int i = 1; i < nums.length; i++) {
                sums[i] = sums[i - 1] + nums[i];
            }
            int count = 0;
            for (int i = 0; i < nums.length; i++) {
                if (nums[i] <= upper && nums[i] >= lower) {
                    count++;
                }
                for (int j = i + 1; j < nums.length; j++) {
                    long sum = sums[j] - (i > 0 ? sums[i - 1] : 0);
                    if (sum <= upper && sum >= lower) {
                        count++;
                    }
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        public int countRangeSum(int[] nums, int lower, int upper) {
```

```
            int n = nums.length;
            long[] sums = new long[n + 1];
            for (int i = 0; i < n; i++) {
                sums[i + 1] = sums[i] + nums[i];
            }
            return countWhileMergeSort(sums, 0, n + 1, lower, upper);
        }

        private int countWhileMergeSort(long[] sums, int start, int end, int lower, int
upper) {
            if (end - start <= 1) {
                return 0;
            }
            int mid = (start + end) / 2;
            int count = countWhileMergeSort(sums, start, mid, lower, upper) +
countWhileMergeSort(sums, mid, end, lower, upper);
            int j = mid;
            int k = mid;
            int t = mid;
            long[] cache = new long[end - start];
            for (int i = start, r = 0; i < mid; i++, r++) {
                while (k < end && sums[k] - sums[i] < lower) {
                    k++;
                }
                while (j < end && sums[j] - sums[i] <= upper) {
                    j++;
                }
                while (t < end && sums[t] < sums[i]) {
                    cache[r++] = sums[t++];
                }
                cache[r] = sums[i];
                count += j - k;
            }
            System.arraycopy(cache, 0, sums, start, t - start);
            return count;
        }
    }

}
```

311. `_328.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 328. Odd Even Linked List
 *
 * Given a singly linked list, group all odd nodes together followed by the even nodes.
Please note here we are talking about the node number and not the value in the nodes.

 You should try to do it in place. The program should run in O(1) space complexity and
O(nodes) time complexity.

 Example:
 Given 1->2->3->4->5->NULL,
 return 1->3->5->2->4->NULL.

 Note:
 The relative order inside both the even and odd groups should remain as it was in the
input.
 The first node is considered odd, the second node even and so on ...
 */
public class _328 {

    public static class Solution1 {
        public ListNode oddEvenList(ListNode head) {
            if (head != null) {
                ListNode odd = head;
                ListNode even = head.next;
                ListNode evenHead = even;

                while (even != null && even.next != null) {
                    odd.next = odd.next.next;
                    even.next = even.next.next;
                    odd = odd.next;
                    even = even.next;
                }

                odd.next = evenHead;
            }
            return head;
        }
    }
}
```

312. `_329.java`

```
package com.fishercoder.solutions;

/**
 * 329. Longest Increasing Path in a Matrix
 *
 * Given an integer matrix, find the length of the longest increasing path.
 * From each cell, you can either move to four directions: left, right, up or down.
 * You may NOT move diagonally or move outside of the boundary (i.e. wrap-around is not
allowed).

 Example 1:

 nums = [
 [9,9,4],
 [6,6,8],
 [2,1,1]
 ]
 Return 4
 The longest increasing path is [1, 2, 6, 9].

 Example 2:

 nums = [
 [3,4,5],
 [3,2,6],
 [2,2,1]
 ]
 Return 4
 The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.
 */

public class _329 {

    public static class Solution1 {
        final int[] dirs = new int[]{0, 1, 0, -1, 0};

        public int longestIncreasingPath(int[][] matrix) {
            if (matrix == null || matrix.length == 0) {
                return 0;
            }
            int max = 0;
            int[][] cache = new int[matrix.length][matrix[0].length];
            for (int i = 0; i < matrix.length; i++) {
                for (int j = 0; j < matrix[0].length; j++) {
                    int len = dfs(matrix, i, j, cache);
                    max = Math.max(len, max);
                }
            }
            return max;
        }
```

```
        int dfs(int[][] matrix, int row, int col, int[][] cache) {
            if (cache[row][col] != 0) {
                return cache[row][col];
            }
            int max = 1;
            for (int i = 0; i < 4; i++) {
                int nextRow = row + dirs[i];
                int nextCol = col + dirs[i + 1];
                if (nextRow < 0 || nextRow >= matrix.length || nextCol < 0 || nextCol >=
matrix[0].length || matrix[nextRow][nextCol] <= matrix[row][col]) {
                    continue;
                }
                int len = 1 + dfs(matrix, nextRow, nextCol, cache);
                max = Math.max(max, len);
            }
            cache[row][col] = max;
            return max;
        }
    }

}
```

313. `_330.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 330. Patching Array
 *
 * Given a sorted positive integer array nums and an integer n,
 * add/patch elements to the array such that any number in range [1, n]
 * inclusive can be formed by the sum of some elements in the array. Return the minimum
number of patches required.

 Example 1:
 nums = [1, 3], n = 6
 Return 1.

 Combinations of nums are [1], [3], [1,3], which form possible sums of: 1, 3, 4.
 Now if we add/patch 2 to nums, the combinations are: [1], [2], [3], [1,3], [2,3],
[1,2,3].
 Possible sums are 1, 2, 3, 4, 5, 6, which now covers the range [1, 6].
 So we only need 1 patch.

 Example 2:
 nums = [1, 5, 10], n = 20
 Return 2.
 The two patches can be [2, 4].

 Example 3:
 nums = [1, 2, 2], n = 5
 Return 0.
 */
public class _330 {

    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/patching-array/ and
https://discuss.leetcode.com/topic/35494/solution-explanation/2
         *
         * Let miss be the smallest sum in [0,n] that we might be missing. Meaning we
already know we
         * can build all sums in [0,miss). Then if we have a number num <= miss in the
given array, we
         * can add it to those smaller sums to build all sums in [0,miss+num). If we
don't, then we must
         * add such a number to the array, and it's best to add miss itself, to maximize
the reach.
         *
         * Example: Let's say the input is nums = [1, 2, 4, 13, 43] and n = 100. We need
to ensure that
         * all sums in the range [1,100] are possible. Using the given numbers 1, 2 and
```

```
4, we can
         * already build all sums from 0 to 7, i.e., the range [0,8). But we can't build
the sum 8, and
         * the next given number (13) is too large. So we insert 8 into the array. Then
we can build all
         * sums in [0,16). Do we need to insert 16 into the array? No! We can already
build the sum 3,
         * and adding the given 13 gives us sum 16. We can also add the 13 to the other
sums, extending
         * our range to [0,29). And so on. The given 43 is too large to help with sum 29,
so we must
         * insert 29 into our array. This extends our range to [0,58). But then the 43
becomes useful
         * and expands our range to [0,101). At which point we're done.
         */

        public int minPatches(int[] nums, int n) {
            long misses = 1;//use long to avoid integer addition overflow
            int patches = 0;
            int i = 0;
            while (misses <= n) {
                if (i < nums.length && nums[i] <= misses) { //miss is covered
                    misses += nums[i++];
                } else { //patch miss to the array
                    misses += misses;
                    patches++;//increase the answer
                }
            }
            return patches;
        }

        public List<Integer> findPatches(int[] nums, int n) {
            long misses = 1;//use long to avoid integer addition overflow
            List<Integer> patches = new ArrayList<>();
            int i = 0;
            while (misses <= n) {
                if (i < nums.length && nums[i] <= misses) { //miss is covered
                    misses += nums[i++];
                } else { //patch miss to the array
                    patches.add((int) misses);//increase the answer
                    misses += misses;
                }
            }
            return patches;
        }
    }

}
```

314. `_331.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 331. Verify Preorder Serialization of a Binary Tree
 *
 * One way to serialize a binary tree is to use pre-order traversal. When we encounter a
non-null node, we record the node's value. If it is a null node, we record using a
sentinel value such as #.

       _9_
      /    \
     3     2
    / \   / \
   4  1  #  6
  / \ / \  / \
  # # # #  # #

 For example, the above binary tree can be serialized to the string
"9,3,4,#,#,1,#,#,2,#,6,#,#", where # represents a null node.

 Given a string of comma separated values, verify whether it is a correct preorder
traversal serialization of a binary tree. Find an algorithm without reconstructing the
tree.

 Each comma separated value in the string must be either an integer or a character '#'
representing null pointer.

 You may assume that the input format is always valid, for example it could never contain
two consecutive commas such as "1,,3".

 Example 1:
 "9,3,4,#,#,1,#,#,2,#,6,#,#"
 Return true

 Example 2:
 "1,#"
 Return false

 Example 3:
 "9,#,#,1"
 Return false


 */
public class _331 {

    public static class Solution1 {
        public boolean isValidSerialization(String preorder) {
```

```
            /**Idea: we keep inserting the string into the stack, if it's a number, we
just push it onto the stack;
             * if it's a "#", we see if the top of the stack is a "#" or not,
             * 1. if it's a "#", we pop it and keep popping numbers from the stack,
             * 2. if it's not a "#", we push the "#" onto the stack*/
            if (preorder == null || preorder.length() == 0) {
                return false;
            }
            String[] pre = preorder.split(",");
            Deque<String> stack = new ArrayDeque<>();
            for (int i = 0; i < pre.length; i++) {
                while (pre[i].equals("#") && !stack.isEmpty() &&
stack.peekLast().equals("#")) {
                    stack.pollLast();
                    if (stack.isEmpty()) {
                        return false;
                    }
                    stack.pollLast();
                }
                stack.addLast(pre[i]);
            }
            return stack.size() == 1 && stack.peekLast().equals("#");
        }
    }

}
```

315. `_332.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * 332. Reconstruct Itinerary
 *
 * Given a list of airline tickets represented by pairs of departure and arrival airports
[from, to],
 * reconstruct the itinerary in order. All of the tickets belong to a man who departs
from JFK. Thus, the itinerary must begin with JFK.

 Note:
 If there are multiple valid itineraries, you should return the itinerary that has the
smallest lexical order when read as a single string.
 For example, the itinerary ["JFK", "LGA"] has a smaller lexical order than ["JFK",
"LGB"].

 All airports are represented by three capital letters (IATA code).

 You may assume all tickets form at least one valid itinerary.
 Example 1:
 tickets = [["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]
 Return ["JFK", "MUC", "LHR", "SFO", "SJC"].

 Example 2:
 tickets = [["JFK","SFO"],["JFK","ATL"],["SFO","ATL"],["ATL","JFK"],["ATL","SFO"]]
 Return ["JFK","ATL","JFK","SFO","ATL","SFO"].
 Another possible reconstruction is ["JFK","SFO","ATL","JFK","ATL","SFO"]. But it is
larger in lexical order.
 */
public class _332 {

    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/36383/share-my-solution */
        public List<String> findItinerary(String[][] tickets) {
            Map<String, PriorityQueue<String>> flights = new HashMap<>();
            LinkedList<String> path = new LinkedList<>();
            for (String[] ticket : tickets) {
                flights.putIfAbsent(ticket[0], new PriorityQueue<>());
                flights.get(ticket[0]).add(ticket[1]);
            }
            dfs("JFK", flights, path);
            return path;
```

```
        }

        public void dfs(String departure, Map<String, PriorityQueue<String>> flights,
            LinkedList path) {
            PriorityQueue<String> arrivals = flights.get(departure);
            while (arrivals != null && !arrivals.isEmpty()) {
                dfs(arrivals.poll(), flights, path);
            }
            path.addFirst(departure);
        }
    }
}
```

316. `_333.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**333. Largest BST Subtree
 *
 * Given a binary tree, find the largest subtree which is a Binary Search Tree (BST),
 * where largest means subtree with largest number of nodes in it.

 Note:
 A subtree must include all of its descendants.
 Here's an example:
      10
     / \
    5  15
   / \   \
  1  8   7

 The Largest BST Subtree in this case is the highlighted one (5,1,8).
 The return value is the subtree's size, which is 3.

 Follow up:
 Can you figure out ways to solve it with O(n) time complexity?
 */

public class _333 {
    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/36995/share-my-o-n-java-code-with-
brief-explanation-and-comments*/
        class Result {  // (size, rangeLower, rangeUpper) -- size of current tree, range
of current tree [rangeLower, rangeUpper]
            int size;
            int lower;
            int upper;

            Result(int size, int lower, int upper) {
                this.size = size;
                this.lower = lower;
                this.upper = upper;
            }
        }

        int max = 0;

        public int largestBSTSubtree(TreeNode root) {
            if (root == null) {
                return 0;
            }
            traverse(root);
            return max;
        }
```

```
        private Result traverse(TreeNode root) {
            if (root == null) {
                return new Result(0, Integer.MAX_VALUE, Integer.MIN_VALUE);
            }
            Result left = traverse(root.left);
            Result right = traverse(root.right);
            if (left.size == -1 || right.size == -1 || root.val <= left.upper || root.val
>= right.lower) {
                return new Result(-1, 0, 0);
            }
            int size = left.size + 1 + right.size;
            max = Math.max(size, max);
            return new Result(size, Math.min(left.lower, root.val), Math.max(right.upper,
root.val));
        }
    }

    public int largestBSTSubtree(TreeNode root) {
        if (root == null) {
            return 0;
        }
        if (isBST(root)) {
            return getNodes(root);
        }
        return Math.max(find(root.left), find(root.right));
    }

    int find(TreeNode root) {
        if (isBST(root)) {
            return getNodes(root);
        }
        return Math.max(find(root.left), find(root.right));
    }

    int getNodes(TreeNode root) {
        if (root == null) {
            return 0;
        }
        return dfsCount(root);
    }

    int dfsCount(TreeNode root) {
        if (root == null) {
            return 0;
        }
        return dfsCount(root.left) + dfsCount(root.right) + 1;
    }

    boolean isBST(TreeNode root) {
        if (root == null) {
            return true;
        }
        return dfs(root, Long.MIN_VALUE, Long.MAX_VALUE);
```

```
    }

    boolean dfs(TreeNode root, long min, long max) {
        if (root == null) {
            return true;
        }
        if (root.val <= min || root.val >= max) {
            return false;
        }
        return dfs(root.left, min, root.val) && dfs(root.right, root.val, max);
    }

}
```

317. `_334.java`

```
package com.fishercoder.solutions;

/**
 * 334. Increasing Triplet Subsequence
 *
 * Given an unsorted array return whether an increasing subsequence of length 3 exists or
not in the array.
 *
 * Formally the function should:
 * Return true if there exists i, j, k
 * such that arr[i] < arr[j] < arr[k] given 0 ≤ i < j < k ≤ n-1 else return false.
 * Your algorithm should run in O(n) time complexity and O(1) space complexity.

 Examples:
 Given [1, 2, 3, 4, 5],
 return true.

 Given [5, 4, 3, 2, 1],
 return false.
 */
public class _334 {

    public static class Solution1 {
        /**Time: O(n^2)
         * Space: O(1)*/
        public boolean increasingTriplet(int[] nums) {
            if (nums == null || nums.length == 0) {
                return false;
            }
            int small = nums[0];
            int medium = Integer.MAX_VALUE;
            for (int i = 1; i < nums.length; i++) {
                small = Math.min(small, nums[i - 1]);
                if (nums[i] > small) {
                    medium = Math.min(medium, nums[i]);
                    for (int j = i + 1; j < nums.length; j++) {
                        if (nums[j] > nums[i] || nums[j] > medium) {
                            return true;
                        }
                    }
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(n)
         * Space: O(1)
         */
```

```
        public boolean increasingTriplet(int[] nums) {
            int small = Integer.MAX_VALUE;
            int big = Integer.MAX_VALUE;
            for (int num : nums) {
                if (num <= small) {
                    small = num;
                } else if (num <= big) {
                    big = num;
                } else {
                    return true;
                }
            }
            return false;
        }
    }

}
```

318. `_335.java`

```
package com.fishercoder.solutions;

/**
 * 335. Self Crossing
 *
 * You are given an array x of n positive numbers.
 * You start at point (0,0) and moves x[0] metres to the north,
 * then x[1] metres to the west,
 * x[2] metres to the south,
 * x[3] metres to the east and so on.
 * In other words, after each move your direction changes counter-clockwise.

 Write a one-pass algorithm with O(1) extra space to determine, if your path crosses
itself, or not.

 Example 1:
 Given x =
 [2, 1, 1, 2]
 ,
 ?????
 ?   ?
 ???????>
 ?

 Return true (self crossing)
 Example 2:
 Given x =
 [1, 2, 3, 4]
 ,
 ????????
 ?      ?
 ?
 ?
 ?????????????>

 Return false (not self crossing)
 Example 3:
 Given x =
 [1, 1, 1, 1]
 ,
 ?????
 ?   ?
 ?????>

 Return true (self crossing)

 */
public class _335 {
    public static class Solution1 {
        /** reference: https://discuss.leetcode.com/topic/38014/java-oms-with-
explanation/2 */
```

```
        public boolean isSelfCrossing(int[] x) {
            int l = x.length;
            if (l <= 3) {
                return false;
            }

            for (int i = 3; i < l; i++) {
                if (x[i] >= x[i - 2] && x[i - 1] <= x[i - 3]) {
                    return true;  //Fourth line crosses first line and onward
                }
                if (i >= 4) {
                    if (x[i - 1] == x[i - 3] && x[i] + x[i - 4] >= x[i - 2]) {
                        return true; // Fifth line meets first line and onward
                    }
                }
                if (i >= 5) {
                    if (x[i - 2] - x[i - 4] >= 0
                        && x[i] >= x[i - 2] - x[i - 4]
                        && x[i - 1] >= x[i - 3] - x[i - 5]
                        && x[i - 1] <= x[i - 3]) {
                        return true;  // Sixth line crosses first line and onward
                    }
                }
            }
            return false;
        }
    }
}
```

319. `_336.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**336. Palindrome Pairs
 *
 * Given a list of unique words, find all pairs of distinct indices (i, j) in the given
list, so that the concatenation of the two words, i.e. words[i] + words[j] is a
palindrome.

 Example 1:
 Given words = ["bat", "tab", "cat"]
 Return [[0, 1], [1, 0]]
 The palindromes are ["battab", "tabbat"]
 Example 2:
 Given words = ["abcd", "dcba", "lls", "s", "sssll"]
 Return [[0, 1], [1, 0], [3, 2], [2, 4]]
 The palindromes are ["dcbaabcd", "abcddcba", "slls", "llssssll"]
 */
public class _336 {

    public static class Solution1 {
        public List<List<Integer>> palindromePairs(String[] words) {
            List<List<Integer>> pairs = new ArrayList();
            Map<String, Integer> map = new HashMap();
            for (int i = 0; i < words.length; i++) {
                map.put(words[i], i);
            }

            for (int i = 0; i < words.length; i++) {
                int l = 0;
                int r = 0;
                while (l <= r) {
                    String s = words[i].substring(l, r);
                    Integer j = map.get(new StringBuilder(s).reverse().toString());
                    if (j != null && j != i && isPalindrome(
                        words[i].substring(l == 0 ? r : 0, l == 0 ? words[i].length() :
l))) {
                        pairs.add(
                            Arrays.asList(l == 0 ? new Integer[] {i, j} : new Integer[]
{j, i}));
                    }
                    if (r < words[i].length()) {
                        r++;
                    } else {
                        l++;
                    }
```

```
                }
            }
            return pairs;
        }

        private boolean isPalindrome(String s) {
            for (int i = 0; i < s.length() / 2; i++) {
                if (s.charAt(i) != s.charAt(s.length() - 1 - i)) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

320. `_337.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**337. House Robber III

 The thief has found himself a new place for his thievery again.
 There is only one entrance to this area, called the "root."
 Besides the root, each house has one and only one parent house.
 After a tour, the smart thief realized that "all houses in this place forms a binary
tree".
 It will automatically contact the police if two directly-linked houses were broken into
on the same night.

Determine the maximum amount of money the thief can rob tonight without alerting the
police.

Example 1:
     3
    / \
   2   3
    \   \
     3   1
Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.

Example 2:
     3
    / \
   4   5
  / \   \
 1   3   1
Maximum amount of money the thief can rob = 4 + 5 = 9.
*/

public class _337 {

    public static class Solution1 {
        //simple recursion without cacheing: 1189 ms
        public int rob(TreeNode root) {
            if (root == null) {
                return 0;
            }

            int val = 0;
            if (root.left != null) {
                val += rob(root.left.left) + rob(root.left.right);
            }
            if (root.right != null) {
```

```
                val += rob(root.right.left) + rob(root.right.right);
            }
            val = Math.max(val + root.val, rob(root.left) + rob(root.right));
            return val;
        }
    }

    public static class Solution2 {
        //same idea, but with cacheing via a hashmap: 8 ms
        public int rob_dp(TreeNode root) {
            Map<TreeNode, Integer> map = new HashMap<>();
            return getMaxValue(root, map);
        }

        private int getMaxValue(TreeNode root, Map<TreeNode, Integer> map) {
            if (root == null) {
                return 0;
            }
            if (map.containsKey(root)) {
                return map.get(root);
            }

            int val = 0;
            if (root.left != null) {
                val += getMaxValue(root.left.left, map) + getMaxValue(root.left.right,
map);
            }
            if (root.right != null) {
                val += getMaxValue(root.right.left, map) + getMaxValue(root.right.right,
map);
            }
            int max = Math.max(root.val + val,
                getMaxValue(root.left, map) + getMaxValue(root.right, map));
            map.put(root, max);
            return max;
        }
    }

}
```

321. `_338.java`

```
package com.fishercoder.solutions;

/**338. Counting Bits
Given a non negative integer number num. For every numbers i in the range 0 ≤ i ≤ num
calculate the number of 1's in their binary representation and return them as an array.

Example:
For num = 5 you should return [0,1,1,2,1,2].

Follow up:

It is very easy to come up with a solution with run time O(n*sizeof(integer)). But can
you do it in linear time O(n) /possibly in a single pass?
Space complexity should be O(n).
Can you do it like a boss? Do it without using any builtin function like
__builtin_popcount in c++ or in any other language.

Hint:

You should make use of what you have produced already.
Divide the numbers in ranges like [2-3], [4-7], [8-15] and so on. And try to generate new
range from previous.
Or does the odd/even status of the number help you in calculating the number of 1s?

*
*
*/
public class _338 {
    public static class Solution1 {
        //use the most regular method to get it AC'ed first
        public int[] countBits(int num) {
            int[] ones = new int[num + 1];
            for (int i = 0; i <= num; i++) {
                ones[i] = countOnes(i);
            }
            return ones;
        }

        private int countOnes(int i) {
            int ones = 0;
            while (i != 0) {
                ones++;
                i &= (i - 1);
            }
            return ones;
        }
    }

    private class Solution2 {
        /**lixx2100's post is cool:https://discuss.leetcode.com/topic/40162/three-line-
java-solution
```

```
        An easy recurrence for this problem is f[i] = f[i / 2] + i % 2
        and then we'll use bit manipulation to express the above recursion function
         right shift by 1 means to divide by 2
        AND with 1 means to modulo 2
        this is so cool!*/
        public int[] countBits(int num) {
            int[] ones = new int[num + 1];
            for (int i = 1; i <= num; i++) {
                ones[i] = ones[i >> 1] + (i & 1);
            }
            return ones;
        }
    }
}
```

322. `_339.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.List;

public class _339 {
  public static class Solution1 {
    private int sum = 0;

    public int depthSum(List<NestedInteger> nestedList) {
      return dfs(nestedList, 1);
    }

    private int dfs(List<NestedInteger> nestedList, int depth) {
      for (NestedInteger ni : nestedList) {
        if (ni.isInteger()) {
          sum += depth * ni.getInteger();
        } else {
          dfs(ni.getList(), depth + 1);
        }
      }
      return sum;
    }
  }

  public static class Solution2 {
    public int depthSum(List<NestedInteger> nestedList) {
      return dfs(nestedList, 1);
    }

    private int dfs(List<NestedInteger> nestedList, int depth) {
      int sum = 0;
      for (NestedInteger ni : nestedList) {
        sum += ni.isInteger() ? depth * ni.getInteger() : dfs(ni.getList(), depth + 1);
      }
      return sum;
    }
  }
}
```

323. `_340.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 340. Longest Substring with At Most K Distinct Characters
 *
 * Given a string, find the length of the longest substring T that contains at most k
distinct characters.

 For example, Given s = “eceba” and k = 2,

 T is "ece" which its length is 3.

 */
public class _340 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/41671/15-lines-java-solution-using-
slide-window
         */
        public int lengthOfLongestSubstringKDistinct(String s, int k) {
            int[] count = new int[256];
            int left = 0;
            int result = 0;
            int num = 0;
            for (int right = 0; right < s.length(); right++) {
                if (count[s.charAt(right)]++ == 0) {
                    num++;
                }
                if (num > k) {
                    while (--count[s.charAt(left++)] > 0) {};
                    num--;
                }
                result = Math.max(result, right - left + 1);
            }
            return result;
        }
    }

    public static class Solution2 {
        /**This is a more generic solution for any characters, not limited to ASCII
characters.*/
        public int lengthOfLongestSubstringKDistinct(String s, int k) {
            Map<Character, Integer> map = new HashMap<>();
            int longest = 0;
            int left = 0;
            for (int i = 0; i < s.length(); i++) {
                char c = s.charAt(i);
```

```
                map.put(c, map.getOrDefault(c, 0) + 1);
                while (map.size() > k) {
                    char leftChar = s.charAt(left);
                    if (map.containsKey(leftChar)) {
                        map.put(leftChar, map.get(leftChar) - 1);
                        if (map.get(leftChar) == 0) {
                            map.remove(leftChar);
                        }
                    }
                    left++;
                }
                longest = Math.max(longest, i - left + 1);
            }
            return longest;
        }
    }

}
```

324. `_341.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 341. Flatten Nested List Iterator
 *
 * Given a nested list of integers, implement an iterator to flatten it.
 * Each element is either an integer, or a list -- whose elements may also be integers or
other lists.
 *
 * Example 1:
 * Given the list [[1,1],2,[1,1]],
 * By calling next repeatedly until hasNext returns false, the order of elements returned
by next should be: [1,1,2,1,1].
 *
 * Example 2:
 * Given the list [1,[4,[6]]],
 * By calling next repeatedly until hasNext returns false, the order of elements returned
by next should be: [1,4,6].
 */
public class _341 {

    public static class Solution1 {
        public static class NestedIterator implements Iterator<Integer> {

            private Queue<Integer> flattenedList;

            public NestedIterator(List<NestedInteger> nestedList) {
                flattenedList = new LinkedList<>();
                constructList(nestedList);
            }

            private void constructList(List<NestedInteger> nestedList) {
                for (NestedInteger nestedInteger : nestedList) {
                    if (nestedInteger.isInteger()) {
                        flattenedList.add(nestedInteger.getInteger());
                    } else {
                        constructList(nestedInteger.getList());
                    }
                }
            }

            @Override
            public Integer next() {
                return flattenedList.poll();
```

```
            }

            @Override
            public boolean hasNext() {
                return !flattenedList.isEmpty();
            }
        }
    }

}
```

325. `_342.java`

```
package com.fishercoder.solutions;

/**
 * 342. Power of Four
 *
 * Given an integer (signed 32 bits), write a function to check whether it is a power of
4.
 *
 * Example:
 * Given num = 16, return true. Given num = 5, return false.
 * Follow up: Could you solve it without loops/recursion?
 * */
public class _342 {
    public static class Solution1 {
        //Just dive a little bit deeper, you can realize that another important feature
of a number
        //that is power of four is that its only single one bit must appear on the odd
position, and power of two won't meet this requirement
        //decimal number 8 has binary format: 0000-0000-0000-0000-0000-0000-0000-1000
        //decimal number 16 has binary format: 0000-0000-0000-0000-0000-0000-0001-0000
        //hex number 0x55555555 has binary format: 1010-1010-1010-1010-1010-1010-1010-
1010
        //thus, doing AND with 0x55555 will check if the only one bit is located on the
odd position, thus ruling out those that are power of 2 but not power of 4
        public boolean isPowerOfFour(int num) {
            return (num > 0 && 1073741824 % num == 0 && (num & 0x55555555) != 0);
        }
    }

    public static class Solution2 {
        public boolean isPowerOfFour(int num) {
            //^ means to match the beginning of a line
            //$ means to match the end of a line
            //* means zero or more of the preceding character
            return Integer.toString(num, 4).matches("^10*$");
        }
    }

    public static class Solution3 {
        //a regular loop method to make it AC'ed
        public boolean isPowerOfFour(int num) {
            if (num < 4 && num != 1) {
                return false;
            }
            while (num != 1) {
                if (num % 4 != 0) {
                    return false;
                }
                num /= 4;
            }
            return true;
```

```
        }
    }

}
```

326. `_343.java`

```
package com.fishercoder.solutions;

/**
 * 343. Integer Break
 *
 * Given a positive integer n, break it into the sum of at least two positive integers
and maximize the product of those integers. Return the maximum product you can get.

 For example, given n = 2, return 1 (2 = 1 + 1); given n = 10, return 36 (10 = 3 + 3 +
4).

 Note: You may assume that n is not less than 2 and not larger than 58.

 Hint:

 There is a simple O(n) solution to this problem.
 You may check the breaking results of n ranging from 7 to 10 to discover the
regularities.
 */
public class _343 {
    public static class Solution1 {
        public int integerBreak(int n) {
            if (n == 2) {
                return 1;
            } else if (n == 3) {
                return 2;
            } else if (n % 3 == 0) {
                return (int) Math.pow(3, n / 3);
            } else if (n % 3 == 1) {
                return 2 * 2 * (int) Math.pow(3, (n - 4) / 3);
            } else {
                return 2 * (int) Math.pow(3, n / 3);
            }
        }
    }
}
```

327. `_344.java`

```
package com.fishercoder.solutions;

/**
 * 344. Reverse String
 *
 * Write a function that takes a string as input and returns the string reversed.

 Example:
 Given s = "hello", return "olleh".
 */
public class _344 {
    public static class Solution1 {
        public String reverseString(String s) {
            int i = 0;
            int j = s.length() - 1;
            char[] chars = s.toCharArray();
            while (i < j) {
                char temp = chars[i];
                chars[i++] = chars[j];
                chars[j--] = temp;
            }
            return new String(chars);
        }
    }
}
```

328. `_345.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 345. Reverse Vowels of a String
 *
 * Write a function that takes a string as input and reverse only the vowels of a string.

 Example 1:
 Given s = "hello", return "holle".

 Example 2:
 Given s = "leetcode", return "leotcede".

 Note:
 The vowels does not include the letter "y".*/
public class _345 {
        public static class Solution1 {
                public String reverseVowels(String s) {
                        StringBuilder sb = new StringBuilder(s);
                        Set<Character> vowels = new HashSet(Arrays.asList('a', 'e', 'i',
'o', 'u', 'A', 'E', 'I', 'O', 'U'));
                        //use two pointers approach would be the fastest
                        int i = 0;
                        int j = s.length() - 1;
                        while (i < j) {
                                char left = s.charAt(i);
                                char right = s.charAt(j);
                                while (i < j && !vowels.contains(left)) {
                                        i++;
                                        left = s.charAt(i);
                                }
                                while (i < j && !vowels.contains(right)) {
                                        j--;
                                        right = s.charAt(j);
                                }
                                char temp = left;
                                sb.setCharAt(i, right);
                                sb.setCharAt(j, temp);
                                i++;
                                j--;
                        }
                        return sb.toString();
                }
        }
}
```

329. `_346.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 346. Moving Average from Data Stream
 *
 * Given a stream of integers and a window size, calculate the moving average of all
integers in the sliding window.

 For example,
 MovingAverage m = new MovingAverage(3);
 m.next(1) = 1
 m.next(10) = (1 + 10) / 2
 m.next(3) = (1 + 10 + 3) / 3
 m.next(5) = (10 + 3 + 5) / 3
 */
public class _346 {

    public static class Solution1 {
        class MovingAverage {

            private Queue<Integer> q;
            private Long sum;
            private int max;

            /**
             * Initialize your data structure here.
             */
            public MovingAverage(int size) {
                q = new LinkedList();
                sum = 0L;
                max = size;
            }

            public double next(int val) {
                if (q.size() < max) {
                    q.offer(val);
                    sum += val;
                    return (double) sum / q.size();
                } else {
                    int first = q.poll();
                    sum -= first;
                    q.offer(val);
                    sum += val;
                    return (double) sum / q.size();
                }
            }
        }
```

```
    }
}
```

330. `_347.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 347. Top K Frequent Elements
 *
 * Given a non-empty array of integers, return the k most frequent elements.

 For example,
 Given [1,1,1,2,2,3] and k = 2, return [1,2].

 Note:
 You may assume k is always valid, 1 ≤ k ≤ number of unique elements.
 Your algorithm's time complexity must be better than O(n log n), where n is the array's
size.*/

public class _347 {

        public static class Solution1 {
                /**
                 * Use buckets to hold numbers of the same frequency
     * It's averaged at 30 ms on Leetcode.
                 */
                public List<Integer> topKFrequent(int[] nums, int k) {
                        Map<Integer, Integer> map = new HashMap();
                        for (int i : nums) {
                                map.put(i, map.getOrDefault(i, 0) + 1);
                        }

                        ArrayList[] bucket = new ArrayList[nums.length + 1];
                        for (Entry<Integer, Integer> e : map.entrySet()) {
                                int frequency = e.getValue();
                                if (bucket[frequency] == null) {
                                        bucket[frequency] = new ArrayList<Integer>();
                                }
                                bucket[frequency].add(e.getKey());
                        }
                        List<Integer> result = new ArrayList<>();
                        for (int i = bucket.length - 1; i >= 0 && result.size() < k; i--)
{
                                if (bucket[i] != null) {
                                        for (int j = 0; j < bucket[i].size(); j++) {
                                                result.add((int) bucket[i].get(j));
                                        }
```

```
                                }
                        }

                        return result;
                }
        }

        public static class Solution2 {
                /**
                 * Use hashtable and heap, it's averaged at 100 ms on Leetocde.
                 */
                public List<Integer> topKFrequent(int[] nums, int k) {
                        // construct the frequency map first, and then iterate through
the map
                        // and put them into the heap, this is O(n)
                        Map<Integer, Integer> map = new HashMap();
                        for (int num : nums) {
                                map.put(num, map.getOrDefault(num, 0) + 1);
                        }

                        // build heap, this is O(logn)
                        Queue<Entry<Integer, Integer>> heap = new PriorityQueue<>((o1,
o2) -> o2.getValue() - o1.getValue());
                        for (Entry<Integer, Integer> entry : map.entrySet()) {
                                heap.offer(entry);
                        }

                        List<Integer> res = new ArrayList();
                        while (k-- > 0) {
                                res.add(heap.poll().getKey());
                        }
                        return res;
                }
        }
}
```

331. `_348.java`

```
package com.fishercoder.solutions;

/**
 * 348. Design Tic-Tac-Toe
 *
 * Design a Tic-tac-toe game that is played between two players on a n x n grid.

 You may assume the following rules:

 A move is guaranteed to be valid and is placed on an empty block.
 Once a winning condition is reached, no more moves is allowed.
 A player who succeeds in placing n of their marks in a horizontal, vertical, or diagonal
row wins the game.

 Example:

 Given n = 3, assume that player 1 is "X" and player 2 is "O" in the board.

 TicTacToe toe = new TicTacToe(3);

 toe.move(0, 0, 1); -> Returns 0 (no one wins)
 |X| | |
 | | | |    // Player 1 makes a move at (0, 0).
 | | | |

 toe.move(0, 2, 2); -> Returns 0 (no one wins)
 |X| |O|
 | | | |    // Player 2 makes a move at (0, 2).
 | | | |

 toe.move(2, 2, 1); -> Returns 0 (no one wins)
 |X| |O|
 | | | |    // Player 1 makes a move at (2, 2).
 | | |X|

 toe.move(1, 1, 2); -> Returns 0 (no one wins)
 |X| |O|
 | |O| |    // Player 2 makes a move at (1, 1).
 | | |X|

 toe.move(2, 0, 1); -> Returns 0 (no one wins)
 |X| |O|
 | |O| |    // Player 1 makes a move at (2, 0).
 |X| |X|

 toe.move(1, 0, 2); -> Returns 0 (no one wins)
 |X| |O|
 |O|O| |    // Player 2 makes a move at (1, 0).
 |X| |X|

 toe.move(2, 1, 1); -> Returns 1 (player 1 wins)
```

```
 |X| |O|
 |O|O| |    // Player 1 makes a move at (2, 1).
 |X|X|X|

 Follow up:
 Could you do better than O(n2) per move() operation?

 Hint:

 Could you trade extra space such that move() operation can be done in O(1)?
 You need two arrays: int rows[n], int cols[n], plus two variables: diagonal,
anti_diagonal.

 */
public class _348 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/44548/java-o-1-solution-easy-to-
understand
         *
         * Key: in order to win a TicTacToe, you must have the entire row or column,
thus, we don't need
         * to keep track of the entire n^2 board. We only need to keep a count for each
row and column.
         * If at any time, a row or column matches the size of the board, then that
player has won.
         */
        public static class TicTacToe {

            private int diagonal;
            /**
             * This is diagonal: |X| | | | |X| | | | |X| So, its condition is always like
this: if (row ==
             * col)
             */

            private int antidiagonal;
            /**
             * This is antidiagonal: | | |X| | |X| | |X| | | So, its condition is always
like this: if
             * (col == size - row - 1)
             */
            private int[] rows;
            private int[] cols;

            /**
             * Initialize your data structure here.
             */
            public TicTacToe(int n) {
                rows = new int[n];
                cols = new int[n];
            }

            /**
```

```
             * Player {player} makes a move at ({row}, {col}).
             *
             * @param row The row of the board.
             * @param col The column of the board.
             * @param player The player, can be either 1 or 2.
             * @return The current winning condition, can be either: 0: No one wins. 1:
Player 1 wins. 2:
             * Player 2 wins.
             */
            public int move(int row, int col, int player) {
                int toAdd = player == 1 ? 1 : -1;

                rows[row] += toAdd;
                cols[col] += toAdd;
                int size = rows.length;

                if (row == col) {
                    diagonal += toAdd;
                }
                if (col == (size - row - 1)) {
                    antidiagonal += toAdd;
                }

                if (Math.abs(rows[row]) == size
                    || Math.abs(cols[col]) == size
                    || Math.abs(antidiagonal) == size
                    || Math.abs(diagonal) == size) {
                    return player;
                }

                return 0;
            }
        }
    }
}
```

332. `_349.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 349. Intersection of Two Arrays
 *
 * Given two arrays, write a function to compute their intersection.
 *
 * Example 1:
 * Input: nums1 = [1,2,2,1], nums2 = [2,2]
 * Output: [2]
 *
 * Example 2:
 * Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
 * Output: [9,4]
 *
 * Note:
 * Each element in the result must be unique.
 * The result can be in any order.
 */
public class _349 {

  public static class Solution1 {
    //credit: https://leetcode.com/articles/intersection-of-two-arrays/
    //Time: O(m+n) on average, O(m*n) in worse case
    //Space: O(m+n)
    public int[] intersection(int[] nums1, int[] nums2) {
      Set<Integer> set1 = Arrays.stream(nums1).boxed().collect(Collectors.toSet());
      Set<Integer> set2 = Arrays.stream(nums2).boxed().collect(Collectors.toSet());
      set1.retainAll(set2);
      int[] intersection = new int[set1.size()];
      int i = 0;
      for (int num : set1) {
        intersection[i++] = num;
      }
      return intersection;
    }
  }

  public static class Solution2 {
      //Time: O(nlgn)
    public int[] intersection(int[] nums1, int[] nums2) {
      Arrays.sort(nums2);
      Set<Integer> intersect = new HashSet();
      for (int i : nums1) {
        if (binarySearch(i, nums2)) {
          intersect.add(i);
```

```
        }
      }
      int[] result = new int[intersect.size()];
      int i = 0;
      for (int num : intersect) {
        result[i++] = num;
      }
      return result;
    }

    private boolean binarySearch(int i, int[] nums) {
      int left = 0;
      int right = nums.length - 1;
      while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == i) {
          return true;
        } else if (nums[mid] > i) {
          right = mid - 1;
        } else {
          left = mid + 1;
        }
      }
      return false;
    }
  }

  public static class Solution3 {
    //use two pointers
    //credit: https://leetcode.com/problems/intersection-of-two-
arrays/discuss/81969/Three-Java-Solutions
    public int[] intersection(int[] nums1, int[] nums2) {
      Arrays.sort(nums1);
      Arrays.sort(nums2);
      int i = 0;
      int j = 0;
      Set<Integer> set = new HashSet<>();
      while (i < nums1.length && j < nums2.length) {
        if (nums1[i] == nums2[j]) {
          set.add(nums1[i++]);
          j++;
        } else if (nums1[i] < nums2[j]) {
          i++;
        } else {
          j++;
        }
      }
      int[] intersection = new int[set.size()];
      int k = 0;
      for (int num : set) {
        intersection[k++] = num;
      }
      return intersection;
    }
```

```
  }
}
```

333. `_350.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Arrays;

/**
 * 350. Intersection of Two Arrays II
 *
 * Given two arrays, write a function to compute their intersection.

 Example:
 Given nums1 = [1, 2, 2, 1], nums2 = [2, 2], return [2, 2].

 Note:
 Each element in the result should appear as many times as it shows in both arrays.
 The result can be in any order.

 Follow up:
 What if the given array is already sorted? How would you optimize your algorithm?
 What if nums1's size is small compared to nums2's size? Which algorithm is better?
 What if elements of nums2 are stored on disk, and the memory is limited such that you
cannot load all elements into the memory at once?
 */
public class _350 {

    public static class Solution1 {
        public int[] intersect(int[] nums1, int[] nums2) {
            Map<Integer, Integer> map = new HashMap();
            for (int i : nums1) {
                map.put(i, map.getOrDefault(i, 0) + 1);
            }

            List<Integer> list = new ArrayList();
            for (int i : nums2) {
                if (map.containsKey(i) && map.get(i) > 0) {
                    list.add(i);
                    map.put(i, map.get(i) - 1);
                }
            }
            return list.stream().mapToInt(i -> i).toArray();
        }
    }

    public static class Solution2 {
        public int[] intersect(int[] nums1, int[] nums2) {
            Arrays.sort(nums1);
            Arrays.sort(nums2);
            int i = 0;
```

```
            int j = 0;
            List<Integer> list = new ArrayList<>();
            while (i < nums1.length && j < nums2.length) {
                if (nums1[i] == nums2[j]) {
                    list.add(nums1[i]);
                    i++;
                    j++;
                } else if (nums1[i] < nums2[j]) {
                    i++;
                } else {
                    j++;
                }
            }
            return list.stream().mapToInt(k -> k).toArray();
        }
    }
}
```

334. `_351.java`

```
package com.fishercoder.solutions;

/**
 * 351. Android Unlock Patterns
 *
 * Given an Android 3x3 key lock screen and two integers m and n, where 1 ≤ m ≤ n ≤ 9,
count the total number of unlock patterns of the Android lock screen, which consist of
minimum of m keys and maximum n keys.

 Rules for a valid pattern:
 Each pattern must connect at least m keys and at most n keys.
 All the keys must be distinct.
 If the line connecting two consecutive keys in the pattern passes through any other
keys, the other keys must have previously selected in the pattern. No jumps through non
selected key is allowed.
 The order of keys used matters.

 Explanation:
 | 1 | 2 | 3 |
 | 4 | 5 | 6 |
 | 7 | 8 | 9 |
 Invalid move: 4 - 1 - 3 - 6
 Line 1 - 3 passes through key 2 which had not been selected in the pattern.

 Invalid move: 4 - 1 - 9 - 2
 Line 1 - 9 passes through key 5 which had not been selected in the pattern.

 Valid move: 2 - 4 - 1 - 3 - 6
 Line 1 - 3 is valid because it passes through key 2, which had been selected in the
pattern

 Valid move: 6 - 5 - 4 - 1 - 9 - 2
 Line 1 - 9 is valid because it passes through key 5, which had been selected in the
pattern.

 Example:
 Given m = 1, n = 1, return 9.


 */
public class _351 {

    public static class Solution1 {
        private int[][] jumps;
        private boolean[] visited;

        public int numberOfPatterns(int m, int n) {
            jumps = new int[10][10];
            jumps[1][3] = jumps[3][1] = 2;
            jumps[4][6] = jumps[6][4] = 5;
            jumps[7][9] = jumps[9][7] = 8;
```

```
            jumps[1][7] = jumps[7][1] = 4;
            jumps[2][8] = jumps[8][2] = jumps[1][9] = jumps[9][1] = 5;
            jumps[9][3] = jumps[3][9] = 6;
            jumps[3][7] = jumps[7][3] = 5;
            visited = new boolean[10];
            int count = 0;
            count += dfs(1, 1, 0, m, n)
                * 4;//1,3,7,9 are symmetric, so we only need to use 1 to do it once and
then multiply the result by 4
            count += dfs(2, 1, 0, m, n)
                * 4;//2,4,6,8 are symmetric, so we only need to use 1 to do it once and
then multiply the result by 4
            count += dfs(5, 1, 0, m, n);
            return count;
        }

        private int dfs(int num, int len, int count, int m, int n) {
            if (len >= m) {
                count++;
            }
            len++;
            if (len > n) {
                return count;
            }
            visited[num] = true;
            for (int next = 1; next <= 9; next++) {
                int jump = jumps[num][next];
                if (!visited[next] && (jump == 0 || visited[jump])) {
                    count = dfs(next, len, count, m, n);
                }
            }
            visited[num] = false;//backtracking
            return count;
        }
    }
}
```

335. `_352.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

/**
 * 352. Data Stream as Disjoint Intervals
 *
 * Given a data stream input of non-negative integers a1, a2, ..., an, ..., summarize the
numbers seen so far as a list of disjoint intervals.

 For example, suppose the integers from the data stream are 1, 3, 7, 2, 6, ..., then the
summary will be:

 [1, 1]
 [1, 1], [3, 3]
 [1, 1], [3, 3], [7, 7]
 [1, 3], [7, 7]
 [1, 3], [6, 7]

 Follow up:
 What if there are lots of merges and the number of disjoint intervals are small compared
to the data stream's size?
 */
public class _352 {

    public static class Solution1 {
        public static class SummaryRanges {

            /**
             * Use treemap, key is the start of the interval.
             */
            TreeMap<Integer, Interval> treeMap;

            /**
             * Initialize your data structure here.
             */
            public SummaryRanges() {
                treeMap = new TreeMap<>();
            }

            public void addNum(int val) {
                if (treeMap.containsKey(val)) {
                    return;
                }
                Integer lower = treeMap.lowerKey(val);
                Integer higher = treeMap.higherKey(val);
                if (lower != null
```

```
                    && higher != null
                    && treeMap.get(lower).end + 1 == val
                    && higher == val + 1) {
                    treeMap.get(lower).end = treeMap.get(higher).end;
                    treeMap.remove(higher);
                } else if (lower != null && treeMap.get(lower).end + 1 >= val) {
                    treeMap.get(lower).end = Math.max(treeMap.get(lower).end, val);
                } else if (higher != null && higher == val + 1) {
                    treeMap.put(val, new Interval(val, treeMap.get(higher).end));
                    treeMap.remove(higher);
                } else {
                    treeMap.put(val, new Interval(val, val));
                }
            }

            public List<Interval> getIntervals() {
                return new ArrayList<>(treeMap.values());
            }
        }
    }
}
```

336. `_353.java`

```
package com.fishercoder.solutions;

import java.util.Deque;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Set;

/**
 * Design a Snake game that is played on a device with screen size = width x height. Play
the game online if you are not familiar with the game.
 * <p>
 * The snake is initially positioned at the top left corner (0,0) with length = 1 unit.
 * <p>
 * You are given a list of food's positions in row-column order. When a snake eats the
food, its length and the game's score both increase by 1.
 * <p>
 * Each food appears one by one on the screen. For example, the second food will not
appear until the first food was eaten by the snake.
 * <p>
 * When a food does appear on the screen, it is guaranteed that it will not appear on a
block occupied by the snake.
 * <p>
 * Example:
 * Given width = 3, height = 2, and food = [[1,2],[0,1]].
 * <p>
 * Snake snake = new Snake(width, height, food);
 * <p>
 * Initially the snake appears at position (0,0) and the food at (1,2).
 * <p>
 * |S| | |
 * | | |F|
 * <p>
 * snake.move("R"); -> Returns 0
 * <p>
 * | |S| |
 * | | |F|
 * <p>
 * snake.move("D"); -> Returns 0
 * <p>
 * | | | |
 * | |S|F|
 * <p>
 * snake.move("R"); -> Returns 1 (Snake eats the first food and right after that, the
second food appears at (0,1) )
 * <p>
 * | |F| |
 * | |S|S|
 * <p>
 * snake.move("U"); -> Returns 1
 * <p>
 * | |F|S|
```

```
 * | | |S|
 * <p>
 * snake.move("L"); -> Returns 2 (Snake eats the second food)
 * <p>
 * | |S|S|
 * | | |S|
 * <p>
 * snake.move("U"); -> Returns -1 (Game over because snake collides with border)
 */
public class _353 {
    public class SnakeGame {
        private Set<Integer> set;//Use a set to hold all occupied points for the snake
body, this is for easy access for the case of eating its own body
        private Deque<Integer> body;//use a queue to hold all occupied points for the
snake body as well, this is for easy access to update the tail
        int[][] food;
        int score;
        int foodIndex;
        int width;
        int height;

        /**
         * Initialize your data structure here.
         *
         * @param width  - screen width
         * @param height - screen height
         * @param food   - A list of food positions
         *               E.g food = [[1,1], [1,0]] means the first food is positioned at
[1,1], the second is at [1,0].
         */
        public SnakeGame(int width, int height, int[][] food) {
            this.set = new HashSet();
            set.add(0);//initially at [0][0]
            this.body = new LinkedList<Integer>();
            body.offerLast(0);
            this.food = food;
            this.width = width;
            this.height = height;
        }

        /**
         * Moves the snake.
         *
         * @param direction - 'U' = Up, 'L' = Left, 'R' = Right, 'D' = Down
         * @return The game's score after the move. Return -1 if game over.
         * Game over when snake crosses the screen boundary or bites its body.
         */
        public int move(String direction) {
            if (score == -1) {
                return -1;
            }

            //compute head
            int rowHead = body.peekFirst() / width;
```

```
            int colHead = body.peekFirst() % width;
            switch (direction) {
                case "U":
                    rowHead--;
                    break;
                case "D":
                    rowHead++;
                    break;
                case "L":
                    colHead--;
                    break;
                default:
                    colHead++;
            }
            int newHead = rowHead * width + colHead;

            set.remove(body.peekLast());//we'll remove the tail from set for now to see
if it hits its tail
            //if it hits the boundary
            if (set.contains(newHead) || rowHead < 0 || colHead < 0 || rowHead >= height
|| colHead >= width) {
                return score = -1;
            }

            //add head for the following two normal cases:
            set.add(newHead);
            body.offerFirst(newHead);

            //normal eat case: keep tail, add head
            if (foodIndex < food.length && rowHead == food[foodIndex][0] && colHead ==
food[foodIndex][1]) {
                set.add(body.peekLast());//old tail does not change, so add it back to
set since we removed it earlier
                foodIndex++;
                return ++score;
            }


            //normal move case without eating: move head and remove tail
            body.pollLast();
            return score;

        }
    }

/**
 * Your SnakeGame object will be instantiated and called as such:
 * SnakeGame obj = new SnakeGame(width, height, food);
 * int param_1 = obj.move(direction);
 */
}
```

337. `_354.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 354. Russian Doll Envelopes
 *
 * You have a number of envelopes with widths and heights given as a pair of integers (w,
h).
 * One envelope can fit into another if and only if both the width and height of one
envelope is greater than
 * the width and height of the other envelope.

 What is the maximum number of envelopes can you Russian doll? (put one inside other)

 Example:
 Given envelopes = [[5,4],[6,4],[6,7],[2,3]], the maximum number of envelopes you can
Russian doll is 3 ([2,3] => [5,4] => [6,7]).

 */
public class _354 {
    public static class Solution1 {
        /** reference: https://discuss.leetcode.com/topic/47469/java-nlogn-solution-with-
explanation */
        public int maxEnvelopes(int[][] envelopes) {
            if (envelopes == null || envelopes.length == 0
                || envelopes[0].length == 0 || envelopes[0].length != 2) {
                return 0;
            }
            Arrays.sort(envelopes, (int[] a, int[] b) -> {
                    if (a[0] == b[0]) {
                        return b[1] - a[1];
                    } else {
                        return a[0] - b[0];
                    }
                }
            );
            int[] dp = new int[envelopes.length];
            int len = 0;
            for (int[] envelope : envelopes) {
                int index = Arrays.binarySearch(dp, 0, len, envelope[1]);
                if (index < 0) {
                    index = -(index + 1);
                }
                dp[index] = envelope[1];
                if (index == len) {
                    len++;
                }
            }
            return len;
        }
```

```
    }
}
```

338. `_355.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Set;

/**
 * 355. Design Twitter
 *
 * Design a simplified version of Twitter where users can post tweets,
 * follow/unfollow another user and is able to see the 10 most recent tweets in the
user's news feed. Your design should support the following methods:
 postTweet(userId, tweetId): Compose a new tweet.
 getNewsFeed(userId): Retrieve the 10 most recent tweet ids in the user's news feed. Each
item in the news feed must be posted by users who the user followed or by the user
herself. Tweets must be ordered from most recent to least recent.
 follow(followerId, followeeId): Follower follows a followee.
 unfollow(followerId, followeeId): Follower unfollows a followee.

 Example:

 Twitter twitter = new Twitter();

 // User 1 posts a new tweet (userId = 5).
 twitter.postTweet(1, 5);

 // User 1's news feed should return a list with 1 tweet userId -> [5].
 twitter.getNewsFeed(1);

 // User 1 follows user 2.
 twitter.follow(1, 2);

 // User 2 posts a new tweet (userId = 6).
 twitter.postTweet(2, 6);

 // User 1's news feed should return a list with 2 tweet ids -> [6, 5].
 // Tweet userId 6 should precede tweet userId 5 because it is posted after tweet userId
5.
 twitter.getNewsFeed(1);

 // User 1 unfollows user 2.
 twitter.unfollow(1, 2);

 // User 1's news feed should return a list with 1 tweet userId -> [5],
 // since user 1 is no longer following user 2.
 twitter.getNewsFeed(1);
 */
```

```
public class _355 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/48100/java-oo-design-with-most-efficient-function-getnewsfeed
         */
        public static class Twitter {

            private static int timestamp = 0;
            private Map<Integer, User> map;

            class Tweet {
                public int time;
                public int id;
                public Tweet next;
                /**have a pointer,
                 * so we could be more memory efficient when retrieving tweets,
                 * think about merging k sorted lists*/

                public Tweet(int id) {
                    this.id = id;
                    time = timestamp++;
                    next = null;
                }
            }

            /**
             * the meat part of this OO design problem,
             * have a User object itself,
             * have follow() and unfollow() method embedded inside it
             */
            class User {
                public int id;
                public Set<Integer> followed;
                public Tweet tweetHead;

                public User(int id) {
                    this.id = id;
                    followed = new HashSet<>();
                    followed.add(id);//follow oneself first
                    this.tweetHead = null;
                }

                public void follow(int followeeId) {
                    followed.add(followeeId);
                }

                public void unfollow(int followeeId) {
                    followed.remove(followeeId);
                }

                public void postTweet(int tweetId) {
                    //every time we post, we prepend it to the head of the tweet
```

```
                    Tweet head = new Tweet(tweetId);
                    head.next = tweetHead;
                    tweetHead = head;//don't forget to overwrite tweetHead with the new
head
                }
            }

            /**
             * Initialize your data structure here.
             */
            public Twitter() {
                map = new HashMap();
            }

            /**
             * Compose a new tweet.
             */
            public void postTweet(int userId, int tweetId) {
                /**update oneself newsFeed first and also all of his followers'
newsFeed*/
                if (!map.containsKey(userId)) {
                    User user = new User(userId);
                    map.put(userId, user);
                }
                map.get(userId).postTweet(tweetId);
            }

            /**
             * Retrieve the 10 most recent tweet ids in the user's news feed. Each item
in the news feed must be posted by users who the user followed or by the user herself.
Tweets must be ordered from most recent to least recent.
             */
            public List<Integer> getNewsFeed(int userId) {
                List<Integer> newsFeed = new LinkedList<>();
                if (!map.containsKey(userId)) {
                    return newsFeed;
                }
                Set<Integer> users = map.get(userId).followed;
                PriorityQueue<Tweet> heap = new PriorityQueue<>(users.size(), (a, b) ->
b.time - a.time);
                for (int user : users) {
                    Tweet tweet = map.get(user).tweetHead;
                    //it's super important to check null before putting into the heap
                    if (tweet != null) {
                        heap.offer(tweet);
                    }
                }

                int count = 0;
                while (!heap.isEmpty() && count < 10) {
                    Tweet tweet = heap.poll();
                    newsFeed.add(tweet.id);
                    count++;
                    if (tweet.next != null) {
```

```
                    heap.offer(tweet.next);
                }
            }

            return newsFeed;
        }

        /**
         * Follower follows a followee. If the operation is invalid, it should be a
no-op.
         */
        public void follow(int followerId, int followeeId) {
            if (!map.containsKey(followeeId)) {
                User user = new User(followeeId);
                map.put(followeeId, user);
            }

            if (!map.containsKey(followerId)) {
                User user = new User(followerId);
                map.put(followerId, user);
            }

            map.get(followerId).follow(followeeId);
        }

        /**
         * Follower unfollows a followee. If the operation is invalid, it should be a
no-op.
         */
        public void unfollow(int followerId, int followeeId) {
            if (!map.containsKey(followerId) || followeeId == followerId) {
                return;
            }
            map.get(followerId).unfollow(followeeId);
        }
        /**
         * Your Twitter object will be instantiated and called as such:
         * Twitter obj = new Twitter();
         * obj.postTweet(userId,tweetId);
         * List<Integer> param_2 = obj.getNewsFeed(userId);
         * obj.follow(followerId,followeeId);
         * obj.unfollow(followerId,followeeId);
         */
    }
}

public static class Solution2 {
    public static class Twitter {

        Map<Integer, User> map;
        private int timestamp;

        private class User {
            private int userId;
```

```
        private Set<Integer> followed;
        private Tweet tweetHead;

        public User(int userId) {
            this.userId = userId;
            this.followed = new HashSet<>();
            this.followed.add(userId);
            this.tweetHead = null;
        }

        public void postTweet(int tweetId) {
            Tweet tweet = new Tweet(tweetId);
            tweet.next = tweetHead;
            tweetHead = tweet;
        }

        public void follow(int followeeId) {
            followed.add(followeeId);
        }

        public void unfollow(int followeeId) {
            followed.remove(followeeId);
        }

    }

    private class Tweet {
        int time;
        int id;
        Tweet next;

        public Tweet(int id) {
            this.id = id;
            time = timestamp++;
            next = null;
        }
    }

    /** Initialize your data structure here. */
    public Twitter() {
        map = new HashMap<>();
        timestamp = 0;
    }

    /** Compose a new tweet. */
    public void postTweet(int userId, int tweetId) {
        if (!map.containsKey(userId)) {
            User user = new User(userId);
            map.put(userId, user);
        }
        map.get(userId).postTweet(tweetId);
    }

    /** Retrieve the 10 most recent tweet ids in the user's news feed. Each item
```

```
in the news feed must be posted by users who the user followed or by the user herself.
Tweets must be ordered from most recent to least recent. */
            public List<Integer> getNewsFeed(int userId) {
                List<Integer> result = new LinkedList<>();
                if (!map.containsKey(userId)) {
                    return result;
                }
                Set<Integer> followeeSet = map.get(userId).followed;
                PriorityQueue<Tweet> maxHeap = new PriorityQueue<>((a, b) -> b.time -
a.time);
                for (int followeeId : followeeSet) {
                    if (map.containsKey(followeeId)) {
                        Tweet tweet = map.get(followeeId).tweetHead;
                        if (tweet != null) {
                            maxHeap.offer(tweet);
                        }
                    }
                }

                int count = 0;
                while (!maxHeap.isEmpty() && count++ < 10) {
                    Tweet tweet = maxHeap.poll();
                    if (tweet != null) {
                        result.add(tweet.id);
                        if (tweet.next != null) {
                            maxHeap.offer(tweet.next);
                        }
                    }
                }
                return result;
            }

            /** Follower follows a followee. If the operation is invalid, it should be a
no-op. */
            public void follow(int followerId, int followeeId) {
                if (!map.containsKey(followerId)) {
                    map.put(followerId, new User(followerId));
                }
                if (!map.containsKey(followeeId)) {
                    map.put(followeeId, new User(followeeId));
                }
                map.get(followerId).follow(followeeId);
            }

            /** Follower unfollows a followee. If the operation is invalid, it should be
a no-op. */
            public void unfollow(int followerId, int followeeId) {
                if (!map.containsKey(followerId) || followeeId == followerId) {
                    return;
                }
                map.get(followerId).unfollow(followeeId);
            }
        }
```

```
    }
}
```

339. `_356.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 356. Line Reflection
 *
 * Given n points on a 2D plane, find if there is such a line parallel to y-axis that
reflect the given points.

 Example 1:
 Given points = [[1,1],[-1,1]], return true.

 Example 2:
 Given points = [[1,1],[-1,-1]], return false.

 Follow up:
 Could you do better than O(n2)?

 Hint:

 Find the smallest and largest x-value for all points.
 If there is a line then it should be at y = (minX + maxX) / 2.
 For each point, make sure that it has a reflected point in the opposite side.
 */
public class _356 {
    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/48172/simple-java-hashset-solution
*/
        public boolean isReflected(int[][] points) {
            int max = Integer.MIN_VALUE;
            int min = Integer.MAX_VALUE;
            Set<String> set = new HashSet<>();
            for (int[] point : points) {
                max = Math.max(max, point[0]);
                min = Math.min(min, point[0]);
                String str = point[0] + "a" + point[1];
                set.add(str);
            }
            int sum = max + min;
            for (int[] p : points) {
                String str = (sum - p[0]) + "a" + p[1];
                if (!set.contains(str)) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

340. `_357.java`

```
package com.fishercoder.solutions;

/**
 * 357. Count Numbers with Unique Digits
 * Given a non-negative integer n, count all numbers with unique digits, x, where 0 ≤ x < 10n.

 Example:
 Given n = 2, return 91. (The answer should be the total numbers in the range of 0 ≤ x < 100, excluding [11,22,33,44,55,66,77,88,99])

 Hint:
 A direct way is to use the backtracking approach.
 Backtracking should contains three states which are
 (the current number, number of steps to get that number and a bitmask which
 represent which number is marked as visited so far in the current number).
 Start with state (0,0,0) and count all valid number till we reach number of steps equals to 10n.
 This problem can also be solved using a dynamic programming approach and some knowledge of combinatorics.
 Let f(k) = count of numbers with unique digits with length equals k.
 f(1) = 10, ..., f(k) = 9 * 9 * 8 * ... (9 - k + 2) [The first factor is 9 because a number cannot start with 0].
 */
public class _357 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/47983/java-dp-o-1-solution Following the hint.
         * Let f(n) = count of number with unique digits of length n. f(1) = 10. (0, 1, 2, 3, ...., 9)
         * f(2) = 9 * 9. Because for each number i from 1, ..., 9, we can pick j to form a 2-digit
         * number ij and there are 9 numbers that are different from i for j to choose from. f(3) = f(2)
         * * 8 = 9 * 9 * 8. Because for each number with unique digits of length 2, say ij, we can pick
         * k to form a 3 digit number ijk and there are 8 numbers that are different from i and j for k
         * to choose from. Similarly f(4) = f(3) * 7 = 9 * 9 * 8 * 7.... ... f(10) = 9 * 9 * 8 * 7 * 6 *
         * ... * 1 f(11) = 0 = f(12) = f(13).... any number with length > 10 couldn't be unique digits
         * number. The problem is asking for numbers from 0 to 10^n. Hence return f(1) + f(2) + .. +
         * f(n) As @4acreg suggests, There are only 11 different ans. You can create a lookup table for
         * it. This problem is O(1) in essence.
         */
        public int countNumbersWithUniqueDigits(int n) {
```

```
            if (n == 0) {
                return 1;
            }
            int res = 10;
            int uniqueDigits = 9;
            int availableNumber = 9;
            while (n-- > 1 && availableNumber > 0) {
                uniqueDigits = uniqueDigits * availableNumber;
                res += uniqueDigits;
                availableNumber--;
            }
            return res;
        }
    }

}
```

341. `_358.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 358. Rearrange String k Distance Apart
 *
 * Given a non-empty string s and an integer k, rearrange the string such that the same
characters are at least distance k from each other.

 All input strings are given in lowercase letters. If it is not possible to rearrange the
string, return an empty string "".
 Example 1:
 s = "aabbcc", k = 3
 Result: "abcabc"
 The same letters are at least distance 3 from each other.

 Example 2:
 s = "aaabc", k = 3
 Answer: ""
 It is not possible to rearrange the string.

 Example 3:
 s = "aaadbbcc", k = 2
 Answer: "abacabcd"
 Another possible answer is: "abcabcda"
 The same letters are at least distance 2 from each other.
 */
public class _358 {

    public static class Solution1 {
        public String rearrangeString(String s, int k) {
            Map<Character, Integer> count = new HashMap<>();
            for (char c : s.toCharArray()) {
                count.put(c, count.getOrDefault(c, 0) + 1);
            }

            PriorityQueue<Map.Entry<Character, Integer>> heap =
                new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());
            heap.addAll(count.entrySet());

            Queue<Map.Entry<Character, Integer>> waitQueue = new LinkedList<>();

            StringBuilder stringBuilder = new StringBuilder();
            while (!heap.isEmpty()) {
                Map.Entry<Character, Integer> entry = heap.poll();
                stringBuilder.append(entry.getKey());
```

```
                entry.setValue(entry.getValue() - 1);
                waitQueue.offer(entry);
                if (waitQueue.size() < k) {
                    continue; //there's only k-1 chars in the waitHeap, not full yet
                }
                Map.Entry<Character, Integer> front = waitQueue.poll();
                if (front.getValue() > 0) {
                    heap.offer(front);
                }
            }

            return stringBuilder.length() == s.length() ? stringBuilder.toString() : "";
        }
    }

}
```

342. `_359.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;
/**
 * 359. Logger Rate Limiter
 *
 * Design a logger system that receive stream of messages along with its timestamps, each
message should be printed if and only if it is not printed in the last 10 seconds.

 Given a message and a timestamp (in seconds granularity), return true if the message
should be printed in the given timestamp, otherwise returns false.

 It is possible that several messages arrive roughly at the same time.

 Example:

 Logger logger = new Logger();

 // logging string "foo" at timestamp 1
 logger.shouldPrintMessage(1, "foo"); returns true;

 // logging string "bar" at timestamp 2
 logger.shouldPrintMessage(2,"bar"); returns true;

 // logging string "foo" at timestamp 3
 logger.shouldPrintMessage(3,"foo"); returns false;

 // logging string "bar" at timestamp 8
 logger.shouldPrintMessage(8,"bar"); returns false;

 // logging string "foo" at timestamp 10
 logger.shouldPrintMessage(10,"foo"); returns false;

 // logging string "foo" at timestamp 11
 logger.shouldPrintMessage(11,"foo"); returns true;*/
public class _359 {

    public static class Solution1 {
        class Logger {

            private Map<String, Integer> map;
            private Set<String> set;

            /**
             * Initialize your data structure here.
             */
            public Logger() {
                map = new HashMap<String, Integer>();
```

```
                set = new HashSet<String>();
            }

            /**
             * Returns true if the message should be printed in the given timestamp,
otherwise returns
             * false. The timestamp is in seconds granularity.
             */
            public boolean shouldPrintMessage(int timestamp, String message) {
                if (!set.contains(message)) {
                    map.put(message, timestamp);
                    set.add(message);
                    return true;
                } else {
                    if (timestamp - map.get(message) < 10) {
                        return false;
                    } else {
                        map.put(message, timestamp);
                        return true;
                    }
                }
            }
        }
    }
}
```

343. `_360.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 360. Sort Transformed Array
 *
 * Given a sorted array of integers nums and integer values a, b and c. Apply a function
of the form f(x) = ax2 + bx + c to each element x in the array.

 The returned array must be in sorted order.

 Expected time complexity: O(n)

 Example:
 nums = [-4, -2, 2, 4], a = 1, b = 3, c = 5,

 Result: [3, 9, 15, 33]

 nums = [-4, -2, 2, 4], a = -1, b = 3, c = 5

 Result: [-23, -5, 1, 7]

 Credits:
 Special thanks to @elmirap for adding this problem and creating all test cases.
 */
public class _360 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/48424/java-o-n-incredibly-short-yet-
easy-to-understand-ac-solution
        //in sum, only two cases: when a >= 0 or when a < 0, this simplifies logic
        public int[] sortTransformedArray(int[] nums, int a, int b, int c) {
            int n = nums.length;
            int[] sorted = new int[n];
            int i = 0;
            int j = n - 1;
            int index = a >= 0 ? n - 1 : 0;
            while (i <= j) {
                if (a >= 0) {
                    sorted[index--] =
                        function(nums[i], a, b, c) >= function(nums[j], a, b, c) ?
function(
                            nums[i++], a, b, c) : function(nums[j--], a, b, c);
                } else {
                    sorted[index++] =
                        function(nums[i], a, b, c) >= function(nums[j], a, b, c) ?
function(
                            nums[j--], a, b, c) : function(nums[i++], a, b, c);
                }
            }
```

```
            return sorted;
        }

        private int function(int num, int a, int b, int c) {
            return a * (num * num) + b * num + c;
        }
    }

    public static class Solution2 {
        public int[] sortTransformedArray(int[] nums, int a, int b, int c) {
            int[] result = new int[nums.length];
            for (int i = 0; i < nums.length; i++) {
                result[i] = function(nums[i], a, b, c);
            }
            Arrays.sort(result);
            return result;
        }

        private int function(int num, int a, int b, int c) {
            return a * (num * num) + b * num + c;
        }
    }

}
```

344. `_361.java`

```
package com.fishercoder.solutions;

/**
 * 361. Bomb Enemy
 *
 * Given a 2D grid, each cell is either a wall 'W', an enemy 'E' or empty '0' (the number zero), return the maximum enemies you can kill using one bomb.
 The bomb kills all the enemies in the same row and column from the planted point until it hits the wall since the wall is too strong to be destroyed.
 Note that you can only put the bomb at an empty cell.

 Example:
 For the given grid

 0 E 0 0
 E 0 W E
 0 E 0 0

 return 3. (Placing a bomb at (1,1) kills 3 enemies)
 */
public class _361 {

    public static class Solution1 {
        public int maxKilledEnemies(char[][] grid) {
            int m = grid.length;
            if (grid == null || m == 0) {
                return 0;
            }
            int n = grid[0].length;

            int[][] max = new int[m][n];

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {

                    if (grid[i][j] == '0') {
                        int count = 0;

                        //count all possible hits in its upward direction
                        for (int k = j - 1; k >= 0; k--) {
                            if (grid[i][k] == 'E') {
                                count++;
                            } else if (grid[i][k] == 'W') {
                                break;
                            }
                        }

                        //count all possible hits in its downward direction
                        for (int k = j + 1; k < n; k++) {
                            if (grid[i][k] == 'E') {
                                count++;
```

```
                        } else if (grid[i][k] == 'W') {
                            break;
                        }
                    }

                    //count all possible hits in its right direction
                    for (int k = i + 1; k < m; k++) {
                        if (grid[k][j] == 'E') {
                            count++;
                        } else if (grid[k][j] == 'W') {
                            break;
                        }
                    }

                    //count all possible hits in its left direction
                    for (int k = i - 1; k >= 0; k--) {
                        if (grid[k][j] == 'E') {
                            count++;
                        } else if (grid[k][j] == 'W') {
                            break;
                        }
                    }

                    max[i][j] = count;
                }
            }
        }

        int result = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (max[i][j] > result) {
                    result = max[i][j];
                }
            }
        }
        return result;
    }
}
```

345. `_362.java`

```
package com.fishercoder.solutions;
/**
 * 362. Design Hit Counter
 *
 * Design a hit counter which counts the number of hits received in the past 5 minutes.

 Each function accepts a timestamp parameter (in seconds granularity) and you may assume
 that calls are being made to the system in chronological order (ie, the timestamp is
monotonically increasing).
 You may assume that the earliest timestamp starts at 1.

 It is possible that several hits arrive roughly at the same time.

 Example:
 HitCounter counter = new HitCounter();

 // hit at timestamp 1.
 counter.hit(1);

 // hit at timestamp 2.
 counter.hit(2);

 // hit at timestamp 3.
 counter.hit(3);

 // get hits at timestamp 4, should return 3.
 counter.getHits(4);

 // hit at timestamp 300.
 counter.hit(300);

 // get hits at timestamp 300, should return 4.
 counter.getHits(300);

 // get hits at timestamp 301, should return 3.
 counter.getHits(301);
 Follow up:
 What if the number of hits per second could be very large? Does your design scale?
 */

public class _362 {

    public static class Solution1 {
        public static class HitCounter {
            /**
             * Reference: https://discuss.leetcode.com/topic/48758/super-easy-design-o-1-
hit-o-s-gethits-no-fancy-data-structure-is-needed,
             * I added one more field k to make it more generic.
             * It basically maintains a window of size 300, use modular to update the
index.
             */
```

```
            private int[] times;
            private int[] hits;
            private int k;

            /**
             * Initialize your data structure here.
             */
            public HitCounter() {
                k = 300;
                times = new int[k];
                hits = new int[k];
            }

            /**
             * Record a hit.
             *
             * @param timestamp - The current timestamp (in seconds granularity).
             */
            public void hit(int timestamp) {
                int index = timestamp % k;
                if (times[index] != timestamp) {
                    times[index] = timestamp;
                    hits[index] = 1;
                } else {
                    hits[index]++;
                }
            }

            /**
             * Return the number of hits in the past 5 minutes.
             *
             * @param timestamp - The current timestamp (in seconds granularity).
             */
            public int getHits(int timestamp) {
                int total = 0;
                for (int i = 0; i < k; i++) {
                    if (timestamp - times[i] < k) {
                        total += hits[i];
                    }
                }
                return total;
            }
        }
    }
}
```

346. `_363.java`

```
package com.fishercoder.solutions;

import java.util.TreeSet;

/**
 * 363. Max Sum of Rectangle No Larger Than K
 *
 * Given a non-empty 2D matrix matrix and an integer k,
 * find the max sum of a rectangle in the matrix such that its sum is no larger than k.

 Example:
 Given matrix = [
 [1,  0, 1],
 [0, -2, 3]
 ]
 k = 2
 The answer is 2. Because the sum of rectangle [[0, 1], [-2, 3]] is 2 and 2 is the max
number no larger than k (k = 2).

 Note:
 The rectangle inside the matrix must have an area > 0.
 What if the number of rows is much larger than the number of columns?
 */
public class _363 {
    public static class Solution1 {
        /** reference: https://discuss.leetcode.com/topic/48854/java-binary-search-
solution-time-complexity-min-m-n-2-max-m-n-log-max-m-n */
        public int maxSumSubmatrix(int[][] matrix, int k) {
            int row = matrix.length;
            if (row == 0) {
                return 0;
            }
            int col = matrix[0].length;
            int m = Math.min(row, col);
            int n = Math.max(row, col);
            //indicating sum up in every row or every column
            boolean colIsBig = col > row;
            int res = Integer.MIN_VALUE;
            for (int i = 0; i < m; i++) {
                int[] array = new int[n];
                // sum from row j to row i
                for (int j = i; j >= 0; j--) {
                    int val = 0;
                    TreeSet<Integer> set = new TreeSet<>();
                    set.add(0);
                    //traverse every column/row and sum up
                    for (int p = 0; p < n; p++) {
                        array[p] = array[p] + (colIsBig ? matrix[j][p] : matrix[p][j]);
                        val = val + array[p];
                        //use  TreeMap to binary search previous sum to get possible
result
```

```
                        Integer subres = set.ceiling(val - k);
                        if (null != subres) {
                            res = Math.max(res, val - subres);
                        }
                        set.add(val);
                    }
                }
            }
            return res;
        }
    }
}
```

347. `_364.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 364. Nested List Weight Sum II
 *
 * Given a nested list of integers, return the sum of all integers in the list weighted
by their depth.

 Each element is either an integer, or a list -- whose elements may also be integers or
other lists.

 Different from the previous question where weight is increasing from root to leaf, now
the weight is defined from bottom up. i.e., the leaf level integers have weight 1, and
the root level integers have the largest weight.

 Example 1:
 Given the list [[1,1],2,[1,1]], return 8. (four 1's at depth 1, one 2 at depth 2)

 Example 2:
 Given the list [1,[4,[6]]], return 17. (one 1 at depth 3, one 4 at depth 2, and one 6 at
depth 1; 1*3 + 4*2 + 6*1 = 17)*/
public class _364 {

    public static class Solution1 {
        public int depthSumInverse(List<NestedInteger> nestedList) {
            Queue<NestedInteger> q = new LinkedList<>();
            for (NestedInteger next : nestedList) {
                q.offer(next);
            }
            int prev = 0;
            int total = 0;

            while (!q.isEmpty()) {
                int size = q.size();
                int levelSum = 0;
                for (int i = 0; i < size; i++) {
                    NestedInteger next = q.poll();
                    if (next.isInteger()) {
                        levelSum += next.getInteger();
                    } else {
                        List<NestedInteger> list = next.getList();
                        for (NestedInteger n : list) {
                            q.offer(n);
                        }
                    }
```

```
                }
                prev += levelSum;
                total += prev;
            }
            return total;
        }
    }

}
```

348. `_365.java`

```
package com.fishercoder.solutions;

/**
 * 365. Water and Jug Problem
 *
 * You are given two jugs with capacities x and y litres.
 * There is an infinite amount of water supply available.
 * You need to determine whether it is possible to measure exactly z litres using these
two jugs.
 *
 If z liters of water is measurable, you must have z liters of water contained within one
or both buckets by the end.

 Operations allowed:

 Fill any of the jugs completely with water.
 Empty any of the jugs.
 Pour water from one jug into another till the other jug is completely full or the first
jug itself is empty.

 Example 1: (From the famous "Die Hard" example)
 Input: x = 3, y = 5, z = 4
 Output: True

 Example 2:
 Input: x = 2, y = 6, z = 5
 Output: False

 */
public class _365 {

    public static class Solution1 {
        public boolean canMeasureWater(int x, int y, int z) {
            if (x + y < z) {
                return false;
            }
            if (x == z || y == z || x + y == z) {
                return true;
            }
            return z % gcd(x, y) == 0;
        }

        int gcd(int x, int y) {
            while (y != 0) {
                int temp = y;
                y = x % y;
                x = temp;
            }
            return x;
        }
```

```
    }
}
```

349. `_366.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 366. Find Leaves of Binary Tree
 *
 * Given a binary tree, collect a tree's nodes as if you were doing this:
 * Collect and remove all leaves, repeat until the tree is empty.

 Example:
 Given binary tree
          1
         / \
        2   3
       / \
      4  5
 Returns [4, 5, 3], [2], [1].

 Explanation:
 1. Removing the leaves [4, 5, 3] would result in this tree:

        1
       /
      2
 2. Now removing the leaf [2] would result in this tree:

      1
 3. Now removing the leaf [1] would result in the empty tree:

      []
 Returns [4, 5, 3], [2], [1].

 */
public class _366 {

    public static class Solution1 {
        List<List<Integer>> result = new ArrayList<>();

        public List<List<Integer>> findLeaves(TreeNode root) {
            dfs(root);
            return result;
        }

        int dfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
```

```
            int level = Math.max(dfs(root.left), dfs(root.right)) + 1;
            if (result.size() < level) {
                result.add(new ArrayList<>());
            }
            result.get(level - 1).add(root.val);
            return level;
        }
    }
}
```

350. `_367.java`

```
package com.fishercoder.solutions;

/**
 * 367. Valid Perfect Square
 *
 * Given a positive integer num, write a function which returns True if num is a perfect
square else False.

 Note: Do not use any built-in library function such as sqrt.

 Example 1:

 Input: 16
 Returns: True
 Example 2:

 Input: 14
 Returns: False
 */
public class _367 {

        public static class Solution1 {
                public boolean isPerfectSquare(int num) {
                        long i = 1;
                        long temp = 1;
                        while (temp < num) {
                                i += 2;
                                temp += i;
                        }
                        return temp == num;
                }
        }
}
```

351. `_368.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 368. Largest Divisible Subset
 *
 * Given a set of distinct positive integers, find the largest subset such that every
pair (Si, Sj) of elements in this subset satisfies: Si % Sj = 0 or Sj % Si = 0.

 If there are multiple solutions, return any subset is fine.

 Example 1:

 nums: [1,2,3]

 Result: [1,2] (of course, [1,3] will also be ok)
 Example 2:

 nums: [1,2,4,8]

 Result: [1,2,4,8]
 */
public class _368 {

    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/49652/classic-dp-solution-similar-
to-lis-o-n-2 */
        public List<Integer> largestDivisibleSubset(int[] nums) {
            int len = nums.length;
            int[] count = new int[len];
            int[] pre = new int[len];
            Arrays.sort(nums);
            int max = 0;
            int index = -1;
            for (int i = 0; i < len; i++) {
                count[i] = 1;
                pre[i] = -1;
                for (int j = i - 1; j >= 0; j--) {
                    if (nums[i] % nums[j] == 0) {
                        if (1 + count[j] > count[i]) {
                            count[i] = count[j] + 1;
                            pre[i] = j;
                        }
                    }
                }
                if (count[i] > max) {
                    max = count[i];
                    index = i;
```

```
                }
            }
            List<Integer> res = new ArrayList<>();
            while (index != -1) {
                res.add(nums[index]);
                index = pre[index];
            }
            return res;
        }
    }

}
```

352. `_369.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 369. Plus One Linked List
 *
 * Given a non-negative integer represented as non-empty a singly linked list of digits,
plus one to the integer.

 You may assume the integer do not contain any leading zero, except the number 0 itself.

 The digits are stored such that the most significant digit is at the head of the list.

 Example:
 Input:
 1->2->3

 Output:
 1->2->4
 */
public class _369 {

    public static class Solution1 {
        public ListNode plusOne(ListNode head) {
            //get the length of the list and take out the value of each node and store
them into an array
            ListNode temp = head;
            int len = 0;
            while (temp != null) {
                len++;
                temp = temp.next;
            }

            int[] nums = new int[len];
            temp = head;
            int j = 0;
            while (temp != null) {
                nums[j++] = temp.val;
                temp = temp.next;
            }

            //plus one into this array: nums
            for (int i = len - 1; i >= 0; i--) {
                if (nums[i] != 9) {
                    nums[i]++;
                    break;
                } else {
                    nums[i] = 0;
                }
            }
```

```
            //still assuming the first value in the list should not be zero as it's
representing a valid number, although it's in a list
            ListNode pre = new ListNode(-1);
            if (nums[0] == 0) {
                //in this case, let's just construct a new linked list and return: only
first node value is 1, all the rest is 0
                ListNode newHead = new ListNode(1);
                ListNode result = newHead;
                int count = 0;
                while (count++ < len) {
                    newHead.next = new ListNode(0);
                    newHead = newHead.next;
                }
                return result;
            } else {
                pre.next = head;
                for (int i = 0; i < len; i++) {
                    head.val = nums[i];
                    head = head.next;
                }
                return pre.next;
            }
        }
    }

}
```

353. `_370.java`

```
package com.fishercoder.solutions;

/**
 * 370. Range Addition
 *
 * Assume you have an array of length n initialized with all 0's and are given k update
operations.
 * Each operation is represented as a triplet: [startIndex, endIndex, inc]
 * which increments each element of subarray A[startIndex ... endIndex] (startIndex and
endIndex inclusive) with inc.
 * Return the modified array after all k operations were executed.

 Example:

 Given:

 length = 5,
 updates = [
 [1,  3,  2],
 [2,  4,  3],
 [0,  2, -2]
 ]

 Output:

 [-2, 0, 3, 5, 3]
 Explanation:

 Initial state:
 [ 0, 0, 0, 0, 0 ]

 After applying operation [1, 3, 2]:
 [ 0, 2, 2, 2, 0 ]

 After applying operation [2, 4, 3]:
 [ 0, 2, 5, 5, 3 ]

 After applying operation [0, 2, -2]:
 [-2, 0, 3, 5, 3 ]

 Hint:
 Thinking of using advanced data structures? You are thinking it too complicated.
 For each update operation, do you really need to update all elements between i and j?
 Update only the first and end element is sufficient.
 The optimal time complexity is O(k + n) and uses O(1) extra space.
 */

public class _370 {
    public static class Solution1 {
        public int[] getModifiedArray(int length, int[][] updates) {
            int[] nums = new int[length];
```

```
            int k = updates.length;
            for (int i = 0; i < k; i++) {
                int start = updates[i][0];
                int end = updates[i][1];
                int inc = updates[i][2];
                nums[start] += inc;
                if (end < length - 1) {
                    nums[end + 1] -= inc;
                }
            }

            int sum = 0;
            for (int i = 0; i < length; i++) {
                sum += nums[i];
                nums[i] = sum;
            }
            return nums;
        }
    }
}
```

354. `_371.java`

```
package com.fishercoder.solutions;

/**
 * 371. Sum of Two Integers
 *
 * Calculate the sum of two integers a and b, but you are not allowed to use the operator
+ and -.

 Example:
 Given a = 1 and b = 2, return 3.
 */
public class _371 {

    public static class Solution1 {
        /** reference: http://stackoverflow.com/questions/9070937/adding-two-numbers-
without-operator-clarification */
        public int getSum(int a, int b) {
            if (b == 0) {
                return a;
            }
            int sum = a ^ b;
            int carry = (a & b) << 1;
            return getSum(sum, carry);
        }
    }
}
```

355. `_372.java`

```
package com.fishercoder.solutions;

/**
 * 372. Super Pow
 *
 * Your task is to calculate ab mod 1337 where a is a positive integer and b is an
extremely large positive integer given in the form of an array.

 Example1:

 a = 2
 b = [3]

 Result: 8
 Example2:

 a = 2
 b = [1,0]

 Result: 1024
 */
public class _372 {
    public static class Solution1 {
        public int superPow(int a, int[] b) {
            if (a % 1337 == 0) {
                return 0;
            }
            int p = 0;
            for (int i : b) {
                p = (p * 10 + i) % 1140;
            }
            if (p == 0) {
                p += 1140;
            }
            return power(a, p, 1337);
        }

        private int power(int a, int n, int mod) {
            a %= mod;
            int result = 1;
            while (n != 0) {
                if ((n & 1) != 0) {
                    result = result * a % mod;
                }
                a = a * a % mod;
                n >>= 1;
            }
            return result;
        }
    }
}
```

356. `_373.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 373. Find K Pairs with Smallest Sums
 *
 * You are given two integer arrays nums1 and nums2 sorted in ascending order and an
integer k.
 * Define a pair (u,v) which consists of one element from the first array and one element
from the second array.
 * Find the k pairs (u1,v1),(u2,v2) ...(uk,vk) with the smallest sums.

 Example 1:
 Given nums1 = [1,7,11], nums2 = [2,4,6],  k = 3
 Return: [1,2],[1,4],[1,6]
 The first 3 pairs are returned from the sequence:
 [1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]

 Example 2:
 Given nums1 = [1,1,2], nums2 = [1,2,3],  k = 2
 Return: [1,1],[1,1]
 The first 2 pairs are returned from the sequence:
 [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]

 Example 3:
 Given nums1 = [1,2], nums2 = [3],  k = 3
 Return: [1,3],[2,3]
 All possible pairs are returned from the sequence:
 [1,3],[2,3]

 */
public class _373 {
    public static class Solution1 {

        final int[][] neighbors = new int[][] {{0, 1}, {1, 0}};

        public List<int[]> kSmallestPairs(int[] nums1, int[] nums2, int k) {
            List<int[]> result = new ArrayList<>();
            if (nums1 == null
                || nums2 == null
                || k == 0
                || nums1.length == 0
                || nums2.length == 0) {
                return result;
            }
            Queue<Pair> meanHeap = new PriorityQueue<>();
            meanHeap.offer(new Pair(0, 0, nums1[0] + nums2[0]));
```

```
            boolean[][] visited = new boolean[nums1.length][nums2.length];
            visited[0][0] = true;//we start form (0,0), so mark it as visited
            while (k > 0 && !meanHeap.isEmpty()) {
                Pair pair = meanHeap.poll();
                result.add(new int[] {nums1[pair.row], nums2[pair.col]});
                k--;
                for (int[] neighbor : neighbors) {
                    int nextRow = pair.row + neighbor[0];
                    int nextCol = pair.col + neighbor[1];
                    if (nextRow < 0
                        || nextCol < 0
                        || nextRow >= nums1.length
                        || nextCol >= nums2.length
                        || visited[nextRow][nextCol]) {
                        continue;
                    }
                    visited[nextRow][nextCol] = true;
                    meanHeap.offer(new Pair(nextRow, nextCol, nums1[nextRow] +
nums2[nextCol]));
                }
            }

            return result;
        }

        class Pair implements Comparable<Pair> {
            int row;
            int col;
            int sum;

            public Pair(int row, int col, int sum) {
                this.row = row;
                this.col = col;
                this.sum = sum;
            }

            @Override
            public int compareTo(Pair that) {
                return this.sum - that.sum;
            }
        }
    }
}
```

357. `_374.java`

```
package com.fishercoder.solutions;

/**
 * 374. Guess Number Higher or Lower
 * We are playing the Guess Game. The game is as follows:
 * I pick a number from 1 to n. You have to guess which number I picked.
 * Every time you guess wrong, I'll tell you whether the number is higher or lower.
 * You call a pre-defined API guess(int num) which returns 3 possible results (-1, 1, or
0):

 -1 : My number is lower
 1 : My number is higher
 0 : Congrats! You got it!
 Example:
 n = 10, I pick 6.
 Return 6.

 */
public class _374 {
    public static class Solution1 {
        /**
         * The core problem/trouble to solve this problem is to figure out the problem
description: this
         * API: guess(int num) means to take your guess num and let you know if your
guessed num is
         * bigger or smaller than the answer. That's why if num > target, it returns -1
which means the
         * target is smaller than your guess!!!
         */

        public int guessNumber(int n) {
            int left = 1;
            int right = n;
            while (left + 1 < right) {
                int mid = left + (right - left) / 2;
                int g = guess(mid);
                if (g == 0) {
                    return mid;
                } else if (g > 0) {
                    left = mid;
                } else {
                    right = mid;
                }
            }
            if (guess(left) == 0) {
                return left;
            }
            return right;
        }

        /**
```

```
         * This is a fake guess method that I wrote just to compile/test, I'll have to
change it to
         * another number other than 6 based on the number to be found.
         */
        private int guess(int num) {
            if (num > 6) {
                return -1;
            } else if (num < 6) {
                return 1;
            } else {
                return 0;
            }
        }
    }

}
```

358. `_375.java`

```
package com.fishercoder.solutions;

/**
 * 375. Guess Number Higher or Lower II
 *
 * We are playing the Guess Game. The game is as follows:
 * I pick a number from 1 to n. You have to guess which number I picked.
 * Every time you guess wrong, I'll tell you whether the number I picked is higher or
lower.
 * However, when you guess a particular number x, and you guess wrong, you pay $x. You
win the game when you guess the number I picked.

 Example:

 n = 10, I pick 8.

 First round:  You guess 5, I tell you that it's higher. You pay $5.
 Second round: You guess 7, I tell you that it's higher. You pay $7.
 Third round:  You guess 9, I tell you that it's lower. You pay $9.

 Game over. 8 is the number I picked.

 You end up paying $5 + $7 + $9 = $21.
 Given a particular n ≥ 1, find out how much money you need to have to guarantee a win.

 Hint:

 The best strategy to play the game is to minimize the maximum loss you could possibly
face. Another strategy is to minimize the expected loss. Here, we are interested in the
first scenario.
 Take a small example (n = 3). What do you end up paying in the worst case?
 Check out this article if you're still stuck.
 The purely recursive implementation of minimax would be worthless for even a small n.
You MUST use dynamic programming.
 As a follow-up, how would you modify your code to solve the problem of minimizing the
expected loss, instead of the worst-case loss?
 */
public class _375 {
    public static class Solution1 {
        public int getMoneyAmount(int n) {
            int[][] table = new int[n + 1][n + 1];
            return dp(table, 1, n);
        }

        private int dp(int[][] table, int s, int e) {
            if (s >= e) {
                return 0;
            }
            if (table[s][e] != 0) {
                return table[s][e];
            }
```

```
            int res = Integer.MAX_VALUE;
            for (int i = s; i <= e; i++) {
                int temp = i + Math.max(dp(table, s, i - 1), dp(table, i + 1, e));
                res = Math.min(res, temp);
            }
            table[s][e] = res;
            return res;
        }
    }

    public static class Solution2 {
        public int getMoneyAmount(int n) {
            if (n == 1) {
                return 0;
            }
            int[][] dp = new int[n + 1][n + 1];
            for (int x = 1; x < n; x++) {
                for (int i = 0; i + x <= n; i++) {
                    int j = i + x;
                    dp[i][j] = Integer.MAX_VALUE;
                    for (int k = i; k <= j; k++) {
                        dp[i][j] = Math.min(dp[i][j], k + Math.max(k - 1 >= i ? dp[i][k -
1] : 0, j >= k + 1 ? dp[k + 1][j] : 0));
                    }
                }
            }
            for (int i = 0; i < n + 1; i++) {
                for (int j = 0; j < n + 1; j++) {
                    System.out.print(dp[i][j] + " ");
                }
                System.out.println();
            }
            return dp[1][n];
        }
    }
}
```

359. `_376.java`

```
package com.fishercoder.solutions;

/**
 * 376. Wiggle Subsequence
 *
 * A sequence of numbers is called a wiggle sequence if the differences between
successive numbers strictly
 * alternate between positive and negative.
 * The first difference (if one exists) may be either positive or negative.
 * A sequence with fewer than two elements is trivially a wiggle sequence.

 For example, [1,7,4,9,2,5] is a wiggle sequence because the differences
 (6,-3,5,-7,3) are alternately positive and negative.
 In contrast, [1,4,7,2,5] and [1,7,4,5,5] are not wiggle sequences,
 the first because its first two differences are positive and the second because its last
difference is zero.

 Given a sequence of integers, return the length of the longest subsequence that is a
wiggle sequence.
 A subsequence is obtained by deleting some number of elements (eventually, also zero)
 from the original sequence, leaving the remaining elements in their original order.

 Examples:
 Input: [1,7,4,9,2,5]
 Output: 6
 The entire sequence is a wiggle sequence.

 Input: [1,17,5,10,13,15,10,5,16,8]
 Output: 7
 There are several subsequences that achieve this length. One is [1,17,10,13,10,16,8].

 Input: [1,2,3,4,5,6,7,8,9]
 Output: 2

 Follow up:
 Can you do it in O(n) time?
 */
public class _376 {

    public static class Solution1 {
        public int wiggleMaxLength(int[] nums) {
            if (nums.length < 2) {
                return nums.length;
            }
            int prevDiff = nums[1] - nums[0];
            int count = (prevDiff != 0) ? 2 : 1;
            for (int i = 2; i < nums.length; i++) {
                int diff = nums[i] - nums[i - 1];
                /**ATTN: prevDiff could be zero. e.g. [3,3,3,2,5]
                 * but diff needs to exactly greater than zero*/
                if ((prevDiff <= 0 && diff > 0) || (prevDiff >= 0) && diff < 0) {
```

```
                    count++;
                    prevDiff = diff;
                }
            }
            return count;
        }
    }
}
```

360. `_377.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 377. Combination Sum IV
 *
 * Given an integer array with all positive numbers and no duplicates,
 * find the number of possible combinations that add up to a positive integer target.

 Example:

 nums = [1, 2, 3]
 target = 4

 The possible combination ways are:
 (1, 1, 1, 1)
 (1, 1, 2)
 (1, 2, 1)
 (1, 3)
 (2, 1, 1)
 (2, 2)
 (3, 1)

 Note that different sequences are counted as different combinations.

 Therefore the output is 7.

 Follow up:
 What if negative numbers are allowed in the given array?
 How does it change the problem?
 What limitation we need to add to the question to allow negative numbers?
 */

public class _377 {

    public static class Solution1 {
        /**
         * this normal backtracking recursive solution will end up in MLE by this
testcase: [4,2,1], 32
         */
        public int combinationSum4(int[] nums, int target) {
            List<List<Integer>> result = new ArrayList();
            Arrays.sort(nums);
            backtracking(nums, target, new ArrayList(), result);
            return result.size();
        }
```

```
        private void backtracking(int[] nums, int target, List<Integer> list,
                                  List<List<Integer>> result) {
            if (target == 0) {
                result.add(new ArrayList(list));
            } else if (target > 0) {
                for (int i = 0; i < nums.length; i++) {
                    list.add(nums[i]);
                    backtracking(nums, target - nums[i], list, result);
                    list.remove(list.size() - 1);
                }
            }
        }
    }

    public static class Solution2 {
        /**
         * Since we don't need to get all of the combinations, instead,
         * we only need to get the possible count, I can use only a count instead of
"List<List<Integer>> result"
         * However, it also ended up in TLE by this testcase: [1,2,3], 32
         */
        public static int count = 0;

        public int combinationSum4(int[] nums, int target) {
            Arrays.sort(nums);
            backtracking(nums, target, new ArrayList());
            return count;
        }

        private void backtracking(int[] nums, int target, List<Integer> list) {
            if (target == 0) {
                count++;
            } else if (target > 0) {
                for (int i = 0; i < nums.length; i++) {
                    list.add(nums[i]);
                    backtracking(nums, target - nums[i], list);
                    list.remove(list.size() - 1);
                }
            }
        }
    }

    public static class Solution3 {
        /**
         * Time: O(n^2)
         * Space: O(n)
         *
         * Since this question doesn't require to return all the combination result,
instead, it just wants one number, we could use DP
         * the idea is similar to Climbing Stairs.
         *
         * The idea is very clear as the code speaks for itself:
         * It's easy to find the routine
```

```
         * dp[0] = 0;
         * dp[1] = 1;
         * ...
         *
         * Reference: https://discuss.leetcode.com/topic/52186/my-3ms-java-dp-solution
         */
        public int combinationSum4(int[] nums, int target) {
            Arrays.sort(nums);
            int[] result = new int[target + 1];
            for (int i = 1; i < result.length; i++) {
                for (int num : nums) {
                    if (num > i) {
                        break;
                    } else if (num == i) {
                        result[i]++;
                    } else {
                        result[i] += result[i - num];
                    }
                }
            }
            return result[target];
        }
    }

    public static class Solution4 {
        /**
         * Time: O(n)
         * Space: O(n)
         *
         * Reference: https://discuss.leetcode.com/topic/52255/java-recursion-solution-
using-hashmap-as-memory
         */
        public static Map<Integer, Integer> map = new HashMap<>();//need to remove public
static before submitting on Leetcode as it doesn't reset static variables

        public int combinationSum4(int[] nums, int target) {
            if (nums == null || nums.length == 0 || target < 0) {
                return 0;
            }
            if (target == 0) {
                return 1;
            }
            if (map.containsKey(target)) {
                return map.get(target);
            }
            int count = 0;
            for (int num : nums) {
                count += combinationSum4(nums, target - num);
            }
            map.put(target, count);
            return count;
        }
    }
}
```

361. `_378.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 378. Kth Smallest Element in a Sorted Matrix
 * Given a n x n matrix where each of the rows and columns are
 * sorted in ascending order, find the kth smallest element in the matrix.

Note that it is the kth smallest element in the sorted order, not the kth distinct
element.

Example:

matrix = [
   [ 1,  5,  9],
   [10, 11, 13],
   [12, 13, 15]
],
k = 8,

return 13.

Note:
You may assume k is always valid, 1 ≤ k ≤ n2.
 */

public class _378 {
    public static class Solution1 {
        /**
         * brute force made it AC'ed, extreme test case needed for OJ
         */
        public int kthSmallest(int[][] matrix, int k) {
            List<Integer> list = new ArrayList();
            int n = matrix.length;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    list.add(matrix[i][j]);
                }
            }
            Collections.sort(list);
            return list.get(k - 1);
        }
    }

    public static class Solution2 {
        /**
         * Binary Search : The idea is to pick a mid number, then compare it with the
elements in each row, we start form
```

```
         * end of row util we find the element is less than the mid, the left side
element is all less than mid; keep tracking elements
         * that less than mid and compare with k, then update the k.
         */
        public int kthSmallestBS(int[][] matrix, int k) {
            int row = matrix.length - 1;
            int col = matrix[0].length - 1;
            int lo = matrix[0][0];
            int hi = matrix[row][col];
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                int count = 0;
                int j = col;
                for (int i = 0; i <= row; i++) {
                    while (j >= 0 && matrix[i][j] > mid) {
                        j--;
                    }
                    count += (j + 1);
                }
                if (count < k) {
                    lo = mid + 1;
                } else {
                    hi = mid;
                }
            }
            return lo;
        }
    }
}
```

362. `_379.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 379. Design Phone Directory
 *
 * Design a Phone Directory which supports the following operations:

 get: Provide a number which is not assigned to anyone.
 check: Check if a number is available or not.
 release: Recycle or release a number.
 Example:

 // Init a phone directory containing a total of 3 numbers: 0, 1, and 2.
 PhoneDirectory directory = new PhoneDirectory(3);

 // It can return any available phone number. Here we assume it returns 0.
 directory.get();

 // Assume it returns 1.
 directory.get();

 // The number 2 is available, so return true.
 directory.check(2);

 // It returns 2, the only number that is left.
 directory.get();

 // The number 2 is no longer available, so return false.
 directory.check(2);

 // Release number 2 back to the pool.
 directory.release(2);

 // Number 2 is available again, return true.
 directory.check(2);
 */
public class _379 {

    public static class Solution1 {
        private class PhoneDirectory {
            private Queue<Integer> phoneDir;
            private Set<Integer> used;

            /**
             * Initialize your data structure here
             *
```

```
             * @param maxNumbers - The maximum numbers that can be stored in the phone
directory.
             */
            public PhoneDirectory(int maxNumbers) {
                phoneDir = new LinkedList();
                int number = 0;
                while (maxNumbers-- > 0) {
                    phoneDir.add(number++);
                }
                used = new HashSet();
            }

            /**
             * Provide a number which is not assigned to anyone.
             *
             * @return - Return an available number. Return -1 if none is available.
             */
            public int get() {
                if (phoneDir.peek() == null) {
                    return -1;
                }
                int newNumber = phoneDir.poll();
                used.add(newNumber);
                return newNumber;
            }

            /**
             * Check if a number is available or not.
             */
            public boolean check(int number) {
                return !used.contains(number);
            }

            /**
             * Recycle or release a number.
             */
            public void release(int number) {
                if (used.remove(number)) {
                    phoneDir.add(number);
                }
            }
        }
    }
}
```

363. `_380.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import java.util.Random;

/**
 * 380. Insert Delete GetRandom O(1)
 * Design a data structure that supports all following operations in average O(1) time.
 *
 * insert(val): Inserts an item val to the set if not already present.
 * remove(val): Removes an item val from the set if present.
 * getRandom: Returns a random element from current set of elements. Each element must
have the same probability of being returned.
 * Example:
 *
 * // Init an empty set.
 * RandomizedSet randomSet = new RandomizedSet();
 *
 * // Inserts 1 to the set. Returns true as 1 was inserted successfully.
 * randomSet.insert(1);
 *
 * // Returns false as 2 does not exist in the set.
 * randomSet.remove(2);
 *
 * // Inserts 2 to the set, returns true. Set now contains [1,2].
 * randomSet.insert(2);
 *
 * // getRandom should return either 1 or 2 randomly.
 * randomSet.getRandom();
 *
 * // Removes 1 from the set, returns true. Set now contains [2].
 * randomSet.remove(1);
 *
 * // 2 was already in the set, so return false.
 * randomSet.insert(2);
 *
 * // Since 2 is the only number in the set, getRandom always return 2.
 * randomSet.getRandom();
 */

public class _380 {

    public static class Solution1 {
        /**
         * credit: https://leetcode.com/problems/insert-delete-getrandom-
o1/discuss/85401/Java-solution-using-a-HashMap-and-an-ArrayList-along-with-a-follow-up.-
(131-ms)
         */
```

```
        public static class RandomizedSet {
            Map<Integer, Integer> locationMap;
            List<Integer> list;
            Random random;

            /**
             * Initialize your data structure here.
             */
            public RandomizedSet() {
                locationMap = new HashMap<>();
                list = new ArrayList<>();
                random = new Random();
            }

            /**
             * Inserts a value to the set. Returns true if the set did not already
contain the specified element.
             */
            public boolean insert(int val) {
                if (locationMap.containsKey(val)) {
                    return false;
                } else {
                    locationMap.put(val, list.size());
                    list.add(val);
                    return true;
                }
            }

            /**
             * Removes a value from the set. Returns true if the set contained the
specified element.
             */
            public boolean remove(int val) {
                if (!locationMap.containsKey(val)) {
                    return false;
                } else {
                    int location = locationMap.get(val);
                    /**if it's not the last one, then swap the last one with this val*/
                    if (location < list.size() - 1) {
                        int lastOne = list.get(list.size() - 1);
                        list.set(location, lastOne);
                        locationMap.put(lastOne, location);
                    }
                    locationMap.remove(val);
                    list.remove(list.size() - 1);
                    return true;
                }
            }

            /**
             * Get a random element from the set.
             */
            public int getRandom() {
                return list.get(random.nextInt(list.size()));
```

```
            }

        }
    }

    /**
     * Your _380 object will be instantiated and called as such:
     * _380 obj = new _380();
     * boolean param_1 = obj.insert(val);
     * boolean param_2 = obj.delete(val);
     * int param_3 = obj.getRandom();
     */
    public static class Solution2 {
        /**This is not ideal and not meeting average O(1) time for all three
operations.*/
        public static class RandomizedSet {

            Map<Integer, Integer> forwardMap;
                //key is auto increment index, value if the inserted val
            Map<Integer, Integer> reverseMap;//the other way around
            int index;
            Random random;

            /**
             * Initialize your data structure here.
             */
            public RandomizedSet() {
                forwardMap = new HashMap();
                reverseMap = new HashMap();
                index = 0;
                random = new Random();
            }

            /**
             * Inserts a value to the set. Returns true if the set did not already
contain the specified
             * element.
             */
            public boolean insert(int val) {
                if (forwardMap.containsValue(val)) {
                    return false;
                } else {
                    forwardMap.put(index, val);
                    reverseMap.put(val, index++);
                    return true;
                }
            }

            /**
             * Deletes a value from the set. Returns true if the set contained the
specified element.
             */
            public boolean remove(int val) {
                if (forwardMap.containsValue(val)) {
```

```
                    int key = reverseMap.get(val);
                    reverseMap.remove(val);
                    forwardMap.remove(key);
                    return true;
                } else {
                    return false;
                }
            }

            /**
             * Get a random element from the set.
             */
            public int getRandom() {
                int max = forwardMap.size();
                if (max == 1) {
                    return forwardMap.get(index - 1);
                }
                int randomNum = random.nextInt(max);
                while (!forwardMap.containsKey(randomNum)) {
                    randomNum = random.nextInt(max);
                }
                return forwardMap.get(randomNum);
            }
        }
    }
}
```

364. `_381.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.Random;

/**381. Insert Delete GetRandom O(1) - Duplicates allowed
 *
Design a data structure that supports all following operations in average O(1) time.

Note: Duplicate elements are allowed.
insert(val): Inserts an item val to the collection.
remove(val): Removes an item val from the collection if present.
getRandom: Returns a random element from current collection of elements. The probability
of each element being returned is linearly related to the number of same value the
collection contains.
Example:

// Init an empty collection.
_381 collection = new _381();

// Inserts 1 to the collection. Returns true as the collection did not contain 1.
collection.insert(1);

// Inserts another 1 to the collection. Returns false as the collection contained 1.
Collection now contains [1,1].
collection.insert(1);

// Inserts 2 to the collection, returns true. Collection now contains [1,1,2].
collection.insert(2);

// getRandom should return 1 with the probability 2/3, and returns 2 with the probability
1/3.
collection.getRandom();

// Removes 1 from the collection, returns true. Collection now contains [1,2].
collection.remove(1);

// getRandom should return 1 and 2 both equally likely.
collection.getRandom();*/
public class _381 {
    public static class Solution1 {

        Map<Integer, Integer> forwardMap;
        //key is the to-be-inserted number, value is its auto-incremented index
        Map<Integer, Integer> reverseMap;//the other way around
        int index;
        Random rand;

        /**
         * Initialize your data structure here.
```

```
         */
        public Solution1() {
            forwardMap = new HashMap();
            reverseMap = new HashMap();
            index = 0;
            rand = new Random();
        }

        /**
         * Inserts a value to the collection. Returns true if the collection did not
already contain the
         * specified element.
         */
        public boolean insert(int val) {
            boolean contains;
            if (reverseMap.containsValue(val)) {
                contains = true;
            } else {
                contains = false;
            }
            forwardMap.put(val,
                index);//this will overwrite the preivous key with a new index if the key
already exists
            reverseMap.put(index, val);
            index++;
            return contains;
        }

        /**
         * Removes a value from the collection. Returns true if the collection contained
the specified
         * element.
         */
        public boolean remove(int val) {
            boolean contains;
            if (reverseMap.containsValue(val)) {
                contains = true;
                if (forwardMap.containsKey(val)) {
                    int i = forwardMap.get(val);
                    forwardMap.remove(val);
                    reverseMap.remove(i);
                } else {
                    //remove the entry in revserve map that has val as its value
                    reverseMap.values().remove(val);
                }
            } else {
                contains = false;
            }
            return contains;
        }

        /**
         * Get a random element from the collection.
         */
```

```
        public int getRandom() {
            int randNum = rand.nextInt(index);
            while (!reverseMap.containsKey(randNum)) {
                randNum = rand.nextInt(index);
            }
            return reverseMap.get(randNum);
        }
    }
}
```

365. `_382.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.HashMap;
import java.util.Map;
import java.util.Random;

/**382. Linked List Random Node
 *
Given a singly linked list, return a random node's value from the linked list. Each node
must have the same probability of being chosen.

Follow up:
What if the linked list is extremely large and its length is unknown to you? Could you
solve this efficiently without using extra space?

Example:

// Init a singly linked list [1,2,3].
ListNode head = new ListNode(1);
head.next = new ListNode(2);
head.next.next = new ListNode(3);
Solution solution = new Solution(head);

// getRandom() should return either 1, 2, or 3 randomly. Each element should have equal
probability of returning.
solution.getRandom();
*/
public class _382 {

    public static class Solution1 {
        private Map<Integer, ListNode> map;
        private Random rand;

        /**
         * @param head The linked list's head. Note that the head is guanranteed to be
not null, so it contains at least one node.
         */
        public Solution1(ListNode head) {
            map = new HashMap();
            rand = new Random();
            int i = 0;
            while (head != null) {
                map.put(i++, head);
                head = head.next;
            }
        }

        /**
         * Returns a random node's value.
```

```
         */
        public int getRandom() {
            return map.get(rand.nextInt(map.size())).val;
        }
    }
}
```

366. `_383.java`

```
package com.fishercoder.solutions;

/**
 * 383 Ransom Note
 *
 * Given an arbitrary ransom note string and another string containing letters
 * from all the magazines, write a function that will return true if
 * the ransom note can be constructed from the magazines ; otherwise, it will return
false.
 * Each letter in the magazine string can only be used once in your ransom note.

 Note:

 You may assume that both strings contain only lowercase letters.
 canConstruct("a", "b") -> false
 canConstruct("aa", "ab") -> false
 canConstruct("aa", "aab") -> true
*/
public class _383 {

    public static class Solution1 {
        public boolean canConstruct(String ransomNote, String magazine) {
            char[] mchars = magazine.toCharArray();
            int[] mcnt = new int[256];
            for (int i = 0; i < mchars.length; i++) {
                mcnt[mchars[i] - 'a']++;
            }

            char[] rchars = ransomNote.toCharArray();
            for (int i = 0; i < rchars.length; i++) {
                if (mcnt[rchars[i] - 'a'] <= 0) {
                    return false;
                }
                mcnt[rchars[i] - 'a']--;
            }
            return true;
        }
    }

}
```

367. `_384.java`

```
package com.fishercoder.solutions;
/**
 * 384. Shuffle an Array
Shuffle a set of numbers without duplicates.

Example:

// Init an array with set 1, 2, and 3.
int[] nums = {1,2,3};
Solution solution = new Solution(nums);

// Shuffle the array [1,2,3] and return its result.
 Any permutation of [1,2,3] must equally likely to be returned.
solution.shuffle();

// Resets the array back to its original configuration [1,2,3].
solution.reset();

// Returns the random shuffling of array [1,2,3].
solution.shuffle();*/
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.Random;

public class _384 {

    public static class Solution1 {
        //Note: the problem states that this is a set without duplicates which makes
building all combinations easier

        private List<List<Integer>> combinations;
        private int[] original;
        private Random random;

        public Solution1(int[] nums) {
            original = nums;
            random = new Random();
            combinations = buildAllComb(nums);
        }

        //insert next value into all possible positions, I wrote this method myself, of
course it could be simplified to not use a queue
        //but it just naturally came into my mind that I used a queue
        private List<List<Integer>> buildAllComb(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null || nums.length == 0) {
                return result;
            }
```

```
            List<Integer> list = new ArrayList<Integer>();
            list.add(nums[0]);
            Queue<List<Integer>> q = new LinkedList();
            q.offer(list);
            for (int i = 1; i < nums.length; i++) {
                int qSize = q.size();
                for (int k = 0; k < qSize; k++) {
                    List<Integer> currList = q.poll();
                    for (int j = 0; j <= currList.size(); j++) {
                        List<Integer> newL = new ArrayList<Integer>(currList);
                        newL.add(j, nums[i]);
                        q.offer(newL);
                    }
                }
            }
            while (!q.isEmpty()) {
                result.add(q.poll());
            }
            return result;
        }

        /**
         * Resets the array to its original configuration and return it.
         */
        public int[] reset() {
            return original;
        }

        /**
         * Returns a random shuffling of the array.
         */
        public int[] shuffle() {
            if (original == null || original.length == 0) {
                return original;
            }
            int randomIndex = random.nextInt(combinations.size());
            List<Integer> list = combinations.get(randomIndex);
            int[] result = new int[list.size()];
            for (int i = 0; i < list.size(); i++) {
                result[i] = list.get(i);
            }
            return result;
        }
    }
}
```

368. `_385.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.Stack;

/**
 * 385. Mini Parser
 *
 * Given a nested list of integers represented as a string,
 * implement a parser to deserialize it.

 Each element is either an integer, or a list --
 whose elements may also be integers or other lists.

 Note: You may assume that the string is well-formed:

 String is non-empty.
 String does not contain white spaces.
 String contains only digits 0-9, [, - ,, ].
 Example 1:

 Given s = "324",

 You should return a NestedInteger object which contains a single integer 324.
 Example 2:

 Given s = "[123,[456,[789]]]",

 Return a NestedInteger object containing a nested list with 2 elements:

 1. An integer containing value 123.
 2. A nested list containing two elements:
 i.  An integer containing value 456.
 ii. A nested list with one element:
 a. An integer containing value 789.*/
public class _385 {

    public static class Solution1 {
        //Lessons: ask the interviewer to clarify the input, for this question, the input
could be "324", "[324]", they are different
        //the former should return a nested integer with one single integer, the latter
should return a nested integer with a list

        //Idea:
        //if it's '[', we just construct a new nested integer and push it onto the stack
        //if it's a number, we parse the whole number and add to the previous nested
integer object
        //if it's ',', we'll just continue;
        //if it's ']', we'll just pop one nested integer from the working stack and
assign it to the result
```

```
        public NestedInteger deserialize(String s) {
            if (s == null || s.isEmpty() || s.length() == 0) {
                return new NestedInteger();
            }
            Stack<NestedInteger> workStack = new Stack<>();
            NestedInteger result = null;
            StringBuilder sb = new StringBuilder();
            int i = 0;
            //if it's just a single number, then we'll just return a nested integer with
one integer
            if (s.charAt(i) != '[') {
                sb.setLength(0);
                while (i < s.length() && ((Character.getNumericValue(s.charAt(i)) < 10
                    && Character.getNumericValue(s.charAt(i)) >= 0) || s.charAt(i) == '-
')) {
                    sb.append(s.charAt(i));
                    i++;
                }
                int num = Integer.parseInt(sb.toString());
                return new NestedInteger(num);
            } else {
                //all other cases, we'll return a nested integer with a list
                while (i < s.length()) {
                    if (s.charAt(i) == '[') {
                        NestedInteger ni = new NestedInteger();
                        // we'll put this one into its last one if there's one on the
workStack
                        if (!workStack.isEmpty()) {
                            NestedInteger lastNi = workStack.pop();
                            lastNi.add(ni);
                            workStack.push(lastNi);// then push it back
                        }
                        workStack.push(ni);
                        i++;
                    } else if (s.charAt(i) == ',') {
                        i++;
                    } else if (s.charAt(i) == ']') {
                        NestedInteger completedNi = workStack.pop();
                        result = completedNi;
                        i++;
                    } else {
                        // then it must be a number
                        sb.setLength(0);
                        while (i < s.length()
                            && ((Character.getNumericValue(s.charAt(i)) < 10 && Character
                            .getNumericValue(s.charAt(i)) >= 0) || s.charAt(i) == '-')) {
                            sb.append(s.charAt(i));
                            i++;
                        }
                        int num = Integer.parseInt(sb.toString());
                        NestedInteger ni = null;
                        if (!workStack.isEmpty()) {
                            ni = workStack.pop();
```

```
                        } else {
                            ni = new NestedInteger();
                        }
                        // case 1: if this one contains one integer
                        if (ni.isInteger()) {
                            // we'll add it to this ni
                            ni.add(new NestedInteger(num));
                        } else if (ni.getList() != null && ni.getList().size() != 0) {
                            // case 2: if this one contains a nested integer
                            // we'll get the last nested integer and add this one to it
                            ni.add(new NestedInteger(num));
                        } else {
                            // case 3: if this is an empty nested integer
                            if (i > 0) {
                                ni.add(new NestedInteger(num));
                            } else {
                                ni.setInteger(num);
                            }
                        }
                        workStack.push(ni);
                        if (i == s.length()) {
                            return ni;// this is for test cases like this: "324", there's
no '[' or ']'
                        }
                    }
                }
            }
            return result;
        }
    }
}
```

369. `_386.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 386. Lexicographical Numbers
 *
 * Given an integer n, return 1 - n in lexicographical order.
 *
 * For example, given 13, return: [1,10,11,12,13,2,3,4,5,6,7,8,9].
 *
 * Please optimize your algorithm to use less time and space. The input size may be as
large as
 * 5,000,000.
 */
public class _386 {
  public static class Solution1 {
    //Radix sort doesn't apply here! Don't confuse yourself!

    //rewrote their solution from Python to
Java:https://discuss.leetcode.com/topic/54986/python-memory-limit-exceeded-for-problem-
386/17
    public List<Integer> lexicalOrder(int n) {
      List<Integer> result = new ArrayList();
      int i = 1;
      while (true) {
        result.add(i);
        if (i * 10 <= n) {
          i *= 10;
        } else {
          while (i % 10 == 9 || i == n) {
            i /= 10;
          }
          if (i == 0) {
            return result;
          }
          i++;
        }
      }
    }
  }
}
```

370. `_387.java`

```
package com.fishercoder.solutions;

/**
 * 387. First Unique Character in a String Given a string, find the first non-repeating
character in
 * it and return it's index. If it doesn't exist, return -1.
 *
 * Examples:
 *
 * s = "leetcode" return 0.
 *
 * s = "loveleetcode", return 2. Note: You may assume the string contain only lowercase
letters.
 */

public class _387 {
  public static class Solution1 {
    public static int firstUniqChar(String s) {
      int[] freq = new int[26];
      for (int i = 0; i < s.length(); i++) {
        freq[s.charAt(i) - 'a']++;
      }
      for (int i = 0; i < s.length(); i++) {
        if (freq[s.charAt(i) - 'a'] == 1) {
          return i;
        }
      }
      return -1;
    }
  }
}
```

371. `_388.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 388. Longest Absolute File Path
 *
 * Suppose we abstract our file system by a string in the following manner:
 *
 * The string "dir\n\tsubdir1\n\tsubdir2\n\t\tfile.ext" represents:
 *
 * dir
 *     subdir1
 *     subdir2
 *         file.ext
 * The directory dir contains an empty sub-directory subdir1 and a sub-directory subdir2
containing a file file.ext.
 *
 * The string
"dir\n\tsubdir1\n\t\tfile1.ext\n\t\tsubsubdir1\n\tsubdir2\n\t\tsubsubdir2\n\t\t\tfile2.ex
t" represents:
 *
 * dir
 *     subdir1
 *         file1.ext
 *         subsubdir1
 *     subdir2
 *         subsubdir2
 *             file2.ext
 * The directory dir contains two sub-directories subdir1 and subdir2. subdir1 contains a
file file1.ext and an empty second-level sub-directory subsubdir1.
 * subdir2 contains a second-level sub-directory subsubdir2 containing a file file2.ext.
 *
 * We are interested in finding the longest (number of characters) absolute path to a
file within our file system.
 * For example, in the second example above, the longest absolute path is
"dir/subdir2/subsubdir2/file2.ext", and its length is 32 (not including the double
quotes).
 *
 * Given a string representing the file system in the above format, return the length of
the longest absolute path to file in the abstracted file system. If there is no file in
the system, return 0.
 *
 * Note:
 *
 * The name of a file contains at least a . and an extension.
 * The name of a directory or sub-directory will not contain a ..
 * Time complexity required: O(n) where n is the size of the input string.
 *
 * Notice that a/aa/aaa/file1.txt is not the longest file path, if there is another path
aaaaaaaaaaaaaaaaaaaaa/sth.png.*/
```

```
public class _388 {
    public static class Solution1 {
        public int lengthLongestPath(String input) {
            Stack<Integer> stack = new Stack();
            int longestLen = 0;
            int currDirLen = 0;
            int i = 0;
            int currLevel = 0;
            int nextLevel = 0;
            boolean isFile = false;
            Character period = '.';
            Character space = ' ';
            while (i < input.length()) {
                currLevel = nextLevel;
                int currStrLen = 0;
                while (i < input.length() && (Character.isLetterOrDigit(input.charAt(i))
                    || period.equals(input.charAt(i)) || space.equals(input.charAt(i))))
{
                    if (period.equals(input.charAt(i))) {
                        isFile = true;
                    }
                    i++;
                    currStrLen++;
                }
                if (isFile) {
                    longestLen = Math.max(longestLen, currDirLen + currStrLen);
                } else {
                    currDirLen += currStrLen + 1;
                    stack.push(currStrLen + 1);
                }

                nextLevel = 0;
                i = i + 1;//increment one to let it pass "\n" and start from "\t"
                while (i < input.length() - 1 && input.substring(i, i + 1).equals("\t"))
{
                    nextLevel++;
                    i = i + 1;
                }

                if (nextLevel < currLevel) {
                    int j = 0;
                    if (isFile) {
                        while (!stack.isEmpty() && j < (currLevel - nextLevel)) {
                            currDirLen -= stack.pop();
                            j++;
                        }
                    } else {
                        while (!stack.isEmpty() && j <= (currLevel - nextLevel)) {
                            currDirLen -= stack.pop();
                            j++;
                        }
                    }
                } else if (nextLevel == currLevel) {
                    if (!isFile && !stack.isEmpty()) {
```

```
                    currDirLen -= stack.pop();
                }
            }

            if (nextLevel == 0) {
                currDirLen = 0;
                stack.clear();
            }

            isFile = false;
        }

        return longestLen;
    }
}
}
```

372. `_389.java`

```
package com.fishercoder.solutions;
/**
 * 389. Find the Difference
 *
 * Given two strings s and t which consist of only lowercase letters.

 String t is generated by random shuffling string s and then add
 one more letter at a random position.

 Find the letter that was added in t.

 Example:

 Input:
 s = "abcd"
 t = "abcde"

 Output:
 e

 Explanation: 'e' is the letter that was added.*/

public class _389 {
    public static class Solution1 {
        public char findTheDifference(String s, String t) {
            int[] counts = new int[128];
            char[] schars = s.toCharArray();
            char[] tchars = t.toCharArray();
            for (int i = 0; i < schars.length; i++) {
                counts[schars[i]]++;
            }
            for (int i = 0; i < tchars.length; i++) {
                counts[tchars[i]]--;
            }
            char result = 'a';
            for (int i = 0; i < 128; i++) {
                if (counts[i] != 0) {
                    result = (char) i;
                }
            }
            return result;
        }
    }
}
```

373. `_390.java`

```
package com.fishercoder.solutions;

/**
 * 390. Elimination Game
 *
 * There is a list of sorted integers from 1 to n.
 * Starting from left to right,
 * remove the first number and every other number afterward until you reach the end of
the list.
 * Repeat the previous step again, but this time from right to left,
 * remove the right most number and every other number from the remaining numbers.
 * We keep repeating the steps again, alternating left to right and right to left, until
a single number remains.
 * Find the last number that remains starting with a list of length n.

 Example:

 Input:
 n = 9,
 1 2 3 4 5 6 7 8 9
 2 4 6 8
 2 6
 6

 Output:
 6

 */
public class _390 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/55870/share-my-solutions-for-
contest-2 instead of
         * literally removing half of the elements in each scan, this solution is just
moving the
         * pointer to point to next start position So brilliant!
         */
        public int lastRemaining(int n) {
            int remaining = n;
            int start = 1;
            int step = 2;
            boolean forward = true;
            while (remaining > 1) {
                remaining /= 2;
                if (forward) {
                    start = start + step * remaining - step / 2;
                } else {
                    start = start - step * remaining + step / 2;
                }
                step *= 2;
```

```
                forward = !forward;
            }
            return start;
        }
    }
}
```

374. `_391.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 391. Perfect Rectangle
 *
 * Given N axis-aligned rectangles where N > 0, determine if they all together form an
exact cover of a rectangular region.
 Each rectangle is represented as a bottom-left point and a top-right point.
 For example, a unit square is represented as [1,1,2,2]. (coordinate of bottom-left point
is (1, 1) and top-right point is (2, 2)).

 Example 1:

 rectangles = [
 [1,1,3,3],
 [3,1,4,2],
 [3,2,4,4],
 [1,3,2,4],
 [2,3,3,4]
 ]

 Return true. All 5 rectangles together form an exact cover of a rectangular region.

 Example 2:

 rectangles = [
 [1,1,2,3],
 [1,3,2,4],
 [3,1,4,2],
 [3,2,4,4]
 ]

 Return false. Because there is a gap between the two rectangular regions.

 Example 3:

 rectangles = [
 [1,1,3,3],
 [3,1,4,2],
 [1,3,2,4],
 [3,2,4,4]
 ]

 Return false. Because there is a gap in the top center.

 Example 4:

 rectangles = [
```

```
 [1,1,3,3],
 [3,1,4,2],
 [1,3,2,4],
 [2,2,4,4]
 ]

 Return false. Because two of the rectangles overlap with each other.
 */
public class _391 {
    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/56052/really-easy-understanding-
solution-o-n-java */
        public boolean isRectangleCover(int[][] rectangles) {
            if (rectangles.length == 0 || rectangles[0].length == 0) {
                return false;
            }

            int x1 = Integer.MAX_VALUE;
            int x2 = Integer.MIN_VALUE;
            int y1 = Integer.MAX_VALUE;
            int y2 = Integer.MIN_VALUE;

            Set<String> set = new HashSet<>();
            int area = 0;

            for (int[] rect : rectangles) {
                x1 = Math.min(rect[0], x1);
                y1 = Math.min(rect[1], y1);
                x2 = Math.max(rect[2], x2);
                y2 = Math.max(rect[3], y2);

                area += (rect[2] - rect[0]) * (rect[3] - rect[1]);

                String s1 = rect[0] + " " + rect[1];
                String s2 = rect[0] + " " + rect[3];
                String s3 = rect[2] + " " + rect[3];
                String s4 = rect[2] + " " + rect[1];

                if (!set.add(s1)) {
                    set.remove(s1);
                }
                if (!set.add(s2)) {
                    set.remove(s2);
                }
                if (!set.add(s3)) {
                    set.remove(s3);
                }
                if (!set.add(s4)) {
                    set.remove(s4);
                }
            }

            if (!set.contains(x1 + " " + y1) || !set.contains(x1 + " " + y2) ||
!set.contains(
```

```
                x2 + " " + y1) || !set.contains(x2 + " " + y2) || set.size() != 4) {
                return false;
            }

            return area == (x2 - x1) * (y2 - y1);
        }
    }
}
```

375. `_392.java`

```
package com.fishercoder.solutions;

/**
 * 392. Is Subsequence
 *
 * Given a string s and a string t, check if s is subsequence of t.

 You may assume that there is only lower case English letters in both s and t. t is
potentially a very long (length ~= 500,000) string, and s is a short string (<=100).

 A subsequence of a string is a new string which is formed from the original string by
deleting some (can be none) of the characters without disturbing the relative positions
of the remaining characters. (ie, "ace" is a subsequence of "abcde" while "aec" is not).

 Example 1:
 s = "abc", t = "ahbgdc"

 Return true.

 Example 2:
 s = "axc", t = "ahbgdc"

 Return false.

 Follow up:
 If there are lots of incoming S, say S1, S2, ... , Sk where k >= 1B, and you want to
check one by one to see if T has its subsequence. In this scenario, how would you change
your code?

 */
public class _392 {

    public static class Solution1 {
        public boolean isSubsequence(String s, String t) {
            int left = 0;
            for (int i = 0; i < s.length(); i++) {
                boolean foundI = false;
                int j = left;
                for (; j < t.length(); j++) {
                    if (s.charAt(i) == t.charAt(j)) {
                        left = j + 1;
                        foundI = true;
                        break;
                    }
                }
                if (j == t.length() && !foundI) {
                    return false;
                }
            }
            return true;
        }
```

```
    }
}
```

376. `_393.java`

```
package com.fishercoder.solutions;

/**
 * 393. UTF-8 Validation
 *
 * A character in UTF8 can be from 1 to 4 bytes long, subjected to the following rules:

 For 1-byte character, the first bit is a 0, followed by its unicode code.
 For n-bytes character, the first n-bits are all one's, the n+1 bit is 0, followed by n-1
bytes with most significant 2 bits being 10.
 This is how the UTF-8 encoding would work:

 Char. number range  |        UTF-8 octet sequence
 (hexadecimal)    |              (binary)
 --------------------+---------------------------------------------
 0000 0000-0000 007F | 0xxxxxxx
 0000 0080-0000 07FF | 110xxxxx 10xxxxxx
 0000 0800-0000 FFFF | 1110xxxx 10xxxxxx 10xxxxxx
 0001 0000-0010 FFFF | 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
 Given an array of integers representing the data, return whether it is a valid utf-8
encoding.

 Note:
 The input is an array of integers. Only the least significant 8 bits of each integer is
used to store the data. This means each integer represents only 1 byte of data.

 Example 1:
 data = [197, 130, 1], which represents the octet sequence: 11000101 10000010 00000001.
 Return true.

 It is a valid utf-8 encoding for a 2-bytes character followed by a 1-byte character.


 Example 2:
 data = [235, 140, 4], which represented the octet sequence: 11101011 10001100 00000100.
 Return false.

 The first 3 bits are all one's and the 4th bit is 0 means it is a 3-bytes character.
 The next byte is a continuation byte which starts with 10 and that's correct.
 But the second continuation byte does not start with 10, so it is invalid.
 */
public class _393 {

    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/58338/bit-manipulation-java-6ms/4
*/
        public boolean validUtf8(int[] data) {
            int count = 0;
            for (int d : data) {
                if (count == 0) {
                    if ((d >> 5) == 0b110) {
```

```
                    count = 1;
                } else if ((d >> 4) == 0b1110) {
                    count = 2;
                } else if ((d >> 3) == 0b11110) {
                    count = 3;
                } else if ((d >> 7) == 1) {
                    return false;
                }
            } else {
                if ((d >> 6) != 0b10) {
                    return false;
                } else {
                    count--;
                }
            }
        }
        return count == 0;
    }
}

}
```

377. `_394.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**394. Decode String
 *
 * Given an encoded string, return it's decoded string.
 * The encoding rule is: k[encoded_string], where the encoded_string inside the square
brackets is being repeated exactly k times.
 * Note that k is guaranteed to be a positive integer.
 * You may assume that the input string is always valid;
 * No extra white spaces, square brackets are well-formed, etc.
 * Furthermore, you may assume that the original data does not contain any digits and
 * that digits are only for those repeat numbers, k. For example, there won't be input
like 3a or 2[4].
 *
 * Examples:
 * s = "3[a]2[bc]", return "aaabcbc".
 * s = "3[a2[c]]", return "accaccacc".
 * s = "2[abc]3[cd]ef", return "abcabccdcdcdef".
 * */

public class _394 {

    public static class Solution1 {
        public String decodeString(String s) {
            Stack<Integer> count = new Stack<>();
            Stack<String> str = new Stack<>();

            int idx = 0;
            str.push("");

            while (idx < s.length()) {
                if (s.charAt(idx) >= '0' && s.charAt(idx) <= '9') {
                    int start = idx;
                    while (s.charAt(idx + 1) >= '0' && s.charAt(idx + 1) <= '9') {
                        idx++;
                    }

                    count.push(Integer.parseInt(s.substring(start, idx + 1)));
                } else if (s.charAt(idx) == '[') {
                    str.push("");
                } else if (s.charAt(idx) == ']') {
                    String st = str.pop();
                    StringBuilder sb = new StringBuilder();
                    int n = count.pop();

                    for (int j = 0; j < n; j++) {
                        sb.append(st);
                    }
```

```
                    str.push(str.pop() + sb.toString());
                } else {
                    str.push(str.pop() + s.charAt(idx));
                }

                idx++;
            }

            return str.pop();
        }
    }
}
```

378. `_395.java`

```
package com.fishercoder.solutions;

/**
 * 395. Longest Substring with At Least K Repeating Characters
 *
 *  Find the length of the longest substring T of a given string
 *  (consists of lowercase letters only)
 *  such that every character in T appears no less than k times.

 Example 1:
 Input:
 s = "aaabb", k = 3

 Output:
 3

 The longest substring is "aaa", as 'a' is repeated 3 times.


 Example 2:
 Input:
 s = "ababbc", k = 2

 Output:
 5

 The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3
times.
 */

public class _395 {
    public static class Solution1 {
        /**Reference: https://discuss.leetcode.com/topic/57372/java-divide-and-conquer-
recursion-solution*/
        public int longestSubstring(String s, int k) {
            return findLongestSubstring(s.toCharArray(), 0, s.length(), k);
        }

        int findLongestSubstring(char[] chars, int start, int end, int k) {
            /**Base case 1 of 2*/
            if (end - start < k) {
                return 0;
            }
            int[] count = new int[26];
            for (int i = start; i < end; i++) {
                int index = chars[i] - 'a';
                count[index]++;
            }

            /**For every character in the above frequency table*/
            for (int i = 0; i < 26; i++) {
```

```
                if (count[i] < k && count[i] > 0) {
                    for (int j = start; j < end; j++) {
                        if (chars[j] == i + 'a') {
                            int left = findLongestSubstring(chars, start, j, k);
                            int right = findLongestSubstring(chars, j + 1, end, k);
                            return Math.max(left, right);
                        }
                    }
                }
            }
            /**Base case 2 of 2:
             * when any characters in this substring has repeated at least k times, then
this entire substring is a valid answer*/
            return end - start;
        }
    }
}
```

379. `_396.java`

```
package com.fishercoder.solutions;

/**
 * 396. Rotate Function
 *
 * Given an array of integers A and let n to be its length.

 Assume Bk to be an array obtained by rotating the array A k positions clock-wise, we
define a "rotation function" F on A as follow:

 F(k) = 0 * Bk[0] + 1 * Bk[1] + ... + (n-1) * Bk[n-1].

 Calculate the maximum value of F(0), F(1), ..., F(n-1).

 Note:
 n is guaranteed to be less than 105.

 Example:

 A = [4, 3, 2, 6]

 F(0) = (0 * 4) + (1 * 3) + (2 * 2) + (3 * 6) = 0 + 3 + 4 + 18 = 25
 F(1) = (0 * 6) + (1 * 4) + (2 * 3) + (3 * 2) = 0 + 4 + 6 + 6 = 16
 F(2) = (0 * 2) + (1 * 6) + (2 * 4) + (3 * 3) = 0 + 6 + 8 + 9 = 23
 F(3) = (0 * 3) + (1 * 2) + (2 * 6) + (3 * 4) = 0 + 2 + 12 + 12 = 26

 So the maximum value of F(0), F(1), F(2), F(3) is F(3) = 26.*/

public class _396 {
    public static class Solution1 {
        /**
         * F(k) = 0 * Bk[0] + 1 * Bk[1] + ... + (n-1) * Bk[n-1]
         */
        public int maxRotateFunction(int[] A) {
            if (A == null || A.length == 0) {
                return 0;
            }
            int[] F = new int[A.length];
            int[] B = A;
            int max = Integer.MIN_VALUE;
            for (int i = 0; i < A.length; i++) {
                F[i] = compute(B);
                max = Math.max(max, F[i]);
                B = rotate(B);
            }
            return max;
        }

        private int compute(int[] b) {
            int sum = 0;
            for (int i = 0; i < b.length; i++) {
```

```
                sum += i * b[i];
            }
            return sum;
        }

        private int[] rotate(int[] a) {
            int first = a[0];
            for (int i = 1; i < a.length; i++) {
                a[i - 1] = a[i];
            }
            a[a.length - 1] = first;
            return a;
        }
    }

    public static class Solution2 {
        /**
         * Reference: https://discuss.leetcode.com/topic/58459/java-o-n-solution-with-
explanation
         */
        public int maxRotateFunction(int[] A) {
            int allSum = 0;
            int len = A.length;
            int F = 0;
            for (int i = 0; i < len; i++) {
                F += i * A[i];
                allSum += A[i];
            }
            int max = F;
            for (int i = len - 1; i >= 1; i--) {
                F = F + allSum - len * A[i];
                max = Math.max(F, max);
            }
            return max;
        }
    }
}
```

380. `_397.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 397. Integer Replacement
 *
 * Given a positive integer n and you can do operations as follow:

 If n is even, replace n with n/2.
 If n is odd, you can replace n with either n + 1 or n - 1.
 What is the minimum number of replacements needed for n to become 1?

 Example 1:

 Input:
 8

 Output:
 3

 Explanation:
 8 -> 4 -> 2 -> 1
 Example 2:

 Input:
 7

 Output:
 4

 Explanation:
 7 -> 8 -> 4 -> 2 -> 1
 or
 7 -> 6 -> 3 -> 2 -> 1*/
public class _397 {

    public static class Solution1 {
        public int integerReplacement(int n) {
            long min = Long.MAX_VALUE;
            Set<long[]> set = new HashSet();
            Queue<long[]> q = new LinkedList();
            long[] pair = new long[] {n, 0};
            q.offer(pair);
            while (!q.isEmpty()) {
                int size = q.size();
                for (int i = 0; i < size; i++) {
```

```
                    long[] curr = q.poll();
                    if (curr[0] == 1) {
                        set.add(curr);
                    } else {

                        if (curr[0] % 2 == 0) {
                            curr[0] /= 2;
                            curr[1]++;
                            q.offer(curr);
                        } else {
                            long[] minus = new long[2];
                            minus[0] = curr[0] - 1;
                            minus[1] = curr[1] + 1;
                            q.offer(minus);

                            long[] plus = new long[2];
                            plus[0] = curr[0] + 1;
                            plus[1] = curr[1] + 1;
                            q.offer(plus);
                        }
                    }
                }
            }

            Iterator<long[]> it = set.iterator();
            while (it.hasNext()) {
                min = Math.min(min, it.next()[1]);
            }
            return (int) min;
        }
    }

}
```

381. `_398.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 398. Random Pick Index
 *
 * Given an array of integers with possible duplicates,
 * randomly output the index of a given target number. You can assume that the given
target number must exist in the array.

 Note:
 The array size can be very large. Solution that uses too much extra space will not pass
the judge.

 Example:

 int[] nums = new int[] {1,2,3,3,3};
 Solution solution = new Solution(nums);

 // pick(3) should return either index 2, 3, or 4 randomly. Each index should have equal
probability of returning.
 solution.pick(3);

 // pick(1) should return 0. Since in the array only nums[0] is equal to 1.
 solution.pick(1);*/
public class _398 {

    //TODO: use reservoir sampling to solve it again

    public static class Solution {
        //brute force
        int[] input;
        java.util.Random rand = new java.util.Random();

        public Solution(int[] nums) {
            input = nums;
        }

        public int pick(int target) {
            List<Integer> list = new ArrayList();
            for (int i = 0; i < input.length; i++) {
                if (input[i] == target) {
                    list.add(i);
                }
            }
            if (list.size() == 1) {
                return list.get(0);
            }
            int randomIndex = rand.nextInt(list.size());
```

```
            return list.get(randomIndex);
        }
    }
}
```

382. `_399.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;

/**
 * 399. Evaluate Division
 *
 * Equations are given in the format A / B = k, where A and B are variables represented
as strings, and k is a real number (floating point number).
 * Given some queries, return the answers. If the answer does not exist, return -1.0.

 Example:
 Given a / b = 2.0, b / c = 3.0.
 queries are: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ? .
 return [6.0, 0.5, -1.0, 1.0, -1.0 ].

 The input is: vector<pair<string, string>> equations, vector<double>& values,
vector<pair<string, string>> queries ,
 where equations.size() == values.size(), and the values are positive. This represents
the equations. Return vector<double>.

 According to the example above:

 equations = [ ["a", "b"], ["b", "c"] ],
 values = [2.0, 3.0],
 queries = [ ["a", "c"], ["b", "a"], ["a", "e"], ["a", "a"], ["x", "x"] ].

 The input is always valid. You may assume that evaluating the queries will result in no
division by zero and there is no contradiction.
 */
public class _399 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/59146/java-ac-solution-using-graph
         *
         * Image a/b = k as a link between node a and b, the weight from a to b is k, the
reverse link
         * is 1/k. Query is to find a path between two nodes.
         */
        public double[] calcEquation(String[][] equations, double[] values, String[][]
queries) {
            Map<String, List<String>> pairs = new HashMap<>();
            Map<String, List<Double>> valuePairs = new HashMap<>();
            for (int i = 0; i < equations.length; i++) {
                String[] equation = equations[i];
                if (!pairs.containsKey(equation[0])) {
```

```
                    pairs.put(equation[0], new ArrayList<>());
                    valuePairs.put(equation[0], new ArrayList<>());
                }
                if (!pairs.containsKey(equation[1])) {
                    pairs.put(equation[1], new ArrayList<>());
                    valuePairs.put(equation[1], new ArrayList<>());
                }
                pairs.get(equation[0]).add(equation[1]);
                pairs.get(equation[1]).add(equation[0]);
                valuePairs.get(equation[0]).add(values[i]);
                valuePairs.get(equation[1]).add(1 / values[i]);
            }

            double[] result = new double[queries.length];
            for (int i = 0; i < queries.length; i++) {
                String[] query = queries[i];
                result[i] = dfs(query[0], query[1], pairs, valuePairs, new HashSet<>(),
1.0);
                if (result[i] == 0.0) {
                    result[i] = -1.0;
                }
            }
            return result;
        }

        private double dfs(String start, String end, Map<String, List<String>> pairs,
            Map<String, List<Double>> valuePairs, HashSet<String> set, double value) {
            if (set.contains(start)) {
                return 0.0;
            }
            if (!pairs.containsKey(start)) {
                return 0.0;
            }
            if (start.equals(end)) {
                return value;
            }
            set.add(start);

            List<String> stringList = pairs.get(start);
            List<Double> valueList = valuePairs.get(start);
            double tmp = 0.0;
            for (int i = 0; i < stringList.size(); i++) {
                tmp = dfs(stringList.get(i), end, pairs, valuePairs, set, value *
valueList.get(i));
                if (tmp != 0.0) {
                    break;
                }
            }
            set.remove(start);
            return tmp;
        }
    }
}
```

383. `_400.java`

```
package com.fishercoder.solutions;

/**
 * 400. Nth Digit
 *
 * Find the nth digit of the infinite integer sequence 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
...

 Note:
 n is positive and will fit within the range of a 32-bit signed integer (n < 231).

 Example 1:
 Input:
 3
 Output:
 3

 Example 2:
 Input:
 11
 Output:
 0

 Explanation:
 The 11th digit of the sequence 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ... is a 0, which is
part of the number 10.
 */
public class _400 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/59314/java-solution:
         *
         * 1. find the length of the number where the nth digit is from 2. find the
actual number where
         * the nth digit is from 3. find the nth digit and return
         */
        public int findNthDigit(int n) {
            int len = 1;
            long count = 9;
            int start = 1;

            while (n > len * count) {
                n -= len * count;
                len += 1;
                count *= 10;
                start *= 10;
            }

            start += (n - 1) / len;
            String s = Integer.toString(start);
```

```
            return Character.getNumericValue(s.charAt((n - 1) % len));
        }
    }
}
```

384. `_401.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 401. Binary Watch
 *
 * A binary watch has 4 LEDs on the top which represent the hours (0-11), and the 6 LEDs
on the bottom represent the minutes (0-59).
 * Each LED represents a zero or one, with the least significant bit on the right.

 For example, the above binary watch reads "3:25".

 Given a non-negative integer n which represents the number of LEDs that are currently
on, return all possible times the watch could represent.

 Example:

 Input: n = 1
 Return: ["1:00", "2:00", "4:00", "8:00", "0:01", "0:02", "0:04", "0:08", "0:16", "0:32"]
 Note:
 The order of output does not matter.
 The hour must not contain a leading zero, for example "01:00" is not valid, it should be
"1:00".
 The minute must be consist of two digits and may contain a leading zero, for example
"10:2" is not valid, it should be "10:02".
 */
public class _401 {

    public static class Solution1 {
        public List<String> readBinaryWatch(int num) {
            List<String> times = new ArrayList<>();
            for (int h = 0; h < 12; h++) {
                for (int m = 0; m < 60; m++) {
                    if (Integer.bitCount(h * 64 + m) == num) {
                        times.add(String.format("%d:%02d", h,
                            m));//%02 means to pad this two-digit decimal number on the
left with zeroes
                    }
                }
            }
            return times;
        }
    }
}
```

385. `_402.java`

```
package com.fishercoder.solutions;

/**
 * 402. Remove K Digits
 *
 * Given a non-negative integer num represented as a string, remove k digits from the
number so that the new number is the smallest possible.

 Note:
 The length of num is less than 10002 and will be ≥ k.
 The given num does not contain any leading zero.
 Example 1:

 Input: num = "1432219", k = 3
 Output: "1219"
 Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is
the smallest.
 Example 2:

 Input: num = "10200", k = 1
 Output: "200"
 Explanation: Remove the leading 1 and the number is 200. Note that the output must not
contain leading zeroes.
 Example 3:

 Input: num = "10", k = 2
 Output: "0"
 Explanation: Remove all the digits from the number and it is left with nothing which is
0.
 */
public class _402 {
    public static class Solution1 {

        /** credit: https://discuss.leetcode.com/topic/59412/a-greedy-method-using-stack-
o-n-time-and-o-n-space */
        public String removeKdigits(String num, int k) {
            int digits = num.length() - k;
            char[] stack = new char[num.length()];
            int top = 0;

            for (int i = 0; i < num.length(); i++) {
                char c = num.charAt(i);
                while (top > 0 && stack[top - 1] > c && k > 0) {
                    top--;
                    k--;
                }
                stack[top++] = c;
            }

            int index = 0;
            while (index < digits && stack[index] == '0') {
```

```
                index++;
            }
            return index == digits ? "0" : new String(stack, index, digits - index);
        }
    }
}
```

386. `_403.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 403. Frog Jump
 *
 * A frog is crossing a river.
 * The river is divided into x units and at each unit there may or may not exist a stone.
 * The frog can jump on a stone, but it must not jump into the water.

 Given a list of stones' positions (in units) in sorted ascending order,
 determine if the frog is able to cross the river by landing on the last stone.
 Initially, the frog is on the first stone and assume the first jump must be 1 unit.

 If the frog's last jump was k units, then its next jump must be either k - 1, k, or k +
1 units.
 Note that the frog can only jump in the forward direction.

 Note:

 The number of stones is ≥ 2 and is < 1,100.
 Each stone's position will be a non-negative integer < 231.
 The first stone's position is always 0.

 Example 1:

 [0,1,3,5,6,8,12,17]

 There are a total of 8 stones.
 The first stone at the 0th unit, second stone at the 1st unit,
 third stone at the 3rd unit, and so on...
 The last stone at the 17th unit.

 Return true. The frog can jump to the last stone by jumping
 1 unit to the 2nd stone, then 2 units to the 3rd stone, then
 2 units to the 4th stone, then 3 units to the 6th stone,
 4 units to the 7th stone, and 5 units to the 8th stone.


 Example 2:

 [0,1,2,3,4,8,9,11]

 Return false. There is no way to jump to the last stone as
 the gap between the 5th and 6th stone is too large.

 */
```

```
public class _403 {

    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/59903/very-easy-to-understand-
java-solution-with-explanations/2
         * and https://leetcode.com/articles/frog-jump/#approach-5-using-dynamic-
programmingaccepted
         */
        public boolean canCross(int[] stones) {
            if (stones.length == 0) {
                return true;
            }
            Map<Integer, Set<Integer>> map = new HashMap<>(stones.length);
            map.put(0, new HashSet<>());
            map.get(0).add(1);
            for (int i = 1; i < stones.length; i++) {
                map.put(stones[i], new HashSet<>());
            }

            for (int i = 0; i < stones.length; i++) {
                int stone = stones[i];
                for (int step : map.get(stone)) {
                    int reach = step + stone;
                    if (reach == stones[stones.length - 1]) {
                        return true;
                    }
                    Set<Integer> set = map.get(reach);
                    if (set != null) {
                        set.add(step);
                        if (step - 1 > 0) {
                            set.add(step - 1);
                        }
                        set.add(step + 1);
                    }
                }
            }
            return false;
        }
    }
}
```

387. `_404.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
/**
 * 404. Sum of Left Leaves
 *
 * Find the sum of all left leaves in a given binary tree.

 Example:

   3
  / \
 9  20
   /  \
  15   7

 There are two left leaves in the binary tree, with values 9 and 15 respectively. Return
24.*/
public class _404 {
    public static class Solution1 {
        public int sumOfLeftLeaves(TreeNode root) {
            int result = 0;
            if (root == null) {
                return result;
            }
            return dfs(root, result, false);
        }

        private int dfs(TreeNode root, int result, boolean left) {
            if (root.left == null && root.right == null && left) {
                result += root.val;
                return result;
            }
            int leftResult = 0;
            if (root.left != null) {
                left = true;
                leftResult = dfs(root.left, result, left);
            }
            int rightResult = 0;
            if (root.right != null) {
                left = false;
                rightResult = dfs(root.right, result, left);
            }
            return leftResult + rightResult;
        }
    }

    public static class Solution2 {

        public int sumOfLeftLeaves(TreeNode root) {
            int sum = 0;
```

```
            if (root == null) {
                return sum;
            }
            if (root.left != null) {
                if (root.left.left == null && root.left.right == null) {
                    sum += root.left.val;
                } else {
                    sum += sumOfLeftLeaves(root.left);
                }
            }
            if (root.right != null) {
                sum += sumOfLeftLeaves(root.right);
            }
            return sum;
        }
    }
}
```

388. `_405.java`

```
package com.fishercoder.solutions;

/**
 * 405. Convert a Number to Hexadecimal

 Given an integer, write an algorithm to convert it to hexadecimal. For negative integer,
two’s complement method is used.

 Note:

 All letters in hexadecimal (a-f) must be in lowercase.
 The hexadecimal string must not contain extra leading 0s.
 If the number is zero, it is represented by a single zero character '0';
 otherwise, the first character in the hexadecimal string will not be the zero character.
 The given number is guaranteed to fit within the range of a 32-bit signed integer.
 You must not use any method provided by the library which converts/formats the number to
hex directly.

 Example 1:
 Input:
 26

 Output:
 "1a"

 Example 2:
 Input:
 -1

 Output:
 "ffffffff"

 */
public class _405 {

    public static class Solution1 {
        public String toHex(int num) {
            char[] hexChars =
                new char[] {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b',
'c', 'd',
                        'e', 'f'};
            String result = "";
            while (num != 0) {
                result = hexChars[(num & 15)] + result;
                num >>>= 4;
            }
            return result.isEmpty() ? "0" : result;
        }
    }
}
```

389. `_406.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Comparator;
import java.util.LinkedList;
import java.util.List;

/**
 * 406. Queue Reconstruction by Height
 *
 * Suppose you have a random list of people standing in a queue.
 * Each person is described by a pair of integers (h, k),
 * where h is the height of the person and k is the number of people in
 * front of this person who have a height greater than or equal to h. Write an algorithm
to reconstruct the queue.

 Note:
 The number of people is less than 1,100.

 Example

 Input:
 [[7,0], [4,4], [7,1], [5,0], [6,1], [5,2]]

 Output:
 [[5,0], [7,0], [5,2], [6,1], [4,4], [7,1]]

 */
public class _406 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/60437/java-solution-using-
priorityqueue-and-linkedlist
         */
        public int[][] reconstructQueue(int[][] people) {
            Arrays.sort(people, new Comparator<int[]>() {
                public int compare(int[] p1, int[] p2) {
                    return p1[0] != p2[0] ? Integer.compare(p2[0], p1[0])
                        : Integer.compare(p1[1], p2[1]);
                }
            });
            List<int[]> list = new LinkedList();
            for (int[] ppl : people) {
                list.add(ppl[1], ppl);
            }
            return list.toArray(new int[people.length][]);
        }
    }
}
```

390. `_407.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 407. Trapping Rain Water II
 *
 * Given an m x n matrix of positive integers representing the height of each unit cell
in a 2D elevation map, compute the volume of water it is able to trap after raining.

 Note:
 Both m and n are less than 110. The height of each unit cell is greater than 0 and is
less than 20,000.

 Example:

 Given the following 3x6 height map:
 [
 [1,4,3,1,3,2],
 [3,2,1,3,2,4],
 [2,3,3,2,3,1]
 ]

 Return 4.

 The above image represents the elevation map [[1,4,3,1,3,2],[3,2,1,3,2,4],[2,3,3,2,3,1]]
before the rain.

 After the rain, water are trapped between the blocks. The total volume of water trapped
is 4.

 */
public class _407 {
    public static class Solution1 {
        /** Reference: https://discuss.leetcode.com/topic/60418/java-solution-using-
priorityqueue */
        public class Cell {
            int row;
            int col;
            int height;

            public Cell(int row, int col, int height) {
                this.row = row;
                this.col = col;
                this.height = height;
            }
        }

        public int trapRainWater(int[][] heights) {
            if (heights == null || heights.length == 0 || heights[0].length == 0) {
                return 0;
```

```
            }

            PriorityQueue<Cell> queue = new PriorityQueue<>(1, (a, b) -> a.height -
b.height);

            int m = heights.length;
            int n = heights[0].length;
            boolean[][] visited = new boolean[m][n];

            // Initially, add all the Cells which are on borders to the queue.
            for (int i = 0; i < m; i++) {
                visited[i][0] = true;
                visited[i][n - 1] = true;
                queue.offer(new Cell(i, 0, heights[i][0]));
                queue.offer(new Cell(i, n - 1, heights[i][n - 1]));
            }

            for (int i = 0; i < n; i++) {
                visited[0][i] = true;
                visited[m - 1][i] = true;
                queue.offer(new Cell(0, i, heights[0][i]));
                queue.offer(new Cell(m - 1, i, heights[m - 1][i]));
            }

            // from the borders, pick the shortest cell visited and check its neighbors:
            // if the neighbor is shorter, collect the water it can trap and update its
height as its height plus the water trapped
            // add all its neighbors to the queue.
            int[][] dirs = new int[][] {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
            int res = 0;
            while (!queue.isEmpty()) {
                Cell cell = queue.poll();
                for (int[] dir : dirs) {
                    int row = cell.row + dir[0];
                    int col = cell.col + dir[1];
                    if (row >= 0 && row < m && col >= 0 && col < n && !visited[row][col])
{
                        visited[row][col] = true;
                        res += Math.max(0, cell.height - heights[row][col]);
                        queue.offer(new Cell(row, col, Math.max(heights[row][col],
cell.height)));
                    }
                }
            }

            return res;
        }
    }
}
```

391. `_408.java`

```
package com.fishercoder.solutions;

/**
 * 408. Valid Word Abbreviation
 *
 * Given a non-empty string s and an abbreviation abbr,
 * return whether the string matches with the given abbreviation.
 * A string such as "word" contains only the following valid abbreviations:

 ["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1",
"1o2", "2r1", "3d", "w3", "4"]
 Notice that only the above abbreviations are valid abbreviations of the string "word".
 Any other string is not a valid abbreviation of "word".

 Note:
 Assume s contains only lowercase letters and abbr contains only lowercase letters and
digits.

 Example 1:
 Given s = "internationalization", abbr = "i12iz4n":

 Return true.
 Example 2:
 Given s = "apple", abbr = "a2e":

 Return false.
 */
public class _408 {

    public static class Solution1 {
        public boolean validWordAbbreviation(String word, String abbr) {
            if (abbr.length() > word.length()) {
                return false;
            } else {
                char[] abbrChars = abbr.toCharArray();
                char[] wordChars = word.toCharArray();
                if (abbr.length() == word.length()) {
                    boolean prevDigit = false;
                    for (int i = 0, j = 0; i < abbrChars.length && j < wordChars.length;
i++, j++) {
                        if (Character.isDigit(abbrChars[i]) && !prevDigit) {
                            prevDigit = true;
                            if (Character.getNumericValue(abbrChars[i]) != 1) {
                                return false;
                            }
                        } else if (Character.isDigit(abbrChars[i]) && prevDigit) {
                            return false;
                        } else if (abbrChars[i] != wordChars[j]) {
                            return false;
                        } else if (prevDigit) {
                            prevDigit = false;
```

```
                        }
                    }
                    return true;
                } else {
                    StringBuilder stringBuilder = new StringBuilder();
                    boolean firstDigit = true;
                    for (int i = 0, j = 0; i < abbrChars.length && j < wordChars.length;
i++) {
                        while (i < abbrChars.length && Character.isDigit(abbrChars[i])) {
                            if (firstDigit && Character.getNumericValue(abbrChars[i]) ==
0) {
                                return false;
                            }
                            stringBuilder.append(abbrChars[i]);
                            i++;
                            firstDigit = false;
                        }
                        firstDigit = true;
                        if (!stringBuilder.toString().isEmpty()) {
                            int number = Integer.valueOf(stringBuilder.toString());
                            j += number;
                            stringBuilder.setLength(0);
                        }
                        if ((i >= abbrChars.length && j < wordChars.length) || (i <
abbrChars.length
                            && j >= wordChars.length)) {
                            return false;
                        }
                        if (i < abbrChars.length
                            && j < wordChars.length
                            && abbrChars[i] != wordChars[j]) {
                            return false;
                        }
                        if (j > wordChars.length && i <= abbrChars.length) {
                            return false;
                        }
                        j++;
                    }
                    return true;
                }
            }
        }
    }
}
```

392. `_409.java`

```
package com.fishercoder.solutions;

/**
 * 409. Longest Palindrome
 *
 * Given a string which consists of lowercase or uppercase letters, find the length of
the longest palindromes that can be built with those letters.
 * This is case sensitive, for example "Aa" is not considered a palindrome here.
 *
 * Note:
 * Assume the length of given string will not exceed 1,010.
 *
 * Example:
 * Input:
 * "abccccdd"
 *
 * Output:
 * 7
 *
 * Explanation:
 * One longest palindrome that can be built is "dccaccd", whose length is 7.
 */
public class _409 {
  public static class Solution1 {
    public int longestPalindrome(String s) {
      int[] counts = new int[56];
      for (char c : s.toCharArray()) {
        if (Character.isUpperCase(c)) {
          counts[c - 'A' + 33]++;
        } else {
          counts[c - 'a']++;
        }
      }
      boolean hasOdd = false;
      int len = 0;
      for (int i = 0; i < 56; i++) {
        if (counts[i] % 2 != 0) {
          hasOdd = true;
          if (counts[i] > 1) {
            len += counts[i] - 1;
          }
        } else {
          len += counts[i];
        }
      }
      return hasOdd ? len + 1 : len;
    }
  }
}
```

393. `_410.java`

```
package com.fishercoder.solutions;

/**
 * 410. Split Array Largest Sum
 *
 * Given an array which consists of non-negative integers and an integer m,
 * you can split the array into m non-empty continuous subarrays.
 * Write an algorithm to minimize the largest sum among these m subarrays.

 Note:
 If n is the length of array, assume the following constraints are satisfied:

 1 ≤ n ≤ 1000
 1 ≤ m ≤ min(50, n)

 Examples:

 Input:
 nums = [7,2,5,10,8]
 m = 2

 Output:
 18

 Explanation:
 There are four ways to split nums into two subarrays.
 The best way is to split it into [7,2,5] and [10,8],
 where the largest sum among the two subarrays is only 18.
 */
public class _410 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/61324/clear-explanation-8ms-binary-
search-java
         *
         * The answer is between maximum value of input array numbers and sum of those
numbers. Use
         * binary search to approach the correct answer. We have l = max number of array;
r = sum of all
         * numbers in the array; Every time we do mid = (l + r) / 2;
         *
         * Use greedy to narrow down left and right boundaries in binary search. 3.1 Cut
the array from
         * left. 3.2 Try our best to make sure that the sum of numbers between each two
cuts (inclusive)
         * is large enough but still less than mid. 3.3 We'll end up with two results:
either we can
         * divide the array into more than m subarrays or we cannot. If we can, it means
that the mid
         * value we pick is too small because we've already tried our best to make sure
```

```
each part holds
         * as many non-negative numbers as we can but we still have numbers left. So, it
is impossible
         * to cut the array into m parts and make sure each parts is no larger than mid.
We should
         * increase m. This leads to l = mid + 1; If we can't, it is either we
successfully divide the
         * array into m parts and the sum of each part is less than mid, or we used up
all numbers
         * before we reach m. Both of them mean that we should lower mid because we need
to find the
         * minimum one. This leads to r = mid - 1;
         */

        public int splitArray(int[] nums, int m) {
            int max = 0;
            long sum = 0;
            for (int num : nums) {
                max = Math.max(num, max);
                sum += num;
            }
            if (m == 1) {
                return (int) sum;
            }
            //binary search
            long l = max;
            long r = sum;
            while (l <= r) {
                long mid = (l + r) / 2;
                if (valid(mid, nums, m)) {
                    r = mid - 1;
                } else {
                    l = mid + 1;
                }
            }
            return (int) l;
        }

        public boolean valid(long target, int[] nums, int m) {
            int count = 1;
            long total = 0;
            for (int num : nums) {
                total += num;
                if (total > target) {
                    total = num;
                    count++;
                    if (count > m) {
                        return false;
                    }
                }
            }
            return true;
        }
```

```
    }
}
```

394. `_411.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 411. Minimum Unique Word Abbreviation
 *
 * A string such as "word" contains the following abbreviations:

 ["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1",
"1o2", "2r1", "3d", "w3", "4"]

 Given a target string and a set of strings in a dictionary,
 find an abbreviation of this target string with the smallest possible length such
 that it does not conflict with abbreviations of the strings in the dictionary.

 Each number or letter in the abbreviation is considered length = 1. For example, the
abbreviation "a32bc" has length = 4.

 Note:

 In the case of multiple answers as shown in the second example below, you may return any
one of them.
 Assume length of target string = m, and dictionary size = n. You may assume that m ≤ 21,
n ≤ 1000, and log2(n) + m ≤ 20.

 Examples:

 "apple", ["blade"] -> "a4" (because "5" or "4e" conflicts with "blade")

 "apple", ["plain", "amber", "blade"] -> "1p3" (other valid answers include "ap3", "a3e",
"2p2", "3le", "3l1").
 *
 */
public class _411 {
    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/61346/trie-bruteforce */
        class Trie {
            Trie[] children = new Trie[26];
            boolean isWord = false;
        }

        Trie root = new Trie();
        List<String> abbrs;

        public String minAbbreviation(String target, String[] dictionary) {
            for (String s : dictionary) {
                addTrie(s);
            }
```

```
        for (int i = 0; i < target.length(); i++) {
            abbrs = new ArrayList<>();
            abbrGenerator(target, 0, "", 0, i + 1);
            for (String s : abbrs) {
                if (search(s, root, 0, 0) == false) {
                    return s;
                }
            }
        }
        return "";
    }

    public void addTrie(String s) {
        Trie cur = root;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (cur.children[c - 'a'] == null) {
                cur.children[c - 'a'] = new Trie();
            }
            cur = cur.children[c - 'a'];
        }
        cur.isWord = true;
    }

    public boolean search(String target, Trie root, int i, int loop) {
        if (root == null) {
            return false;
        }

        if (loop != 0) {
            for (int a = 0; a < 26; a++) {
                if (search(target, root.children[a], i, loop - 1)) {
                    return true;
                }
            }
            return false;
        }
        if (i == target.length()) {
            if (root.isWord) {
                return true;
            }
            return false;
        }
        if (Character.isDigit(target.charAt(i))) {
            int tmp = 0;
            while (i < target.length() && Character.isDigit(target.charAt(i))) {
                tmp = tmp * 10 + target.charAt(i) - '0';
                i++;
            }
            return search(target, root, i, tmp);
        } else {
            return search(target, root.children[target.charAt(i) - 'a'], i + 1, 0);
        }
    }
```

```
        public void abbrGenerator(String target, int i, String tmp, int abbr, int num) {
            if (i == target.length()) {
                if (num == 0 && abbr == 0) {
                    abbrs.add(tmp);
                }
                if (num == 1 && abbr != 0) {
                    abbrs.add(tmp + abbr);
                }
                return;
            }
            if (num <= 0) {
                return;
            }
            char cur = target.charAt(i);
            abbrGenerator(target, i + 1, abbr == 0 ? tmp + cur : tmp + abbr + cur, 0,
                abbr == 0 ? num - 1 : num - 2);
            abbrGenerator(target, i + 1, tmp, abbr + 1, num);
        }
    }

}
```

395. `_412.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 412. Fizz Buzz
 *
 * Write a program that outputs the string representation of numbers from 1 to n.

 But for multiples of three it should output “Fizz” instead of the number and for
 the multiples of five output “Buzz”. For numbers which are multiples of both three and
five output “_412”.

 Example:

 n = 15,

 Return:
 [
 "1",
 "2",
 "Fizz",
 "4",
 "Buzz",
 "Fizz",
 "7",
 "8",
 "Fizz",
 "Buzz",
 "11",
 "Fizz",
 "13",
 "14",
 "_412"
 ]*/
public class _412 {

    public static class Solution1 {
        public List<String> fizzBuzz(int n) {
            List<String> result = new ArrayList();
            for (int i = 1; i <= n; i++) {
                if (i % 3 == 0 && i % 5 == 0) {
                    result.add("_412");
                } else if (i % 3 == 0) {
                    result.add("Fizz");
                } else if (i % 5 == 0) {
                    result.add("Buzz");
                } else {
                    result.add(Integer.toString(i));
                }
```

```
            }
            return result;
        }
    }
}
```

396. `_413.java`

```
package com.fishercoder.solutions;

/**
 * 413. Arithmetic Slices
 *
 * A sequence of number is called arithmetic if it consists of at least three elements
and if the difference between any two consecutive elements is the same.

 For example, these are arithmetic sequence:

 1, 3, 5, 7, 9
 7, 7, 7, 7
 3, -1, -5, -9

 The following sequence is not arithmetic.

 1, 1, 2, 5, 7

 A zero-indexed array A consisting of N numbers is given. A slice of that array is any
pair of integers (P, Q) such that 0 <= P < Q < N.

 A slice (P, Q) of array A is called arithmetic if the sequence:
 A[P], A[p + 1], ..., A[Q - 1], A[Q] is arithmetic. In particular, this means that P + 1
< Q.

 The function should return the number of arithmetic slices in the array A.

 Example:

 A = [1, 2, 3, 4]

 return: 3, for 3 arithmetic slices in A: [1, 2, 3], [2, 3, 4] and [1, 2, 3, 4] itself.
 */
public class _413 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/62884/2ms-java-o-n-time-o-1-space-
solution
        public int numberOfArithmeticSlices(int[] A) {
            int sum = 0;
            int len = 2;
            for (int i = 2; i < A.length; i++) {
                if (A[i] - A[i - 1] == A[i - 1] - A[i - 2]) {
                    len++;
                } else {
                    if (len > 2) {
                        sum += calculateSlices(len);
                    }
                    len = 2;//reset it to 2
                }
            }
```

```
            if (len > 2) {
                sum += calculateSlices(len);
            }
            return sum;
        }

        int calculateSlices(int len) {
            return (len - 1) * (len - 2) / 2;
        }
    }

    class Solution2 {
        //credit: https://discuss.leetcode.com/topic/63302/simple-java-solution-9-lines-
2ms
        public int numberOfArithmeticSlices(int[] A) {
            int sum = 0;
            int curr = 0;
            for (int i = 2; i < A.length; i++) {
                if (A[i] - A[i - 1] == A[i - 1] - A[i - 2]) {
                    curr++;
                    sum += curr;
                } else {
                    curr = 0;
                }
            }
            return sum;
        }
    }

}
```

397. `_414.java`

```
package com.fishercoder.solutions;

/**
 * 414. Third Maximum Number
 *
 * Given a non-empty array of integers, return the third maximum number in this array.
 * If it does not exist, return the maximum number. The time complexity must be in O(n).

 Example 1:
 Input: [3, 2, 1]

 Output: 1

 Explanation: The third maximum is 1.
 Example 2:
 Input: [1, 2]

 Output: 2

 Explanation: The third maximum does not exist, so the maximum (2) is returned instead.
 Example 3:
 Input: [2, 2, 3, 1]

 Output: 1

 Explanation: Note that the third maximum here means the third maximum distinct number.
 Both numbers with value 2 are both considered as second maximum.
 */
public class _414 {

    public static class Solution1 {
        public int thirdMax(int[] nums) {
            long max1 = Long.MIN_VALUE;
            long max2 = Long.MIN_VALUE;
            long max3 = Long.MIN_VALUE;
            for (int i : nums) {
                max1 = Math.max(max1, i);
            }
            for (int i : nums) {
                if (i == max1) {
                    continue;
                }
                max2 = Math.max(max2, i);
            }
            for (int i : nums) {
                if (i == max1 || i == max2) {
                    continue;
                }
                max3 = Math.max(max3, i);
            }
            return (int) (max3 == Long.MIN_VALUE ? max1 : max3);
```

```
        }
    }
}
```

398. `_415.java`

```
package com.fishercoder.solutions;

/**
 * 415. Add Strings
 *
 * Given two non-negative numbers num1 and num2 represented as string, return the sum of
num1 and num2.

 Note:

 1. The length of both num1 and num2 is < 5100.
 2. Both num1 and num2 contains only digits 0-9.
 3. Both num1 and num2 does not contain any leading zero.
 4. You must not use any built-in BigInteger library or convert the inputs to integer
directly.
 */
public class _415 {

    public static class Solution1 {
        public String addStrings(String num1, String num2) {
            if (num1 == null || num1.length() == 0) {
                return num2;
            } else if (num2 == null || num2.length() == 0) {
                return num1;
            }

            int i = num1.length() - 1;
            int j = num2.length() - 1;
            long carry = 0;
            long sum = 0;
            StringBuilder sb = new StringBuilder();
            char[] char1 = num1.toCharArray();
            char[] char2 = num2.toCharArray();
            while (i >= 0 || j >= 0) {
                sum = carry;
                if (i >= 0) {
                    sum += Character.getNumericValue(char1[i--]);
                }
                if (j >= 0) {
                    sum += Character.getNumericValue(char2[j--]);
                }
                carry = sum / 10;
                sb.append(sum % 10);
            }
            if (carry != 0) {
                sb.append(carry);
            }

            return sb.reverse().toString();
        }
    }
```

```
}
```

399. `_416.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 416. Partition Equal Subset Sum
 *
 * Given a non-empty array containing only positive integers,
 * find if the array can be partitioned into two subsets such that the sum of elements in
both subsets is equal.

 Note:

 Each of the array element will not exceed 100.
 The array size will not exceed 200.

 Example 1:

 Input: [1, 5, 11, 5]

 Output: true

 Explanation: The array can be partitioned as [1, 5, 5] and [11].


 Example 2:

 Input: [1, 2, 3, 5]

 Output: false

 Explanation: The array cannot be partitioned into equal sum subsets.
 */
public class _416 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/67539/0-1-knapsack-detailed-
explanation
         */
        public boolean canPartition(int[] nums) {
            int sum = 0;
            for (int num : nums) {
                sum += num;
            }

            if ((sum & 1) == 1) {
                return false;
            }

            sum /= 2;
```

```
            int n = nums.length;
            boolean[][] dp = new boolean[n + 1][sum + 1];
            for (int i = 0; i < dp.length; i++) {
                Arrays.fill(dp[i], false);
            }

            dp[0][0] = true;

            for (int i = 1; i < n + 1; i++) {
                dp[i][0] = true;
            }

            for (int j = 1; j < sum + 1; j++) {
                dp[0][j] = false;
            }

            for (int i = 1; i < n + 1; i++) {
                for (int j = 1; j < sum + 1; j++) {
                    dp[i][j] = dp[i - 1][j];
                    if (j >= nums[i - 1]) {
                        dp[i][j] = (dp[i][j] || dp[i - 1][j - nums[i - 1]]);
                    }
                }
            }

            return dp[n][sum];
        }
    }

}
```

400. `_417.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
/**
 * 417. Pacific Atlantic Water Flow
 *
 * Given the following 5x5 matrix:

  Pacific ~   ~   ~   ~   ~
       ~  1   2   2   3  (5) *
       ~  3   2   3  (4) (4) *
       ~  2   4  (5)  3   1  *
       ~ (6) (7)  1   4   5  *
       ~ (5)  1   1   2   4  *
          *   *   *   *   * Atlantic

Return:

[[0, 4], [1, 3], [1, 4], [2, 2], [3, 0], [3, 1], [4, 0]] (positions with parentheses in
above matrix).*/

public class _417 {
    public static class Solution1 {
        /**
         * Credit: looked at this post: https://discuss.leetcode.com/topic/62379/java-
bfs-dfs-from-ocean
         * <p>
         * One typical trick to work on 2d grid problems is to go through the border and
put proper ones into a queue if using BFS.
         */
        public List<int[]> pacificAtlantic(int[][] matrix) {

            List<int[]> result = new ArrayList();
            if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
                return result;
            }

            int m = matrix.length;
            int n = matrix[0].length;
            boolean[][] pacific = new boolean[m][n];
            boolean[][] atlantic = new boolean[m][n];

            for (int i = 0; i < m; i++) {
                dfs(matrix, pacific, Integer.MIN_VALUE, i, 0);
                dfs(matrix, atlantic, Integer.MIN_VALUE, i, n - 1);
            }

            for (int i = 0; i < n; i++) {
                dfs(matrix, pacific, Integer.MIN_VALUE, 0, i);
                dfs(matrix, atlantic, Integer.MIN_VALUE, m - 1, i);
```

```
            }

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (pacific[i][j] && atlantic[i][j]) {
                        result.add(new int[]{i, j});
                    }
                }
            }

            return result;
        }

        void dfs(int[][] matrix, boolean[][] visited, int height, int x, int y) {
            int m = matrix.length;
            int n = matrix[0].length;
            if (x < 0 || y < 0 || x >= m || y >= n || matrix[x][y] < height || visited[x]
[y]) {
                return;
            }
            visited[x][y] = true;
            dfs(matrix, visited, matrix[x][y], x + 1, y);
            dfs(matrix, visited, matrix[x][y], x, y + 1);
            dfs(matrix, visited, matrix[x][y], x - 1, y);
            dfs(matrix, visited, matrix[x][y], x, y - 1);
        }
    }

}
```

401. `_418.java`

```
package com.fishercoder.solutions;

/**
 * 418. Sentence Screen Fitting
 *
 * Given a rows x cols screen and a sentence represented by a list of non-empty words,
find how many times the given sentence can be fitted on the screen.

 Note:

 A word cannot be split into two lines.
 The order of words in the sentence must remain unchanged.
 Two consecutive words in a line must be separated by a single space.
 Total words in the sentence won't exceed 100.
 Length of each word is greater than 0 and won't exceed 10.
 1 ≤ rows, cols ≤ 20,000.

 Example 1:

 Input:
 rows = 2, cols = 8, sentence = ["hello", "world"]

 Output:
 1

 Explanation:
 hello---
 world---

 The character '-' signifies an empty space on the screen.

 Example 2:

 Input:
 rows = 3, cols = 6, sentence = ["a", "bcd", "e"]

 Output:
 2

 Explanation:
 a-bcd-
 e-a---
 bcd-e-

 The character '-' signifies an empty space on the screen.

 Example 3:

 Input:
 rows = 4, cols = 5, sentence = ["I", "had", "apple", "pie"]
```

```
 Output:
 1

 Explanation:
 I-had
 apple
 pie-I
 had--

 The character '-' signifies an empty space on the screen.

 */
public class _418 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/62455/21ms-18-lines-java-solution
         * <p>
         * 1. String s = String.join(" ", sentence) + " " ;. This line gives us a
formatted sentence to be put to our screen.
         * 2. start is the counter for how many valid characters from s have been put to
our screen.
         * 3. if (s.charAt(start % l) == ' ') is the situation that we don't need an
extra space for current row. The current row could be successfully fitted. So that we
need to increase our counter by using start++.
         * 4. The else is the situation, which the next word can't fit to current row. So
that we need to remove extra characters from next word.
         * 5. start / s.length() is (# of valid characters) / our formatted sentence.
         */
        public int wordsTyping(String[] sentence, int rows, int cols) {
            String s = String.join(" ", sentence) + " ";
            int start = 0;
            int l = s.length();
            for (int i = 0; i < rows; i++) {
                start += cols;
                if (s.charAt(start % l) == ' ') {
                    start++;
                } else {
                    while (start > 0 && s.charAt((start - 1) % l) != ' ') {
                        start--;
                    }
                }
            }
            return start / s.length();
        }
    }
}
```

402. `_419.java`

```
package com.fishercoder.solutions;

/**
 * 419. Battleships in a Board
 *
 * Given an 2D board, count how many battleships are in it.
 * The battleships are represented with 'X's, empty slots are represented with '.'s.
 * You may assume the following rules:

 You receive a valid board, made of only battleships or empty slots.
 Battleships can only be placed horizontally or vertically.
 In other words, they can only be made of the shape 1xN (1 row, N columns) or Nx1 (N
rows, 1 column), where N can be of any size.
 At least one horizontal or vertical cell separates between two battleships - there are
no adjacent battleships.

 Example:

 X..X
 ...X
 ...X

 In the above board there are 2 battleships.

 Invalid Example:

 ...X
 XXXX
 ...X

 This is an invalid board that you will not receive - as battleships will always have a
cell separating between them.

 Follow up:
 Could you do it in one-pass, using only O(1) extra memory and without modifying the
value of the board?

 */
public class _419 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/62970/simple-java-solution,
         * <p>
         * This solution does NOT modify original input.
         * Basically, it only counts the top-left one while ignoring all other parts of
one battleship,
         * using the top-left one as a representative for one battle.
         * This is achieved by counting cells that don't have 'X' to the left and above
them.
         */
```

```
        public int countBattleships(char[][] board) {
            if (board == null || board.length == 0) {
                return 0;
            }
            int count = 0;
            int m = board.length;
            int n = board[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (board[i][j] == '.') {
                        continue;//if it can pass this line, then board[i][j] must be 'X'
                    }
                    if (j > 0 && board[i][j - 1] == 'X') {
                        continue;//then we check if its left is 'X'
                    }
                    if (i > 0 && board[i - 1][j] == 'X') {
                        continue;//also check if its top is 'X'
                    }
                    count++;
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        /**
         * My original solution, actually modified the input. I just undo it at the end.
         */
        public int countBattleships(char[][] board) {
            if (board == null || board.length == 0) {
                return 0;
            }
            int result = 0;
            int m = board.length;
            int n = board[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (board[i][j] == 'X') {
                        result++;
                        dfs(board, i, j, m, n);
                    }
                }
            }

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (board[i][j] == '#') {
                        board[i][j] = 'X';
                    }
                }
            }
            return result;
        }
```

```
        private void dfs(char[][] board, int x, int y, int m, int n) {
            if (x < 0 || x >= m || y < 0 || y >= n || board[x][y] != 'X') {
                return;
            }
            if (board[x][y] == 'X') {
                board[x][y] = '#';
            }
            dfs(board, x + 1, y, m, n);
            dfs(board, x, y + 1, m, n);
            dfs(board, x - 1, y, m, n);
            dfs(board, x, y - 1, m, n);
        }
    }

}
```

403. `_420.java`

```
package com.fishercoder.solutions;

/**
 * 420. Strong Password Checker
 *
 * A password is considered strong if below conditions are all met:

 It has at least 6 characters and at most 20 characters.
 It must contain at least one lowercase letter, at least one uppercase letter, and at
least one digit.
 It must NOT contain three repeating characters in a row ("...aaa..." is weak, but
"...aa...a..." is strong, assuming other conditions are met).
 Write a function strongPasswordChecker(s), that takes a string s as input, and return
the MINIMUM
 change required to make s a strong password. If s is already strong, return 0.

 Insertion, deletion or replace of any one character are all considered as one change.
 */
public class _420 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/63854/o-n-java-solution-by-
analyzing-changes-allowed-to-fix-each-condition
         */
        public int strongPasswordChecker(String s) {
            int res = 0;
            int a = 1;
            int A = 1;
            int d = 1;
            char[] carr = s.toCharArray();
            int[] arr = new int[carr.length];

            for (int i = 0; i < arr.length; ) {
                if (Character.isLowerCase(carr[i])) {
                    a = 0;
                }
                if (Character.isUpperCase(carr[i])) {
                    A = 0;
                }
                if (Character.isDigit(carr[i])) {
                    d = 0;
                }

                int j = i;
                while (i < carr.length && carr[i] == carr[j]) {
                    i++;
                }
                arr[j] = i - j;
            }

            int totalMissing = (a + A + d);
```

```
            if (arr.length < 6) {
                res += totalMissing + Math.max(0, 6 - (arr.length + totalMissing));
            } else {
                int overLen = Math.max(arr.length - 20, 0);
                int leftOver = 0;
                res += overLen;

                for (int k = 1; k < 3; k++) {
                    for (int i = 0; i < arr.length && overLen > 0; i++) {
                        if (arr[i] < 3 || arr[i] % 3 != (k - 1)) {
                            continue;
                        }
                        arr[i] -= Math.min(overLen, k);
                        overLen -= k;
                    }
                }

                for (int i = 0; i < arr.length; i++) {
                    if (arr[i] >= 3 && overLen > 0) {
                        int need = arr[i] - 2;
                        arr[i] -= overLen;
                        overLen -= need;
                    }

                    if (arr[i] >= 3) {
                        leftOver += arr[i] / 3;
                    }
                }

                res += Math.max(totalMissing, leftOver);
            }

            return res;
        }
    }
}
```

404. `_421.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * Given a non-empty array of numbers, a0, a1, a2, … , an-1, where 0 ≤ ai < 231.

 Find the maximum result of ai XOR aj, where 0 ≤ i, j < n.

 Could you do this in O(n) runtime?

 Example:

 Input: [3, 10, 5, 25, 2, 8]

 Output: 28

 Explanation: The maximum result is 5 ^ 25 = 28.
 */
public class _421 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/63213/java-o-n-solution-using-bit-manipulation-and-hashmap/7
        public int findMaximumXOR(int[] nums) {
            int max = 0;
            int mask = 0;
            for (int i = 31; i >= 0; i--) {
                mask |= (1 << i);//the mask will grow like this: 100...000, 110...000, 111...000 to 111...111, each time, we only get the most left part of all numbers in the given array
                System.out.println("mask = " + Integer.toBinaryString(mask));
                Set<Integer> set = new HashSet<>();
                for (int num : nums) {
                    System.out.println("num = " + Integer.toBinaryString(num));
                    set.add(num & mask);
                    System.out.println("mask & num = " + Integer.toBinaryString(mask & num));
                }

                int candidate = max | (1 << i);
                System.out.println("candidate = " + Integer.toBinaryString(candidate));
                /**Reason behind this: if a ^ prefix = candidate, then a ^ candidate = prefix, also prefix ^ candidate = a
                 * in this below code: we use this one: prefix ^ candidate = a*/
                for (int prefix : set) {
                    System.out.println("candidate ^ prefix = " + Integer.toBinaryString(candidate ^ prefix));
                    if (set.contains(candidate ^ prefix)) {
                        max = candidate;
```

```
                    }
                }
                System.out.println("max = " + max);
                System.out.println("i = " + i);
                System.out.println("================================================");
            }
            return max;
        }
    }

}
```

405. `_422.java`

```
package com.fishercoder.solutions;

import java.util.List;

/**
 * 422. Valid Word Square
 *
 * Given a sequence of words, check whether it forms a valid word square.

 A sequence of words forms a valid word square if the kth row and column read the exact same string, where 0 ≤ k < max(numRows, numColumns).

 Note:
 The number of words given is at least 1 and does not exceed 500.
 Word length will be at least 1 and does not exceed 500.
 Each word contains only lowercase English alphabet a-z.
 Example 1:

 Input:
 [
 "abcd",
 "bnrt",
 "crmy",
 "dtye"
 ]

 Output:
 true

 Explanation:
 The first row and first column both read "abcd".
 The second row and second column both read "bnrt".
 The third row and third column both read "crmy".
 The fourth row and fourth column both read "dtye".

 Therefore, it is a valid word square.
 Example 2:

 Input:
 [
 "abcd",
 "bnrt",
 "crm",
 "dt"
 ]

 Output:
 true

 Explanation:
 The first row and first column both read "abcd".
```

```
 The second row and second column both read "bnrt".
 The third row and third column both read "crm".
 The fourth row and fourth column both read "dt".

 Therefore, it is a valid word square.
 Example 3:

 Input:
 [
 "ball",
 "area",
 "read",
 "lady"
 ]

 Output:
 false

 Explanation:
 The third row reads "read" while the third column reads "lead".

 Therefore, it is NOT a valid word square.
 */
public class _422 {

    public static class Solution1 {
        public boolean validWordSquare(List<String> words) {
            for (int i = 0; i < words.size(); i++) {
                String word = words.get(i);
                for (int j = 0; j < word.length(); j++) {
                    if (j >= words.size()) {
                        return false;
                    }
                    if (i >= words.get(j).length()) {
                        return false;
                    }
                    if (word.charAt(j) != words.get(j).charAt(i)) {
                        return false;
                    }
                }
            }
            return true;
        }
    }
}
```

406. `_423.java`

```
package com.fishercoder.solutions;

/**
 * 423. Reconstruct Original Digits from English
 *
 * Given a non-empty string containing an out-of-order English representation of digits
0-9, output the digits in ascending order.

 Note:
 Input contains only lowercase English letters.
 Input is guaranteed to be valid and can be transformed to its original digits. That
means invalid inputs such as "abc" or "zerone" are not permitted.
 Input length is less than 50,000.

 Example 1:
 Input: "owoztneoer"

 Output: "012"
 Example 2:
 Input: "fviefuro"

 Output: "45"

 */
public class _423 {

    public static class Solution1 {
        public String originalDigits(String s) {
            /**we can use one char as a representative to uniquely stand for one number,
             * for some numbers that we cannot find a unique representive, we can dedup.
             * e.g. for number one, if we use 'o' as its representive, then 'o' also
exists in numbers 2, 4 and 0, so
             * we need to dedupe the 'o' in those numbers.
             * Also, the order to dedupe matters:
             * we'll have to dedupe for counts[3], counts[5], counts[7] first before we
dedupe counts[1] and counts[9].*/
            int[] counts = new int[10];
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == 'o') {
                    counts[1]++;//2,4,0
                }
                if (s.charAt(i) == 'w') {
                    counts[2]++;
                }
                if (s.charAt(i) == 'h') {
                    counts[3]++;//8
                }
                if (s.charAt(i) == 'u') {
                    counts[4]++;
                }
                if (s.charAt(i) == 'f') {
```

```
                    counts[5]++;//4
                }
                if (s.charAt(i) == 'x') {
                    counts[6]++;
                }
                if (s.charAt(i) == 'v') {
                    counts[7]++;//5
                }
                if (s.charAt(i) == 'g') {
                    counts[8]++;
                }
                if (s.charAt(i) == 'i') {
                    counts[9]++;//5,6,8
                }
                if (s.charAt(i) == 'z') {
                    counts[0]++;
                }
            }

            counts[3] -= counts[8];
            counts[5] -= counts[4];
            counts[7] -= counts[5];
            counts[1] = counts[1] - (counts[2] + counts[4] + counts[0]);
            counts[9] = counts[9] - (counts[5] + counts[6] + counts[8]);

            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < 10; i++) {
                for (int j = 0; j < counts[i]; j++) {
                    stringBuilder.append(i);
                }
            }
            return stringBuilder.toString();
        }
    }

}
```

407. `_424.java`

```
package com.fishercoder.solutions;

/**
 * 424. Longest Repeating Character Replacement
 *
 * Given a string that consists of only uppercase English letters,
 * you can replace any letter in the string with another letter at most k times.
 * Find the length of a longest substring containing all repeating letters you can get
after performing the above operations.

 Note:
 Both the string's length and k will not exceed 104.

 Example 1:

 Input:
 s = "ABAB", k = 2

 Output:
 4

 Explanation:
 Replace the two 'A's with two 'B's or vice versa.
 Example 2:

 Input:
 s = "AABABBA", k = 1

 Output:
 4

 Explanation:
 Replace the one 'A' in the middle with 'B' and form "AABBBBA".
 The substring "BBBB" has the longest repeating letters, which is 4.
 */
public class _424 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/63494/java-12-lines-o-n-sliding-
window-solution-with-explanation
        public int characterReplacement(String s, int k) {
            int len = s.length();
            int[] count = new int[26];
            int start = 0;
            int maxCount = 0;
            int maxLength = 0;
            for (int end = 0; end < len; end++) {
                maxCount = Math.max(maxCount, ++count[s.charAt(end) - 'A']);
                while (end - start + 1 - maxCount > k) {
                    count[s.charAt(start) - 'A']--;
                    start++;
```

```
                }
                maxLength = Math.max(maxLength, end - start + 1);
            }
            return maxLength;
        }
    }
}
```

408. `_425.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 425. Word Squares
 *
 * Given a set of words (without duplicates), find all word squares you can build from
them.

 A sequence of words forms a valid word square if the kth row and column read the exact
same string, where 0 ≤ k < max(numRows, numColumns).

 For example, the word sequence ["ball","area","lead","lady"] forms a word square because
each word reads the same both horizontally and vertically.

 b a l l
 a r e a
 l e a d
 l a d y

 Note:

 There are at least 1 and at most 1000 words.
 All words will have the exact same length.
 Word length is at least 1 and at most 5.
 Each word contains only lowercase English alphabet a-z.

 Example 1:

 Input:
 ["area","lead","wall","lady","ball"]

 Output:
 [
 [ "wall",
 "area",
 "lead",
 "lady"
 ],
 [ "ball",
 "area",
 "lead",
 "lady"
 ]
 ]

 Explanation:
 The output consists of two word squares. The order of output does not matter (just the
order of words in each word square matters).
```

```
 Example 2:

 Input:
 ["abat","baba","atan","atal"]

 Output:
 [
 [ "baba",
 "abat",
 "baba",
 "atan"
 ],
 [ "baba",
 "abat",
 "baba",
 "atal"
 ]
 ]

 Explanation:
 The output consists of two word squares. The order of output does not matter (just the
order of words in each word square matters).
 */
public class _425 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/63516/explained-my-java-solution-
using-trie-126ms-16-16/2
         */

        class TrieNode {
            List<String> startWith;
            TrieNode[] children;

            TrieNode() {
                startWith = new ArrayList<>();
                children = new TrieNode[26];
            }
        }

        class Trie {
            TrieNode root;

            Trie(String[] words) {
                root = new TrieNode();
                for (String word : words) {
                    TrieNode cur = root;
                    for (char ch : word.toCharArray()) {
                        int index = ch - 'a';
                        if (cur.children[index] == null) {
                            cur.children[index] = new TrieNode();
```

```
                }
                cur.children[index].startWith.add(word);
                cur = cur.children[index];
            }
        }
    }

    List<String> findByPrefix(String prefix) {
        List<String> ans = new ArrayList<>();
        TrieNode cur = root;
        for (char ch : prefix.toCharArray()) {
            int index = ch - 'a';
            if (cur.children[index] == null) {
                return ans;
            }

            cur = cur.children[index];
        }
        ans.addAll(cur.startWith);
        return ans;
    }
}

public List<List<String>> wordSquares(String[] words) {
    List<List<String>> ans = new ArrayList<>();
    if (words == null || words.length == 0) {
        return ans;
    }
    int len = words[0].length();
    Trie trie = new Trie(words);
    List<String> ansBuilder = new ArrayList<>();
    for (String w : words) {
        ansBuilder.add(w);
        search(len, trie, ans, ansBuilder);
        ansBuilder.remove(ansBuilder.size() - 1);
    }

    return ans;
}

private void search(int len, Trie trie, List<List<String>> ans,
                    List<String> ansBuilder) {
    if (ansBuilder.size() == len) {
        ans.add(new ArrayList<>(ansBuilder));
        return;
    }

    int idx = ansBuilder.size();
    StringBuilder prefixBuilder = new StringBuilder();
    for (String s : ansBuilder) {
        prefixBuilder.append(s.charAt(idx));
    }
    List<String> startWith = trie.findByPrefix(prefixBuilder.toString());
    for (String sw : startWith) {
```

```
                ansBuilder.add(sw);
                search(len, trie, ans, ansBuilder);
                ansBuilder.remove(ansBuilder.size() - 1);
            }
        }
    }

}
```

409. `_429.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 429. N-ary Tree Level Order Traversal
 *
 * Given an n-ary tree, return the level order traversal of its nodes' values. (ie, from
left to right, level by level).
 *
 * For example, given a 3-ary tree:
 *
 *                              1
 *                            / | \
 *                           3  2 4
 *                          / \
 *                         5  6
 *
 *  We should return its level order traversal:
 *
 * [
 *      [1],
 *      [3,2,4],
 *      [5,6]
 * ]
 *
 *
 * Note:
 *
 * The depth of the tree is at most 1000.
 * The total number of nodes is at most 5000.
 */
public class _429 {
  public static class Solution1 {
    public List<List<Integer>> levelOrder(Node root) {
      List<List<Integer>> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      Queue<Node> queue = new LinkedList<>();
      queue.offer(root);
      while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
          Node currentNode = queue.poll();
          if (currentNode != null) {
```

```
            level.add(currentNode.val);
            for (Node child : currentNode.children) {
              queue.offer(child);
            }
          }
        }
        result.add(level);
      }
      return result;
    }
  }
}
```

410. `_432.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 432. All O`one Data Structure
 * Implement a data structure supporting the following operations:

 1. Inc(Key) - Inserts a new key with value 1. Or increments an existing key by 1. Key is
guaranteed to be a non-empty string.
 2. Dec(Key) - If Key's value is 1, remove it from the data structure. Otherwise
decrements an existing key by 1. If the key does not exist, this function does nothing.
Key is guaranteed to be a non-empty string.
 3. GetMaxKey() - Returns one of the keys with maximal value. If no element exists,
return an empty string "".
 4. GetMinKey() - Returns one of the keys with minimal value. If no element exists,
return an empty string "".

 Challenge: Perform all these in O(1) time complexity.

 */
public class _432 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/65634/java-ac-all-strict-o-1-not-
average-o-1-easy-to-read/2
         */
        class AllOne {
            // maintain a doubly linked list of Buckets
            private Bucket head;
            private Bucket tail;
            // for accessing a specific Bucket among the Bucket list in O(1) time
            private Map<Integer, Bucket> countBucketMap;
            // keep track of count of keys
            private Map<String, Integer> keyCountMap;

            // each Bucket contains all the keys with the same count
            private class Bucket {
                int count;
                Set<String> keySet;
                Bucket next;
                Bucket pre;

                public Bucket(int cnt) {
                    count = cnt;
                    keySet = new HashSet<>();
```

```
            }
        }

        /**
         * Initialize your data structure here.
         */
        public AllOne() {
            head = new Bucket(Integer.MIN_VALUE);
            tail = new Bucket(Integer.MAX_VALUE);
            head.next = tail;
            tail.pre = head;
            countBucketMap = new HashMap<>();
            keyCountMap = new HashMap<>();
        }

        /**
         * Inserts a new key <Key> with value 1. Or increments an existing key by 1.
         */
        public void inc(String key) {
            if (keyCountMap.containsKey(key)) {
                changeKey(key, 1);
            } else {
                keyCountMap.put(key, 1);
                if (head.next.count != 1) {
                    addBucketAfter(new Bucket(1), head);
                }
                head.next.keySet.add(key);
                countBucketMap.put(1, head.next);
            }
        }

        /**
         * Decrements an existing key by 1. If Key's value is 1, remove it from the
data structure.
         */
        public void dec(String key) {
            if (keyCountMap.containsKey(key)) {
                int count = keyCountMap.get(key);
                if (count == 1) {
                    keyCountMap.remove(key);
                    removeKeyFromBucket(countBucketMap.get(count), key);
                } else {
                    changeKey(key, -1);
                }
            }
        }

        /**
         * Returns one of the keys with maximal value.
         */
        public String getMaxKey() {
            return tail.pre == head ? "" : (String)
tail.pre.keySet.iterator().next();
        }
```

```
            /**
             * Returns one of the keys with Minimal value.
             */
            public String getMinKey() {
                return head.next == tail ? "" : (String)
head.next.keySet.iterator().next();
            }

            // helper function to make change on given key according to offset
            private void changeKey(String key, int offset) {
                int count = keyCountMap.get(key);
                keyCountMap.put(key, count + offset);
                Bucket curBucket = countBucketMap.get(count);
                Bucket newBucket;
                if (countBucketMap.containsKey(count + offset)) {
                    // target Bucket already exists
                    newBucket = countBucketMap.get(count + offset);
                } else {
                    // add new Bucket
                    newBucket = new Bucket(count + offset);
                    countBucketMap.put(count + offset, newBucket);
                    addBucketAfter(newBucket, offset == 1 ? curBucket : curBucket.pre);
                }
                newBucket.keySet.add(key);
                removeKeyFromBucket(curBucket, key);
            }

            private void removeKeyFromBucket(Bucket bucket, String key) {
                bucket.keySet.remove(key);
                if (bucket.keySet.size() == 0) {
                    removeBucketFromList(bucket);
                    countBucketMap.remove(bucket.count);
                }
            }

            private void removeBucketFromList(Bucket bucket) {
                bucket.pre.next = bucket.next;
                bucket.next.pre = bucket.pre;
                bucket.next = null;
                bucket.pre = null;
            }

            // add newBucket after preBucket
            private void addBucketAfter(Bucket newBucket, Bucket preBucket) {
                newBucket.pre = preBucket;
                newBucket.next = preBucket.next;
                preBucket.next.pre = newBucket;
                preBucket.next = newBucket;
            }
        }
    }
}
```

```
/**
 * Your AllOne object will be instantiated and called as such:
 * AllOne obj = new AllOne();
 * obj.inc(key);
 * obj.dec(key);
 * String param_3 = obj.getMaxKey();
 * String param_4 = obj.getMinKey();
 */
```

411. `_434.java`

```
package com.fishercoder.solutions;
/**
 * 434. Number of Segments in a String
 *
 * Count the number of segments in a string,
 * where a segment is defined to be a contiguous sequence of non-space characters.
 *
 * Please note that the string does not contain any non-printable characters.

Example:

Input: "Hello, my name is John"
Output: 5*/
public class _434 {

    public static class Solution1 {
        public int countSegments(String s) {
            if (s == null || s.isEmpty()) {
                return 0;
            }
            String[] segments = s.split(" ");
            int count = 0;
            for (String seg : segments) {
                if (seg.equals("")) {
                    continue;
                }
                count++;
            }
            return count;
        }
    }
}
```

412. `_435.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.Arrays;
import java.util.Collections;

/**
 * 435. Non-overlapping Intervals
 *
 * Given a collection of intervals,
 * find the minimum number of intervals you need to remove to make the rest of the
 * intervals non-overlapping.

 Note:
 You may assume the interval's end point is always bigger than its start point.
 Intervals like [1,2] and [2,3] have borders "touching" but they don't overlap each
other.

 Example 1:
 Input: [ [1,2], [2,3], [3,4], [1,3] ]
 Output: 1
 Explanation: [1,3] can be removed and the rest of intervals are non-overlapping.

 Example 2:
 Input: [ [1,2], [1,2], [1,2] ]
 Output: 2
 Explanation: You need to remove two [1,2] to make the rest of intervals non-overlapping.

 Example 3:
 Input: [ [1,2], [2,3] ]
 Output: 0
 Explanation: You don't need to remove any of the intervals since they're already non-
overlapping.
 */

public class _435 {

    public static class Solution1 {
        /**
         * credit:: https://discuss.leetcode.com/topic/65828/java-solution-with-clear-
explain
         * and https://discuss.leetcode.com/topic/65594/java-least-is-most
         * Sort the intervals by their end time, if equal, then sort by their start time.
         */
        public int eraseOverlapIntervals(Interval[] intervals) {
            Collections.sort(Arrays.asList(intervals), (o1, o2) -> {
                if (o1.end != o2.end) {
                    return o1.end - o2.end;
                } else {
                    return o2.start - o1.start;
```

```
                }
            });
            int end = Integer.MIN_VALUE;
            int count = 0;
            for (Interval interval : intervals) {
                if (interval.start >= end) {
                    end = interval.end;
                } else {
                    count++;
                }
            }
            return count;
        }
    }

}
```

413. `_436.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

/**436. Find Right Interval
 *
 * Given a set of intervals, for each of the interval i, check if there exists an
interval j whose start
 * point is bigger than or equal to the end point of the interval i, which can be called
that j is on the "right" of i.
 *
 * For any interval i, you need to store the minimum interval j's index,
 * which means that the interval j has the minimum start point to build the "right"
relationship for interval i.
 * If the interval j doesn't exist, store -1 for the interval i. Finally, you need output
the stored value of each interval as an array.
 *
 * Note:
 *
 * You may assume the interval's end point is always bigger than its start point.
 * You may assume none of these intervals have the same start point.
 *
 * Example 1:
 * Input: [ [1,2] ]
 * Output: [-1]
 * Explanation: There is only one interval in the collection, so it outputs -1.
 *
 * Example 2:
 * Input: [ [3,4], [2,3], [1,2] ]
 * Output: [-1, 0, 1]
 * Explanation: There is no satisfied "right" interval for [3,4].
 * For [2,3], the interval [3,4] has minimum-"right" start point;
 * For [1,2], the interval [2,3] has minimum-"right" start point.
 *
 * Example 3:
 * Input: [ [1,4], [2,3], [3,4] ]
 * Output: [-1, 2, -1]
 * Explanation: There is no satisfied "right" interval for [1,4] and [3,4].
 * For [2,3], the interval [3,4] has minimum-"right" start point.
 * */

public class _436 {

    public static class Solution1 {
        public int[] findRightInterval(Interval[] intervals) {
            if (intervals == null || intervals.length == 0) {
```

```
                return new int[0];
            }
            int[] result = new int[intervals.length];
            result[0] = -1;
            Interval last = intervals[intervals.length - 1];
            Interval first = intervals[0];
            Map<Interval, Integer> map = new HashMap();
            for (int i = 0; i < intervals.length; i++) {
                map.put(intervals[i], i);
            }

            Collections.sort(Arrays.asList(intervals), (o1, o2) -> o1.start - o2.start);

            for (int i = 1; i < intervals.length; i++) {
                //TODO: use binary search for the minimum start interval for interval[i-
1] instead of a while loop
                int tmp = i - 1;
                int tmpI = i;
                while (tmpI < intervals.length && intervals[tmpI].start <
intervals[tmp].end) {
                    tmpI++;
                }
                if (tmpI < intervals.length) {
                    result[map.get(intervals[tmp])] = map.get(intervals[tmpI]);
                } else {
                    result[map.get(intervals[tmp])] = -1;
                }
            }
            if (result[intervals.length - 1] == 0 && last.end > first.start) {
                result[intervals.length - 1] = -1;
            }
            return result;
        }
    }

}
```

414. `_437.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 437. Path Sum III
 *
 * You are given a binary tree in which each node contains an integer value.

 Find the number of paths that sum to a given value.

 The path does not need to start or end at the root or a leaf, but it must go downwards
(traveling only from parent nodes to child nodes).

 The tree has no more than 1,000 nodes and the values are in the range -1,000,000 to
1,000,000.

 Example:

 root = [10,5,-3,3,2,null,11,3,-2,null,1], sum = 8

      10
     /  \
    5   -3
   / \    \
  3   2   11
 / \   \
3  -2  1

 Return 3. The paths that sum to 8 are:

 1.  5 -> 3
 2.  5 -> 2 -> 1
 3. -3 -> 11
 */

public class _437 {

    public static class Solution1 {
        public int pathSum(TreeNode root, int sum) {
            if (root == null) {
                return 0;
            }
            return pathSumFrom(root, sum) + pathSum(root.left, sum) + pathSum(root.right,
sum);
        }

        private int pathSumFrom(TreeNode root, int sum) {
            if (root == null) {
                return 0;
            }
```

```
            return (root.val == sum ? 1 : 0) + pathSumFrom(root.left, sum - root.val) +
pathSumFrom(root.right, sum - root.val);
        }
    }

}
```

415. `_438.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 438. Find All Anagrams in a String
 *
 * Given a string s and a non-empty string p, find all the start indices of p's anagrams in s.
 Strings consists of lowercase English letters only and the length of both strings s and p will not be larger than 20,100.

 The order of output does not matter.

 Example 1:
 Input:
 s: "cbaebabacd" p: "abc"
 Output:
 [0, 6]

 Explanation:
 The substring with start index = 0 is "cba", which is an anagram of "abc".
 The substring with start index = 6 is "bac", which is an anagram of "abc".

 Example 2:
 Input:
 s: "abab" p: "ab"
 Output:
 [0, 1, 2]

 Explanation:
 The substring with start index = 0 is "ab", which is an anagram of "ab".
 The substring with start index = 1 is "ba", which is an anagram of "ab".
 The substring with start index = 2 is "ab", which is an anagram of "ab".*/

public class _438 {
    public static class Solution1 {
        /**
         * O(m*n) solution, my original and most intuitive one, but sort of brute force, when m is close to n, it becomes O(n^2) runtime complexity.
         */
        public List<Integer> findAnagrams(String s, String p) {
            List<Integer> result = new ArrayList();
            for (int i = 0; i <= s.length() - p.length(); i++) {
                if (isAnagram(s.substring(i, i + p.length()), p)) {
                    result.add(i);
                }
            }
            return result;
        }
```

```
        private boolean isAnagram(String s, String p) {
            int[] c = new int[26];
            for (int i = 0; i < s.length(); i++) {
                c[s.charAt(i) - 'a']++;
                c[p.charAt(i) - 'a']--;
            }

            for (int i : c) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
        }
    }


    public static class Solution2 {
        /**
         * Sliding Window
         */
        public List<Integer> findAnagrams(String s, String p) {
            List<Integer> result = new ArrayList();
            int[] hash = new int[26];
            for (char c : p.toCharArray()) {
                hash[c - 'a']++;
            }
            int start = 0;
            int end = 0;
            int count = p.length();
            while (end < s.length()) {
                if (hash[s.charAt(end) - 'a'] > 0) {
                    count--;
                }
                hash[s.charAt(end) - 'a']--;
                end++;

                if (count == 0) {
                    result.add(start);
                }

                if ((end - start) == p.length()) {
                    if (hash[s.charAt(start) - 'a'] >= 0) {
                        count++;
                    }
                    hash[s.charAt(start) - 'a']++;
                    start++;
                }
            }
            return result;
        }
    }
}
```

416. `_439.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 439. Ternary Expression Parser
 *
 * Given a string representing arbitrarily nested ternary expressions, calculate the
result of the expression.
 * You can always assume that the given expression is valid and only consists of digits
0-9, ?, :, T and F (T and F represent True and False respectively).

 Note:

 The length of the given string is ≤ 10000.
 Each number will contain only one digit.
 The conditional expressions group right-to-left (as usual in most languages).
 The condition will always be either T or F. That is, the condition will never be a
digit.
 The result of the expression will always evaluate to either a digit 0-9, T or F.

 Example 1:

 Input: "T?2:3"

 Output: "2"

 Explanation: If true, then result is 2; otherwise result is 3.


 Example 2:

 Input: "F?1:T?4:5"

 Output: "4"

 Explanation: The conditional expressions group right-to-left. Using parenthesis, it is
read/evaluated as:

 "(F ? 1 : (T ? 4 : 5))"                  "(F ? 1 : (T ? 4 : 5))"
 -> "(F ? 1 : 4)"                or       -> "(T ? 4 : 5)"
 -> "4"                                   -> "4"
 Example 3:

 Input: "T?T?F:5:3"

 Output: "F"

 Explanation: The conditional expressions group right-to-left. Using parenthesis, it is
read/evaluated as:
```

```
 "(T ? (T ? F : 5) : 3)"                  "(T ? (T ? F : 5) : 3)"
 -> "(T ? F : 3)"                or       -> "(T ? F : 5)"
 -> "F"                                   -> "F"
 */
public class _439 {

    public static class Solution1 {
        /**
         * Below is my original solution, but looking at Discuss, a more concise way is
to use just one
         * stack, process it from right to left, example:
https://discuss.leetcode.com/topic/64409/very-easy-1-pass-stack-solution-in-java-no-
string-concat
         */

        public String parseTernary(String expression) {
            Deque<Character> stack = new ArrayDeque<>();
            Deque<Character> tmpStack = new ArrayDeque<>();
            for (char c : expression.toCharArray()) {
                stack.addFirst(c);
            }
            while (!stack.isEmpty()) {
                if (stack.peek() != '?') {
                    tmpStack.addFirst(stack.pollFirst());
                } else {
                    char char1 = tmpStack.removeFirst();
                    tmpStack.removeFirst();//remove ':'
                    char char2 = tmpStack.removeFirst();
                    stack.removeFirst();//remove '?'
                    char judge = stack.removeFirst();
                    tmpStack.addFirst(judge == 'T' ? char1 : char2);
                    while (!tmpStack.isEmpty()) {
                        stack.addFirst(tmpStack.pollFirst());
                    }
                }
                if (stack.size() == 1) {
                    break;
                }
            }
            return Character.toString(stack.removeFirst());
        }
    }

}
```

417. `_440.java`

```
package com.fishercoder.solutions;

/**
 * 440. K-th Smallest in Lexicographical Order
 *
 * Given integers n and k, find the lexicographically k-th smallest integer in the range
from 1 to n.

 Note: 1 ≤ k ≤ n ≤ 109.

 Example:

 Input:
 n: 13   k: 2

 Output:
 10

 Explanation:
 The lexicographical order is [1, 10, 11, 12, 13, 2, 3, 4, 5, 6, 7, 8, 9], so the second
smallest number is 10.
 */
public class _440 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/64624/concise-easy-to-understand-
java-5ms-solution-with-explaination/2
         */
        public int findKthNumber(int n, int k) {
            int curr = 1;
            k = k - 1;
            while (k > 0) {
                int steps = calSteps(n, curr, curr + 1);
                if (steps <= k) {
                    curr += 1;
                    k -= steps;
                } else {
                    curr *= 10;
                    k -= 1;
                }
            }
            return curr;
        }

        //use long in case of overflow
        public int calSteps(int n, long n1, long n2) {
            int steps = 0;
            while (n1 <= n) {
                steps += Math.min(n + 1, n2) - n1;
                n1 *= 10;
```

```
                n2 *= 10;
            }
            return steps;
        }
    }
}
```

418. `_441.java`

```
package com.fishercoder.solutions;

/**
 * 441. Arranging Coins
 *
 * You have a total of n coins that you want to form in a staircase shape, where every k-th row must have exactly k coins.

 Given n, find the total number of full staircase rows that can be formed.

 n is a non-negative integer and fits within the range of a 32-bit signed integer.

 Example 1:

 n = 5

 The coins can form the following rows:
 ¤
 ¤ ¤
 ¤ ¤

 Because the 3rd row is incomplete, we return 2.


 Example 2:

 n = 8

 The coins can form the following rows:
 ¤
 ¤ ¤
 ¤ ¤ ¤
 ¤ ¤

 Because the 4th row is incomplete, we return 3.*/
public class _441 {

    public static class Solution1 {
        public int arrangeCoins(int n) {
            if (n < 2) {
                return n;
            }
            int row = 0;
            int count = 0;
            long sum = 0;
            while (sum < n) {
                row += 1;
                sum += row;
                count++;
            }
            if (sum == n) {
```

```
                return count;
            }
            return count - 1;
        }
    }

}
```

419. `_442.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 442. Find All Duplicates in an Array
 *
 * Given an array of integers, 1 ≤ a[i] ≤ n (n = size of array), some elements appear
twice and others appear once.

 Find all the elements that appear twice in this array.

 Could you do it without extra space and in O(n) runtime?

 Example:
 Input:
 [4,3,2,7,8,2,3,1]

 Output:
 [2,3]
 */
public class _442 {

    public static class Solution1 {
        //O(n) space
        //O(n) time
        public List<Integer> findDuplicates(int[] nums) {
            Set<Integer> set = new HashSet();
            List<Integer> result = new ArrayList();
            for (int i : nums) {
                if (!set.add(i)) {
                    result.add(i);
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        //O(1) space
        //O(n) time
        public List<Integer> findDuplicates(int[] nums) {
            List<Integer> result = new ArrayList();
            for (int i = 0; i < nums.length; i++) {
                int index = Math.abs(nums[i]) - 1;
                if (nums[index] < 0) {
                    result.add(Math.abs(index + 1));
                }
```

```
                nums[index] = -nums[index];
            }
            return result;
        }
    }

}
```

420. `_443.java`

```
package com.fishercoder.solutions;

/**
 * 443. String Compression
 *
 * Given an array of characters, compress it in-place.
 * The length after compression must always be smaller than or equal to the original
array.
 * Every element of the array should be a character (not int) of length 1.
 * After you are done modifying the input array in-place, return the new length of the
array.

 Example 1:
 Input:
 ["a","a","b","b","c","c","c"]

 Output:
 Return 6, and the first 6 characters of the input array should be:
["a","2","b","2","c","3"]

 Explanation:
 "aa" is replaced by "a2". "bb" is replaced by "b2". "ccc" is replaced by "c3".

 Example 2:
 Input:
 ["a"]

 Output:
 Return 1, and the first 1 characters of the input array should be: ["a"]

 Explanation:
 Nothing is replaced.

 Example 3:
 Input:
 ["a","b","b","b","b","b","b","b","b","b","b","b","b","b"]

 Output:
 Return 4, and the first 4 characters of the input array should be: ["a","b","1","2"].

 Explanation:
 Since the character "a" does not repeat, it is not compressed. "bbbbbbbbbbbb" is
replaced by "b12".
 Notice each digit has it's own entry in the array.

 Note:
 All characters have an ASCII value in [35, 126].
 1 <= len(chars) <= 1000.
 */
public class _443 {
    public static class Solution1 {
```

```
        /**This is breaking the rules, it's not in-place.*/
        public int compress(char[] chars) {
            if (chars == null || chars.length == 0) {
                return 0;
            }
            StringBuilder sb = new StringBuilder();
            int count = 1;
            char prev = chars[0];
            for (int i = 1; i < chars.length; i++) {
                if (chars[i] == prev) {
                    count++;
                } else {
                    if (count > 1) {
                        sb.append(prev);
                        sb.append(count);
                    } else if (count == 1) {
                        sb.append(prev);
                    }
                    prev = chars[i];
                    count = 1;
                }
            }
            sb.append(prev);
            if (count > 1) {
                sb.append(count);
            }
            int i = 0;
            for (char c : sb.toString().toCharArray()) {
                chars[i++] = c;
            }
            return sb.length();
        }
    }
}
```

421. `_444.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

/**
 * 444. Sequence Reconstruction
 *
 * Check whether the original sequence org can be uniquely reconstructed from the
sequences in seqs.
 * The org sequence is a permutation of the integers from 1 to n, with 1 ≤ n ≤ 104.
 * Reconstruction means building a shortest common supersequence of the sequences in seqs
 * (i.e., a shortest sequence so that all sequences in seqs are subsequences of it).
 * Determine whether there is only one sequence that can be reconstructed from seqs and
it is the org sequence.

 Example 1:
 Input:
 org: [1,2,3], seqs: [[1,2],[1,3]]

 Output:
 false

 Explanation:
 [1,2,3] is not the only one sequence that can be reconstructed, because [1,3,2] is also
a valid sequence that can be reconstructed.

 Example 2:
 Input:
 org: [1,2,3], seqs: [[1,2]]

 Output:
 false

 Explanation:
 The reconstructed sequence can only be [1,2].

 Example 3:
 Input:
 org: [1,2,3], seqs: [[1,2],[1,3],[2,3]]

 Output:
 true

 Explanation:
 The sequences [1,2], [1,3], and [2,3] can uniquely reconstruct the original sequence
```

```
[1,2,3].

 Example 4:
 Input:
 org: [4,1,5,2,6,3], seqs: [[5,2,6,3],[4,1,5,2]]

 Output:
 true
 */
public class _444 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/65948/java-solution-using-bfs-
topological-sort
         */
        public boolean sequenceReconstruction(int[] org, List<List<Integer>> seqs) {
            Map<Integer, Set<Integer>> map = new HashMap<>();
            Map<Integer, Integer> indegree = new HashMap<>();
            for (List<Integer> seq : seqs) {
                if (seq.size() == 1) {
                    if (!map.containsKey(seq.get(0))) {
                        map.put(seq.get(0), new HashSet<>());
                        indegree.put(seq.get(0), 0);
                    }
                } else {
                    for (int i = 0; i < seq.size() - 1; i++) {
                        if (!map.containsKey(seq.get(i))) {
                            map.put(seq.get(i), new HashSet<>());
                            indegree.put(seq.get(i), 0);
                        }

                        if (!map.containsKey(seq.get(i + 1))) {
                            map.put(seq.get(i + 1), new HashSet<>());
                            indegree.put(seq.get(i + 1), 0);
                        }

                        if (map.get(seq.get(i)).add(seq.get(i + 1))) {
                            indegree.put(seq.get(i + 1), indegree.get(seq.get(i + 1)) +
1);
                        }
                    }
                }
            }

            Queue<Integer> queue = new LinkedList<>();
            for (Integer key : indegree.keySet()) {
                if (indegree.get(key) == 0) {
                    queue.offer(key);
                }
            }

            int index = 0;
            while (!queue.isEmpty()) {
                int size = queue.size();
```

```
                if (size > 1) {
                    return false;
                }
                int curr = queue.poll();
                if (index == org.length || curr != org[index++]) {
                    return false;
                }
                for (int next : map.get(curr)) {
                    indegree.put(next, indegree.get(next) - 1);
                    if (indegree.get(next) == 0) {
                        queue.offer(next);
                    }
                }
            }
            return index == org.length && index == map.size();
        }
    }
}
```

422. `_445.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Stack;

/**
 * 445. Add Two Numbers II
 *
 * You are given two non-empty linked lists representing two non-negative integers.
 * The most significant digit comes first and each of their nodes contain a single digit.
Add the two numbers and return it as a linked list.

 You may assume the two numbers do not contain any leading zero, except the number 0
itself.

 Follow up:
 What if you cannot modify the input lists? In other words, reversing the lists is not
allowed.

 Example:

 Input: (7 -> 2 -> 4 -> 3) + (5 -> 6 -> 4)
 Output: 7 -> 8 -> 0 -> 7
 */
public class _445 {

    public static class Solution1 {
        public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
            Deque<Integer> stack1 = popIntoStack(l1);
            Deque<Integer> stack2 = popIntoStack(l2);

            int sum = 0;
            ListNode list = new ListNode(0);
            while (!stack1.isEmpty() || !stack2.isEmpty()) {
                if (!stack1.isEmpty()) {
                    sum += stack1.removeFirst();
                }
                if (!stack2.isEmpty()) {
                    sum += stack2.removeFirst();
                }
                list.val = sum % 10;
                ListNode head = new ListNode(sum / 10);
                head.next = list;
                list = head;
                sum /= 10;
            }
            return list.val == 0 ? list.next : list;
        }
```

```
    private Deque<Integer> popIntoStack(ListNode head) {
        ListNode tmp = head;
        Deque<Integer> stack = new ArrayDeque<>();
        while (tmp != null) {
            stack.push(tmp.val);
            tmp = tmp.next;
        }
        return stack;
    }
}


public static class Solution2 {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        Stack<Integer> stack1 = popOntoStack(l1);
        Stack<Integer> stack2 = popOntoStack(l2);
        Stack<Integer> resultStack = add(stack1, stack2);
        return buildResult(resultStack);
    }

    private ListNode buildResult(Stack<Integer> stack) {
        ListNode prev = new ListNode(-1);
        ListNode head = new ListNode(stack.pop());
        prev.next = head;
        while (!stack.isEmpty()) {
            head.next = new ListNode(stack.pop());
            head = head.next;
        }
        return prev.next;
    }

    private Stack<Integer> add(Stack<Integer> stack1, Stack<Integer> stack2) {
        Stack<Integer> res = new Stack<>();
        int carry = 0;
        while (!stack1.isEmpty() || !stack2.isEmpty()) {
            if (!stack1.isEmpty()) {
                carry += stack1.pop();
            }
            if (!stack2.isEmpty()) {
                carry += stack2.pop();
            }
            int value = carry;
            if (carry > 9) {
                value = carry % 10;
                carry = 1;
            } else {
                carry = 0;
            }
            res.push(value);
        }
        if (carry != 0) {
            res.add(carry);
        }
```

```
            return res;
        }

        private Stack<Integer> popOntoStack(ListNode head) {
            ListNode temp = head;
            Stack<Integer> stack = new Stack<>();
            while (temp != null) {
                stack.push(temp.val);
                temp = temp.next;
            }
            return stack;
        }
    }
}
```

423. `_446.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 446. Arithmetic Slices II - Subsequence
 *
 * A sequence of numbers is called arithmetic if it consists of at least three elements
and if the difference between any two consecutive elements is the same.

 For example, these are arithmetic sequences:

 1, 3, 5, 7, 9
 7, 7, 7, 7
 3, -1, -5, -9

 The following sequence is not arithmetic.

 1, 1, 2, 5, 7

 A zero-indexed array A consisting of N numbers is given. A subsequence slice of that
array is any sequence of integers (P0, P1, ..., Pk) such that 0 ≤ P0 < P1 < ... < Pk < N.

 A subsequence slice (P0, P1, ..., Pk) of array A is called arithmetic if the sequence
A[P0], A[P1], ..., A[Pk-1], A[Pk] is arithmetic. In particular, this means that k ≥ 2.

 The function should return the number of arithmetic subsequence slices in the array A.

 The input contains N integers. Every integer is in the range of -231 and 231-1 and 0 ≤ N
≤ 1000. The output is guaranteed to be less than 231-1.


 Example:

 Input: [2, 4, 6, 8, 10]

 Output: 7

 Explanation:
 All arithmetic subsequence slices are:
 [2,4,6]
 [4,6,8]
 [6,8,10]
 [2,4,6,8]
 [4,6,8,10]
 [2,4,6,8,10]
 [2,6,10]
 */
public class _446 {
    public static class Solution1 {
```

```
        /**
         * reference: https://discuss.leetcode.com/topic/67413/detailed-explanation-for-
java-o-n-2-solution
         */
        public int numberOfArithmeticSlices(int[] A) {
            int res = 0;
            Map<Integer, Integer>[] map = new Map[A.length];

            for (int i = 0; i < A.length; i++) {
                map[i] = new HashMap<>(i);

                for (int j = 0; j < i; j++) {
                    long diff = (long) A[i] - A[j];
                    if (diff <= Integer.MIN_VALUE || diff > Integer.MAX_VALUE) {
                        continue;
                    }

                    int d = (int) diff;
                    int c1 = map[i].getOrDefault(d, 0);
                    int c2 = map[j].getOrDefault(d, 0);
                    res += c2;
                    map[i].put(d, c1 + c2 + 1);
                }
            }

            return res;
        }
    }
}
```

424. `_447.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 447. Number of Boomerangs
 *
 * Given n points in the plane that are all pairwise distinct,
 * a "boomerang" is a tuple of points (i, j, k) such that the distance
 * between i and j equals the distance between i and k (the order of the tuple matters).
 *
 * Find the number of boomerangs.
 * You may assume that n will be at most 500 and coordinates of points are all in the
range [-10000, 10000] (inclusive).

 Example:
 Input:
 [[0,0],[1,0],[2,0]]

 Output:
 2

 Explanation:
 The two boomerangs are [[1,0],[0,0],[2,0]] and [[1,0],[2,0],[0,0]]*/
public class _447 {

    public static class Solution1 {
        /**
         * Looked at these two posts: https://discuss.leetcode.com/topic/66587/clean-
java-solution-o-n-2-166ms and
         * https://discuss.leetcode.com/topic/66521/share-my-straightforward-solution-
with-hashmap-o-n-2, basically,
         * have a HashMap, key is the distance, value is the number of points that are
this distance apart to this point.
         * Note: we clear up this map every time after we traverse one point with the
rest of the other points.
         * <p>
         * Time complexity: O(n^2)
         * Space complexity: O(n)
         */

        public int numberOfBoomerangs(int[][] points) {
            int result = 0;
            if (points == null || points.length == 0 || points[0].length == 0) {
                return result;
            }
            int totalPts = points.length;
            Map<Long, Integer> map = new HashMap();
            for (int i = 0; i < totalPts; i++) {
                for (int j = 0; j < totalPts; j++) {
```

```
                    if (i == j) {
                        continue;
                    }
                    long d = calcDistance(points[i], points[j]);
                    map.put(d, map.getOrDefault(d, 0) + 1);
                }

                for (int val : map.values()) {
                    result += val * (val - 1);
                }
                map.clear();
            }
            return result;
        }

        private long calcDistance(int[] p1, int[] p2) {
            long x = p2[0] - p1[0];
            long y = p2[1] - p1[1];
            return x * x + y * y;
        }
    }

}
```

425. `_448.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 448. Find All Numbers Disappeared in an Array
 *
 * Given an array of integers where 1 ≤ a[i] ≤ n (n = size of array), some elements
appear twice and others appear once.
 * Find all the elements of [1, n] inclusive that do not appear in this array.
 * Could you do it without extra space and in O(n) runtime? You may assume the returned
list does not count as extra space.

 Example:

 Input:
 [4,3,2,7,8,2,3,1]

 Output:
 [5,6]

 */
public class _448 {

    public static class Solution1 {
        /**
         * O(n) space
         * O(n) time
         */
        public List<Integer> findDisappearedNumbers(int[] nums) {

            int max = Integer.MIN_VALUE;
            for (int i : nums) {
                max = Math.max(max, i);
            }
            max = Math.max(max, nums.length);

            Map<Integer, Integer> map = new HashMap();
            for (int i = 1; i <= max; i++) {
                map.put(i, 0);
            }

            for (int i : nums) {
                if (map.get(i) == 0) {
                    map.put(i, 1);
                } else {
                    map.put(i, map.get(i) + 1);
                }
```

```
            }

            List<Integer> result = new ArrayList();
            for (int i : map.keySet()) {
                if (map.get(i) == 0) {
                    result.add(i);
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * O(1) space
         * O(n) time
         */
        public List<Integer> findDisappearedNumbers(int[] nums) {

            for (int i = 0; i < nums.length; i++) {
                int val = Math.abs(nums[i]) - 1;
                if (nums[val] > 0) {
                    nums[val] = -nums[val];
                }
            }

            List<Integer> result = new ArrayList();
            for (int i = 0; i < nums.length; i++) {
                if (nums[i] > 0) {
                    result.add(i + 1);
                }
            }
            return result;
        }
    }

}
```

426. `_449.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 449. Serialize and Deserialize BST
 *
 * Serialization is the process of converting a data structure or
 * object into a sequence of bits so that it can be stored in a file or memory buffer,
 * or transmitted across a network connection link to be reconstructed later in the same
or another computer environment.
 *
 * Design an algorithm to serialize and deserialize a binary search tree.
 * There is no restriction on how your serialization/deserialization algorithm should
work.
 * You just need to ensure that a binary search tree can be serialized to a string and
this
 * string can be deserialized to the original tree structure.
 * The encoded string should be as compact as possible.

 Note: Do not use class member/global/static variables to store states. Your serialize
and deserialize algorithms should be stateless.
 */

public class _449 {

    public static class Solution1 {
        /**
         * Preorder
         * Reference: https://discuss.leetcode.com/topic/97922/pre-or-post-order-with-
only-keeping-one-bound-beat-98-and-95
         */

        // Encodes a tree to a single string.
        public String serialize(TreeNode root) {
            if (root == null) {
                return null;
            }
            StringBuilder stringBuilder = new StringBuilder();
            return serialize(root, stringBuilder);
        }

        private String serialize(TreeNode root, StringBuilder stringBuilder) {
            if (root == null) {
                return null;
            }
            stringBuilder.append(root.val).append(" ");
            serialize(root.left, stringBuilder);
```

```
            serialize(root.right, stringBuilder);
            return stringBuilder.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.length() == 0) {
                return null;
            }
            String[] values = data.split(" ");
            int[] index = new int[]{0};/**TODO: Why must use an int array, instead of
just an int?*/
            return deserialize(values, index, Integer.MAX_VALUE);
        }

        private TreeNode deserialize(String[] values, int[] index, int maxValue) {
            if (index[0] >= values.length || Integer.valueOf(values[index[0]]) >=
maxValue) {
                return null;
            }
            TreeNode root = new TreeNode(Integer.valueOf(values[index[0]++]));
            root.left = deserialize(values, index, root.val);
            root.right = deserialize(values, index, maxValue);
            return root;
        }
    }

    public static class Solution2 {
        /**
         * Postorder
         * Reference: https://discuss.leetcode.com/topic/97922/pre-or-post-order-with-
only-keeping-one-bound-beat-98-and-95
         */

        public String serialize(TreeNode root) {
            if (root == null) {
                return null;
            }
            StringBuilder sb = new StringBuilder();
            return serialize(root, sb);
        }

        private String serialize(TreeNode root, StringBuilder sb) {
            if (root == null) {
                return null;
            }
            serialize(root.left, sb);
            serialize(root.right, sb);
            sb.append(root.val).append(" ");
            return sb.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
```

```
            if (data == null || data.length() == 0) {
                return null;
            }
            String[] values = data.split(" ");
            int[] index = new int[]{values.length - 1};/**TODO: This is not just one
element any more like in the preorder solution above*/
            return deserialize(values, index, Integer.MIN_VALUE);
        }

        private TreeNode deserialize(String[] values, int[] index, int minValue) {
            if (index[0] < 0 || Integer.valueOf(values[index[0]]) < minValue) {
                return null;
            }
            TreeNode root = new TreeNode(Integer.valueOf(values[index[0]--]));
            root.right = deserialize(values, index, root.val);
            root.left = deserialize(values, index, minValue);
            return root;
        }
    }

    public static class Solution3 {
        /**
         * This is a generic solution that applies to both BT and BST. And also the
easiest to follow.
         */

        // Encodes a tree to a single string.
        public String serialize(TreeNode root) {
            Queue<TreeNode> queue = new LinkedList<>();
            StringBuilder stringBuilder = new StringBuilder();
            if (root == null) {
                return stringBuilder.toString();
            }
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (curr == null) {
                        stringBuilder.append("# ");
                    } else {
                        stringBuilder.append(curr.val + " ");
                        queue.offer(curr.left);
                        queue.offer(curr.right);
                    }
                }
            }
            return stringBuilder.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.length() == 0) {
                return null;
```

```
            }
            String[] nodes = data.split(" ");
            TreeNode root = new TreeNode(Integer.valueOf(nodes[0]));
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            for (int i = 1; i < nodes.length; i++) {
                TreeNode curr = queue.poll();
                if (!nodes[i].equals("#")) {
                    curr.left = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.left);
                }
                if (!nodes[++i].equals("#")) {
                    curr.right = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.right);
                }
            }
            return root;
        }
    }
}
```

427. `_450.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 450. Delete Node in a BST
 *
 * Given a root node reference of a BST and a key, delete the node with the given key in
the BST.
 * Return the root node reference (possibly updated) of the BST.

 Basically, the deletion can be divided into two stages:

 Search for a node to remove.
 If the node is found, delete the node.
 Note: Time complexity should be O(height of tree).

 Example:

 root = [5,3,6,2,4,null,7]
 key = 3

     5
    / \
   3   6
  / \   \
 2  4   7

 Given key to delete is 3. So we find the node with value 3 and delete it.

 One valid answer is [5,4,6,2,null,null,7], shown in the following BST.

     5
    / \
   4   6
  /     \
 2       7

 Another valid answer is [5,2,6,null,4,null,7].

     5
    / \
   2   6
    \   \
    4   7
 */
public class _450 {
    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/65792/recursive-easy-to-understand-
```

```
java-solution
         * Steps:
         * 1. Recursively find the node that has the same value as the key, while setting the left/right nodes equal to the returned subtree
         * 2. Once the node is found, have to handle the below 4 cases
         * a. node doesn't have left or right - return null
         * b. node only has left subtree- return the left subtree
         * c. node only has right subtree- return the right subtree
         * d. node has both left and right - find the minimum value in the right subtree, set that value to the currently found node, then recursively delete the minimum value in the right subtree
         */
        public TreeNode deleteNode(TreeNode root, int key) {
            if (root == null) {
                return root;
            }
            if (root.val > key) {
                root.left = deleteNode(root.left, key);
            } else if (root.val < key) {
                root.right = deleteNode(root.right, key);
            } else {
                if (root.left == null) {
                    return root.right;
                } else if (root.right == null) {
                    return root.left;
                }

                TreeNode minNode = getMin(root.right);
                root.val = minNode.val;
                root.right = deleteNode(root.right, root.val);
            }
            return root;
        }

        private TreeNode getMin(TreeNode node) {
            while (node.left != null) {
                node = node.left;
            }
            return node;
        }
    }

}
```

428. `_451.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 451. Sort Characters By Frequency
 *
 * Given a string, sort it in decreasing order based on the frequency of characters.

 Example 1:

 Input:
 "tree"

 Output:
 "eert"

 Explanation:
 'e' appears twice while 'r' and 't' both appear once.
 So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

 Example 2:

 Input:
 "cccaaa"

 Output:
 "cccaaa"

 Explanation:
 Both 'c' and 'a' appear three times, so "aaaccc" is also a valid answer.
 Note that "cacaca" is incorrect, as the same characters must be together.

 Example 3:

 Input:
 "Aabb"

 Output:
 "bbAa"

 Explanation:
 "bbaA" is also a valid answer, but "Aabb" is incorrect.
 Note that 'A' and 'a' are treated as two different characters.

 */
public class _451 {
```

```
    public static class Solution1 {
        public String frequencySort(String s) {
            Map<Character, Integer> map = new HashMap();
            for (char c : s.toCharArray()) {
                map.put(c, map.getOrDefault(c, 0) + 1);
            }
            List<Map.Entry<Character, Integer>> list = new ArrayList<>(map.entrySet());
            Collections.sort(list, (o1, o2) -> (o2.getValue() - o1.getValue()));
            StringBuilder stringBuilder = new StringBuilder();
            for (Map.Entry<Character, Integer> entry : list) {
                for (int i = 0; i < entry.getValue(); i++) {
                    stringBuilder.append(entry.getKey());
                }
            }
            return stringBuilder.toString();
        }
    }
}
```

429. `_452.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 *452. Minimum Number of Arrows to Burst Balloons
 *
 * There are a number of spherical balloons spread in two-dimensional space.
 * For each balloon, provided input is the start and end coordinates of the horizontal
diameter.
 * Since it's horizontal, y-coordinates don't matter and hence the x-coordinates of start
and end of the diameter suffice.
 * Start is always smaller than end. There will be at most 104 balloons.
 *
 * An arrow can be shot up exactly vertically from different points along the x-axis.
 * A balloon with xstart and xend bursts by an arrow shot at x if xstart ≤ x ≤ xend.
 * There is no limit to the number of arrows that can be shot.
 * An arrow once shot keeps travelling up infinitely.
 * The problem is to find the minimum number of arrows that must be shot to burst all
balloons.

 Example:

 Input:
 [[10,16], [2,8], [1,6], [7,12]]

 Output:
 2

 Explanation:
 One way is to shoot one arrow for example at x = 6 (bursting the balloons [2,8] and
[1,6]) and another arrow at x = 11 (bursting the other two balloons).
 */
public class _452 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/66579/java-greedy-soution/6
         */
        public int findMinArrowShots(int[][] points) {
            if (points == null || points.length == 0) {
                return 0;
            }
            // sort points based on their end point.
            Arrays.sort(points, (p1, p2) -> Integer.compare(p1[1], p2[1]));
            int currentEnd = points[0][1];
            int count = 1;
            for (int[] p : points) {
                // if the point starts after currentEnd, it means this balloons not been
bursted. Then we shot the balloon in its end point. Otherwise, means this balloon has
been bursted, then ignore it.
```

```
                if (p[0] > currentEnd) {
                    count++;
                    currentEnd = p[1];
                } else {
                    continue;
                }
            }
            return count;
        }
    }
}
```

430. `_453.java`

```
package com.fishercoder.solutions;

/**
 * 453. Minimum Moves to Equal Array Elements
 *
 * Given a non-empty integer array of size n,
 * find the minimum number of moves required to make all array elements equal, where a
move is incrementing n - 1 elements by 1.

 Example:

 Input:
 [1,2,3]

 Output:
 3

 Explanation:
 Only three moves are needed (remember each move increments two elements):

 [1,2,3]  =>  [2,3,3]  =>  [3,4,3]  =>  [4,4,4]*/

public class _453 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/66557/java-o-n-solution-short
         * i.e. Add 1 to n-1 elements basically equals to subtracting 1 from one element.
So the easiest way
         * to make all elements in this array equal is to make all of them equal to the
minimum element.
         */
        public int minMoves(int[] nums) {
            if (nums.length == 0) {
                return 0;
            }
            int min = nums[0];
            for (int n : nums) {
                min = Math.min(min, n);
            }
            int res = 0;
            for (int n : nums) {
                res += n - min;
            }
            return res;
        }
    }

}
```

431. `_454.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 454. 4Sum II
 *
 * Given four lists A, B, C, D of integer values,
 * compute how many tuples (i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is
zero.
 * To make problem a bit easier, all A, B, C, D have same length of N where 0 ≤ N ≤ 500.
 * All integers are in the range of -228 to 228 - 1 and the result is guaranteed to be at
most 231 - 1.

 Example:

 Input:
 A = [ 1, 2]
 B = [-2,-1]
 C = [-1, 2]
 D = [ 0, 2]

 Output:
 2

 Explanation:
 The two tuples are:
 1. (0, 0, 0, 1) -> A[0] + B[0] + C[0] + D[1] = 1 + (-2) + (-1) + 2 = 0
 2. (1, 1, 0, 0) -> A[1] + B[1] + C[0] + D[0] = 2 + (-1) + (-1) + 0 = 0*/

public class _454 {

    public static class Solution1 {
        public int fourSumCount(int[] A, int[] B, int[] C, int[] D) {
            Map<Integer, Integer> map = new HashMap();
            int result = 0;
            int len = A.length;
            for (int i = 0; i < len; i++) {
                for (int j = 0; j < len; j++) {
                    int sum = A[i] + B[j];
                    if (map.containsKey(sum)) {
                        map.put(sum, map.get(sum) + 1);
                    } else {
                        map.put(sum, 1);
                    }
                }
            }

            for (int i = 0; i < len; i++) {
                for (int j = 0; j < len; j++) {
```

```
                    int sum = -(C[i] + D[j]);
                    if (map.containsKey(sum)) {
                        result += map.get(sum);
                    }
                }
            }

            return result;
        }
    }
}
```

432. `_455.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 455. Assign Cookies
 *
 * Assume you are an awesome parent and want to give your children some cookies.
 * But, you should give each child at most one cookie.
 * Each child i has a greed factor gi, which is the minimum size of a cookie
 * that the child will be content with;
 * and each cookie j has a size sj. If sj >= gi, we can assign the cookie j to the child
i,
 * and the child i will be content. Your goal is to maximize the number of your content
children and output the maximum number.

 Note:
 You may assume the greed factor is always positive.
 You cannot assign more than one cookie to one child.

 Example 1:
 Input: [1,2,3], [1,1]
 Output: 1
 Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1,
2, 3.
 And even though you have 2 cookies, since their size is both 1, you could only make the
child whose greed factor is 1 content.
 You need to output 1.

 Example 2:
 Input: [1,2], [1,2,3]
 Output: 2
 Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1,
2.
 You have 3 cookies and their sizes are big enough to gratify all of the children,
 You need to output 2.*/

public class _455 {

    public static class Solution1 {
        public int findContentChildren(int[] g, int[] s) {
            Arrays.sort(g);
            Arrays.sort(s);

            int result = 0;
            for (int i = 0, j = 0; i < g.length && j < s.length; ) {
                if (s[j] >= g[i]) {
                    result++;
                    i++;
                }
                j++;
```

```
            }
            return result;
        }
    }

}
```

433. `_456.java`

```
package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;
/**
 * 456. 132 Pattern
 *
 * Given a sequence of n integers a1, a2, ..., an,
 * a 132 pattern is a subsequence ai, aj, ak such that i < j < k and ai < ak < aj.
 * Design an algorithm that takes a list of n numbers as input and checks whether there
is a 132 pattern in the list.

 Note: n will be less than 15,000.

 Example 1:
 Input: [1, 2, 3, 4]

 Output: False

 Explanation: There is no 132 pattern in the sequence.
 Example 2:
 Input: [3, 1, 4, 2]

 Output: True

 Explanation: There is a 132 pattern in the sequence: [1, 4, 2].
 Example 3:
 Input: [-1, 3, 2, 0]

 Output: True

 Explanation: There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and
[-1, 2, 0].
 */

public class _456 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/67881/single-pass-c-o-n-space-and-
time-solution-8-lines-with-detailed-explanation
         * It scans only once, this is the power of using correct data structure.
         * It goes from the right to the left.
         * It keeps pushing elements into the stack,
         * but it also keeps poping elements out of the stack as long as the current
element is bigger than this number.
         */
        public boolean find132pattern(int[] nums) {
            Deque<Integer> stack = new LinkedList<>();

            int s3 = Integer.MIN_VALUE;
```

```
            for (int i = nums.length - 1; i >= 0; i--) {
                if (nums[i] < s3) {
                    return true;
                } else {
                    while (!stack.isEmpty() && nums[i] > stack.peek()) {
                        s3 = Math.max(s3, stack.pop());
                    }
                }
                stack.push(nums[i]);
            }

            return false;
        }
    }

}
```

434. `_457.java`

```
package com.fishercoder.solutions;

/**
 * 457. Circular Array Loop
 *
 * You are given an array of positive and negative integers.
 * If a number n at an index is positive, then move forward n steps.
 * Conversely, if it's negative (-n), move backward n steps.
 *
 * Assume the first element of the array is forward next to the last element,
 * and the last element is backward next to the first element.
 * Determine if there is a loop in this array.
 * A loop starts and ends at a particular index with more than 1 element along the loop.
The loop must be "forward" or "backward'.
 *
 * Example 1: Given the array [2, -1, 1, 2, 2], there is a loop, from index 0 -> 2 -> 3 -
> 0.
 * Example 2: Given the array [-1, 2], there is no loop.
 *
 * Note: The given array is guaranteed to contain no element "0".
 *
 * Can you do it in O(n) time complexity and O(1) space complexity?
 */
public class _457 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/66894/java-slow-fast-pointer-
solution
         */
        public boolean circularArrayLoop(int[] nums) {
            int n = nums.length;
            for (int i = 0; i < n; i++) {
                if (nums[i] == 0) {
                    continue;
                }
                // slow/fast pointer
                int j = i;
                int k = getIndex(i, nums);
                while (nums[k] * nums[i] > 0 && nums[getIndex(k, nums)] * nums[i] > 0) {
                    if (j == k) {
                        // check for loop with only one element
                        if (j == getIndex(j, nums)) {
                            break;
                        }
                        return true;
                    }
                    j = getIndex(j, nums);
                    k = getIndex(getIndex(k, nums), nums);
                }
                // loop not found, set all element along the way to 0
                j = i;
```

```
                int val = nums[i];
                while (nums[j] * val > 0) {
                    int next = getIndex(j, nums);
                    nums[j] = 0;
                    j = next;
                }
            }
            return false;
        }

        public int getIndex(int i, int[] nums) {
            int n = nums.length;
            return i + nums[i] >= 0 ? (i + nums[i]) % n : n + ((i + nums[i]) % n);
        }
    }
}
```

435. `_458.java`

```
package com.fishercoder.solutions;

/**
 * 458. Poor Pigs
 *
 * There are 1000 buckets, one and only one of them contains poison,
 * the rest are filled with water.
 * They all look the same.
 * If a pig drinks that poison it will die within 15 minutes.
 * What is the minimum amount of pigs you need to figure out which bucket contains the
poison within one hour.
 * Answer this question, and write an algorithm for the follow-up general case.

 Follow-up:
 If there are n buckets and a pig drinking poison will die within m minutes,
 how many pigs (x) you need to figure out the "poison" bucket within p minutes? There is
exact one bucket with poison.

 */
public class _458 {

    public static class Solution1 {
        public int poorPigs(int buckets, int minutesToDie, int minutesToTest) {
            if (buckets-- == 1) {
                return 0;
            }
            int base = minutesToTest / minutesToDie + 1;
            int count = 0;
            while (buckets > 0) {
                buckets /= base;
                count++;
            }
            return count;
        }
    }

}
```

436. `_459.java`

```
package com.fishercoder.solutions;

/**
 * 459. Repeated Substring Pattern
 * Given a non-empty string check if it can be constructed by taking a substring of it
and appending multiple copies of the substring together.
 * You may assume the given string consists of lowercase English letters only and its
length will not exceed 10000.

 Example 1:
 Input: "abab"

 Output: True

 Explanation: It's the substring "ab" twice.
 Example 2:
 Input: "aba"

 Output: False
 Example 3:
 Input: "abcabcabcabc"

 Output: True

 Explanation: It's the substring "abc" four times. (And the substring "abcabc" twice.)
 */

public class _459 {

    public static class Solution1 {
        public boolean repeatedSubstringPattern(String s) {
            int len = s.length();
            for (int i = len / 2; i >= 1; i--) {
                String pattern = s.substring(0, i);
                if (len % i == 0) {
                    String formedString = formStringByCopying(pattern, len / i);
                    if (formedString.equals(s)) {
                        return true;
                    }
                }
            }
            return false;
        }

        private String formStringByCopying(String pattern, int k) {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < k; i++) {
                sb.append(pattern);
            }
            return sb.toString();
        }
```

```
    }

    public static class Solution2 {
        /**
         * credit: https://discuss.leetcode.com/topic/68089/repeated-substring-pattern-
simple-java-solution-using-kmp
         */
        public static boolean repeatedSubstringPattern(String str) {
            //build the KMP pattern.
            int n = str.length();
            int cur = 0;
            int j = 1;
            int[] pattern = new int[n];
            pattern[0] = 0;

            while (j < n) {
                if (str.charAt(cur) == str.charAt(j)) {
                    pattern[j++] = ++cur;
                } else {
                    if (cur == 0) {
                        pattern[j++] = 0;
                    } else {
                        cur = pattern[cur - 1];//start from beginning of current matching
pattern.
                    }
                }
            }

            return (pattern[n - 1] > 0 && n % (n - pattern[n - 1]) == 0);
        }
    }

    public static class Solution3 {
        public static boolean repeatedSubstringPattern(String str) {
            String s = str + str;
            return s.substring(1, s.length() - 1).contains(str);
        }
    }
}
```

437. `_460.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.LinkedHashSet;

/**460. LFU Cache

 * Design and implement a data structure for Least Frequently Used (LFU) cache.
 * It should support the following operations: get and put.

 get(key) - Get the value (will always be positive) of the key if the key exists in the
cache, otherwise return -1.
 put(key, value) - Set or insert the value if the key is not already present. When the
cache reaches its capacity,
 it should invalidate the least frequently used item before inserting a new item.
 For the purpose of this problem, when there is a tie (i.e., two or more keys that have
the same frequency),
 the least recently used key would be evicted.

 Follow up:
 Could you do both operations in O(1) time complexity?

 Example:

 LFUCache cache = new LFUCache( 2 /* capacity  );

        cache.put(1, 1);
        cache.put(2, 2);
        cache.get(1);       // returns 1
        cache.put(3, 3);    // evicts key 2
        cache.get(2);       // returns -1 (not found)
        cache.get(3);       // returns 3.
        cache.put(4, 4);    // evicts key 1.
        cache.get(1);       // returns -1 (not found)
        cache.get(3);       // returns 3
        cache.get(4);       // returns 4

 */

public class _460 {
    public static class Solution1 {
        /**
         * Wikipedia: The simplest method to employ an LFU algorithm is to assign a
counter to every
         * block that is loaded into the cache. Each time a reference is made to that
block the counter
         * is increased by one. When the cache reaches capacity and has a new block
waiting to be
         * inserted the system will search for the block with the lowest counter and
remove it from the
         * cache.
```

```
         * Policy to handle frequency ties: based on timestamp, the entries that get set
into cache earlier will be evicted first.
         */

        public static class LFUCache {
            /**
             * credit: https://discuss.leetcode.com/topic/69737/java-o-1-very-easy-
solution-using-3-hashmaps-and-linkedhashset/2
             */

            HashMap<Integer, Integer> keyToValue;
            /**
             * key is the key, value is the value
             */
            HashMap<Integer, Integer> keyToCount;
            /**
             * key is the key, value if the count of the key/value pair
             */
            HashMap<Integer, LinkedHashSet<Integer>> countToLRUKeys;
            /**
             * key is count, value is a set of keys that have the same key, but keeps
insertion order
             */
            int cap;
            int minimumCount;

            public LFUCache(int capacity) {
                this.minimumCount = -1;
                this.cap = capacity;
                this.keyToValue = new HashMap<>();
                this.keyToCount = new HashMap<>();
                this.countToLRUKeys = new HashMap<>();
                this.countToLRUKeys.put(1, new LinkedHashSet<>());
            }

            public int get(int key) {
                if (!keyToValue.containsKey(key)) {
                    return -1;
                }
                int count = keyToCount.get(key);
                keyToCount.put(key, count + 1);
                countToLRUKeys.get(count).remove(key);

                if (count == minimumCount && countToLRUKeys.get(count).size() == 0) {
                    /**This means this key's count equals to current minimumCount
                     * AND
                     * this count doesn't have any entries in the cache.
                     * So, we'll increment minimumCount by 1 to get the next LFU cache
entry
                     * when we need to evict.*/
                    minimumCount++;
                }

                if (!countToLRUKeys.containsKey(count + 1)) {
```

```
                    countToLRUKeys.put(count + 1, new LinkedHashSet<>());
                }
                countToLRUKeys.get(count + 1).add(key);

                return keyToValue.get(key);
            }

            public void put(int key, int value) {
                if (cap <= 0) {
                    return;
                }

                if (keyToValue.containsKey(key)) {
                    /**If the key is already in the cache, we can simply overwrite this
entry;
                     * then call get(key) which will do the update work.*/
                    keyToValue.put(key, value);
                    get(key);
                    return;
                }

                /**If the key is not in the cache, we'll check the size first, evict the
LFU entry first,
                 * then insert this one into cache.*/
                if (keyToValue.size() >= cap) {
                    int evit = countToLRUKeys.get(minimumCount).iterator().next();
                    countToLRUKeys.get(minimumCount).remove(evit);
                    keyToValue.remove(evit);
                }
                keyToValue.put(key, value);
                keyToCount.put(key, 1);
                countToLRUKeys.get(1).add(key);/**Because we put this key/value into
cache for the first time, so its count is 1*/
                minimumCount = 1;/**For the same above reason, minimumCount is of course
1*/
            }
        }
    }

}
```

438. `_461.java`

```
package com.fishercoder.solutions;

/**
 * 461. Hamming Distance
 *
 * The Hamming distance between two integers is the number of positions at which the
corresponding bits are different.

 Given two integers x and y, calculate the Hamming distance.

 Note:
 0 ≤ x, y < 231.

 Example:

 Input: x = 1, y = 4

 Output: 2

 Explanation:
 1   (0 0 0 1)
 4   (0 1 0 0)
 ↑   ↑

 The above arrows point to positions where the corresponding bits are different.
 */
public class _461 {
        public static class Solution1 {
                public int hammingDistance(int x, int y) {
                        int n = x ^ y;
                        int count = 0;
                        while (n != 0) {
                                count++;
                                n &= (n - 1);
                        }
                        return count;
                }
        }
}
```

439. `_462.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 462. Minimum Moves to Equal Array Elements II
 *
 * Given a non-empty integer array,
 * find the minimum number of moves required to make all array elements equal,
 * where a move is incrementing a selected element by 1 or decrementing a selected
element by 1.
 *
 * You may assume the array's length is at most 10,000.

 Example:

 Input:
 [1,2,3]

 Output:
 2

 Explanation:
 Only two moves are needed (remember each move increments or decrements one element):

 [1,2,3]  =>  [2,2,3]  =>  [2,2,2]
 */

public class _462 {

    public static class Solution1 {
        public int minMoves2(int[] nums) {
            /**sort this array, find the median of this array as the pivot*/
            Arrays.sort(nums);
            int result = 0;
            int result1 = 0;
            if (nums.length % 2 != 0) {
                int median = nums[nums.length / 2];
                for (int n : nums) {
                    result += Math.abs(n - median);
                }
                return result;
            } else {
                int median1 = nums[nums.length / 2];
                for (int n : nums) {
                    result1 += Math.abs(n - median1);
                }
                int result2 = 0;
                int median2 = nums[nums.length / 2 - 1];
                for (int n : nums) {
                    result2 += Math.abs(n - median2);
```

```
                }
                result1 = Math.min(result1, result2);
                return result1;
            }
        }
    }

}
```

440. `_463.java`

```
package com.fishercoder.solutions;
/**
 * 463. Island Perimeter
 *
 * You are given a map in form of a two-dimensional integer grid where 1 represents land
and 0 represents water.
 * Grid cells are connected horizontally/vertically (not diagonally).
 * The grid is completely surrounded by water, and there is exactly one island (i.e., one
or more connected land cells).
 * The island doesn't have "lakes" (water inside that isn't connected to the water around
the island).
 * One cell is a square with side length 1.
 * The grid is rectangular, width and height don't exceed 100. Determine the perimeter of
the island.

 Example:

 [[0,1,0,0],
 [1,1,1,0],
 [0,1,0,0],
 [1,1,0,0]]

 Answer: 16
 Explanation: The perimeter is the 16 yellow stripes in the image below:
 */
public class _463 {

    public static class Solution1 {
        /**
         * Inspired by this post: https://discuss.leetcode.com/topic/68983/java-9-line-
solution-add-4-for-each-land-and-remove-2-for-each-internal-edge
         * 1. we increment the count by 4 whenever we encounter an island
         * 2. also, we check in two directions: island's left and island's top, we only
check these two directions,
         * see if this island has any island neighbors, if so, we'll deduct two from it.
         */
        public int islandPerimeter(int[][] grid) {
            int count = 0;
            for (int i = 0; i < grid.length; i++) {
                for (int j = 0; j < grid[0].length; j++) {
                    if (grid[i][j] == 1) {
                        count += 4;
                        if (i > 0 && grid[i - 1][j] == 1) {
                            count -= 2;
                        }
                        if (j > 0 && grid[i][j - 1] == 1) {
                            count -= 2;
                        }
                    }
                }
            }
```

```
            return count;
        }
    }
}
```

441. `_464.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 464. Can I Win
 *
 * In the "100 game," two players take turns adding, to a running total, any integer from
1..10.
 * The player who first causes the running total to reach or exceed 100 wins.
 * What if we change the game so that players cannot re-use integers?
 * For example, two players might take turns drawing from a common pool of numbers of
1..15 without replacement until they reach a total >= 100.
 * Given an integer maxChoosableInteger and another integer desiredTotal, determine if
the first player to move can force a win, assuming both players play optimally.
 *You can always assume that maxChoosableInteger will not be larger than 20 and
desiredTotal will not be larger than 300.

 Example

 Input:
 maxChoosableInteger = 10
 desiredTotal = 11

 Output:
 false

 Explanation:
 No matter which integer the first player choose, the first player will lose.
 The first player can choose an integer from 1 up to 10.
 If the first player choose 1, the second player can only choose integers from 2 up to
10.
 The second player will win by choosing 10 and get a total = 11, which is >=
desiredTotal.
 Same with other integers chosen by the first player, the second player will always win.
 */
public class _464 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/68896/java-solution-using-hashmap-
with-detailed-explanation
         */

        Map<Integer, Boolean> map;
        boolean[] used;

        public boolean canIWin(int maxChoosableInteger, int desiredTotal) {
            int sum = (1 + maxChoosableInteger) * maxChoosableInteger / 2;
            if (sum < desiredTotal) {
                return false;
```

```
            }
            if (desiredTotal <= 0) {
                return true;
            }

            map = new HashMap();
            used = new boolean[maxChoosableInteger + 1];
            return helper(desiredTotal);
        }

        public boolean helper(int desiredTotal) {
            if (desiredTotal <= 0) {
                return false;
            }
            int key = format(used);
            if (!map.containsKey(key)) {
                // try every unchosen number as next step
                for (int i = 1; i < used.length; i++) {
                    if (!used[i]) {
                        used[i] = true;
                        // check whether this lead to a win (i.e. the other player lose)
                        if (!helper(desiredTotal - i)) {
                            map.put(key, true);
                            used[i] = false;
                            return true;
                        }
                        used[i] = false;
                    }
                }
                map.put(key, false);
            }
            return map.get(key);
        }

        // transfer boolean[] to an Integer
        public int format(boolean[] used) {
            int num = 0;
            for (boolean b : used) {
                num <<= 1;
                if (b) {
                    num |= 1;
                }
            }
            return num;
        }
    }
}
```

442. `_465.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Stack;

/**
 * 465. Optimal Account Balancing
 *
 * A group of friends went on holiday and sometimes lent each other money.
 * For example, Alice paid for Bill's lunch for $10.
 * Then later Chris gave Alice $5 for a taxi ride.
 * We can model each transaction as a tuple (x, y, z) which means person x gave person y
$z.
 * Assuming Alice, Bill, and Chris are person 0, 1, and 2 respectively (0, 1, 2 are the
person's ID),
 * the transactions can be represented as [[0, 1, 10], [2, 0, 5]].

 Given a list of transactions between a group of people, return the minimum number of
transactions required to settle the debt.

 Note:

 A transaction will be given as a tuple (x, y, z). Note that x ? y and z > 0.
 Person's IDs may not be linear, e.g. we could have the persons 0, 1, 2 or we could also
have the persons 0, 2, 6.
 Example 1:

 Input:
 [[0,1,10], [2,0,5]]

 Output:
 2

 Explanation:
 Person #0 gave person #1 $10.
 Person #2 gave person #0 $5.

 Two transactions are needed. One way to settle the debt is person #1 pays person #0 and
#2 $5 each.
 Example 2:

 Input:
 [[0,1,10], [1,0,1], [1,2,5], [2,0,5]]

 Output:
 1
```

```
 Explanation:
 Person #0 gave person #1 $10.
 Person #1 gave person #0 $1.
 Person #1 gave person #2 $5.
 Person #2 gave person #0 $5.

 Therefore, person #1 only need to give person #0 $4, and all debt is settled.
 */
public class _465 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/68948/easy-java-solution-with-
explanation
         */
        public int minTransfers(int[][] transactions) {
            if (transactions == null || transactions.length == 0) {
                return 0;
            }
            Map<Integer, Integer> acc = new HashMap<>();
            for (int i = 0; i < transactions.length; i++) {
                int id1 = transactions[i][0];
                int id2 = transactions[i][1];
                int m = transactions[i][2];
                acc.put(id1, acc.getOrDefault(id1, 0) - m);
                acc.put(id2, acc.getOrDefault(id2, 0) + m);
            }
            List<Integer> negs = new ArrayList<>();
            List<Integer> poss = new ArrayList<>();
            for (Integer key : acc.keySet()) {
                int m = acc.get(key);
                if (m == 0) {
                    continue;
                }
                if (m < 0) {
                    negs.add(-m);
                } else {
                    poss.add(m);
                }
            }
            int ans = Integer.MAX_VALUE;
            Stack<Integer> stNeg = new Stack<>();
            Stack<Integer> stPos = new Stack<>();
            for (int i = 0; i < 1000; i++) {
                for (Integer num : negs) {
                    stNeg.push(num);
                }
                for (Integer num : poss) {
                    stPos.push(num);
                }
                int cur = 0;
                while (!stNeg.isEmpty()) {
                    int n = stNeg.pop();
                    int p = stPos.pop();
                    cur++;
```

```
                    if (n == p) {
                        continue;
                    }
                    if (n > p) {
                        stNeg.push(n - p);
                    } else {
                        stPos.push(p - n);
                    }
                }
                ans = Math.min(ans, cur);
                Collections.shuffle(negs);
                Collections.shuffle(poss);
            }
            return ans;
        }
    }
}
```

443. `_466.java`

```
package com.fishercoder.solutions;

/**
 * 466. Count The Repetitions
 *
 * Define S = [s,n] as the string S which consists of n connected strings s. For example,
["abc", 3] ="abcabcabc".

 On the other hand, we define that string s1 can be obtained from string s2 if we can
remove some characters from s2 such that it becomes s1.
 For example, “abc” can be obtained from “abdbec” based on our definition, but it can not
be obtained from “acbbe”.
 You are given two non-empty strings s1 and s2 (each at most 100 characters long) and two
integers 0 ≤ n1 ≤ 106 and 1 ≤ n2 ≤ 106.
 Now consider the strings S1 and S2, where S1=[s1,n1] and S2=[s2,n2]. Find the maximum
integer m such that [S2,m] can be obtained from S1.

 Example:

 Input:
 s1="acb", n1=4
 s2="ab", n2=2

 Return:
 2

 */
public class _466 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/70707/ugly-java-brute-force-
solution-but-accepted-1088ms
         */
        public int getMaxRepetitions(String s1, int n1, String s2, int n2) {
            char[] s1chars = s1.toCharArray();
            char[] s2chars = s2.toCharArray();
            int i = 0;
            int j = 0;
            int count1 = 0;
            int count2 = 0;
            while (count1 < n1) {
                if (s1chars[i] == s2chars[j]) {
                    j++;
                    if (j == s2.length()) {
                        j = 0;
                        count2++;
                    }
                }
                i++;
                if (i == s1.length()) {
```

```
                    i = 0;
                    count1++;
                }
            }
            return count2 / n2;
        }
    }

}
```

444. `_467.java`

```
package com.fishercoder.solutions;

/**
 * 467. Unique Substrings in Wraparound String
 *
 * Consider the string s to be the infinite wraparound string of
"abcdefghijklmnopqrstuvwxyz",
 * so s will look like this:
"...zabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcd....".
 * Now we have another string p.
 * Your job is to find out how many unique non-empty substrings of p are present in s.
 * In particular, your input is the string p and you need to output the number of
different non-empty substrings of p in the string s.
 * Note: p consists of only lowercase English letters and the size of p might be over
10000.

 Example 1:
 Input: "a"
 Output: 1
 Explanation: Only the substring "a" of string "a" is in the string □s.

 Example 2:
 Input: "cac"
 Output: 2
 Explanation: There are two substrings "a", "c" of string "cac" in the string s.

 Example 3:
 Input: "zab"
 Output: 6
 Explanation: There are six substrings "z", "a", "b", "za", "ab", "zab" of string "zab"
in the string s.
 */
public class _467 {
    public static class Solution1 {
        /**
         * A naive solution would definitely result in TLE.
         * Since the pattern keeps repeating, DP is the way to go.
         * Because the input consists merely of lowercase English letters, we could
maintain an array of size 26,
         * keep updating this array, counting the substrings that end with this letter,
keep updating it with the largest one possible.
         * <p>
         * Inspired by this: https://discuss.leetcode.com/topic/70658/concise-java-
solution-using-dp
         */
        public int findSubstringInWraproundString(String p) {
            if (p == null || p.isEmpty()) {
                return 0;
            }
            if (p.length() < 2) {
                return p.length();
```

```
            }
            int count = 0;
            int[] dp = new int[26];
            dp[p.charAt(0) - 'a'] = 1;
            int len = 1;
            for (int i = 1; i < p.length(); i++) {
                if (p.charAt(i) - 1 == p.charAt(i - 1) || (p.charAt(i) == 'a' &&
p.charAt(i - 1) == 'z')) {
                    len++;
                } else {
                    len = 1;
                }
                dp[p.charAt(i) - 'a'] = Math.max(len, dp[p.charAt(i) - 'a']);
            }

            for (int i : dp) {
                count += i;
            }
            return count;
        }
    }

}
```

445. `_468.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 468. Validate IP Address
 *
 *  Write a function to check whether an input string is a valid IPv4 address or IPv6
address or neither.
 *  IPv4 addresses are canonically represented in dot-decimal notation, which consists of
four decimal numbers,
 *  each ranging from 0 to 255, separated by dots ("."), e.g.,172.16.254.1;
 *  Besides, leading zeros in the IPv4 is invalid. For example, the address 172.16.254.01
is invalid.

 IPv6 addresses are represented as eight groups of four hexadecimal digits,
 each group representing 16 bits.
 The groups are separated by colons (":").
 For example, the address 2001:0db8:85a3:0000:0000:8a2e:0370:7334 is a valid one.
 Also, we could omit some leading zeros among four hexadecimal digits and
 some low-case characters in the address to upper-case ones,
 so 2001:db8:85a3:0:0:8A2E:0370:7334 is also a valid IPv6 address(Omit leading zeros and
using upper cases).

 However, we don't replace a consecutive group of zero value with a single empty
 group using two consecutive colons (::) to pursue simplicity.
 For example, 2001:0db8:85a3::8A2E:0370:7334 is an invalid IPv6 address.

 Besides, extra leading zeros in the IPv6 is also invalid.
 For example, the address 02001:0db8:85a3:0000:0000:8a2e:0370:7334 is invalid.

 Note: You may assume there is no extra space or special characters in the input string.

 Example 1:
 Input: "172.16.254.1"
 Output: "IPv4"
 Explanation: This is a valid IPv4 address, return "IPv4".

 Example 2:
 Input: "2001:0db8:85a3:0:0:8A2E:0370:7334"
 Output: "IPv6"
 Explanation: This is a valid IPv6 address, return "IPv6".

 Example 3:
 Input: "256.256.256.256"
 Output: "Neither"
 Explanation: This is neither a IPv4 address nor a IPv6 address.
 validIPAddress */
public class _468 {
```

```
public static class Solution1 {

    static final String NEITHER = "Neither";

    public String validIPAddress(String IP) {
        if (IP.contains(".")) {
            return isValidIPv4(IP);
        } else if (IP.contains(":")) {
            return isValidIPv6(IP);
        } else {
            return NEITHER;
        }
    }

    private String isValidIPv6(String IP) {
        if (getDelimiterCount(IP, ':') != 7) {
            return NEITHER;
        }
        String[] bytes = IP.split("\\:");
        if (bytes.length != 8) {
            return NEITHER;
        }
        for (int i = 0; i < 8; i++) {
            if (hasInvalidIPV6Char(bytes[i])) {
                return NEITHER;
            }
            try {
                if (bytes[i].length() > 4) {
                    return NEITHER;
                }
                int intNum = Integer.parseInt(bytes[i], 16);
                if (intNum < 0) {
                    return NEITHER;
                }
                if (i == 0 && intNum != 0 && bytes[i].charAt(0) == '0') {
                    return NEITHER;
                }
            } catch (Exception e) {
                return NEITHER;
            }

        }
        return "IPv6";
    }

    private String isValidIPv4(String IP) {
        if (getDelimiterCount(IP, '.') != 3) {
            return NEITHER;
        }
        String[] bytes = IP.split("\\.");
        if (bytes.length != 4) {
            return NEITHER;
        }
```

```
            for (String num : bytes) {
                try {
                    int intNum = Integer.parseInt(num);
                    if (intNum > 255 || intNum < 0) {
                        return NEITHER;
                    }
                    if (intNum != 0) {
                        for (int i = 0; i < num.length(); i++) {
                            if (num.charAt(i) == '0') {
                                return NEITHER;
                            } else {
                                break;
                            }
                        }
                    } else if (intNum == 0) {
                        if (num.length() != 1) {
                            return NEITHER;
                        }
                    }
                } catch (Exception e) {
                    return NEITHER;
                }
            }
            return "IPv4";
        }

        private boolean hasInvalidIPV6Char(String str) {
            Set<Character> set = new HashSet<>(Arrays.asList('0', '1', '2', '3', '4',
'5', '6', '7', '8', '9',
                    'a', 'b', 'c', 'd', 'e', 'f', 'A', 'B', 'C', 'D', 'E', 'F'));
            for (char c : str.toCharArray()) {
                if (!set.contains(c)) {
                    return true;
                }
            }
            return false;
        }

        private int getDelimiterCount(String ip, char delimiter) {
            int count = 0;
            for (char c : ip.toCharArray()) {
                if (c == delimiter) {
                    count++;
                }
            }
            return count;
        }
    }
}
```

446. `_469.java`

```
package com.fishercoder.solutions;

import java.util.List;

/**
 * 469. Convex Polygon
 *
 * Given a list of points that form a polygon when joined sequentially, find if this
polygon is convex (Convex polygon definition).

 Note:

 There are at least 3 and at most 10,000 points.
 Coordinates are in the range -10,000 to 10,000.
 You may assume the polygon formed by given points is always a simple polygon (Simple
polygon definition).
 In other words, we ensure that exactly two edges intersect at each vertex, and that
edges otherwise don't intersect each other.

 Example 1:

 [[0,0],[0,1],[1,1],[1,0]]

 Answer: True

 Example 2:

 [[0,0],[0,10],[10,10],[10,0],[5,5]]

 Answer: False

 */
public class _469 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/70706/beyond-my-knowledge-java-
solution-with-in-line-explanation
         */
        public boolean isConvex(List<List<Integer>> points) {
            // For each set of three adjacent points A, B, C, find the cross product AB ·
BC. If the sign of
            // all the cross products is the same, the angles are all positive or
negative (depending on the
            // order in which we visit them) so the polygon is convex.
            boolean gotNegative = false;
            boolean gotPositive = false;
            int numPoints = points.size();
            int B;
            int C;
            for (int A = 0; A < numPoints; A++) {
```

```
                // Trick to calc the last 3 points: n - 1, 0 and 1.
                B = (A + 1) % numPoints;
                C = (B + 1) % numPoints;

                int crossProduct =
                        crossProductLength(
                                points.get(A).get(0), points.get(A).get(1),
                                points.get(B).get(0), points.get(B).get(1),
                                points.get(C).get(0), points.get(C).get(1));
                if (crossProduct < 0) {
                    gotNegative = true;
                } else if (crossProduct > 0) {
                    gotPositive = true;
                }
                if (gotNegative && gotPositive) {
                    return false;
                }
            }

            // If we got this far, the polygon is convex.
            return true;
        }

        // Return the cross product AB x BC.
        // The cross product is a vector perpendicular to AB and BC having length |AB| *
|BC| * Sin(theta) and
        // with direction given by the right-hand rule. For two vectors in the X-Y plane,
the result is a
        // vector with X and Y components 0 so the Z component gives the vector's length
and direction.
        private int crossProductLength(int Ax, int Ay, int Bx, int By, int Cx, int Cy) {
            // Get the vectors' coordinates.
            int BAx = Ax - Bx;
            int BAy = Ay - By;
            int BCx = Cx - Bx;
            int BCy = Cy - By;

            // Calculate the Z coordinate of the cross product.
            return (BAx * BCy - BAy * BCx);
        }
    }

}
```

447. `_471.java`

```
package com.fishercoder.solutions;

/**
 * 471. Encode String with Shortest Length
 *
 Given a non-empty string, encode the string such that its encoded length is the
shortest.

 The encoding rule is: k[encoded_string], where the encoded_string inside the square
 brackets is being repeated exactly k times.

 Note:
 k will be a positive integer and encoded string will not be empty or have extra space.
 You may assume that the input string contains only lowercase English letters. The
string's length is at most 160.
 If an encoding process does not make the string shorter,
 then do not encode it. If there are several solutions, return any of them is fine.

 Example 1:

 Input: "aaa"
 Output: "aaa"
 Explanation: There is no way to encode it such that it is shorter than the input string,
so we do not encode it.
 Example 2:

 Input: "aaaaa"
 Output: "5[a]"
 Explanation: "5[a]" is shorter than "aaaaa" by 1 character.
 Example 3:

 Input: "aaaaaaaaaa"
 Output: "10[a]"
 Explanation: "a9[a]" or "9[a]a" are also valid solutions, both of them have the same
length = 5, which is the same as "10[a]".
 Example 4:

 Input: "aabcaabcd"
 Output: "2[aabc]d"
 Explanation: "aabc" occurs twice, so one answer can be "2[aabc]d".
 Example 5:

 Input: "abbbabbbcabbbabbbc"
 Output: "2[2[abbb]c]"
 Explanation: "abbbabbbc" occurs twice, but "abbbabbbc" can also be encoded to
"2[abbb]c", so one answer can be "2[2[abbb]c]".
 */
public class _471 {

    public static class Solution1 {
        /**
```

```
         * credit: https://discuss.leetcode.com/topic/71963/accepted-solution-in-java
         */
        public String encode(String s) {
            String[][] dp = new String[s.length()][s.length()];

            for (int l = 0; l < s.length(); l++) {
                for (int i = 0; i < s.length() - l; i++) {
                    int j = i + l;
                    String substr = s.substring(i, j + 1);
                    // Checking if string length < 5. In that case, we know that encoding
will not help.
                    if (j - i < 4) {
                        dp[i][j] = substr;
                    } else {
                        dp[i][j] = substr;
                        // Loop for trying all results that we get after dividing the
strings into 2 and combine the   results of 2 substrings
                        for (int k = i; k < j; k++) {
                            if ((dp[i][k] + dp[k + 1][j]).length() < dp[i][j].length()) {
                                dp[i][j] = dp[i][k] + dp[k + 1][j];
                            }
                        }

                        // Loop for checking if string can itself found some pattern in
it which could be repeated.
                        for (int k = 0; k < substr.length(); k++) {
                            String repeatStr = substr.substring(0, k + 1);
                            if (repeatStr != null
                                    && substr.length() % repeatStr.length() == 0
                                    && substr.replaceAll(repeatStr, "").length() == 0) {
                                String ss = substr.length() / repeatStr.length() + "[" +
dp[i][i + k] + "]";

                                if (ss.length() < dp[i][j].length()) {
                                    dp[i][j] = ss;
                                }
                            }
                        }
                    }
                }
            }
            return dp[0][s.length() - 1];
        }
    }

}
```

448. `_472.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 472. Concatenated Words
 *
 * Given a list of words, please write a program that returns all concatenated words in
the given list of words.

 A concatenated word is defined as a string that is comprised entirely of at least two
shorter words in the given array.

 Example:
 Input:
["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]

 Output: ["catsdogcats","dogcatsdog","ratcatdogcat"]

 Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";
 "dogcatsdog" can be concatenated by "dog", "cats" and "dog";
 "ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".
 Note:
 The number of elements of the given array will not exceed 10,000
 The length sum of elements in the given array will not exceed 600,000.
 All the input string will only include lower case letters.
 The returned elements order does not matter.

 */

public class _472 {

        public static class Solution1 {
                private TrieNode root;
                private int maxWordLen;

                public List<String> findAllConcatenatedWordsInADict(String[] words) {
                        ResultType result = buildTrie(words);
                        root = result.root;
                        maxWordLen = result.maxWordLen;

                        List<String> validConcatenatedWords = new ArrayList();
                        for (String word : words) {
                                if (word == null || word.length() == 0) {
                                        continue;
                                }
                                remove(word, root);/** every word is comprised of every
```

```
word itself, thus this word itself needs to be removed first for checking it*/
                                int n = word.length();
                                boolean[] dp = new boolean[n + 1];
                                dp[0] = true;

                                for (int i = 1; i <= n; i++) {
                                        for (int j = 1; j <= i && j <= maxWordLen; j++) {
                                                if (!dp[i - j]) {
                                                        continue;
                                                }

                                                String subWord = word.substring(i - j,
i);
                                                if (contains(subWord, root)) {
                                                        dp[i] = true;
                                                        break;
                                                }
                                        }
                                }

                                if (dp[n]) {
                                        validConcatenatedWords.add(word);
                                }
                                undoRemove(word, root);
                        }
                        return validConcatenatedWords;
                }

                public ResultType buildTrie(String[] words) {
                        ResultType result = new ResultType();

                        TrieNode root = new TrieNode();
                        int maxWordLen = 0;

                        for (String word : words) {
                                maxWordLen = Math.max(maxWordLen, word.length());
                                char[] chars = word.toCharArray();
                                TrieNode node = root;
                                for (int i = 0; i < chars.length; i++) {
                                        char c = chars[i];
                                        if (node.children[c - 'a'] == null) {
                                                node.children[c - 'a'] = new TrieNode();
                                        }
                                        node = node.children[c - 'a'];
                                }
                                node.isWord = true;
                        }

                        result.root = root;
                        result.maxWordLen = maxWordLen;
                        return result;
                }

                public class ResultType {
```

```
                int maxWordLen;
                TrieNode root;
        }

        // Returns true if the word is in the trie.
        public boolean contains(String word, TrieNode root) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                        if (node.children[word.charAt(i) - 'a'] == null) {
                                return false;
                        }
                        node = node.children[word.charAt(i) - 'a'];
                }
                return node.isWord;
        }

        // mark that word on
        public void undoRemove(String word, TrieNode root) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                        node = node.children[word.charAt(i) - 'a'];
                }
                node.isWord = true;
        }

        // mark that word off, we are not really deleting that word
        public void remove(String word, TrieNode root) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                        node = node.children[word.charAt(i) - 'a'];
                }
                node.isWord = false;
        }

        class TrieNode {
                boolean isWord;
                TrieNode[] children = new TrieNode[26];

                public TrieNode() {
                }
        }
}

public static class Solution2 {
        public List<String> findAllConcatenatedWordsInADict(String[] words) {
                List<String> result = new ArrayList<>();
                Set<String> preWords = new HashSet<>();
                /**Words could only be formed by other words that are shorter
than itself, so we sort them based on their lengths first.*/
                Arrays.sort(words, (s1, s2) -> s1.length() - s2.length());

                for (int i = 0; i < words.length; i++) {
                        if (canForm(words[i], preWords)) {
                                result.add(words[i]);
```

```
                        }
                        preWords.add(words[i]);
                }

                return result;
        }

        boolean canForm(String word, Set<String> dict) {
                if (dict.isEmpty()) {
                        return false;
                }
                boolean[] dp = new boolean[word.length() + 1];
                dp[0] = true;
                for (int i = 1; i <= word.length(); i++) {
                        for (int j = 0; j < i; j++) {
                                if (dp[j] && dict.contains(word.substring(j, i)))
{
                                        dp[i] = true;
                                        break;
                                }
                        }
                }
                return dp[word.length()];
        }
    }
}
```

449. `_473.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 473. Matchsticks to Square
 *
 * Remember the story of Little Match Girl? By now, you know exactly what matchsticks the
little match girl has,
 * please find out a way you can make one square by using up all those matchsticks.
 * You should not break any stick, but you can link them up, and each matchstick must be
used exactly one time.
 * Your input will be several matchsticks the girl has, represented with their stick
length.
 * Your output will either be true or false, to represent whether you could make one
square using all the matchsticks the little match girl has.

 Example 1:
 Input: [1,1,2,2,2]
 Output: true
 Explanation: You can form a square with length 2, one side of the square came two sticks
with length 1.

 Example 2:
 Input: [3,3,3,3,4]
 Output: false
 Explanation: You cannot find a way to form a square with all the matchsticks.

 Note:
 The length sum of the given matchsticks is in the range of 0 to 10^9.
 The length of the given matchstick array will not exceed 15.
 */
public class _473 {

    public static class Solution1 {
    /**
     * Partially inspired by: https://discuss.leetcode.com/topic/72107/java-dfs-solution-
with-explanation/2
     * One hidden requirement: you'll have to use up all of the given matchsticks,
nothing could be left behind.
     */
    public boolean makesquare(int[] nums) {
        if (nums == null || nums.length < 4) {
            return false;
        }
        Arrays.sort(nums);
        reverse(nums);
        int sum = 0;
        for (int i : nums) {
            sum += i;
        }
```

```
        if (sum % 4 != 0) {
            return false;
        }

        return dfs(nums, new int[4], 0, sum / 4);
    }

        private boolean dfs(int[] nums, int[] sums, int index, int target) {
            if (index == nums.length) {
                if (sums[0] == target && sums[1] == target && sums[2] == target) {
                    return true;
                }
                return false;
            }
            for (int i = 0; i < 4; i++) {
                if (sums[i] + nums[index] > target) {
                    continue;
                }
                sums[i] += nums[index];
                if (dfs(nums, sums, index + 1, target)) {
                    return true;
                }
                sums[i] -= nums[index];
            }
            return false;
        }

        private void reverse(int[] nums) {
            int left = 0;
            int right = nums.length - 1;
            while (left < right) {
                int tmp = nums[left];
                nums[left] = nums[right];
                nums[right] = tmp;
                left++;
                right--;
            }
        }
    }

}
```

450. `_474.java`

```
package com.fishercoder.solutions;

/**
 * 474. Ones and Zeroes
 *
 * In the computer world, use restricted resource you have to generate maximum benefit is
what we always want to pursue.
 * For now, suppose you are a dominator of m 0s and n 1s respectively.
 * On the other hand, there is an array with strings consisting of only 0s and 1s.
 * Now your task is to find the maximum number of strings that you can form with given m
0s and n 1s.
 * Each 0 and 1 can be used at most once.

 Note:

 The given numbers of 0s and 1s will both not exceed 100. The size of given string array
won't exceed 600.

 Example 1:
 Input: Array = {"10", "0001", "111001", "1", "0"}, m = 5, n = 3
 Output: 4
 Explanation: This are totally 4 strings can be formed by the using of 5 0s and 3 1s,
which are “10,”0001”,”1”,”0”

 Example 2:
 Input: Array = {"10", "0", "1"}, m = 1, n = 1
 Output: 2
 Explanation: You could form "10", but then you'd have nothing left. Better form "0" and
"1".

 */
public class _474 {
    public static class Solution1 {
        public int findMaxForm(String[] strs, int m, int n) {
            int[][] dp = new int[m + 1][n + 1];
            for (String str : strs) {
                int[] count = count(str);
                for (int i = m; i >= count[0]; i--) {
                    for (int j = n; j >= count[1]; j--) {
                        dp[i][j] = Math.max(dp[i][j], dp[i - count[0]][j - count[1]] +
1);
                    }
                }
            }
            return dp[m][n];
        }

        private int[] count(String str) {
            int[] res = new int[2];
            for (int i = 0; i < str.length(); i++) {
                res[str.charAt(i) - '0']++;
```

```
            }
            return res;
        }
    }

}
```

451. `_475.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 475.
 *
 Winter is coming! Your first job during the contest is to design a standard heater with
fixed warm radius to warm all the houses.

 Now, you are given positions of houses and heaters on a horizontal line, find out
minimum radius of heaters so that all houses could be covered by those heaters.

 So, your input will be the positions of houses and heaters seperately, and your expected
output will be the minimum radius standard of heaters.

 Note:
 Numbers of houses and heaters you are given are non-negative and will not exceed 25000.
 Positions of houses and heaters you are given are non-negative and will not exceed 10^9.
 As long as a house is in the heaters' warm radius range, it can be warmed.
 All the heaters follow your radius standard and the warm radius will the same.

 Example 1:
 Input: [1,2,3],[2]
 Output: 1
 Explanation: The only heater was placed in the position 2, and if we use the radius 1
standard, then all the houses can be warmed.

 Example 2:
 Input: [1,2,3,4],[1,4]
 Output: 1
 Explanation: The two heater was placed in the position 1 and 4. We need to use radius 1
standard, then all the houses can be warmed.
 */
public class _475 {

        public static class Solution1 {
                //credit: https://discuss.leetcode.com/topic/71460/short-and-clean-java-
binary-search-solution
                public int findRadius(int[] houses, int[] heaters) {
                        Arrays.sort(heaters);
                        int radius = Integer.MIN_VALUE;
                        for (int house : houses) {
                                int index = Arrays.binarySearch(heaters, house);
                                if (index < 0) {
                                        index = ~index;
                                }
                                int distance1 = index - 1 >= 0 ? house - heaters[index -
1] : Integer.MAX_VALUE;
                                int distance2 = index < heaters.length ? heaters[index] -
house : Integer.MAX_VALUE;
```

```
                                radius = Math.max(radius, Math.min(distance1,
distance2));
                        }
                        return radius;
                }
        }
}
```

452. `_476.java`

```
package com.fishercoder.solutions;

/**
 * 476. Number Complement
 *
 * Given a positive integer, output its complement number. The complement strategy is to
flip the bits of its binary representation.

 Note:
 The given integer is guaranteed to fit within the range of a 32-bit signed integer.
 You could assume no leading zero bit in the integer’s binary representation.
 Example 1:
 Input: 5
 Output: 2
 Explanation: The binary representation of 5 is 101 (no leading zero bits), and its
complement is 010. So you need to output 2.
 Example 2:
 Input: 1
 Output: 0
 Explanation: The binary representation of 1 is 1 (no leading zero bits), and its
complement is 0. So you need to output 0.
 */
public class _476 {

    public static class Solution1 {
        public int findComplement(int num) {
            return ~num & ((Integer.highestOneBit(num) << 1) - 1);
        }
    }

    public static class Solution2 {
        public int findComplement(int num) {
            String str = Integer.toBinaryString(num);
            StringBuilder sb = new StringBuilder();
            char[] chars = str.toCharArray();
            for (int i = 0; i < chars.length; i++) {
                if (chars[i] == '0') {
                    sb.append("1");
                } else {
                    sb.append("0");
                }
            }
            return Integer.parseInt(sb.toString(), 2);
        }
    }
}
```

453. `_477.java`

```
package com.fishercoder.solutions;

/**
 * 477. Total Hamming Distance
 *
 * The Hamming distance between two integers is the number of positions at which the
corresponding bits are different.

 Now your job is to find the total Hamming distance between all pairs of the given
numbers.

 Example:
 Input: 4, 14, 2
 Output: 6
 Explanation: In binary representation, the 4 is 0100, 14 is 1110, and 2 is 0010 (just
 showing the four bits relevant in this case). So the answer will be:
 HammingDistance(4, 14) + HammingDistance(4, 2) + HammingDistance(14, 2) = 2 + 2 + 2 = 6.

 Note:
 Elements of the given array are in the range of 0 to 10^9
 Length of the array will not exceed 10^4.
 */
public class _477 {
        public static class Solution1 {
                public int totalHammingDistance(int[] nums) {
                        int r = 0;
                        for (int i = 0; i < 32; i++) {
                                int one = 0;
                                int zero = 0;
                                int bit = 1 << i;
                                for (int n : nums) {
                                        if ((n & bit) != 0) {
                                                one++;
                                        } else {
                                                zero++;
                                        }
                                }
                                r += one * zero;
                        }
                        return r;
                }
        }

}
```

454. `_479.java`

```
package com.fishercoder.solutions;

/**
 * 479. Largest Palindrome Product
 *
 * Find the largest palindrome made from the product of two n-digit numbers.
 Since the result could be very large, you should return the largest palindrome mod 1337.

 Example:
 Input: 2
 Output: 987

 Explanation: 99 x 91 = 9009, 9009 % 1337 = 987

 Note:
 The range of n is [1,8].
 */
public class _479 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/74125/java-solution-using-assumed-
max-palindrom
         */
        public int largestPalindrome(int n) {
            // if input is 1 then max is 9
            if (n == 1) {
                return 9;
            }

            // if n = 3 then upperBound = 999 and lowerBound = 99
            int upperBound = (int) Math.pow(10, n) - 1;
            int lowerBound = upperBound / 10;
            long maxNumber = (long) upperBound * (long) upperBound;

            // represents the first half of the maximum assumed palindrom.
            // e.g. if n = 3 then maxNumber = 999 x 999 = 998001 so firstHalf = 998
            int firstHalf = (int) (maxNumber / (long) Math.pow(10, n));

            boolean palindromFound = false;
            long palindrom = 0;

            while (!palindromFound) {
                // creates maximum assumed palindrom
                // e.g. if n = 3 first time the maximum assumed palindrom will be 998 899
                palindrom = createPalindrom(firstHalf);

                // here i and palindrom/i forms the two factor of assumed palindrom
                for (long i = upperBound; upperBound > lowerBound; i--) {
                    // if n= 3 none of the factor of palindrom  can be more than 999 or
less than square root of assumed palindrom
                    if (palindrom / i > maxNumber || i * i < palindrom) {
```

```
                    break;
                }

                // if two factors found, where both of them are n-digits,
                if (palindrom % i == 0) {
                    palindromFound = true;
                    break;
                }
            }

            firstHalf--;
        }
        return (int) (palindrom % 1337);
    }

    private long createPalindrom(long num) {
        String str = num + new StringBuilder().append(num).reverse().toString();
        return Long.parseLong(str);
    }
}
}
```

455. `_480.java`

```
package com.fishercoder.solutions;

import java.util.Collections;
import java.util.PriorityQueue;

/**
 * 480. Sliding Window Median
 *
 * Median is the middle value in an ordered integer list. If the size of the list is
even, there is no middle value. So the median is the mean of the two middle value.

 Examples:
 [2,3,4] , the median is 3

 [2,3], the median is (2 + 3) / 2 = 2.5

 Given an array nums, there is a sliding window of size k which is moving from the very
left of the array to the very right.
 You can only see the k numbers in the window. Each time the sliding window moves right
by one position.
 Your job is to output the median array for each window in the original array.

 For example,
 Given nums = [1,3,-1,-3,5,3,6,7], and k = 3.

 Window position                Median
 ---------------               -----
 [1  3  -1] -3  5  3  6  7       1
 1 [3  -1  -3] 5  3  6  7       -1
 1  3 [-1  -3  5] 3  6  7       -1
 1  3  -1 [-3  5  3] 6  7       3
 1  3  -1  -3 [5  3  6] 7       5
 1  3  -1  -3  5 [3  6  7]      6

 Therefore, return the median sliding window as [1,-1,-1,3,5,6].

 Note:
 You may assume k is always valid, ie: 1 ≤ k ≤ input array's size for non-empty array.
 */
public class _480 {
    public static class Solution1 {

        /**
         * You cannot simply use minus sign '-' to denote the descending order, because
e.g. 3 and -3 might both exist in this array,
         * so we'll have to use the original numbers themselves to store in the heaps.
         */
        private PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        private PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
```

```
    public double[] medianSlidingWindow(int[] nums, int k) {
        int n = nums.length - k + 1;
        if (n <= 0) {
            return new double[0];
        }
        double[] result = new double[n];

        for (int i = 0; i <= nums.length; i++) {
            if (i >= k) {
                result[i - k] = getMedian();
                remove(nums[i - k]);
            }
            if (i < nums.length) {
                add(nums[i]);
            }
        }

        return result;
    }

    private double getMedian() {
        if (maxHeap.isEmpty() && minHeap.isEmpty()) {
            return 0;
        }

        if (maxHeap.size() == minHeap.size()) {
            return ((double) maxHeap.peek() + (double) minHeap.peek()) / 2.0;
        } else {
            return (double) minHeap.peek();
        }
    }

    private void remove(int num) {
        if (num < getMedian()) {
            maxHeap.remove(num);
        } else {
            minHeap.remove(num);
        }
        if (maxHeap.size() > minHeap.size()) {
            minHeap.add(maxHeap.poll());
        }
        if (minHeap.size() - maxHeap.size() > 1) {
            maxHeap.add(minHeap.poll());
        }
    }

    private void add(int num) {
        if (num < getMedian()) {
            maxHeap.add(num);
        } else {
            minHeap.add(num);
        }
        if (maxHeap.size() > minHeap.size()) {
            minHeap.add(maxHeap.poll());
```

```
            }
            if (minHeap.size() - maxHeap.size() > 1) {
                maxHeap.add(minHeap.poll());
            }
        }
    }

}
```

456. `_481.java`

```
package com.fishercoder.solutions;

/**
 * 481. Magical String
 *
 * A magical string S consists of only '1' and '2' and obeys the following rules:

 The string S is magical because concatenating the number of contiguous occurrences of characters '1' and '2' generates the string S itself.

 The first few elements of string S is the following: S = "1221121221221121122……"

 If we group the consecutive '1's and '2's in S, it will be:

 1 22 11 2 1 22 1 22 11 2 11 22 ......

 and the occurrences of '1's or '2's in each group are:

 1 2    2 1 1 2 1 2 2 1 2 2 ......

 You can see that the occurrence sequence above is the S itself.

 Given an integer N as input, return the number of '1's in the first N number in the magical string S.

 Note: N will not exceed 100,000.

 Example 1:
 Input: 6
 Output: 3
 Explanation: The first 6 elements of magical string S is "12211" and it contains three 1's, so return 3.
 */
public class _481 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/74917/simple-java-solution-using-one-array-and-two-pointers
         * Algorithm:
         *
         * 1. Create an int array a and initialize the first 3 elements with 1, 2, 2.
         * 2. Create two pointers head and tail. head points to the number which will be used to generate new numbers.
         * tail points to the next empty position to put the new number. Then keep generating new numbers until tail >= n.
         * 3. Need to create the array 1 element more than n to avoid overflow because the last round head might points to a number 2.
         * 4. A trick to flip number back and forth between 1 and 2: num = num ^ 3
         */
        public int magicalString(int n) {
```

```
            if (n <= 0) {
                return 0;
            }
            if (n <= 3) {
                return 1;
            }

            int[] a = new int[n + 1];
            a[0] = 1;
            a[1] = 2;
            a[2] = 2;

            int head = 2;
            int tail = 3;
            int num = 1;
            int result = 1;

            while (tail < n) {
                for (int i = 0; i < a[head]; i++) {
                    a[tail] = num;
                    if (num == 1 && tail < n) {
                        result++;
                    }
                    tail++;
                }
                num = num ^ 3;
                head++;
            }

            return result;
        }
    }

}
```

457. `_482.java`

```
package com.fishercoder.solutions;

/**
 * 482. License Key Formatting
 *
 * Now you are given a string S, which represents a software license key which we would
like to format.
 * The string S is composed of alphanumerical characters and dashes.
 * The dashes split the alphanumerical characters within the string into groups.
 * (i.e. if there are m dashes, the string is split into m+1 groups).
 * The dashes in the given string are possibly misplaced.
 * We want each group of characters to be of length K
 * (except for possibly the first group, which could be shorter, but still must contain
at least one character).
 * To satisfy this requirement, we will reinsert dashes. Additionally, all the lower case
letters in the string must be converted to upper case.
 * So, you are given a non-empty string S, representing a license key to format, and an
integer K.
 * And you need to return the license key formatted according to the description above.

 Example 1:
 Input: S = "2-4A0r7-4k", K = 4
 Output: "24A0-R74K"
 Explanation: The string S has been split into two parts, each part has 4 characters.

 Example 2:
 Input: S = "2-4A0r7-4k", K = 3
 Output: "24-A0R-74K"
 Explanation: The string S has been split into three parts, each part has 3 characters
except the first part as it could be shorter as said above.

 Note:
 The length of string S will not exceed 12,000, and K is a positive integer.
 String S consists only of alphanumerical characters (a-z and/or A-Z and/or 0-9) and
dashes(-).
 String S is non-empty.
 */
public class _482 {
    public static class Solution1 {
        public String licenseKeyFormatting(String S, int K) {
            StringBuilder stringBuilder = new StringBuilder();
            char[] SChars = S.toCharArray();
            for (int i = S.length() - 1, j = 0; i >= 0; ) {
                if (j < K) {
                    if (SChars[i] != '-') {
                        if (SChars[i] >= 'a' && SChars[i] <= 'z') {
                            stringBuilder.append(Character.toUpperCase(SChars[i]));
                        } else {
                            stringBuilder.append(SChars[i]);
                        }
                        j++;
```

```
                    }
                    i--;
                } else if (j == K) {
                    j = 0;
                    stringBuilder.append('-');
                }
            }
            if (stringBuilder.length() > 1 &&
stringBuilder.substring(stringBuilder.length() - 1).equals("-")) {
                return stringBuilder.reverse().substring(1);
            }
            return stringBuilder.reverse().toString();
        }
    }
}
```

458. `_483.java`

```
package com.fishercoder.solutions;

import java.math.BigInteger;

/**
 * 483. Smallest Good Base
 *
 * For an integer n, we call k>=2 a good base of n, if all digits of n base k are 1.

 Now given a string representing n, you should return the smallest good base of n in
string format.

 Example 1:
 Input: "13"
 Output: "3"
 Explanation: 13 base 3 is 111.

 Example 2:
 Input: "4681"
 Output: "8"
 Explanation: 4681 base 8 is 11111.

 Example 3:
 Input: "1000000000000000000"
 Output: "999999999999999999"
 Explanation: 1000000000000000000 base 999999999999999999 is 11.

 Note:
 The range of n is [3, 10^18].
 The string representing n is always valid and will not have leading zeros.

 */
public class _483 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/82130/java-solution-with-hand-
writing-explain
         */
        public String smallestGoodBase(String n) {
            long nn = Long.parseLong(n);
            long res = 0;
            for (int k = 60; k >= 2; k--) {
                long start = 2;
                long end = nn;
                while (start < end) {
                    long m = start + (end - start) / 2;

                    BigInteger left = BigInteger.valueOf(m);
                    left = left.pow(k).subtract(BigInteger.ONE);
                    BigInteger right =
```

```
BigInteger.valueOf(nn).multiply(BigInteger.valueOf(m).subtract(BigInteger.ONE));
                    int cmr = left.compareTo(right);
                    if (cmr == 0) {
                        res = m;
                        break;
                    } else if (cmr < 0) {
                        start = m + 1;
                    } else {
                        end = m;
                    }
                }

                if (res != 0) {
                    break;
                }
            }
            return "" + res;
        }
    }

}
```

459. `_484.java`

```
package com.fishercoder.solutions;

/**
 * 484. Find Permutation
 *
 * By now, you are given a secret signature consisting of character 'D' and 'I'.
 * 'D' represents a decreasing relationship between two numbers, 'I' represents an
increasing relationship between two numbers.
 * And our secret signature was constructed by a special integer array, which contains
uniquely all the different number from 1 to n (n is the length of the secret signature
plus 1).
 * For example, the secret signature "DI" can be constructed by array [2,1,3] or [3,1,2],
 * but won't be constructed by array [3,2,4] or [2,1,3,4], which are both illegal
constructing special string that can't represent the "DI" secret signature.

 On the other hand, now your job is to find the lexicographically smallest permutation of
[1, 2, ... n] could refer to the given secret signature in the input.

 Example 1:
 Input: "I"
 Output: [1,2]
 Explanation: [1,2] is the only legal initial spectial string can construct secret
signature "I", where the number 1 and 2 construct an increasing relationship.

 Example 2:
 Input: "DI"
 Output: [2,1,3]
 Explanation: Both [2,1,3] and [3,1,2] can construct the secret signature "DI",
 but since we want to find the one with the smallest lexicographical permutation, you
need to output [2,1,3]

 Note:
 The input string will only contain the character 'D' and 'I'.
 The length of input string is a positive integer and will not exceed 10,000
 */
public class _484 {
    public static class Solution1 {

        /**
         * credit:https://discuss.leetcode.com/topic/76221/java-o-n-clean-solution-easy-
to-understand
         *
         * For example, given IDIIDD we start with sorted sequence 1234567
         * Then for each k continuous D starting at index i we need to reverse [i, i+k]
portion of the sorted sequence.
         *
         * e.g.
         * IDIIDD
         *
         * 1234567 // sorted
         * 1324765 // answer
```

```
         */
        public int[] findPermutation(String s) {
            int[] result = new int[s.length() + 1];
            for (int i = 0; i <= s.length(); i++) {
                result[i] = i + 1;
            }
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == 'D') {
                    int left = i;
                    while (i < s.length() && s.charAt(i) == 'D') {
                        i++;
                    }
                    reverse(result, left, i);
                }
            }
            return result;
        }

        private void reverse(int[] result, int left, int i) {
            while (left < i) {
                int temp = result[left];
                result[left] = result[i];
                result[i] = temp;
                left++;
                i--;
            }
        }
    }

}
```

460. `_485.java`

```
package com.fishercoder.solutions;

/**
 * 485. Max Consecutive Ones
 *
 * Given a binary array, find the maximum number of consecutive 1s in this array.
 *
 * Example 1:
 *
 * Input: [1,1,0,1,1,1]
 * Output: 3
 * Explanation: The first two digits or the last three digits are consecutive 1s. The
maximum number of consecutive 1s is 3.
 *
 * Note:
 * The input array will only contain 0 and 1.
 * The length of input array is a positive integer and will not exceed 10,000
 * */
public class _485 {
    public static class Solution1 {
        public int findMaxConsecutiveOnes(int[] nums) {
            int maxLen = 0;
            for (int i = 0; i < nums.length; i++) {
                int currentLen = 0;
                while (i < nums.length && nums[i] == 1) {
                    i++;
                    currentLen++;
                }
                maxLen = Math.max(maxLen, currentLen);
            }
            return maxLen;
        }
    }
}
```

461. `_486.java`

```
package com.fishercoder.solutions;

/**
 * 486. Predict the Winner
 *
 * Given an array of scores that are non-negative integers.
 * Player 1 picks one of the numbers from either end of the array followed by the player
2 and then player 1 and so on.
 * Each time a player picks a number, that number will not be available for the next
player.
 * This continues until all the scores have been chosen.
 * The player with the maximum score wins.
 * Given an array of scores, predict whether player 1 is the winner.
 * You can assume each player plays to maximize his score.

 Example 1:
 Input: [1, 5, 2]
 Output: False
 Explanation: Initially, player 1 can choose between 1 and 2.
 If he chooses 2 (or 1), then player 2 can choose from 1 (or 2) and 5. If player 2
chooses 5, then player 1 will be left with 1 (or 2).
 So, final score of player 1 is 1 + 2 = 3, and player 2 is 5.
 Hence, player 1 will never be the winner and you need to return False.

 Example 2:
 Input: [1, 5, 233, 7]
 Output: True
 Explanation: Player 1 first chooses 1. Then player 2 have to choose between 5 and 7. No
matter which number player 2 choose, player 1 can choose 233.
 Finally, player 1 has more score (234) than player 2 (12), so you need to return True
representing player1 can win.

 Note:
 1 <= length of the array <= 20.
 Any scores in the given array are non-negative integers and will not exceed 10,000,000.
 If the scores of both players are equal, then player 1 is still the winner.
 */
public class _486 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/76312/java-1-line-recursion-
solution
         * Explanation
         * So assuming the sum of the array it SUM, so eventually player1 and player2
will split the SUM between themselves.
         * For player1 to win, he/she has to get more than what player2 gets.
         * If we think from the prospective of one player, then what he/she gains each
time is a plus,
         * while, what the other player gains each time is a minus. Eventually if player1
can have a >0 total, player1 can win.
```

```
         *
         * Helper function simulate this process. In each round:
         * if e==s, there is no choice but have to select nums[s]
         * otherwise, this current player has 2 options:
         * --> nums[s]-helper(nums,s+1,e): this player select the front item, leaving the
other player a choice from s+1 to e
         * --> nums[e]-helper(nums,s,e-1): this player select the tail item, leaving the
other player a choice from s to e-1
         * Then take the max of these two options as this player's selection, return it.
         */
        public boolean predictTheWinner(int[] nums) {
            return helper(nums, 0, nums.length - 1, new Integer[nums.length]
[nums.length]) >= 0;
        }

        private int helper(int[] nums, int start, int end, Integer[][] mem) {
            if (mem[start][end] == null) {
                mem[start][end] = start == end ? nums[end] :
                        Math.max(nums[end] - helper(nums, start, end - 1, mem),
                                nums[start] - helper(nums, start + 1, end, mem));
            }
            return mem[start][end];
        }
    }

}
```

462. `_487.java`

```
package com.fishercoder.solutions;

/**
 * Given a binary array, find the maximum number of consecutive 1s in this array.

 Example 1:
 Input: [1,1,0,1,1,1]
 Output: 3
 Explanation: The first two digits or the last three digits are consecutive 1s.
 The maximum number of consecutive 1s is 3.
 Note:

 The input array will only contain 0 and 1.
 The length of input array is a positive integer and will not exceed 10,000
 */
public class _487 {

    public static int findMaxConsecutiveOnes(int[] nums) {
        int maxOnes = 0;
        for (int i = 0; i < nums.length; i++) {
            int newOnes = 0;
            while (i < nums.length && nums[i] == 1) {
                newOnes++;
                i++;
            }
            maxOnes = Math.max(maxOnes, newOnes);
        }
        return maxOnes;
    }

}
```

463. `_488.java`

```
package com.fishercoder.solutions;

/**
 * 488. Zuma Game
 *
 * Think about Zuma Game. You have a row of balls on the table, colored red(R),
yellow(Y), blue(B), green(G), and white(W).
 * You also have several balls in your hand.

 Each time, you may choose a ball in your hand, and insert it into the row (including the
leftmost place and rightmost place).
 Then, if there is a group of 3 or more balls in the same color touching, remove these
balls. Keep doing this until no more balls can be removed.

 Find the minimal balls you have to insert to remove all the balls on the table. If you
cannot remove all the balls, output -1.

 Examples:

 Input: "WRRBBW", "RB"
 Output: -1
 Explanation: WRRBBW -> WRR[R]BBW -> WBBW -> WBB[B]W -> WW

 Input: "WWRRBBWW", "WRBRW"
 Output: 2
 Explanation: WWRRBBWW -> WWRR[R]BBWW -> WWBBWW -> WWBB[B]WW -> WWWW -> empty

 Input:"G", "GGGGG"
 Output: 2
 Explanation: G -> G[G] -> GG[G] -> empty

 Input: "RBYYBBRRB", "YRBGB"
 Output: 3
 Explanation: RBYYBBRRB -> RBYY[Y]BBRRB -> RBBBRRB -> RRRB -> B -> B[B] -> BB[B] -> empty

 Note:

 You may assume that the initial row of balls on the table won’t have any 3 or more
consecutive balls with the same color.
 The number of balls on the table won't exceed 20, and the string represents these balls
is called "board" in the input.
 The number of balls in your hand won't exceed 5, and the string represents these balls
is called "hand" in the input.
 Both input strings will be non-empty and only contain characters 'R','Y','B','G','W'.
 */
public class _488 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/79820/short-java-solution-beats-98
```

```
         * Two layer of recursion, pretty cool!
         */

        int maxcount = 6;   // the max balls you need will not exceed 6 since "The number
of balls in your hand won't exceed 5"

        public int findMinStep(String board, String hand) {
            int[] handCount = new int[26];
            for (int i = 0; i < hand.length(); ++i) {
                ++handCount[hand.charAt(i) - 'A'];
            }
            int result = dfs(board + "#", handCount);  // append a "#" to avoid special
process while j==board.length, make the code shorter.
            return result == maxcount ? -1 : result;
        }

        private int dfs(String s, int[] handCount) {
            s = removeConsecutive(s);
            if (s.equals("#")) {
                return 0;
            }
            int result = maxcount;
            int need = 0;
            for (int i = 0, j = 0; j < s.length(); ++j) {
                if (s.charAt(j) == s.charAt(i)) {
                    continue;
                }
                need = 3 - (j - i);     //balls need to remove current consecutive balls.
                if (handCount[s.charAt(i) - 'A'] >= need) {
                    handCount[s.charAt(i) - 'A'] -= need;
                    result = Math.min(result, need + dfs(s.substring(0, i) +
s.substring(j), handCount));
                    handCount[s.charAt(i) - 'A'] += need;
                }
                i = j;
            }
            return result;
        }

        //remove consecutive balls longer than 3
        private String removeConsecutive(String board) {
            for (int i = 0, j = 0; j < board.length(); ++j) {
                if (board.charAt(j) == board.charAt(i)) {
                    continue;
                }
                if (j - i >= 3) {
                    return removeConsecutive(board.substring(0, i) + board.substring(j));
                } else {
                    i = j;
                }
            }
            return board;
        }
```

```
    }
}
```

464. `_490.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * There is a ball in a maze with empty spaces and walls. The ball can go through empty
spaces by rolling up, down, left or right, but it won't stop rolling until hitting a
wall. When the ball stops, it could choose the next direction.

 Given the ball's start position, the destination and the maze, determine whether the
ball could stop at the destination.

 The maze is represented by a binary 2D array. 1 means the wall and 0 means the empty
space. You may assume that the borders of the maze are all walls. The start and
destination coordinates are represented by row and column indexes.

 Example 1

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (4, 4)

 Output: true
 Explanation: One possible way is : left -> down -> left -> down -> right -> down ->
right.

 Example 2

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (3, 2)

 Output: false
 Explanation: There is no way for the ball to stop at the destination.

 Note:
```

```
 There is only one ball and one destination in the maze.
 Both the ball and the destination exist on an empty space, and they will not be at the same position initially.
 The given maze does not contain border (like the red rectangle in the example pictures), but you could assume the border of the maze are all walls.
 The maze contains at least 2 empty spaces, and both the width and height of the maze won't exceed 100.
 */
public class _490 {
    /**
     * BFS: the key part of this algorithm is that: this is a ball that won't stop rolling until it hits a wall.
     * This means we'll have to store all the empty spaces where the ball was forced to stop into the queue, these are
     * the only places where the next starting points could be.
     * Then we use BFS to traverse through all four directions of each starting point.
     * <p>
     * Also, another point to note is: it must be a stop point for the ball.
     */
    public boolean hasPath(int[][] maze, int[] start, int[] destination) {
        if (start[0] == destination[0] && destination[0] == destination[1]) {
            return true;
        }
        final int[] directions = new int[]{-1, 0, 1, 0, -1};
        Queue<Point> queue = new LinkedList<>();
        queue.offer(new Point(start[0], start[1]));
        int m = maze.length;
        int n = maze[0].length;
        boolean[][] visited = new boolean[m][n];
        visited[start[0]][start[1]] = true;

        while (!queue.isEmpty()) {
            Point curr = queue.poll();
            int x = curr.x;
            int y = curr.y;//keep the original value
            for (int i = 0; i < directions.length - 1; i++) {
                int xx = x;
                int yy = y;//use temp variables to move
                while (xx >= 0 && yy >= 0 && xx < m && yy < n && maze[xx][yy] == 0) {
                    xx += directions[i];
                    yy += directions[i + 1];
                }
                xx -= directions[i];
                yy -= directions[i + 1];
                if (visited[xx][yy]) {
                    continue;
                }
                visited[xx][yy] = true;
                if (destination[0] == xx && destination[1] == yy) {
                    return true;
                }
                queue.offer(new Point(xx, yy));
            }
        }
```

```
        return false;
    }

    class Point {
        int x;
        int y;

        public Point(int x, int y) {
            this.x = x;
            this.y = y;
        }
    }
}
```

465. `_491.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 491. Increasing Subsequences
 *
 * Given an integer array, your task is to find all the different possible increasing
subsequences of the given array,
 * and the length of an increasing subsequence should be at least 2 .

 Example:
 Input: [4, 6, 7, 7]
 Output: [[4, 6], [4, 7], [4, 6, 7], [4, 6, 7, 7], [6, 7], [6, 7, 7], [7,7], [4,7,7]]

 Note:
 The length of the given array will not exceed 15.
 The range of integer in the given array is [-100,100].
 The given array may contain duplicates, and two equal integers should also be considered
as a special case of increasing sequence.
 */
public class _491 {

    public static class Solution1 {
        public List<List<Integer>> findSubsequences(int[] nums) {
            if (nums == null || nums.length == 1) {
                return new ArrayList<>();
            }
            Set<List<Integer>> answer = new HashSet<>();
            List<Integer> list = new ArrayList<>();
            return new ArrayList<>(backtracking(nums, 0, list, answer));
        }

        private Set<List<Integer>> backtracking(int[] nums, int start, List<Integer>
currList,
            Set<List<Integer>> answer) {
            if (currList.size() >= 2) {
                answer.add(new ArrayList<>(currList));
            }
            for (int i = start; i < nums.length; i++) {
                if (currList.size() == 0 || currList.get(currList.size() - 1) <= nums[i])
{
                    currList.add(nums[i]);
                    backtracking(nums, i + 1, currList, answer);
                    currList.remove(currList.size() - 1);
                }
            }
            return answer;
```

```
        }
    }
}
```

466. `_492.java`

```
package com.fishercoder.solutions;

/**
 * 492. Construct the Rectangle
 *
 * For a web developer, it is very important to know how to design a web page's size.
 * So, given a specific rectangular web page’s area, your job by now is to design a
rectangular web page,
 * whose length L and width W satisfy the following requirements:

 1. The area of the rectangular web page you designed must equal to the given target
area.
 2. The width W should not be larger than the length L, which means L >= W.
 3. The difference between length L and width W should be as small as possible.

 You need to output the length L and the width W of the web page you designed in
sequence.
 Example:
 Input: 4
 Output: [2, 2]
 Explanation: The target area is 4, and all the possible ways to construct it are [1,4],
[2,2], [4,1].
 But according to requirement 2, [1,4] is illegal; according to requirement 3,  [4,1] is
not optimal compared to [2,2]. So the length L is 2, and the width W is 2.

 Note:
 The given area won't exceed 10,000,000 and is a positive integer
 The web page's width and length you designed must be positive integers.

 */
public class _492 {

    public static class Solution1 {
        public int[] constructRectangle(int area) {
            int i = 0;
            int j = area;
            int[] result = new int[2];
            while (i <= j) {
                long product = i * j;
                if (product == area) {
                    result[0] = j--;
                    result[1] = i++;
                } else if (product > area) {
                    j--;
                } else {
                    i++;
                }
            }
            return result;
        }
    }
```

```
}
```

467. `_493.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 493. Reverse Pairs
 *
 * Given an array nums, we call (i, j) an important reverse pair if i < j and nums[i] >
2*nums[j].

 You need to return the number of important reverse pairs in the given array.

 Example1:

 Input: [1,3,2,3,1]
 Output: 2
 Example2:

 Input: [2,4,3,5,1]
 Output: 3

 Note:
 The length of the given array will not exceed 50,000.
 All the numbers in the input array are in the range of 32-bit integer.
 */
public class _493 {

    public static class Solution1 {

        /**
         * reference: https://discuss.leetcode.com/topic/78933/very-short-and-clear-
mergesort-bst-java-solutions
         */
        public int reversePairs(int[] nums) {
            return mergeSort(nums, 0, nums.length - 1);
        }

        private int mergeSort(int[] nums, int start, int end) {
            if (start >= end) {
                return 0;
            }
            int mid = start + (end - start) / 2;
            int cnt = mergeSort(nums, start, mid) + mergeSort(nums, mid + 1, end);
            for (int i = start, j = mid + 1; i <= mid; i++) {
                /**it has to be 2.0 instead of 2, otherwise it's going to stack overflow,
i.e. test3 is going to fail*/
                while (j <= end && nums[i] > nums[j] * 2.0) {
                    j++;
                }
                cnt += j - (mid + 1);
            }
```

```
            Arrays.sort(nums, start, end + 1);
            return cnt;
        }
    }
}
```

468. `_494.java`

```
package com.fishercoder.solutions;

/**
 * 494. Target Sum
 *
 *  You are given a list of non-negative integers, a1, a2, ..., an, and a target, S.
 *  Now you have 2 symbols + and -. For each integer, you should choose one from + and -
as its new symbol.
 *  Find out how many ways to assign symbols to make sum of integers equal to target S.

 Example 1:

 Input: nums is [1, 1, 1, 1, 1], S is 3.
 Output: 5
 Explanation:

 -1+1+1+1+1 = 3
 +1-1+1+1+1 = 3
 +1+1-1+1+1 = 3
 +1+1+1-1+1 = 3
 +1+1+1+1-1 = 3

 There are 5 ways to assign symbols to make the sum of nums be target 3.

 Note:

 The length of the given array is positive and will not exceed 20.
 The sum of elements in the given array will not exceed 1000.
 Your output answer is guaranteed to be fitted in a 32-bit integer.

 */
public class _494 {

    public static class Solution1 {
        public int findTargetSumWays(int[] nums, int S) {
            return find(0, nums, S);
        }

        private int find(int p, int[] nums, int sum) {
            if (p == nums.length) {
                if (sum == 0) {
                    return 1;
                } else {
                    return 0;
                }
            }
            return find(p + 1, nums, sum + nums[p]) + find(p + 1, nums, sum - nums[p]);
        }
    }

}
```

469. `_495.java`

```
package com.fishercoder.solutions;

/**
 * 495. Teemo Attacking
 *
 * In LLP world, there is a hero called Teemo and his attacking can make his enemy Ashe be in poisoned condition.
 * Now, given the Teemo's attacking ascending time series towards Ashe and the poisoning time duration per Teemo's attacking,
 * you need to output the total time that Ashe is in poisoned condition.

 You may assume that Teemo attacks at the very beginning of a specific time point, and makes Ashe be in poisoned condition immediately.

 Example 1:
 Input: [1,4], 2
 Output: 4
 Explanation: At time point 1, Teemo starts attacking Ashe and makes Ashe be poisoned immediately.
 This poisoned status will last 2 seconds until the end of time point 2.
 And at time point 4, Teemo attacks Ashe again, and causes Ashe to be in poisoned status for another 2 seconds.
 So you finally need to output 4.

 Example 2:
 Input: [1,2], 2
 Output: 3
 Explanation: At time point 1, Teemo starts attacking Ashe and makes Ashe be poisoned.
 This poisoned status will last 2 seconds until the end of time point 2.
 However, at the beginning of time point 2, Teemo attacks Ashe again who is already in poisoned status.
 Since the poisoned status won't add up together, though the second poisoning attack will still work at time point 2, it will stop at the end of time point 3.
 So you finally need to output 3.

 Note:
 You may assume the length of given time series array won't exceed 10000.
 You may assume the numbers in the Teemo's attacking time series and his poisoning time duration per attacking are non-negative integers, which won't exceed 10,000,000.
 */
public class _495 {

    public static class Solution1 {
        public int findPoisonedDuration(int[] timeSeries, int duration) {
            if (timeSeries == null || timeSeries.length == 0 || duration == 0) {
                return 0;
            }
            int totalDuration = 0;
            for (int i = 0; i < timeSeries.length - 1; i++) {
                if ((timeSeries[i + 1] - timeSeries[i]) >= duration) {
                    totalDuration += duration;
```

```
                } else {
                    totalDuration += (timeSeries[i + 1] - timeSeries[i]);
                }
            }
            totalDuration += duration;//plus the last one duration
            return totalDuration;
        }
    }

}
```

470. `_496.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.Stack;

/**
 * 496. Next Greater Element I
 *
 * You are given two arrays (without duplicates) nums1 and nums2 where nums1’s elements
are subset of nums2.
 * Find all the next greater numbers for nums1's elements in the corresponding places of
nums2.
 * The Next Greater Number of a number x in nums1 is the first greater number to its
right in nums2. If it does not exist, output -1 for this number.

 Example 1:
 Input: nums1 = [4,1,2], nums2 = [1,3,4,2].
 Output: [-1,3,-1]
 Explanation:
 For number 4 in the first array, you cannot find the next greater number for it in the
second array, so output -1.
 For number 1 in the first array, the next greater number for it in the second array is
3.
 For number 2 in the first array, there is no next greater number for it in the second
array, so output -1.

 Example 2:
 Input: nums1 = [2,4], nums2 = [1,2,3,4].
 Output: [3,-1]
 Explanation:
 For number 2 in the first array, the next greater number for it in the second array is
3.
 For number 4 in the first array, there is no next greater number for it in the second
array, so output -1.

 Note:
 All elements in nums1 and nums2 are unique.
 The length of both nums1 and nums2 would not exceed 1000.
 */
public class _496 {

    public static class Solution1 {
        public int[] nextGreaterElement(int[] findNums, int[] nums) {
            Stack<Integer> stack = new Stack();
            Map<Integer, Integer> map = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                while (!stack.isEmpty() && nums[i] > stack.peek()) {
                    map.put(stack.pop(), nums[i]);
                }
                stack.push(nums[i]);
```

```
            }

            while (!stack.isEmpty()) {
                map.put(stack.pop(), -1);
            }

            int[] result = new int[findNums.length];
            for (int i = 0; i < findNums.length; i++) {
                result[i] = map.get(findNums[i]);
            }
            return result;
        }
    }

}
```

471. `_498.java`

```
package com.fishercoder.solutions;

/**
 * 498. Diagonal Traverse
 *
 *  Given a matrix of m x N elements (m rows, N columns), return all elements of the
matrix in diagonal order
 *  as shown in the below image.

 Example:

 Input:
 [
 [ 1, 2, 3 ],
 [ 4, 5, 6 ],
 [ 7, 8, 9 ]
 ]
 Output:  [1,2,4,7,5,3,6,8,9]

 Note:

 The total number of elements of the given matrix will not exceed 10,000.

 */
public class _498 {

public static class Solutoin1 {
    /**
     * Reference: https://discuss.leetcode.com/topic/77865/concise-java-solution/2
     * Just keep walking the matrix, when hitting the four borders (top, bottom, left or
right),
     * just directions and keep walking.
     */
    public int[] findDiagonalOrder(int[][] matrix) {

        if (matrix == null || matrix.length == 0) {
            return new int[0];
        }
        int m = matrix.length;
        int n = matrix[0].length;
        int[] result = new int[m * n];
        int d = 1;
        int i = 0;
        int j = 0;
        for (int k = 0; k < m * n; ) {
            result[k++] = matrix[i][j];
            i -= d;
            j += d;

            if (i >= m) {
                i = m - 1;
```

```
                j += 2;
                d = -d;
            }
            if (j >= n) {
                j = n - 1;
                i += 2;
                d = -d;
            }
            if (i < 0) {
                i = 0;
                d = -d;
            }
            if (j < 0) {
                j = 0;
                d = -d;
            }
        }
        return result;
    }
}

}
```

472. `_499.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 499. The Maze III
 *
 * There is a ball in a maze with empty spaces and walls.
 * The ball can go through empty spaces by rolling up (u), down (d), left (l) or right
(r),
 * but it won't stop rolling until hitting a wall.
 *
 * When the ball stops, it could choose the next direction.
 * There is also a hole in this maze. The ball will drop into the hole if it rolls on to
the hole.
 *
 * Given the ball position, the hole position and the maze,
 * find out how the ball could drop into the hole by moving the shortest distance.
 *
 * The distance is defined by the number of empty spaces traveled by the ball from the
start
 * position (excluded) to the hole (included).
 *
 * Output the moving directions by using 'u', 'd', 'l' and 'r'.
 * Since there could be several different shortest ways, you should output the
lexicographically smallest way.
 *
 * If the ball cannot reach the hole, output "impossible".
 * The maze is represented by a binary 2D array.
 * 1 means the wall and 0 means the empty space.
 * You may assume that the borders of the maze are all walls.
 * The ball and the hole coordinates are represented by row and column indexes.

 Example 1

 Input 1: a maze represented by a 2D array

 0 0 0 0 0
 1 1 0 0 1
 0 0 0 0 0
 0 1 0 0 1
 0 1 0 0 0

 Input 2: ball coordinate (rowBall, colBall) = (4, 3)
 Input 3: hole coordinate (rowHole, colHole) = (0, 1)

 Output: "lul"
 Explanation: There are two shortest ways for the ball to drop into the hole.
 The first way is left -> up -> left, represented by "lul".
 The second way is up -> left, represented by 'ul'.
```

```
 Both ways have shortest distance 6, but the first way is lexicographically smaller
because 'l' < 'u'. So the output is "lul".

 Example 2

 Input 1: a maze represented by a 2D array

 0 0 0 0 0
 1 1 0 0 1
 0 0 0 0 0
 0 1 0 0 1
 0 1 0 0 0

 Input 2: ball coordinate (rowBall, colBall) = (4, 3)
 Input 3: hole coordinate (rowHole, colHole) = (3, 0)
 Output: "impossible"
 Explanation: The ball cannot reach the hole.

 Note:
 There is only one ball and one hole in the maze.
 Both the ball and hole exist on an empty space, and they will not be at the same
position initially.
 The given maze does not contain border (like the red rectangle in the example pictures),
but you could assume the border of the maze are all walls.
 The maze contains at least 2 empty spaces, and the width and the height of the maze
won't exceed 30.*/
public class _499 {
    public static class Solutoin1 {
        /**credit: https://discuss.leetcode.com/topic/77474/similar-to-the-maze-ii-easy-
understanding-java-bfs-solution*/

        public String findShortestWay(int[][] maze, int[] ball, int[] hole) {

            final int[] directions = new int[]{-1, 0, 1, 0, -1};
            Queue<Point> heap = new PriorityQueue<>();
            heap.offer(new Point(ball[0], ball[1], 0, ""));
            int m = maze.length;
            int n = maze[0].length;
            Point[][] points = new Point[m][n];
            for (int i = 0; i < m * n; i++) {
                points[i / n][i % n] = new Point(i / n, i % n);//initialize the length
array
            }
            String[] ds = new String[]{"u", "r", "d", "l"};
            while (!heap.isEmpty()) {
                Point curr = heap.poll();
                if (points[curr.x][curr.y].compareTo(curr) <= 0) {
                    continue;//if we have already found a shorter route
                }
                points[curr.x][curr.y] = curr;
                for (int i = 0; i < directions.length - 1; i++) {
                    int x = curr.x;
                    int y = curr.y;
                    int distance = curr.distance;//use temp variables to move
```

```
                    //we need below while loop to find only "stop" points that could be
put into the queue
                    while (x >= 0 && y >= 0 && x < m && y < n && maze[x][y] == 0 && (x !=
hole[0] || y != hole[1])) {
                        x += directions[i];
                        y += directions[i + 1];
                        distance++;
                    }
                    if (x != hole[0] || y != hole[1]) {
                        x -= directions[i];
                        y -= directions[i + 1];
                        distance--;
                    }
                    heap.offer(new Point(x, y, distance, curr.path + ds[i]));
                }
            }
            return points[hole[0]][hole[1]].distance == Integer.MAX_VALUE ? "impossible"
: points[hole[0]][hole[1]].path;
        }

        class Point implements Comparable<Point> {
            int x;
            int y;
            int distance;
            String path;

            public Point(int x, int y) {
                this.x = x;
                this.y = y;
                this.distance = Integer.MAX_VALUE;
                this.path = "";
            }

            public Point(int x, int y, int distance, String path) {
                this.x = x;
                this.y = y;
                this.distance = distance;
                this.path = path;
            }

            @Override
            public int compareTo(Point o) {
                return (this.distance == o.distance) ? this.path.compareTo(o.path) :
this.distance - o.distance;
            }
        }
    }
}
```

473. `_500.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 500. Keyboard Row
 *
 * Given a List of words, return the words that can be typed using letters of alphabet on
only one row's of American keyboard like the image below.
 *
 * American keyboard
 *
 * Example 1:
 * Input: ["Hello", "Alaska", "Dad", "Peace"]
 * Output: ["Alaska", "Dad"]
 *
 * Note:
 * You may use one character in the keyboard more than once.
 * You may assume the input string will only contain letters of alphabet.
 */
public class _500 {

    public String[] findWords(String[] words) {
        final Set<Character> row1 = new HashSet<>(Arrays.asList('q', 'w', 'e', 'r', 't',
'y', 'u', 'i', 'o', 'p'));
        final Set<Character> row2 = new HashSet<>(Arrays.asList('a', 's', 'd', 'f', 'g',
'h', 'j', 'k', 'l'));
        final Set<Character> row3 = new HashSet<>(Arrays.asList('z', 'x', 'c', 'v', 'b',
'n', 'm'));
        final List<Set<Character>> setList = Arrays.asList(row1, row2, row3);
        List<String> wordList = new ArrayList<>();
        for (String word : words) {
            String lowerCaseWord = word.toLowerCase();
            boolean can = true;
            for (Set<Character> row : setList) {
                for (char c : lowerCaseWord.toCharArray()) {
                    if (!row.contains(c)) {
                        can = false;
                        break;
                    }
                }
                if (can) {
                    wordList.add(word);
                    break;
                }
                can = true;
            }
```

```
        }
        String[] result = new String[wordList.size()];
        for (int i = 0; i < wordList.size(); i++) {
            result[i] = wordList.get(i);
        }
        return result;
    }

}
```

474. `_501.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;

/**
 * 501. Find Mode in Binary Search Tree
 * Given a binary search tree with duplicates. You have to find all the mode(s) in given
binary tree.

 For example:
 Given binary tree [1,null,2,2],
  1
   \
   2
  /
 2
 return [2].

 Note: If a tree has more than one mode, you can return them in any order.

 */
public class _501 {

    public static class Solution1 {
        public int[] findMode(TreeNode root) {
            int[] result = new int[]{};
            Map<Integer, Integer> map = new HashMap();
            if (root == null) {
                return result;
            }
            List<Integer> list = bfs(root, map);
            result = new int[list.size()];
            for (int i = 0; i < list.size(); i++) {
                result[i] = list.get(i);
            }
            return result;
        }

        private List<Integer> bfs(TreeNode root, Map<Integer, Integer> map) {
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
```

```
                TreeNode treeNode = queue.poll();
                if (treeNode.left != null) {
                    queue.offer(treeNode.left);
                }
                if (treeNode.right != null) {
                    queue.offer(treeNode.right);
                }
                map.put(treeNode.val, map.getOrDefault(treeNode.val, 0) + 1);
            }
        }

        int highestFrequency = 0;
        List<Integer> list = new ArrayList<>();
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            if (entry.getValue() > highestFrequency) {
                highestFrequency = entry.getValue();
                list.clear();
                list.add(entry.getKey());
            } else if (entry.getValue() == highestFrequency) {
                list.add(entry.getKey());
            }
        }

        return list;
    }
}

public static class Solution2 {
    public int[] findMode(TreeNode root) {
        Map<Integer, Integer> map = new HashMap<>();
        dfs(root, map);
        int modeCount = 0;
        for (int key : map.keySet()) {
            modeCount = Math.max(modeCount, map.get(key));
        }
        List<Integer> mode = new ArrayList<>();
        for (int key : map.keySet()) {
            if (map.get(key) == modeCount) {
                mode.add(key);
            }
        }
        int[] result = new int[mode.size()];
        for (int i = 0; i < mode.size(); i++) {
            result[i] = mode.get(i);
        }
        return result;
    }

    private void dfs(TreeNode root, Map<Integer, Integer> map) {
        if (root == null) {
            return;
        }
        dfs(root.left, map);
        map.put(root.val, map.getOrDefault(root.val, 0) + 1);
```

```
            dfs(root.right, map);
        }
    }

}
```

475. `_502.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 502. IPO
 *
 * Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most k distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most k distinct projects.

 You are given several projects. For each project i, it has a pure profit Pi and a minimum capital of Ci is needed to start the corresponding project. Initially, you have W capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital.

 To sum up, pick a list of at most k distinct projects from given projects to maximize your final capital, and output your final maximized capital.

 Example 1:
 Input: k=2, W=0, Profits=[1,2,3], Capital=[0,1,1].

 Output: 4

 Explanation: Since your initial capital is 0, you can only start the project indexed 0.
 After finishing it you will obtain profit 1 and your capital becomes 1.
 With capital 1, you can either start the project indexed 1 or the project indexed 2.
 Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum capital.
 Therefore, output the final maximized capital, which is 0 + 1 + 3 = 4.
 Note:
 You may assume all numbers in the input are non-negative integers.
 The length of Profits array and Capital array will not exceed 50,000.
 The answer is guaranteed to fit in a 32-bit signed integer.

 */
public class _502 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/77768/very-simple-greedy-java-solution-using-two-priorityqueues
         */
        public int findMaximizedCapital(int k, int W, int[] Profits, int[] Capital) {
            PriorityQueue<int[]> capitalHeap = new PriorityQueue<>((a, b) -> a[0] - b[0]);
            PriorityQueue<int[]> profitHeap = new PriorityQueue<>((a, b) -> b[1] - a[1]);
            for (int i = 0; i < Profits.length; i++) {
                capitalHeap.add(new int[]{Capital[i], Profits[i]});
```

```
            }
            while (k-- > 0) {
                while (!capitalHeap.isEmpty() && capitalHeap.peek()[0] <= W) {
                    profitHeap.add(capitalHeap.poll());
                }
                if (profitHeap.isEmpty()) {
                    break;
                }
                W += profitHeap.poll()[1];
            }
            return W;
        }
    }

}
```

476. `_503.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 503. Next Greater Element II
 *
 * Given a circular array (the next element of the last element is the first element of
the array),
 * print the Next Greater Number for every element.
 * The Next Greater Number of a number x is the first greater number to its traversing-
order next in the array,
 * which means you could search circularly to find its next greater number. If it doesn't
exist, output -1 for this number.

 Example 1:
 Input: [1,2,1]
 Output: [2,-1,2]
 Explanation: The first 1's next greater number is 2;
 The number 2 can't find next greater number;
 The second 1's next greater number needs to search circularly, which is also 2.
 Note: The length of given array won't exceed 10000.
 */
public class _503 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/77881/typical-ways-to-solve-
circular-array-problems-java-solution
         * Note: we store INDEX into the stack, reversely, the larger index put at the
bottom of the stack, the smaller index at the top
         */
        public int[] nextGreaterElements(int[] nums) {
            if (nums == null || nums.length == 0) {
                return nums;
            }
            int len = nums.length;
            Stack<Integer> stack = new Stack<>();
            for (int i = len - 1; i >= 0; i--) {
                stack.push(i);
                //push all indexes into the stack reversely
            }
            int[] result = new int[len];
            for (int i = len - 1; i >= 0; i--) {
                result[i] = -1;
                //initialize it to be -1 in case we cannot find its next greater element
in the array
                while (!stack.isEmpty() && (nums[stack.peek()] <= nums[i])) {
                    stack.pop();
                }
                if (!stack.isEmpty()) {
```

```
                    result[i] = nums[stack.peek()];
                }
                stack.push(i);
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * credit: https://leetcode.com/articles/next-greater-element-ii/
         */
        public int[] nextGreaterElements(int[] nums) {
            int[] result = new int[nums.length];
            Stack<Integer> stack = new Stack<>();
            for (int i = nums.length * 2 - 1; i >= 0; i--) {
                while (!stack.isEmpty() && nums[stack.peek()] <= nums[i % nums.length]) {
                    stack.pop();
                }
                result[i % nums.length] = stack.isEmpty() ? -1 : nums[stack.peek()];
                stack.push(i % nums.length);
            }
            return result;
        }
    }

}
```

477. `_504.java`

```
package com.fishercoder.solutions;

/**
 * 504. Base 7
 *
 * Given an integer, return its base 7 string representation.
 Example 1:
 Input: 100
 Output: "202"

 Example 2:
 Input: -7
 Output: "-10"

 Note: The input will be in range of [-1e7, 1e7].
 */
public class _504 {

    public static class Solution1 {
        public String convertToBase7(int num) {
            return String.valueOf(Integer.toString(num, 7));
        }
    }
}
```

478. `_505.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 505. The Maze II
 *
 * There is a ball in a maze with empty spaces and walls. The ball can go through empty
spaces by rolling up, down, left or right,
 * but it won't stop rolling until hitting a wall. When the ball stops, it could choose
the next direction.
 * Given the ball's start position, the destination and the maze, find the shortest
distance for the ball to stop at the destination. The distance is defined by the number
of empty spaces traveled by the ball from the start position (excluded) to the
destination (included). If the ball cannot stop at the destination, return -1.
 * The maze is represented by a binary 2D array. 1 means the wall and 0 means the empty
space. You may assume that the borders of the maze are all walls. The start and
destination coordinates are represented by row and column indexes.

 Example 1

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (4, 4)

 Output: 12
 Explanation: One shortest way is : left -> down -> left -> down -> right -> down ->
right.
 The total distance is 1 + 1 + 3 + 1 + 2 + 2 + 2 = 12.

 Example 2

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (3, 2)
```

```
 Output: -1
 Explanation: There is no way for the ball to stop at the destination.

 Note:
 There is only one ball and one destination in the maze.
 Both the ball and the destination exist on an empty space, and they will not be at the same position initially.
 The given maze does not contain border (like the red rectangle in the example pictures), but you could assume the border of the maze are all walls.
 The maze contains at least 2 empty spaces, and both the width and height of the maze won't exceed 100.
 */
public class _505 {

    public static class Solution1 {
        /**
         * The difference between II and I of this problem:
         * the extra array is not boolean type any more, but int type, and it's recording the length of each point to start point.
         */
        public int shortestDistance(int[][] maze, int[] start, int[] destination) {

            final int[] directions = new int[]{-1, 0, 1, 0, -1};
            Queue<Point> queue = new LinkedList<>();
            queue.offer(new Point(start[0], start[1], 0));
            int m = maze.length;
            int n = maze[0].length;
            int[][] length = new int[m][n];
            for (int i = 0; i < m * n; i++) {
                length[i / n][i % n] = Integer.MAX_VALUE;//initialize the length array
            }

            while (!queue.isEmpty()) {
                Point curr = queue.poll();
                if (length[curr.x][curr.y] <= curr.distance) {
                    continue;
                }
                length[curr.x][curr.y] = curr.distance;
                for (int i = 0; i < directions.length - 1; i++) {
                    int x = curr.x;
                    int y = curr.y;
                    int distance = curr.distance;//use temp variables to move
                    //we need below while loop to find only "stop" points that could be put into the queue
                    while (x >= 0 && y >= 0 && x < m && y < n && maze[x][y] == 0) {
                        x += directions[i];
                        y += directions[i + 1];
                        distance++;
                    }
                    x -= directions[i];
                    y -= directions[i + 1];
                    distance--;
                    queue.offer(new Point(x, y, distance));
                }
```

```
            }
            return length[destination[0]][destination[1]] == Integer.MAX_VALUE ? -1 :
length[destination[0]][destination[1]];

        }

        class Point {
            int x;
            int y;
            int distance;

            public Point(int x, int y, int distance) {
                this.x = x;
                this.y = y;
                this.distance = distance;
            }
        }
    }
}
```

479. `_506.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 506. Relative Ranks
 *
 * Given scores of N athletes, find their relative ranks and the people with the top three highest scores,
 * who will be awarded medals: "Gold Medal", "Silver Medal" and "Bronze Medal".

 Example 1:
 Input: [5, 4, 3, 2, 1]
 Output: ["Gold Medal", "Silver Medal", "Bronze Medal", "4", "5"]
 Explanation: The first three athletes got the top three highest scores, so they got "Gold Medal", "Silver Medal" and "Bronze Medal".
 For the left two athletes, you just need to output their relative ranks according to their scores.

 Note:
 N is a positive integer and won't exceed 10,000.
 All the scores of athletes are guaranteed to be unique.
 */
public class _506 {

    public static class Solution1 {
        public String[] findRelativeRanks(int[] nums) {
            int[] tmp = new int[nums.length];
            for (int i = 0; i < nums.length; i++) {
                tmp[i] = nums[i];
            }
            Arrays.sort(tmp);
            Map<Integer, String> rankMap = new HashMap();
            int len = nums.length;
            for (int i = len - 1; i >= 0; i--) {
                if (i == len - 1) {
                    rankMap.put(tmp[i], "Gold Medal");
                } else if (i == len - 2) {
                    rankMap.put(tmp[i], "Silver Medal");
                } else if (i == len - 3) {
                    rankMap.put(tmp[i], "Bronze Medal");
                } else {
                    rankMap.put(tmp[i], String.valueOf(len - i));
                }
            }
            String[] result = new String[len];
            for (int i = 0; i < len; i++) {
                result[i] = rankMap.get(nums[i]);
            }
```

```
            return result;
        }
    }

}
```

480. `_507.java`

```
package com.fishercoder.solutions;

/**
 * 507. Perfect Number
 *
 * We define the Perfect Number is a positive integer that is equal to the sum of all its
positive divisors except itself.
 * Now, given an integer n, write a function that returns true when it is a perfect
number and false when it is not.

 Example:
 Input: 28
 Output: True
 Explanation: 28 = 1 + 2 + 4 + 7 + 14

 Note: The input number n will not exceed 100,000,000. (1e8)
 */
public class _507 {

    public static class Solution1 {
        public boolean checkPerfectNumber(int num) {
            if (num == 1) {
                return false;
            }
            int sum = 0;
            for (int i = 2; i <= Math.sqrt(num); i++) {
                if (num % i == 0) {
                    sum += i;
                    if (i != num / i) {
                        sum += num / i;
                    }
                }
            }
            sum++;
            return sum == num;
        }
    }
}
```

481. `_508.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;

/**
 * 508. Most Frequent Subtree Sum
 *
 * Given the root of a tree, you are asked to find the most frequent subtree sum.
 * The subtree sum of a node is defined as the sum of all the node values formed by the
subtree rooted at that node (including the node itself).
 * So what is the most frequent subtree sum value? If there is a tie, return all the
values with the highest frequency in any order.

 Examples 1
 Input:
    5
  /  \
 2   -3
 return [2, -3, 4], since all the values happen only once, return all of them in any
order.

 Examples 2
 Input:

    5
  /  \
 2   -5
 return [2], since 2 happens twice, however -5 only occur once.

 Note: You may assume the sum of values in any subtree is in the range of 32-bit signed
integer.
 */
public class _508 {

    public static class Solution1 {
        //my purely original but verbose solution
        public int[] findFrequentTreeSum(TreeNode root) {
            if (root == null) {
                return new int[]{};
            }

            Map<TreeNode, Integer> map = new HashMap();
            postOrder(root, map);
```

```
            Map<Integer, Integer> frequencyMap = new HashMap<>();
            for (Map.Entry entry : map.entrySet()) {
                frequencyMap.put((Integer) entry.getValue(),
frequencyMap.getOrDefault(entry.getValue(), 0) + 1);
            }

            List<Map.Entry<Integer, Integer>> list = new LinkedList<>
(frequencyMap.entrySet());
            Collections.sort(list, (o1, o2) -> (o2.getValue()).compareTo(o1.getValue()));

            int mostFrequency = list.get(0).getValue();
            List<Integer> topFrequencyList = new ArrayList<>();
            topFrequencyList.add(list.get(0).getKey());
            int i = 1;
            while (i < list.size() && list.get(i).getValue() == mostFrequency) {
                topFrequencyList.add(list.get(i).getKey());
                i++;
            }

            int[] result = new int[topFrequencyList.size()];
            for (int j = 0; j < topFrequencyList.size(); j++) {
                result[j] = topFrequencyList.get(j);
            }

            return result;
        }

        private int postOrder(TreeNode root, Map<TreeNode, Integer> map) {
            int left = 0;
            int right = 0;
            if (root.left != null) {
                left = postOrder(root.left, map);
            }
            if (root.right != null) {
                right = postOrder(root.right, map);
            }
            if (root.left == null && root.right == null) {
                map.put(root, root.val);
                return root.val;
            }
            int sum = left + right + root.val;
            map.put(root, sum);
            return sum;
        }
    }

    public static class Solution2 {
        //my 2nd purely original but verbose solution
        public int[] findFrequentTreeSum(TreeNode root) {
            Map<Integer, Integer> map = new HashMap<>();
            dfs(root, map);
            List<Map.Entry<Integer, Integer>> entryList = new ArrayList<>
(map.entrySet());
            Collections.sort(entryList, (a, b) -> b.getValue() - a.getValue());
```

```
            List<Integer> list = new ArrayList<>();
            for (int i = 0; i < entryList.size(); i++) {
                if (list.size() == 0) {
                    list.add(entryList.get(i).getKey());
                } else {
                    if (map.get(list.get(0)) == entryList.get(i).getValue()) {
                        list.add(entryList.get(i).getKey());
                    } else {
                        break;
                    }
                }
            }
            int[] result = new int[list.size()];
            for (int i = 0; i < list.size(); i++) {
                result[i] = list.get(i);
            }
            return result;
        }

        private int dfs(TreeNode root, Map<Integer, Integer> map) {
            if (root == null) {
                return 0;
            }
            if (root.left == null && root.right == null) {
                map.put(root.val, map.getOrDefault(root.val, 0) + 1);
                return root.val;
            }
            int leftVal = 0;
            if (root.left != null) {
                leftVal = dfs(root.left, map);
            }
            int rightVal = 0;
            if (root.right != null) {
                rightVal = dfs(root.right, map);
            }
            int val = leftVal + rightVal + root.val;
            map.put(val, map.getOrDefault(val, 0) + 1);
            return val;
        }
    }

    //a more concise and space-efficient solution:
https://discuss.leetcode.com/topic/77775/verbose-java-solution-postorder-traverse-
hashmap-18ms
    //the key difference between the above post and my original solution is that it's
using Frequency as the key of the HashMap
}
```

482. `_509.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 509. Fibonacci Number
 *
 * The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci
sequence,
 * such that each number is the sum of the two preceding ones, starting from 0 and 1.
That is,
 *
 * F(0) = 0,   F(1) = 1
 * F(N) = F(N - 1) + F(N - 2), for N > 1.
 * Given N, calculate F(N).
 *
 * Example 1:
 * Input: 2
 * Output: 1
 * Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.
 *
 * Example 2:
 * Input: 3
 * Output: 2
 * Explanation: F(3) = F(2) + F(1) = 1 + 1 = 2.
 *
 * Example 3:
 * Input: 4
 * Output: 3
 * Explanation: F(4) = F(3) + F(2) = 2 + 1 = 3.
 *
 * Note:
 * 0 ≤ N ≤ 30.
 */
public class _509 {
  public static class Solution1 {
    /**
     * Time: O(n)
     * Space: O(n)
     */
    public int fib(int N) {
      List<Integer> list = new ArrayList<>();
      list.add(0);
      list.add(1);
      for (int i = 2; i <= N; i++) {
        list.add(list.get(i - 1) + list.get(i - 2));
      }
      return list.get(N);
    }
  }
```

```
  public static class Solution2 {
    /**
     * Time: O(n)
     * Space: O(n)
     */
    public int fib(int N) {
      if (N < 2) {
        return N;
      }
      int[] fibNums = new int[N + 1];
      fibNums[0] = 0;
      fibNums[1] = 1;
      for (int i = 2; i <= N; i++) {
        fibNums[i] = fibNums[i - 1] + fibNums[i - 2];
      }
      return fibNums[N];
    }
  }

  public static class Solution3 {
    /**
     * Time: O(n)
     * Space: O(1)
     */
    public int fib(int N) {
      if (N < 2) {
        return N;
      }
      int a = 0;
      int b = 1;
      while (N-- > 1) {
        int sum = a + b;
        a = b;
        b = sum;
      }
      return b;
    }
  }
}
```

483. `_513.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 513. Find Bottom Left Tree Value
 *
 * Given a binary tree, find the leftmost value in the last row of the tree.

 Example 1:
 Input:

   2
  / \
 1   3

 Output:
 1
 Example 2:
 Input:


      1
     / \
    2   3
   /   / \
  4   5   6
     /
    7

 Output:
 7
 Note: You may assume the tree (i.e., the given root node) is not NULL.
 */
public class _513 {

    public static class Solution1 {
        public int findBottomLeftValue(TreeNode root) {
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            TreeNode leftMost = root;
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (i == 0) {
                        leftMost = curr;
                    }
```

```
                    if (curr.left != null) {
                        queue.offer(curr.left);
                    }
                    if (curr.right != null) {
                        queue.offer(curr.right);
                    }
                }
            }
            return leftMost.val;
        }
    }
}
```

484. `_514.java`

```
package com.fishercoder.solutions;

/**
 * 514. Freedom Trail
 *
 * In the video game Fallout 4, the quest "Road to Freedom" requires players to reach a metal dial called
 * the "Freedom Trail Ring", and use the dial to spell a specific keyword in order to open the door.

 Given a string ring, which represents the code engraved on the outer ring and another string key,
 which represents the keyword needs to be spelled.
 You need to find the minimum number of steps in order to spell all the characters in the keyword.

 Initially, the first character of the ring is aligned at 12:00 direction.
 You need to spell all the characters in the string key one by one by
 rotating the ring clockwise or anticlockwise to make each character of the string key
 aligned at 12:00 direction and then by pressing the center button.

 At the stage of rotating the ring to spell the key character key[i]:
 You can rotate the ring clockwise or anticlockwise one place, which counts as 1 step.
 The final purpose of the rotation is to align one of the string ring's characters at the 12:00 direction,
 where this character must equal to the character key[i].
 If the character key[i] has been aligned at the 12:00 direction,
 you need to press the center button to spell, which also counts as 1 step.
 After the pressing, you could begin to spell the next character in the key (next stage), otherwise, you've finished all the spelling.

 Example:
 Input: ring = "godding", key = "gd"
 Output: 4
 Explanation:
 For the first key character 'g', since it is already in place, we just need 1 step to spell this character.
 For the second key character 'd', we need to rotate the ring "godding" anticlockwise by two steps to make it become "ddinggo".
 Also, we need 1 more step for spelling.
 So the final output is 4.

 Note:
 Length of both ring and key will be in range 1 to 100.
 There are only lowercase letters in both strings and might be some duplcate characters in both strings.
 It's guaranteed that string key could always be spelled by rotating the string ring.
 */
public class _514 {

    public static class Solution1 {
```

```
        public int findRotateSteps(String ring, String key) {
            int n = ring.length();
            int m = key.length();
            int[][] dp = new int[m + 1][n];
            for (int i = m - 1; i >= 0; i--) {
                for (int j = 0; j < n; j++) {
                    dp[i][j] = Integer.MAX_VALUE;
                    for (int k = 0; k < n; k++) {
                        if (ring.charAt(k) == key.charAt(i)) {
                            int diff = Math.abs(j - k);
                            int step = Math.min(diff, n - diff);
                            dp[i][j] = Math.min(dp[i][j], step + dp[i + 1][k]);
                        }
                    }
                }
            }
            return dp[0][0] + m;
        }
    }
}
```

485. `_515.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 515. Find Largest Value in Each Tree Row
 *
 * You need to find the largest value in each row of a binary tree.

 Example:
 Input:

      1
     / \
    3   2
   / \   \
  5  3   9

 Output: [1, 3, 9]
 */
public class _515 {

    public static class Solution1 {
        public List<Integer> largestValues(TreeNode root) {
            List<Integer> list = new ArrayList<>();
            Queue<TreeNode> queue = new LinkedList<>();
            if (root != null) {
                queue.offer(root);
                while (!queue.isEmpty()) {
                    int max = Integer.MIN_VALUE;
                    int size = queue.size();
                    for (int i = 0; i < size; i++) {
                        TreeNode curr = queue.poll();
                        max = Math.max(max, curr.val);
                        if (curr.left != null) {
                            queue.offer(curr.left);
                        }
                        if (curr.right != null) {
                            queue.offer(curr.right);
                        }
                    }
                    list.add(max);
                }
            }
            return list;
        }
```

```
    }

    public static class Solution2 {
        public List<Integer> largestValues(TreeNode root) {
            List<Integer> res = new ArrayList<>();
            if (root == null) {
                return res;
            }
            dfs(root, res, 0);
            return res;

        }

        public void dfs(TreeNode root, List<Integer> res, int level) {
            if (root == null) {
                return;
            }
            if (level == res.size()) {
                res.add(root.val);
            }
            res.set(level, Math.max(res.get(level), root.val));
            dfs(root.left, res, level + 1);
            dfs(root.right, res, level + 1);
        }
    }

}
```

486. `_516.java`

```
package com.fishercoder.solutions;

/**
 * 516. Longest Palindromic Subsequence
 *
 * Given a string s, find the longest palindromic subsequence's length in s.
 * You may assume that the maximum length of s is 1000.

 Example 1:
 Input:
 "bbbab"
 Output:
 4
 One possible longest palindromic subsequence is "bbbb".

 Example 2:
 Input:
 "cbbd"
 Output:
 2
 One possible longest palindromic subsequence is "bb".
 */
public class _516 {

    public static class Solution1 {
        /**
         * Inspired by https://discuss.leetcode.com/topic/78603/straight-forward-java-dp-
solution
         * dp[i][j] means the longest palindromic subsequence's length of substring(i, j)
         * so, in the end, we return dp[0][s.length() - 1] which means the longest
palindromic subsequence
         * of this whole string.
         */
        public int longestPalindromeSubseq(String s) {
            int[][] dp = new int[s.length()][s.length()];
            for (int i = s.length() - 1; i >= 0; i--) {
                dp[i][i] = 1;//initialization
                for (int j = i + 1; j < s.length(); j++) {
                    if (s.charAt(i) == s.charAt(j)) {
                        dp[i][j] = dp[i + 1][j - 1] + 2;
                    } else {
                        dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                    }
                }
            }
            return dp[0][s.length() - 1];
        }
    }
}
```

487. `_517.java`

```
package com.fishercoder.solutions;

/**
 * 517. Super Washing Machines
 *
 * You have n super washing machines on a line. Initially, each washing machine has some
dresses or is empty.

 For each move, you could choose any m (1 ? m ? n) washing machines, and pass one dress
of each washing machine to one of its adjacent washing machines at the same time .

 Given an integer array representing the number of dresses in each washing machine from
left to right on the line, you should find the minimum number of moves to make all the
washing machines have the same number of dresses. If it is not possible to do it, return
-1.

 Example1

 Input: [1,0,5]

 Output: 3

 Explanation:
 1st move:    1     0 <-- 5    =>    1     1     4
 2nd move:    1 <-- 1 <-- 4    =>    2     1     3
 3rd move:    2     1 <-- 3    =>    2     2     2
 Example2

 Input: [0,3,0]

 Output: 2

 Explanation:
 1st move:    0 <-- 3     0    =>    1     2     0
 2nd move:    1     2 --> 0    =>    1     1     1
 Example3

 Input: [0,2,0]

 Output: -1

 Explanation:
 It's impossible to make all the three washing machines have the same number of dresses.
 Note:
 The range of n is [1, 10000].
 The range of dresses number in a super washing machine is [0, 1e5].

 */
public class _517 {
    public static class Solution1 {
        /**
```

```
         * Reference: https://discuss.leetcode.com/topic/79938/super-short-easy-java-o-n-
solution
         */
        public int findMinMoves(int[] machines) {
            int total = 0;
            for (int i : machines) {
                total += i;
            }
            if (total % machines.length != 0) {
                return -1;
            }
            int avg = total / machines.length;
            int cnt = 0;
            int max = 0;
            for (int load : machines) {
                cnt += load - avg; //load-avg is "gain/lose"
                max = Math.max(Math.max(max, Math.abs(cnt)), load - avg);
            }
            return max;
        }
    }
}
```

488. `_520.java`

```
package com.fishercoder.solutions;

/**
 * 520. Detect Capital
 *
 * Given a word, you need to judge whether the usage of capitals in it is right or not.
 * We define the usage of capitals in a word to be right when one of the following cases
holds:

 All letters in this word are capitals, like "USA".
 All letters in this word are not capitals, like "leetcode".
 Only the first letter in this word is capital if it has more than one letter, like
"Google".
 Otherwise, we define that this word doesn't use capitals in a right way.

 Example 1:
 Input: "USA"
 Output: True

 Example 2:
 Input: "FlaG"
 Output: False

 Note: The input will be a non-empty word consisting of uppercase and lowercase latin
letters.
 */
public class _520 {
    public static class Solution1 {
        public boolean detectCapitalUse(String word) {
            char[] words = word.toCharArray();
            boolean firstLetterCap = false;
            if (Character.isUpperCase(words[0])) {
                firstLetterCap = true;
            }

            if (firstLetterCap) {
                if (words.length >= 2) {
                    int i = 2;
                    if (Character.isUpperCase(words[1])) {
                        //then all following must be all uppercase
                        while (i < words.length) {
                            if (!Character.isUpperCase(words[i])) {
                                return false;
                            }
                            i++;
                        }
                        return true;
                    } else {
                        //then all following must be all lowercase
                        while (i < words.length) {
                            if (!Character.isLowerCase(words[i])) {
```

```
                                return false;
                            }
                            i++;
                        }
                        return true;
                    }
                }
                return true;
            } else {
                //then all following must be all lowercase
                int i = 1;
                while (i < words.length) {
                    if (!Character.isLowerCase(words[i])) {
                        return false;
                    }
                    i++;
                }
                return true;
            }
        }
    }
}
```

489. `_521.java`

```
package com.fishercoder.solutions;

/**
 * 521. Longest Uncommon Subsequence I
 *
 * Given a group of two strings, you need to find the longest uncommon subsequence of
this group of two strings.
 * The longest uncommon subsequence is defined as the longest subsequence of one of these
strings and this subsequence should not be any subsequence of the other strings.
 * A subsequence is a sequence that can be derived from one sequence by deleting some
characters without changing the order of the remaining elements.
 * Trivially, any string is a subsequence of itself and an empty string is a subsequence
of any string.
 * The input will be two strings, and the output needs to be the length of the longest
uncommon subsequence.
 * If the longest uncommon subsequence doesn't exist, return -1.

 Example 1:
 Input: "aba", "cdc"
 Output: 3
 Explanation: The longest uncommon subsequence is "aba" (or "cdc"),
 because "aba" is a subsequence of "aba",
 but not a subsequence of any other strings in the group of two strings.

 Note:
 Both strings' lengths will not exceed 100.
 Only letters from a ~ z will appear in input strings.
 */
public class _521 {
    public static class Solution1 {
        /**
         * The gotcha point of this question is:
         * 1. if a and b are identical, then there will be no common subsequence, return
-1
         * 2. else if a and b are of equal length, then any one of them will be a
subsequence of the other string
         * 3. else if a and b are of different length, then the longer one is a required
subsequence because the longer string cannot be a subsequence of the shorter one
         * Or in other words, when a.length() != b.length(), no subsequence of b will be
equal to a, so return Math.max(a.length(), b.length())
         */
        public int findLUSlength(String a, String b) {
            if (a.equals(b)) {
                return -1;
            }
            return Math.max(a.length(), b.length());
        }
    }
}
```

490. `_522.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 522. Longest Uncommon Subsequence II
 *
 * Given a list of strings, you need to find the longest uncommon subsequence among them.
 * The longest uncommon subsequence is defined as the longest subsequence of one of these
strings and
 * this subsequence should not be any subsequence of the other strings.
 * A subsequence is a sequence that can be derived from one sequence by deleting some
characters
 * without changing the order of the remaining elements.
 * Trivially, any string is a subsequence of itself and an empty string is a subsequence
of any string.
 * The input will be a list of strings, and the output needs to be the length of the
longest uncommon subsequence.
 * If the longest uncommon subsequence doesn't exist, return -1.

 Example 1:
 Input: "aba", "cdc", "eae"
 Output: 3
 Note:

 All the given strings' lengths will not exceed 10.
 The length of the given list will be in the range of [2, 50].
 */
public class _522 {

    public static class Solution1 {
        /**Idea: if there's such a LUS there in the list, it must be one of the strings
in the given list,
        so we'll just go through the list and check if one string is NOT subsequence of
any others, if so, return it, otherwise, return -1*/
        public int findLUSlength(String[] strs) {
            Arrays.sort(strs, new Comparator<String>() {
                @Override
                public int compare(String o1, String o2) {
                    return o2.length() - o1.length();
                }
            });

            for (int i = 0; i < strs.length; i++) {
                boolean found = true;
                for (int j = 0; j < strs.length; j++) {
                    if (i == j) {
                        continue;
                    } else if (isSubsequence(strs[i], strs[j])) {
                        found = false;
```

```
                        break;
                    }
                }
                if (found) {
                    return strs[i].length();
                }
            }
            return -1;
        }

        private boolean isSubsequence(String a, String b) {
            int i = 0;
            for (int j = 0; i < a.length() && j < b.length(); j++) {
                if (a.charAt(i) == b.charAt(j)) {
                    i++;
                }
            }
            return i == a.length();
        }
    }
}
```

491. `_523.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 523. Continuous Subarray Sum
 *
 * Given a list of non-negative numbers and a target integer k,
 * write a function to check if the array has a continuous subarray of size at least 2
 * that sums up to the multiple of k, that is, sums up to n*k where n is also an integer.

 Example 1:
 Input: [23, 2, 4, 6, 7],  k=6
 Output: True
 Explanation: Because [2, 4] is a continuous subarray of size 2 and sums up to 6.

 Example 2:
 Input: [23, 2, 6, 4, 7],  k=6
 Output: True
 Explanation: Because [23, 2, 6, 4, 7] is an continuous subarray of size 5 and sums up to
42.

 Note:
 The length of the array won't exceed 10,000.
 You may assume the sum of all the numbers is in the range of a signed 32-bit integer.

 */
public class _523 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/80793/java-o-n-time-o-k-space/20
         * "The reason we use modulus is:
         * (a+(n*x))%x is same as (a%x)
         * e.g. in case of the array [23,2,6,4,7] the running sum is [23,25,31,35,42]
         * and the remainders are [5,1,1,5,0].
         * We got reminder 5 at index 0 and at index 3.
         * That means, in between these two indexes we must have added a number which is
multiple of the k.
         * Hope this clarifies your doubt :)"
         */
        public boolean checkSubarraySum(int[] nums, int k) {
            Map<Integer, Integer> map = new HashMap<>();
            map.put(0, -1);
            int sum = 0;
            for (int i = 0; i < nums.length; i++) {
                sum += nums[i];
                if (k != 0) {
                    /**Because if k == 0, sum %= k will throw ArithmeticException.*/
                    sum %= k;
```

```
                }
                Integer prev = map.get(sum);
                if (prev != null) {
                    if (i - prev > 1) {
                        /**This makes sure that it has length at least 2*/
                        return true;
                    }
                } else {
                    map.put(sum, i);
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        public boolean checkSubarraySum(int[] nums, int k) {
            if (nums == null || nums.length == 0) {
                return false;
            }

            //Two continuous zeroes will form a subarray of length 2 with sum 0, 0*k = 0
will always be true
            for (int i = 0; i < nums.length - 1; i++) {
                if (nums[i] == 0 && nums[i + 1] == 0) {
                    return true;
                }
            }

            //then k cannot be zero any more
            if (k == 0 || nums.length < 2) {
                return false;
            }

            int[] preSums = new int[nums.length + 1];
            for (int i = 1; i <= nums.length; i++) {
                preSums[i] = preSums[i - 1] + nums[i - 1];
            }

            for (int i = 1; i <= nums.length; i++) {
                for (int j = 0; j < i - 1; j++) {
                    if ((preSums[i] - preSums[j]) % k == 0) {
                        return true;
                    }
                }
            }
            return false;
        }
    }

}
```

492. `_524.java`

```
package com.fishercoder.solutions;

import java.util.Collections;
import java.util.List;

/**
 * 524. Longest Word in Dictionary through Deleting
 *
 * Given a string and a string dictionary, find the longest string in the dictionary that
 * can be formed by deleting some characters of the given string.
 * If there are more than one possible results,
 * return the longest word with the smallest lexicographical order.
 * If there is no possible result, return the empty string.

 Example 1:
 Input:
 s = "abpcplea", d = ["ale","apple","monkey","plea"]

 Output:
 "apple"
 Example 2:
 Input:
 s = "abpcplea", d = ["a","b","c"]

 Output:
 "a"

 Note:
 All the strings in the input will only contain lower-case letters.
 The size of the dictionary won't exceed 1,000.
 The length of all the strings in the input won't exceed 1,000.
 */
public class _524 {

    public static class Solution1 {
        public String findLongestWord(String s, List<String> d) {
            Collections.sort(d, (a, b) -> a.length() == b.length() ? a.compareTo(b) :
b.length() - a.length());
            for (String dictWord : d) {
                int i = 0;
                for (char c : s.toCharArray()) {
                    if (i < dictWord.length() && dictWord.charAt(i) == c) {
                        i++;
                    }
                }
                if (i == dictWord.length()) {
                    return dictWord;
                }
            }
            return "";
        }
```

```
    }

}
```

493. `_525.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 525. Contiguous Array
 *
 * Given a binary array, find the maximum length of a contiguous subarray with equal
number of 0 and 1.

 Example 1:
 Input: [0,1]
 Output: 2
 Explanation: [0, 1] is the longest contiguous subarray with equal number of 0 and 1.

 Example 2:
 Input: [0,1,0]
 Output: 2
 Explanation: [0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0
and 1.

 Note: The length of the given binary array will not exceed 50,000.
 */
public class _525 {

    public static class Solution1 {
        //credit: https://leetcode.com/articles/contiguous-array/#approach-3-using-
hashmap-accepted
        public int findMaxLength(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            int count = 0;
            int max = 0;
            Map<Integer, Integer> map = new HashMap();
            map.put(0, -1);//initialize the map, which means at index zero, the length of
contiguous subarray is -1
            for (int i = 0; i < nums.length; i++) {
                count += nums[i] == 1 ? 1 : -1;
                if (map.containsKey(count)) {
                    max = Math.max(i - map.get(count), max);
                } else {
                    map.put(count, i);//only when the map does not have this key, we put
it in the map, this avoids overwriting the map
                    //also, it helps us keep the most leftside value in the map to help
us compute the longest contigous array length
                }
            }
            return max;
        }
```

```
    }
}
```

494. `_526.java`

```
package com.fishercoder.solutions;

/**
 * 526. Beautiful Arrangement
 *
 * Suppose you have N integers from 1 to N.
 * We define a beautiful arrangement as an array that is constructed by these N numbers
successfully
 * if one of the following is true for the ith position (1 ≤ i ≤ N) in this array:
 * The number at the ith position is divisible by i.
 * i is divisible by the number at the ith position.
 * Now given N, how many beautiful arrangements can you construct?

 Example 1:

 Input: 2
 Output: 2

 Explanation:

 The first beautiful arrangement is [1, 2]:
 Number at the 1st position (i=1) is 1, and 1 is divisible by i (i=1).
 Number at the 2nd position (i=2) is 2, and 2 is divisible by i (i=2).

 The second beautiful arrangement is [2, 1]:
 Number at the 1st position (i=1) is 2, and 2 is divisible by i (i=1).
 Number at the 2nd position (i=2) is 1, and i (i=2) is divisible by 1.

 Note:
 N is a positive integer and will not exceed 15.
 */
public class _526 {
        public static class Solution1 {
                /**
                 * A good post to look at: https://discuss.leetcode.com/topic/79916/java-
solution-backtracking
                 * and there's a generic template afterwards for backtracking problems
                 */

                int count = 0;

                public int countArrangement(int N) {
                        backtracking(N, new int[N + 1], 1);
                        return count;
                }

                private void backtracking(int N, int[] used, int pos) {
                        if (pos > N) {
                                count++;
                                return;
                        }
```

```
                        for (int i = 1; i <= N; i++) {
                                if (used[i] == 0 && (i % pos == 0 || pos % i == 0)) {
                                        used[i] = 1;
                                        backtracking(N, used, pos + 1);
                                        used[i] = 0;
                                }
                        }
                }
        }

}
```

495. `_527.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.List;

/**
 * 527. Word Abbreviation
 *
 * Given an array of n distinct non-empty strings, you need to generate minimal possible
abbreviations for every word following rules below.

 Begin with the first character and then the number of characters abbreviated, which
followed by the last character.
 If there are any conflict, that is more than one words share the same abbreviation,
 a longer prefix is used instead of only the first character until making the map
 from word to abbreviation become unique.
 In other words, a final abbreviation cannot map to more than one original words.
 If the abbreviation doesn't make the word shorter, then keep it as original.

 Example:
 Input: ["like", "god", "internal", "me", "internet", "interval", "intension", "face",
"intrusion"]
 Output: ["l2e","god","internal","me","i6t","interval","inte4n","f2e","intr4n"]

 Note:
 Both n and the length of each word will not exceed 400.
 The length of each word is greater than 1.
 The words consist of lowercase English letters only.
 The return answers should be in the same order as the original array.
 */
public class _527 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/82613/really-simple-and-
straightforward-java-solution
         */
        public List<String> wordsAbbreviation(List<String> dict) {
            int len = dict.size();
            String[] ans = new String[len];
            int[] prefix = new int[len];
            for (int i = 0; i < len; i++) {
                prefix[i] = 1;
                ans[i] = abbreviate(dict.get(i), 1); // make abbreviation for each string
            }
            for (int i = 0; i < len; i++) {
                while (true) {
                    HashSet<Integer> set = new HashSet<>();
                    for (int j = i + 1; j < len; j++) {
                        if (ans[j].equals(ans[i])) {
```

```
                            set.add(j); // check all strings with the same abbreviation
                        }
                    }
                    if (set.isEmpty()) {
                        break;
                    }
                    set.add(i);
                    for (int k : set) {
                        ans[k] = abbreviate(dict.get(k), ++prefix[k]); // increase the
prefix
                    }
                }
            }
            return Arrays.asList(ans);
        }

        private String abbreviate(String word, int k) {
            if (k + 2 >= word.length()) {
                return word;
            }
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.append(word.substring(0, k));
            stringBuilder.append(word.length() - 1 - k);
            stringBuilder.append(word.substring(word.length() - 1));
            return stringBuilder.toString();
        }
    }

}
```

496. `_529.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 529. Minesweeper
 *
 * Let's play the minesweeper game (Wikipedia, online game)!

 You are given a 2D char matrix representing the game board. 'm' represents an unrevealed mine, 'E' represents an unrevealed empty square, 'B' represents a revealed blank square that has no adjacent (above, below, left, right, and all 4 diagonals) mines, digit ('1' to '8') represents how many mines are adjacent to this revealed square, and finally 'X' represents a revealed mine.

 Now given the next click position (row and column indices) among all the unrevealed squares ('m' or 'E'), return the board after revealing this position according to the following rules:

 If a mine ('m') is revealed, then the game is over - change it to 'X'.
 If an empty square ('E') with no adjacent mines is revealed, then change it to revealed blank ('B') and all of its adjacent unrevealed squares should be revealed recursively.
 If an empty square ('E') with at least one adjacent mine is revealed, then change it to a digit ('1' to '8') representing the number of adjacent mines.
 Return the board when no more squares will be revealed.

 Example 1:
 Input:

 [['E', 'E', 'E', 'E', 'E'],
 ['E', 'E', 'm', 'E', 'E'],
 ['E', 'E', 'E', 'E', 'E'],
 ['E', 'E', 'E', 'E', 'E']]

 Click : [3,0]

 Output:

 [['B', '1', 'E', '1', 'B'],
 ['B', '1', 'm', '1', 'B'],
 ['B', '1', '1', '1', 'B'],
 ['B', 'B', 'B', 'B', 'B']]

 Explanation:


 Example 2:
 Input:

 [['B', '1', 'E', '1', 'B'],
```

```
 ['B', '1', 'm', '1', 'B'],
 ['B', '1', '1', '1', 'B'],
 ['B', 'B', 'B', 'B', 'B']]

 Click : [1,2]

 Output:

 [['B', '1', 'E', '1', 'B'],
 ['B', '1', 'X', '1', 'B'],
 ['B', '1', '1', '1', 'B'],
 ['B', 'B', 'B', 'B', 'B']]

 Explanation:

 Note:
 The range of the input matrix's height and width is [1,50].
 The click position will only be an unrevealed square ('m' or 'E'), which also means the
input board contains at least one clickable square.
 The input board won't be a stage when game is over (some mines have been revealed).
 For simplicity, not mentioned rules should be ignored in this problem. For example, you
don't need to reveal all the unrevealed mines when the game is over, consider any cases
that you will win the game or flag any squares.
 */
public class _529 {

    public static class Solution1 {
        public char[][] updateBoard(char[][] board, int[] click) {
            int m = board.length;
            int n = board[0].length;
            Queue<int[]> queue = new LinkedList();
            queue.offer(click);
            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                int currRow = curr[0];
                int currCol = curr[1];
                if (board[currRow][currCol] == 'M') {
                    /**This also covers the corner case: when click[] happens to be on a
mine, then it'll exit directly.
                     * Otherwise, we'll just continue and mark this cell to be 'M' and
keep processing all 'E' cells in the queue.*/
                    board[currRow][currCol] = 'X';
                } else {
                    /**scan all four directions of this curr cell, count all mines, this
includes 'X' and 'M' */
                    int count = 0;
                    for (int i = -1; i < 2; i++) {
                        for (int j = -1; j < 2; j++) {
                            if (i == 0 && j == 0) {
                                continue;
                            }
                            int nextRow = currRow + i;
                            int nextCol = currCol + j;
                            if (nextRow >= m || nextRow < 0 || nextCol >= n || nextCol <
```

```
0) {
                                continue;
                            }
                            if (board[nextRow][nextCol] == 'M' || board[nextRow][nextCol]
== 'X') {
                                count++;
                            }
                        }
                    }

                    if (count > 0) {
                        /**There are mines around this cell, so update it with the number
of mines*/
                        board[currRow][currCol] = (char) (count + '0');
                    } else {
                        /**There is no mines around this cell, so update it to be 'B'*/
                        board[currRow][currCol] = 'B';

                        /**then we'll also check all of its four surrounding cells, if
it's 'E'. we'll also update it to be 'B' and offer it into the queue*/
                        for (int i = -1; i < 2; i++) {
                            for (int j = -1; j < 2; j++) {
                                if (i == 0 && j == 0) {
                                    continue;
                                }
                                int nextRow = currRow + i;
                                int nextCol = currCol + j;
                                if (nextRow >= m || nextRow < 0 || nextCol >= n ||
nextCol < 0) {
                                    continue;
                                }
                                if (board[nextRow][nextCol] == 'E') {
                                    /**we offer 'E' cells into the queue*/
                                    queue.offer(new int[]{nextRow, nextCol});
                                    /**then update this cell to be 'B' */
                                    board[nextRow][nextCol] = 'B';
                                }
                            }
                        }
                    }
                }
            }
            return board;
        }
    }
}
```

497. `_530.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.Iterator;
import java.util.TreeSet;

/**
 * 530. Minimum Absolute Difference in BST
 *
 * Given a binary search tree with non-negative values, find the minimum absolute
difference between values of any two nodes.

 Example:

 Input:

   1
    \
     3
    /
   2

 Output:
 1

 Explanation:
 The minimum absolute difference is 1, which is the difference between 2 and 1 (or
between 2 and 3).
 Note: There are at least two nodes in this BST.
 */
public class _530 {

    public static class Solution1 {
        public int getMinimumDifference(TreeNode root) {
            TreeSet<Integer> treeset = new TreeSet<>();
            treeset.add(root.val);
            dfs(root, treeset);
            int diff = Integer.MAX_VALUE;
            Iterator<Integer> iterator = treeset.iterator();
            int prev = iterator.next();
            while (iterator.hasNext()) {
                int current = iterator.next();
                diff = Math.min(diff, Math.abs(current - prev));
                prev = current;
            }
            return diff;
        }

        private void dfs(TreeNode root, TreeSet<Integer> treeset) {
            if (root.left != null) {
```

```
                treeset.add(root.left.val);
                dfs(root.left, treeset);
            }
            if (root.right != null) {
                treeset.add(root.right.val);
                dfs(root.right, treeset);
            }
        }
    }
}
```

498. `_531.java`

```
package com.fishercoder.solutions;

/**
 * 531. Lonely Pixel I
 *
 * Given a picture consisting of black and white pixels, find the number of black lonely
pixels.
 * The picture is represented by a 2D char array consisting of 'B' and 'W', which means
black and white pixels respectively.
 * A black lonely pixel is character 'B' that located at a specific position where the
same row and same column don't have any other black pixels.

 Example:
 Input:
 [['W', 'W', 'B'],
 ['W', 'B', 'W'],
 ['B', 'W', 'W']]

 Output: 3
 Explanation: All the three 'B's are black lonely pixels.

 Note:
 The range of width and height of the input 2D array is [1,500].
 */
public class _531 {

    public static class Solution1 {
        public int findLonelyPixel(char[][] picture) {
            int m = picture.length;
            int n = picture[0].length;
            int count = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (picture[i][j] == 'B' && isLonely(i, j, picture, m, n)) {
                        count++;
                    }
                }
            }
            return count;
        }

        private boolean isLonely(int row, int col, char[][] picture, int m, int n) {
            for (int i = 0; i < m; i++) {
                if (i != row) {
                    if (picture[i][col] == 'B') {
                        return false;
                    }
                }
            }

            for (int j = 0; j < n; j++) {
```

```
                if (j != col) {
                    if (picture[row][j] == 'B') {
                        return false;
                    }
                }
            }
            return true;
        }
    }
}
```

499. `_532.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 532. K-diff Pairs in an Array
 *
 * Given an array of integers and an integer k, you need to find the number of unique k-
diff pairs in the array.
 * Here a k-diff pair is defined as an integer pair (i, j), where i and j are both
numbers in the array and their absolute difference is k.

 Example 1:
 Input: [3, 1, 4, 1, 5], k = 2
 Output: 2
 Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5).
 Although we have two 1s in the input, we should only return the number of unique pairs.

 Example 2:
 Input:[1, 2, 3, 4, 5], k = 1
 Output: 4
 Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4,
5).

 Example 3:
 Input: [1, 3, 1, 5, 4], k = 0
 Output: 1
 Explanation: There is one 0-diff pair in the array, (1, 1).

 Note:
 The pairs (i, j) and (j, i) count as the same pair.
 The length of the array won't exceed 10,000.
 All the integers in the given input belong to the range: [-1e7, 1e7].
 */
public class _532 {

    public static class Solution1 {
        public int findPairs(int[] nums, int k) {
            if (nums == null || nums.length == 0 || k < 0) {
                return 0;
            }

            Map<Integer, Integer> map = new HashMap();
            for (int num : nums) {
                map.put(num, map.getOrDefault(num, 0) + 1);
            }

            int answer = 0;
            for (int key : map.keySet()) {
                if (k == 0) {
```

```
                    if (map.get(key) >= 2) {
                        answer++;
                    }
                } else {
                    if (map.containsKey(key + k)) {
                        answer++;
                    }
                }
            }
            return answer;
        }
    }

}
```

500. `_533.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 533. Lonely Pixel II
 *
 * Given a picture consisting of black and white pixels, and a positive integer N,
 * find the number of black pixels located at some specific row R and column C that align
with all the following rules:

 Row R and column C both contain exactly N black pixels.
 For all rows that have a black pixel at column C, they should be exactly the same as row
R
 The picture is represented by a 2D char array consisting of 'B' and 'W', which means
black and white pixels respectively.

 Example:
 Input:
 [['W', 'B', 'W', 'B', 'B', 'W'],
 ['W', 'B', 'W', 'B', 'B', 'W'],
 ['W', 'B', 'W', 'B', 'B', 'W'],
 ['W', 'W', 'B', 'W', 'B', 'W']]

 N = 3
 Output: 6
 Explanation: All the bold 'B' are the black pixels we need (all 'B's at column 1 and 3).
        0    1    2    3    4    5         column index
 0    [['W', 'B', 'W', 'B', 'B', 'W'],
 1     ['W', 'B', 'W', 'B', 'B', 'W'],
 2     ['W', 'B', 'W', 'B', 'B', 'W'],
 3     ['W', 'W', 'B', 'W', 'B', 'W']]
 row index

 Take 'B' at row R = 0 and column C = 1 as an example:
 Rule 1, row R = 0 and column C = 1 both have exactly N = 3 black pixels.
 Rule 2, the rows have black pixel at column C = 1 are row 0, row 1 and row 2. They are
exactly the same as row R = 0.

 Note:
 The range of width and height of the input 2D array is [1,200].
 */
public class _533 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/81686/verbose-java-o-m-n-solution-
hashmap/5
         * This program is very well designed to do things:
         * 1. it scans the entire matrix once, but does two things in this scan:
         * first it records an array of cols that keeps count of how many 'B' are there
```

```
in each column;
         * second, at the end of traversing each column, it puts this entire row as the
key into a HashMap
         * when there N number of 'B's in this row.
         * 2. then we could go through the HashMap keyset:
         * if one row has N number of 'B's, we go through this row's each column to see
if any element in this row is 'B' and also that element's column has N 'B's
         */
        public int findBlackPixel(char[][] picture, int N) {
            if (picture == null || picture.length == 0 || picture[0].length == 0) {
                return 0;
            }
            int m = picture.length;
            int n = picture[0].length;
            int[] cols = new int[n];
            Map<String, Integer> map = new HashMap<>();
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < m; i++) {
                int count = 0;
                for (int j = 0; j < n; j++) {
                    if (picture[i][j] == 'B') {
                        count++;
                        cols[j]++;
                    }
                    stringBuilder.append(picture[i][j]);
                }
                if (count == N) {
                    /**we use this entire row string as key for the map*/
                    map.put(stringBuilder.toString(),
map.getOrDefault(stringBuilder.toString(), 0) + 1);
                }
                stringBuilder.setLength(0);
            }

            int answer = 0;
            for (String key : map.keySet()) {
                if (map.get(key) != N) {
                    continue;
                }
                for (int i = 0; i < n; i++) {
                    if (key.charAt(i) == 'B' && cols[i] == N) {
                        answer += N;
                    }
                }
            }
            return answer;
        }
    }

}
```

501. `_535.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.Random;

/**
 * 535. Encode and Decode TinyURL
 *
 * TinyURL is a URL shortening service where you enter a URL such as
https://leetcode.com/problems/design-tinyurl
 * and it returns a short URL such as http://tinyurl.com/4e9iAk.
 * Design the encode and decode methods for the TinyURL service.
 * There is no restriction on how your encode/decode algorithm should work.
 * You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can
be decoded to the original URL.

 Note: Do not use class member/global/static variables to store states. Your encode and
decode algorithms should be stateless.
 */

public class _535 {

    public static class Solution1 {
        /**
         * Simple counter approach
         * Analysis:
         *      The range of URLs that can be decoded is limited by the range of Integer.
         *      If excessively large number of URLs have to be encoded, after the range
of Integer is exceeded,
         *          integer overflow could lead to overwriting previous URL's encodings.
         *      The length of the URL isn't necessary shorter than incoming URL.
         *      One potential security issue with this problem is that it's very easy to
predict the next code generated,
         *          since this pattern is very easy to be detected.
         */
        public class Codec {
            int i = 0;
            Map<Integer, String> map = new HashMap();
            public static final String PREFIX = "http://tinyurl/";

            public String encode(String longUrl) {
                map.put(i, longUrl);
                return PREFIX + i++;
            }

            public String decode(String shortUrl) {
                return map.get(Integer.parseInt(shortUrl.substring(PREFIX.length())));
            }
        }
    }
```

```
    public static class Solution2 {
        /**
         * Use Java built-in HashCode
         * Analysis:
         *      hashCode() does NOT generate unique codes for different strings,
collision might happen.
         *          As the number of URLs increase, the probability of getting collision
increases which leads to failure.
         */
        public class Codec {
            Map<Integer, String> map = new HashMap<>();
            public static final String PREFIX = "http://tinyurl/";

            // Encodes a URL to a shortened URL.
            public String encode(String longUrl) {
                /**I don't need to create a local variable to cache longUrl.hashCode()
                 * since Java's String cache it already. :) Look at its source code.*/
                map.put(longUrl.hashCode(), longUrl);
                return PREFIX + longUrl.hashCode();
            }

            // Decodes a shortened URL to its original URL.
            public String decode(String shortUrl) {
                return map.get(Integer.parseInt(shortUrl.substring(PREFIX.length())));
            }
        }
    }

    public static class Solution3 {
        /**Use a random number*/
        Map<Integer, String> map = new HashMap<>();
        Random random = new Random();
        public static final String PREFIX = "http://tinyurl/";

        // Encodes a URL to a shortened URL.
        public String encode(String longUrl) {
            int num = random.nextInt(Integer.MAX_VALUE);
            while (map.containsKey(num)) {
                num = random.nextInt(Integer.MAX_VALUE);
            }
            map.put(num, longUrl);
            return PREFIX + num;
        }

        // Decodes a shortened URL to its original URL.
        public String decode(String shortUrl) {
            return map.get(Integer.parseInt(shortUrl.substring(PREFIX.length())));
        }
    }

    public static class Solution4 {
        /**
         * Use a random but fixed length encoding
```

```
         * Analysis:
         *      1. This is the most optimal solution so far.
         *      2. The number of URLs that can be encoded can be as big as Math.pow((10 +
26*2), FIXED_LENGTH)
         *      3. The length of the shortened URL is fixed at a certain length, which
could be a significant reduce for large URLs
         *      4. The performance of this scheme is pretty good, due to much smaller
probability of encountering collision
         *      5. Predicting pattern/encoding isn't possible in this case since random
numbers are used.
         */
        Map<String, String> map = new HashMap<>();
        public static final String PREFIX = "http://tinyurl/";
        public static final int FIXED_LENGTH = 7;
        Random random = new Random();
        String alphabet =
"0123456789abcdefghijklmnopgrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

        // Encodes a URL to a shortened URL.
        public String encode(String longUrl) {
            String shortKey = getRandomFixedLengthKey();
            while (map.containsKey(shortKey)) {
                shortKey = getRandomFixedLengthKey();
            }
            map.put(shortKey, longUrl);
            return PREFIX + shortKey;
        }

        private String getRandomFixedLengthKey() {
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < FIXED_LENGTH; i++) {
                stringBuilder.append(alphabet.charAt(random.nextInt(alphabet.length())));
            }
            return stringBuilder.toString();
        }

        // Decodes a shortened URL to its original URL.
        public String decode(String shortUrl) {
            return map.get(shortUrl.substring(PREFIX.length()));
        }
    }
}
```

502. `_536.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 536. Construct Binary Tree from String
 *
 * You need to construct a binary tree from a string consisting of parenthesis and
integers.
 The whole input represents a binary tree. It contains an integer followed by zero, one
or two pairs of parenthesis.
 The integer represents the root's value and a pair of parenthesis contains a child
binary tree with the same structure.
 You always start to construct the left child node of the parent first if it exists.

 Example:
 Input: "4(2(3)(1))(6(5))"
 Output: return the tree root node representing the following tree:

       4
     /   \
    2     6
   / \   /
  3   1 5

 Note:
 There will only be '(', ')', '-' and '0' ~ '9' in the input string.
 An empty tree is represented by "" instead of "()".
 */

public class _536 {

    public static class Solution1 {
        public TreeNode str2tree(String s) {
            if (s.equals("")) {
                return null;
            }
            int firstParen = s.indexOf("(");
            int val = firstParen == -1 ? Integer.parseInt(s) :
Integer.parseInt(s.substring(0, firstParen));
            TreeNode cur = new TreeNode(val);
            if (firstParen == -1) {
                return cur;
            }
            int start = firstParen;
            int leftParenCount = 0;
            for (int i = start; i < s.length(); i++) {
                if (s.charAt(i) == '(') {
```

```
                    leftParenCount++;
                } else if (s.charAt(i) == ')') {
                    leftParenCount--;
                }
                if (leftParenCount == 0 && start == firstParen) {
                    cur.left = str2tree(s.substring(start + 1, i));
                    start = i + 1;
                } else if (leftParenCount == 0) {
                    cur.right = str2tree(s.substring(start + 1, i));
                }
            }
            return cur;
        }
    }

    public static class Solution2 {
        public TreeNode str2tree(String s) {
            Deque<TreeNode> stack = new ArrayDeque<>();
            for (int i = 0, j = i; i < s.length(); i++, j = i) {
                char c = s.charAt(i);
                if (c == ')') {
                    stack.pop();
                } else if (c >= '0' && c <= '9' || c == '-') {
                    while (i + 1 < s.length() && s.charAt(i + 1) >= '0' && s.charAt(i +
1) <= '9') {
                        i++;
                    }
                    TreeNode curr = new TreeNode(Integer.valueOf(s.substring(j, i + 1)));
                    if (!stack.isEmpty()) {
                        TreeNode parent = stack.peek();
                        if (parent.left != null) {
                            parent.right = curr;
                        } else {
                            parent.left = curr;
                        }
                    }
                    stack.push(curr);
                }
            }
            return stack.isEmpty() ? null : stack.peek();
        }
    }

}
```

503. `_537.java`

```
package com.fishercoder.solutions;

import java.util.stream.Stream;

/**
 * 537. Complex Number Multiplication
 *
 * Given two strings representing two complex numbers.

 You need to return a string representing their multiplication. Note i2 = -1 according to the definition.

 Example 1:
 Input: "1+1i", "1+1i"
 Output: "0+2i"
 Explanation: (1 + i) * (1 + i) = 1 + i2 + 2 * i = 2i, and you need convert it to the form of 0+2i.

 Example 2:
 Input: "1+-1i", "1+-1i"
 Output: "0+-2i"
 Explanation: (1 - i) * (1 - i) = 1 + i2 - 2 * i = -2i, and you need convert it to the form of 0+-2i.

 Note:
 The input strings will not have extra blank.
 The input strings will be given in the form of a+bi, where the integer a and b will both belong to the range of [-100, 100]. And the output should be also in this form.
 */
public class _537 {

    public static class Solution1 {
        public String complexNumberMultiply(String a, String b) {
            String[] part1And2 = a.split("\\+");
            String[] part3And4 = b.split("\\+");
            String product1 = String.valueOf(Integer.parseInt(part1And2[0]) * Integer.parseInt(part3And4[0]));//this is real number multiplication
            String product2 = multiply(part1And2[0], part3And4[1]);
            String product3 = multiply(part3And4[0], part1And2[1]);
            String product4 = multiplyTwoIs(part3And4[1], part1And2[1]);
            String twoISum = sumTwoI(product2, product3);
            String numberValue = String.valueOf(Integer.valueOf(product1) + Integer.valueOf(product4));
            return numberValue + "+" + twoISum;
        }

        private String sumTwoI(String product2, String product3) {
            int number2 = Integer.parseInt(product2.substring(0, product2.length() - 1));
            int number3 = Integer.parseInt(product3.substring(0, product3.length() - 1));
            return String.valueOf(number2 + number3) + "i";
        }
```

```
        private String multiplyTwoIs(String p, String q) {
            int number1 = Integer.parseInt(p.substring(0, p.length() - 1));
            int number2 = Integer.parseInt(q.substring(0, q.length() - 1));
            int numberProduct = number1 * number2;
            return String.valueOf(-numberProduct);
        }

        private String multiply(String p, String withI) {
            int numberPart = Integer.parseInt(withI.substring(0, withI.length() - 1));
            return String.valueOf(numberPart * Integer.valueOf(p)) + "i";
        }
    }

    public static class Solution2 {
        /**
         * (a + bi) * (c + di) could become (ac - bd) + (ad + bc)*i
         * Thus, we have the following function
         */
        public String complexNumberMultiply(String a, String b) {
            int[] coefficients1 =
Stream.of(a.split("\\+|i")).mapToInt(Integer::parseInt).toArray();
            int[] coefficients2 =
Stream.of(b.split("\\+|i")).mapToInt(Integer::parseInt).toArray();
            return (coefficients1[0] * coefficients2[0] - coefficients1[1] *
coefficients2[1]) + "+"
                    + (coefficients1[0] * coefficients2[1] + coefficients1[1] *
coefficients2[0] + "i");
        }
    }

}
```

504. `_538.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.TreeMap;

/**
 * 538. Convert BST to Greater Tree
 *
 * Given a Binary Search Tree (BST), convert it to a Greater Tree such that every key of
the original BST is changed
 * to the original key plus sum of all keys greater than the original key in BST.

 Example:

 Input: The root of a Binary Search Tree like this:
     5
   /   \
  2    13

 Output: The root of a Greater Tree like this:
     18
   /   \
  20   13
 */

public class _538 {

    public static class Solution1 {
        /**
         * Traverse in this order: right -> root -> left
         */
        public TreeNode convertBST(TreeNode root) {
            dfs(root, 0);
            return root;
        }

        private int dfs(TreeNode root, int val) {
            if (root == null) {
                return val;
            }
            root.val += dfs(root.right, val);
            return dfs(root.left, root.val);
        }
    }

    public static class Solution2 {
        //This solution is generic for both BST and regular binary trees
```

```
        public TreeNode convertBST(TreeNode root) {
            if (root == null) {
                return root;
            }
            List<Integer> list = new ArrayList<>();
            putNodeToList(list, root);
            Collections.sort(list);
            int[] sums = new int[list.size()];
            sums[list.size() - 1] = 0;
            for (int i = list.size() - 2; i >= 0; i--) {
                sums[i] = sums[i + 1] + list.get(i + 1);
            }
            TreeMap<Integer, Integer> treeMap = new TreeMap<>();
            for (int i = 0; i < list.size(); i++) {
                treeMap.put(list.get(i), sums[i]);
            }
            TreeNode result = new TreeNode(treeMap.get(list.get(0)));
            return generateResultRoot(root, treeMap, result);
        }

        private TreeNode generateResultRoot(TreeNode root, TreeMap<Integer, Integer>
treeMap, TreeNode result) {
            if (root != null) {
                result.val = treeMap.get(root.val) + root.val;
            }
            if (root.left != null) {
                result.left = new TreeNode(0);
                generateResultRoot(root.left, treeMap, result.left);
            }
            if (root.right != null) {
                result.right = new TreeNode(0);
                generateResultRoot(root.right, treeMap, result.right);
            }
            return result;
        }

        private void putNodeToList(List<Integer> list, TreeNode root) {
            if (root != null) {
                list.add(root.val);
            }
            if (root.left != null) {
                putNodeToList(list, root.left);
            }
            if (root.right != null) {
                putNodeToList(list, root.right);
            }
        }
    }

}
```

505. `_539.java`

```
package com.fishercoder.solutions;

import java.util.List;

/**
 * 539. Minimum Time Difference
 *
 * Given a list of 24-hour clock time points in "Hour:Minutes" format,
 * find the minimum minutes difference between any two time points in the list.

 Example 1:
 Input: ["23:59","00:00"]
 Output: 1

 Note:
 The number of time points in the given list is at least 2 and won't exceed 20000.
 The input time is legal and ranges from 00:00 to 23:59.
 */
public class _539 {

    public static class Soluiton1 {

        public int findMinDifference(List<String> timePoints) {
            /**there are in total 24*60 = 1440 possible time points*/
            final int ALL_POSSIBLE_TIMEPOINTS = 1440;
            boolean[] allTimePoints = new boolean[ALL_POSSIBLE_TIMEPOINTS];
            for (String eachTime : timePoints) {
                String[] timeParts = eachTime.split(":");
                int hour = Integer.valueOf(timeParts[0]);
                int minute = Integer.valueOf(timeParts[1]);
                int value = hour * 60 + minute;
                if (allTimePoints[value]) {
                    return 0;
                }
                allTimePoints[value] = true;
            }

            int min = Integer.MAX_VALUE;
            int prev = 0;
            int first = Integer.MAX_VALUE;
            int last = Integer.MIN_VALUE;
            for (int i = 0; i < ALL_POSSIBLE_TIMEPOINTS; i++) {
                if (allTimePoints[i]) {
                    if (first != Integer.MAX_VALUE) {
                        min = Math.min(min, i - prev);
                    }
                    first = Math.min(first, i);
                    last = Math.max(last, i);
                    prev = i;
                }
            }
```

```
            min = Math.min(min, (ALL_POSSIBLE_TIMEPOINTS - last + first));
            return min;
        }
    }

}
```

506. `_540.java`

```
package com.fishercoder.solutions;

/**
 * 540. Single Element in a Sorted Array
 *
 * Given a sorted array consisting of only integers where every
 * element appears twice except for one element which appears once.
 * Find this single element that appears only once.

 Example 1:
 Input: [1,1,2,3,3,4,4,8,8]
 Output: 2
 Example 2:
 Input: [3,3,7,7,10,11,11]
 Output: 10
 Note: Your solution should run in O(log n) time and O(1) space.
 */
public class _540 {
    public static class Solution1 {
        public int singleNonDuplicate(int[] nums) {
            int result = 0;
            for (int i = 0; i < nums.length; i++) {
                result ^= nums[i];
            }
            return result;
        }
    }

    public static class Solution2 {
        public int singleNonDuplicate(int[] nums) {
            int start = 0;
            int end = nums.length - 1;
            while (start < end) {
                int mid = start + (end - start) / 2;
                if (nums[mid] != nums[mid + 1] && nums[mid] != nums[mid - 1]) {
                    return nums[mid];
                } else if (nums[mid] == nums[mid + 1] && mid % 2 == 0) {
                    start = mid + 1;
                } else if (nums[mid] == nums[mid - 1] && mid % 2 == 1) {
                    start = mid + 1;
                } else {
                    end = mid - 1;
                }
            }
            return nums[start];
        }
    }
}
```

507. `_541.java`

```
package com.fishercoder.solutions;

/**
 * 541. Reverse String II
 *
 * Given a string and an integer k, you need to reverse the first k characters for every
2k characters counting from the start of the string.
 * If there are less than k characters left, reverse all of them.
 * If there are less than 2k but greater than or equal to k characters, then reverse the
first k characters and left the other as original.

 Example:
 Input: s = "abcdefg", k = 2
 Output: "bacdfeg"

 Restrictions:
 The string consists of lower English letters only.
 Length of the given string and k will in the range [1, 10000]
 */
public class _541 {

    public static class Solution1 {
        public String reverseStr(String s, int k) {
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < s.length(); i = i + 2 * k) {
                if (s.length() >= (i + k)) {
                    stringBuilder.append(new StringBuilder(s.substring(i, i +
k)).reverse());
                } else {
                    stringBuilder.append(new StringBuilder(s.substring(i)).reverse());
                    break;
                }
                if ((i + 2 * k) <= s.length()) {
                    stringBuilder.append(s.substring(i + k, i + 2 * k));
                } else {
                    stringBuilder.append(s.substring(i + k));
                }
            }
            return stringBuilder.toString();
        }
    }
}
```

508. `_542.java`

```
package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;
import java.util.List;

/**
 * 542. 01 Matrix
 *
 * Given a matrix consists of 0 and 1, find the distance of the nearest 0 for each cell.

 The distance between two adjacent cells is 1.
 Example 1:
 Input:

 0 0 0
 0 1 0
 0 0 0
 Output:
 0 0 0
 0 1 0
 0 0 0
 Example 2:
 Input:

 0 0 0
 0 1 0
 1 1 1
 Output:
 0 0 0
 0 1 0
 1 2 1

 Note:
 The number of elements of the given matrix will not exceed 10,000.
 There are at least one 0 in the given matrix.
 The cells are adjacent in only four directions: up, down, left and right.
 */
public class _542 {

    public static class Solution1 {

        public List<List<Integer>> updateMatrix(List<List<Integer>> matrix) {
            int m = matrix.size();
            int n = matrix.get(0).size();
            Deque<int[]> deque = new LinkedList<>();
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (matrix.get(i).get(j) == 0) {
                        deque.offer(new int[]{i, j});
                    } else {
```

```
                        matrix.get(i).set(j, Integer.MAX_VALUE);
                    }
                }
            }

            final int[] dirs = new int[]{0, 1, 0, -1, 0};
            while (!deque.isEmpty()) {
                int[] currentCell = deque.poll();
                for (int i = 0; i < dirs.length - 1; i++) {
                    int nextRow = currentCell[0] + dirs[i];
                    int nextCol = currentCell[1] + dirs[i + 1];
                    if (nextRow < 0 || nextCol < 0 || nextRow >= m || nextCol >= n ||
matrix.get(nextRow).get(nextCol) <= matrix.get(currentCell[0]).get(currentCell[1]) + 1) {
                        continue;
                    }
                    deque.offer(new int[]{nextRow, nextCol});
                    matrix.get(nextRow).set(nextCol,
matrix.get(currentCell[0]).get(currentCell[1]) + 1);
                }
            }
            return matrix;
        }
    }

}
```

509. `_543.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 543. Diameter of Binary Tree
 *
 * Given a binary tree, you need to compute the length of the diameter of the tree.
 * The diameter of a binary tree is the length of the longest path between any two nodes
in a tree.
 * This path may or may not pass through the root.

 Example:
 Given a binary tree
     1
    / \
   2   3
  / \
 4   5
 Return 3, which is the length of the path [4,2,1,3] or [5,2,1,3].

 Note: The length of path between two nodes is represented by the number of edges between
them.
 */
public class _543 {

    public static class Solution1 {
        /**This is a very great problem for practicing recursion:
         * 1. What dfs() returns is the max height it should pick from either its left or
right subtree, that's
         * what the int return type stands for;
         * 2. And during the recursion, we can keep updating the global variable:
"diameter";
         * 3. When computing length/height of a subtree, we should take the max of its
left and right, then plus one
         * and left height should be like this
         * int left = dfs(root.left);
         * instead of dfs(root.left) + 1;
         * we'll only plus one at the end
         * */
        int diameter = 0;

        public int diameterOfBinaryTree(TreeNode root) {
            dfs(root);
            return diameter;
        }

        private int dfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
```

```
            int left = dfs(root.left);
            int right = dfs(root.right);
            diameter = Math.max(diameter, left + right);
            return Math.max(left, right) + 1;
        }
    }
}
```

510. `_544.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * During the NBA playoffs, we always arrange the rather strong team to play with the
rather weak team,
 * like make the rank 1 team play with the rank nth team, which is a good strategy to
make the contest more interesting.
 * Now, you're given n teams, you need to output their final contest matches in the form
of a string.
 * The n teams are given in the form of positive integers from 1 to n, which represents
their initial rank.
 * (Rank 1 is the strongest team and Rank n is the weakest team.)
 * We'll use parentheses('(', ')') and commas(',') to represent the contest team pairing
- parentheses('(' , ')')
 * for pairing and commas(',') for partition. During the pairing process in each round,
 * you always need to follow the strategy of making the rather strong one pair with the
rather weak one.

 Example 1:
 Input: 2
 Output: (1,2)

 Explanation:
 Initially, we have the team 1 and the team 2, placed like: 1,2.
 Then we pair the team (1,2) together with '(', ')' and ',', which is the final answer.


 Example 2:
 Input: 4
 Output: ((1,4),(2,3))

 Explanation:
 In the first round, we pair the team 1 and 4, the team 2 and 3 together, as we need to
make the strong team and weak team together.
 And we got (1,4),(2,3).
 In the second round, the winners of (1,4) and (2,3) need to play again to generate the
final winner, so you need to add the paratheses outside them.
 And we got the final answer ((1,4),(2,3)).


 Example 3:
 Input: 8
 Output: (((1,8),(4,5)),((2,7),(3,6)))

 Explanation:
 First round: (1,8),(2,7),(3,6),(4,5)
 Second round: ((1,8),(4,5)),((2,7),(3,6))
 Third round: (((1,8),(4,5)),((2,7),(3,6)))
```

```
 Since the third round will generate the final winner, you need to output the answer
(((1,8),(4,5)),((2,7),(3,6))).

 Note:
 The n is in range [2, 212].
 We ensure that the input n can be converted into the form 2k, where k is a positive
integer.
 */
public class _544 {

    public String findContestMatch(int n) {
        List<String> pairs = new ArrayList<>();
        int left = 1;
        int right = n;
        while (left < right) {
            pairs.add("(" + left + "," + right + ")");
            left++;
            right--;
        }
        if (n == 2) {
            return pairs.get(0);
        }
        return generateFinal(pairs, n / 2);
    }

    private String generateFinal(List<String> pairs, int n) {
        if (n > 2) {
            int size = pairs.size();
            int left = 0;
            int right = size - 1;
            List<String> newPairs = new ArrayList<>();
            while (left < right) {
                String newPair = "(" + pairs.get(left) + "," + pairs.get(right) + ")";
                newPairs.add(newPair);
                left++;
                right--;
            }
            return generateFinal(newPairs, n / 2);
        }
        return "(" + pairs.get(0) + "," + pairs.get(1) + ")";
    }

}
```

511. `_545.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 545. Boundary of Binary Tree
 *
 * Given a binary tree, return the values of its boundary in anti-clockwise direction
starting from root.
 * Boundary includes left boundary, addLeaves, and right boundary in order without
duplicate nodes.
 * Left boundary is defined as the path from root to the left-most node.
 * Right boundary is defined as the path from root to the right-most node.
 * If the root doesn't have left subtree or right subtree,
 * then the root itself is left boundary or right boundary.
 * Note this definition only applies to the input binary tree, and not applies to any
subtrees.
 * The left-most node is defined as a leaf node you could reach when you always firstly
travel
 * to the left subtree if exists. If not, travel to the right subtree.
 * Repeat until you reach a leaf node.
 * The right-most node is also defined by the same way with left and right exchanged.

 Example 1
 Input:
  1
   \
   2
  / \
 3   4

 Ouput:
 [1, 3, 4, 2]

 Explanation:
 The root doesn't have left subtree, so the root itself is left boundary.
 The addLeaves are node 3 and 4.
 The right boundary are node 1,2,4. Note the anti-clockwise direction means you should
output reversed right boundary.
 So order them in anti-clockwise without duplicates and we have [1,3,4,2].


 Example 2
 Input:
     ____1_____
   /           \
  2             3
 / \           /
```

```
4   5        6
   / \      / \
  7  8     9  10

 Ouput:
 [1,2,4,7,8,9,10,6,3]

 Explanation:
 The left boundary are node 1,2,4. (4 is the left-most node according to definition)
 The addLeaves are node 4,7,8,9,10.
 The right boundary are node 1,3,6,10. (10 is the right-most node).
 So order them in anti-clockwise without duplicate nodes we have [1,2,4,7,8,9,10,6,3].

 */
public class _545 {
    public static class Solution1 {
        public List<Integer> boundaryOfBinaryTree(TreeNode root) {
            List<Integer> nodes = new ArrayList<>();
            if (root == null) {
                return nodes;
            }

            nodes.add(root.val);
            leftBoundary(root.left, nodes);
            addLeaves(root.left, nodes);
            addLeaves(root.right, nodes);
            rightBoundary(root.right, nodes);
            return nodes;
        }

        public void leftBoundary(TreeNode root, List<Integer> nodes) {
            if (root == null || (root.left == null && root.right == null)) {
                /**we don't want to add any LEAVES in leftBoundary and rightBoundary
functions either,
                 * that's why we have the later condition in the if branch.*/
                return;
            }
            nodes.add(root.val);// add BEFORE child visit
            if (root.left == null) {
                leftBoundary(root.right, nodes);
            } else {
                leftBoundary(root.left, nodes);
            }
        }

        public void rightBoundary(TreeNode root, List<Integer> nodes) {
            if (root == null || (root.right == null && root.left == null)) {
                return;
            }
            if (root.right == null) {
                rightBoundary(root.left, nodes);
            } else {
                rightBoundary(root.right, nodes);
            }
```

```
            nodes.add(root.val); // add AFTER child visit(reverse)
        }

        public void addLeaves(TreeNode root, List<Integer> nodes) {
            if (root == null) {
                return;
            }
            if (root.left == null && root.right == null) {
                nodes.add(root.val);
                return;
            }
            addLeaves(root.left, nodes);
            addLeaves(root.right, nodes);
        }
    }

}
```

512. `_546.java`

```
package com.fishercoder.solutions;

/**
 * 546. Remove Boxes
 *
 * Given several boxes with different colors represented by different positive numbers.
 * You may experience several rounds to remove boxes until there is no box left.
 * Each time you can choose some continuous boxes with the same color (composed of k
boxes, k >= 1), remove them and get k*k points.
 * Find the maximum points you can get.

 Example 1:
 Input:

 [1, 3, 2, 2, 2, 3, 4, 3, 1]
 Output:
 23

 Explanation:
 [1, 3, 2, 2, 2, 3, 4, 3, 1]
 ----> [1, 3, 3, 4, 3, 1] (3*3=9 points)
 ----> [1, 3, 3, 3, 1] (1*1=1 points)
 ----> [1, 1] (3*3=9 points)
 ----> [] (2*2=4 points)
 Note: The number of boxes n would not exceed 100.

 */

public class _546 {
    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/remove-boxes/#approach-2-using-dp-with-
memorizationaccepted
         *
         * For an entry in dp[l][r][k], l represents the starting index of the subarray,
         * r represents the ending index of the subarray
         * and k represents the number of elements similar to the rth element
         * following it which can be combined to obtain the point information to be
stored in dp[l][r][k].
         */
        public int removeBoxes(int[] boxes) {
            int[][][] dp = new int[100][100][100];
            return calculatePoints(boxes, dp, 0, boxes.length - 1, 0);
        }

        public int calculatePoints(int[] boxes, int[][][] dp, int l, int r, int k) {
            if (l > r) {
                return 0;
            }
            if (dp[l][r][k] != 0) {
                return dp[l][r][k];
```

```
            }
            while (r > l && boxes[r] == boxes[r - 1]) {
                r--;
                k++;
            }
            dp[l][r][k] = calculatePoints(boxes, dp, l, r - 1, 0) + (k + 1) * (k + 1);
            for (int i = l; i < r; i++) {
                if (boxes[i] == boxes[r]) {
                    dp[l][r][k] = Math.max(dp[l][r][k],
                            calculatePoints(boxes, dp, l, i, k + 1) +
calculatePoints(boxes, dp, i + 1, r - 1, 0));
                }
            }
            return dp[l][r][k];
        }
    }
}
```

513. `_547.java`

```
package com.fishercoder.solutions;

/**
 *547. Friend Circles
 *
 *There are N students in a class.
 * Some of them are friends, while some are not. Their friendship is transitive in
nature.
 * For example, if A is a direct friend of B, and B is a direct friend of C, then A is an
indirect friend of C.
 * And we defined a friend circle is a group of students who are direct or indirect
friends.

 Given a N*N matrix m representing the friend relationship between students in the class.
 If m[i][j] = 1, then the ith and jth students are direct friends with each other,
otherwise not.
 And you have to output the total number of friend circles among all the students.

 Example 1:
 Input:
 [[1,1,0],
 [1,1,0],
 [0,0,1]]
 Output: 2
 Explanation:The 0th and 1st students are direct friends, so they are in a friend circle.
 The 2nd student himself is in a friend circle. So return 2.

 Example 2:
 Input:
 [[1,1,0],
 [1,1,1],
 [0,1,1]]
 Output: 1
 Explanation:The 0th and 1st students are direct friends, the 1st and 2nd students are
direct friends,
 so the 0th and 2nd students are indirect friends. All of them are in the same friend
circle, so return 1.

 Note:
 N is in range [1,200].
 m[i][i] = 1 for all students.
 If m[i][j] = 1, then m[j][i] = 1.
 */
public class _547 {

    public static class Solution1 {
        public int findCircleNum(int[][] M) {
            if (M == null || M.length == 0 || M[0].length == 0) {
                return 0;
            }
            int m = M.length;//number of rows in this matrix
```

```
            UnionFind unionFind = new UnionFind(m);
            for (int i = 0; i < m; i++) {
                for (int j = i + 1; j < m; j++) {
                    if (M[i][j] == 1) {
                        unionFind.union(i, j);
                    }
                }
            }
            return unionFind.count;
        }

        class UnionFind {
            int count;
            int[] root;

            public UnionFind(int m) {
                root = new int[m];
                for (int i = 0; i < m; i++) {
                    root[i] = i;
                }
                count = m;
            }

            public void union(int i, int j) {
                int x = find(root, i);
                int y = find(root, j);
                if (x != y) {
                    count--;
                    root[x] = y;//path compression
                }
            }

            public int find(int[] ids, int i) {
                if (ids[i] == i) {
                    return i;
                }
                return find(ids, ids[i]);
            }
        }
    }

}
```

514. `_548.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 548. Split Array with Equal Sum
 *
 * Given an array with n integers, you need to find if there are triplets (i, j, k) which satisfies following conditions:
 * 0 < i, i + 1 < j, j + 1 < k < n - 1
 * Sum of subarrays (0, i - 1), (i + 1, j - 1), (j + 1, k - 1) and (k + 1, n - 1) should be equal.
 * where we define that subarray (L, R) represents a slice of the original array starting from the element indexed L to the element indexed R.

 Example:
 Input: [1,2,1,2,1,2,1]
 Output: True
 Explanation:
 i = 1, j = 3, k = 5.
 sum(0, i - 1) = sum(0, 0) = 1
 sum(i + 1, j - 1) = sum(2, 2) = 1
 sum(j + 1, k - 1) = sum(4, 4) = 1
 sum(k + 1, n - 1) = sum(6, 6) = 1

 Note:
 1 <= n <= 2000.
 Elements in the given array will be in range [-1,000,000, 1,000,000].
 */
public class _548 {

    public static class Solution1 {
        public boolean splitArray(int[] nums) {
            int len = nums.length;
            if (len < 7) {
                return false;
            }
            int[] sum = new int[len];
            sum[0] = nums[0];
            for (int i = 1; i < len; i++) {
                sum[i] = sum[i - 1] + nums[i];
            }
            for (int j = 3; j < len - 3; j++) {
                Set<Integer> set = new HashSet<>();
                for (int i = 1; i < j - 1; i++) {
                    if (sum[i - 1] == sum[j - 1] - sum[i]) {
                        /**this is sum(0, i-1) and sum(i+1, j-1)*/
                        set.add(sum[i - 1]);
                    }
                }
```

```
                for (int k = j + 2; k < len - 1; k++) {
                    if (sum[k - 1] - sum[j] == sum[len - 1] - sum[k] &&
set.contains(sum[k - 1] - sum[j])) {
                        /**this is sum(j+1, k-1) and sum(k+1, len-1)*/
                        return true;
                    }
                }
            }
            return false;
        }
    }

}
```

515. `_549.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 549. Binary Tree Longest Consecutive Sequence II
 *
 * Given a binary tree, you need to find the length of Longest Consecutive Path in Binary
Tree.
 Especially, this path can be either increasing or decreasing. For example, [1,2,3,4] and
[4,3,2,1] are both considered valid,
 but the path [1,2,4,3] is not valid. On the other hand, the path can be in the child-
Parent-child order, where not necessarily be parent-child order.

 Example 1:
 Input:
   1
  / \
 2   3
 Output: 2

 Explanation: The longest consecutive path is [1, 2] or [2, 1].
 Example 2:
 Input:
   2
  / \
 1   3
 Output: 3
 Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].
 Note: All the values of tree nodes are in the range of [-1e7, 1e7].
 */
public class _549 {

    public static class Solution1 {
        int max = 0;

        public int longestConsecutive(TreeNode root) {
            longestPath(root);
            return max;
        }

        private int[] longestPath(TreeNode root) {
            if (root == null) {
                return new int[]{0, 0};
            }
            int increasing = 1;
            int decreasing = 1;
            if (root.left != null) {
                int[] left = longestPath(root.left);
                if (root.val == root.left.val + 1) {
                    decreasing = left[1] + 1;
```

```
                } else if (root.val == root.left.val - 1) {
                    increasing = left[0] + 1;
                }
            }

            if (root.right != null) {
                int[] right = longestPath(root.right);
                if (root.val == root.right.val + 1) {
                    decreasing = Math.max(right[1] + 1, decreasing);
                } else if (root.val == root.right.val - 1) {
                    increasing = Math.max(right[0] + 1, increasing);
                }
            }

            max = Math.max(max, decreasing + increasing - 1);
            return new int[]{increasing, decreasing};
        }
    }
}
```

516. `_551.java`

```
package com.fishercoder.solutions;

/**
 * 551. Student Attendance Record I
 *
 * You are given a string representing an attendance record for a student. The record
only contains the following three characters:

 'A' : Absent.
 'L' : Late.
 'P' : Present.
 A student could be rewarded if his attendance record doesn't contain more than one 'A'
(absent) or more than two continuous 'L' (late).

 You need to return whether the student could be rewarded according to his attendance
record.

 Example 1:
 Input: "PPALLP"
 Output: True

 Example 2:
 Input: "PPALLL"
 Output: False

 */
public class _551 {

    public static class Solution1 {
        public boolean checkRecord(String s) {
            int aCount = 0;
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == 'A') {
                    aCount++;
                    if (aCount > 1) {
                        return false;
                    }
                } else if (s.charAt(i) == 'L') {
                    int continuousLCount = 0;
                    while (i < s.length() && s.charAt(i) == 'L') {
                        i++;
                        continuousLCount++;
                        if (continuousLCount > 2) {
                            return false;
                        }
                    }
                    i--;
                }
            }
            return true;
        }
```

```
    }

}
```

517. `_552.java`

```
package com.fishercoder.solutions;

/**
 * 552. Student Attendance Record II
 *
 * Given a positive integer n, return the number of all possible attendance records with length n,
 * which will be regarded as rewardable. The answer may be very large, return it after mod 109 + 7.

 A student attendance record is a string that only contains the following three characters:

 'A' : Absent.
 'L' : Late.
 'P' : Present.

 A record is regarded as rewardable if it doesn't contain more than one 'A' (absent) or more than two continuous 'L' (late).

 Example 1:

 Input: n = 2
 Output: 8

 Explanation:
 There are 8 records with length 2 will be regarded as rewardable:
 "PP" , "AP", "PA", "LP", "PL", "AL", "LA", "LL"
 Only "AA" won't be regarded as rewardable owing to more than one absent times.

 Note: The value of n won't exceed 100,000.
 */
public class _552 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/86526/improving-the-runtime-from-o-n-to-o-log-n
         */
        public int checkRecord(int n) {
            final int MOD = 1000000007;
            int[][][] f = new int[n + 1][2][3];

            f[0] = new int[][]{{1, 1, 1}, {1, 1, 1}};
            for (int i = 1; i <= n; i++) {
                for (int j = 0; j < 2; j++) {
                    for (int k = 0; k < 3; k++) {
                        int val = f[i - 1][j][2]; // ...P
                        if (j > 0) {
                            val = (val + f[i - 1][j - 1][2]) % MOD; // ...A
                        }
```

```
                        if (k > 0) {
                            val = (val + f[i - 1][j][k - 1]) % MOD; // ...L
                        }
                        f[i][j][k] = val;
                    }
                }
            }
            return f[n][1][2];
        }
    }

}
```

518. `_553.java`

```
package com.fishercoder.solutions;

import java.util.StringJoiner;

/**
 * 553. Optimal Division
 *
 * Given a list of positive integers, the adjacent integers will perform the float
division. For example, [2,3,4] -> 2 / 3 / 4.
 However, you can add any number of parenthesis at any position to change the priority of
operations.
 You should find out how to add parenthesis to get the maximum result,
 and return the corresponding expression in string format.
 Your expression should NOT contain redundant parenthesis.

 Example:
 Input: [1000,100,10,2]
 Output: "1000/(100/10/2)"
 Explanation:
 1000/(100/10/2) = 1000/((100/10)/2) = 200
 However, the bold parenthesis in "1000/((100/10)/2)" are redundant,
 since they don't influence the operation priority. So you should return
"1000/(100/10/2)".

 Other cases:
 1000/(100/10)/2 = 50
 1000/(100/(10/2)) = 50
 1000/100/10/2 = 0.5
 1000/100/(10/2) = 2
 Note:

 The length of the input array is [1, 10].
 Elements in the given array will be in range [2, 1000].
 There is only one optimal division for each test case.
 */
public class _553 {
    public static class Solution1 {
        /**
         * Credit: https://github.com/lydxlx1/LeetCode/blob/master/src/_553.java
         */
        public String optimalDivision(int[] nums) {
            /**https://docs.oracle.com/javase/8/docs/api/java/util/StringJoiner.html:
             * StringJoiner is used to construct a sequence of characters separated by a
delimiter and optionally starting with a supplied prefix and ending with a supplied
suffix.
             * The String "[George:Sally:Fred]" may be constructed as follows:
             StringJoiner sj = new StringJoiner(":", "[", "]");
             sj.add("George").add("Sally").add("Fred");
             String desiredString = sj.toString();*/

            if (nums.length == 1) {
```

```
                return "" + nums[0];
            }
            if (nums.length == 2) {
                return nums[0] + "/" + nums[1];
            }

            /**Tricky one: the solution is fixed: always wrap the one from the second
until the last.
             * Another important thing to note that such way could work is that:
             * the prerequisite is: Elements will be in range [2,1000], so no elements
are smaller than 1.*/
            StringJoiner stringJoiner = new StringJoiner("/");
            for (int i = 1; i < nums.length; i++) {
                stringJoiner.add("" + nums[i]);
            }
            return String.format("%d/(%s)", nums[0], stringJoiner.toString());
        }
    }

}
```

519. `_554.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 554. Brick Wall
 *
 * There is a brick wall in front of you. The wall is rectangular and has several rows of
bricks.
 * The bricks have the same height but different width.
 * You want to draw a vertical line from the top to the bottom and cross the least
bricks.
 * The brick wall is represented by a list of rows.
 * Each row is a list of integers representing the width of each brick in this row from
left to right.
 * If your line go through the edge of a brick,
 * then the brick is not considered as crossed.
 * You need to find out how to draw the line to cross the least bricks and return the
number of crossed bricks.
 * You cannot draw a line just along one of the two vertical edges of the wall, in which
case the line will obviously cross no bricks.

 Example:
 Input:
 [[1,2,2,1],
 [3,1,2],
 [1,3,2],
 [2,4],
 [3,1,2],
 [1,3,1,1]]

 Output: 2
 Explanation:

 Note:
 The width sum of bricks in different rows are the same and won't exceed INT_MAX.
 The number of bricks in each row is in range [1,10,000]. The height of wall is in range
[1,10,000]. Total number of bricks of the wall won't exceed 20,000.
 */
public class _554 {
    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/brick-wall/
         *
         * we make use of a HashMap
         * map which is used to store entries in the form:
         * (sum,count). Here,
         * sum refers to the cumulative sum of the bricks' widths encountered in the
current row, and
```

```
         * count refers to the number of times the corresponding sum is obtained. Thus,
         * sum in a way, represents the positions of the bricks's boundaries relative to
the leftmost boundary.
         *
         * This is done based on the following observation:
         * We will never obtain the same value of sum twice while traversing over a
particular row.
         * Thus, if the sum value is repeated while traversing over the rows, it means
some row's brick boundary coincides with some previous row's brick boundary.
         * This fact is accounted for by incrementing the corresponding count value.
         * But, for every row, we consider the sum only upto the second last brick, since
the last boundary isn't a valid boundary for the solution.
         */

        public int leastBricks(List<List<Integer>> wall) {
            Map<Integer, Integer> map = new HashMap();
            for (List<Integer> row : wall) {
                int sum = 0;
                for (int i = 0; i < row.size() - 1; i++) {
                    //NOTE: i < row.size()-1
                    sum += row.get(i);
                    if (map.containsKey(sum)) {
                        map.put(sum, map.get(sum) + 1);
                    } else {
                        map.put(sum, 1);
                    }
                }
            }
            int result = wall.size();
            for (int key : map.keySet()) {
                result = Math.min(result, wall.size() - map.get(key));
            }
            return result;
        }
    }
}
```

520. `_555.java`

```
package com.fishercoder.solutions;

/**
 * 555. Split Concatenated Strings
 *
 * Given a list of strings, you could concatenate these strings together into a loop,
 * where for each string you could choose to reverse it or not.
 *
 * Among all the possible loops, you need to find the lexicographically biggest string
after cutting the loop,
 * which will make the looped string into a regular one.
 *
 * Specifically, to find the lexicographically biggest string, you need to experience two
phases:
 *
 * 1. Concatenate all the strings into a loop, where you can reverse some strings or not
and connect them in the same order as given.
 * 2. Cut and make one breakpoint in any place of the loop, which will make the looped
string into a regular one starting from
 * the character at the cutpoint.
 *
 * And your job is to find the lexicographically biggest one among all the possible
regular strings.

 Example:
 Input: "abc", "xyz"
 Output: "zyxcba"

 Explanation: You can get the looped string "-abcxyz-", "-abczyx-", "-cbaxyz-", "-cbazyx-
",
 where '-' represents the looped status.
 The answer string came from the fourth looped one,
 where you could cut from the middle character 'a' and get "zyxcba".

 Note:
 The input strings will only contain lowercase letters.
 The total length of all the strings will not over 1,000.

 */
public class _555 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/86477/neat-java-solution
         * and article: https://leetcode.com/articles/split-assembled-strings/#approach-
3-optimized-solution-accepted
         */
        public String splitLoopedString(String[] strs) {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < strs.length; i++) {
```

```
                sb.setLength(0);
                String reverse = sb.append(strs[i]).reverse().toString();
                if (strs[i].compareTo(reverse) < 0) {
                    strs[i] = reverse;
                }
            }
            String result = "";
            for (int i = 0; i < strs.length; i++) {
                sb.setLength(0);
                String reverse = sb.append(strs[i]).reverse().toString();
                for (String str : new String[]{strs[i], reverse}) {
                    for (int k = 0; k < str.length(); k++) {
                        sb.setLength(0);
                        sb.append(str.substring(k));
                        for (int j = i + 1; j < strs.length; j++) {
                            sb.append(strs[j]);
                        }
                        for (int j = 0; j < i; j++) {
                            sb.append(strs[j]);
                        }
                        sb.append(str.substring(0, k));
                        if (sb.toString().compareTo(result) > 0) {
                            result = sb.toString();
                        }
                    }
                }
            }
            return result;
        }
    }
}
```

521. `_556.java`

```
package com.fishercoder.solutions;

/**
 * 556. Next Greater Element III
 *
 * Given a positive 32-bit integer n, you need to find the smallest 32-bit integer which
has exactly the same digits existing
 * in the integer n and is greater in value than n. If no such positive 32-bit integer
exists, you need to return -1.
 *
 * Example 1:
 * Input: 12
 * Output: 21
 *
 * Example 2:
 * Input: 21
 * Output: -1
 */
public class _556 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/85759/this-problem-is-the-same-to-
next-permutation-algorithm-only and https://discuss.leetcode.com/topic/85755/java-
solution-like-next-permutation-problem-o-n
         */

        public int nextGreaterElement(int n) {
            char[] digits = String.valueOf(n).toCharArray();
            int i = digits.length - 2;
            while (i >= 0 && digits[i + 1] <= digits[i]) {
                i--;
            }
            if (i < 0) {
                return -1;
            }
            int j = digits.length - 1;
            while (j >= 0 && digits[j] <= digits[i]) {
                j--;
            }
            swap(digits, i, j);
            reverse(digits, i + 1);
            try {
                return Integer.parseInt(new String(digits));
            } catch (Exception e) {
                return -1;
            }
        }

        private void reverse(char[] a, int start) {
            int i = start;
            int j = a.length - 1;
```

```
            while (i < j) {
                swap(a, i, j);
                i++;
                j--;
            }
        }

        private void swap(char[] a, int i, int j) {
            char temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

}
```

522. `_557.java`

```
package com.fishercoder.solutions;

/**
 * 557. Reverse Words in a String III
 *
 * Given a string, you need to reverse the order of characters in each word within a
sentence while still preserving whitespace and initial word order.

 Example 1:
 Input: "Let's take LeetCode contest"
 Output: "s'teL ekat edoCteeL tsetnoc"
 Note: In the string, each word is separated by single space and there will not be any
extra space in the string.
 */
public class _557 {

    public static class Solution1 {
        public String reverseWords(String s) {
            StringBuilder stringBuilder = new StringBuilder();
            StringBuilder sbUtils = new StringBuilder();
            for (String word : s.split(" ")) {
                sbUtils.setLength(0);
                sbUtils.append(word);
                stringBuilder.append(sbUtils.reverse().toString());
                stringBuilder.append(" ");
            }
            stringBuilder.setLength(stringBuilder.length() - 1);
            return stringBuilder.toString();
        }
    }
}
```

523. `_559.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.List;

/**
 * 559. Maximum Depth of N-ary Tree
 *
 * Given a n-ary tree, find its maximum depth.
 *
 * The maximum depth is the number of nodes along the longest path from the root node
down to the farthest leaf node.
 *
 * For example, given a 3-ary tree:
 *                            1
 *                          / | \
 *                         3  2  4
 *                        / \
 *                       5  6
 *
 * We should return its max depth, which is 3.
 *
 * Note:
 *
 * The depth of the tree is at most 1000.
 * The total number of nodes is at most 5000.
 */
public class _559 {
  public static class Solution1 {
    public int maxDepth(Node root) {
      int maxDepth = 0;
      if (root == null) {
        return maxDepth;
      }
      List<List<Integer>> allPaths = new ArrayList<>();
      List<Integer> currentPath = new ArrayList<>();
      dfs(root, currentPath, allPaths);
      for (List<Integer> path : allPaths) {
        maxDepth = Math.max(path.size(), maxDepth);
      }
      return maxDepth;
    }

    private void dfs(Node root, List<Integer> currentPath, List<List<Integer>> allPaths)
{
      if (root == null) {
        allPaths.add(new ArrayList<>(currentPath));
      }
      if (root.children != null && !root.children.isEmpty()) {
        currentPath.add(root.val);
```

```
        for (Node child : root.children) {
          dfs(child, new ArrayList<>(currentPath), allPaths);
        }
      }
      if (root.children == null || root.children.isEmpty()) {
        currentPath.add(root.val);
        allPaths.add(new ArrayList<>(currentPath));
      }
    }
  }
}
```

524. `_560.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 560. Subarray Sum Equals K
 *
 * Given an array of integers and an integer k, you need to find the total number of
continuous subarrays whose sum equals to k.

 Example 1:
 Input:nums = [1,1,1], k = 2
 Output: 2
 Note:
 The length of the array is in range [1, 20,000].
 The range of numbers in the array is [-1000, 1000] and the range of the integer k is
[-1e7, 1e7].
 */
public class _560 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/87850/java-solution-presum-hashmap
         * We know the key to solve this problem is SUM[i, j].
         * So if we know SUM[0, i - 1] and SUM[0, j],
         * then we can easily get SUM[i, j] via (SUM[0, j] - SUM[0, i-1]).
         * To achieve this, we just need to go through the array,
         * calculate the current sum and save number of all seen PreSum to a HashMap.
         * <p>
         * Time complexity O(n), Space complexity O(n).
         */
        public int subarraySum(int[] nums, int k) {
            Map<Integer, Integer> preSum = new HashMap();
            int sum = 0;
            int result = 0;
            preSum.put(0, 1);
            for (int i = 0; i < nums.length; i++) {
                sum += nums[i];
                if (preSum.containsKey(sum - k)) {
                    result += preSum.get(sum - k);
                }
                preSum.put(sum, preSum.getOrDefault(sum, 0) + 1);
            }
            return result;
        }
    }

}
```

525. `_561.java`

```
package com.fishercoder.solutions;

/**
 * Created by fishercoder on 4/23/17.
 */

import java.util.Arrays;

/**
 * Array Partition I:
 *
 *
 * Given an array of 2n integers, your task is to group these integers into n pairs of integer,
 * say (a1, b1), (a2, b2), ..., (an, bn) which makes sum of min(ai, bi) for all i from 1 to n as large as possible.

 Example 1:
 Input: [1,4,3,2]

 Output: 4
 Explanation: n is 2, and the maximum sum of pairs is 4.

 Note:
 n is a positive integer, which is in the range of [1, 10000].
 All the integers in the array will be in the range of [-10000, 10000].*/
public class _561 {

    public static class Solution1 {
        public int arrayPairSum(int[] nums) {
            Arrays.sort(nums);
            int sum = 0;
            for (int i = 0; i < nums.length; i += 2) {
                sum += nums[i];
            }
            return sum;
        }
    }

}
```

526. `_562.java`

```
package com.fishercoder.solutions;

/**
 * Created by fishercoder on 4/23/17.
 *
 * 562. Longest Line of Consecutive One in Matrix
 *
 * Given a 01 matrix m, find the longest line of consecutive one in the matrix. The line could be horizontal, vertical, diagonal or anti-diagonal.

 Example:

 Input:
 [[0,1,1,0],
  [0,1,1,0],
  [0,0,0,1]]

 Output: 3
 Hint: The number of elements in the given matrix will not exceed 10,000.
 */
public class _562 {

    public static class Solution1 {
        public int longestLine(int[][] M) {
            if (M == null || M.length == 0) {
                return 0;
            }
            int[][] directions = new int[][]{
                    {-1, 0},
                    {-1, 1},
                    {0, 1},
                    {1, 1},
                    {1, 0},
                    {1, -1},
                    {0, -1},
                    {-1, -1},
            };
            int longestLine = 0;
            int m = M.length;
            int n = M[0].length;
            int[][][] cache = new int[m][n][8];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (M[i][j] == 1) {
                        for (int k = 0; k < directions.length; k++) {
                            int nextI = i + directions[k][0];
                            int nextJ = j + directions[k][1];
                            int thisLine = 1;
                            if (nextI >= 0 && nextI < m && nextJ >= 0 && nextJ < n && cache[nextI][nextJ][k] != 0) {
                                thisLine += cache[nextI][nextJ][k];
```

```
                                cache[i][j][k] = thisLine;
                            } else {
                                while (nextI >= 0 && nextI < m && nextJ >= 0 && nextJ < n
&& M[nextI][nextJ] == 1) {
                                    thisLine++;
                                    cache[i][j][k] = thisLine;
                                    nextI += directions[k][0];
                                    nextJ += directions[k][1];
                                }
                            }
                            longestLine = Math.max(longestLine, thisLine);
                        }
                    }
                }
            }
            return longestLine;
        }
    }

}
```

527. `_563.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 563. Binary Tree Tilt
 *
 * Given a binary tree, return the tilt of the whole tree.
 * The tilt of a tree node is defined as the absolute difference between the sum of all
left subtree node values
 * and the sum of all right subtree node values.
 * Null node has tilt 0.
 * The tilt of the whole tree is defined as the sum of all nodes' tilt.

 Example:
 Input:
    1
  /   \
 2     3

 Output: 1

 Explanation:
 Tilt of node 2 : 0
 Tilt of node 3 : 0
 Tilt of node 1 : |2-3| = 1
 Tilt of binary tree : 0 + 0 + 1 = 1

 Note:
 The sum of node values in any subtree won't exceed the range of 32-bit integer.
 All the tilt values won't exceed the range of 32-bit integer.*/
public class _563 {
    public static class Solution1 {

        int tilt = 0;

        public int findTilt(TreeNode root) {
            findTiltDfs(root);
            return tilt;
        }

        public int findTiltDfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int leftTilt = 0;
            if (root.left != null) {
                leftTilt = findTiltDfs(root.left);
            }
            int rightTilt = 0;
            if (root.right != null) {
```

```
                rightTilt = findTiltDfs(root.right);
            }
            if (root.left == null && root.right == null) {
                return root.val;
            }
            tilt += Math.abs(leftTilt - rightTilt);
            return leftTilt + rightTilt + root.val;
        }
    }

}
```

528. `_564.java`

```
package com.fishercoder.solutions;

/**
 * 564. Find the Closest Palindrome
 *
 * Given an integer n, find the closest integer (not including itself), which is a
palindrome.

 The 'closest' is defined as absolute difference minimized between two integers.

 Example 1:

 Input: "123"
 Output: "121"

 Note:

 The input n is a positive integer represented by string, whose length will not exceed
18.
 If there is a tie, return the smaller one as answer.

 */
public class _564 {
    public static class Solution1 {

        public String nearestPalindromic(String n) {
            if (n.length() >= 2 && allNine(n)) {
                String s = "1";
                for (int i = 0; i < n.length() - 1; i++) {
                    s += "0";
                }
                s += "1";
                return s;
            }
            boolean isOdd = (n.length() % 2 != 0);
            String left = n.substring(0, (n.length() + 1) / 2);
            long[] increment = {-1, 0, +1};
            String ret = n;
            long minDiff = Long.MAX_VALUE;
            for (long i : increment) {
                String s = getPalindrom(Long.toString(Long.parseLong(left) + i), isOdd);
                if (n.length() >= 2 && (s.length() != n.length() || Long.parseLong(s) ==
0)) {
                    s = "";
                    for (int j = 0; j < n.length() - 1; j++) {
                        s += "9";
                    }
                }
                long diff = s.equals(n) ? Long.MAX_VALUE : Math.abs(Long.parseLong(s) -
Long.parseLong(n));
                if (diff < minDiff) {
```

```
                    minDiff = diff;
                    ret = s;
                }
            }
            return ret;
        }

        private String getPalindrom(String s, boolean isOdd) {
            String right = new StringBuilder(s).reverse().toString();
            return isOdd ? s.substring(0, s.length() - 1) + right : s + right;
        }

        private boolean allNine(String s) {
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) != '9') {
                    return false;
                }
            }
            return true;
        }
    }
}
```

529. `_565.java`

```
package com.fishercoder.solutions;

/**
 * 565. Array Nesting
 *
 A zero-indexed array A consisting of N different integers is given. The array contains all integers in the range [0, N - 1].

 Sets S[K] for 0 <= K < N are defined as follows:

 S[K] = { A[K], A[A[K]], A[A[A[K]]], ... }.

 Sets S[K] are finite for each K and should NOT contain duplicates.

 Write a function that given an array A consisting of N integers, return the size of the largest set S[K] for this array.

 Example 1:
 Input: A = [5,4,0,3,1,6,2]
 Output: 4
 Explanation:
 A[0] = 5, A[1] = 4, A[2] = 0, A[3] = 3, A[4] = 1, A[5] = 6, A[6] = 2.

 One of the longest S[K]:
 S[0] = {A[0], A[5], A[6], A[2]} = {5, 6, 2, 0}
 Note:
 N is an integer within the range [1, 20,000].
 The elements of A are all distinct.
 Each element of array A is an integer within the range [0, N-1].
 */
public class _565 {

    public static class Solution1 {
        public int arrayNesting(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            boolean[] visited = new boolean[nums.length];
            int answer = 0;
            for (int i : nums) {
                int count = 0;
                int j = i;
                while (j >= 0 && j < nums.length && !visited[j]) {
                    count++;
                    visited[j] = true;
                    j = nums[j];
                }
                answer = Math.max(answer, count);
            }
            return answer;
        }
```

```
    }

}
```

530. `_566.java`

```
package com.fishercoder.solutions;

/**
 * 566. Reshape the Matrix
 *
 In MATLAB, there is a very useful function called 'reshape', which can reshape a matrix into a new one with different size but keep its original data.
 You're given a matrix represented by a two-dimensional array, and two positive integers r and c representing the row number and column number of the wanted reshaped matrix, respectively.
 The reshaped matrix need to be filled with all the elements of the original matrix in the same row-traversing order as they were.
 If the 'reshape' operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

 Example 1:
 Input:
 nums =
 [[1,2],
 [3,4]]
 r = 1, c = 4
 Output:
 [[1,2,3,4]]
 Explanation:
 The row-traversing of nums is [1,2,3,4]. The new reshaped matrix is a 1 * 4 matrix, fill it row by row by using the previous list.

 Example 2:
 Input:
 nums =
 [[1,2],
 [3,4]]
 r = 2, c = 4
 Output:
 [[1,2],
 [3,4]]
 Explanation:
 There is no way to reshape a 2 * 2 matrix to a 2 * 4 matrix. So output the original matrix.

 Note:
 The height and width of the given matrix is in range [1, 100].
 The given r and c are all positive.
 */
public class _566 {

    public static class Solution1 {
        public int[][] matrixReshape(int[][] nums, int r, int c) {
            if (nums == null || nums.length == 0) {
                return nums;
            }
```

```
            int m = nums.length;
            int n = nums[0].length;
            if (r * c > m * n) {
                return nums;
            }
            int k = 0;
            int[][] result = new int[r][c];
            for (int i = 0; i < r; i++) {
                for (int j = 0; j < c; j++, k++) {
                    result[i][j] = nums[k / n][k % n];
                }
            }
            return result;
        }
    }
}
```

531. `_567.java`

```
package com.fishercoder.solutions;

/**
 * 567. Permutation in String
 *
 * Given two strings s1 and s2, write a function to return true if s2 contains the
permutation of s1.
 * In other words, one of the first string's permutations is the substring of the second
string.

 Example 1:
 Input:s1 = "ab" s2 = "eidbaooo"
 Output:True
 Explanation: s2 contains one permutation of s1 ("ba").

 Example 2:
 Input:s1= "ab" s2 = "eidboaoo"
 Output: False

 Note:
 The input strings only contain lower case letters.
 The length of both given strings is in range [1, 10,000].
 */
public class _567 {

    public static class Solution1 {
        /**
         * credit: sliding window: https://discuss.leetcode.com/topic/87845/java-
solution-sliding-window
         */
        public boolean checkInclusion(String s1, String s2) {
            int len1 = s1.length();
            int len2 = s2.length();
            if (len1 > len2) {
                return false;
            }

            int[] count = new int[26];
            for (int i = 0; i < len1; i++) {
                count[s1.charAt(i) - 'a']++;
                count[s2.charAt(i) - 'a']--;
            }

            if (allZeroes(count)) {
                return true;
            }

            for (int i = len1; i < len2; i++) {
                count[s2.charAt(i) - 'a']--;
                count[s2.charAt(i - len1) - 'a']++;
                if (allZeroes(count)) {
```

```
                    return true;
                }
            }

            return false;
        }

        private boolean allZeroes(int[] count) {
            for (int i : count) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

532. `_568.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 568. Maximum Vacation Days
 *
 * LeetCode wants to give one of its best employees the option to travel among N cities
to collect algorithm problems.
 * But all work and no play makes Jack a dull boy,
 * you could take vacations in some particular cities and weeks.
 * Your job is to schedule the traveling to maximize the number of
 * vacation days you could take, but there are certain rules and restrictions you need to
follow.

 Rules and restrictions:
 You can only travel among N cities, represented by indexes from 0 to N-1. Initially, you
are in the city indexed 0 on Monday.
 The cities are connected by flights.
 The flights are represented as a N*N matrix (not necessary symmetrical),
 called flights representing the airline status from the city i to the city j.
 If there is no flight from the city i to the city j, flights[i][j] = 0;
 Otherwise, flights[i][j] = 1. Also, flights[i][i] = 0 for all i.

 You totally have K weeks (each week has 7 days) to travel.
 You can only take flights at most once per day and can only take flights on each week's
Monday morning.
 Since flight time is so short, we don't consider the impact of flight time.
 For each city, you can only have restricted vacation days in different weeks,
 given an N*K matrix called days representing this relationship.
 For the value of days[i][j], it represents the maximum days you could take vacation in
the city i in the week j.
 You're given the flights matrix and days matrix, and you need to output the maximum
vacation days you could take during K weeks.

 Example 1:
 Input:flights = [[0,1,1],[1,0,1],[1,1,0]], days = [[1,3,1],[6,0,3],[3,3,3]]
 Output: 12
 Explanation:
 Ans = 6 + 3 + 3 = 12.

 One of the best strategies is:
 1st week : fly from city 0 to city 1 on Monday, and play 6 days and work 1 day.
 (Although you start at city 0, we could also fly to and start at other cities since it
is Monday.)
 2nd week : fly from city 1 to city 2 on Monday, and play 3 days and work 4 days.
 3rd week : stay at city 2, and play 3 days and work 4 days.


 Example 2:
 Input:flights = [[0,0,0],[0,0,0],[0,0,0]], days = [[1,1,1],[7,7,7],[7,7,7]]
```

```
 Output: 3
 Explanation:
 Ans = 1 + 1 + 1 = 3.

 Since there is no flights enable you to move to another city, you have to stay at city 0
for the whole 3 weeks.
 For each week, you only have one day to play and six days to work.
 So the maximum number of vacation days is 3.


 Example 3:
 Input:flights = [[0,1,1],[1,0,1],[1,1,0]], days = [[7,0,0],[0,7,0],[0,0,7]]
 Output: 21
 Explanation:
 Ans = 7 + 7 + 7 = 21

 One of the best strategies is:
 1st week : stay at city 0, and play 7 days.
 2nd week : fly from city 0 to city 1 on Monday, and play 7 days.
 3rd week : fly from city 1 to city 2 on Monday, and play 7 days.

 Note:
 N and K are positive integers, which are in the range of [1, 100].
 In the matrix flights, all the values are integers in the range of [0, 1].
 In the matrix days, all the values are integers in the range [0, 7].
 You could stay at a city beyond the number of vacation days,
 but you should work on the extra days, which won't be counted as vacation days.
 If you fly from the city A to the city B and take the vacation on that day,
 the deduction towards vacation days will count towards the vacation days of city B in
that week.
 We don't consider the impact of flight hours towards the calculation of vacation days.
 */
public class _568 {

    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/maximum-vacation-days/#approach-2-using-
dfs-with-memoization-accepted
         */
        public int maxVacationDays(int[][] flights, int[][] days) {
            int[][] memo = new int[flights.length][days[0].length];
            for (int[] l : memo) {
                Arrays.fill(l, Integer.MIN_VALUE);
            }
            return dfs(flights, days, 0, 0, memo);
        }

        public int dfs(int[][] flights, int[][] days, int curCity, int weekno, int[][]
memo) {
            if (weekno == days[0].length) {
                return 0;
            }
            if (memo[curCity][weekno] != Integer.MIN_VALUE) {
                return memo[curCity][weekno];
```

```
            }
            int maxvac = 0;
            for (int i = 0; i < flights.length; i++) {
                if (flights[curCity][i] == 1 || i == curCity) {
                    int vac = days[i][weekno] + dfs(flights, days, i, weekno + 1, memo);
                    maxvac = Math.max(maxvac, vac);
                }
            }
            memo[curCity][weekno] = maxvac;
            return maxvac;
        }
    }

}
```

533. `_572.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 572. Subtree of Another Tree
 *
 Given two non-empty binary trees s and t,
 check whether tree t has exactly the same structure and node values with a subtree of s.
 A subtree of s is a tree consists of a node in s and all of this node's descendants. The
tree s could also be considered as a subtree of itself.

 Example 1:
 Given tree s:

     3
    / \
   4   5
  / \
 1   2
 Given tree t:
   4
  / \
 1   2
 Return true, because t has the same structure and node values with a subtree of s.

 Example 2:
 Given tree s:

     3
    / \
   4   5
  / \
 1   2
    /
   0

 Given tree t:
   4
  / \
 1   2
 Return false.
 */
public class _572 {

    public static class Solution1 {
        public boolean isSubtree(TreeNode s, TreeNode t) {
            if (s == null && t == null) {
                return true;
            }
            boolean isSubTree = false;
```

```
            if (s != null && t != null && s.val == t.val) {
                isSubTree = isSameTree(s, t);
            }
            if (isSubTree) {
                return true;
            }
            boolean isSubTreeLeft = false;
            if (s.left != null) {
                isSubTreeLeft = isSubtree(s.left, t);
            }
            if (isSubTreeLeft) {
                return true;
            }
            boolean isSubTreeRight = false;
            if (s.right != null) {
                isSubTreeRight = isSubtree(s.right, t);
            }
            if (isSubTreeRight) {
                return true;
            }
            return false;
        }

        private boolean isSameTree(TreeNode p, TreeNode q) {
            if (p == null || q == null) {
                return p == q;
            }
            return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right,
q.right);
        }
    }

    public static class Solution2 {
        public boolean isSubtree(TreeNode s, TreeNode t) {
            if (s == null) {
                return false;
            }
            if (same(s, t)) {
                return true;
            }
            return isSubtree(s.left, t) || isSubtree(s.right, t);
        }

        private boolean same(TreeNode s, TreeNode t) {
            if (s == null || t == null) {
                return s == t;
            }
            if (s.val != t.val) {
                return false;
            }
            return same(s.left, t.left) && same(s.right, t.right);
        }
```

```
    }
}
```

534. `_573.java`

```
package com.fishercoder.solutions;

/**
 * 573. Squirrel Simulation
 *
 * There's a tree, a squirrel, and several nuts.
 * Positions are represented by the cells in a 2D grid.
 * Your goal is to find the minimal distance for the squirrel to collect all the nuts and
 * put them under the tree one by one.
 * The squirrel can only take at most one nut at one time and can move in four directions
-
 * up, down, left and right, to the adjacent cell.
 * The distance is represented by the number of moves.

 Example 1:

 Input:
 Height : 5
 Width : 7
 Tree position : [2,2]
 Squirrel : [4,4]
 Nuts : [[3,0], [2,5]]
 Output: 12

 Explanation:

 Note:

 All given positions won't overlap.
 The squirrel can take at most one nut at one time.
 The given positions of nuts have no order.
 Height and width are positive integers. 3 <= height * width <= 10,000.
 The given positions contain at least one nut, only one tree and one squirrel.
 */
public class _573 {

    public static class Solution1 {

        /**
         * reference: https://leetcode.com/articles/squirrel-simulation
         *
         * 1. The order in which to pick the nuts does not matter except the first one
         * because for all the other nuts, the squirrel needs to travel back and forth.
         *
         * 2. For the first nut to be picked, there's some distance we might be able to
save, what is this distance?
         * It is the difference between the squirrel's original starting point to the
first nut and that the distance from this
         * first nut to the tree.
         * This is because, only for the first nut, the squirrel does NOT need to travel
back and forth, it only needs to travel from
```

```
         * its starting position to the nut position and then return to the tree.
         *
         * 3. For the rest of all other nuts, the squirrel always needs to go back and
forth.
         *
         * 4. So how can we find the first nut to go to so that we could have the maximum
saved distance?
         * We iterate through all of the nuts and keep updating the savedDist as below:
         */
        public int minDistance(int height, int width, int[] tree, int[] squirrel, int[][]
nuts) {
            int totalDist = 0;
            int savedDist = Integer.MIN_VALUE;
            for (int[] nut : nuts) {
                totalDist += (getDist(nut, tree) * 2);//it needs to travel back and
forth, so times two
                savedDist = Math.max(savedDist, getDist(nut, tree) - getDist(nut,
squirrel));
            }
            return totalDist - savedDist;
        }

        private int getDist(int[] pointA, int[] pointB) {
            return Math.abs(pointA[0] - pointB[0]) + Math.abs(pointA[1] - pointB[1]);
        }
    }
}
```

535. `_575.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 575. Distribute Candies
 *
 Given an integer array with even length, where different numbers in this array represent
different kinds of candies. Each number means one candy of the corresponding kind. You
need to distribute these candies equally in number to brother and sister. Return the
maximum number of kinds of candies the sister could gain.

 Example 1:
 Input: candies = [1,1,2,2,3,3]
 Output: 3
 Explanation:
 There are three different kinds of candies (1, 2 and 3), and two candies for each kind.
 Optimal distribution: The sister has candies [1,2,3] and the brother has candies
[1,2,3], too.
 The sister has three different kinds of candies.
 Example 2:
 Input: candies = [1,1,2,3]
 Output: 2
 Explanation: For example, the sister has candies [2,3] and the brother has candies
[1,1].
 The sister has two different kinds of candies, the brother has only one kind of candies.
 Note:

 The length of the given array is in range [2, 10,000], and will be even.
 The number in given array is in range [-100,000, 100,000].
 */
public class _575 {
    public static class Solution1 {
        public int distributeCandies(int[] candies) {
            Arrays.sort(candies);
            int sisCount = 0;
            for (int i = 0; i < candies.length; i++) {
                int val = candies[i];
                sisCount++;
                if (sisCount >= candies.length / 2) {
                    return candies.length / 2;
                }
                while (i < candies.length && candies[i] == val) {
                    i++;
                }
                i--;
            }
            return sisCount;
        }
    }
}
```

536. `_576.java`

```
package com.fishercoder.solutions;

/**
 * 576. Out of Boundary Paths
 *
 * There is an m by n grid with a ball.
 * Given the start coordinate (i,j) of the ball,
 * you can move the ball to adjacent cell or cross the grid boundary in four directions
(up, down, left, right).
 * However, you can at most move N times. Find out the number of paths to move the ball
out of grid boundary.
 * The answer may be very large, return it after mod 109 + 7.

 Example 1:

 Input:m = 2, n = 2, N = 2, i = 0, j = 0
 Output: 6
 Explanation:

 Example 2:

 Input:m = 1, n = 3, N = 3, i = 0, j = 1
 Output: 12
 Explanation:

 Note:

 Once you move the ball out of boundary, you cannot move it back.
 The length and height of the grid is in range [1,50].
 N is in range [0,50].

 */
public class _576 {
    public static class Solution1 {
        /**
         * reference: https://leetcode.com/articles/out-of-boundary-paths/#approach-2-
recursion-with-memoization-accepted
         */
        public int findPaths(int m, int n, int N, int x, int y) {
            int M = 1000000000 + 7;
            int[][] dp = new int[m][n];
            dp[x][y] = 1;
            int count = 0;
            for (int moves = 1; moves <= N; moves++) {
                int[][] temp = new int[m][n];
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        if (i == m - 1) {
                            count = (count + dp[i][j]) % M;
                        }
                        if (j == n - 1) {
```

```
                            count = (count + dp[i][j]) % M;
                        }
                        if (i == 0) {
                            count = (count + dp[i][j]) % M;
                        }
                        if (j == 0) {
                            count = (count + dp[i][j]) % M;
                        }
                        temp[i][j] = (((i > 0 ? dp[i - 1][j] : 0) + (i < m - 1 ? dp[i +
1][j] : 0)) % M
                                + ((j > 0 ? dp[i][j - 1] : 0) + (j < n - 1 ? dp[i][j + 1]
: 0)) % M) % M;
                    }
                }
                dp = temp;
            }
            return count;
        }
    }

}
```

537. `_581.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 581. Shortest Unsorted Continuous Subarray
 *
 * Given an integer array, you need to find one continuous subarray that if you only sort this subarray in ascending order, then the whole array will be sorted in ascending order, too.

 You need to find the shortest such subarray and output its length.

 Example 1:
 Input: [2, 6, 4, 8, 10, 9, 15]
 Output: 5
 Explanation: You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole array sorted in ascending order.

 Note:
 Then length of the input array is in range [1, 10,000].
 The input array may contain duplicates, so ascending order here means <=.

 */
public class _581 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/89282/java-o-n-time-o-1-space
         * Use start and end to keep track of the minimum subarray nums[start...end] which must be sorted for the entire array nums.
         * If start < end < 0 at the end of the for loop, then the array is already fully sorted.
         *
         * Time: O(n)
         * Space: O(1)
         */
        public int findUnsortedSubarray(int[] nums) {
            int n = nums.length;
            int start = -1;
            int end = -2;
            int min = nums[n - 1];
            int max = nums[0];
            for (int i = 1; i < n; i++) {
                max = Math.max(max, nums[i]);
                min = Math.min(min, nums[n - 1 - i]);
                if (nums[i] < max) {
                    end = i;
                }
                if (nums[n - 1 - i] > min) {
                    start = n - 1 - i;
```

```
                }
            }
            return end - start + 1;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(nlogn)
         * Space: O(n)
         */
        public int findUnsortedSubarray(int[] nums) {
            int[] clones = nums.clone();
            Arrays.sort(clones);
            int start = nums.length;
            int end = 0;
            for (int i = 0; i < nums.length; i++) {
                if (clones[i] != nums[i]) {
                    start = Math.min(start, i);
                    end = Math.max(end, i);
                }
            }
            return (end - start > 0) ? end - start + 1 : 0;
        }
    }

}
```

538. `_582.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;

/**
 * 582. Kill Process
 *
 * Given n processes, each process has a unique PID (process id) and its PPID (parent
process id).

 Each process only has one parent process, but may have one or more children processes.
This is just like a tree structure. Only one process has PPID that is 0, which means this
process has no parent process. All the PIDs will be distinct positive integers.

 We use two list of integers to represent a list of processes, where the first list
contains PID for each process and the second list contains the corresponding PPID.

 Now given the two lists, and a PID representing a process you want to kill, return a
list of PIDs of processes that will be killed in the end. You should assume that when a
process is killed, all its children processes will be killed. No order is required for
the final answer.

 Example 1:
 Input:
 pid =  [1, 3, 10, 5]
 ppid = [3, 0, 5, 3]
 kill = 5
 Output: [5,10]

 Explanation:
     3
   /   \
  1     5
       /
      10

 Kill 5 will also kill 10.

 Note:
 The given kill id is guaranteed to be one of the given PIDs.
 n >= 1.
 */
public class _582 {

    public static class Solution1 {
        public List<Integer> killProcess(List<Integer> pid, List<Integer> ppid, int kill)
```

```
{
            Map<Integer, List<Integer>> map = new HashMap<>();
            for (int i = 0; i < pid.size(); i++) {
                map.putIfAbsent(ppid.get(i), new LinkedList<>());
                map.get(ppid.get(i)).add(pid.get(i));
            }
            List<Integer> result = new LinkedList<>();
            Deque<Integer> stack = new ArrayDeque<>();
            stack.offer(kill);
            while (!stack.isEmpty()) {
                int curr = stack.poll();
                result.add(curr);
                List<Integer> list = map.get(curr);
                if (list != null) {
                    stack.addAll(list);
                }
            }
            return result;
        }
    }

}
```

539. `_583.java`

```
package com.fishercoder.solutions;

/**
 * 583. Delete Operation for Two Strings
 *
 * Given two words word1 and word2, find the minimum number of steps required to make
word1 and word2 the same, where in each step you can delete one character in either
string.

 Example 1:
 Input: "sea", "eat"
 Output: 2

 Explanation: You need one step to make "sea" to "ea" and another step to make "eat" to
"ea".

 Note:
 The length of given words won't exceed 500.
 Characters in given words can only be lower-case letters.
 */
public class _583 {

    public static class Solution1 {

        public int minDistance(String word1, String word2) {
            int m = word1.length();
            int n = word2.length();
            int[][] dp = new int[m + 1][n + 1];
            for (int i = 1; i <= m; i++) {
                for (int j = 1; j <= n; j++) {
                    dp[i][j] = word1.charAt(i - 1) == word2.charAt(j - 1) ? dp[i - 1][j -
1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
            return m + n - 2 * dp[m][n];
        }
    }

}
```

540. `_587.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Point;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 587. Erect the Fence
 *
 * There are some trees, where each tree is represented by (x,y) coordinate in a two-
dimensional garden.
 * Your job is to fence the entire garden using the minimum length of rope as it is
expensive.
 * The garden is well fenced only if all the trees are enclosed.
 * Your task is to help find the coordinates of trees which are exactly located on the
fence perimeter.

 Example 1:
 Input: [[1,1],[2,2],[2,0],[2,4],[3,3],[4,2]]
 Output: [[1,1],[2,0],[4,2],[3,3],[2,4]]
 Explanation:

 Example 2:
 Input: [[1,2],[2,2],[4,2]]
 Output: [[1,2],[2,2],[4,2]]
 Explanation:

 Even you only have trees in a line, you need to use rope to enclose them.

 Note:
 All trees should be enclosed together. You cannot cut the rope to enclose trees that
will separate them in more than one group.
 All input integers will range from 0 to 100.
 The garden has at least one tree.
 All coordinates are distinct.
 Input points have NO order. No order required for output.
 */
public class _587 {
    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/89323/java-solution-convex-hull-
algorithm-gift-wrapping-aka-jarvis-march
         * There are couple of ways to solve Convex Hull problem.
https://en.wikipedia.org/wiki/Convex_hull_algorithms
         * The following code implements Gift wrapping aka Jarvis march algorithm
         * https://en.wikipedia.org/wiki/Gift_wrapping_algorithm and
         * also added logic to handle case of multiple Points in a line
```

```
         * because original Jarvis march algorithm assumes no three points are collinear.
         * It also uses knowledge in this problem https://leetcode.com/problems/convex-
polygon.
         * Disscussion: https://discuss.leetcode.com/topic/70706/beyond-my-knowledge-
java-solution-with-in-line-explanation
         */
        public List<Point> outerTrees(Point[] points) {
            Set<Point> result = new HashSet<>();

            // Find the leftmost point
            Point first = points[0];
            int firstIndex = 0;
            for (int i = 1; i < points.length; i++) {
                if (points[i].x < first.x) {
                    first = points[i];
                    firstIndex = i;
                }
            }
            result.add(first);

            Point cur = first;
            int curIndex = firstIndex;
            do {
                Point next = points[0];
                int nextIndex = 0;
                for (int i = 1; i < points.length; i++) {
                    if (i == curIndex) {
                        continue;
                    }
                    int cross = crossProductLength(cur, points[i], next);
                    if (nextIndex == curIndex || cross > 0
                            // Handle collinear points
                            || (cross == 0 && distance(points[i], cur) > distance(next,
cur))) {
                        next = points[i];
                        nextIndex = i;
                    }
                }
                // Handle collinear points
                for (int i = 0; i < points.length; i++) {
                    if (i == curIndex) {
                        continue;
                    }
                    int cross = crossProductLength(cur, points[i], next);
                    if (cross == 0) {
                        result.add(points[i]);
                    }
                }

                cur = next;
                curIndex = nextIndex;

            } while (curIndex != firstIndex);
```

```
            return new ArrayList<>(result);
        }

        private int crossProductLength(Point A, Point B, Point C) {
            // Get the vectors' coordinates.
            int BAx = A.x - B.x;
            int BAy = A.y - B.y;
            int BCx = C.x - B.x;
            int BCy = C.y - B.y;

            // Calculate the Z coordinate of the cross product.
            return (BAx * BCy - BAy * BCx);
        }

        private int distance(Point p1, Point p2) {
            return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
        }
    }

}
```

541. `_588.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 588. Design In-Memory File System
 *
 * Design an in-memory file system to simulate the following functions:

 ls: Given a path in string format. If it is a file path, return a list that only
contains this file's name.
 If it is a directory path, return the list of file and directory names in this
directory.
 Your output (file and directory names together) should in lexicographic order.

 mkdir: Given a directory path that does not exist,
 you should make a new directory according to the path.
 If the middle directories in the path don't exist either,
 you should create them as well. This function has void return type.

 addContentToFile: Given a file path and file content in string format.
 If the file doesn't exist, you need to create that file containing given content.
 If the file already exists, you need to append given content to original content.
 This function has void return type.

 readContentFromFile: Given a file path, return its content in string format.

 Example:
 Input:
 ["FileSystem","ls","mkdir","addContentToFile","ls","readContentFromFile"]
 [[],["/"],["/a/b/c"],["/a/b/c/d","hello"],["/"],["/a/b/c/d"]]
 Output:
 [null,[],null,null,["a"],"hello"]

 Note:

 You can assume all file or directory paths are absolute paths which begin with / and do
not end with / except that the path is just "/".
 You can assume that all operations will be passed valid parameters and users will not
attempt to retrieve file content or list a directory or file that does not exist.
 You can assume that all directory names and file names only contain lower-case letters,
and same names won't exist in the same directory.

 */
public class _588 {
    public static class Solution1 {
```

```
/**
 * Credit: https://github.com/lydxlx1/LeetCode/blob/master/src/_588.java
 */
public static class FileSystem {

    class TrieNode {
        boolean isFile = false;
        Map<String, TrieNode> map;
        String name;
        StringBuilder stringBuilder;

        TrieNode(String name) {
            this.name = name;
            map = new HashMap<>();
            stringBuilder = new StringBuilder();
        }
    }

    TrieNode root = new TrieNode("");

    TrieNode dfs(String path) {
        TrieNode node = root;
        for (String eachPath : path.split("/")) {
            if (eachPath.isEmpty()) {
                continue;
            }
            if (!node.map.containsKey(eachPath)) {
                node.map.put(eachPath, new TrieNode(eachPath));
            }
            node = node.map.get(eachPath);
        }
        return node;
    }

    public FileSystem() {
    }

    public List<String> ls(String path) {
        TrieNode node = dfs(path);
        if (node.isFile) {
            return Arrays.asList(node.name);
        } else {
            List<String> files = new ArrayList(node.map.keySet());
            Collections.sort(files);
            return files;
        }
    }

    public void mkdir(String path) {
        dfs(path);
    }

    public void addContentToFile(String filePath, String content) {
        TrieNode node = dfs(filePath);
```

```
                node.isFile = true;
                node.stringBuilder.append(content);
            }

            public String readContentFromFile(String filePath) {
                return dfs(filePath).stringBuilder.toString();
            }
        }
    }

/**
 * Your FileSystem object will be instantiated and called as such:
 * FileSystem obj = new FileSystem();
 * List<String> param_1 = obj.ls(path);
 * obj.mkdir(path);
 * obj.addContentToFile(filePath,content);
 * String param_4 = obj.readContentFromFile(filePath);
 */
}
```

542. `_589.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.List;

/**
 * 589. N-ary Tree Preorder Traversal
 *
 * Given an n-ary tree, return the preorder traversal of its nodes' values.
 *
 * For example, given a 3-ary tree:
 *
 *            1
 *          / | \
 *         3  2  4
 *        / \
 *       5   6
 *
 * Return its preorder traversal as: [1,3,5,6,2,4].
 *
 * Note:
 *
 * Recursive solution is trivial, could you do it iteratively?
 */
public class _589 {
  public static class Solution1 {
    public List<Integer> preorder(Node root) {
      List<Integer> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      dfs(root, result);
      return result;
    }

    private void dfs(Node root, List<Integer> result) {
      if (root == null) {
        return;
      }
      result.add(root.val);
      if (root.children.size() > 0) {
        for (Node child : root.children) {
          dfs(child, result);
        }
      }
    }
  }
}
```

543. `_590.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.List;

/**
 * 590. N-ary Tree Postorder Traversal
 *
 * Given an n-ary tree, return the postorder traversal of its nodes' values.
 *
 * For example, given a 3-ary tree:
 *
 *              1
 *            / | \
 *           3  2  4
 *          / \
 *         5   6
 *
 * Return its postorder traversal as: [5,6,3,2,4,1].
 *
 * Note:
 *
 * Recursive solution is trivial, could you do it iteratively?
 */
public class _590 {
  public static class Solution1 {
    public List<Integer> postorder(Node root) {
      List<Integer> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      dfs(root, result);
      result.add(root.val);
      return result;
    }

    private void dfs(Node root, List<Integer> result) {
      if (root == null) {
        return;
      }
      if (root.children.size() > 0) {
        for (Node child : root.children) {
          dfs(child, result);
          result.add(child.val);
        }
      }
    }
  }
}
```

544. `_591.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 591. Tag Validator
 *
 * Given a string representing a code snippet,
 * you need to implement a tag validator to parse the code and return whether it is
valid.
 * A code snippet is valid if all the following rules hold:

 1. The code must be wrapped in a valid closed tag. Otherwise, the code is invalid.

 2. A closed tag (not necessarily valid) has exactly the following format :
<TAG_NAME>TAG_CONTENT</TAG_NAME>.
 Among them, <TAG_NAME> is the start tag, and </TAG_NAME> is the end tag.
 The TAG_NAME in start and end tags should be the same.
 A closed tag is valid if and only if the TAG_NAME and TAG_CONTENT are valid.

 3. A valid TAG_NAME only contain upper-case letters, and has length in range [1,9].
Otherwise, the TAG_NAME is invalid.

 4. A valid TAG_CONTENT may contain other valid closed tags, cdata and any characters
(see note1) EXCEPT unmatched <,
 unmatched start and end tag, and unmatched or closed tags with invalid TAG_NAME.
Otherwise, the TAG_CONTENT is invalid.

 5. A start tag is unmatched if no end tag exists with the same TAG_NAME, and vice versa.
 However, you also need to consider the issue of unbalanced when tags are nested.


6. A < is unmatched if you cannot find a subsequent >. And when you find a < or </,
 all the subsequent characters until the next > should be parsed as TAG_NAME (not
necessarily valid).

7. The cdata has the following format : <![CDATA[CDATA_CONTENT]]>.
 The range of CDATA_CONTENT is defined as the characters between <![CDATA[ and the first
subsequent ]]>.

 8. CDATA_CONTENT may contain any characters.
 The function of cdata is to forbid the validator to parse CDATA_CONTENT,
 so even it has some characters that can be parsed as tag (no matter valid or invalid),
you should treat it as regular characters.

 Valid Code Examples:
 Input: "<DIV>This is the first line <![CDATA[<div>]]></DIV>"

 Output: True
```

```
 Explanation:

 The code is wrapped in a closed tag : <DIV> and </DIV>.

 The TAG_NAME is valid, the TAG_CONTENT consists of some characters and cdata.

 Although CDATA_CONTENT has unmatched start tag with invalid TAG_NAME, it should be
considered as plain text, not parsed as tag.

 So TAG_CONTENT is valid, and then the code is valid. Thus return true.


 Input: "<DIV>>>  ![cdata[]] <![CDATA[<div>]>]]>]]>>]</DIV>"

 Output: True

 Explanation:

 We first separate the code into : start_tag|tag_content|end_tag.

 start_tag -> "<DIV>"

 end_tag -> "</DIV>"

 tag_content could also be separated into : text1|cdata|text2.

 text1 -> ">>  ![cdata[]] "

 cdata -> "<![CDATA[<div>]>]]>", where the CDATA_CONTENT is "<div>]>"

 text2 -> "]]>>]"


 The reason why start_tag is NOT "<DIV>>>" is because of the rule 6.
 The reason why cdata is NOT "<![CDATA[<div>]>]]>]]>" is because of the rule 7.
 Invalid Code Examples:
 Input: "<A>  <B> </A>   </B>"
 Output: False
 Explanation: Unbalanced. If "<A>" is closed, then "<B>" must be unmatched, and vice
versa.

 Input: "<DIV>  div tag is not closed  <DIV>"
 Output: False

 Input: "<DIV>  unmatched <  </DIV>"
 Output: False

 Input: "<DIV> closed tags with invalid tag name  <b>123</b> </DIV>"
 Output: False

 Input: "<DIV> unmatched tags with invalid tag name  </1234567890> and <CDATA[[]]>
</DIV>"
 Output: False
```

```
 Input: "<DIV>  unmatched start tag <B>  and unmatched end tag </C>  </DIV>"
 Output: False

 Note:
 For simplicity, you could assume the input code (including the any characters mentioned
above)
 only contain letters, digits, '<','>','/','!','[',']' and ' '.
 */
public class _591 {

    public static class Solution1 {

        /**
         * Credit: https://discuss.leetcode.com/topic/91300/java-solution-use-startswith-
and-indexof
         */
        public boolean isValid(String code) {
            Deque<String> stack = new ArrayDeque<>();
            for (int i = 0; i < code.length(); ) {
                if (i > 0 && stack.isEmpty()) {
                    return false;
                }
                if (code.startsWith("<![CDATA[", i)) {
                    int j = i + 9;//"<![CDATA[" length is 9
                    i = code.indexOf("]]>", j);
                    if (i < 0) {
                        return false;
                    }
                    i += 3;//"]]>" length is 3
                } else if (code.startsWith("</", i)) {
                    int j = i + 2;
                    i = code.indexOf(">", j);
                    if (i < 0 || i == j || i - j > 9) {
                        return false;
                    }
                    for (int k = j; k < i; k++) {
                        if (!Character.isUpperCase(code.charAt(k))) {
                            return false;
                        }
                    }
                    String s = code.substring(j, i++);
                    if (stack.isEmpty() || !stack.pop().equals(s)) {
                        return false;
                    }
                } else if (code.startsWith("<", i)) {
                    int j = i + 1;
                    i = code.indexOf(">", j);
                    if (i < 0 || i == j || i - j > 9) {
                        return false;
                    }
                    for (int k = j; k < i; k++) {
                        if (!Character.isUpperCase(code.charAt(k))) {
                            return false;
                        }
```

```
                    }
                    String s = code.substring(j, i++);
                    stack.push(s);
                } else {
                    i++;
                }
            }
            return stack.isEmpty();
        }
    }

}
```

545. `_592.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 592. Fraction Addition and Subtraction
 *
 * Given a string representing an expression of fraction addition and subtraction,
 * you need to return the calculation result in string format.
 * The final result should be irreducible fraction.
 * If your final result is an integer,
 * say 2, you need to change it to the format of fraction that has denominator 1.
 * So in this case, 2 should be converted to 2/1.

 Example 1:
 Input:"-1/2+1/2"
 Output: "0/1"

 Example 2:
 Input:"-1/2+1/2+1/3"
 Output: "1/3"

 Example 3:
 Input:"1/3-1/2"
 Output: "-1/6"

 Example 4:
 Input:"5/3+1/3"
 Output: "2/1"

 Note:
 The input string only contains '0' to '9', '/', '+' and '-'. So does the output.
 Each fraction (input and output) has format ±numerator/denominator.
 If the first input fraction or the output is positive, then '+' will be omitted.
 The input only contains valid irreducible fractions,
 where the numerator and denominator of each fraction will always be in the range [1,10].
 If the denominator is 1, it means this fraction is actually an integer in a fraction
format defined above.
 The number of given fractions will be in the range [1,10].
 The numerator and denominator of the final result are guaranteed to be valid and in the
range of 32-bit int.
 */
public class _592 {

    public static class Solution1 {

        /**
         * Credit: https://discuss.leetcode.com/topic/89993/java-solution-fraction-
addition-and-gcd
         */
```

```
        public String fractionAddition(String expression) {
            List<String> nums = new ArrayList<>();
            int i = 0;
            int j = 0;
            while (j <= expression.length()) {
                if (j == expression.length() || j != i && (expression.charAt(j) == '-' ||
expression.charAt(j) == '+')) {
                    if (expression.charAt(i) == '+') {
                        nums.add(expression.substring(i + 1, j));
                    } else {
                        nums.add(expression.substring(i, j));
                    }
                    i = j;
                }
                j++;
            }

            String result = "0/1";
            for (String frac : nums) {
                result = add(result, frac);
            }
            return result;
        }

        private String add(String result, String frac) {
            String[] frac1 = frac.split("/");
            String[] frac2 = result.split("/");
            int n1 = Integer.parseInt(frac1[0]);
            int d1 = Integer.parseInt(frac1[1]);
            int n2 = Integer.parseInt(frac2[0]);
            int d2 = Integer.parseInt(frac2[1]);
            int numerator = n1 * d2 + n2 * d1;
            int denominator = d1 * d2;
            if (numerator == 0) {
                return "0/1";
            }

            boolean negative = numerator * denominator < 0;
            numerator = Math.abs(numerator);
            denominator = Math.abs(denominator);
            int gcd = getGCD(numerator, denominator);

            return (negative ? "-" : "") + (numerator / gcd) + "/" + (denominator / gcd);
        }

        private int getGCD(int a, int b) {
            if (a == 0 || b == 0) {
                return a + b;
            }
            return getGCD(b, a % b);
        }
    }

}
```

546. `_593.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 593. Valid Square
 *
 * Given the coordinates of four points in 2D space, return whether the four points could
construct a square.

 The coordinate (x,y) of a point is represented by an integer array with two integers.

 Example:
 Input: p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,1]
 Output: True
 Note:
 All the input integers are in the range [-10000, 10000].
 A valid square has four equal sides with positive length and four equal angles (90-
degree angles).
 Input points have no order.

 */
public class _593 {
    public static class Solution1 {
        /**
         * Note: I don't need to use backtracking to find all permutations, this is an
overkill.
         * This is the most easy one: https://leetcode.com/articles/kill-process-
2/#approach-3-checking-every-case-accepted
         */

        public boolean validSquare(int[] p1, int[] p2, int[] p3, int[] p4) {
            List<int[]> input = new ArrayList<>(Arrays.asList(p1, p2, p3, p4));
            List<List<int[]>> allPermuations = getAllPermutations(input);
            for (List<int[]> eachPermutation : allPermuations) {
                if (isValid(eachPermutation)) {
                    return true;
                }
            }
            return false;
        }

        private List<List<int[]>> getAllPermutations(List<int[]> input) {
            List<List<int[]>> result = new ArrayList();
            List<int[]> init = new ArrayList<>();
            result.add(init);
            return backTracking(result, input, 0);
        }
```

```
        private List<List<int[]>> backTracking(List<List<int[]>> result, List<int[]>
input, int pos) {
            if (pos == input.size()) {
                return result;
            }
            List<List<int[]>> newResult = new ArrayList<>();
            for (List<int[]> eachList : result) {
                for (int i = 0; i <= eachList.size(); i++) {
                    List<int[]> newList = new ArrayList<>(eachList);
                    newList.add(i, input.get(pos));
                    newResult.add(newList);
                }
            }
            result = newResult;
            return backTracking(result, input, pos + 1);
        }

        private boolean isValid(List<int[]> points) {
            int[] p1 = points.get(0);
            int[] p2 = points.get(1);
            int[] p3 = points.get(2);
            int[] p4 = points.get(3);
            double distance = (Math.pow(p1[0] - p2[0], 2) + Math.pow(p1[1] - p2[1], 2));
            return distance == (Math.pow(p2[0] - p3[0], 2) + Math.pow(p2[1] - p3[1], 2))
                    && distance == (Math.pow(p3[0] - p4[0], 2) + Math.pow(p3[1] - p4[1],
2))
                    && distance == (Math.pow(p4[0] - p1[0], 2) + Math.pow(p4[1] - p1[1],
2))
                    && isRightAngle(p1, p2, p3)
                    && noDuplicate(p1, p2, p3, p4);
        }

        public boolean noDuplicate(int[] p1, int[] p2, int[] p3, int[] p4) {
            return !Arrays.equals(p1, p2)
                    && !Arrays.equals(p1, p3)
                    && !Arrays.equals(p1, p4)
                    && !Arrays.equals(p2, p3)
                    && !Arrays.equals(p2, p4)
                    && !Arrays.equals(p3, p4);
        }

        public boolean isRightAngle(int[] p1, int[] p2, int[] p3) {
            double angle1 = Math.atan2(p2[1] - p1[1], p2[0] - p1[0]);
            double angle2 = Math.atan2(p3[1] - p1[1], p3[0] - p1[0]);
            double degree = Math.toDegrees(angle1 - angle2);
            return degree % 45 == 0;
        }
    }

}
```

547. `_594.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 594. Longest Harmonious Subsequence
 *
 * We define a harmonious array is an array where the difference between its maximum
value and its minimum value is exactly 1.

 Now, given an integer array, you need to find the length of its longest harmonious
subsequence among all its possible subsequences.

 Example 1:
 Input: [1,3,2,2,5,2,3,7]
 Output: 5

 Explanation: The longest harmonious subsequence is [3,2,2,2,3].
 Note: The length of the input array will not exceed 20,000.

 */
public class _594 {
    public static class Solution1 {
        public int findLHS(int[] nums) {
            Map<Integer, Integer> map = new HashMap<>();
            for (int i : nums) {
                map.put(i, map.getOrDefault(i, 0) + 1);
            }
            int max = 0;
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(nums[i] + 1)) {
                    max = Math.max(max, map.get(nums[i]) + map.get(nums[i] + 1));
                }
            }
            return max;
        }
    }
}
```

548. `_598.java`

```
package com.fishercoder.solutions;

/**
 * 598. Range Addition II

 Given an m * n matrix m initialized with all 0's and several update operations.
 Operations are represented by a 2D array,
 and each operation is represented by an array with two positive integers a and b,
 which means m[i][j] should be added by one for all 0 <= i < a and 0 <= j < b.

 You need to count and return the number of maximum integers in the matrix after
performing all the operations.

 Example 1:
 Input:
 m = 3, n = 3
 operations = [[2,2],[3,3]]
 Output: 4

 Explanation:
 Initially, m =
 [[0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]

 After performing [2,2], m =
 [[1, 1, 0],
 [1, 1, 0],
 [0, 0, 0]]

 After performing [3,3], m =
 [[2, 2, 1],
 [2, 2, 1],
 [1, 1, 1]]

 So the maximum integer in m is 2, and there are four of it in m. So return 4.
 Note:
 The range of m and n is [1,40000].
 The range of a is [1,m], and the range of b is [1,n].
 The range of operations size won't exceed 10,000.
 */

public class _598 {

    public static class Solution1 {
        public int maxCount(int m, int n, int[][] ops) {
            int x = m;
            int y = n;
            for (int[] op : ops) {
                x = Math.min(x, op[0]);
                y = Math.min(y, op[1]);
```

```
            }
            return x * y;
        }
    }
}
```

549. `_599.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 599. Minimum Index Sum of Two Lists
 *
 Suppose Andy and Doris want to choose a restaurant for dinner,
 and they both have a list of favorite restaurants represented by strings.
 You need to help them find out their common interest with the least list index sum.
 If there is a choice tie between answers, output all of them with no order requirement.
You could assume there always exists an answer.

 Example 1:
 Input:
 ["Shogun", "Tapioca Express", "Burger King", "KFC"]
 ["Piatti", "The Grill at Torrey Pines", "Hungry Hunter Steakhouse", "Shogun"]
 Output: ["Shogun"]
 Explanation: The only restaurant they both like is "Shogun".

 Example 2:
 Input:
 ["Shogun", "Tapioca Express", "Burger King", "KFC"]
 ["KFC", "Shogun", "Burger King"]
 Output: ["Shogun"]
 Explanation: The restaurant they both like and have the least index sum is "Shogun" with
index sum 1 (0+1).

 Note:
 The length of both lists will be in the range of [1, 1000].
 The length of strings in both lists will be in the range of [1, 30].
 The index is starting from 0 to the list length minus 1.
 No duplicates in both lists.
 */
public class _599 {
    public static class Solution1 {
        public String[] findRestaurant(String[] list1, String[] list2) {
            if (list1 == null || list2 == null) {
                return new String[0];
            }
            Map<String, Integer> map1 = putIntoMap(list1);
            Map<String, Integer> map2 = putIntoMap(list2);
            int leastIndexSum = Integer.MAX_VALUE;
            List<String> resultList = new ArrayList<>();
            for (String key1 : map1.keySet()) {
                if (map2.containsKey(key1)) {
                    int indexSum = map1.get(key1) + map2.get(key1);
                    if (indexSum < leastIndexSum) {
```

```
                        resultList.clear();
                        resultList.add(key1);
                        leastIndexSum = indexSum;
                    } else if (indexSum == leastIndexSum) {
                        resultList.add(key1);
                    }
                }
            }
            String[] result = new String[resultList.size()];
            for (int i = 0; i < resultList.size(); i++) {
                result[i] = resultList.get(i);
            }
            return result;
        }

        private Map<String, Integer> putIntoMap(String[] list) {
            Map<String, Integer> map = new HashMap<>();
            for (int i = 0; i < list.length; i++) {
                map.put(list[i], i);
            }
            return map;
        }
    }
}
```

550. `_600.java`

```
package com.fishercoder.solutions;

/**
 * 600. Non-negative Integers without Consecutive Ones
 *
 * Given a positive integer n, find the number of non-negative integers less than or
equal to n, whose binary representations do NOT contain consecutive ones.

 Example 1:
 Input: 5
 Output: 5
 Explanation:
 Here are the non-negative integers <= 5 with their corresponding binary representations:
 0 : 0
 1 : 1
 2 : 10
 3 : 11
 4 : 100
 5 : 101
 Among them, only integer 3 disobeys the rule (two consecutive ones) and the other 5
satisfy the rule.

 Note: 1 <= n <= 109
 */
public class _600 {

    public static class Solution1 {
        /**
         * Credit: https://leetcode.com/articles/non-negative-integers-without-
consecutive-ones/#approach-3-using-bit-manipulation-accepted
         */
        public int findIntegers(int num) {
            int[] f = new int[32];
            f[0] = 1;
            f[1] = 2;
            for (int i = 2; i < f.length; i++) {
                f[i] = f[i - 1] + f[i - 2];
            }
            int i = 30;
            int sum = 0;
            int prevBit = 0;
            while (i >= 0) {
                if ((num & (1 << i)) != 0) {
                    sum += f[i];
                    if (prevBit == 1) {
                        sum--;
                        break;
                    }
                    prevBit = 1;
                } else {
                    prevBit = 0;
```

```
                }
                i--;
            }
            return sum + 1;
        }
    }

}
```

551. `_604.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 604. Design Compressed String Iterator
 *
 * Design and implement a data structure for a compressed string iterator. It should
support the following operations: next and hasNext.
 The given compressed string will be in the form of each letter followed by a positive
integer representing
 the number of this letter existing in the original uncompressed string.

 next() - if the original string still has uncompressed characters, return the next
letter; Otherwise return a white space.
 hasNext() - Judge whether there is any letter needs to be uncompressed.

 Note:
 Please remember to RESET your class variables declared in StringIterator,
 as static/class variables are persisted across multiple test cases. Please see here for
more details.

 Example:

 StringIterator iterator = new StringIterator("L1e2t1C1o1d1e1");

 iterator.next(); // return 'L'
 iterator.next(); // return 'e'
 iterator.next(); // return 'e'
 iterator.next(); // return 't'
 iterator.next(); // return 'C'
 iterator.next(); // return 'o'
 iterator.next(); // return 'd'
 iterator.hasNext(); // return true
 iterator.next(); // return 'e'
 iterator.hasNext(); // return false
 iterator.next(); // return ' '

 */
public class _604 {
    public static class Solution1 {
        public static class StringIterator {

            Deque<int[]> deque;

            public StringIterator(String compressedString) {
                deque = new ArrayDeque<>();
                int len = compressedString.length();
                int i = 0;
                while (i < len) {
```

```
                    int j = i + 1;
                    while (j < len && Character.isDigit(compressedString.charAt(j))) {
                        j++;
                    }
                    deque.addLast(new int[]{compressedString.charAt(i) - 'A',
Integer.parseInt(compressedString.substring(i + 1, j))});
                    i = j;
                }
            }

            public char next() {
                if (deque.isEmpty()) {
                    return ' ';
                }
                int[] top = deque.peek();
                top[1]--;
                if (top[1] == 0) {
                    deque.pollFirst();
                }
                return (char) ('A' + top[0]);
            }

            public boolean hasNext() {
                return !deque.isEmpty();
            }
        }
    }
}
```

552. `_605.java`

```
package com.fishercoder.solutions;

/**
 * 605. Can Place Flowers
 *
 * Suppose you have a long flowerbed in which some of the plots are planted and some are
not.
 * However, flowers cannot be planted in adjacent plots - they would compete for water
and both would die.

 Given a flowerbed (represented as an array containing 0 and 1, where 0 means empty and 1
means not empty),
 and a number n, return if n new flowers can be planted in it without violating the no-
adjacent-flowers rule.

 Example 1:
 Input: flowerbed = [1,0,0,0,1], n = 1
 Output: True

 Example 2:
 Input: flowerbed = [1,0,0,0,1], n = 2
 Output: False

 Note:
 The input array won't violate no-adjacent-flowers rule.
 The input array size is in the range of [1, 20000].
 n is a non-negative integer which won't exceed the input array size.

 */
public class _605 {

    public static class Solution1 {
        public boolean canPlaceFlowers(int[] flowerbed, int n) {
            int count = 0;
            int i = 0;
            while (i < flowerbed.length) {
                if (flowerbed[i] == 0 && (i == 0 || flowerbed[i - 1] == 0) && (i ==
flowerbed.length - 1 || flowerbed[i + 1] == 0)) {
                    count++;
                    flowerbed[i] = 1;
                }
                if (count >= n) {
                    return true;
                }
                i++;
            }
            if (count >= n) {
                return true;
            }
            return false;
        }
```

```
    }

    public static class Solution2 {
        public boolean canPlaceFlowers(int[] flowerbed, int n) {
            int len = flowerbed.length;
            if (len == 1) {
                if ((flowerbed[0] == 0 && n <= 1) || n == 0) {
                    return true;
                }
                return false;
            }
            if (flowerbed[0] == 0 && flowerbed[1] == 0) {
                flowerbed[0] = 1;
                n--;
            }
            for (int i = 1; i < len - 1; i++) {
                if (flowerbed[i] == 0 && flowerbed[i - 1] == 0 && flowerbed[i + 1] == 0)
{
                    n--;
                    flowerbed[i] = 1;//modify the input, discuss this with interviwer, if
not allowed, then have a copy of this input and modify copy
                }
                if (n <= 0) {
                    return true;
                }
            }
            if (len >= 2 && flowerbed[len - 2] == 0 && flowerbed[len - 1] == 0) {
                n--;
            }
            if (n <= 0) {
                return true;
            }
            return false;
        }
    }

}
```

553. `_606.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 606. Construct String from Binary Tree
 *
 * You need to construct a string consists of parenthesis and integers from a binary tree
with the preorder traversing way.
 The null node needs to be represented by empty parenthesis pair "()".
 And you need to omit all the empty parenthesis pairs that don't affect the one-to-one
mapping relationship between the string
 and the original binary tree.

 Example 1:
 Input: Binary tree: [1,2,3,4]
     1
   /   \
  2     3
 /
4

 Output: "1(2(4))(3)"

 Explanation: Originallay it needs to be "1(2(4())(3()())",
 but you need to omit all the unnecessary empty parenthesis pairs.
 And it will be "1(2(4))(3)".

 Example 2:
 Input: Binary tree: [1,2,3,null,4]
    1
  /   \
 2     3
  \
  4

 Output: "1(2()(4))(3)"

 Explanation: Almost the same as the first example,
 except we can't omit the first parenthesis pair to break the one-to-one mapping
relationship between the input and the output.
 */
public class _606 {

    public static class Solution1 {
        public String tree2str(TreeNode t) {
            if (t == null) {
                return "";
            }
            String result = "" + t.val;
            String left = tree2str(t.left);
```

```
            String right = tree2str(t.right);
            if (left.equals("") && right.equals("")) {
                return result;
            }
            if (left.equals("")) {
                return result + "()(" + right + ")";
            }
            if (right.equals("")) {
                return result + "(" + left + ")";
            }
            return result + "(" + left + ")(" + right + ")";
        }
    }

    public static class Solution2 {
        public String tree2str(TreeNode t) {
            if (t == null) {
                return "";
            }
            StringBuilder sb = new StringBuilder();
            preorder(t, sb);
            return sb.toString();
        }

        private void preorder(TreeNode root, StringBuilder sb) {
            if (root == null) {
                return;
            }
            sb.append(root.val);
            if (root.left != null) {
                sb.append("(");
                preorder(root.left, sb);
                sb.append(")");
            }
            if (root.right != null) {
                if (root.left == null) {
                    sb.append("()");
                }
                sb.append("(");
                preorder(root.right, sb);
                sb.append(")");
            }
        }
    }

}
```

554. `_609.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 609. Find Duplicate File in System
 *
 * Given a list of directory info including directory path, and all the files with
contents in this directory,
 * you need to find out all the groups of duplicate files in the file system in terms of
their paths.
 * A group of duplicate files consists of at least two files that have exactly the same
content.

 A single directory info string in the input list has the following format:

 "root/d1/d2/.../dm f1.txt(f1_content) f2.txt(f2_content) ... fn.txt(fn_content)"

 It means there are n files (f1.txt, f2.txt ... fn.txt with content f1_content,
f2_content ... fn_content, respectively)
 in directory root/d1/d2/.../dm. Note that n >= 1 and m >= 0. If m = 0, it means the
directory is just the root directory.

 The output is a list of group of duplicate file paths.
 For each group, it contains all the file paths of the files that have the same content.
 A file path is a string that has the following format:

 "directory_path/file_name.txt"

 Example 1:

 Input:
 ["root/a 1.txt(abcd) 2.txt(efgh)", "root/c 3.txt(abcd)", "root/c/d 4.txt(efgh)", "root
4.txt(efgh)"]

 Output:
 [["root/a/2.txt","root/c/d/4.txt","root/4.txt"],["root/a/1.txt","root/c/3.txt"]]

 Note:
 No order is required for the final output.
 You may assume the directory name,
 file name and file content only has letters and digits, and the length of file content
is in the range of [1,50].

 The number of files given is in the range of [1,20000].
 You may assume no files or directories share the same name in a same directory.

 You may assume each given directory info represents a unique directory. Directory path
```

```
and file infos are separated by a single blank space.

 Follow up beyond contest:
 Imagine you are given a real file system, how will you search files? DFS or BFS ?
 If the file content is very large (GB level), how will you modify your solution?
 If you can only read the file by 1kb each time, how will you modify your solution?
 What is the time complexity of your modified solution? What is the most time consuming
part and memory consuming part of it? How to optimize?
 How to make sure the duplicated files you find are not false positive?
 */
public class _609 {

    public static class Solution1 {
        public List<List<String>> findDuplicate(String[] paths) {
            Map<String, List<String>> map = new HashMap<>();//key is the file content,
value is the list of directories that has this directory/file
            for (String path : paths) {
                String[] dirAndFiles = path.split(" ");
                for (int i = 1; i < dirAndFiles.length; i++) {
                    String content = dirAndFiles[i].substring(dirAndFiles[i].indexOf("(")
+ 1, dirAndFiles[i].indexOf(")"));
                    if (!map.containsKey(content)) {
                        map.put(content, new ArrayList<>());
                    }
                    List<String> dirs = map.get(content);
                    dirs.add(dirAndFiles[0] + "/" + dirAndFiles[i].substring(0,
dirAndFiles[i].indexOf("(")));
                    map.put(content, dirs);
                }
            }

            List<List<String>> result = new ArrayList<>();
            for (String content : map.keySet()) {
                if (map.get(content).size() > 1) {
                    List<String> dupFile = new ArrayList<>();
                    for (String dir : map.get(content)) {
                        dupFile.add(dir);
                    }
                    result.add(dupFile);
                }
            }
            return result;
        }
    }

    /**Answers to follow-up questions:
     * 1. Imagine you are given a real file system, how will you search files? DFS or BFS
?
     * A: Both BFS and DFS could do the work, but BFS will use extra memory, however, BFS
takes advantage of memory locality, so BFS could be faster.
     *
     * 2. If the file content is very large (GB level), how will you modify your
solution?
     * A: We'll fist map all files according to their sizes, since files with different
```

```
sizes are guaranteed to be different, then
     * we can hash a small part of the files using MD5, SHA256, etc. Only when their md5
or sha256 is the same, we'll compare the contents byte by byte.
     *
     * 3. If you can only read the file by 1kb each time, how will you modify your
solution?
     * A: This is not going to change the solution, we can hash this 1kb chunk, and then
also only compare byte by byte when it's necessary.
     *
     * 4. What is the time complexity of your modified solution? What is the most time
consuming part and memory consuming part of it? How to optimize?
     * A: O(n^2*k), in the worst time, we'll have to compare the file with every other
file, k is the length of the file.
     * Comparing the file (by size, by hash and eventually byte by byte) is the most time
consuming part.
     * Generating hash for every file will be the most memory consuming part.
     *
     * 5. How to make sure the duplicated files you find are not false positive?
     * A: Size comparision, hash detection, byte by byte check, etc. will pretty sure to
rule out false positive.
     * */

}
```

555. `_611.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 611. Valid Triangle Number
 * Given an array consists of non-negative integers,
 * your task is to count the number of triplets chosen
 * from the array that can make triangles if we take them as side lengths of a triangle.

 Example 1:
 Input: [2,2,3,4]
 Output: 3

 Explanation:
 Valid combinations are:
 2,3,4 (using the first 2)
 2,3,4 (using the second 2)
 2,2,3

 Note:
 The length of the given array won't exceed 1000.
 The integers in the given array are in the range of [0, 1000].
 */
public class _611 {
    public static class Solution1 {
        /**
         * Rule: among three sides, we need to find whether the longest of the three is
smaller than the sum of the two shorter one.
         * If so, then these three could form a valid triangle.
         */

        public int triangleNumber(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            Arrays.sort(nums);
            int triplets = 0;
            for (int i = 2; i < nums.length; i++) {
                int left = 0;
                int right = i - 1;
                while (left < right) {
                    if (nums[i] < nums[left] + nums[right]) {
                        triplets += (right - left);
                        right--;
                    } else {
                        left++;
                    }
                }
            }
            return triplets;
```

```
        }
    }

}
```

556. `_616.java`

```
package com.fishercoder.solutions;

/**
 * 616. Add Bold Tag in String
 *
 * Given a string s and a list of strings dict,
 * you need to add a closed pair of bold tag <b> and </b>
 * to wrap the substrings in s that exist in dict.
 * If two such substrings overlap, you need to wrap them together by only one pair of
closed bold tag.
 * Also, if two substrings wrapped by bold tags are consecutive, you need to combine
them.

 Example 1:
 Input:
 s = "abcxyz123"
 dict = ["abc","123"]
 Output:
 "<b>abc</b>xyz<b>123</b>"

 Example 2:
 Input:
 s = "aaabbcc"
 dict = ["aaa","aab","bc"]
 Output:
 "<b>aaabbc</b>c"

 Note:
 The given dict won't contain duplicates, and its length won't exceed 100.
 All the strings in input have length in range [1, 1000].
 */
public class _616 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/92112/java-solution-boolean-array
         */
        public String addBoldTag(String s, String[] dict) {
            boolean[] shouldBold = new boolean[s.length()];
            for (int i = 0, end = 0; i < s.length(); i++) {
                for (String word : dict) {
                    if (s.startsWith(word, i)) {
                        end = Math.max(end, i + word.length());
                    }
                }
                shouldBold[i] = end > i;
            }
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < s.length(); i++) {
                if (!shouldBold[i]) {
                    stringBuilder.append(s.charAt(i));
```

```
                    continue;
                }
                int j = i;
                while (j < s.length() && shouldBold[j]) {
                    j++;
                }
                stringBuilder.append("<b>" + s.substring(i, j) + "</b>");
                i = j - 1;
            }
            return stringBuilder.toString();
        }
    }

}
```

557. `_617.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 617. Merge Two Binary Trees
 *
 * Given two binary trees and imagine that when you put one of them to cover the other,
 * some nodes of the two trees are overlapped while the others are not.
 You need to merge them into a new binary tree.
 The merge rule is that if two nodes overlap,
 then sum node values up as the new value of the merged node.
 Otherwise, the NOT null node will be used as the node of new tree.

 Example 1:
 Input:
 Tree 1                     Tree 2
    1                         2
   / \                       / \
  3   2                     1   3
 /                           \   \
5                             4   7
 Output:
 Merged tree:
     3
    / \
   4   5
  / \   \
 5   4   7
 Note: The merging process must start from the root nodes of both trees.
 */
public class _617 {

    public static class Solution1 {
        public TreeNode mergeTrees(TreeNode t1, TreeNode t2) {
            if (t1 == null) {
                return t2;
            }
            if (t2 == null) {
                return t1;
            }
            TreeNode mergedNode = new TreeNode(t1.val + t2.val);
            mergedNode.left = mergeTrees(t1.left, t2.left);
            mergedNode.right = mergeTrees(t1.right, t2.right);
            return mergedNode;
        }
    }

}
```

558. `_621.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * 621. Task Scheduler
 *
 * Given a char array representing tasks CPU need to do.
 * It contains capital letters A to Z where different letters represent different tasks.
 * Tasks could be done without original order.
 * Each task could be done in one interval. For each interval, CPU could finish one task
or just be idle.
 * However, there is a non-negative cooling interval n that means between two same tasks,
 * there must be at least n intervals that CPU are doing different tasks or just be idle.
 * You need to return the least number of intervals the CPU will take to finish all the
given tasks.

 Example 1:
 Input: tasks = ['A','A','A','B','B','B'], n = 2
 Output: 8
 Explanation: A -> B -> idle -> A -> B -> idle -> A -> B.

 Note:
 The number of tasks is in the range [1, 10000].
 The integer n is in the range [0, 100].
 */
public class _621 {

    public static class Solution1 {

        public int leastInterval(char[] tasks, int n) {
            Map<Character, Integer> map = new HashMap<>();
            for (char c : tasks) {
                map.put(c, map.getOrDefault(c, 0) + 1);
            }
            PriorityQueue<Task> maxHeap = new PriorityQueue<>((a, b) -> b.total -
a.total);
            for (Map.Entry<Character, Integer> entry : map.entrySet()) {
                maxHeap.offer(new Task(entry.getValue(), entry.getKey()));
            }
            int times = 0;
            while (!maxHeap.isEmpty()) {
                int i = 0;
                List<Task> temp = new ArrayList<>();
                while (i <= n) {
                    if (!maxHeap.isEmpty()) {
                        if (maxHeap.peek().total > 1) {
```

```
                            Task curr = maxHeap.poll();
                            temp.add(new Task(curr.total - 1, curr.character));
                        } else {
                            maxHeap.poll();
                        }
                    }
                    times++;
                    if (maxHeap.isEmpty() && temp.size() == 0) {
                        break;
                    }
                    i++;
                }
                for (Task task : temp) {
                    maxHeap.offer(task);
                }
            }
            return times;
        }

        class Task {
            int total;
            char character;

            public Task(int total, char character) {
                this.total = total;
                this.character = character;
            }
        }
    }

}
```

559. `_623.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 623. Add One Row to Tree
 *
 * Given the root of a binary tree,
 * then value v and depth d,
 * you need to add a row of nodes with value v at the given depth d.
 * The root node is at depth 1.

 The adding rule is: given a positive integer depth d,
 for each NOT null tree nodes N in depth d-1,
 create two tree nodes with value v as N's left subtree root and right subtree root.
 And N's original left subtree should be the left subtree of the new left subtree root,
 its original right subtree should be the right subtree of the new right subtree root.
 If depth d is 1 that means there is no depth d-1 at all,
 then create a tree node with value v as the new root of the whole
 original tree, and the original tree is the new root's left subtree.

 Example 1:
 Input:
 A binary tree as following:
      4
    /   \
   2     6
  / \   /
 3   1 5

 v = 1

 d = 2

 Output:
       4
      / \
     1   1
    /     \
   2       6
  / \     /
 3   1   5


 Example 2:
 Input:
 A binary tree as following:
    4
    /
   2
  / \
```

```
 3   1

 v = 1

 d = 3

 Output:
        4
       /
      2
     / \
    1   1
   /     \
  3       1

 Note:
 The given d is in range [1, maximum depth of the given tree + 1].
 The given binary tree has at least one tree node.
 */
public class _623 {

    public static class Solution1 {

        public TreeNode addOneRow(TreeNode root, int v, int d) {
            if (d == 1) {
                TreeNode newRoot = new TreeNode(v);
                newRoot.left = root;
                return newRoot;
            } else {
                dfs(root, v, d);
                return root;
            }
        }

        private void dfs(TreeNode root, int v, int d) {
            if (root == null) {
                return;
            }
            if (d == 2) {
                TreeNode newLeft = new TreeNode(v);
                TreeNode newRight = new TreeNode(v);
                newLeft.left = root.left;
                newRight.right = root.right;
                root.left = newLeft;
                root.right = newRight;
            } else {
                dfs(root.left, v, d - 1);
                dfs(root.right, v, d - 1);
            }
        }
    }

}
```

560. `_624.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 624. Maximum Distance in Arrays
 *
 * Given m arrays, and each array is sorted in ascending order.
 * Now you can pick up two integers from two different arrays (each array picks one)
 * and calculate the distance. We define the distance between two
 * integers a and b to be their absolute difference |a-b|. Your task is to find the
maximum distance.

 Example 1:
 Input:
 [[1,2,3],
 [4,5],
 [1,2,3]]

 Output: 4

 Explanation:

 One way to reach the maximum distance 4 is to pick 1 in the first or third array and
pick 5 in the second array.

 Note:
 Each given array will have at least 1 number. There will be at least two non-empty
arrays.
 The total number of the integers in all the m arrays will be in the range of [2, 10000].
 The integers in the m arrays will be in the range of [-10000, 10000].
 */
public class _624 {

    public static class Solution1 {
        public int maxDistance(int[][] arrays) {
            List<Integer> max = new ArrayList<>();
            for (int[] array : arrays) {
                max.add(array[array.length - 1]);
            }
            Collections.sort(max);
            int ans = Integer.MIN_VALUE;
            for (int[] array : arrays) {
                int big = array[array.length - 1] == max.get(max.size() - 1) ?
max.get(max.size() - 2) : max.get(max.size() - 1);
                ans = Math.max(ans, big - array[0]);
            }
            return ans;
        }
```

```
    }
}
```

561. `_625.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 625. Minimum Factorization
 *
 * Given a positive integer a, find the smallest positive integer b whose multiplication
of each digit equals to a.

 If there is no answer or the answer is not fit in 32-bit signed integer, then return 0.

 Example 1
 Input:
 48
 Output:
 68

 Example 2
 Input:
 15
 Output:
 35
 */
public class _625 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/92854/java-solution-result-array
         * and https://leetcode.com/articles/minimum-factorization/#approach-3-using-
factorizationaccepted
         */
        public int smallestFactorization(int a) {
            //case 1: a < 10
            if (a < 10) {
                return a;
            }

            //case 2: start with 9 and try every possible digit
            List<Integer> resultArray = new ArrayList<>();
            for (int i = 9; i > 1; i--) {
                //if current digit divides a, then store all occurences of current digit
in res
                while (a % i == 0) {
                    a = a / i;
                    resultArray.add(i);
                }
            }

            //if a could not be broken in form of digits, return 0
```

```
            if (a != 0) {
                return 0;
            }

            //get the result from the result array in reverse order
            long result = 0;
            for (int i = resultArray.size() - 1; i >= 0; i--) {
                result = result * 10 + resultArray.get(i);
                if (result > Integer.MAX_VALUE) {
                    return 0;
                }
            }
            return (int) result;
        }
    }

}
```

562. `_628.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 628. Maximum Product of Three Numbers
 *
 * Given an integer array, find three numbers whose product is maximum and output the
maximum product.
 Example 1:
 Input: [1,2,3]
 Output: 6

 Example 2:
 Input: [1,2,3,4]
 Output: 24

 Note:
 The length of the given array will be in range [3,104] and all elements are in the range
[-1000, 1000].
 Multiplication of any three numbers in the input won't exceed the range of 32-bit signed
integer.
 */
public class _628 {

    public static class Solution1 {
        public int maximumProduct(int[] nums) {
            Arrays.sort(nums);
            int product = 1;
            if (nums.length >= 3) {
                for (int i = nums.length - 1; i >= nums.length - 3; i--) {
                    product *= nums[i];
                }
                int anotherProduct = nums[0] * nums
                        [1] * nums[nums.length - 1];
                product = Math.max(product, anotherProduct);
            } else {
                for (int i = 0; i < nums.length; i++) {
                    product *= nums[i];
                }
            }
            return product;
        }
    }
}
```

563. `_629.java`

```
package com.fishercoder.solutions;

/**
 * 629. K Inverse Pairs Array
 *
 * Given two integers n and k, find how many different arrays consist of numbers from 1
to n such that there are exactly k inverse pairs.
 We define an inverse pair as following: For ith and jth element in the array,
 if i < j and a[i] > a[j] then it's an inverse pair; Otherwise, it's not.
 Since the answer may very large, the answer should be modulo 109 + 7.

 Example 1:
 Input: n = 3, k = 0
 Output: 1
 Explanation:
 Only the array [1,2,3] which consists of numbers from 1 to 3 has exactly 0 inverse pair.

 Example 2:
 Input: n = 3, k = 1
 Output: 2
 Explanation:
 The array [1,3,2] and [2,1,3] have exactly 1 inverse pair.

 Note:
 The integer n is in the range [1, 1000] and k is in the range [0, 1000].
 */
public class _629 {

    public static class Solution1 {

        /**
         * reference: https://leetcode.com/articles/k-inverse-pairs-array/#approach-5-
another-optimized-dynamic-programming-approachaccepted
         * and
         * https://discuss.leetcode.com/topic/93815/java-dp-o-nk-solution
         */
        public int kInversePairs(int n, int k) {
            int mod = 1000000007;
            if (k > n * (n - 1) / 2 || k < 0) {
                return 0;
            }
            if (k == 0 || k == n * (n - 1) / 2) {
                return 1;
            }
            long[][] dp = new long[n + 1][k + 1];
            dp[2][0] = 1;
            dp[2][1] = 1;
            for (int i = 3; i <= n; i++) {
                dp[i][0] = 1;
                for (int j = 1; j <= Math.min(k, i * (i - 1) / 2); j++) {
                    dp[i][j] = dp[i][j - 1] + dp[i - 1][j];
```

```
                    if (j >= i) {
                        dp[i][j] -= dp[i - 1][j - i];
                    }
                    dp[i][j] = (dp[i][j] + mod) % mod;
                }
            }
            return (int) dp[n][k];
        }
    }
}
```

564. `_630.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * 630. Course Schedule III
 *
 * There are n different online courses numbered from 1 to n.
 * Each course has some duration(course length) t and closed on dth day.
 * A course should be taken continuously for t days and must be finished before or on the
dth day. You will start at the 1st day.

 Given n online courses represented by pairs (t,d), your task is to find the maximal
number of courses that can be taken.

 Example:
 Input: [[100, 200], [200, 1300], [1000, 1250], [2000, 3200]]
 Output: 3

 Explanation:
 There're totally 4 courses, but you can take 3 courses at most:
 First, take the 1st course, it costs 100 days so you will finish it on the 100th day,
and ready to take the next course on the 101st day.
 Second, take the 3rd course, it costs 1000 days so you will finish it on the 1100th day,
and ready to take the next course on the 1101st day.
 Third, take the 2nd course, it costs 200 days so you will finish it on the 1300th day.
 The 4th course cannot be taken now, since you will finish it on the 3300th day, which
exceeds the closed date.

 Note:
 The integer 1 <= d, t, n <= 10,000.
 You can't take two courses simultaneously.
 */
public class _630 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/93790/short-java-code-using-
priorityqueue
         * Sort by finish date!!! This is greedy! We should take those classes that
finish early first.
         */
        public int scheduleCourse(int[][] courses) {
            Arrays.sort(courses, (a, b) -> a[1] - b[1]);
            int day = 0;
            PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
            for (int[] course : courses) {
                day += course[0];
                maxHeap.offer(course[0]);
                if (day > course[1]) {
                    day -= maxHeap.poll();//drop the previous courses that took the most
```

```
time
                }
            }
            return maxHeap.size();
        }
    }
}
```

565. `_631.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Stack;

/**
 * 631. Design Excel Sum Formula
 *
 * Your task is to design the basic function of Excel and implement the function of sum
formula.
 * Specifically, you need to implement the following functions:

 Excel(int H, char W):
 This is the constructor.
 The inputs represents the height and width of the Excel form.
 H is a positive integer, range from 1 to 26.
 It represents the height.
 W is a character range from 'A' to 'Z'.
 It represents that the width is the number of characters from 'A' to W.
 The Excel form content is represented by a height * width 2D integer array C, it should
be initialized to zero.
 You should assume that the first row of C starts from 1, and the first column of C
starts from 'A'.

 void Set(int row, char column, int val):
 Change the value at C(row, column) to be val.

 int Get(int row, char column):
 Return the value at C(row, column).

 int Sum(int row, char column, List of Strings : numbers):
 This function calculate and set the value at C(row, column),
 where the value should be the sum of cells represented by numbers.
 This function return the sum result at C(row, column).
 This sum formula should exist until this cell is overlapped by another value or another
sum formula.

 numbers is a list of strings that each string represent a cell or a range of cells.
 If the string represent a single cell,
 then it has the following format : ColRow. For example, "F7" represents the cell at (7,
F).
 If the string represent a range of cells,
 then it has the following format : ColRow1:ColRow2.
 The range will always be a rectangle,
 and ColRow1 represent the position of the top-left cell,
 and ColRow2 represents the position of the bottom-right cell.


 Example 1:
 Excel(3,"C");
 // construct a 3*3 2D array with all zero.
```

```
 //   A B C
 // 1 0 0 0
 // 2 0 0 0
 // 3 0 0 0

 Set(1, "A", 2);
 // set C(1,"A") to be 2.
 //   A B C
 // 1 2 0 0
 // 2 0 0 0
 // 3 0 0 0

 Sum(3, "C", ["A1", "A1:B2"]);
 // set C(3,"C") to be the sum of value at C(1,"A")
 //and the values sum of the rectangle range whose top-left cell is C(1,"A") and bottom-
right cell is C(2,"B"). Return 4.
 //   A B C
 // 1 2 0 0
 // 2 0 0 0
 // 3 0 0 4

 Set(2, "B", 2);
 // set C(2,"B") to be 2. Note C(3, "C") should also be changed.
 //   A B C
 // 1 2 0 0
 // 2 0 2 0
 // 3 0 0 6

 Note:
 You could assume that there won't be any circular sum reference.
 For example, A1 = sum(B1) and B1 = sum(A1).
 The test cases are using double-quotes to represent a character.
 Please remember to RESET your class variables declared in class Excel,
 as static/class variables are persisted across multiple test cases. Please see here for
more details.
 */
public class _631 {

    public static class Solution1 {
        /**
         * Credit: https://leetcode.com/articles/design-excel-sum-formula/#approach-1-
using-topological-sortaccepted
         */
        public static class Excel {

            Formula[][] formulas;

            class Formula {
                Formula(HashMap<String, Integer> c, int v) {
                    val = v;
                    cells = c;
                }

                HashMap<String, Integer> cells;
```

```
            int val;
        }

        Stack<int[]> stack = new Stack<>();

        public Excel(int H, char W) {
            formulas = new Formula[H][(W - 'A') + 1];
        }

        public int get(int r, char c) {
            if (formulas[r - 1][c - 'A'] == null) {
                return 0;
            }
            return formulas[r - 1][c - 'A'].val;
        }

        public void set(int r, char c, int v) {
            formulas[r - 1][c - 'A'] = new Formula(new HashMap<String, Integer>(),
v);
            topologicalSort(r - 1, c - 'A');
            execute_stack();
        }

        public int sum(int r, char c, String[] strs) {
            HashMap<String, Integer> cells = convert(strs);
            int summ = calculate_sum(r - 1, c - 'A', cells);
            set(r, c, summ);
            formulas[r - 1][c - 'A'] = new Formula(cells, summ);
            return summ;
        }

        public void topologicalSort(int r, int c) {
            for (int i = 0; i < formulas.length; i++) {
                for (int j = 0; j < formulas[0].length; j++) {
                    if (formulas[i][j] != null && formulas[i][j].cells.containsKey(""
+ (char) ('A' + c) + (r + 1))) {
                        topologicalSort(i, j);
                    }
                }
            }
            stack.push(new int[]{r, c});
        }

        public void execute_stack() {
            while (!stack.isEmpty()) {
                int[] top = stack.pop();
                if (formulas[top[0]][top[1]].cells.size() > 0) {
                    calculate_sum(top[0], top[1], formulas[top[0]][top[1]].cells);
                }
            }
        }

        public HashMap<String, Integer> convert(String[] strs) {
            HashMap<String, Integer> res = new HashMap<>();
```

```
                for (String st : strs) {
                    if (st.indexOf(":") < 0) {
                        res.put(st, res.getOrDefault(st, 0) + 1);
                    } else {
                        String[] cells = st.split(":");
                        int si = Integer.parseInt(cells[0].substring(1));
                        int ei = Integer.parseInt(cells[1].substring(1));
                        char sj = cells[0].charAt(0);
                        char ej = cells[1].charAt(0);
                        for (int i = si; i <= ei; i++) {
                            for (char j = sj; j <= ej; j++) {
                                res.put("" + j + i, res.getOrDefault("" + j + i, 0) + 1);
                            }
                        }
                    }
                }
                return res;
            }

            public int calculate_sum(int r, int c, HashMap<String, Integer> cells) {
                int sum = 0;
                for (String s : cells.keySet()) {
                    int x = Integer.parseInt(s.substring(1)) - 1;
                    int y = s.charAt(0) - 'A';
                    sum += (formulas[x][y] != null ? formulas[x][y].val : 0) *
cells.get(s);
                }
                formulas[r][c] = new Formula(cells, sum);
                return sum;
            }
        }
    }

    /**
     * Your Excel object will be instantiated and called as such:
     * Excel obj = new Excel(H, W);
     * obj.set(r,c,v);
     * int param_2 = obj.get(r,c);
     * int param_3 = obj.sum(r,c,strs);
     */

}
```

566. `_632.java`

```
package com.fishercoder.solutions;

import java.util.List;
import java.util.PriorityQueue;

/**
 * 632. Smallest Range
 *
 * You have k lists of sorted integers in ascending order.
 * Find the smallest range that includes at least one number from each of the k lists.

 We define the range [a,b] is smaller than range [c,d] if b-a < d-c or a < c if b-a == d-
c.

 Example 1:
 Input:[[4,10,15,24,26], [0,9,12,20], [5,18,22,30]]
 Output: [20,24]
 Explanation:
 List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
 List 2: [0, 9, 12, 20], 20 is in range [20,24].
 List 3: [5, 18, 22, 30], 22 is in range [20,24].

 Note:
 The given list may contain duplicates, so ascending order means >= here.
 1 <= k <= 3500
 -105 <= value of elements <= 105.
 For Java users, please note that the input type has been changed to List<List<Integer>>.
 And after you reset the code template, you'll see this point.

 */
public class _632 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/94445/java-code-using-
priorityqueue-similar-to-merge-k-array/2
         */
        public int[] smallestRange(List<List<Integer>> nums) {
            PriorityQueue<int[]> minHeap = new PriorityQueue<>(nums.size(), (a, b) ->
a[0] - b[0]);
            /**int[] array consists of three numbers: value; which list in nums; index of
value in this list*/

            int max = nums.get(0).get(0);
            for (int i = 0; i < nums.size(); i++) {
                minHeap.offer(new int[]{nums.get(i).get(0), i, 0});
                max = Math.max(max, nums.get(i).get(0));
            }
            int minRange = Integer.MAX_VALUE;
            int start = -1;
            while (minHeap.size() == nums.size()) {
                int[] curr = minHeap.poll();
```

```
                if (max - curr[0] < minRange) {
                    minRange = max - curr[0];
                    start = curr[0];
                }
                if (curr[2] + 1 < nums.get(curr[1]).size()) {
                    curr[0] = nums.get(curr[1]).get(curr[2] + 1);
                    curr[2]++;
                    minHeap.offer(curr);
                    max = Math.max(max, curr[0]);
                }
            }
            return new int[]{start, start + minRange};
        }
    }
}
```

567. `_633.java`

```
package com.fishercoder.solutions;

/**
 * 633. Sum of Square Numbers
 *
 * Given a non-negative integer c, your task is to decide whether there're two integers a
and b such that a2 + b2 = c.

 Example 1:
 Input: 5
 Output: True
 Explanation: 1 * 1 + 2 * 2 = 5

 Example 2:
 Input: 3
 Output: False
 */
public class _633 {
    public static class Solution1 {
        public boolean judgeSquareSum(int c) {
            if (c < 0) {
                return false;
            }
            int left = 0;
            int right = (int) (Math.sqrt(c));
            while (left <= right) {
                int curr = left * left + right * right;
                if (curr > c) {
                    right--;
                } else if (curr < c) {
                    left++;
                } else {
                    return true;
                }
            }
            return false;
        }
    }
}
```

568. `_634.java`

```
package com.fishercoder.solutions;

/**
 * 634. Find the Derangement of An Array
 *
 * In combinatorial mathematics, a derangement is a permutation of the elements of a set,
such that no element appears in its original position.

 There's originally an array consisting of n integers from 1 to n in ascending order, you
need to find the number of derangement it can generate.

 Also, since the answer may be very large, you should return the output mod 109 + 7.

 Example 1:
 Input: 3
 Output: 2
 Explanation: The original array is [1,2,3]. The two derangements are [2,3,1] and
[3,1,2].

 Note:
 n is in the range of [1, 106].
 */
public class _634 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/94442/java-5-lines-o-1-space-
solution
         * and https://leetcode.com/articles/find-derangements/#approach-5-using-formula-
accepted
         */
        private static final int M = 1000000007;

        public int findDerangement(int n) {
            long ans = 1;
            for (int i = 1; i <= n; i++) {
                ans = (i * ans % M + (i % 2 == 0 ? 1 : -1)) % M;
            }
            return (int) ans;
        }
    }
}
```

569. `_635.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.LinkedList;
import java.util.List;

/**
 * 635. Design Log Storage System
 *
 * You are given several logs that each log contains a unique id and timestamp.
 * Timestamp is a string that has the following format:
Year:Month:Day:Hour:Minute:Second,
 * for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.
 * Design a log storage system to implement the following functions:

 void Put(int id, string timestamp): Given a log's unique id and timestamp, store the log
in your storage system.

 int[] Retrieve(String start, String end, String granularity):
 Return the id of logs whose timestamps are within the range from start to end.
 Start and end all have the same format as timestamp.
 However, granularity means the time level for consideration.
 For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", granularity =
"Day",
 it means that we need to find the logs within the range from Jan. 1st 2017 to Jan. 2nd
2017.

 Example 1:
 put(1, "2017:01:01:23:59:59");
 put(2, "2017:01:01:22:59:59");
 put(3, "2016:01:01:00:00:00");
 retrieve("2016:01:01:01:01:01","2017:01:01:23:00:00","Year"); // return [1,2,3], because
you need to return all logs within 2016 and 2017.
 retrieve("2016:01:01:01:01:01","2017:01:01:23:00:00","Hour"); // return [1,2], because
you need to return all logs start from 2016:01:01:01 to 2017:01:01:23, where log 3 is
left outside the range.

 Note:
 There will be at most 300 operations of Put or Retrieve.
 Year ranges from [2000,2017]. Hour ranges from [00,23].
 Output for Retrieve has no order required.
 */
public class _635 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/94449/concise-java-solution
         */
        public static class LogSystem {

            /**
```

```
             * These indices denote and string endings of timestamps of different
granularity, i.e.
             * timestamp[1] in timestamps: "2017:01:01:22:59:59"
             * -> 2017: 4, 01: 7, 01: 10, 22: 13, 59: 16, 59: 19
             */

            List<String[]> timestamps;
            List<String> units;
            int[] indices;

            public LogSystem() {
                timestamps = new LinkedList<>();
                units = Arrays.asList("Year", "Month", "Day", "Hour", "Minute",
"Second");
                indices = new int[]{4, 7, 10, 13, 16, 19};
            }

            public void put(int id, String timestamp) {
                timestamps.add(new String[]{Integer.toString(id), timestamp});
            }

            public List<Integer> retrieve(String s, String e, String gra) {
                List<Integer> res = new LinkedList<>();
                int index = units.indexOf(gra);
                int stringEnd = indices[index];
                for (String[] timestamp : timestamps) {
                    if (timestamp[1].substring(0, stringEnd).compareTo(s.substring(0,
stringEnd)) >= 0
                            && timestamp[1].substring(0,
stringEnd).compareTo(e.substring(0, stringEnd)) <= 0) {
                        res.add(Integer.parseInt(timestamp[0]));
                    }
                }
                return res;
            }
        }
    }

}
```

570. `_636.java`

```
package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;
import java.util.List;

/**
 * 636. Exclusive Time of Functions
 *
 * Given the running logs of n functions that are executed in a nonpreemptive single
threaded CPU,
 * find the exclusive time of these functions.
 * Each function has a unique id, start from 0 to n-1.
 * A function may be called recursively or by another function.
 * A log is a string has this format : function_id:start_or_end:timestamp.
 * For example, "0:start:0" means function 0 starts from the very beginning of time 0.
 * "0:end:0" means function 0 ends to the very end of time 0.
 * Exclusive time of a function is defined as the time spent within this function,
 * the time spent by calling other functions should not be considered as this function's
exclusive time.
 * You should return the exclusive time of each function sorted by their function id.

 Example 1:
 Input:
 n = 2
 logs =
 ["0:start:0",
 "1:start:2",
 "1:end:5",
 "0:end:6"]
 Output:[3, 4]
 Explanation:
 Function 0 starts at time 0, then it executes 2 units of time and reaches the end of
time 1.
 Now function 0 calls function 1, function 1 starts at time 2, executes 4 units of time
and end at time 5.
 Function 0 is running again at time 6, and also end at the time 6, thus executes 1 unit
of time.
 So function 0 totally execute 2 + 1 = 3 units of time, and function 1 totally execute 4
units of time.

 Note:
 Input logs will be sorted by timestamp, NOT log id.
 Your output should be sorted by function id, which means the 0th element of your output
corresponds to the exclusive time of function 0.
 Two functions won't start or end at the same time.
 Functions could be called recursively, and will always end.
 1 <= n <= 100
 */
public class _636 {
    public static class Solution1 {
```

```
        /**
         * Based on the example, it's difficult to see how function 2 executes 4 units of
time, actually
         * we can add 1 to all end times to make it easier to understand and AC'ed.
         */
        public int[] exclusiveTime(int n, List<String> logs) {
            /**Stack is the way to go:
             * we keep pushing the logId onto the stack whenever we just encounter this
logId's start timestamp,
             * we'll pop this logId only when we encounter this logId's end timestamp.
             * Meanwhile, we keep a counter called prevTime,
             * whenever the stack is not empty, we'll always deduct prevTime from the
last logId on the stack.*/
            Deque<Integer> stack = new LinkedList<>();
            int[] result = new int[n];
            int prevTime = 0;
            for (String log : logs) {
                String[] parts = log.split(":");
                if (!stack.isEmpty()) {
                    result[stack.peek()] += Integer.parseInt(parts[2]) - prevTime;
                }
                prevTime = Integer.parseInt(parts[2]);
                if (parts[1].equals("start")) {
                    stack.addFirst(Integer.parseInt(parts[0]));//i.e. stack.push()
                } else {
                    prevTime++;
                    //remember to have result pluse 1 to match the problem AC criteria
                    result[stack.pollFirst()]++;//i.e. stack.pop()
                }
            }
            return result;
        }
    }
}
```

571. `_637.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 637. Average of Levels in Binary Tree
 *
 Given a non-empty binary tree, return the average value of the nodes on each level in
the form of an array.

 Example 1:
 Input:
  3
 / \
9  20
  /  \
 15   7

 Output: [3, 14.5, 11]
 Explanation:

 The average value of nodes on level 0 is 3,  on level 1 is 14.5, and on level 2 is 11.
Hence return [3, 14.5, 11].
 Note:
 The range of node's value is in the range of 32-bit signed integer.
 */
public class _637 {

    public static class Solution1 {
        public List<Double> averageOfLevels(TreeNode root) {
            List<Double> result = new ArrayList<>();
            if (root == null) {
                return result;
            }
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                double sum = 0.0;
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (curr != null) {
                        sum += curr.val;
                    }
                    if (curr.left != null) {
                        queue.offer(curr.left);
```

```
                    }
                    if (curr.right != null) {
                        queue.offer(curr.right);
                    }
                }
                result.add(sum / size);
            }
            return result;
        }
    }

}
```

572. `_638.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 638. Shopping Offers
 *
 * In LeetCode Store, there are some kinds of items to sell. Each item has a price.
 * However, there are some special offers, and a special offer consists of one or more
different kinds of items with a sale price.
 * You are given the each item's price, a set of special offers,
 * and the number we need to buy for each item.
 * The job is to output the lowest price you have to pay for exactly certain items as
given,
 * where you could make optimal use of the special offers.
 * Each special offer is represented in the form of an array,
 * the last number represents the price you need to pay for this special offer,
 * other numbers represents how many specific items you could get if you buy this offer.
 * You could use any of special offers as many times as you want.

 Example 1:
 Input: [2,5], [[3,0,5],[1,2,10]], [3,2]
 Output: 14
 Explanation:
 There are two kinds of items, A and B. Their prices are $2 and $5 respectively.
 In special offer 1, you can pay $5 for 3A and 0B
 In special offer 2, you can pay $10 for 1A and 2B.
 You need to buy 3A and 2B, so you may pay $10 for 1A and 2B (special offer #2), and $4
for 2A.

 Example 2:
 Input: [2,3,4], [[1,1,0,4],[2,2,1,9]], [1,2,1]
 Output: 11
 Explanation:
 The price of A is $2, and $3 for B, $4 for C.
 You may pay $4 for 1A and 1B, and $9 for 2A ,2B and 1C.
 You need to buy 1A ,2B and 1C, so you may pay $4 for 1A and 1B (special offer #1), and
$3 for 1B, $4 for 1C.
 You cannot add more items, though only $9 for 2A ,2B and 1C.

 Note:
 There are at most 6 kinds of items, 100 special offers.
 For each item, you need to buy at most 6 of them.
 You are not allowed to buy more items than you want, even if that would lower the
overall price.
 */
public class _638 {
    public static class Solution1 {
        /**
         * reference: https://leetcode.com/articles/shopping-offers/#approach-1-using-
```

```
recursion-accepted
         */
        public int shoppingOffers(List<Integer> price, List<List<Integer>> special,
List<Integer> needs) {
            return shopping(price, special, needs, 0);
        }

        public int shopping(List<Integer> price, List<List<Integer>> special,
List<Integer> needs, int i) {
            if (i == special.size()) {
                return dot(needs, price);
            }
            ArrayList<Integer> clone = new ArrayList(needs);
            int j = 0;
            for (j = 0; j < special.get(i).size() - 1; j++) {
                int diff = clone.get(j) - special.get(i).get(j);
                if (diff < 0) {
                    break;
                }
                clone.set(j, diff);
            }
            if (j == special.get(i).size() - 1) {
                return Math.min(special.get(i).get(j) + shopping(price, special, clone,
i), shopping(price, special, needs, i + 1));
            } else {
                return shopping(price, special, needs, i + 1);
            }
        }

        public int dot(List<Integer> a, List<Integer> b) {
            int sum = 0;
            for (int i = 0; i < a.size(); i++) {
                sum += a.get(i) * b.get(i);
            }
            return sum;
        }
    }
}
```

573. `_639.java`

```
package com.fishercoder.solutions;

/**
 * 639. Decode Ways II
 *
 * A message containing letters from A-Z is being encoded to numbers using the following
mapping way:

 'A' -> 1
 'B' -> 2
 ...
 'Z' -> 26
 Beyond that, now the encoded string can also contain the character '*',
 which can be treated as one of the numbers from 1 to 9.

 Given the encoded message containing digits and the character '*',
 return the total number of ways to decode it.

 Also, since the answer may be very large, you should return the output mod 109 + 7.

 Example 1:
 Input: "*"
 Output: 9
 Explanation: The encoded message can be decoded to the string: "A", "B", "C", "D", "E",
"F", "G", "H", "I".
 Example 2:
 Input: "1*"
 Output: 9 + 9 = 18

 Note:
 The length of the input string will fit in range [1, 105].
 The input string will only contain the character '*' and digits '0' - '9'.

 */
public class _639 {
    public static class Solution1 {
        /**
         * reference: https://leetcode.com/articles/decode-ways-ii/#approach-2-dynamic-
programming-accepted
         */
        int m = 1000000007;

        public int numDecodings(String s) {
            long[] dp = new long[s.length() + 1];
            dp[0] = 1;
            dp[1] = s.charAt(0) == '*' ? 9 : s.charAt(0) == '0' ? 0 : 1;
            for (int i = 1; i < s.length(); i++) {
                if (s.charAt(i) == '*') {
                    dp[i + 1] = 9 * dp[i];
                    if (s.charAt(i - 1) == '1') {
                        dp[i + 1] = (dp[i + 1] + 9 * dp[i - 1]) % m;
```

```
                    } else if (s.charAt(i - 1) == '2') {
                        dp[i + 1] = (dp[i + 1] + 6 * dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '*') {
                        dp[i + 1] = (dp[i + 1] + 15 * dp[i - 1]) % m;
                    }
                } else {
                    dp[i + 1] = s.charAt(i) != '0' ? dp[i] : 0;
                    if (s.charAt(i - 1) == '1') {
                        dp[i + 1] = (dp[i + 1] + dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '2' && s.charAt(i) <= '6') {
                        dp[i + 1] = (dp[i + 1] + dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '*') {
                        dp[i + 1] = (dp[i + 1] + (s.charAt(i) <= '6' ? 2 : 1) * dp[i -
1]) % m;
                    }
                }
            }
            return (int) dp[s.length()];
        }
    }
}
```

574. `_640.java`

```
package com.fishercoder.solutions;

/**
 * 640. Solve the Equation
 *
 * Solve a given equation and return the value of x in the form of string "x=#value".
 * The equation contains only '+', '-' operation, the variable x and its coefficient.

 If there is no solution for the equation, return "No solution".

 If there are infinite solutions for the equation, return "Infinite solutions".

 If there is exactly one solution for the equation, we ensure that the value of x is an
integer.

 Example 1:
 Input: "x+5-3+x=6+x-2"
 Output: "x=2"

 Example 2:
 Input: "x=x"
 Output: "Infinite solutions"

 Example 3:
 Input: "2x=x"
 Output: "x=0"

 Example 4:
 Input: "2x+3x-6x=x+2"
 Output: "x=-1"

 Example 5:
 Input: "x=x+2"
 Output: "No solution"

 */
public class _640 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/95203/concise-java-solution/7
         */
        public String solveEquation(String equation) {
            String[] parts = equation.split("=");
            int[] left = evaluate(parts[0]);
            int[] right = evaluate(parts[1]);
            if (left[0] == right[0] && left[1] == right[1]) {
                return "Infinite solutions";
            } else if (left[0] == right[0]) {
                return "No solution";
            }
            return "x=" + (right[1] - left[1]) / (left[0] - right[0]);
```

```
        }

        private int[] evaluate(String part) {
            int[] result = new int[2];//result[0] is the coefficient for x, result[1] is
the coefficient for constants
            String[] tokens = part.split("(?=[+-])");  // ()for match group; ?= for match
and include in res; [+-] means + or -;
            for (String token : tokens) {
                if (token.equals("+x") || token.equals("x")) {
                    result[0]++;
                } else if (token.equals("-x")) {
                    result[0]--;
                } else if (token.contains("x")) {
                    result[0] += Integer.parseInt(token.substring(0, token.length() -
1));
                } else {
                    result[1] += Integer.parseInt(token);
                }
            }
            return result;
        }
    }
}
```

575. `_642.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/**
 * 642. Design Search Autocomplete System
 *
 * Design a search autocomplete system for a search engine. Users may input a sentence
(at least one word and end with a special character '#'). For each character they type
except '#', you need to return the top 3 historical hot sentences that have prefix the
same as the part of sentence already typed. Here are the specific rules:

 The hot degree for a sentence is defined as the number of times a user typed the exactly
same sentence before.
 The returned top 3 hot sentences should be sorted by hot degree (The first is the
hottest one). If several sentences have the same degree of hot, you need to use ASCII-
code order (smaller one appears first).
 If less than 3 hot sentences exist, then just return as many as you can.
 When the input is a special character, it means the sentence ends, and in this case, you
need to return an empty list.
 Your job is to implement the following functions:

 The constructor function:

 AutocompleteSystem(String[] sentences, int[] times): This is the constructor. The input
is historical data. Sentences is a string array consists of previously typed sentences.
Times is the corresponding times a sentence has been typed. Your system should record
these historical data.

 Now, the user wants to input a new sentence. The following function will provide the
next character the user types:

 List<String> input(char c): The input c is the next character typed by the user. The
character will only be lower-case letters ('a' to 'z'), blank space (' ') or a special
character ('#'). Also, the previously typed sentence should be recorded in your system.
The output will be the top 3 historical hot sentences that have prefix the same as the
part of sentence already typed.


 Example:
 Operation: AutocompleteSystem(["i love you", "island","ironman", "i love leetcode"],
[5,3,2,2])
 The system have already tracked down the following sentences and their corresponding
times:
 "i love you" : 5 times
 "island" : 3 times
```

```
 "ironman" : 2 times
 "i love leetcode" : 2 times
 Now, the user begins another search:

 Operation: input('i')
 Output: ["i love you", "island","i love leetcode"]
 Explanation:
 There are four sentences that have prefix "i". Among them, "ironman" and "i love
leetcode" have same hot degree. Since ' ' has ASCII code 32 and 'r' has ASCII code 114,
"i love leetcode" should be in front of "ironman". Also we only need to output top 3 hot
sentences, so "ironman" will be ignored.

 Operation: input(' ')
 Output: ["i love you","i love leetcode"]
 Explanation:
 There are only two sentences that have prefix "i ".

 Operation: input('a')
 Output: []
 Explanation:
 There are no sentences that have prefix "i a".

 Operation: input('#')
 Output: []
 Explanation:
 The user finished the input, the sentence "i a" should be saved as a historical sentence
in system. And the following input will be counted as a new search.

 Note:
 The input sentence will always start with a letter and end with '#', and only one blank
space will exist between two words.
 The number of complete sentences that to be searched won't exceed 100. The length of
each sentence including those in the historical data won't exceed 100.
 Please use double-quote instead of single-quote when you write test cases even for a
character input.
 Please remember to RESET your class variables declared in class AutocompleteSystem, as
static/class variables are persisted across multiple test cases. Please see here for more
details.
 */
public class _642 {
    public static class Solution1 {

        /**
         * reference: https://discuss.leetcode.com/topic/96150/java-solution-trie-and-
priorityqueue/3
         */
        public class AutocompleteSystem {

            Map<String, Integer> map;
            List<Map.Entry<String, Integer>> answers;
            StringBuilder stringBuilder;

            public AutocompleteSystem(String[] sentences, int[] times) {
                map = new HashMap<>();
```

```
                answers = new ArrayList<>();
                stringBuilder = new StringBuilder();

                for (int i = 0; i < sentences.length; i++) {
                    map.put(sentences[i], map.getOrDefault(sentences[i], 0) + times[i]);
                }
            }

            public List<String> input(char c) {
                List<String> result = new ArrayList<>();
                if (c == '#') {
                    map.put(stringBuilder.toString(),
map.getOrDefault(stringBuilder.toString(), 0) + 1);
                    stringBuilder.setLength(0);
                    answers.clear();/**The user has finished typing, so we'll clean
answers to get ready for next search*/
                } else {
                    stringBuilder.append(c);
                    /**when its length is 1, we find all the prefix that is a match and
put them into answers,
                     * then for the rest, we'll just remove those that are not match with
the prefix any more, we do this logic in else branch*/
                    if (stringBuilder.length() == 1) {
                        for (Map.Entry<String, Integer> entry : map.entrySet()) {
                            if (entry.getKey().startsWith(stringBuilder.toString())) {
                                answers.add(entry);
                            }
                        }
                        Collections.sort(answers, (a, b) -> a.getValue() == b.getValue()
? a.getKey().compareTo(b.getKey()) : b.getValue() - a.getValue());
                    } else {
                        for (Iterator<Map.Entry<String, Integer>> iterator =
answers.iterator(); iterator.hasNext(); ) {
                            if
(!iterator.next().getKey().startsWith(stringBuilder.toString())) {
                                iterator.remove();
                            }
                        }
                    }
                    for (int i = 0; i < 3 && i < answers.size(); i++) {
                        result.add(answers.get(i).getKey());
                    }
                }
                return result;
            }
        }

    }
}
```

576. `_643.java`

```
package com.fishercoder.solutions;

/**
 * 643. Maximum Average Subarray I
 * Given an array consisting of n integers, find the contiguous subarray of given length
k that has the maximum average value.
 * And you need to output the maximum average value.
 * Example 1:
 * Input: [1,12,-5,-6,50,3], k = 4
 * Output: 12.75
 * Explanation: Maximum average is (12-5-6+50)/4 = 51/4 = 12.75
 * Note:
 * 1 <= k <= n <= 30,000.
 * Elements of the given array will be in the range [-10,000, 10,000].
 */
public class _643 {

    public static class Solution1 {
        public double findMaxAverage(int[] nums, int k) {
            double sum = 0;
            double maxAve = Integer.MIN_VALUE;
            for (int i = 0; i < nums.length; i++) {
                if (k <= i) {
                    sum -= nums[i - k];
                }
                sum += nums[i];
                if ((i + 1) >= k) {
                    maxAve = Math.max(maxAve, sum / k);
                }
            }
            return maxAve;
        }
    }
}
```

577. `_644.java`

```
package com.fishercoder.solutions;

/**
 * 644. Maximum Average Subarray II
 * <p>
 * Given an array consisting of n integers, find the contiguous subarray whose length is
greater than
 * or equal to k that has the maximum average value. And you need to output the maximum
average value.
 * <p>
 * Example 1:
 * Input: [1,12,-5,-6,50,3], k = 4
 * Output: 12.75
 * Explanation:
 * when length is 5, maximum average value is 10.8,
 * when length is 6, maximum average value is 9.16667.
 * Thus return 12.75.
 * <p>
 * Note:
 * 1 <= k <= n <= 10,000.
 * Elements of the given array will be in range [-10,000, 10,000].
 * The answer with the calculation error less than 10-5 will be accepted.
 */
public class _644 {
    /**reference: https://leetcode.com/articles/maximum-average-subarray-ii/#approach-2-
using-binary-search-accepted
     * https://discuss.leetcode.com/topic/96123/java-solution-o-nlogm-binary-search-the-
answer/13*/

    /**
     * To understand the idea behind this method, let's look at the following points.
     * Firstly, we know that the value of the average could lie between the range (min,
max)(min,max).
     * Here, minmin and maxmax refer to the minimum and the maximum values out of the
given numsnums array.
     * This is because, the average can't be lesser than the minimum value and can't be
larger than the maximum value.
     * But, in this case, we need to find the maximum average of a subarray with atleast
kk elements.
     * The idea in this method is to try to approximate(guess) the solution and to try to
find if this solution really exists.
     * If it exists, we can continue trying to approximate the solution even to a further
precise value,
     * but choosing a larger number as the next approximation.
     * But, if the initial guess is wrong, and the initial maximum average value(guessed)
isn't possible,
     * we need to try with a smaller number as the next approximate.
     * <p>
     * Now, instead of doing the guesses randomly, we can make use of Binary Search.
     * With minmin and maxmax as the initial numbers to begin with,
     * we can find out the midmid of these two numbers given by (min+max)/2(min+max)/2.
```

```
     * Now, we need to find if a subarray with length greater than or equal to kk is
possible with an average sum greater than this midmid value.
     */
    public double findMaxAverage(int[] nums, int k) {
        double l = -10000;
        double r = 10000;
        while (r - l > 10e-7) {
            double mid = (l + r) / 2;
            if (getMaxSubbaraySumOfSizeK(nums, k, mid) >= 0) {
                l = mid;
            } else {
                r = mid;
            }
        }
        return (l + r) / 2;
    }

    private double getMaxSubbaraySumOfSizeK(int[] nums, int k, double mid) {
        double sum = 0.0;
        for (int i = 0; i <= k - 1; i++) {
            sum += nums[i] - mid;
        }
        double maxSum = sum;
        double prev = nums[0] - mid;
        for (int i = k; i < nums.length; i++) {
            sum = sum - nums[i - k] + nums[i];
            maxSum = Math.max(maxSum, Math.max(sum, sum + prev));
            prev = Math.max(nums[i - k + 1], nums[i - k + 1] + prev) - mid;
        }
        return maxSum;
    }
}
```

578. `_645.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 645. Set Mismatch
 *
 * The set S originally contains numbers from 1 to n.
 * But unfortunately, due to the data error, one of the numbers in
 * the set got duplicated to another number in the set,
 * which results in repetition of one number and loss of another number.

 Given an array nums representing the data status of this set after the error.
 Your task is to firstly find the number occurs twice and then find the number
 that is missing. Return them in the form of an array.

 Example 1:
 Input: nums = [1,2,2,4]
 Output: [2,3]

 Note:
 The given array size will in the range [2, 10000].
 The given array's numbers won't have any order.
 */
public class _645 {
    public static class Solution1 {
        public int[] findErrorNums(int[] nums) {
            int[] result = new int[2];
            Arrays.sort(nums);
            for (int i = 0; i < nums.length - 1; i++) {
                if (nums[i + 1] == nums[i]) {
                    result[0] = nums[i];
                }
            }
            long temp = 0;
            for (int i = 0; i < nums.length; i++) {
                temp += i + 1;
                temp -= nums[i];
            }
            temp += result[0];
            result[1] = (int) temp;
            return result;
        }
    }
}
```

579. `_646.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 646. Maximum Length of Pair Chain
 *
 * You are given n pairs of numbers. In every pair, the first number is always smaller
than the second number.

 Now, we define a pair (c, d) can follow another pair (a, b) if and only if b < c. Chain
of pairs can be formed in this fashion.

 Given a set of pairs, find the length longest chain which can be formed. You needn't use
up all the given pairs. You can select pairs in any order.

 Example 1:
 Input: [[1,2], [2,3], [3,4]]
 Output: 2
 Explanation: The longest chain is [1,2] -> [3,4]

 Note:
 The number of given pairs will be in the range [1, 1000].
 */
public class _646 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/96804/java-o-nlog-n-time-o-1-space
         */
        public int findLongestChain(int[][] pairs) {
            Arrays.sort(pairs, (o1, o2) -> o1[1] - o2[1]);
            int result = 0;
            int n = pairs.length;
            int i = -1;
            while (++i < n) {
                result++;
                int curEnd = pairs[i][1];
                while (i + 1 < n && pairs[i + 1][0] <= curEnd) {
                    /**This means, we'll keep incrementing i until pairs[i+1][0] is
                     * exactly greater than curEnd.*/
                    i++;
                }
            }
            return result;
        }
    }

}
```

580. `_647.java`

```
package com.fishercoder.solutions;

/**
 * 647. Palindromic Substrings
 *
 *  Given a string, your task is to count how many palindromic substrings in this string.
 *  The substrings with different start indexes or end indexes are counted
 *  as different substrings even they consist of same characters.

 Example 1:
 Input: "abc"
 Output: 3
 Explanation: Three palindromic strings: "a", "b", "c".

 Example 2:
 Input: "aaa"
 Output: 6
 Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

 Note:
 The input string length won't exceed 1000.

 */

public class _647 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/96819/java-solution-8-lines-
extendpalindrome
         */
        public int countSubstrings(String s) {
            int count = 0;
            for (int i = 0; i < s.length(); i++) {
                count += extendPalindrome(s, i, i);//odd length
                count += extendPalindrome(s, i, i + 1);//even length
            }
            return count;
        }

        private int extendPalindrome(String s, int left, int right) {
            int count = 0;
            while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right))
{
                count++;
                left--;
                right++;
            }
            return count;
        }
```

```
    }
}
```

581. `_648.java`

```
package com.fishercoder.solutions;

import java.util.List;

/**
 * 648. Replace Words

 * In English, we have a concept called root,
 * which can be followed by some other words to form another longer word
 * - let's call this word successor.
 * For example, the root an, followed by other, which can form another word another.
 * Now, given a dictionary consisting of many roots and a sentence.
 * You need to replace all the successor in the sentence with the root forming it.
 * If a successor has many roots can form it, replace it with the root with the shortest
length.

 You need to output the sentence after the replacement.

 Example 1:
 Input: dict = ["cat", "bat", "rat"]
 sentence = "the cattle was rattled by the battery"
 Output: "the cat was rat by the bat"

 Note:
 The input will only have lower-case letters.
 1 <= dict words number <= 1000
 1 <= sentence words number <= 1000
 1 <= root length <= 100
 1 <= sentence words length <= 1000

 */
public class _648 {
    public static class Solution1 {
        public String replaceWords(List<String> dict, String sentence) {
            String[] tokens = sentence.split(" ");
            TrieNode trie = buildTrie(dict);
            return replaceWords(tokens, trie);
        }

        private String replaceWords(String[] tokens, TrieNode root) {
            StringBuilder stringBuilder = new StringBuilder();
            for (String token : tokens) {
                stringBuilder.append(getShortestReplacement(token, root));
                stringBuilder.append(" ");
            }
            return stringBuilder.substring(0, stringBuilder.length() - 1);
        }

        private String getShortestReplacement(String token, final TrieNode root) {
            TrieNode temp = root;
            StringBuilder stringBuilder = new StringBuilder();
```

```
            for (char c : token.toCharArray()) {
                stringBuilder.append(c);
                if (temp.children[c - 'a'] != null) {
                    if (temp.children[c - 'a'].isWord) {
                        return stringBuilder.toString();
                    }
                    temp = temp.children[c - 'a'];
                } else {
                    return token;
                }
            }
            return token;
        }

        private TrieNode buildTrie(List<String> dict) {
            TrieNode root = new TrieNode(' ');
            for (String word : dict) {
                TrieNode temp = root;
                for (char c : word.toCharArray()) {
                    if (temp.children[c - 'a'] == null) {
                        temp.children[c - 'a'] = new TrieNode(c);
                    }
                    temp = temp.children[c - 'a'];
                }
                temp.isWord = true;
            }
            return root;
        }

        public class TrieNode {
            char val;
            TrieNode[] children;
            boolean isWord;

            public TrieNode(char val) {
                this.val = val;
                this.children = new TrieNode[26];
                this.isWord = false;
            }
        }
    }
}
```

582. `_649.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 649. Dota2 Senate
 *
 * In the world of Dota2, there are two parties: the Radiant and the Dire.

 The Dota2 senate consists of senators coming from two parties.
 Now the senate wants to make a decision about a change in the Dota2 game.
 The voting for this change is a round-based procedure.
 In each round, each senator can exercise one of the two rights:

 1. Ban one senator's right:
 A senator can make another senator lose all his rights in this and all the following
rounds.
 2. Announce the victory:
 If this senator found the senators who still have rights to vote are all from the same
party,
 he can announce the victory and make the decision about the change in the game.
 Given a string representing each senator's party belonging.
 The character 'R' and 'D' represent the Radiant party and the Dire party respectively.
 Then if there are n senators, the size of the given string will be n.

 The round-based procedure starts from the first senator to the last senator in the given
order.
 This procedure will last until the end of voting.
 All the senators who have lost their rights will be skipped during the procedure.

 Suppose every senator is smart enough and will play the best strategy for his own party,
 you need to predict which party will finally announce the victory and make the change in
the Dota2 game.
 The output should be Radiant or Dire.

 Example 1:
 Input: "RD"
 Output: "Radiant"
 Explanation: The first senator comes from Radiant and he can just ban the next senator's
right in the round 1.
 And the second senator can't exercise any rights any more since his right has been
banned.
 And in the round 2, the first senator can just announce the victory since he is the only
guy in the senate who can vote.

 Example 2:
 Input: "RDD"
 Output: "Dire"
 Explanation:
 The first senator comes from Radiant and he can just ban the next senator's right in the
```

```
round 1.
 And the second senator can't exercise any rights anymore since his right has been
banned.
 And the third senator comes from Dire and he can ban the first senator's right in the
round 1.
 And in the round 2, the third senator can just announce the victory since he is the only
guy in the senate who can vote.
 Note:
 The length of the given string will in the range [1, 10,000].

 */
public class _649 {

    public static class Solution1 {
        public String predictPartyVictory(String senate) {
            Queue<Integer> radiantQ = new LinkedList<>();
            Queue<Integer> direQ = new LinkedList<>();
            int len = senate.length();
            for (int i = 0; i < len; i++) {
                if (senate.charAt(i) == 'R') {
                    radiantQ.offer(i);
                } else {
                    direQ.offer(i);
                }
            }
            while (!radiantQ.isEmpty() && !direQ.isEmpty()) {
                int radiantIndex = radiantQ.poll();
                int direIndex = direQ.poll();
                if (radiantIndex < direIndex) {
                    /**Radiant will ban Dire in this case, so we'll add radiant index
back to the queue plus n*/
                    radiantQ.offer(radiantIndex + len);
                } else {
                    direQ.offer(direIndex + len);
                }
            }
            return radiantQ.isEmpty() ? "Dire" : "Radiant";
        }
    }
}
```

583. `_650.java`

```
package com.fishercoder.solutions;

/**
 * 650. 2 Keys Keyboard
 *
 * Initially on a notepad only one character 'A' is present. You can perform two
operations on this notepad for each step:

 Copy All: You can copy all the characters present on the notepad (partial copy is not
allowed).
 Paste: You can paste the characters which are copied last time.
 Given a number n. You have to get exactly n 'A' on the notepad by performing the minimum
number of steps permitted. Output the minimum number of steps to get n 'A'.

 Example 1:
 Input: 3
 Output: 3

 Explanation:
 Intitally, we have one character 'A'.
 In step 1, we use Copy All operation.
 In step 2, we use Paste operation to get 'AA'.
 In step 3, we use Paste operation to get 'AAA'.

 Note:
 The n will be in the range [1, 1000].
 */
public class _650 {

    public static class Solution1 {
        public int minSteps(int n) {
            int[] dp = new int[n + 1];
            for (int i = 2; i <= n; i++) {
                dp[i] = i;//we assign i to dp[i] first, because for a lot of cases, e.g.
for most cases when i is odd, its min steps is i itself, if it's not, we can overwrite it
later
                for (int j = i - 1; j > 1; j--) {
                    //traverse backwards, whenever it's divisible by j, we'll update
dp[i] because it's guaranteed to be smaller when j is smaller.
                    if (i % j == 0) {
                        dp[i] = dp[j] + (i / j);
                        break;
                    }
                }
            }
            return dp[n];
        }
    }
}
```

584. `_651.java`

```
package com.fishercoder.solutions;

/**
 * 651. 4 Keys Keyboard
 *
 * Imagine you have a special keyboard with the following keys:

 Key 1: (A): Prints one 'A' on screen.

 Key 2: (Ctrl-A): Select the whole screen.

 Key 3: (Ctrl-C): Copy selection to buffer.

 Key 4: (Ctrl-V): Print buffer on screen appending it after what has already been
printed.

 Now, you can only press the keyboard for N times (with the above four keys), find out
the maximum numbers of 'A' you can print on screen.

 Example 1:
 Input: N = 3
 Output: 3
 Explanation:
 We can at most get 3 A's on screen by pressing following key sequence:
 A, A, A

 Example 2:
 Input: N = 7
 Output: 9
 Explanation:
 We can at most get 9 A's on screen by pressing following key sequence:
 A, A, A, Ctrl A, Ctrl C, Ctrl V, Ctrl V

 Note:
 1 <= N <= 50
 Answers will be in the range of 32-bit signed integer.
 */
public class _651 {

    public static class Solution1 {
        /**
         * Minimum needs to be more than 3 A's in a row, otherwise "Ctrl A, Ctrl C, Ctrl
V" will make fewer A's than directly
         * copying A's with the equal number of steps.
         * E.g. when n == 5,
         * if we do 5 this: A, A, Ctrl A, Ctrl C, Ctrl V, => this will result in only
AAAA (4 A's)
         * while if we do A, A, A, A, A, => this will result in AAAAA (5 A's)
         * So, at a minimum, we need to have 3 A's, then it's worth to do "Ctrl A, Ctrl
C, Ctrl V"..
         * That's why we have j = 3 in the inner for loop below.
```

```
         */
        public int maxA(int N) {
            int[] dp = new int[N + 1];
            for (int i = 1; i <= N; i++) {
                dp[i] = i;
                for (int j = 3; j < i; j++) {
                    dp[i] = Math.max(dp[i], dp[i - j] * (j - 1));
                }
            }
            return dp[N];
        }
    }

}
```

585. `_652.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

/**
 * 652. Find Duplicate Subtrees
 *
 * Given a binary tree, return all duplicate subtrees. For each kind of duplicate
subtrees, you only need to return the root node of any one of them.

 Two trees are duplicate if they have the same structure with same node values.

 Example 1:
     1
    / \
   2   3
  /   / \
 4   2   4
    /
   4
 The following are two duplicate subtrees:
     2
    /
   4
 and
 4

 Therefore, you need to return above trees' root in the form of a list.
 */
public class _652 {
    public static class Solution1 {

        /**credit: https://discuss.leetcode.com/topic/97584/java-concise-postorder-
traversal-solution*/

        /**
         * You don't actually need to check if every other tree is a duplicate of current
node,
         * just when you go through each node, you'll see whether there's already one in
the map,
         * since map.containsKey() checks this TreeNode.
         */
```

```
        public List<TreeNode> findDuplicateSubtrees(TreeNode root) {
            List<TreeNode> res = new LinkedList<>();
            postorder(root, new HashMap<>(), res);
            return res;
        }

        private String postorder(TreeNode curr, HashMap<String, Integer> map,
List<TreeNode> res) {
            if (curr == null) {
                return "#";
            }
            String serial = curr.val + "," + postorder(curr.left, map, res) + "," +
postorder(curr.right, map, res);
            if (map.getOrDefault(serial, 0) == 1) {
                res.add(curr);
            }
            map.put(serial, map.getOrDefault(serial, 0) + 1);
            return serial;
        }
    }
}
```

586. `_653.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 653. Two Sum IV - Input is a BST
 *
 * Given a Binary Search Tree and a target number,
 * return true if there exist two elements in the BST such that their sum is equal to the
given target.

 Example 1:
 Input:
    5
   / \
  3   6
 / \   \
2   4   7

Target = 9
Output: True

Example 2:
Input:
    5
   / \
  3   6
 / \   \
2   4   7

Target = 28
Output: False

 */
public class _653 {

    public static class Solution1 {
        public boolean findTarget(TreeNode root, int k) {
            if (root == null) {
                return false;
            }
            List<Integer> list = new ArrayList<>();
            dfs(root, list);
            for (int i = 0; i < list.size() - 1; i++) {
                for (int j = i + 1; j < list.size(); j++) {
                    if (list.get(i) + list.get(j) == k) {
                        return true;
                    }
```

```
                }
            }
            return false;
        }

        private void dfs(TreeNode root, List<Integer> list) {
            list.add(root.val);
            if (root.left != null) {
                dfs(root.left, list);
            }
            if (root.right != null) {
                dfs(root.right, list);
            }
        }
    }
}
```

587. `_654.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 654. Maximum Binary Tree
 *
 * Given an integer array with no duplicates. A maximum tree building on this array is
defined as follow:
 *
 * The root is the maximum number in the array.
 * The left subtree is the maximum tree constructed from left part subarray divided by
the maximum number.
 * The right subtree is the maximum tree constructed from right part subarray divided by
the maximum number.
 * Construct the maximum tree by the given array and output the root node of this tree.

 Example 1:
 Input: [3,2,1,6,0,5]
 Output: return the tree root node representing the following tree:

    6
  /   \
 3    5
  \   /
  2  0
   \
   1

 Note:
 The size of the given array will be in the range [1,1000].
 */
public class _654 {

    public static class Solution1 {
        /**
         * Completely my original solution:
         *
         * As the problem states, I always broke the array into two halves and make notes
         * of current max node, then in the recursive call, we can recursively search
         * from its left part to construct its left subtree and its right part to
construct its
         * right subtree.*/
        public TreeNode constructMaximumBinaryTree(int[] nums) {
            int max = Integer.MIN_VALUE;
            int maxIndex = -1;
            for (int i = 0; i < nums.length; i++) {
                if (nums[i] > max) {
                    max = nums[i];
                    maxIndex = i;
                }
```

```
            }
            TreeNode root = new TreeNode(max);
            return constructMaxTree(root, maxIndex, nums, 0, nums.length - 1);
        }

        private TreeNode constructMaxTree(TreeNode root, int rootIndex, int[] nums, int
start, int end) {
            if (rootIndex > start) {
                int max = Integer.MIN_VALUE;
                int maxIndex = -1;
                for (int i = start; i < rootIndex; i++) {
                    if (max < nums[i]) {
                        max = nums[i];
                        maxIndex = i;
                    }
                }
                root.left = constructMaxTree(new TreeNode(max), maxIndex, nums, start,
rootIndex - 1);
            }
            if (rootIndex < end) {
                int max = Integer.MIN_VALUE;
                int maxIndex = -1;
                for (int i = rootIndex + 1; i <= end; i++) {
                    if (max < nums[i]) {
                        max = nums[i];
                        maxIndex = i;
                    }
                }
                root.right = constructMaxTree(new TreeNode(max), maxIndex, nums,
rootIndex + 1, end);
            }
            return root;
        }
    }

    public static class Solution2 {
        /**
         * Completely my original solution as well, but more concise.
         */
        public TreeNode constructMaximumBinaryTree(int[] nums) {
            if (nums == null || nums.length == 0) {
                return null;
            }
            return construct(nums, 0, nums.length - 1);
        }

        TreeNode construct(int[] nums, int start, int end) {
            if (start > end) {
                return null;
            }
            int[] maxArray = findMax(nums, start, end);
            TreeNode root = new TreeNode(maxArray[0]);
            root.left = construct(nums, start, maxArray[1] - 1);
            root.right = construct(nums, maxArray[1] + 1, end);
```

```
            return root;
        }

        int[] findMax(int[] nums, int start, int end) {
            int max = nums[start];
            int maxIndex = start;
            for (int i = start + 1; i <= end; i++) {
                if (max < nums[i]) {
                    maxIndex = i;
                    max = nums[i];
                }
            }
            return new int[]{max, maxIndex};
        }
    }
}
```

588. `_655.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

/**
 * 655. Print Binary Tree
 *
 * Print a binary tree in an m*n 2D string array following these rules:
 * 1. The row number m should be equal to the height of the given binary tree.
 * 2. The column number n should always be an odd number.
 * 3. The root node's value (in string format) should be put in the exactly middle of the
first row it can be put.
 *    The column and the row where the root node belongs will separate the rest space
into two parts (left-bottom part and right-bottom part).
 *    You should print the left subtree in the left-bottom part and print the right
subtree in the right-bottom part.
 *    The left-bottom part and the right-bottom part should have the same size.
 *    Even if one subtree is none while the other is not,
 *    you don't need to print anything for the none subtree but
 *    still need to leave the space as large as that for the other subtree.
 *    However, if two subtrees are none, then you don't need to leave space for both of
them.
 * 4. Each unused space should contain an empty string "".
 * 5. Print the subtrees following the same rules.

 Example 1:
 Input:
   1
  /
 2
 Output:
 [["", "1", ""],
 ["2", "", ""]]

 Example 2:
 Input:
   1
  / \
 2   3
  \
  4
 Output:
 [["", "", "", "1", "", "", ""],
 ["", "2", "", "", "", "3", ""],
 ["", "", "4", "", "", "", ""]]

 Example 3:
```

```
 Input:
       1
      / \
     2   5
    /
   3
  /
 4
 Output:

 [["",  "",  "", "",  "", "", "", "1", "",  "",  "",  "",  "", "", ""]
 ["",  "",  "", "2", "", "", "", "",  "",  "",  "",  "5", "", "", ""]
 ["",  "3", "", "",  "", "", "", "",  "",  "",  "",  "",  "", "", ""]
 ["4", "",  "", "",  "", "", "", "",  "",  "",  "",  "",  "", "", ""]]

 Note: The height of binary tree is in the range of [1, 10]
 */
public class _655 {

    public static class Solution1 {

        /**
         * Reference: https://discuss.leetcode.com/topic/98381/java-recursive-solution
         * and https://leetcode.com/articles/print-binary-tree/
         */
        public List<List<String>> printTree(TreeNode root) {
            List<List<String>> result = new LinkedList<>();
            int height = root == null ? 1 : getHeight(root);
            int rows = height;
            int columns = (int) (Math.pow(2, height) - 1);
            List<String> row = new ArrayList<>();
            for (int i = 0; i < columns; i++) {
                row.add("");
            }
            for (int i = 0; i < rows; i++) {
                result.add(new ArrayList<>(row));
            }
            populateResult(root, result, 0, rows, 0, columns - 1);
            return result;
        }

        private void populateResult(TreeNode root, List<List<String>> result, int row,
int totalRows, int i, int j) {
            if (row == totalRows || root == null) {
                return;
            }
            result.get(row).set((i + j) / 2, Integer.toString(root.val));
            populateResult(root.left, result, row + 1, totalRows, i, (i + j) / 2 - 1);
            populateResult(root.right, result, row + 1, totalRows, (i + j) / 2 + 1, j);
        }

        private int getHeight(TreeNode root) {
            if (root == null) {
                return 0;
```

```
            }
            return 1 + Math.max(getHeight(root.left), getHeight(root.right));
        }
    }
}
```

589. `_656.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 656. Coin Path
 *
 * Given an array A (index starts at 1) consisting of N integers: A1, A2, ..., AN and an integer B.
 * The integer B denotes that from any place (suppose the index is i) in the array A,
 * you can jump to any one of the place in the array A indexed i+1, i+2, …, i+B if this place can be jumped to.
 * Also, if you step on the index i, you have to pay Ai coins. If Ai is -1, it means you can’t jump to the place indexed i in the array.
 * Now, you start from the place indexed 1 in the array A,
 * and your aim is to reach the place indexed N using the minimum coins.
 * You need to return the path of indexes (starting from 1 to N) in the array you should take to get to the place indexed N using minimum coins.
 * If there are multiple paths with the same cost, return the lexicographically smallest such path.
 * If it's not possible to reach the place indexed N then you need to return an empty array.

 Example 1:
 Input: [1,2,4,-1,2], 2
 Output: [1,3,5]

 Example 2:
 Input: [1,2,4,-1,2], 1
 Output: []

 Note:
 Path Pa1, Pa2, ..., Pan is lexicographically smaller than Pb1, Pb2, ..., Pbm,
 if and only if at the first i where Pai and Pbi differ, Pai < Pbi; when no such i exists, then n < m.
 A1 >= 0. A2, ..., AN (if exist) will in the range of [-1, 100].
 Length of A is in the range of [1, 1000].
 B is in the range of [1, 100].
 */

public class _656 {
    public static class Solution1 {

        /**
         * Time: O(n*B)
         * Reference: https://leetcode.com/articles/coin-path/#approach-3-using-dynamic-programming-accepted
         */
        public List<Integer> cheapestJump(int[] A, int B) {
```

```
            int[] next = new int[A.length];
            long[] dp = new long[A.length];
            Arrays.fill(next, -1);
            List<Integer> res = new ArrayList();
            for (int i = A.length - 2; i >= 0; i--) {
                long minCost = Integer.MAX_VALUE;
                for (int j = i + 1; j <= i + B && j < A.length; j++) {
                    if (A[j] >= 0) {
                        long cost = A[i] + dp[j];
                        if (cost < minCost) {
                            minCost = cost;
                            next[i] = j;
                        }
                    }
                }
                dp[i] = minCost;
            }
            int i;
            for (i = 0; i < A.length && next[i] > 0; i = next[i]) {
                res.add(i + 1);
            }
            if (i == A.length - 1 && A[i] >= 0) {
                res.add(A.length);
            } else {
                return new ArrayList<>();
            }
            return res;
        }
    }
}
```

590. `_657.java`

```
package com.fishercoder.solutions;

/**
 * 657. Judge Route Circle
 *
 * Initially, there is a Robot at position (0, 0).
 * Given a sequence of its moves, judge if this robot makes a circle,
 * which means it moves back to the original place.

 The move sequence is represented by a string.
 And each move is represent by a character.
 The valid robot moves are R (Right), L (Left), U (Up) and D (down).
 The output should be true or false representing whether the robot makes a circle.

 Example 1:
 Input: "UD"
 Output: true

 Example 2:
 Input: "LL"
 Output: false
 */

public class _657 {
    public static class Solution1 {
        public boolean judgeCircle(String moves) {
            int hori = 0;
            int verti = 0;
            for (char c : moves.toCharArray()) {
                if (c == 'U') {
                    verti++;
                } else if (c == 'D') {
                    verti--;
                } else if (c == 'L') {
                    hori--;
                } else if (c == 'R') {
                    hori++;
                }
            }
            return verti == 0 && hori == 0;
        }
    }
}
```

591. `_658.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 658. Find K Closest Elements
 *
 * Given a sorted array, two integers k and x,
 * find the k closest elements to x in the array.
 * The result should also be sorted in ascending order.
 * If there is a tie, the smaller elements are always preferred.

 Example 1:
 Input: [1,2,3,4,5], k=4, x=3
 Output: [1,2,3,4]

 Example 2:
 Input: [1,2,3,4,5], k=4, x=-1
 Output: [1,2,3,4]

 Note:
 The value k is positive and will always be smaller than the length of the sorted array.
 Length of the given array is positive and will not exceed 104
 Absolute value of elements in the array and x will not exceed 104
 */
public class _658 {
    public static class Solution1 {
        public List<Integer> findClosestElements(List<Integer> arr, int k, int x) {
            List<Integer> result = new ArrayList<>();
            int index = findInsertPosition(arr, x);
            int right = 0;
            if (arr.get(index) == x) {
                result.add(arr.get(index));
                right = index + 1;
                k--;
            } else {
                right = index;
            }
            int left = index - 1;
            while (k-- > 0) {
                if (left >= 0 && right < arr.size()) {
                    if (x - arr.get(left) <= arr.get(right) - x) {
                        result.add(0, arr.get(left--));
                    } else {
                        result.add(result.size(), arr.get(right++));
                    }
                } else if (left >= 0) {
                    result.add(0, arr.get(left--));
                } else if (right < arr.size()) {
                    result.add(result.size(), arr.get(right++));
```

```
                }
            }
            return result;
        }

        private int findInsertPosition(List<Integer> arr, int x) {
            if (arr == null || arr.size() == 0) {
                return 0;
            }
            int len = arr.size();
            for (int i = 0; i < len; i++) {
                if (arr.get(0) > x) {
                    return 0;
                } else if (arr.get(len - 1) < x) {
                    return len;
                } else if (arr.get(i) < x && arr.get(i + 1) > x) {
                    return i + 1;
                } else if (arr.get(i) == x) {
                    return i;
                }
            }
            return len;
        }
    }
}
```

592. `_659.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 659. Split Array into Consecutive Subsequences
 *
 * You are given an integer array sorted in ascending order (may contain duplicates),
 * you need to split them into several subsequences,
 * where each subsequences consist of at least 3 consecutive integers. Return whether you
can make such a split.

 Example 1:
 Input: [1,2,3,3,4,5]
 Output: True
 Explanation:
 You can split them into two consecutive subsequences :
 1, 2, 3
 3, 4, 5

 Example 2:
 Input: [1,2,3,3,4,4,5,5]
 Output: True
 Explanation:
 You can split them into two consecutive subsequences :
 1, 2, 3, 4, 5
 3, 4, 5

 Example 3:
 Input: [1,2,3,4,4,5]
 Output: False

 Note:
 The length of the input is in range of [1, 10000]
 */

public class _659 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/99187/java-o-n-time-o-n-space
         */
        public boolean isPossible(int[] nums) {
            Map<Integer, Integer> freqMap = new HashMap<>();
            for (int i : nums) {
                freqMap.put(i, freqMap.getOrDefault(i, 0) + 1);
            }
            Map<Integer, Integer> appendFreqMap = new HashMap<>();
            for (int i : nums) {
                if (freqMap.get(i) == 0) {
```

```
                    continue;
                } else if (appendFreqMap.getOrDefault(i, 0) > 0) {
                    appendFreqMap.put(i, appendFreqMap.get(i) - 1);
                    appendFreqMap.put(i + 1, appendFreqMap.getOrDefault(i + 1, 0) + 1);
                } else if (freqMap.getOrDefault(i + 1, 0) > 0 && freqMap.getOrDefault(i +
2, 0) > 0) {
                    freqMap.put(i + 1, freqMap.get(i + 1) - 1);
                    freqMap.put(i + 2, freqMap.get(i + 2) - 1);
                    appendFreqMap.put(i + 3, appendFreqMap.getOrDefault(i + 3, 0) + 1);
                } else {
                    return false;
                }
                freqMap.put(i, freqMap.get(i) - 1);
            }
            return true;
        }
    }
}
```

593. `_660.java`

```
package com.fishercoder.solutions;

/**
 * 660. Remove 9
 *
 * Start from integer 1, remove any integer that contains 9 such as 9, 19, 29...

 So now, you will have a new integer sequence: 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, ...

 Given a positive integer n, you need to return the n-th integer after removing. Note
that 1 will be the first integer.

 Example 1:
 Input: 9
 Output: 10

 Hint: n will not exceed 9 x 10^8.
 */
public class _660 {

    public static class Solution1 {
        public int newInteger(int n) {
            return Integer.parseInt(Integer.toString(n, 9));
        }
    }

    public static class Solution2 {
        public int newInteger(int n) {
            int result = 0;
            int base = 1;
            while (n > 0) {
                result += n % 9 * base;
                n /= 9;
                base *= 10;
            }
            return result;
        }
    }
}
```

594. `_661.java`

```
package com.fishercoder.solutions;

/**
 * 661. Image Smoother
 *
 * Given a 2D integer matrix m representing the gray scale of an image,
 * you need to design a smoother to make the gray scale of each cell becomes the average
gray scale (rounding down) of
 * all the 8 surrounding cells and itself. If a cell has less than 8 surrounding cells,
then use as many as you can.

 Example 1:
 Input:
 [[1,1,1],
 [1,0,1],
 [1,1,1]]

 Output:
 [[0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]

 Explanation:
 For the point (0,0), (0,2), (2,0), (2,2): floor(3/4) = floor(0.75) = 0
 For the point (0,1), (1,0), (1,2), (2,1): floor(5/6) = floor(0.83333333) = 0
 For the point (1,1): floor(8/9) = floor(0.88888889) = 0

 Note:
 The value in the given matrix is in the range of [0, 255].
 The length and width of the given matrix are in the range of [1, 150].

 */
public class _661 {
    public static class Solution1 {
        public int[][] imageSmoother(int[][] M) {
            if (M == null || M.length == 0) {
                return M;
            }
            int m = M.length;
            int n = M[0].length;
            int[][] result = new int[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    bfs(M, i, j, result, m, n);
                }
            }
            return result;
        }

        private void bfs(int[][] M, int i, int j, int[][] result, int m, int n) {
            int sum = M[i][j];
```

```
            int denominator = 1;
            if (j + 1 < n) {
                sum += M[i][j + 1];
                denominator++;
            }
            if (i + 1 < m && j + 1 < n) {
                sum += M[i + 1][j + 1];
                denominator++;
            }
            if (i + 1 < m) {
                sum += M[i + 1][j];
                denominator++;
            }
            if (i + 1 < m && j - 1 >= 0) {
                sum += M[i + 1][j - 1];
                denominator++;
            }
            if (j - 1 >= 0) {
                sum += M[i][j - 1];
                denominator++;
            }
            if (i - 1 >= 0 && j - 1 >= 0) {
                sum += M[i - 1][j - 1];
                denominator++;
            }
            if (i - 1 >= 0) {
                sum += M[i - 1][j];
                denominator++;
            }
            if (i - 1 >= 0 && j + 1 < n) {
                sum += M[i - 1][j + 1];
                denominator++;
            }
            result[i][j] = sum / denominator;
        }
    }
}
```

595. `_662.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.AbstractMap;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;

/**
 * 662. Maximum Width of Binary Tree
 *
 * Given a binary tree, write a function to get the maximum width of the given tree.
 * The width of a tree is the maximum width among all levels.
 * The binary tree has the same structure as a full binary tree, but some nodes are null.
 * The width of one level is defined as the length between the end-nodes
 * (the leftmost and right most non-null nodes in the level,
 * where the null nodes between the end-nodes are also counted into the length
calculation.

 Example 1:
 Input:

      1
    /   \
   3     2
  / \     \
 5   3     9

 Output: 4
 Explanation: The maximum width existing in the third level with the length 4
(5,3,null,9).

 Example 2:
 Input:

     1
    /
   3
  / \
 5   3

 Output: 2
 Explanation: The maximum width existing in the third level with the length 2 (5,3).

 Example 3:
 Input:

     1
```

```
    / \
   3   2
  /
 5

 Output: 2
 Explanation: The maximum width existing in the second level with the length 2 (3,2).
 Example 4:
 Input:

       1
      / \
     3   2
    /     \
   5       9
  /         \
 6           7

 Output: 8
 Explanation:The maximum width existing in the fourth level with the length 8
(6,null,null,null,null,null,null,7).

 Note: Answer will in the range of 32-bit signed integer.
 */
public class _662 {
    public static class Solution1 {
        /**
         * Use a map to store the node to value map,
         * we use root as index 1, then its left child is 2*i-1 and right child is 2*i
         */
        public int widthOfBinaryTree(TreeNode root) {
            if (root == null) {
                return 0;
            }

            Queue<Map.Entry<TreeNode, Integer>> queue = new LinkedList<>();
            queue.offer(new AbstractMap.SimpleEntry<>(root, 1));
            int max = 1;
            while (!queue.isEmpty()) {
                int size = queue.size();
                List<Map.Entry<TreeNode, Integer>> list = new ArrayList<>();
                for (int i = 0; i < size; i++) {
                    Map.Entry<TreeNode, Integer> curr = queue.poll();
                    if (curr.getKey().left != null) {
                        Map.Entry<TreeNode, Integer> newEntry = new
AbstractMap.SimpleEntry<>(curr.getKey().left, curr.getValue() * 2 - 1);
                        queue.offer(newEntry);
                        list.add(newEntry);
                    }
                    if (curr.getKey().right != null) {
                        Map.Entry<TreeNode, Integer> newEntry = new
AbstractMap.SimpleEntry<>(curr.getKey().right, curr.getValue() * 2);
                        queue.offer(newEntry);
                        list.add(newEntry);
```

```
                    }
                }
                if (list.size() > 1) {
                    max = Math.max(list.get(list.size() - 1).getValue() -
list.get(0).getValue() + 1, max);
                }
            }
            return max;
        }
    }
}
```

596. `_663.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 663. Equal Tree Partition
 *
 * Given a binary tree with n nodes,
 * your task is to check if it's possible to partition the tree to two trees which have
the equal sum of values
 * after removing exactly one edge on the original tree.

 Example 1:
 Input:
    5
   / \
  10 10
    /  \
   2   3

 Output: True
 Explanation:
  5
 /
10

 Sum: 15

  10
 /  \
2   3

 Sum: 15


 Example 2:
 Input:
   1
  / \
 2  10
   /  \
  2   20

 Output: False
 Explanation: You can't split the tree into two trees with equal sum after removing
exactly one edge on the tree.

 Note:
```

```
 The range of tree node value is in the range of [-100000, 100000].
 1 <= n <= 10000

 */
public class _663 {
    public static class Solution1 {
        /**
         * The idea is that we use a map to store the sum of each node, then in the end,
         * we check if any node has a sum that is exactly half of total sum.
         */
        public boolean checkEqualTree(TreeNode root) {
            Map<TreeNode, Integer> map = new HashMap<>();
            int totalSum = sumForEachNode(root, map);
            if (totalSum % 2 != 0 || map.size() < 2) {
                return false;
            }
            for (TreeNode key : map.keySet()) {
                if (map.get(key) == totalSum / 2) {
                    return true;
                }
            }
            return false;
        }

        private int sumForEachNode(TreeNode root, Map<TreeNode, Integer> map) {
            if (root == null) {
                return 0;
            }
            if (root.left == null && root.right == null) {
                map.put(root, root.val);
                return root.val;
            }
            int leftVal = 0;
            if (root.left != null) {
                leftVal = sumForEachNode(root.left, map);
            }
            int rightVal = 0;
            if (root.right != null) {
                rightVal = sumForEachNode(root.right, map);
            }
            int val = root.val + leftVal + rightVal;
            map.put(root, val);
            return val;
        }
    }
}
```

597. `_664.java`

```
package com.fishercoder.solutions;

/**
 * 664. Strange Printer
 *
 * There is a strange printer with the following two special requirements:
 * The printer can only print a sequence of the same character each time.
 * At each turn, the printer can print new characters starting from and ending at any
places, and will cover the original existing characters.
 * Given a string consists of lower English letters only, your job is to count the
minimum number of turns the printer needed in order to print it.

 Example 1:
 Input: "aaabbb"
 Output: 2
 Explanation: Print "aaa" first and then print "bbb".

 Example 2:
 Input: "aba"
 Output: 2
 Explanation: Print "aaa" first and then print "b" from the second place of the string,
which will cover the existing character 'a'.

 Hint: Length of the given string will not exceed 100.
 */
public class _664 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/100137/java-solution-dp
         */
        public int strangePrinter(String s) {
            int n = s.length();
            if (n == 0) {
                return 0;
            }

            int[][] dp = new int[101][101];
            for (int i = 0; i < n; i++) {
                dp[i][i] = 1;
            }

            for (int i = 1; i < n; i++) {
                for (int j = 0; j < n - i; j++) {
                    dp[j][j + i] = i + 1;
                    for (int k = j + 1; k <= j + i; k++) {
                        int temp = dp[j][k - 1] + dp[k][j + i];
                        if (s.charAt(k - 1) == s.charAt(j + i)) {
                            temp--;
                        }
                        dp[j][j + i] = Math.min(dp[j][j + i], temp);
                    }
```

```
                }
            }
            return dp[0][n - 1];
        }
    }
}
```

598. `_665.java`

```
package com.fishercoder.solutions;

/**
 * 665. Non-decreasing Array
 *
 *  Given an array with n integers, your task is to check if it could become non-
decreasing by modifying at most 1 element.

 We define an array is non-decreasing if array[i] <= array[i + 1] holds for every i (1 <=
i < n).

 Example 1:

 Input: [4,2,3]
 Output: True
 Explanation: You could modify the first 4 to 1 to get a non-decreasing array.

 Example 2:

 Input: [4,2,1]
 Output: False
 Explanation: You can't get a non-decreasing array by modify at most one element.

 Note: The n belongs to [1, 10,000].
 */
public class _665 {

    public static class Solution1 {
        public boolean checkPossibility(int[] nums) {
            return modifyIMinusOne(nums) || modifyI(nums);
        }

        private boolean modifyI(int[] nums) {
            int[] tmp = nums.clone();
            int len = tmp.length;
            int modifyTimes = 0;
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
                }
                if (tmp[i] < tmp[i - 1]) {
                    modifyTimes++;
                    if (modifyTimes > 1) {
                        return false;
                    }
                    tmp[i] = tmp[i - 1];
                }
            }
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
```

```
                }
                if (tmp[i] < tmp[i - 1]) {
                    return false;
                }
            }
            return true;
        }

        private boolean modifyIMinusOne(int[] nums) {
            int[] tmp = nums.clone();
            int len = tmp.length;
            int modifyTimes = 0;
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
                }
                if (tmp[i] < tmp[i - 1]) {
                    modifyTimes++;
                    if (modifyTimes > 1) {
                        return false;
                    }
                    tmp[i - 1] = tmp[i];
                }
            }
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
                }
                if (tmp[i] < tmp[i - 1]) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

599. `_666.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 666. Path Sum IV
 * If the depth of a tree is smaller than 5, then this tree can be represented by a list
of three-digits integers.

 For each integer in this list:

 The hundreds digit represents the depth D of this node, 1 <= D <= 4.

 The tens digit represents the position P of this node in the level it belongs to, 1 <= P
<= 8.
 The position is the same as that in a full binary tree.

 The units digit represents the value V of this node, 0 <= V <= 9.

 Given a list of ascending three-digits integers representing a binary with the depth
smaller than 5.
 You need to return the totalSum of all paths from the root towards the leaves.

 Example 1:

 Input: [113, 215, 221]
 Output: 12
 Explanation:
 The tree that the list represents is:
   3
  / \
 5   1

 The path totalSum is (3 + 5) + (3 + 1) = 12.

 Example 2:

 Input: [113, 221]
 Output: 4
 Explanation:
 The tree that the list represents is:
 3
  \
  1

 The path totalSum is (3 + 1) = 4.

 */
```

```
public class _666 {
    public static class Solution1 {
        /**OMG, since it's no larger than depth 5, I've got a hardcoded solution here....
         * By "harcoded", I mean the constructTree() method.*/
        public int totalSum = 0;

        public int pathSum(int[] nums) {
            TreeNode root = constructTree(nums);
            if (root == null) {
                return 0;
            }
            computePathSum(root, 0);
            return totalSum;
        }

        private void computePathSum(TreeNode root, int pathSum) {
            pathSum += root.val;
            if (root.left != null) {
                computePathSum(root.left, pathSum);
            }
            if (root.right != null) {
                computePathSum(root.right, pathSum);
            }
            if (root.left == null && root.right == null) {
                totalSum += pathSum;
            }
//            pathSum -= root.val;
            /**this line is not necessary as I'm passing pathSum as a local variable
around, so it's always updated
             it's AC'ed with or without this line*/
        }

        private TreeNode constructTree(int[] nums) {
            if (nums == null || nums.length == 0) {
                return null;
            }
            TreeNode root = new
TreeNode(Integer.parseInt(Integer.toString(nums[0]).substring(2, 3)));
            //depth 2
            for (int i = 1; i < nums.length; i++) {
                if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) == 2 &&
Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 1) {
                    root.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
2 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 2) {
                    root.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                }
            }

            //depth 3
            for (int i = 2; i < nums.length; i++) {
                if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) == 3 &&
```

```
Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 1) {
                    root.left.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
3 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 2) {
                    root.left.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
3 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 3) {
                    root.right.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
3 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 4) {
                    root.right.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                }
            }

            //depth 4
            for (int i = 3; i < nums.length; i++) {
                if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) == 4 &&
Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 1) {
                    root.left.left.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 2) {
                    root.left.left.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 3) {
                    root.left.right.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 4) {
                    root.left.right.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 5) {
                    root.right.left.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 6) {
                    root.right.left.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 7) {
                    root.right.right.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 8) {
                    root.right.right.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                }
            }
```

```
            return root;
        }
    }

    public static class Solution2 {
        public int totalSum = 0;

        public int pathSum(int[] nums) {
            Map<Integer, Integer> map = new HashMap<>();
            for (int i = 0; i < nums.length; i++) {
                int key = nums[i] / 10;
                int value = nums[i] % 10;
                map.put(key, value);
            }
            dfs(nums[0] / 10, 0, map);
            return totalSum;
        }

        private void dfs(int node, int preSum, Map<Integer, Integer> map) {
            int level = node / 10;
            int pos = node % 10;
            int leftChild = (level + 1) * 10 + pos * 2 - 1;
            int rightChild = (level + 1) * 10 + pos * 2;
            int currSum = preSum + map.get(node);
            if (!map.containsKey(leftChild) && !map.containsKey(rightChild)) {
                totalSum += currSum;
                return;
            }

            if (map.containsKey(leftChild)) {
                dfs(leftChild, currSum, map);
            }
            if (map.containsKey(rightChild)) {
                dfs(rightChild, currSum, map);
            }
        }
    }
}
```

600. `_667.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 667. Beautiful Arrangement II
 *
 *  Given two integers n and k, you need to construct a list which contains n different
positive integers ranging from 1 to n
 *  and obeys the following requirement:
 *  Suppose this list is [a1, a2, a3, ... , an],
 *  then the list [|a1 - a2|, |a2 - a3|, |a3 - a4|, ... , |an-1 - an|] has exactly k
distinct integers.
 *  If there are multiple answers, print any of them.

 Example 1:

 Input: n = 3, k = 1
 Output: [1, 2, 3]
 Explanation: The [1, 2, 3] has three different positive integers ranging from 1 to 3,
and the [1, 1] has exactly 1 distinct integer: 1.

 Example 2:

 Input: n = 3, k = 2
 Output: [1, 3, 2]
 Explanation: The [1, 3, 2] has three different positive integers ranging from 1 to 3,
and the [2, 1] has exactly 2 distinct integers: 1 and 2.

 Note:

 The n and k are in the range 1 <= k < n <= 104.
 */

public class _667 {

        public static class Solutoin1 {
                /**This brute force solution will result in TLE as soon as n = 10 and k =
4.*/
                public int[] constructArray(int n, int k) {
                        List<List<Integer>> allPermutaions = findAllPermutations(n);
                        int[] result = new int[n];
                        for (List<Integer> perm : allPermutaions) {
                                if (isBeautifulArrangement(perm, k)) {
                                        convertListToArray(result, perm);
                                        break;
                                }
                        }
```

```
                        return result;
                }

                private void convertListToArray(int[] result, List<Integer> perm) {
                        for (int i = 0; i < perm.size(); i++) {
                                result[i] = perm.get(i);
                        }
                }

                private boolean isBeautifulArrangement(List<Integer> perm, int k) {
                        Set<Integer> diff = new HashSet<>();
                        for (int i = 0; i < perm.size() - 1; i++) {
                                diff.add(Math.abs(perm.get(i) - perm.get(i + 1)));
                        }
                        return diff.size() == k;
                }

                private List<List<Integer>> findAllPermutations(int n) {
                        List<List<Integer>> result = new ArrayList<>();
                        backtracking(new ArrayList<>(), result, n);
                        return result;
                }

                private void backtracking(List<Integer> list, List<List<Integer>> result,
int n) {
                        if (list.size() == n) {
                                result.add(new ArrayList<>(list));
                                return;
                        }
                        for (int i = 1; i <= n; i++) {
                                if (list.contains(i)) {
                                        continue;
                                }
                                list.add(i);
                                backtracking(list, result, n);
                                list.remove(list.size() - 1);
                        }
                }
        }

        public static class Solutoin2 {
                /**This is a very smart solution:
                 * First, we can see that the max value k could reach is n-1 which
                 * comes from a sequence like this:
                 * when n = 8, k = 5, one possible sequence is:
                 * 1, 8, 2, 7, 3, 4, 5, 6
                 * absolute diffs are:
                 *  7, 6, 5, 4, 1, 1, 1
                 *  so, there are total 5 distinct integers.
                 *
                 *  So, we can just form such a sequence by putting the first part first
and
                 *  decrement k along the way, when k becomes 1, we just put the rest
numbers in order.*/
```

```
                public int[] constructArray(int n, int k) {
                        int[] result = new int[n];
                        int left = 1;
                        int right = n;
                        for (int i = 0; i < n && left <= right; i++) {
                                if (k > 1) {
                                        result[i] = k-- % 2 != 0 ? left++ : right--;
                                } else {
                                        result[i] = k % 2 != 0 ? left++ : right--;
                                }
                        }
                        return result;
                }
        }
}
```

601. `_668.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 668. Kth Smallest Number in Multiplication Table
 *
 *  Nearly every one have used the Multiplication Table.
 *  But could you find out the k-th smallest number quickly from the multiplication
table?
 *  Given the height m and the length n of a m * n Multiplication Table,
 *  and a positive integer k, you need to return the k-th smallest number in this table.

 Example 1:
 Input: m = 3, n = 3, k = 5
 Output:
 Explanation:
 The Multiplication Table:
 1      2       3
 2      4       6
 3      6       9

 The 5-th smallest number is 3 (1, 2, 2, 3, 3).


 Example 2:
 Input: m = 2, n = 3, k = 6
 Output:
 Explanation:
 The Multiplication Table:
 1      2       3
 2      4       6

 The 6-th smallest number is 6 (1, 2, 2, 3, 4, 6).

 Note:

 The m and n will be in the range [1, 30000].
 The k will be in the range [1, m * n]
 */

public class _668 {
    public static class Solution1 {
        /**
         * This brute force approach resulted in
         * TLE on Leetcode and
         * OOM error by _668test.test3() when running in my localhost:
         * java.lang.OutOfMemoryError: Java heap space
         * at java.util.Arrays.copyOf(Arrays.java:3210)
         * at java.util.Arrays.copyOf(Arrays.java:3181)
         * at java.util.PriorityQueue.grow(PriorityQueue.java:300)
```

```
         * at java.util.PriorityQueue.offer(PriorityQueue.java:339)
         */
        public int findKthNumber(int m, int n, int k) {
            PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> a - b);
            for (int i = 1; i <= m; i++) {
                for (int j = 1; j <= n; j++) {
                    minHeap.offer(i * j);
                }
            }
            while (k-- > 1) {
                minHeap.poll();
            }
            return minHeap.peek();
        }
    }

    public static class Solution2 {
        /**reference: https://discuss.leetcode.com/topic/101132/java-solution-binary-
search*/
        public int findKthNumber(int m, int n, int k) {
            int low = 1;
            int high = m * n + 1;
            while (low < high) {
                int mid = low + (high - low) / 2;
                int c = count(mid, m, n);
                if (c >= k) {
                    high = mid;
                } else {
                    low = mid + 1;
                }
            }
            return high;
        }

        int count(int v, int m, int n) {
            int count = 0;
            for (int i = 1; i <= m; i++) {
                int temp = Math.min(v / i, n);
                count += temp;
            }
            return count;
        }
    }
}
```

602. `_669.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 669. Trim a Binary Search Tree
 *
 * Given a binary search tree and the lowest and highest boundaries as L and R,
 * trim the tree so that all its elements lies in [L, R] (R >= L).
 * You might need to change the root of the tree, so the result should return the new
root of the trimmed binary search tree.

 Example 1:

 Input:
   1
  / \
 0   2

 L = 1
 R = 2

 Output:
 1
  \
  2

 Example 2:

 Input:
  3
 / \
0   4
 \
 2
 /
1

 L = 1
 R = 3

 Output:
    3
   /
  2
 /
1

 */
public class _669 {
    public static class Solution1 {
```

```
        public TreeNode trimBST(TreeNode root, int L, int R) {
            if (root == null) {
                return root;
            }

            if (root.val > R) {
                return trimBST(root.left, L, R);
            }

            if (root.val < L) {
                return trimBST(root.right, L, R);
            }

            root.left = trimBST(root.left, L, R);
            root.right = trimBST(root.right, L, R);
            return root;
        }
    }
}
```

603. `_670.java`

```
package com.fishercoder.solutions;

/**
 * 670. Maximum Swap
 *
 * Given a non-negative integer, you could swap two digits at most once to get the
maximum valued number.
 * Return the maximum valued number you could get.

 Example 1:
 Input: 2736
 Output: 7236
 Explanation: Swap the number 2 and the number 7.

 Example 2:
 Input: 9973
 Output: 9973
 Explanation: No swap.

 Note:
 The given number is in the range [0, 108]

 */
public class _670 {
    public static class Solution1 {
        public int maximumSwap(int num) {
            String numStr = String.valueOf(num);
            int max = num;
            for (int i = 0; i < numStr.length() - 1; i++) {
                for (int j = i + 1; j < numStr.length(); j++) {
                    if (numStr.charAt(i) < numStr.charAt(j)) {
                        StringBuilder sb = new StringBuilder(numStr);
                        sb.replace(i, i + 1, String.valueOf(numStr.charAt(j)));
                        sb.replace(j, j + 1, String.valueOf(numStr.charAt(i)));
                        max = Math.max(max, Integer.parseInt(sb.toString()));
                    }
                }
            }
            return max;
        }
    }
}
```

604. `_671.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;

/**
 * 671. Second Minimum Node In a Binary Tree
 *
 *  Given a non-empty special binary tree consisting of nodes with the non-negative
value,
 *  where each node in this tree has exactly two or zero sub-node.
 *  If the node has two sub-nodes, then this node's value is the smaller value among its
two sub-nodes.
 *  Given such a binary tree, you need to output the second minimum value in the set made
of all the nodes' value in the whole tree.
 *  If no such second minimum value exists, output -1 instead.

 Example 1:

 Input:
   2
  / \
 2   5
    / \
   5   7

 Output: 5
 Explanation: The smallest value is 2, the second smallest value is 5.

 Example 2:

 Input:
  2
 / \
2  2

 Output: -1
 Explanation: The smallest value is 2, but there isn't any second smallest value.

 */
public class _671 {
    public static class Solution1 {
        public int findSecondMinimumValue(TreeNode root) {
            if (root == null) {
                return -1;
            }
            Set<Integer> set = new TreeSet<>();
            dfs(root, set);
```

```
            Iterator<Integer> iterator = set.iterator();
            int count = 0;
            while (iterator.hasNext()) {
                count++;
                int result = iterator.next();
                if (count == 2) {
                    return result;
                }
            }
            return -1;
        }

        private void dfs(TreeNode root, Set<Integer> set) {
            if (root == null) {
                return;
            }
            set.add(root.val);
            dfs(root.left, set);
            dfs(root.right, set);
            return;
        }
    }
}
```

605. `_672.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 672. Bulb Switcher II
 * There is a room with n lights which are turned on initially and 4 buttons on the wall.
 * After performing exactly m unknown operations towards buttons,
 * you need to return how many different kinds of status of the n lights could be.
 * Suppose n lights are labeled as number [1, 2, 3 ..., n], function of these 4 buttons
are given below:

 Flip all the lights.
 Flip lights with even numbers.
 Flip lights with odd numbers.
 Flip lights with (3k + 1) numbers, k = 0, 1, 2, ...

 Example 1:

 Input: n = 1, m = 1.
 Output: 2
 Explanation: Status can be: [on], [off]

 Example 2:

 Input: n = 2, m = 1.
 Output: 3
 Explanation: Status can be: [on, off], [off, on], [off, off]

 Example 3:

 Input: n = 3, m = 1.
 Output: 4
 Explanation: Status can be: [off, on, off], [on, off, on], [off, off, off], [off, on,
on].

 Note: n and m both fit in range [0, 1000].
 */
public class _672 {
    public static class Solution1 {
        public int flipLights(int n, int m) {
            if (m < 1) {
                return 1;
            }
            int[][] dp = new int[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (j == 0) {
                        dp[i][j] = 2;
                    } else if (i == 0 && j == 1) {
                        dp[i][j] = 3;
```

```
                    } else if (i == 0) {
                        dp[i][j] = 4;
                    } else if (i == 1 && j == 1) {
                        dp[i][j] = 4;
                    } else if (i == 1 && j > 1) {
                        dp[i][j] = 7;
                    } else if (j == 1) {
                        dp[i][j] = 4;
                    } else if (i == 1) {
                        dp[i][j] = 7;
                    } else {
                        dp[i][j] = 8;
                    }
                }
            }
            CommonUtils.print2DIntArray(dp);
            return dp[m - 1][n - 1];
        }
    }

    public static class Solution2 {
        public int flipLights(int n, int m) {
            if (n == 1 && m > 0) {
                return 2;
            } else if (n == 2 && m == 1) {
                return 3;
            } else if ((n > 2 && m == 1) || (n == 2 && m > 1)) {
                return 4;
            } else if (n > 2 && m == 2) {
                return 7;
            } else if (n > 2 && m > 2) {
                return 8;
            } else {
                return 1;
            }
        }
    }
}
```

606. `_673.java`

```
package com.fishercoder.solutions;

/**
 * 673. Number of Longest Increasing Subsequence
 * Given an unsorted array of integers, find the number of longest increasing
subsequence.

 Example 1:

 Input: [1,3,5,4,7]
 Output: 2
 Explanation: The two longest increasing subsequence are [1, 3, 4, 7] and [1, 3, 5, 7].

 Example 2:

 Input: [2,2,2,2,2]
 Output: 5
 Explanation: The length of longest continuous increasing subsequence is 1,
 and there are 5 subsequences' length is 1, so output 5.

 Note: Length of the given array will be not exceed 2000 and the answer is guaranteed to
be fit in 32-bit signed int.
 */
public class _673 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/103020/java-c-simple-dp-
solution-with-explanation
         */
        public int findNumberOfLIS(int[] nums) {
            int n = nums.length;
            int[] cnt = new int[n];
            int[] len = new int[n];
            int max = 0;
            int count = 0;
            for (int i = 0; i < n; i++) {
                len[i] = cnt[i] = 1;
                for (int j = 0; j < i; j++) {
                    if (nums[i] > nums[j]) {
                        if (len[i] == len[j] + 1) {
                            cnt[i] += cnt[j];
                        }
                        if (len[i] < len[j] + 1) {
                            len[i] = len[j] + 1;
                            cnt[i] = cnt[j];
                        }
                    }
                }

                if (max == len[i]) {
                    count += cnt[i];
```

```
                }

                if (len[i] > max) {
                    max = len[i];
                    count = cnt[i];
                }
            }
            return count;
        }
    }
}
```

607. `_674.java`

```
package com.fishercoder.solutions;

/**
 * 674. Longest Continuous Increasing Subsequence
 * Given an unsorted array of integers, find the length of longest continuous increasing
subsequence.

 Example 1:
 Input: [1,3,5,4,7]
 Output: 3
 Explanation: The longest continuous increasing subsequence is [1,3,5],
 its length is 3. Even though [1,3,5,7] is also an increasing subsequence,
 it's not a continuous one where 5 and 7 are separated by 4.

 Example 2:
 Input: [2,2,2,2,2]
 Output: 1
 Explanation: The longest continuous increasing subsequence is [2], its length is 1.

 Note: Length of the array will not exceed 10,000.
 */
public class _674 {
    public static class Solution1 {
        public int findLengthOfLCIS(int[] nums) {
            int longest = 0;
            for (int i = 0; i < nums.length; i++) {
                int len = 1;
                for (int j = i + 1; j < nums.length; j++) {
                    if (nums[j - 1] < nums[j]) {
                        len++;
                        continue;
                    } else {
                        break;
                    }
                }
                longest = Math.max(longest, len);
            }
            return longest;
        }
    }
}
```

608. `_675.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 675. Cut Off Trees for Golf Event
 *
 * You are asked to cut off trees in a forest for a golf event.
 * The forest is represented as a non-negative 2D map, in this map:
 * 0 represents the obstacle can't be reached.
 * 1 represents the ground can be walked through.
 *
 * The place with number bigger than 1 represents a tree can be walked through,
 * and this positive number represents the tree's height.
 *
 * You are asked to cut off all the trees in this forest in the order of tree's height -
 * always cut off the tree with lowest height first.
 * And after cutting, the original place has the tree will become a grass (value 1).
 *
 * You will start from the point (0, 0) and you should output the minimum steps you need
to walk to cut off all the trees.
 *
 * If you can't cut off all the trees, output -1 in that situation.
 * You are guaranteed that no two trees have the same height and there is at least one
tree needs to be cut off.
 *
 * Example 1:
 * Input:
 * [
 * [1,2,3],
 * [0,0,4],
 * [7,6,5]
 * ]
 * Output: 6
 *
 * Example 2:
 * Input:
 * [
 * [1,2,3],
 * [0,0,0],
 * [7,6,5]
 * ]
 * Output: -1
 *
 * Example 3:
 * Input:
 * [
 * [2,3,4],
```

```
 * [0,0,5],
 * [8,7,6]
 * ]
 * Output: 6
 *
 * Explanation: You started from the point (0,0) and you can cut off the tree in (0,0)
directly without walking.
 * Hint: size of the given matrix will not exceed 50x50.
 */

public class _675 {
    public static class Solution1 {
        public int cutOffTree(List<List<Integer>> forest) {
            if (forest == null || forest.isEmpty() || forest.size() == 0 ||
forest.get(0).get(0) == 0) {
                return -1;
            }
            int m = forest.size();
            int n = forest.get(0).size();
            /**cut trees in ascending order*/
            PriorityQueue<Tree> heap = new PriorityQueue<>((a, b) -> a.height -
b.height);
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (forest.get(i).get(j) > 1) {
                        /**This is important: we'll add trees that are only taller than
1!!!*/
                        heap.offer(new Tree(i, j, forest.get(i).get(j)));
                    }
                }
            }

            int sum = 0;
            Tree start = new Tree();
            while (!heap.isEmpty()) {
                Tree curr = heap.poll();
                int step = bfs(forest, curr, start, m, n);
                if (step == -1) {
                    return -1;
                }
                sum += step;
                start = curr;
            }
            return sum;
        }

        private int bfs(List<List<Integer>> forest, Tree target, Tree start, int m, int
n) {
            int[] dirs = new int[]{0, 1, 0, -1, 0};
            boolean[][] visited = new boolean[m][n];
            Queue<Tree> queue = new LinkedList<>();
            queue.offer(start);
            visited[start.x][start.y] = true;
            int step = 0;
```

```
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int k = 0; k < size; k++) {
                    Tree tree = queue.poll();
                    if (tree.x == target.x && tree.y == target.y) {
                        return step;
                    }

                    for (int i = 0; i < 4; i++) {
                        int nextX = tree.x + dirs[i];
                        int nextY = tree.y + dirs[i + 1];
                        if (nextX < 0 || nextY < 0 || nextX >= m || nextY >= n ||
visited[nextX][nextY] || forest.get(nextX).get(nextY) == 0) {
                            continue;
                        }
                        queue.offer(new Tree(nextX, nextY,
forest.get(nextX).get(nextY)));
                        visited[nextX][nextY] = true;
                    }
                }
                step++;
            }
            return -1;
        }

        class Tree {
            int x;
            int y;
            int height;

            public Tree(int x, int y, int height) {
                this.x = x;
                this.y = y;
                this.height = height;
            }

            public Tree() {
                this.x = 0;
                this.y = 0;
                this.height = 0;
            }
        }
    }
}
```

609. `_676.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 676. Implement Magic Dictionary
 * Implement a magic directory with buildDict, and search methods.
 * For the method buildDict, you'll be given a list of non-repetitive words to build a
dictionary.
 * For the method search, you'll be given a word,
 * and judge whether if you modify exactly one character into another character in this
word,
 * the modified word is in the dictionary you just built.

 Example 1:

 Input: buildDict(["hello", "leetcode"]), Output: Null
 Input: search("hello"), Output: False
 Input: search("hhllo"), Output: True
 Input: search("hell"), Output: False
 Input: search("leetcoded"), Output: False

 Note:

 You may assume that all the inputs are consist of lowercase letters a-z.
 For contest purpose, the test data is rather small by now.
 You could think about highly efficient algorithm after the contest.
 Please remember to RESET your class variables declared in class MagicDictionary,
 as static/class variables are persisted across multiple test cases. Please see here for
more details.

 */
public class _676 {

    public static class Solution1 {
        public static class MagicDictionary {

            Set<String> wordSet;

            /**
             * Initialize your data structure here.
             */
            public MagicDictionary() {
                wordSet = new HashSet<>();
            }

            /**
             * Build a dictionary through a list of words
             */
            public void buildDict(String[] dict) {
```

```
                for (String word : dict) {
                    wordSet.add(word);
                }
            }

            /**
             * Returns if there is any word in the trie that equals to the given word
after modifying exactly one character
             */
            public boolean search(String word) {
                for (String candidate : wordSet) {
                    if (modifyOneChar(word, candidate)) {
                        return true;
                    }
                }
                return false;
            }

            private boolean modifyOneChar(String word, String candidate) {
                if (word.length() != candidate.length()) {
                    return false;
                }
                int diff = 0;
                for (int i = 0; i < word.length(); i++) {
                    if (word.charAt(i) != candidate.charAt(i)) {
                        diff++;
                    }
                    if (diff > 1) {
                        return false;
                    }
                }
                return diff == 1;
            }
        }
    }
}
```

610. `_677.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 677. Map Sum Pairs
 *
 *  Implement a MapSum class with insert, and sum methods.

 For the method insert, you'll be given a pair of (string, integer).
 The string represents the key and the integer represents the value.
 If the key already existed, then the original key-value pair will be overridden to the
new one.

 For the method sum, you'll be given a string representing the prefix,
 and you need to return the sum of all the pairs' value whose key starts with the prefix.

 Example 1:

 Input: insert("apple", 3), Output: Null
 Input: sum("ap"), Output: 3
 Input: insert("app", 2), Output: Null
 Input: sum("ap"), Output: 5

 */
public class _677 {
    public static class Solution1 {
        public static class MapSum {

            Map<String, Integer> map;

            /**
             * Initialize your data structure here.
             */
            public MapSum() {
                map = new HashMap<>();
            }

            public void insert(String key, int val) {
                map.put(key, val);
            }

            public int sum(String prefix) {
                int sum = 0;
                for (String key : map.keySet()) {
                    if (key.startsWith(prefix)) {
                        sum += map.get(key);
                    }
                }
                return sum;
```

```
            }
        }

    }
}
```

611. `_678.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 678. Valid Parenthesis String
 *
 *  Given a string containing only three types of characters: '(', ')' and '*',
 *  write a function to check whether this string is valid. We define the validity of a
string by these rules:

 1. Any left parenthesis '(' must have a corresponding right parenthesis ')'.
 2. Any right parenthesis ')' must have a corresponding left parenthesis '('.
 3. Left parenthesis '(' must go before the corresponding right parenthesis ')'.
 4. '*' could be treated as a single right parenthesis ')' or a single left parenthesis
'(' or an empty string.
 5. An empty string is also valid.

 Example 1:
 Input: "()"
 Output: True

 Example 2:
 Input: "(*)"
 Output: True

 Example 3:
 Input: "(*))"
 Output: True

 */
public class _678 {
    public static class Solution1 {
        /**
         * This solution is correct, but will result in TLE by test4
         */
        public boolean checkValidString(String s) {
            if (!s.contains("*")) {
                Stack<Character> stack = new Stack();
                int i = 0;
                while (i < s.length()) {
                    if (s.charAt(i) == '(') {
                        stack.push(s.charAt(i));
                    } else {
                        if (stack.isEmpty()) {
                            return false;
                        } else {
                            stack.pop();
                        }
                    }
                    i++;
```

```
            }
            return stack.isEmpty();
        } else {
            int index = s.indexOf("*");
            String transformedS = s.substring(0, index) + s.substring(index + 1);
            if (checkValidString(transformedS)) {
                return true;
            }
            transformedS = s.substring(0, index) + "(" + s.substring(index + 1);
            if (checkValidString(transformedS)) {
                return true;
            }
            transformedS = s.substring(0, index) + ")" + s.substring(index + 1);
            if (checkValidString(transformedS)) {
                return true;
            }
        }
        return false;
    }
}

public static class Solution2 {
    public boolean checkValidString(String s) {
        return isValid(s, 0, 0);
    }

    private boolean isValid(String s, int start, int cnt) {
        if (cnt < 0) {
            return false;
        }
        for (int i = start; i < s.length(); i++) {
            char c = s.charAt(i);
            if (c == '(') {
                cnt++;
            } else if (c == ')') {
                if (cnt <= 0) {
                    return false;
                }
                cnt--;
            } else if (c == '*') {
                /**Extra caution: start should be i+1, not start+1 !*/
                return isValid(s, i + 1, cnt + 1)
                        || isValid(s, i + 1, cnt - 1)
                        || isValid(s, i + 1, cnt);
            }
        }
        return cnt == 0;
    }
}

public static class Solution3 {
    /**
     * Greedy solution:
     * 1. Let lo mean the lowest possible open left paren
```

```
         * 2. Let hi mean the possibilities of highest possible open left paren, so as
long as s.charAt(i) != ')', it's possible to be a '(', so we'll increment hi by 1
         * 2. If at any time, hi becomes negative, that means this string will never be
valid
         * 3. Each time, we'll let lo be at a minimum of zero
         */
        public boolean checkValidString(String s) {
            int lo = 0;
            int hi = 0;
            for (int i = 0; i < s.length(); i++) {
                lo += s.charAt(i) == '(' ? 1 : -1;
                hi += s.charAt(i) != ')' ? 1 : -1;
                if (hi < 0) {
                    break;
                }
                lo = Math.max(0, lo);
            }
            return lo == 0;
        }
    }
}
```

612. `_679.java`

```
package com.fishercoder.solutions;

import java.util.stream.IntStream;

/**
 * 679. 24 Game
 *
 *  You have 4 cards each containing a number from 1 to 9.
 *  You need to judge whether they could operated through *, /, +, -, (, ) to get the
value of 24.

 Example 1:
 Input: [4, 1, 8, 7]
 Output: True
 Explanation: (8-4) * (7-1) = 24

 Example 2:
 Input: [1, 2, 1, 2]
 Output: False

 Note:
 The division operator / represents real division, not integer division. For example, 4 /
(1 - 2/3) = 12.
 Every operation done is between two numbers.
 In particular, we cannot use - as a unary operator. For example, with [1, 1, 1, 1] as
input, the expression -1 - 1 - 1 - 1 is not allowed.
 You cannot concatenate numbers together. For example, if the input is [1, 2, 1, 2], we
cannot write this as 12 + 12.

 */

public class _679 {
    public static class Solution1 {
        /**Since there are only 4 cards and only 4 operations, we can iterate through all
possible combinations, there's a total of 9216 possibilities:
         * 1. we pick two out of four cards, with order (since order matters for
division), 4 * 3 = 12, then pick one of four operations: 12 * 4 = 48;
         * 2. then we pick two from these three numbers: 12 * 4 * 3 * 4 * 2 = 1152
         * 3. then we pick the remaining two: 1152 * 2 * 4 = 9216 (with order and out of
4 operations)*/
        public boolean judgePoint24(int[] nums) {
            return dfs(IntStream.of(nums).mapToDouble(num -> num).toArray());
        }

        private boolean dfs(double[] nums) {
            if (nums.length == 1) {
                return Math.abs(nums[0] - 24) < 1e-8;//1e-8 means 0.000000001, i.e.
10^(-8)
            }

            for (int i = 0; i < nums.length; i++) {
```

```
                for (int j = 0; j < nums.length; j++) {
                    if (i != j) {
                        int len = 0;
                        double[] a = new double[nums.length - 1];
                        for (int k = 0; k < nums.length; k++) {
                            if (k != i && k != j) {
                                a[len++] = nums[k];
                            }
                        }

                        a[len] = nums[i] + nums[j];
                        if (dfs(a)) {
                            return true;
                        }

                        a[len] = nums[i] - nums[j];
                        if (dfs(a)) {
                            return true;
                        }

                        a[len] = nums[i] * nums[j];
                        if (dfs(a)) {
                            return true;
                        }

                        if (nums[j] > 1e-8) {
                            a[len] = nums[i] / nums[j];
                            if (dfs(a)) {
                                return true;
                            }
                        }
                    }
                }
            }
            return false;
        }
    }
}
```

613. `_680.java`

```
package com.fishercoder.solutions;

/**
 * 680. Valid Palindrome II
 *
 *  Given a non-empty string s, you may delete at most one character. Judge whether you
can make it a palindrome.

 Example 1:
 Input: "aba"
 Output: True

 Example 2:
 Input: "abca"
 Output: True
 Explanation: You could delete the character 'c'.

 Note:
 The string will only contain lowercase characters a-z. The maximum length of the string
is 50000.

 */
public class _680 {
    public static class Solution1 {
        public boolean validPalindrome(String s) {
            int left = 0;
            int right = s.length() - 1;
            int diff = 0;
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) {
                    left++;
                    diff++;
                    if (diff > 1) {
                        break;
                    }
                } else {
                    left++;
                    right--;
                }
            }
            if (diff < 2) {
                return true;
            }
            diff = 0;
            left = 0;
            right = s.length() - 1;
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) {
                    right--;
                    diff++;
                    if (diff > 1) {
```

```
                        break;
                    }
                } else {
                    left++;
                    right--;
                }
            }
            return diff < 2;
        }
    }

    public static class Solution2 {
        public boolean validPalindrome(String s) {
            int left = 0;
            int right = s.length() - 1;
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) {
                    return isValid(s, left + 1, right) || isValid(s, left, right - 1);
                }
                left++;
                right--;
            }
            return true;
        }

        private boolean isValid(String s, int left, int right) {
            while (left < right) {
                if (s.charAt(left++) != s.charAt(right--)) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

614. `_681.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 681. Next Closest Time
 *
 * Given a time represented in the format "HH:MM",
 * form the next closest time by reusing the current digits.
 * There is no limit on how many times a digit can be reused.
 * You may assume the given input string is always valid.
 * For example, "01:34", "12:09" are all valid. "1:34", "12:9" are all invalid.

 Example 1:

 Input: "19:34"
 Output: "19:39"
 Explanation: The next closest time choosing from digits 1, 9, 3, 4, is 19:39,
 which occurs 5 minutes later.  It is not 19:33, because this occurs 23 hours and 59
minutes later.

 Example 2:

 Input: "23:59"
 Output: "22:22"
 Explanation: The next closest time choosing from digits 2, 3, 5, 9, is 22:22.
 It may be assumed that the returned time is next day's time since it is smaller than the
input time numerically.

 */
public class _681 {
    public static class Solution1 {
        public String nextClosestTime(String time) {
            int cur = 60 * Integer.parseInt(time.substring(0, 2));
            cur += Integer.parseInt(time.substring(3));
            Set<Integer> allowed = new HashSet();
            for (char c : time.toCharArray()) {
                if (c != ':') {
                    allowed.add(c - '0');
                }
            }

            while (true) {
                cur = (cur + 1) % (24 * 60);
                int[] digits = new int[]{cur / 60 / 10, cur / 60 % 10, cur % 60 / 10, cur
% 60 % 10};
                search:
                {
                    for (int d : digits) {
                        if (!allowed.contains(d)) {
```

```
                            break search;
                        }
                    }
                    return String.format("%02d:%02d", cur / 60, cur % 60);
                }
            }
        }
    }
}
```

615. `_682.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 682. Baseball Game
 *
 *  You're now a baseball game point recorder.

 Given a list of strings, each string can be one of the 4 following types:

 Integer (one round's score): Directly represents the number of points you get in this
round.
 "+" (one round's score): Represents that the points you get in this round are the sum of
the last two valid round's points.
 "D" (one round's score): Represents that the points you get in this round are the
doubled data of the last valid round's points.
 "C" (an operation, which isn't a round's score): Represents the last valid round's
points you get were invalid and should be removed.

 Each round's operation is permanent and could have an impact on the round before and the
round after.

 You need to return the sum of the points you could get in all the rounds.

 Example 1:

 Input: ["5","2","C","D","+"]
 Output: 15
 Explanation:
 Round 1: You could get 5 points. The sum is: 5.
 Round 2: You could get 2 points. The sum is: 7.
 Operation 1: The round 2's data was invalid. The sum is: 5.
 Round 3: You could get 10 points (the round 2's data has been removed). The sum is: 15.
 Round 4: You could get 5 + 10 = 15 points. The sum is: 30.

 Example 2:

 Input: ["5","-2","4","C","D","9","+","+"]
 Output: 27
 Explanation:
 Round 1: You could get 5 points. The sum is: 5.
 Round 2: You could get -2 points. The sum is: 3.
 Round 3: You could get 4 points. The sum is: 7.
 Operation 1: The round 3's data is invalid. The sum is: 3.
 Round 4: You could get -4 points (the round 3's data has been removed). The sum is: -1.
 Round 5: You could get 9 points. The sum is: 8.
 Round 6: You could get -4 + 9 = 5 points. The sum is 13.
 Round 7: You could get 9 + 5 = 14 points. The sum is 27.

 Note:
```

```
 The size of the input list will be between 1 and 1000.
 Every integer represented in the list will be between -30000 and 30000.
 */
public class _682 {

    public static class Solution1 {
        public int calPoints(String[] ops) {
            Stack<Integer> stack = new Stack<>();
            int sum = 0;
            int firstLast = Integer.MIN_VALUE;
            int secondLast = Integer.MIN_VALUE;
            for (String op : ops) {
                if (op.equals("+")) {
                    if (!stack.isEmpty()) {
                        firstLast = stack.pop();
                    }
                    if (!stack.isEmpty()) {
                        secondLast = stack.pop();
                    }
                    int thisRoundPoints = firstLast + secondLast;

                    if (secondLast != Integer.MIN_VALUE) {
                        stack.push(secondLast);
                    }
                    if (firstLast != Integer.MIN_VALUE) {
                        stack.push(firstLast);
                    }
                    stack.push(thisRoundPoints);
                    sum += thisRoundPoints;

                    firstLast = Integer.MIN_VALUE;
                    secondLast = Integer.MIN_VALUE;
                } else if (op.equals("D")) {
                    if (!stack.isEmpty()) {
                        int thisRoundPoints = stack.peek() * 2;
                        stack.push(thisRoundPoints);
                        sum += thisRoundPoints;
                    }
                } else if (op.equals("C")) {
                    if (!stack.isEmpty()) {
                        int removedData = stack.pop();
                        sum -= removedData;
                    }
                } else {
                    Integer val = Integer.parseInt(op);
                    sum += val;
                    stack.push(val);
                }
            }
            return sum;
        }
    }
}
```

616. `_683.java`

```
package com.fishercoder.solutions;

/**
 * 683. K Empty Slots
 *
 *  There is a garden with N slots. In each slot, there is a flower.
 *
 *  The N flowers will bloom one by one in N days.
 *
 *  In each day, there will be exactly one flower blooming and it will be in the status
of blooming since then.
 *
 *  Given an array flowers consists of number from 1 to N. Each number in the array
represents the place where the flower will open in that day.
 *  For example, flowers[i] = x means that the unique flower that blooms at day i will be
at position x,
 *  where i and x will be in the range from 1 to N.
 *
 *  Also given an integer k, you need to output in which day there exists two flowers in
the status of blooming,
 *  and also the number of flowers between them is k and these flowers are not blooming.
 *  If there isn't such day, output -1.

 Example 1:
 Input:
 flowers: [1,3,2]
 k: 1
 Output: 2
 Explanation: In the second day, the first and the third flower have become blooming.

 Example 2:
 Input:
 flowers: [1,2,3]
 k: 1
 Output: -1

 Note:
 The given array will be in the range [1, 20000].

 */
public class _683 {

    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/104771/java-c-simple-o-n-solution*/
        public int kEmptySlots(int[] flowers, int k) {
            int[] days = new int[flowers.length];
            for (int i = 0; i < flowers.length; i++) {
                days[flowers[i] - 1] = i + 1;
            }
            int left = 0;
            int right = k + 1;
```

```
            int result = Integer.MAX_VALUE;
            for (int i = 0; right < flowers.length; i++) {
                if (days[i] < days[left] || days[i] <= days[right]) {
                    if (i == right) {
                        result = Math.min(result, Math.max(days[left], days[right]));
                    }
                    left = i;
                    right = k + 1 + i;
                }
            }
            return result == Integer.MAX_VALUE ? -1 : result;
        }
    }
}
```

617. `_684.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 684. Redundant Connection
 *
 *  We are given a "tree" in the form of a 2D-array, with distinct values for each node.
 *  In the given 2D-array, each element pair [u, v] represents that v is a child of u in
the tree.
 *  We can remove exactly one redundant pair in this "tree" to make the result a (rooted)
tree.
 *  You need to find and output such a pair. If there are multiple answers for this
question, output the one appearing last in the 2D-array. There is always at least one
answer.

 Example 1:
 Input: [[1,2], [1,3], [2,3]]
 Output: [2,3]
 Explanation: Original tree will be like this:
   1
  / \
 2 - 3


 Example 2:
 Input: [[1,2], [1,3], [3,1]]
 Output: [3,1]
 Explanation: Original tree will be like this:
   1
  / \\
 2   3

 Note:
 The size of the input 2D-array will be between 1 and 1000.
 Every integer represented in the 2D-array will be between 1 and 2000.
 */
public class _684 {

    public static class Solution1 {
        /**
         * This is my original solution. A little verbose.
         */
        class UnionFind {
            int[] ids;
            Set<Integer> nodes;
            Set<Integer> visitedNodes;
            int[] redundantConn;
            int m;
            int n;
```

```
            public UnionFind(int[][] edges) {
                m = edges.length;
                n = edges[0].length;
                nodes = new HashSet<>();
                visitedNodes = new HashSet<>();
                redundantConn = new int[]{};
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        nodes.add(edges[i][j]);
                    }
                }
                ids = new int[nodes.size()];
                for (int i = 0; i < ids.length; i++) {
                    ids[i] = i + 1;
                }
            }

            public int[] union(int[] edge) {
                int x = find(edge[0] - 1);
                int y = find(edge[1] - 1);

                if (x == y && visitedNodes.contains(edge[0]) &&
visitedNodes.contains(edge[1])) {
                    redundantConn = edge;
                }

                if (x != y) {
                    if (x < y) {
                        ids[y] = x + 1;
                    } else {
                        ids[x] = y + 1;
                    }
                }

                visitedNodes.add(edge[0]);
                visitedNodes.add(edge[1]);
                return redundantConn;
            }

            private int find(int id) {
                if (isTree()) {
                    return ids[id];
                }
                if ((id + 1) != ids[id]) {
                    return find(ids[id] - 1);
                }
                return id;
            }

            private boolean isTree() {
                Set<Integer> set = new HashSet<>();
                for (int i : ids) {
                    set.add(i);
```

```
                }
                return set.size() == 1;
            }
        }

        public int[] findRedundantConnection(int[][] edges) {
            UnionFind unionFind = new UnionFind(edges);
            int[] result = new int[]{};
            for (int[] edge : edges) {
                result = unionFind.union(edge);
            }
            return result;
        }
    }
}
```

618. `_685.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 685. Redundant Connection II
 *
 * In this problem, a rooted tree is a directed graph such that, there is exactly one
node (the root) for which all other nodes are
 * descendants of this node, plus every node has exactly one parents, except for the root
node which has no parents.
 * The given input is a directed graph that started as a rooted tree with N nodes (with
distinct values 1, 2, ..., N),
 * with one additional directed edge added. The added edge has two different vertices
chosen from 1 to N, and was not an edge that already existed.
 * The resulting graph is given as a 2D-array of edges. Each element of edges is a pair
[u, v] that
 * represents a directed edge connecting nodes u and v, where u is a parents of child v.
 * Return an edge that can be removed so that the resulting graph is a rooted tree of N
nodes.
 * If there are multiple answers, return the answer that occurs last in the given 2D-
array.

 Example 1:
 Input: [[1,2], [1,3], [2,3]]
 Output: [2,3]
 Explanation: The given directed graph will be like this:
   1
  / \
 v   v
 2-->3

 Example 2:
 Input: [[1,2], [2,3], [3,4], [4,1], [1,5]]
 Output: [4,1]
 Explanation: The given directed graph will be like this:
 5 <- 1 -> 2
      ^    |
      |    v
      4 <- 3

 Note:
 The size of the input 2D-array will be between 3 and 1000.
 Every integer represented in the 2D-array will be between 1 and N, where N is the size
of the input array.
 */
public class _685 {
    public static class Solution1 {
```

```
        /**
         * My original solution, failed by _685Test.test3
         */
        class UnionFind {
            int[] ids;
            Set<Integer> nodes;
            int[][] edges;
            List<LinkedNode> visitedLinkedNodes;
            Set<Integer> visitedValues;
            int[] redundantConn;
            int m;
            int n;

            class LinkedNode {
                List<LinkedNode> parents;//at most, there's one node that has two parents
                int val;

                public LinkedNode(int val) {
                    this.val = val;
                }

                public LinkedNode(int val, LinkedNode parent) {
                    if (parents == null) {
                        parents = new ArrayList<>();
                    }
                    this.parents.add(parent);
                    this.val = val;
                }

                public void addParent(LinkedNode parent) {
                    if (parents == null) {
                        parents = new ArrayList<>();
                    }
                    this.parents.add(parent);
                }
            }

            public UnionFind(int[][] edges) {
                this.edges = edges;
                m = edges.length;
                n = edges[0].length;
                nodes = new HashSet<>();
                visitedLinkedNodes = new ArrayList<>();
                visitedValues = new HashSet<>();
                redundantConn = new int[]{};
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        nodes.add(edges[i][j]);
                    }
                }
                ids = new int[nodes.size()];
                for (int i = 0; i < ids.length; i++) {
                    ids[i] = i + 1;
                }
```

```
            }

            public void union(int[] edge) {
                if (!visitedValues.contains(edge[1])) {
                    LinkedNode parent = new LinkedNode(edge[0]);
                    LinkedNode node = new LinkedNode(edge[1], parent);
                    visitedLinkedNodes.add(node);
                    visitedValues.add(edge[1]);
                } else {
                    for (int i = 0; i < visitedLinkedNodes.size(); i++) {
                        LinkedNode node = visitedLinkedNodes.get(i);
                        if (node.val == edge[1]) {
                            node.addParent(new LinkedNode(edge[0]));
                        }
                    }
                }
            }

            public int[] findRedundantDirectedConnection() {
                int index = hasTwoParents();
                if (index != -1) {
                    List<LinkedNode> parents =
visitedLinkedNodes.get(index).parents;//parents size is fixed, only 2
                    int[] result = new int[2];
                    for (int i = 0; i < parents.size(); i++) {
                        if (hasCycle(visitedLinkedNodes.get(index), parents.get(i))) {
                            result = new int[]{parents.get(i).val,
visitedLinkedNodes.get(index).val};
                            break;
                        }
                    }
                    return result;
                } else {
                    return edges[m - 1];
                }
            }

            private boolean hasCycle(LinkedNode linkedNode, LinkedNode parent) {
                Set<Integer> visited = new HashSet<>();
                visited.add(linkedNode.val);
                while (parent != null) {
                    if (visited.contains(parent.val)) {
                        return true;
                    }
                    visited.add(parent.val);
                    parent = findParent(parent.val);
                }
                return false;
            }

            private LinkedNode findParent(int val) {
                for (int i = 0; i < visitedLinkedNodes.size(); i++) {
                    if (visitedLinkedNodes.get(i).val == val) {
                        return visitedLinkedNodes.get(i).parents.get(0);
```

```
                }
            }
            return null;
        }

        private int hasTwoParents() {
            for (int i = 0; i < visitedLinkedNodes.size(); i++) {
                if (visitedLinkedNodes.get(i).parents.size() > 1) {
                    return i;
                }
            }
            return -1;
        }
    }

    public int[] findRedundantDirectedConnection(int[][] edges) {
        UnionFind unionFind = new UnionFind(edges);
        /**two cases:
         * 1. the entire edges are just one directed loop, in this case, just return
the last edge, see test2 in _685Test.java
         * 2. there's one directed loop, but one node of the loop has two parents, in
this case, what we'll need to do
         * is just to return the edge in this loop that points to the child that has
two parents, see test1 in _685Test.java
         * also, in case 2, use the id of the node that has two parents as the id for
all nodes in this loop, this way, I could know which of its
         * two parents is in the loop and should be the redundant one to return.
         * */
        for (int[] edge : edges) {
            unionFind.union(edge);
        }
        return unionFind.findRedundantDirectedConnection();
    }
}

public static class Solution2 {
    /**credit: https://discuss.leetcode.com/topic/105108/c-java-union-find-with-
explanation-o-n*/
    public int[] findRedundantDirectedConnection(int[][] edges) {
        int[] can1 = {-1, -1};
        int[] can2 = {-1, -1};
        int[] parent = new int[edges.length + 1];
        for (int i = 0; i < edges.length; i++) {
            if (parent[edges[i][1]] == 0) {
                parent[edges[i][1]] = edges[i][0];
            } else {
                can2 = new int[]{edges[i][0], edges[i][1]};
                can1 = new int[]{parent[edges[i][1]], edges[i][1]};
                edges[i][1] = 0;
            }
        }
        for (int i = 0; i < edges.length; i++) {
            parent[i] = i;
        }
```

```
            for (int i = 0; i < edges.length; i++) {
                if (edges[i][1] == 0) {
                    continue;
                }
                int child = edges[i][1];
                int father = edges[i][0];
                if (root(parent, father) == child) {
                    if (can1[0] == -1) {
                        return edges[i];
                    }
                    return can1;
                }
                parent[child] = father;
            }
            return can2;
        }

        int root(int[] parent, int i) {
            while (i != parent[i]) {
                parent[i] = parent[parent[i]];
                i = parent[i];
            }
            return i;
        }
    }
}
```

619. `_686.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 686. Repeated String Match
 *
 * Given two strings A and B, find the minimum number of times A has to be repeated such
that B is a substring of it.
 * If no such solution, return -1.
 * For example, with A = "abcd" and B = "cdabcdab".
 * Return 3, because by repeating A three times (“abcdabcdabcd”),
 * B is a substring of it; and B is not a substring of A repeated two times ("abcdabcd").

 Note:
 The length of A and B will be between 1 and 10000.
 */

public class _686 {
    public static class Solution1 {
        public int repeatedStringMatch(String A, String B) {
            Set<Character> set = new HashSet<>();
            for (char c : A.toCharArray()) {
                set.add(c);
            }
            for (char c : B.toCharArray()) {
                if (!set.contains(c)) {
                    return -1;
                }
            }
            StringBuilder stringBuilder = new StringBuilder(A);
            for (int i = 0; i < B.length(); i++) {
                if (stringBuilder.toString().contains(B)) {
                    return i + 1;
                }
                stringBuilder.append(A);
            }
            return -1;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(N(N+M))
         * Space: O(N + M)
         * */
        public int repeatedStringMatch(String A, String B) {
            int count = 1;
            StringBuilder sb = new StringBuilder(A);
            for (; sb.length() < B.length(); count++) {
```

```
                sb.append(A);
            }
            if (sb.indexOf(B) >= 0) {
                return count;
            }
            sb.append(A);
            if (sb.indexOf(B) >= 0) {
                return count + 1;
            }
            return -1;
        }
    }
}
```

620. `_687.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 687. Longest Univalue Path
 *
 * Given a binary tree, find the length of the longest path where each node in the path
has the same value.
 * This path may or may not pass through the root.

 Note: The length of path between two nodes is represented by the number of edges between
them.

 Example 1:
 Input:
     5
    / \
   4   5
  / \   \
 1  1   5

 Output:
 2

 Example 2:
 Input:

     1
    / \
   4   5
  / \   \
 4  4   5
 Output:
 2

 Note: The given binary tree has not more than 10000 nodes. The height of the tree is not
more than 1000.
 */

public class _687 {
    public static class Solution1 {
        /**
         * Use a one element array to pass in and out is a common technique for handling
tree questions.
         */
        public int longestUnivaluePath(TreeNode root) {
            int[] result = new int[1];
            if (root != null) {
                dfs(root, result);
            }
```

```
            return result[0];
        }

        // calculate longest univalue path from root to leaves
        // In addition, the maximum univalue path cross the root node is calculated and
then global maximum is udpated.
        private int dfs(TreeNode root, int[] result) {
            int leftPath = root.left == null ? 0 : dfs(root.left, result);
            int rightPath = root.right == null ? 0 : dfs(root.right, result);
            int leftResult = (root.left != null && root.left.val == root.val) ? leftPath
+ 1 : 0;
            int rightResult = (root.right != null && root.right.val == root.val) ?
rightPath + 1 : 0;
            result[0] = Math.max(result[0], leftResult + rightResult);
            return Math.max(leftResult, rightResult);
        }
    }
}
```

621. `_688.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.LinkedList;
import java.util.Queue;

/**
 * 688. Knight Probability in Chessboard
 *
 *  On an NxN chessboard, a knight starts at the r-th row and c-th column and attempts to
make exactly K moves.
 *  The rows and columns are 0 indexed, so the top-left square is (0, 0), and the bottom-
right square is (N-1, N-1).
 *  A chess knight has 8 possible moves it can make, as illustrated below.
 *  Each move is two squares in a cardinal direction, then one square in an orthogonal
direction.
 *  Each time the knight is to move, it chooses one of eight possible moves uniformly at
random
 *  (even if the piece would go off the chessboard) and moves there.
 *  The knight continues moving until it has made exactly K moves or has moved off the
chessboard.
 *  Return the probability that the knight remains on the board after it has stopped
moving.

 Example:

 Input: 3, 2, 0, 0
 Output: 0.0625
 Explanation: There are two moves (to (1,2), (2,1)) that will keep the knight on the
board.
 From each of those positions, there are also two moves that will keep the knight on the
board.
 The total probability the knight stays on the board is 0.0625.

 Note:
 N will be between 1 and 25.
 K will be between 0 and 100.
 The knight always initially starts on the board.
 */
public class _688 {

    public static class Solution1 {
        /**
         * This BFS solution results in TLE on Leetcode.
         */
        public double knightProbability(int N, int K, int r, int c) {
            int[][] directions = {{-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2},
{-1, -2}, {-2, -1}};
            Queue<int[]> queue = new LinkedList<>();
            queue.offer(new int[]{r, c});
            int level = K;
```

```
            while (level-- > 0) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    int[] curr = queue.poll();
                    for (int[] direction : directions) {
                        int x = curr[0] + direction[0];
                        int y = curr[1] + direction[1];
                        if (x >= 0 && x < N && y >= 0 && y < N) {
                            queue.offer(new int[]{x, y});
                        }
                    }
                }
            }
            double prob = queue.size();
            for (int i = 0; i < K; i++) {
                prob /= 8;
            }
            return prob;
        }
    }

    public static class Solution2 {
        /**
         * Let f[r][c][k] mean the probability that the knight is still on board after k
steps,
         * we can deduce a recursion from its k-1 steps
         * In addition, instead of using a 3-d array, we can only keep the most recent
two layers,
         * i.e. using only two 2-d arrays.
         */
        public double knightProbability(int N, int K, int r, int c) {
            int[][] directions = {{-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2},
{-1, -2}, {-2, -1}};
            double[][] dp0 = new double[N][N];
            for (double[] row : dp0) {
                Arrays.fill(row, 1);
            }
            for (int k = 0; k < K; k++) {
                double[][] dp1 = new double[N][N];
                for (int i = 0; i < N; i++) {
                    for (int j = 0; j < N; j++) {
                        for (int l = 0; l < directions.length; l++) {
                            int[] direction = directions[l];
                            int x = i + direction[0];
                            int y = j + direction[1];
                            if (x >= 0 && y >= 0 && x < N && y < N) {
                                dp1[i][j] += dp0[x][y];
                            }
                        }
                    }
                }
                dp0 = dp1;
            }
            return dp0[r][c] / Math.pow(8, K);
```

```
        }
    }

    public static void main(String... args) {
        System.out.println((double) 2 / 8);
    }
}
```

622. `_689.java`

```
package com.fishercoder.solutions;

/**
 * 689. Maximum Sum of 3 Non-Overlapping Subarrays
 *
 * In a given array nums of positive integers, find three non-overlapping subarrays with maximum sum.
 * Each subarray will be of size k, and we want to maximize the sum of all 3*k entries.
 * Return the result as a list of indices representing the starting position of each interval (0-indexed).
 * If there are multiple answers, return the lexicographically smallest one.

 Example:
 Input: [1,2,1,2,6,7,5,1], 2
 Output: [0, 3, 5]
 Explanation: Subarrays [1, 2], [2, 6], [7, 5] correspond to the starting indices [0, 3, 5].
 We could have also taken [2, 1], but an answer of [1, 3, 5] would be lexicographically larger.

 Note:
 nums.length will be between 1 and 20000.
 nums[i] will be between 1 and 65535.
 k will be between 1 and floor(nums.length / 3).
 */
public class _689 {
    public static class Solution1 {
        /**we basically need to find the interval (i, i+k-1) as the middle interval, where k <= i <= n-2k
         * then this interval (0, i-1) will be the left interval
         * the interval (i+k, n-1) will be the right interval.
         *
         * Please pay special attention to the variable name I use here: this `k` is not a random one, it's the `k`
         * from the passed in parameter.
         *
         * Credit: https://discuss.leetcode.com/topic/105577/c-java-dp-with-explanation-o-n/*/
        public int[] maxSumOfThreeSubarrays(int[] nums, int k) {
            if (nums == null || nums.length == 0) {
                return new int[]{};
            }
            int n = nums.length;
            int[] sums = new int[n + 1];
            for (int i = 0; i < n; i++) {
                sums[i + 1] = sums[i] + nums[i];
            }

            int[] leftMax = new int[n];
            for (int i = k, total = sums[k] - sums[0]; i < n; i++) {
                if (sums[i + 1] - sums[i + 1 - k] > total) {
```

```
                    leftMax[i] = i + 1 - k;
                    total = sums[i + 1] - sums[i + 1 - k];
                } else {
                    leftMax[i] = leftMax[i - 1];
                }
            }

            int[] rightMax = new int[n];
            rightMax[n - k] = n - k;
            for (int i = n - k - 1, total = sums[n] - sums[n - k]; i >= 0; i--) {
                if (sums[i + k] - sums[i] >= total) {
                    rightMax[i] = i;
                    total = sums[i + k] - sums[i];
                } else {
                    rightMax[i] = rightMax[i + 1];
                }
            }

            //try to find all possible middle intervals
            int[] result = new int[3];
            int max = 0;
            for (int i = k; i <= n - 2 * k; i++) {
                int left = leftMax[i - 1];
                int right = rightMax[i + k];
                int total = (sums[i + k] - sums[i]) + (sums[left + k] - sums[left]) +
(sums[right + k] - sums[right]);
                if (total > max) {
                    max = total;
                    result[0] = left;
                    result[1] = i;
                    result[2] = right;
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**reference: https://leetcode.com/articles/maximum-sum-of-3-non-overlapping-
intervals*/
    }
}
```

623. `_690.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.Employee;

import java.util.List;
import java.util.stream.Collectors;

/**
 * 690. Employee Importance
 *
 * You are given a data structure of employee information, which includes the employee's
unique id,
 * his importance value and his direct subordinates' id.
 * For example, employee 1 is the leader of employee 2, and employee 2 is the leader of
employee 3.
 * They have importance value 15, 10 and 5, respectively.
 * Then employee 1 has a data structure like [1, 15, [2]], and
 * employee 2 has [2, 10, [3]], and employee 3 has [3, 5, []].
 * Note that although employee 3 is also a subordinate of employee 1, the relationship is
not direct.
 * Now given the employee information of a company, and an employee id,
 * you need to return the total importance value of this employee and all his
subordinates.

 Example 1:

 Input: [[1, 5, [2, 3]], [2, 3, []], [3, 3, []]], 1
 Output: 11
 Explanation:
 Employee 1 has importance value 5, and he has two direct subordinates: employee 2 and
employee 3.
 They both have importance value 3. So the total importance value of employee 1 is 5 + 3
+ 3 = 11.

 Note:

 One employee has at most one direct leader and may have several subordinates.
 The maximum number of employees won't exceed 2000.

 */
public class _690 {

    public static class Solution1 {

        int total = 0;

        public int getImportance(List<Employee> employees, int id) {
            Employee manager = employees.stream().filter(e -> e.id ==
id).collect(Collectors.toList()).get(0);
            total += manager.importance;
            manager.subordinates.forEach(subId -> getImportance(employees, subId));
```

```
            /**The above line is equivalent to below for loop*/
//            for (int subId : manager.subordinates) {
//                getImportance(employees, subId);
//            }

            return total;
        }

    }
}
```

624. `_691.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 691. Stickers to Spell Word
 *
 * We are given N different types of stickers. Each sticker has a lowercase English word on it.
 * You would like to spell out the given target string by cutting individual letters from your collection of stickers and rearranging them.
 * You can use each sticker more than once if you want, and you have infinite quantities of each sticker.
 * What is the minimum number of stickers that you need to spell out the target? If the task is impossible, return -1.

 Example 1:
 Input:
 ["with", "example", "science"], "thehat"
 Output:
 3

 Explanation:
 We can use 2 "with" stickers, and 1 "example" sticker.
 After cutting and rearrange the letters of those stickers, we can form the target "thehat".
 Also, this is the minimum number of stickers necessary to form the target string.

 Example 2:
 Input:
 ["notice", "possible"], "basicbasic"
 Output:
 -1

 Explanation:
 We can't form the target "basicbasic" from cutting letters from the given stickers.

 Note:
 stickers has length in the range [1, 50].
 stickers consists of lowercase English words (without apostrophes).
 target has length in the range [1, 15], and consists of lowercase English letters.
 In all test cases, all words were chosen randomly from the 1000 most common US English words, and the target was chosen as a concatenation of two random words.
 The time limit may be more challenging than usual. It is expected that a 50 sticker test case can be solved within 35ms on average.
 */
public class _691 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/106273/c-java-python-dp-
```

```
memoization-with-optimization-29-ms-c/2
         */
        public int minStickers(String[] stickers, String target) {
            int m = stickers.length;
            int[][] mp = new int[m][26];
            Map<String, Integer> dp = new HashMap<>();
            for (int i = 0; i < m; i++) {
                for (char c : stickers[i].toCharArray()) {
                    mp[i][c - 'a']++;
                }
            }
            dp.put("", 0);
            return helper(dp, mp, target);
        }

        private int helper(Map<String, Integer> dp, int[][] mp, String target) {
            if (dp.containsKey(target)) {
                return dp.get(target);
            }
            int ans = Integer.MAX_VALUE;
            int n = mp.length;
            int[] tar = new int[26];
            for (char c : target.toCharArray()) {
                tar[c - 'a']++;
            }
            // try every sticker
            for (int i = 0; i < n; i++) {
                // optimization
                if (mp[i][target.charAt(0) - 'a'] == 0) {
                    continue;
                }
                StringBuilder sb = new StringBuilder();
                // apply a sticker on every character a-z
                for (int j = 0; j < 26; j++) {
                    if (tar[j] > 0) {
                        for (int k = 0; k < Math.max(0, tar[j] - mp[i][j]); k++) {
                            sb.append((char) ('a' + j));
                        }
                    }
                }
                String s = sb.toString();
                int tmp = helper(dp, mp, s);
                if (tmp != -1) {
                    ans = Math.min(ans, 1 + tmp);
                }
            }
            dp.put(target, ans == Integer.MAX_VALUE ? -1 : ans);
            return dp.get(target);
        }
    }
}
```

625. `_692.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.SortedSet;
import java.util.TreeSet;

/**
 * 692. Top K Frequent Words
 *
 * Given a non-empty list of words, return the k most frequent elements.
 * Your answer should be sorted by frequency from highest to lowest.
 * If two words have the same frequency, then the word with the lower alphabetical order
comes first.

 Example 1:
 Input: ["i", "love", "leetcode", "i", "love", "coding"], k = 2
 Output: ["i", "love"]
 Explanation: "i" and "love" are the two most frequent words.
 Note that "i" comes before "love" due to a lower alphabetical order.

 Example 2:
 Input: ["the", "day", "is", "sunny", "the", "the", "the", "sunny", "is", "is"], k = 4
 Output: ["the", "is", "sunny", "day"]
 Explanation: "the", "is", "sunny" and "day" are the four most frequent words,
 with the number of occurrence being 4, 3, 2 and 1 respectively.

 Note:
 You may assume k is always valid, 1 ≤ k ≤ number of unique elements.
 Input words contain only lowercase letters.

 Follow up:
 Try to solve it in O(n log k) time and O(n) extra space.
 Can you solve it in O(n) time with only O(k) extra space?

 */
public class _692 {

    public static class Solution1 {
        /**
         * O(n) extra space
         * O(nlogk) time
         * */
        public List<String> topKFrequent(String[] words, int k) {
            Map<String, Integer> map = new HashMap<>();
            for (String word : words) {
                map.put(word, map.getOrDefault(word, 0) + 1);
            }
```

```
            SortedSet<Map.Entry<String, Integer>> sortedset = new TreeSet<>(
                    (e1, e2) -> {
                        if (e1.getValue() != e2.getValue()) {
                            return e2.getValue() - e1.getValue();
                        } else {
                            return e1.getKey().compareToIgnoreCase(e2.getKey());
                        }
                    });
            sortedset.addAll(map.entrySet());

            List<String> result = new ArrayList<>();
            Iterator<Map.Entry<String, Integer>> iterator = sortedset.iterator();
            while (iterator.hasNext() && k-- > 0) {
                result.add(iterator.next().getKey());
            }
            return result;
        }
    }
}
```

626. `_693.java`

```
package com.fishercoder.solutions;

/**
 * 693. Binary Number with Alternating Bits
 *
 * Given a positive integer, check whether it has alternating bits: namely, if two
adjacent bits will always have different values.

 Example 1:
 Input: 5
 Output: True
 Explanation:
 The binary representation of 5 is: 101

 Example 2:
 Input: 7
 Output: False
 Explanation:
 The binary representation of 7 is: 111.

 Example 3:
 Input: 11
 Output: False
 Explanation:
 The binary representation of 11 is: 1011.

 Example 4:
 Input: 10
 Output: True
 Explanation:
 The binary representation of 10 is: 1010.
 */

public class _693 {
    public static class Solution1 {
        public boolean hasAlternatingBits(int n) {
            String binaryStr = Integer.toBinaryString(n);
            for (int i = 1; i < binaryStr.length(); i++) {
                if (binaryStr.charAt(i - 1) == binaryStr.charAt(i)) {
                    return false;
                }
            }
            return true;
        }
    }

    public static class Solution2 {
        public boolean hasAlternatingBits(int n) {
            return Integer.bitCount(((n >> 1) ^ n) + 1) == 1;
        }
```

```
    }
}
```

627. `_694.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 694. Number of Distinct Islands
 *
 * Given a non-empty 2D array grid of 0's and 1's, an island is a group of 1's
(representing land)
 * connected 4-directionally (horizontal or vertical.) You may assume all four edges of
the grid are surrounded by water.
 * Count the number of distinct islands.
 * An island is considered to be the same as another if and only if
 * one island can be translated (and not rotated or reflected) to equal the other.

 Example 1:

 11000
 11000
 00011
 00011

 Given the above grid map, return 1.

 Example 2:

 11011
 10000
 00001
 11011

 Given the above grid map, return 3.

 Notice that:

 11
 1

 and

 1
 11

 are considered different island shapes, because we do not consider reflection /
rotation.

 Note: The length of each dimension in the given grid does not exceed 50.
```

```
 */
public class _694 {
    public static class Solution1 {
        /**
         * My original idea:
         * my not fully working yet: the equals() and hashcode() methods need to be
refined
         * because HashSet is not really filtering the islands wiht the same shape.
         */
        class Quadrilateral {
            int[] topLeft;
            int[] bottomLeft;
            int[] topRight;
            int[] bottomRight;
            int area;

            public Quadrilateral(int i, int j) {
                this.area = 0;
                this.topLeft = new int[]{i, j};
                this.topRight = new int[]{i, j};
                this.bottomLeft = new int[]{i, j};
                this.bottomRight = new int[]{i, j};
            }

            @Override
            public boolean equals(Object o) {
                if (this == o) {
                    return true;
                }
                if (!(o instanceof Quadrilateral)) {
                    return false;
                }

                Quadrilateral that = (Quadrilateral) o;
                return this.area == that.area && checkDistance(that);
            }

            private boolean checkDistance(Quadrilateral that) {
                int thisTop = computeDistance(this.topLeft, this.topRight);
                int thatTop = computeDistance(that.topLeft, that.topRight);
                if (thisTop != thatTop) {
                    return false;
                }
                int thisRight = computeDistance(this.topRight, this.bottomRight);
                int thatRight = computeDistance(that.topRight, that.bottomRight);
                if (thisRight != thatRight) {
                    return false;
                }
                int thisBottom = computeDistance(this.bottomRight, this.bottomLeft);
                int thatBottom = computeDistance(that.bottomRight, that.bottomLeft);
                if (thisBottom != thatBottom) {
                    return false;
                }
                int thisLeft = computeDistance(this.bottomLeft, this.topLeft);
```

```
                int thatLeft = computeDistance(that.bottomLeft, that.topLeft);
                return thisLeft == thatLeft;
            }

            private int computeDistance(int[] A, int[] B) {
                return (int) (Math.pow(A[0] - B[0], 2) + Math.pow(A[1] - B[1], 2));
            }

            @Override
            public int hashCode() {
                return area + computeDistance(this.topLeft, this.topRight) +
computeDistance(this.topRight, this.bottomRight)
                        + computeDistance(this.bottomRight, this.bottomLeft) +
computeDistance(this.bottomLeft, this.topLeft);
            }

            public void addPoint(int i, int j) {
                //todo: check wether this point (i,j) is in the range, if not, expand the
range
                if (i == topRight[0]) {
                    topRight[1] = Math.max(topRight[1], j);
                }
                if (j == topRight[1]) {
                    topRight[0] = Math.min(topRight[1], i);
                }
                if (i == topLeft[0]) {
                    topLeft[1] = Math.min(topLeft[1], j);
                }
                if (j == topLeft[1]) {
                    topLeft[0] = Math.min(topLeft[0], i);
                }
                if (i == bottomLeft[0]) {
                    bottomLeft[1] = Math.min(bottomLeft[1], j);
                }
                if (j == bottomLeft[1]) {
                    bottomLeft[0] = Math.max(bottomLeft[0], i);
                }
                if (j == bottomRight[1]) {
                    bottomRight[0] = Math.max(bottomRight[0], i);
                }
                if (i == bottomRight[0]) {
                    bottomRight[1] = Math.max(bottomRight[1], j);
                }
            }

            public void addArea() {
                this.area++;
            }
        }

        public int numDistinctIslands(int[][] grid) {
            Set<Quadrilateral> set = new HashSet<>();
            int m = grid.length;
            int n = grid[0].length;
```

```
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        Quadrilateral quadrilateral = dfs(grid, i, j, m, n, new
Quadrilateral(i, j));
                        set.add(quadrilateral);
                    }
                }
            }
            return set.size();
        }

        private Quadrilateral dfs(int[][] grid, int i, int j, int m, int n, Quadrilateral
quadrilateral) {
            if (i < 0 || j < 0 || i >= m || j >= n || grid[i][j] == 0) {
                return quadrilateral;
            }
            grid[i][j] = 0;
            quadrilateral.addPoint(i, j);
            quadrilateral.addArea();
            quadrilateral = dfs(grid, i + 1, j, m, n, quadrilateral);
            quadrilateral = dfs(grid, i - 1, j, m, n, quadrilateral);
            quadrilateral = dfs(grid, i, j + 1, m, n, quadrilateral);
            quadrilateral = dfs(grid, i, j - 1, m, n, quadrilateral);
            return quadrilateral;
        }
    }

    public static class Solution2 {
        int[][] directions = new int[][]{
                {0, 1},
                {1, 0},
                {0, -1},
                {-1, 0}
        };

        public int numDistinctIslands(int[][] grid) {
            int m = grid.length;
            int n = grid[0].length;
            Set<List<List<Integer>>> uniqueShapeIslands = new HashSet<>();
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    List<List<Integer>> island = new ArrayList<>();
                    if (dfs(i, j, i, j, grid, m, n, island)) {
                        uniqueShapeIslands.add(island);
                    }
                }
            }
            return uniqueShapeIslands.size();
        }

        private boolean dfs(int i0, int j0, int i, int j, int[][] grid, int m, int n,
List<List<Integer>> island) {
            if (i < 0 || j < 0 || i >= m || j >= n || grid[i][j] <= 0) {
```

```
                return false;
            }
            island.add(Arrays.asList(i - i0, j - j0));
            grid[i][j] *= -1;
            for (int k = 0; k < 4; k++) {
                dfs(i0, j0, i + directions[k][0], j + directions[k][1], grid, m, n,
island);
            }
            return true;
        }
    }
}
```

628. `_695.java`

```
package com.fishercoder.solutions;

/**
 * 695. Max Area of Island
 *
 * Given a non-empty 2D array grid of 0's and 1's, an island is a group of 1's
(representing land)
 * connected 4-directionally (horizontal or vertical.)
 * You may assume all four edges of the grid are surrounded by water.
 * Find the maximum area of an island in the given 2D array. (If there is no island, the
maximum area is 0.)

 Example 1:

 [[0,0,1,0,0,0,0,1,0,0,0,0,0],
 [0,0,0,0,0,0,0,1,1,1,0,0,0],
 [0,1,1,0,1,0,0,0,0,0,0,0,0],
 [0,1,0,0,1,1,0,0,1,0,1,0,0],
 [0,1,0,0,1,1,0,0,1,1,1,0,0],
 [0,0,0,0,0,0,0,0,0,0,1,0,0],
 [0,0,0,0,0,0,0,1,1,1,0,0,0],
 [0,0,0,0,0,0,0,1,1,0,0,0,0]]

 Given the above grid, return 6. Note the answer is not 11, because the island must be
connected 4-directionally.

 Example 2:

 [[0,0,0,0,0,0,0,0]]

 Given the above grid, return 0.

 Note: The length of each dimension in the given grid does not exceed 50.
 */

public class _695 {

    public static class Solution1 {
        public int maxAreaOfIsland(int[][] grid) {
            if (grid == null || grid.length == 0) {
                return 0;
            }
            int m = grid.length;
            int n = grid[0].length;
            int max = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        int area = dfs(grid, i, j, m, n, 0);
                        max = Math.max(area, max);
                    }
```

```
                }
            }
            return max;
        }

        int dfs(int[][] grid, int i, int j, int m, int n, int area) {
            if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0) {
                return area;
            }
            grid[i][j] = 0;
            area++;
            area = dfs(grid, i + 1, j, m, n, area);
            area = dfs(grid, i, j + 1, m, n, area);
            area = dfs(grid, i - 1, j, m, n, area);
            area = dfs(grid, i, j - 1, m, n, area);
            return area;
        }
    }

}
```

629. `_696.java`

```
package com.fishercoder.solutions;

/**
 * 696. Count Binary Substrings
 *
 * Give a string s, count the number of non-empty (contiguous) substrings that have the same number of 0's and 1's,
 * and all the 0's and all the 1's in these substrings are grouped consecutively.
 * Substrings that occur multiple times are counted the number of times they occur.

 Example 1:

 Input: "00110011"
 Output: 6
 Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01", "1100", "10", "0011", and "01".

 Notice that some of these substrings repeat and are counted the number of times they occur.

 Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.

 Example 2:

 Input: "10101"
 Output: 4
 Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's and 0's.

 Note:
 s.length will be between 1 and 50,000.
 s will only consist of "0" or "1" characters.
 */
public class _696 {
    public static class Solution1 {
        public int countBinarySubstrings(String s) {
            int n = s.length();
            /**a[i][0] denotes from most left up to i (inclusive), how many consecutive 0's
             * a[i][1] denotes from most left up to i (inclusive), how many consecutive 1's*/
            int[][] a = new int[n][2];
            /**a[i][0] denotes from i (inclusive) to the most right, how many consecutive 0's
             * b[i][0] denotes from i (inclusive) to the most right, how many consecutive 1's*/
            int[][] b = new int[n][2];
            for (int i = 0; i < n; i++) {
                if (s.charAt(i) == '0') {
                    a[i][0] = 1 + (i - 1 >= 0 ? a[i - 1][0] : 0);
```

```
                } else {
                    a[i][1] = 1 + (i - 1 >= 0 ? a[i - 1][1] : 0);
                }
            }
            for (int i = n - 1; i >= 0; i--) {
                if (s.charAt(i) == '0') {
                    b[i][0] = 1 + (i + 1 < n ? b[i + 1][0] : 0);
                } else {
                    b[i][1] = 1 + (i + 1 < n ? b[i + 1][1] : 0);
                }

            }
            long ans = 0;
            for (int i = 0; i + 1 < n; i++) {
                ans += Math.min(a[i][0], b[i + 1][1]);
                ans += Math.min(a[i][1], b[i + 1][0]);
            }
            return (int) ans;
        }
    }
}
```

630. `_697.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 697. Degree of an Array
 *
 * Given a non-empty array of non-negative integers nums,
 * the degree of this array is defined as the maximum frequency of any one of its
elements.
 * Your task is to find the smallest possible length of a (contiguous) subarray of nums,
that has the same degree as nums.

 Example 1:

 Input: [1, 2, 2, 3, 1]
 Output: 2
 Explanation:
 The input array has a degree of 2 because both elements 1 and 2 appear twice.
 Of the subarrays that have the same degree:
 [1, 2, 2, 3, 1], [1, 2, 2, 3], [2, 2, 3, 1], [1, 2, 2], [2, 2, 3], [2, 2]
 The shortest length is 2. So return 2.

 Example 2:
 Input: [1,2,2,3,1,4,2]
 Output: 6

 Note:
 nums.length will be between 1 and 50,000.
 nums[i] will be an integer between 0 and 49,999.
 */
public class _697 {
    public static class Solution1 {
        public int findShortestSubArray(int[] nums) {
            Map<Integer, Integer> map = new HashMap<>();
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(nums[i])) {
                    map.put(nums[i], map.get(nums[i]) + 1);
                } else {
                    map.put(nums[i], 1);
                }
            }
            int degree = -1;
            for (int key : map.keySet()) {
                degree = Math.max(degree, map.get(key));
            }
            List<Integer> candidateNums = new ArrayList();
```

```
        for (int key : map.keySet()) {
            if (map.get(key) == degree) {
                candidateNums.add(key);
            }
        }
        int shortest = Integer.MAX_VALUE;
        for (int candidate : candidateNums) {
            shortest = Math.min(shortest, findLength(nums, candidate));
        }
        return shortest;
    }

    int findLength(int[] arr, int candidate) {
        int firstAppearance = Integer.MAX_VALUE;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == candidate) {
                firstAppearance = i;
                break;
            }
        }
        int lastAppearance = arr.length - 1;
        for (int i = arr.length - 1; i > firstAppearance; i--) {
            if (arr[i] == candidate) {
                lastAppearance = i;
                break;
            }
        }
        return (lastAppearance - firstAppearance) + 1;
    }
}

public static class Solution2 {
    public int findShortestSubArray(int[] nums) {
        Map<Integer, Integer> count = new HashMap<>();
        Map<Integer, Integer> left = new HashMap<>();
        Map<Integer, Integer> right = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            count.put(nums[i], count.getOrDefault(nums[i], 0) + 1);
            if (!left.containsKey(nums[i])) {
                left.put(nums[i], i);
            }
            right.put(nums[i], i);
        }

        int result = nums.length;
        int degree = Collections.max(count.values());
        for (int num : count.keySet()) {
            if (count.get(num) == degree) {
                result = Math.min(result, right.get(num) - left.get(num) + 1);
            }
        }
        return result;
    }
```

```
    }
}
```

631. `_698.java`

```
package com.fishercoder.solutions;

/**
 * 698. Partition to K Equal Sum Subsets
 *
 * Given an array of integers nums and a positive integer k,
 * find whether it's possible to divide this array into k non-empty subsets whose sums
are all equal.

 Example 1:

 Input: nums = [4, 3, 2, 3, 5, 2, 1], k = 4
 Output: True
 Explanation: It's possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with
equal sums.

 Note:
 1 <= k <= len(nums) <= 16.
 0 < nums[i] < 10000.
 */
public class _698 {

    public static class Solution1 {
        public boolean canPartitionKSubsets(int[] nums, int k) {
            long sum = 0;
            for (int num : nums) {
                sum += num;
            }
            if (sum % k != 0) {
                return false;
            }
            int equalSum = (int) (sum / k);
            boolean[] visited = new boolean[nums.length];
            return canPartition(nums, visited, 0, k, 0, 0, equalSum);
        }

        private boolean canPartition(int[] nums, boolean[] visited, int startIndex, int
k, int currSum, int currNum, int target) {
            if (k == 1) {
                return true;
            }
            if (currSum == target && currNum > 0) {
                /**Everytime when we get currSum == target, we'll start from index 0 and
look up the numbers that are not used yet
                 * and try to find another sum that could equal to target*/
                return canPartition(nums, visited, 0, k - 1, 0, 0, target);
            }
            for (int i = startIndex; i < nums.length; i++) {
                if (!visited[i]) {
                    visited[i] = true;
                    if (canPartition(nums, visited, i + 1, k, currSum + nums[i],
```

```
currNum++, target)) {
                        return true;
                    }
                    visited[i] = false;
                }
            }
            return false;
        }
    }

}
```

632. `_699.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 699. Falling Squares
 *
 * On an infinite number line (x-axis), we drop given squares in the order they are
given.
 * The i-th square dropped (positions[i] = (left, side_length)) is a
 * square with the left-most point being positions[i][0] and sidelength positions[i][1].
 * The square is dropped with the bottom edge parallel to the number line, and
 * from a higher height than all currently landed squares. We wait for each square to
stick before dropping the next.
 * The squares are infinitely sticky on their bottom edge, and will
 * remain fixed to any positive length surface they touch (either the number line or
another square).
 * Squares dropped adjacent to each other will not stick together prematurely.
 * Return a list ans of heights.
 * Each height ans[i] represents the current highest height of any square we have
dropped,
 * after dropping squares represented by positions[0], positions[1], ..., positions[i].

 Example 1:

 Input: [[1, 2], [2, 3], [6, 1]]
 Output: [2, 5, 5]
 Explanation:


 After the first drop of positions[0] = [1, 2]:
 _aa
 _aa
 -------
 The maximum height of any square is 2.


 After the second drop of positions[1] = [2, 3]:
 __aaa
 __aaa
 __aaa
 _aa__
 _aa__
 --------------
 The maximum height of any square is 5.
 The larger square stays on top of the smaller square despite where its center
 of gravity is, because squares are infinitely sticky on their bottom edge.


 After the third drop of positions[1] = [6, 1]:
```

```
 __aaa
 __aaa
 __aaa
 _aa
 _aa___a
 --------------
 The maximum height of any square is still 5.

 Thus, we return an answer of [2, 5, 5].


 Example 2:

 Input: [[100, 100], [200, 100]]
 Output: [100, 100]
 Explanation: Adjacent squares don't get stuck prematurely - only their bottom edge can
stick to surfaces.

 Note:
 1 <= positions.length <= 1000.
 1 <= positions[0] <= 10^8.
 1 <= positions[1] <= 10^6.
 */

public class _699 {
    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/107107/easy-understood-o-n-2-
solution-with-explanation*/
        public List<Integer> fallingSquares(int[][] positions) {
            List<Interval> intervals = new ArrayList<>();
            List<Integer> result = new ArrayList<>();
            int height = 0;
            for (int[] position : positions) {
                Interval curr = new Interval(position[0], position[0] + position[1] - 1,
position[1]);
                height = Math.max(height, getHeight(intervals, curr));
                result.add(height);
            }
            return result;
        }

        private int getHeight(List<Interval> intervals, Interval curr) {
            int preMaxHeight = 0;
            for (Interval interval : intervals) {
                if (interval.end < curr.start || interval.start > curr.end) {
                    continue;
                }
                preMaxHeight = Math.max(preMaxHeight, interval.height);
            }
            curr.height += preMaxHeight;
            intervals.add(curr);
            return curr.height;
        }
```

```
        class Interval {
            int start;
            int end;
            int height;

            public Interval(int start, int end, int height) {
                this.start = start;
                this.end = end;
                this.height = height;
            }
        }
    }
}
```

633. `_700.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 700. Search in a Binary Search Tree
 *
 * Given the root node of a binary search tree (BST) and a value.
 * You need to find the node in the BST that the node's value equals the given value.
 * Return the subtree rooted with that node. If such node doesn't exist, you should
return NULL.
 *
 * For example,
 *
 * Given the tree:
 *         4
 *        / \
 *       2   7
 *      / \
 *     1   3
 *
 * And the value to search: 2
 * You should return this subtree:
 *
 *       2
 *      / \
 *     1   3
 * In the example above, if we want to search the value 5, since there is no node with
value 5, we should return NULL.
 *
 * Note that an empty tree is represented by NULL,
 * therefore you would see the expected output (serialized tree format) as [], not null.
 * */
public class _700 {
    public static class Solution1 {
        public TreeNode searchBST(TreeNode root, int val) {
            if (root == null) {
                return null;
            } else if (root.val == val) {
                return root;
            } else if (root.val > val) {
                return searchBST(root.left, val);
            } else {
                return searchBST(root.right, val);
            }
        }
    }
}
```

634. `_701.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 701. Insert into a Binary Search Tree
 *
 * Given the root node of a binary search tree (BST) and a value to be inserted into the
tree, insert the value into the BST. Return the root node of the BST after the insertion.
It is guaranteed that the new value does not exist in the original BST.
 * Note that there may exist multiple valid ways for the insertion, as long as the tree
remains a BST after insertion. You can return any of them.
 *
 * For example,
 *
 * Given the tree:
 *         4
 *        / \
 *       2   7
 *      / \
 *     1   3
 * And the value to insert: 5
 *
 *  You can return this binary search tree:
 *
 *          4
 *        /   \
 *       2     7
 *      / \   /
 *     1   3 5
 *
 * This tree is also valid:
 *
 *          5
 *        /   \
 *       2     7
 *      / \
 *     1   3
 *          \
 *           4
 */
public class _701 {
  public static class Solution1 {
    public TreeNode insertIntoBST(TreeNode root, int val) {
      if (root == null) {
        return new TreeNode(val);
      }
      if (root.val < val) {
        root.right = insertIntoBST(root.right, val);
      } else {
        root.left = insertIntoBST(root.left, val);
```

```
      }
      return root;
    }
  }
}
```

635. `_703.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.PriorityQueue;

/**
 * 703. Kth Largest Element in a Stream
 *
 * Design a class to find the kth largest element in a stream. Note that it is the kth
largest element in the sorted order, not the kth distinct element.
 * Your KthLargest class will have a constructor which accepts an integer K and an
integer array nums,
 * which contains initial elements from the stream.
 * For each call to the method KthLargest.add, return the element representing the kth
largest element in the stream.
 *
 * Example:
 *
 * int K = 3;
 * int[] arr = [4,5,8,2];
 * KthLargest kthLargest = new KthLargest(3, arr);
 * kthLargest.add(3);   // returns 4
 * kthLargest.add(5);   // returns 5
 * kthLargest.add(10);  // returns 5
 * kthLargest.add(9);   // returns 8
 * kthLargest.add(4);   // returns 8
 * Note:
 * You may assume that nums' length ≥ K-1 and K ≥ 1.
 */
public class _703 {
  public static class Solution1 {
    public static class KthLargest {
      PriorityQueue<Integer> heap;
      int maxK;

      public KthLargest(int k, int[] nums) {
        heap = new PriorityQueue<>(Collections.reverseOrder());
        for (int num : nums) {
          heap.offer(num);
        }
        maxK = k;
      }

      public int add(int val) {
        List<Integer> tmp = new ArrayList<>();
        int result = 0;
        int tmpK = maxK;
        heap.offer(val);
        while (tmpK-- > 0) {
```

```
          result = heap.poll();
          tmp.add(result);
        }
        for (int num : tmp) {
          heap.offer(num);
        }
        return result;
      }
    }
  }
}
```

636. `_704.java`

```
package com.fishercoder.solutions;

/**
 * 704. Binary Search
 *
 * Given a sorted (in ascending order) integer array nums of n elements and a target
value, write a function to search target in nums.
 * If target exists, then return its index, otherwise return -1.
 *
 * Example 1:
 *
 * Input: nums = [-1,0,3,5,9,12], target = 9
 * Output: 4
 * Explanation: 9 exists in nums and its index is 4
 *
 * Example 2:
 *
 * Input: nums = [-1,0,3,5,9,12], target = 2
 * Output: -1
 * Explanation: 2 does not exist in nums so return -1
 *
 * Note:
 *
 * You may assume that all elements in nums are unique.
 * n will be in the range [1, 10000].
 * The value of each element in nums will be in the range [-9999, 9999].
 */
public class _704 {
  public static class Solution1 {
    public int search(int[] nums, int target) {
      int left = 0;
      int right = nums.length - 1;
      if (target < nums[left] || target > nums[right]) {
        return -1;
      }
      if (nums[left] == target) {
        return left;
      } else if (nums[right] == target) {
        return right;
      }
      while (left <= right) {
        int mid = left + (right - left) / 2;
        if (target == nums[mid]) {
          return mid;
        } else if (target > nums[mid]) {
          left = mid + 1;
        } else {
          right = mid - 1;
        }
      }
      return -1;
```

```
    }
  }
}
```

637. `_705.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 705. Design HashSet
 *
 * Design a HashSet without using any built-in hash table libraries.
 *
 * To be specific, your design should include these functions:
 *
 * add(value): Insert a value into the HashSet.
 * contains(value) : Return whether the value exists in the HashSet or not.
 * remove(value): Remove a value in the HashSet. If the value does not exist in the
HashSet, do nothing.
 *
 * Example:
 *
 * MyHashSet hashSet = new MyHashSet();
 * hashSet.add(1);
 * hashSet.add(2);
 * hashSet.contains(1);    // returns true
 * hashSet.contains(3);    // returns false (not found)
 * hashSet.add(2);
 * hashSet.contains(2);    // returns true
 * hashSet.remove(2);
 * hashSet.contains(2);    // returns false (already removed)
 *
 * Note:
 *
 * All values will be in the range of [0, 1000000].
 * The number of operations will be in the range of [1, 10000].
 * Please do not use the built-in HashSet library.
 */
public class _705 {
  public static class Solution1 {
    class MyHashSet {
      Map<Integer, Integer> map;

      /** Initialize your data structure here. */
      public MyHashSet() {
        map = new HashMap<>();
      }

      public void add(int key) {
        map.put(key, 0);
      }

      public void remove(int key) {
        if (map.containsKey(key)) {
```

```
          map.remove(key);
        }
      }

      /** Returns true if this set contains the specified element */
      public boolean contains(int key) {
        return map.containsKey(key);
      }
    }
  }
}
```

638. `_706.java`

```
package com.fishercoder.solutions;

/**706. Design HashMap
 *
 * Design a HashMap without using any built-in hash table libraries.
 *
 * To be specific, your design should include these functions:
 *
 * put(key, value) : Insert a (key, value) pair into the HashMap. If the value already
exists in the HashMap, update the value.
 * get(key): Returns the value to which the specified key is mapped, or -1 if this map
contains no mapping for the key.
 * remove(key) : Remove the mapping for the value key if this map contains the mapping
for the key.
 *
 * Example:
 *
 * MyHashMap hashMap = new MyHashMap();
 * hashMap.put(1, 1);
 * hashMap.put(2, 2);
 * hashMap.get(1);            // returns 1
 * hashMap.get(3);            // returns -1 (not found)
 * hashMap.put(2, 1);          // update the existing value
 * hashMap.get(2);            // returns 1
 * hashMap.remove(2);          // remove the mapping for 2
 * hashMap.get(2);            // returns -1 (not found)
 *
 * Note:
 *
 * All keys and values will be in the range of [0, 1000000].
 * The number of operations will be in the range of [1, 10000].
 * Please do not use the built-in HashMap library.*/
public class _706 {
    public static class Solution1 {
        /**
         * credit: https://leetcode.com/problems/design-hashmap/discuss/152746/Java-
Solution
         */
        class MyHashMap {

            final ListNode[] nodes = new ListNode[10000];

            public void put(int key, int value) {
                int i = idx(key);
                if (nodes[i] == null) {
                    nodes[i] = new ListNode(-1, -1);
                }
                ListNode prev = find(nodes[i], key);
                if (prev.next == null) {
                    prev.next = new ListNode(key, value);
                } else {
```

```
                prev.next.val = value;
            }
        }

        public int get(int key) {
            int i = idx(key);
            if (nodes[i] == null) {
                return -1;
            }
            ListNode node = find(nodes[i], key);
            return node.next == null ? -1 : node.next.val;
        }

        public void remove(int key) {
            int i = idx(key);
            if (nodes[i] == null) {
                return;
            }
            ListNode prev = find(nodes[i], key);
            if (prev.next == null) {
                return;
            }
            prev.next = prev.next.next;
        }

        int idx(int key) {
            return Integer.hashCode(key) % nodes.length;
        }

        ListNode find(ListNode bucket, int key) {
            ListNode node = bucket;
            ListNode prev = null;
            while (node != null && node.key != key) {
                prev = node;
                node = node.next;
            }
            return prev;
        }

        class ListNode {
            int key;
            int val;
            ListNode next;

            ListNode(int key, int val) {
                this.key = key;
                this.val = val;
            }
        }
    }

/**
 * Your MyHashMap object will be instantiated and called as such:
 * MyHashMap obj = new MyHashMap();
```

```
 * obj.put(key,value);
 * int param_2 = obj.get(key);
 * obj.remove(key);
 */
    }
}
```

639. `_709.java`

```
package com.fishercoder.solutions;

/**709. To Lower Case
 *
 * Implement function ToLowerCase() that has a string parameter str, and returns the same
string in lowercase.
 *
 *  Example 1:
 *
 * Input: "Hello"
 * Output: "hello"
 * Example 2:
 *
 * Input: "here"
 * Output: "here"
 * Example 3:
 *
 * Input: "LOVELY"
 * Output: "lovely"*/
public class _709 {
    public static class Solution1 {
        public String toLowerCase(String str) {
            return str.toLowerCase();
        }
    }
}
```

640. `_712.java`

```
package com.fishercoder.solutions;

/**
 * 712. Minimum ASCII Delete Sum for Two Strings
 *
 * Given two strings s1, s2, find the lowest ASCII sum of deleted characters to make two
strings equal.

 Example 1:
 Input: s1 = "sea", s2 = "eat"
 Output: 231
 Explanation: Deleting "s" from "sea" adds the ASCII value of "s" (115) to the sum.
 Deleting "t" from "eat" adds 116 to the sum.
 At the end, both strings are equal, and 115 + 116 = 231 is the minimum sum possible to
achieve this.

 Example 2:
 Input: s1 = "delete", s2 = "leet"
 Output: 403
 Explanation: Deleting "dee" from "delete" to turn the string into "let",
 adds 100[d]+101[e]+101[e] to the sum.  Deleting "e" from "leet" adds 101[e] to the sum.
 At the end, both strings are equal to "let", and the answer is 100+101+101+101 = 403.
 If instead we turned both strings into "lee" or "eet", we would get answers of 433 or
417, which are higher.

 Note:
 0 < s1.length, s2.length <= 1000.
 All elements of each string will have an ASCII value in [97, 122].
 */
public class _712 {
    public static class Solution1 {
        //credit: https://leetcode.com/articles/minimum-ascii-delete-sum-for-two-strings/
        public int minimumDeleteSum(String s1, String s2) {
            int[][] dp = new int[s1.length() + 1][s2.length() + 1];

            for (int i = s1.length() - 1; i >= 0; i--) {
                dp[i][s2.length()] = dp[i + 1][s2.length()] + s1.codePointAt(i);
            }

            for (int j = s2.length() - 1; j >= 0; j--) {
                dp[s1.length()][j] = dp[s1.length()][j + 1] + s2.codePointAt(j);
            }

            for (int i = s1.length() - 1; i >= 0; i--) {
                for (int j = s2.length() - 1; j >= 0; j--) {
                    if (s1.charAt(i) == s2.charAt(j)) {
                        dp[i][j] = dp[i + 1][j + 1];
                    } else {
                        dp[i][j] = Math.min(dp[i + 1][j] + s1.codePointAt(i), dp[i][j +
1] + s2.codePointAt(j));
                    }
```

```
                }
            }

            return dp[0][0];
        }
    }
}
```

641. `_713.java`

```
package com.fishercoder.solutions;

/**
 * 713. Subarray Product Less Than K
 *
 * Your are given an array of positive integers nums.
 * Count and print the number of (contiguous) subarrays where the product of all the
elements in the subarray is less than k.

 Example 1:
 Input: nums = [10, 5, 2, 6], k = 100
 Output: 8
 Explanation: The 8 subarrays that have product less than 100 are: [10], [5], [2], [6],
[10, 5], [5, 2], [2, 6], [5, 2, 6].
 Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.
 Note:

 0 < nums.length <= 50000.
 0 < nums[i] < 1000.
 0 <= k < 10^6.

 */
public class _713 {
    public static class Solution1 {
        /**O(n^2) solution, accepted initially, then Leetcode added one test case to fail
it.*/
        public int numSubarrayProductLessThanK(int[] nums, int k) {
            int result = 0;
            for (int i = 0; i < nums.length; i++) {
                int product = nums[i];
                if (product < k) {
                    result++;
                    for (int j = i + 1; j < nums.length; j++) {
                        product *= nums[j];
                        if (product < k) {
                            result++;
                        } else {
                            break;
                        }
                    }
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        public int numSubarrayProductLessThanK(int[] nums, int k) {
            if (k < 2) {
                return 0;
            }
```

```
            int result = 0;
            int product = 1;
            for (int i = 0, right = 0; right < nums.length; right++) {
                product *= nums[right];
                while (i < nums.length && product >= k) {
                    product /= nums[i++];
                }
                result += right - i + 1;
            }
            return result;
        }
    }
}
```

642. `_714.java`

```
package com.fishercoder.solutions;

/**
 * 714. Best Time to Buy and Sell Stock with Transaction Fee
 *
 * Your are given an array of integers prices, for which the i-th element is the price of
a given stock on day i;
 * and a non-negative integer fee representing a transaction fee.
 * You may complete as many transactions as you like,
 * but you need to pay the transaction fee for each transaction.
 * You may not buy more than 1 share of a stock at a time (ie. you must sell the stock
share before you buy again.)

 Return the maximum profit you can make.

 Example 1:
 Input: prices = [1, 3, 2, 8, 4, 9], fee = 2
 Output: 8
 Explanation: The maximum profit can be achieved by:
 Buying at prices[0] = 1
 Selling at prices[3] = 8
 Buying at prices[4] = 4
 Selling at prices[5] = 9
 The total profit is ((8 - 1) - 2) + ((9 - 4) - 2) = 8.

 Note:
 0 < prices.length <= 50000.
 0 < prices[i] < 50000.
 0 <= fee < 50000.
 */
public class _714 {
    public static class Solution1 {
        /**O(n) time
         * O(n) space
         * credit: https://discuss.leetcode.com/topic/108009/java-c-clean-code-dp-greedy
         * */
        public int maxProfit(int[] prices, int fee) {
            int n = prices.length;
            if (n < 2) {
                return 0;
            }
            int[] hold = new int[n];
            int[] sell = new int[n];
            hold[0] = -prices[0];
            for (int i = 1; i < prices.length; i++) {
                hold[i] = Math.max(hold[i - 1], sell[i - 1] - prices[i]);
                sell[i] = Math.max(sell[i - 1], hold[i - 1] - fee + prices[i]);
            }
            return sell[n - 1];
        }
    }
```

```
    public static class Solution2 {
        /**O(n) time
         * O(1) space
         * credit: https://leetcode.com/articles/best-time-to-buy-and-sell-stock-with-
transaction-fee/
         *
         * cash: the max profit we could have if we did not have a share of stock in hand
         * hold: the max profit we could have if we hold one share of stack in hand
         *
         * to transition from the i-th day to the i+1 th day, we have two options:
         * 1. sell our stock: cash = Math.max(cash, hold + prices[i] - fee)
         * 2. buy a stock: hold = Math.max(hold, cash - prices[i])
         * */
        public int maxProfit(int[] prices, int fee) {
            int cash = 0;
            int hold = -prices[0];
            for (int i = 1; i < prices.length; i++) {
                cash = Math.max(cash, hold + prices[i] - fee);
                hold = Math.max(hold, cash - prices[i]);
            }
            return cash;
        }
    }
}
```

643. `_716.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Stack;
import java.util.TreeMap;

/**
 * 716. Max Stack
 *
 * Design a max stack that supports push, pop, top, peekMax and popMax.
 *
 push(x) -- Push element x onto stack.
 pop() -- Remove the element on top of the stack and return it.
 top() -- Get the element on the top.
 peekMax() -- Retrieve the maximum element in the stack.
 popMax() -- Retrieve the maximum element in the stack, and remove it. If you find more
than one maximum elements, only remove the top-most one.

 Example 1:
 MaxStack stack = new MaxStack();
 stack.push(5);
 stack.push(1);
 stack.push(5);
 stack.top(); -> 5
 stack.popMax(); -> 5
 stack.top(); -> 1
 stack.peekMax(); -> 5
 stack.pop(); -> 1
 stack.top(); -> 5
 Note:
 -1e7 <= x <= 1e7
 Number of operations won't exceed 10000.
 The last four operations won't be called when stack is empty.
 */
public class _716 {
    public static class Solution1 {
      /**This is O(n) for popMax() and pop() while O(1) for the other three operations
which is UN-acceptable during an interview!
       * We need to do better than O(n) time complexity in order to ace the interview!
       * But O(1) is impossible, so let's aim for O(logn).*/
        public static class MaxStack {

            private int max;
            private Stack<Integer> stack;

            /**
             * initialize your data structure here.
             */
            public MaxStack() {
```

```
        max = Integer.MIN_VALUE;
        stack = new Stack<>();
    }

    public void push(int x) {
        if (x > max) {
            max = x;
        }
        stack.push(x);
    }

    public int pop() {
        if (stack.peek() == max) {
            int result = stack.pop();
            max = findMax();
            return result;
        } else {
            return stack.pop();
        }
    }

    private int findMax() {
        if (!stack.isEmpty()) {
            Iterator<Integer> iterator = stack.iterator();
            int max = stack.peek();
            while (iterator.hasNext()) {
                max = Math.max(max, iterator.next());
            }
            return max;
        } else {
            max = Integer.MIN_VALUE;
            return max;
        }
    }

    public int top() {
        return stack.peek();
    }

    public int peekMax() {
        return max;
    }

    public int popMax() {
        Stack<Integer> tmp = new Stack<>();
        int result = 0;
        while (!stack.isEmpty()) {
            if (stack.peek() != max) {
                tmp.push(stack.pop());
            } else {
                result = stack.pop();
                break;
            }
        }
```

```
                while (!tmp.isEmpty()) {
                    stack.push(tmp.pop());
                }
                max = findMax();
                return result;
            }
        }
    }

public static class Solution2 {
    /** Use a treemap and a doubly linked list to achieve O(logn) time complexity. */

    static class Node {
      int val;
      Node prev;
      Node next;

      public Node(int val) {
        this.val = val;
      }
    }

    static class DoublyLinkedList {
      Node head;
      Node tail;

      public DoublyLinkedList() {
        head = new Node(0);
        tail = new Node(0);
        head.next = tail;
        tail.prev = head;
      }

      public Node add(int val) {
        /**For this doubly linked list, we always add it to the end of the list*/
        Node x = new Node(val);
        x.next = tail;
        x.prev = tail.prev;
        tail.prev.next = x;
        tail.prev = tail.prev.next;
        return x;
      }

      public int pop() {
        /**for pop(), we always pop one from the tail of the doubly linked list*/
        return unlink(tail.prev).val;
      }

      public Node unlink(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
        return node;
      }
```

```
      public int peek() {
        return tail.prev.val;
      }
    }

    public static class MaxStack {
      TreeMap<Integer, List<Node>> treeMap;
      /**
       * the reason we have a list of nodes as treemap's value is because one value could
be pushed
       * multiple times into this MaxStack and we want to keep track of all of them.
       */
      DoublyLinkedList doublyLinkedList;

      /** initialize your data structure here. */
      public MaxStack() {
        treeMap = new TreeMap();
        doublyLinkedList = new DoublyLinkedList();
      }

      public void push(int x) {
        Node node = doublyLinkedList.add(x);
        if (!treeMap.containsKey(x)) {
          treeMap.put(x, new ArrayList<>());
        }
        treeMap.get(x).add(node);
      }

      public int pop() {
        int val = doublyLinkedList.pop();
        List<Node> nodes = treeMap.get(val);
        nodes.remove(nodes.size() - 1);
        if (nodes.isEmpty()) {
          treeMap.remove(val);
        }
        return val;
      }

      public int top() {
        return doublyLinkedList.peek();
      }

      public int peekMax() {
        return treeMap.lastKey();
      }

      public int popMax() {
        int max = treeMap.lastKey();
        List<Node> nodes = treeMap.get(max);
        Node node = nodes.remove(nodes.size() - 1);
        doublyLinkedList.unlink(node);
        if (nodes.isEmpty()) {
          treeMap.remove(max);
        }
```

```
        return max;
      }
    }
  }
}
```

644. `_717.java`

```
package com.fishercoder.solutions;

/**
 * 717. 1-bit and 2-bit Characters
 *
 * We have two special characters. The first character can be represented by one bit 0.
The second character can be represented by two bits (10 or 11).
 * Now given a string represented by several bits. Return whether the last character must
be a one-bit character or not. The given string will always end with a zero.

 Example 1:
 Input:
 bits = [1, 0, 0]
 Output: True
 Explanation:
 The only way to decode it is two-bit character and one-bit character. So the last
character is one-bit character.

 Example 2:
 Input:
 bits = [1, 1, 1, 0]
 Output: False
 Explanation:
 The only way to decode it is two-bit character and two-bit character. So the last
character is NOT one-bit character.

 Note:
 1 <= len(bits) <= 1000.
 bits[i] is always 0 or 1.
 */
public class _717 {
    public static class Solution1 {
        public boolean isOneBitCharacter(int[] bits) {
            int n = bits.length;
            int i = 0;
            while (i < n - 1) {
                if (bits[i] == 0) {
                    i++;
                } else {
                    i += 2;
                }
            }
            return i == n - 1 ? true : false;
        }
    }
}
```

645. `_718.java`

```
package com.fishercoder.solutions;

/**
 * 718. Maximum Length of Repeated Subarray
 *
 * Given two integer arrays A and B, return the maximum length of an subarray that
appears in both arrays.

 Example 1:
 Input:
 A: [1,2,3,2,1]
 B: [3,2,1,4,7]
 Output: 3

 Explanation:
 The repeated subarray with maximum length is [3, 2, 1].
 Note:
 1 <= len(A), len(B) <= 1000
 0 <= A[i], B[i] < 100
 */
public class _718 {
    public static class Solution1 {
        public int findLength(int[] A, int[] B) {
            if (A == null || B == null || A.length == 0 || B.length == 0) {
                return 0;
            }
            int[][] dp = new int[A.length + 1][B.length + 1];
            int result = 0;
            for (int i = A.length - 1; i >= 0; i--) {
                for (int j = B.length - 1; j >= 0; j--) {
                    if (A[i] == B[j]) {
                        dp[i][j] = dp[i + 1][j + 1] + 1;
                    }
                    result = Math.max(result, dp[i][j]);
                }
            }
            return result;
        }
    }
}
```

646. `_719.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * 719. Find K-th Smallest Pair Distance
 *
 * Given an integer array, return the k-th smallest distance among all the pairs.
 * The distance of a pair (A, B) is defined as the absolute difference between A and B.

 Example 1:
 Input:
 nums = [1,3,1]
 k = 1
 Output: 0
 Explanation:
 Here are all the pairs:
 (1,3) -> 2
 (1,1) -> 0
 (3,1) -> 2
 Then the 1st smallest distance pair is (1,1), and its distance is 0.

 Note:
 2 <= len(nums) <= 10000.
 0 <= nums[i] < 1000000.
 1 <= k <= len(nums) * (len(nums) - 1) / 2.
 */

public class _719 {
    public static class Solution1 {
        /**
         * This brute force solution results in TLE of course.
         */
        public int smallestDistancePair(int[] nums, int k) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> a - b);
            for (int i = 0; i < nums.length - 1; i++) {
                for (int j = i + 1; j < nums.length; j++) {
                    minHeap.offer(Math.abs(nums[j] - nums[i]));
                }
            }

            int result = 0;
            while (k-- > 0) {
                result = minHeap.poll();
            }
            return result;
        }
```

```
    }

    public static class Solution2 {
        /**
         * credit: https://leetcode.com/articles/find-k-th-smallest-pair-
distance/#approach-3-binary-search-sliding-window-accepted
         */
        public int smallestDistancePair(int[] nums, int k) {
            Arrays.sort(nums);
            int lo = 0;
            int hi = nums[nums.length - 1] - nums[0];
            while (lo < hi) {
                int mi = (lo + hi) / 2;
                int count = 0;
                int left = 0;
                for (int right = 0; right < nums.length; ++right) {
                    while (nums[right] - nums[left] > mi) {
                        left++;
                    }
                    count += right - left;
                }
                //count = number of pairs with distance <= mi
                if (count >= k) {
                    hi = mi;
                } else {
                    lo = mi + 1;
                }
            }
            return lo;
        }
    }
}
```

647. `_720.java`

```
package com.fishercoder.solutions;

/**
 * 720. Longest Word in Dictionary.
 *
 * Given a list of strings words representing an English Dictionary,
 * find the longest word in words that can be built one character at a time by other
words in words.
 * If there is more than one possible answer, return the longest word with the smallest
lexicographical order.
 * If there is no answer, return the empty string.

 Example 1:
 Input:
 words = ["w","wo","wor","worl", "world"]
 Output: "world"
 Explanation:
 The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

 Example 2:
 Input:
 words = ["a", "banana", "app", "appl", "ap", "apply", "apple"]
 Output: "apple"
 Explanation:
 Both "apply" and "apple" can be built from other words in the dictionary. However,
"apple" is lexicographically smaller than "apply".

 Note:
 All the strings in the input will only contain lowercase letters.
 The length of words will be in the range [1, 1000].
 The length of words[i] will be in the range [1, 30].
 */

public class _720 {
    public static class Solution1 {
        public String longestWord(String[] words) {
            TrieNode root = buildTrie(words);
            return findLongestWord(root, words);
        }

        private String findLongestWord(TrieNode root, String[] words) {
            String longestWord = "";
            for (String word : words) {
                if (longestWord.length() > word.length() || (longestWord.length() ==
word.length() && (longestWord.compareToIgnoreCase(word) < 0))) {
                    continue;
                }
                TrieNode tmp = root;
                boolean validWord = true;
                for (char c : word.toCharArray()) {
                    if (tmp.children[c - 'a'] != null) {
```

```
                        tmp = tmp.children[c - 'a'];
                        if (!tmp.isWord) {
                            validWord = false;
                            break;
                        }
                    }
                }
                if (validWord) {
                    longestWord = word;
                }
            }
            return longestWord;
        }

        private TrieNode buildTrie(String[] words) {
            TrieNode root = new TrieNode(' ');
            for (String word : words) {
                TrieNode tmp = root;
                for (char c : word.toCharArray()) {
                    if (tmp.children[c - 'a'] == null) {
                        tmp.children[c - 'a'] = new TrieNode(c);
                    }
                    tmp = tmp.children[c - 'a'];
                }
                tmp.isWord = true;
            }
            return root;
        }

        class TrieNode {
            char val;
            boolean isWord;
            TrieNode[] children;

            public TrieNode(char val) {
                this.val = val;
                this.isWord = false;
                this.children = new TrieNode[26];
            }
        }
    }
}
```

648. `_721.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.Stack;

/**
 * 721. Accounts Merge
 *
 * Given a list accounts, each element accounts[i] is a list of strings, where the first
element accounts[i][0] is a name, and the rest of the elements are emails representing
emails of the account.
 * Now, we would like to merge these accounts.
 * Two accounts definitely belong to the same person if there is some email that is
common to both accounts.
 * Note that even if two accounts have the same name, they may belong to different people
as people could have the same name.
 * A person can have any number of accounts initially, but all of their accounts
definitely have the same name.
 * After merging the accounts, return the accounts in the following format:
 * the first element of each account is the name, and the rest of the elements are emails
in sorted order.
 * The accounts themselves can be returned in any order.

 Example 1:
 Input:
 accounts = [["John", "johnsmith@mail.com", "john00@mail.com"], ["John",
"johnnybravo@mail.com"], ["John", "johnsmith@mail.com", "john_newyork@mail.com"],
["Mary", "mary@mail.com"]]
 Output: [["John", 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com'],
["John", "johnnybravo@mail.com"], ["Mary", "mary@mail.com"]]

 Explanation:
 The first and third John's are the same person as they have the common email
"johnsmith@mail.com".
 The second John and Mary are different people as none of their email addresses are used
by other accounts.
 We could return these lists in any order, for example the answer [['Mary',
'mary@mail.com'], ['John', 'johnnybravo@mail.com'],
 ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still
be accepted.

 Note:
 The length of accounts will be in the range [1, 1000].
 The length of accounts[i] will be in the range [1, 10].
 The length of accounts[i][j] will be in the range [1, 30].
```

```
 */
public class _721 {

    public static class Solution1 {
        /**credit: https://leetcode.com/articles/accounts-merge/#approach-1-depth-first-search-accepted
         *
         * Time Complexity: O(∑ai*logai) where ai is the length of accounts[i].
         * Without the log factor, this is the complexity to build the graph and search for each component. The log factor is for sorting each component at the end.
         * Space Complexity: O(∑ai) the space used by the graph and search.
         * .*/
        public List<List<String>> accountsMerge(List<List<String>> accounts) {
            Map<String, String> emailToName = new HashMap();
            Map<String, ArrayList<String>> graph = new HashMap();
            for (List<String> account : accounts) {
                String name = "";
                for (String email : account) {
                    if (name == "") {
                        name = email;
                        continue;
                    }
                    graph.computeIfAbsent(email, x -> new ArrayList<>()).add(account.get(1));
                    graph.computeIfAbsent(account.get(1), x -> new ArrayList<>()).add(email);
                    emailToName.put(email, name);
                }
            }

            Set<String> seen = new HashSet();
            List<List<String>> ans = new ArrayList();
            for (String email : graph.keySet()) {
                if (!seen.contains(email)) {
                    seen.add(email);
                    Stack<String> stack = new Stack();
                    stack.push(email);
                    List<String> component = new ArrayList();
                    while (!stack.empty()) {
                        String node = stack.pop();
                        component.add(node);
                        for (String nei : graph.get(node)) {
                            if (!seen.contains(nei)) {
                                seen.add(nei);
                                stack.push(nei);
                            }
                        }
                    }
                    Collections.sort(component);
                    component.add(0, emailToName.get(email));
                    ans.add(component);
                }
            }
            return ans;
```

```
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/articles/accounts-merge/#approach-2-union-find-
accepted
         * DSU stands for Disjoint Set Union: https://en.wikipedia.org/wiki/Disjoint-
set_data_structure
         *
         * Time complexity: O(AlogA)
         * Space complexity: O(A)
         * */
        public List<List<String>> accountsMerge(List<List<String>> accounts) {
            DSU dsu = new DSU();
            Map<String, String> emailToName = new HashMap<>();
            Map<String, Integer> emailToId = new HashMap<>();
            int id = 0;
            for (List<String> account : accounts) {
                String name = "";
                for (String email : account) {
                    if (name.equals("")) {
                        name = email;
                        continue;
                    }
                    emailToName.put(email, name);
                    if (!emailToId.containsKey(email)) {
                        emailToId.put(email, id++);
                    }
                    dsu.union(emailToId.get(account.get(1)), emailToId.get(email));
                }
            }

            Map<Integer, List<String>> ans = new HashMap<>();
            for (String email : emailToName.keySet()) {
                int index = dsu.find(emailToId.get(email));
                ans.computeIfAbsent(index, x -> new ArrayList()).add(email);
            }
            for (List<String> component : ans.values()) {
                Collections.sort(component);
                component.add(0, emailToName.get(component.get(0)));
            }
            return new ArrayList<>(ans.values());
        }

        class DSU {
            int[] parent;

            public DSU() {
                parent = new int[10001];
                for (int i = 0; i <= 10000; i++) {
                    parent[i] = i;
                }
            }
```

```
            public int find(int x) {
                if (parent[x] != x) {
                    parent[x] = find(parent[x]);
                }
                return parent[x];
            }

            public void union(int x, int y) {
                parent[find(x)] = find(y);
            }
        }
    }
}
```

649. `_723.java`

```
package com.fishercoder.solutions;

/**
 * 723. Candy Crush
 *
 * This question is about implementing a basic elimination algorithm for Candy Crush.
 * Given a 2D integer array board representing the grid of candy, different positive
integers board[i][j]
 * represent different types of candies.
 * A value of board[i][j] = 0 represents that the cell at position (i, j) is empty.
 * The given board represents the state of the game following the player's move.
 * Now, you need to restore the board to a stable state by crushing candies according to
the following rules:
 *
 * If three or more candies of the same type are adjacent vertically or horizontally,
 * "crush" them all at the same time - these positions become empty.
 * After crushing all candies simultaneously, if an empty space on the board has candies
on top of itself,
 * then these candies will drop until they hit a candy or bottom at the same time.
 * (No new candies will drop outside the top boundary.)
 * After the above steps, there may exist more candies that can be crushed.
 * If so, you need to repeat the above steps.
 * If there does not exist more candies that can be crushed (ie. the board is stable),
then return the current board.
 * You need to perform the above rules until the board becomes stable, then return the
current board.

 Example 1:

 Input:
 board =
 [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],
[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],
[4,1,4,4,1014]]

 Output:
 [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],
[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]
 Explanation:

 Note:
 The length of board will be in the range [3, 50].
 The length of board[i] will be in the range [3, 50].
 Each board[i][j] will initially start as an integer in the range [1, 2000].

 */
public class _723 {
    public static class Solution1 {
        /**credit: https://leetcode.com/articles/candy-crush/*/
        public int[][] candyCrush(int[][] board) {
            int row = board.length;
```

```
            int col = board[0].length;
            boolean todo = false;
            for (int i = 0; i < row; i++) {
                for (int j = 0; j < col - 2; j++) {
                    int v = Math.abs(board[i][j]);
                    if (v != 0 && v == Math.abs(board[i][j + 1]) && v ==
Math.abs(board[i][j + 2])) {
                        //mark all of them to be negative
                        board[i][j] = board[i][j + 1] = board[i][j + 2] = -v;
                        todo = true;
                    }
                }
            }

            for (int i = 0; i < row - 2; i++) {
                for (int j = 0; j < col; j++) {
                    int v = Math.abs(board[i][j]);
                    if (v != 0 && v == Math.abs(board[i + 1][j]) && v == Math.abs(board[i
+ 2][j])) {
                        board[i + 1][j] = board[i + 2][j] = board[i][j] = -v;
                        todo = true;
                    }
                }
            }

            for (int j = 0; j < col; j++) {
                int wr = row - 1;
                for (int i = row - 1; i >= 0; i--) {
                    if (board[i][j] > 0) {
                        board[wr--][j] = board[i][j];
                    }
                }
                while (wr >= 0) {
                    board[wr--][j] = 0;
                }
            }

            return todo ? candyCrush(board) : board;
        }
    }
}
```

650. `_724.java`

```
package com.fishercoder.solutions;

/**
 * 724. Find Pivot Index
 *
 * Given an array of integers nums, write a method that returns the "pivot" index of this
array.
 * We define the pivot index as the index where the sum of the numbers to the left of the
index is equal
 * to the sum of the numbers to the right of the index.
 * If no such index exists, we should return -1. If there are multiple pivot indexes, you
should return the left-most pivot index.

 Example 1:
 Input:
 nums = [1, 7, 3, 6, 5, 6]
 Output: 3
 Explanation:
 The sum of the numbers to the left of index 3 (nums[3] = 6) is equal to the sum of
numbers to the right of index 3.
 Also, 3 is the first index where this occurs.

 Example 2:
 Input:
 nums = [1, 2, 3]
 Output: -1
 Explanation:
 There is no index that satisfies the conditions in the problem statement.

 Note:
 The length of nums will be in the range [0, 10000].
 Each element nums[i] will be an integer in the range [-1000, 1000].
 */
public class _724 {
    public static class Solution1 {
        /**Space: O(n)
         * Time: O(n)*/
        public int pivotIndex(int[] nums) {
            if (nums == null || nums.length == 0) {
                return -1;
            }
            int[] sums = new int[nums.length];
            sums[0] = nums[0];
            for (int i = 1; i < nums.length; i++) {
                sums[i] = sums[i - 1] + nums[i];
            }
            for (int i = 0; i < nums.length; i++) {
                if (i == 0 && 0 == sums[nums.length - 1] - sums[i] || (i > 0 && sums[i -
1] == sums[nums.length - 1] - sums[i])) {
                    return i;
                }
```

```
            }
            return -1;
        }
    }

    public static class Solution2 {
        /**Space: O(1)
         * Time: O(n)*/
        public int pivotIndex(int[] nums) {
            int total = 0;
            for (int num : nums) {
                total += num;
            }
            int sum = 0;
            for (int i = 0; i < nums.length; sum += nums[i++]) {
                if (sum * 2 == total - nums[i]) {
                    return i;
                }
            }
            return -1;
        }
    }
}
```

651. `_725.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 725. Split Linked List in Parts
 *
 * Given a (singly) linked list with head node root,
 * write a function to split the linked list into k consecutive linked list "parts".
 * The length of each part should be as equal as possible:
 * no two parts should have a size differing by more than 1. This may lead to some parts
being null.
 * The parts should be in order of occurrence in the input list,
 * and parts occurring earlier should always have a size greater than or equal parts
occurring later.
 * Return a List of ListNode's representing the linked list parts that are formed.

 Examples 1->2->3->4, k = 5 // 5 equal parts [ [1], [2], [3], [4], null ]

 Example 1:
 Input:
 root = [1, 2, 3], k = 5
 Output: [[1],[2],[3],[],[]]
 Explanation:
 The input and each element of the output are ListNodes, not arrays.
 For example, the input root has root.val = 1, root.next.val = 2, \root.next.next.val =
3, and root.next.next.next = null.
 The first element output[0] has output[0].val = 1, output[0].next = null.
 The last element output[4] is null, but it's string representation as a ListNode is [].

 Example 2:
 Input:
 root = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], k = 3
 Output: [[1, 2, 3, 4], [5, 6, 7], [8, 9, 10]]
 Explanation:
 The input has been split into consecutive parts with size difference at most 1, and
earlier parts are a larger size than the later parts.

 Note:
 The length of root will be in the range [0, 1000].
 Each value of a node in the input will be an integer in the range [0, 999].
 k will be an integer in the range [1, 50].

 */
public class _725 {
    public static class Solution1 {
        /**My very original solution, but verbose.*/
        public ListNode[] splitListToParts(ListNode root, int k) {
            int len = getLength(root);
            int aveSize = len / k;
            int extra = len % k;
```

```
            ListNode[] result = new ListNode[k];
            for (int i = 0; i < k; i++) {
                result[i] = root;
                int aveSizeTmp = aveSize;
                aveSizeTmp += extra-- > 0 ? 1 : 0;
                int aveSizeTmp2 = aveSizeTmp;
                while (aveSizeTmp-- > 0) {
                    root = root.next;
                }
                if (result[i] != null) {
                    ListNode tmp = result[i];
                    while (tmp.next != null && aveSizeTmp2-- > 1) {
                        tmp = tmp.next;
                    }
                    tmp.next = null;
                }
            }
            return result;
        }

        private int getLength(ListNode root) {
            int len = 0;
            ListNode tmp = root;
            while (tmp != null) {
                len++;
                tmp = tmp.next;
            }
            return len;
        }
    }

    public static class Solution2 {
        /**More concise version*/
        public ListNode[] splitListToParts(ListNode root, int k) {
            int len = getLength(root);
            int aveSize = len / k;
            int extra = len % k;
            ListNode[] result = new ListNode[k];
            ListNode prev = null;
            for (int i = 0; i < k; i++, extra--) {
                result[i] = root;
                for (int j = 0; j < aveSize + (extra > 0 ? 1 : 0); j++) {
                    prev = root;
                    root = root.next;
                }
                if (prev != null) {
                    prev.next = null;
                }
            }
            return result;
        }

        private int getLength(ListNode root) {
            int len = 0;
```

```
            ListNode tmp = root;
            while (tmp != null) {
                len++;
                tmp = tmp.next;
            }
            return len;
        }
    }
}
```

652. `_727.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 727. Minimum Window Subsequence
 *
 * Given strings S and T, find the minimum (contiguous) substring W of S, so that T is a
subsequence of W.
 * If there is no such window in S that covers all characters in T,
 * return the empty string "". If there are multiple such minimum-length windows, return
the one with the left-most starting index.

 Example 1:
 Input:
 S = "abcdebdde", T = "bde"
 Output: "bcde"

 Explanation:
 "bcde" is the answer because it occurs before "bdde" which has the same length.
 "deb" is not a smaller window because the elements of T in the window must occur in
order.

 Note:
 All the strings in the input will only contain lowercase letters.
 The length of S will be in the range [1, 20000].
 The length of T will be in the range [1, 100].
 */
public class _727 {
    public static class Solution1 {
        /**
         * This naive brute force results in TLE.
         */
        public String minWindow(String S, String T) {
            String result = S;
            for (int i = 0; i < S.length(); i++) {
                for (int j = i + T.length(); j <= S.length(); j++) {
                    String sub = S.substring(i, j);
                    if (sub.length() < result.length() && isSubsequence(T, sub)) {
                        result = sub;
                    }
                }
            }
            return result.equals(S) ? "" : result;
        }

        private boolean isSubsequence(String T, String sub) {
            int i = 0;
            for (int j = 0; i < T.length() && j < sub.length(); j++) {
                if (T.charAt(i) == sub.charAt(j)) {
                    i++;
```

```
                }
            }
            return i == T.length();
        }
    }

    public static class Solution2 {
        /**credit: https://github.com/lydxlx1/LeetCode/blob/master/src/_727.java*/
        public String minWindow(String S, String T) {
            int[][] dp = new int[S.length() + 1][T.length() + 1];
            int INFINITY = 1 << 29;
            Arrays.fill(dp[0], INFINITY);
            dp[0][0] = 0;
            for (int i = 1; i <= S.length(); i++) {
                for (int j = 1; j <= T.length(); j++) {
                    dp[i][j] = dp[i - 1][j] + 1;
                    if (S.charAt(i - 1) == T.charAt(j - 1)) {
                        dp[i][j] = dp[i - 1][j - 1] + 1;
                    }
                }
            }
            int ans = INFINITY;
            int tail = -1;
            for (int i = 1; i <= S.length(); i++) {
                if (dp[i][T.length()] < ans) {
                    ans = dp[i][T.length()];
                    tail = i;
                }
            }
            return ans == INFINITY ? "" : S.substring(tail - ans, tail);
        }

    }
}
```

653. `_728.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 728. Self Dividing Numbers
 *
 * A self-dividing number is a number that is divisible by every digit it contains.
 * For example, 128 is a self-dividing number because 128 % 1 == 0, 128 % 2 == 0, and 128
% 8 == 0.
 * Also, a self-dividing number is not allowed to contain the digit zero.
 * Given a lower and upper number bound, output a list of every possible self dividing
number, including the bounds if possible.

 Example 1:
 Input:
 left = 1, right = 22
 Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 22]
 Note:

 The boundaries of each input argument are 1 <= left <= right <= 10000.
 */
public class _728 {
    public static class Solution1 {
        public List<Integer> selfDividingNumbers(int left, int right) {
            List<Integer> result = new ArrayList<>();
            for (int num = left; num <= right; num++) {
                if (isSelfDividing(num)) {
                    result.add(num);
                }
            }
            return result;
        }

        private boolean isSelfDividing(int num) {
            int tmp = num;
            while (tmp != 0) {
                int digit = tmp % 10;
                if (digit == 0 || num % digit != 0) {
                    return false;
                }
                tmp /= 10;
            }
            return true;
        }
    }
}
```

654. `_729.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

/**
 * 729. My Calendar I
 *
 * Implement a MyCalendar class to store your events. A new event can be added if adding
the event will not cause a double booking.
 * Your class will have the method, book(int start, int end).
 * Formally, this represents a booking on the half open interval [start, end),
 * the range of real numbers x such that start <= x < end.
 * A double booking happens when two events have some non-empty intersection (ie., there
is some time that is common to both events.)
 * For each call to the method MyCalendar.book,
 * return true if the event can be added to the calendar successfully without causing a
double booking.
 * Otherwise, return false and do not add the event to the calendar.

 Your class will be called like this: MyCalendar cal = new MyCalendar();
MyCalendar.book(start, end)

 Example 1:
 MyCalendar();
 MyCalendar.book(10, 20); // returns true
 MyCalendar.book(15, 25); // returns false
 MyCalendar.book(20, 30); // returns true

 Explanation:

 The first event can be booked.  The second can't because time 15 is already booked by
another event.
 The third event can be booked, as the first event takes every time less than 20, but not
including 20.
 Note:

 The number of calls to MyCalendar.book per test case will be at most 1000.
 In calls to MyCalendar.book(start, end), start and end are integers in the range [0,
10^9].

 */
public class _729 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/111205/java-8-liner-treemap
         */
        public static class MyCalendar {
            TreeMap<Integer, Integer> calendar;
```

```
            public MyCalendar() {
                calendar = new TreeMap<>();
            }

            public boolean book(int start, int end) {
                Integer floorKey = calendar.floorKey(start);
                if (floorKey != null && calendar.get(floorKey) > start) {
                    return false;
                }
                Integer ceilingKey = calendar.ceilingKey(start);
                if (ceilingKey != null && ceilingKey < end) {
                    return false;
                }
                calendar.put(start, end);
                return true;
            }
        }
    }

    public static class Solution2 {
        public class MyCalendar {
            List<int[]> calendar;

            MyCalendar() {
                calendar = new ArrayList();
            }

            public boolean book(int start, int end) {
                for (int i = 0; i < calendar.size(); i++) {
                    if (calendar.get(i)[0] < end && start < calendar.get(i)[1]) {
                        return false;
                    }
                }
                calendar.add(new int[]{start, end});
                return true;
            }
        }
    }
}
```

655. `_733.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 733. Flood Fill
 *
 * An image is represented by a 2-D array of integers, each integer representing the
pixel value of the image (from 0 to 65535).
 * Given a coordinate (sr, sc) representing the starting pixel (row and column) of the
flood fill, and a pixel value newColor, "flood fill" the image.
 * To perform a "flood fill", consider the starting pixel, plus any pixels connected 4-
directionally to the starting
 * pixel of the same color as the starting pixel,
 * plus any pixels connected 4-directionally to those pixels (also with the same color as
the starting pixel), and so on.
 * Replace the color of all of the aforementioned pixels with the newColor.
 * At the end, return the modified image.

 Example 1:
 Input:
 image = [[1,1,1],[1,1,0],[1,0,1]]
 sr = 1, sc = 1, newColor = 2
 Output: [[2,2,2],[2,2,0],[2,0,1]]

 Explanation:
 From the center of the image (with position (sr, sc) = (1, 1)), all pixels connected
 by a path of the same color as the starting pixel are colored with the new color.
 Note the bottom corner is not colored 2, because it is not 4-directionally connected
 to the starting pixel.

 Note:
 The length of image and image[0] will be in the range [1, 50].
 The given starting pixel will satisfy 0 <= sr < image.length and 0 <= sc <
image[0].length.
 The value of each color in image[i][j] and newColor will be an integer in [0, 65535].
 */
public class _733 {
    public static class Solution1 {
        public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
            int[] directions = new int[]{0, 1, 0, -1, 0};
            int m = image.length;
            int n = image[0].length;
            int originalValue = image[sr][sc];
            image[sr][sc] = newColor;

            boolean[][] visited = new boolean[m][n];

            Queue<int[]> queue = new LinkedList<>();
            queue.offer(new int[]{sr, sc});
```

```
            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                visited[curr[0]][curr[1]] = true;
                for (int i = 0; i < directions.length - 1; i++) {
                    int nextR = curr[0] + directions[i];
                    int nextC = curr[1] + directions[i + 1];
                    if (nextR < 0 || nextC < 0 || nextR >= m || nextC >= n ||
image[nextR][nextC] != originalValue || visited[nextR][nextC]) {
                        continue;
                    }
                    image[nextR][nextC] = newColor;
                    queue.offer(new int[]{nextR, nextC});
                }
            }
            return image;
        }
    }
}
```

656. `_734.java`

```
package com.fishercoder.solutions;

/**
 * 734. Sentence Similarity
 *
 * Given two sentences words1, words2 (each represented as an array of strings), and a
list of similar word pairs pairs, determine if two sentences are similar.
 * For example, "great acting skills" and "fine drama talent" are similar, if the similar
word pairs are pairs = [["great", "fine"], ["acting","drama"], ["skills","talent"]].
 * Note that the similarity relation is not transitive. For example, if "great" and
"fine" are similar, and "fine" and "good" are similar, "great" and "good" are not
necessarily similar.
 * Also, a word is always similar with itself. For example, the sentences words1 =
["great"], words2 = ["great"], pairs = [] are similar, even though there are no specified
similar word pairs.

 Note:

 The length of words1 and words2 will not exceed 1000.
 The length of pairs will not exceed 2000.
 The length of each pairs[i] will be 2.
 The length of each words[i] and pairs[i][j] will be in the range [1, 20].
 */
public class _734 {
    public static class Solution1 {
        public boolean areSentencesSimilar(String[] words1, String[] words2, String[][]
pairs) {
            if (words1.length != words2.length) {
                return false;
            }
            for (int i = 0; i < words1.length; i++) {
                if (!isSimilar(words1[i], words2[i], pairs)) {
                    return false;
                }
            }
            return true;
        }

        private boolean isSimilar(String word1, String word2, String[][] pairs) {
            if (word1.equals(word2)) {
                return true;
            }
            for (int i = 0; i < pairs.length; i++) {
                String[] pair = pairs[i];
                if (pair[0].equals(word1) && pair[1].equals(word2)) {
                    return true;
                }
                if (pair[0].equals(word2) && pair[1].equals(word1)) {
                    return true;
                }
            }
```

```
            return false;
        }
    }
}
```

657. `_735.java`

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 735. Asteroid Collision
 *
 * We are given an array asteroids of integers representing asteroids in a row.
 * For each asteroid, the absolute value represents its size, and the sign represents its
direction
 * (positive meaning right, negative meaning left). Each asteroid moves at the same
speed.
 * Find out the state of the asteroids after all collisions.
 * If two asteroids meet, the smaller one will explode. If both are the same size, both
will explode. Two asteroids moving in the same direction will never meet.

 Example 1:
 Input:
 asteroids = [5, 10, -5]
 Output: [5, 10]
 Explanation:
 The 10 and -5 collide resulting in 10.  The 5 and 10 never collide.

 Example 2:
 Input:
 asteroids = [8, -8]
 Output: []
 Explanation:
 The 8 and -8 collide exploding each other.

 Example 3:
 Input:
 asteroids = [10, 2, -5]
 Output: [10]
 Explanation:
 The 2 and -5 collide resulting in -5.  The 10 and -5 collide resulting in 10.

 Example 4:
 Input:
 asteroids = [-2, -1, 1, 2]
 Output: [-2, -1, 1, 2]
 Explanation:
 The -2 and -1 are moving left, while the 1 and 2 are moving right.
 Asteroids moving the same direction never meet, so no asteroids will meet each other.

 Note:
 The length of asteroids will be at most 10000.
 Each asteroid will be a non-zero integer in the range [-1000, 1000]..

 */
public class _735 {
```

```
    public static class Solution1 {
        public int[] asteroidCollision(int[] asteroids) {
            Stack<Integer> stack = new Stack();
            for (int i = 0; i < asteroids.length; i++) {
                if (!stack.isEmpty() && stack.peek() > 0 && asteroids[i] < 0) {
                    if (Math.abs(stack.peek()) < Math.abs(asteroids[i])) {
                        stack.pop();
                        stack.push(asteroids[i]);
                        collide(stack);
                    } else if (Math.abs(stack.peek()) == Math.abs(asteroids[i])) {
                        stack.pop();
                    }
                } else {
                    stack.push(asteroids[i]);
                }
            }
            int[] result = new int[stack.size()];
            int i = stack.size();
            while (!stack.isEmpty()) {
                result[--i] = stack.pop();
            }
            return result;
        }

        private void collide(Stack<Integer> stack) {
            do {
                Integer top = stack.pop();
                if (!stack.isEmpty() && stack.peek() * top < 0) {
                    if (stack.peek() < Math.abs(top)) {
                        stack.pop();
                        stack.push(top);
                    } else if (stack.peek() == Math.abs(top)) {
                        stack.pop();
                        break;
                    } else {
                        break;
                    }
                } else if (stack.isEmpty() || stack.peek() * top > 0) {
                    stack.push(top);
                    break;
                }
            } while (!stack.isEmpty());
        }
    }
}
```

658. `_737.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 737. Sentence Similarity II
 *
 * Given two sentences words1, words2 (each represented as an array of strings), and a
list of similar word pairs pairs, determine if two sentences are similar.
 * For example, words1 = ["great", "acting", "skills"] and words2 = ["fine", "drama",
"talent"] are similar,
 * if the similar word pairs are pairs = [["great", "good"], ["fine", "good"],
["acting","drama"], ["skills","talent"]].
 * Note that the similarity relation is transitive.
 *
 * For example, if "great" and "good" are similar, and "fine" and "good" are similar,
then "great" and "fine" are similar.
 * Similarity is also symmetric. For example, "great" and "fine" being similar is the
same as "fine" and "great" being similar.
 * Also, a word is always similar with itself. For example, the sentences words1 =
["great"], words2 = ["great"], pairs = [] are similar, even though there are no specified
similar word pairs.
 * Finally, sentences can only be similar if they have the same number of words. So a
sentence like words1 = ["great"] can never be similar to words2 = ["doubleplus","good"].

 Note:

 The length of words1 and words2 will not exceed 1000.
 The length of pairs will not exceed 2000.
 The length of each pairs[i] will be 2.
 The length of each words[i] and pairs[i][j] will be in the range [1, 20].

 */
public class _737 {
    public static class Solution1 {
        public boolean areSentencesSimilarTwo(String[] words1, String[] words2, String[]
[] pairs) {
            if (words1.length != words2.length) {
                return false;
            }

            Map<String, String> map = new HashMap<>();
            for (String[] pair : pairs) {
                if (!map.containsKey(pair[0])) {
                    map.put(pair[0], pair[0]);
                }
                if (!map.containsKey(pair[1])) {
                    map.put(pair[1], pair[1]);
                }
```

```
                setParent(map, pair[0], pair[1]);
            }

            for (int i = 0; i < words1.length; i++) {
                String parent1 = getParent(map, words1[i]);
                String parent2 = getParent(map, words2[i]);
                if (!parent1.equals(parent2)) {
                    return false;
                }
            }
            return true;
        }

        private void setParent(Map<String, String> map, String word1, String word2) {
            String parent1 = getParent(map, word1);
            String parent2 = getParent(map, word2);
            map.put(parent1, parent2);
        }

        private String getParent(Map<String, String> map, String word) {
            if (!map.containsKey(word)) {
                return word;
            }
            while (!word.equals(map.get(word))) {
                word = map.get(word);
            }
            return word;
        }
    }
}
```

659. `_738.java`

```
package com.fishercoder.solutions;

/**
 * 738. Monotone Increasing Digits
 *
 * Given a non-negative integer N, find the largest number that is less than or equal to
N with monotone increasing digits.
 * (Recall that an integer has monotone increasing digits if and only if each pair of
adjacent digits x and y satisfy x <= y.)

 Example 1:
 Input: N = 10
 Output: 9

 Example 2:
 Input: N = 1234
 Output: 1234

 Example 3:
 Input: N = 332
 Output: 299

 Note: N is an integer in the range [0, 10^9].
 */
public class _738 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/112808/simple-python-solution-w-
explanation/2
         */
        public int monotoneIncreasingDigits(int N) {
            String s = Integer.toString(N);
            int index = -1;
            for (int i = s.length() - 2; i >= 0; i--) {
                if (s.charAt(i) > s.charAt(i + 1) || (index != -1 && s.charAt(index) ==
s.charAt(i))) {
                    index = i;
                }
            }
            return index == -1 ? N : N - Integer.parseInt(s.substring(index + 1,
s.length())) - 1;
        }
    }
}
```

660. `_739.java`

```
package com.fishercoder.solutions;

/**
 * 739. Daily Temperatures
 *
 * Given a list of daily temperatures, produce a list that,
 * for each day in the input, tells you how many days you would have to wait until a
warmer temperature.
 * If there is no future day for which this is possible, put 0 instead.
 * For example, given the list temperatures = [73, 74, 75, 71, 69, 72, 76, 73], your
output should be [1, 1, 4, 2, 1, 1, 0, 0].
 *
 * Note: The length of temperatures will be in the range [1, 30000]. Each temperature
will be an integer in the range [30, 100].
 */
public class _739 {

    public static class Solution1 {
        public int[] dailyTemperatures(int[] temperatures) {
            if (temperatures == null || temperatures.length == 0) {
                return temperatures;
            }
            int[] result = new int[temperatures.length];
            for (int i = 0; i < temperatures.length; i++) {
                for (int j = i + 1; j < temperatures.length; j++) {
                    if (temperatures[j] > temperatures[i]) {
                        result[i] = j - i;
                        break;
                    }
                }
            }
            return result;
        }
    }
}
```

661. `_740.java`

```
package com.fishercoder.solutions;

/**
 * 740. Delete and Earn
 *
 * Given an array nums of integers, you can perform operations on the array.
 * In each operation, you pick any nums[i] and delete it to earn nums[i] points. After,
you must delete every element equal to nums[i] - 1 or nums[i] + 1.
 * You start with 0 points. Return the maximum number of points you can earn by applying
such operations.

 Example 1:
 Input: nums = [3, 4, 2]
 Output: 6
 Explanation:
 Delete 4 to earn 4 points, consequently 3 is also deleted.
 Then, delete 2 to earn 2 points. 6 total points are earned.

 Example 2:
 Input: nums = [2, 2, 3, 3, 3, 4]
 Output: 9
 Explanation:
 Delete 3 to earn 3 points, deleting both 2's and the 4.
 Then, delete 3 again to earn 3 points, and 3 again to earn 3 points.
 9 total points are earned.

 Note:
 The length of nums is at most 20000.
 Each element nums[i] is an integer in the range [1, 10000].*/

public class _740 {
  public static class Solution1 {
    /**
     * Since the number is within range [1, 10000], we can build another array:
     * each number in the array denotes the total sum of this number that appears in this
array
     * and
     * use the numbers themselves in the indices of another array
     *
     * credit: https://leetcode.com/problems/delete-and-earn/discuss/109895/JavaC++-
Clean-Code-with-Explanation*/
    public int deleteAndEarn(int[] nums) {
      int n = 10001;
      int[] values = new int[n];
      for (int num : nums) {
        values[num] += num;
      }

      int take = 0;
      int skip = 0;
      for (int i = 0; i < n; i++) {
```

```
        int takeI = skip + values[i];
        int skipI = Math.max(skip, take);
        take = takeI;
        skip = skipI;
      }
      return Math.max(take, skip);
    }
  }
}
```

662. `_743.java`

```
package com.fishercoder.solutions;

/**
 * 743. Network Delay Time
 *
 * There are N network nodes, labelled 1 to N.
 *
 *  Given times, a list of travel times as directed edges times[i] = (u, v, w), where u
is the source node, v is the target node, and w is the time it takes for a signal to
travel from source to target.
 *
 * Now, we send a signal from a certain node K. How long will it take for all nodes to
receive the signal? If it is impossible, return -1.
 *
 * Note:
 *  * N will be in the range [1, 100].
 *  * K will be in the range [1, N].
 *  * The length of times will be in the range [1, 6000].
 *  * All edges times[i] = (u, v, w) will have 1 <= u, v <= N and 0 <= w <= 100.
 */

public class _743 {
    public static class Solution1 {
        public static final int MAX = 100_00_00;

        private int[][] parseTimes(int[][] times, int N) {
            int[][] weights = new int[N + 1][N + 1];

            for (int i = 0; i < N + 1; ++i) {
                for (int j = 0; j < N + 1; ++j) {
                    weights[i][j] = MAX;
                }
            }

            for (int[] time : times) {
                int u = time[0];
                int v = time[1];
                int w = time[2];
                weights[u][v] = w;
            }

            return weights;
        }

        private int findMinUnvisitedIndex(int[] a, boolean[] visited) {
            int min = Integer.MAX_VALUE;
            int minIdx = 1;

            for (int i = 1; i < a.length; ++i) {
                if (a[i] < min && !visited[i]) {
                    min = a[i];
```

```
                    minIdx = i;
                }
            }
            return minIdx;
        }

        public int networkDelayTime(int[][] times, int N, int K) {
            // Time from u to v is weights[u][v]
            int[][] weights = parseTimes(times, N);

            // Time from node K to note i is totalTime[i]
            int[] totalTime = new int[N + 1];
            for (int j = 0; j <= N; ++j) {
                totalTime[j] = MAX;
            }
            totalTime[K] = 0;

            boolean[] visited = new boolean[N + 1];
            int visitCount = 0;

            while (visitCount != N) {
                int node = findMinUnvisitedIndex(totalTime, visited);

                // The smallest time to a node is more than our designated max
                // so it must be unreachable
                if (totalTime[node] >= MAX) {
                    return -1;
                }

                visited[node] = true;
                visitCount += 1;

                // Update the time to each node if we try to visit it from our current
node
                for (int i = 1; i <= N; ++i) {
                    totalTime[i] = Math.min(weights[node][i] + totalTime[node],
totalTime[i]);
                }
            }

            // What is the maximum time to any particular node
            int maxVal = Integer.MIN_VALUE;
            for (int k = 1; k <= N; ++k) {
                maxVal = Math.max(totalTime[k], maxVal);
            }

            return maxVal;
        }
    }
}
```

663. `_744.java`

```
package com.fishercoder.solutions;

/**
 * 744. Find Smallest Letter Greater Than Target
 *
 * Given a list of sorted characters letters containing only lowercase letters, and given
a target letter target, find the smallest element in the list that is larger than the
given target.
 * Letters also wrap around. For example, if the target is target = 'z' and letters =
['a', 'b'], the answer is 'a'.

 Examples:
 Input:
 letters = ["c", "f", "j"]
 target = "a"
 Output: "c"

 Input:
 letters = ["c", "f", "j"]
 target = "c"
 Output: "f"

 Input:
 letters = ["c", "f", "j"]
 target = "d"
 Output: "f"

 Input:
 letters = ["c", "f", "j"]
 target = "g"
 Output: "j"

 Input:
 letters = ["c", "f", "j"]
 target = "j"
 Output: "c"

 Input:
 letters = ["c", "f", "j"]
 target = "k"
 Output: "c"

 Note:
 letters has a length in range [2, 10000].
 letters consists of lowercase letters, and contains at least 2 unique letters.
 target is a lowercase letter.
 */
public class _744 {
    public static class Solution1 {
        public char nextGreatestLetter(char[] letters, char target) {
            if (letters[0] > target) {
```

```
                return letters[0];
            }
            int left = 0;
            int right = letters.length - 1;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (letters[mid] > target) {
                    while (letters[mid] > target) {
                        mid--;
                    }
                    return letters[++mid];
                } else {
                    left = mid + 1;
                }
            }
            if (right < letters.length && letters[right] > target) {
                return letters[right];
            }
            return letters[0];
        }
    }
}
```

664. `_746.java`

```
package com.fishercoder.solutions;

/**
 * 746. Min Cost Climbing Stairs
 *
 * On a staircase, the i-th step has some non-negative cost cost[i] assigned (0 indexed).
 * Once you pay the cost, you can either climb one or two steps.
 * You need to find minimum cost to reach the top of the floor, and you can either start
from the step with index 0, or the step with index 1.

 Example 1:
 Input: cost = [10, 15, 20]
 Output: 15
 Explanation: Cheapest is start on cost[1], pay that cost and go to the top.

 Example 2:
 Input: cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
 Output: 6
 Explanation: Cheapest is start on cost[0], and only step on 1s, skipping cost[3].

 Note:
 cost will have a length in the range [2, 1000].
 Every cost[i] will be an integer in the range [0, 999].
 */

public class _746 {
  public static class Solution1 {
    public int minCostClimbingStairs(int[] cost) {
      int[] dp = new int[cost.length];
      dp[0] = cost[0];
      dp[1] = cost[1];
      for (int i = 2; i < cost.length; i++) {
        dp[i] = Math.min(dp[i - 1] + cost[i], dp[i - 2] + cost[i]);
      }
      return Math.min(dp[cost.length - 1], dp[cost.length - 2]);
    }
  }
}
```

665. `_747.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 747. Largest Number Greater Than Twice of Others
 *
 * In a given integer array nums, there is always exactly one largest element.
 * Find whether the largest element in the array is at least twice as much as every other
number in the array.
 * If it is, return the index of the largest element, otherwise return -1.

 Example 1:
 Input: nums = [3, 6, 1, 0]
 Output: 1
 Explanation: 6 is the largest integer, and for every other number in the array x,
 6 is more than twice as big as x.  The index of value 6 is 1, so we return 1.

 Example 2:
 Input: nums = [1, 2, 3, 4]
 Output: -1
 Explanation: 4 isn't at least as big as twice the value of 3, so we return -1.

 Note:
 nums will have a length in the range [1, 50].
 Every nums[i] will be an integer in the range [0, 99].
 */
public class _747 {

  public static class Solution1 {
    public int dominantIndex(int[] nums) {
      Map<Integer, Integer> map = new HashMap<>();
      int max;
      int secondMax;
      for (int i = 0; i < nums.length; i++) {
        map.put(nums[i], i);
      }
      Arrays.sort(nums);
      max = nums[nums.length - 1];
      secondMax = nums[nums.length - 2];
      if (max >= 2 * secondMax) {
        return map.get(max);
      } else {
        return -1;
      }
    }
  }

  public static class Solution2 {
```

```
    public int dominantIndex(int[] nums) {
      int max = Integer.MIN_VALUE;
      int maxIndex = -1;
      for (int i = 0; i < nums.length; i++) {
        if (nums[i] > max) {
          max = nums[i];
          maxIndex = i;
        }
      }
      for (int i = 0; i < nums.length; i++) {
        if (nums[i] * 2 > max && i != maxIndex) {
          return -1;
        }
      }
      return maxIndex;
    }
  }
}
```

666. `_748.java`

```
package com.fishercoder.solutions;

/**
 * 748. Shortest Completing Word
 *
 * Find the minimum length word from a given dictionary words,
 * which has all the letters from the string licensePlate. Such a word is said to
complete the given string licensePlate
 *
 * Here, for letters we ignore case. For example, "P" on the licensePlate still matches
"p" on the word.
 * It is guaranteed an answer exists. If there are multiple answers, return the one that
occurs first in the array.
 * The license plate might have the same letter occurring multiple times. For example,
 * given a licensePlate of "PP", the word "pair" does not complete the licensePlate, but
the word "supper" does.

 Example 1:
 Input: licensePlate = "1s3 PSt", words = ["step", "steps", "stripe", "stepple"]
 Output: "steps"
 Explanation: The smallest length word that contains the letters "S", "P", "S", and "T".
 Note that the answer is not "step", because the letter "s" must occur in the word twice.
 Also note that we ignored case for the purposes of comparing whether a letter exists in
the word.

 Example 2:
 Input: licensePlate = "1s3 456", words = ["looks", "pest", "stew", "show"]
 Output: "pest"
 Explanation: There are 3 smallest length words that contains the letters "s".
 We return the one that occurred first.

 Note:
 licensePlate will be a string with length in range [1, 7].
 licensePlate will contain digits, spaces, or letters (uppercase or lowercase).
 words will have a length in the range [10, 1000].
 Every words[i] will consist of lowercase letters, and have length in range [1, 15].

 */
public class _748 {

  public static class Solution1 {
    public String shortestCompletingWord(String licensePlate, String[] words) {
      int[] counts = new int[26];
      for (char c : licensePlate.toCharArray()) {
        if (Character.isAlphabetic(c)) {
          counts[Character.toLowerCase(c) - 'a']++;
        }
      }
      String result = "";
      for (String word : words) {
        if (isComplete(word, counts)) {
```

```
          if (result.equals("")) {
            result = word;
          } else if (word.length() < result.length()) {
            result = word;
          }
        }
      }
      return result;
    }

    private boolean isComplete(String word, int[] counts) {
      int[] tmp = counts.clone();
      for (char c : word.toCharArray()) {
        if (tmp[c - 'a'] > 0) {
          tmp[c - 'a']--;
        }
      }
      for (int i : tmp) {
        if (i != 0) {
          return false;
        }
      }
      return true;
    }
  }
}
```

667. `_749.java`

```
package com.fishercoder.solutions;

/**
 * 749. Contain Virus
 *
 * A virus is spreading rapidly, and your task is to quarantine the infected area by
installing walls.
 * The world is modeled as a 2-D array of cells, where 0 represents uninfected cells,
 * and 1 represents cells contaminated with the virus.
 * A wall (and only one wall) can be installed between any two 4-directionally adjacent
cells, on the shared boundary.
 * Every night, the virus spreads to all neighboring cells in all four directions unless
blocked by a wall.
 * Resources are limited. Each day, you can install walls around only one region --
 * the affected area (continuous block of infected cells) that threatens the most
uninfected cells the following night. There will never be a tie.
 * Can you save the day? If so, what is the number of walls required? If not, and the
world becomes fully infected, return the number of walls used.

 Example 1:
 Input: grid =
 [[0,1,0,0,0,0,0,1],
 [0,1,0,0,0,0,0,1],
 [0,0,0,0,0,0,0,1],
 [0,0,0,0,0,0,0,0]]

 Output: 10
 Explanation:
 There are 2 contaminated regions.
 On the first day, add 5 walls to quarantine the viral region on the left. The board
after the virus spreads is:

 [[0,1,0,0,0,0,1,1],
 [0,1,0,0,0,0,1,1],
 [0,0,0,0,0,0,1,1],
 [0,0,0,0,0,0,0,1]]

 On the second day, add 5 walls to quarantine the viral region on the right. The virus is
fully contained.


 Example 2:
 Input: grid =
 [[1,1,1],
 [1,0,1],
 [1,1,1]]

 Output: 4
 Explanation: Even though there is only one cell saved, there are 4 walls built.
 Notice that walls are only built on the shared boundary of two different cells.
```

```
 Example 3:
 Input: grid =
 [[1,1,1,0,0,0,0,0,0],
 [1,0,1,0,1,1,1,1,1],
 [1,1,1,0,0,0,0,0,0]]
 Output: 13
 Explanation: The region on the left only builds two new walls.
 Note:
 The number of rows and columns of grid will each be in the range [1, 50].
 Each grid[i][j] will be either 0 or 1.
 Throughout the described process, there is always a contiguous viral region that will
infect strictly more uncontaminated squares in the next round.
 */
public class _749 {
  public static class Solution1 {
    public int containVirus(int[][] grid) {
      return -1;
    }
  }
}
```

668. `_750.java`

```
package com.fishercoder.solutions;

/**
 * 750. Number Of Corner Rectangles
 *
 * Given a grid where each entry is only 0 or 1, find the number of corner rectangles.
 * A corner rectangle is 4 distinct 1s on the grid that form an axis-aligned rectangle.
 * Note that only the corners need to have the value 1. Also, all four 1s used must be
distinct.

 Example 1:
 Input: grid =
 [[1, 0, 0, 1, 0],
 [0, 0, 1, 0, 1],
 [0, 0, 0, 1, 0],
 [1, 0, 1, 0, 1]]
 Output: 1
 Explanation: There is only one corner rectangle, with corners grid[1][2], grid[1][4],
grid[3][2], grid[3][4].

 Example 2:
 Input: grid =
 [[1, 1, 1],
 [1, 1, 1],
 [1, 1, 1]]
 Output: 9
 Explanation: There are four 2x2 rectangles, four 2x3 and 3x2 rectangles, and one 3x3
rectangle.

 Example 3:
 Input: grid =
 [[1, 1, 1, 1]]
 Output: 0
 Explanation: Rectangles must have four distinct corners.

 Note:
 The number of rows and columns of grid will each be in the range [1, 200].
 Each grid[i][j] will be either 0 or 1.
 The number of 1s in the grid will be at most 6000.*/
public class _750 {
  public static class Solution1 {
    public int countCornerRectangles(int[][] grid) {
      if (grid == null || grid.length < 2) {
        return 0;
      }
      int m = grid.length;
      int n = grid[0].length;
      int count = 0;
      for (int i = 0; i < m - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
          if (grid[i][j] == 1) {
```

```
            for (int jNext = j + 1; jNext < n; jNext++) {
              if (grid[i][jNext] == 1) {
                for (int iNext = i + 1; iNext < m; iNext++) {
                  if (grid[iNext][j] == 1 && grid[iNext][jNext] == 1) {
                    count++;
                  }
                }
              }
            }
          }
        }
      }
      return count;
    }
  }
}
```

669. `_754.java`

```
package com.fishercoder.solutions;

/**
 * 754. Reach a Number
 *
 * You are standing at position 0 on an infinite number line. There is a goal at position target.
 * On each move, you can either go left or right. During the n-th move (starting from 1), you take n steps.
 * Return the minimum number of steps required to reach the destination.

 Example 1:
 Input: target = 3
 Output: 2
 Explanation:
 On the first move we step from 0 to 1.
 On the second step we step from 1 to 3.

 Example 2:
 Input: target = 2
 Output: 3
 Explanation:
 On the first move we step from 0 to 1.
 On the second move we step  from 1 to -1.
 On the third move we step from -1 to 2.

 Note:
 target will be a non-zero integer in the range [-10^9, 10^9].
 */

public class _754 {
  public static class Solution1 {
    /**Two case:
     * 1. go to the right, and reach the goal exactly.
     * 2. go over the goal by several steps:
     *    by even number, then you can choose one of the steps that went right to go back to the left (the step is half of what you went over)
     *    by odd number, then you keep going until you are over by an even number.*/
    public int reachNumber(int target) {
      int absTarget = Math.abs(target);
      int steps = 1;
      int sum = 0;
      while (sum < absTarget || (sum - absTarget) % 2 == 1) {
        sum += steps;
        steps++;
      }
      return steps - 1;
    }
  }
}
```

670. `_755.java`

```
package com.fishercoder.solutions;

/**
 * 755. Pour Water
 *
 * We are given an elevation map, heights[i] representing the height of the terrain at
that index.
 * The width at each index is 1. After V units of water fall at index K, how much water
is at each index?
 * Water first drops at index K and rests on top of the highest terrain or water at that
index.
 * Then, it flows according to the following rules:

 If the droplet would eventually fall by moving left, then move left.
 Otherwise, if the droplet would eventually fall by moving right, then move right.
 Otherwise, rise at it's current position.

 Here, "eventually fall" means that the droplet will eventually be at a lower level if it
moves in that direction. Also, "level" means the height of the terrain plus any water in
that column.
 We can assume there's infinitely high terrain on the two sides out of bounds of the
array. Also, there could not be partial water being spread out evenly on more than 1 grid
block - each unit of water has to be in exactly one block.

 Example 1:
 Input: heights = [2,1,1,2,1,2,2], V = 4, K = 3
 Output: [2,2,2,3,2,2,2]
 Explanation:
 #       #
 #       #
 ##  # ###
 #########
 0123456    <- index

 The first drop of water lands at index K = 3:

 #       #
 #   w   #
 ##  # ###
 #########
 0123456

 When moving left or right, the water can only move to the same level or a lower level.
 (By level, we mean the total height of the terrain plus any water in that column.)
 Since moving left will eventually make it fall, it moves left.
 (A droplet "made to fall" means go to a lower height than it was at previously.)

 #       #
 #       #
 ## w# ###
 #########
```

```
0123456
```

Since moving left will not make it fall, it stays in place. The next droplet falls:

```
#       #
#   w   #
## w# ###
#########
0123456
```

Since the new droplet moving left will eventually make it fall, it moves left.
Notice that the droplet still preferred to move left,
even though it could move right (and moving right makes it fall quicker.)

```
#       #
#  w    #
## w# ###
#########
0123456
```

```
#       #
#       #
##ww# ###
#########
0123456
```

After those steps, the third droplet falls.
Since moving left would not eventually make it fall, it tries to move right.
Since moving right would eventually make it fall, it moves right.

```
#       #
#   w   #
##ww# ###
#########
0123456
```

```
#       #
#       #
##ww#w###
#########
0123456
```

Finally, the fourth droplet falls.
Since moving left would not eventually make it fall, it tries to move right.
Since moving right would not eventually make it fall, it stays in place:

```
#       #
#   w   #
##ww#w###
#########
0123456
```

The final answer is [2,2,2,3,2,2,2]:

```
 #
 #######
 #######
 0123456


 Example 2:
 Input: heights = [1,2,3,4], V = 2, K = 2
 Output: [2,3,3,4]
 Explanation:
 The last droplet settles at index 1, since moving further left would not cause it to
eventually fall to a lower height.


 Example 3:
 Input: heights = [3,1,3], V = 5, K = 1
 Output: [4,4,4]
 Note:

 heights will have length in [1, 100] and contain integers in [0, 99].
 V will be in range [0, 2000].
 K will be in range [0, heights.length - 1].
 */

public class _755 {
  public static class Solution1 {
    public int[] pourWater(int[] heights, int V, int K) {
      int index;
      while (V > 0) {
        index = K;
        for (int i = K - 1; i >= 0; i--) {
          if (heights[i] > heights[index]) {
            break;
          } else if (heights[i] < heights[index]) {
            index = i;
          }
        }
        if (index != K) {
          heights[index]++;
          V--;
          continue;
        }

        for (int i = K + 1; i < heights.length; i++) {
          if (heights[i] > heights[index]) {
            break;
          } else if (heights[i] < heights[index]) {
            index = i;
          }
        }
        heights[index]++;
        V--;
      }
      return heights;
```

```
    }
  }
}
```

671. `_756.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 756. Pyramid Transition Matrix
 *
 * We are stacking blocks to form a pyramid. Each block has a color which is a one letter
string, like `'Z'`.
 * For every block of color `C` we place not in the bottom row,
 * we are placing it on top of a left block of color `A` and right block of color `B`.
 * We are allowed to place the block there only if `(A, B, C)` is an allowed triple.
 * We start with a bottom row of bottom,
 * represented as a single string. We also start with a list of allowed triples allowed.
 * Each allowed triple is represented as a string of length 3.
 * Return true if we can build the pyramid all the way to the top, otherwise false.

 Example 1:
 Input: bottom = "XYZ", allowed = ["XYD", "YZE", "DEA", "FFF"]
 Output: true
 Explanation:
 We can stack the pyramid like this:
     A
    / \
   D   E
  / \ / \
 X   Y   Z

 This works because ('X', 'Y', 'D'), ('Y', 'Z', 'E'), and ('D', 'E', 'A') are allowed
triples.

 Example 2:
 Input: bottom = "XXYX", allowed = ["XXX", "XXY", "XYX", "XYY", "YXZ"]
 Output: false
 Explanation:
 We can't stack the pyramid to the top.
 Note that there could be allowed triples (A, B, C) and (A, B, D) with C != D.

 Note:
 bottom will be a string with length in range [2, 8].
 allowed will have length in range [0, 200].
 Letters in all strings will be chosen from the set {'A', 'B', 'C', 'D', 'E', 'F', 'G'}.
 */
public class _756 {
  public static class Solution1 {
    /**credit: https://discuss.leetcode.com/topic/116042/java-solution-map-backtracking*/
    public boolean pyramidTransition(String bottom, List<String> allowed) {
      Map<String, List<String>> map = new HashMap<>();
```

```
      for (String s : allowed) {
        String key = s.substring(0, 2);
        if (!map.containsKey(key)) {
          map.put(key, new ArrayList<>());
        }
        map.get(key).add(s.substring(2));
      }

      return helper(bottom, map);
    }

    private boolean helper(String bottom, Map<String, List<String>> map) {
      if (bottom.length() == 1) {
        return true;
      }
      for (int i = 0; i < bottom.length() - 1; i++) {
        if (!map.containsKey(bottom.substring(i, i + 2))) {
          return false;
        }
      }
      List<String> ls = new ArrayList<>();
      getList(bottom, 0, new StringBuilder(), ls, map);
      for (String s : ls) {
        if (helper(s, map)) {
          return true;
        }
      }
      return false;
    }

    private void getList(String bottom, int idx, StringBuilder sb, List<String> ls,
        Map<String, List<String>> map) {
      if (idx == bottom.length() - 1) {
        ls.add(sb.toString());
        return;
      }
      for (String s : map.get(bottom.substring(idx, idx + 2))) {
        sb.append(s);
        getList(bottom, idx + 1, sb, ls, map);
        sb.deleteCharAt(sb.length() - 1);
      }
    }
  }
}
```

672. `_758.java`

```
package com.fishercoder.solutions;

/**
 * 758. Bold Words in String
 *
 * Given a set of keywords words and a string S, make all appearances of all keywords in
S bold. Any letters between <b> and </b> tags become bold.
 * The returned string should use the least number of tags possible, and of course the
tags should form a valid combination.
 * For example, given that words = ["ab", "bc"] and S = "aabcd", we should return
"a<b>abc</b>d".
 * Note that returning "a<b>a<b>b</b>c</b>d" would use more tags, so it is incorrect.

 Note:

 words has length in range [0, 50].
 words[i] has length in range [1, 10].
 S has length in range [0, 500].
 All characters in words[i] and S are lowercase letters.

 */

public class _758 {
  public static class Solution1 {
    /**Interestingly, this problem is exactly the same as 616, using 616's code could get
it AC'ed.*/
    public String boldWords(String[] words, String S) {
      boolean[] shouldBold = new boolean[S.length()];
      for (int i = 0, end = 0; i < S.length(); i++) {
        for (String word : words) {
          if (S.startsWith(word, i)) {
            end = Math.max(end, i + word.length());
          }
        }
        shouldBold[i] = end > i;
      }
      StringBuilder stringBuilder = new StringBuilder();
      for (int i = 0; i < S.length(); i++) {
        if (!shouldBold[i]) {
          stringBuilder.append(S.charAt(i));
          continue;
        }
        int j = i;
        while (j < S.length() && shouldBold[j]) {
          j++;
        }
        stringBuilder.append("<b>" + S.substring(i, j) + "</b>");
        i = j - 1;
      }
      return stringBuilder.toString();
    }
```

```
  }
}
```

673. `_760.java`

```
package com.fishercoder.solutions;

/**
 * 760. Find Anagram Mappings
 *
 * Given two lists Aand B, and B is an anagram of A. B is an anagram of A means B is made by randomizing the order of the elements in A.
 * We want to find an index mapping P, from A to B. A mapping P[i] = j means the ith element in A appears in B at index j.
 * These lists A and B may contain duplicates. If there are multiple answers, output any of them.

 For example, given

 A = [12, 28, 46, 32, 50]
 B = [50, 12, 32, 46, 28]

 We should return
 [1, 4, 3, 2, 0]
 as P[0] = 1 because the 0th element of A appears at B[1], and P[1] = 4 because the 1st element of A appears at B[4], and so on.

 Note:

 A, B have equal lengths in range [1, 100].
 A[i], B[i] are integers in range [0, 10^5].
 */
public class _760 {
  public static class Solution1 {
    public int[] anagramMappings(int[] A, int[] B) {
      int[] result = new int[A.length];
      for (int i = 0; i < A.length; i++) {
        for (int j = 0; j < B.length; j++) {
          if (A[i] == B[j]) {
            result[i] = j;
          }
        }
      }
      return result;
    }
  }
}
```

674. `_762.java`

```
package com.fishercoder.solutions;

/**
 * 762. Prime Number of Set Bits in Binary Representation
 *
 * Given two integers L and R, find the count of numbers in the range [L, R] (inclusive)
having a prime number of set bits in their binary representation.
 * (Recall that the number of set bits an integer has is the number of 1s present when
written in binary.
 * For example, 21 written in binary is 10101 which has 3 set bits. Also, 1 is not a
prime.)

 Example 1:

 Input: L = 6, R = 10
 Output: 4

 Explanation:
 6 -> 110 (2 set bits, 2 is prime)
 7 -> 111 (3 set bits, 3 is prime)
 9 -> 1001 (2 set bits , 2 is prime)
 10->1010 (2 set bits , 2 is prime)

 Example 2:

 Input: L = 10, R = 15
 Output: 5

 Explanation:
 10 -> 1010 (2 set bits, 2 is prime)
 11 -> 1011 (3 set bits, 3 is prime)
 12 -> 1100 (2 set bits, 2 is prime)
 13 -> 1101 (3 set bits, 3 is prime)
 14 -> 1110 (3 set bits, 3 is prime)
 15 -> 1111 (4 set bits, 4 is not prime)

 Note:

 L, R will be integers L <= R in the range [1, 10^6].
 R - L will be at most 10000.
 */

public class _762 {
  public static class Solution1 {
    public int countPrimeSetBits(int L, int R) {
      int count = 0;
      for (int i = L; i <= R; i++) {
        if (hasPrimeNumberSetBits(i)) {
          count++;
        }
      }
```

```
      return count;
    }

    private boolean hasPrimeNumberSetBits(int num) {
      int k = getSetBits(num);
      if (k <= 1) {
        return false;
      }
      for (int i = 2; i * i <= k; i++) {
        if (k % i == 0) {
          return false;
        }
      }
      return true;
    }

    private int getSetBits(int n) {
      int bits = 0;
      while (n != 0) {
        bits++;
        n &= (n - 1);
      }
      return bits;
    }
  }
}
```

675. `_763.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 763. Partition Labels
 *
 * A string S of lowercase letters is given.
 * We want to partition this string into as many parts as possible so that each letter
appears
 * in at most one part, and return a list of integers representing the size of these
parts.

 Example 1:
 Input: S = "ababcbacadefegdehijhklij"
 Output: [9,7,8]
 Explanation:
 The partition is "ababcbaca", "defegde", "hijhklij".
 This is a partition so that each letter appears in at most one part.
 A partition like "ababcbacadefegde", "hijhklij" is incorrect, because it splits S into
less parts.

 Note:
 S will have length in range [1, 500].
 S will consist of lowercase letters ('a' to 'z') only.
 */
public class _763 {

  public static class Solution1 {
    public List<Integer> partitionLabels(String S) {
      List<Integer> result = new ArrayList<>();
      int[] last = new int[26];
      /**This is the key step:
       * we find the last occurrence of each letter and record them in last[]*/
      for (int i = 0; i < S.length(); i++) {
        last[S.charAt(i) - 'a'] = i;
      }
      /**record the last end index of the current substring*/
      int end = 0;
      int start = 0;
      for (int i = 0; i < S.length(); i++) {
        end = Math.max(end, last[S.charAt(i) - 'a']);
        if (end == i) {
          result.add(end - start + 1);
          start = end + 1;
        }
      }
      return result;
    }
  }
```

```
}
```

676. `_764.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 764. Largest Plus Sign
 *
 * In a 2D grid from (0, 0) to (N-1, N-1), every cell contains a 1,
 * except those cells in the given list mines which are 0.
 * What is the largest axis-aligned plus sign of 1s contained in the grid? Return the
order of the plus sign.
 * If there is none, return 0.
 *
 * An "axis-aligned plus sign of 1s of order k" has some center grid[x][y] = 1 along with
4 arms of length k-1 going up, down, left, and right, and made of 1s.
 * This is demonstrated in the diagrams below.
 * Note that there could be 0s or 1s beyond the arms of the plus sign,
 * only the relevant area of the plus sign is checked for 1s.

 Examples of Axis-Aligned Plus Signs of Order k:

 Order 1:
 000
 010
 000

 Order 2:
 00000
 00100
 01110
 00100
 00000

 Order 3:
 0000000
 0001000
 0001000
 0111110
 0001000
 0001000
 0000000


 Example 1:
 Input: N = 5, mines = [[4, 2]]
 Output: 2
 Explanation:
 11111
 11111
 11111
```

```
 11111
 11011
 In the above grid, the largest plus sign can only be order 2.  One of them is marked in
bold.

 Example 2:
 Input: N = 2, mines = []
 Output: 1
 Explanation:
 There is no plus sign of order 2, but there is of order 1.

 Example 3:
 Input: N = 1, mines = [[0, 0]]
 Output: 0
 Explanation:
 There is no plus sign, so return 0.

 Note:
 N will be an integer in the range [1, 500].
 mines will have length at most 5000.
 mines[i] will be length 2 and consist of integers in the range [0, N-1].
 (Additionally, programs submitted in C, C++, or C# will be judged with a slightly
smaller time limit.)

 */

public class _764 {
  public static class Solution1 {
    /**Brute force
     *
     * Time: O(N^3)
     * Space: O(mines.length)*/
    public int orderOfLargestPlusSign(int N, int[][] mines) {
      Set<Integer> banned = new HashSet<>();
      for (int[] mine : mines) {
        banned.add(mine[0] * N + mine[1]);
      }
      int result = 0;
      for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++) {
          int k = 0;
          while (k <= row && row < N - k && k <= col && col < N - k
              && !banned.contains((row - k) * N + col)
              && !banned.contains((row + k) * N + col)
              && !banned.contains(row * N + col - k)
              && !banned.contains(row * N + col + k)) {
            k++;
          }
          result = Math.max(result, k);
        }
      }
      return result;
    }
  }
```

```
  public static class Solution2 {
    /**Dp
     *
     * Time: O(N^2)
     * Space: O(N^2)
     * Credit: https://leetcode.com/articles/largest-plus-sign/*/
    public int orderOfLargestPlusSign(int N, int[][] mines) {
      Set<Integer> banned = new HashSet<>();
      for (int[] mine : mines) {
        banned.add(mine[0] * N + mine[1]);
      }

      int[][] dp = new int[N][N];

      for (int row = 0; row < N; row++) {
        int count = 0;
        for (int col = 0; col < N; col++) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = count;
        }

        count = 0;
        for (int col = N - 1; col >= 0; col--) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = Math.min(dp[row][col], count);
        }
      }

      int result = 0;
      for (int col = 0; col < N; col++) {
        int count = 0;
        for (int row = 0; row < N; row++) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = Math.min(dp[row][col], count);
        }

        count = 0;
        for (int row = N - 1; row >= 0; row--) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = Math.min(dp[row][col], count);
          result = Math.max(result, dp[row][col]);
        }
      }
      return result;
    }
  }
}
```

677. `_765.java`

```
package com.fishercoder.solutions;

/**
 * 765. Couples Holding Hands
 *
 * N couples sit in 2N seats arranged in a row and want to hold hands.
 * We want to know the minimum number of swaps so that every couple is sitting side by
side.
 * A swap consists of choosing any two people, then they stand up and switch seats.
 * The people and seats are represented by an integer from 0 to 2N-1, the couples are
numbered in order,
 * the first couple being (0, 1), the second couple being (2, 3), and so on with the last
couple being (2N-2, 2N-1).
 * The couples' initial seating is given by row[i] being the value of the person who is
initially sitting in the i-th seat.

 Example 1:

 Input: row = [0, 2, 1, 3]
 Output: 1
 Explanation: We only need to swap the second (row[1]) and third (row[2]) person.

 Example 2:

 Input: row = [3, 2, 0, 1]
 Output: 0
 Explanation: All couples are already seated side by side.

 Note:
 len(row) is even and in the range of [4, 60].
 row is guaranteed to be a permutation of 0...len(row)-1.
 */

public class _765 {
  public static class Solution1 {
    public int minSwapsCouples(int[] row) {
      int swaps = 0;
      for (int i = 0; i < row.length - 1; i += 2) {
        int coupleValue = row[i] % 2 == 0 ? row[i] + 1 : row[i] - 1;
        if (row[i + 1] != coupleValue) {
          swaps++;
          int coupleIndex = findIndex(row, coupleValue);
          swap(row, coupleIndex, i + 1);
        }
      }
      return swaps;
    }

    private void swap(int[] row, int i, int j) {
      int tmp = row[i];
      row[i] = row[j];
```

```
      row[j] = tmp;
    }

    private int findIndex(int[] row, int value) {
      for (int i = 0; i < row.length; i++) {
        if (row[i] == value) {
          return i;
        }
      }
      return -1;
    }
  }
}
```

678. `_766.java`

```
package com.fishercoder.solutions;

/**766. Toeplitz Matrix
 *
 * A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same
element.

 Now given an M x N matrix, return True if and only if the matrix is Toeplitz.

 Example 1:

 Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
 Output: True
 Explanation:
 1234
 5123
 9512

 In the above grid, the diagonals are "[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3,
3]", "[4]",
 and in each diagonal all elements are the same, so the answer is True.

 Example 2:

 Input: matrix = [[1,2],[2,2]]
 Output: False
 Explanation:
 The diagonal "[1, 2]" has different elements.

 Note:

 matrix will be a 2D array of integers.
 matrix will have a number of rows and columns in range [1, 20].
 matrix[i][j] will be integers in range [0, 99].

 */
public class _766 {
  public static class Solution1 {
    public boolean isToeplitzMatrix(int[][] matrix) {
      int m = matrix.length;
      int n = matrix[0].length;
      int i = 0;
      int j = 0;
      int sameVal = matrix[i][j];
      while (++i < m && ++j < n) {
        if (matrix[i][j] != sameVal) {
          return false;
        }
      }

      for (i = 1, j = 0; i < m; i++) {
```

```
        int tmpI = i;
        int tmpJ = j;
        sameVal = matrix[i][j];
        while (++tmpI < m && ++tmpJ < n) {
          if (matrix[tmpI][tmpJ] != sameVal) {
            return false;
          }
        }
      }
      for (i = 0, j = 1; j < n; j++) {
        int tmpJ = j;
        int tmpI = i;
        sameVal = matrix[tmpI][tmpJ];
        while (++tmpI < m && ++tmpJ < n) {
          if (matrix[tmpI][tmpJ] != sameVal) {
            return false;
          }
        }
      }
      return true;
    }
  }
}
```

679. `_767.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * 767. Reorganize String
 *
 * Given a string S, check if the letters can be rearranged so that two characters that
are adjacent to each other are not the same.

 If possible, output any possible result.  If not possible, return the empty string.

 Example 1:

 Input: S = "aab"
 Output: "aba"

 Example 2:

 Input: S = "aaab"
 Output: ""

 Note:

 S will consist of lowercase letters and have length in range [1, 500].
 */

public class _767 {
  public static class Solution1 {
    public String reorganizeString(String S) {
      Map<Character, Integer> map = new HashMap<>();
      for (char c : S.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
      }
      int len = S.length();
      for (char c : map.keySet()) {
        if ((len % 2 == 0 && map.get(c) > len / 2) || (len % 2 != 0 && map.get(c) >= len
/ 2 + 2)) {
          return "";
        }
      }
      PriorityQueue<CustChar> queue = new PriorityQueue<>((a, b) -> b.count - a.count);
      for (char c : map.keySet()) {
        queue.offer(new CustChar(c, map.get(c)));
      }

      StringBuilder sb = new StringBuilder();
      while (!queue.isEmpty()) {
        CustChar curr = queue.poll();
```

```
        char c = curr.c;
        if (sb.length() > 0 && sb.charAt(sb.length() - 1) != c) {
          sb.append(c);
          if (curr.count > 1) {
            queue.offer(new CustChar(c, curr.count - 1));
          }
        } else if (sb.length() == 0) {
          sb.append(c);
          if (curr.count > 1) {
            queue.offer(new CustChar(c, curr.count - 1));
          }
        } else if (sb.length() > 0 && sb.charAt(sb.length() - 1) == c &&
!queue.isEmpty()) {
          CustChar next = queue.poll();
          sb.append(next.c);
          if (next.count > 1) {
            queue.offer(new CustChar(next.c, next.count - 1));
          }
          queue.offer(curr);
        }
      }
      return sb.toString();
    }

    class CustChar {
      Character c;
      int count;

      public CustChar(Character c, int count) {
        this.c = c;
        this.count = count;
      }
    }
  }
}
```

680. `_769.java`

```
package com.fishercoder.solutions;

/**
 * 769. Max Chunks To Make Sorted

 Given an array arr that is a permutation of [0, 1, ..., arr.length - 1], we split the
array into some number of "chunks" (partitions), and individually sort each chunk.
 After concatenating them, the result equals the sorted array.

 What is the most number of chunks we could have made?

 Example 1:

 Input: arr = [4,3,2,1,0]
 Output: 1
 Explanation:
 Splitting into two or more chunks will not return the required result.
 For example, splitting into [4, 3], [2, 1, 0] will result in [3, 4, 0, 1, 2], which
isn't sorted.

 Example 2:

 Input: arr = [1,0,2,3,4]
 Output: 4
 Explanation:
 We can split into two chunks, such as [1, 0], [2, 3, 4].
 However, splitting into [1, 0], [2], [3], [4] is the highest number of chunks possible.

 Note:

 arr will have length in range [1, 10].
 arr[i] will be a permutation of [0, 1, ..., arr.length - 1].

 */
public class _769 {
  public static class Solution1 {
    /**credit: https://leetcode.com/problems/max-chunks-to-make-
sorted/discuss/113520/Java-solution-left-max-and-right-min.*/
    public int maxChunksToSorted(int[] arr) {
      int len = arr.length;

      int[] maxOfLeft = new int[len];
      maxOfLeft[0] = arr[0];
      for (int i = 1; i < len; i++) {
        maxOfLeft[i] = Math.max(arr[i], maxOfLeft[i - 1]);
      }

      int[] minOfRight = new int[len];
      minOfRight[len - 1] = arr[len - 1];
      for (int i = len - 2; i >= 0; i--) {
        minOfRight[i] = Math.min(minOfRight[i + 1], arr[i]);
```

```
      }

      int result = 0;
      for (int i = 0; i < len - 1; i++) {
        if (maxOfLeft[i] <= minOfRight[i + 1]) {
          result++;
        }
      }
      return result + 1;
    }
  }

  public static class Solution2 {
    /**credit: https://leetcode.com/articles/max-chunks-to-make-sorted-i/*/
    public int maxChunksToSorted(int[] arr) {
      int ans = 0;
      int max = 0;
      for (int i = 0; i < arr.length; ++i) {
        max = Math.max(max, arr[i]);
        if (max == i) {
          ans++;
        }
      }
      return ans;
    }
  }
}
```

681. `_771.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 771. Jewels and Stones

 You're given strings J representing the types of stones that are jewels, and S
representing the stones you have.
 Each character in S is a type of stone you have.  You want to know how many of the
stones you have are also jewels.
 The letters in J are guaranteed distinct, and all characters in J and S are letters.
 Letters are case sensitive, so "a" is considered a different type of stone from "A".

 Example 1:
 Input: J = "aA", S = "aAAbbbb"
 Output: 3

 Example 2:
 Input: J = "z", S = "ZZ"
 Output: 0

 Note:
 S and J will consist of letters and have length at most 50.
 The characters in J are distinct.
 */

public class _771 {
  public static class Solution1 {
    public int numJewelsInStones(String J, String S) {
      Set<Character> set = new HashSet<>();
      for (char c : J.toCharArray()) {
        set.add(c);
      }
      int count = 0;
      for (char c : S.toCharArray()) {
        if (set.contains(c)) {
          count++;
        }
      }
      return count;
    }
  }
}
```

682. `_776.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 776. Split BST
 *
 * Given a Binary Search Tree (BST) with root node root, and a target value V,
 * split the tree into two subtrees where one subtree has nodes that are all smaller or
equal to the target value,
 * while the other subtree has all nodes that are greater than the target value.
 * It's not necessarily the case that the tree contains a node with value V.
 * Additionally, most of the structure of the original tree should remain.
 * Formally, for any child C with parent P in the original tree,
 * if they are both in the same subtree after the split, then node C should still have
the parent P.
 * You should output the root TreeNode of both subtrees after splitting, in any order.

 Example 1:

 Input: root = [4,2,6,1,3,5,7], V = 2
 Output: [[2,1],[4,3,6,null,null,5,7]]

 Explanation:
 Note that root, output[0], and output[1] are TreeNode objects, not arrays.

 The given tree [4,2,6,1,3,5,7] is represented by the following diagram:

       4
    /     \
   2      6
  /  \   / \
 1   3  5   7

 while the diagrams for the outputs are:

   4
 /   \
3     6      and    2
     / \           /
    5   7         1

 Note:

 The size of the BST will not exceed 50.
 The BST is always valid and each node's value is different.
 */

public class _776 {
  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/119481/recursive-java-solution */
```

```
    public TreeNode[] splitBST(TreeNode root, int V) {
      TreeNode small = new TreeNode(0);
      TreeNode big = new TreeNode(0);
      split(root, V, small, big);
      return new TreeNode[] {small.right, big.left};
    }

    private void split(TreeNode root, int v, TreeNode small, TreeNode big) {
      if (root == null) {
        return;
      }
      if (root.val <= v) {
        small.right = root;
        TreeNode right = root.right;
        root.right = null;
        split(right, v, root, big);
      } else {
        big.left = root;
        TreeNode left = root.left;
        root.left = null;
        split(left, v, small, root);
      }
    }
  }

  public static class Solution2 {
    /** credit: https://leetcode.com/articles/split-bst/ */
    public TreeNode[] splitBST(TreeNode root, int V) {
      if (root == null) {
        return new TreeNode[] {null, null};
      } else if (root.val <= V) {
        TreeNode[] result = splitBST(root.right, V);
        root.right = result[0];
        result[0] = root;
        return result;
      } else {
        TreeNode[] result = splitBST(root.left, V);
        root.left = result[1];
        result[1] = root;
        return result;
      }
    }
  }
}
```

683. `_779.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 779. K-th Symbol in Grammar
 *
 * On the first row, we write a 0. Now in every subsequent row,
 * we look at the previous row and replace each occurrence of 0 with 01, and each
occurrence of 1 with 10.
 * Given row N and index K, return the K-th indexed symbol in row N. (The values of K are
1-indexed.) (1 indexed).

 Examples:
 Input: N = 1, K = 1
 Output: 0

 Input: N = 2, K = 1
 Output: 0

 Input: N = 2, K = 2
 Output: 1

 Input: N = 4, K = 5
 Output: 1

 Explanation:
 row 1: 0
 row 2: 01
 row 3: 0110
 row 4: 01101001

 Note:

 N will be an integer in the range [1, 30].
 K will be an integer in the range [1, 2^(N-1)].
 */

public class _779 {
  public static class Solution1 {
    /**Time: O(2^n)
     * Space: O(2^n)
     * This will result int TLE.*/
    public int kthGrammar(int N, int K) {
      List<List<Integer>> lists = new ArrayList<>();
      lists.add(Arrays.asList(0));
      for (int i = 1; i <= N; i++) {
        List<Integer> curr = new ArrayList<>();
        List<Integer> prev = lists.get(i - 1);
```

```
        for (int j = 0; j < prev.size(); j++) {
          if (prev.get(j) == 0) {
            curr.add(0);
            curr.add(1);
          } else {
            curr.add(1);
            curr.add(0);
          }
        }
        lists.add(curr);
      }
      return lists.get(N).get(K - 1);
    }
  }

  public static class Solution2 {
    /**Time: O(logn)
     * Space: O(1)*/
    public int kthGrammar(int N, int K) {
      return Integer.bitCount(K - 1) % 2;
    }

  }

}
```

684. `_783.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 783. Minimum Distance Between BST Nodes
 *
 * Given a Binary Search Tree (BST) with the root node root, return the minimum
difference between the values of any two different nodes in the tree.

 Example :

 Input: root = [4,2,6,1,3,null,null]
 Output: 1
 Explanation:
 Note that root is a TreeNode object, not an array.

 The given tree [4,2,6,1,3,null,null] is represented by the following diagram:

      4
    /   \
   2     6
  / \
 1  3

 while the minimum difference in this tree is 1, it occurs between node 1 and node 2,
also between node 3 and node 2.

 Note:

 The size of the BST will be between 2 and 100.
 The BST is always valid, each node's value is an integer, and each node's value is
different.
 */

public class _783 {
  public static class Solution1 {
    public int minDiffInBST(TreeNode root) {
      List<Integer> inorder = new ArrayList<>();
      inorder(root, inorder);
      return findMinDiff(inorder);
    }

    private int findMinDiff(List<Integer> inorder) {
      int minDiff = Integer.MAX_VALUE;
      for (int i = 1; i < inorder.size(); i++) {
        minDiff = Math.min(minDiff, inorder.get(i) - inorder.get(i - 1));
      }
      return minDiff;
```

```
    }

    private void inorder(TreeNode root, List<Integer> inorder) {
      if (root == null) {
        return;
      }
      inorder(root.left, inorder);
      inorder.add(root.val);
      inorder(root.right, inorder);
    }
  }
}
```

685. `_784.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 784. Letter Case Permutation
 *
 * Given a string S, we can transform every letter individually to be lowercase or
uppercase to create another string.
 * Return a list of all possible strings we could create.

 Examples:
 Input: S = "a1b2"
 Output: ["a1b2", "a1B2", "A1b2", "A1B2"]

 Input: S = "3z4"
 Output: ["3z4", "3Z4"]

 Input: S = "12345"
 Output: ["12345"]

 Note:

 S will be a string with length at most 12.
 S will consist only of letters or digits.

 */

public class _784 {
  public static class Solution1 {
    public List<String> letterCasePermutation(String S) {
      Set<String> result = new HashSet<>();
      result.add(S);
      for (int i = 0; i < S.length(); i++) {
        if (Character.isAlphabetic(S.charAt(i))) {
          Set<String> newResult = new HashSet<>();
          for (String word : result) {
            if (Character.isUpperCase(word.charAt(i))) {
              StringBuilder sb = new StringBuilder();
              for (int j = 0; j < i; j++) {
                sb.append(word.charAt(j));
              }
              sb.append(Character.toLowerCase(word.charAt(i)));
              for (int j = i + 1; j < word.length(); j++) {
                sb.append(word.charAt(j));
              }
              newResult.add(sb.toString());
            } else {
```

```
              StringBuilder sb = new StringBuilder();
              for (int j = 0; j < i; j++) {
                sb.append(word.charAt(j));
              }
              sb.append(Character.toUpperCase(word.charAt(i)));
              for (int j = i + 1; j < word.length(); j++) {
                sb.append(word.charAt(j));
              }
              newResult.add(sb.toString());
            }
          }
          result.addAll(newResult);
        }
      }
      return new ArrayList<>(result);
    }
  }
}
```

686. `_788.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 788. Rotated Digits
 *
 * X is a good number if after rotating each digit individually by 180 degrees,
 * we get a valid number that is different from X.
 * A number is valid if each digit remains a digit after rotation.
 * 0, 1, and 8 rotate to themselves;
 * 2 and 5 rotate to each other;
 * 6 and 9 rotate to each other,
 * and the rest of the numbers do not rotate to any other number.

 Now given a positive number N, how many numbers X from 1 to N are good?

 Example:
 Input: 10
 Output: 4

 Explanation:
 There are four good numbers in the range [1, 10] : 2, 5, 6, 9.
 Note that 1 and 10 are not good numbers, since they remain unchanged after rotating.

 Note: N  will be in range [1, 10000].
 */
public class _788 {
  public static class Solution1 {
    public int rotatedDigits(int N) {
      int count = 0;
      Map<Character, String> map = new HashMap<>();
      map.put('0', "0");
      map.put('1', "1");
      map.put('8', "8");
      map.put('2', "5");
      map.put('5', "2");
      map.put('6', "9");
      map.put('9', "6");
      for (int i = 1; i <= N; i++) {
        if (isRotatedNumber(i, map)) {
          count++;
        }
      }
      return count;
    }

    private boolean isRotatedNumber(int num, Map<Character, String> map) {
      String originalNum = String.valueOf(num);
      StringBuilder sb = new StringBuilder();
```

```
      for (char c : String.valueOf(num).toCharArray()) {
        if (!map.containsKey(c)) {
          return false;
        } else {
          sb.append(map.get(c));
        }
      }
      return !originalNum.equals(sb.toString());
    }
  }
}
```

687. `_789.java`

```
package com.fishercoder.solutions;

/**
 * 789. Escape The Ghosts
 *
 * You are playing a simplified Pacman game. You start at the point (0, 0), and your
destination is
 * (target[0], target[1]). There are several ghosts on the map, the i-th ghost starts at
(ghosts[i][0], ghosts[i][1]).
 *
 * Each turn, you and all ghosts simultaneously *may* move in one of 4 cardinal
directions: north, east, west, or
 * south, going from the previous point to a new point 1 unit of distance away.
 *
 * You escape if and only if you can reach the target before any ghost reaches you (for
any given moves the ghosts
 * may take.)  If you reach any square (including the target) at the same time as a
ghost, it doesn't count as an
 * escape.
 *
 * Return True if and only if it is possible to escape.
 */

public class _789 {

    public static class Solution {
        public boolean escapeGhosts(int[][] ghosts, int[] target) {
            int[] currPos = {0, 0};
            int selfDist = getDist(currPos, target);

            for (int[] ghost : ghosts) {
                int ghostDist = getDist(ghost, target);
                if (ghostDist <= selfDist) {
                    return false;
                }
            }
            return true;
        }

        private int getDist(int[] p1, int[] p2) {
            return Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);
        }
    }
}
```

688. `_791.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 791. Custom Sort String

 S and T are strings composed of lowercase letters. In S, no letter occurs more than
once.

 S was sorted in some custom order previously. We want to permute the characters of T so
that they match the order that S was sorted. More specifically, if x occurs before y in
S, then x should occur before y in the returned string.

 Return any permutation of T (as a string) that satisfies this property.

 Example :
 Input:
 S = "cba"
 T = "abcd"
 Output: "cbad"
 Explanation:
 "a", "b", "c" appear in S, so the order of "a", "b", "c" should be "c", "b", and "a".
 Since "d" does not appear in S, it can be at any position in T. "dcba", "cdba", "cbda"
are also valid outputs.

 Note:

 S has length at most 26, and no character is repeated in S.
 T has length at most 200.
 S and T consist of lowercase letters only.

 */
public class _791 {
  public static class Solution1 {
    public String customSortString(String S, String T) {
      Map<Character, Integer> map = new HashMap<>();
      for (char c : T.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
      }
      StringBuilder sb = new StringBuilder();
      for (char c : S.toCharArray()) {
        if (map.containsKey(c)) {
          int count = map.get(c);
          while (count-- > 0) {
            sb.append(c);
          }
          map.remove(c);
        }
      }
```

```
      for (char c : map.keySet()) {
        int count = map.get(c);
        while (count-- > 0) {
          sb.append(c);
        }
      }
      return sb.toString();
    }
  }
}
```

689. `_796.java`

```
package com.fishercoder.solutions;

/**
 * 796. Rotate String

 We are given two strings, A and B.

 A shift on A consists of taking string A and moving the leftmost character to the
rightmost position. For example, if A = 'abcde', then it will be 'bcdea' after one shift
on A. Return True if and only if A can become B after some number of shifts on A.

 Example 1:
 Input: A = 'abcde', B = 'cdeab'
 Output: true

 Example 2:
 Input: A = 'abcde', B = 'abced'
 Output: false

 Note:

 A and B will have length at most 100.
 */
public class _796 {
  public static class Solution1 {
    public boolean rotateString(String A, String B) {
      if (A.length() != B.length()) {
        return false;
      }
      for (int i = 0; i < A.length(); i++) {
        if ((A.substring(i) + A.substring(0, i)).equals(B)) {
          return true;
        }
      }
      return false;
    }
  }
}
```

690. `_799.java`

```
package com.fishercoder.solutions;

/**
 * 799. Champagne Tower

 We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row.  Each glass holds one cup (250ml) of champagne.
 Then, some champagne is poured in the first glass at the top.
 When the top most glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it.
 When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on.
 (A glass at the bottom row has it's excess champagne fall on the floor.)
 For example, after one cup of champagne is poured, the top most glass is full.
 After two cups of champagne are poured, the two glasses on the second row are half full.
 After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now.
 After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.
 Now after pouring some non-negative integer cups of champagne, return how full the j-th glass in the i-th row is (both i and j are 0 indexed.)

 Example 1:
 Input: poured = 1, query_glass = 1, query_row = 1
 Output: 0.0
 Explanation: We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

 Example 2:
 Input: poured = 2, query_glass = 1, query_row = 1
 Output: 0.5
 Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

 Note:
 poured will be in the range of [0, 10 ^ 9].
 query_glass and query_row will be in the range of [0, 99].

 *
 */
public class _799 {
  public static class Solution1 {
    public double champagneTower(int poured, int queryRow, int queryGlass) {
      double[][] dp = new double[101][101];
      dp[0][0] = poured;
      for (int row = 0; row <= queryRow; row++) {
        for (int col = 0; col <= row; col++) {
          double quantity = (dp[row][col] - 1.0) / 2.0;
```

```
          if (quantity > 0) {
            dp[row + 1][col] += quantity;
            dp[row + 1][col + 1] += quantity;
          }
        }
      }
      return Math.min(dp[queryRow][queryGlass], 1.0);
    }
  }
}
```

691. `_800.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 800. Similar RGB Color

 In the following, every capital letter represents some hexadecimal digit from 0 to f.

 The red-green-blue color "#AABBCC" can be written as "#ABC" in shorthand.  For example,
"#15c" is shorthand for the color "#1155cc".

 Now, say the similarity between two colors "#ABCDEF" and "#UVWXYZ" is -(AB - UV)^2 - (CD
- WX)^2 - (EF - YZ)^2.

 Given the color "#ABCDEF", return a 7 character color that is most similar to #ABCDEF,
and has a shorthand (that is, it can be represented as some "#XYZ"

 Example 1:
 Input: color = "#09f166"
 Output: "#11ee66"
 Explanation:
 The similarity is -(0x09 - 0x11)^2 -(0xf1 - 0xee)^2 - (0x66 - 0x66)^2 = -64 -9 -0 = -73.
 This is the highest among any shorthand color.

 Note:

 color is a string of length 7.
 color is a valid RGB color: for i > 0, color[i] is a hexadecimal digit from 0 to f
 Any answer which has the same (highest) similarity as the best answer will be accepted.
 All inputs and outputs should use lowercase letters, and the output is 7 characters.

 */
public class _800 {
  public static class Solution1 {
    public String similarRGB(String color) {
      List<String> allShortHandCombinations = computeAllShorthandCombinations();
      int minSimilarity = Integer.MIN_VALUE;
      String result = "";
      for (String candidate : allShortHandCombinations) {
        int similarity = computeSimilarity(candidate, color);
        if (similarity > minSimilarity) {
          result = candidate;
          minSimilarity = similarity;
        }
      }
      return result;
    }
```

```
    private int computeSimilarity(String candidate, String color) {
      return -(Integer.parseInt(candidate.substring(1, 3), 16) - Integer.parseInt(
          color.substring(1, 3), 16)) * (Integer.parseInt(candidate.substring(1, 3), 16)
          - Integer.parseInt(color.substring(1, 3), 16))
          - (Integer.parseInt(candidate.substring(3, 5), 16) - Integer.parseInt(
          color.substring(3, 5), 16)) * (Integer.parseInt(candidate.substring(3, 5), 16)
          - Integer.parseInt(color.substring(3, 5), 16))
          - (Integer.parseInt(candidate.substring(5, 7), 16) - Integer.parseInt(
          color.substring(5, 7), 16)) * (Integer.parseInt(candidate.substring(5, 7), 16)
          - Integer.parseInt(color.substring(5, 7), 16));
    }

    private List<String> computeAllShorthandCombinations() {
      List<String> result = new ArrayList<>();
      List<Character> hexNumber = new ArrayList<>(
          Arrays.asList('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c',
'd', 'e',
              'f'));
      for (int i = 0; i < hexNumber.size(); i++) {
        for (int j = 0; j < hexNumber.size(); j++) {
          for (int k = 0; k < hexNumber.size(); k++) {
            StringBuilder sb = new StringBuilder();
            sb.append("#");
            sb.append(hexNumber.get(i));
            sb.append(hexNumber.get(i));
            sb.append(hexNumber.get(j));
            sb.append(hexNumber.get(j));
            sb.append(hexNumber.get(k));
            sb.append(hexNumber.get(k));
            result.add(sb.toString());
          }
        }
      }
      return result;
    }
  }
}
```

692. `_804.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 804. Unique Morse Code Words

 International Morse Code defines a standard encoding where each letter is mapped to a
series of dots and dashes, as follows: "a" maps to ".-", "b" maps to "-...", "c" maps to
"-.-.", and so on.

 For convenience, the full table for the 26 letters of the English alphabet is given
below:

 [".-","-...","-.-.","-..",".","..-.","--.","....","..",".---","-.-",".-..","--","-.","--
-",".--.","--.-",".-.","...","-","..-","...-",".--","-..-","-.--","--.."]

 Now, given a list of words, each word can be written as a concatenation of the Morse
code of each letter. For example, "cab" can be written as "-.-.-....-", (which is the
concatenation "-.-." + "-..." + ".-"). We'll call such a concatenation, the
transformation of a word.

 Return the number of different transformations among all words we have.

 Example:
 Input: words = ["gin", "zen", "gig", "msg"]
 Output: 2
 Explanation:
 The transformation of each word is:
 "gin" -> "--...-."
 "zen" -> "--...-."
 "gig" -> "--....--."
 "msg" -> "--....--."

 There are 2 different transformations, "--...-." and "--....--.".

 Note:
 The length of words will be at most 100.
 Each words[i] will have length in range [1, 12].
 words[i] will only consist of lowercase letters.
 */
public class _804 {
  public static class Solution1 {
    public int uniqueMorseRepresentations(String[] words) {
      String[] morseCodes =
          new String[] {".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..",
".---",
              "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.", "...", "-", "..-",
"...-",
              ".--", "-..-", "-.--", "--.."};
```

```
      Set<String> concatenation = new HashSet<>();
      StringBuilder sb = new StringBuilder();
      for (String word : words) {
        sb.setLength(0);
        for (char c : word.toCharArray()) {
          sb.append(morseCodes[c - 'a']);
        }
        concatenation.add(sb.toString());
      }
      return concatenation.size();
    }
  }
}
```

693. `_806.java`

```
package com.fishercoder.solutions;

/**
 * 806. Number of Lines To Write String

 We are to write the letters of a given string S, from left to right into lines.
 Each line has maximum width 100 units, and if writing a letter would cause the width of
the line to exceed 100 units, it is written on the next line.
 We are given an array widths, an array where widths[0] is the width of 'a', widths[1] is
the width of 'b', ..., and widths[25] is the width of 'z'.

 Now answer two questions:
 how many lines have at least one character from S,
 and what is the width used by the last such line?
 Return your answer as an integer list of length 2.

 Example :
 Input:
 widths = [10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10]
 S = "abcdefghijklmnopqrstuvwxyz"
 Output: [3, 60]
 Explanation:
 All letters have the same length of 10. To write all 26 letters,
 we need two full lines and one line with 60 units.

 Example :
 Input:
 widths = [4,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10]
 S = "bbbcccdddaaa"
 Output: [2, 4]
 Explanation:
 All letters except 'a' have the same length of 10, and
 "bbbcccdddaa" will cover 9 * 10 + 2 * 4 = 98 units.
 For the last 'a', it is written on the second line because
 there is only 2 units left in the first line.
 So the answer is 2 lines, plus 4 units in the second line.

 Note:
 The length of S will be in the range [1, 1000].
 S will only contain lowercase letters.
 widths is an array of length 26.
 widths[i] will be in the range of [2, 10].

 */
public class _806 {
  public static class Solution1 {
    public int[] numberOfLines(int[] widths, String S) {
      int numOfLines = 1;
      int offsetInCurrentLine = 0;
      for (char c : S.toCharArray()) {
        if (offsetInCurrentLine + widths[c - 'a'] < 100) {
```

```
          offsetInCurrentLine += widths[c - 'a'];
        } else if (offsetInCurrentLine + widths[c - 'a'] == 100) {
          numOfLines++;
          offsetInCurrentLine = 0;
        } else {
          numOfLines++;
          offsetInCurrentLine = widths[c - 'a'];
        }
      }
      return new int[] {numOfLines, offsetInCurrentLine};
    }
  }
}
```

694. `_811.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 811. Subdomain Visit Count

 A website domain like "discuss.leetcode.com" consists of various subdomains.
 At the top level, we have "com", at the next level, we have "leetcode.com", and at the
lowest level, "discuss.leetcode.com".
 When we visit a domain like "discuss.leetcode.com", we will also visit the parent
domains "leetcode.com" and "com" implicitly.
 Now, call a "count-paired domain" to be a count (representing the number of visits this
domain received), followed by a space,
 followed by the address.
 An example of a count-paired domain might be "9001 discuss.leetcode.com".
 We are given a list cpdomains of count-paired domains. We would like a list of count-
paired domains, (in the same format as the input, and in any order),
 that explicitly counts the number of visits to each subdomain.

 Example 1:
 Input:
 ["9001 discuss.leetcode.com"]
 Output:
 ["9001 discuss.leetcode.com", "9001 leetcode.com", "9001 com"]
 Explanation:
 We only have one website domain: "discuss.leetcode.com". As discussed above, the
subdomain "leetcode.com" and "com" will also be visited. So they will all be visited 9001
times.

 Example 2:
 Input:
 ["900 google.mail.com", "50 yahoo.com", "1 intel.mail.com", "5 wiki.org"]
 Output:
 ["901 mail.com","50 yahoo.com","900 google.mail.com","5 wiki.org","5 org","1
intel.mail.com","951 com"]
 Explanation:
 We will visit "google.mail.com" 900 times, "yahoo.com" 50 times, "intel.mail.com" once
and "wiki.org" 5 times. For the subdomains, we will visit "mail.com" 900 + 1 = 901 times,
"com" 900 + 50 + 1 = 951 times, and "org" 5 times.

 Notes:

 The length of cpdomains will not exceed 100.
 The length of each domain name will not exceed 100.
 Each address will have either 1 or 2 "." characters.
 The input count in any count-paired domain will not exceed 10000.
 */
```

```
public class _811 {
  public static class Solution1 {
    public List<String> subdomainVisits(String[] cpdomains) {
      Map<String, Integer> map = new HashMap<>();
      for (String each : cpdomains) {
        String[] pair = each.split(" ");
        String[] subDomains = pair[1].split("\\.");
        StringBuilder sb = new StringBuilder();
        for (int i = subDomains.length - 1; i >= 0; i--) {
          if (i < subDomains.length - 1) {
            sb.insert(0, ".");
          }
          sb.insert(0, subDomains[i]);
          map.put(sb.toString(), map.getOrDefault(sb.toString(), 0) +
Integer.parseInt(pair[0]));
        }
      }
      List<String> result = new ArrayList<>();
      for (String key : map.keySet()) {
        result.add(map.get(key) + " " + key);
      }
      return result;
    }
  }
}
```

695. `_814.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 814. Binary Tree Pruning
 *
 * We are given the head node root of a binary tree, where additionally every node's
value is either a 0 or a 1.
 * Return the same tree where every subtree (of the given tree) not containing a 1 has
been removed.
 * (Recall that the subtree of a node X is X, plus every node that is a descendant of X.)
 *
 * Example 1:
 * Input: [1,null,0,0,1]
 * Output: [1,null,0,null,1]
 * Explanation:
 * Only the red nodes satisfy the property "every subtree not containing a 1".
 * The diagram on the right represents the answer.
 * 1                              1
 *  \                              \
 *   0             ---->            0
 *  / \                              \
 * 0   1                              1
 *
 *
 * Example 2:
 * Input: [1,0,1,0,0,0,1]
 * Output: [1,null,1,null,1]
 *              1                 1
 *             / \                 \
 *            0   1    ---->        1
 *           / \  / \                \
 *          0  0 0  1                1
 *
 *
 * Example 3:
 * Input: [1,1,0,1,1,0,1,0]
 * Output: [1,1,0,1,1,null,1]
 *               1                     1
 *              / \                   / \
 *             1   0     ----->      1  0
 *            / \ / \                / \  \
 *           1  1 0 1               1  1   1
 *          /
 *         0
 *
 *
 * Note:
 *
 * The binary tree will have at most 100 nodes.
```

```
 * The value of each node will only be 0 or 1.*/
public class _814 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/binary-tree-
pruning/discuss/122730/C%2B%2BJavaPython-Self-Explaining-Solution-and-2-lines*/
        public TreeNode pruneTree(TreeNode root) {
            if (root == null) {
                return root;
            }
            root.left = pruneTree(root.left);
            root.right = pruneTree(root.right);
            if (root.left == null && root.right == null && root.val == 0) {
                return null;
            }
            return root;
        }
    }
}
```

696. `_819.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 819. Most Common Word

 Given a paragraph and a list of banned words, return the most frequent word that is not
in the list of banned words.
 It is guaranteed there is at least one word that isn't banned, and that the answer is
unique.
 Words in the list of banned words are given in lowercase, and free of punctuation.
 Words in the paragraph are not case sensitive.  The answer is in lowercase.

 Example:
 Input:
 paragraph = "Bob hit a ball, the hit BALL flew far after it was hit."
 banned = ["hit"]
 Output: "ball"
 Explanation:
 "hit" occurs 3 times, but it is a banned word.
 "ball" occurs twice (and no other word does), so it is the most frequent non-banned word
in the paragraph.
 Note that words in the paragraph are not case sensitive,
 that punctuation is ignored (even if adjacent to words, such as "ball,"),
 and that "hit" isn't the answer even though it occurs more because it is banned.

 Note:

 1 <= paragraph.length <= 1000.
 1 <= banned.length <= 100.
 1 <= banned[i].length <= 10.
 The answer is unique, and written in lowercase (even if its occurrences in paragraph may
have uppercase symbols, and even if it is a proper noun.)
 paragraph only consists of letters, spaces, or the punctuation symbols !?',;.
 Different words in paragraph are always separated by a space.
 There are no hyphens or hyphenated words.
 Words only consist of letters, never apostrophes or other punctuation symbols.
 */
public class _819 {
  public static class Solution1 {
    public String mostCommonWord(String paragraph, String[] banned) {
      Set<String> bannedSet = new HashSet(Arrays.asList(banned));
      String[] words = paragraph.replaceAll("[^a-zA-Z ]",
"").toLowerCase().split("\\s+");
      Map<String, Integer> map = new HashMap<>();
      Arrays.stream(words)
```

```
          .filter(word -> !bannedSet.contains(word))
          .forEach(word -> map.put(word, map.getOrDefault(word, 0) + 1));
      String result = "";
      int freq = 0;
      for (String key : map.keySet()) {
        if (map.get(key) > freq) {
          result = key;
          freq = map.get(key);
        }
      }
      return result;
    }
  }
}
```

697. `_821.java`

```
package com.fishercoder.solutions;

import java.util.TreeSet;

/**
 * 821. Shortest Distance to a Character

  Given a string S and a character C,
  return an array of integers representing the shortest distance from the character C in
the string.

 Example 1:

 Input: S = "loveleetcode", C = 'e'
 Output: [3, 2, 1, 0, 1, 0, 0, 1, 2, 2, 1, 0]

 Note:

 S string length is in [1, 10000].
 C is a single character, and guaranteed to be in string S.
 All letters in S and C are lowercase.

 */
public class _821 {

  public static class Solution1 {
    public int[] shortestToChar(String S, char C) {
      int[] result = new int[S.length()];
      TreeSet<Integer> cIndices = new TreeSet();
      for (int i = 0; i < S.length(); i++) {
        if (S.charAt(i) == C) {
          cIndices.add(i);
        }
      }
      for (int i = 0; i < S.length(); i++) {
        int leftDist = Integer.MAX_VALUE;
        if (cIndices.floor(i) != null) {
          leftDist = Math.abs(cIndices.floor(i) - i);
        }
        int rightDist = Integer.MAX_VALUE;
        if (cIndices.ceiling(i) != null) {
          rightDist = Math.abs(cIndices.ceiling(i) - i);
        }
        result[i] = Math.min(leftDist, rightDist);
      }
      return result;
    }
  }
}
```

698. `_824.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 824. Goat Latin
 *
 * A sentence S is given, composed of words separated by spaces. Each word consists of
lowercase and uppercase letters only.
 *
 * We would like to convert the sentence to "Goat Latin" (a made-up language similar to
Pig Latin.)
 *
 * The rules of Goat Latin are as follows:
 *
 *     If a word begins with a vowel (a, e, i, o, or u), append "ma" to the end of the
word.
 *     For example, the word 'apple' becomes 'applema'.
 *
 *     If a word begins with a consonant (i.e. not a vowel), remove the first letter and
append it to the end, then add "ma".
 *     For example, the word "goat" becomes "oatgma".
 *
 *     Add one letter 'a' to the end of each word per its word index in the sentence,
starting with 1.
 *     For example, the first word gets "a" added to the end, the second word gets "aa"
added to the end and so on.
 *
 * Return the final sentence representing the conversion from S to Goat Latin.
 *
 * Example 1:
 *
 * Input: "I speak Goat Latin"
 * Output: "Imaa peaksmaaa oatGmaaaa atinLmaaaaa"
 *
 * Example 2:
 *
 * Input: "The quick brown fox jumped over the lazy dog"
 * Output: "heTmaa uickqmaaa rownbmaaaa oxfmaaaaa umpedjmaaaaaa overmaaaaaaa hetmaaaaaaaa
azylmaaaaaaaaa ogdmaaaaaaaaaa"
 *
 * Notes:
 *
 *     S contains only uppercase, lowercase and spaces. Exactly one space between each
word.
 *     1 <= S.length <= 100.
 */
public class _824 {
```

```
  public static class Solution1 {
    public String toGoatLatin(String S) {
      StringBuilder sb = new StringBuilder();
      Set<Character> vowels =
          new HashSet(Arrays.asList('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'));
      String[] words = S.split(" ");
      for (int i = 0; i < words.length; i++) {
        if (vowels.contains(words[i].charAt(0))) {
          String newWord = words[i] + "ma";
          int j = i + 1;
          while (j-- > 0) {
            newWord += 'a';
          }
          sb.append(newWord);
          sb.append(" ");
        } else {
          StringBuilder subSb = new StringBuilder(words[i].substring(1));
          subSb.append(words[i].charAt(0));
          subSb.append("ma");
          int j = i + 1;
          while (j-- > 0) {
            subSb.append("a");
          }
          sb.append(subSb.toString());
          sb.append(" ");
        }
      }
      return sb.substring(0, sb.length() - 1);
    }
  }
}
```

699. `_830.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 830. Positions of Large Groups
 *
 * In a string S of lowercase letters, these letters form consecutive groups of the same
character.
 * For example, a string like S = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z" and
"yy".
 * Call a group large if it has 3 or more characters.  We would like the starting and
ending positions of every large group.
 * The final answer should be in lexicographic order.
 *
 * Example 1:
 * Input: "abbxxxxzzy"
 * Output: [[3,6]]
 * Explanation: "xxxx" is the single large group with starting  3 and ending positions 6.
 *
 * Example 2:
 * Input: "abc"
 * Output: []
 * Explanation: We have "a","b" and "c" but no large group.
 *
 * Example 3:
 * Input: "abcdddeeeeaabbbcd"
 * Output: [[3,5],[6,9],[12,14]]
 *
 * Note:  1 <= S.length <= 1000
 */
public class _830 {
  public static class Solution1 {
    public List<List<Integer>> largeGroupPositions(String S) {
      List<List<Integer>> result = new ArrayList<>();
      char[] chars = S.toCharArray();
      for (int i = 0; i < chars.length; ) {
        char first = chars[i];
        int j = i + 1;
        while (j < chars.length && first == chars[j]) {
          j++;
        }
        if ((j - i) >= 3) {
          result.add(Arrays.asList(i, j - 1));
        }
        i = j;
      }
      return result;
    }
```

```
  }
}
```

700. `_832.java`

```
package com.fishercoder.solutions;

/**
 * 832. Flipping an Image
 *
 * Given a binary matrix A, we want to flip the image horizontally, then invert it, and
return the resulting image.
 * To flip an image horizontally means that each row of the image is reversed.  For
example, flipping [1, 1, 0] horizontally results in [0, 1, 1].
 * To invert an image means that each 0 is replaced by 1, and each 1 is replaced by 0.
For example, inverting [0, 1, 1] results in [1, 0, 0].
 *
 * Example 1:
 * Input: [[1,1,0],[1,0,1],[0,0,0]]
 * Output: [[1,0,0],[0,1,0],[1,1,1]]
 * Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
 * Then, invert the image: [[1,0,0],[0,1,0],[1,1,1]]
 *
 * Example 2:
 * Input: [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]
 * Output: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
 * Explanation: First reverse each row: [[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]].
 * Then invert the image: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
 *
 * Notes:
 * 1 <= A.length = A[0].length <= 20
 * 0 <= A[i][j] <= 1
 */
public class _832 {
  public static class Solution1 {
    public int[][] flipAndInvertImage(int[][] A) {
      int m = A.length;
      int n = A[0].length;
      int[][] result = new int[m][n];
      for (int i = 0; i < m; i++) {
        int[] flipped = (reverse(A[i]));
        result[i] = invert(flipped);
      }
      return result;
    }

    private int[] invert(int[] flipped) {
      int[] result = new int[flipped.length];
      for (int i = 0; i < flipped.length; i++) {
        if (flipped[i] == 0) {
          result[i] = 1;
        } else {
          result[i] = 0;
        }
      }
      return result;
```

```
    }

    private int[] reverse(int[] nums) {
      for (int i = 0, j = nums.length - 1; i < j; i++, j--) {
        int tmp = nums[i];
        nums[i] = nums[j];
        nums[j] = tmp;
      }
      return nums;
    }
  }
}
```

701. `_836.java`

```
package com.fishercoder.solutions;

/**
 * 836. Rectangle Overlap
 *
 * A rectangle is represented as a list [x1, y1, x2, y2], where (x1, y1) are the
coordinates of
 * its bottom-left corner, and (x2, y2) are the coordinates of its top-right corner.
 * Two rectangles overlap if the area of their intersection is positive.
 * To be clear, two rectangles that only touch at the corner or edges do not overlap.
 * Given two (axis-aligned) rectangles, return whether they overlap.
 *
 * Example 1:
 * Input: rec1 = [0,0,2,2], rec2 = [1,1,3,3]
 * Output: true
 *
 * Example 2:
 * Input: rec1 = [0,0,1,1], rec2 = [1,0,2,1]
 * Output: false
 *
 * Notes:
 * Both rectangles rec1 and rec2 are lists of 4 integers.
 * All coordinates in rectangles will be between -10^9 and 10^9.
 * */
public class _836 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/rectangle-
overlap/discuss/132340/C%2B%2BJavaPython-1-line-Solution-1D-to-2D*/
        public boolean isRectangleOverlap(int[] rec1, int[] rec2) {
            return rec1[0] < rec2[2] && rec2[0] < rec1[2] && rec1[1] < rec2[3] && rec2[1]
< rec1[3];
        }
    }
}
```

702. `_840.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 840. Magic Squares In Grid
 *
 * A 3 x 3 magic square is a 3 x 3 grid filled with distinct numbers from 1 to 9 such
that each row,
 * column, and both diagonals all have the same sum.
 *
 * Given an grid of integers, how many 3 x 3 "magic square" subgrids are there?  (Each
subgrid is contiguous).
 *
 * Example 1:
 *
 * Input: [[4,3,8,4],
 *         [9,5,1,9],
 *         [2,7,6,2]]
 *
 * Output: 1
 *
 * Explanation:
 * The following subgrid is a 3 x 3 magic square:
 * 438
 * 951
 * 276
 *
 * while this one is not:
 * 384
 * 519
 * 762
 *
 * In total, there is only one magic square inside the given grid.
 * Note:
 *
 * 1 <= grid.length <= 10
 * 1 <= grid[0].length <= 10
 * 0 <= grid[i][j] <= 15
 */
public class _840 {
  public static class Solution1 {
    public int numMagicSquaresInside(int[][] grid) {
      int m = grid.length;
      int n = grid[0].length;
      int count = 0;
      for (int i = 0; i < m - 2; i++) {
        for (int j = 0; j < n - 2; j++) {
          Set<Integer> set = new HashSet<>();
          int sum = grid[i][j] + grid[i][j + 1] + grid[i][j + 2];
```

```
          if (sum == grid[i + 1][j] + grid[i + 1][j + 1] + grid[i + 1][j + 2]
              && sum == grid[i + 2][j] + grid[i + 2][j + 1] + grid[i + 2][j + 2]

              && sum == grid[i][j] + grid[i + 1][j] + grid[i + 2][j]
              && sum == grid[i][j + 1] + grid[i + 1][j + 1] + grid[i + 2][j + 1]
              && sum == grid[i][j + 2] + grid[i + 1][j + 2] + grid[i + 2][j + 2]

              && sum == grid[i][j] + grid[i + 1][j + 1] + grid[i + 2][j + 2]
              && sum == grid[i][j + 2] + grid[i + 1][j + 1] + grid[i + 2][j]

              && set.add(grid[i][j]) && isLegit(grid[i][j])
              && set.add(grid[i][j + 1]) && isLegit(grid[i][j + 1])
              && set.add(grid[i][j + 2]) && isLegit(grid[i][j + 2])
              && set.add(grid[i + 1][j]) && isLegit(grid[i + 1][j])
              && set.add(grid[i + 1][j + 1]) && isLegit(grid[i + 1][j + 1])
              && set.add(grid[i + 1][j + 2]) && isLegit(grid[i + 1][j + 2])
              && set.add(grid[i + 2][j]) && isLegit(grid[i + 2][j])
              && set.add(grid[i + 2][j + 1]) && isLegit(grid[i + 2][j + 1])
              && set.add(grid[i + 2][j + 2]) && isLegit(grid[i + 2][j + 2])
          ) {
            count++;
          }
        }
      }
      return count;
    }

    private boolean isLegit(int num) {
      return num <= 9 && num >= 1;
    }
  }
}
```

703. `_844.java`

```
package com.fishercoder.solutions;

/**
 * 844. Backspace String Compare
 *
 * Given two strings S and T, return if they are equal when both are typed into empty
text editors. # means a backspace character.
 *
 * Example 1:
 * Input: S = "ab#c", T = "ad#c"
 * Output: true
 * Explanation: Both S and T become "ac".
 *
 *  Example 2:
 * Input: S = "ab##", T = "c#d#"
 * Output: true
 * Explanation: Both S and T become "".
 *
 *  Example 3:
 * Input: S = "a##c", T = "#a#c"
 * Output: true
 * Explanation: Both S and T become "c".
 *
 * Example 4:
 * Input: S = "a#c", T = "b"
 * Output: false
 * Explanation: S becomes "c" while T becomes "b".
 * Note:
 *
 * 1 <= S.length <= 200
 * 1 <= T.length <= 200
 * S and T only contain lowercase letters and '#' characters.
 * Follow up:
 *
 * Can you solve it in O(N) time and O(1) space?
 */
public class _844 {
  public static class Solution1 {
    public boolean backspaceCompare(String S, String T) {
      String processedS = process(S);
      String processedT = process(T);
      return processedS.equals(processedT);
    }

    private String process(String str) {
      StringBuilder sb = new StringBuilder();
      for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) == '#') {
          if (sb.length() > 0) {
            sb.deleteCharAt(sb.length() - 1);
          }
```

```
        } else {
          sb.append(str.charAt(i));
        }
      }
      return sb.reverse().toString();
    }
  }
}
```

704. `_849.java`

```
package com.fishercoder.solutions;

/**
 * 849. Maximize Distance to Closest Person
 *
 * In a row of seats, 1 represents a person sitting in that seat, and 0 represents that
the seat is empty.
 * There is at least one empty seat, and at least one person sitting.
 * Alex wants to sit in the seat such that the distance between him and the closest
person to him is maximized.
 * Return that maximum distance to closest person.
 *
 * Example 1:
 * Input: [1,0,0,0,1,0,1]
 * Output: 2
 * Explanation:
 * If Alex sits in the second open seat (seats[2]), then the closest person has distance
2.
 * If Alex sits in any other open seat, the closest person has distance 1.
 * Thus, the maximum distance to the closest person is 2.
 *
 * Example 2:
 * Input: [1,0,0,0]
 * Output: 3
 * Explanation:
 * If Alex sits in the last seat, the closest person is 3 seats away.
 * This is the maximum distance possible, so the answer is 3.
 * Note:
 *
 * 1 <= seats.length <= 20000
 * seats contains only 0s or 1s, at least one 0, and at least one 1.
 * */
public class _849 {
    public static class Solution1 {
        int maxDist = 0;

        public int maxDistToClosest(int[] seats) {
            for (int i = 0; i < seats.length; i++) {
                if (seats[i] == 0) {
                    extend(seats, i);
                }
            }
            return maxDist;
        }

        private void extend(int[] seats, int position) {
            int left = position - 1;
            int right = position + 1;
            int leftMinDistance = 1;
            while (left >= 0) {
                if (seats[left] == 0) {
```

```
                    leftMinDistance++;
                    left--;
                } else {
                    break;
                }
            }
            int rightMinDistance = 1;
            while (right < seats.length) {
                if (seats[right] == 0) {
                    rightMinDistance++;
                    right++;
                } else {
                    break;
                }
            }
            int maxReach = 0;
            if (position == 0) {
                maxReach = rightMinDistance;
            } else if (position == seats.length - 1) {
                maxReach = leftMinDistance;
            } else {
                maxReach = Math.min(leftMinDistance, rightMinDistance);
            }
            maxDist = Math.max(maxDist, maxReach);
        }
    }
}
```

705. `_852.java`

```
package com.fishercoder.solutions;

/**852. Peak Index in a Mountain Array
 *
 * Let's call an array A a mountain if the following properties hold:
 *
 * A.length >= 3
 * There exists some 0 < i < A.length - 1 such that A[0] < A[1] < ... A[i-1] < A[i] >
A[i+1] > ... > A[A.length - 1]
 * Given an array that is definitely a mountain, return any i such that A[0] < A[1] < ...
A[i-1] < A[i] > A[i+1] > ... > A[A.length - 1].
 *
 * Example 1:
 *
 * Input: [0,1,0]
 * Output: 1
 * Example 2:
 *
 * Input: [0,2,1,0]
 * Output: 1
 * Note:
 *
 * 3 <= A.length <= 10000
 * 0 <= A[i] <= 10^6
 * A is a mountain, as defined above.*/
public class _852 {
    public static class Solution1 {
        public int peakIndexInMountainArray(int[] A) {
            for (int i = 1; i < A.length - 1; i++) {
                if (A[i] > A[i + 1]) {
                    return i;
                }
            }
            return -1;
        }
    }
}
```

706. `_859.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 859. Buddy Strings
 *
 * Given two strings A and B of lowercase letters, return true if and only if we can swap
two letters in A so that the result equals B.
 *
 *
 *
 * Example 1:
 *
 * Input: A = "ab", B = "ba"
 * Output: true
 * Example 2:
 *
 * Input: A = "ab", B = "ab"
 * Output: false
 * Example 3:
 *
 * Input: A = "aa", B = "aa"
 * Output: true
 * Example 4:
 *
 * Input: A = "aaaaaaabc", B = "aaaaaaacb"
 * Output: true
 * Example 5:
 *
 * Input: A = "", B = "aa"
 * Output: false
 *
 *
 * Note:
 *
 * 0 <= A.length <= 20000
 * 0 <= B.length <= 20000
 * A and B consist only of lowercase letters.
 */
public class _859 {
  public static class Solution1 {
    public boolean buddyStrings(String A, String B) {
      if (A.length() != B.length()) {
        return false;
      }
      Character c1 = null;
      Character c2 = null;
      Set<Character> set = new HashSet<>();
      int count = 0;
```

```
      for (int i = 0; i < A.length(); i++) {
        if (A.charAt(i) != B.charAt(i)) {
          if (count > 2) {
            return false;
          }
          if (c1 == null) {
            c1 = B.charAt(i);
            c2 = A.charAt(i);
            count++;
            continue;
          }
          if (c1 != A.charAt(i) || c2 != B.charAt(i)) {
            return false;
          }
          count++;
        }
        set.add(A.charAt(i));
      }
      return count == 2 || (count == 0 && set.size() < A.length());
    }
  }
}
```

707. `_860.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 860. Lemonade Change
 *
 * At a lemonade stand, each lemonade costs $5.
 *
 * Customers are standing in a queue to buy from you,
 * and order one at a time (in the order specified by bills).
 * Each customer will only buy one lemonade and pay with either a $5, $10, or $20 bill.
 * You must provide the correct change to each customer,
 * so that the net transaction is that the customer pays $5.
 * Note that you don't have any change in hand at first.
 * Return true if and only if you can provide every customer with correct change.
 *
 * Example 1:
 * Input: [5,5,5,10,20]
 * Output: true
 * Explanation:
 * From the first 3 customers, we collect three $5 bills in order.
 * From the fourth customer, we collect a $10 bill and give back a $5.
 * From the fifth customer, we give a $10 bill and a $5 bill.
 * Since all customers got correct change, we output true.
 *
 * Example 2:
 * Input: [5,5,10]
 * Output: true
 *
 * Example 3:
 * Input: [10,10]
 * Output: false
 *
 * Example 4:
 * Input: [5,5,10,10,20]
 * Output: false
 * Explanation:
 * From the first two customers in order, we collect two $5 bills.
 * For the next two customers in order, we collect a $10 bill and give back a $5 bill.
 * For the last customer, we can't give change of $15 back because we only have two $10
bills.
 * Since not every customer received correct change, the answer is false.
 *
 *
 * Note:
 * 0 <= bills.length <= 10000
 * bills[i] will be either 5, 10, or 20.
 */
public class _860 {
```

```
  public static class Solution1 {
    public boolean lemonadeChange(int[] bills) {
      Map<Integer, Integer> map = new HashMap<>();
      for (int bill : bills) {
        if (bill == 5) {
          map.put(5, map.getOrDefault(5, 0) + 1);
        } else if (bill == 10) {
          if (!map.containsKey(5)) {
            return false;
          } else {
            map.put(5, map.get(5) - 1);
            if (map.get(5) == 0) {
              map.remove(5);
            }
            map.put(10, map.getOrDefault(10, 0) + 1);
          }
        } else {
          if (!map.containsKey(5)) {
            return false;
          } else {
            if (!map.containsKey(10)) {
              if (!map.containsKey(5) || map.get(5) < 3) {
                return false;
              } else {
                map.put(5, map.get(5) - 3);
                if (map.get(5) == 0) {
                  map.remove(5);
                }
              }
            } else {
              if (!map.containsKey(5)) {
                return false;
              } else {
                map.put(5, map.get(5) - 1);
                if (map.get(5) == 0) {
                  map.remove(5);
                }
                map.put(10, map.get(10) - 1);
                if (map.get(10) == 0) {
                  map.remove(10);
                }
              }
            }
          }
          map.put(20, map.getOrDefault(20, 0) + 1);
        }
      }
      return true;
    }
  }
}
```

708. `_867.java`

```
package com.fishercoder.solutions;

/**
 * 867. Transpose Matrix
 *
 * Given a matrix A, return the transpose of A.
 * The transpose of a matrix is the matrix flipped over it's main diagonal, switching the
row and column indices of the matrix.
 *
 * Example 1:
 *
 * Input: [[1,2,3],[4,5,6],[7,8,9]]
 * Output: [[1,4,7],[2,5,8],[3,6,9]]
 * Example 2:
 *
 * Input: [[1,2,3],[4,5,6]]
 * Output: [[1,4],[2,5],[3,6]]
 *
 * Note:
 *
 * 1 <= A.length <= 1000
 * 1 <= A[0].length <= 1000
 * */
public class _867 {
    public static class Solution1 {
        public int[][] transpose(int[][] A) {
            int[][] result = new int[A[0].length][A.length];
            for (int i = 0; i < A.length; i++) {
                for (int j = 0; j < A[0].length; j++) {
                    result[j][i] = A[i][j];
                }
            }
            return result;
        }
    }
}
```

709. `_868.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 868. Binary Gap
 *
 * Given a positive integer N, find and return the longest distance between two
consecutive 1's in the binary representation of N.
 *
 * If there aren't two consecutive 1's, return 0.
 *
 * Example 1:
 * Input: 22
 * Output: 2
 * Explanation:
 * 22 in binary is 0b10110.
 * In the binary representation of 22, there are three ones, and two consecutive pairs of
1's.
 * The first consecutive pair of 1's have distance 2.
 * The second consecutive pair of 1's have distance 1.
 * The answer is the largest of these two distances, which is 2.
 *
 * Example 2:
 * Input: 5
 * Output: 2
 * Explanation:
 * 5 in binary is 0b101.
 *
 * Example 3:
 * Input: 6
 * Output: 1
 * Explanation:
 * 6 in binary is 0b110.
 *
 * Example 4:
 * Input: 8
 * Output: 0
 * Explanation:
 * 8 in binary is 0b1000.
 * There aren't any consecutive pairs of 1's in the binary representation of 8, so we
return 0.
 *
 * Note:
 * 1 <= N <= 10^9
 */
public class _868 {
  public static class Solution1 {
    public int binaryGap(int N) {
      String bin = Integer.toBinaryString(N);
```

```
      List<Integer> oneIndexes = new ArrayList<>();
      for (int i = 0; i < bin.length(); i++) {
        if (bin.charAt(i) == '1') {
          oneIndexes.add(i);
        }
      }
      int maxGap = 0;
      for (int i = 0; i < oneIndexes.size() - 1; i++) {
        maxGap = Math.max(oneIndexes.get(i + 1) - oneIndexes.get(i), maxGap);
      }
      return maxGap;
    }
  }
}
```

710. `_872.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 872. Leaf-Similar Trees
 *
 * Consider all the leaves of a binary tree.  From left to right order, the values of
those leaves form a leaf value sequence.
 *
 *          3
 *         / \
 *        5   1
 *       / \ / \
 *      6  2 9 8
 *        / \
 *       7  4
 *
 * For example, in the given tree above, the leaf value sequence is (6, 7, 4, 9, 8).
 *
 * Two binary trees are considered leaf-similar if their leaf value sequence is the same.
 *
 * Return true if and only if the two given trees with head nodes root1 and root2 are
leaf-similar.
 *
 * Note:
 *
 * Both of the given trees will have between 1 and 100 nodes.
 */
public class _872 {
  public static class Solution1 {
    public boolean leafSimilar(TreeNode root1, TreeNode root2) {
      List<Integer> leaves1 = new ArrayList<>();
      List<Integer> leaves2 = new ArrayList<>();
      preorder(root1, leaves1);
      preorder(root2, leaves2);
      return leaves1.equals(leaves2);
    }

    private void preorder(TreeNode root,
        List<Integer> leaves) {
      if (root == null) {
        return;
      }
      if (root.left == null && root.right == null) {
        leaves.add(root.val);
      }
      preorder(root.left, leaves);
      preorder(root.right, leaves);
```

```
    }
  }
}
```

711. `_876.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 876. Middle of the Linked List
 *
 * Given a non-empty, singly linked list with head node head, return a middle node of
linked list.
 *
 * If there are two middle nodes, return the second middle node.
 *
 * Example 1:
 *
 * Input: [1,2,3,4,5]
 * Output: Node 3 from this list (Serialization: [3,4,5])
 * The returned node has value 3.  (The judge's serialization of this node is [3,4,5]).
 * Note that we returned a ListNode object ans, such that:
 * ans.val = 3, ans.next.val = 4, ans.next.next.val = 5, and ans.next.next.next = NULL.
 *
 *  Example 2:
 *
 * Input: [1,2,3,4,5,6]
 * Output: Node 4 from this list (Serialization: [4,5,6])
 * Since the list has two middle nodes with values 3 and 4, we return the second one.
 *
 * Note:
 *
 * The number of nodes in the given list will be between 1 and 100.
 */
public class _876 {
  public static class Solution1 {
    public ListNode middleNode(ListNode head) {
      ListNode fast = head;
      ListNode slow = head;
      while (fast != null && fast.next != null) {
        fast = fast.next.next;
        slow = slow.next;
      }
      return slow;
    }
  }
}
```

712. `_884.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 884. Uncommon Words from Two Sentences
 *
 * We are given two sentences A and B.  (A sentence is a string of space separated words.
Each word consists only of lowercase letters.)
 *
 * A word is uncommon if it appears exactly once in one of the sentences, and does not
appear in the other sentence.
 *
 * Return a list of all uncommon words.
 *
 * You may return the list in any order.
 *
 *
 *
 * Example 1:
 *
 * Input: A = "this apple is sweet", B = "this apple is sour"
 * Output: ["sweet","sour"]
 * Example 2:
 *
 * Input: A = "apple apple", B = "banana"
 * Output: ["banana"]
 *
 *
 * Note:
 *
 * 0 <= A.length <= 200
 * 0 <= B.length <= 200
 * A and B both contain only spaces and lowercase letters.
 */
public class _884 {
  public static class Solution1 {
    public String[] uncommonFromSentences(String A, String B) {
      Map<String, Integer> map = new HashMap<>();
      for (String word : A.split(" ")) {
        map.put(word, map.getOrDefault(word, 0) + 1);
      }

      for (String word : B.split(" ")) {
        map.put(word, map.getOrDefault(word, 0) + 1);
      }
      List<String> result = new ArrayList<>();
      for (String key : map.keySet()) {
```

```
        if (map.get(key) == 1) {
          result.add(key);
        }
      }
      String[] strs = new String[result.size()];
      result.toArray(strs);
      return strs;
    }
  }
}
```

713. `_885.java`

```
package com.fishercoder.solutions;

/**
 * 885. Spiral Matrix III
 *
 * On a 2 dimensional grid with R rows and C columns, we start at (r0, c0) facing east.
 * Here, the north-west corner of the grid is at the first row and column, and the south-east corner of the grid is at the last row and column.
 * Now, we walk in a clockwise spiral shape to visit every position in this grid.
 * Whenever we would move outside the boundary of the grid, we continue our walk outside the grid (but may return to the grid boundary later.)
 * Eventually, we reach all R * C spaces of the grid.
 * Return a list of coordinates representing the positions of the grid in the order they were visited.
 *
 * Example 1:
 * Input: R = 1, C = 4, r0 = 0, c0 = 0
 * Output: [[0,0],[0,1],[0,2],[0,3]]
 *
 * Example 2:
 * Input: R = 5, C = 6, r0 = 1, c0 = 4
 * Output: [[1,4],[1,5],[2,5],[2,4],[2,3],[1,3],[0,3],[0,4],[0,5],[3,5],[3,4],[3,3],[3,2],[2,2],[1,2],[0,2],[4,5],[4,4],[4,3],[4,2],[4,1],[3,1],[2,1],[1,1],[0,1],[4,0],[3,0],[2,0],[1,0],[0,0]]
 *
 * Note:
 * 1 <= R <= 100
 * 1 <= C <= 100
 * 0 <= r0 < R
 * 0 <= c0 < C
 * */
public class _885 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/spiral-matrix-iii/discuss/158977/Java-15-lines-concise-solution-with-comments*/
        public int[][] spiralMatrixIII(int R, int C, int r0, int c0) {
            int[] directions = new int[]{0, 1, 0, -1, 0};
            int[][] result = new int[R * C][2];
            int i = 0;
            result[i++] = new int[]{r0, c0};
            int len = 0;
            int d = 0;
            while (i < R * C) {
                if (d == 0 || d == 2) {
                    //plus one when moving east or west
                    len++;
                }
                for (int k = 0; k < len; k++) {
                    r0 += directions[d];
                    c0 += directions[d + 1];
                    if (r0 >= 0 && r0 < R && c0 >= 0 && c0 < C) {
```

```
                        result[i++] = new int[]{r0, c0};
                    }
                }
                d = (d + 1) % 4;
            }
            return result;
        }
    }
}
```

714. `_888.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;

/**
 * 888. Fair Candy Swap
 *
 * Alice and Bob have candy bars of different sizes: A[i] is the size of the i-th bar of
candy that Alice has, and B[j]
 * is the size of the j-th bar of candy that Bob has.
 *
 * Since they are friends, they would like to exchange one candy bar each so that after
the exchange, they both have the
 * same total amount of candy.  (The total amount of candy a person has is the sum of the
sizes of candy bars they have.)
 *
 * Return an integer array ans where ans[0] is the size of the candy bar that Alice must
exchange, and ans[1] is the
 * size of the candy bar that Bob must exchange.
 *
 * If there are multiple answers, you may return any one of them.  It is guaranteed an
answer exists.
 *
 *
 *
 * Example 1:
 *
 * Input: A = [1,1], B = [2,2]
 * Output: [1,2]
 * Example 2:
 *
 * Input: A = [1,2], B = [2,3]
 * Output: [1,2]
 * Example 3:
 *
 * Input: A = [2], B = [1,3]
 * Output: [2,3]
 * Example 4:
 *
 * Input: A = [1,2,5], B = [2,4]
 * Output: [5,4]
 */

public class _888 {
    public static class Solution1 {
        public int[] fairCandySwap(int[] A, int[] B) {
            int aSum = 0;
            int bSum = 0;
            int diff = 0;
            int[] ans = new int[2];
            for (int bar : A) {
```

```
                aSum += bar;
            }
            for (int bar : B) {
                bSum += bar;
            }
            diff = aSum - bSum;
            HashSet<Integer> set = new HashSet<>();
            for (int bar : A) {
                set.add(bar);
            }
            for (int bar : B) {
                if (set.contains(bar + diff / 2)) {
                    ans[0] = bar + diff / 2;
                    ans[1] = bar;
                    break;
                }
            }
            return ans;
        }
    }
}
```

715. `_890.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * 890. Find and Replace Pattern
 *
 * You have a list of words and a pattern, and you want to know which words in words
matches the pattern.
 * A word matches the pattern if there exists a permutation of letters p so that after
replacing every letter x in the pattern with p(x), we get the desired word.
 * (Recall that a permutation of letters is a bijection from letters to letters: every
letter maps to another letter, and no two letters map to the same letter.)
 * Return a list of the words in words that match the given pattern.
 * You may return the answer in any order.
 *
 *  Example 1:
 *
 * Input: words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"
 * Output: ["mee","aqq"]
 * Explanation: "mee" matches the pattern because there is a permutation {a -> m, b -> e,
...}.
 * "ccc" does not match the pattern because {a -> c, b -> c, ...} is not a permutation,
 * since a and b map to the same letter.
 *
 * Note:
 * 1 <= words.length <= 50
 * 1 <= pattern.length = words[i].length <= 20
 */
public class _890 {
  public static class Solution1 {
    public List<String> findAndReplacePattern(String[] words, String pattern) {
      List<String> result = new ArrayList<>();
      for (String word : words) {
        Map<Character, Character> map = new HashMap<>();
        Set<Character> set = new HashSet<>();
        boolean match = true;
        for (int i = 0; i < pattern.length(); i++) {
          if (map.containsKey(pattern.charAt(i))) {
            if (word.charAt(i) != map.get(pattern.charAt(i))) {
              match = false;
              break;
            }
          } else {
            map.put(pattern.charAt(i), word.charAt(i));
            if (!set.add(word.charAt(i))) {
```

```
              match = false;
            }
          }
        }
        if (match) {
          result.add(word);
        }
      }
      return result;
    }
  }
}
```

716. `_893.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 893. Groups of Special-Equivalent Strings
 *
 * You are given an array A of strings.
 * A move onto S consists of swapping any two even indexed characters of S, or any two
odd indexed characters of S.
 * Two strings S and T are special-equivalent if after any number of moves onto S, S ==
T.
 * For example, S = "zzxy" and T = "xyzz" are special-equivalent because we may make the
moves "zzxy" -> "xzzy" -> "xyzz"
 * that swap S[0] and S[2], then S[1] and S[3].
 *
 * Now, a group of special-equivalent strings from A is a non-empty subset of A such
that:
 * Every pair of strings in the group are special equivalent, and;
 * The group is the largest size possible (ie., there isn't a string S not in the group
such that S is special equivalent to every string in the group)
 * Return the number of groups of special-equivalent strings from A.
 *
 * Example 1:
 * Input: ["abcd","cdab","cbad","xyzz","zzxy","zzyx"]
 * Output: 3
 * Explanation:
 * One group is ["abcd", "cdab", "cbad"], since they are all pairwise special equivalent,
and none of the other strings are all pairwise special equivalent to these.
 * The other two groups are ["xyzz", "zzxy"] and ["zzyx"].  Note that in particular,
"zzxy" is not special equivalent to "zzyx".
 *
 * Example 2:
 * Input: ["abc","acb","bac","bca","cab","cba"]
 * Output: 3
 *
 * Note:
 * 1 <= A.length <= 1000
 * 1 <= A[i].length <= 20
 * All A[i] have the same length.
 * All A[i] consist of only lowercase letters.
 * */
public class _893 {
    public static class Solution1 {
        /**my original solution, a bit lengthy:
         * generate a unique signaure as key for each equivelant group and sum them up*/
        public int numSpecialEquivGroups(String[] A) {
            return
```

```
Arrays.stream(A).map(this::getCommonKey).collect(Collectors.toSet()).size();
        }

        private String getCommonKey(String word) {
            char[] oddIndexed = new char[word.length() / 2];
            char[] evenIndexed = new char[word.length() / 2 + (word.length() % 2 == 0 ? 0
: 1)];
            char[] array = word.toCharArray();
            for (int i = 0; i < array.length - 1; i += 2) {
                evenIndexed[i / 2] = array[i];
                oddIndexed[i / 2] = array[i + 1];
            }
            if (word.length() % 2 != 0) {
                evenIndexed[evenIndexed.length - 1] = array[array.length - 1];
            }
            Arrays.sort(oddIndexed);
            Arrays.sort(evenIndexed);
            return new StringBuffer().append(new String(evenIndexed)).append(new
String(oddIndexed)).toString();
        }
    }

    public static class Solution2 {
        /**
         * more concise solution: https://leetcode.com/problems/groups-of-special-
equivalent-strings/discuss/163413/Java-Concise-Set-Solution
         * but somehow a bit slower than mine: 12 ms vs 7ms
         * I guess due to the problem constraint and this: "1 <= A[i].length <= 20" to
have made this problem simpler
         */
        public int numSpecialEquivGroups(String[] A) {
            Set<String> set = new HashSet<>();
            for (String str : A) {
                int[] odd = new int[26];
                int[] even = new int[26];
                for (int i = 0; i < str.length(); i++) {
                    if (i % 2 == 0) {
                        even[str.charAt(i) - 'a']++;
                    } else {
                        odd[str.charAt(i) - 'a']++;
                    }
                }
                String key = Arrays.toString(even) + Arrays.toString(odd);
                set.add(key);
            }
            return set.size();
        }
    }
}
```

717. `_896.java`

```
package com.fishercoder.solutions;

/**
 * 896. Monotonic Array
 *
 * An array is monotonic if it is either monotone increasing or monotone decreasing.
 *
 * An array A is monotone increasing if for all i <= j, A[i] <= A[j].
 * An array A is monotone decreasing if for all i <= j, A[i] >= A[j].
 *
 * Return true if and only if the given array A is monotonic.
 *
 * Example 1:
 * Input: [1,2,2,3]
 * Output: true
 *
 * Example 2:
 * Input: [6,5,4,4]
 * Output: true
 *
 * Example 3:
 * Input: [1,3,2]
 * Output: false
 *
 * Example 4:
 * Input: [1,2,4,5]
 * Output: true
 *
 * Example 5:
 * Input: [1,1,1]
 * Output: true
 *
 * Note:
 *
 * 1 <= A.length <= 50000
 * -100000 <= A[i] <= 100000
 */
public class _896 {
    public static class Solution1 {
        public boolean isMonotonic(int[] A) {
            int i = 0;
            for (; i < A.length - 1; i++) {
                if (A[i] <= A[i + 1]) {
                    continue;
                } else {
                    break;
                }
            }
            if (i == A.length - 1) {
                return true;
            }
```

```
            i = 0;
            for (; i < A.length - 1; i++) {
                if (A[i] >= A[i + 1]) {
                    continue;
                } else {
                    break;
                }
            }
            return i == A.length - 1;
        }
    }
}
```

718. `_897.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 897. Increasing Order Search Tree
 *
 * Given a tree, rearrange the tree in in-order so that the leftmost node in the tree is
now the root of the tree, and every node has no left child and only 1 right child.
 *
 * Example 1:
 * Input: [5,3,6,2,4,null,8,1,null,null,null,7,9]
 *
 *        5
 *       / \
 *     3    6
 *    / \    \
 *   2   4    8
 *  /        / \
 * 1        7   9
 *
 * Output: [1,null,2,null,3,null,4,null,5,null,6,null,7,null,8,null,9]
 *
 *  1
 *   \
 *    2
 *     \
 *      3
 *       \
 *        4
 *         \
 *          5
 *           \
 *            6
 *             \
 *              7
 *               \
 *                8
 *                 \
 *                  9
 * Note:
 *
 * The number of nodes in the given tree will be between 1 and 100.
 * Each node will have a unique integer value from 0 to 1000.
 *
 */
public class _897 {
  public static class Solution1 {
    public TreeNode increasingBST(TreeNode root) {
```

```
      List<Integer> inorderList = new ArrayList<>();
      inorderTraversal(root, inorderList);
      return constructTree(inorderList);
    }

    private TreeNode constructTree(List<Integer> inorderList) {
      if (inorderList.isEmpty() || inorderList.size() == 0) {
        return null;
      }
      TreeNode root = new TreeNode(inorderList.get(0));
      TreeNode tmp = root;
      for (int i = 1; i < inorderList.size(); i++) {
        tmp.right = new TreeNode(inorderList.get(i));
        tmp = tmp.right;
      }
      return root;
    }

    private void inorderTraversal(TreeNode root, List<Integer> inorderList) {
      if (root == null) {
        return;
      }
      if (root.left != null) {
        inorderTraversal(root.left, inorderList);
      }
      inorderList.add(root.val);
      if (root.right != null) {
        inorderTraversal(root.right, inorderList);
      }
    }
  }
}
```

719. `_900.java`

```
package com.fishercoder.solutions;

/**
 * 900. RLE Iterator
 *
 * Write an iterator that iterates through a run-length encoded sequence.
 *
 * The iterator is initialized by RLEIterator(int[] A), where A is a run-length encoding
of some sequence.
 * More specifically, for all even i, A[i] tells us the number of times that the non-
negative integer value A[i+1] is repeated in the sequence.
 *
 * The iterator supports one function: next(int n),
 * which exhausts the next n elements (n >= 1) and returns the last element exhausted in
this way.
 * If there is no element left to exhaust, next returns -1 instead.
 *
 * For example, we start with A = [3,8,0,9,2,5], which is a run-length encoding of the
 * sequence [8,8,8,5,5].
 * This is because the sequence can be read as "three eights, zero nines, two fives".
 *
 * Example 1:
 *
 * Input: ["RLEIterator","next","next","next","next"], [[[3,8,0,9,2,5]],[2],[1],[1],[2]]
 * Output: [null,8,8,5,-1]
 * Explanation:
 * RLEIterator is initialized with RLEIterator([3,8,0,9,2,5]).
 * This maps to the sequence [8,8,8,5,5].
 * RLEIterator.next is then called 4 times:
 * .next(2) exhausts 2 terms of the sequence, returning 8.  The remaining sequence is now
[8, 5, 5].
 * .next(1) exhausts 1 term of the sequence, returning 8.  The remaining sequence is now
[5, 5].
 * .next(1) exhausts 1 term of the sequence, returning 5.  The remaining sequence is now
[5].
 * .next(2) exhausts 2 terms, returning -1.  This is because the first term exhausted was
5,
 *  but the second term did not exist.  Since the last term exhausted does not exist, we
return -1.
 *
 * Note:
 *
 * 0 <= A.length <= 1000
 * A.length is an even integer.
 * 0 <= A[i] <= 10^9
 * There are at most 1000 calls to RLEIterator.next(int n) per test case.
 * Each call to RLEIterator.next(int n) will have 1 <= n <= 10^9.
 */
public class _900 {
  public static class Solution1 {
    public static class RLEIterator {
```

```
      int index;
      int[] array;

      public RLEIterator(int[] A) {
        index = 0;
        array = A;
      }

      public int next(int n) {
        int lastElement = -1;
        while (n > 0 && index < array.length) {
          if (array[index] > n) {
            array[index] -= n;
            lastElement = array[index + 1];
            break;
          } else if (array[index] == n) {
            array[index] = 0;
            lastElement = array[index + 1];
            index += 2;
            break;
          } else {
            n -= array[index];
            index += 2;
          }
        }
        return lastElement;
      }

    }
  }
}
```

720. `_905.java`

```
package com.fishercoder.solutions;

/**
 * 905. Sort Array By Parity
 *
 * Given an array A of non-negative integers, return an array consisting of all the even
elements of A, followed by all the odd elements of A.
 *
 * You may return any answer array that satisfies this condition.
 *
 * Example 1:
 *
 * Input: [3,1,2,4]
 * Output: [2,4,3,1]
 * The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.
 *
 * Note:
 *
 * 1 <= A.length <= 5000
 * 0 <= A[i] <= 5000
 * */
public class _905 {
    public static class Solution1 {
        public int[] sortArrayByParity(int[] A) {
            for (int i = 0, j = A.length - 1; i < j; i++) {
                if (A[i] % 2 == 0) {
                    continue;
                } else {
                    while (j > i && A[j] % 2 != 0) {
                        j--;
                    }
                    swap(A, i, j);
                }
            }
            return A;
        }

        private void swap(int[] A, int i, int j) {
            int tmp = A[i];
            A[i] = A[j];
            A[j] = tmp;
        }
    }
}
```

721. `_908.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 908. Smallest Range I
 *
 * Given an array A of integers, for each integer A[i] we may choose any x with -K <= x
<= K, and add x to A[i].
 *
 * After this process, we have some array B.
 *
 * Return the smallest possible difference between the maximum value of B and the minimum
value of B.
 *
 *
 *
 * Example 1:
 *
 * Input: A = [1], K = 0
 * Output: 0
 * Explanation: B = [1]
 * Example 2:
 *
 * Input: A = [0,10], K = 2
 * Output: 6
 * Explanation: B = [2,8]
 * Example 3:
 *
 * Input: A = [1,3,6], K = 3
 * Output: 0
 * Explanation: B = [3,3,3] or B = [4,4,4]
 *
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * 0 <= A[i] <= 10000
 * 0 <= K <= 10000
 */
public class _908 {
    public static class Solution1 {
        public int smallestRangeI(int[] A, int K) {
          Arrays.sort(A);
          int smallestPlus = A[0] + K;
          int biggestMinus = A[A.length - 1] - K;
          int diff = biggestMinus - smallestPlus;
          if (diff > 0) {
              return diff;
          } else {
              return 0;
```

```
            }
        }
    }

    public static class Solution2 {
        public int smallestRangeI(int[] A, int K) {
            int min = A[0];
            int max = A[0];

            for (int k : A) {
              min = Math.min(min, k);
              max = Math.max(max, k);
            }

            return Math.max(max - min - 2 * K, 0);
        }
    }
}
```

722. `_912.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 912. Sort an Array
 *
 * Given an array of integers nums, sort the array in ascending order.
 *
 * Example 1:
 *
 * Input: [5,2,3,1]
 * Output: [1,2,3,5]
 * Example 2:
 *
 * Input: [5,1,1,2,0,0]
 * Output: [0,0,1,1,2,5]
 *
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * -50000 <= A[i] <= 50000
 * */
public class _912 {
    public static class Solution1 {
        public int[] sortArray(int[] nums) {
            Arrays.sort(nums);
            return nums;
        }
    }
}
```

723. `_914.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 914. X of a Kind in a Deck of Cards
 *
 * In a deck of cards, each card has an integer written on it.
 * Return true if and only if you can choose X >= 2 such that it is possible to split the
entire deck into 1 or more groups of cards, where:
 * Each group has exactly X cards.
 * All the cards in each group have the same integer.
 *
 * Example 1:
 * Input: [1,2,3,4,4,3,2,1]
 * Output: true
 * Explanation: Possible partition [1,1],[2,2],[3,3],[4,4]
 *
 * Example 2:
 * Input: [1,1,1,2,2,2,3,3]
 * Output: false
 * Explanation: No possible partition.
 *
 * Example 3:
 * Input: [1]
 * Output: false
 * Explanation: No possible partition.
 *
 * Example 4:
 * Input: [1,1]
 * Output: true
 * Explanation: Possible partition [1,1]
 *
 * Example 5:
 * Input: [1,1,2,2,2,2]
 * Output: true
 * Explanation: Possible partition [1,1],[2,2],[2,2]
 */
public class _914 {
    public static class Solution1 {
        public boolean hasGroupsSizeX(int[] deck) {
            //Size too small for partitions
            if (deck.length < 2) {
                return false;
            }

            //Track repetitions of values in deck array
            Map<Integer, Integer> mapReps = new HashMap<>();
            for (int card : deck) {
                if (!mapReps.containsKey(card)) {
```

```
                    mapReps.put(card, 1);
                } else {
                    mapReps.put(card, mapReps.get(card) + 1);
                }
            }

            //Create array of map values
            int num = 0;
            int[] arrReps = new int[mapReps.size()];
            for (Map.Entry<Integer, Integer> e : mapReps.entrySet()) {
                arrReps[num++] = e.getValue();
            }

            //Find greatest common denominator
            num = arrGCD(arrReps, arrReps.length);

            //If gcd of all repetitions is greater than 1, it's partitionable.
            return num > 1;
        }

        private int gcd(int a, int b) {
            return b == 0 ? a : gcd(b, a % b);
        }

        private int arrGCD(int[] arr, int n) {
            int result = arr[0];
            for (int i = 1; i < n; i++) {
                result = gcd(arr[i], result);
            }

            return result;
        }
    }
}
```

724. `_917.java`

```
package com.fishercoder.solutions;

/**
 * 917. Reverse Only Letters
 *
 * Given a string S, return the "reversed" string where all characters
 * that are not a letter stay in the same place, and all letters reverse their positions.
 *
 * Example 1:
 *
 * Input: "ab-cd"
 * Output: "dc-ba"
 * Example 2:
 *
 * Input: "a-bC-dEf-ghIj"
 * Output: "j-Ih-gfE-dCba"
 * Example 3:
 *
 * Input: "Test1ng-Leet=code-Q!"
 * Output: "Qedo1ct-eeLg=ntse-T!"
 *
 *
 * Note:
 *
 * S.length <= 100
 * 33 <= S[i].ASCIIcode <= 122
 * S doesn't contain \ or "
 * */
public class _917 {
    public static class Solution1 {
        public String reverseOnlyLetters(String S) {
            char[] array = S.toCharArray();
            for (int i = 0, j = array.length - 1; i < j;) {
                if (Character.isLetter(array[i]) && Character.isLetter(array[j])) {
                    char temp = array[i];
                    array[i++] = array[j];
                    array[j--] = temp;
                } else if (Character.isLetter(array[i])) {
                    j--;
                } else if (Character.isLetter(array[j])) {
                    i++;
                } else {
                    i++;
                    j--;
                }
            }
            return new String(array);
        }
    }
}
```

725. `_922.java`

```
package com.fishercoder.solutions;

/**
 * 922. Sort Array By Parity II
 *
 * Given an array A of non-negative integers, half of the integers in A are odd, and half of the integers are even.
 * Sort the array so that whenever A[i] is odd, i is odd; and whenever A[i] is even, i is even.
 * You may return any answer array that satisfies this condition.
 *
 * Example 1:
 *
 * Input: [4,2,5,7]
 * Output: [4,5,2,7]
 * Explanation: [4,7,2,5], [2,5,4,7], [2,7,4,5] would also have been accepted.
 *
 *
 * Note:
 * 2 <= A.length <= 20000
 * A.length % 2 == 0
 * 0 <= A[i] <= 1000
 * */
public class _922 {
    public static class Solution1 {
        public int[] sortArrayByParityII(int[] A) {
            for (int i = 0, j = 1; i < A.length - 1 && j < A.length;) {
                if (A[i] % 2 != 0 && A[j] % 2 == 0) {
                    int tmp = A[i];
                    A[i] = A[j];
                    A[j] = tmp;
                    i += 2;
                    j += 2;
                }
                while (i < A.length - 1 && A[i] % 2 == 0) {
                    i += 2;
                }
                while (j < A.length && A[j] % 2 != 0) {
                    j += 2;
                }
            }
            return A;
        }
    }
}
```

726. `_925.java`

```
package com.fishercoder.solutions;

/**
 * 925. Long Pressed Name
 *
 * Your friend is typing his name into a keyboard.
 * Sometimes, when typing a character c, the key might get long pressed,
 * and the character will be typed 1 or more times.
 *
 * You examine the typed characters of the keyboard.
 * Return True if it is possible that it was your friends name,
 * with some characters (possibly none) being long pressed.
 *
 * Example 1:
 *
 * Input: name = "alex", typed = "aaleex"
 * Output: true
 * Explanation: 'a' and 'e' in 'alex' were long pressed.
 * Example 2:
 *
 * Input: name = "saeed", typed = "ssaaedd"
 * Output: false
 * Explanation: 'e' must have been pressed twice, but it wasn't in the typed output.
 * Example 3:
 *
 * Input: name = "leelee", typed = "lleeelee"
 * Output: true
 * Example 4:
 *
 * Input: name = "laiden", typed = "laiden"
 * Output: true
 * Explanation: It's not necessary to long press any character.
 *
 * Note:
 * name.length <= 1000
 * typed.length <= 1000
 * The characters of name and typed are lowercase letters.
 * */
public class _925 {
    public static class Solution1 {
        public boolean isLongPressedName(String name, String typed) {
            int i = 0;
            for (int j = 0; i < name.length() && j < typed.length(); i++) {
                if (name.charAt(i) != typed.charAt(j)) {
                    return false;
                } else if (i < name.length() - 1 && name.charAt(i) != name.charAt(i + 1))
{
                    j++;
                    while (j < typed.length() && name.charAt(i) == typed.charAt(j)) {
                        j++;
                    }
```

```
                } else {
                    j++;
                }

            }
            return i == name.length();
        }
    }
}
```

727. `_929.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 929. Unique Email Addresses
 *
 * Every email consists of a local name and a domain name, separated by the @ sign.
 * For example, in alice@leetcode.com, alice is the local name, and leetcode.com is the
domain name.
 * Besides lowercase letters, these emails may contain '.'s or '+'s.
 * If you add periods ('.') between some characters in the local name part of an email
address, mail sent there will be forwarded to the same address without dots in the local
name.
 * For example, "alice.z@leetcode.com" and "alicez@leetcode.com" forward to the same
email address.  (Note that this rule does not apply for domain names.)
 * If you add a plus ('+') in the local name, everything after the first plus sign will
be ignored.
 * This allows certain emails to be filtered, for example m.y+name@email.com will be
forwarded to my@email.com.  (Again, this rule does not apply for domain names.)
 * It is possible to use both of these rules at the same time.
 * Given a list of emails, we send one email to each address in the list.  How many
different addresses actually receive mails?
 *
 * Example 1:
 *
 * Input:
["test.email+alex@leetcode.com","test.e.mail+bob.cathy@leetcode.com","testemail+david@lee
.tcode.com"]
 * Output: 2
 *
 * Explanation: "testemail@leetcode.com" and "testemail@lee.tcode.com" actually receive
mails
 *
 * Note:
 *
 * 1 <= emails[i].length <= 100
 * 1 <= emails.length <= 100
 * Each emails[i] contains exactly one '@' character.
 * All local and domain names are non-empty.
 * Local names do not start with a '+' character.
 * */
public class _929 {
    public static class Solution1 {
        public int numUniqueEmails(String[] emails) {
            Map<String, Set<String>> map = new HashMap<>();
            for (String email : emails) {
                String[] parts = email.split("@");
```

```
                if (!map.containsKey(parts[1])) {
                    map.put(parts[1], new HashSet<>());
                }
                String filteredLocalName = parts[0].substring(0, parts[0].indexOf('+'));
                filteredLocalName = filteredLocalName.replace(".", "");
                map.get(parts[1]).add(filteredLocalName);
            }
            return map.keySet().stream().mapToInt(key -> map.get(key).size()).sum();
        }
    }
}
```

728. `_933.java`

```
package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;

/**
 * 933. Number of Recent Calls
 *
 * Write a class RecentCounter to count recent requests.
 * It has only one method: ping(int t), where t represents some time in milliseconds.
 * Return the number of pings that have been made from 3000 milliseconds ago until now.
 * Any ping with time in [t - 3000, t] will count, including the current ping.
 * It is guaranteed that every call to ping uses a strictly larger value of t than
before.
 *
 * Example 1:
 * Input: inputs = ["RecentCounter","ping","ping","ping","ping"], inputs = [[],[1],[100],
[3001],[3002]]
 * Output: [null,1,2,3,3]
 *
 * Note:
 * Each test case will have at most 10000 calls to ping.
 * Each test case will call ping with strictly increasing values of t.
 * Each call to ping will have 1 <= t <= 10^9.
 * */
public class _933 {
    public static class Solution1 {
        public static class RecentCounter {

            Deque<Integer> deque;

            public RecentCounter() {
                deque = new LinkedList<>();
            }

            public int ping(int t) {
                while (!deque.isEmpty() && t - deque.getFirst() > 3000) {
                    deque.removeFirst();
                }
                deque.addLast(t);
                return deque.size();
            }
        }
    }
}
```

729. `_935.java`

```
package com.fishercoder.solutions;

/**
 * 935. Knight Dialer
 *
 * A chess knight can move as indicated in the chess diagram below:
 *
 * https://assets.leetcode.com/uploads/2018/10/12/knight.png
 *
 * |---|---|---|
 * | 1 | 2 | 3 |
 * |---|---|---|
 * | 4 | 5 | 6 |
 * |---|---|---|
 * | 7 | 8 | 9 |
 * |---|---|---|
 * | x | 0 | x |
 * |---|---|---|
 *
 * This time, we place our chess knight on any numbered key of a phone pad (indicated
above), and the knight makes N-1 hops.
 * Each hop must be from one key to another numbered key.
 *
 * Each time it lands on a key (including the initial placement of the knight), it
presses the number of that key, pressing N digits total.
 *
 * How many distinct numbers can you dial in this manner?
 *
 * Since the answer may be large, output the answer modulo 10^9 + 7.
 *
 * Note:
 *  * 1 <= N <= 5000
 */

public class _935 {
    /*
     * The intuition is to calculate the number of ways
     * we can reach a key k after i hops, based on the number of ways we can reach keys x
after i-1 hops
     * s.t. the knight can move from x to k in one move
     * For example,
     * We can reach 6 in 3 ways after 1 hop (1 -> 6, 7 -> 6 or 0 -> 6)
     * We can reach 8 in 2 ways after 1 hop (1 -> 8 or 3 -> 8)
     * Thus, we can reach 1 in 5 ways after 2 hops:
     * . 1. 1 -> 6 -> 1
     * . 2. 7 -> 6 -> 1
     * . 3. 0 -> 6 -> 1
     *   4. 1 -> 8 -> 1
     *   5. 3 -> 8 -> 1
     */
    public static class Solution1 {
```

```
        private static final int MOD = 1000_000_007;

        // whereFromHere[i] is an array of keys that can be reached from the ith digit
        private static final int[][] whereFromHere = {
            {4, 6},    {6, 8},  {7, 9},    {4, 8},  // 0, 1, 2, 3
            {3, 9, 0}, {},      {1, 7, 0},          // 4, 5, 6
            {2, 6},    {1, 3},  {2, 4}              // 7, 8, 9
        };

        public int knightDialer(int N) {
            // a[i] is the number of ways we can end up on the ith digit
            // The initial array is for N = 1, i.e. for 0 hops.
            long[] a = new long[]{1, 1, 1, 1, 1, 1, 1, 1, 1, 1};

            // Simulate N - 1 hops
            for (int i = 0; i < N - 1; ++i) {
                long[] tmp = new long[10];

                // For each digit
                for (int j = 0; j < 10; j++) {
                    // Which other digits can we reach?
                    for (int k : whereFromHere[j]) {
                        tmp[j] = (tmp[j] + a[k]) % MOD;
                    }
                }

                // Sanity checks based on symmetry of the keypad
                assert tmp[1] == tmp[3];
                assert tmp[4] == tmp[6];
                assert tmp[7] == tmp[9];

                a = tmp;
            }

            long ans = 0;
            for (long k : a) {
                ans = (ans + k) % MOD;
            }

            return (int) ans;
        }
    }
}
```

730. `_937.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

/**
 * 937. Reorder Log Files
 *
 * You have an array of logs.  Each log is a space delimited string of words.
 *
 * For each log, the first word in each log is an alphanumeric identifier.  Then, either:
 *
 * Each word after the identifier will consist only of lowercase letters, or;
 * Each word after the identifier will consist only of digits.
 * We will call these two varieties of logs letter-logs and digit-logs.  It is guaranteed
that each log has at least one word after its identifier.
 *
 * Reorder the logs so that all of the letter-logs come before any digit-log.
 * The letter-logs are ordered lexicographically ignoring identifier, with the identifier
used in case of ties.
 * The digit-logs should be put in their original order.
 *
 * Return the final order of the logs.
 *
 * Example 1:
 *
 * Input: ["a1 9 2 3 1","g1 act car","zo4 4 7","ab1 off key dog","a8 act zoo"]
 * Output: ["g1 act car","a8 act zoo","ab1 off key dog","a1 9 2 3 1","zo4 4 7"]
 *
 * Note:
 * 0 <= logs.length <= 100
 * 3 <= logs[i].length <= 100
 * logs[i] is guaranteed to have an identifier, and a word after the identifier.
 */
public class _937 {
  public static class Solution1 {
    public String[] reorderLogFiles(String[] logs) {
      TreeMap<String, String> letterLogMap = new TreeMap<>();
      List<String> digitLogList = new ArrayList<>();
      for (String log : logs) {
        int firstSpaceIndex = log.indexOf(' ');
        String id = log.substring(0, firstSpaceIndex);
        if (Character.isAlphabetic(log.charAt(firstSpaceIndex + 1))) {
          String key = log.substring(firstSpaceIndex + 1) + id;
          letterLogMap.put(key, log);
        } else {
          digitLogList.add(log);
        }
      }
      String[] reorderedLogs = new String[logs.length];
```

```
      int i = 0;
      for (String key : letterLogMap.keySet()) {
        reorderedLogs[i++] = letterLogMap.get(key);
      }
      for (String log : digitLogList) {
        reorderedLogs[i++] = log;
      }
      return reorderedLogs;
    }
  }
}
```

731. `_938.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 938. Range Sum of BST
 *
 * Given the root node of a binary search tree, return the sum of values of all nodes
with value between L and R (inclusive).
 *
 * The binary search tree is guaranteed to have unique values.
 *
 * Example 1:
 *
 * Input: root = [10,5,15,3,7,null,18], L = 7, R = 15
 * Output: 32
 *
 *
 * Example 2:
 *
 * Input: root = [10,5,15,3,7,13,18,1,null,6], L = 6, R = 10
 * Output: 23
 *
 * Note:
 *
 * The number of nodes in the tree is at most 10000.
 * The final answer is guaranteed to be less than 2^31.
 */
public class _938 {
  public static class Solution1 {
    public int rangeSumBST(TreeNode root, int L, int R) {
      if (root == null) {
        return 0;
      }
      List<Integer> list = new ArrayList<>();
      dfs(root, L, R, list);
      return list.stream().mapToInt(num -> num).sum();
    }

    private void dfs(TreeNode root, int l, int r, List<Integer> list) {
      if (root == null) {
        return;
      }
      if (root.val <= r && root.val >= l) {
        list.add(root.val);
      }
      dfs(root.left, l, r, list);
      dfs(root.right, l, r, list);
    }
```

```
  }
}
```

732. `_941.java`

```
package com.fishercoder.solutions;

/**
 * 941. Valid Mountain Array
 *
 * Given an array A of integers, return true if and only if it is a valid mountain array.
 *
 * Recall that A is a mountain array if and only if:
 *
 * A.length >= 3
 * There exists some i with 0 < i < A.length - 1 such that:
 * A[0] < A[1] < ... A[i-1] < A[i]
 * A[i] > A[i+1] > ... > A[B.length - 1]
 *
 *
 * Example 1:
 *
 * Input: [2,1]
 * Output: false
 * Example 2:
 *
 * Input: [3,5,5]
 * Output: false
 * Example 3:
 *
 * Input: [0,3,2,1]
 * Output: true
 *
 *
 * Note:
 *
 * 0 <= A.length <= 10000
 * 0 <= A[i] <= 10000
 * */
public class _941 {
    public static class Solution1 {
        public boolean validMountainArray(int[] A) {
            int i = 0;
            for (; i < A.length - 1; i++) {
                if (A[i] < A[i + 1]) {
                    continue;
                } else if (A[i] == A[i + 1]) {
                    return false;
                } else {
                    break;
                }
            }
            if (i == 0 || i >= A.length - 1) {
                return false;
            }
            for (; i < A.length - 1; i++) {
```

```
                if (A[i] > A[i + 1]) {
                    continue;
                } else {
                    return false;
                }
            }
            return i == A.length - 1;
        }
    }
}
```

733. `_942.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Queue;

/**
 * 942. DI String Match
 *
 * Given a string S that only contains "I" (increase) or "D" (decrease), let N =
S.length.
 *
 * Return any permutation A of [0, 1, ..., N] such that for all i = 0, ..., N-1:
 *
 * If S[i] == "I", then A[i] < A[i+1]
 * If S[i] == "D", then A[i] > A[i+1]
 *
 * Example 1:
 * Input: "IDID"
 * Output: [0,4,1,3,2]
 *
 * Example 2:
 * Input: "III"
 * Output: [0,1,2,3]
 *
 * Example 3:
 * Input: "DDI"
 * Output: [3,2,0,1]
 *
 * Note:
 * 1 <= S.length <= 10000
 * S only contains characters "I" or "D".
 */
public class _942 {
  public static class Solution1 {
    public int[] diStringMatch(String S) {
      Queue<Integer> deque = new ArrayDeque<>();
      for (int i = 0; i <= S.length(); i++) {
        deque.add(i);
      }
      int[] result = new int[S.length() + 1];
      for (int i = 0; i <= S.length(); i++) {
        if (i == S.length()) {
          result[i] = ((ArrayDeque<Integer>) deque).pollLast();
        } else if (S.charAt(i) == 'I') {
          result[i] = ((ArrayDeque<Integer>) deque).pollFirst();
        } else {
          result[i] = ((ArrayDeque<Integer>) deque).pollLast();
        }
      }
      return result;
    }
```

```
  }
}
```

734. `_944.java`

```
package com.fishercoder.solutions;

/**
 * 944. Delete Columns to Make Sorted
 *
 * We are given an array A of N lowercase letter strings, all of the same length.
 * Now, we may choose any set of deletion indices, and for each string, we delete all the characters in those indices.
 * For example, if we have an array A = ["abcdef","uvwxyz"] and deletion indices {0, 2, 3},
 * then the final array after deletions is ["bef", "vyz"], and the remaining columns of A are ["b","v"], ["e","y"],
 * and ["f","z"].  (Formally, the c-th column is [A[0][c], A[1][c], ..., A[A.length-1][c]].)
 *
 * Suppose we chose a set of deletion indices D such that after deletions, each remaining column in A is in non-decreasing sorted order.
 *
 * Return the minimum possible value of D.length.
 *
 * Example 1:
 *
 * Input: ["cba","daf","ghi"]
 * Output: 1
 * Explanation:
 * After choosing D = {1}, each column ["c","d","g"] and ["a","f","i"] are in non-decreasing sorted order.
 * If we chose D = {}, then a column ["b","a","h"] would not be in non-decreasing sorted order.
 * Example 2:
 *
 * Input: ["a","b"]
 * Output: 0
 * Explanation: D = {}
 * Example 3:
 *
 * Input: ["zyx","wvu","tsr"]
 * Output: 3
 * Explanation: D = {0, 1, 2}
 *
 *
 * Note:
 *
 * 1 <= A.length <= 100
 * 1 <= A[i].length <= 1000
 * */
public class _944 {
  public static class Solution1 {
    public int minDeletionSize(String[] A) {
      if (A == null || A.length == 0) {
        return 0;
```

```
      }
      int deletion = 0;
      for (int i = 0; i < A[0].length(); i++) {
        for (int j = 0; j < A.length - 1; j++) {
          if (A[j].charAt(i) > A[j + 1].charAt(i)) {
            deletion++;
            break;
          }
        }
      }
      return deletion;
    }
  }
}
```

735. `_950.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;

/**
 * 950. Reveal Cards In Increasing Order
 *
 * In a deck of cards, every card has a unique integer.  You can order the deck in any
order you want.
 * Initially, all the cards start face down (unrevealed) in one deck.
 * Now, you do the following steps repeatedly, until all cards are revealed:
 * Take the top card of the deck, reveal it, and take it out of the deck.
 * If there are still cards in the deck, put the next top card of the deck at the bottom
of the deck.
 * If there are still unrevealed cards, go back to step 1.  Otherwise, stop.
 * Return an ordering of the deck that would reveal the cards in increasing order.
 *
 * The first entry in the answer is considered to be the top of the deck.
 *
 * Example 1:
 * Input: [17,13,11,2,3,5,7]
 * Output: [2,13,3,11,5,17,7]
 *
 * Explanation:
 * We get the deck in the order [17,13,11,2,3,5,7] (this order doesn't matter), and
reorder it.
 * After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the
deck.
 * We reveal 2, and move 13 to the bottom.  The deck is now [3,11,5,17,7,13].
 * We reveal 3, and move 11 to the bottom.  The deck is now [5,17,7,13,11].
 * We reveal 5, and move 17 to the bottom.  The deck is now [7,13,11,17].
 * We reveal 7, and move 13 to the bottom.  The deck is now [11,17,13].
 * We reveal 11, and move 17 to the bottom.  The deck is now [13,17].
 * We reveal 13, and move 17 to the bottom.  The deck is now [17].
 * We reveal 17.
 * Since all the cards revealed are in increasing order, the answer is correct.
 *
 * Note:
 * 1 <= A.length <= 1000
 * 1 <= A[i] <= 10^6
 * A[i] != A[j] for all i != j
 */
public class _950 {
  public static class Solution1 {
    public int[] deckRevealedIncreasing(int[] deck) {
      Arrays.sort(deck);
      Deque<Integer> deque = new ArrayDeque<>();
      for (int i = deck.length - 1; i >= 0; i--) {
        if (i != deck.length - 1) {
```

```
          deque.addFirst(deque.pollLast());
        }
        deque.addFirst(deck[i]);
      }
      int[] result = new int[deck.length];
      int i = 0;
      while (!deque.isEmpty()) {
        result[i++] = deque.pollFirst();
      }
      return result;
    }
  }
}
```

736. `_951.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 951. Flip Equivalent Binary Trees
 *
 * For a binary tree T, we can define a flip operation as follows: choose any node, and
swap the left and right child subtrees.
 *
 * A binary tree X is flip equivalent to a binary tree Y if and only if we can make X
equal to Y after some number of flip operations.
 *
 * Write a function that determines whether two binary trees are flip equivalent.  The
trees are given by root nodes root1 and root2.
 *
 * Note:
 *  * Each tree will have at most 100 nodes.
 *  * Each value in each tree will be a unique integer in the range [0, 99].
 */

public class _951 {
    public static class Solution1 {
        public boolean flipEquiv(TreeNode root1, TreeNode root2) {
            if (root1 == null && root2 == null) {
                return true;
            }
            if (root1 == null || root2 == null) {
                return false;
            }

            if (root1.val != root2.val) {
                return false;
            }

            return (
                (flipEquiv(root1.left, root2.left) && flipEquiv(root1.right,
root2.right))
                || (flipEquiv(root1.left, root2.right) && flipEquiv(root1.right,
root2.left))
            );
        }
    }
}
```

737. `_953.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 953. Verifying an Alien Dictionary
 *
 * In an alien language, surprisingly they also use english lowercase letters, but
possibly in a different order. The order of the alphabet is some permutation of lowercase
letters.
 *
 * Given a sequence of words written in the alien language, and the order of the
alphabet, return true if and only if the given words are sorted lexicographicaly in this
alien language.
 *
 * Example 1:
 *
 * Input: words = ["hello","leetcode"], order = "hlabcdefgijkmnopqrstuvwxyz"
 * Output: true
 * Explanation: As 'h' comes before 'l' in this language, then the sequence is sorted.
 *
 * Example 2:
 *
 * Input: words = ["word","world","row"], order = "worldabcefghijkmnpqstuvxyz"
 * Output: false
 * Explanation: As 'd' comes after 'l' in this language, then words[0] > words[1], hence
the sequence is unsorted.
 *
 * Example 3:
 *
 * Input: words = ["apple","app"], order = "abcdefghijklmnopqrstuvwxyz"
 * Output: false
 * Explanation: The first three characters "app" match, and the second string is shorter
(in size.) According to lexicographical rules "apple" > "app", because 'l' > 'ø', where
'ø' is defined as the blank character which is less than any other character (More info).
 *
 *
 * Note:
 * 1 <= words.length <= 100
 * 1 <= words[i].length <= 20
 * order.length == 26
 * All characters in words[i] and order are english lowercase letters.
 */
public class _953 {
  public static class Solution1 {
    public boolean isAlienSorted(String[] words, String order) {
      if (words.length == 1) {
        return true;
      }
```

```
      Map<Character, Integer> map = new HashMap<>();
      for (int i = 0; i < order.length(); i++) {
        map.put(order.charAt(i), i);
      }

      for (int i = 0; i < words.length - 1; i++) {
        String firstWord = words[i];
        String secondWord = words[i + 1];
        if (!sorted(firstWord, secondWord, map)) {
          return false;
        }
      }
      return true;
    }

    private boolean sorted(String firstWord, String secondWord, Map<Character, Integer>
map) {
      for (int i = 0; i < Math.min(firstWord.length(), secondWord.length()); i++) {
        if (firstWord.charAt(i) == secondWord.charAt(i)) {
          continue;
        } else {
          if (map.get(firstWord.charAt(i)) > map.get(secondWord.charAt(i))) {
            return false;
          } else {
            return true;
          }
        }
      }
      return firstWord.length() <= secondWord.length();
    }
  }
}
```

738. `_954.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 954. Array of Doubled Pairs
 *
 * Given an array of integers A with even length, return true if and only if it is
possible to reorder it
 * such that A[2 * i + 1] = 2 * A[2 * i] for every 0 <= i < len(A) / 2.
 *
 * Example 1:
 * Input: [3,1,3,6]
 * Output: false
 *
 * Example 2:
 * Input: [2,1,2,6]
 * Output: false
 *
 * Example 3:
 * Input: [4,-2,2,-4]
 * Output: true
 * Explanation: We can take two groups, [-2,-4] and [2,4] to form [-2,-4,2,4] or
[2,4,-2,-4].
 *
 * Example 4:
 * Input: [1,2,4,16,8,4]
 * Output: false
 *
 * Note:
 * 0 <= A.length <= 30000
 * A.length is even
 * -100000 <= A[i] <= 100000
 * */
public class _954 {
    public static class Solution1 {
        public boolean canReorderDoubled(int[] A) {
            for (int i = 0; i < A.length; i++) {
                if (A[i] < 0) {
                    A[i] = -A[i];
                }
            }
            Arrays.sort(A);
            Map<Integer, Integer> map = new HashMap<>();
            for (int num : A) {
                map.put(num, map.getOrDefault(num, 0) + 1);
            }
            for (int num : A) {
                if (num != 0) {
```

```
                    if (map.get(num) < 0) {
                        return false;
                    } else if (map.get(num) == 0) {
                        continue;
                    } else {
                        int count = map.get(num);
                        map.put(num, 0);
                        int doubleNum = num * 2;
                        if (!map.containsKey(doubleNum)) {
                            return false;
                        } else {
                            map.put(doubleNum, map.get(doubleNum) - count);
                        }
                    }
                } else if (map.get(num) % 2 != 0) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

739. `_961.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 961. N-Repeated Element in Size 2N Array
 *
 * In a array A of size 2N, there are N+1 unique elements, and exactly one of these
elements is repeated N times.
 *
 * Return the element repeated N times.
 *
 * Example 1:
 *
 * Input: [1,2,3,3]
 * Output: 3
 * Example 2:
 *
 * Input: [2,1,2,5,3,2]
 * Output: 2
 * Example 3:
 *
 * Input: [5,1,5,2,5,3,5,4]
 * Output: 5
 *
 *
 * Note:
 *
 * 4 <= A.length <= 10000
 * 0 <= A[i] < 10000
 * A.length is even
 * */
public class _961 {
    public static class Solution1 {
        public int repeatedNTimes(int[] A) {
            Set<Integer> set = new HashSet<>();
            for (int num : A) {
                if (!set.add(num)) {
                    return num;
                }
            }
            return -1;
        }
    }
}
```

740. `_965.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 965. Univalued Binary Tree
 *
 * A binary tree is univalued if every node in the tree has the same value.
 *
 * Return true if and only if the given tree is univalued.
 *
 * Example 1:
 *
 *         1
 *        / \
 *       1   1
 *      / \   \
 *     1  1   1
 *
 * Input: [1,1,1,1,1,null,1]
 * Output: true
 *
 *
 * Example 2:
 *        2
 *       / \
 *      2   2
 *     / \
 *    5  2
 *
 * Input: [2,2,2,5,2]
 * Output: false
 *
 *
 * Note:
 *
 * The number of nodes in the given tree will be in the range [1, 100].
 * Each node's value will be an integer in the range [0, 99].
 */
public class _965 {
  public static class Solution1 {
    public boolean isUnivalTree(TreeNode root) {
      if (root == null) {
        return true;
      }
      return dfs(root, root.val);
    }

    private boolean dfs(TreeNode root, int value) {
      if (root == null) {
        return true;
```

```
      }
      if (root.val != value) {
        return false;
      }
      return dfs(root.left, value) && dfs(root.right, value);
    }
  }
}
```

741. `_966.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 966. Vowel Spellchecker
 *
 * Given a wordlist, we want to implement a spellchecker that converts a query word into
a correct word.
 *
 * For a given query word, the spell checker handles two categories of spelling mistakes:
 *
 * Capitalization: If the query matches a word in the wordlist (case-insensitive), then
the query word is returned with
 * the same case as the case in the wordlist.
 * Example: wordlist = ["yellow"], query = "YellOw": correct = "yellow"
 * Example: wordlist = ["Yellow"], query = "yellow": correct = "Yellow"
 * Example: wordlist = ["yellow"], query = "yellow": correct = "yellow"
 *
 * Vowel Errors: If after replacing the vowels ('a', 'e', 'i', 'o', 'u') of the query
word with any vowel individually,
 * it matches a word in the wordlist (case-insensitive), then the query word is returned
with the same case as the
 * match in the wordlist.
 * Example: wordlist = ["YellOw"], query = "yollow": correct = "YellOw"
 * Example: wordlist = ["YellOw"], query = "yeellow": correct = "" (no match)
 * Example: wordlist = ["YellOw"], query = "yllw": correct = "" (no match)
 *
 * In addition, the spell checker operates under the following precedence rules:
 *
 * When the query exactly matches a word in the wordlist (case-sensitive), you should
return the same word back.
 * When the query matches a word up to capitlization, you should return the first such
match in the wordlist.
 * When the query matches a word up to vowel errors, you should return the first such
match in the wordlist.
 * If the query has no matches in the wordlist, you should return the empty string.
 *
 * Given some queries, return a list of words answer, where answer[i] is the correct word
for query = queries[i].
 *
 * Example 1:
 *
 * Input: wordlist = ["KiTe","kite","hare","Hare"], queries =
["kite","Kite","KiTe","Hare","HARE","Hear","hear","keti","keet","keto"]
 * Output: ["kite","KiTe","KiTe","Hare","hare","","","KiTe","","KiTe"]
 *
 * Note:
```

```
 *
 * 1 <= wordlist.length <= 5000
 * 1 <= queries.length <= 5000
 * 1 <= wordlist[i].length <= 7
 * 1 <= queries[i].length <= 7
 * All strings in wordlist and queries consist only of english letters.
 *
 * */

public class _966 {

    public static class Solution1 {
        public String[] spellchecker(String[] wordlist, String[] queries) {
            Map<String, String> caseMap = new HashMap<>();
            Set<String> set = new HashSet<>();

            // Case Part Mapping
            for (String word : wordlist) {
                if (!caseMap.containsKey(word.toLowerCase())) {
                    caseMap.put(word.toLowerCase(), word);
                }

                set.add(word);
            }

            // Vowel Part Mapping
            Map<String, String> vowelMap = new HashMap<>();
            for (String word : wordlist) {
                String genericVal = makeGenericVowel(word);
                if (!vowelMap.containsKey(genericVal)) {
                    vowelMap.put(genericVal, word);
                }
            }

            String[] ans = new String[queries.length];

            for (int i = 0; i < queries.length; i++) {
                if (set.contains(queries[i])) {
                    ans[i] = queries[i];
                } else if (caseMap.containsKey(queries[i].toLowerCase())) {
                    ans[i] = caseMap.get(queries[i].toLowerCase());
                } else if (vowelMap.containsKey(makeGenericVowel(queries[i]))) {
                    ans[i] = vowelMap.get(makeGenericVowel(queries[i]));
                } else {
                    ans[i] = "";
                }
            }

            return ans;
        }

        private String makeGenericVowel(String s) {
            String vowel = "aeiou";
            char[] ch = s.toLowerCase().toCharArray();
```

```
            for (int i = 0; i < ch.length; i++) {
                if (vowel.indexOf(ch[i]) != -1) {
                    ch[i] = '#';
                }
            }

            return String.valueOf(ch);
        }
    }
}
```

742. `_970.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 970. Powerful Integers
 *
 * Given two non-negative integers x and y, an integer is powerful if it is equal to x^i
+ y^j for some integers i >= 0 and j >= 0.
 *
 * Return a list of all powerful integers that have value less than or equal to bound.
 *
 * You may return the answer in any order.  In your answer, each value should occur at
most once.
 *
 * Example 1:
 *
 * Input: x = 2, y = 3, bound = 10
 * Output: [2,3,4,5,7,9,10]
 * Explanation:
 * 2 = 2^0 + 3^0
 * 3 = 2^1 + 3^0
 * 4 = 2^0 + 3^1
 * 5 = 2^1 + 3^1
 * 7 = 2^2 + 3^1
 * 9 = 2^3 + 3^0
 * 10 = 2^0 + 3^2
 *
 * Example 2:
 *
 * Input: x = 3, y = 5, bound = 15
 * Output: [2,4,6,8,10,14]
 *
 *
 * Note:
 * 1 <= x <= 100
 * 1 <= y <= 100
 * 0 <= bound <= 10^6
 */
public class _970 {
  public static class Solution1 {
    /**This approach results in Time Limit Exceeded since it's apparently doing
     * redundant checks.*/
    public List<Integer> powerfulIntegers(int x, int y, int bound) {
      Set<Integer> result = new HashSet<>();
      int small = x;
      int big = y;
```

```
      if (x > y) {
        small = y;
        big = x;
      }
      int maxPower = bound / small;
      for (int i = 0; i <= maxPower + 1; i++) {
        for (int j = 0; j <= maxPower + 1; j++) {
          int sum = (int) (Math.pow(small, i) + Math.pow(big, j));
          if (sum <= bound) {
            result.add(sum);
          }
        }
      }
      List<Integer> list = new ArrayList<>(result);
      Collections.sort(list);
      return list;
    }
  }

  public static class Solution2 {
    /** credit: https://leetcode.com/problems/powerful-
integers/discuss/214212/JavaC%2B%2BPython-Brute-Force */
    public List<Integer> powerfulIntegers(int x, int y, int bound) {
      Set<Integer> result = new HashSet<>();
      for (int i = 1; i < bound; i *= x > 1 ? x : bound + 1) {
        for (int j = 1; i + j <= bound; j *= y > 1 ? y : bound + 1) {
          result.add(i + j);
        }
      }
      return new ArrayList<>(result);
    }
  }
}
```

743. `_973.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 973. K Closest Points to Origin
 *
 * We have a list of points on the plane.  Find the K closest points to the origin (0,
0).
 *
 * (Here, the distance between two points on a plane is the Euclidean distance.)
 *
 * You may return the answer in any order.  The answer is guaranteed to be unique (except
for the order that it is in.)
 *
 *
 *
 * Example 1:
 *
 * Input: points = [[1,3],[-2,2]], K = 1
 * Output: [[-2,2]]
 * Explanation:
 * The distance between (1, 3) and the origin is sqrt(10).
 * The distance between (-2, 2) and the origin is sqrt(8).
 * Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
 * We only want the closest K = 1 points from the origin, so the answer is just [[-2,2]].
 * Example 2:
 *
 * Input: points = [[3,3],[5,-1],[-2,4]], K = 2
 * Output: [[3,3],[-2,4]]
 * (The answer [[-2,4],[3,3]] would also be accepted.)
 *
 *
 * Note:
 *
 * 1 <= K <= points.length <= 10000
 * -10000 < points[i][0] < 10000
 * -10000 < points[i][1] < 10000
 * */
public class _973 {

    public static class Solution1 {
        public int[][] kClosest(int[][] points, int K) {
            int[][] ans = new int[K][2];

            PriorityQueue<int[]> pq = new PriorityQueue<>((o1, o2) -> {
                double dist1 = getDistance(o1);
                double dist2 = getDistance(o2);

                if (dist1 > dist2) {
                    return 1;
```

```
                } else if (dist1 < dist2) {
                    return -1;
                } else {
                    return 0;
                }
            });

            for (int[] point : points) {
                pq.add(point);
            }

            for (int i = 0; i < K; i++) {
                ans[i] = pq.poll();
            }

            return ans;
        }

        private double getDistance(int[] point) {
            return Math.sqrt(Math.pow(point[0], 2) + Math.pow(point[1], 2));
        }
    }
}
```

744. `_974.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 974. Subarray Sums Divisible by K
 *
 * Given an array A of integers, return the number of (contiguous, non-empty) subarrays
that have a sum
 * divisible by K.
 * */

public class _974 {
    public static class Solution1 {
        public int subarraysDivByK(int[] A, int K) {
            int count = 0;
            int sum = 0;
            Map<Integer, Integer> map = new HashMap<>();
            map.put(0, 1);

            for (int i = 0; i < A.length; i++) {
                sum = (sum + A[i]) % K;
                if (sum < 0) {
                    sum += K;
                }

                count += map.getOrDefault(sum, 0);
                map.put(sum, map.getOrDefault(sum, 0) + 1);
            }

            return count;
        }
    }
}
```

745. `_976.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 976. Largest Perimeter Triangle
 *
 * Given an array A of positive lengths, return the largest perimeter of a triangle with
non-zero area, formed from 3
 * of these lengths.
 *
 * If it is impossible to form any triangle of non-zero area, return 0.
 * */

public class _976 {

    public static class Solution1 {
        public int largestPerimeter(int[] A) {
            Arrays.sort(A);
            int n = A.length;

            for (int i = n - 1; i > 1; i--) {
                if (A[i] < A[i - 1] + A[i - 2]) {
                    return A[i] + A[i - 1] + A[i - 2];
                }
            }

            return 0;
        }
    }
}
```

746. `_977.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 977. Squares of a Sorted Array
 *
 * Given an array of integers A sorted in non-decreasing order, return an array of the
squares of each number, also in sorted non-decreasing order.
 *
 * Example 1:
 * Input: [-4,-1,0,3,10]
 * Output: [0,1,9,16,100]
 *
 * Example 2:
 * Input: [-7,-3,2,3,11]
 * Output: [4,9,9,49,121]
 *
 *
 * Note:
 * 1 <= A.length <= 10000
 * -10000 <= A[i] <= 10000
 * A is sorted in non-decreasing order.
 */
public class _977 {
  public static class Solution1 {
    public int[] sortedSquares(int[] A) {
      int[] result = new int[A.length];
      for (int i = 0; i < A.length; i++) {
        result[i] = (int) Math.pow(A[i], 2);
      }
      Arrays.sort(result);
      return result;
    }
  }
}
```

747. `_979.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 979. Distribute Coins in Binary Tree
 *
 * Given the root of a binary tree with N nodes, each node in the tree has node.val
coins, and there are N coins total.
 * In one move, we may choose two adjacent nodes and move one coin from one node to
another.  (The move may be from parent to child, or from child to parent.)
 * Return the number of moves required to make every node have exactly one coin.
 *
 * Example 1:
 *
 *      3
 *     / \
 *    0  0
 *
 * Input: [3,0,0]
 * Output: 2
 * Explanation: From the root of the tree, we move one coin to its left child, and one
coin to its right child.
 *
 * Example 2:
 *
 *    0
 *   / \
 *  3  0
 *
 * Input: [0,3,0]
 * Output: 3
 * Explanation: From the left child of the root, we move two coins to the root [taking
two moves].  Then, we move one coin from the root of the tree to the right child.
 *
 * Example 3:
 *
 *    0
 *   / \
 *  1  2
 *
 * Input: [1,0,2]
 * Output: 2
 *
 * Example 4:
 *
 *     1
 *    / \
 *   0  0
 *    \
 *    3
```

```
 *
 * Input: [1,0,0,null,3]
 * Output: 4
 *
 *
 * Note:
 * 1<= N <= 100
 * 0 <= node.val <= N
 * */
public class _979 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/distribute-coins-in-binary-
tree/discuss/221930/JavaC%2B%2BPython-Recursive-Solution*/
        int moves = 0;

        public int distributeCoins(TreeNode root) {
            dfs(root);
            return moves;
        }

        int dfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int left = dfs(root.left);
            int right = dfs(root.right);
            moves += Math.abs(left) + Math.abs(right);
            return root.val + left + right - 1;
        }
    }
}
```

748. `_985.java`

```
package com.fishercoder.solutions;

/**
 * 985. Sum of Even Numbers After Queries
 *
 * We have an array A of integers, and an array queries of queries.
 * For the i-th query val = queries[i][0], index = queries[i][1], we add val to A[index].
Then, the answer to the i-th query is the sum of the even values of A.
 * (Here, the given index = queries[i][1] is a 0-based index, and each query permanently
modifies the array A.)
 * Return the answer to all queries.  Your answer array should have answer[i] as the
answer to the i-th query.
 *
 * Example 1:
 *
 * Input: A = [1,2,3,4], queries = [[1,0],[-3,1],[-4,0],[2,3]]
 * Output: [8,6,2,4]
 * Explanation:
 * At the beginning, the array is [1,2,3,4].
 * After adding 1 to A[0], the array is [2,2,3,4], and the sum of even values is 2 + 2 +
4 = 8.
 * After adding -3 to A[1], the array is [2,-1,3,4], and the sum of even values is 2 + 4
= 6.
 * After adding -4 to A[0], the array is [-2,-1,3,4], and the sum of even values is -2 +
4 = 2.
 * After adding 2 to A[3], the array is [-2,-1,3,6], and the sum of even values is -2 + 6
= 4.
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * -10000 <= A[i] <= 10000
 * 1 <= queries.length <= 10000
 * -10000 <= queries[i][0] <= 10000
 * 0 <= queries[i][1] < A.length
 */
public class _985 {
  public static class Solution1 {
    public int[] sumEvenAfterQueries(int[] A, int[][] queries) {
      int[] result = new int[A.length];
      for (int i = 0; i < A.length; i++) {
        int col = queries[i][1];
        A[col] = A[col] + queries[i][0];
        result[i] = computeEvenSum(A);
      }
      return result;
    }

    private int computeEvenSum(int[] A) {
      int sum = 0;
      for (int num : A) {
```

```
        if (num % 2 == 0) {
          sum += num;
        }
      }
      return sum;
    }
  }
}
```

749. `_987.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.List;
import java.util.ArrayList;
import java.util.PriorityQueue;
import java.util.TreeMap;

/**
 * 987. Vertical Order Traversal of a Binary Tree
 *
 * Given a binary tree, return the vertical order traversal of its nodes values.
 * For each node at position (X, Y), its left and right children respectively will be at
positions (X-1, Y-1) and (X+1, Y-1).
 * Running a vertical line from X = -infinity to X = +infinity, whenever the vertical
line touches some nodes, we report the values of the nodes in order from top to bottom
(decreasing Y coordinates).
 * If two nodes have the same position, then the value of the node that is reported first
is the value that is smaller.
 * Return an list of non-empty reports in order of X coordinate.  Every report will have
a list of values of nodes.
 *
 * Example 1:
 *
 *        3
 *       / \
 *      9  20
 *        /  \
 *       15  7
 *
 * Input: [3,9,20,null,null,15,7]
 * Output: [[9],[3,15],[20],[7]]
 * Explanation:
 * Without loss of generality, we can assume the root node is at position (0, 0):
 * Then, the node with value 9 occurs at position (-1, -1);
 * The nodes with values 3 and 15 occur at positions (0, 0) and (0, -2);
 * The node with value 20 occurs at position (1, -1);
 * The node with value 7 occurs at position (2, -2).
 *
 *
 * Example 2:
 *
 *       1
 *      / \
 *     2   3
 *    / \ / \
 *   4  5 6 7
 *
 * Input: [1,2,3,4,5,6,7]
 * Output: [[4],[2],[1,5,6],[3],[7]]
```

```
 * Explanation:
 * The node with value 5 and the node with value 6 have the same position according to
the given scheme.
 * However, in the report "[1,5,6]", the node value of 5 comes first since 5 is smaller
than 6.
 *
 * Note:
 *
 * The tree will have between 1 and 1000 nodes.
 * Each node's value will be between 0 and 1000.
 * */
public class _987 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/vertical-order-traversal-of-a-binary-
tree/discuss/231148/Java-TreeMap-Solution*/
        public List<List<Integer>> verticalTraversal(TreeNode root) {
            TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>> map = new
TreeMap<>();
            dfs(root, 0, 0, map);
            List<List<Integer>> list = new ArrayList<>();
            for (TreeMap<Integer, PriorityQueue<Integer>> ys : map.values()) {
                list.add(new ArrayList<>());
                for (PriorityQueue<Integer> nodes : ys.values()) {
                    while (!nodes.isEmpty()) {
                        list.get(list.size() - 1).add(nodes.poll());
                    }
                }
            }
            return list;
        }

        private void dfs(TreeNode root, int x, int y, TreeMap<Integer, TreeMap<Integer,
PriorityQueue<Integer>>> map) {
            if (root == null) {
                return;
            }
            if (!map.containsKey(x)) {
                map.put(x, new TreeMap<>());
            }
            if (!map.get(x).containsKey(y)) {
                map.get(x).put(y, new PriorityQueue<>());
            }
            map.get(x).get(y).offer(root.val);
            dfs(root.left, x - 1, y + 1, map);
            dfs(root.right, x + 1, y + 1, map);
        }
    }
}
```

750. `_989.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 989. Add to Array-Form of Integer
 *
 * For a non-negative integer X, the array-form of X is an array of its digits in left to
right order.  For example, if X = 1231, then the array form is [1,2,3,1].
 *
 * Given the array-form A of a non-negative integer X, return the array-form of the
integer X+K.
 *
 * Example 1:
 *
 * Input: A = [1,2,0,0], K = 34
 * Output: [1,2,3,4]
 * Explanation: 1200 + 34 = 1234
 * Example 2:
 *
 * Input: A = [2,7,4], K = 181
 * Output: [4,5,5]
 * Explanation: 274 + 181 = 455
 * Example 3:
 *
 * Input: A = [2,1,5], K = 806
 * Output: [1,0,2,1]
 * Explanation: 215 + 806 = 1021
 * Example 4:
 *
 * Input: A = [9,9,9,9,9,9,9,9,9,9], K = 1
 * Output: [1,0,0,0,0,0,0,0,0,0,0]
 * Explanation: 9999999999 + 1 = 10000000000
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * 0 <= A[i] <= 9
 * 0 <= K <= 10000
 * If A.length > 1, then A[0] != 0
 */
public class _989 {
  public static class Solution1 {
    public List<Integer> addToArrayForm(int[] A, int K) {
      List<Integer> kDigitsReversed = new ArrayList<>();
      int divisor = 10;
      while (K != 0) {
        kDigitsReversed.add(K % divisor);
        K /= 10;
```

```
      }
      List<Integer> result = new ArrayList<>();
      int prevFlow = 0;
      for (int i = A.length - 1, j = 0; i >= 0 || j < kDigitsReversed.size(); i --, j++)
{
        int sum;
        if (i >= 0 && j < kDigitsReversed.size()) {
          sum = A[i] + kDigitsReversed.get(j);
        } else if (i >= 0) {
          sum = A[i];
        } else {
          sum = kDigitsReversed.get(j);
        }
        int flow = 0;
        if (prevFlow != 0) {
          sum += prevFlow;
        }
        if (sum > 9) {
          flow = 1;
        }
        sum %= 10;
        prevFlow = flow;
        result.add(sum);
      }
      if (prevFlow != 0) {
        result.add(prevFlow);
      }
      Collections.reverse(result);
      return result;
    }
  }
}
```

751. `_993.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 993. Cousins in Binary Tree
 *
 * In a binary tree, the root node is at depth 0, and children of each depth k node are
at depth k+1.
 * Two nodes of a binary tree are cousins if they have the same depth, but have different
parents.
 * We are given the root of a binary tree with unique values, and the values x and y of
two different nodes in the tree.
 * Return true if and only if the nodes corresponding to the values x and y are cousins.
 *
 * Example 1:
 *         1
 *        / \
 *       2   3
 *      /
 *     4
 *
 * Input: root = [1,2,3,4], x = 4, y = 3
 * Output: false
 *
 * Example 2:
 *          1
 *         / \
 *        2   3
 *        \    \
 *        4    5
 *
 * Input: root = [1,2,3,null,4,null,5], x = 5, y = 4
 * Output: true
 *
 * Example 3:
 *           1
 *          / \
 *         2  3
 *         \
 *         4
 *
 * Input: root = [1,2,3,null,4], x = 2, y = 3
 * Output: false
 *
 *
 * Note:
```

```
 *
 * The number of nodes in the tree will be between 2 and 100.
 * Each node has a unique integer value from 1 to 100.
 */
public class _993 {
  public static class Solution1 {
    public boolean isCousins(TreeNode root, int x, int y) {
      Queue<TreeNode> queue = new LinkedList<>();
      queue.offer(root);
      while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
          TreeNode current = queue.poll();
          if (current.left != null) {
            queue.offer(current.left);
          }
          if (current.right != null) {
            queue.offer(current.right);
          }
          if (current.left != null && current.right != null) {
            if (current.left.val == x && current.right.val == y
                || current.left.val == y && current.right.val == x) {
              return false;
            }
          }
        }
        if (checkQueue(queue, x, y)) {
          return true;
        }
      }
      return false;
    }

    private boolean checkQueue(Queue<TreeNode> queue, int x, int y) {
      Set<Integer> set = new HashSet<>();
      Queue<TreeNode> tmp = new LinkedList<>(queue);
      while (!tmp.isEmpty()) {
        set.add(tmp.poll().val);
      }
      return set.contains(x) && set.contains(y);
    }
  }
}
```

752. `_994.java`

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 994. Rotting Oranges
 *
 * In a given grid, each cell can have one of three values:
 *
 * the value 0 representing an empty cell;
 * the value 1 representing a fresh orange;
 * the value 2 representing a rotten orange.
 * Every minute, any fresh orange that is adjacent (4-directionally) to a
 * rotten orange becomes rotten.
 * Return the minimum number of minutes that must elapse until no cell has a fresh
orange.
 * If this is impossible, return -1 instead.
 *
 * Example 1:
 * Input: [[2,1,1],[1,1,0],[0,1,1]]
 * Output: 4
 *
 * Example 2:
 * Input: [[2,1,1],[0,1,1],[1,0,1]]
 * Output: -1
 * Explanation:  The orange in the bottom left corner (row 2, column 0) is never rotten,
because rotting only happens 4-directionally.
 *
 * Example 3:
 * Input: [[0,2]]
 * Output: 0
 * Explanation:  Since there are already no fresh oranges at minute 0, the answer is just
0.
 *
 * Note:
 * 1 <= grid.length <= 10
 * 1 <= grid[0].length <= 10
 * grid[i][j] is only 0, 1, or 2.
 */
public class _994 {
    public static class Solution1 {
        int[] directions = new int[]{0, 1, 0, -1, 0};

        public int orangesRotting(int[][] grid) {
            Queue<int[]> rottens = new LinkedList<>();
            for (int i = 0; i < grid.length; i++) {
                for (int j = 0; j < grid[0].length; j++) {
                    if (grid[i][j] == 2) {
                        rottens.add(new int[]{i, j});
                    }
```

```
                }
            }
            int times = 0;
            while (!rottens.isEmpty()) {
                int size = rottens.size();
                boolean counted = false;
                for (int k = 0; k < size; k++) {
                    int[] rotten = rottens.poll();
                    for (int i = 0; i < 4; i++) {
                        int x = rotten[0] + directions[i];
                        int y = rotten[1] + directions[i + 1];
                        if (x >= 0 && x < grid.length && y >= 0 && y < grid[0].length &&
grid[x][y] == 1) {
                            grid[x][y] = 2;
                            if (!counted) {
                                times++;
                            }
                            counted = true;
                            rottens.add(new int[]{x, y});
                        }
                    }
                }
            }
            for (int i = 0; i < grid.length; i++) {
                for (int j = 0; j < grid[0].length; j++) {
                    if (grid[i][j] == 1) {
                        return -1;
                    }
                }
            }
            return times;
        }
    }
}
```

753. `_997.java`

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 997. Find the Town Judge
 *
 * In a town, there are N people labelled from 1 to N.
 * There is a rumor that one of these people is secretly the town judge.
 *
 * If the town judge exists, then:
 *
 * The town judge trusts nobody.
 * Everybody (except for the town judge) trusts the town judge.
 * There is exactly one person that satisfies properties 1 and 2.
 * You are given trust, an array of pairs trust[i] = [a, b] representing that the person
labelled a trusts the person labelled b.
 *
 * If the town judge exists and can be identified, return the label of the town judge.
 * Otherwise, return -1.
 *
 * Example 1:
 * Input: N = 2, trust = [[1,2]]
 * Output: 2
 *
 * Example 2:
 * Input: N = 3, trust = [[1,3],[2,3]]
 * Output: 3
 *
 * Example 3:
 * Input: N = 3, trust = [[1,3],[2,3],[3,1]]
 * Output: -1
 *
 * Example 4:
 * Input: N = 3, trust = [[1,2],[2,3]]
 * Output: -1
 *
 * Example 5:
 * Input: N = 4, trust = [[1,3],[1,4],[2,3],[2,4],[4,3]]
 * Output: 3
 *
 * Note:
 * 1 <= N <= 1000
 * trust.length <= 10000
 * trust[i] are all different
 * trust[i][0] != trust[i][1]
 * 1 <= trust[i][0], trust[i][1] <= N
 */
public class _997 {
  public static class Solution1 {
```

```
    public int findJudge(int N, int[][] trust) {
      int[] trustPoints = new int[N];
      Set<Integer> trustOthers = new HashSet<>();
      for (int[] eachTrust : trust) {
        trustPoints[eachTrust[1] - 1]++;
        trustOthers.add(eachTrust[0]);
      }
      int judge = -1;
      for (int i = 0; i < trustPoints.length; i++) {
        if (trustPoints[i] == N - 1 && !trustOthers.contains(i + 1)) {
            judge = i + 1;
        }
      }
      return judge;
    }
  }
}
```

754. `_999.java`

```
package com.fishercoder.solutions;

/**
 * 999. Available Captures for Rook
 *
 * On an 8 x 8 chessboard, there is one white rook.
 * There also may be empty squares, white bishops, and black pawns.
 * These are given as characters 'R', '.', 'B', and 'p' respectively.
 * Uppercase characters represent white pieces, and lowercase characters represent black
pieces.
 *
 * The rook moves as in the rules of Chess:
 * it chooses one of four cardinal directions (north, east, west, and south),
 * then moves in that direction until it chooses to stop, reaches the edge of the board,
 * or captures an opposite colored pawn by moving to the same square it occupies.
 * Also, rooks cannot move into the same square as other friendly bishops.
 *
 * Return the number of pawns the rook can capture in one move.
 *
 * Example 1:
 *
 * Input:[
 * 8 [".",".",".",".",".",".",".","."],
 * 7 [".",".",".","p",".",".",".","."],
 * 6 [".",".",".","R",".",".",".","p"],
 * 5 [".",".",".",".",".",".",".","."],
 * 4 [".",".",".",".",".",".",".","."],
 * 3 [".",".",".","p",".",".",".","."],
 * 2 [".",".",".",".",".",".",".","."],
 * 1 [".",".",".",".",".",".",".","."]]
 *     a   b   c   d   e   f   g   h
 *
 * Output: 3
 * Explanation:
 * In this example the rook is able to capture all the pawns.
 *
 * Example 2:
 *
 * Input:[
 * 8 [".",".",".",".",".",".",".","."],
 * 7 [".","p","p","p","p","p",".","."],
 * 6 [".","p","p","B","p","p",".","."],
 * 5 [".","p","B","R","B","p",".","."],
 * 4 [".","p","p","B","p","p",".","."],
 * 3 [".","p","p","p","p","p",".","."],
 * 2 [".",".",".",".",".",".",".","."],
 * 1 [".",".",".",".",".",".",".","."]]
 *     a   b   c   d   e   f   g   h
 *
 * Output: 0
 * Explanation:
```

```
 * Bishops are blocking the rook to capture any pawn.
 *
 * Example 3:
 *
 * Input: [
 * 8 [".",".",".",".",".",".",".","."],
 * 7 [".",".",".","p",".",".",".","."],
 * 6 [".",".",".","p",".",".",".","."],
 * 5 ["p","p",".","R",".","p","B","."],
 * 4 [".",".",".",".",".",".",".","."],
 * 3 [".",".",".","B",".",".",".","."],
 * 2 [".",".",".","p",".",".",".","."],
 * 1 [".",".",".",".",".",".",".","."]]
 *     a   b   c   d   e   f   g   h
 *
 * Output: 3
 * Explanation:
 * The rook can capture the pawns at positions b5, d6 and f5.
 *
 * Note:
 * board.length == board[i].length == 8
 * board[i][j] is either 'R', '.', 'B', or 'p'
 * There is exactly one cell with board[i][j] == 'R'
 */
public class _999 {
  public static class Solution1 {
    int[] directions = new int[] {0, 1, 0, -1, 0};

    public int numRookCaptures(char[][] board) {
      int m = board.length;
      int n = board[0].length;
      int rowR = -1;
      int colR = -1;
      for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
          if (board[i][j] == 'R') {
            rowR = i;
            colR = j;
            break;
          }
        }
      }
      int count = 0;
      for (int i = 0; i < 4; i++) {
        int neighborRow = rowR + directions[i];
        int neighborCol = colR + directions[i + 1];
        if (neighborRow >= 0 && neighborRow < m
            && neighborCol >= 0 && neighborCol < n
            && board[neighborRow][neighborCol] != 'B') {
          if (directions[i] == 0 && directions[i + 1] == 1) {
            while (neighborCol < n) {
              if (board[neighborRow][neighborCol] == 'p') {
                count++;
                break;
```

```
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborCol++;
            }
          }
        } else if (directions[i] == 1 && directions[i + 1] == 0) {
          while (neighborRow < m) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborRow++;
            }
          }
        } else if (directions[i] == 0 && directions[i + 1] == -1) {
          while (neighborCol >= 0) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborCol--;
            }
          }
        } else {
          while (neighborRow >= 0) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborRow--;
            }
          }
        }
      }
    }

    return count;
  }
 }
}
```

755. `_1002.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1002. Find Common Characters
 *
 * Given an array A of strings made only from lowercase letters,
 * return a list of all characters that show up in all strings within the list (including
duplicates).
 * For example, if a character occurs 3 times in all strings but not 4 times, you need to
include that character three times in the final answer.
 *
 * You may return the answer in any order.
 *
 * Example 1:
 * Input: ["bella","label","roller"]
 * Output: ["e","l","l"]
 *
 * Example 2:
 * Input: ["cool","lock","cook"]
 * Output: ["c","o"]
 *
 * Note:
 *
 * 1 <= A.length <= 100
 * 1 <= A[i].length <= 100
 * A[i][j] is a lowercase letter
 */
public class _1002 {
  public static class Solution1 {
    public List<String> commonChars(String[] A) {
      int[][] charCount = new int[A.length][26];
      for (int i = 0; i < A.length; i++) {
        for (char c : A[i].toCharArray()) {
          charCount[i][c - 'a']++;
        }
      }
      List<String> result = new ArrayList<>();
      for (int i = 0; i < 26; i++) {
        while (charCount[0][i] != 0) {
          char c = (char) (i + 'a');
          boolean valid = true;
          charCount[0][i]--;
          for (int j = 1; j < A.length; j++) {
            if (charCount[j][i] == 0) {
              valid = false;
              break;
            } else {
              charCount[j][i]--;
```

```
            }
          }
          if (!valid) {
            break;
          } else {
            result.add("" + c);
          }
        }
      }
      return result;
    }
  }
}
```

756. `_1003.java`

```
package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 1003. Check If Word Is Valid After Substitutions
 *
 * We are given that the string "abc" is valid.
 *
 * From any valid string V, we may split V into two pieces X and Y such that X + Y (X
concatenated with Y) is equal to V.  (X or Y may be empty.)  Then, X + "abc" + Y is also
valid.
 *
 * If for example S = "abc", then examples of valid strings are: "abc", "aabcbc",
"abcabc", "abcabcababcc".  Examples of invalid strings are: "abccba", "ab", "cababc",
"bac".
 *
 * Return true if and only if the given string S is valid.
 *
 * Note:
 *  * 1 <= S.length <= 20000
 *  * S[i] is 'a', 'b', or 'c'
 */

public class _1003 {
    public static class Solution1 {
        public boolean isValid(String S) {
            Deque<Character> stack = new ArrayDeque<>();

            for (char c : S.toCharArray()) {
                if (c == 'c') {
                    if (stack.isEmpty() || stack.pop() != 'b') {
                        return false;
                    }
                    if (stack.isEmpty() || stack.pop() != 'a') {
                        return false;
                    }
                } else {
                    stack.push(c);
                }
            }

            return stack.isEmpty();
        }
    }
}
```

757. `_1005.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * 1005. Maximize Sum Of Array After K Negations
 *
 * Given an array A of integers, we must modify the array in the following way:
 * we choose an i and replace A[i] with -A[i], and we repeat this process K times in
total.
 * (We may choose the same index i multiple times.)
 *
 * Return the largest possible sum of the array after modifying it in this way.
 *
 * Example 1:
 * Input: A = [4,2,3], K = 1
 * Output: 5
 * Explanation: Choose indices (1,) and A becomes [4,-2,3].
 *
 * Example 2:
 * Input: A = [3,-1,0,2], K = 3
 * Output: 6
 * Explanation: Choose indices (1, 2, 2) and A becomes [3,1,0,2].
 * Example 3:
 *
 * Input: A = [2,-3,-1,5,-4], K = 2
 * Output: 13
 * Explanation: Choose indices (1, 4) and A becomes [2,3,-1,5,4].
 *
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * 1 <= K <= 10000
 * -100 <= A[i] <= 100
 * */
public class _1005 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/maximize-sum-of-array-after-k-
negations/discuss/252228/A-very-simple-java-solution*/
        public int largestSumAfterKNegations(int[] A, int K) {
            PriorityQueue<Integer> minHeap = new PriorityQueue<>();
            for (int num : A) {
                minHeap.offer(num);
            }
            while (K-- > 0) {
                minHeap.offer(-minHeap.poll());
            }
            int sum = 0;
            while (!minHeap.isEmpty()) {
```

```
                sum += minHeap.poll();
            }
            return sum;
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/problems/maximize-sum-of-array-after-k-
negations/discuss/252254/JavaC%2B%2BPython-Sort*/
        public int largestSumAfterKNegations(int[] A, int K) {
            Arrays.sort(A);
            for (int i = 0; i < A.length && K > 0 && A[i] < 0; i++, K--) {
                A[i] = -A[i];
            }
            int sum = 0;
            int min = Integer.MAX_VALUE;
            for (int i = 0; i < A.length; i++) {
                min = Math.min(min, A[i]);
                sum += A[i];
            }
            return K % 2 == 0 ? sum : sum - min * 2;
        }
    }
}
```

758. `_1008.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 1008. Construct Binary Search Tree from Preorder Traversal
 *
 * Return the root node of a binary search tree that matches the given preorder
traversal.
 * (Recall that a binary search tree is a binary tree where for every node, any
descendant of node.left has a value < node.val, and any descendant of node.right has a
value > node.val.  Also recall that a preorder traversal displays the value of the node
first, then traverses node.left, then traverses node.right.)
 *
 *
 * Example 1:
 *
 * Input: [8,5,1,7,10,12]
 * Output: [8,5,10,1,7,null,12]
 *
 *              8
 *             / \
 *            5  10
 *           / \   \
 *          1   7  12
 *
 * Note:
 *
 * 1 <= preorder.length <= 100
 * The values of preorder are distinct.
 * */
public class _1008 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/construct-binary-search-tree-from-
preorder-traversal/discuss/252232/JavaC%2B%2BPython-O(N)-Solution*/
        int i = 0;

        public TreeNode bstFromPreorder(int[] preorder) {
            return bstFromPreorder(preorder, Integer.MAX_VALUE);
        }

        private TreeNode bstFromPreorder(int[] preorder, int bound) {
            if (i == preorder.length || preorder[i] > bound) {
                return null;
            }
            TreeNode root = new TreeNode(preorder[i++]);
            root.left = bstFromPreorder(preorder, root.val);
            root.right = bstFromPreorder(preorder, bound);
            return root;
        }
```

```
    }
}
```

759. `_1009.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1009. Complement of Base 10 Integer
 *
 * Every non-negative integer N has a binary representation.
 * For example, 5 can be represented as "101" in binary,
 * 11 as "1011" in binary, and so on.
 *
 * Note that except for N = 0, there are no leading zeroes in any binary representation.
 *
 * The complement of a binary representation is the number in binary you get when
 * changing every 1 to a 0 and 0 to a 1.  For example, the complement of "101" in binary
is "010" in binary.
 *
 * For a given number N in base-10, return the complement of it's binary representation
as a base-10 integer.
 *
 * Example 1:
 * Input: 5
 * Output: 2
 * Explanation: 5 is "101" in binary, with complement "010" in binary, which is 2 in
base-10.
 *
 * Example 2:
 * Input: 7
 * Output: 0
 * Explanation: 7 is "111" in binary, with complement "000" in binary, which is 0 in
base-10.
 *
 * Example 3:
 * Input: 10
 * Output: 5
 * Explanation: 10 is "1010" in binary, with complement "0101" in binary, which is 5 in
base-10.
 *
 * Note:
 * 0 <= N < 10^9
 * */
public class _1009 {
    public static class Solution1 {
        public int bitwiseComplement(int N) {
            if (N == 0) {
                return 1;
            }
            List<Integer> list = new ArrayList<>();
            while (N != 0) {
                list.add(N & 1);
```

```
            N >>= 1;
        }
        int result = 0;
        int exp = list.size() - 1;
        for (int i = list.size() - 1; i >= 0; i--) {
            if (list.get(i) == 0) {
                result += Math.pow(2, exp);
            }
            exp--;
        }
        return result;
    }
}
}
```

760. `_1010.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1010. Pairs of Songs With Total Durations Divisible by 60
 *
 * In a list of songs, the i-th song has a duration of time[i] seconds.
 *
 * Return the number of pairs of songs for which their total duration in seconds is
divisible by 60.
 * Formally, we want the number of indices i < j with (time[i] + time[j]) % 60 == 0.
 *
 * Example 1:
 * Input: [30,20,150,100,40]
 * Output: 3
 * Explanation: Three pairs have a total duration divisible by 60:
 * (time[0] = 30, time[2] = 150): total duration 180
 * (time[1] = 20, time[3] = 100): total duration 120
 * (time[1] = 20, time[4] = 40): total duration 60
 *
 * Example 2:
 * Input: [60,60,60]
 * Output: 3
 * Explanation: All three pairs have a total duration of 120, which is divisible by 60.
 *
 * Note:
 *
 * 1 <= time.length <= 60000
 * 1 <= time[i] <= 500
 * */
public class _1010 {
    public static class Solution1 {
        /**Credit: https://leetcode.com/problems/pairs-of-songs-with-total-durations-
divisible-by-60/discuss/256726/Java-O(n)-code-w-comment-similar-to-Two-Sum
         *
         * Think of Problem 1: Two Sum
         * Assume target is 60, each item in time % 60.
         * Then this problem becomes very similar to Problem 1.
         * */
        public int numPairsDivisibleBy60(int[] time) {
            int result = 0;
            Map<Integer, Integer> map = new HashMap<>();
            for (int t : time) {
                int d = (60 - t % 60) % 60;
                if (map.containsKey(d)) {
                    result += map.get(d);
                }
                map.put(t % 60, map.getOrDefault(t % 60, 0) + 1);
            }
```

```
            return result;
        }
    }
}
```

761. `_1011.java`

```
package com.fishercoder.solutions;

/**
 * 1011. Capacity To Ship Packages Within D Days
 *
 * A conveyor belt has packages that must be shipped from one port to another within D
days.
 *
 * The i-th package on the conveyor belt has a weight of weights[i].
 * Each day, we load the ship with packages on the conveyor belt (in the order given by
weights).
 * We may not load more weight than the maximum weight capacity of the ship.
 *
 * Return the least weight capacity of the ship that will result in all the packages on
the conveyor belt being shipped within D days.
 *
 * Note:
 *  * 1 <= D <= weights.length <= 50000
 *  * 1 <= weights[i] <= 500
 */

public class _1011 {
    public static class Solution1 {
        public int daysToShip(int[] weights, int capacity) {
            int days = 0;
            int currentShip = 0;

            for (int k : weights) {
                if (currentShip + k > capacity) {
                    currentShip = 0;
                    days += 1;
                }
                currentShip += k;
            }

            return days + 1;
        }

        public int shipWithinDays(int[] weights, int D) {

            int sum = 0;
            int max = 0;

            for (int k : weights) {
                sum += k;
                max = Math.max(max, k);
            }

            // Minimum possible capacity needs to be as much as the heaviest package
            int lower = max;
            // Maximum possible capacity is the total weight of all packages
```

```
            int upper = sum;

            if (daysToShip(weights, lower) <= D) {
                return lower;
            }

            // Guess is for capacity
            int currentGuess;
            int bestGuess = -1;

            // Binary search
            while (lower <= upper) {
                currentGuess = (upper + lower) / 2;
                if (daysToShip(weights, currentGuess) <= D) {
                    bestGuess = currentGuess;
                    upper = currentGuess - 1;
                } else {
                    lower = currentGuess + 1;
                }
            }

            return bestGuess;
        }
    }
}
```

762. `_1013.java`

```
package com.fishercoder.solutions;

/**
 * 1013. Partition Array Into Three Parts With Equal Sum
 *
 * Given an array A of integers, return true if and only if we can partition the array
into three non-empty parts with equal sums.
 * Formally, we can partition the array if we can find indexes i+1 < j with
 * (A[0] + A[1] + ... + A[i] == A[i+1] + A[i+2] + ... + A[j-1] == A[j] + A[j-1] + ... +
A[A.length - 1])
 *
 * Example 1:
 * Input: [0,2,1,-6,6,-7,9,1,2,0,1]
 * Output: true
 * Explanation: 0 + 2 + 1 = -6 + 6 - 7 + 9 + 1 = 2 + 0 + 1
 *
 * Example 2:
 * Input: [0,2,1,-6,6,7,9,-1,2,0,1]
 * Output: false
 *
 * Example 3:
 * Input: [3,3,6,5,-2,2,5,1,-9,4]
 * Output: true
 * Explanation: 3 + 3 = 6 = 5 - 2 + 2 + 5 + 1 - 9 + 4
 *
 * Note:
 *
 * 3 <= A.length <= 50000
 * -10000 <= A[i] <= 10000*/
public class _1013 {
    public static class Solution1 {
        public boolean canThreePartsEqualSum(int[] A) {
            int sum = 0;
            for (int i = 0; i < A.length; i++) {
                sum += A[i];
            }
            if (sum % 3 != 0) {
                return false;
            }
            int equalSum = sum / 3;
            int left = 0;
            int leftSum = 0;
            while (left < A.length - 2 && leftSum != equalSum) {
                leftSum += A[left++];
            }
            if (left > A.length - 2 || leftSum != equalSum) {
                return false;
            }

            int right = A.length - 1;
            int rightSum = 0;
```

```
            while (right > left && rightSum != equalSum) {
                rightSum += A[right--];
            }
            if (right < left || rightSum != equalSum) {
                return false;
            }
            int middleSum = 0;
            for (int i = left; i <= right; i++) {
                middleSum += A[i];
            }
            return middleSum == equalSum;
        }
    }
}
```

763. `_1014.java`

```
package com.fishercoder.solutions;

/**
 * 1014. Best Sightseeing Pair
 *
 * Given an array A of positive integers, A[i] represents the value of the i-th
sightseeing spot,
 * and two sightseeing spots i and j have distance j - i between them.
 *
 * The score of a pair (i < j) of sightseeing spots is (A[i] + A[j] + i - j) :
 * the sum of the values of the sightseeing spots, minus the distance between them.
 *
 * Return the maximum score of a pair of sightseeing spots.
 *
 * Note:
 *  * 2 <= A.length <= 50000
 *  * 1 <= A[i] <= 1000
 */

public class _1014 {
    public static class Solution1 {
        public int maxScoreSightseeingPair(int[] A) {
            int bestPrevious = A[0];
            int maxSum = 0;

            for (int i = 1; i < A.length; ++i) {
                maxSum = Math.max(maxSum, bestPrevious + A[i] - i);
                bestPrevious = Math.max(bestPrevious, A[i] + i);
            }

            return maxSum;
        }
    }
}
```

764. `_1018.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1018. Binary Prefix Divisible By 5
 *
 * Given an array A of 0s and 1s, consider N_i: the i-th subarray from A[0] to A[i]
interpreted as a binary number (from most-significant-bit to least-significant-bit.)
 *
 * Return a list of booleans answer, where answer[i] is true if and only if N_i is
divisible by 5.
 *
 * Example 1:
 *
 * Input: [0,1,1]
 * Output: [true,false,false]
 * Explanation:
 * The input numbers in binary are 0, 01, 011; which are 0, 1, and 3 in base-10.  Only
the first number is divisible by 5, so answer[0] is true.
 * Example 2:
 *
 * Input: [1,1,1]
 * Output: [false,false,false]
 * Example 3:
 *
 * Input: [0,1,1,1,1,1]
 * Output: [true,false,false,false,true,false]
 * Example 4:
 *
 * Input: [1,1,1,0,1]
 * Output: [false,false,false,false,false]
 *
 *
 * Note:
 *
 * 1 <= A.length <= 30000
 * A[i] is 0 or 1
 * */
public class _1018 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/binary-prefix-divisible-by-
5/discuss/266051/Java-beats-100*/
        public List<Boolean> prefixesDivBy5(int[] A) {
            List<Boolean> result = new ArrayList<>(A.length);
            int remainder = 0;
            for (int a : A) {
                remainder = ((remainder << 1) + a) % 5;
                result.add(remainder == 0);
            }
```

```
            return result;
        }
    }
}
```

765. `_1020.java`

```
package com.fishercoder.solutions;

/**
 * 1020. Number of Enclaves
 *
 * Given a 2D array A, each cell is 0 (representing sea) or 1 (representing land)
 *
 * A move consists of walking from one land square 4-directionally to another land
square, or off the boundary of the grid.
 *
 * Return the number of land squares in the grid for which we cannot walk off the
boundary of the grid in any number of moves.
 */

public class _1020 {
    public static class Solution1 {
        public void walk(int[][] A, boolean[][] visited, int x, int y) {
            if (x >= A.length || x < 0 || y >= A[0].length || y < 0) {
                return;
            }
            if (visited[x][y]) {
                return;
            }
            if (A[x][y] == 0) {
                return;
            }

            visited[x][y] = true;

            walk(A, visited, x - 1, y);
            walk(A, visited, x, y - 1);
            walk(A, visited, x, y + 1);
            walk(A, visited, x + 1, y);
        }

        public int numEnclaves(int[][] A) {
            int n = A.length;
            int m = A[0].length;
            boolean[][] visited = new boolean[n][m];

            for (int i = 0; i < n; ++i) {
                walk(A, visited, i, 0);
                walk(A, visited, i, m - 1);
            }

            for (int j = 0; j < m; ++j) {
                walk(A, visited, 0, j);
                walk(A, visited, n - 1, j);
            }

            int unreachables = 0;
```

```
            for (int i = 0; i < n; ++i) {
                for (int j = 0; j < m; ++j) {
                    if (A[i][j] == 1 && !visited[i][j]) {
                        ++unreachables;
                    }
                }
            }

            return unreachables;
        }
    }
}
```

766. `_1021.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**1021. Remove Outermost Parentheses
 *
 * A valid parentheses string is either empty (""), "(" + A + ")", or A + B, where A and
B are valid parentheses strings, and + represents string concatenation.  For example, "",
"()", "(())()", and "((()()))" are all valid parentheses strings.
 *
 * A valid parentheses string S is primitive if it is nonempty, and there does not exist
a way to split it into S = A+B, with A and B nonempty valid parentheses strings.
 *
 * Given a valid parentheses string S, consider its primitive decomposition: S = P_1 +
P_2 + ... + P_k, where P_i are primitive valid parentheses strings.
 *
 * Return S after removing the outermost parentheses of every primitive string in the
primitive decomposition of S.
 *
 *
 * Example 1:
 * Input: "(()())(())"
 * Output: "()()()"
 * Explanation:
 * The input string is "(()())(())", with primitive decomposition "(()())" + "(())".
 * After removing outer parentheses of each part, this is "()()" + "()" = "()()()".
 *
 * Example 2:
 * Input: "(()())(())(()(()))"
 * Output: "()()()()(())"
 * Explanation:
 * The input string is "(()())(())(()(()))", with primitive decomposition "(()())" + "
(())" + "(()(()))".
 * After removing outer parentheses of each part, this is "()()" + "()" + "()(())" = "()
()()()(())".
 *
 * Example 3:
 * Input: "()()"
 * Output: ""
 * Explanation:
 * The input string is "()()", with primitive decomposition "()" + "()".
 * After removing outer parentheses of each part, this is "" + "" = "".
 *
 * Note:
 *
 * S.length <= 10000
 * S[i] is "(" or ")"
 * S is a valid parentheses string
 * */
public class _1021 {
```

```
    public static class Solution1 {
        public String removeOuterParentheses(String S) {
            List<String> primitives = new ArrayList<>();
            for (int i = 1; i < S.length(); i++) {
                int initialI = i - 1;
                int left = 1;
                while (i < S.length() && left > 0) {
                    if (S.charAt(i) == '(') {
                        left++;
                    } else {
                        left--;
                    }
                    i++;
                }
                primitives.add(S.substring(initialI, i));
            }
            StringBuilder sb = new StringBuilder();
            for (String primitive : primitives) {
                sb.append(primitive.substring(1, primitive.length() - 1));
            }
            return sb.toString();
        }
    }
}
```

767. `_1022.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * Source: https://leetcode.com/problems/sum-of-root-to-leaf-binary-numbers/
 *
 * 1022. Sum of Root To Leaf Binary Numbers
 *
 * Given a binary tree, each node has value 0 or 1.  Each root-to-leaf path represents a
binary number starting with the most significant bit.  For example, if the path is 0 -> 1
-> 1 -> 0 -> 1, then this could represent 01101 in binary, which is 13.
 *
 * For all leaves in the tree, consider the numbers represented by the path from the root
to that leaf.
 *
 * Return the sum of these numbers.
 *
 * Example 1:
 *
 *        1
 *       / \
 *      0   1
 *     / \ / \
 *    0  1 0 1
 *
 * Input: [1,0,1,0,1,0,1]
 * Output: 22
 * Explanation: (100) + (101) + (110) + (111) = 4 + 5 + 6 + 7 = 22
 *
 * Note:
 *
 * The number of nodes in the tree is between 1 and 1000.
 * node.val is 0 or 1.
 * The answer will not exceed 2^31 - 1.
 * */
public class _1022 {
    public static class Solution1 {
        public int sumRootToLeaf(TreeNode root) {
            List<List<Integer>> paths = new ArrayList<>();
            dfs(root, paths, new ArrayList<>());
            int sum = 0;
            for (List<Integer> list : paths) {
                int num = 0;
                for (int i : list) {
                    num = (num << 1) + i;
                }
                sum += num;
```

```
            }
            return sum;
        }

        private void dfs(TreeNode root, List<List<Integer>> paths, List<Integer> path) {
            path.add(root.val);
            if (root.left != null) {
                dfs(root.left, paths, path);
                path.remove(path.size() - 1);
            }
            if (root.right != null) {
                dfs(root.right, paths, path);
                path.remove(path.size() - 1);
            }
            if (root.left == null && root.right == null) {
                paths.add(new ArrayList<>(path));
            }
        }
    }
}
```

768. `_1029.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1029. Two City Scheduling
 *
 * There are 2N people a company is planning to interview.
 * The cost of flying the i-th person to city A is costs[i][0], and the cost of flying
the i-th person to city B is costs[i][1].
 * Return the minimum cost to fly every person to a city such that exactly N people
arrive in each city.
 *
 * Example 1:
 *
 * Input: [[10,20],[30,200],[400,50],[30,20]]
 * Output: 110
 * Explanation:
 * The first person goes to city A for a cost of 10.
 * The second person goes to city A for a cost of 30.
 * The third person goes to city B for a cost of 50.
 * The fourth person goes to city B for a cost of 20.
 *
 * The total minimum cost is 10 + 30 + 50 + 20 = 110 to have half the people interviewing
in each city.
 *
 * Note:
 *
 * 1 <= costs.length <= 100
 * It is guaranteed that costs.length is even.
 * 1 <= costs[i][0], costs[i][1] <= 1000
 * */
public class _1029 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/two-city-scheduling/discuss/280173/Java-
4-lines-intuitive-solution
         * and
         * https://leetcode.com/problems/two-city-scheduling/discuss/278771/Java-sort-
solution*/
        public int twoCitySchedCost(int[][] costs) {
            Arrays.sort(costs, (a, b) -> (a[0] - a[1] - (b[0] - b[1])));
            int cost = 0;
            for (int i = 0; i < costs.length; i++) {
                if (i < costs.length / 2) {
                    cost += costs[i][0];
                } else {
                    cost += costs[i][1];
                }
            }
            return cost;
        }
```

```
    }
}
```

769. `_1030.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 1030. Matrix Cells in Distance Order
 *
 * We are given a matrix with R rows and C columns has cells with integer coordinates (r,
c), where 0 <= r < R and 0 <= c < C.
 *
 * Additionally, we are given a cell in that matrix with coordinates (r0, c0).
 *
 * Return the coordinates of all cells in the matrix, sorted by their distance from (r0,
c0) from smallest distance to largest distance.  Here, the distance between two cells
(r1, c1) and (r2, c2) is the Manhattan distance, |r1 - r2| + |c1 - c2|.  (You may return
the answer in any order that satisfies this condition.)
 *
 * Example 1:
 * Input: R = 1, C = 2, r0 = 0, c0 = 0
 * Output: [[0,0],[0,1]]
 * Explanation: The distances from (r0, c0) to other cells are: [0,1]
 *
 * Example 2:
 * Input: R = 2, C = 2, r0 = 0, c0 = 1
 * Output: [[0,1],[0,0],[1,1],[1,0]]
 * Explanation: The distances from (r0, c0) to other cells are: [0,1,1,2]
 * The answer [[0,1],[1,1],[0,0],[1,0]] would also be accepted as correct.
 *
 * Example 3:
 * Input: R = 2, C = 3, r0 = 1, c0 = 2
 * Output: [[1,2],[0,2],[1,1],[0,1],[1,0],[0,0]]
 * Explanation: The distances from (r0, c0) to other cells are: [0,1,1,2,2,3]
 * There are other answers that would also be accepted as correct, such as [[1,2],[1,1],
[0,2],[1,0],[0,1],[0,0]].
 *
 * Note:
 *
 * 1 <= R <= 100
 * 1 <= C <= 100
 * 0 <= r0 < R
 * 0 <= c0 < C
 * */
public class _1030 {
    public static class Solution1 {
        public int[][] allCellsDistOrder(int R, int C, int r0, int c0) {
            int[][] result = new int[R * C][2];
            Queue<int[]> queue = new LinkedList<>();
            queue.offer(new int[]{r0, c0});
```

```
            boolean[][] visited = new boolean[R][C];
            int i = 0;
            while (!queue.isEmpty()) {
                int[] cell = queue.poll();
                int row = cell[0];
                int col = cell[1];

                if (row < 0 || row >= R || col < 0 || col >= C || visited[row][col]) {
                    continue;
                }

                visited[row][col] = true;
                result[i++] = new int[]{row, col};
                queue.offer(new int[]{row, col + 1});
                queue.offer(new int[]{row + 1, col});
                queue.offer(new int[]{row - 1, col});
                queue.offer(new int[]{row, col - 1});

            }
            return result;
        }
    }
}
```

770. `_1033.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1033. Moving Stones Until Consecutive
 *
 * Three stones are on a number line at positions a, b, and c.
 *
 * Each turn, you pick up a stone at an endpoint (ie., either the lowest or highest
position stone),
 * and move it to an unoccupied position between those endpoints.
 * Formally, let's say the stones are currently at positions x, y, z with x < y < z.
 * You pick up the stone at either position x or position z, and move that stone to an
integer position k, with x < k < z and k != y.
 *
 * The game ends when you cannot make any more moves, ie. the stones are in consecutive
positions.
 *
 * When the game ends, what is the minimum and maximum number of moves that you could
have made?
 * Return the answer as an length 2 array: answer = [minimum_moves, maximum_moves]
 *
 * Note:
 *  * 1 <= a <= 100
 *  *  1 <= b <= 100
 *  *  1 <= c <= 100
 *  *  a != b, b != c, c != a
 */

public class _1033 {
    public static class Solution1 {
        private int minMoves(int x, int y, int z) {
            // already consecutive integers, nothing to be done
            if (x + 1 == y && y + 1 == z) {
                return 0;
            }
            // one of the following (sample) cases:
            // 1, 2, 8 (8 -> 3)
            // 1, 7, 8 (1 -> 6)
            // 1, 3, 8 (8 -> 2)
            // 1, 6, 8 (1 -> 7)
            if (y - x <= 2 || z - y <= 2) {
                return 1;
            }

            // move z to y + 1, x to y - 1
            return 2;
        }

        private int maxMoves(int x, int y, int z) {
```

```
            return z - x - 2;
        }

        public int[] numMovesStones(int a, int b, int c) {
            int[] t = {a, b, c};
            Arrays.sort(t);

            int min = minMoves(t[0], t[1], t[2]);
            int max = maxMoves(t[0], t[1], t[2]);

            return new int[]{min, max};
        }
    }
}
```

771. `_1037.java`

```
package com.fishercoder.solutions;

/**
 * 1037. Valid Boomerang
 *
 * A boomerang is a set of 3 points that are all distinct and not in a straight line.
 *
 * Given a list of three points in the plane, return whether these points are a
boomerang.
 *
 * Example 1:
 * Input: [[1,1],[2,3],[3,2]]
 * Output: true
 *
 * Example 2:
 * Input: [[1,1],[2,2],[3,3]]
 * Output: false
 *
 * Note:
 * points.length == 3
 * points[i].length == 2
 * 0 <= points[i][j] <= 100
 * */
public class _1037 {
    public static class Solution1 {
        public boolean isBoomerang(int[][] points) {
            return (points[1][1] - points[0][1]) * (points[2][0] - points[0][0]) !=
(points[2][1] - points[0][1]) * (points[1][0] - points[0][0]);
        }
    }
}
```

772. `_1038.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 1038. Binary Search Tree to Greater Sum Tree
 *
 * Given the root of a binary search tree with distinct values,
 * modify it so that every node has a new value equal to the sum of the values of the
original tree that are greater than or equal to node.val.
 * As a reminder, a binary search tree is a tree that satisfies these constraints:
 *
 * The left subtree of a node contains only nodes with keys less than the node's key.
 * The right subtree of a node contains only nodes with keys greater than the node's key.
 * Both the left and right subtrees must also be binary search trees.
 *
 *
 * Example 1:
 *                             4(30)
 *                           /    \
 *                       1(36)   6(21)
 *                     /   \     /    \
 *                 0(36)  2(35) 5(26) 7(15)
 *                          \          \
 *                         3(33)        8(8)
 *
 * Input: [4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]
 * Output: [30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]
 *
 *
 * Note:
 *
 * The number of nodes in the tree is between 1 and 100.
 * Each node will have value between 0 and 100.
 * The given tree is a binary search tree.*/
public class _1038 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/binary-search-tree-to-greater-sum-
tree/discuss/286725/JavaC%2B%2BPython-Revered-Inorder-Traversal*/
        int greaterSum = 0;

        public TreeNode bstToGst(TreeNode root) {
            if (root.right != null) {
                bstToGst(root.right);
            }
            greaterSum = root.val = greaterSum + root.val;
            if (root.left != null) {
                bstToGst(root.left);
            }
            return root;
        }
```

```
    }
}
```

773. `_1043.java`

```
package com.fishercoder.solutions;

/**
 * 1043. Partition Array for Maximum Sum
 *
 * Given an integer array A, you partition the array into (contiguous) subarrays of
length at most K.  After partitioning, each subarray has their values changed to become
the maximum value of that subarray.
 * Return the largest sum of the given array after partitioning.
 *
 * Example 1:
 * Input: A = [1,15,7,9,2,5,10], K = 3
 * Output: 84
 * Explanation: A becomes [15,15,15,9,10,10,10]
 *
 * Note:
 * 1 <= K <= A.length <= 500
 * 0 <= A[i] <= 10^6
 * */
public class _1043 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/partition-array-for-maximum-
sum/discuss/290863/JavaC%2B%2BPython-DP*/
        public int maxSumAfterPartitioning(int[] A, int K) {
            int N = A.length;
            int[] dp = new int[N];
            for (int i = 0; i < N; i++) {
                int curMax = 0;
                for (int k = 1; k <= K && i - k + 1 >= 0; k++) {
                    curMax = Math.max(curMax, A[i - k + 1]);
                    dp[i] = Math.max(dp[i], (i >= k ? dp[i - k] : 0) + curMax * k);
                }
            }
            return dp[N - 1];
        }
    }
}
```

774. `_1046.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 1046. Last Stone Weight
 *
 * We have a collection of rocks, each rock has a positive integer weight.
 *
 * Each turn, we choose the two heaviest rocks and smash them together.
 * Suppose the stones have weights x and y with x <= y.  The result of this smash is:
 *
 * If x == y, both stones are totally destroyed;
 * If x != y, the stone of weight x is totally destroyed, and the stone of weight y has
new weight y-x.
 * At the end, there is at most 1 stone left.  Return the weight of this stone (or 0 if
there are no stones left.)
 *
 * Example 1:
 * Input: [2,7,4,1,8,1]
 * Output: 1
 * Explanation:
 * We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then,
 * we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then,
 * we combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
 * we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of last
stone.
 *
 * Note:
 * 1 <= stones.length <= 30
 * 1 <= stones[i] <= 1000
 * */
public class _1046 {
    public static class Solution1 {
        public int lastStoneWeight(int[] stones) {
            PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> b - a);
            for (int stone : stones) {
                heap.offer(stone);
            }
            while (!heap.isEmpty()) {
                if (heap.size() >= 2) {
                    int one = heap.poll();
                    int two = heap.poll();
                    int diff = one - two;
                    heap.offer(diff);
                } else {
                    return heap.poll();
                }
            }
            return -1;
        }
```

```
    }
}
```

775. `_1047.java`

```
package com.fishercoder.solutions;

/**
 * 1047. Remove All Adjacent Duplicates In String
 *
 * Given a string S of lowercase letters, a duplicate removal consists of choosing two
adjacent and equal letters, and removing them.
 *
 * We repeatedly make duplicate removals on S until we no longer can.
 *
 * Return the final string after all such duplicate removals have been made.  It is
guaranteed the answer is unique.
 *
 *
 *
 * Example 1:
 *
 * Input: "abbaca"
 * Output: "ca"
 * Explanation:
 * For example, in "abbaca" we could remove "bb" since the letters are adjacent and
equal, and this is the only possible move.
 * The result of this move is that the string is "aaca", of which only "aa" is possible,
so the final string is "ca".
 *
 *
 * Note:
 *
 * 1 <= S.length <= 20000
 * S consists only of English lowercase letters.*/
public class _1047 {
    public static class Solution1 {
        public String removeDuplicates(String S) {
            StringBuilder sb = new StringBuilder(S);
            for (int i = 0; i < S.length() - 1; i++) {
                if (S.charAt(i) == S.charAt(i + 1)) {
                    return removeDuplicates(S.substring(0, i) + S.substring(i + 2));
                }
            }
            return sb.toString();
        }
    }
}
```

776. `_1049.java`

```
package com.fishercoder.solutions;

/**
 * 1049. Last Stone Weight II
 *
 * We have a collection of rocks, each rock has a positive integer weight.
 * Each turn, we choose any two rocks and smash them together.
 * Suppose the stones have weights x and y with x <= y.  The result of this smash is:
 * If x == y, both stones are totally destroyed;
 * If x != y, the stone of weight x is totally destroyed, and the stone of weight y has
new weight y-x.
 * At the end, there is at most 1 stone left.  Return the smallest possible weight of
this stone (the weight is 0 if there are no stones left.)
 *
 * Example 1:
 * Input: [2,7,4,1,8,1]
 * Output: 1
 *
 * Explanation:
 * We can combine 2 and 4 to get 2 so the array converts to [2,7,1,8,1] then,
 * we can combine 7 and 8 to get 1 so the array converts to [2,1,1,1] then,
 * we can combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
 * we can combine 1 and 1 to get 0 so the array converts to [1] then that's the optimal
value.
 *
 * Note:
 * 1 <= stones.length <= 30
 * 1 <= stones[i] <= 100
 * */
public class _1049 {
    public static class Solution1 {
        public int lastStoneWeightII(int[] stones) {
            //TODO: implement it
            return 1;
        }
    }
}
```

777. `_1051.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1051. Height Checker
 *
 * Students are asked to stand in non-decreasing order of heights for an annual photo.
 * Return the minimum number of students not standing in the right positions.
 * (This is the number of students that must move in order for all students to be
standing in non-decreasing order of height.)
 *
 * Example 1:
 * Input: [1,1,4,2,1,3]
 * Output: 3
 * Explanation:
 * Students with heights 4, 3 and the last 1 are not standing in the right positions.
 *
 * Note:
 *
 * 1 <= heights.length <= 100
 * 1 <= heights[i] <= 100
 * */
public class _1051 {
    public static class Solution1 {
        public int heightChecker(int[] heights) {
            int[] originals = Arrays.copyOf(heights, heights.length);
            Arrays.sort(heights);
            int count = 0;
            for (int i = 0; i < originals.length; i++) {
                if (heights[i] != originals[i]) {
                    count++;
                }
            }
            return count;
        }
    }
}
```

778. `_1055.java`

```
package com.fishercoder.solutions;

import java.util.stream.IntStream;

/**
 * 1055. Fixed Point
 *
 * Given an array A of distinct integers sorted in ascending order,
 * return the smallest index i that satisfies A[i] == i.  Return -1 if no such i exists.
 *
 * Example 1:
 * Input: [-10,-5,0,3,7]
 * Output: 3
 * Explanation:
 * For the given array, A[0] = -10, A[1] = -5, A[2] = 0, A[3] = 3, thus the output is 3.
 *
 * Example 2:
 * Input: [0,2,5,8,17]
 * Output: 0
 * Explanation:
 * A[0] = 0, thus the output is 0.
 *
 * Example 3:
 * Input: [-10,-5,3,4,7,9]
 * Output: -1
 * Explanation:
 * There is no such i that A[i] = i, thus the output is -1.
 *
 * Note:
 *
 * 1 <= A.length < 10^4
 * -10^9 <= A[i] <= 10^9
 * */
public class _1055 {
    public static class Solution1 {
        public int fixedPoint(int[] A) {
            return IntStream.range(0, A.length).filter(i -> A[i] ==
i).findFirst().orElse(-1);
        }
    }
}
```

779. `_1056.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1056. Confusing Number
 *
 * Given a number N, return true if and only if it is a confusing number, which satisfies
the following condition:
 * We can rotate digits by 180 degrees to form new digits. When 0, 1, 6, 8, 9 are rotated
180 degrees,
 * they become 0, 1, 9, 8, 6 respectively. When 2, 3, 4, 5 and 7 are rotated 180 degrees,
 * they become invalid. A confusing number is a number that when rotated 180 degrees
becomes a different number with each digit valid.
 *
 * Example 1:
 * Input: 6
 * Output: true
 * Explanation:
 * We get 9 after rotating 6, 9 is a valid number and 9!=6.
 *
 *
 * Example 2:
 * Input: 89
 * Output: true
 * Explanation:
 * We get 68 after rotating 89, 86 is a valid number and 86!=89.
 *
 *
 * Example 3:
 * Input: 11
 * Output: false
 * Explanation:
 * We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus
11 is not a confusing number.
 *
 * Example 4:
 * Input: 25
 * Output: false
 * Explanation:
 * We get an invalid number after rotating 25.
 *
 *  Note:
 *
 * 0 <= N <= 10^9
 * After the rotation we can ignore leading zeros, for example if after rotation we have
0008 then this number is considered as just 8.*/
public class _1056 {
    public static class Solution1 {
        Map<Integer, Integer> map = new HashMap<Integer, Integer>() {{
```

```
            put(0, 0);
            put(1, 1);
            put(8, 8);
            put(6, 9);
            put(9, 6);
        }
        };

        public boolean confusingNumber(int N) {
            if (N == 0) {
                return false;
            }
            int newNumber = 0;
            int originalN = N;
            while (N != 0) {
                newNumber *= 10;
                int digit = N % 10;
                if (!map.containsKey(digit)) {
                    return false;
                }
                digit = map.get(digit);
                newNumber += digit;
                N /= 10;
            }
            return newNumber != originalN;
        }
    }
}
```

780. `_1065.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

/**
 * 1065. Index Pairs of a String
 *
 * Given a text string and words (a list of strings),
 * return all index pairs [i, j] so that the substring text[i]...text[j] is in the list
of words.
 *
 * Example 1:
 * Input: text = "thestoryofleetcodeandme", words = ["story","fleet","leetcode"]
 * Output: [[3,7],[9,13],[10,17]]
 *
 * Example 2:
 * Input: text = "ababa", words = ["aba","ab"]
 * Output: [[0,1],[0,2],[2,3],[2,4]]
 * Explanation:
 * Notice that matches can overlap, see "aba" is found in [0,2] and [2,4].
 *
 * Note:
 *
 * All strings contains only lowercase English letters.
 * It's guaranteed that all strings in words are different.
 * 1 <= text.length <= 100
 * 1 <= words.length <= 20
 * 1 <= words[i].length <= 50
 * Return the pairs [i,j] in sorted order (i.e. sort them by their first coordinate in
case of ties sort them by their second coordinate).
 * */
public class _1065 {
    public static class Solution1 {
        public int[][] indexPairs(String text, String[] words) {
            List<List<Integer>> lists = new ArrayList<>();
            for (String word : words) {
                lists.addAll(findAllMatchsForThisWord(word, text));
            }
            if (lists.isEmpty()) {
                return new int[][]{};
            }
            Collections.sort(lists, (o1, o2) -> {
                if (o1.get(0) > o2.get(0)) {
                    return 1;
                } else if (o1.get(0) < o2.get(0)) {
                    return -1;
                } else {
                    if (o1.get(1) > o2.get(1)) {
```

```
                        return 1;
                    } else {
                        return -1;
                    }
                }
            });
            int[][] result = new int[lists.size()][lists.get(0).size()];
            for (int i = 0; i < lists.size(); i++) {
                result[i][0] = lists.get(i).get(0);
                result[i][1] = lists.get(i).get(1);
            }
            return result;
        }

        private List<List<Integer>> findAllMatchsForThisWord(String word, String text) {
            List<List<Integer>> lists = new ArrayList<>();
            for (int i = 0; i <= text.length() - word.length(); i++) {
                if (text.substring(i, i + word.length()).equals(word)) {
                    lists.add(Arrays.asList(i, i + word.length() - 1));
                }
            }
            return lists;
        }
    }
}
```

781. `_1071.java`

```
package com.fishercoder.solutions;

/**
 * 1071. Greatest Common Divisor of Strings
 *
 * For strings S and T, we say "T divides S" if and only if S = T + ... + T  (T
concatenated with itself 1 or more times)
 * Return the largest string X such that X divides str1 and X divides str2.
 *
 * Example 1:
 * Input: str1 = "ABCABC", str2 = "ABC"
 * Output: "ABC"
 *
 * Example 2:
 * Input: str1 = "ABABAB", str2 = "ABAB"
 * Output: "AB"
 *
 * Example 3:
 * Input: str1 = "LEET", str2 = "CODE"
 * Output: ""
 *
 * Note:
 *
 * 1 <= str1.length <= 1000
 * 1 <= str2.length <= 1000
 * str1[i] and str2[i] are English uppercase letters.
 * */
public class _1071 {
    public static class Solution1 {
        public String gcdOfStrings(String str1, String str2) {
            if (str1.isEmpty() || str2.isEmpty()) {
                return "";
            }
            String commomDivisor = str2;
            while (commomDivisor != null && !commomDivisor.isEmpty()) {
                if (isDivisor(str1, commomDivisor)) {
                    return commomDivisor;
                } else {
                    commomDivisor = findNextShorterCommonDivisor(str2, commomDivisor);
                }
            }
            return "";
        }

        private String findNextShorterCommonDivisor(String str2, String commomDivisor) {
            int length = nextPossibleLength(str2, commomDivisor.length() - 1);
            while (length > 1) {
                if (isDivisor(str2, length)) {
                    return str2.substring(0, length);
                }
                length = nextPossibleLength(str2, length - 1);
```

```
            }
            return null;
        }

        private boolean isDivisor(String str2, int length) {
            String commonDivisorCandidate = str2.substring(0, length);
            for (int i = 0; i < str2.length() - length; i += length) {
                if (!str2.substring(i, i + length).equals(commonDivisorCandidate)) {
                    return false;
                }
            }
            return true;
        }

        private boolean isDivisor(String str1, String commomDivisor) {
            if (str1.length() == commomDivisor.length()) {
                return str1.equals(commomDivisor);
            }
            int i = 0;
            for (; i < str1.length() - commomDivisor.length(); i +=
commomDivisor.length()) {
                if (!str1.substring(i, i + commomDivisor.length()).equals(commomDivisor))
{
                    return false;
                }
            }
            return i == (str1.length() - commomDivisor.length());
        }

        private int nextPossibleLength(String str2, int bound) {
            if (bound <= 0) {
                return -1;
            }
            int len = bound;
            while (str2.length() % len != 0) {
                len--;
            }
            return len;
        }
    }
}
```

782. `_1078.java`

```
package com.fishercoder.solutions;

import java.util.stream.Collectors;
import java.util.stream.IntStream;

/**
 * 5083. Occurrences After Bigram
 * <p>
 * Given words first and second, consider occurrences in some text of the form "first
second third",
 * where second comes immediately after first, and third comes immediately after second.
 * For each such occurrence, add "third" to the answer, and return the answer.
 * <p>
 * Example 1:
 * Input: text = "alice is a good girl she is a good student", first = "a", second =
"good"
 * Output: ["girl","student"]
 * <p>
 * Example 2:
 * Input: text = "we will we will rock you", first = "we", second = "will"
 * Output: ["we","rock"]
 * <p>
 * Note:
 * 1 <= text.length <= 1000
 * text consists of space separated words, where each word consists of lowercase English
letters.
 * 1 <= first.length, second.length <= 10
 * first and second consist of lowercase English letters.
 */
public class _1078 {
    public static class Solution1 {
        public String[] findOcurrences(String text, String first, String second) {
            String[] words = text.split(" ");
            return IntStream
                    .range(0, words.length - 2)
                    .filter(i -> words[i].equals(first) && words[i + 1].equals(second))
                    .mapToObj(i -> words[i + 2])
                    .collect(Collectors.toList())
                    .stream()
                    .toArray(String[]::new);
        }
    }
}
```

783. `_1079.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.stream.IntStream;

/**
 * 5087. Letter Tile Possibilities
 *
 * You have a set of tiles, where each tile has one letter tiles[i] printed on it.
 * Return the number of possible non-empty sequences of letters you can make.
 *
 * Example 1:
 * Input: "AAB"
 * Output: 8
 * Explanation: The possible sequences are "A", "B", "AA", "AB", "BA", "AAB", "ABA",
"BAA".
 *
 * Example 2:
 * Input: "AAABBC"
 * Output: 188
 *
 * Note:
 * 1. 1 <= tiles.length <= 7
 * 2. tiles consists of uppercase English letters.
 * */
public class _1079 {
    public static class Solution1 {
        public int numTilePossibilities(String tiles) {
            char[] chars = tiles.toCharArray();
            Arrays.sort(chars);
            boolean[] used = new boolean[chars.length];
            StringBuilder sb = new StringBuilder();
            List<String> result = new ArrayList<>();
            dfs(chars, used, sb, result);
            return result.size();
        }

        private void dfs(char[] chars, boolean[] used, StringBuilder sb, List<String>
result) {
            if (sb.length() != 0) {
                result.add(sb.toString());
            }
            IntStream.range(0, chars.length)
                    .filter(i -> !used[i])
                    .filter(i -> i <= 0 || chars[i - 1] != chars[i] || used[i - 1])
                    .forEach(i -> {
                        used[i] = true;
                        sb.append(chars[i]);
                        dfs(chars, used, sb, result);
```

```
                        used[i] = false;
                        sb.deleteCharAt(sb.length() - 1);
                    });
        }
    }
}
```

784. `_1085.java`

```
package com.fishercoder.solutions;

/**
 * 1085. Sum of Digits in the Minimum Number
 *
 * Given an array A of positive integers, let S be the sum of the digits of the minimal element of A.
 * Return 0 if S is odd, otherwise return 1.
 *
 * Example 1:
 * Input: [34,23,1,24,75,33,54,8]
 * Output: 0
 * Explanation:
 * The minimal element is 1, and the sum of those digits is S = 1 which is odd, so the answer is 0.
 *
 * Example 2:
 * Input: [99,77,33,66,55]
 * Output: 1
 * Explanation:
 * The minimal element is 33, and the sum of those digits is S = 3 + 3 = 6 which is even, so the answer is 1.
 *
 * Note:
 * 1 <= A.length <= 100
 * 1 <= A[i].length <= 100
 * */
public class _1085 {
    public static class Solution1 {
        public int sumOfDigits(int[] A) {
            int smallestNumber = A[0];
            for (int i = 1; i < A.length; i++) {
                smallestNumber = Math.min(smallestNumber, A[i]);
            }
            int sum = 0;
            while (smallestNumber > 0) {
                sum += smallestNumber % 10;
                smallestNumber /= 10;
            }
            return sum % 2 == 0 ? 1 : 0;
        }
    }
}
```

785. `_1086.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.PriorityQueue;
import java.util.TreeMap;

/**
 * 1086. High Five
 *
 * Given a list of scores of different students, return the average score of each
student's top five scores in the order of each student's id.
 * Each entry items[i] has items[i][0] the student's id, and items[i][1] the student's
score.
 * The average score is calculated using integer division.
 *
 * Example 1:
 * Input: [[1,91],[1,92],[2,93],[2,97],[1,60],[2,77],[1,65],[1,87],[1,100],[2,100],
[2,76]]
 * Output: [[1,87],[2,88]]
 * Explanation:
 * The average of the student with id = 1 is 87.
 * The average of the student with id = 2 is 88.6. But with integer division their
average converts to 88.
 *
 * Note:
 * 1 <= items.length <= 1000
 * items[i].length == 2
 * The IDs of the students is between 1 to 1000
 * The score of the students is between 1 to 100
 * For each student, there are at least 5 scores
 * */
public class _1086 {
    public static class Solution1 {
        public int[][] highFive(int[][] items) {
            TreeMap<Integer, List<Integer>> map = new TreeMap<>();
            for (int[] studentToScore : items) {
                if (map.containsKey(studentToScore[0])) {
                    map.get(studentToScore[0]).add(studentToScore[1]);
                } else {
                    List<Integer> list = new ArrayList<>();
                    list.add(studentToScore[1]);
                    map.put(studentToScore[0], list);
                }
            }
            int[][] result = new int[map.size()][2];
            for (int id : map.keySet()) {
                List<Integer> scores = map.get(id);
                Collections.sort(scores);
                int sum = 0;
```

```
                for (int i = scores.size() - 1; i >= scores.size() - 5 && i >= 0; i--) {
                    sum += scores.get(i);
                }
                result[id - 1][0] = id;
                result[id - 1][1] = sum / 5;
            }
            return result;
        }
    }

    public static class Solution2 {
        public int[][] highFive(int[][] items) {
            TreeMap<Integer, PriorityQueue<Integer>> treeMap = new TreeMap<>();
            for (int[] studentToScores : items) {
                if (treeMap.containsKey(studentToScores[0])) {
                    PriorityQueue<Integer> maxHeap = treeMap.get(studentToScores[0]);
                    maxHeap.offer(studentToScores[1]);
                    if (maxHeap.size() > 5) {
                        maxHeap.poll();
                    }
                    treeMap.put(studentToScores[0], maxHeap);
                } else {
                    PriorityQueue<Integer> maxHeap = new PriorityQueue<>();
                    maxHeap.offer(studentToScores[1]);
                    treeMap.put(studentToScores[0], maxHeap);
                }
            }
            int[][] result = new int[treeMap.size()][2];
            for (int id : treeMap.keySet()) {
                result[id - 1][0] = id;
                int sum = 0;
                PriorityQueue<Integer> maxHeap = treeMap.get(id);
                while (!maxHeap.isEmpty()) {
                    sum += maxHeap.poll();
                }
                result[id - 1][1] = sum / 5;
            }
            return result;
        }
    }
}
```

786. `_1089.java`

```
package com.fishercoder.solutions;

/**
 * 1089. Duplicate Zeros
 *
 * Given a fixed length array arr of integers, duplicate each occurrence of zero,
shifting the remaining elements to the right.
 *
 * Note that elements beyond the length of the original array are not written.
 *
 * Do the above modifications to the input array in place, do not return anything from
your function.
 *
 * Example 1:
 * Input: [1,0,2,3,0,4,5,0]
 * Output: null
 * Explanation: After calling your function, the input array is modified to:
[1,0,0,2,3,0,0,4]
 *
 * Example 2:
 * Input: [1,2,3]
 * Output: null
 * Explanation: After calling your function, the input array is modified to: [1,2,3]
 *
 * Note:
 *
 * 1 <= arr.length <= 10000
 * 0 <= arr[i] <= 9
 * */
public class _1089 {
    public static class Solution1 {
        public void duplicateZeros(int[] arr) {
            for (int i = 0; i < arr.length - 1; i++) {
                if (arr[i] == 0) {
                    duplicateZeros(arr, i + 1);
                    i++;
                }
            }
        }

        private void duplicateZeros(int[] arr, int zeroIndex) {
            int tmp = arr[zeroIndex];
            arr[zeroIndex] = 0;
            for (int i = zeroIndex + 1; i < arr.length; i++) {
                int next = arr[i];
                arr[i] = tmp;
                tmp = next;
            }
        }
    }
}
```

787. `_1099.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1099. Two Sum Less Than K
 *
 * Given an array A of integers and integer K,
 * return the maximum S such that there exists i < j with A[i] + A[j] = S and S < K.
 * If no i, j exist satisfying this equation, return -1.
 *
 * Example 1:
 * Input: A = [34,23,1,24,75,33,54,8], K = 60
 * Output: 58
 * Explanation:
 * We can use 34 and 24 to sum 58 which is less than 60.
 *
 * Example 2:
 * Input: A = [10,20,30], K = 15
 * Output: -1
 * Explanation:
 * In this case it's not possible to get a pair sum less that 15.
 *
 * Note:
 * 1 <= A.length <= 100
 * 1 <= A[i] <= 1000
 * 1 <= K <= 2000
 * */
public class _1099 {
    public static class Solution1 {
        /**
         * Time: O(n^2)
         * Space: O(1)
         * */
        public int twoSumLessThanK(int[] A, int K) {
            int maxSum = Integer.MIN_VALUE;
            for (int i = 0; i < A.length - 1; i++) {
                for (int j = i + 1; j < A.length; j++) {
                    if (A[i] + A[j] < K) {
                        maxSum = Math.max(maxSum, A[i] + A[j]);
                    }
                }
            }
            return maxSum == Integer.MIN_VALUE ? -1 : maxSum;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(nlogn)
         * Space: O(1)
```

```
         * */
        public int twoSumLessThanK(int[] A, int K) {
            Arrays.sort(A);
            int left = 0;
            int right = A.length - 1;
            int sum = Integer.MIN_VALUE;
            while (left < right) {
                int newSum = A[left] + A[right];
                if (newSum < K && newSum > sum) {
                    sum = newSum;
                } else if (newSum >= K) {
                    right--;
                } else {
                    left++;
                }
            }
            return sum == Integer.MIN_VALUE ? -1 : sum;
        }
    }
}
```

788. `_1108.java`

```
package com.fishercoder.solutions;

/**
 * 1108. Defanging an IP Address
 *
 * Given a valid (IPv4) IP address, return a defanged version of that IP address.
 * A defanged IP address replaces every period "." with "[.]".
 *
 * Example 1:
 * Input: address = "1.1.1.1"
 * Output: "1[.]1[.]1[.]1"
 *
 * Example 2:
 * Input: address = "255.100.50.0"
 * Output: "255[.]100[.]50[.]0"
 *
 * Constraints:
 * The given address is a valid IPv4 address.
 * */
public class _1108 {
    public static class Solution1 {
        public String defangIPaddr(String address) {
            //String.replaceAll() takes in a regex which needs to be escaped
            return address.replaceAll("\\.", "\$$
\\.\
$$");
        }
    }

    public static class Solution2 {
        public String defangIPaddr(String address) {
            //String.replace() takes in a string which does NOT need to be escaped
            return address.replace(".", "[.]");
        }
    }
}
```

789. `_1119.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 1119. Remove Vowels from a String
 *
 * Given a string S, remove the vowels 'a', 'e', 'i', 'o', and 'u' from it, and return
the new string.
 *
 * Example 1:
 * Input: "leetcodeisacommunityforcoders"
 * Output: "ltcdscmmntyfrcdrs"
 *
 * Example 2:
 * Input: "aeiou"
 * Output: ""
 *
 * Note:
 * S consists of lowercase English letters only.
 * 1 <= S.length <= 1000
 * */
public class _1119 {
    public static class Solution1 {
        public String removeVowels(String S) {
            Set<Character> vowels = new HashSet<>(Arrays.asList('a', 'e', 'i', 'o',
'u'));
            StringBuilder stringBuilder = new StringBuilder();
            for (char c : S.toCharArray()) {
                if (!vowels.contains(c)) {
                    stringBuilder.append(c);
                }
            }
            return stringBuilder.toString();
        }
    }

    public static class Solution2 {
        public String removeVowels(String S) {
            return S.replaceAll("[aeiou]", "");
        }
    }
}
```

790. `_1122.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.TreeMap;

/**
 * 1122. Relative Sort Array
 *
 * Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all elements in
arr2 are also in arr1.
 * Sort the elements of arr1 such that the relative ordering of items in arr1 are the
same as in arr2.
 * Elements that don't appear in arr2 should be placed at the end of arr1 in ascending
order.
 *
 * Example 1:
 * Input: arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
 * Output: [2,2,2,1,4,3,3,9,6,7,19]
 *
 * Constraints:
 * arr1.length, arr2.length <= 1000
 * 0 <= arr1[i], arr2[i] <= 1000
 * Each arr2[i] is distinct.
 * Each arr2[i] is in arr1.
 * */
public class _1122 {
    public static class Solution1 {
        public int[] relativeSortArray(int[] arr1, int[] arr2) {
            TreeMap<Integer, Integer> map = new TreeMap<>();
            Arrays.stream(arr1).forEach(num -> map.put(num, map.getOrDefault(num, 0) +
1));
            int[] result = new int[arr1.length];
            int i = 0;
            for (int num : arr2) {
                int count = map.get(num);
                while (count-- > 0) {
                    result[i++] = num;
                }
                map.remove(num);
            }
            for (int key : map.keySet()) {
                int count = map.get(key);
                while (count-- > 0) {
                    result[i++] = key;
                }
            }
            return result;
        }
    }
}
```

791. `_1128.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1128. Number of Equivalent Domino Pairs
 *
 * Given a list of dominoes, dominoes[i] = [a, b] is equivalent to dominoes[j] = [c, d]
if and only if either (a==c and b==d), or (a==d and b==c) - that is, one domino can be
rotated to be equal to another domino.
 * Return the number of pairs (i, j) for which 0 <= i < j < dominoes.length, and
dominoes[i] is equivalent to dominoes[j].
 *
 * Example 1:
 *
 * Input: dominoes = [[1,2],[2,1],[3,4],[5,6]]
 * Output: 1
 *
 * Constraints:
 * 1 <= dominoes.length <= 40000
 * 1 <= dominoes[i][j] <= 9
 * */
public class _1128 {
    public static class Solution1 {
        public int numEquivDominoPairs(int[][] dominoes) {
            int m = dominoes.length;
            Map<Integer, Integer> map = new HashMap<>();
            int count = 0;
            for (int i = 0; i < m; i++) {
                int smaller = Math.min(dominoes[i][0], dominoes[i][1]);
                int bigger = Math.max(dominoes[i][0], dominoes[i][1]);
                int key = smaller * 10 + bigger;
                count += map.getOrDefault(key, 0);
                map.put(key, map.getOrDefault(key, 0) + 1);
            }
            return count;
        }

    }
}
```

792. `_1133.java`

```
package com.fishercoder.solutions;

import java.util.TreeMap;

/**
 * 1133. Largest Unique Number
 *
 * Given an array of integers A, return the largest integer that only occurs once.
 * If no integer occurs once, return -1.
 *
 * Example 1:
 * Input: [5,7,3,9,4,9,8,3,1]
 * Output: 8
 * Explanation:
 * The maximum integer in the array is 9 but it is repeated. The number 8 occurs only
once, so it's the answer.
 *
 * Example 2:
 * Input: [9,9,8,8]
 * Output: -1
 * Explanation:
 * There is no number that occurs only once.
 *
 * Note:
 * 1 <= A.length <= 2000
 * 0 <= A[i] <= 1000
 * */
public class _1133 {
    public static class Solution1 {
        public int largestUniqueNumber(int[] A) {
            TreeMap<Integer, Integer> treeMap = new TreeMap<>((a, b) -> b - a);
            for (int num : A) {
                treeMap.put(num, treeMap.getOrDefault(num, 0) + 1);
            }
            for (int key : treeMap.keySet()) {
                if (treeMap.get(key) == 1) {
                    return key;
                }
            }
            return -1;
        }
    }

    public static class Solution2 {
        public int largestUniqueNumber(int[] A) {
            int[] count = new int[1001];
            for (int num : A) {
                count[num]++;
            }
            for (int i = 1000; i >= 0; i--) {
                if (count[i] == 1) {
```

```
                    return i;
                }
            }
            return -1;
        }
    }
}
```

793. `_1134.java`

```
package com.fishercoder.solutions;

/**
 * 1134. Armstrong Number
 *
 * The k-digit number N is an Armstrong number if and only if the k-th power of each
digit sums to N.
 * Given a positive integer N, return true if and only if it is an Armstrong number.
 *
 * Example 1:
 * Input: 153
 * Output: true
 * Explanation:
 * 153 is a 3-digit number, and 153 = 1^3 + 5^3 + 3^3.
 *
 * Example 2:
 * Input: 123
 * Output: false
 * Explanation:
 * 123 is a 3-digit number, and 123 != 1^3 + 2^3 + 3^3 = 36.
 *
 * Note:
 * 1 <= N <= 10^8
 * */
public class _1134 {
    public static class Solution1 {
        public boolean isArmstrong(int N) {
            int numOfDigits = 0;
            int copyN = N;
            while (copyN > 0) {
                copyN /= 10;
                numOfDigits++;
            }
            int sum = 0;
            copyN = N;
            while (N > 0) {
                int digit = N % 10;
                sum += Math.pow(digit, numOfDigits);
                N /= 10;
            }
            return sum == copyN;
        }
    }
}
```

794. `_1137.java`

```
package com.fishercoder.solutions;

import java.util.stream.IntStream;

/**
 * 1137. N-th Tribonacci Number
 *
 * The Tribonacci sequence Tn is defined as follows:
 *
 * T0 = 0, T1 = 1, T2 = 1, and Tn+3 = Tn + Tn+1 + Tn+2 for n >= 0.
 *
 * Given n, return the value of Tn.
 *
 * Example 1:
 * Input: n = 4
 * Output: 4
 *
 * Explanation:
 * T_3 = 0 + 1 + 1 = 2
 * T_4 = 1 + 1 + 2 = 4
 *
 * Example 2:
 * Input: n = 25
 * Output: 1389537
 *
 * Constraints:
 * 0 <= n <= 37
 * The answer is guaranteed to fit within a 32-bit integer, ie. answer <= 2^31 - 1.
 * */
public class _1137 {
    public static class Solution1 {
        public int tribonacci(int n) {
            if (n <= 1) {
                return n;
            }
            int[] numbers = new int[n + 1];
            numbers[0] = 0;
            numbers[1] = 1;
            numbers[2] = 1;
            IntStream.rangeClosed(3, n).forEach(i -> numbers[i] = numbers[i - 1] +
numbers[i - 2] + numbers[i - 3]);
            return numbers[n];
        }
    }
}
```

795. `_1150.java`

```
package com.fishercoder.solutions;

/**
 * 1150. Check If a Number Is Majority Element in a Sorted Array
 *
 * Given an array nums sorted in non-decreasing order, and a number target, return True
if and only if target is a majority element.
 * A majority element is an element that appears more than N/2 times in an array of
length N.
 *
 * Example 1:
 * Input: nums = [2,4,5,5,5,5,5,6,6], target = 5
 * Output: true
 * Explanation:
 * The value 5 appears 5 times and the length of the array is 9.
 * Thus, 5 is a majority element because 5 > 9/2 is true.
 *
 * Example 2:
 * Input: nums = [10,100,101,101], target = 101
 * Output: false
 * Explanation:
 * The value 101 appears 2 times and the length of the array is 4.
 * Thus, 101 is not a majority element because 2 > 4/2 is false.
 *
 * Note:
 * 1 <= nums.length <= 1000
 * 1 <= nums[i] <= 10^9
 * 1 <= target <= 10^9
 **/
public class _1150 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/check-if-a-number-is-majority-element-
in-a-sorted-array/discuss/358130/Java-just-one-binary-search-O(logN))-0ms-beats-100*/
        public boolean isMajorityElement(int[] nums, int target) {
            int firstIndex = findFirstOccur(nums, target);
            int plusHalfIndex = firstIndex + nums.length / 2;
            return plusHalfIndex < nums.length && nums[plusHalfIndex] == target;
        }

        private int findFirstOccur(int[] nums, int target) {
            int left = 0;
            int right = nums.length;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (nums[mid] < target) {
                    left = mid + 1;
                } else if (nums[mid] >= target) {
                    right = mid;
                }
            }
            return left;
```

```
        }
    }
}
```

796. `_1154.java`

```
package com.fishercoder.solutions;

import java.util.Calendar;

/**
 * 1154. Day of the Year
 *
 * Given a string date representing a Gregorian calendar date formatted as YYYY-MM-DD,
return the day number of the year.
 *
 * Example 1:
 *
 * Input: date = "2019-01-09"
 * Output: 9
 * Explanation: Given date is the 9th day of the year in 2019.
 * Example 2:
 *
 * Input: date = "2019-02-10"
 * Output: 41
 * Example 3:
 *
 * Input: date = "2003-03-01"
 * Output: 60
 * Example 4:
 *
 * Input: date = "2004-03-01"
 * Output: 61
 *
 * Constraints:
 *
 * date.length == 10
 * date[4] == date[7] == '-', and all other date[i]'s are digits
 * date represents a calendar date between Jan 1st, 1900 and Dec 31, 2019.
 * */
public class _1154 {
    public static class Solution1 {
        Calendar cal = Calendar.getInstance();

        public int dayOfYear(String date) {
            int year = Integer.parseInt(date.substring(0, 4));
            int month = Integer.parseInt(date.substring(5, 7));
            int day = Integer.parseInt(date.substring(8, 10));
            int thirtyDays = 30;
            int thirtyOneDays = 31;
            if (month == 1) {
                return day;
            } else if (month == 2) {
                return day + thirtyOneDays;
            } else {
                int daysInFeb = isLeapYear(year) ? 29 : 28;
                if (month == 3) {
```

```
                    return thirtyOneDays + daysInFeb + day;
                } else if (month == 4) {
                    return 2 * thirtyOneDays + daysInFeb + day;
                } else if (month == 5) {
                    return 2 * thirtyOneDays + daysInFeb + day + thirtyDays;
                } else if (month == 6) {
                    return 3 * thirtyOneDays + daysInFeb + day + thirtyDays;
                } else if (month == 7) {
                    return 3 * thirtyOneDays + daysInFeb + day + 2 * thirtyDays;
                } else if (month == 8) {
                    return 4 * thirtyOneDays + daysInFeb + day + 2 * thirtyDays;
                } else if (month == 9) {
                    return 5 * thirtyOneDays + daysInFeb + day + 2 * thirtyDays;
                } else if (month == 10) {
                    return 5 * thirtyOneDays + daysInFeb + day + 3 * thirtyDays;
                } else if (month == 11) {
                    return 6 * thirtyOneDays + daysInFeb + day + 3 * thirtyDays;
                } else {
                    return 6 * thirtyOneDays + daysInFeb + day + 4 * thirtyDays;
                }
            }
        }

        private boolean isLeapYear(int year) {
            cal.set(Calendar.YEAR, year);
            return cal.getActualMaximum(Calendar.DAY_OF_YEAR) > 365;
        }
    }
}
```

797. `_1160.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1160. Find Words That Can Be Formed by Characters
 *
 * You are given an array of strings words and a string chars.
 * A string is good if it can be formed by characters from chars (each character can only
be used once).
 * Return the sum of lengths of all good strings in words.
 *
 * Example 1:
 *
 * Input: words = ["cat","bt","hat","tree"], chars = "atach"
 * Output: 6
 * Explanation:
 * The strings that can be formed are "cat" and "hat" so the answer is 3 + 3 = 6.
 *
 * Example 2:
 *
 * Input: words = ["hello","world","leetcode"], chars = "welldonehoneyr"
 * Output: 10
 * Explanation:
 * The strings that can be formed are "hello" and "world" so the answer is 5 + 5 = 10.
 *
 * Note:
 *
 * 1 <= words.length <= 1000
 * 1 <= words[i].length, chars.length <= 100
 * All strings contain lowercase English letters only.
 * */
public class _1160 {
    public static class Solution1 {
        public int countCharacters(String[] words, String chars) {
            int length = 0;
            Map<Character, Integer> map = new HashMap<>();
            for (char c : chars.toCharArray()) {
                int count = map.getOrDefault(c, 0);
                map.put(c, count + 1);

            }
            for (String word : words) {
                if (canForm(word, map)) {
                    length += word.length();
                }
            }
            return length;
        }
```

```
        private boolean canForm(String word, final Map<Character, Integer> map) {
            Map<Character, Integer> tmp = new HashMap<>(map);
            for (Character c : word.toCharArray()) {
                if (tmp.containsKey(c) && tmp.get(c) > 0) {
                    int count = tmp.get(c);
                    tmp.put(c, count - 1);
                } else {
                    return false;
                }
            }
            return true;
        }

    }
}
```

798. `_1165.java`

```
package com.fishercoder.solutions;

/**
 * 1165. Single-Row Keyboard
 *
 * There is a special keyboard with all keys in a single row.
 * Given a string keyboard of length 26 indicating the layout of the keyboard (indexed
from 0 to 25),
 * initially your finger is at index 0.
 * To type a character, you have to move your finger to the index of the desired
character. The time taken to move your finger from index i to index j is |i - j|.
 *
 * You want to type a string word. Write a function to calculate how much time it takes
to type it with one finger.
 *
 * Example 1:
 * Input: keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"
 * Output: 4
 * Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0
again to write 'a'.
 * Total time = 2 + 1 + 1 = 4.
 *
 * Example 2:
 * Input: keyboard = "pqrstuvwxyzabcdefghijklmno", word = "leetcode"
 * Output: 73
 *
 * Constraints:
 * keyboard.length == 26
 * keyboard contains each English lowercase letter exactly once in some order.
 * 1 <= word.length <= 10^4
 * word[i] is an English lowercase letter.
 * */
public class _1165 {
    public static class Solution1 {
        public int calculateTime(String keyboard, String word) {
            int time = 0;
            int fromIndex = 0;
            for (char c : word.toCharArray()) {
                int fingerMoves = Math.abs(fromIndex - keyboard.indexOf(c));
                fromIndex = keyboard.indexOf(c);
                time += fingerMoves;
            }
            return time;
        }
    }
}
```

799. `_1170.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1170. Compare Strings by Frequency of the Smallest Character
 *
 * Let's define a function f(s) over a non-empty string s,
 * which calculates the frequency of the smallest character in s.
 * For example, if s = "dcce" then f(s) = 2 because the smallest character is "c" and its
frequency is 2.
 * Now, given string arrays queries and words,
 * return an integer array answer,
 * where each answer[i] is the number of words such that f(queries[i]) < f(W), where W is
a word in words.
 *
 * Example 1:
 * Input: queries = ["cbd"], words = ["zaaaz"]
 * Output: [1]
 * Explanation: On the first query we have f("cbd") = 1, f("zaaaz") = 3 so f("cbd") <
f("zaaaz").
 *
 * Example 2:
 * Input: queries = ["bbb","cc"], words = ["a","aa","aaa","aaaa"]
 * Output: [1,2]
 * Explanation: On the first query only f("bbb") < f("aaaa"). On the second query both
f("aaa") and f("aaaa") are both > f("cc").
 *
 * Constraints:
 * 1 <= queries.length <= 2000
 * 1 <= words.length <= 2000
 * 1 <= queries[i].length, words[i].length <= 10
 * queries[i][j], words[i][j] are English lowercase letters.
 * */
public class _1170 {
    public static class Solution1 {
        /**
         * Use simple iteration when finding counts
         * Time: O(n^m) where m is the size of queries and n is the size of words
         * Space: O(max(m, n) where m is the size of queries and n is the size of words)
         * */
        public int[] numSmallerByFrequency(String[] queries, String[] words) {
            int[] queriesMinFrequecies = new int[queries.length];
            for (int i = 0; i < queries.length; i++) {
                queriesMinFrequecies[i] = computeLowestFrequency(queries[i]);
            }

            int[] wordsMinFrequecies = new int[words.length];
            for (int i = 0; i < words.length; i++) {
                wordsMinFrequecies[i] = computeLowestFrequency(words[i]);
            }
```

```
        Arrays.sort(wordsMinFrequecies);

        int[] result = new int[queries.length];
        for (int i = 0; i < result.length; i++) {
            result[i] = search(wordsMinFrequecies, queriesMinFrequecies[i]);
        }
        return result;
    }

    private int search(int[] nums, int target) {
        int count = 0;
        for (int i = nums.length - 1; i >= 0; i--) {
            if (nums[i] > target) {
                count++;
            } else {
                break;
            }
        }
        return count;
    }

    private int computeLowestFrequency(String string) {
        char[] str = string.toCharArray();
        Arrays.sort(str);
        String sortedString = new String(str);
        int frequency = 1;
        for (int i = 1; i < sortedString.length(); i++) {
            if (sortedString.charAt(i) == sortedString.charAt(0)) {
                frequency++;
            } else {
                break;
            }
        }
        return frequency;
    }
}

public static class Solution2 {
    /**
     * Use binary search when finding counts
     * Time: O(n^logn) where m is the size of queries and n is the size of words
     * Space: O(max(m, n) where m is the size of queries and n is the size of words)
     * */
    public int[] numSmallerByFrequency(String[] queries, String[] words) {
        int[] queriesMinFrequecies = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            queriesMinFrequecies[i] = computeLowestFrequency(queries[i]);
        }

        int[] wordsMinFrequecies = new int[words.length];
        for (int i = 0; i < words.length; i++) {
            wordsMinFrequecies[i] = computeLowestFrequency(words[i]);
        }
        Arrays.sort(wordsMinFrequecies);
```

```
            int[] result = new int[queries.length];
            for (int i = 0; i < result.length; i++) {
                result[i] = binarySearch(wordsMinFrequecies, queriesMinFrequecies[i]);
            }
            return result;
        }

        private int binarySearch(int[] nums, int target) {
            int left = 0;
            int right = nums.length - 1;
            while (left <= right) {
                int mid = (left + right) / 2;
                if (nums[mid] <= target) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
            return nums.length - left;
        }

        private int computeLowestFrequency(String string) {
            char[] str = string.toCharArray();
            Arrays.sort(str);
            String sortedString = new String(str);
            int frequency = 1;
            for (int i = 1; i < sortedString.length(); i++) {
                if (sortedString.charAt(i) == sortedString.charAt(0)) {
                    frequency++;
                } else {
                    break;
                }
            }
            return frequency;
        }
    }
}
```

800. `_1184.java`

```
package com.fishercoder.solutions;

/**
 * 1184. Distance Between Bus Stops
 *
 * A bus has n stops numbered from 0 to n - 1 that form a circle. We know the distance
between all pairs of neighboring stops where distance[i] is the distance between the
stops number i and (i + 1) % n.
 * The bus goes along both directions i.e. clockwise and counterclockwise.
 * Return the shortest distance between the given start and destination stops.
 *
 * Example 1:
 * Input: distance = [1,2,3,4], start = 0, destination = 1
 * Output: 1
 * Explanation: Distance between 0 and 1 is 1 or 9, minimum is 1.
 *
 * Example 2:
 * Input: distance = [1,2,3,4], start = 0, destination = 2
 * Output: 3
 * Explanation: Distance between 0 and 2 is 3 or 7, minimum is 3.
 *
 * Example 3:
 * Input: distance = [1,2,3,4], start = 0, destination = 3
 * Output: 4
 * Explanation: Distance between 0 and 3 is 6 or 4, minimum is 4.
 *
 * Constraints:
 * 1 <= n <= 10^4
 * distance.length == n
 * 0 <= start, destination < n
 * 0 <= distance[i] <= 10^4
 * */
public class _1184 {
    public static class Solution1 {
        public int distanceBetweenBusStops(int[] distance, int start, int destination) {
            if (start > destination) {
                int tmp = start;
                start = destination;
                destination = tmp;
            }
            int clockwise = 0;
            for (int i = start; i < destination; i++) {
                clockwise += distance[i];
            }
            int counterClockwise = 0;
            for (int i = destination; i < distance.length; i++) {
                counterClockwise += distance[i];
            }
            for (int i = 0; i < start; i++) {
                counterClockwise += distance[i];
            }
```

```
            return Math.min(clockwise, counterClockwise);
        }
    }
}
```

801. `_1185.java`

```
package com.fishercoder.solutions;

/**
 * 1185. Day of the Week
 *
 * Given a date, return the corresponding day of the week for that date.
 * The input is given as three integers representing the day, month and year
respectively.
 * Return the answer as one of the following values {"Sunday", "Monday", "Tuesday",
"Wednesday", "Thursday", "Friday", "Saturday"}.
 *
 * Example 1:
 * Input: day = 31, month = 8, year = 2019
 * Output: "Saturday"
 *
 * Example 2:
 * Input: day = 18, month = 7, year = 1999
 * Output: "Sunday"
 *
 * Example 3:
 * Input: day = 15, month = 8, year = 1993
 * Output: "Sunday"
 *
 * Constraints:
 * The given dates are valid dates between the years 1971 and 2100.
 * */
public class _1185 {
    public static class Solution1 {
        /**
         * Time: O(1)
         * Space: O(1)
         *
         * Plain and simple algorithm:
         * based on the fact that 1/1/1971 is a Friday and calculate the given day.
         * */
        public String dayOfTheWeek(int day, int month, int year) {
            String[] daysInTheWeek = new String[]{"Sunday", "Monday", "Tuesday",
"Wednesday", "Thursday", "Friday", "Saturday"};
            int[] daysInTheMonth = new int[]{31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31};
            int numberOfDays = 0;
            for (int i = 1971; i < year; i++) {
                numberOfDays += i % 4 == 0 ? 366 : 365;
            }
            if (year % 4 == 0) {
                daysInTheMonth[1] = 29;
            }
            for (int i = 0; i < month - 1; i++) {
                numberOfDays += daysInTheMonth[i];
            }
            numberOfDays += day - 1;
```

```
            return daysInTheWeek[(numberOfDays + 5) % 7];
        }
    }
}
```

802. `_1189.java`

```
package com.fishercoder.solutions;

/**
 * 1189. Maximum Number of Balloons
 *
 * Given a string text, you want to use the characters of text to form as many instances
of the word "balloon" as possible.
 * You can use each character in text at most once. Return the maximum number of
instances that can be formed.
 *
 * Example 1:
 * Input: text = "nlaebolko"
 * Output: 1
 *
 * Example 2:
 * Input: text = "loonbalxballpoon"
 * Output: 2
 *
 * Example 3:
 * Input: text = "leetcode"
 * Output: 0
 *
 * Constraints:
 * 1 <= text.length <= 10^4
 * text consists of lower case English letters only.
 * */
public class _1189 {
    public static class Solution1 {
        public int maxNumberOfBalloons(String text) {
            int[] counts = new int[26];
            for (char c : text.toCharArray()) {
                counts[c - 'a']++;
            }
            return Math.min(counts[0], Math.min(counts[1], Math.min(counts[11] / 2,
Math.min(counts[14] / 2, counts[13]))));
        }
    }
}
```

803. `_1196.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1196. How Many Apples Can You Put into the Basket
 *
 * You have some apples, where arr[i] is the weight of the i-th apple.  You also have a
basket that can carry up to 5000 units of weight.
 * Return the maximum number of apples you can put in the basket.
 *
 * Example 1:
 * Input: arr = [100,200,150,1000]
 * Output: 4
 * Explanation: All 4 apples can be carried by the basket since their sum of weights is
1450.
 *
 * Example 2:
 * Input: arr = [900,950,800,1000,700,800]
 * Output: 5
 * Explanation: The sum of weights of the 6 apples exceeds 5000 so we choose any 5 of
them.
 *
 * Constraints:
 * 1 <= arr.length <= 10^3
 * 1 <= arr[i] <= 10^3
 * */
public class _1196 {
    public static class Solution1 {
        public int maxNumberOfApples(int[] arr) {
            Arrays.sort(arr);
            int sum = 0;
            int i = 0;
            for (; i < arr.length; i++) {
                sum += arr[i];
                if (sum > 5000) {
                    break;
                }
            }
            return i;
        }
    }
}
```

804. `_1198.java`

```
package com.fishercoder.solutions;

/**
 * 1198. Find Smallest Common Element in All Rows
 *
 * Given a matrix mat where every row is sorted in increasing order, return the smallest
common element in all rows.
 * If there is no common element, return -1.
 *
 * Example 1:
 * Input: mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]
 * Output: 5
 *
 * Constraints:
 * 1 <= mat.length, mat[i].length <= 500
 * 1 <= mat[i][j] <= 10^4
 * mat[i] is sorted in increasing order.
 * */
public class _1198 {
    public static class Solution1 {
        public int smallestCommonElement(int[][] mat) {
            int m = mat.length;
            int n = mat[0].length;
            for (int j = 0; j < n; j++) {
                int minCommon = mat[0][j];
                //we'll start from the second row
                int i = 1;
                for (; i < m; i++) {
                    if (thisRowHasThisNumber(mat[i], minCommon)) {
                        continue;
                    } else {
                        break;
                    }
                }
                if (i == m) {
                    return minCommon;
                }
            }
            return -1;
        }

        private boolean thisRowHasThisNumber(int[] nums, int target) {
            int left = 0;
            int right = nums.length - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;
                if (target == nums[mid]) {
                    return true;
                } else if (target > nums[mid]) {
                    left = mid + 1;
                } else {
```

```
                    right = mid - 1;
                }
            }
            return false;
        }
    }
}
```

805. `_1200.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 1200. Minimum Absolute Difference
 *
 * Given an array of distinct integers arr, find all pairs of elements with the minimum
absolute difference of any two elements.
 * Return a list of pairs in ascending order(with respect to pairs), each pair [a, b]
follows
 * a, b are from arr
 * a < b
 * b - a equals to the minimum absolute difference of any two elements in arr
 *
 * Example 1:
 * Input: arr = [4,2,1,3]
 * Output: [[1,2],[2,3],[3,4]]
 * Explanation: The minimum absolute difference is 1. List all pairs with difference
equal to 1 in ascending order.
 *
 * Example 2:
 * Input: arr = [1,3,6,10,15]
 * Output: [[1,3]]
 *
 * Example 3:
 * Input: arr = [3,8,-10,23,19,-4,-14,27]
 * Output: [[-14,-10],[19,23],[23,27]]
 *
 * Constraints:
 * 2 <= arr.length <= 10^5
 * -10^6 <= arr[i] <= 10^6
 * */
public class _1200 {
    public static class Solution1 {
        /**
         * Time: O(nlogn) due to sorting
         * Space: O(k) where k is the distinct number of differences between two numbers
in the given array
         * */
        public List<List<Integer>> minimumAbsDifference(int[] arr) {
            Arrays.sort(arr);
            int minimumDiff = arr[1] - arr[0];
            List result = new ArrayList();
            result.add(Arrays.asList(arr[0], arr[1]));
            for (int i = 1; i < arr.length - 1; i++) {
                int diff = arr[i + 1] - arr[i];
                if (minimumDiff == diff) {
                    result.add(Arrays.asList(arr[i], arr[i + 1]));
```

```
                } else if (minimumDiff > diff) {
                    minimumDiff = diff;
                    result = new ArrayList();
                    result.add(Arrays.asList(arr[i], arr[i + 1]));
                }
            }
            return result;
        }
    }
}
```

806. `_1207.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 1207. Unique Number of Occurrences
 *
 * Given an array of integers arr,
 * write a function that returns true if and only if the number of occurrences of each
value in the array is unique.
 *
 * Example 1:
 * Input: arr = [1,2,2,1,1,3]
 * Output: true
 * Explanation: The value 1 has 3 occurrences, 2 has 2 and 3 has 1. No two values have
the same number of occurrences.
 *
 * Example 2:
 * Input: arr = [1,2]
 * Output: false
 *
 * Example 3:
 * Input: arr = [-3,0,1,-3,1,1,1,-3,10,0]
 * Output: true
 *
 * Constraints:
 * 1 <= arr.length <= 1000
 * -1000 <= arr[i] <= 1000
 * */
public class _1207 {
    public static class Solution1 {
        public boolean uniqueOccurrences(int[] arr) {
            Map<Integer, Integer> map = new HashMap<>();
            Arrays.stream(arr).forEach(num -> {
                map.put(num, map.containsKey(num) ? map.get(num) + 1 : 1);
            });
            Set<Integer> set = new HashSet<>();
            return map.keySet().stream().mapToInt(key -> key).allMatch(key ->
set.add(map.get(key)));
        }
    }
}
```

807. `_1213.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1213. Intersection of Three Sorted Arrays
 *
 * Given three integer arrays arr1, arr2 and arr3 sorted in strictly increasing order,
 * return a sorted array of only the integers that appeared in all three arrays.
 *
 * Example 1:
 * Input: arr1 = [1,2,3,4,5], arr2 = [1,2,5,7,9], arr3 = [1,3,4,5,8]
 * Output: [1,5]
 * Explanation: Only 1 and 5 appeared in the three arrays.
 *
 * Constraints:
 *
 * 1 <= arr1.length, arr2.length, arr3.length <= 1000
 * 1 <= arr1[i], arr2[i], arr3[i] <= 2000
 * */
public class _1213 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/intersection-of-three-sorted-
arrays/discuss/397603/Simple-Java-solution-beats-100*/
        public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
            List<Integer> result = new ArrayList();
            int i = 0;
            int j = 0;
            int k = 0;
            while (i < arr1.length && j < arr2.length && k < arr3.length) {
                if (arr1[i] == arr2[j] && arr1[i] == arr3[k]) {
                    result.add(arr1[i]);
                    i++;
                    j++;
                    k++;
                } else if (arr1[i] < arr2[j]) {
                    i++;
                } else if (arr2[j] < arr3[k]) {
                    j++;
                } else {
                    k++;
                }
            }
            return result;
        }
    }
}
```

808. `_1217.java`

```
package com.fishercoder.solutions;

/**
 * 1217. Play with Chips
 *
 * There are some chips, and the i-th chip is at position chips[i].
 * You can perform any of the two following types of moves any number of times (possibly
zero) on any chip:
 *
 * Move the i-th chip by 2 units to the left or to the right with a cost of 0.
 * Move the i-th chip by 1 unit to the left or to the right with a cost of 1.
 *
 * There can be two or more chips at the same position initially.
 *
 * Return the minimum cost needed to move all the chips to the same position (any
position).
 *
 * Example 1:
 * Input: chips = [1,2,3]
 * Output: 1
 * Explanation: Second chip will be moved to positon 3 with cost 1. First chip will be
moved to position 3 with cost 0. Total cost is 1.
 *
 * Example 2:
 * Input: chips = [2,2,2,3,3]
 * Output: 2
 * Explanation: Both fourth and fifth chip will be moved to position two with cost 1.
Total minimum cost will be 2.
 *
 * Constraints:
 * 1 <= chips.length <= 100
 * 1 <= chips[i] <= 10^9
 * */
public class _1217 {
    public static class Solution1 {
        public int minCostToMoveChips(int[] chips) {
            //TODO: implement it
            return -1;
        }
    }
}
```

809. `_1228.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 1228. Missing Number In Arithmetic Progression
 *
 * In some array arr, the values were in arithmetic progression: the values arr[i+1] -
arr[i] are all equal for every 0 <= i < arr.length - 1.
 * Then, a value from arr was removed that was not the first or last value in the array.
 * Return the removed value.
 *
 * Example 1:
 * Input: arr = [5,7,11,13]
 * Output: 9
 * Explanation: The previous array was [5,7,9,11,13].
 *
 * Example 2:
 * Input: arr = [15,13,12]
 * Output: 14
 * Explanation: The previous array was [15,14,13,12].
 *
 * Constraints:
 * 3 <= arr.length <= 1000
 * 0 <= arr[i] <= 10^5
 * */
public class _1228 {
    public static class Solution1 {
        /**A super verbose and inefficient but working way...*/
        public int missingNumber(int[] arr) {
            Arrays.sort(arr);
            Map<Integer, List<Integer>> map = new HashMap<>();
            for (int i = 0; i < arr.length - 1; i++) {
                int diff = arr[i + 1] - arr[i];
                List<Integer> list = map.getOrDefault(diff, new ArrayList<>());
                list.add(i);
                map.put(diff, list);
            }
            int smallDiff = arr[arr.length - 1];
            int bigDiff = 0;
            for (int key : map.keySet()) {
                smallDiff = Math.min(smallDiff, key);
                bigDiff = Math.max(bigDiff, key);
            }
            return arr[map.get(bigDiff).get(0)] + smallDiff;
        }
    }
```

```
    public static class Solution2 {
        /**credit: https://leetcode.com/problems/missing-number-in-arithmetic-
progression/discuss/408474/JavaC%2B%2BPython-Arithmetic-Sum-and-Binary-Search*/
        public int missingNumber(int[] arr) {
            int min = arr[0];
            int max = arr[0];
            int sum = 0;
            for (int num : arr) {
                max = Math.max(max, num);
                min = Math.min(min, num);
                sum += num;
            }
            return (max + min) * (arr.length + 1) / 2 - sum;
        }
    }
}
```

810. `_1232.java`

```
package com.fishercoder.solutions;

/**
 * 1232. Check If It Is a Straight Line
 *
 * You are given an array coordinates, coordinates[i] = [x, y],
 * where [x, y] represents the coordinate of a point.
 * Check if these points make a straight line in the XY plane.
 *
 * Example 1:
 * Input: coordinates = [[1,2],[2,3],[3,4],[4,5],[5,6],[6,7]]
 * Output: true
 *
 * Example 2:
 * Input: coordinates = [[1,1],[2,2],[3,4],[4,5],[5,6],[7,7]]
 * Output: false
 *
 * Constraints:
 * 2 <= coordinates.length <= 1000
 * coordinates[i].length == 2
 * -10^4 <= coordinates[i][0], coordinates[i][1] <= 10^4
 * coordinates contains no duplicate point.
 * */
public class _1232 {
    public static class Solution1 {
        /**
         * To check if they share the same slope, we use this formula:
         *
         * check whether (y4 - y3)/(x4- x3) equals to (y2 - y1)/(x2 - x1)
         * considering denominator could be zero, we'll change it to use multiplication
instead of division,
         * thus it becomes
         * check whether (y4 - y3)*(x2 - x1) equals (x4 - x3)*(y2 - y1)
         * */
        public boolean checkStraightLine(int[][] coordinates) {
            for (int i = 2; i < coordinates.length - 1; i++) {
                if ((coordinates[1][0] - coordinates[0][0]) * (coordinates[i + 1][1] -
coordinates[i][1])
                        != (coordinates[1][1] - coordinates[0][1]) * (coordinates[i + 1]
[0] - coordinates[i][0])) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

811. `_1237.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 1237. Find Positive Integer Solution for a Given Equation
 *
 * Given a function  f(x, y) and a value z, return all positive integer pairs x and y
where f(x,y) == z.
 * The function is constantly increasing, i.e.:
 *
 * f(x, y) < f(x + 1, y)
 * f(x, y) < f(x, y + 1)
 * The function interface is defined like this:
 *
 * interface CustomFunction {
 * public:
 *   // Returns positive integer f(x, y) for any given positive integer x and y.
 *   int f(int x, int y);
 * };
 * For custom testing purposes you're given an integer function_id and a target z as
input,
 * where function_id represent one function from an secret internal list, on the examples
you'll know only two functions from the list.
 * You may return the solutions in any order.
 *
 * Example 1:
 * Input: function_id = 1, z = 5
 * Output: [[1,4],[2,3],[3,2],[4,1]]
 * Explanation: function_id = 1 means that f(x, y) = x + y
 *
 * Example 2:
 * Input: function_id = 2, z = 5
 * Output: [[1,5],[5,1]]
 * Explanation: function_id = 2 means that f(x, y) = x * y
 *
 * Constraints:
 * 1 <= function_id <= 9
 * 1 <= z <= 100
 * It's guaranteed that the solutions of f(x, y) == z will be on the range 1 <= x, y <=
1000
 * It's also guaranteed that f(x, y) will fit in 32 bit signed integer if 1 <= x, y <=
1000
 */
public class _1237 {

    // This is the custom function interface.
    // You should not implement it, or speculate about its implementation
    abstract class CustomFunction {
```

```
    // Returns f(x, y) for any given positive integers x and y.
    // Note that f(x, y) is increasing with respect to both x and y.
    // i.e. f(x, y) < f(x + 1, y), f(x, y) < f(x, y + 1)
    public abstract int f(int x, int y);
}

public static class Solution1 {
    /**Time: O(x*y)
     * Space: O(1)*/
    public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
        List<List<Integer>> result = new ArrayList<>();
        for (int x = 1; x <= 1000; x++) {
            for (int y = 1; y <= 1000; y++) {
                if (customfunction.f(x, y) == z) {
                    result.add(Arrays.asList(x, y));
                }
            }
        }
        return result;
    }
}

public static class Solution2 {
    /**
     * linear search
     *
     * Time: O(x + y)
     * Space: O(1)
     */
    public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
        List<List<Integer>> result = new ArrayList<>();
        int x = 1;
        int y = 1000;
        while (x < 1001 && y > 0) {
            int functionResult = customfunction.f(x, y);
            if (functionResult < z) {
                x++;
            } else if (functionResult > z) {
                y--;
            } else {
                result.add(Arrays.asList(x++, y--));
            }
        }
        return result;
    }
}

public static class Solution3 {
    /**
     * binary search
     *
     * Time: O(xlogy)
     * Space: O(1)
     * */
```

```
        public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
            List<List<Integer>> result = new ArrayList<>();
            for (int x = 1; x <= 1000; x++) {
                int left = 1;
                int right = 1001;
                while (left < right) {
                    int y = (left + right) / 2;
                    if (customfunction.f(x, y) < z) {
                        left = y + 1;
                    } else {
                        right = y;
                    }
                }
                if (customfunction.f(x, left) == z) {
                    result.add(Arrays.asList(x, left));
                }
            }
            return result;
        }
    }

}
```

812. `_1243.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 1243. Array Transformation
 *
 * Given an initial array arr, every day you produce a new array using the array of the
previous day.
 *
 * On the i-th day, you do the following operations on the array of day i-1 to produce
the array of day i:
 *
 * If an element is smaller than both its left neighbor and its right neighbor, then this
element is incremented.
 * If an element is bigger than both its left neighbor and its right neighbor, then this
element is decremented.
 * The first and last elements never change.
 * After some days, the array does not change. Return that final array.
 *
 * Example 1:
 * Input: arr = [6,2,3,4]
 * Output: [6,3,3,4]
 * Explanation:
 * On the first day, the array is changed from [6,2,3,4] to [6,3,3,4].
 * No more operations can be done to this array.
 *
 * Example 2:
 * Input: arr = [1,6,3,4,3,5]
 * Output: [1,4,4,4,4,5]
 * Explanation:
 * On the first day, the array is changed from [1,6,3,4,3,5] to [1,5,4,3,4,5].
 * On the second day, the array is changed from [1,5,4,3,4,5] to [1,4,4,4,4,5].
 * No more operations can be done to this array.
 *
 * Constraints:
 * 1 <= arr.length <= 100
 * 1 <= arr[i] <= 100
 * */
public class _1243 {
    public static class Solution1 {
        public List<Integer> transformArray(int[] arr) {
            int[] copy;
            do {
                copy = Arrays.copyOf(arr, arr.length);
                for (int i = 1; i < arr.length - 1; i++) {
                    if (copy[i] < copy[i - 1] && copy[i] < copy[i + 1]) {
                        arr[i]++;
                    } else if (copy[i] > copy[i - 1] && copy[i] > copy[i + 1]) {
```

```
                        arr[i]--;
                    }
                }
            } while (!Arrays.equals(copy, arr));
            return Arrays.stream(arr)
                    .boxed()
                    .collect(Collectors.toList());
        }
    }
}
```

813. `_1252.java`

```
package com.fishercoder.solutions;

/**
 * 1252. Cells with Odd Values in a Matrix
 *
 * Given n and m which are the dimensions of a matrix initialized by zeros and given an
array indices where indices[i] = [ri, ci]. For each pair of [ri, ci] you have to
increment all cells in row ri and column ci by 1.
 * Return the number of cells with odd values in the matrix after applying the increment
to all indices.
 *
 * Example 1:
 * 0, 0, 0     1, 2, 1    1, 3, 1
 * 0, 0, 0     0, 1, 0    1, 3 ,1
 *
 * Input: n = 2, m = 3, indices = [[0,1],[1,1]]
 * Output: 6
 * Explanation: Initial matrix = [[0,0,0],[0,0,0]].
 * After applying first increment it becomes [[1,2,1],[0,1,0]].
 * The final matrix will be [[1,3,1],[1,3,1]] which contains 6 odd numbers.
 *
 * Example 2:
 * 0, 0    0, 1    2, 2
 * 0, 0    1, 2    2, 2
 *
 * Input: n = 2, m = 2, indices = [[1,1],[0,0]]
 * Output: 0
 * Explanation: Final matrix = [[2,2],[2,2]]. There is no odd number in the final matrix.
 *
 *
 * Constraints:
 * 1 <= n <= 50
 * 1 <= m <= 50
 * 1 <= indices.length <= 100
 * 0 <= indices[i][0] < n
 * 0 <= indices[i][1] < m
 * */
public class _1252 {
    public static class Solution1 {
        /**
         * Time: O(m*n + k) where k is the length of indices
         * Space: O(m*n)
         * */
        public int oddCells(int n, int m, int[][] indices) {
            int[][] matrix = new int[n][m];
            for (int i = 0; i < indices.length; i++) {
                addOneToRow(matrix, indices[i][0]);
                addOneToColumn(matrix, indices[i][1]);
            }
            int oddNumberCount = 0;
            for (int i = 0; i < matrix.length; i++) {
```

```
                for (int j = 0; j < matrix[0].length; j++) {
                    if (matrix[i][j] % 2 != 0) {
                        oddNumberCount++;
                    }
                }
            }
            return oddNumberCount;
        }

        private void addOneToColumn(int[][] matrix, int columnIndex) {
            for (int i = 0; i < matrix.length; i++) {
                matrix[i][columnIndex] += 1;
            }
        }

        private void addOneToRow(int[][] matrix, int rowIndex) {
            for (int j = 0; j < matrix[0].length; j++) {
                matrix[rowIndex][j] += 1;
            }
        }
    }

    public static class Solution2 {
        /**
         * Time: O(m*n + k) where k is the length of indices
         * Space: O(m + n)
         */
        public int oddCells(int n, int m, int[][] indices) {
            boolean[] row = new boolean[n];
            boolean[] column = new boolean[m];
            for (int[] index : indices) {
                row[index[0]] ^= true;
                column[index[1]] ^= true;
            }
            int oddNumberCount = 0;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < m; j++) {
                    oddNumberCount += row[i] ^ column[j] ? 1 : 0;
                }
            }
            return oddNumberCount;
        }
    }
}
```

814. `_1260.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

/**
 * 1260. Shift 2D Grid
 *
 * Given a 2D grid of size m x n and an integer k. You need to shift the grid k times.
 *
 * In one shift operation:
 * Element at grid[i][j] becomes at grid[i][j + 1].
 * Element at grid[i][n - 1] becomes at grid[i + 1][0].
 * Element at grid[n - 1][n - 1] becomes at grid[0][0].
 * Return the 2D grid after applying shift operation k times.
 *
 * Example 1:
 * Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 1
 * Output: [[9,1,2],[3,4,5],[6,7,8]]
 *
 * Example 2:
 * Input: grid = [[3,8,1,9],[19,7,2,5],[4,6,11,10],[12,0,21,13]], k = 4
 * Output: [[12,0,21,13],[3,8,1,9],[19,7,2,5],[4,6,11,10]]
 *
 * Example 3:
 * Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 9
 * Output: [[1,2,3],[4,5,6],[7,8,9]]
 *
 * Constraints:
 * m == grid.length
 * n == grid[i].length
 * 1 <= m <= 50
 * 1 <= n <= 50
 * -1000 <= grid[i][j] <= 1000
 * 0 <= k <= 100
 * */
public class _1260 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/shift-2d-grid/discuss/431102/JavaPython-
3-simple-code-using-mod*/
        public List<List<Integer>> shiftGrid(int[][] grid, int k) {
            int m = grid.length;
            int n = grid[0].length;
            int totalNumbers = m * n;
            int start = totalNumbers - k % totalNumbers;
            LinkedList<List<Integer>> result = new LinkedList<>();
            for (int i = start; i < totalNumbers + start; i++) {
                int moveIndex = i % totalNumbers;
                int moveRow = moveIndex / n;
                int moveColumn = moveIndex % n;
```

```
                if ((i - start) % n == 0) {
                    result.add(new ArrayList<>());
                }
                result.peekLast().add(grid[moveRow][moveColumn]);
            }
            return result;
        }
    }
}
```

815. `_1266.java`

```
package com.fishercoder.solutions;

/**
 * 1266. Minimum Time Visiting All Points
 *
 * On a plane there are n points with integer coordinates points[i] = [xi, yi].
 * Your task is to find the minimum time in seconds to visit all points.
 *
 * You can move according to the next rules:
 * In one second always you can either move vertically, horizontally by one unit or
diagonally
 * (it means to move one unit vertically and one unit horizontally in one second).
 * You have to visit the points in the same order as they appear in the array.
 *
 * Example 1:
 * Input: points = [[1,1],[3,4],[-1,0]]
 * Output: 7
 * Explanation: One optimal path is [1,1] -> [2,2] -> [3,3] -> [3,4] -> [2,3] -> [1,2] ->
[0,1] -> [-1,0]
 * Time from [1,1] to [3,4] = 3 seconds
 * Time from [3,4] to [-1,0] = 4 seconds
 * Total time = 7 seconds
 *
 * Example 2:
 * Input: points = [[3,2],[-2,2]]
 * Output: 5
 *
 * Constraints:
 * points.length == n
 * 1 <= n <= 100
 * points[i].length == 2
 * -1000 <= points[i][0], points[i][1] <= 1000
 * */
public class _1266 {
    public static class Solution1 {
        /**
         * Time: O(n)
         * Space: O(1)
         *
         * credit: https://leetcode.com/problems/minimum-time-visiting-all-
points/discuss/436142/Sum-of-Chebyshev-distance-between-two-consecutive-points
         * */
        public int minTimeToVisitAllPoints(int[][] points) {
            int minTime = 0;
            for (int i = 0; i < points.length - 1; i++) {
                minTime += chebyshevDistance(points[i], points[i + 1]);
            }
            return minTime;
        }

        private int chebyshevDistance(int[] pointA, int[] pointB) {
```

```
            return Math.max(Math.abs(pointA[0] - pointB[0]), Math.abs(pointA[1] -
pointB[1]));
        }
    }
}
```

816. `_1267.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 1267. Count Servers that Communicate
 *
 * You are given a map of a server center, represented as a m * n integer matrix grid,
where 1 means that on that cell there is a server and 0 means that it is no server.
 * Two servers are said to communicate if they are on the same row or on the same column.
 *
 * Return the number of servers that communicate with any other server.
 *
 * Example 1:
 * Input: grid = [[1,0],[0,1]]
 * Output: 0
 * Explanation: No servers can communicate with others.
 *
 * Example 2:
 * Input: grid = [[1,0],[1,1]]
 * Output: 3
 * Explanation: All three servers can communicate with at least one other server.
 *
 * Example 3:
 * Input: grid = [[1,1,0,0],[0,0,1,0],[0,0,1,0],[0,0,0,1]]
 * Output: 4
 * Explanation: The two servers in the first row can communicate with each other.
 * The two servers in the third column can communicate with each other. The server at
right bottom corner can't communicate with any other server.
 *
 * Constraints:
 * m == grid.length
 * n == grid[i].length
 * 1 <= m <= 250
 * 1 <= n <= 250
 * grid[i][j] == 0 or 1
 * */
public class _1267 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/count-servers-that-
communicate/discuss/436188/Java-or-Clean-And-Simple-or-Beats-100*/
        public int countServers(int[][] grid) {
            int m = grid.length;
            int n = grid[0].length;
            int[] rowCount = new int[m];
            int[] columnCount = new int[n];
            int total = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        rowCount[i]++;
```

```
                        columnCount[j]++;
                        total++;
                    }
                }
            }
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1 && rowCount[i] == 1 && columnCount[j] == 1) {
                        total--;
                    }
                }
            }
            return total;
        }
    }
}
```

817. `_1271.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 1271. Hexspeak
 *
 * A decimal number can be converted to its Hexspeak representation by first converting
it to an uppercase hexadecimal string,
 * then replacing all occurrences of the digit 0 with the letter O, and the digit 1 with
the letter I.
 * Such a representation is valid if and only if it consists only of the letters in the
set {"A", "B", "C", "D", "E", "F", "I", "O"}.
 * Given a string num representing a decimal integer N, return the Hexspeak
representation of N if it is valid, otherwise return "ERROR".
 *
 * Example 1:
 * Input: num = "257"
 * Output: "IOI"
 * Explanation:  257 is 101 in hexadecimal.
 *
 * Example 2:
 * Input: num = "3"
 * Output: "ERROR"
 *
 * Constraints:
 * 1 <= N <= 10^12
 * There are no leading zeros in the given string.
 * All answers must be in uppercase letters.
 * */
public class _1271 {
    public static class Solution1 {
        public String toHexspeak(String num) {
            long numInt = Long.parseLong(num);
            String hexString = Long.toHexString(numInt);
            StringBuilder sb = new StringBuilder();
            Set<Character> set = new HashSet<>(Arrays.asList('A', 'B', 'C', 'D', 'E',
'F', '1', '0', 'a', 'b', 'c', 'd', 'e', 'f'));
            for (char c : hexString.toCharArray()) {
                if (!set.contains(c)) {
                    return "ERROR";
                } else if (c == '1') {
                    sb.append("I");
                } else if (c == '0') {
                    sb.append("O");
                } else {
                    sb.append(Character.toUpperCase(c));
                }
            }
```

```
            return sb.toString();
        }
    }
}
```

818. `_1277.java`

```
package com.fishercoder.solutions;

/**
 * 1277. Count Square Submatrices with All Ones
 *
 * Given a m * n matrix of ones and zeros, return how many square submatrices have all
ones.
 *
 * Example 1:
 * Input: matrix =
 * [
 *   [0,1,1,1],
 *   [1,1,1,1],
 *   [0,1,1,1]
 * ]
 * Output: 15
 * Explanation:
 * There are 10 squares of side 1.
 * There are 4 squares of side 2.
 * There is  1 square of side 3.
 * Total number of squares = 10 + 4 + 1 = 15.
 *
 * Example 2:
 * Input: matrix =
 * [
 *   [1,0,1],
 *   [1,1,0],
 *   [1,1,0]
 * ]
 * Output: 7
 * Explanation:
 * There are 6 squares of side 1.
 * There is 1 square of side 2.
 * Total number of squares = 6 + 1 = 7.
 *
 * Constraints:
 * 1 <= arr.length <= 300
 * 1 <= arr[0].length <= 300
 * 0 <= arr[i][j] <= 1
 * */
public class _1277 {
    public static class Solution1 {
        /**
         * In-place solution.
         *
         * credit: https://leetcode.com/problems/count-square-submatrices-with-all-
ones/discuss/441306/Python-DP-solution
         * */
        public int countSquares(int[][] matrix) {
            int m = matrix.length;
            int n = matrix[0].length;
```

```
            int count = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (matrix[i][j] > 0 && i > 0 && j > 0) {
                        matrix[i][j] = Math.min(matrix[i - 1][j - 1], Math.min(matrix[i - 1][j], matrix[i][j - 1])) + 1;
                    }
                    count += matrix[i][j];
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        /**
         * Use m*n extra space solution.
         *
         * credit: https://leetcode.com/problems/count-square-submatrices-with-all-ones/discuss/441312/Java-Simple-DP-solution
         */
        public int countSquares(int[][] matrix) {
            int m = matrix.length;
            int n = matrix[0].length;
            int[][] dp = new int[m][n];
            for (int i = 0; i < m; i++) {
                if (matrix[i][0] == 1) {
                    dp[i][0] = 1;
                }
            }
            for (int j = 0; j < n; j++) {
                if (matrix[0][j] == 1) {
                    dp[0][j] = 1;
                }
            }
            for (int i = 1; i < m; i++) {
                for (int j = 1; j < n; j++) {
                    if (matrix[i][j] == 1) {
                        dp[i][j] = Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1])) + 1;
                    }
                }
            }
            int count = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    count += dp[i][j];
                }
            }
            return count;
        }
    }
}
```

819. `_1281.java`

```
package com.fishercoder.solutions;

/**
 * 1281. Subtract the Product and Sum of Digits of an Integer
 *
 * Given an integer number n, return the difference between the product of its digits and
the sum of its digits.
 *
 * Example 1:
 * Input: n = 234
 * Output: 15
 * Explanation:
 * Product of digits = 2 * 3 * 4 = 24
 * Sum of digits = 2 + 3 + 4 = 9
 * Result = 24 - 9 = 15
 *
 * Example 2:
 * Input: n = 4421
 * Output: 21
 * Explanation:
 * Product of digits = 4 * 4 * 2 * 1 = 32
 * Sum of digits = 4 + 4 + 2 + 1 = 11
 * Result = 32 - 11 = 21
 *
 * Constraints:
 * 1 <= n <= 10^5
 * */
public class _1281 {
    public static class Solution1 {
        public int subtractProductAndSum(int n) {
            int sum = 0;
            int product = 1;
            while (n > 0) {
                int digit = n % 10;
                n /= 10;
                sum += digit;
                product *= digit;
            }
            return product - sum;
        }
    }
}
```

820. `_1282.java`

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 1282. Group the People Given the Group Size They Belong To
 *
 * There are n people whose IDs go from 0 to n - 1 and each person belongs exactly to one
group.
 * Given the array groupSizes of length n telling the group size each person belongs to,
return the groups there are and the people's IDs each group includes.
 * You can return any solution in any order and the same applies for IDs.
 * Also, it is guaranteed that there exists at least one solution.
 *
 * Example 1:
 * Input: groupSizes = [3,3,3,3,3,1,3]
 * Output: [[5],[0,1,2],[3,4,6]]
 * Explanation:
 * Other possible solutions are [[2,1,6],[5],[0,4,3]] and [[5],[0,6,2],[4,3,1]].
 *
 * Example 2:
 * Input: groupSizes = [2,1,3,3,3,2]
 * Output: [[1],[0,5],[2,3,4]]
 *
 * Constraints:
 * groupSizes.length == n
 * 1 <= n <= 500
 * 1 <= groupSizes[i] <= n
 * */
public class _1282 {
    public static class Solution1 {
        public List<List<Integer>> groupThePeople(int[] groupSizes) {
            Map<Integer, List<Integer>> map = new HashMap<>();
            for (int i = 0; i < groupSizes.length; i++) {
                List<Integer> list = map.getOrDefault(groupSizes[i], new ArrayList<>());
                list.add(i);
                map.put(groupSizes[i], list);
            }
            List<List<Integer>> result = new ArrayList<>();
            for (int key : map.keySet()) {
                List<Integer> list = map.get(key);
                int i = 0;
                do {
                    result.add(list.subList(i, i + key));
                    i += key;
                } while (i + key <= list.size());
            }
            return result;
```

```
        }
    }
}
```

821. `_1287.java`

```
package com.fishercoder.solutions;

/**
 * 1287. Element Appearing More Than 25% In Sorted Array
 *
 * Given an integer array sorted in non-decreasing order, there is exactly one integer in
the array that occurs more than 25% of the time.
 * Return that integer.
 *
 * Example 1:
 * Input: arr = [1,2,2,6,6,6,6,7,10]
 * Output: 6
 *
 * Constraints:
 *
 * 1 <= arr.length <= 10^4
 * 0 <= arr[i] <= 10^5
 * */
public class _1287 {
    public static class Solution1 {
        public int findSpecialInteger(int[] arr) {
            int quarter = arr.length / 4;
            for (int i = 0; i < arr.length - quarter; i++) {
                if (arr[i] == arr[i + quarter]) {
                    return arr[i];
                }
            }
            return -1;
        }
    }
}
```

822. `_1290.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 1290. Convert Binary Number in a Linked List to Integer
 *
 * Given head which is a reference node to a singly-linked list.
 * The value of each node in the linked list is either 0 or 1.
 * The linked list holds the binary representation of a number.
 *
 * Return the decimal value of the number in the linked list.
 *
 * Example 1:
 * Input: head = [1,0,1]
 * Output: 5
 * Explanation: (101) in base 2 = (5) in base 10
 *
 * Example 2:
 * Input: head = [0]
 * Output: 0
 *
 * Example 3:
 * Input: head = [1]
 * Output: 1
 *
 * Example 4:
 * Input: head = [1,0,0,1,0,0,1,1,1,0,0,0,0,0,0]
 * Output: 18880
 *
 * Example 5:
 * Input: head = [0,0]
 * Output: 0
 *
 * Constraints:
 *
 * The Linked List is not empty.
 * Number of nodes will not exceed 30.
 * Each node's value is either 0 or 1.
 * */
public class _1290 {
    public static class Solution1 {
        public int getDecimalValue(ListNode head) {
            StringBuilder sb = new StringBuilder();
            while (head != null) {
                sb.append(head.val);
                head = head.next;
            }
            return Integer.parseInt(sb.toString(), 2);
        }
```

```
    }
}
```

823. `_1295.java`

```
package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1295. Find Numbers with Even Number of Digits
 *
 * Given an array nums of integers, return how many of them contain an even number of
digits.
 *
 * Example 1:
 * Input: nums = [12,345,2,6,7896]
 * Output: 2
 * Explanation:
 * 12 contains 2 digits (even number of digits).
 * 345 contains 3 digits (odd number of digits).
 * 2 contains 1 digit (odd number of digits).
 * 6 contains 1 digit (odd number of digits).
 * 7896 contains 4 digits (even number of digits).
 * Therefore only 12 and 7896 contain an even number of digits.
 *
 * Example 2:
 * Input: nums = [555,901,482,1771]
 * Output: 1
 * Explanation:
 * Only 1771 contains an even number of digits.
 *
 * Constraints:
 * 1 <= nums.length <= 500
 * 1 <= nums[i] <= 10^5
 * */
public class _1295 {
    public static class Solution1 {
        public int findNumbers(int[] nums) {
            int count = 0;
            for (int num : nums) {
                if (String.valueOf(num).length() % 2 == 0) {
                    count++;
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        public int findNumbers(int[] nums) {
            return (int) Arrays.stream(nums).filter(num -> String.valueOf(num).length() %
2 == 0).count();
        }
    }
}
```

824. `_1296.java`

```
package com.fishercoder.solutions;

import java.util.TreeMap;

/**
 * 1296. Divide Array in Sets of K Consecutive Numbers
 *
 * Given an array of integers nums and a positive integer k,
 * find whether it's possible to divide this array into sets of k consecutive numbers
 * Return True if its possible otherwise return False.
 *
 * Example 1:
 * Input: nums = [1,2,3,3,4,4,5,6], k = 4
 * Output: true
 * Explanation: Array can be divided into [1,2,3,4] and [3,4,5,6].
 *
 * Example 2:
 * Input: nums = [3,2,1,2,3,4,3,4,5,9,10,11], k = 3
 * Output: true
 * Explanation: Array can be divided into [1,2,3] , [2,3,4] , [3,4,5] and [9,10,11].
 *
 * Example 3:
 * Input: nums = [3,3,2,2,1,1], k = 3
 * Output: true
 *
 * Example 4:
 * Input: nums = [1,2,3,4], k = 3
 * Output: false
 * Explanation: Each array should be divided in subarrays of size 3.
 *
 * Constraints:
 * 1 <= nums.length <= 10^5
 * 1 <= nums[i] <= 10^9
 * 1 <= k <= nums.length
 * */
public class _1296 {
    public static class Solution1 {
        public boolean isPossibleDivide(int[] nums, int k) {
            if (nums.length % k != 0) {
                return false;
            }
            TreeMap<Integer, Integer> treeMap = new TreeMap();
            int min = nums[0];
            for (int num : nums) {
                treeMap.put(num, treeMap.getOrDefault(num, 0) + 1);
                min = Math.min(min, num);
            }
            while (!treeMap.isEmpty()) {
                if (!isConsecutiveK(treeMap, min, k)) {
                    return false;
                }
```

```
                min = findNextMin(treeMap);
                if (min == Integer.MIN_VALUE) {
                    break;
                }
            }
            return true;
        }

        private int findNextMin(TreeMap<Integer, Integer> treeMap) {
            return treeMap.isEmpty() ? Integer.MIN_VALUE :
treeMap.entrySet().iterator().next().getKey();
        }

        private boolean isConsecutiveK(TreeMap<Integer, Integer> treeMap, int min, int k)
{
            int count = treeMap.get(min);
            treeMap.remove(min);
            for (int i = 1; i < k; i++) {
                int key = min + i;
                if (!treeMap.containsKey(key) || treeMap.get(key) < count) {
                    return false;
                } else if (treeMap.get(key) > count) {
                    treeMap.put(key, treeMap.get(key) - count);
                } else {
                    treeMap.remove(key);
                }
            }
            return true;
        }
    }
}
```

825. `_1297.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 1297. Maximum Number of Occurrences of a Substring
 *
 * Given a string s, return the maximum number of ocurrences of any substring under the
following rules:
 *
 * The number of unique characters in the substring must be less than or equal to
maxLetters.
 * The substring size must be between minSize and maxSize inclusive.
 *
 * Example 1:
 * Input: s = "aababcaab", maxLetters = 2, minSize = 3, maxSize = 4
 * Output: 2
 * Explanation: Substring "aab" has 2 ocurrences in the original string.
 * It satisfies the conditions, 2 unique letters and size 3 (between minSize and
maxSize).
 *
 * Example 2:
 * Input: s = "aaaa", maxLetters = 1, minSize = 3, maxSize = 3
 * Output: 2
 * Explanation: Substring "aaa" occur 2 times in the string. It can overlap.
 *
 * Example 3:
 * Input: s = "aabcabcab", maxLetters = 2, minSize = 2, maxSize = 3
 * Output: 3
 *
 * Example 4:
 * Input: s = "abcde", maxLetters = 2, minSize = 3, maxSize = 3
 * Output: 0
 *
 * Constraints:
 * 1 <= s.length <= 10^5
 * 1 <= maxLetters <= 26
 * 1 <= minSize <= maxSize <= min(26, s.length)
 * s only contains lowercase English letters.
 * */
public class _1297 {
    public static class Solution1 {
        public int maxFreq(String s, int maxLetters, int minSize, int maxSize) {
            int maxFreq = 0;
            for (int size = minSize; size <= maxSize; size++) {
                maxFreq = Math.max(maxFreq, getMaxFreqWithThisSize(s, maxLetters, size));
            }
            return maxFreq;
```

```
        }

        private int getMaxFreqWithThisSize(String str, int maxLetters, int size) {
            Map<String, Integer> map = new HashMap<>();
            for (int i = 0; i <= str.length() - size; i++) {
                String substring = str.substring(i, i + size);
                Set<Character> set = new HashSet<>();
                for (int j = 0; j < substring.length(); j++) {
                    set.add(substring.charAt(j));
                    if (set.size() > maxLetters) {
                        break;
                    }
                }
                if (set.size() <= maxLetters) {
                    map.put(substring, map.getOrDefault(substring, 0) + 1);
                }
            }
            int max = 0;
            for (String key : map.keySet()) {
                max = Math.max(max, map.get(key));
            }
            return max;
        }
    }
}
```

826. `_1299.java`

```
package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 1299. Replace Elements with Greatest Element on Right Side
 *
 * Given an array arr, replace every element in that array with the greatest element
among the elements to its right, and replace the last element with -1.
 * After doing so, return the array.
 *
 * Example 1:
 * Input: arr = [17,18,5,4,6,1]
 * Output: [18,6,6,6,1,-1]
 * */
public class _1299 {
    public static class Solution1 {
        public int[] replaceElements(int[] arr) {
            PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> b - a);
            for (int num : arr) {
                heap.offer(num);
            }
            int[] result = new int[arr.length];
            for (int i = 0; i < arr.length - 1; i++) {
                heap.remove(arr[i]);
                result[i] = heap.peek();
            }
            result[arr.length - 1] = -1;
            return result;
        }
    }
}
```

827. `_1300.java`

```
package com.fishercoder.solutions;

/**
 * 1300. Sum of Mutated Array Closest to Target
 *
 * Given an integer array arr and a target value target, return the integer value such
that when we change all the integers larger than value in the given array to be equal to
value,
 * the sum of the array gets as close as possible (in absolute difference) to target.
 * In case of a tie, return the minimum such integer.
 * Notice that the answer is not neccesarilly a number from arr.
 *
 * Example 1:
 * Input: arr = [4,9,3], target = 10
 * Output: 3
 * Explanation: When using 3 arr converts to [3, 3, 3] which sums 9 and that's the
optimal answer.
 *
 * Example 2:
 * Input: arr = [2,3,5], target = 10
 * Output: 5
 *
 * Example 3:
 * Input: arr = [60864,25176,27249,21296,20204], target = 56803
 * Output: 11361
 * */
public class _1300 {
    public static class Solution1 {
        public int findBestValue(int[] arr, int target) {
            int ave = target / arr.length;
            int min = findMin(arr);
            int max = findMax(arr);
            //if ave is the best value, what's the difference to target?
            int closetDiff = findClosestDiffIfReplaceWithVal(arr, ave, target);
            int bestValue = ave;
            //extend candidate towards the right to see how close the sum could be to
target
            int candidateOnTheRight = ave;
            while (candidateOnTheRight <= max) {
                int thisOne = findClosestDiffIfReplaceWithVal(arr, ++candidateOnTheRight,
target);
                if (thisOne >= closetDiff) {
                    break;
                } else {
                    closetDiff = thisOne;
                    bestValue = candidateOnTheRight;
                }
            }

            //extend candidate towards the left to see how close the sum could be to
target
```

```
            int candidateOnTheLeft = ave;
            while (candidateOnTheLeft >= min) {
                int thisOne = findClosestDiffIfReplaceWithVal(arr, --candidateOnTheLeft,
target);
                if (thisOne >= closetDiff) {
                    break;
                } else {
                    closetDiff = thisOne;
                    bestValue = candidateOnTheLeft;
                }
            }
            return bestValue;
        }

        private int findClosestDiffIfReplaceWithVal(int[] arr, int replaceValue, int
target) {
            int sum = 0;
            for (int i = 0; i < arr.length; i++) {
                if (arr[i] > replaceValue) {
                    sum += replaceValue;
                } else {
                    sum += arr[i];
                }
            }
            return Math.abs(sum - target);
        }

        private int findMax(int[] arr) {
            int max = arr[0];
            for (int i = 1; i < arr.length; i++) {
                max = Math.max(max, arr[i]);
            }
            return max;
        }

        private int findMin(int[] arr) {
            int min = arr[0];
            for (int i = 1; i < arr.length; i++) {
                min = Math.min(min, arr[i]);
            }
            return min;
        }
    }
}
```

828. `_1302.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 1302. Deepest Leaves Sum
 *
 * Given a binary tree, return the sum of values of its deepest leaves.
 *
 * Example 1:
 *                1
 *               / \
 *              2   3
 *             / \   \
 *            4  5   6
 *           /        \
 *          7         8
 *
 * Input: root = [1,2,3,4,5,null,6,7,null,null,null,null,8]
 * Output: 15
 *
 * Constraints:
 * The number of nodes in the tree is between 1 and 10^4.
 * The value of nodes is between 1 and 100.
 * */
public class _1302 {
    public static class Solution1 {
        public int deepestLeavesSum(TreeNode root) {
            int depth = maxDepth(root);
            return bfs(root, depth);
        }

        private int bfs(TreeNode root, int depth) {
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            int currentLevel = 0;
            int sum = 0;
            while (!queue.isEmpty()) {
                int size = queue.size();
                currentLevel++;
                for (int i = 0; i < size; i++) {
                    TreeNode currNode = queue.poll();
                    if (currentLevel == depth) {
                        sum += currNode.val;
                    }
                    if (currNode.left != null) {
                        queue.offer(currNode.left);
                    }
```

```
                    if (currNode.right != null) {
                        queue.offer(currNode.right);
                    }
                }
            }
            return sum;
        }

        private int maxDepth(TreeNode root) {
            if (root == null) {
                return 0;
            }
            return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
        }
    }
}
```

829. `_1304.java`

```
package com.fishercoder.solutions;

/**
 * 1304. Find N Unique Integers Sum up to Zero
 *
 * Given an integer n, return any array containing n unique integers such that they add
up to 0.
 *
 * Example 1:
 * Input: n = 5
 * Output: [-7,-1,1,3,4]
 * Explanation: These arrays also are accepted [-5,-1,1,2,3] , [-3,-1,2,-2,4].
 *
 * Example 2:
 * Input: n = 3
 * Output: [-1,0,1]
 *
 * Example 3:
 * Input: n = 1
 * Output: [0]
 * */
public class _1304 {
    public static class Solution1 {
        public int[] sumZero(int n) {
            int[] result = new int[n];
            int start = -n / 2;
            for (int i = 0; i < n / 2; i++) {
                result[i] = start++;
            }
            if (n % 2 == 0) {
                start++;
            }
            for (int i = n / 2; i < n; i++) {
                result[i] = start++;
            }
            return result;
        }
    }
}
```

830. `_1305.java`

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 1305. All Elements in Two Binary Search Trees
 *
 * Given two binary search trees root1 and root2.
 * Return a list containing all the integers from both trees sorted in ascending order.
 *
 * Example 1:
 * Input: root1 = [2,1,4], root2 = [1,0,3]
 * Output: [0,1,1,2,3,4]
 *
 * Example 2:
 * Input: root1 = [0,-10,10], root2 = [5,1,7,0,2]
 * Output: [-10,0,0,1,2,5,7,10]
 *
 * Example 3:
 * Input: root1 = [], root2 = [5,1,7,0,2]
 * Output: [0,1,2,5,7]
 *
 * Example 4:
 * Input: root1 = [0,-10,10], root2 = []
 * Output: [-10,0,10]
 *
 * Example 5:
 * Input: root1 = [1,null,8], root2 = [8,1]
 * Output: [1,1,8,8]
 *
 * Constraints:
 * Each tree has at most 5000 nodes.
 * Each node's value is between [-10^5, 10^5].*/
public class _1305 {
    public static class Solution1 {
        public List<Integer> getAllElements(TreeNode root1, TreeNode root2) {
            List<Integer> list1 = getAllNodes(root1);
            List<Integer> list2 = getAllNodes(root2);
            List<Integer> merged = new ArrayList<>();
            merged.addAll(list1);
            merged.addAll(list2);
            Collections.sort(merged);
            return merged;
        }

        private List<Integer> getAllNodes(TreeNode root) {
            List<Integer> list = new ArrayList<>();
```

```
            return inorder(root, list);
        }

        List<Integer> inorder(TreeNode root, List<Integer> result) {
            if (root == null) {
                return result;
            }
            inorder(root.left, result);
            result.add(root.val);
            return inorder(root.right, result);
        }
    }
}
```

831. `_1309.java`

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1309. Decrypt String from Alphabet to Integer Mapping
 *
 * Given a string s formed by digits ('0' - '9') and '#' . We want to map s to English
lowercase characters as follows:
 * Characters ('a' to 'i') are represented by ('1' to '9') respectively.
 * Characters ('j' to 'z') are represented by ('10#' to '26#') respectively.
 * Return the string formed after mapping.
 * It's guaranteed that a unique mapping will always exist.
 *
 * Example 1:
 * Input: s = "10#11#12"
 * Output: "jkab"
 * Explanation: "j" -> "10#" , "k" -> "11#" , "a" -> "1" , "b" -> "2".
 *
 * Example 2:
 * Input: s = "1326#"
 * Output: "acz"
 *
 * Example 3:
 * Input: s = "25#"
 * Output: "y"
 *
 * Example 4:
 * Input: s = "12345678910#11#12#13#14#15#16#17#18#19#20#21#22#23#24#25#26#"
 * Output: "abcdefghijklmnopqrstuvwxyz"
 *
 * Constraints:
 * 1 <= s.length <= 1000
 * s[i] only contains digits letters ('0'-'9') and '#' letter.
 * s will be valid string such that mapping is always possible.
 * */
public class _1309 {
    public static class Solution1 {
        //TODO: very silly solution, optimze it
        public String freqAlphabets(String s) {
            Map<String, String> map = new HashMap<>();
            map.put("1", "a");
            map.put("2", "b");
            map.put("3", "c");
            map.put("4", "d");
            map.put("5", "e");
            map.put("6", "f");
            map.put("7", "g");
            map.put("8", "h");
            map.put("9", "i");
```

```
            map.put("10#", "j");
            map.put("11#", "k");
            map.put("12#", "l");
            map.put("13#", "m");
            map.put("14#", "n");
            map.put("15#", "o");
            map.put("16#", "p");
            map.put("17#", "q");
            map.put("18#", "r");
            map.put("19#", "s");
            map.put("20#", "t");
            map.put("21#", "u");
            map.put("22#", "v");
            map.put("23#", "w");
            map.put("24#", "x");
            map.put("25#", "y");
            map.put("26#", "z");
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < s.length(); ) {
                if (Integer.parseInt("" + s.charAt(i)) == 1 && i + 1 < s.length() && i +
2 < s.length() && s.charAt(i + 2) == '#') {
                    sb.append(map.get(s.substring(i, i + 3)));
                    i += 3;
                } else if (Integer.parseInt("" + s.charAt(i)) == 2 && i + 1 < s.length()
&& i + 2 < s.length() && s.charAt(i + 2) == '#') {
                    sb.append(map.get(s.substring(i, i + 3)));
                    i += 3;
                } else {
                    sb.append(map.get("" + s.charAt(i)));
                    i++;
                }
            }
            return sb.toString();
        }
    }
}
```

832. `_1317.java`

```
package com.fishercoder.solutions;

/**
 * 1317. Convert Integer to the Sum of Two No-Zero Integers
 *
 * Given an integer n. No-Zero integer is a positive integer which doesn't contain any 0
in its decimal representation.
 *
 * Return a list of two integers [A, B] where:
 * A and B are No-Zero integers.
 * A + B = n
 * It's guarateed that there is at least one valid solution. If there are many valid
solutions you can return any of them.
 *
 * Example 1:
 * Input: n = 2
 * Output: [1,1]
 * Explanation: A = 1, B = 1. A + B = n and both A and B don't contain any 0 in their
decimal representation.
 *
 * Example 2:
 * Input: n = 11
 * Output: [2,9]
 *
 * Example 3:
 * Input: n = 10000
 * Output: [1,9999]
 *
 * Example 4:
 * Input: n = 69
 * Output: [1,68]
 *
 * Example 5:
 * Input: n = 1010
 * Output: [11,999]
 *
 * Constraints:
 * 2 <= n <= 10^4
 * */
public class _1317 {
    public static class Solution1 {
        public int[] getNoZeroIntegers(int n) {
            int left = 1;
            int right = n - 1;
            while (left <= right) {
                if (noZero(left) && noZero(right)) {
                    return new int[]{left, right};
                } else {
                    left++;
                    right--;
                }
```

```
            }
            return null;
        }

        private boolean noZero(int num) {
            while (num != 0) {
                if (num % 10 == 0) {
                    return false;
                } else {
                    num /= 10;
                }
            }
            return true;
        }
    }
}
```

833. `Combined_Java_Source.java`

```
// ============================================================
// FILE 1: _1.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1. Two Sum
 *
 * Given an array of integers, return indices of the two numbers such that they add up to
a specific target.
 * You may assume that each input would have exactly one solution, and you may not use
the same element twice.
 *
 * Example:
 * Given nums = [2, 7, 11, 15], target = 9,
 * Because nums[0] + nums[1] = 2 + 7 = 9,
 * return [0, 1].
 */
public class _1 {

    public static class Solution1 {
        public int[] twoSum(int[] nums, int target) {
            Map<Integer, Integer> map = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(target - nums[i])) {
                    return new int[]{map.get(target - nums[i]), i};
                } else {
                    map.put(nums[i], i);
                }
            }
            return new int[]{-1, -1};
        }
    }

}

// ============================================================
// FILE 2: _2.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 2. Add Two Numbers
```

```
 You are given two linked lists representing two non-negative numbers.
 The digits are stored in reverse order and each of their nodes contain a single digit.
 Add the two numbers and return it as a linked list.

Input: (2 -> 4 -> 3) + (5 -> 6 -> 4)
Output: 7 -> 0 -> 8
 */

public class _2 {
    public static class Solution1 {
        public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
            ListNode result = new ListNode(0);
            ListNode tmp = result;
            int sum = 0;
            while (l1 != null || l2 != null) {
                sum /= 10;
                if (l1 != null) {
                    sum += l1.val;
                    l1 = l1.next;
                }
                if (l2 != null) {
                    sum += l2.val;
                    l2 = l2.next;
                }
                tmp.next = new ListNode(sum % 10);
                tmp = tmp.next;
            }
            if (sum / 10 == 1) {
                tmp.next = new ListNode(1);//this means there's a carry, so we add
additional 1, e.g. [5] + [5] = [0, 1]
            }
            return result.val == 0 ? result.next : result;
        }
    }

}

// ============================================================
// FILE 3: _3.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 3. Longest Substring Without Repeating Characters
 *
 * Given a string, find the length of the longest substring without repeating characters.
```

```
 Examples:

 Given "abcabcbb", the answer is "abc", which the length is 3.

 Given "bbbbb", the answer is "b", with the length of 1.

 Given "pwwkew", the answer is "wke", with the length of 3.

 Note that the answer must be a substring, "pwke" is a subsequence and not a substring.
 */

public class _3 {

    public static class Solution1 {
        public int lengthOfLongestSubstring(String s) {
            int result = 0;
            Map<Character, Integer> map = new HashMap();
            for (int i = 0, j = i; j < s.length(); ) {
                if (!map.containsKey(s.charAt(j)) || (map.containsKey(s.charAt(j)) &&
map.get(s.charAt(j)) == 0)) {
                    map.put(s.charAt(j), 1);
                    result = Math.max(j - i + 1, result);
                    j++;
                } else {
                    map.put(s.charAt(i), map.get(s.charAt(i)) - 1);
                    i++;
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * Sliding Window
         * O(n) time
         * O(min(m,n)) or O(k) space
         */
        public int lengthOfLongestSubstring(String s) {
            int n = s.length();
            Set<Character> set = new HashSet<>();
            int result = 0;
            int i = 0;
            int j = 0;
            while (i < n && j < n) {
                /**Try to extend the range i, j*/
                if (!set.contains(s.charAt(j))) {
                    set.add(s.charAt(j++));
                    result = Math.max(result, j - i);
                } else {
                    set.remove(s.charAt(i++));
                }
            }
            return result;
```

```
        }
    }

    public static class Solution3 {
        /**
         * Sliding Window
         * O(n) time
         * O(n) space
         */
        public int lengthOfLongestSubstring(String s) {
            if (s.length() == 0) {
                return 0;
            }
            int max = 0;
            Map<Character, Integer> map = new HashMap<>();
            /**Try to extend the range (i, j)*/
            for (int i = 0, j = 0; i < s.length(); i++) {
                if (map.containsKey(s.charAt(i))) {
                    j = Math.max(j, map.get(s.charAt(i)) + 1);
                }
                map.put(s.charAt(i), i);
                max = Math.max(max, i + 1 - j);
            }
            return max;
        }
    }

    public static class Solution4 {
        /**
         * Sliding Window Optimized
         * O(n) time
         * O(n) space
         */
        public int lengthOfLongestSubstring(String s) {
            if (s.length() == 0) {
                return 0;
            }
            int max = 0;
            int[] index = new int[128];
            /**Try to extend the range (i, j)*/
            for (int i = 0, j = 0; j < s.length(); j++) {
                i = Math.max(index[s.charAt(j)], i);
                max = Math.max(max, j - i + 1);
                index[s.charAt(j)] = j + 1;
            }
            return max;
        }
    }
}

// ============================================================
// FILE 4: _4.java
// ============================================================
```

```
package com.fishercoder.solutions;

import static java.lang.Math.max;
import static java.lang.Math.min;

/**
 4. Median of Two Sorted Arrays

 There are two sorted arrays nums1 and nums2 of size m and n respectively.

 Find the median of the two sorted arrays. The overall run time complexity should be
O(log (m+n)).

 Example 1:
 nums1 = [1, 3]
 nums2 = [2]
 The median is 2.0

 Example 2:
 nums1 = [1, 2]
 nums2 = [3, 4]
 The median is (2 + 3)/2 = 2.5
 */
public class _4 {

    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/28602/concise-java-solution-based-
on-binary-search
         *
         * The key point of this problem is to ignore half part of A and B each step
recursively by comparing the median of remaining A and B:

         if (aMid < bMid) Keep [aRight + bLeft]
         else Keep [bRight + aLeft]

         As the following: time=O(log(m + n))
         */
        public double findMedianSortedArrays(int[] A, int[] B) {
            int m = A.length;
            int n = B.length;
            int l = (m + n + 1) / 2;
            int r = (m + n + 2) / 2;
            return (getkth(A, 0, B, 0, l) + getkth(A, 0, B, 0, r)) / 2.0;
        }

        public double getkth(int[] A, int aStart, int[] B, int bStart, int k) {
            if (aStart > A.length - 1) {
                return B[bStart + k - 1];
            }
            if (bStart > B.length - 1) {
                return A[aStart + k - 1];
            }
            if (k == 1) {
                return Math.min(A[aStart], B[bStart]);
```

```
            }

            int aMid = Integer.MAX_VALUE;
            int bMid = Integer.MAX_VALUE;
            if (aStart + k / 2 - 1 < A.length) {
                aMid = A[aStart + k / 2 - 1];
            }
            if (bStart + k / 2 - 1 < B.length) {
                bMid = B[bStart + k / 2 - 1];
            }

            if (aMid < bMid) {
                return getkth(A, aStart + k / 2, B, bStart, k - k / 2);// Check: aRight +
bLeft
            } else {
                return getkth(A, aStart, B, bStart + k / 2, k - k / 2);// Check: bRight +
aLeft
            }
        }
    }

    public static class Solution2 {
        /**
         * Reference: https://leetcode.com/discuss/28843/my-accepted-java-solution:
         * Basic Idea is very similar to K-selection. it's easier to understand if you
imagine this to be chopping off the last K elements from a total of len(A) + len(B)
elements,
         * where K = (len(A) + len(B))/2.
         * we want to remove K, but each time we can remove only at most K/2 elements,
         * because we can only be sure that these elements are not within the first
(len(A) + len(B)) -K elements.
         */
        public double findMedianSortedArrays(int[] nums1, int[] nums2) {
            int K = nums1.length + nums2.length;
            if (K % 2 == 0) {
                return (findMedianSortedArrays(nums1, nums2, (K - K / 2)) +
findMedianSortedArrays(nums1, nums2, (K - (K / 2 + 1)))) / 2;
            } else {
                return findMedianSortedArrays(nums1, nums2, K - (K / 2 + 1));
            }
        }

        // k is the number of elements to REMOVE, or "Chop off"
        public double findMedianSortedArrays(int[] A, int[] B, int K) {

            int lowA = 0;
            int lowB = 0;
            int highA = A.length;
            int highB = B.length;
            int midA;
            int midB;
            while (K > 0 && highA > 0 && highB > 0) {
                int chopA = max(1, min(K / 2, (highA) / 2));
                int chopB = max(1, min(K / 2, (highB) / 2));
```

```
                midA = highA - chopA;
                midB = highB - chopB;
                if (A[midA] < B[midB]) { // here A[0 .. midA] < B[midB], and we know that
B[0 .. midB-1] < B[midB], so B[midB..highB] can not possibly be within the first (len(A)
+ len(B) - K) elements, and can be safely removed.
                    highB = midB;
                    K = K - chopB;
                } else {
                    highA = midA;
                    K = K - chopA;
                }
            }

            if (highA == 0 && highB == 0) {
                return 0;
            }
            if (highA == 0) {
                return B[highB - 1 - K];
            }
            if (highB == 0) {
                return A[highA - 1 - K];
            }
            return max(A[highA - 1], B[highB - 1]);
        }
    }
}

// ============================================================
// FILE 5: _5.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 5. Longest Palindromic Substring
 *
 * Given a string s, find the longest palindromic substring in s. You may assume that the
maximum length of s is 1000.

 Example:
 Input: "babad"
 Output: "bab"
 Note: "aba" is also a valid answer.

 Example:
 Input: "cbbd"
 Output: "bb"
 */
public class _5 {

    public static class Solution1 {
        private int low;
        private int maxLen;
```

```
        public String longestPalindrome(String s) {
            int len = s.length();
            if (len < 2) {
                return s;
            }

            for (int i = 0; i < len - 1; i++) {
                extendPalindrome(s, i, i); // assume odd length, try to extend Palindrome
as possible
                extendPalindrome(s, i, i + 1); // assume even length.
            }
            return s.substring(low, low + maxLen);
        }

        private void extendPalindrome(String s, int left, int right) {
            while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right))
{
                left--;
                right++;
            }
            if (maxLen < right - left - 1) {
                low = left + 1;
                maxLen = right - left - 1;
            }
        }
    }
}

// ============================================================
// FILE 6: _6.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 6. ZigZag Conversion
 *
 * The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows
like this:
 * (you may want to display this pattern in a fixed font for better legibility)

 P   A   H   N
 A P L S I I G
 Y   I   R

 And then read line by line: "PAHNAPLSIIGYIR"
 Write the code that will take a string and make this conversion given a number of rows:

 string convert(string text, int nRows);
 convert("PAYPALISHIRING", 3) should return "PAHNAPLSIIGYIR".
 */

public class _6 {
```

```
        public static class Solution1 {
                public String convert(String s, int numRows) {
                        StringBuilder[] sb = new StringBuilder[numRows];
                        char[] c = s.toCharArray();
                        int len = s.length();
                        for (int i = 0; i < numRows; i++) {
                                sb[i] = new StringBuilder();//this is an important step
to initialize it
                        }
                        int i = 0;
                        while (i < len) {
                                for (int index = 0; index < numRows && i < len; index++)
{
                                        sb[index].append(c[i++]);// vertically down
                                }

                                for (int index = numRows - 2; index >= 1 && i < len;
index--) {
                                        /**Why it should start from numRows - 2? Think of
the example when numRows = 3
                                         the starting point of obliquely going up is 1,
which is numRows-2.*/
                                        sb[index].append(c[i++]);// obliquely up
                                }
                        }

                        for (i = 1; i < numRows; i++) {
                                sb[0].append(sb[i]);
                        }
                        return sb[0].toString();
                }
        }

}

// ============================================================
// FILE 7: _7.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 7. Reverse Integer

 Given a 32-bit signed integer, reverse digits of an integer.

 Example 1:
 Input: 123
 Output: 321

 Example 2:
 Input: -123
 Output: -321
```

```
 Example 3:
 Input: 120
 Output: 21

 Note:
 Assume we are dealing with an environment which could only store integers within the 32-
bit signed integer range: [−231,  231 − 1].
 For the purpose of this problem, assume that your function returns 0 when the reversed
integer overflows.
 */
public class _7 {

    public static class Solution1 {
        /**This solution is NOT meeting the problem requirement actually:
         * Assume we are dealing with an environment which could only store integers
within the 32-bit signed integer range: [−231,  231 − 1].*/
        public int reverse(int x) {
            long result = 0;
            while (x != 0) {
                result = result * 10 + x % 10;
                x /= 10;
                System.out.println("result = " + result);
                if (result > Integer.MAX_VALUE || result < Integer.MIN_VALUE) {
                    System.out.println("break out..");
                    return 0;
                }
            }
            return (int) result;
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/problems/reverse-integer/discuss/4060/My-
accepted-15-lines-of-code-for-Java*/
        public int reverse(int x) {
            int result = 0;
            while (x != 0) {
                int lastDigit = x % 10;
                int newResult = result * 10 + lastDigit;
                if (result != (newResult - lastDigit) / 10) {
                    return 0;
                }
                x /= 10;
                result = newResult;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 8: _8.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 8. String to Integer (atoi)
 *
 * Implement atoi to convert a string to an integer.
 *
 * Hint: Carefully consider all possible input cases. If you want a challenge, please do
not see below and ask yourself what are the possible input cases.
 * Notes: It is intended for this problem to be specified vaguely (ie, no given input
specs). You are responsible to gather all the input requirements up front.
 *
 * Requirements for atoi:
 * The function first discards as many whitespace characters as necessary until the first
non-whitespace character is found.
 * Then, starting from this character, takes an optional initial plus or minus sign
followed by as many numerical digits as possible, and interprets them as a numerical
value.
 * The string can contain additional characters after those that form the integral
number, which are ignored and have no effect on the behavior of this function.
 * If the first sequence of non-whitespace characters in str is not a valid integral
number,
 * or if no such sequence exists because either str is empty or it contains only
whitespace characters, no conversion is performed.
 * If no valid conversion could be performed, a zero value is returned. If the correct
value is out of the range of representable values, INT_MAX (2147483647) or INT_MIN
(-2147483648) is returned.*/

public class _8 {

        public static class Solution1 {
                /**
                 * four corner cases:
                 * 1. discards all leading zeroes
                 * 2. sign of the number
                 * 3. overflow
                 * 4. invalid input
                 * */
                public int myAtoi(String str) {
                        int pointer = 0;
                        int result = 0;
                        while (pointer < str.length() &&
Character.isWhitespace(str.charAt(pointer))) {
                                pointer++;
                        }
                        if (pointer == str.length()) {
                                return 0;
                        }
                        boolean negativeFlag = (str.charAt(pointer) == '-');
                        if (str.charAt(pointer) == '+' || str.charAt(pointer) == '-') {
                                pointer++;
                        }
                        for (; pointer < str.length(); pointer++) {
                                if (str.charAt(pointer) > '9' || str.charAt(pointer) <
```

```
'0') {
                                        break;
                                } else {
                                        int digit = str.charAt(pointer) - '0';
                                        if (!negativeFlag && result > (Integer.MAX_VALUE
- digit) / 10) {
                                                return Integer.MAX_VALUE;
                                        } else if (negativeFlag && result <
(Integer.MIN_VALUE + digit) / 10) {
                                                return Integer.MIN_VALUE;
                                        }
                                        result = result * 10 + (negativeFlag ? -digit :
digit);
                                }
                        }
                        return result;
                }
        }

}

// ============================================================
// FILE 9: _9.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 9. Palindrome Number
 *
 * Determine whether an integer is a palindrome. Do this without extra space.
 *
 * Some hints:
 *
 * Could negative integers be palindromes? (ie, -1)
 *
 * If you are thinking of converting the integer to string, note the restriction of using
extra space.
 *
 * You could also try reversing an integer. However, if you have solved the problem
"Reverse Integer", you know that the reversed integer might overflow. How would you
handle such case?
 *
 * There is a more generic way of solving this problem.
 */
public class _9 {

    /**credit: https://discuss.leetcode.com/topic/8090/9-line-accepted-java-code-without-
the-need-of-handling-overflow
     * reversing only half and then compare if they're equal.*/
    public static class Solution1 {
        public boolean isPalindrome(int x) {
            if (x < 0) {
                return false;
```

```
            } else if (x == 0) {
                return true;
            } else if (x % 10 == 0) {
                return false;
            }
            int reversed = 0;
            while (x > reversed) {
                int digit = x % 10;
                reversed *= 10;
                reversed += digit;
                x /= 10;
            }
            return (x == reversed || x == reversed / 10);
        }
    }

}

// ============================================================
// FILE 10: _10.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 10. Regular Expression Matching
 *
 * Implement regular expression matching with support for '.' and '*'.
 *
 * '.' Matches any single character.
 * '*' Matches zero or more of the preceding element.
 *
 * The matching should cover the entire input string (not partial).
 *
 * The function prototype should be:
 * bool isMatch(const char *s, const char *p)
 *
 * Some examples:
 * isMatch("aa","a") → false
 * isMatch("aa","aa") → true
 * isMatch("aaa","aa") → false
 * isMatch("aa", "a*") → true
 * isMatch("aa", ".*") → true
 * isMatch("ab", ".*") → true
 * isMatch("aab", "c*a*b") → true
 */

public class _10 {

    public static class Solution1 {
        public boolean isMatch(String s, String p) {
            if (s == null || p == null) {
                return false;
            }
```

```
            boolean[][] dp = new boolean[s.length() + 1][p.length() + 1];
            dp[0][0] = true;
            for (int i = 0; i < p.length(); i++) { //here's the p's length, not s's
                if (p.charAt(i) == '*' && dp[0][i - 1]) {
                    dp[0][i + 1] = true; //here's y axis should be i+1
                }
            }
            for (int i = 0; i < s.length(); i++) {
                for (int j = 0; j < p.length(); j++) {
                    if (p.charAt(j) == '.' || p.charAt(j) == s.charAt(i)) {
                        dp[i + 1][j + 1] = dp[i][j];
                    }
                    if (p.charAt(j) == '*') {
                        if (p.charAt(j - 1) != s.charAt(i) && p.charAt(j - 1) != '.') {
                            dp[i + 1][j + 1] = dp[i + 1][j - 1];
                        } else {
                            dp[i + 1][j + 1] = (dp[i + 1][j] || dp[i][j + 1] || dp[i + 1]
[j - 1]);
                        }
                    }
                }
            }
            return dp[s.length()][p.length()];
        }
    }

}

// ============================================================
// FILE 11: _11.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 11. Container With Most Water
 *
 * Given n non-negative integers a1, a2, ..., an,
 * where each represents a point at coordinate (i, ai).
 * n vertical lines are drawn such that the two endpoints of line i is at (i, ai) and (i,
0).
 * Find two lines, which together with x-axis forms a container, such that the container
contains the most water.

 Note: You may not slant the container and n is at least 2.
 */
public class _11 {

    public static class Solution1 {
        public int maxArea(int[] height) {
            int max = 0;
            int i = 0;
            int j = height.length - 1;
            while (i < j) {
```

```
                max = Math.max(Math.min(height[i], height[j]) * (j - i), max);
                if (height[i] <= height[j]) {
                    // we need to find the shorter one,
                    // then calculate its area
                    i++;
                } else {
                    j--;
                }
            }
            return max;
        }
    }

}

// ============================================================
// FILE 12: _12.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 12. Integer to Roman
 *
 * Given an integer, convert it to a roman numeral.
 * Input is guaranteed to be within the range from 1 to 3999.
 */
public class _12 {

    public static class Solution1 {
        public String intToRoman(int num) {
            String[] M = new String[]{"", "M", "MM", "MMM"};
            String[] C = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
            String[] X = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
            String[] I = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
            return M[num / 1000] + C[(num % 1000) / 100] + X[(num % 100) / 10] + I[num %
10];
        }
    }

}

// ============================================================
// FILE 13: _13.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 13. Roman to Integer
 *
```

```
 * Given a roman numeral, convert it to an integer.
 * Input is guaranteed to be within the range from 1 to 3999.
 * */

public class _13 {

    public static class Solution1 {
        public int romanToInt(String s) {
            Map<Character, Integer> map = new HashMap();
            map.put('I', 1);
            map.put('V', 5);
            map.put('X', 10);
            map.put('L', 50);
            map.put('C', 100);
            map.put('D', 500);
            map.put('M', 1000);

            int result = 0;
            for (int i = 0; i < s.length(); i++) {
                if (i > 0 && map.get(s.charAt(i)) > map.get(s.charAt(i - 1))) {
                    result += map.get(s.charAt(i)) - 2 * map.get(s.charAt(i - 1));
                } else {
                    result += map.get(s.charAt(i));
                }
            }
            return result;
        }
    }

}

// ============================================================
// FILE 14: _14.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 14. Longest Common Prefix
 *
 * Write a function to find the longest common prefix string amongst an array of strings.
 *
 * If there is no common prefix, return an empty string "".
 *
 * Example 1:
 * Input: ["flower","flow","flight"]
 * Output: "fl"
 *
 * Example 2:
 * Input: ["dog","racecar","car"]
 * Output: ""
 * Explanation: There is no common prefix among the input strings.
 *
 * Note:
```

```
 * All given inputs are in lowercase letters a-z.
 */

public class _14 {

    public static class Solution1 {
        //horizontal scan
        public String longestCommonPrefix(String[] strs) {
            if (strs.length == 0) {
                return "";
            }
            String prefix = strs[0];
            for (int i = 1; i < strs.length; i++) {
                while (strs[i].indexOf(prefix) != 0) {
                    prefix = prefix.substring(0, prefix.length() - 1);
                    if (prefix.isEmpty()) {
                        return "";
                    }
                }
            }
            return prefix;
        }
    }

    public static class Solution2 {
        //vertical scan
        public String longestCommonPrefix(String[] strs) {
            if (strs.length == 0) {
                return "";
            }
            for (int i = 0; i < strs[0].length(); i++) {
                char c = strs[0].charAt(i);
                for (int j = 1; j < strs.length; j++) {
                    if (i == strs[j].length() || strs[j].charAt(i) != c) {
                        return strs[0].substring(0, i);
                    }
                }
            }
            return strs[0];
        }
    }
}

// ============================================================
// FILE 15: _15.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
```

```
 * 15. 3Sum
 *
 * Given an array S of n integers, are there elements a, b, c in S such that a + b + c =
0?
 * Find all unique triplets in the array which gives the sum of zero.
 *
 * Note: The solution set must not contain duplicate triplets.

 For example, given array S = [-1, 0, 1, 2, -1, -4],

 A solution set is:
 [
 [-1, 0, 1],
 [-1, -1, 2]
 ]
 */

public class _15 {

    public static class Solution1 {
        public List<List<Integer>> threeSum(int[] nums) {
            Arrays.sort(nums);

            List<List<Integer>> result = new ArrayList<>();
            for (int i = 0; i < nums.length - 2; i++) {
                if (i >= 1 && nums[i] == nums[i - 1]) {
                    continue;
                }
                int left = i + 1;
                int right = nums.length - 1;
                while (left < right) {
                    int sum = nums[i] + nums[left] + nums[right];
                    if (sum == 0) {
                        result.add(Arrays.asList(nums[i], nums[left], nums[right]));

                        while (left < right && nums[left] == nums[left + 1]) {
                            left++;
                        }

                        while (left < right && nums[right] == nums[right - 1]) {
                            right--;
                        }
                        left++;
                        right--;
                    } else if (sum > 0) {
                        right--;
                    } else {
                        left++;
                    }
                }
            }
            return result;
        }
    }
```

```
}

// ============================================================
// FILE 16: _16.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 16. 3Sum Closest
 *
 * Given an array S of n integers, find three integers in S such that the sum is closest
to a given number, target.
 * Return the sum of the three integers. You may assume that each input would have
exactly one solution.
 * For example, given array S = {-1 2 1 -4}, and target = 1.
 * The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).
 */

public class _16 {

    public static class Solution1 {
        public int threeSumClosest(int[] nums, int target) {
            Arrays.sort(nums);
            int len = nums.length;
            if (len < 3) {
                int sum = 0;
                for (int i : nums) {
                    sum += i;
                }
                return sum;
            }
            int sum = nums[0] + nums[1] + nums[2];
            for (int i = 0; i < len - 2; i++) {
                int left = i + 1;
                int right = len - 1;
                while (left < right) {
                    int thisSum = nums[i] + nums[left] + nums[right];
                    if (Math.abs(target - thisSum) < Math.abs(target - sum)) {
                        sum = thisSum;
                        if (sum == target) {
                            return sum;
                        }
                    } else if (target > thisSum) {
                        left++;
                    } else {
                        right--;
                    }
                }
            }
            return sum;
        }
```

```
    }

}

// ============================================================
// FILE 17: _17.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 17. Letter Combinations of a Phone Number
 *
 * Given a digit string, return all possible letter combinations that the number could
represent.
 * A mapping of digit to letters (just like on the telephone buttons) is given below.

 Input:Digit string "23"
 Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"].

 Note:
 Although the above answer is in lexicographical order, your answer could be in any order
you want.
 */

public class _17 {

    public static class Solution1 {
        public List<String> letterCombinations(String digits) {
            List<String> result = new ArrayList();
            if (digits.length() == 0) {
                return result;
            }

            String[] digits2Letters = new String[]{"", "", "abc", "def", "ghi", "jkl",
"mno", "pqrs", "tuv", "wxyz"};

            result.add("");//this line is important, otherwise result is empty and Java
will default it to an empty String
            for (int i = 0; i < digits.length(); i++) {
                result = combine(digits2Letters[digits.charAt(i) - '0'], result);
            }

            return result;
        }

        List<String> combine(String letters, List<String> result) {
            List<String> newResult = new ArrayList();

            for (int i = 0; i < letters.length(); i++) {
                //the order of the two for loops doesn't matter, you could swap them and
```

```
it still works.
                for (String str : result) {
                    newResult.add(str + letters.charAt(i));
                }
            }
            return newResult;
        }
    }
}

// ============================================================
// FILE 18: _18.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 18. 4Sum
 *
 * Given an array S of n integers, are there elements a, b, c, and d in S such that a + b
+ c + d = target?
 * Find all unique quadruplets in the array which gives the sum of target.

 Note: The solution set must not contain duplicate quadruplets.

 For example, given array S = [1, 0, -1, 0, -2, 2], and target = 0.

 A solution set is:
 [
 [-1,  0, 0, 1],
 [-2, -1, 1, 2],
 [-2,  0, 0, 2]
 ]*/

public class _18 {

    public static class Solution1 {
        public List<List<Integer>> fourSum(int[] nums, int target) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null || nums.length == 0) {
                return result;
            }
            Arrays.sort(nums);
            for (int i = 0; i < nums.length - 3; i++) {
                if (i > 0 && nums[i - 1] == nums[i]) {
                    continue;
                }
                for (int j = i + 1; j < nums.length - 2; j++) {
                    if (j > i + 1 && nums[j - 1] == nums[j]) {
                        continue;
```

```
                    }
                    int left = j + 1;
                    int right = nums.length - 1;
                    while (left < right) {
                        int sum = nums[i] + nums[j] + nums[left] + nums[right];
                        if (sum == target) {
                            result.add(Arrays.asList(nums[i], nums[j], nums[left],
nums[right]));
                            while (left + 1 < right && nums[left] == nums[left + 1]) {
                                left++;
                            }
                            while (right - 1 > left && nums[right] == nums[right - 1]) {
                                right--;
                            }
                            left++;
                            right--;
                        } else if (sum > target) {
                            right--;
                        } else {
                            left++;
                        }
                    }
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 19: _19.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;
import com.fishercoder.common.utils.CommonUtils;

/**19. Remove Nth Node From End of List
 *
 * Given a linked list, remove the nth node from the end of list and return its head.
 * For example, Given linked list: 1->2->3->4->5, and n = 2.
 * After removing the second node from the end, the linked list becomes 1->2->3->5.

Note:
Given n will always be valid.
Try to do this in one pass.
 */

public class _19 {

    public static class Solution1 {
        /**
         * Naive/most straightforward approach:
```

```
         * go through the list, find its total length, then go through the list a second
time:
         * this time, pause at the delta point, then assign its next.next pointer to
next.
         * This approach has to traverse the list twice, not one-pass.
         */
        public ListNode removeNthFromEnd(ListNode head, int n) {
            ListNode temp = head;
            int len = 0;
            while (temp != null) {
                temp = temp.next;
                len++;
            }
            if (n == len) {
                return head.next;
            }

            temp = head;
            int cut = len - n;
            while (cut-- > 1) {
                temp = temp.next;
            }
            if (temp.next != null) {
                temp.next = temp.next.next;
                return head;
            }
            return null;
        }
    }

    public static class Solution2 {
        public ListNode removeNthFromEnd(ListNode head, int n) {
            //this approach uses two pointers, fast moves first for n nodes, when fast
reaches n, then we start to move slow
            //then, when fast reaches null, slow reaches the point where the node should
be deleted.
            ListNode dummy = new ListNode(-1);
            dummy.next = head;
            ListNode slow = head;
            ListNode fast = head;
            int tempN = n;
            while (tempN-- > 0) {
                fast = fast.next;
            }

            if (fast == null) {
                if (n > 0) {
                    // this is for cases like this: [1,2] 2 or [1,2,3,4] 4, namely,
remove the head of
                    // the list and return the second node from the original list
                    dummy.next = dummy.next.next;
                }
                return dummy.next;
            }
```

```
            fast = fast.next;//we'll have to move fast pointer one node forward before
moving the two together, this way,
            //when fast reaches null, slow will be at the previous node to the node that
should be deleted, thus, we can change the next pointer easily

            while (fast != null) {
                fast = fast.next;
                slow = slow.next;
            }

            if (slow.next != null) {
                slow.next = slow.next.next;
            }
            return dummy.next;
        }
    }

    public static class Solution3 {
        //a more concise version using the same idea
        public ListNode removeNthFromEnd(ListNode head, int n) {
            ListNode dummy = new ListNode(-1);
            dummy.next = head;
            ListNode slow = dummy;
            ListNode fast = dummy;
            while (fast.next != null) {
                if (n <= 0) {
                    slow = slow.next;
                }
                fast = fast.next;
                n--;
            }
            if (slow.next != null) {
                slow.next = slow.next.next;
            }
            return dummy.next;
        }
    }
}

// ============================================================
// FILE 20: _20.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 20. Valid Parentheses
 *
 * Given a string containing just the characters '(', ')', '{', '}', '[' and ']',
determine if the input string is valid.
 *
```

```
 * An input string is valid if:
 *
 * 1. Open brackets must be closed by the same type of brackets.
 * 2. Open brackets must be closed in the correct order.
 *
 * Note that an empty string is also considered valid.
 *
 * Example 1:
 * Input: "()"
 * Output: true
 *
 * Example 2:
 * Input: "()[]{}"
 * Output: true
 *
 * Example 3:
 * Input: "(]"
 * Output: false
 *
 * Example 4:
 * Input: "([)]"
 * Output: false
 *
 * Example 5:
 * Input: "{[]}"
 * Output: true
 * */
public class _20 {
    public static class Solution1 {
        public boolean isValid(String s) {
            Stack<Character> stack = new Stack<>();
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == '(' || s.charAt(i) == '{' || s.charAt(i) == '[') {
                    stack.push(s.charAt(i));
                } else {
                    if (stack.isEmpty()) {
                        return false;
                    } else {
                        if (stack.peek() == '(' && s.charAt(i) != ')') {
                            return false;
                        } else if (stack.peek() == '{' && s.charAt(i) != '}') {
                            return false;
                        } else if (stack.peek() == '[' && s.charAt(i) != ']') {
                            return false;
                        }
                        stack.pop();
                    }
                }
            }
            return stack.isEmpty();
        }
    }

    public static class Solution2 {
```

```
        /**
         * A more concise solution:
         * credit: https://leetcode.com/problems/valid-parentheses/discuss/9178/Short-
java-solution
         * */
        public boolean isValid(String s) {
            Stack<Character> stack = new Stack<>();
            for (char c : s.toCharArray()) {
                if (c == '(') {
                    stack.push(')');
                } else if (c == '{') {
                    stack.push('}');
                } else if (c == '[') {
                    stack.push(']');
                } else if (stack.isEmpty() || stack.pop() != c) {
                    return false;
                }
            }
            return stack.isEmpty();
        }
    }
}

// ============================================================
// FILE 21: _21.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 21. Merge Two Sorted Lists
 *
 * Merge two sorted linked lists and return it as a new list.
 * The new list should be made by splicing together the nodes of the first two lists.
 *
 * Example:
 * Input: 1->2->4, 1->3->4
 * Output: 1->1->2->3->4->4
 *
 * */

public class _21 {
    public static class Solution1 {
        /**
         * recursive solution
         * Time: O(m+n)
         * Space: O(m+n)
         * */
        public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
            if (l1 == null) {
                return l2;
            }
```

```
            if (l2 == null) {
                return l1;
            }
            if (l1.val < l2.val) {
                l1.next = mergeTwoLists(l1.next, l2);
                return l1;
            } else {
                l2.next = mergeTwoLists(l1, l2.next);
                return l2;
            }
        }
    }

    public static class Solution2 {
        /**
         * iterative solution
         * Time: O(m+n)
         * Space: O(1)
         */
        public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
            ListNode pre = new ListNode(-1);
            ListNode curr = pre;

            while (l1 != null && l2 != null) {
                if (l1.val < l2.val) {
                    curr.next = l1;
                    l1 = l1.next;
                } else {
                    curr.next = l2;
                    l2 = l2.next;
                }
                curr = curr.next;
            }

            curr.next = l1 == null ? l2 : l1;
            return pre.next;
        }
    }

}

// ============================================================
// FILE 22: _22.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 22. Generate Parentheses
 *
 * Given n pairs of parentheses, write a function to generate all combinations of well-
```

```
formed parentheses.
 *
 * For example, given n = 3, a solution set is:

 [
 "((()))",
 "(()())",
 "(())()",
 "()(())",
 "()()()"
 ]*/

public class _22 {
    public static class Solution1 {
        public List<String> generateParenthesis(int n) {
            List<String> result = new ArrayList();
            backtrack(result, "", 0, 0, n);
            return result;
        }

        void backtrack(List<String> result, String str, int left, int right, int max) {
            if (str.length() == max * 2) {
                result.add(str);
                return;
            }

            if (left < max) {
                backtrack(result, str + "(", left + 1, right, max);
            }

            if (right < left) {
                backtrack(result, str + ")", left, right + 1, max);
            }
        }
    }

    public static class Solution2 {
        public List<String> generateParenthesis(int n) {
            List<String> result = new ArrayList();
            if (n == 0) {
                return result;
            }
            helper(result, "", n, n);
            return result;
        }

        void helper(List<String> result, String par, int left, int right) {
            if (left > 0) {
                helper(result, par + "(", left - 1, right);
            }
            if (right > left) {
                helper(result, par + ")", left, right - 1);
            }
            if (right == 0) {
```

```
                result.add(par);
            }
        }
    }
}

// ============================================================
// FILE 23: _23.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.Comparator;
import java.util.PriorityQueue;

/**
 * 23. Merge k Sorted Lists
 *
 * Merge k sorted linked lists and return it as one sorted list. Analyze and describe its
complexity.
 *
 * Example:
 *
 * Input:
 * [
 *   1->4->5,
 *   1->3->4,
 *   2->6
 * ]
 * Output: 1->1->2->3->4->4->5->6
 *
 * */

public class _23 {
    public static class Solution1 {
        public ListNode mergeKLists(ListNode[] lists) {
            PriorityQueue<ListNode> heap = new PriorityQueue((Comparator<ListNode>) (o1,
o2) -> o1.val - o2.val);

            for (ListNode node : lists) {
                if (node != null) {
                    heap.offer(node);
                }
            }

            ListNode pre = new ListNode(-1);
            ListNode temp = pre;
            while (!heap.isEmpty()) {
                ListNode curr = heap.poll();
                temp.next = new ListNode(curr.val);
                if (curr.next != null) {
                    heap.offer(curr.next);
```

```
                }
                temp = temp.next;
            }
            return pre.next;
        }
    }

}

// ============================================================
// FILE 24: _24.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 24. Swap Nodes in Pairs
 *
 * Given a linked list, swap every two adjacent nodes and return its head.

 For example,
 Given 1->2->3->4, you should return the list as 2->1->4->3.

 Your algorithm should use only constant space. You may not modify the values in the
list, only nodes itself can be changed.

 */
public class _24 {
    public static class Solution1 {
        public ListNode swapPairs(ListNode head) {
            if (head == null || head.next == null) {
                return head;
            }
            ListNode second = head.next;
            ListNode third = second.next;
            second.next = head;
            head.next = swapPairs(third);
            return second;
        }
    }

}

// ============================================================
// FILE 25: _25.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
```

```
 * 25. Reverse Nodes in k-Group
 *
 * Given a linked list, reverse the nodes of a linked list k at a time and return its
modified list.
 * k is a positive integer and is less than or equal to the length of the linked list.
 * If the number of nodes is not a multiple of k then left-out nodes in the end should
remain as it is.
 * You may not alter the values in the nodes, only nodes itself may be changed.
 * Only constant memory is allowed.

 For example,
 Given this linked list: 1->2->3->4->5

 For k = 2, you should return: 2->1->4->3->5

 For k = 3, you should return: 3->2->1->4->5
 */

public class _25 {

    /**We use recursion to go all the way until the end: when the number of nodes are
smaller than k;
     * then we start to reverse each group of k nodes from the end towards the start.*/
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode curr = head;
        int count = 0;
        while (curr != null && count != k) {
            //find the k+1 node
            curr = curr.next;
            count++;
        }

        if (count == k) {
            /**after this below recursive call finishes, it'll return head;
             * then this returned "head" will become "curr", while the head
             * in its previous callstack is the real head after this call.
             * Setting up a break point will make all of this crystal clear.*/
            curr = reverseKGroup(curr, k);

            while (count-- > 0) {
                ListNode temp = head.next;
                head.next = curr;
                curr = head;
                head = temp;
            }
            head = curr;
        }
        return head;//we run out of nodes before we hit count == k, so we'll just
directly return head in this case as well
    }

}

// ============================================================
```

```java
// FILE 26: _26.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 26. Remove Duplicates from Sorted Array
 *
 * Given a sorted array nums, remove the duplicates in-place such that each element
appear only once and return the new length.
 * Do not allocate extra space for another array, you must do this by modifying the input
array in-place with O(1) extra memory.
 *
 * Example 1:
 * Given nums = [1,1,2],
 * Your function should return length = 2, with the first two elements of nums being 1
and 2 respectively.
 * It doesn't matter what you leave beyond the returned length.
 *
 * Example 2:
 * Given nums = [0,0,1,1,1,2,2,3,3,4],
 * Your function should return length = 5, with the first five elements of nums being
modified to 0, 1, 2, 3, and 4 respectively.
 * It doesn't matter what values are set beyond the returned length.
 *
 * Clarification:
 * Confused why the returned value is an integer but your answer is an array?
 * Note that the input array is passed in by reference, which means modification to the
input array will be known to the caller as well.
 * Internally you can think of this:
 * // nums is passed in by reference. (i.e., without making a copy)
 * int len = removeDuplicates(nums);
 *
 * // any modification to nums in your function would be known by the caller.
 * // using the length returned by your function, it prints the first len elements.
 * for (int i = 0; i < len; i++) {
 *     print(nums[i]);
 * }
 * */

public class _26 {

  public static class Solution1 {
    /**Key: It doesn't matter what you leave beyond the returned length.*/
    public int removeDuplicates(int[] nums) {
      int i = 0;
      for (int j = 1; j < nums.length; j++) {
        if (nums[i] != nums[j]) {
          i++;
          nums[i] = nums[j];
        }
      }
      return i + 1;
    }
```

```
  }
}

// ============================================================
// FILE 27: _27.java
// ============================================================

package com.fishercoder.solutions;

/**27. Remove Element

Given an array and a value, remove all instances of that value in place and return the
new length.

Do not allocate extra space for another array, you must do this in place with constant
memory.

The order of elements can be changed. It doesn't matter what you leave beyond the new
length.

Example:
Given input array nums = [3,2,2,3], val = 3

Your function should return length = 2, with the first two elements of nums being 2.
 */

public class _27 {

  public static class Solution1 {
    public int removeElement(int[] nums, int val) {
      int i = 0;
      for (int j = 0; j < nums.length; j++) {
        if (nums[j] != val) {
          nums[i++] = nums[j];
        }
      }
      return i;
    }
  }
}

// ============================================================
// FILE 28: _28.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 28. Implement strStr()
 *
 * Implement strStr().
 *
 * Returns the index of the first occurrence of needle in haystack, or -1 if needle is
not part of haystack.
```

```
 */

public class _28 {

  public static class Solution1 {
    public int strStr(String haystack, String needle) {
      if (haystack == null || needle == null || haystack.length() < needle.length()) {
        return -1;
      }

      for (int i = 0; i <= haystack.length() - needle.length(); i++) {
        if (haystack.substring(i, i + needle.length()).equals(needle)) {
          return i;
        }
      }
      return -1;
    }
  }

}

// ============================================================
// FILE 29: _29.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 29. Divide Two Integers
 *
 * Divide two integers without using multiplication, division and mod operator.
 * If it is overflow, return MAX_INT.
 */
public class _29 {

  public static class Solution1 {
    public int divide(int dividend, int divisor) {
      if (divisor == 0 || (dividend == Integer.MIN_VALUE && divisor == -1)) {
        return Integer.MAX_VALUE;
      }
      if (dividend != Integer.MIN_VALUE
          && Math.abs(dividend) < Math.abs(divisor)) {
        return 0;
      }
      if (divisor == Integer.MIN_VALUE) {
        return (dividend == Integer.MIN_VALUE) ? 1 : 0;
      }

      boolean flag = (dividend < 0) ^ (divisor < 0);
      dividend = -Math.abs(dividend);
      divisor = -Math.abs(divisor);
      int[] num = new int[40];
      int[] multiple = new int[40];
      num[1] = divisor;
```

```
      multiple[1] = 1;

      for (int i = 2; i < 32 && num[i - 1] < 0; ++i) {
        num[i] = num[i - 1] << 1;
        multiple[i] = multiple[i - 1] << 1;
      }

      int result = 0;
      int index = 1;
      while (num[index] < 0) {
        ++index;
      }
      index -= 1;

      while (dividend <= divisor) {
        while (dividend <= num[index]) {
          result += multiple[index];
          dividend -= num[index];
        }
        --index;
      }
      return !flag ? result : -result;
    }
  }
}

// ============================================================
// FILE 30: _30.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 30. Substring with Concatenation of All Words
 *
 * You are given a string, s, and a list of words, words, that are all of the same
length.
 * Find all starting indices of substring(s) in s that is a concatenation of each word in
words exactly once and without any intervening characters.

 For example, given:
 s: "barfoothefoobarman"
 words: ["foo", "bar"]

 You should return the indices: [0,9].
 (order does not matter).
 */
public class _30 {
```

```
  public static class Solution1 {
    /**TODO: this one is not AC'ed. fix this one.*/
    public List<Integer> findSubstring(String s, String[] words) {
      Map<String, Integer> map = new HashMap<>();
      for (String word : words) {
        map.put(word, 1);
      }
      List<Integer> result = new ArrayList<>();
      int startIndex = 0;
      int wordLen = words.length;
      for (int i = 0; i < s.length(); i++) {
        startIndex = i;
        Map<String, Integer> clone = new HashMap<>(map);
        int matchedWord = 0;
        for (int j = i + 1; j < s.length(); j++) {
          String word = s.substring(i, j);
          if (clone.containsKey(word) && clone.get(word) == 1) {
            clone.put(word, 0);
            i = j;
            matchedWord++;
          }
          if (matchedWord == wordLen) {
            boolean all = true;
            for (String key : clone.keySet()) {
              if (clone.get(key) != 0) {
                all = false;
                break;
              }
            }
            if (all) {
              result.add(startIndex);
            }
            matchedWord = 0;
          }
        }
      }
      return result;
    }
  }

}

// ============================================================
// FILE 31: _31.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 31. Next Permutation

Implement next permutation, which rearranges numbers into the lexicographically next
greater permutation of numbers.
```

```
If such arrangement is not possible, it must rearrange it as the lowest possible order
(ie, sorted in ascending order).

The replacement must be in-place, do not allocate extra memory.

Here are some examples. Inputs are in the left-hand column and its corresponding outputs
are in the right-hand column.
1,2,3 → 1,3,2
3,2,1 → 1,2,3
1,1,5 → 1,5,1
 */

public class _31 {
  public static class Solution1 {
    /**
     * Leetcode has a very good article to illustrate this problem and with animation:
     * https://leetcode.com/articles/next-permutation/
     * 1. if the array is already in decrementing order, then there's no next larger
permutation possible.
     * 2. if not, start from the end of the array, find the first pair of numbers that
break the decrementing order
     * 3. then from that index going to the right again, find the element that is closest
bigger than this number, swap them
     * 4. reverse the right half of this array after this index
     */

    public void nextPermutation(int[] nums) {
      int i = nums.length - 2;
      while (i >= 0 && nums[i] >= nums[i + 1]) {
        i--;
      }
      if (i >= 0) {
        int j = nums.length - 1;
        while (j >= 0 && nums[i] >= nums[j]) {
          j--;
        }

        swap(nums, i, j);
      }

      reverse(nums, i + 1);
    }

    private void reverse(int[] nums, int start) {
      int end = nums.length - 1;
      while (start <= end) {
        int tmp = nums[start];
        nums[start++] = nums[end];
        nums[end--] = tmp;
      }
    }

    private void swap(int[] nums, int i, int j) {
      int tmp = nums[i];
```

```
      nums[i] = nums[j];
      nums[j] = tmp;
    }
  }
}

// ============================================================
// FILE 32: _32.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 32. Longest Valid Parentheses
 *
 * Given a string containing just the characters '(' and ')', find the length of the
longest valid (well-formed) parentheses substring.
 * For "(()", the longest valid parentheses substring is "()", which has length = 2.
 * Another example is ")()())", where the longest valid parentheses substring is "()()",
which has length = 4.
 */
public class _32 {
  public static class Solution1 {
    public int longestValidParentheses(String s) {
      int result = 0;
      Stack<Integer> stack = new Stack();
      stack.push(-1);
      for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == '(') {
          stack.push(i);
        } else {
          stack.pop();
          if (stack.isEmpty()) {
            stack.push(i);
          } else {
            result = Math.max(result, i - stack.peek());
          }
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 33: _33.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 33. Search in Rotated Sorted Array
```

```
 *
 * Suppose a sorted array is rotated at some pivot unknown to you beforehand.
 * (i.e., 0 1 2 4 5 6 7 might become 4 5 6 7 0 1 2).
 * You are given a target value to search. If found in the array return its index,
otherwise return -1.
 * You may assume no duplicate exists in the array.
 */
public class _33 {

    public static class Solution1 {

        public int search(int[] nums, int target) {
            if (nums == null || nums.length == 0) {
                return -1;
            }
            int minIdx = findMinIdx(nums);
            if (target == nums[minIdx]) {
                return minIdx;
            }
            int m = nums.length;
            int start = (target <= nums[m - 1]) ? minIdx : 0;
            int end = (target > nums[m - 1]) ? minIdx : m - 1;

            while (start <= end) {
                int mid = start + (end - start) / 2;
                if (nums[mid] == target) {
                    return mid;
                } else if (target > nums[mid]) {
                    start = mid + 1;
                } else {
                    end = mid - 1;
                }
            }
            return -1;
        }

        private int findMinIdx(int[] nums) {
            int start = 0;
            int end = nums.length - 1;

            while (start < end) {
                int mid = start + (end - start) / 2;
                if (nums[mid] > nums[end]) {
                    start = mid + 1;
                } else {
                    end = mid;
                }
            }
            return start;
        }
    }

    public static class Solution2 {
        public int search(int[] nums, int target) {
```

```
            if (nums == null || nums.length == 0) {
                return -1;
            }
            int lo = 0;
            int hi = nums.length - 1;
            while (lo < hi) {
                int mid = (lo + hi) / 2;
                if (nums[mid] == target) {
                    return mid;
                }

                if (nums[lo] <= nums[mid]) {
                    if (target >= nums[lo] && target < nums[mid]) {
                        hi = mid - 1;
                    } else {
                        lo = mid + 1;
                    }
                } else {
                    if (target > nums[mid] && target <= nums[hi]) {
                        lo = mid + 1;
                    } else {
                        hi = mid - 1;
                    }
                }
            }
            return nums[lo] == target ? lo : -1;
        }
    }
}

// ============================================================
// FILE 34: _34.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 34. Search for a Range
 *
 * Given an array of integers sorted in ascending order, find the starting and ending
position of a given target value.
 * Your algorithm's runtime complexity must be in the order of O(log n).
 * If the target is not found in the array, return [-1, -1].

 For example,
 Given [5, 7, 7, 8, 8, 10] and target value 8,
 return [3, 4].

 */
public class _34 {

    public int[] searchRange(int[] nums, int target) {
        int[] range = new int[2];
        range[0] = -1;
```

```
        range[1] = -1;
        if (nums == null || nums.length == 0) {
            return range;
        }
        int start = 0;
        int end = nums.length - 1;
        while (start + 1 < end) {
            int mid = start + (end - start) / 2;
            if (nums[mid] == target) {
                int left = mid;
                while (left - 1 >= 0 && nums[left] == nums[left - 1]) {
                    left--;
                }
                range[0] = left;
                int right = mid;
                while (right + 1 < nums.length && nums[right] == nums[right + 1]) {
                    right++;
                }
                range[1] = right;
                break;
            } else if (nums[mid] < target) {
                start = mid;
            } else {
                end = mid;
            }
        }

        if (nums[start] == target) {
            range[0] = start;
            while (start + 1 < nums.length && nums[start] == nums[start + 1]) {
                start++;
            }
            range[1] = start;
        } else if (nums[end] == target) {
            range[1] = end;
            while (end - 1 >= 0 && nums[end] == nums[end - 1]) {
                end--;
            }
            range[0] = end;
        }
        return range;
    }
}

// ============================================================
// FILE 35: _35.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 35. Search Insert Position
 *
 * Given a sorted array and a target value,
```

```
 * return the index if the target is found.
 * If not, return the index where it would be if it were inserted in order.

 You may assume no duplicates in the array.

 Here are few examples.
 [1,3,5,6], 5 → 2
 [1,3,5,6], 2 → 1
 [1,3,5,6], 7 → 4
 [1,3,5,6], 0 → 0
 */

public class _35 {

    public static class Solution1 {
        public int searchInsert(int[] nums, int target) {
            int len = nums.length;
            for (int i = 0; i < len; i++) {
                if (target <= nums[i]) {
                    return i;
                }
            }
            return len;
        }
    }

}

// ============================================================
// FILE 36: _36.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 36. Valid Sudoku
 *
 * Determine if a Sudoku is valid, according to: Sudoku Puzzles - The Rules.
 * The Sudoku board could be partially filled, where empty cells are filled with the
character '.'.
 *
 * A partially filled sudoku which is valid.

 Note:
 A valid Sudoku board (partially filled) is not necessarily solvable. Only the filled
cells need to be validated.*/
public class _36 {

  public static class Solution1 {
    public boolean isValidSudoku(char[][] board) {
      for (int i = 0; i < 9; i++) {
        if (!isValidRowOrColumn(board, i)) {
          return false;
        }
```

```
    }

    for (int j = 0; j < 9; j++) {
      if (!isValidCol(board, j)) {
        return false;
      }
    }

    for (int i = 0; i < 7; i = i + 3) {
      for (int j = 0; j < 7; j = j + 3) {
        if (!isValidSquare(board, i, j)) {
          return false;
        }
      }
    }
    return true;
  }

  boolean isValidRowOrColumn(char[][] board, int index) {
    int[] nums = new int[9];
    for (int i = 0; i < 9; i++) {
      nums[i] = 1;
    }
    for (int j = 0; j < 9; j++) {
      if (board[index][j] != '.') {
        nums[Character.getNumericValue(board[index][j]) - 1]--;
      }
    }
    for (int i : nums) {
      if (i < 0) {
        return false;
      }
    }
    return true;
  }

  boolean isValidCol(char[][] board, int col) {
    int[] nums = new int[9];
    for (int i = 0; i < 9; i++) {
      nums[i] = 1;
    }
    for (int i = 0; i < 9; i++) {
      if (board[i][col] != '.') {
        nums[Character.getNumericValue(board[i][col]) - 1]--;
      }
    }
    for (int i : nums) {
      if (i < 0) {
        return false;
      }
    }
    return true;
  }
```

```
    boolean isValidSquare(char[][] board, int row, int col) {
      int[] nums = new int[9];
      for (int i = 0; i < 9; i++) {
        nums[i] = 1;
      }
      for (int i = row; i < row + 3; i++) {
        for (int j = col; j < col + 3; j++) {
          if (board[i][j] != '.') {
            nums[Character.getNumericValue(board[i][j]) - 1]--;
          }
        }
      }
      for (int i : nums) {
        if (i < 0) {
          return false;
        }
      }
      return true;
    }
  }
}

// ============================================================
// FILE 37: _37.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 37. Sudoku Solver
 *
 * Determine if a 9x9 Sudoku board is valid. Only the filled cells need to be validated
according to the following rules:
 *
 * Each row must contain the digits 1-9 without repetition.
 * Each column must contain the digits 1-9 without repetition.
 * Each of the 9 3x3 sub-boxes of the grid must contain the digits 1-9 without
repetition.
 *
 * The Sudoku board could be partially filled, where empty cells are filled with the
character '.'.
 *
 * Example 1:
 *
 * Input:
 * [
 *   ["5","3",".",".","7",".",".",".","."],
 *   ["6",".",".","1","9","5",".",".","."],
 *   [".","9","8",".",".",".",".","6","."],
 *   ["8",".",".",".","6",".",".",".","3"],
 *   ["4",".",".","8",".","3",".",".","1"],
 *   ["7",".",".",".","2",".",".",".","6"],
 *   [".","6",".",".",".",".","2","8","."],
 *   [".",".",".","4","1","9",".",".","5"],
```

```
 *   [".",".",".",".","8",".",".","7","9"]
 * ]
 * Output: true
 * Example 2:
 *
 * Input:
 * [
 *   ["8","3",".",".","7",".",".",".","."],
 *   ["6",".",".","1","9","5",".",".","."],
 *   [".","9","8",".",".",".",".","6","."],
 *   ["8",".",".",".","6",".",".",".","3"],
 *   ["4",".",".","8",".","3",".",".","1"],
 *   ["7",".",".",".","2",".",".",".","6"],
 *   [".","6",".",".",".",".","2","8","."],
 *   [".",".",".","4","1","9",".",".","5"],
 *   [".",".",".",".","8",".",".","7","9"]
 * ]
 * Output: false
 * Explanation: Same as Example 1, except with the 5 in the top left corner being
 *     modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.
 * Note:
 *
 * A Sudoku board (partially filled) could be valid but is not necessarily solvable.
 * Only the filled cells need to be validated according to the mentioned rules.
 * The given board contain only digits 1-9 and the character '.'.
 * The given board size is always 9x9.
 *
 */
public class _37 {

    public static class Solution1 {
        public void solveSudoku(char[][] board) {
            if (board == null || board.length == 0) {
                return;
            }
            solve(board);
        }

        private boolean solve(char[][] board) {
            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    if (board[i][j] == '.') {
                        for (char c = '1'; c <= '9'; c++) {
                            //try 1 to 9
                            if (isValid(board, i, j, c)) {
                                board[i][j] = c;

                                if (solve(board)) {
                                    return true;
                                } else {
                                    board[i][j] = '.';//recover it to be '.'
                                }
                            }
                        }
```

```
                        return false;
                    }
                }
            }
            return true;
        }

        private boolean isValid(char[][] board, int row, int col, char c) {
            for (int i = 0; i < 9; i++) {
                if (board[i][col] != '.' && board[i][col] == c) {
                    return false;//check row
                }
                if (board[row][i] != '.' && board[row][i] == c) {
                    return false;//check column
                }
                if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] != '.' && board[3
* (row / 3) + i / 3][3 * (col / 3) + i % 3] == c) {
                    return false; //check 3*3 block
                }
            }
            return true;
        }
    }

}

// ============================================================
// FILE 38: _38.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 38. Count and Say
 *
 * The count-and-say sequence is the sequence of integers beginning as follows:
 1, 11, 21, 1211, 111221, ...

 1 is read off as "one 1" or 11.
 11 is read off as "two 1s" or 21.
 21 is read off as "one 2, then one 1" or 1211.
 Given an integer n, generate the nth sequence.

 Note: The sequence of integers will be represented as a string.*/

public class _38 {
  public static class Solution1 {
    public String countAndSay(int n) {
      StringBuilder curr = new StringBuilder("1");
      StringBuilder prev;
      int count;
      char say;
      for (int i = 1; i < n; i++) {
        prev = curr;
```

```
        curr = new StringBuilder();
        count = 1;
        say = prev.charAt(0);

        for (int j = 1, len = prev.length(); j < len; j++) {
          if (prev.charAt(j) != say) {
            curr.append(count).append(say);
            count = 1;
            say = prev.charAt(j);
          } else {
            count++;
          }
        }
        curr.append(count).append(say);
      }
      return curr.toString();
    }
  }

}

// ============================================================
// FILE 39: _39.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 39. Combination Sum
 *
 * Given a set of candidate numbers (C) (without duplicates) and a target number (T),
 * find all unique combinations in C where the candidate numbers sums to T.

 The same repeated number may be chosen from C unlimited number of times.

 Note:
 All numbers (including target) will be positive integers.
 The solution set must not contain duplicate combinations.
 For example, given candidate set [2, 3, 6, 7] and target 7,
 A solution set is:
 [
 [7],
 [2, 2, 3]
 ]*/
public class _39 {

    public static class Solution1 {
        public List<List<Integer>> combinationSum(int[] candidates, int target) {
            List<List<Integer>> result = new ArrayList();
            Arrays.sort(candidates);
```

```
            backtracking(candidates, target, 0, new ArrayList(), result);
            return result;
        }

        void backtracking(int[] candidates, int target, int start, List<Integer> curr,
List<List<Integer>> result) {
            if (target > 0) {
                for (int i = start; i < candidates.length; i++) {
                    if (candidates[i] > target) {
                        continue;//pruning
                    }
                    curr.add(candidates[i]);
                    backtracking(candidates, target - candidates[i], i, curr, result);
                    curr.remove(curr.size() - 1);
                }
            } else if (target == 0) {
                result.add(new ArrayList(curr));
            }
        }
    }
}

// ============================================================
// FILE 40: _40.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
/**
 * 40. Combination Sum II
 *
 * Given a collection of candidate numbers (C) and a target number (T), find all unique
combinations in C where the candidate numbers sums to T.
 * Each number in C may only be used once in the combination.

 Note:
 All numbers (including target) will be positive integers.
 The solution set must not contain duplicate combinations.
 For example, given candidate set [10, 1, 2, 7, 6, 1, 5] and target 8,
 A solution set is:
 [
 [1, 7],
 [1, 2, 5],
 [2, 6],
 [1, 1, 6]
 ]
 */
public class _40 {

    public static class Solution1 {
      public List<List<Integer>> combinationSum2(int[] candidates, int target) {
```

```
        List<List<Integer>> result = new ArrayList();
        Arrays.sort(candidates);
        backtracking(candidates, 0, result, target, new ArrayList());
        return result;
      }

      void backtracking(int[] candidates, int start, List<List<Integer>> result, int
target,
          List<Integer> curr) {
        if (target > 0) {
          for (int i = start; i < candidates.length; i++) {
            if (candidates[i] > target || (i > start && candidates[i - 1] ==
candidates[i])) {
              continue;
            }
            curr.add(candidates[i]);
            backtracking(candidates, i + 1, result, target - candidates[i], curr);
            curr.remove(curr.size() - 1);
          }
        } else if (target == 0) {
          result.add(new ArrayList(curr));
        }
      }
    }
}

// ============================================================
// FILE 41: _41.java
// ============================================================

package com.fishercoder.solutions;

/**
 *41. First Missing Positive
 *
 *Given an unsorted integer array, find the first missing positive integer.

 For example,
 Given [1,2,0] return 3,
 and [3,4,-1,1] return 2.

 Your algorithm should run in O(n) time and uses constant space.
 */

public class _41 {

  public static class Solution1 {
    /**
     * Time: O(n) Space: O(1)
     *
     * Idea: put every number in its right position, e.g. put 5 in nums[4].
     */
    public int firstMissingPositive(int[] nums) {
      int i = 0;
```

```
      while (i < nums.length) {
        if (nums[i] > 0 && nums[i] != i + 1 && nums[i] - 1 < nums.length && nums[i] !=
nums[nums[i]
            - 1]) {
          swap(nums, i, nums[i] - 1);
        } else {
          i++;
        }
      }

      for (int j = 0; j < nums.length; j++) {
        if (nums[j] != j + 1) {
          return j + 1;
        }
      }

      return nums.length + 1;
    }

    void swap(int[] nums, int i, int j) {
      int temp = nums[i];
      nums[i] = nums[j];
      nums[j] = temp;
    }
  }
}

// ============================================================
// FILE 42: _42.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 42. Trapping Rain Water
 * Given n non-negative integers representing an elevation map where the width of each
bar is 1,
 * compute how much water it is able to trap after raining.

 For example,
 Given [0,1,0,2,1,0,1,3,2,1,2,1], return 6.

 The above elevation map is represented by array [0,1,0,2,1,0,1,3,2,1,2,1].
 In this case, 6 units of rain water (blue section) are being trapped.
 */

public class _42 {

    public static class Solution1 {
        /**
         * O(n) time and O(1) space, awesome!
         *
         * 1. first scan to find the max height index
         * 2. then scan from left up to max index and find all the water units up to the
```

```
max height
         * 3. then scan from right down to max index and find all the water units down to
the max height
         * 4. return the sum of those above two
         *
         * reference: https://discuss.leetcode.com/topic/22976/my-accepted-java-solution
         */
        public int trap(int[] height) {
            if (height == null || height.length <= 2) {
                return 0;
            }

            int max = height[0];
            int maxIndex = 0;
            for (int i = 0; i < height.length; i++) {
                if (height[i] > max) {
                    max = height[i];
                    maxIndex = i;
                }
            }

            int water = 0;

            int leftMax = height[0];
            for (int i = 0; i < maxIndex; i++) {
                if (height[i] > leftMax) {
                    leftMax = height[i];
                } else {
                    water += leftMax - height[i];
                }
            }

            int rightMax = height[height.length - 1];
            for (int i = height.length - 1; i > maxIndex; i--) {
                if (height[i] > rightMax) {
                    rightMax = height[i];
                } else {
                    water += rightMax - height[i];
                }
            }

            return water;
        }
    }
}

// ============================================================
// FILE 43: _43.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 43. Multiply Strings
```

```
 *
 * Given two non-negative integers num1 and num2 represented as strings, return the
product of num1 and num2.

 Note:

 The length of both num1 and num2 is < 110.
 Both num1 and num2 contains only digits 0-9.
 Both num1 and num2 does not contain any leading zero.
 You must not use any built-in BigInteger library or convert the inputs to integer
directly.

 */
public class _43 {

    /**Inspired by https://discuss.leetcode.com/topic/30508/easiest-java-solution-with-
graph-explanation
     * Basically, the rule we can find is that products of each two digits will land in
this position in the final product:
     * i+j and i+j+1*/
    public String multiply(String num1, String num2) {
        if (isZero(num1) || isZero(num2)) {
            return "0";
        }
        int[] a1 = new int[num1.length()];
        int[] a2 = new int[num2.length()];
        int[] product = new int[num1.length() + num2.length()];

        for (int i = a1.length - 1; i >= 0; i--) {
            for (int j = a2.length - 1; j >= 0; j--) {
                int thisProduct = Character.getNumericValue(num1.charAt(i)) *
Character.getNumericValue(num2.charAt(j));
                product[i + j + 1] += thisProduct % 10;
                if (product[i + j + 1] >= 10) {
                    product[i + j + 1] %= 10;
                    product[i + j]++;
                }
                product[i + j] += thisProduct / 10;
                if (product[i + j] >= 10) {
                    product[i + j] %= 10;
                    product[i + j - 1]++;
                }
            }
        }

        StringBuilder stringBuilder = new StringBuilder();
        for (int i = 0; i < product.length; i++) {
            if (i == 0 && product[i] == 0) {
                continue;
            }
            stringBuilder.append(product[i]);
        }
        return stringBuilder.toString();
    }
```

```
    private boolean isZero(String num) {
        for (char c : num.toCharArray()) {
            if (c != '0') {
                return false;
            }
        }
        return true;
    }

}

// ============================================================
// FILE 44: _44.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 44. Wildcard Matching
 * Implement wildcard pattern matching with support for '?' and '*'.

 '?' Matches any single character.
 '*' Matches any sequence of characters (including the empty sequence).

 The matching should cover the entire input string (not partial).

 The function prototype should be:
 bool isMatch(const char *s, const char *p)

 Some examples:
 isMatch("aa","a") → false
 isMatch("aa","aa") → true
 isMatch("aaa","aa") → false
 isMatch("aa", "*") → true
 isMatch("aa", "a*") → true
 isMatch("ab", "?*") → true
 isMatch("aab", "c*a*b") → false
 */
public class _44 {

  public static class Solution1 {
    public boolean isMatch(String s, String p) {
      boolean[][] match = new boolean[s.length() + 1][p.length() + 1];
      match[s.length()][p.length()] = true;
      for (int i = p.length() - 1; i >= 0; i--) {
        if (p.charAt(i) != '*') {
          break;
        } else {
          match[s.length()][i] = true;
        }
      }

      for (int i = s.length() - 1; i >= 0; i--) {
```

```
        for (int j = p.length() - 1; j >= 0; j--) {
          if (s.charAt(i) == p.charAt(j) || p.charAt(j) == '?') {
            match[i][j] = match[i + 1][j + 1];
          } else if (p.charAt(j) == '*') {
            match[i][j] = match[i + 1][j] || match[i][j + 1];
          } else {
            match[i][j] = false;
          }
        }
      }
      return match[0][0];
    }
  }
}

// ======================================================
// FILE 45: _45.java
// ======================================================

package com.fishercoder.solutions;

/**
 * 45. Jump Game II
 *
 * Given an array of non-negative integers, you are initially positioned at the first
index of the array.
 * Each element in the array represents your maximum jump length at that position.
 * Your goal is to reach the last index in the minimum number of jumps.

 For example:
 Given array A = [2,3,1,1,4]

 The minimum number of jumps to reach the last index is 2. (Jump 1 step from index 0 to
1, then 3 steps to the last index.)

 Note:
 You can assume that you can always reach the last index.

 */
public class _45 {

  public static class Solution1 {
    public int jump(int[] nums) {
      int stepCount = 0;
      int lastJumpMax = 0;
      int currentJumpMax = 0;
      for (int i = 0; i < nums.length - 1; i++) {
        currentJumpMax = Math.max(currentJumpMax, i + nums[i]);
        if (i == lastJumpMax) {
          stepCount++;
          lastJumpMax = currentJumpMax;
        }
      }
      return stepCount;
```

```
    }
  }
}

// ============================================================
// FILE 46: _46.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 46. Permutations
 *
 * Given a collection of distinct numbers, return all possible permutations.

 For example,
 [1,2,3] have the following permutations:
 [
 [1,2,3],
 [1,3,2],
 [2,1,3],
 [2,3,1],
 [3,1,2],
 [3,2,1]
 ]

 */

public class _46 {

    public static class Solution1 {
        public List<List<Integer>> permute(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            result.add(new ArrayList<>());
            return backtracking(result, nums, 0);
        }

        private List<List<Integer>> backtracking(List<List<Integer>> result, int[] nums,
int pos) {
            if (pos == nums.length) {
                return result;
            }
            List<List<Integer>> newResult = new ArrayList();
            for (List<Integer> eachList : result) {
                for (int i = 0; i <= eachList.size(); i++) {
                    //attn: i starts from 0
                    List<Integer> newList = new ArrayList(eachList);
                    newList.add(i, nums[pos]);
                    newResult.add(newList);
                }
            }
```

```
            result = newResult;
            return backtracking(result, nums, pos + 1);
        }
    }

}

// ============================================================
// FILE 47: _47.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 47. Permutations II
 *
 * Given a collection of numbers that might contain duplicates, return all possible
unique permutations.

 For example,
 [1,1,2] have the following unique permutations:
 [
 [1,1,2],
 [1,2,1],
 [2,1,1]
 ]*/
public class _47 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/31445/really-easy-java-solution-
much-easier-than-the-solutions-with-very-high-vote
         */
        public List<List<Integer>> permuteUnique(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null || nums.length == 0) {
                return result;
            }
            boolean[] used = new boolean[nums.length];
            List<Integer> list = new ArrayList();
            Arrays.sort(nums);
            dfs(nums, used, list, result);
            return result;
        }


        private void dfs(int[] nums, boolean[] used, List<Integer> list,
List<List<Integer>> result) {
            if (list.size() == nums.length) {
                result.add(new ArrayList(list));
                return;
```

```
            }
            for (int i = 0; i < nums.length; i++) {
                if (used[i]) {
                    continue;
                }
                if (i > 0 && nums[i - 1] == nums[i] && !used[i - 1]) {
                    continue;
                }
                /**
                 * For this line, both !used[i-1] and used[i-1] will AC. It is because
the first one makes sure when
                 * duplicates are selected, the order is ascending (index from small to
large). However,
                 * the second one means the descending order.
                 */
                used[i] = true;
                list.add(nums[i]);
                dfs(nums, used, list, result);
                used[i] = false;
                list.remove(list.size() - 1);
            }
        }
    }
}

// ============================================================
// FILE 48: _48.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 48. Rotate Image

 You are given an n x n 2D matrix representing an image.

 Rotate the image by 90 degrees (clockwise).

 Note:
 You have to rotate the image in-place, which means you have to modify the
 input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

 Example 1:
 Given input matrix =
 [
 [1,2,3],
 [4,5,6],
 [7,8,9]
 ],
 rotate the input matrix in-place such that it becomes:
 [
 [7,4,1],
```

```
 [8,5,2],
 [9,6,3]
 ]

 Example 2:
 Given input matrix =
 [
 [ 5, 1, 9,11],
 [ 2, 4, 8,10],
 [13, 3, 6, 7],
 [15,14,12,16]
 ],
 rotate the input matrix in-place such that it becomes:
 [
 [15,13, 2, 5],
 [14, 3, 4, 1],
 [12, 6, 8, 9],
 [16, 7,10,11]
 ]
 */
public class _48 {

    /**Note: this is an n*n matrix, in other words, it's a square, this makes it easier
as well.*/

    public static class Solution1 {
        //Time: O(n^2)
        //Space: O(1)
        public void rotate(int[][] matrix) {
            /**First swap the elements on the diagonal, then reverse each row:
             * 1, 2, 3                    1, 4, 7                      7, 4, 1
             * 4, 5, 6         becomes    2, 5, 8           becomes    8, 5, 2
             * 7, 8, 9                    3, 6, 9                      9, 6, 3
             **/
            int m = matrix.length;
            for (int i = 0; i < m; i++) {
                for (int j = i; j < m; j++) {
                    /**ATTN: j starts from i, so that the diagonal changes with itself,
results in no change.*/
                    int tmp = matrix[i][j];
                    matrix[i][j] = matrix[j][i];
                    matrix[j][i] = tmp;
                }
            }
            CommonUtils.print2DIntArray(matrix);

            /**then reverse*/
            for (int i = 0; i < m; i++) {
                int left = 0;
                int right = m - 1;
                while (left < right) {
                    int tmp = matrix[i][left];
                    matrix[i][left] = matrix[i][right];
                    matrix[i][right] = tmp;
```

```
                left++;
                right--;
            }
        }
    }
}

public static class Solution2 {
    /**First swap the rows bottom up, then swap the element on the diagonal:
     *
     * 1, 2, 3                          7, 8, 9                          7, 4, 1
     * 4, 5, 6           becomes        4, 5, 6           becomes        8, 5, 2
     * 7, 8, 9                          1, 2, 3                          9, 6, 3
     *
     * Time: O(n^2)
     * Space: O(1)
     */
    public void rotate(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        int top = 0;
        int bottom = n - 1;
        while (top < bottom) {
            int[] tmp = matrix[top];
            matrix[top] = matrix[bottom];
            matrix[bottom] = tmp;
            top++;
            bottom--;
        }

        for (int i = 0; i < m; i++) {
            for (int j = i + 1; j < n; j++) {
                int tmp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = tmp;
            }
        }
    }
}

public static class Solution3 {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n / 2; i++) {
            for (int j = i; j < n - i - 1; j++) {
                //save the top
                int top = matrix[i][j];

                //move left to top
                matrix[i][j] = matrix[n - 1 - j][i];

                //move bottom to left
                matrix[n - 1 - j][i] = matrix[n - i - 1][n - 1 - j];
```

```
                    //move right to bottom
                    matrix[n - i - 1][n - 1 - j] = matrix[j][n - i - 1];

                    //move top to right
                    matrix[j][n - i - 1] = top;
                }
            }
        }
    }

}

// ============================================================
// FILE 49: _49.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 49. Group Anagrams
 *
 * Given an array of strings, group anagrams together.

 For example, given: ["eat", "tea", "tan", "ate", "nat", "bat"],
 Return:

 [
 ["ate", "eat","tea"],
 ["nat","tan"],
 ["bat"]
 ]

 Note: All inputs will be in lower-case.
 */

public class _49 {

  public static class Solution1 {
    public List<List<String>> groupAnagrams(String[] strs) {
      Map<String, List<String>> map = new HashMap<>();
      for (String word : strs) {
        char[] c = word.toCharArray();
        Arrays.sort(c);
        String key = new String(c);
        if (!map.containsKey(key)) {
          map.put(key, new ArrayList<>());
        }
        map.get(key).add(word);
```

```
      }
      return new ArrayList<>(map.values());
    }
  }
}

// ============================================================
// FILE 50: _50.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 50. Pow(x, n)
 *
 * Implement pow(x, n).

 Example 1:

 Input: 2.00000, 10
 Output: 1024.00000

 Example 2:

 Input: 2.10000, 3
 Output: 9.26100
 */
public class _50 {

  public static class Solution1 {
    /**
     * Time: O(logn)
     * Space: O(logn)
     */
    public double myPow(double x, int n) {
      long N = n;
      if (N < 0) {
        x = 1 / x;
        N = -N;
      }
      return fastPow(x, N);
    }

    private double fastPow(double x, long n) {
      if (n == 0) {
        return 1.0;
      }
      double half = fastPow(x, n / 2);
      if (n % 2 == 0) {
        return half * half;
      } else {
        return half * half * x;
      }
    }
```

```
  }

  public static class Solution2 {
    /**
     * Time: O(logn)
     * Space: O(1)
     */
    public double myPow(double x, int n) {
      long N = n;
      if (N < 0) {
        x = 1 / x;
        N = -N;
      }
      double answer = 1;
      double currentProduct = x;
      for (long i = N; i > 0; i /= 2) {
        if (i % 2 == 1) {
          answer = answer * currentProduct;
        }
        currentProduct *= currentProduct;
      }
      return answer;
    }
  }
}

// ============================================================
// FILE 51: _51.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 51. N-Queens
 *
 * The n-queens puzzle is the problem of placing n queens on an n×n chessboard such that
no two queens attack each other.

 Given an integer n, return all distinct solutions to the n-queens puzzle.
 Each solution contains a distinct board configuration of the n-queens' placement, where
'Q' and '.' both indicate a queen and an empty space respectively.

 For example,
 There exist two distinct solutions to the 4-queens puzzle:

 [
 [".Q..",  // Solution 1
 "...Q",
 "Q...",
 "..Q."],
```

```
 ["..Q.",  // Solution 2
 "Q...",
 "...Q",
 ".Q.."]
 ]
 */
public class _51 {

  public static class Solution1 {

    public List<List<String>> solveNQueens(int n) {
      List<List<String>> result = new ArrayList<>();
      if (n <= 0) {
        return result;
      }
      search(n, new ArrayList<>(), result);
      return result;
    }

    private void search(int n, ArrayList<Integer> col, List<List<String>> result) {
      if (col.size() == n) {
        result.add(drawChessBoard(col));
        return;
      }

      for (int i = 0; i < n; i++) {
        if (!isValid(col, i)) {
          continue;
        }
        col.add(i);
        search(n, col, result);
        col.remove(col.size() - 1);
      }
    }

    private boolean isValid(ArrayList<Integer> col, int next) {
      int row = col.size();
      for (int i = 0; i < row; i++) {
        if (next == col.get(i)) {
          return false;
        }

        if (i - row == col.get(i) - next) {
          return false;
        }

        if (i - row == next - col.get(i)) {
          return false;
        }
      }
      return true;
    }

    private ArrayList<String> drawChessBoard(ArrayList<Integer> col) {
```

```
      ArrayList<String> chessBoard = new ArrayList<>();

      for (int i = 0; i < col.size(); i++) {
        String row = "";
        for (int j = 0; j < col.size(); j++) {
          if (col.get(j) == i) {
            row += "Q";
          } else {
            row += ".";
          }
        }
        chessBoard.add(row);
      }
      return chessBoard;
    }
  }
}

// ============================================================
// FILE 52: _52.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 52. N-Queens II
 *
 * Follow up for N-Queens problem.
 * Now, instead outputting board configurations, return the total number of distinct
solutions.
 */
public class _52 {

  public static class Solution1 {
    /**credit: https://discuss.leetcode.com/topic/29626/easiest-java-solution-1ms-98-22*/
    int count = 0;

    public int totalNQueens(int n) {
      boolean[] cols = new boolean[n];
      boolean[] diagnol = new boolean[2 * n];
      boolean[] antiDiagnol = new boolean[2 * n];
      backtracking(0, cols, diagnol, antiDiagnol, n);
      return count;
    }

    private void backtracking(int row, boolean[] cols, boolean[] diagnol, boolean[]
antiDiagnol,
        int n) {
      if (row == n) {
        count++;
      }
      for (int col = 0; col < n; col++) {
        int x = col - row + n;
        int y = col + row;
```

```
        if (cols[col] || diagnol[x] || antiDiagnol[y]) {
          continue;
        }
        cols[col] = true;
        diagnol[x] = true;
        antiDiagnol[y] = true;
        backtracking(row + 1, cols, diagnol, antiDiagnol, n);
        cols[col] = false;
        diagnol[x] = false;
        antiDiagnol[y] = false;
      }
    }
  }
}

// ============================================================
// FILE 53: _53.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 53. Maximum Subarray
 *
 * Find the contiguous subarray within an array (containing at least one number) which
has the largest sum.
 * For example, given the array [-2,1,-3,4,-1,2,1,-5,4], the contiguous subarray
[4,-1,2,1] has the largest sum = 6.
 */

public class _53 {

  public static class Solution1 {
    /**credit: https://discuss.leetcode.com/topic/5000/accepted-o-n-solution-in-java*/
    public int maxSubArray(int[] nums) {
      int maxSoFar = nums[0];
      int maxEndingHere = nums[0];
      for (int i = 1; i < nums.length; i++) {
        maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);
        maxSoFar = Math.max(maxEndingHere, maxSoFar);
      }
      return maxSoFar;
    }
  }
}

// ============================================================
// FILE 54: _54.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
```

```java
/**
 * 54. Spiral Matrix
 *
 * Given a matrix of m x n elements (m rows, n columns), return all elements of the
matrix in spiral order.

 For example,
 Given the following matrix:

 [
 [ 1, 2, 3 ],
 [ 4, 5, 6 ],
 [ 7, 8, 9 ]
 ]

 You should return [1,2,3,6,9,8,7,4,5].
 */
public class _54 {

  public static class Solution1 {
    /**credit: https://leetcode.com/problems/spiral-matrix/discuss/20599/Super-Simple-
and-Easy-to-Understand-Solution/185257*/
    public List<Integer> spiralOrder(int[][] matrix) {
      if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
        return new ArrayList<>();
      }
      int m = matrix.length;
      int n = matrix[0].length;
      List<Integer> result = new ArrayList();
      int left = 0;
      int right = n - 1;
      int top = 0;
      int bottom = m - 1;
      while (result.size() < m * n) {
        for (int j = left; j <= right && result.size() < m * n; j++) {
          result.add(matrix[top][j]);
        }
        top++;
        for (int i = top; i <= bottom && result.size() < m * n; i++) {
          result.add(matrix[i][right]);
        }
        right--;
        for (int j = right; j >= left && result.size() < m * n; j--) {
          result.add(matrix[bottom][j]);
        }
        bottom--;
        for (int i = bottom; i >= top && result.size() < m * n; i--) {
          result.add(matrix[i][left]);
        }
        left++;
      }
      return result;
    }
```

```
  }
}

// ============================================================
// FILE 55: _55.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 55. Jump Game
 *
 * Given an array of non-negative integers, you are initially positioned at the first
index of the array.
 * Each element in the array represents your maximum jump length at that position.
 * Determine if you are able to reach the last index.
 *
 * For example:
 * A = [2,3,1,1,4], return true.
 * A = [3,2,1,0,4], return false.*/
public class _55 {

    public static class Solution1 {
        public boolean canJump(int[] nums) {
            int farthest = nums[0];
            for (int i = 0; i < nums.length; i++) {
                if (i <= farthest && nums[i] + i > farthest) {
                    //i <= farthest is to make sure that this current i is within the
current range
                    // nums[i]+i > farthest is to make sure that it's necessary to update
farthest with current nums[i]+i
                    farthest = nums[i] + i;
                }
            }
            return farthest >= nums.length - 1;
        }
    }
}

// ============================================================
// FILE 56: _56.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 56. Merge Intervals
 *
 * Given a collection of intervals, merge all overlapping intervals.
```

```
 For example,
 Given [1,3],[2,6],[8,10],[15,18],
 return [1,6],[8,10],[15,18].
 */
public class _56 {

  public static class Solution1 {
    public List<Interval> merge(List<Interval> intervals) {
        if (intervals.size() <= 1) {
            return intervals;
        }

        Collections.sort(intervals, (o1, o2) -> o1.start - o2.start);

        List<Interval> result = new ArrayList();
        for (int i = 0; i < intervals.size(); i++) {
            int start = intervals.get(i).start;
            int end = intervals.get(i).end;
            while (i < intervals.size() && end >= intervals.get(i).start) {
                end = Math.max(end, intervals.get(i).end);
                i++;
            }
            result.add(new Interval(start, end));
            i--;
        }
        return result;
    }
  }

}

// ============================================================
// FILE 57: _57.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.ArrayList;
import java.util.List;

/**
 * 57. Insert Interval
 *
 * Given a set of non-overlapping intervals, insert a new interval into the intervals
(merge if necessary).

 You may assume that the intervals were initially sorted according to their start times.

 Example 1:
 Given intervals [1,3],[6,9], insert and merge [2,5] in as [1,5],[6,9].
```

```
 Example 2:
 Given [1,2],[3,5],[6,7],[8,10],[12,16], insert and merge [4,9] in as [1,2],[3,10],
[12,16].

 This is because the new interval [4,9] overlaps with [3,5],[6,7],[8,10].
 */
public class _57 {

  public static class Solution1 {
    public List<Interval> insert(List<Interval> intervals, Interval newInterval) {
      List<Interval> result = new ArrayList<>();
      int i = 0;
      // add all the intervals ending before newInterval starts
      while (i < intervals.size() && intervals.get(i).end < newInterval.start) {
        result.add(intervals.get(i++));
      }
      // merge all overlapping intervals to one considering newInterval
      while (i < intervals.size() && intervals.get(i).start <= newInterval.end) {
        newInterval = new Interval( // we could mutate newInterval here also
            Math.min(newInterval.start, intervals.get(i).start),
            Math.max(newInterval.end, intervals.get(i).end));
        i++;
      }
      result.add(newInterval);
      // add all the rest
      while (i < intervals.size()) {
        result.add(intervals.get(i++));
      }
      return result;
    }
  }
}

// ============================================================
// FILE 58: _58.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 58. Length of Last Word
 *
 * Given a string s consists of upper/lower-case alphabets and empty space characters '
', return the length of last word in the string.
 * If the last word does not exist, return 0.

 Note: A word is defined as a character sequence consists of non-space characters only.

 For example,
 Given s = "Hello World",
 return 5.

 */
public class _58 {
```

```
    public static class Solution1 {
        public int lengthOfLastWord(String s) {
            if (s == null || s.length() == 0) {
                return 0;
            }
            s = s.trim();
            int n = s.length() - 1;
            while (n >= 0 && s.charAt(n) != ' ') {
                n--;
            }
            return s.length() - n - 1;
        }
    }

}

// ============================================================
// FILE 59: _59.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 59. Spiral Matrix II
 *
 * Given an integer n, generate a square matrix filled with elements from 1 to n2 in
spiral order.

 For example,
 Given n = 3,

 You should return the following matrix:
 [
 [ 1, 2, 3 ],
 [ 8, 9, 4 ],
 [ 7, 6, 5 ]
 ]
 */
public class _59 {

  public static class Solution1 {
    /**credit: https://leetcode.com/problems/spiral-matrix-ii/discuss/22289/My-Super-
Simple-Solution.-Can-be-used-for-both-Spiral-Matrix-I-and-II/21907*/
    public int[][] generateMatrix(int n) {
      int[][] matrix = new int[n][n];
      if (n == 0) {
        return matrix;
      }
      int value = 1;
      int top = 0;
      int bottom = n - 1;
      int left = 0;
      int right = n - 1;
      while (left <= right && top <= bottom) {
```

```
        for (int j = left; j <= right; j++) {
          matrix[top][j] = value++;
        }
        top++;
        for (int i = top; i <= bottom; i++) {
          matrix[i][right] = value++;
        }
        right--;
        for (int j = right; j >= left; j--) {
          matrix[bottom][j] = value++;
        }
        bottom--;
        for (int i = bottom; i >= top; i--) {
          matrix[i][left] = value++;
        }
        left++;
      }
      return matrix;
    }
  }
}

// ============================================================
// FILE 60: _60.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 60. Permutation Sequence
 *
 * The set [1,2,3,…,n] contains a total of n! unique permutations.

 By listing and labeling all of the permutations in order,
 We get the following sequence (ie, for n = 3):

 "123"
 "132"
 "213"
 "231"
 "312"
 "321"
 Given n and k, return the kth permutation sequence.

 Note: Given n will be between 1 and 9 inclusive.
 */
public class _60 {

    public static class Solution1 {
        public String getPermutation(int n, int k) {
            int[] nums = new int[n + 1];
            int permcount = 1;
            for (int i = 0; i < n; i++) {
                nums[i] = i + 1; // put 1, 2, 3 ... n into nums[]
```

```
                permcount *= (i + 1);
            }

            k--;
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < n; i++) {
                permcount = permcount / (n - i);
                int idx = k / permcount;// the index that this position should
                // choose
                sb.append(nums[idx]);
                // left shift nums[] by one bit
                for (int j = idx; j < n - i; j++) {
                    nums[j] = nums[j + 1];
                }
                k %= permcount;
            }
            return sb.toString();
        }
    }
}

// ============================================================
// FILE 61: _61.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 61. Rotate List
 *
 * Given a list, rotate_naive the list to the right by k places, where k is non-negative.

 For example:
 Given 1->2->3->4->5->NULL and k = 2,
 return 4->5->1->2->3->NULL.
 */
public class _61 {

  public static class Solution1 {
    //credit: https://discuss.leetcode.com/topic/26364/clean-java-solution-with-brief-
explanation
    //link the tail of the linked list to the head to form a circle, then count to find
the pint and cut it
    public ListNode rotateRight(ListNode head, int k) {
      if (head == null) {
        return head;
      }
      ListNode copyHead = head;
      int len = 1;
      while (copyHead.next != null) {
        copyHead = copyHead.next;
        len++;
```

```
      }
      copyHead.next = head;//link the tail and head to make it a circle
      for (int i = len - k % len; i > 1; i--) {
        head = head.next;
      }
      copyHead = head.next;
      head.next = null;//break the circle
      return copyHead;
    }
  }
}

// ============================================================
// FILE 62: _62.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 62. Unique Paths

A robot is located at the top-left corner of a m x n grid (marked 'Start' in the diagram
below).
The robot can only move either down or right at any point in time. The robot is trying to
reach
the bottom-right corner of the grid (marked 'Finish' in the diagram below).

How many possible unique paths are there?
 */
public class _62 {

  public static class Solution1 {
    /**
     * Another typical DP question, use a 2d array: the first row and the first column
need to be
     * initialized to be 1 since there's only one way to reach every position in the
first row and
     * the first column: either from left or top.
     */
    public int uniquePaths(int m, int n) {
      int[][] dp = new int[m][n];
      for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
          if (i == 0 || j == 0) {
            dp[i][j] = 1;
          } else {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
          }
        }
      }
      return dp[m - 1][n - 1];
    }
  }
}
```

```
// ============================================================
// FILE 63: _63.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 63. Unique Paths II

 Follow up for "Unique Paths":

Now consider if some obstacles are added to the grids. How many unique paths would there be?

An obstacle and empty space is marked as 1 and 0 respectively in the grid.

For example,
There is one obstacle in the middle of a 3x3 grid as illustrated below.

[
  [0,0,0],
  [0,1,0],
  [0,0,0]
]
The total number of unique paths is 2.

Note: m and n will be at most 100.*/
public class _63 {
  public static class Solution1 {
    /**
     * Idea: grid[i][j] has to be set to zero if obstacleGrid[i][j] == 1, otherwise, we can get
     * dp[i][j] from its top and left dp.
     */
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
      if (obstacleGrid == null || obstacleGrid.length == 0) {
        return 0;
      }

      int height = obstacleGrid.length;
      int width = obstacleGrid[0].length;
      int[][] dp = new int[height][width];
      dp[0][0] = obstacleGrid[0][0] == 1 ? 0 : 1;
      for (int i = 1; i < height; i++) {
        dp[i][0] = obstacleGrid[i][0] == 1 ? 0 : dp[i - 1][0];
      }
      for (int j = 1; j < width; j++) {
        dp[0][j] = obstacleGrid[0][j] == 1 ? 0 : dp[0][j - 1];
      }

      for (int i = 1; i < height; i++) {
        for (int j = 1; j < width; j++) {
          if (obstacleGrid[i][j] == 1) {
```

```
            dp[i][j] = 0;
          } else {
            int paths = 0;
            if (obstacleGrid[i - 1][j] == 0) {
              paths += dp[i - 1][j];
            }
            if (obstacleGrid[i][j - 1] == 0) {
              paths += dp[i][j - 1];
            }
            dp[i][j] = paths;
          }
        }
      }
      return dp[height - 1][width - 1];
    }
  }
}

// ============================================================
// FILE 64: _64.java
// ============================================================

package com.fishercoder.solutions;

/**64. Minimum Path Sum

Given a m x n grid filled with non-negative numbers, find a path from top left to bottom
right which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.*/
public class _64 {
  public static class Solution1 {
    /**
     * Same idea as _70: have to initialize the first row and the first column and start
the for
     * loop from i==1 and j==1 for the rest of the matrix.
     */
    public int minPathSum(int[][] grid) {
      if (grid == null || grid.length == 0) {
        return 0;
      }

      int height = grid.length;
      int width = grid[0].length;
      int[][] dp = new int[height][width];
      dp[0][0] = grid[0][0];
      for (int i = 1; i < height; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
      }
      for (int j = 1; j < width; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
      }
      for (int i = 1; i < height; i++) {
        for (int j = 1; j < width; j++) {
```

```
          dp[i][j] = Math.min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j];
        }
      }
      return dp[height - 1][width - 1];
    }
  }
}

// ============================================================
// FILE 65: _65.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 65. Valid Number
 *
 * Validate if a given string is numeric.
 *
 * Some examples: "0" => true " 0.1 " => true "abc" => false "1 a" => false "2e10" =>
true
 *
 * Note: It is intended for the problem statement to be ambiguous. You should gather all
 * requirements up front before implementing one.
 */

public class _65 {
  /**credit: https://discuss.leetcode.com/topic/9490/clear-java-solution-with-ifs*/
  public static class Solution1 {
    public boolean isNumber(String s) {
      s = s.trim();

      boolean pointSeen = false;
      boolean eSeen = false;
      boolean numberSeen = false;
      boolean numberAfterE = true;
      for (int i = 0; i < s.length(); i++) {
        if ('0' <= s.charAt(i) && s.charAt(i) <= '9') {
          numberSeen = true;
          numberAfterE = true;
        } else if (s.charAt(i) == '.') {
          if (eSeen || pointSeen) {
            return false;
          }
          pointSeen = true;
        } else if (s.charAt(i) == 'e') {
          if (eSeen || !numberSeen) {
            return false;
          }
          numberAfterE = false;
          eSeen = true;
        } else if (s.charAt(i) == '-' || s.charAt(i) == '+') {
          if (i != 0 && s.charAt(i - 1) != 'e') {
            return false;
```

```
          }
        } else {
          return false;
        }
      }

      return numberSeen && numberAfterE;
    }
  }

  public static class Solution2 {
    /** credit: https://discuss.leetcode.com/topic/2973/java-solution-with-one-line */
    public boolean isNumber(String s) {
      return s.matches("(\\s*)[+-]?((\\.[0-9]+)|([0-9]+(\\.[0-9]*)?))(e[+-]?[0-9]+)?
(\\s*)");
    }
  }
}

// ============================================================
// FILE 66: _66.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 66. Plus One
 *
 * Given a non-empty array of digits representing a non-negative integer, plus one to the
integer.
 * The digits are stored such that the most significant digit is at the head of the list,
 * and each element in the array contain a single digit.
 *
 * You may assume the integer does not contain any leading zero, except the number 0
itself.
 *
 * Example 1:
 * Input: [1,2,3]
 * Output: [1,2,4]
 * Explanation: The array represents the integer 123.
 *
 * Example 2:
 * Input: [4,3,2,1]
 * Output: [4,3,2,2]
 * Explanation: The array represents the integer 4321.
 */
public class _66 {

  public static class Solution1 {
      /**credit: https://leetcode.com/problems/plus-one/discuss/24082/My-Simple-Java-
Solution*/
    public int[] plusOne(int[] digits) {
        int len = digits.length;
        for (int i = len - 1; i >= 0; i--) {
```

```
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        int[] newNumber = new int[len + 1];
        newNumber[0] = 1;
        return newNumber;
    }
  }
}

// ============================================================
// FILE 67: _67.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 67. Add Binary
 *
 * Given two binary strings, return their sum (also a binary string).
 * For example,
 * a = "11"
 * b = "1"
 * Return "100".
 */

public class _67 {
  public static class Solution1 {
    /**
     * credit: https://discuss.leetcode.com/topic/13698/short-ac-solution-in-java-with-
explanation
     * 1. use StringBuilder.reverse() function! Nice!
     * 2. if a numeric number is represented/stored in String, how to get its value: use
Character.getNumericValue(s.charAt(i))
     * 3. directly adding/subtracting chars will end up working with their ASCII numbers,
e.g. chars[0] = 'a', chars[1] = 'b', then chars[0] + chars[1] will become 195.
     */
    public String addBinary(String a, String b) {
      int carry = 0;
      int i = a.length() - 1;
      int j = b.length() - 1;
      StringBuilder sb = new StringBuilder();
      while (i >= 0 || j >= 0) {
        int sum = carry;
        if (i >= 0) {
          sum += a.charAt(i--) - '0';
        }
        if (j >= 0) {
          sum += b.charAt(j--) - '0';
        }
        sb.append(sum % 2);
```

```
        carry = sum / 2;
      }
      if (carry != 0) {
        sb.append(carry);
      }
      return sb.reverse().toString();
    }
  }
}

// ============================================================
// FILE 68: _68.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 68. Text Justification

 Given an array of words and a length L, format the text such that each line has exactly
L characters and is fully (left and right) justified.
 You should pack your words in a greedy approach; that is, pack as many words as you can
in each line.
 Pad extra spaces ' ' when necessary so that each line has exactly L characters.
 Extra spaces between words should be distributed as evenly as possible.
 If the number of spaces on a line do not divide evenly between words,
 the empty slots on the left will be assigned more spaces than the slots on the right.
 For the last line of text, it should be left justified and no extra space is inserted
between words.

 For example,
 words: ["This", "is", "an", "example", "of", "text", "justification."]
 L: 16.

 Return the formatted lines as:
 [
 "This    is    an",
 "example  of text",
 "justification.  "
 ]

 Note: Each word is guaranteed not to exceed L in length.

 Corner Cases:
 A line other than the last line might contain only one word. What should you do in this
case?
 In this case, that line should be left-justified.
 */
public class _68 {

  public static class Solution1 {
```

```
    public List<String> fullJustify(String[] words, int L) {
        ArrayList<String> result = new ArrayList();
        if (words == null || words.length == 0) {
            return result;
        }
        int count = 0;
        int last = 0;
        for (int i = 0; i < words.length; i++) {
            if (count + words[i].length() + (i - last) > L) {
                int spaceNum = 0;
                int extraNum = 0;
                if (i - last - 1 > 0) {
                    spaceNum = (L - count) / (i - last - 1);
                    extraNum = (L - count) % (i - last - 1);
                }
                StringBuilder sb = new StringBuilder();
                for (int j = last; j < i; j++) {
                    sb.append(words[j]);
                    if (j < i - 1) {
                        for (int k = 0; k < spaceNum; k++) {
                            sb.append(" ");
                        }
                        if (extraNum > 0) {
                            sb.append(" ");
                        }
                        extraNum--;
                    }
                }
                for (int j = sb.length(); j < L; j++) {
                    sb.append(" ");
                }
                result.add(sb.toString());
                count = 0;
                last = i;
            }
            count += words[i].length();
        }
        StringBuilder sb = new StringBuilder();
        for (int i = last; i < words.length; i++) {
            sb.append(words[i]);
            if (sb.length() < L) {
                sb.append(" ");
            }
        }
        for (int i = sb.length(); i < L; i++) {
            sb.append(" ");
        }
        result.add(sb.toString());
        return result;
    }
  }

}
```

```
// ============================================================
// FILE 69: _69.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 69. Sqrt(x)
 *
 * Implement int sqrt(int x).
 * Compute and return the square root of x.
 */

public class _69 {
    public static class Solution1 {
        /**A few key points:
         * 1. all variable use long type, otherwise overflow, just cast to int before
returning
         * 2. left start from 0, not 1
         * 3. right start from x/2 + 1, not from x*/
        public int mySqrt(int x) {
            long left = 0;
            long right = x / 2 + 1;
            while (left <= right) {
                long mid = left + (right - left) / 2;
                long result = mid * mid;
                if (result == (long) x) {
                    return (int) mid;
                } else if (result > x) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }
            return (int) right;
        }
    }
}

// ============================================================
// FILE 70: _70.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 70. Climbing Stairs

 You are climbing a stair case. It takes n steps to reach to the top.

 Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to
the top?

 Note: Given n will be a positive integer.
```

```
 Example 1:

 Input: 2
 Output:  2
 Explanation:  There are two ways to climb to the top.

 1. 1 step + 1 step
 2. 2 steps

 Example 2:

 Input: 3
 Output:  3
 Explanation:  There are three ways to climb to the top.

 1. 1 step + 1 step + 1 step
 2. 1 step + 2 steps
 3. 2 steps + 1 step

 */

public class _70 {
  public static class Solution1 {
    //classical dp problem
    public int climbStairs(int n) {
      if (n < 1) {
        return 0;
      } else if (n < 4) {
        return n;
      } else {
        int[] dp = new int[n + 1];
        //the number of ways to reach step n could be calculated from n-1 and n-2
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
          dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
      }
    }
  }
}

// ============================================================
// FILE 71: _71.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Deque;
import java.util.HashSet;
import java.util.LinkedList;
```

```
import java.util.Set;

/**
 * 71. Simplify Path

 Given an absolute path for a file (Unix-style), simplify it.

 For example,
 path = "/home/", => "/home"
 path = "/a/./b/../../c/", => "/c"

 Corner Cases:
 Did you consider the case where path = "/../"?
 In this case, you should return "/".
 Another corner case is the path might contain multiple slashes '/' together, such as
"/home//foo/".
 In this case, you should ignore redundant slashes and return "/home/foo".
 */
public class _71 {

  public static class Solution1 {
    public String simplifyPath(String path) {
      Deque<String> stack = new LinkedList<>();
      Set<String> skipSet = new HashSet<>(Arrays.asList("..", ".", ""));
      for (String dir : path.split("/")) {
        if (dir.equals("..") && !stack.isEmpty()) {
          stack.pop();
        } else if (!skipSet.contains(dir)) {
          stack.push(dir);
        }
      }
      String result = "";
      for (String dir : stack) {
        result = "/" + dir + result;
      }
      return result.isEmpty() ? "/" : result;
    }
  }
}

// ============================================================
// FILE 72: _72.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 72. Edit Distance
 *
 * Given two words word1 and word2,
 * find the minimum number of steps required to convert word1 to word2.
 * (each operation is counted as 1 step.)

 You have the following 3 operations permitted on a word:
```

```
 a) Insert a character
 b) Delete a character
 c) Replace a character
 */

public class _72 {

    public static class Solution1 {
        public int minDistance(String word1, String word2) {
            int m = word1.length();
            int n = word2.length();
            if (m == 0) {
                return n;
            }
            if (n == 0) {
                return m;
            }
            char[] str1 = word1.toCharArray();
            char[] str2 = word2.toCharArray();

            int[][] table = new int[m + 1][n + 1];
            for (int i = 0; i < m + 1; i++) {
                table[i][0] = i;
            }
            for (int j = 0; j < n + 1; j++) {
                table[0][j] = j;
            }
            for (int i = 1; i < m + 1; i++) {
                for (int j = 1; j < n + 1; j++) {
                    int cost = 0;
                    if (str1[i - 1] != str2[j - 1]) {
                        cost = 1;
                    }
                    table[i][j] = Math.min(Math.min(table[i - 1][j] + 1, table[i][j - 1]
+ 1),
                            table[i - 1][j - 1] + cost);
                }
            }
            return table[m][n];
        }
    }
}

// ============================================================
// FILE 73: _73.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 73. Set Matrix Zeroes
 *
 * Given a m x n matrix, if an element is 0, set its entire row and column to 0. Do it in
```

```
place.

 Follow up:

 Did you use extra space?
 A straight forward solution using O(mn) space is probably a bad idea.
 A simple improvement uses O(m + n) space, but still not the best solution.
 Could you devise a constant space solution?
 */
public class _73 {

    public static class Solution1 {
        /**
         * Space: O(m*n)
         */
        public void setZeroes(int[][] matrix) {
            if (matrix == null || matrix.length == 0) {
                return;
            }
            int height = matrix.length;
            int width = matrix[0].length;
            boolean[][] zero = new boolean[height][width];
            for (int i = 0; i < height; i++) {
                for (int j = 0; j < width; j++) {
                    if (matrix[i][j] == 0) {
                        zero[i][j] = true;
                    }
                }
            }
            for (int i = 0; i < height; i++) {
                for (int j = 0; j < width; j++) {
                    if (zero[i][j]) {
                        for (int k = 0; k < height; k++) {
                            matrix[k][j] = 0;
                        }
                        for (int k = 0; k < width; k++) {
                            matrix[i][k] = 0;
                        }
                    }
                }
            }
        }
    }

    public static class Solution2 {
        /**
         * Space: O(m+n)
         */
        public void setZeroes(int[][] matrix) {
            if (matrix == null || matrix.length == 0) {
                return;
            }
            int m = matrix.length;
            int n = matrix[0].length;
```

```
        boolean[] row = new boolean[m];
        boolean[] col = new boolean[n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    row[i] = true;
                    col[j] = true;
                }
            }
        }

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (row[i] && col[j]) {
                    for (int k = 0; k < m; k++) {
                        matrix[k][j] = 0;
                    }
                    for (int k = 0; k < n; k++) {
                        matrix[i][k] = 0;
                    }
                }
            }
        }
    }
}

public static class Solution3 {
    /**
     * Space: O(1)
     */
    public void setZeroes(int[][] matrix) {
        if (matrix == null || matrix.length == 0) {
            return;
        }
        int m = matrix.length;
        int n = matrix[0].length;
        boolean firstRow = false;
        boolean firstCol = false;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    if (i == 0) {
                        firstRow = true;
                    }
                    if (j == 0) {
                        firstCol = true;
                    }
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }

        for (int i = 1; i < m; i++) {
```

```
                for (int j = 1; j < n; j++) {
                    if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                        matrix[i][j] = 0;
                    }
                }
            }

            if (firstRow) {
                for (int j = 0; j < n; j++) {
                    matrix[0][j] = 0;
                }
            }

            if (firstCol) {
                for (int i = 0; i < m; i++) {
                    matrix[i][0] = 0;
                }
            }
        }
    }

    public static class Solution4 {
        /**
         * Space: O(1)
         * credit: https://leetcode.com/problems/set-matrix-zeroes/discuss/26014/Any-
shorter-O(1)-space-solution
         */
        public void setZeroes(int[][] matrix) {
            int col0 = 1;
            int m = matrix.length;
            int n = matrix[0].length;
            /**the first iteration (first nested for loop) is to check from top row to
bottom row:
             * keep the first column state into variable col0;
             * then starting from the second column, check all the rest of the columns
and mark its top cell and its most-left cell if it
             * s a zero.*/
            for (int i = 0; i < m; i++) {
                if (matrix[i][0] == 0) {
                    col0 = 0;
                }

                for (int j = 1; j < n; j++) {
                    if (matrix[i][j] == 0) {
                        matrix[i][0] = 0;
                        matrix[0][j] = 0;
                    }
                }
            }

            /**the second iteration (second nested for loop) is to check from bottom row
to the top row
             * from the right-most column to the second left-most column: as long as its
left-most column cell or its top row cell is zero, then set that cell to be zero
```

```
             * at last, check col0 variable, if it's zero, mark that row cell as zero*/
            for (int i = m - 1; i >= 0; i--) {
                for (int j = n - 1; j >= 1; j--) {
                    if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                        matrix[i][j] = 0;
                    }
                }
                if (col0 == 0) {
                    matrix[i][0] = 0;
                }
            }
        }
    }
}

// ============================================================
// FILE 74: _74.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 74. Search a 2D Matrix
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix.
 * This matrix has the following properties:

 Integers in each row are sorted from left to right.
 The first integer of each row is greater than the last integer of the previous row.
 For example,

 Consider the following matrix:

 [
 [1,   3,  5,  7],
 [10, 11, 16, 20],
 [23, 30, 34, 50]
 ]

 Given target = 3, return true.

 */
public class _74 {

    public static class Solution1 {
        public boolean searchMatrix(int[][] matrix, int target) {
            if (matrix == null || matrix.length == 0
                || matrix[0].length == 0
                || matrix[0][0] > target
                || matrix[matrix.length - 1][matrix[0].length - 1] < target) {
                return false;
            }
            int m = matrix.length;
            int n = matrix[0].length;
```

```
            int left = 0;
            int right = m * n - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;
                int row = mid / n;
                int col = mid % n;
                if (matrix[row][col] == target) {
                    return true;
                } else if (matrix[row][col] > target) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 75: _75.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 75. Sort Colors
 *
 * Given an array with n objects colored red, white or blue,
 * sort them so that objects of the same color are adjacent, with the colors in the order
red, white and blue.

 Here, we will use the integers 0, 1, and 2 to represent the color red, white, and blue
respectively.

 Note:
 You are not suppose to use the library's sort function for this problem.

 Follow up:

 A rather straight forward solution is a two-pass algorithm using counting sort.
 First, iterate the array counting number of 0's, 1's, and 2's,
 then overwrite array with total number of 0's, then 1's and followed by 2's.
 Could you come up with an one-pass algorithm using only constant space?
 */

public class _75 {

  public static class Solution1 {
    public void sortColors(int[] nums) {
      int zero = 0;
      int two = nums.length - 1;
      for (int i = 0; i <= two; ) {
        if (nums[i] == 0 && i > zero) {
```

```
          swap(nums, i, zero++);
        } else if (nums[i] == 2 && i < two) {
          swap(nums, i, two--);
        } else {
          i++;
        }
      }
    }

    void swap(int[] nums, int m, int n) {
      int temp = nums[m];
      nums[m] = nums[n];
      nums[n] = temp;
    }
  }
}

// ============================================================
// FILE 76: _76.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 76. Minimum Window Substring
 *
 * Given a string S and a string T, find the minimum window in S which will contain all
the characters in T in complexity O(n).

 For example,
 S = "ADOBECODEBANC"
 T = "ABC"

 Minimum window is "BANC".

 Note:

 If there is no such window in S that covers all characters in T, return the empty string
"".
 If there are multiple such windows, you are guaranteed that there will always be only
one unique minimum window in S.
 */

public class _76 {

  public static class Solution1 {
    public String minWindow(String s, String t) {
      int[] counts = new int[256];
      for (char c : t.toCharArray()) {
        counts[c]++;
      }

      int start = 0;
      int end = 0;
```

```
      int minStart = 0;
      int minLen = Integer.MAX_VALUE;
      int counter = t.length();
      while (end < s.length()) {
        if (counts[s.charAt(end)] > 0) {
          counter--;
        }

        counts[s.charAt(end)]--;
        end++;

        while (counter == 0) {
          if (end - start < minLen) {
            minStart = start;
            minLen = end - start;
          }
          counts[s.charAt(start)]++;
          if (counts[s.charAt(start)] > 0) {
            counter++;
          }
          start++;
        }
      }

      if (minLen == Integer.MAX_VALUE) {
        return "";
      }
      return s.substring(minStart, minStart + minLen);
    }
  }
}

// ============================================================
// FILE 77: _77.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 77. Combinations
 *
 * Given two integers n and k, return all possible combinations of k numbers out of 1 ...
n.

 For example,
 If n = 4 and k = 2, a solution is:

 [
 [2,4],
 [3,4],
 [2,3],
```

```
 [1,2],
 [1,3],
 [1,4],
 ]
 */

public class _77 {

    public static class Solution1 {
        public List<List<Integer>> combine(int n, int k) {
            List<List<Integer>> result = new ArrayList();
            int[] nums = new int[n];
            for (int i = 0; i < n; i++) {
                nums[i] = i + 1;
            }
            backtracking(k, 0, nums, new ArrayList(), result);
            return result;
        }

        void backtracking(int k, int start, int[] nums, List<Integer> curr,
List<List<Integer>> result) {
            if (curr.size() == k) {
                result.add(new ArrayList(curr));
            } else if (curr.size() < k) {
                for (int i = start; i < nums.length; i++) {
                    curr.add(nums[i]);
                    backtracking(k, i + 1, nums, curr, result);
                    curr.remove(curr.size() - 1);
                }
            }
        }
    }
}

// ============================================================
// FILE 78: _78.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 78. Subsets
 *
 * Given a set of distinct integers, nums, return all possible subsets.
 * Note: The solution set must not contain duplicate subsets.

 For example,
 If nums = [1,2,3], a solution is:
 [
 [3],
 [1],
```

```
 [2],
 [1,2,3],
 [1,3],
 [2,3],
 [1,2],
 []
 ]
 */

public class _78 {

    public static class Solution1 {
        public List<List<Integer>> subsets(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null) {
                return result;
            }
            result.add(new ArrayList());
            for (int i = 0; i < nums.length; i++) {
                List<List<Integer>> temp = new ArrayList();
                //you'll have to create a new one here, otherwise, it'll throw
ConcurrentModificationException.
                for (List<Integer> list : result) {
                    List<Integer> newList = new ArrayList(list);
                    newList.add(nums[i]);
                    temp.add(newList);
                }
                result.addAll(temp);
            }
            return result;
        }
    }

    public static class Solution2 {
        /**This is the most straightforward solution and easy to follow.*/
        public List<List<Integer>> subsets(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            backtracking(result, new ArrayList(), nums, 0);
            return result;
        }

        void backtracking(List<List<Integer>> result, List<Integer> list, int[] nums, int
start) {
            result.add(new ArrayList(list));
            for (int i = start; i < nums.length; i++) {
                list.add(nums[i]);
                backtracking(result, list, nums, i + 1);
                list.remove(list.size() - 1);
            }
        }
    }

    public static class Solution3 {
        /**
```

```
         * This is just a slight modification of Solution2, pay close to attention to
notice the difference between them.
         */
        public List<List<Integer>> subsets(int[] nums) {
            List<List<Integer>> result = new ArrayList<>();
            List<Integer> list = new ArrayList<>();
            result.add(list);
            backtracking(result, list, nums, 0);
            return result;
        }

        private void backtracking(List<List<Integer>> result, List<Integer> list, int[]
nums, int start) {
            for (int i = start; i < nums.length; i++) {
                list.add(nums[i]);
                result.add(new ArrayList<>(list));
                backtracking(result, list, nums, i + 1);
                list.remove(list.size() - 1);
            }
        }
    }
}

// ============================================================
// FILE 79: _79.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 79. Word Search
 *
 * Given a 2D board and a word, find if the word exists in the grid.
 * The word can be constructed from letters of sequentially adjacent cell,
 * where "adjacent" cells are those horizontally or vertically neighboring.
 * The same letter cell may not be used more than once.

 For example,
 Given board =
 [
 ['A','B','C','E'],
 ['S','F','C','S'],
 ['A','D','E','E']
 ]

 word = "ABCCED", -> returns true,
 word = "SEE", -> returns true,
 word = "ABCB", -> returns false.
 */

public class _79 {

    public static class Solution1 {
```

```
      public boolean exist(char[][] board, String word) {
        int m = board.length;
        int n = board[0].length;
        boolean[][] visited = new boolean[m][n];
        for (int i = 0; i < m; i++) {
          for (int j = 0; j < n; j++) {
            if (dfs(board, visited, i, j, word, 0)) {
              return true;
            }
          }
        }
        return false;
      }

      final int[] dirs = new int[] {0, 1, 0, -1, 0};

      boolean dfs(char[][] board, boolean[][] visited, int row, int col, String word, int
index) {
        if (index >= word.length() || word.charAt(index) != board[row][col]) {
          return false;
        } else if (index == word.length() - 1 && word.charAt(index) == board[row][col]) {
          visited[row][col] = true;
          return true;
        }
        visited[row][col] = true;//set it to true for this case
        boolean result = false;
        for (int i = 0; i < 4; i++) {
          int nextRow = row + dirs[i];
          int nextCol = col + dirs[i + 1];
          if (nextRow < 0 || nextRow >= board.length || nextCol < 0 || nextCol >=
board[0].length || visited[nextRow][nextCol]) {
            continue;
          }
          result = dfs(board, visited, nextRow, nextCol, word, index + 1);
          if (result) {
            return result;
          } else {
            visited[nextRow][nextCol] = false;//set it back to false if this road doesn't
work to allow it for other paths, this is backtracking!!!
          }
        }
        return result;
      }
    }

    public static class Solution2 {
        //credit: https://discuss.leetcode.com/topic/21142/my-java-solution

        boolean[][] visited;

        public boolean exist(char[][] board, String word) {
            int m = board.length;
            int n = board[0].length;
            visited = new boolean[m][n];
```

```
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (word.charAt(0) == board[i][j] && search(board, word, i, j, 0)) {
                        return true;
                    }
                }
            }
            return false;
        }

        boolean search(char[][] board, String word, int i, int j, int pos) {
            if (pos == word.length()) {
                return true;
            }
            if (i < 0 || j < 0 || i >= board.length || j >= board[0].length ||
word.charAt(pos) != board[i][j] || visited[i][j]) {
                return false;
            }
            visited[i][j] = true;
            if (search(board, word, i + 1, j, pos + 1)
                || search(board, word, i - 1, j, pos + 1)
                || search(board, word, i, j + 1, pos + 1)
                || search(board, word, i, j - 1, pos + 1)) {
                return true;
            }

            visited[i][j] = false;
            return false;
        }
    }
}

// ============================================================
// FILE 80: _80.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 80. Remove Duplicates from Sorted Array II
 *
 * Follow up for "Remove Duplicates":
 What if duplicates are allowed at most twice?

 For example,
 Given sorted array nums = [1,1,1,2,2,3],

 Your function should return length = 5,
 with the first five elements of nums being 1, 1, 2, 2 and 3. It doesn't matter what you
leave beyond the new length.
 */
```

```
public class _80 {

  public static class Solution1 {
    public int removeDuplicates(int[] nums) {
      int counter = 0;
      int len = nums.length;
      if (len == 0) {
        return 0;
      }
      if (len == 1) {
        return 1;
      }
      if (len == 2) {
        return 2;
      }

      List<Integer> a = new ArrayList();
      a.add(nums[0]);
      a.add(nums[1]);
      for (int i = 2; i < len; i++) {
        if (nums[i] != nums[i - 1]) {
          a.add(nums[i]);
        } else if (nums[i] != nums[i - 2]) {
          a.add(nums[i]);
        }
      }

      counter = a.size();
      for (int i = 0; i < counter; i++) {
        nums[i] = a.get(i);
      }
      return counter;
    }
  }

}

// ============================================================
// FILE 81: _81.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 81. Search in Rotated Sorted Array II
 *
 * Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.
 *
 * (i.e., [0,0,1,2,2,5,6] might become [2,5,6,0,0,1,2]).
 *
 * You are given a target value to search. If found in the array return true, otherwise
return false.
 *
```

```
 * Example 1:
 *
 * Input: nums = [2,5,6,0,0,1,2], target = 0
 * Output: true
 * Example 2:
 *
 * Input: nums = [2,5,6,0,0,1,2], target = 3
 * Output: false
 * Follow up:
 *
 * This is a follow up problem to Search in Rotated Sorted Array, where nums may contain
duplicates.
 * Would this affect the run-time complexity? How and why?
 */
public class _81 {

  public static class Solution1 {
    public boolean search(int[] nums, int target) {
      int start = 0;
      int end = nums.length - 1;

      //check each num so we will check start == end
      //We always get a sorted part and a half part
      //we can check sorted part to decide where to go next
      while (start <= end) {
        int mid = start + (end - start) / 2;
        if (nums[mid] == target) {
          return true;
        }

        //if left part is sorted
        if (nums[start] < nums[mid]) {
          if (target < nums[start] || target > nums[mid]) {
            //target is in rotated part
            start = mid + 1;
          } else {
            end = mid - 1;
          }
        } else if (nums[start] > nums[mid]) {
          //right part is rotated

          //target is in rotated part
          if (target < nums[mid] || target > nums[end]) {
            end = mid - 1;
          } else {
            start = mid + 1;
          }
        } else {
          //duplicates, we know nums[mid] != target, so nums[start] != target
          //based on current information, we can only move left pointer to skip one cell
          //thus in the worst case, we would have target: 2, and array like 11111111,
then
          //the running time would be O(n)
          start++;
```

```
        }
      }
      return false;
    }
  }
}

// ============================================================
// FILE 82: _82.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 82. Remove Duplicates from Sorted List II
 *
 *  Given a sorted linked list, delete all nodes that have duplicate numbers, leaving
only distinct numbers from the original list.

 For example,
 Given 1->2->3->3->4->4->5, return 1->2->5.
 Given 1->1->1->2->3, return 2->3.

 */
public class _82 {
  public static class Solution1 {
    public ListNode deleteDuplicates(ListNode head) {
      if (head == null) {
        return head;
      }
      ListNode fakeHead = new ListNode(-1);
      fakeHead.next = head;
      ListNode pre = fakeHead;
      ListNode curr = head;
      while (curr != null) {
        while (curr.next != null && curr.val == curr.next.val) {
          curr = curr.next;
        }
        if (pre.next == curr) {
          pre = pre.next;
        } else {
          pre.next = curr.next;
        }
        curr = curr.next;
      }
      return fakeHead.next;
    }
  }

}

// ============================================================
```

```java
// FILE 83: _83.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 83. Remove Duplicates from Sorted List
 *
 * Given a sorted linked list, delete all duplicates such that each element appear only
once.

 For example,
 Given 1->1->2, return 1->2.
 Given 1->1->2->3->3, return 1->2->3.
 */
public class _83 {
  public static class Solution1 {
    public ListNode deleteDuplicates(ListNode head) {
      ListNode ret = new ListNode(-1);
      ret.next = head;
      while (head != null) {
        while (head.next != null && head.next.val == head.val) {
          head.next = head.next.next;
        }
        head = head.next;
      }
      return ret.next;
    }
  }

  public static class Solution2 {
    public ListNode deleteDuplicates(ListNode head) {
      ListNode curr = head;
      while (curr != null && curr.next != null) {
        if (curr.val == curr.next.val) {
          curr.next = curr.next.next;
        } else {
          curr = curr.next;
        }
      }
      return head;
    }
  }
}

// ============================================================
// FILE 84: _84.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;
```

```
/**
 * 84. Largest Rectangle in Histogram
 *
 * Given n non-negative integers representing the histogram's bar height where
 * the width of each bar is 1, find the area of largest rectangle in the histogram.

 Above is a histogram where width of each bar is 1, given height = [2,1,5,6,2,3].
 The largest rectangle is shown in the shaded area, which has area = 10 unit.

 For example,
 Given heights = [2,1,5,6,2,3],
 return 10.
 */
public class _84 {

  public static class Solution1 {

    /**
     * credit: https://leetcode.com/articles/largest-rectangle-histogram/#approach-5-
using-stack-accepted
     * and https://discuss.leetcode.com/topic/7599/o-n-stack-based-java-solution
     */
    public int largestRectangleArea(int[] heights) {
      int len = heights.length;
      Stack<Integer> s = new Stack<>();
      int maxArea = 0;
      for (int i = 0; i <= len; i++) {
        int h = (i == len ? 0 : heights[i]);
        if (s.isEmpty() || h >= heights[s.peek()]) {
          s.push(i);
        } else {
          int tp = s.pop();
          maxArea = Math.max(maxArea, heights[tp] * (s.isEmpty() ? i : i - 1 -
s.peek()));
          i--;
        }
      }
      return maxArea;
    }
  }

}

// ============================================================
// FILE 85: _85.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 85. Maximal Rectangle
```

```
 *
 * Given a 2D binary matrix filled with 0's and 1's, find the largest rectangle
containing only 1's and return its area.

 For example, given the following matrix:

 1 0 1 0 0
 1 0 1 1 1
 1 1 1 1 1
 1 0 0 1 0
 Return 6.

 */
public class _85 {
  public static class Solution1 {
    public int maximalRectangle(char[][] matrix) {
      if (matrix.length == 0) {
        return 0;
      }
      int m = matrix.length;
      int n = matrix[0].length;
      int[] left = new int[n];
      int[] right = new int[n];
      int[] height = new int[n];
      Arrays.fill(left, 0);
      Arrays.fill(right, n);
      Arrays.fill(height, 0);
      int maxA = 0;
      for (int i = 0; i < m; i++) {
        int currLeft = 0;
        int currRight = n;

        //compute height, this can be achieved from either side
        for (int j = 0; j < n; j++) {
          if (matrix[i][j] == '1') {
            height[j]++;
          } else {
            height[j] = 0;
          }
        }

        //compute left, from left to right
        for (int j = 0; j < n; j++) {
          if (matrix[i][j] == '1') {
            left[j] = Math.max(left[j], currLeft);
          } else {
            left[j] = 0;
            currLeft = j + 1;
          }
        }

        //compute right, from right to left
        for (int j = n - 1; j >= 0; j--) {
          if (matrix[i][j] == '1') {
```

```
            right[j] = Math.min(right[j], currRight);
          } else {
            right[j] = n;
            currRight = j;
          }
        }

        //compute rectangle area, this can be achieved from either side
        for (int j = 0; j < n; j++) {
          maxA = Math.max(maxA, (right[j] - left[j]) * height[j]);
        }
      }
      return maxA;
    }
  }
}

// ======================================================
// FILE 86: _86.java
// ======================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 86. Partition List
 *
 * Given a linked list and a value x, partition it such that all nodes less than x come
before nodes greater than or equal to x.

 You should preserve the original relative order of the nodes in each of the two
partitions.

 For example,
 Given 1->4->3->2->5->2 and x = 3,
 return 1->2->2->4->3->5.
 */
public class _86 {

  public static class Solution1 {
    public ListNode partition(ListNode head, int x) {
      if (head == null || head.next == null) {
        return head;
      }
      ListNode left = new ListNode(0);
      ListNode right = new ListNode(0);
      ListNode less = left;
      ListNode greater = right;
      while (head != null) {
        if (head.val < x) {
          less.next = head;
          less = less.next;
        } else {
```

```
          greater.next = head;
          greater = greater.next;
        }
        head = head.next;
      }
      greater.next = null;
      less.next = right.next;
      return left.next;
    }
  }
}

// ============================================================
// FILE 87: _87.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 87. Scramble String
 *
 * Given a string s1, we may represent it as a binary tree by partitioning it to two non-
empty substrings recursively.

 Below is one possible representation of s1 = "great":

     great
    /    \
   gr    eat
  / \    /  \
 g   r  e   at
            / \
           a   t
 To scramble the string, we may choose any non-leaf node and swap its two children.

 For example, if we choose the node "gr" and swap its two children, it produces a
scrambled string "rgeat".

     rgeat
    /    \
   rg    eat
  / \    /  \
 r   g  e   at
            / \
           a   t
 We say that "rgeat" is a scrambled string of "great".

 Similarly, if we continue to swap the children of nodes "eat" and "at", it produces a
scrambled string "rgtae".

      rgtae
     /    \
   rg    tae
  / \    /  \
```

```
 r   g  ta  e
       / \
      t   a
 We say that "rgtae" is a scrambled string of "great".

 Given two strings s1 and s2 of the same length, determine if s2 is a scrambled string of
s1.
 */
public class _87 {

    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/19158/accepted-java-solution */
        public boolean isScramble(String s1, String s2) {
          if (s1.equals(s2)) {
            return true;
          }
          if (s1.length() != s2.length()) {
            return false;
          }

          int[] letters = new int[26];
          for (int i = 0; i < s1.length(); i++) {
            letters[s1.charAt(i) - 'a']++;
            letters[s2.charAt(i) - 'a']--;
          }

          for (int i : letters) {
            if (i != 0) {
              return false;
            }
          }

          for (int i = 1; i < s1.length(); i++) {
            if (isScramble(s1.substring(0, i), s2.substring(0, i)) && isScramble(
                s1.substring(i), s2.substring(i))) {
              return true;
            }
            if (isScramble(s1.substring(0, i), s2.substring(s2.length() - i)) &&
isScramble(
                s1.substring(i), s2.substring(0, s2.length() - i))) {
              return true;
            }
          }
          return false;
        }
    }
}

// ============================================================
// FILE 88: _88.java
// ============================================================

package com.fishercoder.solutions;
```

```java
/**
 * 88. Merge Sorted Array
 *
 * Given two sorted integer arrays nums1 and nums2, merge nums2 into nums1 as one sorted
array.
 *
 * Note: You may assume that nums1 has enough space (size that is greater or equal to m +
n) to hold
 * additional elements from nums2. The number of elements initialized in nums1 and nums2
are m and n
 * respectively.
 */

public class _88 {

  public static class Solution1 {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
      int i = m - 1;
      int j = n - 1;
      int k = m + n - 1;
      while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j]) {
          nums1[k--] = nums1[i--];
        } else {
          nums1[k--] = nums2[j--];
        }
      }
      while (j >= 0) {
        nums1[k--] = nums2[j--];
      }
    }
  }
}

// ============================================================
// FILE 89: _89.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 89. Gray Code
 *
 * The gray code is a binary numeral system where two successive values differ in only
one bit.
 * Given a non-negative integer n representing the total number of bits in the code,
 * print the sequence of gray code.
 * A gray code sequence must begin with 0.

 For example, given n = 2, return [0,1,3,2]. Its gray code sequence is:
```

```
 00 - 0
 01 - 1
 11 - 3
 10 - 2

 Note:

 For a given n, a gray code sequence is not uniquely defined.

 For example, [0,2,3,1] is also a valid gray code sequence according to the above
definition.

 For now, the judge is able to judge based on one instance of gray code sequence. Sorry
about that.
 */

public class _89 {

  public static class Solution1 {
    public List<Integer> grayCode(int n) {
      List<Integer> result = new ArrayList();
      for (int i = 0; i < (1 << n); i++) {
        result.add(i ^ (i >> 1));
      }
      return result;
    }
  }

  public static class Solution2 {
    public List<Integer> grayCode(int n) {
      List<Integer> result = new ArrayList();
      for (int i = 0; i < Math.pow(2, n); i++) {
        result.add(i ^ (i >> 1));
      }
      return result;
    }
  }

  public static void main(String... args) {
    System.out.println("----------------------------------------------------------------------
------------------------");
    System.out.println("How to understand i << n? It means n to the power of two, see
below. So we have an equivalent solution, which is solution2.");
    System.out.println("1 << 2: " + (1 << 2));
    System.out.println("1 << 3: " + (1 << 3));
    System.out.println("1 << 4: " + (1 << 4));
    System.out.println("1 << 5: " + (1 << 5));
    System.out.println("1 << 6: " + (1 << 6));
    System.out.println("----------------------------------------------------------------------
------------------------");
    System.out.println("How to understand i >> 1? It means to shift the number i to the
right by 1 bit, see below");
    System.out.println("0 >> 1: " + (0 >> 1));
    System.out.println("1 >> 1: " + (1 >> 1));
```

```
    System.out.println("2 >> 1: " + (2 >> 1));
    System.out.println("3 >> 1: " + (3 >> 1));
    System.out.println("4 >> 1: " + (4 >> 1));
    System.out.println("5 >> 1: " + (5 >> 1));
    System.out.println("6 >> 1: " + (6 >> 1));
  }

}

// ============================================================
// FILE 90: _90.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 90. Subsets II
 *
 *Given a collection of integers that might contain duplicates, nums, return all possible
subsets.
 Note: The solution set must not contain duplicate subsets.

 For example,
 If nums = [1,2,2], a solution is:
 [
 [2],
 [1],
 [1,2,2],
 [2,2],
 [1,2],
 []
 ]
 */
public class _90 {

    public static class Solution1 {
        public List<List<Integer>> subsetsWithDup(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            List<Integer> empty = new ArrayList();
            result.add(empty);
            if (nums == null) {
                return result;
            }
            Arrays.sort(nums);
            for (int i = 0; i < nums.length; i++) {
                Set<List<Integer>> temp = new HashSet();
                for (List<Integer> list : result) {
                    List<Integer> newList = new ArrayList(list);
```

```
                    newList.add(nums[i]);
                    temp.add(newList);
                }
                result.addAll(temp);
            }
            Set<List<Integer>> resultSet = new HashSet();
            resultSet.addAll(result);
            result.clear();
            result.addAll(resultSet);
            return result;
        }
    }

    public static class Solution2 {
        public List<List<Integer>> subsetsWithDup(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            Arrays.sort(nums);
            backtrack(nums, 0, new ArrayList(), result);
            return result;
        }

        void backtrack(int[] nums, int start, List<Integer> curr, List<List<Integer>>
result) {
            result.add(new ArrayList(curr));
            for (int i = start; i < nums.length; i++) {
                if (i > start && nums[i] == nums[i - 1]) {
                    continue;
                }
                curr.add(nums[i]);
                backtrack(nums, i + 1, curr, result);
                curr.remove(curr.size() - 1);
            }
        }
    }

    public static class Solution3 {
        public List<List<Integer>> subsetsWithDup(int[] nums) {
            List<List<Integer>> result = new ArrayList<>();
            List<Integer> list = new ArrayList<>();
            result.add(list);
            Arrays.sort(nums);
            backtracking(nums, 0, result, list);
            return result;
        }

        private void backtracking(int[] nums, int start, List<List<Integer>> result,
List<Integer> list) {
            for (int i = start; i < nums.length; i++) {
                if (i > start && nums[i] == nums[i - 1]) {
                    continue;
                }
                list.add(nums[i]);
                result.add(new ArrayList<>(list));
                backtracking(nums, i + 1, result, list);
```

```
                list.remove(list.size() - 1);
            }
        }
    }
}

// ============================================================
// FILE 91: _91.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 91. Decode Ways
 *
 * A message containing letters from A-Z is being encoded to numbers using the following
mapping:

 'A' -> 1
 'B' -> 2
 ...
 'Z' -> 26
 Given an encoded message containing digits, determine the total number of ways to decode
it.

 For example,
 Given encoded message "12", it could be decoded as "AB" (1 2) or "L" (12).

 The number of ways decoding "12" is 2.
 */
public class _91 {
    /**Credit: https://discuss.leetcode.com/topic/35840/java-clean-dp-solution-with-
explanation
     * I used a dp array of size n + 1 to save subproblem solutions.
     * dp[0] means an empty string will have one way to decode,
     * dp[1] means the way to decode a string of size 1.
     *
     * I then check one digit and two digit combination and save the results along the
way.
     * In the end, dp[n] will be the end result.*/

    public static class Solution1 {
      public int numDecodings(String s) {
        if (s == null || s.length() == 0) {
          return 0;
        }
        int[] dp = new int[s.length() + 1];
        dp[0] = 1;
        dp[1] = (s.charAt(0) != '0') ? 1 : 0;
        for (int i = 2; i <= s.length(); i++) {
          int first = Integer.valueOf(s.substring(i - 1, i));
          int second = Integer.valueOf(s.substring(i - 2, i));
          if (first > 0 && first <= 9) {
            dp[i] += dp[i - 1];
```

```
          }
          if (second >= 10 && second <= 26) {
            dp[i] += dp[i - 2];
          }
        }
        return dp[s.length()];
      }
    }
}

// ============================================================
// FILE 92: _92.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 92. Reverse Linked List II
 *
 * Reverse a linked list from position m to n. Do it in-place and in one-pass.
 *
 * For example: Given 1->2->3->4->5->NULL, m = 2 and n = 4,
 *
 * return 1->4->3->2->5->NULL.
 *
 * Note: Given m, n satisfy the following condition: 1 ≤ m ≤ n ≤ length of list.
 */
public class _92 {

  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/8976/simple-java-solution-with-clear-
explanation */
    public ListNode reverseBetween(ListNode head, int m, int n) {
      // use four nodes, pre, start, then, dummy
      // just reverse the nodes along the way
      ListNode dummy = new ListNode(-1);
      dummy.next = head;
      ListNode pre = dummy;
      for (int i = 0; i < m - 1; i++) {
        pre = pre.next;
      }

      ListNode start = pre.next;// start is the node prior to reversing, in the given
example,
      // start is node with value 1
      ListNode then = start.next;// then is the node that we'll start to reverse, in the
given
      // example, it's 2

      for (int i = 0; i < n - m; i++) {
        // pay special attention to this for loop, it's assigning then.next to
start.next, it
```

```
        // didn't initialize a new node
        // this does exactly what I desired to do, but I just didn't figure out how to
implement
        // it, thumbs up to the OP!
        start.next = then.next;
        then.next = pre.next;
        pre.next = then;
        then = start.next;
      }

      return dummy.next;
    }
  }
}

// ============================================================
// FILE 93: _93.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 93. Restore IP Addresses
 *
 * Given a string containing only digits, restore it by returning all possible valid IP
address
 * combinations.
 *
 * For example: Given "25525511135",
 *
 * return ["255.255.11.135", "255.255.111.35"]. (Order does not matter)
 */
public class _93 {

  public static class Solution1 {
    public List<String> restoreIpAddresses(String s) {
      List<String> allValidIpAddresses = new ArrayList<>();
      if (s == null || s.length() > 12 || s.length() < 4) {
        return allValidIpAddresses;
      }
      backtracking(s, new ArrayList<>(), allValidIpAddresses, 0);
      return allValidIpAddresses;
    }

    private void backtracking(String s, ArrayList<String> bytes, List<String> result, int
pos) {
      if (bytes.size() == 4) {
        if (pos != s.length()) {
          return;
        }
        StringBuilder stringBuilder = new StringBuilder();
```

```
        for (int i = 0; i < 4; i++) {
          stringBuilder.append(bytes.get(i));
          stringBuilder.append(".");
        }
        stringBuilder.setLength(stringBuilder.length() - 1);
        result.add(stringBuilder.toString());
        return;
      }

      for (int i = pos; i < pos + 4 && i < s.length(); i++) {
        String oneByte = s.substring(pos, i + 1);
        if (!isValid(oneByte)) {
          continue;
        }
        bytes.add(oneByte);
        backtracking(s, bytes, result, i + 1);
        bytes.remove(bytes.size() - 1);
      }
    }

    private boolean isValid(String oneByte) {
      if (oneByte.charAt(0) == '0') {
        return oneByte.equals("0");
      }
      int num = Integer.valueOf(oneByte);
      return (num >= 0 && num < 256);
    }
  }
}

// ============================================================
// FILE 94: _94.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

/**
 * 94. Binary Tree Inorder Traversal
 *
 * Given a binary tree, return the inorder traversal of its nodes' values.

 For example:
 Given binary tree [1,null,2,3],
 1
  \
   2
  /
 3
```

```
 return [1,3,2].

 Note: Recursive solution is trivial, could you do it iteratively?*/

public class _94 {

    public static class Solution1 {
        public List<Integer> inorderTraversal(TreeNode root) {
            return inorder(root, new ArrayList());
        }

        List<Integer> inorder(TreeNode root, List<Integer> result) {
            if (root == null) {
                return result;
            }
            inorder(root.left, result);
            result.add(root.val);
            return inorder(root.right, result);
        }
    }

    public static class Solution2 {
        //iterative approach
        public List<Integer> inorderTraversal(TreeNode root) {
            List<Integer> result = new ArrayList();
            Stack<TreeNode> stack = new Stack();
            while (root != null || !stack.isEmpty()) {
                while (root != null) {
                    stack.push(root);
                    root = root.left;
                }
                root = stack.pop();
                result.add(root.val);
                root = root.right;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 95: _95.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 95. Unique Binary Search Trees II
 *
```

```
 * Given an integer n, generate all structurally unique BST's (binary search trees) that
store values 1...n.

 For example,
 Given n = 3, your program should return all 5 unique BST's shown below.

 1         3     3      2      1
  \       /     /      / \      \
   3     2     1      1   3      2
  /     /       \                 \
 2     1         2                 3
 */
public class _95 {

        public static class Solution1 {
                public List<TreeNode> generateTrees(int n) {
                        List<TreeNode> result = new ArrayList();
                        if (n == 0) {
                                return result;
                        }
                        return generateTrees(1, n);
                }

                private List<TreeNode> generateTrees(int start, int end) {
                        List<TreeNode> result = new ArrayList();
                        if (start > end) {
                                result.add(null);
                                return result;
                        }
                        if (start == end) {
                                result.add(new TreeNode(start));
                                return result;
                        }

                        for (int i = start; i <= end; i++) {
                                List<TreeNode> leftList = generateTrees(start, i - 1);
                                List<TreeNode> rightList = generateTrees(i + 1, end);
                                for (TreeNode left : leftList) {
                                        for (TreeNode right : rightList) {
                                                TreeNode root = new TreeNode(i);
                                                root.left = left;
                                                root.right = right;
                                                result.add(root);
                                        }
                                }
                        }
                        return result;
                }
        }
}

// ============================================================
// FILE 96: _96.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 96. Unique Binary Search Trees
 *
 * Given n, how many structurally unique BST's (binary search trees) that store values
1...n?

 For example,
 Given n = 3, there are a total of 5 unique BST's.

 1         3     3      2      1
 \       /     /      / \      \
 3      2     1      1   3      2
 /     /       \                 \
 2    1         2                 3

 */
public class _96 {

    public static class Solution1 {
        public int numTrees(int n) {
            int[] G = new int[n + 1];
            G[0] = G[1] = 1;

            for (int i = 2; i <= n; ++i) {
                for (int j = 1; j <= i; ++j) {
                    int temp = G[j - 1] * G[i - j];
                    G[i] = G[i] + temp;
                }
            }
            return G[n];
        }
    }
}

// ============================================================
// FILE 97: _97.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 97. Interleaving String
 *
 * Given s1, s2, s3, find whether s3 is formed by the interleaving of s1 and s2.
 * For example,
 * Given:
 * s1 = "aabcc",
 * s2 = "dbbca",
 * When s3 = "aadbbcbcac", return true.
 * When s3 = "aadbbbaccc", return false.
 */
```

```
public class _97 {
  public static class Solution1 {
    public boolean isInterleave(String s1, String s2, String s3) {
      int m = s1.length();
      int n = s2.length();
      if (m + n != s3.length()) {
        return false;
      }

      boolean[][] dp = new boolean[m + 1][n + 1];

      dp[0][0] = true;

      for (int i = 0; i < m; i++) {
        if (s1.charAt(i) == s3.charAt(i)) {
          dp[i + 1][0] = true;
        } else {
          //if one char fails, that means it breaks, the rest of the chars won't matter
any more.
          //Mian and I found one missing test case on Lintcode: ["b", "aabccc",
"aabbbcb"]
          //if we don't break, here, Lintcode could still accept this code, but Leetcode
fails it.
          break;
        }
      }

      for (int j = 0; j < n; j++) {
        if (s2.charAt(j) == s3.charAt(j)) {
          dp[0][j + 1] = true;
        } else {
          break;
        }
      }

      for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
          int k = i + j - 1;
          dp[i][j] = (s1.charAt(i - 1) == s3.charAt(k) && dp[i - 1][j])
              || (s2.charAt(j - 1) == s3.charAt(k) && dp[i][j - 1]);
        }
      }

      return dp[m][n];
    }
  }
}

// ============================================================
// FILE 98: _98.java
// ============================================================

package com.fishercoder.solutions;
```

```
import com.fishercoder.common.classes.TreeNode;

/**
 * 98. Validate Binary Search Tree
 *
 * Given a binary tree, determine if it is a valid binary search tree (BST).

 Assume a BST is defined as follows:

 The left subtree of a node contains only nodes with keys less than the node's key.
 The right subtree of a node contains only nodes with keys greater than the node's key.
 Both the left and right subtrees must also be binary search trees.

 Example 1:
   2
  / \
 1   3
 Binary tree [2,1,3], return true.

 Example 2:
   1
  / \
 2   3
 Binary tree [1,2,3], return false.
 */
public class _98 {

    public static class Solution1 {

        public boolean isValidBST(TreeNode root) {
            return valid(root, null, null);
        }

        boolean valid(TreeNode root, Integer min, Integer max) {
            if (root == null) {
                return true;
            }
            if ((min != null && root.val <= min) || (max != null && root.val >= max)) {
                return false;
            }
            return valid(root.left, min, root.val) && valid(root.right, root.val, max);
        }
    }


    public static class Solution2 {
        public boolean isValidBST(TreeNode root) {
            if (root == null) {
                return true;
            }
            return dfs(root.left, Long.MIN_VALUE, root.val) && dfs(root.right, root.val,
Long.MAX_VALUE);
        }
```

```
        private boolean dfs(TreeNode root, long minValue, long maxValue) {
            if (root == null) {
                return true;
            }
            if (root != null && (root.val <= minValue || root.val >= maxValue)) {
                return false;
            }
            boolean leftResult = true;
            boolean rightResult = true;
            if (root.left != null) {
                leftResult = dfs(root.left, minValue, root.val);
            }
            if (root.right != null) {
                rightResult = dfs(root.right, root.val, maxValue);
            }
            return leftResult && rightResult;
        }
    }

}

// ==========================================================
// FILE 99: _99.java
// ==========================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
/**
 * 99. Recover Binary Search Tree
 *
 * Two elements of a binary search tree (BST) are swapped by mistake.
 * Recover the tree without changing its structure.

 Example 1:

 Input: [1,3,null,null,2]

    1
   /
  3
  \
  2

 Output: [3,1,null,null,2]

   3
  /
 1
  \
  2
 Example 2:

 Input: [3,1,4,null,null,2]
```

```
    3
   / \
  1   4
 /
2

 Output: [2,1,4,null,null,3]

    2
   / \
  1   4
 /
3

 Follow up:
 A solution using O(n) space is pretty straight forward.
 Could you devise a constant space solution?
 */

public class _99 {
  public static class Solution1 {
    TreeNode firstElement = null;
    TreeNode secondElement = null;

    TreeNode prevElement = new TreeNode(Integer.MIN_VALUE);

    public void recoverTree(TreeNode root) {
      traverseTree(root);

      //swap the two elements
      int temp = firstElement.val;
      firstElement.val = secondElement.val;
      secondElement.val = temp;
    }

    private void traverseTree(TreeNode root) {
      if (root == null) {
        return;
      }

      traverseTree(root.left);

      //prevElement means the one previous to the current root, refer to in-order
traversal, previous element must be smaller than the current root
      //if it's bigger, then we find the first element, thus we store it in the variable
called firstElement
      if (firstElement == null && prevElement.val >= root.val) {
        firstElement = prevElement;
      }

      if (firstElement != null && prevElement.val >= root.val) {
        secondElement = root;
      }
```

```
      //this is the last step in the "do some business logic", so we'll always to have
update the previous node to be the current root before it traverses the right subtree
      //since the current root will be the new previous node for the right subtree.
      prevElement = root;

      traverseTree(root.right);
    }

  }
}

// ============================================================
// FILE 100: _100.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 100. Same Tree
 *
 * Given two binary trees, write a function to check if they are equal or not. Two binary
trees are
 * considered equal if they are structurally identical and the nodes have the same value.
 *
 * Example 1:
 *
 * Input:     1         1
 *           / \       / \
 *          2   3     2   3
 *
 *         [1,2,3],   [1,2,3]
 *
 * Output: true
 *
 * Example 2:
 *
 * Input:     1         1
 *           /           \
 *          2             2
 *
 *         [1,2],     [1,null,2]
 *
 * Output: false
 *
 * Example 3:
 *
 * Input:     1         1
 *           / \       / \
 *          2   1     1   2
 *
 *         [1,2,1],   [1,1,2]
```

```
 *
 * Output: false
 */

public class _100 {

  public static class Solution1 {
    public boolean isSameTree(TreeNode p, TreeNode q) {
      if (p == null || q == null) {
        return p == q;
      }
      return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right,
q.right);
    }
  }
}

// ============================================================
// FILE 101: _101.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**101. Symmetric Tree

Given a binary tree, check whether it is a mirror of itself (ie, symmetric around its
center).

For example, this binary tree [1,2,2,3,4,4,3] is symmetric:

    1
   / \
  2   2
 / \ / \
3  4 4  3

But the following [1,2,2,null,3,null,3] is not:

    1
   / \
  2   2
   \   \
   3    3

Note:
Bonus points if you could solve it both recursively and iteratively.
 */

public class _101 {
        public static class Solution1 {
                public boolean isSymmetric(TreeNode root) {
                        if (root == null) {
```

```
                                return true;
                        }
                        return isSymmetric(root.left, root.right);
                }

                private boolean isSymmetric(TreeNode left, TreeNode right) {
                        if (left == null || right == null) {
                                return left == right;
                        }
                        if (left.val != right.val) {
                                return false;
                        }
                        return isSymmetric(left.left, right.right) &&
isSymmetric(left.right, right.left);
                }
        }
}

// ============================================================
// FILE 102: _102.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 102. Binary Tree Level Order Traversal

Given a binary tree, return the level order traversal of its nodes' values. (ie, from
left to right, level by level).

For example:
Given binary tree [3,9,20,null,null,15,7],

    3
   / \
  9  20
    /  \
   15   7

return its level order traversal as:

[
  [3],
  [9,20],
  [15,7]
]
*/
```

```
public class _102 {

  public static class Solution1 {
    public List<List<Integer>> levelOrder(TreeNode root) {
      List<List<Integer>> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      Queue<TreeNode> queue = new LinkedList();
      queue.offer(root);
      while (!queue.isEmpty()) {
        List<Integer> thisLevel = new ArrayList();
        int size = queue.size();
        for (int i = 0; i < size; i++) {
          TreeNode curr = queue.poll();
          thisLevel.add(curr.val);
          if (curr.left != null) {
            queue.offer(curr.left);
          }
          if (curr.right != null) {
            queue.offer(curr.right);
          }
        }
        result.add(thisLevel);
      }
      return result;
    }
  }
}

// ============================================================
// FILE 103: _103.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 103. Binary Tree Zigzag Level Order Traversal
 *
Given a binary tree, return the zigzag level order traversal of its nodes' values.
 (ie, from left to right, then right to left for the next level and alternate between).

For example:
Given binary tree [3,9,20,null,null,15,7],
    3
   / \
```

```
  9  20
    /  \
   15   7
return its zigzag level order traversal as:
[
  [3],
  [20,9],
  [15,7]
]
*/
public class _103 {
    public static class Solution1 {
        public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
            Queue<TreeNode> q = new LinkedList();
            List<List<Integer>> levels = new ArrayList();
            if (root == null) {
                return levels;
            }
            q.offer(root);
            boolean forward = true;
            while (!q.isEmpty()) {
                int size = q.size();
                List<Integer> level = new ArrayList();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = q.poll();
                    level.add(curr.val);
                    if (curr.left != null) {
                        q.offer(curr.left);
                    }
                    if (curr.right != null) {
                        q.offer(curr.right);
                    }
                }
                if (forward) {
                    forward = false;
                    levels.add(level);
                } else {
                    Collections.reverse(level);
                    levels.add(level);
                    forward = true;
                }
            }
            return levels;
        }
    }
}

// ============================================================
// FILE 104: _104.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
```

```java
/**
 * 104. Maximum Depth of Binary Tree
 *
 * Given a binary tree, find its maximum depth.
 * The maximum depth is the number of nodes along the longest path from the root node
down to the farthest leaf node.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 * Given binary tree [3,9,20,null,null,15,7],
 *
 *     3
 *    / \
 *   9  20
 *     /  \
 *    15  7
 *
 * return its depth = 3.
 */
public class _104 {

  public static class Solution1 {
    public int maxDepth(TreeNode root) {
      if (root == null) {
        return 0;
      }
      return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
    }
  }

}

// ============================================================
// FILE 105: _105.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 105. Construct Binary Tree from Preorder and Inorder Traversal
 * Given preorder and inorder traversal of a tree, construct the binary tree.

 Note:
 You may assume that duplicates do not exist in the tree.
 */
public class _105 {
```

```
        public static class Solution1 {
                /**
                 * credit: https://discuss.leetcode.com/topic/29838/5ms-java-clean-
solution-with-caching use
                 * HashMap as the cache so that accessing inorder index becomes O(1) time
Note: The first
                 * element of preorder array is the root!
                 */
                public TreeNode buildTree(int[] preorder, int[] inorder) {
                        Map<Integer, Integer> inorderMap = new HashMap();
                        for (int i = 0; i < inorder.length; i++) {
                                inorderMap.put(inorder[i], i);
                        }

                        /**At the beginning, both start from 0 to nums.length-1*/
                        return buildTree(preorder, 0, preorder.length - 1, inorderMap, 0,
inorder.length - 1);
                }

                private TreeNode buildTree(int[] preorder, int preStart, int preEnd,
                                Map<Integer, Integer> inorderMap, int inStart, int inEnd)
{
                        if (preStart > preEnd || inStart > inEnd) {
                                return null;
                        }

                        TreeNode root = new TreeNode(preorder[preStart]);
                        int inRoot = inorderMap.get(preorder[preStart]);
                        int numsLeft = inRoot - inStart;

                        /**It's easy to understand and remember:
                         * for the indices of inorder array:
                         * root.left should be inStart and inRoot-1 as new start and end
indices
                         * root.right should be inRoot+1 and inEnd as new start and end
indices
                         *
                         * since inRoot is being used already in this recursion call,
that's why we use inRoot-1 and inRoot+1
                         * this part is the same for both Leetcode 105 and Leetcode
106.*/
                        root.left = buildTree(preorder, preStart + 1, preStart +
numsLeft, inorderMap, inStart, inRoot - 1);
                        root.right = buildTree(preorder, preStart + numsLeft + 1, preEnd,
inorderMap, inRoot + 1, inEnd);
                        return root;
                }
        }
}

// ============================================================
// FILE 106: _106.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 106. Construct Binary Tree from Inorder and Postorder Traversal
 *
 * Given inorder and postorder traversal of a tree, construct the binary tree.

 Note:
 You may assume that duplicates do not exist in the tree.
 */
public class _106 {
    public static class Solution1 {

        /**
         * https://discuss.leetcode.com/topic/3296/my-recursive-java-code-with-o-n-time-
and-o-n-space
         *
         * Note: the last element of postorder array is the root!
         *
         * The idea is to take the last element in postorder as the root; find the
position of the root
         * in the inorder array; then locate the range for left sub-tree and right sub-
tree and do
         * recursion, use a hashmap to record the index of root in the inorder array.
         */
        public TreeNode buildTree(int[] inorder, int[] postorder) {
            if (inorder == null || postorder == null || inorder.length !=
postorder.length) {
                return null;
            }
            HashMap<Integer, Integer> inorderMap = new HashMap<>();
            for (int i = 0; i < inorder.length; i++) {
                inorderMap.put(inorder[i], i);
            }
            /**At the beginning, both start from 0 to nums.length-1*/
            return buildTreeRecursively(inorderMap, 0, inorder.length - 1, postorder, 0,
                postorder.length - 1);
        }

        private TreeNode buildTreeRecursively(Map<Integer, Integer> inorderMap, int
inorderStart,
            int inorderEnd, int[] postorder, int postorderStart, int postorderEnd) {
            if (postorderStart > postorderEnd || inorderStart > inorderEnd) {
                return null;
            }
            TreeNode root = new TreeNode(postorder[postorderEnd]);
            int inRoot = inorderMap.get(postorder[postorderEnd]);
            int numsLeft = inRoot - inorderStart;
```

```
            /**It's easy to understand and remember:
             * for the indices of inorder array:
             * inStart and inRoot-1 as new start and end indices
             * inRoot+1 and inEnd as new start and end indices
             *
             * this is easy to understand and remember: since inRoot is already been used
in this recursion call, so we're going to use inRoot-1 and inRoot+1 for next recursion
call
             *
             * for the indices of postorder array:
             * postorderStart and postorderStart+numsLeft-1 should be the new start and
end indices
             * postorderStart+numsLeft and postorderEnd-1 should be the new start and end
indices
             *
             * this is also easy to understand and remember:
             * since the last one in postorder is the root and we have used it in this
recursion call already, so the end is definitely postorderEnd-1;
             * then the postorderEnd for root.left is contiguous to the postorderStart of
root.right, :)*/
            root.left = buildTreeRecursively(inorderMap, inorderStart, inRoot - 1,
postorder, postorderStart, postorderStart + numsLeft - 1);
            root.right = buildTreeRecursively(inorderMap, inRoot + 1, inorderEnd,
postorder, postorderStart + numsLeft, postorderEnd - 1);
            return root;
        }
    }
}

// ============================================================
// FILE 107: _107.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**107. Binary Tree Level Order Traversal II

Given a binary tree, return the bottom-up level order traversal of its nodes' values.
(ie, from left to right, level by level from leaf to root).

For example:
Given binary tree [3,9,20,null,null,15,7],

    3
   / \
  9  20
```

```
    /  \
   15   7

return its bottom-up level order traversal as:

[
  [15,7],
  [9,20],
  [3]
]
*/

public class _107 {
        public static class Solution1 {
                public List<List<Integer>> levelOrder(TreeNode root) {
                        List<List<Integer>> result = new ArrayList();
                        if (root == null) {
                                return result;
                        }

                        Queue<TreeNode> q = new LinkedList();
                        q.offer(root);
                        while (!q.isEmpty()) {
                                List<Integer> thisLevel = new ArrayList<Integer>();
                                int qSize = q.size();
                                for (int i = 0; i < qSize; i++) {
                                        TreeNode curr = q.poll();
                                        thisLevel.add(curr.val);
                                        if (curr.left != null) {
                                                q.offer(curr.left);
                                        }
                                        if (curr.right != null) {
                                                q.offer(curr.right);
                                        }
                                }
                                result.add(thisLevel);
                        }
                        Collections.reverse(result);
                        return result;
                }
        }
}

// ============================================================
// FILE 108: _108.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 108. Convert Sorted Array to Binary Search Tree
 *
```

```
 * Given an array where elements are sorted in ascending order, convert it to a height
balanced BST.
 */
public class _108 {

  public static class Solution1 {
    public TreeNode sortedArrayToBST(int[] num) {
      return rec(num, 0, num.length - 1);
    }

    public TreeNode rec(int[] num, int low, int high) {
      if (low > high) {
        return null;
      }
      int mid = low + (high - low) / 2;
      TreeNode root = new TreeNode(num[mid]);
      root.left = rec(num, low, mid - 1);
      root.right = rec(num, mid + 1, high);
      return root;
    }
  }
}

// ============================================================
// FILE 109: _109.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;
import com.fishercoder.common.classes.TreeNode;

/**
 * 109. Convert Sorted List to Binary Search Tree
 *
 * Given a singly linked list where elements are sorted in ascending order, convert it to
a height balanced BST.
 */
public class _109 {

    public static class Solution1 {

        public TreeNode sortedListToBST(ListNode head) {
            return toBstRecursively(head, null);
        }

        public TreeNode toBstRecursively(ListNode start, ListNode end) {
            if (start == end) {
                return null;
            } else {
                ListNode mid = start;
                ListNode fast = start;
                while (fast != end && fast.next != end) {
                    mid = mid.next;
```

```
                    fast = fast.next.next;
                }

                TreeNode root = new TreeNode(mid.val);
                root.left = toBstRecursively(start, mid);
                root.right = toBstRecursively(mid.next, end);
                return root;
            }
        }
    }

}

// ============================================================
// FILE 110: _110.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 110. Balanced Binary Tree
 *
 * Given a binary tree, determine if it is height-balanced.
 * For this problem, a height-balanced binary tree is defined as a binary tree in which
 * the depth of the two subtrees of every node never differ by more than 1.

 Example 1:
 Given the following tree [3,9,20,null,null,15,7]:

    3
   / \
  9  20
  /  \
 15  7

 Return true.

 Example 2:
 Given the following tree [1,2,2,3,3,null,null,4,4]:

     1
    / \
   2   2
  / \
 3   3
/ \
4 4

 Return false.
 */

public class _110 {
```

```
    public static class Solution1 {
        //recursively get the height of each subtree of each node, compare their
difference, if greater than 1, then return false
        //although this is working, but it's not efficient, since it repeatedly computes
the heights of each node every time
        //Its time complexity is O(n^2).
        public boolean isBalanced(TreeNode root) {
            if (root == null) {
                return true;
            }
            if (Math.abs(getH(root.left) - getH(root.right)) > 1) {
                return false;
            } else {
                return isBalanced(root.left) && isBalanced(root.right);
            }
        }

        private int getH(TreeNode root) {
            if (root == null) {
                return 0;//base case
            }
            int leftH = getH(root.left);
            int rightH = getH(root.right);
            return Math.max(leftH, rightH) + 1;
        }
    }

    public static class Solution2 {

        public boolean isBalanced(TreeNode root) {
            return getH(root) != -1;
        }

        private int getH(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int leftH = getH(root.left);
            if (leftH == -1) {
                return -1;
            }
            int rightH = getH(root.right);
            if (rightH == -1) {
                return -1;
            }
            if (Math.abs(leftH - rightH) > 1) {
                return -1;
            }
            return Math.max(leftH, rightH) + 1;
        }
    }

}
```

```
// ============================================================
// FILE 111: _111.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 111. Minimum Depth of Binary Tree
 *
 * Given a binary tree, find its minimum depth.
 * The minimum depth is the number of nodes along the shortest path from the root node
down to the nearest leaf node.
 * */

public class _111 {
    public static class Solution1 {
      /**DFS*/
      public int minDepth(TreeNode root) {
        if (root == null) {
          return 0;
        }
        int left = minDepth(root.left);
        int right = minDepth(root.right);
        if (left == 0) {
          return right + 1;
        }
        if (right == 0) {
          return left + 1;
        }
        return Math.min(left, right) + 1;
      }
    }

  public static class Solution2 {
    /**BFS*/
    public int minDepth(TreeNode root) {
      if (root == null) {
        return 0;
      }
      Queue<TreeNode> q = new LinkedList();
      q.offer(root);
      int level = 0;
      while (!q.isEmpty()) {
        level++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
          TreeNode curr = q.poll();
          if (curr.left != null) {
```

```
            q.offer(curr.left);
          }
          if (curr.right != null) {
            q.offer(curr.right);
          }
          if (curr.left == null && curr.right == null) {
            return level;
          }
        }
      }
      return level;
    }
  }
}

// ============================================================
// FILE 112: _112.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 112. Path Sum

Given a binary tree and a sum, determine if the tree has a root-to-leaf path such that
adding up all the values along the path equals the given sum.

For example:
Given the below binary tree and sum = 22,

              5
             / \
            4   8
           /   / \
          11  13  4
         /  \      \
        7    2      1

 return true, as there exist a root-to-leaf path 5->4->11->2 which sum is 22.*/
public class _112 {
  public static class Solution1 {
    public boolean hasPathSum(TreeNode root, int sum) {
      if (root == null) {
        return false;
      }
      if (root.val == sum && root.left == null && root.right == null) {
        return true;
      }
      return hasPathSum(root.left, sum - root.val) || hasPathSum(root.right, sum -
root.val);
    }
  }
```

```
}

// ============================================================
// FILE 113: _113.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 113. Path Sum II

 Given a binary tree and a sum, find all root-to-leaf paths where each path's sum equals
the given sum.

For example:
Given the below binary tree and sum = 22,
              5
             / \
            4   8
           /   / \
          11  13  4
         /  \    / \
        7    2  5   1

 return
[
   [5,4,11,2],
   [5,8,4,5]
]
*/
public class _113 {

  public static class Solution1 {
    public List<List<Integer>> pathSum(TreeNode root, int sum) {
      List<List<Integer>> allPaths = new ArrayList();
      if (root == null) {
        return allPaths;
      }
      dfs(root, new ArrayList(), allPaths, sum);
      return allPaths;
    }

    private void dfs(TreeNode root, List<Integer> path, List<List<Integer>> allPaths, int
sum) {
      path.add(root.val);
      if (root.left != null) {
        dfs(root.left, path, allPaths, sum - root.val);
      }
      if (root.right != null) {
```

```
        dfs(root.right, path, allPaths, sum - root.val);
      }
      if (root.left == null && root.right == null) {
        /**Check if sum equals root.val, not sum equals zero!*/
        if (sum == root.val) {
          allPaths.add(new ArrayList(path));
        }
      }
      path.remove(path.size() - 1);
    }
  }
}

// ============================================================
// FILE 114: _114.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 114. Flatten Binary Tree to Linked List
 *
 * Given a binary tree, flatten it to a linked list in-place.

 For example,
 Given

   1
  / \
 2  5
/ \  \
3 4  6

 The flattened tree should look like:
  1
   \
   2
    \
    3
     \
     4
      \
      5
       \
       6

 Hints:
 If you notice carefully in the flattened tree, each node's right child points to the
next node of a pre-order traversal.
 */
public class _114 {
```

```
    public static class Solution1 {
        public void flatten(TreeNode root) {
            while (root != null) {
                if (root.left != null) {
                    TreeNode previousNode = root.left;
                    while (previousNode.right != null) {
                        previousNode = previousNode.right;
                    }
                    previousNode.right = root.right;
                    root.right = root.left;
                    root.left = null;
                }
                root = root.right;
            }
        }
    }
}

// ============================================================
// FILE 115: _115.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 115. Distinct Subsequences
 *
 * Given a string S and a string T, count the number of distinct subsequences of S which
equals T. A
 * subsequence of a string is a new string which is formed from the original string by
deleting some
 * (can be none) of the characters without disturbing the relative positions of the
remaining
 * characters. (ie, "ACE" is a subsequence of "ABCDE" while "AEC" is not).
 *
 * Here is an example: S = "rabbbit", T = "rabbit" Return 3.
 */

public class _115 {
  public static class Solution1 {
    public int numDistinct(String s, String t) {
      int m = s.length();
      int n = t.length();
      int[][] dp = new int[m + 1][n + 1];

      char[] schar = s.toCharArray();
      char[] tchar = t.toCharArray();

      for (int i = 0; i <= m; i++) {
        dp[i][0] = 1;
      }

      for (int j = 1; j <= n; j++) {
        dp[0][j] = 0;
```

```
      }

      for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
          if (schar[i - 1] == tchar[j - 1]) {
            dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1];
          } else {
            dp[i][j] = dp[i - 1][j];
          }
        }
      }
      return dp[m][n];
    }
  }
}

// ============================================================
// FILE 116: _116.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeLinkNode;

/**
 * 116. Populating Next Right Pointers in Each Node

 Given a binary tree

 struct TreeLinkNode {
     TreeLinkNode *left;
     TreeLinkNode *right;
     TreeLinkNode *next;
 }
 Populate each next pointer to point to its next right node. If there is no next right
node, the next pointer should be set to NULL.

 Initially, all next pointers are set to NULL.

 Note:

 You may only use constant extra space.
 You may assume that it is a perfect binary tree (ie, all leaves are at the same level,
and every parent has two children).
 For example,
 Given the following perfect binary tree,
      1
    /  \
   2    3
  / \  / \
 4  5  6  7

 After calling your function, the tree should look like:
      1 -> NULL
```

```
    /  \
   2 -> 3 -> NULL
  / \  / \
 4->5->6->7 -> NULL
 */

public class _116 {
  public static class Solution1 {
    /**
     * credit: https://discuss.leetcode.com/topic/1106/o-1-space-o-n-complexity-
iterative-solution
     * based on level order traversal
     */
    public void connect(TreeLinkNode root) {

      TreeLinkNode head = null; //head of the next level
      TreeLinkNode prev = null; //the leading node on the next level
      TreeLinkNode curr = root;  //current node of current level

      while (curr != null) {
        while (curr != null) { //iterate on the current level
          //left child
          if (curr.left != null) {
            if (prev != null) {
              prev.next = curr.left;
            } else {
              head = curr.left;
            }
            prev = curr.left;
          }
          //right child
          if (curr.right != null) {
            if (prev != null) {
              prev.next = curr.right;
            } else {
              head = curr.right;
            }
            prev = curr.right;
          }
          //move to next node
          curr = curr.next;
        }
        //move to next level
        curr = head;
        head = null;
        prev = null;
      }
    }
  }
}

// ============================================================
// FILE 117: _117.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeLinkNode;

/**
 * 117. Populating Next Right Pointers in Each Node II
 *
 * Follow up for problem "Populating Next Right Pointers in Each Node".
 * What if the given tree could be any binary tree? Would your previous solution still
work?

 Note:

 You may only use constant extra space.
 For example,
 Given the following binary tree,
      1
    /  \
   2    3
  / \    \
 4   5    7
 After calling your function, the tree should look like:
      1 -> NULL
    /  \
   2 -> 3 -> NULL
  / \    \
 4-> 5 -> 7 -> NULL */

public class _117 {
    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/1106/o-1-space-o-n-complexity-
iterative-solution
        O(1) space, based on level order traversal*/
        public void connect(TreeLinkNode root) {

            TreeLinkNode head = null; //head of the next level
            TreeLinkNode prev = null; //the leading node on the next level
            TreeLinkNode cur = root;  //current node of current level

            while (cur != null) {

                while (cur != null) { //iterate on the current level
                    //left child
                    if (cur.left != null) {
                        if (prev != null) {
                            prev.next = cur.left;
                        } else {
                            head = cur.left;
                        }
                        prev = cur.left;
                    }
                    //right child
                    if (cur.right != null) {
```

```
                        if (prev != null) {
                            prev.next = cur.right;
                        } else {
                            head = cur.right;
                        }
                        prev = cur.right;
                    }
                    //move to next node
                    cur = cur.next;
                }

                //move to next level
                cur = head;
                head = null;
                prev = null;
            }
        }
    }
}

// ============================================================
// FILE 118: _118.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 118. Pascal's Triangle
 * Given numRows, generate the first numRows of Pascal's triangle.

 For example, given numRows = 5,
 Return

 [
     [1],
    [1,1],
   [1,2,1],
  [1,3,3,1],
 [1,4,6,4,1]
 ]

 */
public class _118 {

  public static class Solution1 {
    /**fill out values from left to right*/
    public List<List<Integer>> generate(int numRows) {
      List<List<Integer>> result = new ArrayList();
      List<Integer> row = new ArrayList();
      for (int i = 0; i < numRows; i++) {
        row.add(0, 1);
```

```
        for (int j = 1; j < row.size() - 1; j++) {
          row.set(j, row.get(j) + row.get(j + 1));
        }
        result.add(new ArrayList(row));
      }
      return result;
    }
  }

  public static class Solution2 {
    /**fill out values from right to left
     * credit: https://leetcode.com/problems/pascals-triangle/discuss/38141/My-concise-
solution-in-Java/36127*/
    public List<List<Integer>> generate(int numRows) {
      List<List<Integer>> result = new ArrayList();
      List<Integer> row = new ArrayList();
      for (int i = 0; i < numRows; i++) {
        for (int j = row.size() - 1; j >= 1; j--) {
          row.set(j, row.get(j) + row.get(j - 1));
        }
        row.add(1);
        result.add(new ArrayList<>(row));
      }
      return result;
    }
  }
}

// ============================================================
// FILE 119: _119.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 119. Pascal's Triangle II
 * Given an index k, return the kth row of the Pascal's triangle.

 For example, given k = 3,
 Return [1,3,3,1].

 Note:
 Could you optimize your algorithm to use only O(k) extra space?
 */

public class _119 {

  public static class Solution1 {
    public List<Integer> getRow(int rowIndex) {
      if (rowIndex < 0) {
        return new ArrayList();
```

```
      }
      List<List<Integer>> result = new ArrayList();
      List<Integer> row = new ArrayList();
      row.add(1);
      result.add(row);
      for (int i = 1; i <= rowIndex; i++) {
        List<Integer> newRow = new ArrayList();
        newRow.add(1);
        List<Integer> lastRow = result.get(i - 1);
        for (int j = 1; j < lastRow.size(); j++) {
          newRow.add(lastRow.get(j - 1) + lastRow.get(j));
        }
        newRow.add(1);
        result.add(newRow);
      }
      return result.get(result.size() - 1);
    }
  }

  public static class Solution2 {
    /** O(k) space */
    public List<Integer> getRow(int rowIndex) {
      List<Integer> row = new ArrayList<>();
      for (int i = 0; i <= rowIndex; i++) {
        row.add(0, 1);
        for (int j = 1; j < row.size() - 1; j++) {
          row.set(j, row.get(j) + row.get(j + 1));
        }
      }
      return row;
    }
  }
}

// ============================================================
// FILE 120: _120.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;

/**
 * 120. Triangle

  Given a triangle, find the minimum path sum from top to bottom. Each step you may move
to adjacent numbers on the row below.

 For example, given the following triangle
 [
 [2],
 [3,4],
 [6,5,7],
 [4,1,8,3]
```

```
 ]
 The minimum path sum from top to bottom is 11 (i.e., 2 + 3 + 5 + 1 = 11).

 Note:
 Bonus point if you are able to do this using only O(n) extra space, where n is the total
number of rows in the triangle.*/

public class _120 {
  public static class Solution1 {
    public int minimumTotal(List<List<Integer>> triangle) {
      int n = triangle.size();
      List<Integer> cache = triangle.get(n - 1);

      for (int layer = n - 2; layer >= 0; layer--) {
        //for each layer
        for (int i = 0; i <= layer; i++) {
          //check its very node
          int value = Math.min(cache.get(i), cache.get(i + 1)) +
triangle.get(layer).get(i);
          cache.set(i, value);
        }
      }
      return cache.get(0);
    }
  }
}

// ============================================================
// FILE 121: _121.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 121. Best Time to Buy and Sell Stock
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 *
 * If you were only permitted to complete at most one transaction (ie, buy one and sell
one share of the stock),
 * design an algorithm to find the maximum profit.
 *
 * Example 1:
 * Input: [7, 1, 5, 3, 6, 4]
 * Output: 5
 *
 * max. difference = 6-1 = 5 (not 7-1 = 6, as selling price needs to be larger than
buying price)
 *
 * Example 2:
 * Input: [7, 6, 4, 3, 1]
 * Output: 0
 *
```

```
 * In this case, no transaction is done, i.e. max profit = 0.
 */

public class _121 {

    public static class Solution1 {
        /**
         * The key here is that you'll have to buy first, before you can sell. That
means, if the lower
         * price comes after a higher price, their combination won't work! Since you
cannot sell first
         * before you buy it.
         */
        public int maxProfit(int[] prices) {
            if (prices == null || prices.length == 0) {
                return 0;
            }
            int minBuy = prices[0];
            int maxProfit = 0;
            for (int i = 1; i < prices.length; i++) {
                if (prices[i] < minBuy) {
                    minBuy = prices[i];
                } else {
                    maxProfit = Math.max(maxProfit, prices[i] - minBuy);
                }
            }
            return maxProfit;
        }
    }
}

// ============================================================
// FILE 122: _122.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 122. Best Time to Buy and Sell Stock II
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 * Design an algorithm to find the maximum profit. You may complete as many transactions
as you like
 * (ie, buy one and sell one share of the stock multiple times).
 * However, you may not engage in multiple transactions at the same time (ie, you must
sell the stock before you buy again).
 * */

public class _122 {
    public static class Solution1 {
        //peak and valley approach
        public int maxProfit(int[] prices) {
            int pro = 0;
```

```
            int i = 0;
            while (i < prices.length - 1) {
                while (i < prices.length - 1 && prices[i] >= prices[i + 1]) {
                    i++;
                }
                int valley = prices[i];
                while (i < prices.length - 1 && prices[i] <= prices[i + 1]) {
                    i++;
                }
                int peak = prices[i];
                pro += peak - valley;
            }
            return pro;
        }
    }

    public static class Solution2 {
        //simple one pass approach: the above solution could be simplied as below
        public int maxProfit(int[] prices) {
            int pro = 0;
            for (int i = 0; i < prices.length - 1; i++) {
                if (prices[i + 1] > prices[i]) {
                    pro += prices[i + 1] - prices[i];
                }
            }
            return pro;
        }
    }
}

// ============================================================
// FILE 123: _123.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 123. Best Time to Buy and Sell Stock III
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 * Design an algorithm to find the maximum profit. You may complete at most two
transactions.

Note:
You may not engage in multiple transactions at the same time (ie, you must sell the stock
before you buy again).
 */

public class _123 {

    public static class Solution1 {
        //this is a very clear solution and very highly upvoted in Discuss, but not
extensible to K solution.
```

```
        public int maxProfit(int[] prices) {
            int buy1 = Integer.MIN_VALUE;
            int buy2 = Integer.MIN_VALUE;
            int sell1 = 0;
            int sell2 = 0;
            for (int i = 0; i < prices.length; i++) {
                buy1 = Math.max(buy1, -prices[i]);
                sell1 = Math.max(sell1, buy1 + prices[i]);
                buy2 = Math.max(buy2, sell1 - prices[i]);
                sell2 = Math.max(sell2, buy2 + prices[i]);
            }
            return sell2;
        }
    }
}

// ============================================================
// FILE 124: _124.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 124. Binary Tree Maximum Path Sum

 Given a binary tree, find the maximum path sum.
 For this problem, a path is defined as any sequence of nodes from some starting node to
any node
 in the tree along the parent-child connections.

 The path must contain at least one node and does not need to go through the root.

 For example:
 Given the below binary tree,

   1
  / \
 2   3

 Return 6.
 */
public class _124 {

  public static class Solution1 {
    int max = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
      dfs(root);
      return max;
```

```
    }

    private int dfs(TreeNode root) {
      if (root == null) {
        return 0;
      }

      int left = Math.max(dfs(root.left), 0);
      int right = Math.max(dfs(root.right), 0);

      max = Math.max(max, root.val + left + right);

      return root.val + Math.max(left, right);
    }
  }

  public static class Solution2 {
    /**
     * This one uses a map to cache, but surprisingly, it's 10% slower than all
submissions compared
     * with solution1
     */
    int max = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
      Map<TreeNode, Integer> map = new HashMap<>();
      dfs(root, map);
      return max;
    }

    private int dfs(TreeNode root, Map<TreeNode, Integer> map) {
      if (root == null) {
        return 0;
      }
      if (map.containsKey(root)) {
        return map.get(root);
      }
      int left = Math.max(0, dfs(root.left, map));
      int right = Math.max(0, dfs(root.right, map));
      max = Math.max(max, root.val + left + right);
      int pathSum = root.val + Math.max(left, right);
      map.put(root, pathSum);
      return pathSum;
    }
  }
}

// ============================================================
// FILE 125: _125.java
// ============================================================

package com.fishercoder.solutions;

/**
```

```
 * 125. Valid Palindrome

 Given a string, determine if it is a palindrome, considering only alphanumeric
characters and ignoring cases.

 For example,
 "A man, a plan, a canal: Panama" is a palindrome.
 "race a car" is not a palindrome.

 Note:
 Have you consider that the string might be empty? This is a good question to ask during
an interview.
 For the purpose of this problem, we define empty string as valid palindrome.
 */

public class _125 {

  public static class Solution1 {
    public boolean isPalindrome(String s) {
      int i = 0;
      int j = s.length() - 1;
      char[] chars = s.toCharArray();
      while (i < j) {
        while (i < j && !Character.isLetterOrDigit(chars[i])) {
          i++;
        }
        while (i < j && !Character.isLetterOrDigit(chars[j])) {
          j--;
        }
        if (Character.toLowerCase(chars[i]) != Character.toLowerCase(chars[j])) {
          return false;
        }
        i++;
        j--;
      }
      return true;
    }
  }
}

// ============================================================
// FILE 126: _126.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
```

```
/**
 * 126. Word Ladder II

 Given two words (beginWord and endWord), and a dictionary's word list,
 find all shortest transformation sequence(s) from beginWord to endWord, such that:

 Only one letter can be changed at a time
 Each transformed word must exist in the word list. Note that beginWord is not a
transformed word.

 For example,
 Given:
 beginWord = "hit"
 endWord = "cog"
 wordList = ["hot","dot","dog","lot","log","cog"]

 Return
 [
 ["hit","hot","dot","dog","cog"],
 ["hit","hot","lot","log","cog"]
 ]

 Note:
 Return an empty list if there is no such transformation sequence.
 All words have the same length.
 All words contain only lowercase alphabetic characters.
 You may assume no duplicates in the word list.
 You may assume beginWord and endWord are non-empty and are not the same.
 */

public class _126 {

    public static class Solution1 {
        /** Reference: https://discuss.leetcode.com/topic/2857/share-two-similar-java-
solution-that-accpted-by-oj */

        Map<String, List<String>> map;
        List<List<String>> results;

        public List<List<String>> findLadders(String start, String end, List<String>
dict) {
            results = new ArrayList<>();
            if (dict.size() == 0) {
                return results;
            }

            int min = Integer.MAX_VALUE;

            Queue<String> queue = new ArrayDeque<>();
            queue.add(start);

            map = new HashMap<>();

            Map<String, Integer> ladder = new HashMap<>();
```

```
            for (String string : dict) {
                ladder.put(string, Integer.MAX_VALUE);
            }
            ladder.put(start, 0);

            dict.add(end);
            //BFS: Dijisktra search
            while (!queue.isEmpty()) {

                String word = queue.poll();

                int step = ladder.get(word)
                    + 1;//'step' indicates how many steps are needed to travel to one
word.

                if (step > min) {
                    break;
                }

                for (int i = 0; i < word.length(); i++) {
                    StringBuilder builder = new StringBuilder(word);
                    for (char ch = 'a'; ch <= 'z'; ch++) {
                        builder.setCharAt(i, ch);
                        String newWord = builder.toString();
                        if (ladder.containsKey(newWord)) {

                            if (step > ladder.get(newWord)) {
                                //Check if it is the shortest path to one word.
                                continue;
                            } else if (step < ladder.get(newWord)) {
                                queue.add(newWord);
                                ladder.put(newWord, step);
                            } else {
                                // It is a KEY line. If one word already appeared in one
ladder,
                                // Do not insert the same word inside the queue twice.
Otherwise it gets TLE.
                            }
                            if (map.containsKey(newWord)) {
                                //Build adjacent Graph
                                map.get(newWord).add(word);
                            } else {
                                List<String> list = new LinkedList();
                                list.add(word);
                                map.put(newWord, list);
                                //It is possible to write three lines in one:
                                //map.put(new_word,new LinkedList<String>
(Arrays.asList(new String[]{word})));
                                //Which one is better?
                            }

                            if (newWord.equals(end)) {
                                min = step;
                            }
```

```
                        }
                        //End if dict contains new_word
                    }
                    //End:Iteration from 'a' to 'z'
                }
                //End:Iteration from the first to the last
            }
            //End While

            //BackTracking
            LinkedList<String> result = new LinkedList<>();
            backTrace(end, start, result);

            return results;
        }

        private void backTrace(String word, String start, List<String> list) {
            if (word.equals(start)) {
                list.add(0, start);
                results.add(new ArrayList<>(list));
                list.remove(0);
                return;
            }
            list.add(0, word);
            if (map.get(word) != null) {
                for (String s : map.get(word)) {
                    backTrace(s, start, list);
                }
            }
            list.remove(0);
        }
    }
}

// ==========================================================
// FILE 127: _127.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 127. Word Ladder
 *
 *  Given two words (beginWord and endWord), and a dictionary's word list,
 *  find the length of shortest transformation sequence from beginWord to endWord, such
that:
 *  Only one letter can be changed at a time.
 *  Each transformed word must exist in the word list. Note that beginWord is not a
transformed word.
```

```
 For example,

 Given:
 beginWord = "hit"
 endWord = "cog"
 wordList = ["hot","dot","dog","lot","log","cog"]

 As one shortest transformation is "hit" -> "hot" -> "dot" -> "dog" -> "cog", return its
length 5.

 Note:

 Return 0 if there is no such transformation sequence.
 All words have the same length.
 All words contain only lowercase alphabetic characters.
 You may assume no duplicates in the word list.
 You may assume beginWord and endWord are non-empty and are not the same.
 */

public class _127 {
    public static class Solution1 {

        public int ladderLength(String beginWord, String endWord, List<String> wordList)
{
            Set<String> beginSet = new HashSet<>();
            Set<String> endSet = new HashSet<>();
            Set<String> visited = new HashSet<>();
            Set<String> dict = new HashSet<>(wordList);
            int len = 1;

            beginSet.add(beginWord);

            if (dict.contains(endWord)) {
                endSet.add(endWord);
            }

            while (!beginSet.isEmpty() && !endSet.isEmpty()) {
                Set<String> nextBeginSet = new HashSet<>();
                for (String word : beginSet) {
                    char[] chars = word.toCharArray();
                    for (int i = 0; i < chars.length; i++) {
                        for (char c = 'a'; c <= 'z'; c++) {
                            char old = chars[i];
                            chars[i] = c;
                            String newWord = new String(chars);
                            if (endSet.contains(newWord)) {
                                return len + 1;
                            }

                            if (!visited.contains(newWord) && dict.contains(newWord)) {
                                visited.add(newWord);
                                nextBeginSet.add(newWord);
                            }
                            chars[i] = old;
```

```
                        }
                    }
                }

                beginSet = nextBeginSet;
                len++;
            }
            return 0;
        }
    }
}

// ============================================================
// FILE 128: _128.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 128. Longest Consecutive Sequence

 Given an unsorted array of integers, find the length of the longest consecutive elements
sequence.

 For example,
 Given [100, 4, 200, 1, 3, 2],
 The longest consecutive elements sequence is [1, 2, 3, 4]. Return its length: 4.

 Your algorithm should run in O(n) complexity.
 */
public class _128 {
  public static class Solution1 {
    public int longestConsecutive(int[] nums) {
      Map<Integer, Integer> map = new HashMap();
      //<value, index>
      UnionFind uf = new UnionFind(nums);
      for (int i = 0; i < nums.length; i++) {
        if (map.containsKey(nums[i])) {
          continue;
        }
        map.put(nums[i], i);
        if (map.containsKey(nums[i] - 1)) {
          uf.union(i, map.get(nums[i] - 1));
          //note: we want to union this index and nums[i]-1's root index which we can get
from the map
        }
        if (map.containsKey(nums[i] + 1)) {
          uf.union(i, map.get(nums[i] + 1));
        }
```

```
      }
      return uf.maxUnion();
    }

    class UnionFind {
      int[] ids;

      public UnionFind(int[] nums) {
        ids = new int[nums.length];
        for (int i = 0; i < nums.length; i++) {
          ids[i] = i;
        }
      }

      public void union(int i, int j) {
        int x = find(ids, i);
        int y = find(ids, j);
        ids[x] = y;
      }

      public int find(int[] ids, int i) {
        while (i != ids[i]) {
          ids[i] = ids[ids[i]];
          i = ids[i];
        }
        return i;
      }

      public boolean connected(int i, int j) {
        return find(ids, i) == find(ids, j);
      }

      public int maxUnion() {
        //this is O(n)
        int max = 0;
        int[] count = new int[ids.length];
        for (int i = 0; i < ids.length; i++) {
          count[find(ids, i)]++;
          max = max < count[find(ids, i)] ? count[find(ids, i)] : max;
        }
        return max;
      }
    }
  }

  public static class Solution2 {
    //inspired by this solution: https://discuss.leetcode.com/topic/25493/simple-fast-
java-solution-using-set
    public int longestConsecutive(int[] nums) {
      if (nums == null || nums.length == 0) {
        return 0;
      }

      Set<Integer> set = new HashSet();
```

```
      for (int i : nums) {
        set.add(i);
      }
      int max = 1;

      for (int num : nums) {
        if (set.remove(num)) {
          int val = num;
          int count = 1;
          while (set.remove(val - 1)) {
            val--;//we find all numbers that are smaller than num and remove them from
the set
          }
          count += num - val;

          val = num;
          while (set.remove(val + 1)) {
            val++;//then we find all numbers that are bigger than num and also remove
them from the set
          }
          count += val - num;

          max = Math.max(max, count);
        }
      }
      return max;
    }
  }
}

// ============================================================
// FILE 129: _129.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 129. Sum Root to Leaf Numbers
 *
 * Given a binary tree containing digits from 0-9 only, each root-to-leaf path could
represent a number.

 An example is the root-to-leaf path 1->2->3 which represents the number 123.

 Find the total sum of all root-to-leaf numbers.

 For example,

 1
```

```
 / \
 2   3
 The root-to-leaf path 1->2 represents the number 12.
 The root-to-leaf path 1->3 represents the number 13.

 Return the sum = 12 + 13 = 25.

 */
public class _129 {
  public static class Solution1 {
    public int sumNumbers(TreeNode root) {
      if (root == null) {
        return 0;
      }
      List<Integer> allNumbers = new ArrayList();
      dfs(root, new StringBuilder(), allNumbers);
      int sum = 0;
      for (int i : allNumbers) {
        sum += i;
      }
      return sum;
    }

    private void dfs(TreeNode root, StringBuilder sb, List<Integer> allNumbers) {
      sb.append(root.val);
      if (root.left != null) {
        dfs(root.left, sb, allNumbers);
      }
      if (root.right != null) {
        dfs(root.right, sb, allNumbers);
      }
      if (root.left == null && root.right == null) {
        allNumbers.add(Integer.parseInt(sb.toString()));
      }
      sb.deleteCharAt(sb.length() - 1);
    }
  }

  public static class Solution2 {
    public int sumNumbers(TreeNode root) {
      return dfs(root, 0);
    }

    private int dfs(TreeNode root, int sum) {
      if (root == null) {
        return 0;
      }
      if (root.left == null && root.right == null) {
        return sum * 10 + root.val;
      }
      return dfs(root.left, sum * 10 + root.val) + dfs(root.right, sum * 10 + root.val);
    }
  }
```

```
}

// ============================================================
// FILE 130: _130.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 130. Surrounded Regions
 *
 * Given a 2D board containing 'X' and 'O' (the letter O), capture all regions surrounded
by 'X'.

 A region is captured by flipping all 'O's into 'X's in that surrounded region.

 For example,
 X X X X
 X O O X
 X X O X
 X O X X

 After running your function, the board should be:

 X X X X
 X X X X
 X X X X
 X O X X
 */
public class _130 {

    public static class Solution1 {

        /**
         * I won't call this problem hard, it's just confusing, you'll definitely want to
clarify what
         * the problem means before coding. This problem eactually means: any grid that
is 'O' but on
         * the four edges, will never be marked to 'X'; furthermore, any grid that is 'O'
and that is
         * connected with the above type of 'O' will never be marked to 'X' as well; only
all other
         * nodes that has any one direct neighbor that is an 'X' will be marked to 'X'.
         */

        int[] dirs = new int[] {0, 1, 0, -1, 0};

        public void solve(char[][] board) {
            if (board == null || board.length == 0 || board[0].length == 0) {
                return;
            }
```

```
            int m = board.length;
            int n = board[0].length;
            Queue<int[]> queue = new LinkedList();
            //check first row and last row and mark all those '0' on these two rows to be
'+' to let them be different from other 'O',
            //at the same time, we put them into the queue to get ready for a BFS to mark
all those adjacent 'O' nodes to '+' as well
            for (int j = 0; j < n; j++) {
                if (board[0][j] == 'O') {
                    board[0][j] = '+';
                    queue.offer(new int[] {0, j});
                }
                if (board[m - 1][j] == 'O') {
                    board[m - 1][j] = '+';
                    queue.offer(new int[] {m - 1, j});
                }
            }

            //check first column and last column too
            for (int i = 0; i < m; i++) {
                if (board[i][0] == 'O') {
                    board[i][0] = '+';
                    queue.offer(new int[] {i, 0});
                }
                if (board[i][n - 1] == 'O') {
                    board[i][n - 1] = '+';
                    queue.offer(new int[] {i, n - 1});
                }
            }

            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                for (int i = 0; i < 4; i++) {
                    int x = curr[0] + dirs[i];
                    int y = curr[1] + dirs[i + 1];
                    if (x >= 0 && x < m && y >= 0 && y < n && board[x][y] == 'O') {
                        board[x][y] = '+';
                        queue.offer(new int[] {x, y});
                    }
                }
            }

            //now we can safely mark all other 'O' to 'X', also remember to put those '+'
back to 'O'
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (board[i][j] == 'O') {
                        board[i][j] = 'X';
                    } else if (board[i][j] == '+') {
                        board[i][j] = 'O';
                    }
                }
            }
        }
```

```
    }
}

// ============================================================
// FILE 131: _131.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 131. Palindrome Partitioning

 Given a string s, partition s such that every substring of the partition is a
palindrome.

 Return all possible palindrome partitioning of s.

 For example, given s = "aab",
 Return

 [
 ["aa","b"],
 ["a","a","b"]
 ]

 */
public class _131 {

  public static class Solution1 {
    public List<List<String>> partition(String s) {
      List<List<String>> result = new ArrayList();
      int n = s.length();
      boolean[][] dp = new boolean[n][n];
      for (int i = 0; i < n; i++) {
        for (int j = 0; j <= i; j++) {
          if (s.charAt(j) == s.charAt(i) && (j + 1 >= i - 1 || dp[j + 1][i - 1])) {
            // j+1 >= i-1 means j and i are adjance to each other or only one char apart
from each other
            //dp[j+1][i-1] means its inner substring is a palindrome, so as long as
s.charAt(j) == s.charAt(i), then dp[j][i] must be a palindrome.
            dp[j][i] = true;
          }
        }
      }

      for (boolean[] list : dp) {
        for (boolean b : list) {
          System.out.print(b + ", ");
        }
        System.out.println();
      }
```

```
      System.out.println();

      backtracking(s, 0, dp, new ArrayList(), result);

      return result;
    }

    void backtracking(String s, int start, boolean[][] dp, List<String> temp,
        List<List<String>> result) {
      if (start == s.length()) {
        List<String> newTemp = new ArrayList(temp);
        result.add(newTemp);
      }
      for (int i = start; i < s.length(); i++) {
        if (dp[start][i]) {
          temp.add(s.substring(start, i + 1));
          backtracking(s, i + 1, dp, temp, result);
          temp.remove(temp.size() - 1);
        }
      }
    }
  }
}

// ============================================================
// FILE 132: _132.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 132. Palindrome Partitioning II

 Given a string s, partition s such that every substring of the partition is a
palindrome.

 Return the minimum cuts needed for a palindrome partitioning of s.

 For example, given s = "aab",
 Return 1 since the palindrome partitioning ["aa","b"] could be produced using 1 cut.

 */
public class _132 {

  /**This solution is cooler than Jiuzhang:
https://discuss.leetcode.com/topic/32575/easiest-java-dp-solution-97-36*/

    public static class Solution1 {
      //cut[i] stands for the minimum number of cut needed to cut [0, i] into palindromes
      //we initiazlie cut[i] with its max possible value which is i, this is because a
single char is naturally a palindrome, so, we'll cut this string into all single-char
substrings, which is the max cuts needed

      //dp[j][i] == true stands for s.substring(j,i) is a palindrome
```

```
      public int minCut(String s) {
        int n = s.length();
        char[] c = s.toCharArray();
        boolean[][] dp = new boolean[n][n];
        int[] cut = new int[n];

        for (int i = 0; i < n; i++) {
          cut[i] = i;
          for (int j = 0; j <= i; j++) {
            if (c[i] == c[j] && (j + 1 > i - 1 || dp[j + 1][i - 1])) {
              dp[j][i] = true;
              if (j == 0) {
                cut[i] = 0;
              } else {
                cut[i] = (cut[i] < cut[j - 1] + 1) ? cut[i] : cut[j - 1] + 1;
              }
            }
          }
        }

        return cut[n - 1];
      }
    }
}

// ============================================================
// FILE 133: _133.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.UndirectedGraphNode;

import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;

/**
 * 133. Clone Graph

 Clone an undirected graph.
 Each node in the graph contains a label and a list of its neighbors.


 OJ's undirected graph serialization:
 Nodes are labeled uniquely.

 We use # as a separator for each node, and , as a separator for node label and each
neighbor of the node.
 As an example, consider the serialized graph {0,1,2#1,2#2,2}.

 The graph has a total of three nodes, and therefore contains three parts as separated by
#.
```

```
 First node is labeled as 0. Connect node 0 to both nodes 1 and 2.
 Second node is labeled as 1. Connect node 1 to node 2.
 Third node is labeled as 2. Connect node 2 to node 2 (itself), thus forming a self-
cycle.
 Visually, the graph looks like the following:

    1
   / \
  /   \
 0 --- 2
      / \
      \_/

 */
public class _133 {

  public static class Solution1 {
    public UndirectedGraphNode cloneGraph(UndirectedGraphNode node) {
      if (node == null) {
        return node;
      }

      Map<Integer, UndirectedGraphNode> map = new HashMap();
      Queue<UndirectedGraphNode> queue = new LinkedList();
      UndirectedGraphNode root = new UndirectedGraphNode(node.label);
      map.put(root.label, root);
      queue.offer(node);
      //remember to offer the original input node into the queue which contains all the
information
      while (!queue.isEmpty()) {
        UndirectedGraphNode curr = queue.poll();
        for (UndirectedGraphNode eachNode : curr.neighbors) {
          if (!map.containsKey(eachNode.label)) {
            map.put(eachNode.label, new UndirectedGraphNode(eachNode.label));
            queue.offer(eachNode);
          }
          map.get(curr.label).neighbors.add(map.get(eachNode.label));
        }
      }
      return root;
    }
  }
}

// ============================================================
// FILE 134: _134.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 134. Gas Station
 *
```

```
 * There are N gas stations along a circular route, where the amount of gas at station i
is gas[i].
 You have a car with an unlimited gas tank and it costs cost[i] of gas to travel from
station i to its next station (i+1).
 You begin the journey with an empty tank at one of the gas stations.

 Return the starting gas station's index if you can travel around the circuit once,
otherwise return -1.

 Note:
 The solution is guaranteed to be unique.
 */
public class _134 {

  public static class Solution1 {
    /** Credit: https://discuss.leetcode.com/topic/5088/my-ac-is-o-1-space-o-n-running-
time-solution-does-anybody-have-posted-this-solution */
    public int canCompleteCircuit(int[] gas, int[] cost) {
      int start = gas.length - 1;
      int end = 0;
      int sum = gas[start] - cost[start];
      while (start > end) {
        if (sum >= 0) {
          sum += gas[end] - cost[end];
          end++;
        } else {
          start--;
          sum += gas[start] - cost[start];
        }
      }
      return sum >= 0 ? start : -1;
    }
  }

}

// ============================================================
// FILE 135: _135.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 135. Candy

 There are N children standing in a line. Each child is assigned a rating value.

 You are giving candies to these children subjected to the following requirements:

 Each child must have at least one candy.
 Children with a higher rating get more candies than their neighbors.
 What is the minimum candies you must give?
 */
public class _135 {
```

```
  public static class Solution1 {
    public int candy(int[] ratings) {
      int[] candy = new int[ratings.length];
      for (int i = 0; i < ratings.length; i++) {
        candy[i] = 1;
      }

      for (int i = 0; i < ratings.length - 1; i++) {
        if (ratings[i] < ratings[i + 1]) {
          candy[i + 1] = candy[i] + 1;
        }
      }

      for (int i = ratings.length - 1; i > 0; i--) {
        if (ratings[i] < ratings[i - 1]) {
          candy[i - 1] = Math.max(candy[i - 1], candy[i] + 1);
        }
      }

      int sum = 0;
      for (int i : candy) {
        sum += i;
      }

      return sum;
    }
  }
}

// ============================================================
// FILE 136: _136.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;

/**136. Single Number

 Given a non-empty array of integers, every element appears twice except for one. Find
that single one.

 Note:
 Your algorithm should have a linear runtime complexity. Could you implement it without
using extra memory?

 Example 1:
 Input: [2,2,1]
 Output: 1

 Example 2:
```

```
 Input: [4,1,2,1,2]
 Output: 4
*/

public class _136 {

  public static class Solution1 {
    /**
     * Approach 1: use set, since this problem explicitly states that every element
appears twice
     * and only one appears once so, we could safely remove the ones that are already in
the set,
     * O(n) time and O(n) space. HashTable approach works similarly like this one, but it
could be
     * more easily extend to follow-up questions.
     */
    public int singleNumber(int[] nums) {
      Set<Integer> set = new HashSet();
      for (int i : nums) {
        if (!set.add(i)) {
          set.remove(i);
        }
      }
      return set.iterator().next();
    }
  }

  public static class Solution2 {
    /**
     * Approach 2: bit manipulation, use exclusive or ^ to solve this problem: we're
using the trick
     * here: every number ^ itself will become zero, so, the only remaining element will
be the one
     * that appeared only once.
     */
    public int singleNumber(int[] nums) {
      int res = 0;
      for (int i : nums) {
        res ^= i;
      }
      return res;
    }
  }
}

// ============================================================
// FILE 137: _137.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
```

```
/**
 * 137. Single Number II

Given an array of integers, every element appears three times except for one. Find that
single one.

Note:
Your algorithm should have a linear runtime complexity. Could you implement it without
using extra memory?
*/
public class _137 {

  public static class Solution1 {
    public int singleNumber(int[] nums) {
      Map<Integer, Integer> map = new HashMap();
      for (int i : nums) {
        map.put(i, map.getOrDefault(i, 0) + 1);
      }
      for (int key : map.keySet()) {
        if (map.get(key) != 3) {
          return key;
        }
      }
      return 0;
    }
  }

  public static class Solution2 {
    /** Credit: https://discuss.leetcode.com/topic/11877/detailed-explanation-and-
generalization-of-the-bitwise-operation-method-for-single-numbers/2 */
    public int singleNumber(int[] nums) {
      int counter1 = 0;
      int counter2 = 0;
      int mask = 0;
      for (int num : nums) {
        counter2 ^= counter1 & num;
        counter1 ^= num;
        mask = ~(counter1 & counter2);
        counter1 &= mask;
        counter2 &= mask;
      }
      return counter1;
    }
  }
}

// ============================================================
// FILE 138: _138.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
```

```java
/**
 * 138. Copy List with Random Pointer
 *
 * A linked list is given such that each node contains an additional random
 * pointer which could point to any node in the list or null.
 * Return a deep copy of the list.
 * */

public class _138 {
  public static class Solution1 {
    public Node copyRandomList(Node head) {
      /**Key is the original nodes, value is the new nodes we're deep copying to.*/
      Map<Node, Node> map = new HashMap();
      Node node = head;

      //loop for the first time: copy the node themselves with only labels
      while (node != null) {
        map.put(node, new Node(node.val));
        node = node.next;
      }

      //loop for the second time: copy random and next pointers
      node = head;
      while (node != null) {
        map.get(node).next = map.get(node.next);
        map.get(node).random = map.get(node.random);
        node = node.next;
      }

      return map.get(head);
    }

    // Definition for singly-linked list with a random pointer.
    class Node {
      int val;

      Node next;
      Node random;

      Node(int x) {
        this.val = x;
      }
    }
  }
}

// ============================================================
// FILE 139: _139.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;
```

```
/**
 * 139. Word Break
 *
 * Given a non-empty string s and a dictionary wordDict containing a list of non-empty
words,
 * determine if s can be segmented into a space-separated sequence of one or more
dictionary words.
 * You may assume the dictionary does not contain duplicate words.

 For example, given
 s = "leetcode",
 dict = ["leet", "code"].

 Return true because "leetcode" can be segmented as "leet code".

 UPDATE (2017/1/4):
 The wordDict parameter had been changed to a list of strings (instead of a set of
strings).
 Please reload the code definition to get the latest changes.
 */

public class _139 {

    public static class Solution1 {
        /** this beats 70.46% submission. */
        public boolean wordBreak(String s, List<String> wordDict) {
            int n = s.length();
            boolean[] dp = new boolean[n + 1];
            dp[0] = true;
            for (int i = 1; i <= n; i++) {
                for (int j = 0; j < i; j++) {
                    if (dp[j] && wordDict.contains(s.substring(j, i))) {
                        dp[i] = true;
                        break;
                    }
                }
            }
            return dp[n];
        }
    }

    public static class Solution2 {
        /**
         * Added pruning.
         * this beats 89.91% submissions.
         */
        public boolean wordBreak(String s, List<String> wordDict) {
            int maxLen = Integer.MIN_VALUE;
            for (String word : wordDict) {
                maxLen = (word.length() > maxLen) ? word.length() : maxLen;
            }

            int n = s.length();
```

```
            boolean[] dp = new boolean[n + 1];
            dp[0] = true;
            for (int i = 1; i <= n; i++) {
                for (int j = 0; j < i; j++) {
                    if ((i - j) > maxLen) {
                        continue;
                    }
                    if (dp[j] && wordDict.contains(s.substring(j, i))) {
                        dp[i] = true;
                        break;
                    }
                }
            }
            return dp[n];
        }
    }

    public static class Solution3 {
        /**
         * Added pruning, plus start from the end to check.
         * This beats 95.20% submissions.
         */
        public boolean wordBreak(String s, List<String> wordDict) {
            int maxLen = Integer.MIN_VALUE;
            for (String word : wordDict) {
                maxLen = (word.length() > maxLen) ? word.length() : maxLen;
            }

            int n = s.length();
            boolean[] dp = new boolean[n + 1];
            dp[0] = true;
            for (int i = 1; i <= n; i++) {
                for (int lastWordLength = 1; lastWordLength <= i && lastWordLength <=
maxLen;
                    lastWordLength++) {
                    if (!dp[i - lastWordLength]) {
                        continue;
                    }
                    String sub = s.substring(i - lastWordLength, i);
                    if (wordDict.contains(sub)) {
                        dp[i] = true;
                        break;
                    }
                }
            }
            return dp[n];
        }
    }

}

// ============================================================
// FILE 140: _140.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

/**
 * 140. Word Break II

 Given a non-empty string s and a dictionary wordDict containing a list of non-empty
words,
 add spaces in s to construct a sentence where each word is a valid dictionary word.
 Return all such possible sentences.

 Note:

 The same word in the dictionary may be reused multiple times in the segmentation.
 You may assume the dictionary does not contain duplicate words.

 Example 1:

 Input:
 s = "catsanddog"
 wordDict = ["cat", "cats", "and", "sand", "dog"]
 Output:
 [
 "cats and dog",
 "cat sand dog"
 ]

 Example 2:

 Input:
 s = "pineapplepenapple"
 wordDict = ["apple", "pen", "applepen", "pine", "pineapple"]
 Output:
 [
 "pine apple pen apple",
 "pineapple pen apple",
 "pine applepen apple"
 ]
 Explanation: Note that you are allowed to reuse a dictionary word.

 Example 3:

 Input:
 s = "catsandog"
 wordDict = ["cats", "dog", "sand", "and", "cat"]
 Output:
 []
 */

public class _140 {
```

```
  public static class Solution1 {
    public List<String> wordBreak(String s, List<String> wordDict) {
      return dfs(s, wordDict, new HashMap<>());
    }

    List<String> dfs(String s, List<String> wordDict, HashMap<String, List<String>> map)
{
      if (map.containsKey(s)) {
        return map.get(s);
      }

      ArrayList<String> result = new ArrayList<>();
      if (s.length() == 0) {
        result.add("");
        return result;
      }

      for (String word : wordDict) {
        if (s.startsWith(word)) {
          List<String> subList = dfs(s.substring(word.length()), wordDict, map);
          for (String sub : subList) {
            result.add(word + (sub.length() == 0 ? "" : " ") + sub);
          }
        }
      }
      map.put(s, result);
      return result;
    }
  }
}

// ============================================================
// FILE 141: _141.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.HashSet;
import java.util.Set;

/**
 * 141. Linked List Cycle

Given a linked list, determine if it has a cycle in it.

Follow up:
Can you solve it without using extra space?
 */
public class _141 {

        public static class Solution1 {
                public boolean hasCycle(ListNode head) {
```

```
                        Set<ListNode> set = new HashSet();
                        while (head != null) {
                                if (!set.add(head)) {
                                        return true;
                                }
                                head = head.next;
                        }
                        return false;
                }
        }

        public static class Solution2 {
                public boolean hasCycle(ListNode head) {
                        ListNode slow = head;
                        ListNode fast = head;
                        while (fast != null && fast.next != null) {
                                fast = fast.next.next;
                                slow = slow.next;
                                if (fast == slow) {
                                        return true;
                                }
                        }
                        return false;
                }
        }
}

// ============================================================
// FILE 142: _142.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 142. Linked List Cycle II

 Given a linked list, return the node where the cycle begins. If there is no cycle,
return null.

 Note: Do not modify the linked list.

 Follow up:
 Can you solve it without using extra space?
 */
public class _142 {

  public static class Solution1 {
    public ListNode detectCycle(ListNode head) {
      ListNode slow = head;
      ListNode fast = head;
      while (fast != null && fast.next != null) {
        slow = slow.next;
```

```
        fast = fast.next.next;
        if (slow == fast) {
          ListNode slow2 = head;
          while (slow2 != slow) {
            slow = slow.next;
            slow2 = slow2.next;
          }
          return slow;
        }
      }
      return null;
    }
  }
}

// ============================================================
// FILE 143: _143.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 143. Reorder List
 *
 * Given a singly linked list L: L0→L1→…→Ln-1→Ln,
 reorder it to: L0→Ln→L1→Ln-1→L2→Ln-2→…

 You must do this in-place without altering the nodes' values.

 For example,
 Given {1,2,3,4}, reorder it to {1,4,2,3}.
 */
public class _143 {

    public static class Solution1 {
        public void reorderList(ListNode head) {
            if (head == null || head.next == null) {
                return;
            }
            /* first we use two pointers to separate this list into two parts */
            ListNode slowNode = head;
            ListNode fastNode = head;
            while (fastNode.next != null) {
                fastNode = fastNode.next;
                if (fastNode.next != null) {
                    fastNode = fastNode.next;
                } else {
                    break;
                }
                slowNode = slowNode.next;
            }
            // two sublist heads
```

```
            ListNode head1 = head;
            ListNode head2 = slowNode.next;
            // detach the two sublists;
            slowNode.next = null;

            // reverse the second sublist
            ListNode cur = head2;
            ListNode post = cur.next;
            cur.next = null;
            while (post != null) {
                ListNode temp = post.next;
                post.next = cur;
                cur = post;
                post = temp;
            }
            head2 = cur;// the new head of the reversed sublist

            // merge the two sublists as required
            ListNode p = head1;
            ListNode q = head2;
            while (q != null) {
                ListNode temp1 = p.next;
                ListNode temp2 = q.next;
                p.next = q;
                q.next = temp1;
                p = temp1;
                q = temp2;
            }
        }
    }
}

// ============================================================
// FILE 144: _144.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;

/**
 * 144. Binary Tree Preorder Traversal

 Given a binary tree, return the preorder traversal of its nodes' values.

 For example:
 Given binary tree {1,#,2,3},
 1
  \
```

```
   2
  /
 3
 return [1,2,3].

 Note: Recursive solution is trivial, could you do it iteratively?*/

public class _144 {
  public static class Solution1 {
    public List<Integer> preorderTraversal(TreeNode root) {
      List<Integer> list = new ArrayList();
      if (root == null) {
        return list;
      }
      Deque<TreeNode> stack = new ArrayDeque<>();
      stack.push(root);
      while (!stack.isEmpty()) {
        TreeNode curr = stack.pop();
        list.add(curr.val);
        /**We push right nodes onto the stack first, since they'll be popped out later
than
         * the left nodes, to meet the preorder: root -> left -> right. */
        if (curr.right != null) {
          stack.push(curr.right);
        }
        if (curr.left != null) {
          stack.push(curr.left);
        }
      }
      return list;
    }
  }

  public static class Solution2 {
    public List<Integer> preorderTraversal(TreeNode root) {
      List<Integer> list = new ArrayList();
      return pre(root, list);
    }

    List<Integer> pre(TreeNode root, List<Integer> list) {
      if (root == null) {
        return list;
      }
      list.add(root.val);
      pre(root.left, list);
      pre(root.right, list);
      return list;
    }
  }
}

// ============================================================
// FILE 145: _145.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Stack;

/**
 * 145. Binary Tree Postorder Traversal

 Given a binary tree, return the postorder traversal of its nodes' values.

 For example:
 Given binary tree {1,#,2,3},
 1
  \
   2
  /
 3
 return [3,2,1].

 Note: Recursive solution is trivial, could you do it iteratively?*/

public class _145 {
  public static class Solution1 {
    /**
     * A tricky one: Modify the code for pre-order traversal
     * so that it becomes root->right->left,
     * and then reverse the result to get left->right->root.
     */
    public static List<Integer> postorderTraversal(TreeNode root) {
      List<Integer> result = new ArrayList();
      if (root == null) {
        return result;
      }
      Stack<TreeNode> stack = new Stack();
      stack.push(root);
      while (!stack.isEmpty()) {
        root = stack.pop();
        result.add(root.val);
        if (root.left != null) {
          stack.push(root.left);
        }
        if (root.right != null) {
          stack.push(root.right);
        }
      }
      Collections.reverse(result);
      return result;
    }
  }
```

```
  public static class Solution2 {
    public List<Integer> postorderTraversal(TreeNode root) {
      List<Integer> result = new ArrayList();
      return post(root, result);
    }

    List<Integer> post(TreeNode root, List<Integer> result) {
      if (root == null) {
        return result;
      }
      post(root.left, result);
      post(root.right, result);
      result.add(root.val);
      return result;
    }
  }
}

// ============================================================
// FILE 146: _146.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * 146. LRU Cache
 *
 * Design and implement a data structure for Least Recently Used (LRU) cache. It should
support the following operations: get and put.

 get(key) - Get the value (will always be positive) of the key if the key exists in the
cache, otherwise return -1.
 put(key, value) - Set or insert the value if the key is not already present.
 When the cache reached its capacity, it should invalidate the least recently used item
before inserting a new item.

 Follow up:
 Could you do both operations in O(1) time complexity?

 Example:

 LRUCache cache = new LRUCache(2);//capacity

        cache.put(1, 1);
        cache.put(2, 2);
        cache.get(1);       // returns 1
        cache.put(3, 3);    // evicts key 2
        cache.get(2);       // returns -1 (not found)
        cache.put(4, 4);    // evicts key 1
```

```
        cache.get(1);       // returns -1 (not found)
        cache.get(3);       // returns 3
        cache.get(4);       // returns 4
        */

public class _146 {

    public class Solution1 {
        public class LRUCache {
            /**
             * The shortest implementation is to use LinkedHashMap:
             * specify a size of the LinkedHashMap;
             * override the removeEldestEntry method when its size exceeds max size:
             *
https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashMap.html#removeEldestEntry-
java.util.Map.Entry-
             * in the constructor, set the last boolean variable to be true: it means the
ordering mode,
             * if we set it to be true, it means in access order, false, means it's in
insertion order:
             *
https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashMap.html#LinkedHashMap-int-
float-boolean-
             */

            private Map<Integer, Integer> cache;
            private final int max;

            public LRUCache(int capacity) {
                max = capacity;
                cache = new LinkedHashMap<Integer, Integer>(capacity, 1.0f, true) {
                    public boolean removeEldestEntry(Map.Entry eldest) {
                        return cache.size() > max;
                    }
                };
            }

            public int get(int key) {
                return cache.getOrDefault(key, -1);
            }

            public void set(int key, int value) {
                cache.put(key, value);
            }
        }
    }

    public class Solution2 {
        public class LRUCache {
            /**
             * The more verbose solution is to write a doubly linked list plus a map.
             */
            private class Node {
                int key;
```

```
                int value;

                LRUCache.Node prev;
                LRUCache.Node next;

                Node(int k, int v) {
                    this.key = k;
                    this.value = v;
                }

                Node() {
                    this.key = 0;
                    this.value = 0;
                }
            }

            private int capacity;
            private int count;
            private LRUCache.Node head;
            private LRUCache.Node tail;
            private Map<Integer, LRUCache.Node> map;
            // ATTN: the value should be Node type! This is the whole point of having a
class called Node!

            public LRUCache(int capacity) {
                this.capacity = capacity;
                this.count = 0;// we need a count to keep track of the number of elements
in the cache so
                // that we know when to evict the LRU one from the cache
                this.map = new HashMap();
                head = new LRUCache.Node();
                tail = new LRUCache.Node();
                head.next = tail;
                tail.prev = head;
            }

            public int get(int key) {
                LRUCache.Node node = map.get(key);
                // HashMap allows value to be null, this is superior than HashTable!
                if (node == null) {
                    return -1;
                } else {

                    /**Do two operations: this makes the process more clear:
                     * remove the old node first, and then
                     * just add the node again.
                     * This will guarantee that this node will be at the latest position:
                     * the most recently used position.*/
                    remove(node);
                    add(node);

                    return node.value;
                }
            }
```

```
            public void set(int key, int value) {
                LRUCache.Node node = map.get(key);
                if (node == null) {
                    node = new LRUCache.Node(key, value);
                    map.put(key, node);
                    add(node);
                    count++;

                    if (count > capacity) {
                        /** ATTN: It's tail.prev, not tail, because tail is always an
invalid node, it
                         doesn't contain anything, it's always the tail.prev that is the
last node in the
                         cache*/
                        LRUCache.Node toDelete = tail.prev;
                        map.remove(toDelete.key);
                        remove(toDelete);
                        count--;
                    }
                } else {
                    remove(node);
                    node.value = value;
                    add(node);
                }
            }

            private void remove(LRUCache.Node node) {
                LRUCache.Node next = node.next;
                LRUCache.Node prev = node.prev;
                prev.next = next;
                next.prev = prev;
            }

            private void add(LRUCache.Node node) {
                // ATTN: we'll always add the node into the first position: head.next!!!!
                LRUCache.Node next = head.next;
                head.next = node;
                node.next = next;
                node.prev = head;
                next.prev = node;
            }
        }
    }
}

// ============================================================
// FILE 147: _147.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;
```

```java
import java.util.ArrayList;
import java.util.List;

/**
 * 147. Insertion Sort List
 *
 * Sort a linked list using insertion sort.
 *
 * Algorithm of Insertion Sort:
 *
 *     Insertion sort iterates, consuming one input element each repetition, and growing
a sorted output list.
 *     At each iteration, insertion sort removes one element from the input data, finds
the location it belongs within the sorted list, and inserts it there.
 *     It repeats until no input elements remain.
 *
 *
 * Example 1:
 *
 * Input: 4->2->1->3
 * Output: 1->2->3->4
 *
 * Example 2:
 *
 * Input: -1->5->3->4->0
 * Output: -1->0->3->4->5
 */
public class _147 {

  public static class Solution1 {
    public ListNode insertionSortList(ListNode head) {
      ListNode temp = head;
      List<Integer> list = new ArrayList<>();
      while (temp != null) {
        list.add(temp.val);
        temp = temp.next;
      }
      Integer[] nums = list.toArray(new Integer[list.size()]);
      for (int i = 1; i < list.size(); i++) {
        for (int j = i - 1; j >= 0; j--) {
          if (nums[j] > nums[j + 1]) {
            int tempNum = nums[j];
            nums[j] = nums[j + 1];
            nums[j + 1] = tempNum;
          }
        }
      }
      ListNode newHead = head;
      for (int i = 0; i < nums.length; i++) {
        newHead.val = nums[i];
        newHead = newHead.next;
      }
      return head;
    }
```

```
  }
}

// ============================================================
// FILE 148: _148.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 148. Sort List
 *
 * Sort a linked list in O(n log n) time using constant space complexity.
 */
public class _148 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/18100/java-merge-sort-solution
         * But this is not using constant space.
         */
        public ListNode sortList(ListNode head) {
            if (head == null || head.next == null) {
                return head;
            }

            //Step 1: split the list into halves
            ListNode prev = null;
            ListNode slow = head;
            ListNode fast = head;
            while (fast != null && fast.next != null) {
                prev = slow;
                fast = fast.next.next;
                slow = slow.next;
            }
            prev.next = null;

            //step 2: sort each half
            ListNode l1 = sortList(head);
            ListNode l2 = sortList(slow);

            //step 3: merge the two halves
            return merge(l1, l2);
        }

        private ListNode merge(ListNode l1, ListNode l2) {
            ListNode result = new ListNode(0);
            ListNode tmp = result;

            while (l1 != null && l2 != null) {
                if (l1.val < l2.val) {
                    tmp.next = l1;
```

```
                    l1 = l1.next;
                } else {
                    tmp.next = l2;
                    l2 = l2.next;
                }
                tmp = tmp.next;
            }

            if (l1 != null) {
                tmp.next = l1;
            }
            if (l2 != null) {
                tmp.next = l2;
            }
            return result.next;
        }
    }
}

// ============================================================
// FILE 149: _149.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Point;
import java.util.HashMap;
import java.util.Map;

/**
 * 149. Max Points on a Line
 *
 * Given n points on a 2D plane, find the maximum number of points that lie on the same
straight line.
 *
 * Example 1:
 * Input: [[1,1],[2,2],[3,3]]
 * Output: 3
 * Explanation:
 * ^
 * |
 * |        o
 * |     o
 * |  o
 * +------------->
 * 0  1  2  3  4
 *
 * Example 2:
 * Input: [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
 * Output: 4
 * Explanation:
 * ^
 * |
 * |  o
```

```
 * |     o        o
 * |        o
 * |  o        o
 * +------------------->
 * 0  1  2  3  4  5  6
 *
 */
public class _149 {

  public static class Solution1 {
    /** credit: https://leetcode.com/problems/max-points-on-a-line/discuss/47113/A-java-solution-with-notes */
    public int maxPoints(Point[] points) {
      if (points == null) {
        return 0;
      }
      if (points.length <= 2) {
        return points.length;
      }

      Map<Integer, Map<Integer, Integer>> map = new HashMap<>();
      int result = 0;
      for (int i = 0; i < points.length; i++) {
        map.clear();
        int overlap = 0;
        int max = 0;
        for (int j = i + 1; j < points.length; j++) {
          int x = points[j].x - points[i].x;
          int y = points[j].y - points[i].y;
          if (x == 0 && y == 0) {
            overlap++;
            continue;
          }
          int gcd = generateGCD(x, y);
          if (gcd != 0) {
            x /= gcd;
            y /= gcd;
          }

          if (map.containsKey(x)) {
            if (map.get(x).containsKey(y)) {
              map.get(x).put(y, map.get(x).get(y) + 1);
            } else {
              map.get(x).put(y, 1);
            }
          } else {
            Map<Integer, Integer> m = new HashMap<>();
            m.put(y, 1);
            map.put(x, m);
          }
          max = Math.max(max, map.get(x).get(y));
        }
        result = Math.max(result, max + overlap + 1);
      }
```

```
      return result;
    }

    private int generateGCD(int a, int b) {
      if (b == 0) {
        return a;
      } else {
        return generateGCD(b, a % b);
      }
    }
  }
}

// ============================================================
// FILE 150: _150.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;
import java.util.Stack;

/**
 * 150. Evaluate Reverse Polish Notation
 *
 * Evaluate the value of an arithmetic expression in Reverse Polish Notation.
 *
 * Valid operators are +, -, *, /. Each operand may be an integer or another expression.
 *
 * Note:
 *
 *     Division between two integers should truncate toward zero.
 *     The given RPN expression is always valid. That means the expression would always
evaluate to a result and there won't be any divide by zero operation.
 *
 * Example 1:
 *
 * Input: ["2", "1", "+", "3", "*"]
 * Output: 9
 * Explanation: ((2 + 1) * 3) = 9
 *
 * Example 2:
 *
 * Input: ["4", "13", "5", "/", "+"]
 * Output: 6
 * Explanation: (4 + (13 / 5)) = 6
 *
 * Example 3:
 *
 * Input: ["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"]
 * Output: 22
 * Explanation:
 *   ((10 * (6 / ((9 + 3) * -11))) + 17) + 5
```

```
 * = ((10 * (6 / (12 * -11))) + 17) + 5
 * = ((10 * (6 / -132)) + 17) + 5
 * = ((10 * 0) + 17) + 5
 * = (0 + 17) + 5
 * = 17 + 5
 * = 22
 */
public class _150 {

    public static class Solution1 {
        public int evalRPN(String[] tokens) {
            Stack<String> stack = new Stack<>();
            Set<String> op = new HashSet();
            op.add("+");
            op.add("-");
            op.add("*");
            op.add("/");

            int exp = 0;
            String operand1 = "";
            String operand2 = "";
            for (int i = 0; i < tokens.length; ) {
                while ((i < tokens.length) && (!op.contains(tokens[i]))) {
                    stack.push(tokens[i]);
                    i++;
                }
                if (i == tokens.length) {
                    if (!stack.isEmpty()) {
                        return Integer.parseInt(stack.pop());
                    }
                } else if (op.contains(tokens[i])) {
                    if (!stack.isEmpty()) {
                        operand2 = stack.pop();
                    }
                    if (!stack.isEmpty() && !op.contains(stack.peek())) {
                        operand1 = stack.pop();
                    }
                    if (tokens[i].equals("+")) {
                        exp = Integer.parseInt(operand1) + Integer.parseInt(operand2);
                    } else if (tokens[i].equals("-")) {
                        exp = Integer.parseInt(operand1) - Integer.parseInt(operand2);
                    } else if (tokens[i].equals("*")) {
                        exp = Integer.parseInt(operand1) * Integer.parseInt(operand2);
                    } else if (tokens[i].equals("/")) {
                        exp = Integer.parseInt(operand1) / Integer.parseInt(operand2);
                    } else {
                        exp = Integer.parseInt(operand2);
                    }
                    stack.push(String.valueOf(exp));
                    i++;
                }
            }
            return Integer.parseInt(stack.pop());
        }
```

```
    }
}

// ============================================================
// FILE 151: _151.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 151. Reverse Words in a String

  Given an input string, reverse the string word by word.
  For example,
  Given s = "the sky is blue",
  return "blue is sky the".

 Clarification:

 What constitutes a word?
 A sequence of non-space characters constitutes a word.
 Could the input string contain leading or trailing spaces?
 Yes. However, your reversed string should not contain leading or trailing spaces.
 How about multiple spaces between two words?
 Reduce them to a single space in the reversed string.
 */

public class _151 {
  public static class Solution1 {
    public String reverseWords(String s) {
      s.trim();
      if (s == null || s.length() == 0) {
        return "";
      }
      String[] words = s.split(" ");
      if (words == null || words.length == 0) {
        return "";
      }
      Deque<String> stack = new ArrayDeque<>();
      for (String word : words) {
        if (!word.equals("")) {
          stack.offer(word);
        }
      }
      StringBuilder stringBuilder = new StringBuilder();
      while (!stack.isEmpty()) {
        stringBuilder.append(stack.pollLast()).append(" ");
      }
      return stringBuilder.substring(0, stringBuilder.length() - 1).toString();
    }
  }
```

```
}

// ============================================================
// FILE 152: _152.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 152. Maximum Product Subarray

 Find the contiguous subarray within an array (containing at least one number) which has
the largest product.

 For example, given the array [2,3,-2,4],
 the contiguous subarray [2,3] has the largest product = 6.
 */
public class _152 {
  public static class Solution1 {
    public int maxProduct(int[] nums) {
      int pos = nums[0];
      int neg = nums[0];
      int max = nums[0];
      for (int i = 1; i < nums.length; i++) {
        int temp = pos;
        pos = Math.max(nums[i], nums[i] * ((nums[i] >= 0) ? pos : neg));
        neg = Math.min(nums[i], nums[i] * ((nums[i] >= 0) ? neg : temp));
        max = Math.max(max, pos);
      }
      return max;
    }
  }
}

// ============================================================
// FILE 153: _153.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 153. Find Minimum in Rotated Sorted Array

  Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.

 (i.e.,  [0,1,2,4,5,6,7] might become  [4,5,6,7,0,1,2]).

 Find the minimum element.

 You may assume no duplicate exists in the array.

 Example 1:
 Input: [3,4,5,1,2]
```

```
 Output: 1

 Example 2:
 Input: [4,5,6,7,0,1,2]
 Output: 0

 */
public class _153 {
  public static class Solution1 {
    public int findMin(int[] nums) {
      int left = 0;
      int right = nums.length - 1;
      if (nums[left] < nums[right]) {
        return nums[left];
      }
      int min = nums[0];
      while (left + 1 < right) {
        int mid = left + (right - left) / 2;
        min = Math.min(min, nums[mid]);
        if (nums[mid] > nums[left]) {
          min = Math.min(nums[left], min);
          left = mid + 1;
        } else if (nums[mid] < nums[left]) {
          right = mid - 1;
        }
      }
      min = Math.min(min, Math.min(nums[left], nums[right]));
      return min;
    }
  }
}

// ============================================================
// FILE 154: _154.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 154. Find Minimum in Rotated Sorted Array II

 Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.

 (i.e.,  [0,1,2,4,5,6,7] might become  [4,5,6,7,0,1,2]).

 Find the minimum element.

 The array may contain duplicates.

 Example 1:
 Input: [1,3,5]
 Output: 1
```

```
 Example 2:
 Input: [2,2,2,0,1]
 Output: 0

 Note:
 This is a follow up problem to Find Minimum in Rotated Sorted Array.
 Would allow duplicates affect the run-time complexity? How and why?

 */
public class _154 {
  public static class Solution1 {
    public int findMin(int[] nums) {
      int left = 0;
      int right = nums.length - 1;
      if (nums[left] < nums[right]) {
        return nums[left];
      }
      int min = nums[0];
      while (left + 1 < right) {
        int mid = left + (right - left) / 2;
        min = Math.min(min, nums[mid]);
        if (nums[mid] > nums[left]) {
          min = Math.min(nums[left], min);
          left = mid + 1;
        } else if (nums[mid] < nums[left]) {
          right = mid - 1;
        } else {
          left++;
        }
      }
      min = Math.min(min, Math.min(nums[left], nums[right]));
      return min;
    }
  }
}

// ============================================================
// FILE 155: _155.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 155. Min Stack
 * Design a stack that supports push, pop, top, and retrieving the minimum element in
constant time.

 * push(x) -- Push element x onto stack.
 * pop() -- Removes the element on top of the stack.
 * top() -- Get the top element.
 * getMin() -- Retrieve the minimum element in the stack.
```

```
 * Example:
 * MinStack minStack = new MinStack();
 * minStack.push(-2);
 * minStack.push(0);
 * minStack.push(-3);
 * minStack.getMin();   --> Returns -3.
 * minStack.pop();
 * minStack.top();      --> Returns 0.
 * minStack.getMin();   --> Returns -2.
 */

public class _155 {

    public static class Solution1 {
        class MinStack {
            private Stack<Integer> stack;
            private int min;

            /**
             * initialize your data structure here.
             */
            public MinStack() {
                stack = new Stack();
                min = Integer.MAX_VALUE;
            }

            public void push(int x) {
                /**All the trick happens here, we push the second minimum number onto the
stack before we push the newer one,
                 * this way, when popping, we could always get the next minimum one in
constant time.*/
                if (x <= min) {
                    stack.push(min);
                    min = x;
                }
                stack.push(x);
            }

            public void pop() {
                /**if the value on the top of the stack happens to be the current
minimum, we'll pop twice and change
                 * the current min value to be the last min value */
                if (min == stack.pop()) {
                    min = stack.pop();
                }
            }

            public int top() {
                return stack.peek();
            }

            public int getMin() {
                return min;
            }
```

```
        }
    }
}

// ============================================================
// FILE 156: _156.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 156. Binary Tree Upside Down
 *
 * Given a binary tree where all the right nodes are either leaf nodes with a sibling
 * (a left node that shares the same parent node) or empty,
 * flip it upside down and turn it into a tree where the original right nodes turned into
left leaf nodes. Return the new root.

 For example:
 Given a binary tree {1,2,3,4,5},
    1
   / \
  2  3
 / \
4   5

 return the root of the binary tree [4,5,2,#,#,3,1].
    4
   / \
  5  2
    / \
   3  1

 confused what "{1,#,2,3}" means? > read more on how binary tree is serialized on OJ.
 */
public class _156 {

    public static class Solution1 {
        public TreeNode upsideDownBinaryTree(TreeNode root) {
            if (root == null || root.left == null && root.right == null) {
                return root;
            }
            TreeNode newRoot = upsideDownBinaryTree(root.left);
            root.left.left = root.right;
            root.left.right = root;
            root.left = null;
            root.right = null;
            return newRoot;
        }
    }
}
```

```java
// ============================================================
// FILE 157: _157.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 157. Read N Characters Given Read4
 *
 * The API: int read4(char *buf) reads 4 characters at a time from a file.
 *
 * The return value is the actual number of characters read. For example, it returns 3 if
there is only 3 characters left in the file.
 *
 * By using the read4 API, implement the function int read(char *buf, int n) that reads n
characters from the file.
 *
 * Example 1:
 *
 * Input: buf = "abc", n = 4
 * Output: "abc"
 * Explanation: The actual number of characters read is 3, which is "abc".
 *
 * Example 2:
 *
 * Input: buf = "abcde", n = 5
 * Output: "abcde"
 *
 * Note:
 * The read function will only be called once for each test case.
 *
 */
public class _157 {
    public static class Solution1 {
        public int read(char[] buf, int n) {
            int index = 0;
            int next = 0;
            char[] buffer = new char[4];
            while (index < n && (next = read4(buffer)) != 0) {
                for (int i = 0; i < next && index < n; index++, i++) {
                    buf[index] = buffer[i];
                }
            }
            return index;
        }

        private int read4(char[] buffer) {
            //this is a fake method to make Eclipse happy
            return 0;
        }
    }
}

// ============================================================
```

```java
// FILE 158: _158.java
// ============================================================

package com.fishercoder.solutions;

/**
 158. Read N Characters Given Read4 II - Call multiple times
 The API: int read4(char *buf) reads 4 characters at a time from a file.

 The return value is the actual number of characters read. For example, it returns 3 if there is only 3 characters left in the file.

 By using the read4 API, implement the function int read(char *buf, int n) that reads n characters from the file.

 Note:
 The read function may be called multiple times.

 Example 1:

 Given buf = "abc"
 read("abc", 1) // returns "a"
 read("abc", 2); // returns "bc"
 read("abc", 1); // returns ""

 Example 2:

 Given buf = "abc"
 read("abc", 4) // returns "abc"
 read("abc", 1); // returns ""
 */
public class _158 {

  public static class Solution1 {
    /**
     * @param buf Destination buffer
     * @param n   Maximum number of characters to read
     * @return The number of characters read
     */
    private int buffPtr = 0;
    private int buffCnt = 0;
    private char[] buff = new char[4];

    public int read(char[] buf, int n) {
      int ptr = 0;
      while (ptr < n) {
        if (buffPtr == 0) {
          buffCnt = read4(buff);
        }
        if (buffCnt == 0) {
          break;
        }
        while (ptr < n && buffPtr < buffCnt) {
          buf[ptr++] = buff[buffPtr++];
```

```
        }
        if (buffPtr >= buffCnt) {
          buffPtr = 0;
        }
      }
      return ptr;
    }

    //This is a fake method to make IDE happy.
    private int read4(char[] buff) {
      return 1;
    }
  }
}

// ============================================================
// FILE 159: _159.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 159. Longest Substring with At Most Two Distinct Characters

 Given a string s , find the length of the longest substring t  that contains at most 2
distinct characters.

 Example 1:

 Input: "eceba"
 Output: 3
 Explanation: t is "ece" which its length is 3.

 Example 2:

 Input: "ccaabbb"
 Output: 5
 Explanation: t is "aabbb" which its length is 5.
 */
public class _159 {
  public static class Solution1 {
    public int lengthOfLongestSubstringTwoDistinct(String s) {
      if (s.length() < 1) {
        return 0;
      }
      Map<Character, Integer> index = new HashMap<>();
      int lo = 0;
      int hi = 0;
      int maxLength = 0;
      while (hi < s.length()) {
        if (index.size() <= 2) {
```

```
          char c = s.charAt(hi);
          index.put(c, hi);
          hi++;
        }
        if (index.size() > 2) {
          int leftMost = s.length();
          for (int i : index.values()) {
            leftMost = Math.min(leftMost, i);
          }
          char c = s.charAt(leftMost);
          index.remove(c);
          lo = leftMost + 1;
        }
        maxLength = Math.max(maxLength, hi - lo);
      }
      return maxLength;
    }
  }
}

// ============================================================
// FILE 160: _160.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.HashSet;
import java.util.Set;

/**
 * 160. Intersection of Two Linked Lists
 Write a program to find the node at which the intersection of two singly linked lists
begins.


 For example, the following two linked lists:

 A:          a1 → a2
                    ↘
                    c1 → c2 → c3
                    ↗
 B:     b1 → b2 → b3
 begin to intersect at node c1.

 Example 1:
 A:             4 → 1
                    ↘
                    8 → 4 → 5
                    ↗
 B:          5 → 0 → 1
```

```
 Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3
 Output: Reference of the node with value = 8
 Input Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).
 From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,0,1,8,4,5].
 There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.


 Example 2:
 A:         0 -> 9 → 1
                     ↘
                      2 → 4
                     ↗
 B:                 3

 Input: intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
 Output: Reference of the node with value = 2
 Input Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect).
 From the head of A, it reads as [0,9,1,2,4]. From the head of B, it reads as [3,2,4].
 There are 3 nodes before the intersected node in A; There are 1 node before the intersected node in B.


 Example 3:
 A:          2 → 6 -> 4

 B:          1 -> 5

 Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
 Output: null
 Input Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5].
 Since the two lists do not intersect, intersectVal must be 0, while skipA and skipB can be arbitrary values.
 Explanation: The two lists do not intersect, so return null.


 Notes:

 If the two linked lists have no intersection at all, return null.
 The linked lists must retain their original structure after the function returns.
 You may assume there are no cycles anywhere in the entire linked structure.
 Your code should preferably run in O(n) time and use only O(1) memory.
 */

public class _160 {

    public static class Solution1 {

        public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
```

```
            int lenA = findLen(headA);
            int lenB = findLen(headB);
            /**align headA and headB to the same starting point and then move together
until we find the intersection point*/
            while (lenA < lenB) {
                headB = headB.next;
                lenB--;
            }

            while (lenB < lenA) {
                headA = headA.next;
                lenA--;
            }

            while (headA != headB) {
                headA = headA.next;
                headB = headB.next;
            }

            return headA;
        }

        private int findLen(ListNode head) {
            int len = 0;
            while (head != null) {
                head = head.next;
                len++;
            }
            return len;
        }

    }

    public static class Solution2 {
        /**
         * Most optimal solution:
         *
         * O(m+n) time
         * O(1) space
         * credit: https://discuss.leetcode.com/topic/28067/java-solution-without-
knowing-the-difference-in-len*/
        public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
            if (headA == null || headB == null) {
                return null;
            }

            ListNode a = headA;
            ListNode b = headB;

            /**if a and b have different lengths, then it will stop the loop after second
iteration*/
            while (a != b) {
                /**for the first iteration, it'll just reset the pointer to the head of
another linkedlist*/
```

```
                a = a == null ? headB : a.next;
                b = b == null ? headA : b.next;
            }
            return a;
        }
    }

    public static class Solution3 {
        /**
         * O(m+n) time
         * O(Math.max(m, n)) space
         * */
        public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
            Set<ListNode> set = new HashSet<>();
            while (headA != null) {
                set.add(headA);
                headA = headA.next;
            }

            while (headB != null) {
                if (set.contains(headB)) {
                    return headB;
                }
                headB = headB.next;
            }
            return null;
        }
    }
}

// ============================================================
// FILE 161: _161.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 161. One Edit Distance
 *
 * Given two strings s and t, determine if they are both one edit distance apart.
 *
 * Note:
 *
 * There are 3 possiblities to satisify one edit distance apart:
 *
 *     Insert a character into s to get t
 *     Delete a character from s to get t
 *     Replace a character of s to get t
 *
 * Example 1:
 * Input: s = "ab", t = "acb"
 * Output: true
 * Explanation: We can insert 'c' into s to get t.
 *
```

```
 * Example 2:
 * Input: s = "cab", t = "ad"
 * Output: false
 * Explanation: We cannot get t from s by only one step.
 *
 * Example 3:
 * Input: s = "1203", t = "1213"
 * Output: true
 * Explanation: We can replace '0' with '1' to get t.
 */
public class _161 {
  public static class Solution1 {
    public boolean isOneEditDistance(String s, String t) {
      char[] schar = s.toCharArray();
      char[] tchar = t.toCharArray();

      if (Math.abs(s.length() - t.length()) == 1) {
        char[] longer = (s.length() > t.length()) ? schar : tchar;
        char[] shorter = (longer == schar) ? tchar : schar;

        int diffCnt = 0;
        int i = 0;
        int j = 0;
        for (; i < shorter.length && j < longer.length; ) {
          if (longer[j] != shorter[i]) {
            diffCnt++;
            j++;
          } else {
            i++;
            j++;
          }
        }
        return diffCnt == 1
            || diffCnt
            == 0;//it could be the last char of the longer is the different one, in that
case, diffCnt remains to be zero
      } else if (s.length() == t.length()) {
        int diffCnt = 0;
        for (int i = 0; i < s.length(); i++) {
          if (schar[i] != tchar[i]) {
            diffCnt++;
          }
          if (diffCnt > 1) {
            return false;
          }
        }
        return diffCnt == 1;
      }
      return false;
    }
  }
}

// ============================================================
```

```
// FILE 162: _162.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 162. Find Peak Element

 A peak element is an element that is greater than its neighbors.

 Given an input array nums, where nums[i] ≠ nums[i+1], find a peak element and return its
index.

 The array may contain multiple peaks, in that case return the index to any one of the
peaks is fine.

 You may imagine that nums[-1] = nums[n] = -∞.

 Example 1:
 Input: nums = [1,2,3,1]
 Output: 2
 Explanation: 3 is a peak element and your function should return the index number 2.

 Example 2:
 Input: nums = [1,2,1,3,5,6,4]
 Output: 1 or 5
 Explanation: Your function can return either index number 1 where the peak element is 2,
 or index number 5 where the peak element is 6.

 Note:
 Your solution should be in logarithmic complexity.
 */

public class _162 {

  public static class Solution1 {
    /**
     * credit: https://discuss.leetcode.com/topic/29329/java-solution-and-explanation-
using-invariants
     *
     * Basically, we need to keep this invariant:
     * nums[left] > nums[left-1], then we could return left as the result
     * or nums[right] > nums[right+1], then we could return right as the result
     *
     * Time: O(Ologn)
     */
    public int findPeakElement(int[] nums) {
      if (nums == null || nums.length == 0) {
        return 0;
      }
      int left = 0;
      int right = nums.length - 1;
      while (left + 1 < right) {
        int mid = left + (right - left) / 2;
```

```
        if (nums[mid] < nums[mid + 1]) {
          left = mid;
        } else {
          right = mid;
        }
      }
      return (left == nums.length - 1 || nums[left] > nums[left + 1]) ? left : right;
    }
  }

  public static class Solution2 {
    /**
     * My original O(n) solution.
     */
    public int findPeakElement(int[] nums) {
      if (nums == null || nums.length == 0) {
        return 0;
      }
      int n = nums.length;
      int result = 0;
      for (int i = 0; i < n; i++) {
        if (i == 0 && n > 1 && nums[i] > nums[i + 1]) {
          result = i;
          break;
        } else if (i == n - 1 && i > 0 && nums[i] > nums[i - 1]) {
          result = i;
          break;
        } else if (i > 0 && i < n - 1 && nums[i] > nums[i - 1] && nums[i] > nums[i + 1])
{
          result = i;
          break;
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 163: _163.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 163. Missing Ranges
 *
 * Given a sorted integer array where the range of elements are in the inclusive range
[lower, upper], return its missing ranges.
 * For example, given [0, 1, 3, 50, 75], lower = 0 and upper = 99, return ["2", "4->49",
"51->74", "76->99"].
```

```
 */
public class _163 {
    public static class Solution1 {
        public List<String> findMissingRanges(int[] nums, int lower, int upper) {
            List<String> result = new ArrayList<>();
            long low = (long) lower - 1;
            long up = 0;
            for (int i = 0; i <= nums.length; i++) {
                if (i == nums.length) {
                    up = (long) upper + 1;
                } else {
                    up = nums[i];
                }
                if (up == low + 2) {
                    result.add(low + 1 + "");
                } else if (up > low + 2) {
                    result.add((low + 1) + "->" + (up - 1));
                }
                low = up;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 164: _164.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 164. Maximum Gap
 *
 * Given an unsorted array, find the maximum difference between the successive elements
in its sorted form.
 * Return 0 if the array contains less than 2 elements.
 *
 * Example 1:
 * Input: [3,6,9,1]
 * Output: 3
 * Explanation: The sorted form of the array is [1,3,6,9], either
 *              (3,6) or (6,9) has the maximum difference 3.
 *
 * Example 2:
 * Input: [10]
 * Output: 0
 * Explanation: The array contains less than 2 elements, therefore return 0.
 *
 * Note:
 *  You may assume all elements in the array are non-negative integers and fit in the 32-
bit signed integer range.
```

```
 *  Try to solve it in linear time/space.
 */
public class _164 {
  public static class Solution1 {
    /** brute force solution */
    public int maximumGap(int[] nums) {
      if (nums.length < 2) {
        return 0;
      }

      Arrays.sort(nums);
      int max = Integer.MIN_VALUE;
      for (int i = 1; i < nums.length; ) {
        while (i < nums.length && nums[i] == nums[i - 1]) {
          i++;
        }
        if (i == nums.length) {
          i--;
          max = (nums[i] - nums[i - 1] > max) ? nums[i] - nums[i - 1] : max;
          break;
        } else {
          max = (nums[i] - nums[i - 1] > max) ? nums[i] - nums[i - 1] : max;
        }
        if (nums[i] != nums[i - 1]) {
          i++;
        }
      }
      return max;
    }
  }
}

// ============================================================
// FILE 165: _165.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 165. Compare Version Numbers

 Compare two version numbers version1 and version2.
 If version1 > version2 return 1, if version1 < version2 return -1, otherwise return 0.

 You may assume that the version strings are non-empty and contain only digits and the .
character.
 The . character does not represent a decimal point and is used to separate number
sequences.
 For instance, 2.5 is not "two and a half" or "half way to version three", it is the
fifth second-level revision of the second first-level revision.

 Here is an example of version numbers ordering:

 0.1 < 1.1 < 1.2 < 13.37
 */
```

```
public class _165 {
  public static class Solution1 {
    public int compareVersion(String version1, String version2) {
      String[] v1s = version1.split(
          "\\.");//escaping it is very important! Otherwise, it's not going to work as
expected!
      String[] v2s = version2.split("\\.");
      int len = (v1s.length < v2s.length) ? v2s.length : v1s.length;
      for (int i = 0; i < len; i++) {
        if (v1s.length == i) {
          while (i < len) {
            if (Integer.parseInt(v2s[i]) > 0) {
              return -1;
            }
            i++;
          }
        } else if (v2s.length == i) {
          while (i < len) {
            if (Integer.parseInt(v1s[i]) > 0) {
              return 1;
            }
            i++;
          }
        } else {
          if (Integer.parseInt(v1s[i]) > Integer.parseInt(v2s[i])) {
            return 1;
          } else if (Integer.parseInt(v2s[i]) > Integer.parseInt(v1s[i])) {
            return -1;
          }
        }
      }
      return 0;
    }
  }
}

// ============================================================
// FILE 166: _166.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 166. Fraction to Recurring Decimal
 *
 * Given two integers representing the numerator and denominator of a fraction, return
the fraction in string format.

 If the fractional part is repeating, enclose the repeating part in parentheses.
```

```
 For example,

 Given numerator = 1, denominator = 2, return "0.5".
 Given numerator = 2, denominator = 1, return "2".
 Given numerator = 2, denominator = 3, return "0.(6)".

 */
public class _166 {
  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/33311/simple-and-short-solution-in-
java */
    public String fractionToDecimal(int numerator, int denominator) {
      String sign =
          (numerator >= 0 && denominator >= 0) || (numerator < 0 && denominator < 0) ? ""
: "-";
      if (numerator == 0) {
        return "0";
      }
      long num = Math.abs((long) numerator);
      long deno = Math.abs((long) denominator);
      StringBuilder stringBuilder = new StringBuilder();
      stringBuilder.append(sign);
      long integral = Math.abs(num / deno);
      stringBuilder.append(integral);
      if (numerator % denominator == 0) {
        return stringBuilder.toString();
      } else {
        stringBuilder.append(".");
      }
      long remainder = num % deno;

      Map<Long, Integer> map = new HashMap<>();
      while (!map.containsKey(remainder)) {
        map.put(remainder, stringBuilder.length());
        long n = remainder * 10 / deno;
        remainder = remainder * 10 % deno;
        if (remainder != 0 || (remainder == 0 && !map.containsKey(remainder))) {
          stringBuilder.append(n);
        }
      }
      if (remainder != 0) {
        stringBuilder.insert(map.get(remainder), "(");
        stringBuilder.append(")");
      }
      return stringBuilder.toString();
    }
  }
}

// ============================================================
// FILE 167: _167.java
// ============================================================

package com.fishercoder.solutions;
```

```
/**
 * 167. Two Sum II - Input array is sorted

 Given an array of integers that is already sorted in ascending order, find two numbers
such that they add up to a specific target number.

 The function twoSum should return indices of the two numbers such that they add up to
the target, where index1 must be less than index2.

 Note:

 Your returned answers (both index1 and index2) are not zero-based.
 You may assume that each input would have exactly one solution and you may not use the
same element twice.

 Example:

 Input: numbers = [2,7,11,15], target = 9
 Output: [1,2]
 Explanation: The sum of 2 and 7 is 9. Therefore index1 = 1, index2 = 2.

 */

public class _167 {
    public static class Solution1 {
        public int[] twoSum(int[] numbers, int target) {
            int left = 0;
            int right = numbers.length - 1;
            while (left < right) {
                long sum = numbers[left] + numbers[right];
                if (sum > target) {
                    right--;
                } else if (sum < target) {
                    left++;
                } else {
                    return new int[]{left + 1, right + 1};
                }
            }
            return new int[]{-1, -1};
        }
    }
}

// ============================================================
// FILE 168: _168.java
// ============================================================

package com.fishercoder.solutions;
/**
 *
 * 168. Excel Sheet Column Title

 Given a positive integer, return its corresponding column title as appear in an Excel
```

```
sheet.

 For example:

 1 -> A
 2 -> B
 3 -> C
 ...
 26 -> Z
 27 -> AA
 28 -> AB
 ...

 Example 1:

 Input: 1
 Output: "A"

 Example 2:

 Input: 28
 Output: "AB"

 Example 3:

 Input: 701
 Output: "ZY"

 */
public class _168 {
  public static class Solution1 {
    public String convertToTitle(int n) {
      /**Get the right most digit first, move to the left, e.g. when n = 28, we get 'B'
first, then we get 'A'.*/
      StringBuilder sb = new StringBuilder();
      while (n != 0) {
        int temp = (n - 1) % 26;
        sb.append((char) (temp + 65));
        n = (n - 1) / 26;
      }
      return sb.reverse().toString();
    }
  }
}

// ============================================================
// FILE 169: _169.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
```

```
/**
 * 169. Majority Element

 Given an array of size n, find the majority element. The majority element is the element
that appears more than ⌊ n/2 ⌋ times.

 You may assume that the array is non-empty and the majority element always exist in the
array.

 Example 1:

 Input: [3,2,3]
 Output: 3

 Example 2:

 Input: [2,2,1,1,1,2,2]
 Output: 2
 */
public class _169 {
  public static class Solution1 {
    /**Moore Voting Algorithm*/
    public int majorityElement(int[] nums) {
      int count = 1;
      int majority = nums[0];
      for (int i = 1; i < nums.length; i++) {
        if (count == 0) {
          count++;
          majority = nums[i];
        } else if (nums[i] == majority) {
          count++;
        } else {
          count--;
        }
      }
      return majority;
    }
  }

  public static class Solution2 {
    public int majorityElement(int[] nums) {
      Map<Integer, Integer> map = new HashMap();
      for (int i : nums) {
        map.put(i, map.getOrDefault(i, 0) + 1);
        if (map.get(i) > nums.length / 2) {
          return i;
        }
      }
      return -1;
    }
  }

  public static class Solution3 {
```

```
    //This is O(nlogn) time.
    public int majorityElement(int[] nums) {
      Arrays.sort(nums);
      return nums[nums.length / 2];
    }
  }

  public static class Solution4 {
    //bit manipulation
    public int majorityElement(int[] nums) {
      int[] bit = new int[32];//because an integer is 32 bits, so we use an array of 32
long
      for (int num : nums) {
        for (int i = 0; i < 32; i++) {
          if ((num >> (31 - i) & 1) == 1) {
            bit[i]++;//this is to compute each number's ones frequency
          }
        }
      }
      int res = 0;
      //this below for loop is to construct the majority element: since every bit of this
element would have appeared more than n/2 times
      for (int i = 0; i < 32; i++) {
        bit[i] = bit[i] > nums.length / 2 ? 1
            : 0;//we get rid of those that bits that are not part of the majority number
        res += bit[i] * (1 << (31 - i));
      }
      return res;
    }
  }
}

// ============================================================
// FILE 170: _170.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 170. Two Sum III - Data structure design
 *
 * Design and implement a TwoSum class. It should support the following operations: add
and find.
 *
 * add - Add the number to an internal data structure.
 * find - Find if there exists any pair of numbers which sum is equal to the value.
 *
 * Example 1:
 * add(1); add(3); add(5);
```

```
 * find(4) -> true
 * find(7) -> false
 *
 * Example 2:
 * add(3); add(1); add(2);
 * find(3) -> true
 * find(6) -> false
 */

public class _170 {
  public static class Solution1 {
    private Map<Integer, Integer> map = new HashMap();
    private List<Integer> list = new ArrayList();

    // Add the number to an internal data structure.
    public void add(int number) {
      list.add(number);
      map.put(number, map.getOrDefault(number, 0) + 1);
    }

    // Find if there exists any pair of numbers which sum is equal to the value.
    public boolean find(int value) {
      for (int i = 0; i < list.size(); i++) {
        int val1 = list.get(i);
        int val2 = value - val1;
        if (map.containsKey(val2)) {
          if (val1 == val2) {
            if (map.get(val2) > 1) {
              return true;
            }
          } else {
            return true;
          }
        }
      }
      return false;
    }
  }
}

// ============================================================
// FILE 171: _171.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 170. Two Sum III - Data structure design
 *
 * Given a column title as appear in an Excel sheet, return its corresponding column
number.

 For example:
```

```
 A -> 1
 B -> 2
 C -> 3
 ...
 Z -> 26
 AA -> 27
 AB -> 28

 */
public class _171 {

  public static class Solution1 {
    public int titleToNumber(String s) {
      char[] c = s.toCharArray();
      int result = 0;
      for (int i = s.length() - 1; i >= 0; i--) {
        result +=
            (c[i] - 64) * ((int) Math.pow(26, s.length() - i - 1));//The ASCII value of A
is 65
      }
      return result;
    }
  }
}

// ============================================================
// FILE 172: _172.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 172. Factorial Trailing Zeroes

 Given an integer n, return the number of trailing zeroes in n!.

 Example 1:

 Input: 3
 Output: 0
 Explanation: 3! = 6, no trailing zero.

 Example 2:

 Input: 5
 Output: 1
 Explanation: 5! = 120, one trailing zero.

 Note: Your solution should be in logarithmic time complexity.

 */
public class _172 {

  public static class Solution1 {
    public int trailingZeroes(int n) {
```

```
      int result = 0;
      while (n > 4) {
        n /= 5;
        result += n;
      }
      return result;
    }
  }
}

// ============================================================
// FILE 173: _173.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

/**
 * 173. Binary Search Tree Iterator
 *
 * Implement an iterator over a binary search tree (BST). Your iterator will be
initialized with the root node of a BST.
 *
 * Calling next() will return the next smallest number in the BST.
 *
 * Note: next() and hasNext() should run in average O(1) time and uses O(h) memory, where
h is the height of the tree.
 */
public class _173 {

  public static class Solution1 {

    public static class BSTIterator {
      private Queue<Integer> queue;

      public BSTIterator(TreeNode root) {
        queue = new LinkedList<>();
        if (root != null) {
          dfs(root, queue);
        }
      }

      private void dfs(TreeNode root, Queue<Integer> q) {
        if (root.left != null) {
          dfs(root.left, q);
        }
        q.offer(root.val);
        if (root.right != null) {
          dfs(root.right, q);
```

```
        }
      }

      public boolean hasNext() {
        return !queue.isEmpty();
      }

      public int next() {
        return queue.poll();
      }
    }
  }

  public static class Solution2 {
    public static class BSTIterator {
      /**
       * This is a super cool/clever idea: use a stack to store all the current left
nodes of the BST, when pop(), we
       * push all its right nodes into the stack if there are any.
       * This way, we use only O(h) memory for this iterator, this is a huge saving when
the tree is huge
       * since h could be much smaller than n. Cheers!
       */

      private Stack<TreeNode> stack;

      public BSTIterator(TreeNode root) {
        stack = new Stack();
        pushToStack(root, stack);
      }

      private void pushToStack(TreeNode root, Stack<TreeNode> stack) {
        while (root != null) {
          stack.push(root);
          root = root.left;
        }
      }

      public boolean hasNext() {
        return !stack.isEmpty();
      }

      public int next() {
        TreeNode curr = stack.pop();
        pushToStack(curr.right, stack);
        return curr.val;
      }
    }
  }
}

// ============================================================
// FILE 174: _174.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 174. Dungeon Game

 The demons had captured the princess (P) and imprisoned her in the bottom-right corner
of a dungeon. The dungeon consists of M x N rooms laid out in a 2D grid. Our valiant
knight (K) was initially positioned in the top-left room and must fight his way through
the dungeon to rescue the princess.

 The knight has an initial health point represented by a positive integer. If at any
point his health point drops to 0 or below, he dies immediately.

 Some of the rooms are guarded by demons, so the knight loses health (negative integers)
upon entering these rooms; other rooms are either empty (0's) or contain magic orbs that
increase the knight's health (positive integers).

 In order to reach the princess as quickly as possible, the knight decides to move only
rightward or downward in each step.

 Write a function to determine the knight's minimum initial health so that he is able to
rescue the princess.

 For example, given the dungeon below, the initial health of the knight must be at least
7 if he follows the optimal path RIGHT-> RIGHT -> DOWN -> DOWN.
 -2 (K)         -3      3
 -5     -10     1
 10     30      -5 (P)

 Note:

 The knight's health has no upper bound.
 Any room can contain threats or power-ups, even the first room the knight enters and the
bottom-right room where the princess is imprisoned.

 */
public class _174 {

  public static class Solution1 {
    /** This problem should fill the dp matrix from bottom right. */
    public int calculateMinimumHP(int[][] dungeon) {
      if (dungeon == null || dungeon.length == 0) {
        return 0;
      }

      int height = dungeon.length;
      int width = dungeon[0].length;
      int[][] dp = new int[height][width];
      dp[height - 1][width - 1] =
          (dungeon[height - 1][width - 1] > 0) ? 1 : 1 - dungeon[height - 1][width - 1];
```

```
      //fill the last column
      for (int i = height - 2; i >= 0; i--) {
        int temp = dp[i + 1][width - 1] - dungeon[i][width - 1];
        dp[i][width - 1] = Math.max(1, temp);
      }

      //fill the last row
      for (int j = width - 2; j >= 0; j--) {
        int temp = dp[height - 1][j + 1] - dungeon[height - 1][j];
        dp[height - 1][j] = Math.max(temp, 1);
      }

      for (int i = height - 2; i >= 0; i--) {
        for (int j = width - 2; j >= 0; j--) {
          int down = Math.max(1, dp[i + 1][j] - dungeon[i][j]);
          int right = Math.max(1, dp[i][j + 1] - dungeon[i][j]);
          dp[i][j] = Math.min(down, right);
        }
      }

      CommonUtils.printMatrix(dp);
      return dp[0][0];
    }
  }
}

// ============================================================
// FILE 175: _179.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 179. Largest Number

 Given a list of non negative integers, arrange them such that they form the largest
number.

 Example 1:

 Input: [10,2]
 Output: "210"

 Example 2:

 Input: [3,30,34,5,9]
 Output: "9534330"

 Note: The result may be very large, so you need to return a string instead of an
integer.
 */
```

```
public class _179 {

  public static class Solution1 {
    public String largestNumber(int[] num) {
      if (num.length == 0) {
        return "";
      }
      if (num.length == 1) {
        return Integer.toString(num[0]);
      }
      String[] str = new String[num.length];
      for (int i = 0; i < num.length; i++) {
        str[i] = Integer.toString(num[i]);
      }
      Arrays.sort(str, new StringComparator());
      StringBuilder sb = new StringBuilder("");
      for (int i = num.length - 1; i >= 0; i--) {
        sb.append(str[i]);
      }
      if (sb.charAt(0) == '0') {
        return "0";
      }
      return sb.toString();
    }

    class StringComparator implements Comparator<String> {
      public int compare(String s1, String s2) {
        if (s1.length() == 0 && s2.length() == 0) {
          return 0;
        }
        if (s2.length() == 0) {
          return 1;
        }
        if (s1.length() == 0) {
          return -1;
        }
        for (int i = 0; i < s1.length() && i < s2.length(); i++) {
          if (s1.charAt(i) > s2.charAt(i)) {
            return 1;
          } else if (s1.charAt(i) < s2.charAt(i)) {
            return -1;
          }
        }
        if (s1.length() == s2.length()) {
          return 0;
        }
        if (s1.length() > s2.length()) {
          if (s1.charAt(0) < s1.charAt(s2.length())) {
            return 1;
          } else if (s1.charAt(0) > s1.charAt(s2.length())) {
            return -1;
          } else {
            return compare(s1.substring(s2.length()), s2);
          }
```

```
        } else {
          if (s2.charAt(0) < s2.charAt(s1.length())) {
            return -1;
          } else if (s2.charAt(0) > s2.charAt(s1.length())) {
            return 1;
          } else {
            return compare(s1, s2.substring(s1.length()));
          }
        }
      }
    }
  }
}

// ============================================================
// FILE 176: _186.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 186. Reverse Words in a String II

 Given an input string, reverse the string word by word. A word is defined as a sequence
of non-space characters.

 The input string does not contain leading or trailing spaces and the words are always
separated by a single space.

 For example,
 Given s = "the sky is blue",
 return "blue is sky the".

 Could you do it in-place without allocating extra space?
 */
public class _186 {
  public static class Solution1 {
    public void reverseWords(char[] s) {
      // Three steps to reverse
      // 1, reverse the whole sentence
      reverse(s, 0, s.length - 1);
      // 2, reverse each word
      int start = 0;
      for (int i = 0; i < s.length; i++) {
        if (s[i] == ' ') {
          reverse(s, start, i - 1);
          start = i + 1;
        }
      }
      // 3, reverse the last word, if there is only one word this will solve the corner
case
      reverse(s, start, s.length - 1);
    }
```

```
    private void reverse(char[] s, int start, int end) {
      while (start < end) {
        char temp = s[start];
        s[start++] = s[end];
        s[end--] = temp;
      }
    }
  }

}

// ============================================================
// FILE 177: _187.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 187. Repeated DNA Sequences
 *
 * All DNA is composed of a series of nucleotides abbreviated as A, C, G, and T,
 * for example: "ACGAATTCCG". When studying DNA, it is sometimes useful to identify
repeated sequences within the DNA.
 * Write a function to find all the 10-letter-long sequences (substrings) that occur more
than once in a DNA molecule.

 For example,

 Given s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT",

 Return:
 ["AAAAACCCCC", "CCCCCAAAAA"].
 */
public class _187 {
    public static class Solution1 {
        public List<String> findRepeatedDnaSequences(String s) {
            Map<String, Integer> map = new HashMap();
            for (int i = 0; i < s.length() - 9; i++) {
                String sequence = s.substring(i, i + 10);
                map.put(sequence, map.getOrDefault(sequence, 0) + 1);
            }
            List<String> repeatedSequences = new ArrayList<>();
            for (Map.Entry<String, Integer> entry : map.entrySet()) {
                if (entry.getValue() > 1) {
                    repeatedSequences.add(entry.getKey());
                }
            }
            return repeatedSequences;
        }
```

```
    }
}

// ============================================================
// FILE 178: _188.java
// ============================================================

package com.fishercoder.solutions;

/**

 188. Best Time to Buy and Sell Stock IV

 Say you have an array for which the ith element is the price of a given stock on day i.

 Design an algorithm to find the maximum profit. You may complete at most k transactions.

 Note:
 You may not engage in multiple transactions at the same time (ie, you must sell the
stock before you buy again).

 Example 1:
 Input: [2,4,1], k = 2
 Output: 2
 Explanation: Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = 4-2 = 2.

 Example 2:
 Input: [3,2,6,5,0,3], k = 2
 Output: 7
 Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4.
 Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

 */
public class _188 {
  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/8984/a-concise-dp-solution-in-java */
    public int maxProfit(int k, int[] prices) {
      int len = prices.length;
      if (k >= len / 2) {
        return quickSolve(prices);
      }

      int[][] t = new int[k + 1][len];
      for (int i = 1; i <= k; i++) {
        int tmpMax = -prices[0];
        for (int j = 1; j < len; j++) {
          t[i][j] = Math.max(t[i][j - 1], prices[j] + tmpMax);
          tmpMax = Math.max(tmpMax, t[i - 1][j - 1] - prices[j]);
        }
      }
      return t[k][len - 1];
    }

    private int quickSolve(int[] prices) {
```

```
      int len = prices.length;
      int profit = 0;
      for (int i = 1; i < len; i++) {
        // as long as there is a price gap, we gain a profit.
        if (prices[i] > prices[i - 1]) {
          profit += prices[i] - prices[i - 1];
        }
      }
      return profit;
    }
  }
}

// ============================================================
// FILE 179: _189.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 189. Rotate Array

 Given an array, rotate the array to the right by k steps, where k is non-negative.

 Example 1:
 Input: [1,2,3,4,5,6,7] and k = 3
 Output: [5,6,7,1,2,3,4]
 Explanation:
 rotate 1 steps to the right: [7,1,2,3,4,5,6]
 rotate 2 steps to the right: [6,7,1,2,3,4,5]
 rotate 3 steps to the right: [5,6,7,1,2,3,4]

 Example 2:
 Input: [-1,-100,3,99] and k = 2
 Output: [3,99,-1,-100]
 Explanation:
 rotate 1 steps to the right: [99,-1,-100,3]
 rotate 2 steps to the right: [3,99,-1,-100]

 Note:
 Try to come up as many solutions as you can, there are at least 3 different ways to
solve this problem.
 Could you do it in-place with O(1) extra space?
 * */

public class _189 {

  public static class Solution1 {
    /**
     * O(n*k) time
     * O(1) space
     */
    public void rotate(int[] nums, int k) {
      int tmp;
```

```
      for (int i = 0; i < k; i++) {
        tmp = nums[nums.length - 1];
        for (int j = nums.length - 1; j > 0; j--) {
          nums[j] = nums[j - 1];
        }
        nums[0] = tmp;
      }
    }
  }

  public static class Solution2 {
    /**
     * using an extra array of the same size to copy it
     * O(n) time
     * O(n) space
     */
    public void rotate(int[] nums, int k) {
      int len = nums.length;
      int[] tmp = new int[len];
      for (int i = 0; i < len; i++) {
        tmp[(i + k) % len] = nums[i];
      }
      for (int i = 0; i < len; i++) {
        nums[i] = tmp[i];
      }
    }
  }

  public static class Solution3 {
    /**
     * reverse three times
     * O(n) time
     * O(1) space
     */
    public void rotate(int[] nums, int k) {
      int len = nums.length;
      k %= len;
      reverse(nums, 0, len - 1);
      reverse(nums, 0, k - 1);
      reverse(nums, k, len - 1);
    }

    private void reverse(int[] nums, int start, int end) {
      while (start < end) {
        int tmp = nums[start];
        nums[start] = nums[end];
        nums[end] = tmp;
        start++;
        end--;
      }
    }
  }
}
```

```
// ============================================================
// FILE 180: _190.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 190. Reverse Bits
 * Reverse bits of a given 32 bits unsigned integer.

 Example:
 Input: 43261596
 Output: 964176192

 Explanation: 43261596 represented in binary as 00000010100101000001111010011100,
 return 964176192 represented in binary as 00111001011110000010100101000000.

 Follow up:
 If this function is called many times, how would you optimize it?
 */

public class _190 {
    /**delimiting the binary string into 4 bits array will make it easier to
see/visualize:
     * original binary format:
     * 0000,0010,1001,0100,0001,1110,1001,1100,
     * after reversing, becomes:
     * 0011,1001,0111,1000,0010,1001,0100,0000
     * The most right side digit shifted to the most left side, the 2nd right side digit
shifted to the 2nd left side, so forth..*/

    /**This post: http://stackoverflow.com/questions/2811319/difference-between-and
     * gives a good explanation between logical right shift: ">>>" and arithmetic right
shift: ">>".
     * Basically, ">>" preserves the most left bit and treats it as the sign for this
number,
     * e.g. -2 represented in 8 bits is 11111110, thus -2 >> 1 will become 11111111, i.e.
-1
     * notice its sign bit (the most left one bit) is preserved
     * However, logical right shift ">>>" doesn't care about the first bit on the most
left,
     * it simply shifts every bit to the right.
     * e.g. -2 >>> 1 would become 11111111111111111111111111111111, i.e. 2147483647*/

 public static class Solution1 {
      // you need treat n as an unsigned value
      public int reverseBits(int n) {
        int res = 0;
        for (int i = 0; i < 32; i++) {
          res += n & 1;//get the most right bit each time
          n >>>= 1;//do UN-signed right shift by 1 each time
          if (i < 31) {
            res <<=
                1;//shift this number to the left by 1 each time, so that eventually,
```

```
this number is reversed
          }
        }
        return res;
      }
    }
}

// ============================================================
// FILE 181: _191.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 191. Number of 1 Bits
 *
 * Write a function that takes an unsigned integer and returns the number of ’1' bits it
has (also known as the Hamming weight).
 * For example, the 32-bit integer ’11' has binary representation
00000000000000000000000000001011, so the function should return 3.*/

public class _191 {

    public static class Solution1 {
        /**Doing bitwise AND operation between n and n-1 will always flip the least
significant 1 bit in n to zero
         example run for the above editorial solution: input 5, n will be 5&4 and becomes
4,
         then in the next run, n will become 4&3 which is 0, thus exit the while loop.*/
        public int hammingWeight(int n) {
            int bits = 0;
            while (n != 0) {
                bits++;
                n &= (n - 1);
            }
            return bits;
        }
    }

    public static class Solution2 {
        public int hammingWeight(int n) {
            int bits = 0;
            int mask = 1;
            for (int i = 0; i < 32; i++) {
                if ((n & mask) != 0) {
                    bits++;
                }
                mask <<= 1;
            }
            return bits;
        }
    }
```

```
    public static class Solution3 {
        public int hammingWeight(int n) {
            int bits = 0;
            for (int i = 0; i < 32; i++) {
                if ((n & 1) == 1) {
                    bits++;
                }
                if (n == 0) {
                    return bits;
                }
                /**must use unsigned right shift operator since the problem says this is
an unsigned value*/
                n >>>= 1;
            }
            return bits;
        }
    }

    public static class Solution4 {
        public int hammingWeight(int n) {
            int bits = 0;
            for (int i = 0; i < 32; i++) {
                bits += n & 1;
                n >>>= 1;
            }
            return bits;
        }
    }
}

// ============================================================
// FILE 182: _198.java
// ============================================================

package com.fishercoder.solutions;

/**198. House Robber

 You are a professional robber planning to rob houses along a street.
 Each house has a certain amount of money stashed,
 the only constraint stopping you from robbing each of them is that adjacent houses have
security
 system connected and it will automatically contact the police if two adjacent houses
were broken into on the same night.
 Given a list of non-negative integers representing the amount of money of each house,
 determine the maximum amount of money you can rob tonight without alerting the police.

 Example 1:
 Input: [1,2,3,1]
 Output: 4
 Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
 Total amount you can rob = 1 + 3 = 4.

 Example 2:
```

```
 Input: [2,7,9,3,1]
 Output: 12
 Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money =
1).
 Total amount you can rob = 2 + 9 + 1 = 12.
 */
public class _198 {

    public static class Solution1 {
        public int rob(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            if (nums.length == 1) {
                return nums[0];
            }
            if (nums.length == 2) {
                return Math.max(nums[0], nums[1]);
            }
            int[] dp = new int[nums.length];
            dp[0] = nums[0];
            dp[1] = Math.max(nums[0], nums[1]);
            for (int i = 2; i < nums.length; i++) {
                dp[i] = Math.max(dp[i - 2] + nums[i], dp[i - 1]);
            }
            return dp[nums.length - 1];
        }
    }
}

// ============================================================
// FILE 183: _199.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**199. Binary Tree Right Side View

Given a binary tree, imagine yourself standing on the right side of it, return the values
of the nodes you can see ordered from top to bottom.

For example:
Given the following binary tree,

   1            <---
 /   \
2     3         <---
```

```
 \     \
  5     4       <---

You should return [1, 3, 4]. */

public class _199 {

        public static class Solution1 {
                /**credit: https://leetcode.com/problems/binary-tree-right-side-
view/discuss/56012/My-simple-accepted-solution(JAVA)*/
                public List<Integer> rightSideView(TreeNode root) {
                        List<Integer> result = new ArrayList<>();
                        rightView(root, result, 0);
                        return result;
                }

                void rightView(TreeNode curr, List<Integer> result, int currDepth) {
                        if (curr == null) {
                                return;
                        }
                        if (currDepth == result.size()) {
                                result.add(curr.val);
                        }
                        rightView(curr.right, result, currDepth + 1);
                        rightView(curr.left, result, currDepth + 1);
                }
        }

        public static class Solution2 {
                /**BFS the tree*/
                public List<Integer> rightSideView(TreeNode root) {
                        List<Integer> result = new ArrayList<>();
                        if (root == null) {
                                return result;
                        }
                        Queue<TreeNode> q = new LinkedList<>();
                        q.offer(root);
                        while (!q.isEmpty()) {
                                int size = q.size();
                                for (int i = 0; i < size; i++) {
                                        TreeNode curr = q.poll();
                                        if (i == size - 1) {
                                                result.add(curr.val);
                                        }
                                        if (curr.left != null) {
                                                q.offer(curr.left);
                                        }
                                        if (curr.right != null) {
                                                q.offer(curr.right);
                                        }
                                }
                        }
                        return result;
                }
```

```
        }

}

// ============================================================
// FILE 184: _200.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 200. Number of Islands
 *
 * Given a 2d grid map of '1's (land) and '0's (water),
 * count the number of islands.
 * An island is surrounded by water and is formed by connecting adjacent lands
horizontally or vertically.
 * You may assume all four edges of the grid are all surrounded by water.

 Example 1:
 11110
 11010
 11000
 00000
 Answer: 1

 Example 2:
 11000
 11000
 00100
 00011
 Answer: 3

 */
public class _200 {

    public static class Solution1 {

        public int numIslands(char[][] grid) {
            if (grid == null || grid.length == 0) {
                return 0;
            }
            int count = 0;
            int m = grid.length;
            int n = grid[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == '1') {
                        count++;
                        dfs(grid, i, j, m, n);
                    }
                }
            }
            return count;
```

```
        }

        void dfs(char[][] grid, int i, int j, int m, int n) {
            if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == '0') {
                return;
            }
            grid[i][j] = '0';
            dfs(grid, i + 1, j, m, n);
            dfs(grid, i, j + 1, m, n);
            dfs(grid, i - 1, j, m, n);
            dfs(grid, i, j - 1, m, n);
        }
    }

    public static class Solution2 {

        class UnionFind {
            int count;
            int m;
            int n;
            int[] ids;

            public UnionFind(char[][] grid) {
                m = grid.length;
                n = grid[0].length;
                ids = new int[m * n];
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        if (grid[i][j] == '1') {
                            count++;
                            ids[i * n + j] = i * n + j;
                        }
                    }
                }
            }

            public void union(int i, int j) {
                int x = find(ids, i);
                int y = find(ids, j);
                if (x != y) {
                    /**note: this is when x != y, only in this case, we should union
these two nodes, which makes sense naturally.*/
                    count--;
                    ids[x] = y;//ids[y] = x; //also works
                }
            }

            public int find(int[] ids, int i) {
                if (ids[i] == i) {
                    return i;
                }
                return find(ids, ids[i]);
            }
        }
```

```
        public int numIslands(char[][] grid) {
            if (grid == null || grid.length == 0 || grid[0].length == 0) {
                return 0;
            }
            int[] dirs = new int[]{0, 1, 0, -1, 0};
            UnionFind uf = new UnionFind(grid);
            int m = grid.length;
            int n = grid[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == '1') {
                        for (int k = 0; k < 4; k++) {
                            int x = i + dirs[k];
                            int y = j + dirs[k + 1];
                            if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] == '1')
{
                                int id1 = i * n + j;
                                int id2 = x * n + y;
                                uf.union(id1, id2);
                            }
                        }
                    }
                }
            }
            return uf.count;
        }
    }
}

// ============================================================
// FILE 185: _201.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 201. Bitwise AND of Numbers Range
 *
 * Given a range [m, n] where 0 <= m <= n <= 2147483647, return the bitwise AND of all
numbers in this range, inclusive.
 *
 * For example, given the range [5, 7], you should return 4.
 */
public class _201 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/28538/java-python-easy-solution-
with-explanation
         * Bitwise AND operation within range actually turns out to be doing some
operations with just these two boundary numbers: m and n
         * e.g. 5 and 7, in binary, they are 101 and 111, the result for [5,7] is 5&6&7
which is 101&110&111
```

```
         * this we can understand it to be shifting the digits of m and n from left to
right until they become the same, then we pad that number with zeroes on the right side
         */
        public int rangeBitwiseAnd(int m, int n) {
            int i = 0;
            while (m != n) {
                m >>= 1;
                n >>= 1;
                i++;
            }
            return (n << i);
        }
    }
}

// ============================================================
// FILE 186: _202.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 202. Happy Number
 *
 * Write an algorithm to determine if a number is "happy".
 * A happy number is a number defined by the following process:
 * Starting with any positive integer,
 * replace the number by the sum of the squares of its digits,
 * and repeat the process until the number equals 1 (where it will stay),
 * or it loops endlessly in a cycle which does not include 1.
 * Those numbers for which this process ends in 1 are happy numbers.
 *
 * Example: 19 is a happy number
 *
 * 12 + 92 = 82
 * 82 + 22 = 68
 * 62 + 82 = 100
 * 12 + 02 + 02 = 1
 */
public class _202 {
    public static class Solution1 {
        public boolean isHappy(int n) {
            if (n == 1) {
                return true;
            }
            Set<Integer> set = new HashSet();
            while (n != 1) {
                String str = String.valueOf(n);
                n = 0;
                for (int i = 0; i < str.length(); i++) {
                    int temp = Character.getNumericValue(str.charAt(i));
```

```
                    n += temp * temp;
                }
                if (n == 1) {
                    return true;
                }
                if (!set.add(n)) {
                    return false;
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 187: _203.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 203. Remove Linked List Elements
 *
 * Remove all elements from a linked list of integers that have value val.
 *
 * Example
 * Given: 1 --> 2 --> 6 --> 3 --> 4 --> 5 --> 6, val = 6
 * Return: 1 --> 2 --> 3 --> 4 --> 5
 */
public class _203 {
    public static class Solution1 {
        /**
         * This is a very good question to test your understanding of
pointers/memory/addresses, although it's marked as EASY.
         * All the three nodes: dummy, curr and prev are indispensable.
         *
         * 1. Eventually, we should return dummy.next as the final result.
         * 2. we assign head to curr, dummy to prev
         * 3. and then we use prev and curr to traverse through the list and do the work
         * 4. each time, we only move one node forward, so we don't need another while
loop inside the while loop
         * 5. KEY: if(curr.val == val), then curr is the node we want to remove, so,
we'll assign curr.next to prev.next, thus, prev won't have that node
         * else, we'll keep moving prev forward, so, just do prev = prev.next
         * but, for both cases, we'll also move curr forward, so we put curr = curr.next
in the outside.
         */
        public ListNode removeElements(ListNode head, int val) {
            ListNode dummy = new ListNode(-1);
            dummy.next = head;
            ListNode curr = head;
            ListNode prev = dummy;
```

```
            while (curr != null) {
                if (curr.val == val) {
                    prev.next = curr.next;
                } else {
                    prev = prev.next;
                }
                curr = curr.next;
            }
            return dummy.next;
        }
    }
}

// ============================================================
// FILE 188: _204.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 204. Count Primes
 *
 * Description:

 Count the number of prime numbers less than a non-negative number, n.

 Hint:

 Let's start with a isPrime function. To determine if a number is prime, we need to check if it is not divisible by any number less than n.
 The runtime complexity of isPrime function would be O(n) and hence counting the total prime numbers up to n would be O(n2). Could we do better?

 As we know the number must not be divisible by any number > n / 2,
 we can immediately cut the total iterations half by dividing only up to n / 2. Could we still do better?

 Let's write down all of 12's factors:

 2 × 6 = 12
 3 × 4 = 12
 4 × 3 = 12
 6 × 2 = 12
 As you can see, calculations of 4 × 3 and 6 × 2 are not necessary. Therefore, we only need to consider factors up to √n because,
 if n is divisible by some number p, then n = p × q and since p ≤ q, we could derive that p ≤ √n.

 Our total runtime has now improved to O(n1.5), which is slightly better. Is there a faster approach?

 public int countPrimes(int n) {
     int count = 0;
     for (int i = 1; i < n; i++) {
```

```
        if (isPrime(i)) count++;
    }
 return count;
 }

 private boolean isPrime(int num) {
    if (num <= 1) return false;
    // Loop's ending condition is i * i <= num instead of i <= sqrt(num)
    // to avoid repeatedly calling an expensive function sqrt().
    for (int i = 2; i * i <= num; i++) {
       if (num % i == 0) return false;
    }
    return true;
 }
```

The Sieve of Eratosthenes is one of the most efficient ways to find all prime numbers up to n.
But don't let that name scare you, I promise that the concept is surprisingly simple.

Sieve of Eratosthenes: algorithm steps for primes below 121. "Sieve of Eratosthenes Animation" by SKopp is licensed under CC BY 2.0.
We start off with a table of n numbers. Let's look at the first number, 2. We know all multiples of 2 must not be primes, so we mark them off as non-primes. Then we look at the next number, 3. Similarly, all multiples of 3 such as 3 × 2 = 6, 3 × 3 = 9, ... must not be primes, so we mark them off as well. Now we look at the next number, 4, which was already marked off.
What does this tell you? Should you mark off all multiples of 4 as well?
4 is not a prime because it is divisible by 2, which means all multiples of 4 must also be divisible by 2 and were already marked off.
So we can skip 4 immediately and go to the next number, 5.
Now, all multiples of 5 such as 5 × 2 = 10, 5 × 3 = 15, 5 × 4 = 20, 5 × 5 = 25, ... can be marked off.
There is a slight optimization here, we do not need to start from 5 × 2 = 10. Where should we start marking off?

In fact, we can mark off multiples of 5 starting at 5 × 5 = 25, because 5 × 2 = 10 was already marked off by multiple of 2,
similarly 5 × 3 = 15 was already marked off by multiple of 3.
Therefore, if the current number is p, we can always mark off multiples of p starting at p2, then in increments of p: p2 + p, p2 + 2p, ...
Now what should be the terminating loop condition?

It is easy to say that the terminating loop condition is p < n, which is certainly correct but not efficient. Do you still remember Hint #3?
Yes, the terminating loop condition can be p < √n, as all non-primes ≥ √n must have already been marked off. When the loop terminates,
all the numbers in the table that are non-marked are prime.

The Sieve of Eratosthenes uses an extra O(n) memory and its runtime complexity is O(n log log n).
For the more mathematically inclined readers, you can read more about its algorithm complexity on Wikipedia.

```
 public int countPrimes(int n) {
```

```
     boolean[] isPrime = new boolean[n];
     for (int i = 2; i < n; i++) {
        isPrime[i] = true;
    }

     // Loop's ending condition is i * i < n instead of i < sqrt(n)
     // to avoid repeatedly calling an expensive function sqrt().

     for (int i = 2; i * i < n; i++) {
         if (!isPrime[i]) continue;
         for (int j = i * i; j < n; j += i) {
            isPrime[j] = false;
         }
    }
     int count = 0;
     for (int i = 2; i < n; i++) {
        if (isPrime[i]) count++;
     }

    return count;
 }
 */
public class _204 {

    public static class Solution1 {
        public int countPrimes(int n) {
            boolean[] notPrime = new boolean[n];
            int count = 0;
            for (int i = 2; i < n; i++) {
                if (!notPrime[i]) {
                    count++;
                    for (int j = 2; i * j < n; j++) {
                        notPrime[i * j] = true;
                    }
                }
            }
            return count;
        }
    }
}

// ============================================================
// FILE 189: _205.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
/**
 * 205. Isomorphic Strings
 *
 * Given two strings s and t, determine if they are isomorphic.
```

```
 Two strings are isomorphic if the characters in s can be replaced to get t.

 All occurrences of a character must be replaced with another character while preserving
the order of characters. No two characters may map to the same character but a character
may map to itself.

 For example,
 Given "egg", "add", return true.

 Given "foo", "bar", return false.

 Given "paper", "title", return true.

 Note:
 You may assume both s and t have the same length.*/
public class _205 {
    /**space should be O(1) since it only has alphabetic letters which are capped at
52.*/

    public static class Solution1 {
        public boolean isIsomorphic(String s, String t) {
            if (s == null || s.length() == 0) {
                return (t == null || t.length() == 0);
            }
            if (t == null || t.length() == 0) {
                return (s == null || s.length() == 0);
            }
            char[] schar = s.toCharArray();
            char[] tchar = t.toCharArray();
            Map<Character, Character> map = new HashMap();
            if (s.length() != t.length()) {
                return false;
            }
            for (int i = 0; i < s.length(); i++) {
                if (map.containsKey(schar[i])) {
                    if (map.get(schar[i]) != tchar[i]) {
                        return false;
                    }
                } else {
                    if (map.containsValue(tchar[i])) {
                        return false;//this line is necessary for this case: ("ab", "aa")
                    }
                    map.put(schar[i], tchar[i]);
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 190: _206.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 206. Reverse Linked List
 *
 * Reverse a singly linked list.
 *
 * Example:
 * Input: 1->2->3->4->5->NULL
 * Output: 5->4->3->2->1->NULL
 *
 * Follow up:
 * A linked list can be reversed either iteratively or recursively. Could you implement
both?
 * */
public class _206 {

    public static class Solution1 {
        public ListNode reverseList(ListNode head) {
            ListNode prev = null;
            ListNode curr = head;
            while (curr != null) {
                ListNode next = curr.next;
                curr.next = prev;
                prev = curr;
                curr = next;
            }
            return prev;
        }
    }

    public static class Solution2 {
        public ListNode reverseList(ListNode head) {
            return reverse(head, null);
        }

        ListNode reverse(ListNode head, ListNode newHead) {
            if (head == null) {
                return newHead;
            }
            ListNode next = head.next;
            head.next = newHead;
            newHead = head;
            head = next;
            return reverse(head, newHead);
        }
    }

}

// ============================================================
// FILE 191: _207.java
```

```
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;

/**
 * 207. Course Schedule
 *
 * There are a total of n courses you have to take, labeled from 0 to n - 1.
 Some courses may have prerequisites, for example to take course 0 you have to first take
course 1,
 which is expressed as a pair: [0,1]
 Given the total number of courses and a list of prerequisite pairs, is it possible for
you to finish all courses?

 For example:
 2, [[1,0]]
 There are a total of 2 courses to take.
 To take course 1 you should have finished course 0. So it is possible.

 2, [[1,0],[0,1]]
 There are a total of 2 courses to take.
 To take course 1 you should have finished course 0,
 and to take course 0 you should also have finished course 1. So it is impossible.

 Note:
 The input prerequisites is a graph represented by a list of edges, not adjacency
matrices. Read more about how a graph is represented.
 You may assume that there are no duplicate edges in the input prerequisites.
 click to show more hints.

 Hints:
 This problem is equivalent to finding if a cycle exists in a directed graph. If a cycle
exists, no topological ordering exists and therefore it will be impossible to take all
courses.
 Topological Sort via DFS - A great video tutorial (21 minutes) on Coursera explaining
the basic concepts of Topological Sort.
 Topological sort could also be done via BFS.
 */
public class _207 {

    public static class Solution1 {
        public boolean canFinish(int numCourses, int[][] prerequisites) {
            int[] indegree = new int[numCourses];
            for (int[] prereq : prerequisites) {
                indegree[prereq[0]]++;
            }
            Set<Integer> zeroDegree = new HashSet();
            for (int i = 0; i < numCourses; i++) {
                if (indegree[i] == 0) {
                    zeroDegree.add(i);
```

```
                }
            }
            if (zeroDegree.isEmpty()) {
                return false;
            }

            while (!zeroDegree.isEmpty()) {
                Iterator<Integer> it = zeroDegree.iterator();
                int course = it.next();
                zeroDegree.remove(course);
                for (int[] prereq : prerequisites) {
                    if (prereq[1] == course) {
                        indegree[prereq[0]]--;
                        if (indegree[prereq[0]] == 0) {
                            zeroDegree.add(prereq[0]);
                        }
                    }
                }
            }

            for (int i : indegree) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 192: _208.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 208. Implement Trie (Prefix Tree)
 *
 * Implement a trie with insert, search, and startsWith methods.

 Note:
 You may assume that all inputs are consist of lowercase letters a-z.
 */

public class _208 {
    public static class Solution1 {
        class TrieNode {

            char val;
            boolean isWord;
            TrieNode[] children = new TrieNode[26];

            // Initialize your data structure here.
```

```
            public TrieNode() {
            }

            public TrieNode(char c) {
                this.val = c;
            }
        }

        public class Trie {
            private TrieNode root;

            public Trie() {
                root = new TrieNode();
                root.val = ' ';//initialize root to be an empty char, this is a common
practice as how Wiki defines Trie data structure as well
            }

            // Inserts a word into the trie.
            public void insert(String word) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                    if (node.children[word.charAt(i) - 'a'] == null) {
                        node.children[word.charAt(i) - 'a'] = new
TrieNode(word.charAt(i));
                    }
                    node = node.children[word.charAt(i) - 'a'];
                }
                node.isWord = true;
            }

            // Returns if the word is in the trie.
            public boolean search(String word) {
                TrieNode node = root;
                for (int i = 0; i < word.length(); i++) {
                    if (node.children[word.charAt(i) - 'a'] == null) {
                        return false;
                    }
                    node = node.children[word.charAt(i) - 'a'];
                }
                return node.isWord;
            }

            // Returns if there is any word in the trie
            // that starts with the given prefix.
            public boolean startsWith(String prefix) {
                TrieNode node = root;
                for (int i = 0; i < prefix.length(); i++) {
                    if (node.children[prefix.charAt(i) - 'a'] == null) {
                        return false;
                    }
                    node = node.children[prefix.charAt(i) - 'a'];
                }
                return true;
            }
```

```
        }

        // Your Trie object will be instantiated and called as such:
        // Trie trie = new Trie();
        // trie.insert("somestring");
        // trie.search("key");
    }
}

// ============================================================
// FILE 193: _209.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 209. Minimum Size Subarray Sum
 *
 * Given an array of n positive integers and a positive integer s,
 * find the minimal length of a contiguous subarray of which the sum ≥ s.
 * If there isn't one, return 0 instead.

 For example, given the array [2,3,1,2,4,3] and s = 7,
 the subarray [4,3] has the minimal length under the problem constraint.

 click to show more practice.

 More practice:
 If you have figured out the O(n) solution,
 try coding another solution of which the time complexity is O(n log n).
 */
public class _209 {

    public static class Solution1 {
        public int minSubArrayLen(int s, int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            int i = 0;
            int j = 0;
            int min = Integer.MAX_VALUE;
            int sum = 0;
            while (j < nums.length) {
                sum += nums[j++];

                while (sum >= s) {
                    min = Math.min(min, j - i);
                    sum -= nums[i++];
                }
            }
            return min == Integer.MAX_VALUE ? 0 : min;
        }
    }
}
```

```
// ============================================================
// FILE 194: _210.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 210. Course Schedule II
 *
 * There are a total of n courses you have to take, labeled from 0 to n - 1.
 Some courses may have prerequisites, for example to take course 0 you have to first take
course 1,
 which is expressed as a pair: [0,1]
 Given the total number of courses and a list of prerequisite pairs,
 return the ordering of courses you should take to finish all courses.

 There may be multiple correct orders,
 you just need to return one of them.
 If it is impossible to finish all courses, return an empty array.

 For example:

 2, [[1,0]]
 There are a total of 2 courses to take.
 To take course 1 you should have finished course 0. So the correct course order is [0,1]

 4, [[1,0],[2,0],[3,1],[3,2]]
 There are a total of 4 courses to take.
 To take course 3 you should have finished both courses 1 and 2.
 Both courses 1 and 2 should be taken after you finished course 0.
 So one correct course order is [0,1,2,3]. Another correct ordering is[0,2,1,3].

 Note:
 The input prerequisites is a graph represented by a list of edges, not adjacency
matrices. Read more about how a graph is represented.
 You may assume that there are no duplicate edges in the input prerequisites.
 click to show more hints.

 Hints:
 This problem is equivalent to finding the topological order in a directed graph. If a
cycle exists, no topological ordering exists and therefore it will be impossible to take
all courses.
 Topological Sort via DFS - A great video tutorial (21 minutes) on Coursera explaining
the basic concepts of Topological Sort.
 Topological sort could also be done via BFS.
 */
public class _210 {
```

```
    public static class Solution1 {
        public int[] findOrder(int numCourses, int[][] prerequisites) {
            int[] inDegree = new int[numCourses];
            for (int[] course : prerequisites) {
                inDegree[course[0]]++;
            }

            Set<Integer> zeroDegree = new HashSet();
            Queue<Integer> queue = new LinkedList();
            for (int i = 0; i < numCourses; i++) {
                if (inDegree[i] == 0) {
                    zeroDegree.add(i);
                    queue.offer(i);
                }
            }

            if (zeroDegree.isEmpty()) {
                return new int[0];
            }

            while (!zeroDegree.isEmpty()) {
                Iterator<Integer> it = zeroDegree.iterator();
                int course = it.next();
                zeroDegree.remove(course);
                for (int[] pre : prerequisites) {
                    if (course == pre[1]) {
                        inDegree[pre[0]]--;
                        if (inDegree[pre[0]] == 0) {
                            zeroDegree.add(pre[0]);
                            queue.offer(pre[0]);
                        }
                    }
                }
            }

            for (int i = 0; i < numCourses; i++) {
                if (inDegree[i] != 0) {
                    return new int[0];
                }
            }

            int[] result = new int[queue.size()];
            int i = 0;
            while (!queue.isEmpty()) {
                result[i++] = queue.poll();
            }
            return result;
        }
    }
}

// ============================================================
// FILE 195: _211.java
```

```
// ============================================================

package com.fishercoder.solutions;

/**
 * 211. Add and Search Word - Data structure design
 *
 * Design a data structure that supports the following two operations:

 void addWord(word)
 bool search(word)
 search(word) can search a literal word or a regular expression string containing only
letters a-z or .. A . means it can represent any one letter.

 For example:

 addWord("bad")
 addWord("dad")
 addWord("mad")
 search("pad") -> false
 search("bad") -> true
 search(".ad") -> true
 search("b..") -> true
 Note:
 You may assume that all words are consist of lowercase letters a-z.

 You should be familiar with how a Trie works. If not, please work on this problem:
Implement Trie (Prefix Tree) first.
 */
public class _211 {
  public static class Solution1 {
    public static class WordDictionary {
      WordNode root;

      /** Initialize your data structure here. */
      public WordDictionary() {
        root = new WordNode();
      }

      public void addWord(String word) {
        char[] chars = word.toCharArray();
        addWord(chars, 0, root);
      }

      private void addWord(char[] chars, int index, WordNode parent) {
        char c = chars[index];
        int idx = c - 'a';
        WordNode node = parent.children[idx];
        if (node == null) {
          node = new WordNode();
          parent.children[idx] = node;
        }
        if (chars.length == index + 1) {
          node.isLeaf = true;
```

```
          return;
        }
        addWord(chars, ++index, node);
      }

      public boolean search(String word) {
        return search(word.toCharArray(), 0, root);
      }

      /** This is also a beautifully designed recursive function. */
      private boolean search(char[] chars, int index, WordNode parent) {
        if (index == chars.length) {
          if (parent.isLeaf) {
            return true;
          }
          return false;
        }
        WordNode[] childNodes = parent.children;
        char c = chars[index];
        if (c == '.') {
          for (int i = 0; i < childNodes.length; i++) {
            WordNode n = childNodes[i];
            if (n != null) {
              boolean b = search(chars, index + 1, n);
              if (b) {
                return true;
              }
            }
          }
          return false;
        }
        WordNode node = childNodes[c - 'a'];
        if (node == null) {
          return false;
        }
        return search(chars, ++index, node);
      }

      /** This is a cool/standard design for a Trie node class. */
      private class WordNode {
        boolean isLeaf;
        WordNode[] children = new WordNode[26];
      }
    }

    /**
     * Your WordDictionary object will be instantiated and called as such:
     * WordDictionary obj = new WordDictionary();
     * obj.addWord(word);
     * boolean param_2 = obj.search(word);
     */
  }
}
```

```
// ============================================================
// FILE 196: _212.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 212. Word Search II
 * Given a 2D board and a list of words from the dictionary, find all words in the board.

 Each word must be constructed from letters of sequentially adjacent cell, where
"adjacent" cells are those horizontally or vertically neighboring. The same letter cell
may not be used more than once in a word.

 For example,
 Given words = ["oath","pea","eat","rain"] and board =

 [
 ['o','a','a','n'],
 ['e','t','a','e'],
 ['i','h','k','r'],
 ['i','f','l','v']
 ]
 Return ["eat","oath"].
 Note:
 You may assume that all inputs are consist of lowercase letters a-z.

 You would need to optimize your backtracking to pass the larger test. Could you stop
backtracking earlier?

 If the current candidate does not exist in all words' prefix, you could stop
backtracking immediately.
 What kind of data structure could answer such query efficiently?
 Does a hash table work? Why or why not? How about a Trie?
 If you would like to learn how to implement a basic trie, please work on this problem:
Implement Trie (Prefix Tree) first.*/
public class _212 {

    public static class Solution1 {
        public List<String> findWords(char[][] board, String[] words) {
            TrieNode root = buildTrie(words);
            List<String> result = new ArrayList();
            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    dfs(root, board, i, j, result);
                }
            }
            return result;
        }

        private void dfs(TrieNode root, char[][] board, int i, int j, List<String>
```

```
result) {
            char c = board[i][j];

            if (c == '#' || root.children[c - 'a'] == null) {
                return;
            }

            if (root.children[c - 'a'].word != null) {
                result.add(root.children[c - 'a'].word);
                root.children[c - 'a'].word = null;//de-duplicate
            }
            board[i][j] = '#';//mark it as visited to avoid cycles
            if (i > 0) {
                dfs(root.children[c - 'a'], board, i - 1, j, result);
            }
            if (j > 0) {
                dfs(root.children[c - 'a'], board, i, j - 1, result);
            }
            if (i + 1 < board.length) {
                dfs(root.children[c - 'a'], board, i + 1, j, result);
            }
            if (j + 1 < board[0].length) {
                dfs(root.children[c - 'a'], board, i, j + 1, result);
            }

            board[i][j] = c;
        }

        private TrieNode root;

        class TrieNode {
            String word;
            TrieNode[] children = new TrieNode[26];
        }

        private TrieNode buildTrie(String[] words) {
            TrieNode root = new TrieNode();
            for (String word : words) {
                char[] chars = word.toCharArray();
                TrieNode temp = root;
                for (char c : chars) {
                    if (temp.children[c - 'a'] == null) {
                        temp.children[c - 'a'] = new TrieNode();
                    }
                    temp = temp.children[c - 'a'];
                }
                temp.word = word;
            }
            return root;
        }
    }
}

// ==========================================================
```

```
// FILE 197: _213.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 213. House Robber II

 Note: This is an extension of House Robber.

 After robbing those houses on that street,
 the thief has found himself a new place for his thievery
 so that he will not get too much attention.
 This time, all houses at this place are arranged in a circle.
 That means the first house is the neighbor of the last one.
 Meanwhile, the security system for these houses remain the same as for those in the
previous street.

Given a list of non-negative integers representing the amount of money of each house,
 determine the maximum amount of money you can rob tonight without alerting the police.
*/
public class _213 {
    public static class Solution1 {
        /**
         * Another dp problem:
         * separate them into two cases:
         * 1. rob from house 1 to n-1, get max1
         * 2. rob from house 2 to n, get max2 take the max from the above two max
         */
        public int rob(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            if (nums.length == 1) {
                return nums[0];
            }
            if (nums.length == 2) {
                return Math.max(nums[0], nums[1]);
            }
            int[] dp = new int[nums.length - 1];

            //rob 1 to n-1
            dp[0] = nums[0];
            dp[1] = Math.max(nums[0], nums[1]);
            for (int i = 2; i < nums.length - 1; i++) {
                dp[i] = Math.max(dp[i - 2] + nums[i], dp[i - 1]);
            }
            int prevMax = dp[nums.length - 2];

            //rob 2 to n
            dp = new int[nums.length - 1];
            dp[0] = nums[1];
            dp[1] = Math.max(nums[1], nums[2]);
            for (int i = 3; i < nums.length; i++) {
                dp[i - 1] = Math.max(dp[i - 3] + nums[i], dp[i - 2]);
```

```
            }
            return Math.max(prevMax, dp[nums.length - 2]);
        }
    }
}

// ============================================================
// FILE 198: _214.java
// ============================================================

package com.fishercoder.solutions;

/**
 214. Shortest Palindrome

 Given a string S, you are allowed to convert it to a palindrome
 by adding characters in front of it.
 Find and return the shortest palindrome you can find by performing this transformation.

 For example:

 Given "aacecaaa", return "aaacecaaa".

 Given "abcd", return "dcbabcd".
 */
public class _214 {

    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/27261/clean-kmp-solution-with-
super-detailed-explanation*/
        /**
         * TODO: read it explanation and understand KMP completely.
         */
        public String shortestPalindrome(String s) {
            String temp = s + "#" + new StringBuilder(s).reverse().toString();
            int[] table = getTable(temp);
            //get the maximum palin part in s starts from 0
            return new StringBuilder(s.substring(table[table.length -
1])).reverse().toString() + s;
        }

        public int[] getTable(String s) {
            //get lookup table
            int[] table = new int[s.length()];

            //pointer that points to matched char in prefix part
            int index = 0;
            //skip index 0, we will not match a string with itself
            for (int i = 1; i < s.length(); i++) {
                if (s.charAt(index) == s.charAt(i)) {
                    //we can extend match in prefix and postfix
                    table[i] = table[i - 1] + 1;
                    index++;
                } else {
```

```
                    //match failed, we try to match a shorter substring

                    //by assigning index to table[i-1], we will shorten the match string
length, and jump to the
                    //prefix part that we used to match postfix ended at i - 1
                    index = table[i - 1];

                    while (index > 0 && s.charAt(index) != s.charAt(i)) {
                        //we will try to shorten the match string length until we revert
to the beginning of match (index 1)
                        index = table[index - 1];
                    }

                    //when we are here may either found a match char or we reach the
boundary and still no luck
                    //so we need check char match
                    if (s.charAt(index) == s.charAt(i)) {
                        //if match, then extend one char
                        index++;
                    }
                    table[i] = index;
                }
            }
            return table;
        }
    }
}

// ============================================================
// FILE 199: _215.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Collections;
import java.util.PriorityQueue;

/**
 * 215. Kth Largest Element in an Array
 *
 * Find the kth largest element in an unsorted array. Note that it is the kth largest
element in the sorted order, not the kth distinct element.

 For example,
 Given [3,2,1,5,6,4] and k = 2, return 5.

 Note:
 You may assume k is always valid, 1 ≤ k ≤ array's length.

 */
public class _215 {

    public static class Solution1 {
```

```
        public int findKthLargest(int[] nums, int k) {
            Arrays.sort(nums);
            return nums[nums.length - k];
        }
    }

    public static class Solution2 {
        public int findKthLargest(int[] nums, int k) {
            PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
            for (int i : nums) {
                maxHeap.offer(i);
            }
            while (k-- > 1) {
                maxHeap.poll();
            }
            return maxHeap.poll();
        }
    }

    public static class Solution3 {
        /**Quick Select algorithm
         * Time: O(n) in average, O(n^2) in worst case
         *
         * Reference: https://discuss.leetcode.com/topic/14611/java-quick-select*/
        public int findKthLargest(int[] nums, int k) {
            int start = 0;
            int end = nums.length - 1;
            int index = nums.length - k;
            while (start < end) {
                int pivot = partition(nums, start, end);
                if (pivot < index) {
                    start = pivot + 1;
                } else if (pivot > index) {
                    end = pivot - 1;
                } else {
                    return nums[pivot];
                }
            }
            return nums[start];
        }

        int partition(int[] nums, int start, int end) {
            int pivot = start;
            while (start <= end) {
                while (start <= end && nums[start] <= nums[pivot]) {
                    start++;
                }
                while (start <= end && nums[end] > nums[pivot]) {
                    end--;
                }
                if (start > end) {
                    break;
                }
```

```
                swap(nums, start, end);
            }
            swap(nums, end, pivot);
            return end;
        }

        void swap(int[] nums, int i, int j) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }
    }
}

// ============================================================
// FILE 200: _216.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 216. Combination Sum III
 *
 * Find all possible combinations of k numbers that add up to a number n,
 * given that only numbers from 1 to 9 can be used and each combination should be a
unique set of numbers.

 Example 1:
 Input: k = 3, n = 7
 Output: [[1,2,4]]

 Example 2:
 Input: k = 3, n = 9
 Output: [[1,2,6], [1,3,5], [2,3,4]]
 */
public class _216 {

    public static class Solution1 {
        public List<List<Integer>> combinationSum3(int k, int n) {
            List<List<Integer>> result = new ArrayList();
            int[] nums = new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9};
            backtracking(k, n, nums, 0, new ArrayList(), result);
            return result;
        }

        void backtracking(int k, int n, int[] nums, int start, List<Integer> curr,
List<List<Integer>> result) {
            if (n > 0) {
                for (int i = start; i < nums.length; i++) {
                    curr.add(nums[i]);
                    backtracking(k, n - nums[i], nums, i + 1, curr, result);
```

```
                    curr.remove(curr.size() - 1);
                }
            } else if (n == 0 && curr.size() == k) {
                result.add(new ArrayList(curr));
            }
        }
    }
}

// ============================================================
// FILE 201: _217.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 217. Contains Duplicate
 *
 * Given an array of integers, find if the array contains any
 * duplicates. Your function should return true if any value appears at least twice in
the array,
 * and it should return false if every element is distinct.
 */
public class _217 {
    public static class Solution1 {
        public boolean containsDuplicate(int[] nums) {
            if (nums == null || nums.length == 0) {
                return false;
            }
            Set<Integer> set = new HashSet();
            for (int i : nums) {
                if (!set.add(i)) {
                    return true;
                }
            }
            return false;
        }
    }

}

// ============================================================
// FILE 202: _218.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.TreeMap;
```

```
/**
 * 218. The Skyline Problem
 *
 * A city's skyline is the outer contour of the silhouette formed by all the buildings in
that city when viewed from a distance.
 * Now suppose you are given the locations and height of all the buildings as shown on a
cityscape photo (Figure A),
 * write a program to output the skyline formed by these buildings collectively (Figure
B).
 *
 * The geometric information of each building is represented by a triplet of integers
[Li, Ri, Hi],
 * where Li and Ri are the x coordinates of the left and right edge of the ith building,
respectively,
 * and Hi is its height. It is guaranteed that 0 ≤ Li, Ri ≤ INT_MAX, 0 < Hi ≤ INT_MAX,
and Ri - Li > 0.
 * You may assume all buildings are perfect rectangles grounded on an absolutely flat
surface at height 0.
 *
 * For instance, the dimensions of all buildings in Figure A are recorded as: [ [2 9 10],
[3 7 15], [5 12 12], [15 20 10], [19 24 8] ] .
 * The output is a list of "key points" (red dots in Figure B) in the format of [
[x1,y1], [x2, y2], [x3, y3], ... ]
 * that uniquely defines a skyline.
 * A key point is the left endpoint of a horizontal line segment.
 * Note that the last key point, where the rightmost building ends,
 * is merely used to mark the termination of the skyline, and always has zero height.
 * Also, the ground in between any two adjacent buildings should be considered part of
the skyline contour.

 For instance, the skyline in Figure B should be represented as:[ [2 10], [3 15], [7 12],
[12 0], [15 10], [20 8], [24, 0] ].

 Notes:

 The number of buildings in any input list is guaranteed to be in the range [0, 10000].
 The input list is already sorted in ascending order by the left x position Li.
 The output list must be sorted by the x position.
 There must be no consecutive horizontal lines of equal height in the output skyline. For
instance, [...[2 3], [4 5], [7 5], [11 5], [12 7]...] is not acceptable; the three lines
of height 5 should be merged into one in the final output as such: [...[2 3], [4 5], [12
7], ...]*/

/**This video is super clear and helpful: https://www.youtube.com/watch?v=GSBLe8cKu0s

 Algorithm:
First observation: all the points in the final result come from the four angles that each
building has
Scan through the horizontal lines
Use a PriorityQueue to hold each building, and make the PriorityQueue to sort on the
height of the buildings
whenever we encounter the start of a building, we push it into the PriorityQueue,
whenever we finished scanning that building, we remove it from the PriorityQueue
Also, in the scan process, we'll keep updating the maxHeight in the PriorityQueue if we
```

```
find a new maxHeight which means the building will be overshadowed by the new higher one

Three edge cases (see the graph illustration in the above video at 12’18”):
when two buildings have the same start point, the one with higher height shows up in the final result
when two buildings have the same end point, the one with higher height shows up in the final result
when the start point of one building is is also the end point of another building, the one with higher height shows up in the final result

 We use TreeMap over a normal PriorityQueue:
For the sake of efficiency (better time complexity), we’ll use TreeMap which supports O(logn) for remove() operation,
 this is the reason we choose TreeMap over a normal PriorityQueue in Java (PriorityQueue supports add() and getMaxVal() in both O(logn) time, however, for remove(), it does NOT.)
But TreeMap in Java supports all the three operations in O(logn) time.*/

public class _218 {

    public static class Solution1 {

        class BuildingPoint implements Comparable<BuildingPoint> {
            int x;
            boolean isStart;
            int h;

            public BuildingPoint(int x, boolean isStart, int h) {
                this.x = x;
                this.h = h;
                this.isStart = isStart;
            }

            @Override
            public int compareTo(BuildingPoint o) {
                if (this.x != o.x) {
                    return this.x - o.x;
                } else {
                    if (this.isStart && o.isStart) {
                        return o.h - this.h;
                    } else if (this.isStart && !o.isStart) {
                        return -this.h - o.h;
                    } else if (!this.isStart && !o.isStart) {
                        return this.h - o.h;
                    } else {
                        return this.h + o.h;
                    }
                }
            }
        }

        public List<int[]> getSkyline(int[][] buildings) {
            BuildingPoint[] bps = new BuildingPoint[buildings.length * 2];
            int index = 0;
            for (int[] building : buildings) {
```

```
                BuildingPoint bp1 = new BuildingPoint(building[0], true, building[2]);
                BuildingPoint bp2 = new BuildingPoint(building[1], false, building[2]);
                bps[index++] = bp1;
                bps[index++] = bp2;
            }

            //this is one key step:
            Arrays.sort(bps);

            List<int[]> result = new ArrayList();
            TreeMap<Integer, Integer> treeMap = new TreeMap();
            treeMap.put(0, 1);
            int prevMaxH = 0;
            for (BuildingPoint bp : bps) {
                //if it's a starting point, we'll add it into the final result
                if (bp.isStart) {
                    if (treeMap.containsKey(bp.h)) {
                        treeMap.put(bp.h, treeMap.get(bp.h) + 1);
                    } else {
                        treeMap.put(bp.h, 1);
                    }
                } else if (!bp.isStart) {
                    //if it's an ending point, we'll decrement/remove this entry
                    if (treeMap.containsKey(bp.h) && treeMap.get(bp.h) > 1) {
                        treeMap.put(bp.h, treeMap.get(bp.h) - 1);
                    } else {
                        treeMap.remove(bp.h);
                    }
                }

                int currMaxH = treeMap.lastKey();
                if (currMaxH != prevMaxH) {
                    result.add(new int[]{bp.x, currMaxH});
                    prevMaxH = currMaxH;
                }

            }

            return result;
        }
    }
}

// ============================================================
// FILE 203: _219.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;
```

```java
/**
 * 219. Contains Duplicate II
 Given an array of integers and an integer k,
 find out whether there are two distinct indices i and j in the array
 such that nums[i] = nums[j] and the absolute difference between i and j is at most k.

 Example 1:
 Input: nums = [1,2,3,1], k = 3
 Output: true

 Example 2:
 Input: nums = [1,0,1,1], k = 1
 Output: true

 Example 3:
 Input: nums = [1,2,3,1,2,3], k = 2
 Output: false
 */
public class _219 {

    public static class Solution1 {
        public boolean containsNearbyDuplicate(int[] nums, int k) {
            Map<Integer, Integer> map = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(nums[i])) {
                    if (i - map.get(nums[i]) <= k) {
                        return true;
                    } else {
                        map.put(nums[i], i);
                    }
                } else {
                    map.put(nums[i], i);
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        public boolean containsNearbyDuplicate(int[] nums, int k) {
            Set<Integer> set = new HashSet<>();
            for (int i = 0; i < nums.length; i++) {
                if (!set.add(nums[i])) {
                    return true;
                }
                if (set.size() > k) {
                    set.remove(nums[i - k]);
                }
            }
            return false;
        }
    }
}
```

```java
// ============================================================
// FILE 204: _220.java
// ============================================================

package com.fishercoder.solutions;

import java.util.TreeSet;

/**
 * 220. Contains Duplicate III
 *
 * Given an array of integers, find out whether there are two
 * distinct indices i and j in the array such that the difference between nums[i] and
nums[j] is at
 * most t and the difference between i and j is at most k.
 */
public class _220 {

    public static class Solution1 {
        /**
         * TreeSet: per Java doc, is a NavigableSet implementation based on a TreeMap.
The elements are ordered
         * using their natural ordering, or by a Comparator provided at set creation
time, depending on
         * which constructor is used. This implementation provides guaranteed log(n) time
cost for the
         * basic operations (add, remove and contains).
         */

        /**
         * TreeSet turns out to be a super handy data structure for this problem. It
implements Set
         * interface and keeps elements in sorted order, plus it has two very handy APIs:
         * https://docs.oracle.com/javase/7/docs/api/java/util/TreeSet.html#ceiling(E):
Returns the
         * least element in this set greater than or equal to the given element, or null
if there is no
         * such element.
         * https://docs.oracle.com/javase/7/docs/api/java/util/TreeSet.html#floor(E):
Returns the
         * greatest element in this set less than or equal to the given element, or null
if there is no
         * such element.
         *
         * Idea: loop through this array, keep adding each element into the TreeSet, also
keep an eye on
         * the size of the TreeSet, if it's greater than the required distance of k, then
we remove the
         * left-most or oldest one from the set. For each element, we get the current
floor and the
         * current ceiling and see if it works, if it does, then we return true
immediately, otherwise,
         * we continue. Return false when we finished the loop.
         */
```

```
        public boolean containsNearbyAlmostDuplicate(int[] nums, int k, int t) {
            /**case to Long to avoid Integer overflow.*/
            TreeSet<Long> set = new TreeSet<>();
            for (int i = 0; i < nums.length; ++i) {
                Long s = set.ceiling((long) nums[i]);
                if (s != null && s - nums[i] <= t) {
                    return true;
                }

                Long g = set.floor((long) nums[i]);
                if (g != null && nums[i] - g <= t) {
                    return true;
                }

                set.add((long) nums[i]);
                if (set.size() > k) {
                    set.remove((long) nums[i - k]);
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 205: _221.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 221. Maximal Square
 *
 * Given a 2D binary matrix filled with 0's and 1's, find the largest square containing
only 1's and return its area.

 For example, given the following matrix:

 1 0 1 0 0
 1 0 1 1 1
 1 1 1 1 1
 1 0 0 1 0
 Return 4.
 */
public class _221 {

    public static class Solution1 {
        /**
         * The idea is pretty straightforward: use a 2d dp table to store the
intermediate results
         */
        public int maximalSquare(char[][] matrix) {
            if (matrix == null || matrix.length == 0) {
```

```
                return 0;
            }
            int m = matrix.length;
            int n = matrix[0].length;
            int max = Integer.MIN_VALUE;
            int[][] dp = new int[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (i == 0 || j == 0) {
                        dp[i][j] = (matrix[i][j] == '1') ? 1 : 0;
                    } else {
                        if (matrix[i][j] == '0') {
                            dp[i][j] = 0;
                        } else {
                            dp[i][j] = Math.min(dp[i - 1][j], Math.min(dp[i][j - 1], dp[i
- 1][j - 1])) + 1;
                        }
                    }
                    max = (max < dp[i][j]) ? dp[i][j] : max;
                }
            }
            return max * max;
        }
    }
}

// ============================================================
// FILE 206: _222.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 222. Count Complete Tree Nodes
 * Given a complete binary tree, count the number of nodes.
 * Definition of a complete binary tree from Wikipedia:
 * In a complete binary tree every level,
 * except possibly the last, is completely filled,
 * and all nodes in the last level are as far left as possible.
 * It can have between 1 and 2h nodes inclusive at the last level h.
 */
public class _222 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/21317/accepted-easy-understand-
java-solution/2
         */
        public int countNodes(TreeNode root) {
            int leftH = getLeftHeight(root);
            int rightH = getRightHeight(root);
            if (leftH == rightH) {
```

```
                return (1 << leftH) - 1;
            } else {
                return 1 + countNodes(root.left) + countNodes(root.right);
            }
        }

        private int getRightHeight(TreeNode root) {
            int height = 0;
            while (root != null) {
                root = root.right;
                height++;
            }
            return height;
        }

        private int getLeftHeight(TreeNode root) {
            int height = 0;
            while (root != null) {
                root = root.left;
                height++;
            }
            return height;
        }
    }

}

// ============================================================
// FILE 207: _223.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 223. Rectangle Area
 *
 * Find the total area covered by two rectilinear rectangles in a 2D plane.
 * Each rectangle is defined by its bottom left corner and top right corner as shown in
the figure.
 * Rectangle Area

 Example:

 Input: A = -3, B = 0, C = 3, D = 4, E = 0, F = -1, G = 9, H = 2
 Output: 45

 Note: Assume that the total area is never beyond the maximum possible value of int.*/
public class _223 {

    public static class Solution1 {
        public int computeArea(int A, int B, int C, int D, int E, int F, int G, int H) {
            int areaA = (C - A) * (D - B);
            int areaB = (G - E) * (H - F);
```

```
            int top = Math.min(D, H);
            int bottom = Math.max(B, F);
            int left = Math.max(A, E);
            int right = Math.min(C, G);

            int overlap = 0;
            if (top > bottom && right > left) {
                overlap = (top - bottom) * (right - left);
            }
            return areaA + areaB - overlap;
        }
    }
}

// ============================================================
// FILE 208: _224.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

/**
 * 224. Basic Calculator
 *
 * Implement a basic calculator to evaluate a simple expression string.
 * The expression string may contain open ( and closing parentheses ), the plus + or
minus sign -, non-negative integers and empty spaces .
 * You may assume that the given expression is always valid.

 Some examples:
 "1 + 1" = 2
 " 2-1 + 2 " = 3
 "(1+(4+5+2)-3)+(6+8)" = 23
 Note: Do not use the eval built-in library function.
 */
public class _224 {

    public static class Solution1 {

        public int calculate(String s) {
            if (s == null || s.isEmpty()) {
                return 0;
            }

            s = s.replaceAll("\\s", "");
            char[] chars = s.toCharArray();
            List<String> filteredStr = new ArrayList();
            for (int i = 0; i < chars.length; ) {
                StringBuilder sb = new StringBuilder();
                while (i < chars.length && Character.isDigit(chars[i])) {
                    sb.append(chars[i]);
```

```
                    i++;
                }
                if (i == chars.length) {
                    if (sb.toString().length() != 0) {
                        filteredStr.add(sb.toString());
                    }
                } else {
                    if (sb.toString().length() != 0) {
                        filteredStr.add(sb.toString());
                    }
                    if (chars[i] == '+' || chars[i] == '-' || chars[i] == '(' || chars[i]
== ')') {
                        filteredStr.add(String.valueOf(chars[i]));
                    }
                    i++;
                }
            }

            for (String str : filteredStr) {
                System.out.print(str);
            }

            Stack<String> stack1 = new Stack();
            Stack<String> stack2 = new Stack();
            for (int i = 0; i < filteredStr.size(); ) {
                while (i < filteredStr.size() && !filteredStr.get(i).equals(")")) {
                    stack1.push(filteredStr.get(i));
                    i++;
                }
                if (i != filteredStr.size()) {
                    while (!stack1.isEmpty() && !stack1.peek().equals("(")) {
                        stack2.push(stack1.pop());
                    }
                    stack1.pop();
                    int exp = 0;
                    while (!stack2.isEmpty()) {
                        if (stack2.size() == 1) {
                            stack1.push(stack2.pop());
                            break;
                        }
                        int operand1 = Integer.parseInt(stack2.pop());
                        String operator = stack2.pop();
                        int operand2 = Integer.parseInt(stack2.pop());
                        if (operator.equals("+")) {
                            exp = operand1 + operand2;
                        } else if (operator.equals("-")) {
                            exp = operand1 - operand2;
                        }
                        stack2.push(String.valueOf(exp));
                    }
                    i++;
                }
            }
```

```
            if (stack1.size() == 1) {
                return Integer.parseInt(stack1.pop());
            }

            while (!stack1.isEmpty()) {
                stack2.push(stack1.pop());
            }
            while (!stack2.isEmpty()) {
                if (stack2.size() == 1) {
                    stack1.push(stack2.pop());
                    break;
                }
                int exp = 0;
                int operand1 = Integer.parseInt(stack2.pop());
                String operator = stack2.pop();
                int operand2 = Integer.parseInt(stack2.pop());
                if (operator.equals("+")) {
                    exp = operand1 + operand2;
                } else if (operator.equals("-")) {
                    exp = operand1 - operand2;
                }
                stack2.push(String.valueOf(exp));
            }
            return Integer.parseInt(stack1.pop());
        }
    }

}

// ============================================================
// FILE 209: _225.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;
/**
 * 225. Implement Stack using Queues
 *
 * Implement the following operations of a stack using queues.

 push(x) -- Push element x onto stack.
 pop() -- Removes the element on top of the stack.
 top() -- Get the top element.
 empty() -- Return whether the stack is empty.

 Notes:
 You must use only standard operations of a queue -- which means only push to back,
peek/pop from front, size, and is empty operations are valid.
 Depending on your language, queue may not be supported natively. You may simulate a
queue by using a list or deque (double-ended queue), as long as you use only standard
operations of a queue.
 You may assume that all operations are valid (for example, no pop or top operations will
```

```
be called on an empty stack).
 Update (2015-06-11):
 The class name of the Java function had been updated to MyStack instead of Stack.*/

public class _225 {

    public static class Solution1 {
        class MyStack {

            Queue<Integer> q = new LinkedList();

            // Push element x onto stack.
            public void push(int x) {
                q.offer(x);
                for (int i = 1; i < q.size(); i++) {
                    q.offer(q.remove());
                }
            }

            // Removes the element on top of the stack.
            public void pop() {
                q.poll();
            }

            // Get the top element.
            public int top() {
                return q.peek();
            }

            // Return whether the stack is empty.
            public boolean empty() {
                return q.isEmpty();
            }
        }
    }
}

// ==========================================================
// FILE 210: _226.java
// ==========================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**226. Invert Binary Tree

Invert a binary tree.

     4
   /   \
```

```
  2     7
 / \   / \
1   3 6   9

to

     4
   /   \
  7     2
 / \   / \
9   6 3   1

Trivia:
This problem was inspired by this original tweet by Max Howell:
Google: 90% of our engineers use the software you wrote (Homebrew),
 but you can t invert a binary tree on a whiteboard so fuck off.
 */
public class _226 {

        public static class Solution1 {
                public TreeNode invertTree(TreeNode root) {
                        if (root == null) {
                                return root;
                        }
                        Queue<TreeNode> q = new LinkedList();
                        q.offer(root);
                        while (!q.isEmpty()) {
                                TreeNode curr = q.poll();
                                TreeNode temp = curr.left;
                                curr.left = curr.right;
                                curr.right = temp;
                                if (curr.left != null) {
                                        q.offer(curr.left);
                                }
                                if (curr.right != null) {
                                        q.offer(curr.right);
                                }
                        }
                        return root;
                }
        }

        public static class Solution2 {
                public TreeNode invertTree(TreeNode root) {
                        if (root == null) {
                                return root;
                        }
                        TreeNode temp = root.left;
                        root.left = root.right;
                        root.right = temp;
                        invertTree(root.left);
                        invertTree(root.right);
                        return root;
                }
```

```
        }

        public static class Solution3 {
                //more concise version
                public TreeNode invertTree(TreeNode root) {
                        if (root == null) {
                                return root;
                        }
                        TreeNode temp = root.left;
                        root.left = invertTree(root.right);
                        root.right = invertTree(temp);
                        return root;
                }
        }
}

// ============================================================
// FILE 211: _227.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 227. Basic Calculator II
 *
 * Implement a basic calculator to evaluate a simple expression string.

 The expression string contains only non-negative integers, +, -, *, / operators and
empty spaces . The integer division should truncate toward zero.

 You may assume that the given expression is always valid.

 Some examples:
 "3+2*2" = 7
 " 3/2 " = 1
 " 3+5 / 2 " = 5
 Note: Do not use the eval built-in library function.
 */
public class _227 {

    public static class Solution1 {
        public int calculate(String s) {
            if (s == null || s.length() == 0) {
                return 0;
            }
            int len = s.length();
            Deque<Integer> stack = new ArrayDeque<>();
            int num = 0;
            char sign = '+';
            for (int i = 0; i < len; i++) {
                if (Character.isDigit(s.charAt(i))) {
```

```
                    num = num * 10 + s.charAt(i) - '0';
                }
                if ((!Character.isDigit(s.charAt(i))) && ' ' != s.charAt(i) || i == len -
1) {
                    if (sign == '+') {
                        stack.addLast(num);
                    } else if (sign == '-') {
                        stack.addLast(-num);
                    } else if (sign == '/') {
                        stack.addLast(stack.pollLast() / num);
                    } else if (sign == '*') {
                        stack.addLast(stack.pollLast() * num);
                    }
                    sign = s.charAt(i);
                    num = 0;
                }
            }
            int result = 0;
            while (!stack.isEmpty()) {
                result += stack.poll();
            }
            return result;
        }
    }
}

// ============================================================
// FILE 212: _228.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 228. Summary Ranges
 *
 * Given a sorted integer array without duplicates, return the summary of its ranges.
 *
 * For example, given [0,1,2,4,5,7], return ["0->2","4->5","7"].
 */
public class _228 {

    public static class Solution1 {
        public List<String> summaryRanges(int[] nums) {
            List<String> result = new ArrayList<>();
            for (int i = 0; i < nums.length; i++) {
                String start = String.valueOf(nums[i]);
                int tmpI = i;
                while ((i + 1) < nums.length && (nums[i] + 1) == nums[i + 1]) {
                    i++;
                }
                if (tmpI == i) {
```

```
                    result.add(start);
                } else {
                    result.add(start + "->" + String.valueOf(nums[i]));
                }
            }
            return result;
        }
    }

}

// ============================================================
// FILE 213: _229.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 229. Majority Element II
 *
 * Given an integer array of size n, find all elements that appear more than ⌊ n/3 ⌋
times.
 * The algorithm should run in linear time and in O(1) space.

 Hint:
 How many majority elements could it possibly have?
 Do you have a better hint? Suggest it!
 */
public class _229 {

    public static class Solution1 {
        public List<Integer> majorityElement(int[] nums) {
            Map<Integer, Integer> counterMap = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                if (counterMap.containsKey(nums[i])) {
                    counterMap.put(nums[i], counterMap.get(nums[i]) + 1);
                } else {
                    counterMap.put(nums[i], 1);
                }
            }
            int size = nums.length;
            List<Integer> result = new ArrayList<>();
            for (Integer i : counterMap.keySet()) {
                int threshold = size / 3;
                if (counterMap.get(i) > threshold) {
                    result.add(i);
                }
            }
            return result;
```

```
    }
}

public static class Solution2 {
    /**Moore Voting algorithm*/
    public List<Integer> majorityElement(int[] nums) {
        List<Integer> result = new ArrayList<>();
        if (nums == null || nums.length == 0) {
            return result;
        }
        int count1 = 0;
        int count2 = 0;
        int candidate1 = 0;
        int candidate2 = 1;
        for (int num : nums) {
            if (num == candidate1) {
                count1++;
            } else if (num == candidate2) {
                count2++;
            } else if (count1 == 0) {
                candidate1 = num;
                count1 = 1;
            } else if (count2 == 0) {
                candidate2 = num;
                count2 = 1;
            } else {
                count1--;
                count2--;
            }
        }
        count1 = 0;
        count2 = 0;
        for (int num : nums) {
            if (num == candidate1) {
                count1 += 2;
            } else {
                count1--;
            }
            if (num == candidate2) {
                count2 += 2;
            } else {
                count2--;
            }
        }
        if (count1 > 0) {
            result.add(candidate1);
        }
        if (count2 > 0) {
            result.add(candidate2);
        }
        return result;
    }
}
```

```
}

// ============================================================
// FILE 214: _230.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 230. Kth Smallest Element in a BST
 *
 * Given a binary search tree, write a function kthSmallest to find the kth smallest
element in it.

 Note:
 You may assume k is always valid, 1 ? k ? BST's total elements.

 Follow up:
 What if the BST is modified (insert/delete operations) often and you need to find the
kth smallest frequently? How would you optimize the kthSmallest routine?

 */
public class _230 {

    public static class Solution1 {
        public int kthSmallest(TreeNode root, int k) {
            List<Integer> inorderList = new ArrayList<>();
            inorder(root, inorderList);
            return inorderList.get(k - 1);
        }

        private void inorder(TreeNode root, List<Integer> inorderList) {
            if (root == null) {
                return;
            }
            if (root.left != null) {
                inorder(root.left, inorderList);
            }
            inorderList.add(root.val);
            if (root.right != null) {
                inorder(root.right, inorderList);
            }
            return;
        }
    }

    public static class Solution2 {
        /**
         * Inorder traversal gives the natural ordering of a BST, no need to sort.
```

```
         */
        int count = 0;
        int result = Integer.MIN_VALUE;

        public int kthSmallest(TreeNode root, int k) {
            inorder(root, k);
            return result;
        }

        private void inorder(TreeNode root, int k) {
            if (root == null) {
                return;
            }
            inorder(root.left, k);
            count++;
            if (count == k) {
                result = root.val;
                return;
            }
            inorder(root.right, k);
        }
    }

}

// ============================================================
// FILE 215: _231.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 231. Power of Two
 *
 * Given an integer, write a function to determine if it is a power of two.
 */

public class _231 {
    public static class Solution1 {
        public boolean isPowerOfTwo(int n) {
            //after writing out the binary representation of some numbers: 1,2,4,8,16,32,
you can easily figure out that
            //every number that is power of two has only one bit that is 1
            //then we can apply that cool trick that we learned from {@link easy._191}:
n&(n-1) which will clear the least significant bit in n to zero
            return n > 0 && (n & (n - 1)) == 0;
        }
    }
}

// ============================================================
// FILE 216: _232.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.Stack;
/**
 * 232. Implement Queue using Stacks
 *
 * Implement the following operations of a queue using stacks.

 push(x) -- Push element x to the back of queue.
 pop() -- Removes the element from in front of queue.
 peek() -- Get the front element.
 empty() -- Return whether the queue is empty.

 Notes:
 You must use only standard operations of a stack -- which means only push to top,
peek/pop from top, size, and is empty operations are valid.
 Depending on your language, stack may not be supported natively. You may simulate a
stack by using a list or deque (double-ended queue), as long as you use only standard
operations of a stack.
 You may assume that all operations are valid (for example, no pop or peek operations
will be called on an empty queue).
 */
public class _232 {

    public static class Solution1 {
        class MyQueue {

            Stack<Integer> input = new Stack();
            Stack<Integer> output = new Stack();

            // Push element x to the back of queue.
            public void push(int x) {
                input.push(x);
            }

            // Removes the element from in front of queue.
            public int pop() {
                peek();
                return output.pop();
            }

            // Get the front element.
            public int peek() {
                if (output.isEmpty()) {
                    while (!input.isEmpty()) {
                        output.push(input.pop());
                    }
                }
                return output.peek();
            }

            // Return whether the queue is empty.
            public boolean empty() {
                return input.isEmpty() && output.isEmpty();
```

```
            }
        }
    }
}


// ============================================================
// FILE 217: _233.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 233. Number of Digit One
 *
 * Given an integer n, count the total number of digit 1 appearing in all non-negative
integers less than or equal to n.

 For example:
 Given n = 13,
 Return 6, because digit 1 occurred in the following numbers: 1, 10, 11, 12, 13.

 Hint: Beware of overflow.
 */
public class _233 {
    public static class Solution1 {
        public int countDigitOne(int n) {
            int count = 0;
            for (long k = 1; k <= n; k *= 10) {
                long r = n / k;
                long m = n % k;
                // sum up the count of ones on every place k
                count += (r + 8) / 10 * k + (r % 10 == 1 ? m + 1 : 0);
            }
            return count;
        }
    }
}

// ============================================================
// FILE 218: _234.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 234. Palindrome Linked List
 *
 * Given a singly linked list, determine if it is a palindrome.
```

```
 Follow up:
 Could you do it in O(n) time and O(1) space?
 */

public class _234 {
    public static class Solution1 {
        /**O(n) time
         * O(1) space
         * */
        public boolean isPalindrome(ListNode head) {
            if (head == null) {
                return true;
            }

            ListNode slow = head;
            ListNode fast = head;
            while (fast.next != null && fast.next.next != null) {
                fast = fast.next.next;
                slow = slow.next;
            }

            ListNode secondHalfHead = reverse(slow.next);
            ListNode firstHalfHead = head;
            while (firstHalfHead != null && secondHalfHead != null) {
                if (firstHalfHead.val != secondHalfHead.val) {
                    return false;
                }
                firstHalfHead = firstHalfHead.next;
                secondHalfHead = secondHalfHead.next;
            }
            return true;
        }

        private ListNode reverse(ListNode head) {
            ListNode newHead = null;
            while (head != null) {
                ListNode next = head.next;
                head.next = newHead;
                newHead = head;
                head = next;
            }
            return newHead;
        }
    }

    public static class Solution2 {
        /**O(n) time
         * O(n) space
         * */
        public boolean isPalindrome(ListNode head) {
            int len = 0;
            ListNode fast = head;
            ListNode slow = head;
```

```
            List<Integer> firstHalf = new ArrayList<>();
            while (fast != null && fast.next != null) {
                fast = fast.next.next;
                firstHalf.add(slow.val);
                slow = slow.next;
                len += 2;
            }
            if (fast != null) {
                len++;
            }
            if (len % 2 != 0) {
                slow = slow.next;
            }
            int i = firstHalf.size() - 1;
            while (slow != null) {
                if (firstHalf.get(i--) != slow.val) {
                    return false;
                }
                slow = slow.next;
            }
            return true;
        }
    }

}

// ============================================================
// FILE 219: _235.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 235. Lowest Common Ancestor of a Binary Search Tree
 *
 * Given a binary search tree (BST), find the lowest common ancestor (LCA) of two given
nodes in the BST.
 * According to the definition of LCA on Wikipedia:
 * “The lowest common ancestor is defined between two nodes v and w as the lowest node in
T that has both v and w as descendants
 * (where we allow a node to be a descendant of itself).”

          _______6______
         /              \
      ___2__          ___8__
     /      \        /      \
     0      4        7       9
   /  \
  3   5

 For example, the lowest common ancestor (LCA) of nodes 2 and 8 is 6.
 Another example is LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself
```

```
according to the LCA definition.

 */
public class _235 {

    public static class Solution1 {
        public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
            if (root == null || p == root || q == root) {
                return root;
            }
            if ((root.val - p.val) * (root.val - q.val) > 0) {
                if (root.val - p.val > 0) {
                    return lowestCommonAncestor(root.left, p, q);
                }
                return lowestCommonAncestor(root.right, p, q);
            }
            return root;
        }
    }

}

// ============================================================
// FILE 220: _236.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 236. Lowest Common Ancestor of a Binary Tree
 *
 * Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the
tree.
 * According to the definition of LCA on Wikipedia:
 * “The lowest common ancestor is defined between two nodes v and w as the lowest node in
T
 * that has both v and w as descendants (where we allow a node to be a descendant of
itself).”

           _______3______
          /              \
      ___5__          ___1__
    /      \        /      \
   6      _2       0       8
  / \
 7   4

 For example, the lowest common ancestor (LCA) of nodes 5 and 1 is 3.
 Another example is LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself
according to the LCA definition.*/

public class _236 {
```

```
    public static class Solution1 {

        public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
            if (root == null || root == p || root == q) {
                return root;
            }
            TreeNode left = lowestCommonAncestor(root.left, p, q);
            TreeNode right = lowestCommonAncestor(root.right, p, q);
            if (left != null && right != null) {
                return root;
            }
            return left != null ? left : right;
        }
    }

}

// ============================================================
// FILE 221: _237.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 237. Delete Node in a Linked List
 *
 * Write a function to delete a node (except the tail) in a singly linked list, given
only access to that node.
 *
 * Given linked list -- head = [4,5,1,9], which looks like following:
 *
 * Example 1:
 * Input: head = [4,5,1,9], node = 5
 * Output: [4,1,9]
 * Explanation: You are given the second node with value 5, the linked list should become
4 -> 1 -> 9 after calling your function.
 *
 * Example 2:
 * Input: head = [4,5,1,9], node = 1
 * Output: [4,5,9]
 * Explanation: You are given the third node with value 1, the linked list should become
4 -> 5 -> 9 after calling your function.
 *
 * Note:
 * The linked list will have at least two elements.
 * All of the nodes' values will be unique.
 * The given node will not be the tail and it will always be a valid node of the linked
list.
 * Do not return anything from your function.
*/
public class _237 {
```

```
    public static class Solution1 {
        public void deleteNode(ListNode node) {
            node.val = node.next.val;
            node.next = node.next.next;
        }
    }
}

// ============================================================
// FILE 222: _238.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 238. Product of Array Except Self
 *
 * Given an array of n integers where n > 1, nums,
 * return an array output such that output[i] is equal to the product of all the elements
of nums except nums[i].
 * Solve it without division and in O(n).
 * For example, given [1,2,3,4], return [24,12,8,6].

 Follow up:
 Could you solve it with constant space complexity?
 (Note: The output array does not count as extra space for the purpose of space
complexity analysis.)
 */

public class _238 {

    public static class Solution1 {
        /**
         * Very straightforward idea: iterate through the array twice:
         * first time: get res[i] = res[i-1]*nums[i-1]
         * second time: have a variable called right, which means all the numbers product
to its right, then do
         * res[i] *= right;
         * right *= nums[i];
         * that's it.
         *
         * This could be very well illustrated with this example: [1,2,3,4]
         */
        public int[] productExceptSelf(int[] nums) {
            int n = nums.length;
            int[] result = new int[n];
            result[0] = 1;
            for (int i = 1; i < n; i++) {
                result[i] = result[i - 1] * nums[i - 1];
            }
            int right = 1;
            for (int i = n - 1; i >= 0; i--) {
                result[i] *= right;
                right *= nums[i];
```

```
            }
            return result;
        }
    }
}

// ============================================================
// FILE 223: _239.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 239. Sliding Window Maximum
 *
 * Given an array nums, there is a sliding window of size k which is moving from the very
left of the array to the very right.
 * You can only see the k numbers in the window. Each time the sliding window moves right
by one position.

 For example:

 Given nums = [1,3,-1,-3,5,3,6,7], and k = 3.

 Window position                Max
 ---------------               -----
 [1  3  -1] -3  5  3  6  7      3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
 Therefore, return the max sliding window as [3,3,5,5,6,7].

 Note:
 You may assume k is always valid, ie: 1 ≤ k ≤ input array's size for non-empty array.

 Follow up:
 Could you solve it in linear time?

 Hint:
 How about using a data structure such as deque (double-ended queue)?
 The queue size need not be the same as the window’s size.
 Remove redundant elements and the queue should store only elements that need to be
considered.
 */
public class _239 {

    public static class Solution1 {
        public int[] maxSlidingWindow(int[] nums, int k) {
            if (nums == null || nums.length == 0 || k == 0) {
                return new int[0];
```

```
            }
            PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> b - a);
            int[] res = new int[nums.length - k + 1];
            for (int i = 0; i < nums.length; i++) {
                if (i < k) {
                    heap.offer(nums[i]);
                    if (i == k - 1) {
                        res[0] = heap.peek();
                    }
                } else {
                    heap.remove(nums[i - k]);
                    heap.offer(nums[i]);
                    res[i - k + 1] = heap.peek();
                }
            }
            return res;
        }
    }
}

// ============================================================
// FILE 224: _240.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 240. Search a 2D Matrix II
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix.
 * This matrix has the following properties:

 Integers in each row are sorted in ascending from left to right.
 Integers in each column are sorted in ascending from top to bottom.
 For example,

 Consider the following matrix:

 [
 [1,   4,  7, 11, 15],
 [2,   5,  8, 12, 19],
 [3,   6,  9, 16, 22],
 [10, 13, 14, 17, 24],
 [18, 21, 23, 26, 30]
 ]
 Given target = 5, return true.

 Given target = 20, return false.
 */
public class _240 {

    public static class Solution1 {
        public boolean searchMatrix(int[][] matrix, int target) {
            if (matrix == null || matrix.length == 0) {
```

```
                return false;
            }
            int m = matrix.length;
            int n = matrix[0].length;
            int x = 0;
            int y = n - 1;
            while (x < m && y >= 0) {
                if (target == matrix[x][y]) {
                    return true;
                } else if (target > matrix[x][y]) {
                    x++;
                } else {
                    y--;
                }
            }
            return false;
        }
    }

}

// ============================================================
// FILE 225: _241.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.List;

/**
 * 241. Different Ways to Add Parentheses
 *
 * Given a string of numbers and operators, return all possible results from computing
all the different possible ways to group numbers and operators. The valid operators are
+, - and *.


 Example 1
 Input: "2-1-1".

 ((2-1)-1) = 0
 (2-(1-1)) = 2
 Output: [0, 2]


 Example 2
 Input: "2*3-4*5"

 (2*(3-(4*5))) = -34
 ((2*3)-(4*5)) = -14
 ((2*(3-4))*5) = -10
 (2*((3-4)*5)) = -10
 (((2*3)-4)*5) = 10
```

```
 Output: [-34, -14, -10, -10, 10]

 */
public class _241 {
    public static class Solution1 {
        /**Time:  O(n * 4^n / n^(3/2)) ~= n * Catalan numbers = n * (C(2n, n) - C(2n, n -
1)),
         due to the size of the results is Catalan numbers,
         and every way of evaluation is the length of the string,
         so the time complexity is at most n * Catalan numbers.
         Space: O(n * 4^n / n^(3/2)), the cache size of lookup is at most n * Catalan
numbers.*/

        /**
         * Credit: https://discuss.leetcode.com/topic/19901/a-recursive-java-solution-
284-ms
         */
        public List<Integer> diffWaysToCompute(String input) {
            List<Integer> answer = new LinkedList<>();
            int len = input.length();
            for (int i = 0; i < len; i++) {
                if (input.charAt(i) == '+'
                        || input.charAt(i) == '-'
                        || input.charAt(i) == '*') {
                    String part1 = input.substring(0, i);
                    String part2 = input.substring(i + 1);
                    List<Integer> answer1 = diffWaysToCompute(part1);
                    List<Integer> answer2 = diffWaysToCompute(part2);
                    for (int a1 : answer1) {
                        for (int a2 : answer2) {
                            int result = 0;
                            switch (input.charAt(i)) {
                                case '+':
                                    result = a1 + a2;
                                    break;
                                case '-':
                                    result = a1 - a2;
                                    break;
                                case '*':
                                    result = a1 * a2;
                                    break;
                                default:
                                    break;
                            }
                            answer.add(result);
                        }
                    }
                }
            }
            if (answer.size() == 0) {
                answer.add(Integer.valueOf(input));
            }
            return answer;
        }
```

```
    }
}

// ============================================================
// FILE 226: _242.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 242. Valid Anagram
 * Given two strings s and t, write a function to determine if t is an anagram of s.

 For example,
 s = "anagram", t = "nagaram", return true.
 s = "rat", t = "car", return false.

 Note:
 You may assume the string contains only lowercase alphabets.

 Follow up:
 What if the inputs contain unicode characters? How would you adapt your solution to such
case?
 */

public class _242 {

    public static class Solution1 {
        public boolean isAnagram(String s, String t) {
            char[] schar = s.toCharArray();
            char[] tchar = t.toCharArray();
            Arrays.sort(schar);
            Arrays.sort(tchar);
            return new String(schar).equals(new String(tchar));
        }
    }

    public static class Solution2 {
        public boolean isAnagram(String s, String t) {
            if (s == null || t == null || s.length() != t.length()) {
                return false;
            }
            int[] counts = new int[26];
            for (int i = 0; i < s.length(); i++) {
                counts[s.charAt(i) - 'a']++;
                counts[t.charAt(i) - 'a']--;
            }
            for (int i : counts) {
                if (i != 0) {
                    return false;
                }
            }
```

```
            return true;
        }
    }
}

// ============================================================
// FILE 227: _243.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 243. Shortest Word Distance
 *
 * Given a list of words and two words word1 and word2, return the shortest distance
between these two words in the list.

 For example,
 Assume that words = ["practice", "makes", "perfect", "coding", "makes"].

 Given word1 = “coding”, word2 = “practice”, return 3.
 Given word1 = "makes", word2 = "coding", return 1.

 Note:
 You may assume that word1 does not equal to word2, and word1 and word2 are both in the
list.*/
public class _243 {
    public static class Solution1 {
        public int shortestDistance(String[] words, String word1, String word2) {
            int p = -1;
            int q = -1;
            int min = Integer.MAX_VALUE;
            for (int i = 0; i < words.length; i++) {
                if (words[i].equals(word1)) {
                    p = i;
                }
                if (words[i].equals(word2)) {
                    q = i;
                }
                if (p != -1 && q != -1) {
                    min = Math.min(min, Math.abs(p - q));
                }
            }
            return min;

        }
    }
}

// ============================================================
// FILE 228: _244.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 244. Shortest Word Distance II
 *
 * This is a follow up of Shortest Word Distance. The only difference is now you are
given the list of words and your method will be called repeatedly many times with
different parameters. How would you optimize it?
 * Design a class which receives a list of words in the constructor, and implements a
method that takes two words word1 and word2 and return the shortest distance between
these two words in the list.

 For example,
 Assume that words = ["practice", "makes", "perfect", "coding", "makes"].

 Given word1 = “coding”, word2 = “practice”, return 3.
 Given word1 = "makes", word2 = "coding", return 1.

 Note:
 You may assume that word1 does not equal to word2, and word1 and word2 are both in the
list.
 */
public class _244 {
    public static class Solution1 {
        class WordDistance {

            private Map<String, List<Integer>> map;

            public WordDistance(String[] words) {
                map = new HashMap<>();
                for (int i = 0; i < words.length; i++) {
                    String w = words[i];
                    if (map.containsKey(w)) {
                        map.get(w).add(i);
                    } else {
                        List<Integer> list = new ArrayList<>();
                        list.add(i);
                        map.put(w, list);
                    }
                }
            }

            public int shortest(String word1, String word2) {
                List<Integer> list1 = map.get(word1);
                List<Integer> list2 = map.get(word2);
                int result = Integer.MAX_VALUE;
                for (int i = 0, j = 0; i < list1.size() && j < list2.size(); ) {
                    int index1 = list1.get(i);
                    int index2 = list2.get(j);
                    if (index1 < index2) {
```

```
                        result = Math.min(result, index2 - index1);
                        i++;
                    } else {
                        result = Math.min(result, index1 - index2);
                        j++;
                    }
                }
                return result;
            }
        }
    }
}

// ============================================================
// FILE 229: _245.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 245. Shortest Word Distance III
 *
 * This is a follow up of Shortest Word Distance. The only difference is now word1 could
be the same as word2.
 * Given a list of words and two words word1 and word2, return the shortest distance
between these two words in the list.
 * word1 and word2 may be the same and they represent two individual words in the list.

 For example,
 Assume that words = ["practice", "makes", "perfect", "coding", "makes"].

 Given word1 = “makes”, word2 = “coding”, return 1.
 Given word1 = "makes", word2 = "makes", return 3.

 Note:
 You may assume word1 and word2 are both in the list.
 */
public class _245 {

    public static class Solution1 {
        public int shortestWordDistance(String[] words, String word1, String word2) {
            int p1 = -1;
            int p2 = -1;
            int min = Integer.MAX_VALUE;
            for (int i = 0; i < words.length; i++) {
                if (words[i].equals(word1)) {
                    if (word1.equals(word2)) {
                        if (p1 != -1 && i - p1 < min) {
                            min = i - p1;
                        }
                        p1 = i;
                    } else {
                        p1 = i;
                        if (p2 != -1 && p1 - p2 < min) {
```

```
                            min = p1 - p2;
                        }
                    }
                } else if (words[i].equals(word2)) {
                    p2 = i;
                    if (p1 != -1 && p2 - p1 < min) {
                        min = p2 - p1;
                    }
                }
            }
            return min;
        }
    }
}

// ============================================================
// FILE 230: _246.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 246. Strobogrammatic Number
 *
 * A strobogrammatic number is a number that looks the same when rotated 180 degrees
(looked at upside down).
 * Write a function to determine if a number is strobogrammatic. The number is
represented as a string.
 * For example, the numbers "69", "88", and "818" are all strobogrammatic.
 */

public class _246 {

    public static class Solution1 {
        public boolean isStrobogrammatic(String num) {
            int i = 0;
            int j = num.length() - 1;
            Map<Character, Character> map = new HashMap();
            map.put('8', '8');
            map.put('1', '1');
            map.put('0', '0');
            if (j == 0) {
                return map.containsKey(num.charAt(i));
            }

            map.put('9', '6');
            map.put('6', '9');
            while (i < j) {
                if (!map.containsKey(num.charAt(i)) || !map.containsKey(num.charAt(j))) {
```

```
                    return false;
                }
                if (map.get(num.charAt(i)) != num.charAt(j)) {
                    return false;
                }
                i++;
                j--;
            }
            return map.containsKey(num.charAt(i));
        }
    }


    public static class Solution2 {
        public boolean isStrobogrammatic(String num) {
            Set<Character> set = new HashSet();
            set.add('0');
            set.add('1');
            set.add('6');
            set.add('8');
            set.add('9');
            char[] nums = num.toCharArray();
            int i = 0;
            int j = num.length() - 1;
            while (i <= j) {
                if (!set.contains(nums[i]) || !set.contains(nums[j])) {
                    return false;
                }
                if (nums[i] == '6' && nums[j] != '9') {
                    return false;
                } else if (nums[i] == '9' && nums[j] != '6') {
                    return false;
                } else if (nums[i] == '1' && nums[j] != '1') {
                    return false;
                } else if (nums[i] == '8' && nums[j] != '8') {
                    return false;
                } else if (nums[i] == '0' && nums[j] != '0') {
                    return false;
                } else {
                    i++;
                    j--;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 231: _247.java
// ============================================================

package com.fishercoder.solutions;
```

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class _247 {
    public static class Solution1 {
        public List<String> findStrobogrammatic(int n) {
            return recursiveHelper(n, n);
        }

        private List<String> recursiveHelper(int n, int m) {
            if (n == 0) {
                return new ArrayList<>(Arrays.asList(""));
            }
            if (n == 1) {
                return new ArrayList<>(Arrays.asList("0", "1", "8"));
            }

            List<String> list = recursiveHelper(n - 2, m);
            List<String> res = new ArrayList<>();

            for (int i = 0; i < list.size(); i++) {
                String s = list.get(i);
                if (n != m) {
                    res.add("0" + s + "0");
                }
                res.add("1" + s + "1");
                res.add("6" + s + "9");
                res.add("8" + s + "8");
                res.add("9" + s + "6");
            }
            return res;
        }
    }
}

// ============================================================
// FILE 232: _248.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

public class _248 {

    public static class Solution1 {
        /**Credit: https://discuss.leetcode.com/topic/31386/concise-java-solution
         *
         Construct char arrays from low.length() to high.length()
         Add stro pairs from outside
         When left > right, add eligible count
         */
```

```
        private static final char[][] pairs = {{'0', '0'}, {'1', '1'}, {'6', '9'}, {'8',
'8'}, {'9', '6'}};

        public int strobogrammaticInRange(String low, String high) {
            int[] count = {0};
            for (int len = low.length(); len <= high.length(); len++) {
                char[] c = new char[len];
                dfs(low, high, c, 0, len - 1, count);
            }
            return count[0];
        }

        public void dfs(String low, String high , char[] c, int left, int right, int[]
count) {
            if (left > right) {
                String s = new String(c);
                if ((s.length() == low.length() && s.compareTo(low) < 0)
                        || (s.length() == high.length() && s.compareTo(high) > 0)) {
                    return;
                }
                count[0]++;
                return;
            }
            for (char[] p : pairs) {
                c[left] = p[0];
                c[right] = p[1];
                if (c.length != 1 && c[0] == '0') {
                    continue;
                }
                if (left == right && p[0] != p[1]) {
                    continue;
                }
                dfs(low, high, c, left + 1, right - 1, count);
            }
        }
    }

    public static class Solution2 {
        Map<Character, Character> map = new HashMap<>();

        {
            map.put('1', '1');
            map.put('8', '8');
            map.put('6', '9');
            map.put('9', '6');
            map.put('0', '0');
        }

        String low = "";
        String high = "";

        public int strobogrammaticInRange(String low, String high) {
            this.low = low;
```

```
            this.high = high;
            int result = 0;
            for (int n = low.length(); n <= high.length(); n++) {
                int[] count = new int[1];
                strobogrammaticInRange(new char[n], count, 0, n - 1);
                result += count[0];
            }
            return result;
        }

        private void strobogrammaticInRange(char[] arr, int[] count, int lo, int hi) {
            if (lo > hi) {
                String s = new String(arr);
                if ((arr[0] != '0' || arr.length == 1) && compare(low, s) && compare(s,
high)) {
                    count[0]++;
                }
                return;
            }
            for (Character c : map.keySet()) {
                arr[lo] = c;
                arr[hi] = map.get(c);
                if ((lo == hi && c == map.get(c)) || lo < hi) {
                    strobogrammaticInRange(arr, count, lo + 1, hi - 1);
                }
            }
        }

        private boolean compare(String a, String b) {
            if (a.length() != b.length()) {
                return a.length() < b.length();
            }
            int i = 0;
            while (i < a.length() && a.charAt(i) == b.charAt(i)) {
                i++;
            }
            return i == a.length() ? true : a.charAt(i) <= b.charAt(i);
        }
    }
}

// ==========================================================
// FILE 233: _249.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class _249 {
```

```
    public static class Solution1 {
        public List<List<String>> groupStrings(String[] strings) {

            List<List<String>> result = new ArrayList<List<String>>();
            Map<String, List<String>> map = new HashMap<String, List<String>>();

            for (String word : strings) {
                String key = "";
                int offset = word.charAt(0) - 'a';
                for (int i = 1; i < word.length(); i++) {
                    key += (word.charAt(i) - offset + 26) % 26;
                }

                if (!map.containsKey(key)) {
                    map.put(key, new ArrayList<>());
                }
                map.get(key).add(word);
            }

            for (List<String> list : map.values()) {
                Collections.sort(list);
                result.add(list);
            }

            return result;
        }
    }

}

// ============================================================
// FILE 234: _250.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

public class _250 {

    public static class Solution1 {
        public int countUnivalSubtrees(TreeNode root) {
            int[] count = new int[1];
            helper(root, count);
            return count[0];
        }

        private boolean helper(TreeNode node, int[] count) {
            if (node == null) {
                return true;
            }
            boolean left = helper(node.left, count);
            boolean right = helper(node.right, count);
```

```
            if (left && right) {
                if (node.left != null && node.val != node.left.val) {
                    return false;
                }
                if (node.right != null && node.val != node.right.val) {
                    return false;
                }
                count[0]++;
                return true;
            }
            return false;
        }
    }
}

// ==========================================================
// FILE 235: _251.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

public class _251 {

    public static class Solution1 {
        class Vector2D implements Iterator<Integer> {
            private Queue<Integer> cache;
            private List<List<Integer>> vec2d;

            public Vector2D(List<List<Integer>> vec2d) {
                this.vec2d = vec2d;
                this.cache = new LinkedList<Integer>();
                if (vec2d != null && vec2d.size() > 0) {
                    for (List<Integer> list : vec2d) {
                        for (int i : list) {
                            cache.offer(i);
                        }
                    }
                }
            }

            @Override
            public Integer next() {
                return cache.poll();
            }

            @Override
            public boolean hasNext() {
                return !cache.isEmpty();
            }
```

```
        }
    }
}

// ============================================================
// FILE 236: _252.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class _252 {
    public static class Solution1 {
        public boolean canAttendMeetings(Interval[] intervals) {

            List<Interval> list = new ArrayList();
            for (Interval interval : intervals) {
                list.add(interval);
            }

            Collections.sort(list, (o1, o2) -> {
                if (o1.start > o2.start) {
                    return 1;
                } else {
                    return -1;
                }
            });

            for (int i = 0; i < list.size() - 1; i++) {
                if (list.get(i).end > list.get(i + 1).start) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 237: _253.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.Arrays;
import java.util.PriorityQueue;
```

```
public class _253 {
    public static class Solution1 {

        public int minMeetingRooms(Interval[] intervals) {
            if (intervals == null || intervals.length == 0) {
                return 0;
            }

            // Sort the intervals by start time
            Arrays.sort(intervals, (a, b) -> a.start - b.start);

            // Use a min heap to track the minimum end time of merged intervals
            PriorityQueue<Interval> heap = new PriorityQueue<>(intervals.length, (a, b) -
> a.end - b.end);

            // start with the first meeting, put it to a meeting room
            heap.offer(intervals[0]);

            for (int i = 1; i < intervals.length; i++) {
                // get the meeting room that finishes earliest
                Interval interval = heap.poll();

                if (intervals[i].start >= interval.end) {
                    // if the current meeting starts right after
                    // there's no need for a new room, merge the interval
                    interval.end = intervals[i].end;
                } else {
                    // otherwise, this meeting needs a new room
                    heap.offer(intervals[i]);
                }

                // don't forget to put the meeting room back
                heap.offer(interval);
            }

            return heap.size();
        }
    }
}


// ============================================================
// FILE 238: _254.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

public class _254 {

    public static class Solution1 {
        public List<List<Integer>> getFactors(int n) {
```

```
            List<List<Integer>> result = new ArrayList<>();
            helper(result, new ArrayList<>(), n, 2);
            return result;
        }

        public void helper(List<List<Integer>> result, List<Integer> item, int n, int
start) {
            if (n <= 1) {
                if (item.size() > 1) {
                    result.add(new ArrayList<>(item));
                }
                return;
            }

            for (int i = start; i <= n; ++i) {
                if (n % i == 0) {
                    item.add(i);
                    helper(result, item, n / i, i);
                    item.remove(item.size() - 1);
                }
            }
        }
    }
}

// ==========================================================
// FILE 239: _255.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.Stack;

public class _255 {

    public static class Solution1 {
        public boolean verifyPreorder(int[] preorder) {
            int low = Integer.MIN_VALUE;
            Stack<Integer> stack = new Stack();
            for (int p : preorder) {
                if (p < low) {
                    return false;
                }
                while (!stack.empty() && p > stack.peek()) {
                    low = stack.pop();
                }
                stack.push(p);
            }
            return true;
        }
    }

}
```

```
// ============================================================
// FILE 240: _256.java
// ============================================================

package com.fishercoder.solutions;

public class _256 {

    public static class Solution1 {
        public int minCost(int[][] costs) {
            if (costs == null || costs.length == 0) {
                return 0;
            }
            for (int i = 1; i < costs.length; i++) {
                costs[i][0] += Math.min(costs[i - 1][1], costs[i - 1][2]);
                costs[i][1] += Math.min(costs[i - 1][0], costs[i - 1][2]);
                costs[i][2] += Math.min(costs[i - 1][1], costs[i - 1][0]);
            }
            int n = costs.length - 1;
            return Math.min(Math.min(costs[n][0], costs[n][1]), costs[n][2]);
        }
    }
}

// ============================================================
// FILE 241: _257.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

public class _257 {
    public static class Solution1 {
        //a very typical/good question to test your recursion/dfs understanding.
        public List<String> binaryTreePaths_more_concise(TreeNode root) {
            List<String> paths = new ArrayList<>();
            if (root == null) {
                return paths;
            }
            dfs(root, paths, "");
            return paths;
        }

        private void dfs(TreeNode root, List<String> paths, String path) {
            if (root.left == null && root.right == null) {
                paths.add(path + root.val);
                return;
            }
            path += root.val + "->";
            if (root.left != null) {
```

```
                dfs(root.left, paths, path);
            }
            if (root.right != null) {
                dfs(root.right, paths, path);
            }
        }
    }

    public static class Solution2 {
        public List<String> binaryTreePaths(TreeNode root) {
            List<String> paths = new ArrayList<>();
            dfs(root, paths, new StringBuilder());
            return paths;
        }

        private void dfs(TreeNode root, List<String> paths, StringBuilder sb) {
            if (root == null) {
                return;
            }
            if (root.left == null && root.right == null) {
                sb.append(root.val);
                paths.add(sb.toString());
                return;
            }
            sb.append(root.val + "->");
            String curr = sb.toString();
            if (root.left != null) {
                dfs(root.left, paths, sb);
            }
            sb.setLength(0);
            sb.append(curr);
            if (root.right != null) {
                dfs(root.right, paths, sb);
            }
        }
    }
}

// ============================================================
// FILE 242: _258.java
// ============================================================

package com.fishercoder.solutions;

public class _258 {

    public static class Solution1 {
        //only three cases as the code shows
        public int addDigits(int num) {
            if (num == 0) {
                return 0;
            }
            if (num % 9 == 0) {
                return 9;
```

```
            }
            return num % 9;
        }
    }
}

// ============================================================
// FILE 243: _259.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

public class _259 {

    public static class Solution1 {
        /**
         * Basically, very similar to 3Sum, but the key is that you'll have to add result
by (right-left), not just increment result by 1!
         */
        public int threeSumSmaller(int[] nums, int target) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            int result = 0;
            Arrays.sort(nums);
            for (int i = 0; i < nums.length - 2; i++) {
                int left = i + 1;
                int right = nums.length - 1;
                while (left < right) {
                    int sum = nums[i] + nums[left] + nums[right];
                    if (sum < target) {
                        result += right - left;//this line is key!
                        left++;
                    } else {
                        right--;
                    }
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 244: _260.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
```

```
/**
 * 260. Single Number III
 *
 * Given an array of numbers nums,
 * in which exactly two elements appear only once and all the other elements appear
exactly twice.
 * Find the two elements that appear only once.

For example:

Given nums = [1, 2, 1, 3, 2, 5], return [3, 5].

Note:
The order of the result is not important. So in the above example, [5, 3] is also
correct.
Your algorithm should run in linear runtime complexity. Could you implement it using only
constant space complexity?
*/

public class _260 {

    public static class Solution1 {
        public int[] singleNumber(int[] nums) {
            Map<Integer, Integer> map = new HashMap();
            for (int i : nums) {
                map.put(i, map.getOrDefault(i, 0) + 1);
            }

            int[] res = new int[2];
            int index = 0;
            for (int key : map.keySet()) {
                if (map.get(key) == 1) {
                    res[index++] = key;
                }
                if (index == 2) {
                    break;
                }
            }
            return res;
        }
    }

    public static class Solution2 {
        /**Credit: https://discuss.leetcode.com/topic/21605/accepted-c-java-o-n-time-o-1-
space-easy-solution-with-detail-explanations/2
         *
         * some more explanation about this algorithm:
         * two's complement: one number's two's complement number is computed as below:
         * reverse all bits of this number and then add one:
         * e.g. decimal number 2, in binary format: 0010 (4 bits)
         * reversing every single bit becomes 1101,
         * then add 1 to it, it becomes 1110
         *
         * so
```

```
         * num &= -num, in this case, 2 &= -2 becomes 2
         * */
        public int[] singleNumber(int[] nums) {
            int diff = 0;
            for (int num : nums) {
                diff ^= num;
            }

            //get least significant set bit
            diff &= -diff;

            int[] result = new int[2];
            for (int num : nums) {
                if ((num & diff) == 0) {
                    result[0] ^= num;
                } else {
                    result[1] ^= num;
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 245: _261.java
// ============================================================

package com.fishercoder.solutions;

/**
 *261. Graph Valid Tree
 *
 * Given n nodes labeled from 0 to n - 1 and a list of undirected edges
 * (each edge is a pair of nodes),
 * write a function to check whether these edges make up a valid tree.

 For example:

 Given n = 5 and edges = [[0, 1], [0, 2], [0, 3], [1, 4]], return true.

 Given n = 5 and edges = [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]], return false.

 Hint:

 Given n = 5 and edges = [[0, 1], [1, 2], [3, 4]], what should your return? Is this case
a valid tree?
 According to the definition of tree on Wikipedia:
 “a tree is an undirected graph in which any two vertices are connected by exactly one
path.
 In other words, any connected graph without simple cycles is a tree.”

 Note: you can assume that no duplicate edges will appear in edges.
 Since all edges are undirected, [0, 1] is the same as [1, 0] and thus will not appear
```

```
together in edges.
 */
public class _261 {

    public static class Solution1 {
        public boolean validTree(int n, int[][] edges) {
            int[] nums = new int[n];
            for (int i = 0; i < n; i++) {
                nums[i] = i;
            }

            for (int i = 0; i < edges.length; i++) {
                int x = find(nums, edges[i][0]);
                int y = find(nums, edges[i][1]);

                if (x == y) {
                    return false;
                }

                //union
                nums[x] = y;
            }

            return edges.length == n - 1;
        }

        int find(int[] nums, int i) {
            if (nums[i] == i) {
                return i;
            }
            return find(nums, nums[i]);
        }
    }
}

// ============================================================
// FILE 246: _263.java
// ============================================================

package com.fishercoder.solutions;

public class _263 {

    public static class Solution1 {
        public boolean isUgly(int num) {
            if (num == 0) {
                return false;
            }
            int[] divisors = new int[]{5, 3, 2};
            for (int divisor : divisors) {
                while (num % divisor == 0) {
                    num /= divisor;
                }
            }
```

```
            return num == 1;
        }
    }

}

// ============================================================
// FILE 247: _264.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 264. Ugly Number II
 *
 * Write a program to find the n-th ugly number.

 Ugly numbers are positive numbers whose prime factors only include 2, 3, 5. For example,
1, 2, 3, 4, 5, 6, 8, 9, 10, 12 is the sequence of the first 10 ugly numbers.

 Note that 1 is typically treated as an ugly number, and n does not exceed 1690.
 */
public class _264 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/21791/o-n-java-solution
         */
        public int nthUglyNumber(int n) {
            int[] ugly = new int[n];
            ugly[0] = 1;
            int index2 = 0;
            int index3 = 0;
            int index5 = 0;
            int factor2 = 2;
            int factor3 = 3;
            int factor5 = 5;
            for (int i = 1; i < n; i++) {
                int min = Math.min(Math.min(factor2, factor3), factor5);
                ugly[i] = min;
                if (factor2 == min) {
                    factor2 = 2 * ugly[++index2];
                }
                if (factor3 == min) {
                    factor3 = 3 * ugly[++index3];
                }
                if (factor5 == min) {
                    factor5 = 5 * ugly[++index5];
                }
            }
            return ugly[n - 1];
        }
    }
}
```

```
// ============================================================
// FILE 248: _265.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 265. Paint House II
 *
 * There are a row of n houses, each house can be painted with one of the k colors.
 * The cost of painting each house with a certain color is different.
 * You have to paint all the houses such that no two adjacent houses have the same color.
 * The cost of painting each house with a certain color is represented by a n x k cost
matrix.
 *
 * For example, costs[0][0] is the cost of painting house 0 with color 0;
 * costs[1][2] is the cost of painting house 1 with color 2,
 * and so on...
 *
 * Find the minimum cost to paint all houses.

 Note:
 All costs are positive integers.

 Follow up:
 Could you solve it in O(nk) runtime?
 */
public class _265 {

        public static class Solution1 {
                public int minCostII(int[][] costs) {
                        if (costs == null || costs.length == 0) {
                                return 0;
                        }

                        int n = costs.length;
                        int k = costs[0].length;
                        // min1 is the index of the 1st-smallest cost till previous house
                        // min2 is the index of the 2nd-smallest cost till previous house
                        int min1 = -1;
                        int min2 = -1;

                        for (int i = 0; i < n; i++) {
                                int last1 = min1;
                                int last2 = min2;
                                min1 = -1;
                                min2 = -1;

                                for (int j = 0; j < k; j++) {
                                        if (j != last1) {
                                                // current color j is different to last
min1
                                                costs[i][j] += last1 < 0 ? 0 : costs[i -
```

```
1][last1];
                                        } else {
                                                costs[i][j] += last2 < 0 ? 0 : costs[i -
1][last2];
                                        }

                                        // find the indices of 1st and 2nd smallest cost
of painting current house i
                                        if (min1 < 0 || costs[i][j] < costs[i][min1]) {
                                                min2 = min1;
                                                min1 = j;
                                        } else if (min2 < 0 || costs[i][j] < costs[i]
[min2]) {
                                                min2 = j;
                                        }
                                }
                        }
                        return costs[n - 1][min1];
                }
        }

}

// ======================================================
// FILE 249: _266.java
// ======================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

public class _266 {

    public static class Solution1 {
        public boolean canPermutePalindrome(String s) {

            char[] chars = s.toCharArray();
            Map<Character, Integer> map = new HashMap<>();
            for (char c : chars) {
                if (!map.containsKey(c)) {
                    map.put(c, 1);
                } else {
                    map.put(c, map.get(c) + 1);
                }
            }
            int evenCount = 0;
            for (Map.Entry<Character, Integer> e : map.entrySet()) {
                if (e.getValue() % 2 != 0) {
                    evenCount++;
                }
                if (evenCount > 1) {
                    return false;
                }
```

```
            }
            return true;
        }
    }

}

// ============================================================
// FILE 250: _267.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class _267 {

    public static class Solution1 {
        public List<String> generatePalindromes(String s) {
            int odd = 0;
            String mid = "";
            List<String> res = new ArrayList();
            List<Character> list = new ArrayList();
            Map<Character, Integer> map = new HashMap();

            // step 1. build character count map and count odds
            for (int i = 0; i < s.length(); i++) {
                char c = s.charAt(i);
                map.put(c, map.containsKey(c) ? map.get(c) + 1 : 1);
                odd += map.get(c) % 2 != 0 ? 1 : -1;
            }

            // cannot form any palindromic string
            if (odd > 1) {
                return res;
            }

            // step 2. add half count of each character to list
            for (Map.Entry<Character, Integer> entry : map.entrySet()) {
                char key = entry.getKey();
                int val = entry.getValue();

                if (val % 2 != 0) {
                    mid += key;
                }

                for (int i = 0; i < val / 2; i++) {
                    list.add(key);
                }
            }
```

```
            // step 3. generate all the permutations
            getPerm(list, mid, new boolean[list.size()], new StringBuilder(), res);

            return res;
        }

        // generate all unique permutation from list
        void getPerm(List<Character> list, String mid, boolean[] used, StringBuilder sb,
                     List<String> res) {
            if (sb.length() == list.size()) {
                // form the palindromic string
                res.add(sb.toString() + mid + sb.reverse().toString());
                sb.reverse();
                return;
            }

            for (int i = 0; i < list.size(); i++) {
                // avoid duplication
                if (i > 0 && list.get(i) == list.get(i - 1) && !used[i - 1]) {
                    continue;
                }

                if (!used[i]) {
                    used[i] = true;
                    sb.append(list.get(i));
                    // recursion
                    getPerm(list, mid, used, sb, res);
                    // backtracking
                    used[i] = false;
                    sb.deleteCharAt(sb.length() - 1);
                }
            }
        }
    }
}

// ============================================================
// FILE 251: _268.java
// ============================================================

package com.fishercoder.solutions;

public class _268 {

    public static class Solution1 {
        /**
         * we could take advantage of the array indices
         * then a number xor with itself is zero, so after we xor the entire array with
all of its indices, the missing number will show up.
         */
        public int missingNumber(int[] nums) {
            int xor = 0;
            int i = 0;
            for (; i < nums.length; i++) {
```

```
                xor = xor ^ i ^ nums[i];
            }
            return xor ^ i;
        }
    }

}

// ============================================================
// FILE 252: _269.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

/**
 * 269. Alien Dictionary
 *
 * There is a new alien language which uses the latin alphabet.
 * However, the order among letters are unknown to you.
 * You receive a list of words from the dictionary, where words are sorted
lexicographically by the rules of this new language.
 * Derive the order of letters in this language.

 For example,
 Given the following words in dictionary,

 [
 "wrt",
 "wrf",
 "er",
 "ett",
 "rftt"
 ]
 The correct order is: "wertf".

 Note:
 You may assume all letters are in lowercase.
 If the order is invalid, return an empty string.
 There may be multiple valid order of letters, return any one of them is fine.
 */
public class _269 {
    public static class Solution1 {

        /**
         * reference: https://discuss.leetcode.com/topic/28308/java-ac-solution-using-bfs
         */
        public String alienOrder(String[] words) {
```

```
            Map<Character, Set<Character>> map = new HashMap();
            Map<Character, Integer> degree = new HashMap<>();
            String result = "";
            if (words == null || words.length == 0) {
                return result;
            }
            for (String s : words) {
                for (char c : s.toCharArray()) {
                    degree.put(c, 0);//keeps overwriting it, the purpose is to create one
entry
                    //for each letter in the degree map
                }
            }
            for (int i = 0; i < words.length - 1; i++) {
                String cur = words[i];
                String next = words[i + 1];
                int length = Math.min(cur.length(), next.length());
                for (int j = 0; j < length; j++) {
                    char c1 = cur.charAt(j);
                    char c2 = next.charAt(j);
                    if (c1 != c2) {
                        Set<Character> set = new HashSet<>();
                        if (map.containsKey(c1)) {
                            set = map.get(c1);
                        }
                        if (!set.contains(c2)) {
                            set.add(c2);
                            map.put(c1, set);
                            degree.put(c2, degree.get(c2) + 1);
                        }
                        break;
                    }
                }
            }
            Queue<Character> queue = new LinkedList<>();
            for (char c : degree.keySet()) {
                if (degree.get(c) == 0) {
                    queue.add(c);
                }
            }
            while (!queue.isEmpty()) {
                char c = queue.remove();
                result += c;
                if (map.containsKey(c)) {
                    for (char c2 : map.get(c)) {
                        degree.put(c2, degree.get(c2) - 1);
                        if (degree.get(c2) == 0) {
                            queue.add(c2);
                        }
                    }
                }
            }
            if (result.length() != degree.size()) {
                return "";
```

```
            }
            return result;
        }
    }

}

// ============================================================
// FILE 253: _270.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

public class _270 {

    public static class Solution1 {
        //A general tree solution, this finished in 1 ms
        public int closestValue(TreeNode root, double target) {
            if (root == null) {
                return 0;
            }
            double delta = Double.MAX_VALUE;
            return dfs(root, target, delta, root.val);
        }

        private int dfs(TreeNode root, double target, double delta, int closestVal) {
            if (Math.abs(root.val - target) < delta) {
                closestVal = root.val;
                delta = Math.abs(root.val - target);
            }
            int leftVal = closestVal;
            if (root.left != null) {
                leftVal = dfs(root.left, target, delta, closestVal);
            }
            int rightVal = closestVal;
            if (root.right != null) {
                rightVal = dfs(root.right, target, delta, closestVal);
            }
            return (Math.abs(leftVal - target) > Math.abs(rightVal - target)) ? rightVal
: leftVal;
        }
    }

    public static class Solution2 {
        // BST solution
        // we can tailor the solution to use the BST feature: left subtrees are always
smaller than the root the right subtrees
        //this finished in 0 ms
        public int closestValue(TreeNode root, double target) {
            if (root == null) {
                return 0;
            }
```

```
            return dfs(root, target, root.val);
        }

        private int dfs(TreeNode root, double target, int minVal) {
            if (root == null) {
                return minVal;
            }
            if (Math.abs(root.val - target) < Math.abs(minVal - target)) {
                minVal = root.val;
            }
            if (target < root.val) {
                minVal = dfs(root.left, target, minVal);
            } else {
                minVal = dfs(root.right, target, minVal);
            }
            return minVal;
        }
    }

    public static class Solution3 {
        //a more concise solution
        public int closestValue(TreeNode root, double target) {
            if (root == null) {
                return 0;
            }
            return dfs(root, target, root.val);
        }

        private int dfs(TreeNode root, double target, int minVal) {
            if (root == null) {
                return minVal;
            }
            if (Math.abs(root.val - target) < Math.abs(minVal - target)) {
                minVal = root.val;
            }
            minVal = dfs(root.left, target, minVal);
            minVal = dfs(root.right, target, minVal);
            return minVal;
        }
    }

    public static class Solution4 {
        //BST iterative solution
        public int closestValue(TreeNode root, double target) {
            long minVal = Long.MAX_VALUE;
            while (root != null) {
                if (Math.abs(root.val - target) < Math.abs(minVal - target)) {
                    minVal = root.val;
                }
                if (target < root.val) {
                    root = root.left;
                } else {
                    root = root.right;
                }
```

```
            }
            return minVal == Long.MAX_VALUE ? 0 : (int) minVal;
        }
    }
}

// ============================================================
// FILE 254: _271.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 271. Encode and Decode Strings
 *
 * Design an algorithm to encode a list of strings to a string.
 * The encoded string is then sent over the network and is decoded back to the original
list of strings.

 Machine 1 (sender) has the function:

 string encode(vector<string> strs) {
 // ... your code
 return encoded_string;
 }
 Machine 2 (receiver) has the function:
 vector<string> decode(string s) {
 //... your code
 return strs;
 }
 So Machine 1 does:

 string encoded_string = encode(strs);
 and Machine 2 does:

 vector<string> strs2 = decode(encoded_string);
 strs2 in Machine 2 should be the same as strs in Machine 1.

 Implement the encode and decode methods.

 Note:
 The string may contain any possible characters out of 256 valid ascii characters. Your
algorithm should be generalized enough to work on any possible characters.
 Do not use class member/global/static variables to store states. Your encode and decode
algorithms should be stateless.
 Do not rely on any library method such as eval or serialize methods. You should
implement your own encode/decode algorithm.
 */
public class _271 {
    public static class Solution1 {
        // Encodes a list of strings to a single string.
```

```
        public String encode(List<String> strs) {
            StringBuilder sb = new StringBuilder();
            for (String s : strs) {
                sb.append(s.length()).append('/').append(s);
            }
            return sb.toString();
        }

        // Decodes a single string to a list of strings.
        public List<String> decode(String s) {
            List<String> result = new ArrayList<>();
            int i = 0;
            while (i < s.length()) {
                int slash = s.indexOf('/', i);
                int size = Integer.valueOf(s.substring(i, slash));
                result.add(s.substring(slash + 1, slash + 1 + size));
                i = slash + size + 1;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 255: _272.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

/**
 * 272. Closest Binary Search Tree Value II
 *
 * Given a non-empty binary search tree and a target value, find k values in the BST that
are closest to the target.

 Note:
 Given target value is a floating point.
 You may assume k is always valid, that is: k ≤ total nodes.
 You are guaranteed to have only one unique set of k values in the BST that are closest
to the target.
 Follow up:
 Assume that the BST is balanced, could you solve it in less than O(n) runtime (where n =
total nodes)?

 Hint:

 Consider implement these two helper functions:
 getPredecessor(N), which returns the next smaller node to N.
```

```
 getSuccessor(N), which returns the next larger node to N.
 Try to assume that each node has a parent pointer, it makes the problem much easier.
 Without parent pointer we just need to keep track of the path from the root to the
current node using a stack.
 You would need two stacks to track the path in finding predecessor and successor node
separately.

 */
public class _272 {

    public static class Solution1 {
        public List<Integer> closestKValues(TreeNode root, double target, int k) {
            List<Integer> res = new ArrayList();

            Stack<Integer> s1 = new Stack(); // predecessors
            Stack<Integer> s2 = new Stack(); // successors

            inorder(root, target, false, s1);
            inorder(root, target, true, s2);

            while (k-- > 0) {
                if (s1.isEmpty()) {
                    res.add(s2.pop());
                } else if (s2.isEmpty()) {
                    res.add(s1.pop());
                } else if (Math.abs(s1.peek() - target) < Math.abs(s2.peek() - target)) {
                    res.add(s1.pop());
                } else {
                    res.add(s2.pop());
                }
            }

            return res;
        }

        // inorder traversal
        void inorder(TreeNode root, double target, boolean reverse, Stack<Integer> stack)
{
            if (root == null) {
                return;
            }

            inorder(reverse ? root.right : root.left, target, reverse, stack);
            // early terminate, no need to traverse the whole tree
            if ((reverse && root.val <= target) || (!reverse && root.val > target)) {
                return;
            }
            // track the value of current node
            stack.push(root.val);
            inorder(reverse ? root.left : root.right, target, reverse, stack);
        }
    }
}
```

```java
// ============================================================
// FILE 256: _273.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 273. Integer to English Words
 *
 * Convert a non-negative integer to its english words representation. Given input is guaranteed to be less than 231 - 1.

 For example,
 123 -> "One Hundred Twenty Three"
 12345 -> "Twelve Thousand Three Hundred Forty Five"
 1234567 -> "One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"

 Hint:
 Did you see a pattern in dividing the number into chunk of words? For example, 123 and 123000.
 Group the number by thousands (3 digits). You can write a helper function that takes a number less than 1000 and convert just that chunk to words.
 There are many edge cases. What are some good test cases? Does your code work with input such as 0? Or 1000010? (middle chunk is zero and should not be printed out)
 */
public class _273 {

    public static class Solution1 {
        private String[] belowTen = new String[]{"Zero", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine"};
        private String[] belowTwenty = new String[]{"Ten", "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"};
        private String[] belowHundred = new String[]{"Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"};
        private String[] overThousand = new String[]{"Thousand", "Million", "Billion"};

        public String numberToWords(int num) {
            String result;
            if (num == 0) {
                return belowTen[num];
            }

            result = hundredHelper(num % 1000);
            num = num / 1000;
            int i = 0;
            while (i < 3 && num > 0) {
                if (num % 1000 > 0) {
                    result = hundredHelper(num % 1000) + overThousand[i] + " " + result;
                }
                num = num / 1000;
                i++;
            }

            return result.trim();
```

```
        }

        private String hundredHelper(int num) {
            String nstr = "";
            if (num >= 100) {
                nstr = belowTen[num / 100] + " Hundred ";
            }
            num = num % 100;
            if (num >= 20) {
                if (num % 10 != 0) {
                    nstr = nstr + belowHundred[num / 10 - 2] + " " + belowTen[num % 10] +
" ";
                } else {
                    nstr = nstr + belowHundred[num / 10 - 2] + " ";
                }
            } else if (num >= 10) {
                nstr = nstr + belowTwenty[num % 10] + " ";
            } else if (num > 0) {
                nstr = nstr + belowTen[num] + " ";
            }
            return nstr;
        }
    }
}

// ============================================================
// FILE 257: _274.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 274. H-Index
 *
 * Given an array of citations (each citation is a non-negative integer) of a researcher,
write a function to compute the researcher's h-index.
 * According to the definition of h-index on Wikipedia: "A scientist has index h if h of
his/her N papers have at least h citations each, and the other N − h papers have no more
than h citations each."
 * For example, given citations = [3, 0, 6, 1, 5], which means the researcher has 5
papers in total and each of them had received 3, 0, 6, 1, 5 citations respectively.
 * Since the researcher has 3 papers with at least 3 citations each and the remaining two
with no more than 3 citations each, his h-index is 3.

 Note: If there are several possible values for h, the maximum one is taken as the h-
index.

 Hint:

 An easy approach is to sort the array first.
 What are the possible values of h-index?
 A faster approach is to use extra space.
```

```
 */
public class _274 {
    public static class Solution1 {
        public int hIndex(int[] citations) {
            if (citations == null || citations.length == 0) {
                return 0;
            }

            Arrays.sort(citations);
            for (int i = 0; i < citations.length; i++) {
                if (citations[i] >= citations.length - i) {
                    return citations.length - i;
                }
            }
            return 0;
        }
    }
}

// ============================================================
// FILE 258: _275.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 275. H-Index II
 *
 * Follow up for H-Index: What if the citations array is sorted in ascending order?
 * Could you optimize your algorithm?
 */
public class _275 {
    public static class Solution1 {
        public int hIndex(int[] citations) {
            int left = 0;
            int len = citations.length;
            int right = len - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;
                if (citations[mid] >= (len - mid)) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }
            return len - left;
        }
    }
}

// ============================================================
// FILE 259: _276.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 276. Paint Fence

 There is a fence with n posts, each post can be painted with one of the k colors.

 You have to paint all the posts such that no more than two adjacent fence posts have the
same color.

 Return the total number of ways you can paint the fence.

 Note:
 n and k are non-negative integers.*/

public class _276 {
    public static class Solution1 {
        public int numWays(int n, int k) {
            if (n == 0) {
                return 0;
            } else if (n == 1) {
                return k;
            }
            int sameColorCnt = k;
            int diffColorCnt = k * (k - 1);
            for (int i = 2; i < n; i++) {
                int temp = diffColorCnt;
                diffColorCnt = (diffColorCnt + sameColorCnt) * (k - 1);
                sameColorCnt = temp;
            }
            return sameColorCnt + diffColorCnt;
        }
    }
}

// ============================================================
// FILE 260: _277.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 277. Find the Celebrity
 *
 * Suppose you are at a party with n people (labeled from 0 to n - 1) and among them,
there may exist one celebrity.
 * The definition of a celebrity is that all the other n - 1 people know him/her but
he/she does not know any of them.
 * Now you want to find out who the celebrity is or verify that there is not one.
 * The only thing you are allowed to do is to ask questions like: "Hi, A. Do you know B?"
to get information of whether A knows B.
 * You need to find out the celebrity (or verify there is not one) by asking as few
questions as possible (in the asymptotic sense).
 * You are given a helper function bool knows(a, b) which tells you whether A knows B.
```

```
Implement a function int findCelebrity(n), your function should minimize the number of
calls to knows.
 *
 * Note: There will be exactly one celebrity if he/she is in the party. Return the
celebrity's label if there is a celebrity in the party. If there is no celebrity, return
-1.
 */
public class _277 {

    public static class Solution1 {
        public int findCelebrity(int n) {
            int candidate = 0;
            for (int i = 1; i < n; i++) {
                if (knows(candidate, i)) {
                    candidate = i;
                }
            }
            for (int i = 0; i < n; i++) {
                if (i != candidate && (knows(candidate, i) || !knows(i, candidate))) {
                    return -1;
                }
            }
            return candidate;
        }

        //this is a mock-up method to make IDE happy.s
        private boolean knows(int i, int candidate) {
            return false;
        }
    }
}

// ============================================================
// FILE 261: _278.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 278. First Bad Version
 *
 * You are a product manager and currently leading a team to develop a new product.
 * Unfortunately, the latest version of your product fails the quality check.
 * Since each version is developed based on the previous version, all the versions after
a bad version are also bad.
 * Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one,
 * which causes all the following ones to be bad.
 *
 * You are given an API bool isBadVersion(version) which will return whether version is
bad.
 * Implement a function to find the first bad version. You should minimize the number of
calls to the API.*/
public class _278 {
```

```
        public static class Solution1 {
                public int firstBadVersion(int n) {
                        int left = 1;
                        int right = n;
                        if (isBadVersion(left)) {
                                return left;
                        }

                        while (left + 1 < right) {
                                int mid = left + (right - left) / 2;
                                if (isBadVersion(mid)) {
                                        right = mid;
                                } else {
                                        left = mid;
                                }
                        }

                        if (isBadVersion(left)) {
                                return left;
                        }
                        return right;
                }

                private boolean isBadVersion(int left) {
                        //this is a fake method to make Eclipse happy
                        return false;
                }
        }
}

// ============================================================
// FILE 262: _279.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 279. Perfect Squares
 *
 * Given a positive integer n, find the least number of perfect square numbers (for
example, 1, 4, 9, 16, ...) which sum to n.
 * For example, given n = 12, return 3 because 12 = 4 + 4 + 4; given n = 13, return 2
because 13 = 4 + 9.
 */
public class _279 {
    public static class Solution1 {
        public int numSquares(int n) {
            int result = n;
            int num = 2;
            while (num * num <= n) {
                int temp1 = n / (num * num);
                int temp2 = n % (num * num);
```

```
                result = Math.min(result, temp1 + numSquares(temp2));
                num++;
            }
            return result;
        }
    }

    public static class Solution2 {
        //DP solution
        public int numSquares(int n) {
            int[] dp = new int[n + 1];
            Arrays.fill(dp, Integer.MAX_VALUE);
            dp[0] = 0;
            dp[1] = 1;
            for (int i = 1; i <= n; i++) {
                int min = Integer.MAX_VALUE;
                int j = 1;
                while (i - j * j >= 0) {
                    min = Math.min(min, dp[i - j * j] + 1);
                    j++;
                }
                dp[i] = min;
            }
            return dp[n];
        }
    }

}

// ============================================================
// FILE 263: _280.java
// ============================================================

package com.fishercoder.solutions;

/**
 * Given an unsorted array nums, reorder it in-place such that nums[0] <= nums[1] >=
nums[2] <= nums[3]....
For example, given nums = [3, 5, 2, 1, 6, 4], one possible answer is [1, 6, 2, 5, 3, 4]*/
public class _280 {
    public static class Solution1 {
        public void wiggleSort(int[] nums) {
            for (int i = 1; i < nums.length; i++) {
                if ((i % 2 == 0 && nums[i] > nums[i - 1]) || (i % 2 == 1 && nums[i] <
nums[i - 1])) {
                    swap(nums, i);
                }
            }
        }

        void swap(int[] nums, int i) {
            int temp = nums[i - 1];
            nums[i - 1] = nums[i];
            nums[i] = temp;
```

```
        }
    }
}

// ============================================================
// FILE 264: _281.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 281. Zigzag Iterator
 *
 * Given two 1d vectors, implement an iterator to return their elements alternately.

 For example, given two 1d vectors:

 v1 = [1, 2]
 v2 = [3, 4, 5, 6]
 By calling next repeatedly until hasNext returns false, the order of elements returned
by next should be: [1, 3, 2, 4, 5, 6].

 Follow up: What if you are given k 1d vectors? How well can your code be extended to
such cases?

 Clarification for the follow up question - Update (2015-09-18):
 The "Zigzag" order is not clearly defined and is ambiguous for k > 2 cases.
 If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic". For example,
given the following input:

 [1,2,3]
 [4,5,6,7]
 [8,9]
 It should return [1,4,8,2,5,9,3,6,7].
 */
public class _281 {

    public static class Solution1 {
        public static class ZigzagIterator {
            private Iterator<Integer> i;
            private Iterator<Integer> j;
            private Iterator<Integer> tmp;

            public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
                i = v2.iterator();
                j = v1.iterator();
            }

            public int next() {
```

```
                if (j.hasNext()) {
                    tmp = j;
                    j = i;
                    i = tmp;
                }
                return i.next();
            }

            public boolean hasNext() {
                return i.hasNext() || j.hasNext();
            }
        }
    }

    public static class Solution2 {
        public static class ZigzagIterator {

            Queue<Iterator<Integer>> queue;

            public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
                queue = new LinkedList<>();
                if (v1 != null && !v1.isEmpty()) {
                    Iterator<Integer> iterator1 = v1.iterator();
                    queue.offer(iterator1);
                }
                if (v2 != null && !v2.isEmpty()) {
                    Iterator<Integer> iterator2 = v2.iterator();
                    queue.offer(iterator2);
                }
            }

            public boolean hasNext() {
                return !queue.isEmpty();
            }

            public int next() {
                Iterator<Integer> iterator = queue.poll();
                int next = iterator.next();
                if (iterator.hasNext()) {
                    queue.offer(iterator);
                }
                return next;
            }
        }
    }
}

// ============================================================
// FILE 265: _282.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
```

```
import java.util.List;

/**
 * 282. Expression Add Operators
 *
 * Given a string that contains only digits 0-9 and a target value,
 * return all possibilities to add binary operators (not unary) +, -, or * between the
digits
 * so they evaluate to the target value.

 Examples:
 "123", 6 -> ["1+2+3", "1*2*3"]
 "232", 8 -> ["2*3+2", "2+3*2"]
 "105", 5 -> ["1*0+5","10-5"]
 "00", 0 -> ["0+0", "0-0", "0*0"]
 "3456237490", 9191 -> []
 */
public class _282 {

    public static class Solution1 {
        public List<String> addOperators(String num, int target) {
            List<String> res = new ArrayList<String>();
            StringBuilder sb = new StringBuilder();
            dfs(res, sb, num, 0, target, 0, 0);
            return res;

        }

        private void dfs(List<String> res, StringBuilder sb, String num, int pos, int
target, long prev, long multi) {
            if (pos == num.length()) {
                if (target == prev) {
                    res.add(sb.toString());
                }
                return;
            }
            for (int i = pos; i < num.length(); i++) {
                if (num.charAt(pos) == '0' && i != pos) {
                    break;
                }
                long curr = Long.parseLong(num.substring(pos, i + 1));
                int len = sb.length();
                if (pos == 0) {
                    dfs(res, sb.append(curr), num, i + 1, target, curr, curr);
                    sb.setLength(len);
                } else {
                    dfs(res, sb.append("+").append(curr), num, i + 1, target, prev +
curr, curr);
                    sb.setLength(len);

                    dfs(res, sb.append("-").append(curr), num, i + 1, target, prev -
curr, -curr);
                    sb.setLength(len);
```

```
                    dfs(res, sb.append("*").append(curr), num, i + 1, target, prev -
multi + multi * curr, multi * curr);
                    sb.setLength(len);
                }
            }
        }
    }

}

// ============================================================
// FILE 266: _283.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 283. Move Zeroes
 *
 * Given an array nums, write a function to move all 0's to the end of it while
maintaining the relative order of the non-zero elements.
 *
 * For example, given nums = [0, 1, 0, 3, 12], after calling your function, nums should
be [1, 3, 12, 0, 0].
 *
 * Note:
 * You must do this in-place without making a copy of the array.
 * Minimize the total number of operations.*/
public class _283 {
    public static class Solution1 {
        public void moveZeroes(int[] nums) {
            //keep the last non-zero index and keep overwriting it, then append zeroes to
fill the end
            int j = 0;
            int i = 0;
            for (; j < nums.length; j++) {
                if (nums[j] != 0) {
                    nums[i++] = nums[j];
                }
            }
            for (; i < nums.length; i++) {
                nums[i] = 0;
            }
        }
    }

    public static class Solution2 {
        public void moveZeroes(int[] nums) {
            //this solutoin is the most optimal since it minimizes the number of
operations
            //the idea is to swap the non-zero element to the first zero number position
            for (int i = 0, j = 0; i < nums.length && j < nums.length; j++) {
                if (nums[j] != 0) {
                    int temp = nums[i];
```

```
                    nums[i++] = nums[j];
                    nums[j] = temp;
                }
            }
        }
    }

    //then I came up with this solution and got it AC'ed! Cheers!
    //basically, find the next non-zero number and swap it with the current zero number
    //Apparently it's not the most optimal, since this is basically an O(n^2) solution,
then I turned to Editorial solutions
    public static class Solution3 {
        public void moveZeroes(int[] nums) {
            for (int i = 0; i < nums.length - 1; i++) {
                if (nums[i] == 0) {
                    int j = i + 1;
                    while (j < nums.length && nums[j] == 0) {
                        j++;
                    }
                    if (j >= nums.length) {
                        return;
                    } else {
                        int temp = nums[j];
                        nums[j] = nums[i];
                        nums[i] = temp;
                    }
                }
            }
        }
    }

}

// ============================================================
// FILE 267: _284.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

/**
 * 284. Peeking Iterator
 *
 * Given an Iterator class interface with methods: next() and hasNext(),
 * design and implement a PeekingIterator that support
 * the peek() operation -- it essentially peek() at the element that will be returned by
the next call to next().

 Here is an example. Assume that the iterator is initialized to the beginning of the
queue: [1, 2, 3].
```

```
 Call next() gets you 1, the first element in the queue.

 Now you call peek() and it returns 2, the next element. Calling next() after that still
return 2.

 You call next() the final time and it returns 3, the last element. Calling hasNext()
after that should return false.

 Follow up: How would you extend your design to be generic and work with all types, not
just integer?
 */
public class _284 {
    public static class Solution1 {
        public static class PeekingIterator implements Iterator<Integer> {

            private Queue<Integer> queue;

            public PeekingIterator(Iterator<Integer> iterator) {
                // initialize any member here.
                queue = new LinkedList<>();
                while (iterator.hasNext()) {
                    queue.add(iterator.next());
                }
            }

            // Returns the next element in the iteration without advancing the iterator.
            public Integer peek() {
                return queue.peek();
            }

            // hasNext() and next() should behave the same as in the Iterator interface.
            // Override them if needed.
            @Override
            public Integer next() {
                return queue.poll();
            }

            @Override
            public boolean hasNext() {
                return !queue.isEmpty();
            }
        }
    }
}

// ============================================================
// FILE 268: _285.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.Iterator;
```

```
import java.util.Map;
import java.util.TreeMap;

/**285. Inorder Successor in BST

Given a binary search tree and a node in it, find the in-order successor of that node in the BST.

Note: If the given node has no in-order successor in the tree, return null. */
public class _285 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/25698/java-python-solution-o-h-time-and-o-1-space-iterative
         * The inorder traversal of a BST is the nodes in ascending order.
         * To find a successor, you just need to find the smallest one that is larger than the given value since there are no duplicate values in a BST.
         * It's just like the binary search in a sorted list.
         * <p>
         * The time complexity should be O(h) where h is the depth of the result node.
         * succ is a pointer that keeps the possible successor.
         * Whenever you go left the current root is the new possible successor, otherwise the it remains the same.
         * <p>
         * Only in a balanced BST O(h) = O(log n). In the worst case h can be as large as n.
         */
        public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
            TreeNode successor = null;
            while (root != null) {
                if (p.val < root.val) {
                    successor = root;
                    root = root.left;
                } else {
                    root = root.right;
                }
            }
            return successor;
        }
    }

    public static class Solution2 {
        public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
            TreeMap<Integer, TreeNode> map = new TreeMap<>();
            inorderTraversal(root, map);
            Iterator<Map.Entry<Integer, TreeNode>> iterator = map.entrySet().iterator();
            while (iterator.hasNext()) {
                Map.Entry<Integer, TreeNode> entry = iterator.next();
                if (entry.getValue() == p) {
                    if (iterator.hasNext()) {
                        return iterator.next().getValue();
                    } else {
                        return null;
```

```
                    }
                }
            }
            return null;
        }

        private void inorderTraversal(TreeNode root, TreeMap<Integer, TreeNode> map) {
            if (root == null) {
                return;
            }
            inorderTraversal(root.left, map);
            map.put(root.val, root);
            inorderTraversal(root.right, map);
            return;
        }
    }

}

// ============================================================
// FILE 269: _286.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * You are given a m x n 2D grid initialized with these three possible values.

 -1 - A wall or an obstacle.
 0 - A gate.
 INF - Infinity means an empty room. We use the value 231 - 1 = 2147483647 to represent
INF as you may assume that the distance to a gate is less than 2147483647.
 Fill each empty room with the distance to its nearest gate. If it is impossible to reach
a gate, it should be filled with INF.

 For example, given the 2D grid:
 INF  -1  0  INF
 INF INF INF  -1
 INF  -1 INF  -1
 0  -1 INF INF

 After running your function, the 2D grid should be:
 3  -1   0   1
 2   2   1  -1
 1  -1   2  -1
 0  -1   3   4

 */
public class _286 {
    public static class Solution1 {
```

```
        int[] dirs = new int[]{0, 1, 0, -1, 0};

        public void wallsAndGates(int[][] rooms) {
            if (rooms == null || rooms.length == 0 || rooms[0].length == 0) {
                return;
            }
            int m = rooms.length;
            int n = rooms[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (rooms[i][j] == 0) {
                        bfs(rooms, i, j, m, n);
                    }
                }
            }
        }

        void bfs(int[][] rooms, int i, int j, int m, int n) {
            for (int k = 0; k < 4; k++) {
                int x = dirs[k] + i;
                int y = dirs[k + 1] + j;
                if (x >= 0 && y >= 0 && x < m && y < n && rooms[x][y] > rooms[i][j] + 1)
{
                    rooms[x][y] = rooms[i][j] + 1;
                    bfs(rooms, x, y, m, n);
                }
            }
        }

    }

    public static class Solution2 {

        //push all gates into the queue first, and then put all its neighbours into the
queue with one distance to the gate, then continue to push the rest of the nodes into the
queue, and put all their neighbours into the queue with the nodes' value plus one until
the queue is empty
        int[] dirs = new int[]{0, 1, 0, -1, 0};

        public void wallsAndGates(int[][] rooms) {
            if (rooms == null || rooms.length == 0 || rooms[0].length == 0) {
                return;
            }
            int m = rooms.length;
            int n = rooms[0].length;
            Queue<int[]> queue = new LinkedList();
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (rooms[i][j] == 0) {
                        queue.offer(new int[]{i, j});
                    }
                }
            }
```

```
            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                for (int k = 0; k < 4; k++) {
                    int x = curr[0] + dirs[k];
                    int y = curr[1] + dirs[k + 1];
                    if (x >= 0 && x < m && y >= 0 && y < n && rooms[x][y] ==
Integer.MAX_VALUE) {
                        rooms[x][y] = rooms[curr[0]][curr[1]] + 1;
                        queue.offer(new int[]{x, y});
                    }
                }
            }
        }

    }
}

// ============================================================
// FILE 270: _287.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 287. Find the Duplicate Number
 *
 * Given an array nums containing n + 1 integers where each integer is between 1 and n
(inclusive),
 * prove that at least one duplicate number must exist. Assume that there is only one
duplicate number, find the duplicate one.
 *
 Note:
 You must not modify the array (assume the array is read only).
 You must use only constant, O(1) extra space.
 Your runtime complexity should be less than O(n2).
 There is only one duplicate number in the array, but it could be repeated more than
once.

 */
public class _287 {

    public static class Solution1 {
        /**no-brainer, used O(n) space*/
        public int findDuplicate(int[] nums) {
            Set<Integer> set = new HashSet<>();
            int dup = 0;
            for (int i = 0; i < nums.length; i++) {
                if (!set.add(nums[i])) {
                    dup = nums[i];
                    break;
                }
```

```
            }
            return dup;
        }
    }

    public static class Solution2 {
        /** O(1) space */
        public int findDuplicate(int[] nums) {
            int slow = 0;
            int fast = 0;
            int finder = 0;
            while (true) {
                slow = nums[slow];
                fast = nums[nums[fast]];

                if (slow == fast) {
                    break;
                }
            }

            while (true) {
                slow = nums[slow];
                finder = nums[finder];
                if (slow == finder) {
                    return slow;
                }
            }
        }
    }
}

// ============================================================
// FILE 271: _288.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;

/**An abbreviation of a word follows the form <first letter><number><last letter>. Below
are some examples of word abbreviations:

 a) it                      --> it    (no abbreviation)

 1
 b) d|o|g                   --> d1g

 1    1  1
 1---5----0----5--8
 c) i|nternationalizatio|n  --> i18n
```

```
 1
 1---5----0
 d) l|ocalizatio|n          --> l10n
 Assume you have a dictionary and given a word, find whether its abbreviation is unique
in the dictionary. A word's abbreviation is unique if no other word from the dictionary
has the same abbreviation.

 Example:
 Given dictionary = [ "deer", "door", "cake", "card" ]

 isUnique("dear") ->
 false

 isUnique("cart") ->
 true

 isUnique("cane") ->
 false

 isUnique("make") ->
 true
 */
public class _288 {
    public static class Solution1 {

        public class ValidWordAbbr {
            private Map<String, String> dict;

            public ValidWordAbbr(String[] dictionary) {
                dict = new HashMap();
                for (String word : dictionary) {
                    String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                    if (dict.containsKey(key) && !dict.get(key).equals(word)) {
                        dict.put(key, "");
                    } else {
                        dict.put(key, word);
                    }
                }
            }

            public boolean isUnique(String word) {
                String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                if (!dict.containsKey(key)) {
                    return true;
                } else {
                    return dict.get(key) != "" && dict.get(key).equals(word);
                }
            }
        }
    }
```

```
    public static class Solution2 {
        public class ValidWordAbbr {

            private Map<String, Set<String>> dict;

            public ValidWordAbbr(String[] dictionary) {
                dict = new HashMap();
                for (String word : dictionary) {
                    String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                    if (dict.containsKey(key)) {
                        Set<String> set = dict.get(key);
                        set.add(word);
                        dict.put(key, set);
                    } else {
                        Set<String> set = new HashSet();
                        set.add(word);
                        dict.put(key, set);
                    }
                }
            }

            public boolean isUnique(String word) {
                String key = word.length() <= 2 ? word : (word.charAt(0) +
String.valueOf(word.length() - 2) + word.charAt(word.length() - 1));
                if (!dict.containsKey(key)) {
                    return true;
                } else {
                    Set<String> set = dict.get(key);
                    if (set.size() != 1) {
                        return false;
                    }
                    Iterator<String> it = set.iterator();
                    return it.next().equals(word);
                }
            }
        }
    }
}

// ============================================================
// FILE 272: _289.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 289. Game of Life
 *
 * According to the Wikipedia's article: "The Game of Life, also known simply as Life,
 * is a cellular automaton devised by the British mathematician John Horton Conway in
1970."

 Given a board with m by n cells, each cell has an initial state live (1) or dead (0).
```

```
 Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the
 following four rules (taken from the above Wikipedia article):

 Any live cell with fewer than two live neighbors dies, as if caused by under-population.
 Any live cell with two or three live neighbors lives on to the next generation.
 Any live cell with more than three live neighbors dies, as if by over-population..
 Any dead cell with exactly three live neighbors becomes a live cell, as if by
reproduction.
 Write a function to compute the next state (after one update) of the board given its
current state.

 Follow up:
 Could you solve it in-place? Remember that the board needs to be updated at the same
time:
 You cannot update some cells first and then use their updated values to update other
cells.
 In this question, we represent the board using a 2D array.
 In principle, the board is infinite, which would cause problems when the active area
encroaches the
 border of the array. How would you address these problems?*/

public class _289 {
    public static class Solution1 {
        public void gameOfLife(int[][] board) {
            int height = board.length;
            int width = board[0].length;
            int[][] next = new int[height][width];
            int[][] directions = {{-1, -1}, {-1, 0}, {-1, 1}, {0, 1}, {1, 1}, {1, 0}, {1,
-1}, {0, -1}};

            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    int liveCellsCount = 0;
                    //count all its live cells

                    for (int[] dir : directions) {
                        int x = i + dir[0];
                        int y = j + dir[1];
                        if (x >= 0 && y >= 0 && x < height && y < width && board[x][y] ==
1) {
                            liveCellsCount++;
                        }
                    }

                    if (board[i][j] == 1) {
                        if (liveCellsCount <= 3 && liveCellsCount >= 2) {
                            next[i][j] = 1;
                        }
                    } else if (board[i][j] == 0) {
                        if (liveCellsCount == 3) {
                            next[i][j] = 1;
                        }
                    }
                }
```

```
            }

            for (int i = 0; i < board.length; i++) {
                for (int j = 0; j < board[0].length; j++) {
                    board[i][j] = next[i][j];
                }
            }
        }
    }
}

// ============================================================
// FILE 273: _290.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 290. Word Pattern
 *
 * Given a pattern and a string str, find if str follows the same pattern.
 * Here follow means a full match, such that there is a bijection between a letter in
pattern and a non-empty word in str.

 Examples:
 pattern = "abba", str = "dog cat cat dog" should return true.
 pattern = "abba", str = "dog cat cat fish" should return false.
 pattern = "aaaa", str = "dog cat cat dog" should return false.
 pattern = "abba", str = "dog dog dog dog" should return false.

 Notes:
 You may assume pattern contains only lowercase letters, and str contains lowercase
letters separated by a single space.
 */

public class _290 {

    public static class Solution1 {
        public boolean wordPattern(String pattern, String str) {
            String[] words = str.split(" ");
            char[] patterns = pattern.toCharArray();
            Map<Character, String> map = new HashMap();
            if (patterns.length != words.length) {
                return false;
            }
            for (int i = 0; i < patterns.length; i++) {
                if (map.containsKey(patterns[i])) {
                    if (!map.get(patterns[i]).equals(words[i])) {
                        return false;
                    }
                } else {
```

```
                    if (map.containsValue(words[i])) {
                        return false;//this is for this case: "abba", "dog dog dog dog"
                    }
                    map.put(patterns[i], words[i]);
                }
            }
            return true;
        }
    }

}

// ============================================================
// FILE 274: _291.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 291. Word Pattern II
 *
 * Given a pattern and a string str, find if str follows the same pattern.
 * Here follow means a full match, such that there is a bijection between a letter in
pattern and a non-empty substring in str.

 Examples:
 pattern = "abab", str = "redblueredblue" should return true.
 pattern = "aaaa", str = "asdasdasdasd" should return true.
 pattern = "aabb", str = "xyzabcxzyabc" should return false.

 Notes:
 You may assume both pattern and str contains only lowercase letters.
 */
public class _291 {

    public static class Solution1 {
        /**
         * We can try recursively:
         * say pattern is "abab", str is "redblueredblue"
         * first we try if "a" matches with "r", "b" matches with "e", we find it's not,
so we try to see if "b" matches "ed", and so on ...
         * then eventually, we find this pattern:
         * "a" matches "red"
         * "b" matches "blue"
         * then we'll just finish the str check based on this pattern
         * */
        public boolean wordPatternMatch(String pattern, String str) {
            Map<Character, String> map = new HashMap();
            Set<String> set = new HashSet();
```

```
            return isMatch(str, 0, pattern, 0, map, set);
        }

        private boolean isMatch(String str, int i, String pattern, int j, Map<Character,
String> map, Set<String> set) {
            //base case
            if (i == str.length() && j == pattern.length()) {
                return true;
            }
            if (i == str.length() || j == pattern.length()) {
                return false;
            }

            char c = pattern.charAt(j);

            if (map.containsKey(c)) {
                String s = map.get(c);

                //check to see if we can use s to match str.substring(i, i + s.length())
                if (!str.startsWith(s, i)) {
                    return false;
                }

                //if it's match, great, then let's check the rest
                return isMatch(str, i + s.length(), pattern, j + 1, map, set);
            }

            for (int k = i; k < str.length(); k++) {
                String p = str.substring(i, k + 1);

                if (set.contains(p)) {
                    continue;
                }

                map.put(c, p);
                set.add(p);

                //continue to match the rest
                if (isMatch(str, k + 1, pattern, j + 1, map, set)) {
                    return true;
                }

                //backtracking
                map.remove(c);
                set.remove(p);
            }

            //we've tried everything, but still no luck
            return false;
        }
    }
}

// ==========================================================
```

```
// FILE 275: _292.java
// ============================================================

package com.fishercoder.solutions;

/**You are playing the following Nim Game with your friend:
 * There is a heap of stones on the table, each time one of you take turns to remove 1 to
3 stones.
 * The one who removes the last stone will be the winner. You will take the first turn to
remove the stones.
 * Both of you are very clever and have optimal strategies for the game.
 * Write a function to determine whether you can win the game given the number of stones
in the heap.
 * For example, if there are 4 stones in the heap, then you will never win the game: no
matter 1, 2, or 3 stones you remove,
 * the last stone will always be removed by your friend.

 Hint:
 If there are 5 stones in the heap, could you figure out a way to remove the stones such
that you will always be the winner?*/

public class _292 {

    public static class Solution1 {
        /**
         * 1. If there are only 1 or 2 or 3 stones, you could always win by taking 1 or 2
or 3 stones;
         * 2. If there are 4 stones, you could never win because no matter you tak 1 or 2
or 3 stones, you could never take the 4th one;
         * 3. If there are 5 or 6 or 7 stones, you could always win because no matter how
your opponent works, you'll always get the last one;
         * 4. Then we could deduce that as long as the number is not divisible by 4, you
could always win.
         */

        public boolean canWinNim(int n) {
            return n % 4 != 0;
        }
    }
}

// ============================================================
// FILE 276: _293.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
/**
 * 293. Flip Game
 *
 * You are playing the following Flip Game with your friend: Given a string that contains
only
```

```
 * these two characters: + and -, you and your friend take turns to flip two consecutive
"++" into "--".
 * The game ends when a person can no longer make a move and therefore the other person
will be the winner.
 Write a function to compute all possible states of the string after one valid move.
 For example, given s = "++++", after one move, it may become one of the following
states:

 [
 "--++",
 "+--+",
 "++--"
 ]
 If there is no valid move, return an empty list [].*/
public class _293 {

    public static class Solutoin1 {
        public List<String> generatePossibleNextMoves(String s) {
            List<String> result = new ArrayList<>();
            for (int i = 1; i < s.length(); i++) {
                if (s.charAt(i) == '+' && s.charAt(i - 1) == '+') {
                    result.add(s.substring(0, i - 1) + "--" + s.substring(i + 1));
                }
            }
            return result;
        }
    }
}

// ================================================================
// FILE 277: _294.java
// ================================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * You are playing the following Flip Game with your friend:
 * Given a string that contains only these two characters: + and -,
 * you and your friend take turns to flip two consecutive "++" into "--".
 * The game ends when a person can no longer make a move and therefore the other person
will be the winner.

 Write a function to determine if the starting player can guarantee a win.

 For example, given s = "++++", return true. The starting player can guarantee a win by
flipping the middle "++" to become "+--+".

 Follow up:
 Derive your algorithm's runtime complexity.
 */
public class _294 {
```

```
    public static class Solution1 {
        public boolean canWin(String s) {
            List<String> res = new ArrayList<>();
            char[] charArray = s.toCharArray();
            for (int i = 0; i < s.length() - 1; i++) {
                if (charArray[i] == '+' && charArray[i + 1] == '+') {
                    //change these two bits to '-'
                    charArray[i] = '-';
                    charArray[i + 1] = '-';
                    res.add(String.valueOf(charArray));
                    //change these two bits back to '+' for its next move
                    charArray[i] = '+';
                    charArray[i + 1] = '+';
                }
            }
            /**The above part is the same of Flip Game I.
             * The only added part is the following piece of logic (so-called
backtracking.)*/
            for (String str : res) {
                if (!canWin(str)) {
                    return true;
                }
            }
            return false;
        }
    }

}

// ============================================================
// FILE 278: _295.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Collections;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 295. Find Median from Data Stream
 *
 * Median is the middle value in an ordered integer list.
 * If the size of the list is even, there is no middle value.
 * So the median is the mean of the two middle value.

 Examples:
 [2,3,4] , the median is 3

 [2,3], the median is (2 + 3) / 2 = 2.5

 Design a data structure that supports the following two operations:
```

```
 void addNum(int num) - Add a integer number from the data stream to the data structure.
 double findMedian() - Return the median of all elements so far.
 For example:

 addNum(1)
 addNum(2)
 findMedian() -> 1.5
 addNum(3)
 findMedian() -> 2
 */
public class _295 {
    /**A few key points for both following solutions:
     *
     * 1. always keep one queue one element more than the other if the number is odd,
offer into that one
     * first, then poll from that queue and offer into the other queue, then check
whether that queue is smaller
     * in size than the other, if so, poll one from the other queue and offer it into
this queue
     *
     * 2. only need to check whether this bigger queue size is greater than the other
queue when returning.*/

    public static class Solution1 {
        public static class MedianFinder {
            private Queue<Long> large;
            private Queue<Long> small;

            public MedianFinder() {
                large = new PriorityQueue<>();
                small = new PriorityQueue<>(Collections.reverseOrder());
            }

            // Adds a number into the data structure.
            public void addNum(int num) {
                large.offer((long) num);
                small.offer(large.poll());
                if (large.size() < small.size()) {
                    large.offer(small.poll());
                }
            }

            // Returns the median of current data stream
            public double findMedian() {
                if (large.size() > small.size()) {
                    return large.peek();
                }
                return (large.peek() + small.peek()) / 2.0;
            }
        }
    }

    public static class Solution2 {
        public static class MedianFinder {
```

```
            /**
             * credit: https://discuss.leetcode.com/topic/27521/short-simple-java-c-
python-o-log-n-o-1
             * The idea is for sure to use two heaps, one is max heap, one is min heap,
we always let the max heap be one element
             * bigger than min heap if the total number of elements is not even.
             * we could always get the median in O(1) time.
             * 1. use Long type to avoid overflow
             * 2. negate the numbers for small heap to save the effort for writing a
reverse comparator, brilliant!
             */

            private Queue<Long> large;
            private Queue<Long> small;

            /**
             * initialize your data structure here.
             */
            public MedianFinder() {
                large = new PriorityQueue<>();
                small = new PriorityQueue<>();
            }

            // Adds a number into the data structure.
            public void addNum(int num) {
                large.offer((long) num);
                small.offer(-large.poll());
                if (large.size() < small.size()) {
                    large.offer(-small.poll());
                }
            }

            // Returns the median of current data stream
            public double findMedian() {
                if (large.size() > small.size()) {
                    return large.peek();
                }
                return (large.peek() - small.peek()) / 2.0;
            }

        }
    }
}

// ============================================================
// FILE 279: _296.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
```

```
/**
 * 296: Best Meeting Point
 *
 * A group of two or more people wants to meet and minimize the total travel distance.
 * You are given a 2D grid of values 0 or 1, where each 1 marks the home of someone in
the group.
 * The distance is calculated using Manhattan Distance, where distance(p1, p2) = |p2.x -
p1.x| + |p2.y - p1.y|.

 For example, given three people living at (0,0), (0,4), and (2,2):

 1 - 0 - 0 - 0 - 1
 |   |   |   |   |
 0 - 0 - 0 - 0 - 0
 |   |   |   |   |
 0 - 0 - 1 - 0 - 0
 The point (0,2) is an ideal meeting point, as the total travel distance of 2+2+2=6 is
minimal. So return 6.
 */
public class _296 {
        public static class Solution1 {
                public int minTotalDistance(int[][] grid) {
                        int m = grid.length;
                        int n = grid[0].length;

                        List<Integer> I = new ArrayList(m);
                        List<Integer> J = new ArrayList(n);

                        for (int i = 0; i < m; i++) {
                                for (int j = 0; j < n; j++) {
                                        if (grid[i][j] == 1) {
                                                I.add(i);
                                                J.add(j);
                                        }
                                }
                        }

                        return getMin(I) + getMin(J);
                }

                private int getMin(List<Integer> list) {
                        int ret = 0;

                        Collections.sort(list);

                        int i = 0;
                        int j = list.size() - 1;
                        while (i < j) {
                                ret += list.get(j--) - list.get(i++);
                        }

                        return ret;
                }
        }
```

```
}

// ============================================================
// FILE 280: _297.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 297. Serialize and Deserialize Binary Tree
 *
 * Serialization is the process of converting a data structure or object into a sequence
of bits
 * so that it can be stored in a file or memory buffer,
 * or transmitted across a network connection link to be reconstructed later in the same
or another computer environment.
 * Design an algorithm to serialize and deserialize a binary tree.
 * There is no restriction on how your serialization/deserialization algorithm should
work.
 * You just need to ensure that a binary tree can be serialized to a string and this
string can
 * be deserialized to the original tree structure.

 For example, you may serialize the following tree

     1
    / \
   2   3
  / \
 4   5

 as "[1,2,3,null,null,4,5]",
 just the same as how LeetCode OJ serializes a binary tree.
 You do not necessarily need to follow this format, so please be creative and come up
with different approaches yourself.

 Note: Do not use class member/global/static variables to store states. Your serialize
and deserialize algorithms should be stateless.
 */
public class _297 {

    public static class Solution1 {
        /**
         * The idea is very straightforward:
         * use "#" as the terminator, do BFS, level order traversal to store all nodes
values into a StringBuilder.
         * When deserializing, also use a queue: pop the root into the queue first, then
use a for loop to construct each node,
         * then eventually just return the root.
```

```
         */

        // Encodes a tree to a single string.
        public String serialize(TreeNode root) {
            if (root == null) {
                return "";
            }

            StringBuilder sb = new StringBuilder();
            Queue<TreeNode> queue = new LinkedList();
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (curr == null) {
                        sb.append("# ");
                        continue;
                    }
                    sb.append(curr.val);
                    sb.append(" ");
                    queue.offer(curr.left);
                    queue.offer(curr.right);
                }
            }
            return sb.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.isEmpty()) {
                return null;
            }

            String[] nodes = data.split(" ");
            TreeNode root = new TreeNode(Integer.valueOf(nodes[0]));
            Queue<TreeNode> queue = new LinkedList();
            queue.offer(root);
            for (int i = 1; i < nodes.length; i++) {
                TreeNode curr = queue.poll();
                if (!nodes[i].equals("#")) {
                    curr.left = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.left);
                }
                if (!nodes[++i].equals("#")) {
                    curr.right = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.right);
                }
            }
            return root;
        }
    }
}
```

```
// ============================================================
// FILE 281: _298.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * Given a binary tree, find the length of the longest consecutive sequence path.

 The path refers to any sequence of nodes from some starting node to any node in the tree
along the parent-child connections.
 The longest consecutive path need to be from parent to child (cannot be the reverse).

 For example,
   1
   \
   3
  / \
 2  4
     \
     5
 Longest consecutive sequence path is 3-4-5, so return 3.

   2
    \
    3
   /
  2
 /
1
 Longest consecutive sequence path is 2-3,not3-2-1, so return 2.
 */
public class _298 {

    public static class Solution1 {
        private int max = 1;

        public int longestConsecutive(TreeNode root) {
            if (root == null) {
                return 0;
            }
            dfs(root, 0, root.val);
            return max;
        }

        private void dfs(TreeNode root, int curr, int target) {
            if (root == null) {
                return;
            }
            if (root.val == target) {
                curr++;
            } else {
```

```
                curr = 1;
            }
            max = Math.max(max, curr);
            dfs(root.left, curr, root.val + 1);
            dfs(root.right, curr, root.val + 1);
        }
    }
}

// ============================================================
// FILE 282: _299.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 299. Bulls and Cows
 *
 * You are playing the following Bulls and Cows game with your friend:
 * You write down a number and ask your friend to guess what the number is.
 * Each time your friend makes a guess, you provide a hint that indicates how many digits
in said guess match your secret number exactly in
 * both digit and position (called "bulls") and how many digits match the secret number
but locate in the wrong position (called "cows").
 * Your friend will use successive guesses and hints to eventually derive the secret
number.

 For example:

 Secret number:  "1807"
 Friend's guess: "7810"
 Hint: 1 bull and 3 cows. (The bull is 8, the cows are 0, 1 and 7.)
 Write a function to return a hint according to the secret number and friend's guess, use
A to indicate the bulls and B to indicate the cows. In the above example, your function
should return "1A3B".

 Please note that both secret number and friend's guess may contain duplicate digits, for
example:

 Secret number:  "1123"
 Friend's guess: "0111"
 In this case, the 1st 1 in friend's guess is a bull, the 2nd or 3rd 1 is a cow, and your
function should return "1A1B".
 You may assume that the secret number and your friend's guess only contain digits, and
their lengths are always equal.*/
public class _299 {
    public static class Solution1 {
        public String getHint(String secret, String guess) {
            int[] secretCows = new int[10];
            int[] guessCows = new int[10];
            int bulls = 0;
            for (int i = 0; i < secret.length(); i++) {
                if (guess.charAt(i) == secret.charAt(i)) {
                    bulls++;
```

```
                } else {
                    secretCows[Character.getNumericValue(secret.charAt(i))]++;
                    guessCows[Character.getNumericValue(guess.charAt(i))]++;
                }
            }
            int cows = 0;
            for (int i = 0; i < 11; i++) {
                cows += Math.min(secretCows[i], guessCows[i]);
            }
            return bulls + "A" + cows + "B";
        }
    }
}

// ============================================================
// FILE 283: _300.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 300. Longest Increasing Subsequence
 *
 * Given an unsorted array of integers, find the length of longest increasing
subsequence.
 For example,
 Given [10, 9, 2, 5, 3, 7, 101, 18],
 The longest increasing subsequence is [2, 3, 7, 101], therefore the length is 4.
 Note that there may be more than one LIS combination, it is only necessary for you to
return the length.

 Your algorithm should run in O(n2) complexity.

 Follow up: Could you improve it to O(nlogn) time complexity?
 */
public class _300 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/28719/short-java-solution-using-dp-
o-n-log-n
         * The idea is that as you iterate the sequence,
         * you keep track of the minimum value a subsequence of given length might end
with,
         * for all so far possible subsequence lengths.
         * So dp[i] is the minimum value a subsequence of length i+1 might end with.
         * Having this info, for each new number we iterate to,
         * we can determine the longest subsequence where it can be appended using binary
search.
         * The final answer is the length of the longest subsequence we found so far.
         */
```

```
        public int lengthOfLIS(int[] nums) {
            int[] dp = new int[nums.length];
            int len = 0;
            for (int x : nums) {
                /**Java Doc of this binarySearch API:
                 * @return index of the search key, if it is contained in the array
                 *         within the specified range;
                 *         otherwise, <tt>(-(<i>insertion point</i>) - 1)</tt>.  The
                 *         <i>insertion point</i> is defined as the point at which the
                 *         key would be inserted into the array: the index of the first
                 *         element in the range greater than the key,
                 *         or <tt>toIndex</tt> if all
                 *         elements in the range are less than the specified key.  Note
                 *         that this guarantees that the return value will be &gt;= 0 if
                 *         and only if the key is found.*/
                int index = Arrays.binarySearch(dp, 0, len, x);
                if (index < 0) {
                    index = -(index + 1);
                }
                dp[index] = x;
                if (index == len) {
                    len++;
                }
            }
            return len;
        }
    }
}

// ============================================================
// FILE 284: _301.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.Set;

/**
 * 301. Remove Invalid Parentheses
 *
 * Remove the minimum number of invalid parentheses in order to make the input string
valid. Return all possible results.

 Note: The input string may contain letters other than the parentheses ( and ).

 Examples:
 "()())()" -> ["()()()", "(())()"]
 "(a)())()" -> ["(a)()()", "(a())()"]
 ")(" -> [""]
```

```
 */
public class _301 {

	public static class Solution1 {

		public List<String> removeInvalidParentheses(String s) {
			List<String> result = new ArrayList<>();
			if (s == null) {
				return result;
			}

			Set<String> visited = new HashSet();
			Queue<String> q = new LinkedList();

			q.offer(s);
			visited.add(s);

			boolean found = false;

			while (!q.isEmpty()) {
				String curr = q.poll();
				if (isValid(curr)) {
					found = true;
					result.add(curr);
				}

				if (found) {
					continue;//this means if the initial input is
already a valid one, we'll just directly return it and there's actually only one valid
result
				}

				for (int i = 0; i < curr.length(); i++) {
					if (curr.charAt(i) != '(' && curr.charAt(i) !=
')') {
						continue;//this is to rule out those non-
parentheses characters
					}

					String next = curr.substring(0, i) +
curr.substring(i + 1);
					if (!visited.contains(next)) {
						q.offer(next);
						visited.add(next);
					}
				}

			}
			return result;
		}

		private boolean isValid(String str) {
			char[] chars = str.toCharArray();
			int count = 0;
```

```
                        for (int i = 0; i < chars.length; i++) {
                                char c = chars[i];
                                if (c == '(') {
                                        count++;
                                }
                                if (c == ')') {
                                        count--;
                                        if (count == -1) {
                                                return false;
                                        }
                                }
                        }
                        return count == 0;
                }
        }
}

// ============================================================
// FILE 285: _302.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 302. Smallest Rectangle Enclosing Black Pixels
 *
 * An image is represented by a binary matrix with 0 as a white pixel and 1 as a black
pixel.
 * The black pixels are connected, i.e., there is only one black region.
 * Pixels are connected horizontally and vertically.
 * Given the location (x, y) of one of the black pixels, return the area of the smallest
(axis-aligned) rectangle that encloses all black pixels.

 For example, given the following image:

 [
 "0010",
 "0110",
 "0100"
 ]
 and x = 0, y = 2,
 Return 6.
 */
public class _302 {
    public static class Solution1 {
        private char[][] image;

        public int minArea(char[][] iImage, int x, int y) {
            image = iImage;
            int m = image.length;
            int n = image[0].length;
            int left = searchColumns(0, y, 0, m, true);
            int right = searchColumns(y + 1, n, 0, m, false);
            int top = searchRows(0, x, left, right, true);
```

```
            int bottom = searchRows(x + 1, m, left, right, false);
            return (right - left) * (bottom - top);
        }

        private int searchColumns(int i, int j, int top, int bottom, boolean opt) {
            while (i != j) {
                int k = top;
                int mid = (i + j) / 2;
                while (k < bottom && image[k][mid] == '0') {
                    ++k;
                }
                if (k < bottom == opt) {
                    j = mid;
                } else {
                    i = mid + 1;
                }
            }
            return i;
        }

        private int searchRows(int i, int j, int left, int right, boolean opt) {
            while (i != j) {
                int k = left;
                int mid = (i + j) / 2;
                while (k < right && image[mid][k] == '0') {
                    ++k;
                }
                if (k < right == opt) {
                    j = mid;
                } else {
                    i = mid + 1;
                }
            }
            return i;
        }
    }
}

// ============================================================
// FILE 286: _303.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 303. Range Sum Query - Immutable
 * Given an integer array nums, find the sum of the elements between indices i and j (i ≤
j), inclusive.

 Example:
 Given nums = [-2, 0, 3, -5, 2, -1]

 sumRange(0, 2) -> 1
 sumRange(2, 5) -> -1
```

```
 sumRange(0, 5) -> -3

 Note:
 You may assume that the array does not change.
 There are many calls to sumRange function.
 */
public class _303 {
    public static class NumArray {
        int[] sums;

        public NumArray(int[] nums) {
            sums = new int[nums.length];
            for (int i = 0; i < nums.length; i++) {
                if (i == 0) {
                    sums[i] = nums[i];
                } else {
                    sums[i] = sums[i - 1] + nums[i];
                }
            }
        }

        public int sumRange(int i, int j) {
            if (i == 0) {
                return sums[j];
            }
            return sums[j] - sums[i - 1];
        }
    }
}

// Your NumArray object will be instantiated and called as such:
// NumArray numArray = new NumArray(nums);
// numArray.sumRange(0, 1);
// numArray.sumRange(1, 2);

// ============================================================
// FILE 287: _304.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 304. Range Sum Query 2D - Immutable
 *
 * Given a 2D matrix matrix, find the sum of the elements inside the rectangle defined by
its upper left corner (row1, col1) and lower right corner (row2, col2).

 Range Sum Query 2D
 The above rectangle (with the red border) is defined by (row1, col1) = (2, 1) and (row2,
col2) = (4, 3), which contains sum = 8.

 Example:
 Given matrix = [
 [3, 0, 1, 4, 2],
```

```
 [5, 6, 3, 2, 1],
 [1, 2, 0, 1, 5],
 [4, 1, 0, 1, 7],
 [1, 0, 3, 0, 5]
 ]

 sumRegion(2, 1, 4, 3) -> 8
 sumRegion(1, 1, 2, 2) -> 11
 sumRegion(1, 2, 2, 4) -> 12
 Note:
 You may assume that the matrix does not change.
 There are many calls to sumRegion function.
 You may assume that row1 ≤ row2 and col1 ≤ col2.
 */
public class _304 {

    public static class Solution1 {
        public class NumMatrix {

            public NumMatrix(int[][] matrix) {
                if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
                    return;
                }

                /**The dimensions of this tot matrix is actually 1 bigger than the given
matrix, cool!*/
                tot = new int[matrix.length + 1][matrix[0].length + 1];
                for (int i = 0; i < matrix.length; i++) {
                    for (int j = 0; j < matrix[0].length; j++) {
                        tot[i + 1][j + 1] =
                                matrix[i][j] + tot[i + 1][j] + tot[i][j + 1] - tot[i][j];
                    }
                }
            }

            public int sumRegion(int row1, int col1, int row2, int col2) {
                return tot[row2 + 1][col2 + 1] - tot[row2 + 1][col1] - tot[row1][col2 +
1]
                        + tot[row1][col1];
            }

            int[][] tot;
        }
    }
/**
 * Your NumMatrix object will be instantiated and called as such:
 * NumMatrix obj = new NumMatrix(matrix);
 * int param_1 = obj.sumRegion(row1,col1,row2,col2);
 */
}

// ============================================================
// FILE 288: _305.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 305. Number of Islands II
 *
 * A 2d grid map of m rows and n columns is initially filled with water.
 * We may perform an addLand operation which turns the water at position (row, col) into
a land.
 * Given a list of positions to operate, count the number of islands after each addLand
operation.
 * An island is surrounded by water and is formed by connecting adjacent lands
horizontally or vertically.
 * You may assume all four edges of the grid are all surrounded by water.

 Example:

 Given m = 3, n = 3, positions = [[0,0], [0,1], [1,2], [2,1]].
 Initially, the 2d grid grid is filled with water. (Assume 0 represents water and 1
represents land).

 0 0 0
 0 0 0
 0 0 0
 Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land.

 1 0 0
 0 0 0   Number of islands = 1
 0 0 0
 Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land.

 1 1 0
 0 0 0   Number of islands = 1
 0 0 0
 Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land.

 1 1 0
 0 0 1   Number of islands = 2
 0 0 0
 Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land.

 1 1 0
 0 0 1   Number of islands = 3
 0 1 0
 We return the result as an array: [1, 1, 2, 3]

 Challenge:

 Can you do it in time complexity O(k log mn), where k is the length of the positions?
 */
public class _305 {
```

```
public static class Solution1 {

    public int find(int[] father, int id) {
        int tf = father[id];
        while (tf != father[tf]) {
            tf = father[tf];
        }
        int cur = id;
        int tmp;
        while (father[cur] != tf) {
            tmp = father[cur];
            father[cur] = tf;
            cur = tmp;
        }
        return tf;
    }

    public void union(int[] father, int[] sz, int id1, int id2) {
        int tf1 = find(father, id1);
        int tf2 = find(father, id2);
        if (tf1 != tf2) {
            if (sz[tf1] > sz[tf2]) {
                father[tf2] = tf1;
                sz[tf1] += sz[tf2];
            } else {
                father[tf1] = tf2;
                sz[tf2] += sz[tf1];
            }
        }
    }

    public List<Integer> numIslands2(int m, int n, int[][] positions) {
        if (m == 0 || n == 0) {
            return new ArrayList<>();
        }
        ArrayList<Integer> res = new ArrayList();
        int[] father = new int[m * n];
        for (int i = 0; i < father.length; i++) {
            father[i] = -1;
        }
        int[] sz = new int[m * n];
        int[] dr = {0, 0, -1, 1};
        int[] dc = {-1, 1, 0, 0};
        int r;
        int c;
        int nr;
        int nc;
        int count = 0;
        for (int i = 0; i < positions.length; i++) {
            r = positions[i][0];
            c = positions[i][1];
            count++;
            father[r * n + c] = r * n + c;
            sz[r * n + c] = 1;
```

```
                for (int j = 0; j < 4; j++) {
                    nr = r + dr[j];
                    nc = c + dc[j];
                    if (nr >= 0 && nr < m && nc >= 0 && nc < n && father[nr * n + nc] !=
-1) {
                        if (find(father, r * n + c) != find(father, nr * n + nc)) {
                            count--;
                            union(father, sz, r * n + c, nr * n + nc);
                        }
                    }
                }
                res.add(count);
            }
            return res;
        }
    }
}

// ============================================================
// FILE 289: _306.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 306. Additive Number
 *
 * Additive number is a string whose digits can form additive sequence.

 A valid additive sequence should contain at least three numbers.
 Except for the first two numbers, each subsequent number in the sequence must be the sum
of the preceding two.

 For example:
 "112358" is an additive number because the digits can form an additive sequence: 1, 1,
2, 3, 5, 8.

 1 + 1 = 2, 1 + 2 = 3, 2 + 3 = 5, 3 + 5 = 8
 "199100199" is also an additive number, the additive sequence is: 1, 99, 100, 199.
 1 + 99 = 100, 99 + 100 = 199
 Note: Numbers in the additive sequence cannot have leading zeros, so sequence 1, 2, 03
or 1, 02, 3 is invalid.

 Given a string containing only digits '0'-'9', write a function to determine if it's an
additive number.

 Follow up:
 How would you handle overflow for very large input integers?
 */
public class _306 {
    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/29856/java-recursive-and-
iterative-solutions/2 */
        public boolean isAdditiveNumber(String num) {
```

```
            int n = num.length();
            for (int i = 1; i <= n / 2; ++i) {
                for (int j = 1; Math.max(j, i) <= n - i - j; ++j) {
                    if (isValid(i, j, num)) {
                        return true;
                    }
                }
            }
            return false;
        }

        private boolean isValid(int i, int j, String num) {
            if (num.charAt(0) == '0' && i > 1) {
                return false;
            }
            if (num.charAt(i) == '0' && j > 1) {
                return false;
            }
            String sum;
            Long x1 = Long.parseLong(num.substring(0, i));
            Long x2 = Long.parseLong(num.substring(i, i + j));
            for (int start = i + j; start != num.length(); start += sum.length()) {
                x2 = x2 + x1;
                x1 = x2 - x1;
                sum = x2.toString();
                if (!num.startsWith(sum, start)) {
                    return false;
                }
            }
            return true;
        }
    }

}

// ==========================================================
// FILE 290: _307.java
// ==========================================================

package com.fishercoder.solutions;

/**
 * 307. Range Sum Query - Mutable
 *
 * Given an integer array nums, find the sum of the elements between indices i and j (i ≤
j), inclusive.

 The update(i, val) function modifies nums by updating the element at index i to val.
 Example:
 Given nums = [1, 3, 5]

 sumRange(0, 2) -> 9
 update(1, 2)
 sumRange(0, 2) -> 8
```

```
 Note:
 The array is only modifiable by the update function.
 You may assume the number of calls to update and sumRange function is distributed
evenly.
 */
public class _307 {

        public static class Solution1 {
                class NumArray {
                        class SegmentTreeNode {
                                SegmentTreeNode left;
                                SegmentTreeNode right;
                                int start;
                                int end;
                                int sum;

                                public SegmentTreeNode(int start, int end) {
                                        this.start = start;
                                        this.end = end;
                                        this.left = null;
                                        this.right = null;
                                        this.sum = 0;
                                }
                        }

                        private SegmentTreeNode root = null;

                        public NumArray(int[] nums) {
                                root = buildSegmentTree(nums, 0, nums.length - 1);
                        }

                        SegmentTreeNode buildSegmentTree(int[] nums, int start, int end)
{
                                if (start > end) {
                                        return null;
                                } else {
                                        SegmentTreeNode root = new SegmentTreeNode(start,
end);
                                        if (start == end) {
                                                root.sum = nums[start];
                                        } else {
                                                int mid = start + (end - start) / 2;
                                                root.left = buildSegmentTree(nums, start,
mid);
                                                root.right = buildSegmentTree(nums, mid +
1, end);
                                                root.sum = root.left.sum +
root.right.sum;
                                        }
                                        return root;
                                }
                        }

                        void update(int i, int val) {
```

```
                        update(root, i, val);
                }

                void update(SegmentTreeNode root, int pos, int val) {
                        if (root.start == root.end) {
                                root.sum = val;
                        } else {
                                int mid = root.start + (root.end - root.start) /
2;
                                if (pos <= mid) {
                                        update(root.left, pos, val);
                                } else {
                                        update(root.right, pos, val);
                                }
                                root.sum = root.left.sum + root.right.sum;
                        }
                }

                public int sumRange(int i, int j) {
                        return sumRange(root, i, j);
                }

                int sumRange(SegmentTreeNode root, int start, int end) {
                        if (root.end == end && root.start == start) {
                                return root.sum;
                        } else {
                                int mid = root.start + (root.end - root.start) /
2;
                                if (end <= mid) {
                                        return sumRange(root.left, start, end);
                                } else if (start >= mid + 1) {
                                        return sumRange(root.right, start, end);
                                } else {
                                        return sumRange(root.right, mid + 1, end)
+ sumRange(root.left, start, mid);
                                }
                        }
                }
        }
}


// ============================================================
// FILE 291: _308.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 308. Range Sum Query 2D - Mutable
 *
 * Given a 2D matrix matrix,
 * find the sum of the elements inside the rectangle defined by its upper left corner
```

```
(row1, col1) and lower right corner (row2, col2).

 Range Sum Query 2D
 The above rectangle (with the red border) is defined by (row1, col1) = (2, 1) and (row2,
col2) = (4, 3), which contains sum = 8.

 Example:
 Given matrix = [
 [3, 0, 1, 4, 2],
 [5, 6, 3, 2, 1],
 [1, 2, 0, 1, 5],
 [4, 1, 0, 1, 7],
 [1, 0, 3, 0, 5]
 ]

 sumRegion(2, 1, 4, 3) -> 8
 update(3, 2, 2)
 sumRegion(2, 1, 4, 3) -> 10
 Note:
 The matrix is only modifiable by the update function.
 You may assume the number of calls to update and sumRegion function is distributed
evenly.
 You may assume that row1 ≤ row2 and col1 ≤ col2.
 */
public class _308 {
    public static class Solution1 {
        public class NumMatrix {
            int[][] nums;
            int[][] tree;
            int height;
            int width;

            public NumMatrix(int[][] matrix) {
                if (matrix.length == 0 || matrix[0].length == 0) {
                    return;
                }
                height = matrix.length;
                width = matrix[0].length;
                this.nums = new int[height][width];
                this.tree = new int[height + 1][width + 1];
                for (int i = 0; i < height; i++) {
                    for (int j = 0; j < width; j++) {
                        update(i, j, matrix[i][j]);
                    }
                }
            }

            public void update(int rowIndex, int colIndex, int newVal) {
                if (height == 0 || width == 0) {
                    return;
                }
                int delta = newVal - nums[rowIndex][colIndex];
                nums[rowIndex][colIndex] = newVal;
                for (int i = rowIndex + 1; i <= height; i += i & (-i)) {
```

```
                    for (int j = colIndex + 1; j <= width; j += j & (-j)) {
                        tree[i][j] += delta;//just use its previous value plus delta is
good
                    }
                }
            }

            public int sumRegion(int row1, int col1, int row2, int col2) {
                if (height == 0 || width == 0) {
                    return 0;
                }
                return sum(row2 + 1, col2 + 1) + sum(row1, col1) - sum(row1, col2 + 1) -
sum(
                    row2 + 1, col1);
            }

            private int sum(int row, int col) {
                int sum = 0;
                for (int i = row; i > 0; i -= i & (-i)) {
                    for (int j = col; j > 0; j -= j & (-j)) {
                        sum += tree[i][j];
                    }
                }
                return sum;
            }
        }

        /**
         * Your NumMatrix object will be instantiated and called as such:
         * NumMatrix obj = new NumMatrix(matrix);
         * obj.update(row,col,val);
         * int param_2 = obj.sumRegion(row1,col1,row2,col2);
         */
    }
}

// ============================================================
// FILE 292: _309.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 309. Best Time to Buy and Sell Stock with Cooldown
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.

 Design an algorithm to find the maximum profit. You may complete as many transactions as
you like (ie, buy one and sell one share of the stock multiple times) with the following
restrictions:

 You may not engage in multiple transactions at the same time (ie, you must sell the
stock before you buy again).
```

```
 After you sell your stock, you cannot buy stock on next day. (ie, cooldown 1 day)
 Example:

 prices = [1, 2, 3, 0, 2]
 maxProfit = 3
 transactions = [buy, sell, cooldown, buy, sell]
 */
public class _309 {
    public static class Solution1 {
        /**
         * The series of problems are typical dp. The key for dp is to find the variables
to
         * represent the states and deduce the transition function.
         *
         * Of course one may come up with a O(1) space solution directly, but I think it
is better
         * to be generous when you think and be greedy when you implement.
         *
         * The natural states for this problem is the 3 possible transactions : buy,
sell, rest.
         * Here rest means no transaction on that day (aka cooldown).
         *
         * Then the transaction sequences can end with any of these three states.
         *
         * For each of them we make an array, buy[n], sell[n] and rest[n].
         *
         * buy[i] means before day i what is the maxProfit for any sequence end with buy.
         *
         * sell[i] means before day i what is the maxProfit for any sequence end with
sell.
         *
         * rest[i] means before day i what is the maxProfit for any sequence end with
rest.
         *
         * Then we want to deduce the transition functions for buy sell and rest. By
definition we
         * have:
         *
         * buy[i] = max(rest[i-1]-price, buy[i-1])
         * sell[i] = max(buy[i-1]+price, sell[i-1])
         * rest[i] = max(sell[i-1], buy[i-1], rest[i-1])
         *
         * Where price is the price of day i. All of these are very straightforward. They
simply represents :
         *
         * (1) We have to `rest` before we `buy` and
         * (2) we have to `buy` before we `sell`
         * One tricky point is how do you make sure you sell before you buy, since from
the equations it seems that [buy, rest, buy] is entirely possible.
         *
         * Well, the answer lies within the fact that buy[i] <= rest[i] which means
rest[i] =
         * max(sell[i-1], rest[i-1]). That made sure [buy, rest, buy] is never occurred.
         *
```

```
         * A further observation is that and rest[i] <= sell[i] is also true therefore
         *
         * rest[i] = sell[i-1] Substitute this in to buy[i] we now have 2 functions
instead of 3:
         *
         * buy[i] = max(sell[i-2]-price, buy[i-1]) sell[i] = max(buy[i-1]+price, sell[i-
1]) This is
         * better than 3, but
         *
         * we can do even better
         *
         * Since states of day i relies only on i-1 and i-2 we can reduce the O(n) space
to O(1).
         * And here we are at our final solution:
         */
        public int maxProfit(int[] prices) {
            int sell = 0;
            int prevSell = 0;
            int buy = Integer.MIN_VALUE;
            int prevBuy;
            for (int price : prices) {
                prevBuy = buy;
                buy = Math.max(prevSell - price, prevBuy);
                prevSell = sell;
                sell = Math.max(prevBuy + price, prevSell);
            }
            return sell;
        }
    }

    public static class Solution2 {
        /**Surprisingly, this solution is even much faster than the one above provided by
the author.*/
        /**
         * Here I share my no brainer weapon when it comes to this kind of problems.
         *
         * 1. Define States
         *
         * To represent the decision at index i:
         *
         * buy[i]: Max profit till index i. The series of transaction is ending with a
buy.
         * sell[i]: Max profit till index i. The series of transaction is ending with a
sell.
         *
         * 2. Define Recursion
         *
         * buy[i]: To make a decision whether to buy at i, we either take a rest, by just
using the
         * old decision at i - 1, or sell at/before i - 2, then buy at i, We cannot sell
at i - 1,
         * then buy at i, because of cooldown.
         * sell[i]: To make a decision whether to sell at i, we either take a rest, by
just using the old decision at i - 1,
```

```
         * or buy at/before i - 1, then sell at i.
         *
         * So we get the following formula:
         *
         * buy[i] = Math.max(buy[i - 1], sell[i - 2] - prices[i]);
         * sell[i] = Math.max(sell[i - 1], buy[i - 1] + prices[i]);
         *
         * 3. Optimize to O(1) Space
         *
         * DP solution only depending on i - 1 and i - 2 can be optimized using O(1)
space.
         *
         * Let b2, b1, b0 represent buy[i - 2], buy[i - 1], buy[i]
         * Let s2, s1, s0 represent sell[i - 2], sell[i - 1], sell[i]
         *
         * Then arrays turn into Fibonacci like recursion:
         *
         * b0 = Math.max(b1, s2 - prices[i]);
         * s0 = Math.max(s1, b1 + prices[i]);
         *
         * 4. Write Code in 5 Minutes
         *
         * First we define the initial states at i = 0:
         *
         * We can buy. The max profit at i = 0 ending with a buy is -prices[0].
         * We cannot sell. The max profit at i = 0 ending with a sell is 0.
         */
        public int maxProfit(int[] prices) {
            if (prices == null || prices.length <= 1) {
                return 0;
            }

            int b0 = -prices[0];
            int b1 = b0;
            int b2 = b0;
            int s0 = 0;
            int s1 = s0;
            int s2 = s0;

            for (int i = 1; i < prices.length; i++) {
                b0 = Math.max(b1, s2 - prices[i]);
                s0 = Math.max(s1, b1 + prices[i]);
                b2 = b1;
                b1 = b0;
                s2 = s1;
                s1 = s0;
            }
            return s0;
        }
    }
}

// ============================================================
// FILE 293: _310.java
```

```
// ============================================================

package com.fishercoder.solutions;


import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 310. Minimum Height Trees
 *
 * For a undirected graph with tree characteristics, we can choose any node as the root.
 * The result graph is then a rooted tree. Among all possible rooted trees, those with
minimum height are called minimum height trees (MHTs).
 * Given such a graph, write a function to find all the MHTs and return a list of their
root labels.

 Format
 The graph contains n nodes which are labeled from 0 to n - 1. You will be given the
number n and a list of undirected edges (each edge is a pair of labels).

 You can assume that no duplicate edges will appear in edges. Since all edges are
undirected, [0, 1] is the same as [1, 0] and thus will not appear together in edges.

 Example 1:

 Given n = 4, edges = [[1, 0], [1, 2], [1, 3]]

   0
   |
   1
  / \
 2   3
 return [1]

 Example 2:

 Given n = 6, edges = [[0, 3], [1, 3], [2, 3], [4, 3], [5, 4]]

 0  1  2
  \ | /
    3
    |
    4
    |
    5
 return [3, 4]

 Hint:

 How many MHTs can a graph have at most?
```

```
 Note:

 (1) According to the definition of tree on Wikipedia: “a tree is an undirected graph in
which any two vertices are connected by exactly one path. In other words, any connected
graph without simple cycles is a tree.”

 (2) The height of a rooted tree is the number of edges on the longest downward path
between the root and a leaf.
 */
public class _310 {

    public static class Solution1 {
        public List<Integer> findMinHeightTrees(int n, int[][] edges) {
            if (n == 1) {
                return Collections.singletonList(0);
            }

            List<Set<Integer>> adj = new ArrayList<>(n);
            for (int i = 0; i < n; ++i) {
                adj.add(new HashSet<>());
            }
            for (int[] edge : edges) {
                adj.get(edge[0]).add(edge[1]);
                adj.get(edge[1]).add(edge[0]);
            }

            List<Integer> leaves = new ArrayList<>();
            for (int i = 0; i < n; ++i) {
                if (adj.get(i).size() == 1) {
                    leaves.add(i);
                }
            }

            while (n > 2) {
                n -= leaves.size();
                List<Integer> newLeaves = new ArrayList<>();
                for (int i : leaves) {
                    int j = adj.get(i).iterator().next();
                    adj.get(j).remove(i);
                    if (adj.get(j).size() == 1) {
                        newLeaves.add(j);
                    }
                }
                leaves = newLeaves;
            }
            return leaves;
        }
    }
}

// ============================================================
// FILE 294: _311.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 311. Sparse Matrix Multiplication
 *
 * Given two sparse matrices A and B, return the result of AB.

 You may assume that A's column number is equal to B's row number.

 Example:

 A = [
 [ 1, 0, 0],
 [-1, 0, 3]
 ]

 B = [
 [ 7, 0, 0 ],
 [ 0, 0, 0 ],
 [ 0, 0, 1 ]
 ]

      |  1 0 0 |   | 7 0 0 |   |  7 0 0 |
 AB = | -1 0 3 | x | 0 0 0 | = | -7 0 3 |
                   | 0 0 1 |

 */
public class _311 {

    public static class Solution1 {
        public int[][] multiply(int[][] A, int[][] B) {
            int m = A.length;
            int n = A[0].length;
            int p = B[0].length;
            int[][] C = new int[m][p];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (A[i][j] != 0) {
                        for (int k = 0; k < p; k++) {
                            if (B[j][k] != 0) {
                                C[i][k] += A[i][j] * B[j][k];
                            }
                        }
                    }
                }
            }
            return C;
        }
    }
}

// ============================================================
// FILE 295: _312.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 312. Burst Balloons
 *
 * Given n balloons, indexed from 0 to n-1.
 * Each balloon is painted with a number on it represented by array nums.
 * You are asked to burst all the balloons.
 * If the you burst balloon i you will get nums[left] * nums[i] * nums[right] coins.
 * Here left and right are adjacent indices of i. After the burst, the left and right
then becomes adjacent.

 Find the maximum coins you can collect by bursting the balloons wisely.

 Note:
 (1) You may imagine nums[-1] = nums[n] = 1. They are not real therefore you can not
burst them.
 (2) 0 ≤ n ≤ 500, 0 ≤ nums[i] ≤ 100

 Example:

 Given [3, 1, 5, 8]

 Return 167

 nums = [3,1,5,8] --> [3,5,8] -->   [3,8]   -->  [8]  --> []
 coins =  3*1*5      +  3*5*8    +  1*3*8      + 1*8*1   = 167
 */
public class _312 {

    public static class Solution1 {
        public int maxCoins(int[] iNums) {
            int[] nums = new int[iNums.length + 2];
            int n = 1;
            for (int x : iNums) {
                if (x > 0) {
                    nums[n++] = x;
                }
            }
            nums[0] = nums[n++] = 1;

            int[][] dp = new int[n][n];
            for (int k = 2; k < n; ++k) {
                for (int left = 0; left < n - k; ++left) {
                    int right = left + k;
                    for (int i = left + 1; i < right; ++i) {
                        dp[left][right] = Math.max(dp[left][right],
                            nums[left] * nums[i] * nums[right] + dp[left][i] + dp[i]
[right]);
                    }
                }
            }
            return dp[0][n - 1];
```

```
        }
    }
}

// ============================================================
// FILE 296: _313.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 313. Super Ugly Number
 *
 * Write a program to find the nth super ugly number.
 * Super ugly numbers are positive numbers whose all prime factors
 * are in the given prime list primes of size k.
 * For example, [1, 2, 4, 7, 8, 13, 14, 16, 19, 26, 28, 32]
 * is the sequence of the first 12 super ugly numbers given primes = [2, 7, 13, 19] of
size 4.
 *
 * Note:
 * (1) 1 is a super ugly number for any given primes.
 * (2) The given numbers in primes are in ascending order.
 * (3) 0 < k ≤ 100, 0 < n ≤ 106, 0 < primes[i] < 1000.
 * (4) The nth super ugly number is guaranteed to fit in a 32-bit signed integer.
 */
public class _313 {

    public static class Solution1 {
        public int nthSuperUglyNumber(int n, int[] primes) {
            int[] ret = new int[n];
            ret[0] = 1;

            int[] indexes = new int[primes.length];

            for (int i = 1; i < n; i++) {
                ret[i] = Integer.MAX_VALUE;

                for (int j = 0; j < primes.length; j++) {
                    ret[i] = Math.min(ret[i], primes[j] * ret[indexes[j]]);
                }

                for (int j = 0; j < indexes.length; j++) {
                    if (ret[i] == primes[j] * ret[indexes[j]]) {
                        indexes[j]++;
                    }
                }
            }
            return ret[n - 1];
        }
    }

}
```

```
// ============================================================
// FILE 297: _314.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.TreeMap;

/**
 * 314. Binary Tree Vertical Order Traversal
 *
 * Given a binary tree, return the vertical order traversal of its nodes' values. (ie,
from top to bottom, column by column).

 If two nodes are in the same row and column, the order should be from left to right.

 Examples:

 Given binary tree [3,9,20,null,null,15,7],
    3
   /\
  /  \
 9  20
    /\
   /  \
  15   7
 return its vertical order traversal as:
 [
 [9],
 [3,15],
 [20],
 [7]
 ]

 Given binary tree [3,9,8,4,0,1,7],
    3
   /\
  /  \
 9   8
/\   /\
/  \/  \
4  01   7
 return its vertical order traversal as:
 [
 [4],
 [9],
 [3,0,1],
```

```
 [8],
 [7]
 ]

 Given binary tree [3,9,8,4,0,1,7,null,null,null,2,5] (0's right child is 2 and 1's left
child is 5),
      3
     /\
    /  \
   9   8
  /\  /\
 /  \/  \
4  01   7
   /\
  /  \
 5   2
 return its vertical order traversal as:
 [
 [4],
 [9,5],
 [3,0,1],
 [8,2],
 [7]
 ]
 */
public class _314 {
    public static class Solution1 {
        public List<List<Integer>> verticalOrder(TreeNode root) {
            List<List<Integer>> result = new ArrayList();
            if (root == null) {
                return result;
            }
            Queue<TreeNode> bfsQ = new LinkedList();
            Queue<Integer> indexQ = new LinkedList();
            TreeMap<Integer, List<Integer>> map = new TreeMap();
            bfsQ.offer(root);
            indexQ.offer(
                0);//we set the root as index 0, left will be negative, right will be
positive
            while (!bfsQ.isEmpty()) {
                int qSize = bfsQ.size();
                for (int i = 0; i < qSize; i++) {
                    TreeNode curr = bfsQ.poll();
                    int index = indexQ.poll();
                    if (map.containsKey(index)) {
                        map.get(index).add(curr.val);
                    } else if (!map.containsKey(index)) {
                        List<Integer> list = new ArrayList();
                        list.add(curr.val);
                        map.put(index, list);
                    }
                    if (curr.left != null) {
                        bfsQ.offer(curr.left);
                        indexQ.offer(index - 1);
```

```
                }
                if (curr.right != null) {
                    bfsQ.offer(curr.right);
                    indexQ.offer(index + 1);
                }
            }
        }
        for (int i : map.keySet()) {
            result.add(map.get(i));
        }
        return result;
    }
}

public static class Solution2 {
    public List<List<Integer>> verticalOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList();
        if (root == null) {
            return result;
        }
        Queue<TreeNode> bfsQ = new LinkedList();
        Queue<Integer> indexQ = new LinkedList();
        HashMap<Integer, List<Integer>> map = new HashMap();
        bfsQ.offer(root);
        indexQ.offer(
            0);//we set the root as index 0, left will be negative, right will be
positive
        int min = 0;
        int max = 0;
        while (!bfsQ.isEmpty()) {
            int qSize = bfsQ.size();
            for (int i = 0; i < qSize; i++) {
                TreeNode curr = bfsQ.poll();
                int index = indexQ.poll();
                if (map.containsKey(index)) {
                    map.get(index).add(curr.val);
                } else if (!map.containsKey(index)) {
                    List<Integer> list = new ArrayList();
                    list.add(curr.val);
                    map.put(index, list);
                }
                if (curr.left != null) {
                    bfsQ.offer(curr.left);
                    indexQ.offer(index - 1);
                    min = Math.min(min, index - 1);
                }
                if (curr.right != null) {
                    bfsQ.offer(curr.right);
                    indexQ.offer(index + 1);
                    max = Math.max(max, index + 1);
                }
            }
        }
        for (int i = min; i <= max; i++) {
```

```
                result.add(map.get(i));
            }
            return result;
        }
    }

}

// ============================================================
// FILE 298: _315.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.List;

/**
 * 315. Count of Smaller Numbers After Self
 *
 * You are given an integer array nums and you have to return a new counts array.
 * The counts array has the property where counts[i] is the number of smaller elements to
the right of nums[i].

 Example:

 Given nums = [5, 2, 6, 1]

 To the right of 5 there are 2 smaller elements (2 and 1).
 To the right of 2 there is only 1 smaller element (1).
 To the right of 6 there is 1 smaller element (1).
 To the right of 1 there is 0 smaller element.
 Return the array [2, 1, 1, 0].

 */
public class _315 {

    public static class Solution1 {
        class Node {
            int val;
            int sum;
            int dup = 1;
            Node left;
            Node right;

            public Node(int v, int s) {
                this.val = v;
                this.sum = s;
            }
        }

        public List<Integer> countSmaller(int[] nums) {
            Integer[] ans = new Integer[nums.length];
            Node root = null;
```

```
            for (int i = nums.length - 1; i >= 0; i--) {
                root = insertNode(nums[i], root, i, 0, ans);
            }
            return Arrays.asList(ans);
        }

        Node insertNode(int val, Node node, int i, int prevSum, Integer[] ans) {
            if (node == null) {
                node = new Node(val, 0);
                ans[i] = prevSum;
            } else if (val == node.val) {
                node.dup += 1;
                ans[i] = prevSum + node.sum;
            } else if (val > node.val) {
                node.right = insertNode(val, node.right, i, prevSum + node.sum +
node.dup, ans);
            } else {
                node.sum += 1;
                node.left = insertNode(val, node.left, i, prevSum, ans);
            }

            return node;
        }
    }
}

// ============================================================
// FILE 299: _316.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 316. Remove Duplicate Letters
 *
 * Given a string which contains only lowercase letters,
 * remove duplicate letters so that every letter appear once and only once.
 * You must make sure your result is the smallest in lexicographical order among all
possible results.

 Example:
 Given "bcabc"
 Return "abc"

 Given "cbacdcbc"
 Return "acdb"
 */
public class _316 {
    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/32259/java-solution-using-stack-
with-comments/2 */
```

```
        public String removeDuplicateLetters(String s) {
            int[] res = new int[26]; //will contain number of occurences of character
(i+'a')
            boolean[] visited =
                new boolean[26]; //will contain if character (i+'a') is present in
current result Stack
            char[] ch = s.toCharArray();
            for (char c : ch) {  //count number of occurences of character
                res[c - 'a']++;
            }
            Deque<Character> st = new ArrayDeque<>(); // answer stack
            int index;
            for (char c : ch) {
                index = c - 'a';
                res[index]--;   //decrement number of characters remaining in the string
to be analysed
                if (visited[index]) {
                    //if character is already present in stack, dont bother
                    continue;
                }
                //if current character is smaller than last character in stack which
occurs later in the string again
                //it can be removed and  added later e.g stack = bc remaining string abc
then a can pop b and then c
                while (!st.isEmpty() && c < st.peek() && res[st.peek() - 'a'] != 0) {
                    visited[st.pop() - 'a'] = false;
                }
                st.push(c); //add current character and mark it as visited
                visited[index] = true;
            }

            StringBuilder sb = new StringBuilder();
            //pop character from stack and build answer string from back
            while (!st.isEmpty()) {
                sb.insert(0, st.pop());
            }
            return sb.toString();
        }
    }

    public static class Solution2 {
        /**
         * Credit: https://discuss.leetcode.com/topic/31404/a-short-o-n-recursive-greedy-
solution
         */
        public String removeDuplicateLetters(String s) {
            int[] count = new int[26];
            int pos = 0; // the position for the smallest s[i]
            for (int i = 0; i < s.length(); i++) {
                count[s.charAt(i) - 'a']++;
            }
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) < s.charAt(pos)) {
                    pos = i;
```

```
                }
                count[s.charAt(i) - 'a']--;
                if (count[s.charAt(i) - 'a'] == 0) {
                    break;
                }
            }
            return s.length() == 0 ? "" : s.charAt(pos) + removeDuplicateLetters(
                s.substring(pos + 1).replaceAll("" + s.charAt(pos), ""));
        }
    }
}

// ============================================================
// FILE 300: _317.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 317. Shortest Distance from All Buildings
 *
 * You want to build a house on an empty land which reaches all buildings in the shortest amount of distance.
 * You can only move up, down, left and right. You are given a 2D grid of values 0, 1 or 2, where:

 Each 0 marks an empty land which you can pass by freely.
 Each 1 marks a building which you cannot pass through.
 Each 2 marks an obstacle which you cannot pass through.

 For example, given three buildings at (0,0), (0,4), (2,2), and an obstacle at (0,2):

 1 - 0 - 2 - 0 - 1
 |   |   |   |   |
 0 - 0 - 0 - 0 - 0
 |   |   |   |   |
 0 - 0 - 1 - 0 - 0

 The point (1,2) is an ideal empty land to build a house, as the total travel distance of 3+3+1=7 is minimal. So return 7.

 Note:
 There will be at least one building. If it is not possible to build such house according to the above rules, return -1.
 */

public class _317 {
    public static class Solution1 {
        public int shortestDistance(int[][] grid) {
            int m = grid.length;
            if (m == 0) {
```

```
                return -1;
            }
            int n = grid[0].length;
            int[][] reach = new int[m][n];
            int[][] distance = new int[m][n];
            int[] shift = new int[] {0, 1, 0, -1,
                0};//how these five elements is ordered is important since it denotes the
neighbor of the current node
            int numBuilding = 0;

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        numBuilding++;
                        int dist = 1;
                        boolean[][] visited = new boolean[m][n];

                        Queue<int[]> q = new LinkedList<int[]>();
                        q.offer(new int[] {i, j});
                        while (!q.isEmpty()) {
                            int size = q.size();
                            for (int l = 0; l < size; l++) {
                                int[] current = q.poll();
                                for (int k = 0; k < 4; k++) {
                                    int nextRow = current[0] + shift[k];
                                    int nextCol = current[1] + shift[k + 1];
                                    if (nextRow >= 0
                                        && nextRow < m
                                        && nextCol >= 0
                                        && nextCol < n
                                        && !visited[nextRow][nextCol]
                                        && grid[nextRow][nextCol] == 0) {
                                        distance[nextRow][nextCol] += dist;
                                        visited[nextRow][nextCol] = true;
                                        reach[nextRow][nextCol]++;
                                        q.offer(new int[] {nextRow, nextCol});
                                    }
                                }
                            }
                            dist++;
                        }
                    }
                }
            }

            int result = Integer.MAX_VALUE;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 0 && reach[i][j] == numBuilding && distance[i][j] <
result) {
                        result = distance[i][j];
                    }
                }
            }
```

```
            return result == Integer.MAX_VALUE ? -1 : result;
        }
    }

}

// ============================================================
// FILE 301: _318.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**318. Maximum Product of Word Lengths
 *
Given a string array words, find the maximum value of length(word[i]) * length(word[j])
where the two words do not share common letters. You may assume that each word will
contain only lower case letters. If no such two words exist, return 0.

Example 1:
Given ["abcw", "baz", "foo", "bar", "xtfn", "abcdef"]
Return 16
The two words can be "abcw", "xtfn".

Example 2:
Given ["a", "ab", "abc", "d", "cd", "bcd", "abcd"]
Return 4
The two words can be "ab", "cd".

Example 3:
Given ["a", "aa", "aaa", "aaaa"]
Return 0
No such pair of words.*/
public class _318 {
    public static class Solution1 {
        //Inspired by this awesome post: https://discuss.leetcode.com/topic/35539/java-
easy-version-to-understand
        //Idea: this question states that all words consisted of lower case (total only
26 unique chars),
        //this is a big hint that we could use integer (total 32 bits) to represent each
char
        //values[i] means how many unique characters this string words[i] has
        public int maxProduct(String[] words) {
            if (words == null || words.length == 0) {
                return 0;
            }
            int len = words.length;
            int[] values = new int[len];
            for (int i = 0; i < words.length; i++) {
                String word = words[i];
                for (int j = 0; j < words[i].length(); j++) {
```

```
                    values[i] |= 1 << (word.charAt(j)
                        - 'a');//the reason for left shift by this number "word.charAt(j)
-'a'" is for 'a', otherwise 'a' - 'a' will be zero and 'a' will be missed out.
                }
            }
            int maxProduct = 0;
            for (int i = 0; i < words.length; i++) {
                for (int j = 0; j < words.length; j++) {
                    //check if values[i] AND values[j] equals to zero, this means they
share NO common chars
                    if ((values[i] & values[j]) == 0
                        && words[i].length() * words[j].length() > maxProduct) {
                        maxProduct = words[i].length() * words[j].length();
                    }
                }
            }
            return maxProduct;
        }
    }

    public static void main(String... strings) {
        _318 test = new _318();
        String[] words = new String[]{"abcw", "baz", "foo", "bar", "xtfn", "abcdef"};

        //The following is to understand what does left shift by 1 mean:
        //the tricky part is to understand how it's written for me:
        // "x << y" means left shift x by y bits
        //left shift is equivalent to multiplication of powers of 2, so "4 << 1" equals
to " 4 * 2^1"
        //similarly, "4 << 3" equals to "4 * 2^3" which equals "4 * 8"
        String sample = "f";
        int bits = 0;
        int shiftLeftByHowMany = 0;
        int shiftLeftResult = 0;
        for (int j = 0; j < sample.length(); j++) {
            shiftLeftByHowMany = sample.charAt(j) - 'a';
            shiftLeftResult = 1 << shiftLeftByHowMany;
            bits |= 1 << (sample.charAt(j) - 'a');
            //this means shift left 1 by "sample.charAt(j) -'a'" bits
            System.out.println("nonShiftLeft = " + shiftLeftByHowMany + "\tnonShiftLeft
binary form is: " + Integer.toBinaryString(shiftLeftByHowMany)
                    + "\nshiftLeft = " + shiftLeftResult + "\tshiftLeft binary form is: "
+ Integer.toBinaryString(shiftLeftResult)
                    + "\nbits = " + bits + "\tbits binary form is: " +
Integer.toBinaryString(bits));
            System.out.println(shiftLeftResult == (1 * Math.pow(2, shiftLeftByHowMany)));
        }

        //similarly, right shift is written like this: "x >> y", means shift x by y bits
        //4 >> 3 equals 4 * 2^3, see below:
        System.out.println(4 * 8 == (4 * Math.pow(2, 3)));
    }
}
```

```
// ============================================================
// FILE 302: _319.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 319. Bulb Switcher
 *
 * There are n bulbs that are initially off. You first turn on all the bulbs.
 * Then, you turn off every second bulb. On the third round,
 * you toggle every third bulb (turning on if it's off or turning off if it's on).
 * For the ith round, you toggle every i bulb. For the nth round, you only
 * toggle the last bulb. Find how many bulbs are on after n rounds.

 Example:

 Given n = 3.

 At first, the three bulbs are [off, off, off].
 After first round, the three bulbs are [on, on, on].
 After second round, the three bulbs are [on, off, on].
 After third round, the three bulbs are [on, off, off].

 So you should return 1, because there is only one bulb is on.
 */
public class _319 {

    public static class Solution1 {
        public int bulbSwitch(int n) {
            if (n < 2) {
                return n;
            }
            return (int) Math.sqrt(n);
        }
    }

}

// ============================================================
// FILE 303: _320.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 320. Generalized Abbreviation
 *
 * Write a function to generate the generalized abbreviations of a word.

 Example:
```

```
 Given word = "word", return the following list (order does not matter):
 ["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1",
"1o2", "2r1", "3d", "w3", "4"]
 */
public class _320 {

    public static class Solution1 {
        public List<String> generateAbbreviations(String word) {
            List<String> result = new ArrayList<>();
            backtrack(word, result, 0, "", 0);
            return result;
        }

        private void backtrack(String word, List<String> result, int position, String
current,
            int count) {
            if (position == word.length()) {
                if (count > 0) {
                    current += count;
                }
                result.add(current);
            } else {
                backtrack(word, result, position + 1, current, count + 1);
                backtrack(word, result, position + 1,
                    current + (count > 0 ? count : "") + word.charAt(position), 0);
            }
        }
    }

}

// ============================================================
// FILE 304: _321.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 321. Create Maximum Number
 *
 * Given two arrays of length m and n with digits 0-9 representing two numbers.
 * Create the maximum number of length k <= m + n from digits of the two.
 * The relative order of the digits from the same array must be preserved.
 * Return an array of the k digits. You should try to optimize your time and space
complexity.

 Example 1:
 nums1 = [3, 4, 6, 5]
 nums2 = [9, 1, 2, 5, 8, 3]
 k = 5
 return [9, 8, 6, 5, 3]

 Example 2:
```

```
 nums1 = [6, 7]
 nums2 = [6, 0, 4]
 k = 5
 return [6, 7, 6, 0, 4]

 Example 3:
 nums1 = [3, 9]
 nums2 = [8, 9]
 k = 3
 return [9, 8, 9]
 */
public class _321 {
    public static class Solution1 {
        public int[] maxNumber(int[] nums1, int[] nums2, int k) {
            int n = nums1.length;
            int m = nums2.length;
            int[] ans = new int[k];
            for (int i = Math.max(0, k - m); i <= k && i <= n; ++i) {
                //what is this and why?
                int[] candidate = merge(maxArray(nums1, i), maxArray(nums2, k - i), k);
                if (greater(candidate, 0, ans, 0)) {
                    ans = candidate;
                }
            }
            return ans;
        }

        private boolean greater(int[] nums1, int i, int[] nums2, int j) {
            while (i < nums1.length && j < nums2.length && nums1[i] == nums2[j]) {
                i++;
                j++;
            }
            return j == nums2.length || (i < nums1.length && nums1[i] > nums2[j]);
        }

        private int[] merge(int[] nums1, int[] nums2, int k) {
            int[] ans = new int[k];
            for (int i = 0, j = 0, r = 0; r < k; r++) {
                ans[r] = greater(nums1, i, nums2, j) ? nums1[i++] : nums2[j++];
            }
            return ans;
        }

        private int[] maxArray(int[] nums, int k) {
            int n = nums.length;
            int[] ans = new int[k];
            for (int i = 0, j = 0; i < n; i++) {
                while (n - i + j > k && j > 0 && ans[j - 1] < nums[i]) {
                    j--;
                }
                if (j < k) {
                    ans[j++] = nums[i];
                }
            }
```

```
            return ans;
        }
    }
}

// ============================================================
// FILE 305: _322.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 322. Coin Change
 *
 * You are given coins of different denominations and a total amount of money amount.
 * Write a function to compute the fewest number of coins that you need to make up that
amount.
 * If that amount of money cannot be made up by any combination of the coins, return -1.

 Example 1:
 coins = [1, 2, 5], amount = 11
 return 3 (11 = 5 + 5 + 1)

 Example 2:
 coins = [2], amount = 3
 return -1.

 Note:
 You may assume that you have an infinite number of each kind of coin.
 */
public class _322 {

    public static class Solution1 {
        public int coinChange(int[] coins, int amount) {
            if (amount < 1) {
                return 0;
            }
            int[] count = new int[amount];
            int result = helper(coins, amount, count);
            return result;
        }

        //remaining means the remaining coins after the last step;
        //count[remaining] means the minimum number of coins to sum up to remaining
        private int helper(int[] coins, int remaining, int[] count) {
            if (remaining < 0) {
                return -1;//not valid case, thus, per problem description, we should
return -1
            }
            if (remaining == 0) {
                return 0;//completed, this is also a base case for this recursive
function
```

```
            }
            if (count[remaining - 1] != 0) {
                return count[remaining - 1];//already computed, so reuse it.
            }
            int min = Integer.MAX_VALUE;
            for (int coin : coins) {
                int res = helper(coins, remaining - coin, count);
                if (res >= 0 && res < min) {
                    min = 1 + res;
                }
            }
            return count[remaining - 1] = (min == Integer.MAX_VALUE) ? -1 : min;
        }
    }

    public static class Solution2 {
        //dp solution
        public int coinChange(int[] coins, int amount) {
            int max = amount + 1;
            int[] dp = new int[max];
            Arrays.fill(dp, max);// initial the dp array with amount + 1 which is not
valid case.
            dp[0] = 0;//initial amount 0 = 0;
            for (int i = 1; i <= amount; i++) {
                for (int j = 0; j < coins.length; j++) {
                    if (coins[j] <= i) {
                        //the dp[coins[j]] will ba a valid case, then if dp[i - coins[j]]
is  valid
                        dp[i] = Math.min(dp[i], dp[i - coins[j]] + 1);
                        // then we update dp[i], otherwise dp[i] = max;
                    }
                }
            }
            return dp[amount] > amount ? -1 : dp[amount];
        }
    }

}

// ============================================================
// FILE 306: _323.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 323. Number of Connected Components in an Undirected Graph
 *
 *  Given n nodes labeled from 0 to n - 1 and a list of undirected edges (each edge is a
```

```
pair of nodes), write a function to find the number of connected components in an
undirected graph.

 Example 1:

 0          3
 |          |
 1 --- 2    4

 Given n = 5 and edges = [[0, 1], [1, 2], [3, 4]], return 2.

 Example 2:

 0           4
 |           |
 1 --- 2 --- 3

 Given n = 5 and edges = [[0, 1], [1, 2], [2, 3], [3, 4]], return 1.

 Note:
 You can assume that no duplicate edges will appear in edges. Since all edges are
undirected, [0, 1] is the same as [1, 0] and thus will not appear together in edges.
 */
public class _323 {
    public static class Solution1 {

        public int countComponents(int n, int[][] edges) {
            if (n <= 1) {
                return n;
            }

            List<List<Integer>> adList = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                adList.add(new ArrayList<>());
            }

            for (int[] edge : edges) {
                adList.get(edge[0]).add(edge[1]);
                adList.get(edge[1]).add(edge[0]);
            }

            for (List<Integer> list : adList) {
                for (int i : list) {
                    System.out.print(i + ", ");
                }
                System.out.println();
            }

            boolean[] visited = new boolean[n];
            int count = 0;
            for (int i = 0; i < n; i++) {
                if (!visited[i]) {
                    count++;
                    Queue<Integer> q = new LinkedList<>();
```

```
                    q.offer(i);
                    while (!q.isEmpty()) {
                        int index = q.poll();
                        visited[index] = true;
                        for (int j : adList.get(index)) {
                            if (!visited[j]) {
                                q.offer(j);
                            }
                        }
                    }
                }
            }

            return count;
        }
    }

}

// ============================================================
// FILE 307: _324.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Collections;
import java.util.PriorityQueue;

/**
 * 324. Wiggle Sort II
 *
 * Given an unsorted array nums, reorder it such that nums[0] < nums[1] > nums[2] <
nums[3]....

 Example:
 (1) Given nums = [1, 5, 1, 1, 6, 4], one possible answer is [1, 4, 1, 5, 1, 6].
 (2) Given nums = [1, 3, 2, 2, 3, 1], one possible answer is [2, 3, 1, 3, 1, 2].

 Note:
 You may assume all input has valid answer.

 Follow Up:
 Can you do it in O(n) time and/or in-place with O(1) extra space?
 */
public class _324 {

    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/41464/step-by-step-explanation-of-
index-mapping-in-java */
        public void wiggleSort(int[] nums) {
            int median = findKthLargest(nums, (nums.length + 1) / 2);
            int n = nums.length;

            int left = 0;
```

```
            int i = 0;
            int right = n - 1;

            while (i <= right) {

                if (nums[newIndex(i, n)] > median) {
                    swap(nums, newIndex(left++, n), newIndex(i++, n));
                } else if (nums[newIndex(i, n)] < median) {
                    swap(nums, newIndex(right--, n), newIndex(i, n));
                } else {
                    i++;
                }
            }
        }

        private int findKthLargest(int[] nums, int k) {
            PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
            for (int i : nums) {
                maxHeap.offer(i);
            }
            while (k-- > 1) {
                maxHeap.poll();
            }
            return maxHeap.poll();
        }

        private void swap(int[] nums, int i, int j) {
            int tmp = nums[i];
            nums[i] = nums[j];
            nums[j] = tmp;
        }

        private int newIndex(int index, int n) {
            return (1 + 2 * index) % (n | 1);
        }
    }
}

// ============================================================
// FILE 308: _325.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 325. Maximum Size Subarray Sum Equals k
 *
 * Given an array nums and a target value k, find the maximum length of a subarray that
sums to k. If there isn't one, return 0 instead.
 *
```

```
 * Note:
 * The sum of the entire nums array is guaranteed to fit within the 32-bit signed integer
range.
 *
 * Example 1:
 * Given nums = [1, -1, 5, -2, 3], k = 3,
 * return 4. (because the subarray [1, -1, 5, -2] sums to 3 and is the longest)
 *
 * Example 2:
 * Given nums = [-2, -1, 2, 1], k = 1,
 * return 2. (because the subarray [-1, 2] sums to 1 and is the longest)
 *
 * Follow Up:
 * Can you do it in O(n) time?*/

public class _325 {

    public static class Solution1 {
        public int maxSubArrayLen(int[] nums, int k) {
            Map<Integer, Integer> map = new HashMap();
            int sum = 0;
            int max = 0;
            for (int i = 0; i < nums.length; i++) {
                sum += nums[i];
                if (sum == k) {
                    max = i + 1;
                } else if (map.containsKey(sum - k)) {
                    max = Math.max(max, i - map.get(sum - k));
                }
                if (!map.containsKey(sum)) {
                    map.put(sum, i);
                }
            }
            return max;
        }
    }

}

// ============================================================
// FILE 309: _326.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 326. Power of Three
 *
 * Given an integer, write a function to determine if it is a power of three.
 *
 * Follow up:
 * Could you do it without using any loop / recursion?
*/
```

```
public class _326 {
    public static class Solution1 {
        //regular method that has a loop
        public boolean isPowerOfThree(int n) {
            if (n < 3 && n != 1) {
                return false;
            }
            while (n != 1) {
                if (n % 3 != 0) {
                    return false;
                }
                n /= 3;
            }
            return true;
        }
    }

    public static class Solution2 {
        //find the max possible integer that is a power of 3, then do modulor with this
number
        public boolean isPowerOfThree(int n) {
            return (n > 0 && 1162261467 % n == 0);
        }
    }

    public static class Solution3 {
        //Editorial solution: it's pretty elegant to use base conversion which can be
easily extended to any radix k
        //Idea: for a number in base 10, if it's power of 10, then it must be in this
format: 10, 100, 1000... with a leading one and all trailing zeros
        //similarly, if a number is power of 3, then in its base 3 format, it must be in
this format as well: 10, 100, 1000, 1000...
        //some Java built-in function could help us along the way:
        public boolean isPowerOfThree(int n) {
            return Integer.toString(n, 3).matches("^10*$");
        }
    }

}

// ============================================================
// FILE 310: _327.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 327. Count of Range Sum
 *
 * Given an integer array nums, return the number of range sums that lie in [lower,
upper] inclusive.
 * Range sum S(i, j) is defined as the sum of the elements in nums between indices i and
j (i ≤ j), inclusive.
```

```
 Note:
 A naive algorithm of O(n2) is trivial. You MUST do better than that.

 Example:
 Given nums = [-2, 5, -1], lower = -2, upper = 2, Return 3.
 The three ranges are : [0, 0], [2, 2], [0, 2] and their respective sums are: -2, -1, 2.
 */
public class _327 {

    public static class Solution1 {
        /**
         * Time: O(n^2)
         * This results in TLE on Leetcode by the last test case.
         */
        public int countRangeSum(int[] nums, int lower, int upper) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            long[] sums = new long[nums.length];
            sums[0] = nums[0];
            for (int i = 1; i < nums.length; i++) {
                sums[i] = sums[i - 1] + nums[i];
            }
            int count = 0;
            for (int i = 0; i < nums.length; i++) {
                if (nums[i] <= upper && nums[i] >= lower) {
                    count++;
                }
                for (int j = i + 1; j < nums.length; j++) {
                    long sum = sums[j] - (i > 0 ? sums[i - 1] : 0);
                    if (sum <= upper && sum >= lower) {
                        count++;
                    }
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        public int countRangeSum(int[] nums, int lower, int upper) {
            int n = nums.length;
            long[] sums = new long[n + 1];
            for (int i = 0; i < n; i++) {
                sums[i + 1] = sums[i] + nums[i];
            }
            return countWhileMergeSort(sums, 0, n + 1, lower, upper);
        }

        private int countWhileMergeSort(long[] sums, int start, int end, int lower, int
upper) {
            if (end - start <= 1) {
                return 0;
            }
```

```
            int mid = (start + end) / 2;
            int count = countWhileMergeSort(sums, start, mid, lower, upper) +
countWhileMergeSort(sums, mid, end, lower, upper);
            int j = mid;
            int k = mid;
            int t = mid;
            long[] cache = new long[end - start];
            for (int i = start, r = 0; i < mid; i++, r++) {
                while (k < end && sums[k] - sums[i] < lower) {
                    k++;
                }
                while (j < end && sums[j] - sums[i] <= upper) {
                    j++;
                }
                while (t < end && sums[t] < sums[i]) {
                    cache[r++] = sums[t++];
                }
                cache[r] = sums[i];
                count += j - k;
            }
            System.arraycopy(cache, 0, sums, start, t - start);
            return count;
        }
    }

}

// ============================================================
// FILE 311: _328.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 328. Odd Even Linked List
 *
 * Given a singly linked list, group all odd nodes together followed by the even nodes.
Please note here we are talking about the node number and not the value in the nodes.

 You should try to do it in place. The program should run in O(1) space complexity and
O(nodes) time complexity.

 Example:
 Given 1->2->3->4->5->NULL,
 return 1->3->5->2->4->NULL.

 Note:
 The relative order inside both the even and odd groups should remain as it was in the
input.
 The first node is considered odd, the second node even and so on ...
 */
public class _328 {
```

```
    public static class Solution1 {
        public ListNode oddEvenList(ListNode head) {
            if (head != null) {
                ListNode odd = head;
                ListNode even = head.next;
                ListNode evenHead = even;

                while (even != null && even.next != null) {
                    odd.next = odd.next.next;
                    even.next = even.next.next;
                    odd = odd.next;
                    even = even.next;
                }

                odd.next = evenHead;
            }
            return head;
        }
    }
}

// ============================================================
// FILE 312: _329.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 329. Longest Increasing Path in a Matrix
 *
 * Given an integer matrix, find the length of the longest increasing path.
 * From each cell, you can either move to four directions: left, right, up or down.
 * You may NOT move diagonally or move outside of the boundary (i.e. wrap-around is not
allowed).

 Example 1:

 nums = [
 [9,9,4],
 [6,6,8],
 [2,1,1]
 ]
 Return 4
 The longest increasing path is [1, 2, 6, 9].

 Example 2:

 nums = [
 [3,4,5],
 [3,2,6],
 [2,2,1]
 ]
 Return 4
```

```
 The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.
 */

public class _329 {

    public static class Solution1 {
        final int[] dirs = new int[]{0, 1, 0, -1, 0};

        public int longestIncreasingPath(int[][] matrix) {
            if (matrix == null || matrix.length == 0) {
                return 0;
            }
            int max = 0;
            int[][] cache = new int[matrix.length][matrix[0].length];
            for (int i = 0; i < matrix.length; i++) {
                for (int j = 0; j < matrix[0].length; j++) {
                    int len = dfs(matrix, i, j, cache);
                    max = Math.max(len, max);
                }
            }
            return max;
        }

        int dfs(int[][] matrix, int row, int col, int[][] cache) {
            if (cache[row][col] != 0) {
                return cache[row][col];
            }
            int max = 1;
            for (int i = 0; i < 4; i++) {
                int nextRow = row + dirs[i];
                int nextCol = col + dirs[i + 1];
                if (nextRow < 0 || nextRow >= matrix.length || nextCol < 0 || nextCol >=
matrix[0].length || matrix[nextRow][nextCol] <= matrix[row][col]) {
                    continue;
                }
                int len = 1 + dfs(matrix, nextRow, nextCol, cache);
                max = Math.max(max, len);
            }
            cache[row][col] = max;
            return max;
        }
    }

}

// ============================================================
// FILE 313: _330.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
```

```
/**
 * 330. Patching Array
 *
 * Given a sorted positive integer array nums and an integer n,
 * add/patch elements to the array such that any number in range [1, n]
 * inclusive can be formed by the sum of some elements in the array. Return the minimum
number of patches required.

 Example 1:
 nums = [1, 3], n = 6
 Return 1.

 Combinations of nums are [1], [3], [1,3], which form possible sums of: 1, 3, 4.
 Now if we add/patch 2 to nums, the combinations are: [1], [2], [3], [1,3], [2,3],
[1,2,3].
 Possible sums are 1, 2, 3, 4, 5, 6, which now covers the range [1, 6].
 So we only need 1 patch.

 Example 2:
 nums = [1, 5, 10], n = 20
 Return 2.
 The two patches can be [2, 4].

 Example 3:
 nums = [1, 2, 2], n = 5
 Return 0.
 */
public class _330 {

    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/patching-array/ and
https://discuss.leetcode.com/topic/35494/solution-explanation/2
         *
         * Let miss be the smallest sum in [0,n] that we might be missing. Meaning we
already know we
         * can build all sums in [0,miss). Then if we have a number num <= miss in the
given array, we
         * can add it to those smaller sums to build all sums in [0,miss+num). If we
don't, then we must
         * add such a number to the array, and it's best to add miss itself, to maximize
the reach.
         *
         * Example: Let's say the input is nums = [1, 2, 4, 13, 43] and n = 100. We need
to ensure that
         * all sums in the range [1,100] are possible. Using the given numbers 1, 2 and
4, we can
         * already build all sums from 0 to 7, i.e., the range [0,8). But we can't build
the sum 8, and
         * the next given number (13) is too large. So we insert 8 into the array. Then
we can build all
         * sums in [0,16). Do we need to insert 16 into the array? No! We can already
build the sum 3,
         * and adding the given 13 gives us sum 16. We can also add the 13 to the other
```

```
sums, extending
         * our range to [0,29). And so on. The given 43 is too large to help with sum 29,
so we must
         * insert 29 into our array. This extends our range to [0,58). But then the 43
becomes useful
         * and expands our range to [0,101). At which point we're done.
         */

        public int minPatches(int[] nums, int n) {
            long misses = 1;//use long to avoid integer addition overflow
            int patches = 0;
            int i = 0;
            while (misses <= n) {
                if (i < nums.length && nums[i] <= misses) { //miss is covered
                    misses += nums[i++];
                } else { //patch miss to the array
                    misses += misses;
                    patches++;//increase the answer
                }
            }
            return patches;
        }

        public List<Integer> findPatches(int[] nums, int n) {
            long misses = 1;//use long to avoid integer addition overflow
            List<Integer> patches = new ArrayList<>();
            int i = 0;
            while (misses <= n) {
                if (i < nums.length && nums[i] <= misses) { //miss is covered
                    misses += nums[i++];
                } else { //patch miss to the array
                    patches.add((int) misses);//increase the answer
                    misses += misses;
                }
            }
            return patches;
        }
    }

}

// ============================================================
// FILE 314: _331.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 331. Verify Preorder Serialization of a Binary Tree
 *
 * One way to serialize a binary tree is to use pre-order traversal. When we encounter a
```

```
non-null node, we record the node's value. If it is a null node, we record using a
sentinel value such as #.

       _9_
      /   \
     3     2
    / \   / \
   4  1  #  6
  / \ / \  / \
  # # # #  # #

 For example, the above binary tree can be serialized to the string
"9,3,4,#,#,1,#,#,2,#,6,#,#", where # represents a null node.

 Given a string of comma separated values, verify whether it is a correct preorder
traversal serialization of a binary tree. Find an algorithm without reconstructing the
tree.

 Each comma separated value in the string must be either an integer or a character '#'
representing null pointer.

 You may assume that the input format is always valid, for example it could never contain
two consecutive commas such as "1,,3".

 Example 1:
 "9,3,4,#,#,1,#,#,2,#,6,#,#"
 Return true

 Example 2:
 "1,#"
 Return false

 Example 3:
 "9,#,#,1"
 Return false


 */
public class _331 {

    public static class Solution1 {
        public boolean isValidSerialization(String preorder) {
            /**Idea: we keep inserting the string into the stack, if it's a number, we
just push it onto the stack;
             * if it's a "#", we see if the top of the stack is a "#" or not,
             * 1. if it's a "#", we pop it and keep popping numbers from the stack,
             * 2. if it's not a "#", we push the "#" onto the stack*/
            if (preorder == null || preorder.length() == 0) {
                return false;
            }
            String[] pre = preorder.split(",");
            Deque<String> stack = new ArrayDeque<>();
            for (int i = 0; i < pre.length; i++) {
                while (pre[i].equals("#") && !stack.isEmpty() &&
```

```
stack.peekLast().equals("#")) {
                    stack.pollLast();
                    if (stack.isEmpty()) {
                        return false;
                    }
                    stack.pollLast();
                }
                stack.addLast(pre[i]);
            }
            return stack.size() == 1 && stack.peekLast().equals("#");
        }
    }

}

// ============================================================
// FILE 315: _332.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * 332. Reconstruct Itinerary
 *
 * Given a list of airline tickets represented by pairs of departure and arrival airports
[from, to],
 * reconstruct the itinerary in order. All of the tickets belong to a man who departs
from JFK. Thus, the itinerary must begin with JFK.

 Note:
 If there are multiple valid itineraries, you should return the itinerary that has the
smallest lexical order when read as a single string.
 For example, the itinerary ["JFK", "LGA"] has a smaller lexical order than ["JFK",
"LGB"].

 All airports are represented by three capital letters (IATA code).

 You may assume all tickets form at least one valid itinerary.
 Example 1:
 tickets = [["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]
 Return ["JFK", "MUC", "LHR", "SFO", "SJC"].

 Example 2:
 tickets = [["JFK","SFO"],["JFK","ATL"],["SFO","ATL"],["ATL","JFK"],["ATL","SFO"]]
 Return ["JFK","ATL","JFK","SFO","ATL","SFO"].
```

```
 Another possible reconstruction is ["JFK","SFO","ATL","JFK","ATL","SFO"]. But it is
larger in lexical order.
 */
public class _332 {

    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/36383/share-my-solution */
        public List<String> findItinerary(String[][] tickets) {
            Map<String, PriorityQueue<String>> flights = new HashMap<>();
            LinkedList<String> path = new LinkedList<>();
            for (String[] ticket : tickets) {
                flights.putIfAbsent(ticket[0], new PriorityQueue<>());
                flights.get(ticket[0]).add(ticket[1]);
            }
            dfs("JFK", flights, path);
            return path;
        }

        public void dfs(String departure, Map<String, PriorityQueue<String>> flights,
            LinkedList path) {
            PriorityQueue<String> arrivals = flights.get(departure);
            while (arrivals != null && !arrivals.isEmpty()) {
                dfs(arrivals.poll(), flights, path);
            }
            path.addFirst(departure);
        }
    }
}

// ============================================================
// FILE 316: _333.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**333. Largest BST Subtree
 *
 * Given a binary tree, find the largest subtree which is a Binary Search Tree (BST),
 * where largest means subtree with largest number of nodes in it.

 Note:
 A subtree must include all of its descendants.
 Here's an example:
      10
     / \
    5  15
   / \   \
  1  8   7

 The Largest BST Subtree in this case is the highlighted one (5,1,8).
 The return value is the subtree's size, which is 3.
```

```
 Follow up:
 Can you figure out ways to solve it with O(n) time complexity?
 */

public class _333 {
    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/36995/share-my-o-n-java-code-with-
brief-explanation-and-comments*/
        class Result {  // (size, rangeLower, rangeUpper) -- size of current tree, range
of current tree [rangeLower, rangeUpper]
            int size;
            int lower;
            int upper;

            Result(int size, int lower, int upper) {
                this.size = size;
                this.lower = lower;
                this.upper = upper;
            }
        }

        int max = 0;

        public int largestBSTSubtree(TreeNode root) {
            if (root == null) {
                return 0;
            }
            traverse(root);
            return max;
        }

        private Result traverse(TreeNode root) {
            if (root == null) {
                return new Result(0, Integer.MAX_VALUE, Integer.MIN_VALUE);
            }
            Result left = traverse(root.left);
            Result right = traverse(root.right);
            if (left.size == -1 || right.size == -1 || root.val <= left.upper || root.val
>= right.lower) {
                return new Result(-1, 0, 0);
            }
            int size = left.size + 1 + right.size;
            max = Math.max(size, max);
            return new Result(size, Math.min(left.lower, root.val), Math.max(right.upper,
root.val));
        }
    }

    public int largestBSTSubtree(TreeNode root) {
        if (root == null) {
            return 0;
        }
        if (isBST(root)) {
            return getNodes(root);
```

```
        }
        return Math.max(find(root.left), find(root.right));
    }

    int find(TreeNode root) {
        if (isBST(root)) {
            return getNodes(root);
        }
        return Math.max(find(root.left), find(root.right));
    }

    int getNodes(TreeNode root) {
        if (root == null) {
            return 0;
        }
        return dfsCount(root);
    }

    int dfsCount(TreeNode root) {
        if (root == null) {
            return 0;
        }
        return dfsCount(root.left) + dfsCount(root.right) + 1;
    }

    boolean isBST(TreeNode root) {
        if (root == null) {
            return true;
        }
        return dfs(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    boolean dfs(TreeNode root, long min, long max) {
        if (root == null) {
            return true;
        }
        if (root.val <= min || root.val >= max) {
            return false;
        }
        return dfs(root.left, min, root.val) && dfs(root.right, root.val, max);
    }

}

// ============================================================
// FILE 317: _334.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 334. Increasing Triplet Subsequence
 *
 * Given an unsorted array return whether an increasing subsequence of length 3 exists or
```

```
not in the array.
 *
 * Formally the function should:
 * Return true if there exists i, j, k
 * such that arr[i] < arr[j] < arr[k] given 0 ≤ i < j < k ≤ n-1 else return false.
 * Your algorithm should run in O(n) time complexity and O(1) space complexity.

 Examples:
 Given [1, 2, 3, 4, 5],
 return true.

 Given [5, 4, 3, 2, 1],
 return false.
 */
public class _334 {

    public static class Solution1 {
        /**Time: O(n^2)
         * Space: O(1)*/
        public boolean increasingTriplet(int[] nums) {
            if (nums == null || nums.length == 0) {
                return false;
            }
            int small = nums[0];
            int medium = Integer.MAX_VALUE;
            for (int i = 1; i < nums.length; i++) {
                small = Math.min(small, nums[i - 1]);
                if (nums[i] > small) {
                    medium = Math.min(medium, nums[i]);
                    for (int j = i + 1; j < nums.length; j++) {
                        if (nums[j] > nums[i] || nums[j] > medium) {
                            return true;
                        }
                    }
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(n)
         * Space: O(1)
         */
        public boolean increasingTriplet(int[] nums) {
            int small = Integer.MAX_VALUE;
            int big = Integer.MAX_VALUE;
            for (int num : nums) {
                if (num <= small) {
                    small = num;
                } else if (num <= big) {
                    big = num;
                } else {
```

```
                    return true;
                }
            }
            return false;
        }
    }

}

// ============================================================
// FILE 318: _335.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 335. Self Crossing
 *
 * You are given an array x of n positive numbers.
 * You start at point (0,0) and moves x[0] metres to the north,
 * then x[1] metres to the west,
 * x[2] metres to the south,
 * x[3] metres to the east and so on.
 * In other words, after each move your direction changes counter-clockwise.

 Write a one-pass algorithm with O(1) extra space to determine, if your path crosses
itself, or not.

 Example 1:
 Given x =
 [2, 1, 1, 2]
 ,
 ?????
 ?   ?
 ???????>
 ?

 Return true (self crossing)
 Example 2:
 Given x =
 [1, 2, 3, 4]
 ,
 ????????
 ?      ?
 ?
 ?
 ?????????????>

 Return false (not self crossing)
 Example 3:
 Given x =
 [1, 1, 1, 1]
 ,
 ?????
```

```
 ?   ?
 ?????>

 Return true (self crossing)

 */
public class _335 {
    public static class Solution1 {
        /** reference: https://discuss.leetcode.com/topic/38014/java-oms-with-
explanation/2 */
        public boolean isSelfCrossing(int[] x) {
            int l = x.length;
            if (l <= 3) {
                return false;
            }

            for (int i = 3; i < l; i++) {
                if (x[i] >= x[i - 2] && x[i - 1] <= x[i - 3]) {
                    return true;  //Fourth line crosses first line and onward
                }
                if (i >= 4) {
                    if (x[i - 1] == x[i - 3] && x[i] + x[i - 4] >= x[i - 2]) {
                        return true; // Fifth line meets first line and onward
                    }
                }
                if (i >= 5) {
                    if (x[i - 2] - x[i - 4] >= 0
                        && x[i] >= x[i - 2] - x[i - 4]
                        && x[i - 1] >= x[i - 3] - x[i - 5]
                        && x[i - 1] <= x[i - 3]) {
                        return true;  // Sixth line crosses first line and onward
                    }
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 319: _336.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**336. Palindrome Pairs
 *
 * Given a list of unique words, find all pairs of distinct indices (i, j) in the given
```

```
list, so that the concatenation of the two words, i.e. words[i] + words[j] is a
palindrome.

 Example 1:
 Given words = ["bat", "tab", "cat"]
 Return [[0, 1], [1, 0]]
 The palindromes are ["battab", "tabbat"]
 Example 2:
 Given words = ["abcd", "dcba", "lls", "s", "sssll"]
 Return [[0, 1], [1, 0], [3, 2], [2, 4]]
 The palindromes are ["dcbaabcd", "abcddcba", "slls", "llssssll"]
 */
public class _336 {

    public static class Solution1 {
        public List<List<Integer>> palindromePairs(String[] words) {
            List<List<Integer>> pairs = new ArrayList();
            Map<String, Integer> map = new HashMap();
            for (int i = 0; i < words.length; i++) {
                map.put(words[i], i);
            }

            for (int i = 0; i < words.length; i++) {
                int l = 0;
                int r = 0;
                while (l <= r) {
                    String s = words[i].substring(l, r);
                    Integer j = map.get(new StringBuilder(s).reverse().toString());
                    if (j != null && j != i && isPalindrome(
                        words[i].substring(l == 0 ? r : 0, l == 0 ? words[i].length() :
l))) {
                        pairs.add(
                            Arrays.asList(l == 0 ? new Integer[] {i, j} : new Integer[]
{j, i}));
                    }
                    if (r < words[i].length()) {
                        r++;
                    } else {
                        l++;
                    }
                }
            }
            return pairs;
        }

        private boolean isPalindrome(String s) {
            for (int i = 0; i < s.length() / 2; i++) {
                if (s.charAt(i) != s.charAt(s.length() - 1 - i)) {
                    return false;
                }
            }
            return true;
        }
    }
```

```
}

// ============================================================
// FILE 320: _337.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**337. House Robber III

 The thief has found himself a new place for his thievery again.
 There is only one entrance to this area, called the "root."
 Besides the root, each house has one and only one parent house.
 After a tour, the smart thief realized that "all houses in this place forms a binary
tree".
 It will automatically contact the police if two directly-linked houses were broken into
on the same night.

Determine the maximum amount of money the thief can rob tonight without alerting the
police.

Example 1:
     3
    / \
   2   3
    \   \
     3   1
Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.

Example 2:
     3
    / \
   4   5
  / \   \
 1   3   1
Maximum amount of money the thief can rob = 4 + 5 = 9.
*/

public class _337 {

    public static class Solution1 {
        //simple recursion without cacheing: 1189 ms
        public int rob(TreeNode root) {
            if (root == null) {
                return 0;
            }

            int val = 0;
            if (root.left != null) {
```

```
                val += rob(root.left.left) + rob(root.left.right);
            }
            if (root.right != null) {
                val += rob(root.right.left) + rob(root.right.right);
            }
            val = Math.max(val + root.val, rob(root.left) + rob(root.right));
            return val;
        }
    }

    public static class Solution2 {
        //same idea, but with cacheing via a hashmap: 8 ms
        public int rob_dp(TreeNode root) {
            Map<TreeNode, Integer> map = new HashMap<>();
            return getMaxValue(root, map);
        }

        private int getMaxValue(TreeNode root, Map<TreeNode, Integer> map) {
            if (root == null) {
                return 0;
            }
            if (map.containsKey(root)) {
                return map.get(root);
            }

            int val = 0;
            if (root.left != null) {
                val += getMaxValue(root.left.left, map) + getMaxValue(root.left.right,
map);
            }
            if (root.right != null) {
                val += getMaxValue(root.right.left, map) + getMaxValue(root.right.right,
map);
            }
            int max = Math.max(root.val + val,
                getMaxValue(root.left, map) + getMaxValue(root.right, map));
            map.put(root, max);
            return max;
        }
    }

}

// ============================================================
// FILE 321: _338.java
// ============================================================

package com.fishercoder.solutions;

/**338. Counting Bits
Given a non negative integer number num. For every numbers i in the range 0 ≤ i ≤ num
calculate the number of 1's in their binary representation and return them as an array.

Example:
```

```
For num = 5 you should return [0,1,1,2,1,2].

Follow up:

It is very easy to come up with a solution with run time O(n*sizeof(integer)). But can
you do it in linear time O(n) /possibly in a single pass?
Space complexity should be O(n).
Can you do it like a boss? Do it without using any builtin function like
__builtin_popcount in c++ or in any other language.

Hint:

You should make use of what you have produced already.
Divide the numbers in ranges like [2-3], [4-7], [8-15] and so on. And try to generate new
range from previous.
Or does the odd/even status of the number help you in calculating the number of 1s?

*
*
*/
public class _338 {
    public static class Solution1 {
        //use the most regular method to get it AC'ed first
        public int[] countBits(int num) {
            int[] ones = new int[num + 1];
            for (int i = 0; i <= num; i++) {
                ones[i] = countOnes(i);
            }
            return ones;
        }

        private int countOnes(int i) {
            int ones = 0;
            while (i != 0) {
                ones++;
                i &= (i - 1);
            }
            return ones;
        }
    }

    private class Solution2 {
        /**lixx2100's post is cool:https://discuss.leetcode.com/topic/40162/three-line-
java-solution
        An easy recurrence for this problem is f[i] = f[i / 2] + i % 2
        and then we'll use bit manipulation to express the above recursion function
         right shift by 1 means to divide by 2
        AND with 1 means to modulo 2
        this is so cool!*/
        public int[] countBits(int num) {
            int[] ones = new int[num + 1];
            for (int i = 1; i <= num; i++) {
                ones[i] = ones[i >> 1] + (i & 1);
            }
```

```
            return ones;
        }
    }
}

// ============================================================
// FILE 322: _339.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.List;

public class _339 {
  public static class Solution1 {
    private int sum = 0;

    public int depthSum(List<NestedInteger> nestedList) {
      return dfs(nestedList, 1);
    }

    private int dfs(List<NestedInteger> nestedList, int depth) {
      for (NestedInteger ni : nestedList) {
        if (ni.isInteger()) {
          sum += depth * ni.getInteger();
        } else {
          dfs(ni.getList(), depth + 1);
        }
      }
      return sum;
    }
  }

  public static class Solution2 {
    public int depthSum(List<NestedInteger> nestedList) {
      return dfs(nestedList, 1);
    }

    private int dfs(List<NestedInteger> nestedList, int depth) {
      int sum = 0;
      for (NestedInteger ni : nestedList) {
        sum += ni.isInteger() ? depth * ni.getInteger() : dfs(ni.getList(), depth + 1);
      }
      return sum;
    }
  }
}

// ============================================================
// FILE 323: _340.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 340. Longest Substring with At Most K Distinct Characters
 *
 * Given a string, find the length of the longest substring T that contains at most k
distinct characters.

 For example, Given s = “eceba” and k = 2,

 T is "ece" which its length is 3.

 */
public class _340 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/41671/15-lines-java-solution-using-
slide-window
         */
        public int lengthOfLongestSubstringKDistinct(String s, int k) {
            int[] count = new int[256];
            int left = 0;
            int result = 0;
            int num = 0;
            for (int right = 0; right < s.length(); right++) {
                if (count[s.charAt(right)]++ == 0) {
                    num++;
                }
                if (num > k) {
                    while (--count[s.charAt(left++)] > 0) {};
                    num--;
                }
                result = Math.max(result, right - left + 1);
            }
            return result;
        }
    }

    public static class Solution2 {
        /**This is a more generic solution for any characters, not limited to ASCII
characters.*/
        public int lengthOfLongestSubstringKDistinct(String s, int k) {
            Map<Character, Integer> map = new HashMap<>();
            int longest = 0;
            int left = 0;
            for (int i = 0; i < s.length(); i++) {
                char c = s.charAt(i);
                map.put(c, map.getOrDefault(c, 0) + 1);
                while (map.size() > k) {
                    char leftChar = s.charAt(left);
```

```
                    if (map.containsKey(leftChar)) {
                        map.put(leftChar, map.get(leftChar) - 1);
                        if (map.get(leftChar) == 0) {
                            map.remove(leftChar);
                        }
                    }
                    left++;
                }
                longest = Math.max(longest, i - left + 1);
            }
            return longest;
        }
    }

}

// ============================================================
// FILE 324: _341.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 341. Flatten Nested List Iterator
 *
 * Given a nested list of integers, implement an iterator to flatten it.
 * Each element is either an integer, or a list -- whose elements may also be integers or
other lists.
 *
 * Example 1:
 * Given the list [[1,1],2,[1,1]],
 * By calling next repeatedly until hasNext returns false, the order of elements returned
by next should be: [1,1,2,1,1].
 *
 * Example 2:
 * Given the list [1,[4,[6]]],
 * By calling next repeatedly until hasNext returns false, the order of elements returned
by next should be: [1,4,6].
 */
public class _341 {

    public static class Solution1 {
        public static class NestedIterator implements Iterator<Integer> {

            private Queue<Integer> flattenedList;

            public NestedIterator(List<NestedInteger> nestedList) {
```

```
                flattenedList = new LinkedList<>();
                constructList(nestedList);
            }

            private void constructList(List<NestedInteger> nestedList) {
                for (NestedInteger nestedInteger : nestedList) {
                    if (nestedInteger.isInteger()) {
                        flattenedList.add(nestedInteger.getInteger());
                    } else {
                        constructList(nestedInteger.getList());
                    }
                }
            }

            @Override
            public Integer next() {
                return flattenedList.poll();
            }

            @Override
            public boolean hasNext() {
                return !flattenedList.isEmpty();
            }
        }
    }

}

// ============================================================
// FILE 325: _342.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 342. Power of Four
 *
 * Given an integer (signed 32 bits), write a function to check whether it is a power of
4.
 *
 * Example:
 * Given num = 16, return true. Given num = 5, return false.
 * Follow up: Could you solve it without loops/recursion?
 * */
public class _342 {
    public static class Solution1 {
        //Just dive a little bit deeper, you can realize that another important feature
of a number
        //that is power of four is that its only single one bit must appear on the odd
position, and power of two won't meet this requirement
        //decimal number 8 has binary format: 0000-0000-0000-0000-0000-0000-0000-1000
        //decimal number 16 has binary format: 0000-0000-0000-0000-0000-0000-0001-0000
        //hex number 0x55555555 has binary format: 1010-1010-1010-1010-1010-1010-1010-
1010
```

```
        //thus, doing AND with 0x55555 will check if the only one bit is located on the
odd position, thus ruling out those that are power of 2 but not power of 4
        public boolean isPowerOfFour(int num) {
            return (num > 0 && 1073741824 % num == 0 && (num & 0x55555555) != 0);
        }
    }

    public static class Solution2 {
        public boolean isPowerOfFour(int num) {
            //^ means to match the beginning of a line
            //$ means to match the end of a line
            //* means zero or more of the preceding character
            return Integer.toString(num, 4).matches("^10*$");
        }
    }

    public static class Solution3 {
        //a regular loop method to make it AC'ed
        public boolean isPowerOfFour(int num) {
            if (num < 4 && num != 1) {
                return false;
            }
            while (num != 1) {
                if (num % 4 != 0) {
                    return false;
                }
                num /= 4;
            }
            return true;
        }
    }

}

// ============================================================
// FILE 326: _343.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 343. Integer Break
 *
 * Given a positive integer n, break it into the sum of at least two positive integers
and maximize the product of those integers. Return the maximum product you can get.

 For example, given n = 2, return 1 (2 = 1 + 1); given n = 10, return 36 (10 = 3 + 3 +
4).

 Note: You may assume that n is not less than 2 and not larger than 58.

 Hint:

 There is a simple O(n) solution to this problem.
```

```
 You may check the breaking results of n ranging from 7 to 10 to discover the
regularities.
 */
public class _343 {
    public static class Solution1 {
        public int integerBreak(int n) {
            if (n == 2) {
                return 1;
            } else if (n == 3) {
                return 2;
            } else if (n % 3 == 0) {
                return (int) Math.pow(3, n / 3);
            } else if (n % 3 == 1) {
                return 2 * 2 * (int) Math.pow(3, (n - 4) / 3);
            } else {
                return 2 * (int) Math.pow(3, n / 3);
            }
        }
    }
}

// ============================================================
// FILE 327: _344.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 344. Reverse String
 *
 * Write a function that takes a string as input and returns the string reversed.

 Example:
 Given s = "hello", return "olleh".
 */
public class _344 {
    public static class Solution1 {
        public String reverseString(String s) {
            int i = 0;
            int j = s.length() - 1;
            char[] chars = s.toCharArray();
            while (i < j) {
                char temp = chars[i];
                chars[i++] = chars[j];
                chars[j--] = temp;
            }
            return new String(chars);
        }
    }
}

// ============================================================
// FILE 328: _345.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 345. Reverse Vowels of a String
 *
 * Write a function that takes a string as input and reverse only the vowels of a string.

 Example 1:
 Given s = "hello", return "holle".

 Example 2:
 Given s = "leetcode", return "leotcede".

 Note:
 The vowels does not include the letter "y".*/
public class _345 {
        public static class Solution1 {
                public String reverseVowels(String s) {
                        StringBuilder sb = new StringBuilder(s);
                        Set<Character> vowels = new HashSet(Arrays.asList('a', 'e', 'i',
'o', 'u', 'A', 'E', 'I', 'O', 'U'));
                        //use two pointers approach would be the fastest
                        int i = 0;
                        int j = s.length() - 1;
                        while (i < j) {
                                char left = s.charAt(i);
                                char right = s.charAt(j);
                                while (i < j && !vowels.contains(left)) {
                                        i++;
                                        left = s.charAt(i);
                                }
                                while (i < j && !vowels.contains(right)) {
                                        j--;
                                        right = s.charAt(j);
                                }
                                char temp = left;
                                sb.setCharAt(i, right);
                                sb.setCharAt(j, temp);
                                i++;
                                j--;
                        }
                        return sb.toString();
                }
        }
}

// ============================================================
// FILE 329: _346.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 346. Moving Average from Data Stream
 *
 * Given a stream of integers and a window size, calculate the moving average of all
integers in the sliding window.

 For example,
 MovingAverage m = new MovingAverage(3);
 m.next(1) = 1
 m.next(10) = (1 + 10) / 2
 m.next(3) = (1 + 10 + 3) / 3
 m.next(5) = (10 + 3 + 5) / 3
 */
public class _346 {

    public static class Solution1 {
        class MovingAverage {

            private Queue<Integer> q;
            private Long sum;
            private int max;

            /**
             * Initialize your data structure here.
             */
            public MovingAverage(int size) {
                q = new LinkedList();
                sum = 0L;
                max = size;
            }

            public double next(int val) {
                if (q.size() < max) {
                    q.offer(val);
                    sum += val;
                    return (double) sum / q.size();
                } else {
                    int first = q.poll();
                    sum -= first;
                    q.offer(val);
                    sum += val;
                    return (double) sum / q.size();
                }
            }
        }
    }
}
```

```
// ============================================================
// FILE 330: _347.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 347. Top K Frequent Elements
 *
 * Given a non-empty array of integers, return the k most frequent elements.

 For example,
 Given [1,1,1,2,2,3] and k = 2, return [1,2].

 Note:
 You may assume k is always valid, 1 ≤ k ≤ number of unique elements.
 Your algorithm's time complexity must be better than O(n log n), where n is the array's
size.*/

public class _347 {

        public static class Solution1 {
                /**
                 * Use buckets to hold numbers of the same frequency
     * It's averaged at 30 ms on Leetcode.
                 */
                public List<Integer> topKFrequent(int[] nums, int k) {
                        Map<Integer, Integer> map = new HashMap();
                        for (int i : nums) {
                                map.put(i, map.getOrDefault(i, 0) + 1);
                        }

                        ArrayList[] bucket = new ArrayList[nums.length + 1];
                        for (Entry<Integer, Integer> e : map.entrySet()) {
                                int frequency = e.getValue();
                                if (bucket[frequency] == null) {
                                        bucket[frequency] = new ArrayList<Integer>();
                                }
                                bucket[frequency].add(e.getKey());
                        }
                        List<Integer> result = new ArrayList<>();
                        for (int i = bucket.length - 1; i >= 0 && result.size() < k; i--)
{
                                if (bucket[i] != null) {
                                        for (int j = 0; j < bucket[i].size(); j++) {
                                                result.add((int) bucket[i].get(j));
```

```
                                        }
                                }
                        }

                        return result;
                }
        }

        public static class Solution2 {
                /**
                 * Use hashtable and heap, it's averaged at 100 ms on Leetocde.
                 */
                public List<Integer> topKFrequent(int[] nums, int k) {
                        // construct the frequency map first, and then iterate through
the map
                        // and put them into the heap, this is O(n)
                        Map<Integer, Integer> map = new HashMap();
                        for (int num : nums) {
                                map.put(num, map.getOrDefault(num, 0) + 1);
                        }

                        // build heap, this is O(logn)
                        Queue<Entry<Integer, Integer>> heap = new PriorityQueue<>((o1,
o2) -> o2.getValue() - o1.getValue());
                        for (Entry<Integer, Integer> entry : map.entrySet()) {
                                heap.offer(entry);
                        }

                        List<Integer> res = new ArrayList();
                        while (k-- > 0) {
                                res.add(heap.poll().getKey());
                        }
                        return res;
                }
        }
}

// ============================================================
// FILE 331: _348.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 348. Design Tic-Tac-Toe
 *
 * Design a Tic-tac-toe game that is played between two players on a n x n grid.

 You may assume the following rules:

 A move is guaranteed to be valid and is placed on an empty block.
 Once a winning condition is reached, no more moves is allowed.
 A player who succeeds in placing n of their marks in a horizontal, vertical, or diagonal
row wins the game.
```

```
 Example:

 Given n = 3, assume that player 1 is "X" and player 2 is "O" in the board.

 TicTacToe toe = new TicTacToe(3);

 toe.move(0, 0, 1); -> Returns 0 (no one wins)
 |X| | |
 | | | |    // Player 1 makes a move at (0, 0).
 | | | |

 toe.move(0, 2, 2); -> Returns 0 (no one wins)
 |X| |O|
 | | | |    // Player 2 makes a move at (0, 2).
 | | | |

 toe.move(2, 2, 1); -> Returns 0 (no one wins)
 |X| |O|
 | | | |    // Player 1 makes a move at (2, 2).
 | | |X|

 toe.move(1, 1, 2); -> Returns 0 (no one wins)
 |X| |O|
 | |O| |    // Player 2 makes a move at (1, 1).
 | | |X|

 toe.move(2, 0, 1); -> Returns 0 (no one wins)
 |X| |O|
 | |O| |    // Player 1 makes a move at (2, 0).
 |X| |X|

 toe.move(1, 0, 2); -> Returns 0 (no one wins)
 |X| |O|
 |O|O| |    // Player 2 makes a move at (1, 0).
 |X| |X|

 toe.move(2, 1, 1); -> Returns 1 (player 1 wins)
 |X| |O|
 |O|O| |    // Player 1 makes a move at (2, 1).
 |X|X|X|

 Follow up:
 Could you do better than O(n2) per move() operation?

 Hint:

 Could you trade extra space such that move() operation can be done in O(1)?
 You need two arrays: int rows[n], int cols[n], plus two variables: diagonal,
anti_diagonal.

 */
public class _348 {
    public static class Solution1 {
```

```
        /**
         * credit: https://discuss.leetcode.com/topic/44548/java-o-1-solution-easy-to-
understand
         *
         * Key: in order to win a TicTacToe, you must have the entire row or column,
thus, we don't need
         * to keep track of the entire n^2 board. We only need to keep a count for each
row and column.
         * If at any time, a row or column matches the size of the board, then that
player has won.
         */
        public static class TicTacToe {

            private int diagonal;
            /**
             * This is diagonal: |X| | | | |X| | | | |X| So, its condition is always like
this: if (row ==
             * col)
             */

            private int antidiagonal;
            /**
             * This is antidiagonal: | | |X| | |X| | |X| | | So, its condition is always
like this: if
             * (col == size - row - 1)
             */
            private int[] rows;
            private int[] cols;

            /**
             * Initialize your data structure here.
             */
            public TicTacToe(int n) {
                rows = new int[n];
                cols = new int[n];
            }

            /**
             * Player {player} makes a move at ({row}, {col}).
             *
             * @param row The row of the board.
             * @param col The column of the board.
             * @param player The player, can be either 1 or 2.
             * @return The current winning condition, can be either: 0: No one wins. 1:
Player 1 wins. 2:
             * Player 2 wins.
             */
            public int move(int row, int col, int player) {
                int toAdd = player == 1 ? 1 : -1;

                rows[row] += toAdd;
                cols[col] += toAdd;
                int size = rows.length;
```

```
                if (row == col) {
                    diagonal += toAdd;
                }
                if (col == (size - row - 1)) {
                    antidiagonal += toAdd;
                }

                if (Math.abs(rows[row]) == size
                    || Math.abs(cols[col]) == size
                    || Math.abs(antidiagonal) == size
                    || Math.abs(diagonal) == size) {
                    return player;
                }

                return 0;
            }
        }
    }
}

// ============================================================
// FILE 332: _349.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 349. Intersection of Two Arrays
 *
 * Given two arrays, write a function to compute their intersection.
 *
 * Example 1:
 * Input: nums1 = [1,2,2,1], nums2 = [2,2]
 * Output: [2]
 *
 * Example 2:
 * Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
 * Output: [9,4]
 *
 * Note:
 * Each element in the result must be unique.
 * The result can be in any order.
 */
public class _349 {

  public static class Solution1 {
    //credit: https://leetcode.com/articles/intersection-of-two-arrays/
    //Time: O(m+n) on average, O(m*n) in worse case
    //Space: O(m+n)
```

```
    public int[] intersection(int[] nums1, int[] nums2) {
      Set<Integer> set1 = Arrays.stream(nums1).boxed().collect(Collectors.toSet());
      Set<Integer> set2 = Arrays.stream(nums2).boxed().collect(Collectors.toSet());
      set1.retainAll(set2);
      int[] intersection = new int[set1.size()];
      int i = 0;
      for (int num : set1) {
        intersection[i++] = num;
      }
      return intersection;
    }
  }

  public static class Solution2 {
      //Time: O(nlgn)
    public int[] intersection(int[] nums1, int[] nums2) {
      Arrays.sort(nums2);
      Set<Integer> intersect = new HashSet();
      for (int i : nums1) {
        if (binarySearch(i, nums2)) {
          intersect.add(i);
        }
      }
      int[] result = new int[intersect.size()];
      int i = 0;
      for (int num : intersect) {
        result[i++] = num;
      }
      return result;
    }

    private boolean binarySearch(int i, int[] nums) {
      int left = 0;
      int right = nums.length - 1;
      while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == i) {
          return true;
        } else if (nums[mid] > i) {
          right = mid - 1;
        } else {
          left = mid + 1;
        }
      }
      return false;
    }
  }

  public static class Solution3 {
    //use two pointers
    //credit: https://leetcode.com/problems/intersection-of-two-
arrays/discuss/81969/Three-Java-Solutions
    public int[] intersection(int[] nums1, int[] nums2) {
      Arrays.sort(nums1);
```

```
      Arrays.sort(nums2);
      int i = 0;
      int j = 0;
      Set<Integer> set = new HashSet<>();
      while (i < nums1.length && j < nums2.length) {
        if (nums1[i] == nums2[j]) {
          set.add(nums1[i++]);
          j++;
        } else if (nums1[i] < nums2[j]) {
          i++;
        } else {
          j++;
        }
      }
      int[] intersection = new int[set.size()];
      int k = 0;
      for (int num : set) {
        intersection[k++] = num;
      }
      return intersection;
    }
  }
}

// ============================================================
// FILE 333: _350.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Arrays;

/**
 * 350. Intersection of Two Arrays II
 *
 * Given two arrays, write a function to compute their intersection.

 Example:
 Given nums1 = [1, 2, 2, 1], nums2 = [2, 2], return [2, 2].

 Note:
 Each element in the result should appear as many times as it shows in both arrays.
 The result can be in any order.

 Follow up:
 What if the given array is already sorted? How would you optimize your algorithm?
 What if nums1's size is small compared to nums2's size? Which algorithm is better?
 What if elements of nums2 are stored on disk, and the memory is limited such that you
cannot load all elements into the memory at once?
 */
```

```
public class _350 {

    public static class Solution1 {
        public int[] intersect(int[] nums1, int[] nums2) {
            Map<Integer, Integer> map = new HashMap();
            for (int i : nums1) {
                map.put(i, map.getOrDefault(i, 0) + 1);
            }

            List<Integer> list = new ArrayList();
            for (int i : nums2) {
                if (map.containsKey(i) && map.get(i) > 0) {
                    list.add(i);
                    map.put(i, map.get(i) - 1);
                }
            }
            return list.stream().mapToInt(i -> i).toArray();
        }
    }

    public static class Solution2 {
        public int[] intersect(int[] nums1, int[] nums2) {
            Arrays.sort(nums1);
            Arrays.sort(nums2);
            int i = 0;
            int j = 0;
            List<Integer> list = new ArrayList<>();
            while (i < nums1.length && j < nums2.length) {
                if (nums1[i] == nums2[j]) {
                    list.add(nums1[i]);
                    i++;
                    j++;
                } else if (nums1[i] < nums2[j]) {
                    i++;
                } else {
                    j++;
                }
            }
            return list.stream().mapToInt(k -> k).toArray();
        }
    }
}

// ============================================================
// FILE 334: _351.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 351. Android Unlock Patterns
 *
 * Given an Android 3x3 key lock screen and two integers m and n, where 1 ≤ m ≤ n ≤ 9,
count the total number of unlock patterns of the Android lock screen, which consist of
```

```
minimum of m keys and maximum n keys.

 Rules for a valid pattern:
 Each pattern must connect at least m keys and at most n keys.
 All the keys must be distinct.
 If the line connecting two consecutive keys in the pattern passes through any other
keys, the other keys must have previously selected in the pattern. No jumps through non
selected key is allowed.
 The order of keys used matters.

 Explanation:
 | 1 | 2 | 3 |
 | 4 | 5 | 6 |
 | 7 | 8 | 9 |
 Invalid move: 4 - 1 - 3 - 6
 Line 1 - 3 passes through key 2 which had not been selected in the pattern.

 Invalid move: 4 - 1 - 9 - 2
 Line 1 - 9 passes through key 5 which had not been selected in the pattern.

 Valid move: 2 - 4 - 1 - 3 - 6
 Line 1 - 3 is valid because it passes through key 2, which had been selected in the
pattern

 Valid move: 6 - 5 - 4 - 1 - 9 - 2
 Line 1 - 9 is valid because it passes through key 5, which had been selected in the
pattern.

 Example:
 Given m = 1, n = 1, return 9.


 */
public class _351 {

    public static class Solution1 {
        private int[][] jumps;
        private boolean[] visited;

        public int numberOfPatterns(int m, int n) {
            jumps = new int[10][10];
            jumps[1][3] = jumps[3][1] = 2;
            jumps[4][6] = jumps[6][4] = 5;
            jumps[7][9] = jumps[9][7] = 8;
            jumps[1][7] = jumps[7][1] = 4;
            jumps[2][8] = jumps[8][2] = jumps[1][9] = jumps[9][1] = 5;
            jumps[9][3] = jumps[3][9] = 6;
            jumps[3][7] = jumps[7][3] = 5;
            visited = new boolean[10];
            int count = 0;
            count += dfs(1, 1, 0, m, n)
                * 4;//1,3,7,9 are symmetric, so we only need to use 1 to do it once and
then multiply the result by 4
            count += dfs(2, 1, 0, m, n)
```

```
                * 4;//2,4,6,8 are symmetric, so we only need to use 1 to do it once and
then multiply the result by 4
            count += dfs(5, 1, 0, m, n);
            return count;
        }

        private int dfs(int num, int len, int count, int m, int n) {
            if (len >= m) {
                count++;
            }
            len++;
            if (len > n) {
                return count;
            }
            visited[num] = true;
            for (int next = 1; next <= 9; next++) {
                int jump = jumps[num][next];
                if (!visited[next] && (jump == 0 || visited[jump])) {
                    count = dfs(next, len, count, m, n);
                }
            }
            visited[num] = false;//backtracking
            return count;
        }
    }
}

// ============================================================
// FILE 335: _352.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

/**
 * 352. Data Stream as Disjoint Intervals
 *
 * Given a data stream input of non-negative integers a1, a2, ..., an, ..., summarize the
numbers seen so far as a list of disjoint intervals.

 For example, suppose the integers from the data stream are 1, 3, 7, 2, 6, ..., then the
summary will be:

 [1, 1]
 [1, 1], [3, 3]
 [1, 1], [3, 3], [7, 7]
 [1, 3], [7, 7]
 [1, 3], [6, 7]
```

```
 Follow up:
 What if there are lots of merges and the number of disjoint intervals are small compared
to the data stream's size?
 */
public class _352 {

    public static class Solution1 {
        public static class SummaryRanges {

            /**
             * Use treemap, key is the start of the interval.
             */
            TreeMap<Integer, Interval> treeMap;

            /**
             * Initialize your data structure here.
             */
            public SummaryRanges() {
                treeMap = new TreeMap<>();
            }

            public void addNum(int val) {
                if (treeMap.containsKey(val)) {
                    return;
                }
                Integer lower = treeMap.lowerKey(val);
                Integer higher = treeMap.higherKey(val);
                if (lower != null
                    && higher != null
                    && treeMap.get(lower).end + 1 == val
                    && higher == val + 1) {
                    treeMap.get(lower).end = treeMap.get(higher).end;
                    treeMap.remove(higher);
                } else if (lower != null && treeMap.get(lower).end + 1 >= val) {
                    treeMap.get(lower).end = Math.max(treeMap.get(lower).end, val);
                } else if (higher != null && higher == val + 1) {
                    treeMap.put(val, new Interval(val, treeMap.get(higher).end));
                    treeMap.remove(higher);
                } else {
                    treeMap.put(val, new Interval(val, val));
                }
            }

            public List<Interval> getIntervals() {
                return new ArrayList<>(treeMap.values());
            }
        }
    }
}

// ============================================================
// FILE 336: _353.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.Deque;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Set;

/**
 * Design a Snake game that is played on a device with screen size = width x height. Play
the game online if you are not familiar with the game.
 * <p>
 * The snake is initially positioned at the top left corner (0,0) with length = 1 unit.
 * <p>
 * You are given a list of food's positions in row-column order. When a snake eats the
food, its length and the game's score both increase by 1.
 * <p>
 * Each food appears one by one on the screen. For example, the second food will not
appear until the first food was eaten by the snake.
 * <p>
 * When a food does appear on the screen, it is guaranteed that it will not appear on a
block occupied by the snake.
 * <p>
 * Example:
 * Given width = 3, height = 2, and food = [[1,2],[0,1]].
 * <p>
 * Snake snake = new Snake(width, height, food);
 * <p>
 * Initially the snake appears at position (0,0) and the food at (1,2).
 * <p>
 * |S| | |
 * | | |F|
 * <p>
 * snake.move("R"); -> Returns 0
 * <p>
 * | |S| |
 * | | |F|
 * <p>
 * snake.move("D"); -> Returns 0
 * <p>
 * | | | |
 * | |S|F|
 * <p>
 * snake.move("R"); -> Returns 1 (Snake eats the first food and right after that, the
second food appears at (0,1) )
 * <p>
 * | |F| |
 * | |S|S|
 * <p>
 * snake.move("U"); -> Returns 1
 * <p>
 * | |F|S|
 * | | |S|
 * <p>
 * snake.move("L"); -> Returns 2 (Snake eats the second food)
```

```
 * <p>
 * | |S|S|
 * | | |S|
 * <p>
 * snake.move("U"); -> Returns -1 (Game over because snake collides with border)
 */
public class _353 {
    public class SnakeGame {
        private Set<Integer> set;//Use a set to hold all occupied points for the snake
body, this is for easy access for the case of eating its own body
        private Deque<Integer> body;//use a queue to hold all occupied points for the
snake body as well, this is for easy access to update the tail
        int[][] food;
        int score;
        int foodIndex;
        int width;
        int height;

        /**
         * Initialize your data structure here.
         *
         * @param width  - screen width
         * @param height - screen height
         * @param food   - A list of food positions
         *               E.g food = [[1,1], [1,0]] means the first food is positioned at
[1,1], the second is at [1,0].
         */
        public SnakeGame(int width, int height, int[][] food) {
            this.set = new HashSet();
            set.add(0);//initially at [0][0]
            this.body = new LinkedList<Integer>();
            body.offerLast(0);
            this.food = food;
            this.width = width;
            this.height = height;
        }

        /**
         * Moves the snake.
         *
         * @param direction - 'U' = Up, 'L' = Left, 'R' = Right, 'D' = Down
         * @return The game's score after the move. Return -1 if game over.
         * Game over when snake crosses the screen boundary or bites its body.
         */
        public int move(String direction) {
            if (score == -1) {
                return -1;
            }

            //compute head
            int rowHead = body.peekFirst() / width;
            int colHead = body.peekFirst() % width;
            switch (direction) {
                case "U":
```

```
                    rowHead--;
                    break;
                case "D":
                    rowHead++;
                    break;
                case "L":
                    colHead--;
                    break;
                default:
                    colHead++;
            }
            int newHead = rowHead * width + colHead;

            set.remove(body.peekLast());//we'll remove the tail from set for now to see
if it hits its tail
            //if it hits the boundary
            if (set.contains(newHead) || rowHead < 0 || colHead < 0 || rowHead >= height
|| colHead >= width) {
                return score = -1;
            }

            //add head for the following two normal cases:
            set.add(newHead);
            body.offerFirst(newHead);

            //normal eat case: keep tail, add head
            if (foodIndex < food.length && rowHead == food[foodIndex][0] && colHead ==
food[foodIndex][1]) {
                set.add(body.peekLast());//old tail does not change, so add it back to
set since we removed it earlier
                foodIndex++;
                return ++score;
            }


            //normal move case without eating: move head and remove tail
            body.pollLast();
            return score;

        }
    }

/**
 * Your SnakeGame object will be instantiated and called as such:
 * SnakeGame obj = new SnakeGame(width, height, food);
 * int param_1 = obj.move(direction);
 */
}

// ============================================================
// FILE 337: _354.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.Arrays;

/**
 * 354. Russian Doll Envelopes
 *
 * You have a number of envelopes with widths and heights given as a pair of integers (w,
h).
 * One envelope can fit into another if and only if both the width and height of one
envelope is greater than
 * the width and height of the other envelope.

 What is the maximum number of envelopes can you Russian doll? (put one inside other)

 Example:
 Given envelopes = [[5,4],[6,4],[6,7],[2,3]], the maximum number of envelopes you can
Russian doll is 3 ([2,3] => [5,4] => [6,7]).

 */
public class _354 {
    public static class Solution1 {
        /** reference: https://discuss.leetcode.com/topic/47469/java-nlogn-solution-with-
explanation */
        public int maxEnvelopes(int[][] envelopes) {
            if (envelopes == null || envelopes.length == 0
                || envelopes[0].length == 0 || envelopes[0].length != 2) {
                return 0;
            }
            Arrays.sort(envelopes, (int[] a, int[] b) -> {
                    if (a[0] == b[0]) {
                        return b[1] - a[1];
                    } else {
                        return a[0] - b[0];
                    }
                }
            );
            int[] dp = new int[envelopes.length];
            int len = 0;
            for (int[] envelope : envelopes) {
                int index = Arrays.binarySearch(dp, 0, len, envelope[1]);
                if (index < 0) {
                    index = -(index + 1);
                }
                dp[index] = envelope[1];
                if (index == len) {
                    len++;
                }
            }
            return len;
        }
    }
}

// ============================================================
```

```
// FILE 338: _355.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Set;

/**
 * 355. Design Twitter
 *
 * Design a simplified version of Twitter where users can post tweets,
 * follow/unfollow another user and is able to see the 10 most recent tweets in the
user's news feed. Your design should support the following methods:
 postTweet(userId, tweetId): Compose a new tweet.
 getNewsFeed(userId): Retrieve the 10 most recent tweet ids in the user's news feed. Each
item in the news feed must be posted by users who the user followed or by the user
herself. Tweets must be ordered from most recent to least recent.
 follow(followerId, followeeId): Follower follows a followee.
 unfollow(followerId, followeeId): Follower unfollows a followee.

 Example:

 Twitter twitter = new Twitter();

 // User 1 posts a new tweet (userId = 5).
 twitter.postTweet(1, 5);

 // User 1's news feed should return a list with 1 tweet userId -> [5].
 twitter.getNewsFeed(1);

 // User 1 follows user 2.
 twitter.follow(1, 2);

 // User 2 posts a new tweet (userId = 6).
 twitter.postTweet(2, 6);

 // User 1's news feed should return a list with 2 tweet ids -> [6, 5].
 // Tweet userId 6 should precede tweet userId 5 because it is posted after tweet userId
5.
 twitter.getNewsFeed(1);

 // User 1 unfollows user 2.
 twitter.unfollow(1, 2);

 // User 1's news feed should return a list with 1 tweet userId -> [5],
 // since user 1 is no longer following user 2.
 twitter.getNewsFeed(1);
 */
```

```
public class _355 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/48100/java-oo-design-with-most-efficient-function-getnewsfeed
         */
        public static class Twitter {

            private static int timestamp = 0;
            private Map<Integer, User> map;

            class Tweet {
                public int time;
                public int id;
                public Tweet next;
                /**have a pointer,
                 * so we could be more memory efficient when retrieving tweets,
                 * think about merging k sorted lists*/

                public Tweet(int id) {
                    this.id = id;
                    time = timestamp++;
                    next = null;
                }
            }

            /**
             * the meat part of this OO design problem,
             * have a User object itself,
             * have follow() and unfollow() method embedded inside it
             */
            class User {
                public int id;
                public Set<Integer> followed;
                public Tweet tweetHead;

                public User(int id) {
                    this.id = id;
                    followed = new HashSet<>();
                    followed.add(id);//follow oneself first
                    this.tweetHead = null;
                }

                public void follow(int followeeId) {
                    followed.add(followeeId);
                }

                public void unfollow(int followeeId) {
                    followed.remove(followeeId);
                }

                public void postTweet(int tweetId) {
                    //every time we post, we prepend it to the head of the tweet
```

```
                    Tweet head = new Tweet(tweetId);
                    head.next = tweetHead;
                    tweetHead = head;//don't forget to overwrite tweetHead with the new
head
                }
            }

            /**
             * Initialize your data structure here.
             */
            public Twitter() {
                map = new HashMap();
            }

            /**
             * Compose a new tweet.
             */
            public void postTweet(int userId, int tweetId) {
                /**update oneself newsFeed first and also all of his followers'
newsFeed*/
                if (!map.containsKey(userId)) {
                    User user = new User(userId);
                    map.put(userId, user);
                }
                map.get(userId).postTweet(tweetId);
            }

            /**
             * Retrieve the 10 most recent tweet ids in the user's news feed. Each item
in the news feed must be posted by users who the user followed or by the user herself.
Tweets must be ordered from most recent to least recent.
             */
            public List<Integer> getNewsFeed(int userId) {
                List<Integer> newsFeed = new LinkedList<>();
                if (!map.containsKey(userId)) {
                    return newsFeed;
                }
                Set<Integer> users = map.get(userId).followed;
                PriorityQueue<Tweet> heap = new PriorityQueue<>(users.size(), (a, b) ->
b.time - a.time);
                for (int user : users) {
                    Tweet tweet = map.get(user).tweetHead;
                    //it's super important to check null before putting into the heap
                    if (tweet != null) {
                        heap.offer(tweet);
                    }
                }

                int count = 0;
                while (!heap.isEmpty() && count < 10) {
                    Tweet tweet = heap.poll();
                    newsFeed.add(tweet.id);
                    count++;
                    if (tweet.next != null) {
```

```
                    heap.offer(tweet.next);
                }
            }

            return newsFeed;
        }

        /**
         * Follower follows a followee. If the operation is invalid, it should be a
no-op.
         */
        public void follow(int followerId, int followeeId) {
            if (!map.containsKey(followeeId)) {
                User user = new User(followeeId);
                map.put(followeeId, user);
            }

            if (!map.containsKey(followerId)) {
                User user = new User(followerId);
                map.put(followerId, user);
            }

            map.get(followerId).follow(followeeId);
        }

        /**
         * Follower unfollows a followee. If the operation is invalid, it should be a
no-op.
         */
        public void unfollow(int followerId, int followeeId) {
            if (!map.containsKey(followerId) || followeeId == followerId) {
                return;
            }
            map.get(followerId).unfollow(followeeId);
        }
        /**
         * Your Twitter object will be instantiated and called as such:
         * Twitter obj = new Twitter();
         * obj.postTweet(userId,tweetId);
         * List<Integer> param_2 = obj.getNewsFeed(userId);
         * obj.follow(followerId,followeeId);
         * obj.unfollow(followerId,followeeId);
         */
    }
}

public static class Solution2 {
    public static class Twitter {

        Map<Integer, User> map;
        private int timestamp;

        private class User {
            private int userId;
```

```
        private Set<Integer> followed;
        private Tweet tweetHead;

        public User(int userId) {
            this.userId = userId;
            this.followed = new HashSet<>();
            this.followed.add(userId);
            this.tweetHead = null;
        }

        public void postTweet(int tweetId) {
            Tweet tweet = new Tweet(tweetId);
            tweet.next = tweetHead;
            tweetHead = tweet;
        }

        public void follow(int followeeId) {
            followed.add(followeeId);
        }

        public void unfollow(int followeeId) {
            followed.remove(followeeId);
        }

    }

    private class Tweet {
        int time;
        int id;
        Tweet next;

        public Tweet(int id) {
            this.id = id;
            time = timestamp++;
            next = null;
        }
    }

    /** Initialize your data structure here. */
    public Twitter() {
        map = new HashMap<>();
        timestamp = 0;
    }

    /** Compose a new tweet. */
    public void postTweet(int userId, int tweetId) {
        if (!map.containsKey(userId)) {
            User user = new User(userId);
            map.put(userId, user);
        }
        map.get(userId).postTweet(tweetId);
    }

    /** Retrieve the 10 most recent tweet ids in the user's news feed. Each item
```

```
in the news feed must be posted by users who the user followed or by the user herself.
Tweets must be ordered from most recent to least recent. */
            public List<Integer> getNewsFeed(int userId) {
                List<Integer> result = new LinkedList<>();
                if (!map.containsKey(userId)) {
                    return result;
                }
                Set<Integer> followeeSet = map.get(userId).followed;
                PriorityQueue<Tweet> maxHeap = new PriorityQueue<>((a, b) -> b.time -
a.time);
                for (int followeeId : followeeSet) {
                    if (map.containsKey(followeeId)) {
                        Tweet tweet = map.get(followeeId).tweetHead;
                        if (tweet != null) {
                            maxHeap.offer(tweet);
                        }
                    }
                }

                int count = 0;
                while (!maxHeap.isEmpty() && count++ < 10) {
                    Tweet tweet = maxHeap.poll();
                    if (tweet != null) {
                        result.add(tweet.id);
                        if (tweet.next != null) {
                            maxHeap.offer(tweet.next);
                        }
                    }
                }
                return result;
            }

            /** Follower follows a followee. If the operation is invalid, it should be a
no-op. */
            public void follow(int followerId, int followeeId) {
                if (!map.containsKey(followerId)) {
                    map.put(followerId, new User(followerId));
                }
                if (!map.containsKey(followeeId)) {
                    map.put(followeeId, new User(followeeId));
                }
                map.get(followerId).follow(followeeId);
            }

            /** Follower unfollows a followee. If the operation is invalid, it should be
a no-op. */
            public void unfollow(int followerId, int followeeId) {
                if (!map.containsKey(followerId) || followeeId == followerId) {
                    return;
                }
                map.get(followerId).unfollow(followeeId);
            }
        }
    }
```

```
}

// ============================================================
// FILE 339: _356.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 356. Line Reflection
 *
 * Given n points on a 2D plane, find if there is such a line parallel to y-axis that
reflect the given points.

 Example 1:
 Given points = [[1,1],[-1,1]], return true.

 Example 2:
 Given points = [[1,1],[-1,-1]], return false.

 Follow up:
 Could you do better than O(n2)?

 Hint:

 Find the smallest and largest x-value for all points.
 If there is a line then it should be at y = (minX + maxX) / 2.
 For each point, make sure that it has a reflected point in the opposite side.
 */
public class _356 {
    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/48172/simple-java-hashset-solution
*/
        public boolean isReflected(int[][] points) {
            int max = Integer.MIN_VALUE;
            int min = Integer.MAX_VALUE;
            Set<String> set = new HashSet<>();
            for (int[] point : points) {
                max = Math.max(max, point[0]);
                min = Math.min(min, point[0]);
                String str = point[0] + "a" + point[1];
                set.add(str);
            }
            int sum = max + min;
            for (int[] p : points) {
                String str = (sum - p[0]) + "a" + p[1];
                if (!set.contains(str)) {
                    return false;
                }
            }
            return true;
```

```
        }
    }
}

// ============================================================
// FILE 340: _357.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 357. Count Numbers with Unique Digits
 * Given a non-negative integer n, count all numbers with unique digits, x, where 0 ≤ x < 10n.

 Example:
 Given n = 2, return 91. (The answer should be the total numbers in the range of 0 ≤ x < 100, excluding [11,22,33,44,55,66,77,88,99])

 Hint:
 A direct way is to use the backtracking approach.
 Backtracking should contains three states which are
 (the current number, number of steps to get that number and a bitmask which
 represent which number is marked as visited so far in the current number).
 Start with state (0,0,0) and count all valid number till we reach number of steps equals to 10n.
 This problem can also be solved using a dynamic programming approach and some knowledge of combinatorics.
 Let f(k) = count of numbers with unique digits with length equals k.
 f(1) = 10, ..., f(k) = 9 * 9 * 8 * ... (9 - k + 2) [The first factor is 9 because a number cannot start with 0].
 */
public class _357 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/47983/java-dp-o-1-solution Following the hint.
         * Let f(n) = count of number with unique digits of length n. f(1) = 10. (0, 1, 2, 3, ...., 9)
         * f(2) = 9 * 9. Because for each number i from 1, ..., 9, we can pick j to form a 2-digit
         * number ij and there are 9 numbers that are different from i for j to choose from. f(3) = f(2)
         * * 8 = 9 * 9 * 8. Because for each number with unique digits of length 2, say ij, we can pick
         * k to form a 3 digit number ijk and there are 8 numbers that are different from i and j for k
         * to choose from. Similarly f(4) = f(3) * 7 = 9 * 9 * 8 * 7.... ... f(10) = 9 * 9 * 8 * 7 * 6 *
         * ... * 1 f(11) = 0 = f(12) = f(13).... any number with length > 10 couldn't be unique digits
         * number. The problem is asking for numbers from 0 to 10^n. Hence return f(1) + f(2) + .. +
```

```
         * f(n) As @4acreg suggests, There are only 11 different ans. You can create a
lookup table for
         * it. This problem is O(1) in essence.
         */
        public int countNumbersWithUniqueDigits(int n) {
            if (n == 0) {
                return 1;
            }
            int res = 10;
            int uniqueDigits = 9;
            int availableNumber = 9;
            while (n-- > 1 && availableNumber > 0) {
                uniqueDigits = uniqueDigits * availableNumber;
                res += uniqueDigits;
                availableNumber--;
            }
            return res;
        }
    }

}

// ============================================================
// FILE 341: _358.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 358. Rearrange String k Distance Apart
 *
 * Given a non-empty string s and an integer k, rearrange the string such that the same
characters are at least distance k from each other.

 All input strings are given in lowercase letters. If it is not possible to rearrange the
string, return an empty string "".
 Example 1:
 s = "aabbcc", k = 3
 Result: "abcabc"
 The same letters are at least distance 3 from each other.

 Example 2:
 s = "aaabc", k = 3
 Answer: ""
 It is not possible to rearrange the string.

 Example 3:
 s = "aaadbbcc", k = 2
```

```
 Answer: "abacabcd"
 Another possible answer is: "abcabcda"
 The same letters are at least distance 2 from each other.
 */
public class _358 {

    public static class Solution1 {
        public String rearrangeString(String s, int k) {
            Map<Character, Integer> count = new HashMap<>();
            for (char c : s.toCharArray()) {
                count.put(c, count.getOrDefault(c, 0) + 1);
            }

            PriorityQueue<Map.Entry<Character, Integer>> heap =
                new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());
            heap.addAll(count.entrySet());

            Queue<Map.Entry<Character, Integer>> waitQueue = new LinkedList<>();

            StringBuilder stringBuilder = new StringBuilder();
            while (!heap.isEmpty()) {
                Map.Entry<Character, Integer> entry = heap.poll();
                stringBuilder.append(entry.getKey());
                entry.setValue(entry.getValue() - 1);
                waitQueue.offer(entry);
                if (waitQueue.size() < k) {
                    continue; //there's only k-1 chars in the waitHeap, not full yet
                }
                Map.Entry<Character, Integer> front = waitQueue.poll();
                if (front.getValue() > 0) {
                    heap.offer(front);
                }
            }

            return stringBuilder.length() == s.length() ? stringBuilder.toString() : "";
        }
    }

}

// ============================================================
// FILE 342: _359.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;
/**
 * 359. Logger Rate Limiter
 *
 * Design a logger system that receive stream of messages along with its timestamps, each
```

```
message should be printed if and only if it is not printed in the last 10 seconds.

 Given a message and a timestamp (in seconds granularity), return true if the message
should be printed in the given timestamp, otherwise returns false.

 It is possible that several messages arrive roughly at the same time.

 Example:

 Logger logger = new Logger();

 // logging string "foo" at timestamp 1
 logger.shouldPrintMessage(1, "foo"); returns true;

 // logging string "bar" at timestamp 2
 logger.shouldPrintMessage(2,"bar"); returns true;

 // logging string "foo" at timestamp 3
 logger.shouldPrintMessage(3,"foo"); returns false;

 // logging string "bar" at timestamp 8
 logger.shouldPrintMessage(8,"bar"); returns false;

 // logging string "foo" at timestamp 10
 logger.shouldPrintMessage(10,"foo"); returns false;

 // logging string "foo" at timestamp 11
 logger.shouldPrintMessage(11,"foo"); returns true;*/
public class _359 {

    public static class Solution1 {
        class Logger {

            private Map<String, Integer> map;
            private Set<String> set;

            /**
             * Initialize your data structure here.
             */
            public Logger() {
                map = new HashMap<String, Integer>();
                set = new HashSet<String>();
            }

            /**
             * Returns true if the message should be printed in the given timestamp,
otherwise returns
             * false. The timestamp is in seconds granularity.
             */
            public boolean shouldPrintMessage(int timestamp, String message) {
                if (!set.contains(message)) {
                    map.put(message, timestamp);
                    set.add(message);
                    return true;
```

```
                } else {
                    if (timestamp - map.get(message) < 10) {
                        return false;
                    } else {
                        map.put(message, timestamp);
                        return true;
                    }
                }
            }
        }
    }
}

// ======================================================
// FILE 343: _360.java
// ======================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 360. Sort Transformed Array
 *
 * Given a sorted array of integers nums and integer values a, b and c. Apply a function
of the form f(x) = ax2 + bx + c to each element x in the array.

 The returned array must be in sorted order.

 Expected time complexity: O(n)

 Example:
 nums = [-4, -2, 2, 4], a = 1, b = 3, c = 5,

 Result: [3, 9, 15, 33]

 nums = [-4, -2, 2, 4], a = -1, b = 3, c = 5

 Result: [-23, -5, 1, 7]

 Credits:
 Special thanks to @elmirap for adding this problem and creating all test cases.
 */
public class _360 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/48424/java-o-n-incredibly-short-yet-
easy-to-understand-ac-solution
        //in sum, only two cases: when a >= 0 or when a < 0, this simplifies logic
        public int[] sortTransformedArray(int[] nums, int a, int b, int c) {
            int n = nums.length;
            int[] sorted = new int[n];
            int i = 0;
            int j = n - 1;
```

```
            int index = a >= 0 ? n - 1 : 0;
            while (i <= j) {
                if (a >= 0) {
                    sorted[index--] =
                        function(nums[i], a, b, c) >= function(nums[j], a, b, c) ?
function(
                            nums[i++], a, b, c) : function(nums[j--], a, b, c);
                } else {
                    sorted[index++] =
                        function(nums[i], a, b, c) >= function(nums[j], a, b, c) ?
function(
                            nums[j--], a, b, c) : function(nums[i++], a, b, c);
                }
            }
            return sorted;
        }

        private int function(int num, int a, int b, int c) {
            return a * (num * num) + b * num + c;
        }
    }

    public static class Solution2 {
        public int[] sortTransformedArray(int[] nums, int a, int b, int c) {
            int[] result = new int[nums.length];
            for (int i = 0; i < nums.length; i++) {
                result[i] = function(nums[i], a, b, c);
            }
            Arrays.sort(result);
            return result;
        }

        private int function(int num, int a, int b, int c) {
            return a * (num * num) + b * num + c;
        }
    }

}

// ============================================================
// FILE 344: _361.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 361. Bomb Enemy
 *
 * Given a 2D grid, each cell is either a wall 'W', an enemy 'E' or empty '0' (the number
zero), return the maximum enemies you can kill using one bomb.
 The bomb kills all the enemies in the same row and column from the planted point until
it hits the wall since the wall is too strong to be destroyed.
 Note that you can only put the bomb at an empty cell.
```

```
 Example:
 For the given grid

 0 E 0 0
 E 0 W E
 0 E 0 0

 return 3. (Placing a bomb at (1,1) kills 3 enemies)
 */
public class _361 {

    public static class Solution1 {
        public int maxKilledEnemies(char[][] grid) {
            int m = grid.length;
            if (grid == null || m == 0) {
                return 0;
            }
            int n = grid[0].length;

            int[][] max = new int[m][n];

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {

                    if (grid[i][j] == '0') {
                        int count = 0;

                        //count all possible hits in its upward direction
                        for (int k = j - 1; k >= 0; k--) {
                            if (grid[i][k] == 'E') {
                                count++;
                            } else if (grid[i][k] == 'W') {
                                break;
                            }
                        }

                        //count all possible hits in its downward direction
                        for (int k = j + 1; k < n; k++) {
                            if (grid[i][k] == 'E') {
                                count++;
                            } else if (grid[i][k] == 'W') {
                                break;
                            }
                        }

                        //count all possible hits in its right direction
                        for (int k = i + 1; k < m; k++) {
                            if (grid[k][j] == 'E') {
                                count++;
                            } else if (grid[k][j] == 'W') {
                                break;
                            }
                        }
```

```
                        //count all possible hits in its left direction
                        for (int k = i - 1; k >= 0; k--) {
                            if (grid[k][j] == 'E') {
                                count++;
                            } else if (grid[k][j] == 'W') {
                                break;
                            }
                        }

                        max[i][j] = count;
                    }
                }
            }

            int result = 0;

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (max[i][j] > result) {
                        result = max[i][j];
                    }
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 345: _362.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 362. Design Hit Counter
 *
 * Design a hit counter which counts the number of hits received in the past 5 minutes.

 Each function accepts a timestamp parameter (in seconds granularity) and you may assume
 that calls are being made to the system in chronological order (ie, the timestamp is
monotonically increasing).
 You may assume that the earliest timestamp starts at 1.

 It is possible that several hits arrive roughly at the same time.

 Example:
 HitCounter counter = new HitCounter();

 // hit at timestamp 1.
 counter.hit(1);

 // hit at timestamp 2.
 counter.hit(2);
```

```
 // hit at timestamp 3.
 counter.hit(3);

 // get hits at timestamp 4, should return 3.
 counter.getHits(4);

 // hit at timestamp 300.
 counter.hit(300);

 // get hits at timestamp 300, should return 4.
 counter.getHits(300);

 // get hits at timestamp 301, should return 3.
 counter.getHits(301);
 Follow up:
 What if the number of hits per second could be very large? Does your design scale?
 */

public class _362 {

    public static class Solution1 {
        public static class HitCounter {
            /**
             * Reference: https://discuss.leetcode.com/topic/48758/super-easy-design-o-1-
hit-o-s-gethits-no-fancy-data-structure-is-needed,
             * I added one more field k to make it more generic.
             * It basically maintains a window of size 300, use modular to update the
index.
             */
            private int[] times;
            private int[] hits;
            private int k;

            /**
             * Initialize your data structure here.
             */
            public HitCounter() {
                k = 300;
                times = new int[k];
                hits = new int[k];
            }

            /**
             * Record a hit.
             *
             * @param timestamp - The current timestamp (in seconds granularity).
             */
            public void hit(int timestamp) {
                int index = timestamp % k;
                if (times[index] != timestamp) {
                    times[index] = timestamp;
                    hits[index] = 1;
                } else {
                    hits[index]++;
```

```
                }
            }

            /**
             * Return the number of hits in the past 5 minutes.
             *
             * @param timestamp - The current timestamp (in seconds granularity).
             */
            public int getHits(int timestamp) {
                int total = 0;
                for (int i = 0; i < k; i++) {
                    if (timestamp - times[i] < k) {
                        total += hits[i];
                    }
                }
                return total;
            }
        }
    }
}

// ============================================================
// FILE 346: _363.java
// ============================================================

package com.fishercoder.solutions;

import java.util.TreeSet;

/**
 * 363. Max Sum of Rectangle No Larger Than K
 *
 * Given a non-empty 2D matrix matrix and an integer k,
 * find the max sum of a rectangle in the matrix such that its sum is no larger than k.

 Example:
 Given matrix = [
 [1,  0, 1],
 [0, -2, 3]
 ]
 k = 2
 The answer is 2. Because the sum of rectangle [[0, 1], [-2, 3]] is 2 and 2 is the max
number no larger than k (k = 2).

 Note:
 The rectangle inside the matrix must have an area > 0.
 What if the number of rows is much larger than the number of columns?
 */
public class _363 {
    public static class Solution1 {
        /** reference: https://discuss.leetcode.com/topic/48854/java-binary-search-
solution-time-complexity-min-m-n-2-max-m-n-log-max-m-n */
        public int maxSumSubmatrix(int[][] matrix, int k) {
            int row = matrix.length;
```

```
            if (row == 0) {
                return 0;
            }
            int col = matrix[0].length;
            int m = Math.min(row, col);
            int n = Math.max(row, col);
            //indicating sum up in every row or every column
            boolean colIsBig = col > row;
            int res = Integer.MIN_VALUE;
            for (int i = 0; i < m; i++) {
                int[] array = new int[n];
                // sum from row j to row i
                for (int j = i; j >= 0; j--) {
                    int val = 0;
                    TreeSet<Integer> set = new TreeSet<>();
                    set.add(0);
                    //traverse every column/row and sum up
                    for (int p = 0; p < n; p++) {
                        array[p] = array[p] + (colIsBig ? matrix[j][p] : matrix[p][j]);
                        val = val + array[p];
                        //use  TreeMap to binary search previous sum to get possible
result
                        Integer subres = set.ceiling(val - k);
                        if (null != subres) {
                            res = Math.max(res, val - subres);
                        }
                        set.add(val);
                    }
                }
            }
            return res;
        }
    }
}

// ============================================================
// FILE 347: _364.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 364. Nested List Weight Sum II
 *
 * Given a nested list of integers, return the sum of all integers in the list weighted
by their depth.

 Each element is either an integer, or a list -- whose elements may also be integers or
```

```
other lists.

 Different from the previous question where weight is increasing from root to leaf, now
the weight is defined from bottom up. i.e., the leaf level integers have weight 1, and
the root level integers have the largest weight.

 Example 1:
 Given the list [[1,1],2,[1,1]], return 8. (four 1's at depth 1, one 2 at depth 2)

 Example 2:
 Given the list [1,[4,[6]]], return 17. (one 1 at depth 3, one 4 at depth 2, and one 6 at
depth 1; 1*3 + 4*2 + 6*1 = 17)*/
public class _364 {

    public static class Solution1 {
        public int depthSumInverse(List<NestedInteger> nestedList) {
            Queue<NestedInteger> q = new LinkedList<>();
            for (NestedInteger next : nestedList) {
                q.offer(next);
            }
            int prev = 0;
            int total = 0;

            while (!q.isEmpty()) {
                int size = q.size();
                int levelSum = 0;
                for (int i = 0; i < size; i++) {
                    NestedInteger next = q.poll();
                    if (next.isInteger()) {
                        levelSum += next.getInteger();
                    } else {
                        List<NestedInteger> list = next.getList();
                        for (NestedInteger n : list) {
                            q.offer(n);
                        }
                    }
                }
                prev += levelSum;
                total += prev;
            }
            return total;
        }
    }

}

// ============================================================
// FILE 348: _365.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 365. Water and Jug Problem
```

```
 *
 * You are given two jugs with capacities x and y litres.
 * There is an infinite amount of water supply available.
 * You need to determine whether it is possible to measure exactly z litres using these
two jugs.
 *
 If z liters of water is measurable, you must have z liters of water contained within one
or both buckets by the end.

 Operations allowed:

 Fill any of the jugs completely with water.
 Empty any of the jugs.
 Pour water from one jug into another till the other jug is completely full or the first
jug itself is empty.

 Example 1: (From the famous "Die Hard" example)
 Input: x = 3, y = 5, z = 4
 Output: True

 Example 2:
 Input: x = 2, y = 6, z = 5
 Output: False

 */
public class _365 {

    public static class Solution1 {
        public boolean canMeasureWater(int x, int y, int z) {
            if (x + y < z) {
                return false;
            }
            if (x == z || y == z || x + y == z) {
                return true;
            }
            return z % gcd(x, y) == 0;
        }

        int gcd(int x, int y) {
            while (y != 0) {
                int temp = y;
                y = x % y;
                x = temp;
            }
            return x;
        }
    }
}

// ============================================================
// FILE 349: _366.java
// ============================================================

package com.fishercoder.solutions;
```

```
import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 366. Find Leaves of Binary Tree
 *
 * Given a binary tree, collect a tree's nodes as if you were doing this:
 * Collect and remove all leaves, repeat until the tree is empty.

 Example:
 Given binary tree
     1
    / \
   2   3
  / \
 4  5
 Returns [4, 5, 3], [2], [1].

 Explanation:
 1. Removing the leaves [4, 5, 3] would result in this tree:

   1
  /
 2
 2. Now removing the leaf [2] would result in this tree:

 1
 3. Now removing the leaf [1] would result in the empty tree:

 []
 Returns [4, 5, 3], [2], [1].

 */
public class _366 {

    public static class Solution1 {
        List<List<Integer>> result = new ArrayList<>();

        public List<List<Integer>> findLeaves(TreeNode root) {
            dfs(root);
            return result;
        }

        int dfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int level = Math.max(dfs(root.left), dfs(root.right)) + 1;
            if (result.size() < level) {
                result.add(new ArrayList<>());
            }
```

```
            result.get(level - 1).add(root.val);
            return level;
        }
    }
}

// ============================================================
// FILE 350: _367.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 367. Valid Perfect Square
 *
 * Given a positive integer num, write a function which returns True if num is a perfect
square else False.

 Note: Do not use any built-in library function such as sqrt.

 Example 1:

 Input: 16
 Returns: True
 Example 2:

 Input: 14
 Returns: False
 */
public class _367 {

        public static class Solution1 {
                public boolean isPerfectSquare(int num) {
                        long i = 1;
                        long temp = 1;
                        while (temp < num) {
                                i += 2;
                                temp += i;
                        }
                        return temp == num;
                }
        }
}

// ============================================================
// FILE 351: _368.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

```
/**
 * 368. Largest Divisible Subset
 *
 * Given a set of distinct positive integers, find the largest subset such that every
pair (Si, Sj) of elements in this subset satisfies: Si % Sj = 0 or Sj % Si = 0.

 If there are multiple solutions, return any subset is fine.

 Example 1:

 nums: [1,2,3]

 Result: [1,2] (of course, [1,3] will also be ok)
 Example 2:

 nums: [1,2,4,8]

 Result: [1,2,4,8]
 */
public class _368 {

    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/49652/classic-dp-solution-similar-
to-lis-o-n-2 */
        public List<Integer> largestDivisibleSubset(int[] nums) {
            int len = nums.length;
            int[] count = new int[len];
            int[] pre = new int[len];
            Arrays.sort(nums);
            int max = 0;
            int index = -1;
            for (int i = 0; i < len; i++) {
                count[i] = 1;
                pre[i] = -1;
                for (int j = i - 1; j >= 0; j--) {
                    if (nums[i] % nums[j] == 0) {
                        if (1 + count[j] > count[i]) {
                            count[i] = count[j] + 1;
                            pre[i] = j;
                        }
                    }
                }
                if (count[i] > max) {
                    max = count[i];
                    index = i;
                }
            }
            List<Integer> res = new ArrayList<>();
            while (index != -1) {
                res.add(nums[index]);
                index = pre[index];
            }
            return res;
        }
```

```
    }

}

// ============================================================
// FILE 352: _369.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 369. Plus One Linked List
 *
 * Given a non-negative integer represented as non-empty a singly linked list of digits,
plus one to the integer.

 You may assume the integer do not contain any leading zero, except the number 0 itself.

 The digits are stored such that the most significant digit is at the head of the list.

 Example:
 Input:
 1->2->3

 Output:
 1->2->4
 */
public class _369 {

    public static class Solution1 {
        public ListNode plusOne(ListNode head) {
            //get the length of the list and take out the value of each node and store
them into an array
            ListNode temp = head;
            int len = 0;
            while (temp != null) {
                len++;
                temp = temp.next;
            }

            int[] nums = new int[len];
            temp = head;
            int j = 0;
            while (temp != null) {
                nums[j++] = temp.val;
                temp = temp.next;
            }

            //plus one into this array: nums
            for (int i = len - 1; i >= 0; i--) {
                if (nums[i] != 9) {
                    nums[i]++;
```

```
                    break;
                } else {
                    nums[i] = 0;
                }
            }

            //still assuming the first value in the list should not be zero as it's
representing a valid number, although it's in a list
            ListNode pre = new ListNode(-1);
            if (nums[0] == 0) {
                //in this case, let's just construct a new linked list and return: only
first node value is 1, all the rest is 0
                ListNode newHead = new ListNode(1);
                ListNode result = newHead;
                int count = 0;
                while (count++ < len) {
                    newHead.next = new ListNode(0);
                    newHead = newHead.next;
                }
                return result;
            } else {
                pre.next = head;
                for (int i = 0; i < len; i++) {
                    head.val = nums[i];
                    head = head.next;
                }
                return pre.next;
            }
        }
    }

}

// ============================================================
// FILE 353: _370.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 370. Range Addition
 *
 * Assume you have an array of length n initialized with all 0's and are given k update
operations.
 * Each operation is represented as a triplet: [startIndex, endIndex, inc]
 * which increments each element of subarray A[startIndex ... endIndex] (startIndex and
endIndex inclusive) with inc.
 * Return the modified array after all k operations were executed.

 Example:

 Given:

 length = 5,
```

```
 updates = [
 [1,  3,  2],
 [2,  4,  3],
 [0,  2, -2]
 ]

 Output:

 [-2, 0, 3, 5, 3]
 Explanation:

 Initial state:
 [ 0, 0, 0, 0, 0 ]

 After applying operation [1, 3, 2]:
 [ 0, 2, 2, 2, 0 ]

 After applying operation [2, 4, 3]:
 [ 0, 2, 5, 5, 3 ]

 After applying operation [0, 2, -2]:
 [-2, 0, 3, 5, 3 ]

 Hint:
 Thinking of using advanced data structures? You are thinking it too complicated.
 For each update operation, do you really need to update all elements between i and j?
 Update only the first and end element is sufficient.
 The optimal time complexity is O(k + n) and uses O(1) extra space.
 */

public class _370 {
    public static class Solution1 {
        public int[] getModifiedArray(int length, int[][] updates) {
            int[] nums = new int[length];
            int k = updates.length;
            for (int i = 0; i < k; i++) {
                int start = updates[i][0];
                int end = updates[i][1];
                int inc = updates[i][2];
                nums[start] += inc;
                if (end < length - 1) {
                    nums[end + 1] -= inc;
                }
            }

            int sum = 0;
            for (int i = 0; i < length; i++) {
                sum += nums[i];
                nums[i] = sum;
            }
            return nums;
        }
    }
}
```

```
// ============================================================
// FILE 354: _371.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 371. Sum of Two Integers
 *
 * Calculate the sum of two integers a and b, but you are not allowed to use the operator
+ and -.

 Example:
 Given a = 1 and b = 2, return 3.
 */
public class _371 {

    public static class Solution1 {
        /** reference: http://stackoverflow.com/questions/9070937/adding-two-numbers-
without-operator-clarification */
        public int getSum(int a, int b) {
            if (b == 0) {
                return a;
            }
            int sum = a ^ b;
            int carry = (a & b) << 1;
            return getSum(sum, carry);
        }
    }
}

// ============================================================
// FILE 355: _372.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 372. Super Pow
 *
 * Your task is to calculate ab mod 1337 where a is a positive integer and b is an
extremely large positive integer given in the form of an array.

 Example1:

 a = 2
 b = [3]

 Result: 8
 Example2:

 a = 2
 b = [1,0]
```

```
 Result: 1024
 */
public class _372 {
    public static class Solution1 {
        public int superPow(int a, int[] b) {
            if (a % 1337 == 0) {
                return 0;
            }
            int p = 0;
            for (int i : b) {
                p = (p * 10 + i) % 1140;
            }
            if (p == 0) {
                p += 1140;
            }
            return power(a, p, 1337);
        }

        private int power(int a, int n, int mod) {
            a %= mod;
            int result = 1;
            while (n != 0) {
                if ((n & 1) != 0) {
                    result = result * a % mod;
                }
                a = a * a % mod;
                n >>= 1;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 356: _373.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 373. Find K Pairs with Smallest Sums
 *
 * You are given two integer arrays nums1 and nums2 sorted in ascending order and an
integer k.
 * Define a pair (u,v) which consists of one element from the first array and one element
from the second array.
 * Find the k pairs (u1,v1),(u2,v2) ...(uk,vk) with the smallest sums.
```

```
 Example 1:
 Given nums1 = [1,7,11], nums2 = [2,4,6],  k = 3
 Return: [1,2],[1,4],[1,6]
 The first 3 pairs are returned from the sequence:
 [1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]

 Example 2:
 Given nums1 = [1,1,2], nums2 = [1,2,3],  k = 2
 Return: [1,1],[1,1]
 The first 2 pairs are returned from the sequence:
 [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]

 Example 3:
 Given nums1 = [1,2], nums2 = [3],  k = 3
 Return: [1,3],[2,3]
 All possible pairs are returned from the sequence:
 [1,3],[2,3]

 */
public class _373 {
    public static class Solution1 {

        final int[][] neighbors = new int[][] {{0, 1}, {1, 0}};

        public List<int[]> kSmallestPairs(int[] nums1, int[] nums2, int k) {
            List<int[]> result = new ArrayList<>();
            if (nums1 == null
                || nums2 == null
                || k == 0
                || nums1.length == 0
                || nums2.length == 0) {
                return result;
            }
            Queue<Pair> meanHeap = new PriorityQueue<>();
            meanHeap.offer(new Pair(0, 0, nums1[0] + nums2[0]));
            boolean[][] visited = new boolean[nums1.length][nums2.length];
            visited[0][0] = true;//we start form (0,0), so mark it as visited
            while (k > 0 && !meanHeap.isEmpty()) {
                Pair pair = meanHeap.poll();
                result.add(new int[] {nums1[pair.row], nums2[pair.col]});
                k--;
                for (int[] neighbor : neighbors) {
                    int nextRow = pair.row + neighbor[0];
                    int nextCol = pair.col + neighbor[1];
                    if (nextRow < 0
                        || nextCol < 0
                        || nextRow >= nums1.length
                        || nextCol >= nums2.length
                        || visited[nextRow][nextCol]) {
                        continue;
                    }
                    visited[nextRow][nextCol] = true;
                    meanHeap.offer(new Pair(nextRow, nextCol, nums1[nextRow] +
nums2[nextCol]));
```

```
                }
            }

            return result;
        }

        class Pair implements Comparable<Pair> {
            int row;
            int col;
            int sum;

            public Pair(int row, int col, int sum) {
                this.row = row;
                this.col = col;
                this.sum = sum;
            }

            @Override
            public int compareTo(Pair that) {
                return this.sum - that.sum;
            }
        }
    }
}

// ============================================================
// FILE 357: _374.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 374. Guess Number Higher or Lower
 * We are playing the Guess Game. The game is as follows:
 * I pick a number from 1 to n. You have to guess which number I picked.
 * Every time you guess wrong, I'll tell you whether the number is higher or lower.
 * You call a pre-defined API guess(int num) which returns 3 possible results (-1, 1, or
0):

 -1 : My number is lower
 1 : My number is higher
 0 : Congrats! You got it!
 Example:
 n = 10, I pick 6.
 Return 6.

 */
public class _374 {
    public static class Solution1 {
        /**
         * The core problem/trouble to solve this problem is to figure out the problem
description: this
         * API: guess(int num) means to take your guess num and let you know if your
guessed num is
```

```
         * bigger or smaller than the answer. That's why if num > target, it returns -1
which means the
         * target is smaller than your guess!!!
         */

        public int guessNumber(int n) {
            int left = 1;
            int right = n;
            while (left + 1 < right) {
                int mid = left + (right - left) / 2;
                int g = guess(mid);
                if (g == 0) {
                    return mid;
                } else if (g > 0) {
                    left = mid;
                } else {
                    right = mid;
                }
            }
            if (guess(left) == 0) {
                return left;
            }
            return right;
        }

        /**
         * This is a fake guess method that I wrote just to compile/test, I'll have to
change it to
         * another number other than 6 based on the number to be found.
         */
        private int guess(int num) {
            if (num > 6) {
                return -1;
            } else if (num < 6) {
                return 1;
            } else {
                return 0;
            }
        }
    }

}

// ============================================================
// FILE 358: _375.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 375. Guess Number Higher or Lower II
 *
 * We are playing the Guess Game. The game is as follows:
 * I pick a number from 1 to n. You have to guess which number I picked.
```

```
 * Every time you guess wrong, I'll tell you whether the number I picked is higher or
lower.
 * However, when you guess a particular number x, and you guess wrong, you pay $x. You
win the game when you guess the number I picked.

 Example:

 n = 10, I pick 8.

 First round:  You guess 5, I tell you that it's higher. You pay $5.
 Second round: You guess 7, I tell you that it's higher. You pay $7.
 Third round:  You guess 9, I tell you that it's lower. You pay $9.

 Game over. 8 is the number I picked.

 You end up paying $5 + $7 + $9 = $21.
 Given a particular n ≥ 1, find out how much money you need to have to guarantee a win.

 Hint:

 The best strategy to play the game is to minimize the maximum loss you could possibly
face. Another strategy is to minimize the expected loss. Here, we are interested in the
first scenario.
 Take a small example (n = 3). What do you end up paying in the worst case?
 Check out this article if you're still stuck.
 The purely recursive implementation of minimax would be worthless for even a small n.
You MUST use dynamic programming.
 As a follow-up, how would you modify your code to solve the problem of minimizing the
expected loss, instead of the worst-case loss?
 */
public class _375 {
    public static class Solution1 {
        public int getMoneyAmount(int n) {
            int[][] table = new int[n + 1][n + 1];
            return dp(table, 1, n);
        }

        private int dp(int[][] table, int s, int e) {
            if (s >= e) {
                return 0;
            }
            if (table[s][e] != 0) {
                return table[s][e];
            }
            int res = Integer.MAX_VALUE;
            for (int i = s; i <= e; i++) {
                int temp = i + Math.max(dp(table, s, i - 1), dp(table, i + 1, e));
                res = Math.min(res, temp);
            }
            table[s][e] = res;
            return res;
        }
    }
```

```
    public static class Solution2 {
        public int getMoneyAmount(int n) {
            if (n == 1) {
                return 0;
            }
            int[][] dp = new int[n + 1][n + 1];
            for (int x = 1; x < n; x++) {
                for (int i = 0; i + x <= n; i++) {
                    int j = i + x;
                    dp[i][j] = Integer.MAX_VALUE;
                    for (int k = i; k <= j; k++) {
                        dp[i][j] = Math.min(dp[i][j], k + Math.max(k - 1 >= i ? dp[i][k -
1] : 0, j >= k + 1 ? dp[k + 1][j] : 0));
                    }
                }
            }
            for (int i = 0; i < n + 1; i++) {
                for (int j = 0; j < n + 1; j++) {
                    System.out.print(dp[i][j] + " ");
                }
                System.out.println();
            }
            return dp[1][n];
        }
    }
}

// ============================================================
// FILE 359: _376.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 376. Wiggle Subsequence
 *
 * A sequence of numbers is called a wiggle sequence if the differences between
successive numbers strictly
 * alternate between positive and negative.
 * The first difference (if one exists) may be either positive or negative.
 * A sequence with fewer than two elements is trivially a wiggle sequence.

 For example, [1,7,4,9,2,5] is a wiggle sequence because the differences
 (6,-3,5,-7,3) are alternately positive and negative.
 In contrast, [1,4,7,2,5] and [1,7,4,5,5] are not wiggle sequences,
 the first because its first two differences are positive and the second because its last
difference is zero.

 Given a sequence of integers, return the length of the longest subsequence that is a
wiggle sequence.
 A subsequence is obtained by deleting some number of elements (eventually, also zero)
 from the original sequence, leaving the remaining elements in their original order.

 Examples:
```

```
 Input: [1,7,4,9,2,5]
 Output: 6
 The entire sequence is a wiggle sequence.

 Input: [1,17,5,10,13,15,10,5,16,8]
 Output: 7
 There are several subsequences that achieve this length. One is [1,17,10,13,10,16,8].

 Input: [1,2,3,4,5,6,7,8,9]
 Output: 2

 Follow up:
 Can you do it in O(n) time?
 */
public class _376 {

    public static class Solution1 {
        public int wiggleMaxLength(int[] nums) {
            if (nums.length < 2) {
                return nums.length;
            }
            int prevDiff = nums[1] - nums[0];
            int count = (prevDiff != 0) ? 2 : 1;
            for (int i = 2; i < nums.length; i++) {
                int diff = nums[i] - nums[i - 1];
                /**ATTN: prevDiff could be zero. e.g. [3,3,3,2,5]
                 * but diff needs to exactly greater than zero*/
                if ((prevDiff <= 0 && diff > 0) || (prevDiff >= 0) && diff < 0) {
                    count++;
                    prevDiff = diff;
                }
            }
            return count;
        }
    }
}

// ================================================================
// FILE 360: _377.java
// ================================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 377. Combination Sum IV
 *
 * Given an integer array with all positive numbers and no duplicates,
 * find the number of possible combinations that add up to a positive integer target.
```

```
 Example:

 nums = [1, 2, 3]
 target = 4

 The possible combination ways are:
 (1, 1, 1, 1)
 (1, 1, 2)
 (1, 2, 1)
 (1, 3)
 (2, 1, 1)
 (2, 2)
 (3, 1)

 Note that different sequences are counted as different combinations.

 Therefore the output is 7.

 Follow up:
 What if negative numbers are allowed in the given array?
 How does it change the problem?
 What limitation we need to add to the question to allow negative numbers?
 */

public class _377 {

    public static class Solution1 {
        /**
         * this normal backtracking recursive solution will end up in MLE by this
testcase: [4,2,1], 32
         */
        public int combinationSum4(int[] nums, int target) {
            List<List<Integer>> result = new ArrayList();
            Arrays.sort(nums);
            backtracking(nums, target, new ArrayList(), result);
            return result.size();
        }

        private void backtracking(int[] nums, int target, List<Integer> list,
                                  List<List<Integer>> result) {
            if (target == 0) {
                result.add(new ArrayList(list));
            } else if (target > 0) {
                for (int i = 0; i < nums.length; i++) {
                    list.add(nums[i]);
                    backtracking(nums, target - nums[i], list, result);
                    list.remove(list.size() - 1);
                }
            }
        }
    }

    public static class Solution2 {
```

```
        /**
         * Since we don't need to get all of the combinations, instead,
         * we only need to get the possible count, I can use only a count instead of
"List<List<Integer>> result"
         * However, it also ended up in TLE by this testcase: [1,2,3], 32
         */
        public static int count = 0;

        public int combinationSum4(int[] nums, int target) {
            Arrays.sort(nums);
            backtracking(nums, target, new ArrayList());
            return count;
        }

        private void backtracking(int[] nums, int target, List<Integer> list) {
            if (target == 0) {
                count++;
            } else if (target > 0) {
                for (int i = 0; i < nums.length; i++) {
                    list.add(nums[i]);
                    backtracking(nums, target - nums[i], list);
                    list.remove(list.size() - 1);
                }
            }
        }
    }

    public static class Solution3 {
        /**
         * Time: O(n^2)
         * Space: O(n)
         *
         * Since this question doesn't require to return all the combination result,
instead, it just wants one number, we could use DP
         * the idea is similar to Climbing Stairs.
         *
         * The idea is very clear as the code speaks for itself:
         * It's easy to find the routine
         * dp[0] = 0;
         * dp[1] = 1;
         * ...
         *
         * Reference: https://discuss.leetcode.com/topic/52186/my-3ms-java-dp-solution
         */
        public int combinationSum4(int[] nums, int target) {
            Arrays.sort(nums);
            int[] result = new int[target + 1];
            for (int i = 1; i < result.length; i++) {
                for (int num : nums) {
                    if (num > i) {
                        break;
                    } else if (num == i) {
                        result[i]++;
                    } else {
```

```
                        result[i] += result[i - num];
                    }
                }
            }
            return result[target];
        }
    }

    public static class Solution4 {
        /**
         * Time: O(n)
         * Space: O(n)
         *
         * Reference: https://discuss.leetcode.com/topic/52255/java-recursion-solution-
using-hashmap-as-memory
         */
        public static Map<Integer, Integer> map = new HashMap<>();//need to remove public
static before submitting on Leetcode as it doesn't reset static variables

        public int combinationSum4(int[] nums, int target) {
            if (nums == null || nums.length == 0 || target < 0) {
                return 0;
            }
            if (target == 0) {
                return 1;
            }
            if (map.containsKey(target)) {
                return map.get(target);
            }
            int count = 0;
            for (int num : nums) {
                count += combinationSum4(nums, target - num);
            }
            map.put(target, count);
            return count;
        }
    }
}

// ============================================================
// FILE 361: _378.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 378. Kth Smallest Element in a Sorted Matrix
 * Given a n x n matrix where each of the rows and columns are
 * sorted in ascending order, find the kth smallest element in the matrix.
```

```
Note that it is the kth smallest element in the sorted order, not the kth distinct
element.

Example:

matrix = [
   [ 1,  5,  9],
   [10, 11, 13],
   [12, 13, 15]
],
k = 8,

return 13.

Note:
You may assume k is always valid, 1 ≤ k ≤ n2.
 */

public class _378 {
    public static class Solution1 {
        /**
         * brute force made it AC'ed, extreme test case needed for OJ
         */
        public int kthSmallest(int[][] matrix, int k) {
            List<Integer> list = new ArrayList();
            int n = matrix.length;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    list.add(matrix[i][j]);
                }
            }
            Collections.sort(list);
            return list.get(k - 1);
        }
    }

    public static class Solution2 {
        /**
         * Binary Search : The idea is to pick a mid number, then compare it with the
elements in each row, we start form
         * end of row util we find the element is less than the mid, the left side
element is all less than mid; keep tracking elements
         * that less than mid and compare with k, then update the k.
         */
        public int kthSmallestBS(int[][] matrix, int k) {
            int row = matrix.length - 1;
            int col = matrix[0].length - 1;
            int lo = matrix[0][0];
            int hi = matrix[row][col];
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                int count = 0;
                int j = col;
                for (int i = 0; i <= row; i++) {
```

```
                    while (j >= 0 && matrix[i][j] > mid) {
                        j--;
                    }
                    count += (j + 1);
                }
                if (count < k) {
                    lo = mid + 1;
                } else {
                    hi = mid;
                }
            }
            return lo;
        }
    }
}

// ============================================================
// FILE 362: _379.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 379. Design Phone Directory
 *
 * Design a Phone Directory which supports the following operations:

 get: Provide a number which is not assigned to anyone.
 check: Check if a number is available or not.
 release: Recycle or release a number.
 Example:

 // Init a phone directory containing a total of 3 numbers: 0, 1, and 2.
 PhoneDirectory directory = new PhoneDirectory(3);

 // It can return any available phone number. Here we assume it returns 0.
 directory.get();

 // Assume it returns 1.
 directory.get();

 // The number 2 is available, so return true.
 directory.check(2);

 // It returns 2, the only number that is left.
 directory.get();

 // The number 2 is no longer available, so return false.
 directory.check(2);
```

```
 // Release number 2 back to the pool.
 directory.release(2);

 // Number 2 is available again, return true.
 directory.check(2);
 */
public class _379 {

    public static class Solution1 {
        private class PhoneDirectory {
            private Queue<Integer> phoneDir;
            private Set<Integer> used;

            /**
             * Initialize your data structure here
             *
             * @param maxNumbers - The maximum numbers that can be stored in the phone
directory.
             */
            public PhoneDirectory(int maxNumbers) {
                phoneDir = new LinkedList();
                int number = 0;
                while (maxNumbers-- > 0) {
                    phoneDir.add(number++);
                }
                used = new HashSet();
            }

            /**
             * Provide a number which is not assigned to anyone.
             *
             * @return - Return an available number. Return -1 if none is available.
             */
            public int get() {
                if (phoneDir.peek() == null) {
                    return -1;
                }
                int newNumber = phoneDir.poll();
                used.add(newNumber);
                return newNumber;
            }

            /**
             * Check if a number is available or not.
             */
            public boolean check(int number) {
                return !used.contains(number);
            }

            /**
             * Recycle or release a number.
             */
            public void release(int number) {
```

```
                if (used.remove(number)) {
                    phoneDir.add(number);
                }
            }
        }
    }
}

// ============================================================
// FILE 363: _380.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.List;
import java.util.ArrayList;
import java.util.Map;
import java.util.Random;

/**
 * 380. Insert Delete GetRandom O(1)
 * Design a data structure that supports all following operations in average O(1) time.
 *
 * insert(val): Inserts an item val to the set if not already present.
 * remove(val): Removes an item val from the set if present.
 * getRandom: Returns a random element from current set of elements. Each element must
have the same probability of being returned.
 * Example:
 *
 * // Init an empty set.
 * RandomizedSet randomSet = new RandomizedSet();
 *
 * // Inserts 1 to the set. Returns true as 1 was inserted successfully.
 * randomSet.insert(1);
 *
 * // Returns false as 2 does not exist in the set.
 * randomSet.remove(2);
 *
 * // Inserts 2 to the set, returns true. Set now contains [1,2].
 * randomSet.insert(2);
 *
 * // getRandom should return either 1 or 2 randomly.
 * randomSet.getRandom();
 *
 * // Removes 1 from the set, returns true. Set now contains [2].
 * randomSet.remove(1);
 *
 * // 2 was already in the set, so return false.
 * randomSet.insert(2);
 *
 * // Since 2 is the only number in the set, getRandom always return 2.
 * randomSet.getRandom();
 */
```

```
public class _380 {

    public static class Solution1 {
        /**
         * credit: https://leetcode.com/problems/insert-delete-getrandom-
o1/discuss/85401/Java-solution-using-a-HashMap-and-an-ArrayList-along-with-a-follow-up.-
(131-ms)
         */
        public static class RandomizedSet {
            Map<Integer, Integer> locationMap;
            List<Integer> list;
            Random random;

            /**
             * Initialize your data structure here.
             */
            public RandomizedSet() {
                locationMap = new HashMap<>();
                list = new ArrayList<>();
                random = new Random();
            }

            /**
             * Inserts a value to the set. Returns true if the set did not already
contain the specified element.
             */
            public boolean insert(int val) {
                if (locationMap.containsKey(val)) {
                    return false;
                } else {
                    locationMap.put(val, list.size());
                    list.add(val);
                    return true;
                }
            }

            /**
             * Removes a value from the set. Returns true if the set contained the
specified element.
             */
            public boolean remove(int val) {
                if (!locationMap.containsKey(val)) {
                    return false;
                } else {
                    int location = locationMap.get(val);
                    /**if it's not the last one, then swap the last one with this val*/
                    if (location < list.size() - 1) {
                        int lastOne = list.get(list.size() - 1);
                        list.set(location, lastOne);
                        locationMap.put(lastOne, location);
                    }
                    locationMap.remove(val);
                    list.remove(list.size() - 1);
```

```
                    return true;
                }
            }

            /**
             * Get a random element from the set.
             */
            public int getRandom() {
                return list.get(random.nextInt(list.size()));
            }

        }
    }

    /**
     * Your _380 object will be instantiated and called as such:
     * _380 obj = new _380();
     * boolean param_1 = obj.insert(val);
     * boolean param_2 = obj.delete(val);
     * int param_3 = obj.getRandom();
     */
    public static class Solution2 {
        /**This is not ideal and not meeting average O(1) time for all three
operations.*/
        public static class RandomizedSet {

            Map<Integer, Integer> forwardMap;
                //key is auto increment index, value if the inserted val
            Map<Integer, Integer> reverseMap;//the other way around
            int index;
            Random random;

            /**
             * Initialize your data structure here.
             */
            public RandomizedSet() {
                forwardMap = new HashMap();
                reverseMap = new HashMap();
                index = 0;
                random = new Random();
            }

            /**
             * Inserts a value to the set. Returns true if the set did not already
contain the specified
             * element.
             */
            public boolean insert(int val) {
                if (forwardMap.containsValue(val)) {
                    return false;
                } else {
                    forwardMap.put(index, val);
                    reverseMap.put(val, index++);
                    return true;
```

```
                }
            }

            /**
             * Deletes a value from the set. Returns true if the set contained the
specified element.
             */
            public boolean remove(int val) {
                if (forwardMap.containsValue(val)) {
                    int key = reverseMap.get(val);
                    reverseMap.remove(val);
                    forwardMap.remove(key);
                    return true;
                } else {
                    return false;
                }
            }

            /**
             * Get a random element from the set.
             */
            public int getRandom() {
                int max = forwardMap.size();
                if (max == 1) {
                    return forwardMap.get(index - 1);
                }
                int randomNum = random.nextInt(max);
                while (!forwardMap.containsKey(randomNum)) {
                    randomNum = random.nextInt(max);
                }
                return forwardMap.get(randomNum);
            }
        }
    }
}

// ============================================================
// FILE 364: _381.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.Random;

/**381. Insert Delete GetRandom O(1) - Duplicates allowed
 *
Design a data structure that supports all following operations in average O(1) time.

Note: Duplicate elements are allowed.
insert(val): Inserts an item val to the collection.
remove(val): Removes an item val from the collection if present.
getRandom: Returns a random element from current collection of elements. The probability
```

```
of each element being returned is linearly related to the number of same value the
collection contains.
Example:

// Init an empty collection.
_381 collection = new _381();

// Inserts 1 to the collection. Returns true as the collection did not contain 1.
collection.insert(1);

// Inserts another 1 to the collection. Returns false as the collection contained 1.
Collection now contains [1,1].
collection.insert(1);

// Inserts 2 to the collection, returns true. Collection now contains [1,1,2].
collection.insert(2);

// getRandom should return 1 with the probability 2/3, and returns 2 with the probability
1/3.
collection.getRandom();

// Removes 1 from the collection, returns true. Collection now contains [1,2].
collection.remove(1);

// getRandom should return 1 and 2 both equally likely.
collection.getRandom();*/
public class _381 {
    public static class Solution1 {

        Map<Integer, Integer> forwardMap;
        //key is the to-be-inserted number, value is its auto-incremented index
        Map<Integer, Integer> reverseMap;//the other way around
        int index;
        Random rand;

        /**
         * Initialize your data structure here.
         */
        public Solution1() {
            forwardMap = new HashMap();
            reverseMap = new HashMap();
            index = 0;
            rand = new Random();
        }

        /**
         * Inserts a value to the collection. Returns true if the collection did not
already contain the
         * specified element.
         */
        public boolean insert(int val) {
            boolean contains;
            if (reverseMap.containsValue(val)) {
                contains = true;
```

```
            } else {
                contains = false;
            }
            forwardMap.put(val,
                index);//this will overwrite the preivous key with a new index if the key
already exists
            reverseMap.put(index, val);
            index++;
            return contains;
        }

        /**
         * Removes a value from the collection. Returns true if the collection contained
the specified
         * element.
         */
        public boolean remove(int val) {
            boolean contains;
            if (reverseMap.containsValue(val)) {
                contains = true;
                if (forwardMap.containsKey(val)) {
                    int i = forwardMap.get(val);
                    forwardMap.remove(val);
                    reverseMap.remove(i);
                } else {
                    //remove the entry in revserve map that has val as its value
                    reverseMap.values().remove(val);
                }
            } else {
                contains = false;
            }
            return contains;
        }

        /**
         * Get a random element from the collection.
         */
        public int getRandom() {
            int randNum = rand.nextInt(index);
            while (!reverseMap.containsKey(randNum)) {
                randNum = rand.nextInt(index);
            }
            return reverseMap.get(randNum);
        }
    }
}

// ============================================================
// FILE 365: _382.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;
```

```
import java.util.HashMap;
import java.util.Map;
import java.util.Random;

/**382. Linked List Random Node
 *
Given a singly linked list, return a random node's value from the linked list. Each node
must have the same probability of being chosen.

Follow up:
What if the linked list is extremely large and its length is unknown to you? Could you
solve this efficiently without using extra space?

Example:

// Init a singly linked list [1,2,3].
ListNode head = new ListNode(1);
head.next = new ListNode(2);
head.next.next = new ListNode(3);
Solution solution = new Solution(head);

// getRandom() should return either 1, 2, or 3 randomly. Each element should have equal
probability of returning.
solution.getRandom();
*/
public class _382 {

    public static class Solution1 {
        private Map<Integer, ListNode> map;
        private Random rand;

        /**
         * @param head The linked list's head. Note that the head is guanranteed to be
not null, so it contains at least one node.
         */
        public Solution1(ListNode head) {
            map = new HashMap();
            rand = new Random();
            int i = 0;
            while (head != null) {
                map.put(i++, head);
                head = head.next;
            }
        }

        /**
         * Returns a random node's value.
         */
        public int getRandom() {
            return map.get(rand.nextInt(map.size())).val;
        }
    }
}
```

```
// ============================================================
// FILE 366: _383.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 383 Ransom Note
 *
 * Given an arbitrary ransom note string and another string containing letters
 * from all the magazines, write a function that will return true if
 * the ransom note can be constructed from the magazines ; otherwise, it will return
false.
 * Each letter in the magazine string can only be used once in your ransom note.

 Note:

 You may assume that both strings contain only lowercase letters.
 canConstruct("a", "b") -> false
 canConstruct("aa", "ab") -> false
 canConstruct("aa", "aab") -> true
*/
public class _383 {

    public static class Solution1 {
        public boolean canConstruct(String ransomNote, String magazine) {
            char[] mchars = magazine.toCharArray();
            int[] mcnt = new int[256];
            for (int i = 0; i < mchars.length; i++) {
                mcnt[mchars[i] - 'a']++;
            }

            char[] rchars = ransomNote.toCharArray();
            for (int i = 0; i < rchars.length; i++) {
                if (mcnt[rchars[i] - 'a'] <= 0) {
                    return false;
                }
                mcnt[rchars[i] - 'a']--;
            }
            return true;
        }
    }

}

// ============================================================
// FILE 367: _384.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 384. Shuffle an Array
```

```
Shuffle a set of numbers without duplicates.

Example:

// Init an array with set 1, 2, and 3.
int[] nums = {1,2,3};
Solution solution = new Solution(nums);

// Shuffle the array [1,2,3] and return its result.
 Any permutation of [1,2,3] must equally likely to be returned.
solution.shuffle();

// Resets the array back to its original configuration [1,2,3].
solution.reset();

// Returns the random shuffling of array [1,2,3].
solution.shuffle();*/
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.Random;

public class _384 {

    public static class Solution1 {
        //Note: the problem states that this is a set without duplicates which makes
building all combinations easier

        private List<List<Integer>> combinations;
        private int[] original;
        private Random random;

        public Solution1(int[] nums) {
            original = nums;
            random = new Random();
            combinations = buildAllComb(nums);
        }

        //insert next value into all possible positions, I wrote this method myself, of
course it could be simplified to not use a queue
        //but it just naturally came into my mind that I used a queue
        private List<List<Integer>> buildAllComb(int[] nums) {
            List<List<Integer>> result = new ArrayList();
            if (nums == null || nums.length == 0) {
                return result;
            }

            List<Integer> list = new ArrayList<Integer>();
            list.add(nums[0]);
            Queue<List<Integer>> q = new LinkedList();
            q.offer(list);
            for (int i = 1; i < nums.length; i++) {
                int qSize = q.size();
```

```
                for (int k = 0; k < qSize; k++) {
                    List<Integer> currList = q.poll();
                    for (int j = 0; j <= currList.size(); j++) {
                        List<Integer> newL = new ArrayList<Integer>(currList);
                        newL.add(j, nums[i]);
                        q.offer(newL);
                    }
                }
            }
            while (!q.isEmpty()) {
                result.add(q.poll());
            }
            return result;
        }

        /**
         * Resets the array to its original configuration and return it.
         */
        public int[] reset() {
            return original;
        }

        /**
         * Returns a random shuffling of the array.
         */
        public int[] shuffle() {
            if (original == null || original.length == 0) {
                return original;
            }
            int randomIndex = random.nextInt(combinations.size());
            List<Integer> list = combinations.get(randomIndex);
            int[] result = new int[list.size()];
            for (int i = 0; i < list.size(); i++) {
                result[i] = list.get(i);
            }
            return result;
        }
    }
}


// ============================================================
// FILE 368: _385.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.NestedInteger;

import java.util.Stack;

/**
 * 385. Mini Parser
 *
```

```
 * Given a nested list of integers represented as a string,
 * implement a parser to deserialize it.

 Each element is either an integer, or a list --
 whose elements may also be integers or other lists.

 Note: You may assume that the string is well-formed:

 String is non-empty.
 String does not contain white spaces.
 String contains only digits 0-9, [, - ,, ].
 Example 1:

 Given s = "324",

 You should return a NestedInteger object which contains a single integer 324.
 Example 2:

 Given s = "[123,[456,[789]]]",

 Return a NestedInteger object containing a nested list with 2 elements:

 1. An integer containing value 123.
 2. A nested list containing two elements:
 i.  An integer containing value 456.
 ii. A nested list with one element:
 a. An integer containing value 789.*/
public class _385 {

    public static class Solution1 {
        //Lessons: ask the interviewer to clarify the input, for this question, the input
could be "324", "[324]", they are different
        //the former should return a nested integer with one single integer, the latter
should return a nested integer with a list

        //Idea:
        //if it's '[', we just construct a new nested integer and push it onto the stack
        //if it's a number, we parse the whole number and add to the previous nested
integer object
        //if it's ',', we'll just continue;
        //if it's ']', we'll just pop one nested integer from the working stack and
assign it to the result

        public NestedInteger deserialize(String s) {
            if (s == null || s.isEmpty() || s.length() == 0) {
                return new NestedInteger();
            }
            Stack<NestedInteger> workStack = new Stack<>();
            NestedInteger result = null;
            StringBuilder sb = new StringBuilder();
            int i = 0;
            //if it's just a single number, then we'll just return a nested integer with
one integer
            if (s.charAt(i) != '[') {
```

```
                sb.setLength(0);
                while (i < s.length() && ((Character.getNumericValue(s.charAt(i)) < 10
                    && Character.getNumericValue(s.charAt(i)) >= 0) || s.charAt(i) == '-
')) {
                    sb.append(s.charAt(i));
                    i++;
                }
                int num = Integer.parseInt(sb.toString());
                return new NestedInteger(num);
            } else {
                //all other cases, we'll return a nested integer with a list
                while (i < s.length()) {
                    if (s.charAt(i) == '[') {
                        NestedInteger ni = new NestedInteger();
                        // we'll put this one into its last one if there's one on the
workStack
                        if (!workStack.isEmpty()) {
                            NestedInteger lastNi = workStack.pop();
                            lastNi.add(ni);
                            workStack.push(lastNi);// then push it back
                        }
                        workStack.push(ni);
                        i++;
                    } else if (s.charAt(i) == ',') {
                        i++;
                    } else if (s.charAt(i) == ']') {
                        NestedInteger completedNi = workStack.pop();
                        result = completedNi;
                        i++;
                    } else {
                        // then it must be a number
                        sb.setLength(0);
                        while (i < s.length()
                            && ((Character.getNumericValue(s.charAt(i)) < 10 && Character
                            .getNumericValue(s.charAt(i)) >= 0) || s.charAt(i) == '-')) {
                            sb.append(s.charAt(i));
                            i++;
                        }
                        int num = Integer.parseInt(sb.toString());
                        NestedInteger ni = null;
                        if (!workStack.isEmpty()) {
                            ni = workStack.pop();
                        } else {
                            ni = new NestedInteger();
                        }
                        // case 1: if this one contains one integer
                        if (ni.isInteger()) {
                            // we'll add it to this ni
                            ni.add(new NestedInteger(num));
                        } else if (ni.getList() != null && ni.getList().size() != 0) {
                            // case 2: if this one contains a nested integer
                            // we'll get the last nested integer and add this one to it
                            ni.add(new NestedInteger(num));
                        } else {
```

```
                            // case 3: if this is an empty nested integer
                            if (i > 0) {
                                ni.add(new NestedInteger(num));
                            } else {
                                ni.setInteger(num);
                            }
                        }
                        workStack.push(ni);
                        if (i == s.length()) {
                            return ni;// this is for test cases like this: "324", there's
no '[' or ']'
                        }
                    }
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 369: _386.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 386. Lexicographical Numbers
 *
 * Given an integer n, return 1 - n in lexicographical order.
 *
 * For example, given 13, return: [1,10,11,12,13,2,3,4,5,6,7,8,9].
 *
 * Please optimize your algorithm to use less time and space. The input size may be as
large as
 * 5,000,000.
 */
public class _386 {
  public static class Solution1 {
    //Radix sort doesn't apply here! Don't confuse yourself!

    //rewrote their solution from Python to
Java:https://discuss.leetcode.com/topic/54986/python-memory-limit-exceeded-for-problem-
386/17
    public List<Integer> lexicalOrder(int n) {
      List<Integer> result = new ArrayList();
      int i = 1;
      while (true) {
        result.add(i);
        if (i * 10 <= n) {
          i *= 10;
```

```
        } else {
          while (i % 10 == 9 || i == n) {
            i /= 10;
          }
          if (i == 0) {
            return result;
          }
          i++;
        }
      }
    }
  }
}

// ============================================================
// FILE 370: _387.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 387. First Unique Character in a String Given a string, find the first non-repeating
character in
 * it and return it's index. If it doesn't exist, return -1.
 *
 * Examples:
 *
 * s = "leetcode" return 0.
 *
 * s = "loveleetcode", return 2. Note: You may assume the string contain only lowercase
letters.
 */

public class _387 {
  public static class Solution1 {
    public static int firstUniqChar(String s) {
      int[] freq = new int[26];
      for (int i = 0; i < s.length(); i++) {
        freq[s.charAt(i) - 'a']++;
      }
      for (int i = 0; i < s.length(); i++) {
        if (freq[s.charAt(i) - 'a'] == 1) {
          return i;
        }
      }
      return -1;
    }
  }
}

// ============================================================
// FILE 371: _388.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 388. Longest Absolute File Path
 *
 * Suppose we abstract our file system by a string in the following manner:
 *
 * The string "dir\n\tsubdir1\n\tsubdir2\n\t\tfile.ext" represents:
 *
 * dir
 *     subdir1
 *     subdir2
 *         file.ext
 * The directory dir contains an empty sub-directory subdir1 and a sub-directory subdir2
containing a file file.ext.
 *
 * The string
"dir\n\tsubdir1\n\t\tfile1.ext\n\t\tsubsubdir1\n\tsubdir2\n\t\tsubsubdir2\n\t\t\tfile2.ex
t" represents:
 *
 * dir
 *     subdir1
 *         file1.ext
 *         subsubdir1
 *     subdir2
 *         subsubdir2
 *             file2.ext
 * The directory dir contains two sub-directories subdir1 and subdir2. subdir1 contains a
file file1.ext and an empty second-level sub-directory subsubdir1.
 * subdir2 contains a second-level sub-directory subsubdir2 containing a file file2.ext.
 *
 * We are interested in finding the longest (number of characters) absolute path to a
file within our file system.
 * For example, in the second example above, the longest absolute path is
"dir/subdir2/subsubdir2/file2.ext", and its length is 32 (not including the double
quotes).
 *
 * Given a string representing the file system in the above format, return the length of
the longest absolute path to file in the abstracted file system. If there is no file in
the system, return 0.
 *
 * Note:
 *
 * The name of a file contains at least a . and an extension.
 * The name of a directory or sub-directory will not contain a ..
 * Time complexity required: O(n) where n is the size of the input string.
 *
 * Notice that a/aa/aaa/file1.txt is not the longest file path, if there is another path
aaaaaaaaaaaaaaaaaaaaa/sth.png.*/
public class _388 {
    public static class Solution1 {
        public int lengthLongestPath(String input) {
```

```
            Stack<Integer> stack = new Stack();
            int longestLen = 0;
            int currDirLen = 0;
            int i = 0;
            int currLevel = 0;
            int nextLevel = 0;
            boolean isFile = false;
            Character period = '.';
            Character space = ' ';
            while (i < input.length()) {
                currLevel = nextLevel;
                int currStrLen = 0;
                while (i < input.length() && (Character.isLetterOrDigit(input.charAt(i))
                    || period.equals(input.charAt(i)) || space.equals(input.charAt(i))))
{
                    if (period.equals(input.charAt(i))) {
                        isFile = true;
                    }
                    i++;
                    currStrLen++;
                }
                if (isFile) {
                    longestLen = Math.max(longestLen, currDirLen + currStrLen);
                } else {
                    currDirLen += currStrLen + 1;
                    stack.push(currStrLen + 1);
                }

                nextLevel = 0;
                i = i + 1;//increment one to let it pass "\n" and start from "\t"
                while (i < input.length() - 1 && input.substring(i, i + 1).equals("\t"))
{
                    nextLevel++;
                    i = i + 1;
                }

                if (nextLevel < currLevel) {
                    int j = 0;
                    if (isFile) {
                        while (!stack.isEmpty() && j < (currLevel - nextLevel)) {
                            currDirLen -= stack.pop();
                            j++;
                        }
                    } else {
                        while (!stack.isEmpty() && j <= (currLevel - nextLevel)) {
                            currDirLen -= stack.pop();
                            j++;
                        }
                    }
                } else if (nextLevel == currLevel) {
                    if (!isFile && !stack.isEmpty()) {
                        currDirLen -= stack.pop();
                    }
                }
```

```
                if (nextLevel == 0) {
                    currDirLen = 0;
                    stack.clear();
                }

                isFile = false;
            }

            return longestLen;
        }
    }
}

// ============================================================
// FILE 372: _389.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 389. Find the Difference
 *
 * Given two strings s and t which consist of only lowercase letters.

 String t is generated by random shuffling string s and then add
 one more letter at a random position.

 Find the letter that was added in t.

 Example:

 Input:
 s = "abcd"
 t = "abcde"

 Output:
 e

 Explanation: 'e' is the letter that was added.*/

public class _389 {
    public static class Solution1 {
        public char findTheDifference(String s, String t) {
            int[] counts = new int[128];
            char[] schars = s.toCharArray();
            char[] tchars = t.toCharArray();
            for (int i = 0; i < schars.length; i++) {
                counts[schars[i]]++;
            }
            for (int i = 0; i < tchars.length; i++) {
                counts[tchars[i]]--;
            }
            char result = 'a';
            for (int i = 0; i < 128; i++) {
```

```
                if (counts[i] != 0) {
                    result = (char) i;
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 373: _390.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 390. Elimination Game
 *
 * There is a list of sorted integers from 1 to n.
 * Starting from left to right,
 * remove the first number and every other number afterward until you reach the end of
the list.
 * Repeat the previous step again, but this time from right to left,
 * remove the right most number and every other number from the remaining numbers.
 * We keep repeating the steps again, alternating left to right and right to left, until
a single number remains.
 * Find the last number that remains starting with a list of length n.

 Example:

 Input:
 n = 9,
 1 2 3 4 5 6 7 8 9
 2 4 6 8
 2 6
 6

 Output:
 6

 */
public class _390 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/55870/share-my-solutions-for-
contest-2 instead of
         * literally removing half of the elements in each scan, this solution is just
moving the
         * pointer to point to next start position So brilliant!
         */
        public int lastRemaining(int n) {
            int remaining = n;
            int start = 1;
```

```
            int step = 2;
            boolean forward = true;
            while (remaining > 1) {
                remaining /= 2;
                if (forward) {
                    start = start + step * remaining - step / 2;
                } else {
                    start = start - step * remaining + step / 2;
                }
                step *= 2;
                forward = !forward;
            }
            return start;
        }
    }
}

// ============================================================
// FILE 374: _391.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 391. Perfect Rectangle
 *
 * Given N axis-aligned rectangles where N > 0, determine if they all together form an
exact cover of a rectangular region.
 Each rectangle is represented as a bottom-left point and a top-right point.
 For example, a unit square is represented as [1,1,2,2]. (coordinate of bottom-left point
is (1, 1) and top-right point is (2, 2)).

 Example 1:

 rectangles = [
 [1,1,3,3],
 [3,1,4,2],
 [3,2,4,4],
 [1,3,2,4],
 [2,3,3,4]
 ]

 Return true. All 5 rectangles together form an exact cover of a rectangular region.

 Example 2:

 rectangles = [
 [1,1,2,3],
 [1,3,2,4],
 [3,1,4,2],
 [3,2,4,4]
```

```
 ]

 Return false. Because there is a gap between the two rectangular regions.

 Example 3:

 rectangles = [
 [1,1,3,3],
 [3,1,4,2],
 [1,3,2,4],
 [3,2,4,4]
 ]

 Return false. Because there is a gap in the top center.

 Example 4:

 rectangles = [
 [1,1,3,3],
 [3,1,4,2],
 [1,3,2,4],
 [2,2,4,4]
 ]

 Return false. Because two of the rectangles overlap with each other.
 */
public class _391 {
    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/56052/really-easy-understanding-
solution-o-n-java */
        public boolean isRectangleCover(int[][] rectangles) {
            if (rectangles.length == 0 || rectangles[0].length == 0) {
                return false;
            }

            int x1 = Integer.MAX_VALUE;
            int x2 = Integer.MIN_VALUE;
            int y1 = Integer.MAX_VALUE;
            int y2 = Integer.MIN_VALUE;

            Set<String> set = new HashSet<>();
            int area = 0;

            for (int[] rect : rectangles) {
                x1 = Math.min(rect[0], x1);
                y1 = Math.min(rect[1], y1);
                x2 = Math.max(rect[2], x2);
                y2 = Math.max(rect[3], y2);

                area += (rect[2] - rect[0]) * (rect[3] - rect[1]);

                String s1 = rect[0] + " " + rect[1];
                String s2 = rect[0] + " " + rect[3];
                String s3 = rect[2] + " " + rect[3];
```

```
                String s4 = rect[2] + " " + rect[1];

                if (!set.add(s1)) {
                    set.remove(s1);
                }
                if (!set.add(s2)) {
                    set.remove(s2);
                }
                if (!set.add(s3)) {
                    set.remove(s3);
                }
                if (!set.add(s4)) {
                    set.remove(s4);
                }
            }

            if (!set.contains(x1 + " " + y1) || !set.contains(x1 + " " + y2) ||
!set.contains(
                x2 + " " + y1) || !set.contains(x2 + " " + y2) || set.size() != 4) {
                return false;
            }

            return area == (x2 - x1) * (y2 - y1);
        }
    }
}

// ============================================================
// FILE 375: _392.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 392. Is Subsequence
 *
 * Given a string s and a string t, check if s is subsequence of t.

 You may assume that there is only lower case English letters in both s and t. t is
potentially a very long (length ~= 500,000) string, and s is a short string (<=100).

 A subsequence of a string is a new string which is formed from the original string by
deleting some (can be none) of the characters without disturbing the relative positions
of the remaining characters. (ie, "ace" is a subsequence of "abcde" while "aec" is not).

 Example 1:
 s = "abc", t = "ahbgdc"

 Return true.

 Example 2:
 s = "axc", t = "ahbgdc"

 Return false.
```

```
 Follow up:
 If there are lots of incoming S, say S1, S2, ... , Sk where k >= 1B, and you want to
check one by one to see if T has its subsequence. In this scenario, how would you change
your code?

 */
public class _392 {

    public static class Solution1 {
        public boolean isSubsequence(String s, String t) {
            int left = 0;
            for (int i = 0; i < s.length(); i++) {
                boolean foundI = false;
                int j = left;
                for (; j < t.length(); j++) {
                    if (s.charAt(i) == t.charAt(j)) {
                        left = j + 1;
                        foundI = true;
                        break;
                    }
                }
                if (j == t.length() && !foundI) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 376: _393.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 393. UTF-8 Validation
 *
 * A character in UTF8 can be from 1 to 4 bytes long, subjected to the following rules:

 For 1-byte character, the first bit is a 0, followed by its unicode code.
 For n-bytes character, the first n-bits are all one's, the n+1 bit is 0, followed by n-1
bytes with most significant 2 bits being 10.
 This is how the UTF-8 encoding would work:

 Char. number range  |        UTF-8 octet sequence
 (hexadecimal)    |              (binary)
 --------------------+---------------------------------------------
 0000 0000-0000 007F | 0xxxxxxx
 0000 0080-0000 07FF | 110xxxxx 10xxxxxx
 0000 0800-0000 FFFF | 1110xxxx 10xxxxxx 10xxxxxx
 0001 0000-0010 FFFF | 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
```

```
 Given an array of integers representing the data, return whether it is a valid utf-8
encoding.

 Note:
 The input is an array of integers. Only the least significant 8 bits of each integer is
used to store the data. This means each integer represents only 1 byte of data.

 Example 1:
 data = [197, 130, 1], which represents the octet sequence: 11000101 10000010 00000001.
 Return true.

 It is a valid utf-8 encoding for a 2-bytes character followed by a 1-byte character.


 Example 2:
 data = [235, 140, 4], which represented the octet sequence: 11101011 10001100 00000100.
 Return false.

 The first 3 bits are all one's and the 4th bit is 0 means it is a 3-bytes character.
 The next byte is a continuation byte which starts with 10 and that's correct.
 But the second continuation byte does not start with 10, so it is invalid.
 */
public class _393 {

    public static class Solution1 {
        /** credit: https://discuss.leetcode.com/topic/58338/bit-manipulation-java-6ms/4
*/
        public boolean validUtf8(int[] data) {
            int count = 0;
            for (int d : data) {
                if (count == 0) {
                    if ((d >> 5) == 0b110) {
                        count = 1;
                    } else if ((d >> 4) == 0b1110) {
                        count = 2;
                    } else if ((d >> 3) == 0b11110) {
                        count = 3;
                    } else if ((d >> 7) == 1) {
                        return false;
                    }
                } else {
                    if ((d >> 6) != 0b10) {
                        return false;
                    } else {
                        count--;
                    }
                }
            }
            return count == 0;
        }
    }

}
```

```java
// ============================================================
// FILE 377: _394.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;

/**394. Decode String
 *
 * Given an encoded string, return it's decoded string.
 * The encoding rule is: k[encoded_string], where the encoded_string inside the square brackets is being repeated exactly k times.
 * Note that k is guaranteed to be a positive integer.
 * You may assume that the input string is always valid;
 * No extra white spaces, square brackets are well-formed, etc.
 * Furthermore, you may assume that the original data does not contain any digits and
 * that digits are only for those repeat numbers, k. For example, there won't be input like 3a or 2[4].
 *
 * Examples:
 * s = "3[a]2[bc]", return "aaabcbc".
 * s = "3[a2[c]]", return "accaccacc".
 * s = "2[abc]3[cd]ef", return "abcabccdcdcdef".
 * */

public class _394 {

    public static class Solution1 {
        public String decodeString(String s) {
            Stack<Integer> count = new Stack<>();
            Stack<String> str = new Stack<>();

            int idx = 0;
            str.push("");

            while (idx < s.length()) {
                if (s.charAt(idx) >= '0' && s.charAt(idx) <= '9') {
                    int start = idx;
                    while (s.charAt(idx + 1) >= '0' && s.charAt(idx + 1) <= '9') {
                        idx++;
                    }

                    count.push(Integer.parseInt(s.substring(start, idx + 1)));
                } else if (s.charAt(idx) == '[') {
                    str.push("");
                } else if (s.charAt(idx) == ']') {
                    String st = str.pop();
                    StringBuilder sb = new StringBuilder();
                    int n = count.pop();

                    for (int j = 0; j < n; j++) {
                        sb.append(st);
                    }
```

```
                    str.push(str.pop() + sb.toString());
                } else {
                    str.push(str.pop() + s.charAt(idx));
                }

                idx++;
            }

            return str.pop();
        }
    }
}

// ============================================================
// FILE 378: _395.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 395. Longest Substring with At Least K Repeating Characters
 *
 *  Find the length of the longest substring T of a given string
 *  (consists of lowercase letters only)
 *  such that every character in T appears no less than k times.

 Example 1:
 Input:
 s = "aaabb", k = 3

 Output:
 3

 The longest substring is "aaa", as 'a' is repeated 3 times.


 Example 2:
 Input:
 s = "ababbc", k = 2

 Output:
 5

 The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3
times.
 */

public class _395 {
    public static class Solution1 {
        /**Reference: https://discuss.leetcode.com/topic/57372/java-divide-and-conquer-
recursion-solution*/
        public int longestSubstring(String s, int k) {
            return findLongestSubstring(s.toCharArray(), 0, s.length(), k);
```

```
        }

        int findLongestSubstring(char[] chars, int start, int end, int k) {
            /**Base case 1 of 2*/
            if (end - start < k) {
                return 0;
            }
            int[] count = new int[26];
            for (int i = start; i < end; i++) {
                int index = chars[i] - 'a';
                count[index]++;
            }

            /**For every character in the above frequency table*/
            for (int i = 0; i < 26; i++) {
                if (count[i] < k && count[i] > 0) {
                    for (int j = start; j < end; j++) {
                        if (chars[j] == i + 'a') {
                            int left = findLongestSubstring(chars, start, j, k);
                            int right = findLongestSubstring(chars, j + 1, end, k);
                            return Math.max(left, right);
                        }
                    }
                }
            }
            /**Base case 2 of 2:
             * when any characters in this substring has repeated at least k times, then
this entire substring is a valid answer*/
            return end - start;
        }
    }
}

// ============================================================
// FILE 379: _396.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 396. Rotate Function
 *
 * Given an array of integers A and let n to be its length.

 Assume Bk to be an array obtained by rotating the array A k positions clock-wise, we
define a "rotation function" F on A as follow:

 F(k) = 0 * Bk[0] + 1 * Bk[1] + ... + (n-1) * Bk[n-1].

 Calculate the maximum value of F(0), F(1), ..., F(n-1).

 Note:
 n is guaranteed to be less than 105.
```

```
 Example:

 A = [4, 3, 2, 6]

 F(0) = (0 * 4) + (1 * 3) + (2 * 2) + (3 * 6) = 0 + 3 + 4 + 18 = 25
 F(1) = (0 * 6) + (1 * 4) + (2 * 3) + (3 * 2) = 0 + 4 + 6 + 6 = 16
 F(2) = (0 * 2) + (1 * 6) + (2 * 4) + (3 * 3) = 0 + 6 + 8 + 9 = 23
 F(3) = (0 * 3) + (1 * 2) + (2 * 6) + (3 * 4) = 0 + 2 + 12 + 12 = 26

 So the maximum value of F(0), F(1), F(2), F(3) is F(3) = 26.*/

public class _396 {
    public static class Solution1 {
        /**
         * F(k) = 0 * Bk[0] + 1 * Bk[1] + ... + (n-1) * Bk[n-1]
         */
        public int maxRotateFunction(int[] A) {
            if (A == null || A.length == 0) {
                return 0;
            }
            int[] F = new int[A.length];
            int[] B = A;
            int max = Integer.MIN_VALUE;
            for (int i = 0; i < A.length; i++) {
                F[i] = compute(B);
                max = Math.max(max, F[i]);
                B = rotate(B);
            }
            return max;
        }

        private int compute(int[] b) {
            int sum = 0;
            for (int i = 0; i < b.length; i++) {
                sum += i * b[i];
            }
            return sum;
        }

        private int[] rotate(int[] a) {
            int first = a[0];
            for (int i = 1; i < a.length; i++) {
                a[i - 1] = a[i];
            }
            a[a.length - 1] = first;
            return a;
        }
    }

    public static class Solution2 {
        /**
         * Reference: https://discuss.leetcode.com/topic/58459/java-o-n-solution-with-
explanation
         */
```

```
        public int maxRotateFunction(int[] A) {
            int allSum = 0;
            int len = A.length;
            int F = 0;
            for (int i = 0; i < len; i++) {
                F += i * A[i];
                allSum += A[i];
            }
            int max = F;
            for (int i = len - 1; i >= 1; i--) {
                F = F + allSum - len * A[i];
                max = Math.max(F, max);
            }
            return max;
        }
    }
}

// ============================================================
// FILE 380: _397.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 397. Integer Replacement
 *
 * Given a positive integer n and you can do operations as follow:

 If n is even, replace n with n/2.
 If n is odd, you can replace n with either n + 1 or n - 1.
 What is the minimum number of replacements needed for n to become 1?

 Example 1:

 Input:
 8

 Output:
 3

 Explanation:
 8 -> 4 -> 2 -> 1
 Example 2:

 Input:
 7
```

```
 Output:
 4

 Explanation:
 7 -> 8 -> 4 -> 2 -> 1
 or
 7 -> 6 -> 3 -> 2 -> 1*/
public class _397 {

    public static class Solution1 {
        public int integerReplacement(int n) {
            long min = Long.MAX_VALUE;
            Set<long[]> set = new HashSet();
            Queue<long[]> q = new LinkedList();
            long[] pair = new long[] {n, 0};
            q.offer(pair);
            while (!q.isEmpty()) {
                int size = q.size();
                for (int i = 0; i < size; i++) {
                    long[] curr = q.poll();
                    if (curr[0] == 1) {
                        set.add(curr);
                    } else {

                        if (curr[0] % 2 == 0) {
                            curr[0] /= 2;
                            curr[1]++;
                            q.offer(curr);
                        } else {
                            long[] minus = new long[2];
                            minus[0] = curr[0] - 1;
                            minus[1] = curr[1] + 1;
                            q.offer(minus);

                            long[] plus = new long[2];
                            plus[0] = curr[0] + 1;
                            plus[1] = curr[1] + 1;
                            q.offer(plus);
                        }
                    }
                }
            }

            Iterator<long[]> it = set.iterator();
            while (it.hasNext()) {
                min = Math.min(min, it.next()[1]);
            }
            return (int) min;
        }
    }

}

// ============================================================
```

```
// FILE 381: _398.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 398. Random Pick Index
 *
 * Given an array of integers with possible duplicates,
 * randomly output the index of a given target number. You can assume that the given
target number must exist in the array.

 Note:
 The array size can be very large. Solution that uses too much extra space will not pass
the judge.

 Example:

 int[] nums = new int[] {1,2,3,3,3};
 Solution solution = new Solution(nums);

 // pick(3) should return either index 2, 3, or 4 randomly. Each index should have equal
probability of returning.
 solution.pick(3);

 // pick(1) should return 0. Since in the array only nums[0] is equal to 1.
 solution.pick(1);*/
public class _398 {

    //TODO: use reservoir sampling to solve it again

    public static class Solution {
        //brute force
        int[] input;
        java.util.Random rand = new java.util.Random();

        public Solution(int[] nums) {
            input = nums;
        }

        public int pick(int target) {
            List<Integer> list = new ArrayList();
            for (int i = 0; i < input.length; i++) {
                if (input[i] == target) {
                    list.add(i);
                }
            }
            if (list.size() == 1) {
                return list.get(0);
            }
            int randomIndex = rand.nextInt(list.size());
```

```
            return list.get(randomIndex);
        }
    }
}

// ============================================================
// FILE 382: _399.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;

/**
 * 399. Evaluate Division
 *
 * Equations are given in the format A / B = k, where A and B are variables represented
as strings, and k is a real number (floating point number).
 * Given some queries, return the answers. If the answer does not exist, return -1.0.

 Example:
 Given a / b = 2.0, b / c = 3.0.
 queries are: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ? .
 return [6.0, 0.5, -1.0, 1.0, -1.0 ].

 The input is: vector<pair<string, string>> equations, vector<double>& values,
vector<pair<string, string>> queries ,
 where equations.size() == values.size(), and the values are positive. This represents
the equations. Return vector<double>.

 According to the example above:

 equations = [ ["a", "b"], ["b", "c"] ],
 values = [2.0, 3.0],
 queries = [ ["a", "c"], ["b", "a"], ["a", "e"], ["a", "a"], ["x", "x"] ].

 The input is always valid. You may assume that evaluating the queries will result in no
division by zero and there is no contradiction.
 */
public class _399 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/59146/java-ac-solution-using-graph
         *
         * Image a/b = k as a link between node a and b, the weight from a to b is k, the
reverse link
         * is 1/k. Query is to find a path between two nodes.
         */
        public double[] calcEquation(String[][] equations, double[] values, String[][]
```

```
queries) {
            Map<String, List<String>> pairs = new HashMap<>();
            Map<String, List<Double>> valuePairs = new HashMap<>();
            for (int i = 0; i < equations.length; i++) {
                String[] equation = equations[i];
                if (!pairs.containsKey(equation[0])) {
                    pairs.put(equation[0], new ArrayList<>());
                    valuePairs.put(equation[0], new ArrayList<>());
                }
                if (!pairs.containsKey(equation[1])) {
                    pairs.put(equation[1], new ArrayList<>());
                    valuePairs.put(equation[1], new ArrayList<>());
                }
                pairs.get(equation[0]).add(equation[1]);
                pairs.get(equation[1]).add(equation[0]);
                valuePairs.get(equation[0]).add(values[i]);
                valuePairs.get(equation[1]).add(1 / values[i]);
            }

            double[] result = new double[queries.length];
            for (int i = 0; i < queries.length; i++) {
                String[] query = queries[i];
                result[i] = dfs(query[0], query[1], pairs, valuePairs, new HashSet<>(),
1.0);
                if (result[i] == 0.0) {
                    result[i] = -1.0;
                }
            }
            return result;
        }

        private double dfs(String start, String end, Map<String, List<String>> pairs,
            Map<String, List<Double>> valuePairs, HashSet<String> set, double value) {
            if (set.contains(start)) {
                return 0.0;
            }
            if (!pairs.containsKey(start)) {
                return 0.0;
            }
            if (start.equals(end)) {
                return value;
            }
            set.add(start);

            List<String> stringList = pairs.get(start);
            List<Double> valueList = valuePairs.get(start);
            double tmp = 0.0;
            for (int i = 0; i < stringList.size(); i++) {
                tmp = dfs(stringList.get(i), end, pairs, valuePairs, set, value *
valueList.get(i));
                if (tmp != 0.0) {
                    break;
                }
            }
```

```
            set.remove(start);
            return tmp;
        }
    }
}

// ============================================================
// FILE 383: _400.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 400. Nth Digit
 *
 * Find the nth digit of the infinite integer sequence 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
...

 Note:
 n is positive and will fit within the range of a 32-bit signed integer (n < 231).

 Example 1:
 Input:
 3
 Output:
 3

 Example 2:
 Input:
 11
 Output:
 0

 Explanation:
 The 11th digit of the sequence 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ... is a 0, which is
part of the number 10.
 */
public class _400 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/59314/java-solution:
         *
         * 1. find the length of the number where the nth digit is from 2. find the
actual number where
         * the nth digit is from 3. find the nth digit and return
         */
        public int findNthDigit(int n) {
            int len = 1;
            long count = 9;
            int start = 1;

            while (n > len * count) {
                n -= len * count;
```

```
                len += 1;
                count *= 10;
                start *= 10;
            }

            start += (n - 1) / len;
            String s = Integer.toString(start);
            return Character.getNumericValue(s.charAt((n - 1) % len));
        }
    }
}

// ============================================================
// FILE 384: _401.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 401. Binary Watch
 *
 * A binary watch has 4 LEDs on the top which represent the hours (0-11), and the 6 LEDs
on the bottom represent the minutes (0-59).
 * Each LED represents a zero or one, with the least significant bit on the right.

 For example, the above binary watch reads "3:25".

 Given a non-negative integer n which represents the number of LEDs that are currently
on, return all possible times the watch could represent.

 Example:

 Input: n = 1
 Return: ["1:00", "2:00", "4:00", "8:00", "0:01", "0:02", "0:04", "0:08", "0:16", "0:32"]
 Note:
 The order of output does not matter.
 The hour must not contain a leading zero, for example "01:00" is not valid, it should be
"1:00".
 The minute must be consist of two digits and may contain a leading zero, for example
"10:2" is not valid, it should be "10:02".
 */
public class _401 {

    public static class Solution1 {
        public List<String> readBinaryWatch(int num) {
            List<String> times = new ArrayList<>();
            for (int h = 0; h < 12; h++) {
                for (int m = 0; m < 60; m++) {
                    if (Integer.bitCount(h * 64 + m) == num) {
                        times.add(String.format("%d:%02d", h,
                            m));//%02 means to pad this two-digit decimal number on the
```

```
left with zeroes
                    }
                }
            }
            return times;
        }
    }
}

// ============================================================
// FILE 385: _402.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 402. Remove K Digits
 *
 * Given a non-negative integer num represented as a string, remove k digits from the
number so that the new number is the smallest possible.

 Note:
 The length of num is less than 10002 and will be ≥ k.
 The given num does not contain any leading zero.
 Example 1:

 Input: num = "1432219", k = 3
 Output: "1219"
 Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is
the smallest.
 Example 2:

 Input: num = "10200", k = 1
 Output: "200"
 Explanation: Remove the leading 1 and the number is 200. Note that the output must not
contain leading zeroes.
 Example 3:

 Input: num = "10", k = 2
 Output: "0"
 Explanation: Remove all the digits from the number and it is left with nothing which is
0.
 */
public class _402 {
    public static class Solution1 {

        /** credit: https://discuss.leetcode.com/topic/59412/a-greedy-method-using-stack-
o-n-time-and-o-n-space */
        public String removeKdigits(String num, int k) {
            int digits = num.length() - k;
            char[] stack = new char[num.length()];
            int top = 0;

            for (int i = 0; i < num.length(); i++) {
```

```
                char c = num.charAt(i);
                while (top > 0 && stack[top - 1] > c && k > 0) {
                    top--;
                    k--;
                }
                stack[top++] = c;
            }

            int index = 0;
            while (index < digits && stack[index] == '0') {
                index++;
            }
            return index == digits ? "0" : new String(stack, index, digits - index);
        }
    }
}

// ============================================================
// FILE 386: _403.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 403. Frog Jump
 *
 * A frog is crossing a river.
 * The river is divided into x units and at each unit there may or may not exist a stone.
 * The frog can jump on a stone, but it must not jump into the water.

 Given a list of stones' positions (in units) in sorted ascending order,
 determine if the frog is able to cross the river by landing on the last stone.
 Initially, the frog is on the first stone and assume the first jump must be 1 unit.

 If the frog's last jump was k units, then its next jump must be either k - 1, k, or k +
1 units.
 Note that the frog can only jump in the forward direction.

 Note:

 The number of stones is ≥ 2 and is < 1,100.
 Each stone's position will be a non-negative integer < 231.
 The first stone's position is always 0.

 Example 1:

 [0,1,3,5,6,8,12,17]

 There are a total of 8 stones.
```

```
 The first stone at the 0th unit, second stone at the 1st unit,
 third stone at the 3rd unit, and so on...
 The last stone at the 17th unit.

 Return true. The frog can jump to the last stone by jumping
 1 unit to the 2nd stone, then 2 units to the 3rd stone, then
 2 units to the 4th stone, then 3 units to the 6th stone,
 4 units to the 7th stone, and 5 units to the 8th stone.


 Example 2:

 [0,1,2,3,4,8,9,11]

 Return false. There is no way to jump to the last stone as
 the gap between the 5th and 6th stone is too large.

 */
public class _403 {

    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/59903/very-easy-to-understand-
java-solution-with-explanations/2
         * and https://leetcode.com/articles/frog-jump/#approach-5-using-dynamic-
programmingaccepted
         */
        public boolean canCross(int[] stones) {
            if (stones.length == 0) {
                return true;
            }
            Map<Integer, Set<Integer>> map = new HashMap<>(stones.length);
            map.put(0, new HashSet<>());
            map.get(0).add(1);
            for (int i = 1; i < stones.length; i++) {
                map.put(stones[i], new HashSet<>());
            }

            for (int i = 0; i < stones.length; i++) {
                int stone = stones[i];
                for (int step : map.get(stone)) {
                    int reach = step + stone;
                    if (reach == stones[stones.length - 1]) {
                        return true;
                    }
                    Set<Integer> set = map.get(reach);
                    if (set != null) {
                        set.add(step);
                        if (step - 1 > 0) {
                            set.add(step - 1);
                        }
                        set.add(step + 1);
                    }
                }
```

```
            }
            return false;
        }
    }
}

// ============================================================
// FILE 387: _404.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
/**
 * 404. Sum of Left Leaves
 *
 * Find the sum of all left leaves in a given binary tree.

 Example:

   3
  / \
 9  20
   /  \
  15   7

 There are two left leaves in the binary tree, with values 9 and 15 respectively. Return
24.*/
public class _404 {
    public static class Solution1 {
        public int sumOfLeftLeaves(TreeNode root) {
            int result = 0;
            if (root == null) {
                return result;
            }
            return dfs(root, result, false);
        }

        private int dfs(TreeNode root, int result, boolean left) {
            if (root.left == null && root.right == null && left) {
                result += root.val;
                return result;
            }
            int leftResult = 0;
            if (root.left != null) {
                left = true;
                leftResult = dfs(root.left, result, left);
            }
            int rightResult = 0;
            if (root.right != null) {
                left = false;
                rightResult = dfs(root.right, result, left);
            }
            return leftResult + rightResult;
```

```
        }
    }

    public static class Solution2 {

        public int sumOfLeftLeaves(TreeNode root) {
            int sum = 0;
            if (root == null) {
                return sum;
            }
            if (root.left != null) {
                if (root.left.left == null && root.left.right == null) {
                    sum += root.left.val;
                } else {
                    sum += sumOfLeftLeaves(root.left);
                }
            }
            if (root.right != null) {
                sum += sumOfLeftLeaves(root.right);
            }
            return sum;
        }
    }
}

// ============================================================
// FILE 388: _405.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 405. Convert a Number to Hexadecimal

 Given an integer, write an algorithm to convert it to hexadecimal. For negative integer,
two’s complement method is used.

 Note:

 All letters in hexadecimal (a-f) must be in lowercase.
 The hexadecimal string must not contain extra leading 0s.
 If the number is zero, it is represented by a single zero character '0';
 otherwise, the first character in the hexadecimal string will not be the zero character.
 The given number is guaranteed to fit within the range of a 32-bit signed integer.
 You must not use any method provided by the library which converts/formats the number to
hex directly.

 Example 1:
 Input:
 26

 Output:
 "1a"
```

```
 Example 2:
 Input:
 -1

 Output:
 "ffffffff"

 */
public class _405 {

    public static class Solution1 {
        public String toHex(int num) {
            char[] hexChars =
                new char[] {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b',
'c', 'd',
                        'e', 'f'};
            String result = "";
            while (num != 0) {
                result = hexChars[(num & 15)] + result;
                num >>>= 4;
            }
            return result.isEmpty() ? "0" : result;
        }
    }
}

// ============================================================
// FILE 389: _406.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Comparator;
import java.util.LinkedList;
import java.util.List;

/**
 * 406. Queue Reconstruction by Height
 *
 * Suppose you have a random list of people standing in a queue.
 * Each person is described by a pair of integers (h, k),
 * where h is the height of the person and k is the number of people in
 * front of this person who have a height greater than or equal to h. Write an algorithm
to reconstruct the queue.

 Note:
 The number of people is less than 1,100.

 Example

 Input:
 [[7,0], [4,4], [7,1], [5,0], [6,1], [5,2]]
```

```
 Output:
 [[5,0], [7,0], [5,2], [6,1], [4,4], [7,1]]

 */
public class _406 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/60437/java-solution-using-
priorityqueue-and-linkedlist
         */
        public int[][] reconstructQueue(int[][] people) {
            Arrays.sort(people, new Comparator<int[]>() {
                public int compare(int[] p1, int[] p2) {
                    return p1[0] != p2[0] ? Integer.compare(p2[0], p1[0])
                        : Integer.compare(p1[1], p2[1]);
                }
            });
            List<int[]> list = new LinkedList();
            for (int[] ppl : people) {
                list.add(ppl[1], ppl);
            }
            return list.toArray(new int[people.length][]);
        }
    }
}

// ============================================================
// FILE 390: _407.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 407. Trapping Rain Water II
 *
 * Given an m x n matrix of positive integers representing the height of each unit cell
in a 2D elevation map, compute the volume of water it is able to trap after raining.

 Note:
 Both m and n are less than 110. The height of each unit cell is greater than 0 and is
less than 20,000.

 Example:

 Given the following 3x6 height map:
 [
 [1,4,3,1,3,2],
 [3,2,1,3,2,4],
 [2,3,3,2,3,1]
 ]
```

```
 Return 4.

 The above image represents the elevation map [[1,4,3,1,3,2],[3,2,1,3,2,4],[2,3,3,2,3,1]]
before the rain.

 After the rain, water are trapped between the blocks. The total volume of water trapped
is 4.

 */
public class _407 {
    public static class Solution1 {
        /** Reference: https://discuss.leetcode.com/topic/60418/java-solution-using-
priorityqueue */
        public class Cell {
            int row;
            int col;
            int height;

            public Cell(int row, int col, int height) {
                this.row = row;
                this.col = col;
                this.height = height;
            }
        }

        public int trapRainWater(int[][] heights) {
            if (heights == null || heights.length == 0 || heights[0].length == 0) {
                return 0;
            }

            PriorityQueue<Cell> queue = new PriorityQueue<>(1, (a, b) -> a.height -
b.height);

            int m = heights.length;
            int n = heights[0].length;
            boolean[][] visited = new boolean[m][n];

            // Initially, add all the Cells which are on borders to the queue.
            for (int i = 0; i < m; i++) {
                visited[i][0] = true;
                visited[i][n - 1] = true;
                queue.offer(new Cell(i, 0, heights[i][0]));
                queue.offer(new Cell(i, n - 1, heights[i][n - 1]));
            }

            for (int i = 0; i < n; i++) {
                visited[0][i] = true;
                visited[m - 1][i] = true;
                queue.offer(new Cell(0, i, heights[0][i]));
                queue.offer(new Cell(m - 1, i, heights[m - 1][i]));
            }

            // from the borders, pick the shortest cell visited and check its neighbors:
            // if the neighbor is shorter, collect the water it can trap and update its
```

```
height as its height plus the water trapped
            // add all its neighbors to the queue.
            int[][] dirs = new int[][] {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
            int res = 0;
            while (!queue.isEmpty()) {
                Cell cell = queue.poll();
                for (int[] dir : dirs) {
                    int row = cell.row + dir[0];
                    int col = cell.col + dir[1];
                    if (row >= 0 && row < m && col >= 0 && col < n && !visited[row][col])
{
                        visited[row][col] = true;
                        res += Math.max(0, cell.height - heights[row][col]);
                        queue.offer(new Cell(row, col, Math.max(heights[row][col],
cell.height)));
                    }
                }
            }

            return res;
        }
    }
}

// ============================================================
// FILE 391: _408.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 408. Valid Word Abbreviation
 *
 * Given a non-empty string s and an abbreviation abbr,
 * return whether the string matches with the given abbreviation.
 * A string such as "word" contains only the following valid abbreviations:

 ["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1",
"1o2", "2r1", "3d", "w3", "4"]
 Notice that only the above abbreviations are valid abbreviations of the string "word".
 Any other string is not a valid abbreviation of "word".

 Note:
 Assume s contains only lowercase letters and abbr contains only lowercase letters and
digits.

 Example 1:
 Given s = "internationalization", abbr = "i12iz4n":

 Return true.
 Example 2:
 Given s = "apple", abbr = "a2e":

 Return false.
```

```
 */
public class _408 {

    public static class Solution1 {
        public boolean validWordAbbreviation(String word, String abbr) {
            if (abbr.length() > word.length()) {
                return false;
            } else {
                char[] abbrChars = abbr.toCharArray();
                char[] wordChars = word.toCharArray();
                if (abbr.length() == word.length()) {
                    boolean prevDigit = false;
                    for (int i = 0, j = 0; i < abbrChars.length && j < wordChars.length;
i++, j++) {
                        if (Character.isDigit(abbrChars[i]) && !prevDigit) {
                            prevDigit = true;
                            if (Character.getNumericValue(abbrChars[i]) != 1) {
                                return false;
                            }
                        } else if (Character.isDigit(abbrChars[i]) && prevDigit) {
                            return false;
                        } else if (abbrChars[i] != wordChars[j]) {
                            return false;
                        } else if (prevDigit) {
                            prevDigit = false;
                        }
                    }
                    return true;
                } else {
                    StringBuilder stringBuilder = new StringBuilder();
                    boolean firstDigit = true;
                    for (int i = 0, j = 0; i < abbrChars.length && j < wordChars.length;
i++) {
                        while (i < abbrChars.length && Character.isDigit(abbrChars[i])) {
                            if (firstDigit && Character.getNumericValue(abbrChars[i]) ==
0) {
                                return false;
                            }
                            stringBuilder.append(abbrChars[i]);
                            i++;
                            firstDigit = false;
                        }
                        firstDigit = true;
                        if (!stringBuilder.toString().isEmpty()) {
                            int number = Integer.valueOf(stringBuilder.toString());
                            j += number;
                            stringBuilder.setLength(0);
                        }
                        if ((i >= abbrChars.length && j < wordChars.length) || (i <
abbrChars.length
                            && j >= wordChars.length)) {
                            return false;
                        }
                        if (i < abbrChars.length
```

```
                            && j < wordChars.length
                            && abbrChars[i] != wordChars[j]) {
                            return false;
                        }
                        if (j > wordChars.length && i <= abbrChars.length) {
                            return false;
                        }
                        j++;
                    }
                    return true;
                }
            }
        }
    }
}

// ============================================================
// FILE 392: _409.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 409. Longest Palindrome
 *
 * Given a string which consists of lowercase or uppercase letters, find the length of
the longest palindromes that can be built with those letters.
 * This is case sensitive, for example "Aa" is not considered a palindrome here.
 *
 * Note:
 * Assume the length of given string will not exceed 1,010.
 *
 * Example:
 * Input:
 * "abccccdd"
 *
 * Output:
 * 7
 *
 * Explanation:
 * One longest palindrome that can be built is "dccaccd", whose length is 7.
 */
public class _409 {
  public static class Solution1 {
    public int longestPalindrome(String s) {
      int[] counts = new int[56];
      for (char c : s.toCharArray()) {
        if (Character.isUpperCase(c)) {
          counts[c - 'A' + 33]++;
        } else {
          counts[c - 'a']++;
        }
      }
      boolean hasOdd = false;
```

```
      int len = 0;
      for (int i = 0; i < 56; i++) {
        if (counts[i] % 2 != 0) {
          hasOdd = true;
          if (counts[i] > 1) {
            len += counts[i] - 1;
          }
        } else {
          len += counts[i];
        }
      }
      return hasOdd ? len + 1 : len;
    }
  }
}

// ============================================================
// FILE 393: _410.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 410. Split Array Largest Sum
 *
 * Given an array which consists of non-negative integers and an integer m,
 * you can split the array into m non-empty continuous subarrays.
 * Write an algorithm to minimize the largest sum among these m subarrays.

 Note:
 If n is the length of array, assume the following constraints are satisfied:

 1 ≤ n ≤ 1000
 1 ≤ m ≤ min(50, n)

 Examples:

 Input:
 nums = [7,2,5,10,8]
 m = 2

 Output:
 18

 Explanation:
 There are four ways to split nums into two subarrays.
 The best way is to split it into [7,2,5] and [10,8],
 where the largest sum among the two subarrays is only 18.
 */
public class _410 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/61324/clear-explanation-8ms-binary-
```

```
search-java
         *
         * The answer is between maximum value of input array numbers and sum of those
numbers. Use
         * binary search to approach the correct answer. We have l = max number of array;
r = sum of all
         * numbers in the array; Every time we do mid = (l + r) / 2;
         *
         * Use greedy to narrow down left and right boundaries in binary search. 3.1 Cut
the array from
         * left. 3.2 Try our best to make sure that the sum of numbers between each two
cuts (inclusive)
         * is large enough but still less than mid. 3.3 We'll end up with two results:
either we can
         * divide the array into more than m subarrays or we cannot. If we can, it means
that the mid
         * value we pick is too small because we've already tried our best to make sure
each part holds
         * as many non-negative numbers as we can but we still have numbers left. So, it
is impossible
         * to cut the array into m parts and make sure each parts is no larger than mid.
We should
         * increase m. This leads to l = mid + 1; If we can't, it is either we
successfully divide the
         * array into m parts and the sum of each part is less than mid, or we used up
all numbers
         * before we reach m. Both of them mean that we should lower mid because we need
to find the
         * minimum one. This leads to r = mid - 1;
         */

        public int splitArray(int[] nums, int m) {
            int max = 0;
            long sum = 0;
            for (int num : nums) {
                max = Math.max(num, max);
                sum += num;
            }
            if (m == 1) {
                return (int) sum;
            }
            //binary search
            long l = max;
            long r = sum;
            while (l <= r) {
                long mid = (l + r) / 2;
                if (valid(mid, nums, m)) {
                    r = mid - 1;
                } else {
                    l = mid + 1;
                }
            }
            return (int) l;
        }
```

```
        public boolean valid(long target, int[] nums, int m) {
            int count = 1;
            long total = 0;
            for (int num : nums) {
                total += num;
                if (total > target) {
                    total = num;
                    count++;
                    if (count > m) {
                        return false;
                    }
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 394: _411.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 411. Minimum Unique Word Abbreviation
 *
 * A string such as "word" contains the following abbreviations:

 ["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1",
"1o2", "2r1", "3d", "w3", "4"]

 Given a target string and a set of strings in a dictionary,
 find an abbreviation of this target string with the smallest possible length such
 that it does not conflict with abbreviations of the strings in the dictionary.

 Each number or letter in the abbreviation is considered length = 1. For example, the
abbreviation "a32bc" has length = 4.

 Note:

 In the case of multiple answers as shown in the second example below, you may return any
one of them.
 Assume length of target string = m, and dictionary size = n. You may assume that m ≤ 21,
n ≤ 1000, and log2(n) + m ≤ 20.

 Examples:

 "apple", ["blade"] -> "a4" (because "5" or "4e" conflicts with "blade")
```

```
 "apple", ["plain", "amber", "blade"] -> "1p3" (other valid answers include "ap3", "a3e",
"2p2", "3le", "3l1").
 *
 */
public class _411 {
    public static class Solution1 {
        /** Credit: https://discuss.leetcode.com/topic/61346/trie-bruteforce */
        class Trie {
            Trie[] children = new Trie[26];
            boolean isWord = false;
        }

        Trie root = new Trie();
        List<String> abbrs;

        public String minAbbreviation(String target, String[] dictionary) {
            for (String s : dictionary) {
                addTrie(s);
            }

            for (int i = 0; i < target.length(); i++) {
                abbrs = new ArrayList<>();
                abbrGenerator(target, 0, "", 0, i + 1);
                for (String s : abbrs) {
                    if (search(s, root, 0, 0) == false) {
                        return s;
                    }
                }
            }
            return "";
        }

        public void addTrie(String s) {
            Trie cur = root;
            for (int i = 0; i < s.length(); i++) {
                char c = s.charAt(i);
                if (cur.children[c - 'a'] == null) {
                    cur.children[c - 'a'] = new Trie();
                }
                cur = cur.children[c - 'a'];
            }
            cur.isWord = true;
        }

        public boolean search(String target, Trie root, int i, int loop) {
            if (root == null) {
                return false;
            }

            if (loop != 0) {
                for (int a = 0; a < 26; a++) {
                    if (search(target, root.children[a], i, loop - 1)) {
                        return true;
                    }
```

```
                }
                return false;
            }
            if (i == target.length()) {
                if (root.isWord) {
                    return true;
                }
                return false;
            }
            if (Character.isDigit(target.charAt(i))) {
                int tmp = 0;
                while (i < target.length() && Character.isDigit(target.charAt(i))) {
                    tmp = tmp * 10 + target.charAt(i) - '0';
                    i++;
                }
                return search(target, root, i, tmp);
            } else {
                return search(target, root.children[target.charAt(i) - 'a'], i + 1, 0);
            }
        }

        public void abbrGenerator(String target, int i, String tmp, int abbr, int num) {
            if (i == target.length()) {
                if (num == 0 && abbr == 0) {
                    abbrs.add(tmp);
                }
                if (num == 1 && abbr != 0) {
                    abbrs.add(tmp + abbr);
                }
                return;
            }
            if (num <= 0) {
                return;
            }
            char cur = target.charAt(i);
            abbrGenerator(target, i + 1, abbr == 0 ? tmp + cur : tmp + abbr + cur, 0,
                abbr == 0 ? num - 1 : num - 2);
            abbrGenerator(target, i + 1, tmp, abbr + 1, num);
        }
    }

}

// ============================================================
// FILE 395: _412.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 412. Fizz Buzz
```

```
 *
 * Write a program that outputs the string representation of numbers from 1 to n.

 But for multiples of three it should output “Fizz” instead of the number and for
 the multiples of five output “Buzz”. For numbers which are multiples of both three and
five output “_412”.

 Example:

 n = 15,

 Return:
 [
 "1",
 "2",
 "Fizz",
 "4",
 "Buzz",
 "Fizz",
 "7",
 "8",
 "Fizz",
 "Buzz",
 "11",
 "Fizz",
 "13",
 "14",
 "_412"
 ]*/
public class _412 {

    public static class Solution1 {
        public List<String> fizzBuzz(int n) {
            List<String> result = new ArrayList();
            for (int i = 1; i <= n; i++) {
                if (i % 3 == 0 && i % 5 == 0) {
                    result.add("_412");
                } else if (i % 3 == 0) {
                    result.add("Fizz");
                } else if (i % 5 == 0) {
                    result.add("Buzz");
                } else {
                    result.add(Integer.toString(i));
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 396: _413.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 413. Arithmetic Slices
 *
 * A sequence of number is called arithmetic if it consists of at least three elements and if the difference between any two consecutive elements is the same.

 For example, these are arithmetic sequence:

 1, 3, 5, 7, 9
 7, 7, 7, 7
 3, -1, -5, -9

 The following sequence is not arithmetic.

 1, 1, 2, 5, 7

 A zero-indexed array A consisting of N numbers is given. A slice of that array is any pair of integers (P, Q) such that 0 <= P < Q < N.

 A slice (P, Q) of array A is called arithmetic if the sequence:
 A[P], A[p + 1], ..., A[Q - 1], A[Q] is arithmetic. In particular, this means that P + 1 < Q.

 The function should return the number of arithmetic slices in the array A.

 Example:

 A = [1, 2, 3, 4]

 return: 3, for 3 arithmetic slices in A: [1, 2, 3], [2, 3, 4] and [1, 2, 3, 4] itself.
 */
public class _413 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/62884/2ms-java-o-n-time-o-1-space-solution
        public int numberOfArithmeticSlices(int[] A) {
            int sum = 0;
            int len = 2;
            for (int i = 2; i < A.length; i++) {
                if (A[i] - A[i - 1] == A[i - 1] - A[i - 2]) {
                    len++;
                } else {
                    if (len > 2) {
                        sum += calculateSlices(len);
                    }
                    len = 2;//reset it to 2
                }
            }
            if (len > 2) {
                sum += calculateSlices(len);
            }
```

```
            return sum;
        }

        int calculateSlices(int len) {
            return (len - 1) * (len - 2) / 2;
        }
    }

    class Solution2 {
        //credit: https://discuss.leetcode.com/topic/63302/simple-java-solution-9-lines-
2ms
        public int numberOfArithmeticSlices(int[] A) {
            int sum = 0;
            int curr = 0;
            for (int i = 2; i < A.length; i++) {
                if (A[i] - A[i - 1] == A[i - 1] - A[i - 2]) {
                    curr++;
                    sum += curr;
                } else {
                    curr = 0;
                }
            }
            return sum;
        }
    }

}

// ============================================================
// FILE 397: _414.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 414. Third Maximum Number
 *
 * Given a non-empty array of integers, return the third maximum number in this array.
 * If it does not exist, return the maximum number. The time complexity must be in O(n).

 Example 1:
 Input: [3, 2, 1]

 Output: 1

 Explanation: The third maximum is 1.
 Example 2:
 Input: [1, 2]

 Output: 2

 Explanation: The third maximum does not exist, so the maximum (2) is returned instead.
 Example 3:
 Input: [2, 2, 3, 1]
```

```
 Output: 1

 Explanation: Note that the third maximum here means the third maximum distinct number.
 Both numbers with value 2 are both considered as second maximum.
 */
public class _414 {

    public static class Solution1 {
        public int thirdMax(int[] nums) {
            long max1 = Long.MIN_VALUE;
            long max2 = Long.MIN_VALUE;
            long max3 = Long.MIN_VALUE;
            for (int i : nums) {
                max1 = Math.max(max1, i);
            }
            for (int i : nums) {
                if (i == max1) {
                    continue;
                }
                max2 = Math.max(max2, i);
            }
            for (int i : nums) {
                if (i == max1 || i == max2) {
                    continue;
                }
                max3 = Math.max(max3, i);
            }
            return (int) (max3 == Long.MIN_VALUE ? max1 : max3);
        }
    }
}

// ============================================================
// FILE 398: _415.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 415. Add Strings
 *
 * Given two non-negative numbers num1 and num2 represented as string, return the sum of
num1 and num2.

 Note:

 1. The length of both num1 and num2 is < 5100.
 2. Both num1 and num2 contains only digits 0-9.
 3. Both num1 and num2 does not contain any leading zero.
 4. You must not use any built-in BigInteger library or convert the inputs to integer
directly.
 */
public class _415 {
```

```
    public static class Solution1 {
        public String addStrings(String num1, String num2) {
            if (num1 == null || num1.length() == 0) {
                return num2;
            } else if (num2 == null || num2.length() == 0) {
                return num1;
            }

            int i = num1.length() - 1;
            int j = num2.length() - 1;
            long carry = 0;
            long sum = 0;
            StringBuilder sb = new StringBuilder();
            char[] char1 = num1.toCharArray();
            char[] char2 = num2.toCharArray();
            while (i >= 0 || j >= 0) {
                sum = carry;
                if (i >= 0) {
                    sum += Character.getNumericValue(char1[i--]);
                }
                if (j >= 0) {
                    sum += Character.getNumericValue(char2[j--]);
                }
                carry = sum / 10;
                sb.append(sum % 10);
            }
            if (carry != 0) {
                sb.append(carry);
            }

            return sb.reverse().toString();
        }
    }

}

// ============================================================
// FILE 399: _416.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 416. Partition Equal Subset Sum
 *
 * Given a non-empty array containing only positive integers,
 * find if the array can be partitioned into two subsets such that the sum of elements in
both subsets is equal.

 Note:
```

```
 Each of the array element will not exceed 100.
 The array size will not exceed 200.

 Example 1:

 Input: [1, 5, 11, 5]

 Output: true

 Explanation: The array can be partitioned as [1, 5, 5] and [11].


 Example 2:

 Input: [1, 2, 3, 5]

 Output: false

 Explanation: The array cannot be partitioned into equal sum subsets.
 */
public class _416 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/67539/0-1-knapsack-detailed-
explanation
         */
        public boolean canPartition(int[] nums) {
            int sum = 0;
            for (int num : nums) {
                sum += num;
            }

            if ((sum & 1) == 1) {
                return false;
            }

            sum /= 2;

            int n = nums.length;
            boolean[][] dp = new boolean[n + 1][sum + 1];
            for (int i = 0; i < dp.length; i++) {
                Arrays.fill(dp[i], false);
            }

            dp[0][0] = true;

            for (int i = 1; i < n + 1; i++) {
                dp[i][0] = true;
            }

            for (int j = 1; j < sum + 1; j++) {
                dp[0][j] = false;
            }
```

```
            for (int i = 1; i < n + 1; i++) {
                for (int j = 1; j < sum + 1; j++) {
                    dp[i][j] = dp[i - 1][j];
                    if (j >= nums[i - 1]) {
                        dp[i][j] = (dp[i][j] || dp[i - 1][j - nums[i - 1]]);
                    }
                }
            }

            return dp[n][sum];
        }
    }

}

// ============================================================
// FILE 400: _417.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
/**
 * 417. Pacific Atlantic Water Flow
 *
 * Given the following 5x5 matrix:

  Pacific ~   ~   ~   ~   ~
       ~  1   2   2   3  (5) *
       ~  3   2   3  (4) (4) *
       ~  2   4  (5)  3   1  *
       ~ (6) (7)  1   4   5  *
       ~ (5)  1   1   2   4  *
          *   *   *   *   * Atlantic

Return:

[[0, 4], [1, 3], [1, 4], [2, 2], [3, 0], [3, 1], [4, 0]] (positions with parentheses in
above matrix).*/

public class _417 {
    public static class Solution1 {
        /**
         * Credit: looked at this post: https://discuss.leetcode.com/topic/62379/java-
bfs-dfs-from-ocean
         * <p>
         * One typical trick to work on 2d grid problems is to go through the border and
put proper ones into a queue if using BFS.
         */
        public List<int[]> pacificAtlantic(int[][] matrix) {

            List<int[]> result = new ArrayList();
            if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
```

```
                return result;
            }

            int m = matrix.length;
            int n = matrix[0].length;
            boolean[][] pacific = new boolean[m][n];
            boolean[][] atlantic = new boolean[m][n];

            for (int i = 0; i < m; i++) {
                dfs(matrix, pacific, Integer.MIN_VALUE, i, 0);
                dfs(matrix, atlantic, Integer.MIN_VALUE, i, n - 1);
            }

            for (int i = 0; i < n; i++) {
                dfs(matrix, pacific, Integer.MIN_VALUE, 0, i);
                dfs(matrix, atlantic, Integer.MIN_VALUE, m - 1, i);
            }

            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (pacific[i][j] && atlantic[i][j]) {
                        result.add(new int[]{i, j});
                    }
                }
            }

            return result;
        }

        void dfs(int[][] matrix, boolean[][] visited, int height, int x, int y) {
            int m = matrix.length;
            int n = matrix[0].length;
            if (x < 0 || y < 0 || x >= m || y >= n || matrix[x][y] < height || visited[x]
[y]) {
                return;
            }
            visited[x][y] = true;
            dfs(matrix, visited, matrix[x][y], x + 1, y);
            dfs(matrix, visited, matrix[x][y], x, y + 1);
            dfs(matrix, visited, matrix[x][y], x - 1, y);
            dfs(matrix, visited, matrix[x][y], x, y - 1);
        }
    }

}

// ============================================================
// FILE 401: _418.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 418. Sentence Screen Fitting
```

```
 *
 * Given a rows x cols screen and a sentence represented by a list of non-empty words,
find how many times the given sentence can be fitted on the screen.

 Note:

 A word cannot be split into two lines.
 The order of words in the sentence must remain unchanged.
 Two consecutive words in a line must be separated by a single space.
 Total words in the sentence won't exceed 100.
 Length of each word is greater than 0 and won't exceed 10.
 1 ≤ rows, cols ≤ 20,000.

 Example 1:

 Input:
 rows = 2, cols = 8, sentence = ["hello", "world"]

 Output:
 1

 Explanation:
 hello---
 world---

 The character '-' signifies an empty space on the screen.

 Example 2:

 Input:
 rows = 3, cols = 6, sentence = ["a", "bcd", "e"]

 Output:
 2

 Explanation:
 a-bcd-
 e-a---
 bcd-e-

 The character '-' signifies an empty space on the screen.

 Example 3:

 Input:
 rows = 4, cols = 5, sentence = ["I", "had", "apple", "pie"]

 Output:
 1

 Explanation:
 I-had
 apple
 pie-I
```

```
 had--

 The character '-' signifies an empty space on the screen.

 */
public class _418 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/62455/21ms-18-lines-java-solution
         * <p>
         * 1. String s = String.join(" ", sentence) + " " ;. This line gives us a
formatted sentence to be put to our screen.
         * 2. start is the counter for how many valid characters from s have been put to
our screen.
         * 3. if (s.charAt(start % l) == ' ') is the situation that we don't need an
extra space for current row. The current row could be successfully fitted. So that we
need to increase our counter by using start++.
         * 4. The else is the situation, which the next word can't fit to current row. So
that we need to remove extra characters from next word.
         * 5. start / s.length() is (# of valid characters) / our formatted sentence.
         */
        public int wordsTyping(String[] sentence, int rows, int cols) {
            String s = String.join(" ", sentence) + " ";
            int start = 0;
            int l = s.length();
            for (int i = 0; i < rows; i++) {
                start += cols;
                if (s.charAt(start % l) == ' ') {
                    start++;
                } else {
                    while (start > 0 && s.charAt((start - 1) % l) != ' ') {
                        start--;
                    }
                }
            }
            return start / s.length();
        }
    }
}

// ============================================================
// FILE 402: _419.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 419. Battleships in a Board
 *
 * Given an 2D board, count how many battleships are in it.
 * The battleships are represented with 'X's, empty slots are represented with '.'s.
 * You may assume the following rules:
```

```
 You receive a valid board, made of only battleships or empty slots.
 Battleships can only be placed horizontally or vertically.
 In other words, they can only be made of the shape 1xN (1 row, N columns) or Nx1 (N
rows, 1 column), where N can be of any size.
 At least one horizontal or vertical cell separates between two battleships - there are
no adjacent battleships.

 Example:

 X..X
 ...X
 ...X

 In the above board there are 2 battleships.

 Invalid Example:

 ...X
 XXXX
 ...X

 This is an invalid board that you will not receive - as battleships will always have a
cell separating between them.

 Follow up:
 Could you do it in one-pass, using only O(1) extra memory and without modifying the
value of the board?

 */
public class _419 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/62970/simple-java-solution,
         * <p>
         * This solution does NOT modify original input.
         * Basically, it only counts the top-left one while ignoring all other parts of
one battleship,
         * using the top-left one as a representative for one battle.
         * This is achieved by counting cells that don't have 'X' to the left and above
them.
         */
        public int countBattleships(char[][] board) {
            if (board == null || board.length == 0) {
                return 0;
            }
            int count = 0;
            int m = board.length;
            int n = board[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (board[i][j] == '.') {
                        continue;//if it can pass this line, then board[i][j] must be 'X'
                    }
```

```
                if (j > 0 && board[i][j - 1] == 'X') {
                    continue;//then we check if its left is 'X'
                }
                if (i > 0 && board[i - 1][j] == 'X') {
                    continue;//also check if its top is 'X'
                }
                count++;
            }
        }
        return count;
    }
}

public static class Solution2 {
    /**
     * My original solution, actually modified the input. I just undo it at the end.
     */
    public int countBattleships(char[][] board) {
        if (board == null || board.length == 0) {
            return 0;
        }
        int result = 0;
        int m = board.length;
        int n = board[0].length;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 'X') {
                    result++;
                    dfs(board, i, j, m, n);
                }
            }
        }

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == '#') {
                    board[i][j] = 'X';
                }
            }
        }
        return result;
    }

    private void dfs(char[][] board, int x, int y, int m, int n) {
        if (x < 0 || x >= m || y < 0 || y >= n || board[x][y] != 'X') {
            return;
        }
        if (board[x][y] == 'X') {
            board[x][y] = '#';
        }
        dfs(board, x + 1, y, m, n);
        dfs(board, x, y + 1, m, n);
        dfs(board, x - 1, y, m, n);
        dfs(board, x, y - 1, m, n);
```

```
        }
    }

}

// ============================================================
// FILE 403: _420.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 420. Strong Password Checker
 *
 * A password is considered strong if below conditions are all met:

 It has at least 6 characters and at most 20 characters.
 It must contain at least one lowercase letter, at least one uppercase letter, and at
least one digit.
 It must NOT contain three repeating characters in a row ("...aaa..." is weak, but
"...aa...a..." is strong, assuming other conditions are met).
 Write a function strongPasswordChecker(s), that takes a string s as input, and return
the MINIMUM
 change required to make s a strong password. If s is already strong, return 0.

 Insertion, deletion or replace of any one character are all considered as one change.
 */
public class _420 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/63854/o-n-java-solution-by-
analyzing-changes-allowed-to-fix-each-condition
         */
        public int strongPasswordChecker(String s) {
            int res = 0;
            int a = 1;
            int A = 1;
            int d = 1;
            char[] carr = s.toCharArray();
            int[] arr = new int[carr.length];

            for (int i = 0; i < arr.length; ) {
                if (Character.isLowerCase(carr[i])) {
                    a = 0;
                }
                if (Character.isUpperCase(carr[i])) {
                    A = 0;
                }
                if (Character.isDigit(carr[i])) {
                    d = 0;
                }

                int j = i;
                while (i < carr.length && carr[i] == carr[j]) {
```

```
                    i++;
                }
                arr[j] = i - j;
            }

            int totalMissing = (a + A + d);

            if (arr.length < 6) {
                res += totalMissing + Math.max(0, 6 - (arr.length + totalMissing));
            } else {
                int overLen = Math.max(arr.length - 20, 0);
                int leftOver = 0;
                res += overLen;

                for (int k = 1; k < 3; k++) {
                    for (int i = 0; i < arr.length && overLen > 0; i++) {
                        if (arr[i] < 3 || arr[i] % 3 != (k - 1)) {
                            continue;
                        }
                        arr[i] -= Math.min(overLen, k);
                        overLen -= k;
                    }
                }

                for (int i = 0; i < arr.length; i++) {
                    if (arr[i] >= 3 && overLen > 0) {
                        int need = arr[i] - 2;
                        arr[i] -= overLen;
                        overLen -= need;
                    }

                    if (arr[i] >= 3) {
                        leftOver += arr[i] / 3;
                    }
                }

                res += Math.max(totalMissing, leftOver);
            }

            return res;
        }
    }
}

// ============================================================
// FILE 404: _421.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
```

```
 * Given a non-empty array of numbers, a0, a1, a2, … , an-1, where 0 ≤ ai < 231.

 Find the maximum result of ai XOR aj, where 0 ≤ i, j < n.

 Could you do this in O(n) runtime?

 Example:

 Input: [3, 10, 5, 25, 2, 8]

 Output: 28

 Explanation: The maximum result is 5 ^ 25 = 28.
 */
public class _421 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/63213/java-o-n-solution-using-bit-
manipulation-and-hashmap/7
        public int findMaximumXOR(int[] nums) {
            int max = 0;
            int mask = 0;
            for (int i = 31; i >= 0; i--) {
                mask |= (1 << i);//the mask will grow like this: 100...000, 110...000,
111...000 to 111...111, each time, we only get the most left part of all numbers in the
given array
                System.out.println("mask = " + Integer.toBinaryString(mask));
                Set<Integer> set = new HashSet<>();
                for (int num : nums) {
                    System.out.println("num = " + Integer.toBinaryString(num));
                    set.add(num & mask);
                    System.out.println("mask & num = " + Integer.toBinaryString(mask &
num));
                }

                int candidate = max | (1 << i);
                System.out.println("candidate = " + Integer.toBinaryString(candidate));
                /**Reason behind this: if a ^ prefix = candidate, then a ^ candidate =
prefix, also prefix ^ candidate = a
                 * in this below code: we use this one: prefix ^ candidate = a*/
                for (int prefix : set) {
                    System.out.println("candidate ^ prefix = " +
Integer.toBinaryString(candidate ^ prefix));
                    if (set.contains(candidate ^ prefix)) {
                        max = candidate;
                    }
                }
                System.out.println("max = " + max);
                System.out.println("i = " + i);
                System.out.println("================================================");
            }
            return max;
        }
    }
```

```
}

// ============================================================
// FILE 405: _422.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;

/**
 * 422. Valid Word Square
 *
 * Given a sequence of words, check whether it forms a valid word square.

 A sequence of words forms a valid word square if the kth row and column read the exact
same string, where 0 ≤ k < max(numRows, numColumns).

 Note:
 The number of words given is at least 1 and does not exceed 500.
 Word length will be at least 1 and does not exceed 500.
 Each word contains only lowercase English alphabet a-z.
 Example 1:

 Input:
 [
 "abcd",
 "bnrt",
 "crmy",
 "dtye"
 ]

 Output:
 true

 Explanation:
 The first row and first column both read "abcd".
 The second row and second column both read "bnrt".
 The third row and third column both read "crmy".
 The fourth row and fourth column both read "dtye".

 Therefore, it is a valid word square.
 Example 2:

 Input:
 [
 "abcd",
 "bnrt",
 "crm",
 "dt"
 ]

 Output:
```

```
 true

 Explanation:
 The first row and first column both read "abcd".
 The second row and second column both read "bnrt".
 The third row and third column both read "crm".
 The fourth row and fourth column both read "dt".

 Therefore, it is a valid word square.
 Example 3:

 Input:
 [
 "ball",
 "area",
 "read",
 "lady"
 ]

 Output:
 false

 Explanation:
 The third row reads "read" while the third column reads "lead".

 Therefore, it is NOT a valid word square.
 */
public class _422 {

    public static class Solution1 {
        public boolean validWordSquare(List<String> words) {
            for (int i = 0; i < words.size(); i++) {
                String word = words.get(i);
                for (int j = 0; j < word.length(); j++) {
                    if (j >= words.size()) {
                        return false;
                    }
                    if (i >= words.get(j).length()) {
                        return false;
                    }
                    if (word.charAt(j) != words.get(j).charAt(i)) {
                        return false;
                    }
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 406: _423.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 423. Reconstruct Original Digits from English
 *
 * Given a non-empty string containing an out-of-order English representation of digits
0-9, output the digits in ascending order.

 Note:
 Input contains only lowercase English letters.
 Input is guaranteed to be valid and can be transformed to its original digits. That
means invalid inputs such as "abc" or "zerone" are not permitted.
 Input length is less than 50,000.

 Example 1:
 Input: "owoztneoer"

 Output: "012"
 Example 2:
 Input: "fviefuro"

 Output: "45"

 */
public class _423 {

    public static class Solution1 {
        public String originalDigits(String s) {
            /**we can use one char as a representative to uniquely stand for one number,
             * for some numbers that we cannot find a unique representive, we can dedup.
             * e.g. for number one, if we use 'o' as its representive, then 'o' also
exists in numbers 2, 4 and 0, so
             * we need to dedupe the 'o' in those numbers.
             * Also, the order to dedupe matters:
             * we'll have to dedupe for counts[3], counts[5], counts[7] first before we
dedupe counts[1] and counts[9].*/
            int[] counts = new int[10];
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == 'o') {
                    counts[1]++;//2,4,0
                }
                if (s.charAt(i) == 'w') {
                    counts[2]++;
                }
                if (s.charAt(i) == 'h') {
                    counts[3]++;//8
                }
                if (s.charAt(i) == 'u') {
                    counts[4]++;
                }
                if (s.charAt(i) == 'f') {
                    counts[5]++;//4
                }
                if (s.charAt(i) == 'x') {
```

```
                    counts[6]++;
                }
                if (s.charAt(i) == 'v') {
                    counts[7]++;//5
                }
                if (s.charAt(i) == 'g') {
                    counts[8]++;
                }
                if (s.charAt(i) == 'i') {
                    counts[9]++;//5,6,8
                }
                if (s.charAt(i) == 'z') {
                    counts[0]++;
                }
            }

            counts[3] -= counts[8];
            counts[5] -= counts[4];
            counts[7] -= counts[5];
            counts[1] = counts[1] - (counts[2] + counts[4] + counts[0]);
            counts[9] = counts[9] - (counts[5] + counts[6] + counts[8]);

            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < 10; i++) {
                for (int j = 0; j < counts[i]; j++) {
                    stringBuilder.append(i);
                }
            }
            return stringBuilder.toString();
        }
    }

}

// ============================================================
// FILE 407: _424.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 424. Longest Repeating Character Replacement
 *
 * Given a string that consists of only uppercase English letters,
 * you can replace any letter in the string with another letter at most k times.
 * Find the length of a longest substring containing all repeating letters you can get
after performing the above operations.

 Note:
 Both the string's length and k will not exceed 104.

 Example 1:

 Input:
```

```
 s = "ABAB", k = 2

 Output:
 4

 Explanation:
 Replace the two 'A's with two 'B's or vice versa.
 Example 2:

 Input:
 s = "AABABBA", k = 1

 Output:
 4

 Explanation:
 Replace the one 'A' in the middle with 'B' and form "AABBBBA".
 The substring "BBBB" has the longest repeating letters, which is 4.
 */
public class _424 {

    public static class Solution1 {
        //credit: https://discuss.leetcode.com/topic/63494/java-12-lines-o-n-sliding-
window-solution-with-explanation
        public int characterReplacement(String s, int k) {
            int len = s.length();
            int[] count = new int[26];
            int start = 0;
            int maxCount = 0;
            int maxLength = 0;
            for (int end = 0; end < len; end++) {
                maxCount = Math.max(maxCount, ++count[s.charAt(end) - 'A']);
                while (end - start + 1 - maxCount > k) {
                    count[s.charAt(start) - 'A']--;
                    start++;
                }
                maxLength = Math.max(maxLength, end - start + 1);
            }
            return maxLength;
        }
    }
}

// ============================================================
// FILE 408: _425.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 425. Word Squares
```

```
 *
 * Given a set of words (without duplicates), find all word squares you can build from
them.

 A sequence of words forms a valid word square if the kth row and column read the exact
same string, where 0 ≤ k < max(numRows, numColumns).

 For example, the word sequence ["ball","area","lead","lady"] forms a word square because
each word reads the same both horizontally and vertically.

 b a l l
 a r e a
 l e a d
 l a d y

 Note:

 There are at least 1 and at most 1000 words.
 All words will have the exact same length.
 Word length is at least 1 and at most 5.
 Each word contains only lowercase English alphabet a-z.

 Example 1:

 Input:
 ["area","lead","wall","lady","ball"]

 Output:
 [
 [ "wall",
 "area",
 "lead",
 "lady"
 ],
 [ "ball",
 "area",
 "lead",
 "lady"
 ]
 ]

 Explanation:
 The output consists of two word squares. The order of output does not matter (just the
order of words in each word square matters).


 Example 2:

 Input:
 ["abat","baba","atan","atal"]

 Output:
 [
 [ "baba",
```

```
 "abat",
 "baba",
 "atan"
 ],
 [ "baba",
 "abat",
 "baba",
 "atal"
 ]
 ]

 Explanation:
 The output consists of two word squares. The order of output does not matter (just the order of words in each word square matters).
 */
public class _425 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/63516/explained-my-java-solution-using-trie-126ms-16-16/2
         */

        class TrieNode {
            List<String> startWith;
            TrieNode[] children;

            TrieNode() {
                startWith = new ArrayList<>();
                children = new TrieNode[26];
            }
        }

        class Trie {
            TrieNode root;

            Trie(String[] words) {
                root = new TrieNode();
                for (String word : words) {
                    TrieNode cur = root;
                    for (char ch : word.toCharArray()) {
                        int index = ch - 'a';
                        if (cur.children[index] == null) {
                            cur.children[index] = new TrieNode();
                        }
                        cur.children[index].startWith.add(word);
                        cur = cur.children[index];
                    }
                }
            }

            List<String> findByPrefix(String prefix) {
                List<String> ans = new ArrayList<>();
                TrieNode cur = root;
```

```
                for (char ch : prefix.toCharArray()) {
                    int index = ch - 'a';
                    if (cur.children[index] == null) {
                        return ans;
                    }

                    cur = cur.children[index];
                }
                ans.addAll(cur.startWith);
                return ans;
            }
        }

        public List<List<String>> wordSquares(String[] words) {
            List<List<String>> ans = new ArrayList<>();
            if (words == null || words.length == 0) {
                return ans;
            }
            int len = words[0].length();
            Trie trie = new Trie(words);
            List<String> ansBuilder = new ArrayList<>();
            for (String w : words) {
                ansBuilder.add(w);
                search(len, trie, ans, ansBuilder);
                ansBuilder.remove(ansBuilder.size() - 1);
            }

            return ans;
        }

        private void search(int len, Trie trie, List<List<String>> ans,
                            List<String> ansBuilder) {
            if (ansBuilder.size() == len) {
                ans.add(new ArrayList<>(ansBuilder));
                return;
            }

            int idx = ansBuilder.size();
            StringBuilder prefixBuilder = new StringBuilder();
            for (String s : ansBuilder) {
                prefixBuilder.append(s.charAt(idx));
            }
            List<String> startWith = trie.findByPrefix(prefixBuilder.toString());
            for (String sw : startWith) {
                ansBuilder.add(sw);
                search(len, trie, ans, ansBuilder);
                ansBuilder.remove(ansBuilder.size() - 1);
            }
        }
    }

}
```

```
// ============================================================
// FILE 409: _429.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 429. N-ary Tree Level Order Traversal
 *
 * Given an n-ary tree, return the level order traversal of its nodes' values. (ie, from
left to right, level by level).
 *
 * For example, given a 3-ary tree:
 *
 *                              1
 *                            / | \
 *                           3  2 4
 *                          / \
 *                         5  6
 *
 *  We should return its level order traversal:
 *
 * [
 *      [1],
 *      [3,2,4],
 *      [5,6]
 * ]
 *
 *
 * Note:
 *
 * The depth of the tree is at most 1000.
 * The total number of nodes is at most 5000.
 */
public class _429 {
  public static class Solution1 {
    public List<List<Integer>> levelOrder(Node root) {
      List<List<Integer>> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      Queue<Node> queue = new LinkedList<>();
      queue.offer(root);
      while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
          Node currentNode = queue.poll();
```

```
          if (currentNode != null) {
            level.add(currentNode.val);
            for (Node child : currentNode.children) {
              queue.offer(child);
            }
          }
        }
        result.add(level);
      }
      return result;
    }
  }
}

// ============================================================
// FILE 410: _432.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 432. All O`one Data Structure
 * Implement a data structure supporting the following operations:

 1. Inc(Key) - Inserts a new key with value 1. Or increments an existing key by 1. Key is
guaranteed to be a non-empty string.
 2. Dec(Key) - If Key's value is 1, remove it from the data structure. Otherwise
decrements an existing key by 1. If the key does not exist, this function does nothing.
Key is guaranteed to be a non-empty string.
 3. GetMaxKey() - Returns one of the keys with maximal value. If no element exists,
return an empty string "".
 4. GetMinKey() - Returns one of the keys with minimal value. If no element exists,
return an empty string "".

 Challenge: Perform all these in O(1) time complexity.

 */
public class _432 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/65634/java-ac-all-strict-o-1-not-
average-o-1-easy-to-read/2
         */
        class AllOne {
            // maintain a doubly linked list of Buckets
            private Bucket head;
            private Bucket tail;
```

```
            // for accessing a specific Bucket among the Bucket list in O(1) time
            private Map<Integer, Bucket> countBucketMap;
            // keep track of count of keys
            private Map<String, Integer> keyCountMap;

            // each Bucket contains all the keys with the same count
            private class Bucket {
                int count;
                Set<String> keySet;
                Bucket next;
                Bucket pre;

                public Bucket(int cnt) {
                    count = cnt;
                    keySet = new HashSet<>();
                }
            }

            /**
             * Initialize your data structure here.
             */
            public AllOne() {
                head = new Bucket(Integer.MIN_VALUE);
                tail = new Bucket(Integer.MAX_VALUE);
                head.next = tail;
                tail.pre = head;
                countBucketMap = new HashMap<>();
                keyCountMap = new HashMap<>();
            }

            /**
             * Inserts a new key <Key> with value 1. Or increments an existing key by 1.
             */
            public void inc(String key) {
                if (keyCountMap.containsKey(key)) {
                    changeKey(key, 1);
                } else {
                    keyCountMap.put(key, 1);
                    if (head.next.count != 1) {
                        addBucketAfter(new Bucket(1), head);
                    }
                    head.next.keySet.add(key);
                    countBucketMap.put(1, head.next);
                }
            }

            /**
             * Decrements an existing key by 1. If Key's value is 1, remove it from the
data structure.
             */
            public void dec(String key) {
                if (keyCountMap.containsKey(key)) {
                    int count = keyCountMap.get(key);
                    if (count == 1) {
```

```
                    keyCountMap.remove(key);
                    removeKeyFromBucket(countBucketMap.get(count), key);
                } else {
                    changeKey(key, -1);
                }
            }
        }

        /**
         * Returns one of the keys with maximal value.
         */
        public String getMaxKey() {
            return tail.pre == head ? "" : (String)
tail.pre.keySet.iterator().next();
        }

        /**
         * Returns one of the keys with Minimal value.
         */
        public String getMinKey() {
            return head.next == tail ? "" : (String)
head.next.keySet.iterator().next();
        }

        // helper function to make change on given key according to offset
        private void changeKey(String key, int offset) {
            int count = keyCountMap.get(key);
            keyCountMap.put(key, count + offset);
            Bucket curBucket = countBucketMap.get(count);
            Bucket newBucket;
            if (countBucketMap.containsKey(count + offset)) {
                // target Bucket already exists
                newBucket = countBucketMap.get(count + offset);
            } else {
                // add new Bucket
                newBucket = new Bucket(count + offset);
                countBucketMap.put(count + offset, newBucket);
                addBucketAfter(newBucket, offset == 1 ? curBucket : curBucket.pre);
            }
            newBucket.keySet.add(key);
            removeKeyFromBucket(curBucket, key);
        }

        private void removeKeyFromBucket(Bucket bucket, String key) {
            bucket.keySet.remove(key);
            if (bucket.keySet.size() == 0) {
                removeBucketFromList(bucket);
                countBucketMap.remove(bucket.count);
            }
        }

        private void removeBucketFromList(Bucket bucket) {
            bucket.pre.next = bucket.next;
            bucket.next.pre = bucket.pre;
```

```
                bucket.next = null;
                bucket.pre = null;
            }

            // add newBucket after preBucket
            private void addBucketAfter(Bucket newBucket, Bucket preBucket) {
                newBucket.pre = preBucket;
                newBucket.next = preBucket.next;
                preBucket.next.pre = newBucket;
                preBucket.next = newBucket;
            }
        }
    }
}

/**
 * Your AllOne object will be instantiated and called as such:
 * AllOne obj = new AllOne();
 * obj.inc(key);
 * obj.dec(key);
 * String param_3 = obj.getMaxKey();
 * String param_4 = obj.getMinKey();
 */

// ============================================================
// FILE 411: _434.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 434. Number of Segments in a String
 *
 * Count the number of segments in a string,
 * where a segment is defined to be a contiguous sequence of non-space characters.
 *
 * Please note that the string does not contain any non-printable characters.

Example:

Input: "Hello, my name is John"
Output: 5*/
public class _434 {

    public static class Solution1 {
        public int countSegments(String s) {
            if (s == null || s.isEmpty()) {
                return 0;
            }
            String[] segments = s.split(" ");
            int count = 0;
            for (String seg : segments) {
                if (seg.equals("")) {
                    continue;
                }
```

```
                count++;
            }
            return count;
        }
    }
}

// ============================================================
// FILE 412: _435.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.Arrays;
import java.util.Collections;

/**
 * 435. Non-overlapping Intervals
 *
 * Given a collection of intervals,
 * find the minimum number of intervals you need to remove to make the rest of the
 * intervals non-overlapping.

 Note:
 You may assume the interval's end point is always bigger than its start point.
 Intervals like [1,2] and [2,3] have borders "touching" but they don't overlap each
other.

 Example 1:
 Input: [ [1,2], [2,3], [3,4], [1,3] ]
 Output: 1
 Explanation: [1,3] can be removed and the rest of intervals are non-overlapping.

 Example 2:
 Input: [ [1,2], [1,2], [1,2] ]
 Output: 2
 Explanation: You need to remove two [1,2] to make the rest of intervals non-overlapping.

 Example 3:
 Input: [ [1,2], [2,3] ]
 Output: 0
 Explanation: You don't need to remove any of the intervals since they're already non-
overlapping.
 */

public class _435 {

    public static class Solution1 {
        /**
         * credit:: https://discuss.leetcode.com/topic/65828/java-solution-with-clear-
explain
         * and https://discuss.leetcode.com/topic/65594/java-least-is-most
```

```
         * Sort the intervals by their end time, if equal, then sort by their start time.
         */
        public int eraseOverlapIntervals(Interval[] intervals) {
            Collections.sort(Arrays.asList(intervals), (o1, o2) -> {
                if (o1.end != o2.end) {
                    return o1.end - o2.end;
                } else {
                    return o2.start - o1.start;
                }
            });
            int end = Integer.MIN_VALUE;
            int count = 0;
            for (Interval interval : intervals) {
                if (interval.start >= end) {
                    end = interval.end;
                } else {
                    count++;
                }
            }
            return count;
        }
    }

}

// ==========================================================
// FILE 413: _436.java
// ==========================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Interval;

import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

/**436. Find Right Interval
 *
 * Given a set of intervals, for each of the interval i, check if there exists an
interval j whose start
 * point is bigger than or equal to the end point of the interval i, which can be called
that j is on the "right" of i.
 *
 * For any interval i, you need to store the minimum interval j's index,
 * which means that the interval j has the minimum start point to build the "right"
relationship for interval i.
 * If the interval j doesn't exist, store -1 for the interval i. Finally, you need output
the stored value of each interval as an array.
 *
 * Note:
 *
 * You may assume the interval's end point is always bigger than its start point.
```

```
 * You may assume none of these intervals have the same start point.
 *
 * Example 1:
 * Input: [ [1,2] ]
 * Output: [-1]
 * Explanation: There is only one interval in the collection, so it outputs -1.
 *
 * Example 2:
 * Input: [ [3,4], [2,3], [1,2] ]
 * Output: [-1, 0, 1]
 * Explanation: There is no satisfied "right" interval for [3,4].
 * For [2,3], the interval [3,4] has minimum-"right" start point;
 * For [1,2], the interval [2,3] has minimum-"right" start point.
 *
 * Example 3:
 * Input: [ [1,4], [2,3], [3,4] ]
 * Output: [-1, 2, -1]
 * Explanation: There is no satisfied "right" interval for [1,4] and [3,4].
 * For [2,3], the interval [3,4] has minimum-"right" start point.
 * */

public class _436 {

    public static class Solution1 {
        public int[] findRightInterval(Interval[] intervals) {
            if (intervals == null || intervals.length == 0) {
                return new int[0];
            }
            int[] result = new int[intervals.length];
            result[0] = -1;
            Interval last = intervals[intervals.length - 1];
            Interval first = intervals[0];
            Map<Interval, Integer> map = new HashMap();
            for (int i = 0; i < intervals.length; i++) {
                map.put(intervals[i], i);
            }

            Collections.sort(Arrays.asList(intervals), (o1, o2) -> o1.start - o2.start);

            for (int i = 1; i < intervals.length; i++) {
                //TODO: use binary search for the minimum start interval for interval[i-
1] instead of a while loop
                int tmp = i - 1;
                int tmpI = i;
                while (tmpI < intervals.length && intervals[tmpI].start <
intervals[tmp].end) {
                    tmpI++;
                }
                if (tmpI < intervals.length) {
                    result[map.get(intervals[tmp])] = map.get(intervals[tmpI]);
                } else {
                    result[map.get(intervals[tmp])] = -1;
                }
            }
```

```
            if (result[intervals.length - 1] == 0 && last.end > first.start) {
                result[intervals.length - 1] = -1;
            }
            return result;
        }
    }

}

// ============================================================
// FILE 414: _437.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 437. Path Sum III
 *
 * You are given a binary tree in which each node contains an integer value.

 Find the number of paths that sum to a given value.

 The path does not need to start or end at the root or a leaf, but it must go downwards
(traveling only from parent nodes to child nodes).

 The tree has no more than 1,000 nodes and the values are in the range -1,000,000 to
1,000,000.

 Example:

 root = [10,5,-3,3,2,null,11,3,-2,null,1], sum = 8

      10
     /  \
    5   -3
   / \    \
  3   2   11
 / \   \
3  -2  1

 Return 3. The paths that sum to 8 are:

 1.  5 -> 3
 2.  5 -> 2 -> 1
 3. -3 -> 11
 */

public class _437 {

    public static class Solution1 {
        public int pathSum(TreeNode root, int sum) {
            if (root == null) {
```

```
                return 0;
            }
            return pathSumFrom(root, sum) + pathSum(root.left, sum) + pathSum(root.right,
sum);
        }

        private int pathSumFrom(TreeNode root, int sum) {
            if (root == null) {
                return 0;
            }
            return (root.val == sum ? 1 : 0) + pathSumFrom(root.left, sum - root.val) +
pathSumFrom(root.right, sum - root.val);
        }
    }

}

// ============================================================
// FILE 415: _438.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 438. Find All Anagrams in a String
 *
 * Given a string s and a non-empty string p, find all the start indices of p's anagrams
in s.
 Strings consists of lowercase English letters only and the length of both strings s and
p will not be larger than 20,100.

 The order of output does not matter.

 Example 1:
 Input:
 s: "cbaebabacd" p: "abc"
 Output:
 [0, 6]

 Explanation:
 The substring with start index = 0 is "cba", which is an anagram of "abc".
 The substring with start index = 6 is "bac", which is an anagram of "abc".

 Example 2:
 Input:
 s: "abab" p: "ab"
 Output:
 [0, 1, 2]

 Explanation:
 The substring with start index = 0 is "ab", which is an anagram of "ab".
```

```
 The substring with start index = 1 is "ba", which is an anagram of "ab".
 The substring with start index = 2 is "ab", which is an anagram of "ab".*/

public class _438 {
    public static class Solution1 {
        /**
         * O(m*n) solution, my original and most intuitive one, but sort of brute force,
when m is close to n, it becomes O(n^2) runtime complexity.
         */
        public List<Integer> findAnagrams(String s, String p) {
            List<Integer> result = new ArrayList();
            for (int i = 0; i <= s.length() - p.length(); i++) {
                if (isAnagram(s.substring(i, i + p.length()), p)) {
                    result.add(i);
                }
            }
            return result;
        }

        private boolean isAnagram(String s, String p) {
            int[] c = new int[26];
            for (int i = 0; i < s.length(); i++) {
                c[s.charAt(i) - 'a']++;
                c[p.charAt(i) - 'a']--;
            }

            for (int i : c) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
        }
    }


    public static class Solution2 {
        /**
         * Sliding Window
         */
        public List<Integer> findAnagrams(String s, String p) {
            List<Integer> result = new ArrayList();
            int[] hash = new int[26];
            for (char c : p.toCharArray()) {
                hash[c - 'a']++;
            }
            int start = 0;
            int end = 0;
            int count = p.length();
            while (end < s.length()) {
                if (hash[s.charAt(end) - 'a'] > 0) {
                    count--;
                }
                hash[s.charAt(end) - 'a']--;
```

```
                end++;

                if (count == 0) {
                    result.add(start);
                }

                if ((end - start) == p.length()) {
                    if (hash[s.charAt(start) - 'a'] >= 0) {
                        count++;
                    }
                    hash[s.charAt(start) - 'a']++;
                    start++;
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 416: _439.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 439. Ternary Expression Parser
 *
 * Given a string representing arbitrarily nested ternary expressions, calculate the
result of the expression.
 * You can always assume that the given expression is valid and only consists of digits
0-9, ?, :, T and F (T and F represent True and False respectively).

 Note:

 The length of the given string is ≤ 10000.
 Each number will contain only one digit.
 The conditional expressions group right-to-left (as usual in most languages).
 The condition will always be either T or F. That is, the condition will never be a
digit.
 The result of the expression will always evaluate to either a digit 0-9, T or F.

 Example 1:

 Input: "T?2:3"

 Output: "2"

 Explanation: If true, then result is 2; otherwise result is 3.
```

```
 Example 2:

 Input: "F?1:T?4:5"

 Output: "4"

 Explanation: The conditional expressions group right-to-left. Using parenthesis, it is
read/evaluated as:

 "(F ? 1 : (T ? 4 : 5))"                  "(F ? 1 : (T ? 4 : 5))"
 -> "(F ? 1 : 4)"                or       -> "(T ? 4 : 5)"
 -> "4"                                   -> "4"
 Example 3:

 Input: "T?T?F:5:3"

 Output: "F"

 Explanation: The conditional expressions group right-to-left. Using parenthesis, it is
read/evaluated as:

 "(T ? (T ? F : 5) : 3)"                  "(T ? (T ? F : 5) : 3)"
 -> "(T ? F : 3)"                or       -> "(T ? F : 5)"
 -> "F"                                   -> "F"
 */
public class _439 {

    public static class Solution1 {
        /**
         * Below is my original solution, but looking at Discuss, a more concise way is
to use just one
         * stack, process it from right to left, example:
https://discuss.leetcode.com/topic/64409/very-easy-1-pass-stack-solution-in-java-no-
string-concat
         */

        public String parseTernary(String expression) {
            Deque<Character> stack = new ArrayDeque<>();
            Deque<Character> tmpStack = new ArrayDeque<>();
            for (char c : expression.toCharArray()) {
                stack.addFirst(c);
            }
            while (!stack.isEmpty()) {
                if (stack.peek() != '?') {
                    tmpStack.addFirst(stack.pollFirst());
                } else {
                    char char1 = tmpStack.removeFirst();
                    tmpStack.removeFirst();//remove ':'
                    char char2 = tmpStack.removeFirst();
                    stack.removeFirst();//remove '?'
                    char judge = stack.removeFirst();
                    tmpStack.addFirst(judge == 'T' ? char1 : char2);
                    while (!tmpStack.isEmpty()) {
                        stack.addFirst(tmpStack.pollFirst());
```

```
                    }
                }
                if (stack.size() == 1) {
                    break;
                }
            }
            return Character.toString(stack.removeFirst());
        }
    }

}

// ============================================================
// FILE 417: _440.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 440. K-th Smallest in Lexicographical Order
 *
 * Given integers n and k, find the lexicographically k-th smallest integer in the range
from 1 to n.

 Note: 1 ≤ k ≤ n ≤ 109.

 Example:

 Input:
 n: 13   k: 2

 Output:
 10

 Explanation:
 The lexicographical order is [1, 10, 11, 12, 13, 2, 3, 4, 5, 6, 7, 8, 9], so the second
smallest number is 10.
 */
public class _440 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/64624/concise-easy-to-understand-
java-5ms-solution-with-explaination/2
         */
        public int findKthNumber(int n, int k) {
            int curr = 1;
            k = k - 1;
            while (k > 0) {
                int steps = calSteps(n, curr, curr + 1);
                if (steps <= k) {
                    curr += 1;
                    k -= steps;
                } else {
```

```
                    curr *= 10;
                    k -= 1;
                }
            }
            return curr;
        }

        //use long in case of overflow
        public int calSteps(int n, long n1, long n2) {
            int steps = 0;
            while (n1 <= n) {
                steps += Math.min(n + 1, n2) - n1;
                n1 *= 10;
                n2 *= 10;
            }
            return steps;
        }
    }
}

// ============================================================
// FILE 418: _441.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 441. Arranging Coins
 *
 * You have a total of n coins that you want to form in a staircase shape, where every k-
th row must have exactly k coins.

 Given n, find the total number of full staircase rows that can be formed.

 n is a non-negative integer and fits within the range of a 32-bit signed integer.

 Example 1:

 n = 5

 The coins can form the following rows:
 ¤
 ¤ ¤
 ¤ ¤

 Because the 3rd row is incomplete, we return 2.


 Example 2:

 n = 8

 The coins can form the following rows:
 ¤
```

```
 ¤ ¤
 ¤ ¤ ¤
 ¤ ¤

 Because the 4th row is incomplete, we return 3.*/
public class _441 {

    public static class Solution1 {
        public int arrangeCoins(int n) {
            if (n < 2) {
                return n;
            }
            int row = 0;
            int count = 0;
            long sum = 0;
            while (sum < n) {
                row += 1;
                sum += row;
                count++;
            }
            if (sum == n) {
                return count;
            }
            return count - 1;
        }
    }

}

// ============================================================
// FILE 419: _442.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 442. Find All Duplicates in an Array
 *
 * Given an array of integers, 1 ≤ a[i] ≤ n (n = size of array), some elements appear
twice and others appear once.

 Find all the elements that appear twice in this array.

 Could you do it without extra space and in O(n) runtime?

 Example:
 Input:
 [4,3,2,7,8,2,3,1]
```

```
 Output:
 [2,3]
 */
public class _442 {

    public static class Solution1 {
        //O(n) space
        //O(n) time
        public List<Integer> findDuplicates(int[] nums) {
            Set<Integer> set = new HashSet();
            List<Integer> result = new ArrayList();
            for (int i : nums) {
                if (!set.add(i)) {
                    result.add(i);
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        //O(1) space
        //O(n) time
        public List<Integer> findDuplicates(int[] nums) {
            List<Integer> result = new ArrayList();
            for (int i = 0; i < nums.length; i++) {
                int index = Math.abs(nums[i]) - 1;
                if (nums[index] < 0) {
                    result.add(Math.abs(index + 1));
                }
                nums[index] = -nums[index];
            }
            return result;
        }
    }

}

// ============================================================
// FILE 420: _443.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 443. String Compression
 *
 * Given an array of characters, compress it in-place.
 * The length after compression must always be smaller than or equal to the original
array.
 * Every element of the array should be a character (not int) of length 1.
 * After you are done modifying the input array in-place, return the new length of the
array.
```

```
 Example 1:
 Input:
 ["a","a","b","b","c","c","c"]

 Output:
 Return 6, and the first 6 characters of the input array should be:
["a","2","b","2","c","3"]

 Explanation:
 "aa" is replaced by "a2". "bb" is replaced by "b2". "ccc" is replaced by "c3".

 Example 2:
 Input:
 ["a"]

 Output:
 Return 1, and the first 1 characters of the input array should be: ["a"]

 Explanation:
 Nothing is replaced.

 Example 3:
 Input:
 ["a","b","b","b","b","b","b","b","b","b","b","b","b"]

 Output:
 Return 4, and the first 4 characters of the input array should be: ["a","b","1","2"].

 Explanation:
 Since the character "a" does not repeat, it is not compressed. "bbbbbbbbbbbb" is
replaced by "b12".
 Notice each digit has it's own entry in the array.

 Note:
 All characters have an ASCII value in [35, 126].
 1 <= len(chars) <= 1000.
 */
public class _443 {
    public static class Solution1 {
        /**This is breaking the rules, it's not in-place.*/
        public int compress(char[] chars) {
            if (chars == null || chars.length == 0) {
                return 0;
            }
            StringBuilder sb = new StringBuilder();
            int count = 1;
            char prev = chars[0];
            for (int i = 1; i < chars.length; i++) {
                if (chars[i] == prev) {
                    count++;
                } else {
                    if (count > 1) {
                        sb.append(prev);
                        sb.append(count);
```

```
                    } else if (count == 1) {
                        sb.append(prev);
                    }
                    prev = chars[i];
                    count = 1;
                }
            }
            sb.append(prev);
            if (count > 1) {
                sb.append(count);
            }
            int i = 0;
            for (char c : sb.toString().toCharArray()) {
                chars[i++] = c;
            }
            return sb.length();
        }
    }
}

// ============================================================
// FILE 421: _444.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

/**
 * 444. Sequence Reconstruction
 *
 * Check whether the original sequence org can be uniquely reconstructed from the
sequences in seqs.
 * The org sequence is a permutation of the integers from 1 to n, with 1 ≤ n ≤ 104.
 * Reconstruction means building a shortest common supersequence of the sequences in seqs
 * (i.e., a shortest sequence so that all sequences in seqs are subsequences of it).
 * Determine whether there is only one sequence that can be reconstructed from seqs and
it is the org sequence.

 Example 1:
 Input:
 org: [1,2,3], seqs: [[1,2],[1,3]]

 Output:
 false

 Explanation:
 [1,2,3] is not the only one sequence that can be reconstructed, because [1,3,2] is also
```

```
a valid sequence that can be reconstructed.

 Example 2:
 Input:
 org: [1,2,3], seqs: [[1,2]]

 Output:
 false

 Explanation:
 The reconstructed sequence can only be [1,2].

 Example 3:
 Input:
 org: [1,2,3], seqs: [[1,2],[1,3],[2,3]]

 Output:
 true

 Explanation:
 The sequences [1,2], [1,3], and [2,3] can uniquely reconstruct the original sequence
[1,2,3].

 Example 4:
 Input:
 org: [4,1,5,2,6,3], seqs: [[5,2,6,3],[4,1,5,2]]

 Output:
 true
 */
public class _444 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/65948/java-solution-using-bfs-
topological-sort
         */
        public boolean sequenceReconstruction(int[] org, List<List<Integer>> seqs) {
            Map<Integer, Set<Integer>> map = new HashMap<>();
            Map<Integer, Integer> indegree = new HashMap<>();
            for (List<Integer> seq : seqs) {
                if (seq.size() == 1) {
                    if (!map.containsKey(seq.get(0))) {
                        map.put(seq.get(0), new HashSet<>());
                        indegree.put(seq.get(0), 0);
                    }
                } else {
                    for (int i = 0; i < seq.size() - 1; i++) {
                        if (!map.containsKey(seq.get(i))) {
                            map.put(seq.get(i), new HashSet<>());
                            indegree.put(seq.get(i), 0);
                        }

                        if (!map.containsKey(seq.get(i + 1))) {
                            map.put(seq.get(i + 1), new HashSet<>());
```

```
                            indegree.put(seq.get(i + 1), 0);
                        }

                        if (map.get(seq.get(i)).add(seq.get(i + 1))) {
                            indegree.put(seq.get(i + 1), indegree.get(seq.get(i + 1)) +
1);
                        }
                    }
                }
            }

            Queue<Integer> queue = new LinkedList<>();
            for (Integer key : indegree.keySet()) {
                if (indegree.get(key) == 0) {
                    queue.offer(key);
                }
            }

            int index = 0;
            while (!queue.isEmpty()) {
                int size = queue.size();
                if (size > 1) {
                    return false;
                }
                int curr = queue.poll();
                if (index == org.length || curr != org[index++]) {
                    return false;
                }
                for (int next : map.get(curr)) {
                    indegree.put(next, indegree.get(next) - 1);
                    if (indegree.get(next) == 0) {
                        queue.offer(next);
                    }
                }
            }
            return index == org.length && index == map.size();
        }
    }
}

// ==========================================================
// FILE 422: _445.java
// ==========================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Stack;

/**
 * 445. Add Two Numbers II
```

```
 *
 * You are given two non-empty linked lists representing two non-negative integers.
 * The most significant digit comes first and each of their nodes contain a single digit.
Add the two numbers and return it as a linked list.

 You may assume the two numbers do not contain any leading zero, except the number 0
itself.

 Follow up:
 What if you cannot modify the input lists? In other words, reversing the lists is not
allowed.

 Example:

 Input: (7 -> 2 -> 4 -> 3) + (5 -> 6 -> 4)
 Output: 7 -> 8 -> 0 -> 7
 */
public class _445 {

    public static class Solution1 {
        public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
            Deque<Integer> stack1 = popIntoStack(l1);
            Deque<Integer> stack2 = popIntoStack(l2);

            int sum = 0;
            ListNode list = new ListNode(0);
            while (!stack1.isEmpty() || !stack2.isEmpty()) {
                if (!stack1.isEmpty()) {
                    sum += stack1.removeFirst();
                }
                if (!stack2.isEmpty()) {
                    sum += stack2.removeFirst();
                }
                list.val = sum % 10;
                ListNode head = new ListNode(sum / 10);
                head.next = list;
                list = head;
                sum /= 10;
            }
            return list.val == 0 ? list.next : list;
        }

        private Deque<Integer> popIntoStack(ListNode head) {
            ListNode tmp = head;
            Deque<Integer> stack = new ArrayDeque<>();
            while (tmp != null) {
                stack.push(tmp.val);
                tmp = tmp.next;
            }
            return stack;
        }
    }
```

```
public static class Solution2 {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        Stack<Integer> stack1 = popOntoStack(l1);
        Stack<Integer> stack2 = popOntoStack(l2);
        Stack<Integer> resultStack = add(stack1, stack2);
        return buildResult(resultStack);
    }

    private ListNode buildResult(Stack<Integer> stack) {
        ListNode prev = new ListNode(-1);
        ListNode head = new ListNode(stack.pop());
        prev.next = head;
        while (!stack.isEmpty()) {
            head.next = new ListNode(stack.pop());
            head = head.next;
        }
        return prev.next;
    }

    private Stack<Integer> add(Stack<Integer> stack1, Stack<Integer> stack2) {
        Stack<Integer> res = new Stack<>();
        int carry = 0;
        while (!stack1.isEmpty() || !stack2.isEmpty()) {
            if (!stack1.isEmpty()) {
                carry += stack1.pop();
            }
            if (!stack2.isEmpty()) {
                carry += stack2.pop();
            }
            int value = carry;
            if (carry > 9) {
                value = carry % 10;
                carry = 1;
            } else {
                carry = 0;
            }
            res.push(value);
        }
        if (carry != 0) {
            res.add(carry);
        }
        return res;
    }

    private Stack<Integer> popOntoStack(ListNode head) {
        ListNode temp = head;
        Stack<Integer> stack = new Stack<>();
        while (temp != null) {
            stack.push(temp.val);
            temp = temp.next;
        }
        return stack;
    }
}
```

```
}

// ============================================================
// FILE 423: _446.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 446. Arithmetic Slices II - Subsequence
 *
 * A sequence of numbers is called arithmetic if it consists of at least three elements
and if the difference between any two consecutive elements is the same.

 For example, these are arithmetic sequences:

 1, 3, 5, 7, 9
 7, 7, 7, 7
 3, -1, -5, -9

 The following sequence is not arithmetic.

 1, 1, 2, 5, 7

 A zero-indexed array A consisting of N numbers is given. A subsequence slice of that
array is any sequence of integers (P0, P1, ..., Pk) such that 0 ≤ P0 < P1 < ... < Pk < N.

 A subsequence slice (P0, P1, ..., Pk) of array A is called arithmetic if the sequence
A[P0], A[P1], ..., A[Pk-1], A[Pk] is arithmetic. In particular, this means that k ≥ 2.

 The function should return the number of arithmetic subsequence slices in the array A.

 The input contains N integers. Every integer is in the range of -231 and 231-1 and 0 ≤ N
≤ 1000. The output is guaranteed to be less than 231-1.


 Example:

 Input: [2, 4, 6, 8, 10]

 Output: 7

 Explanation:
 All arithmetic subsequence slices are:
 [2,4,6]
 [4,6,8]
 [6,8,10]
 [2,4,6,8]
 [4,6,8,10]
 [2,4,6,8,10]
 [2,6,10]
```

```
 */
public class _446 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/67413/detailed-explanation-for-
java-o-n-2-solution
         */
        public int numberOfArithmeticSlices(int[] A) {
            int res = 0;
            Map<Integer, Integer>[] map = new Map[A.length];

            for (int i = 0; i < A.length; i++) {
                map[i] = new HashMap<>(i);

                for (int j = 0; j < i; j++) {
                    long diff = (long) A[i] - A[j];
                    if (diff <= Integer.MIN_VALUE || diff > Integer.MAX_VALUE) {
                        continue;
                    }

                    int d = (int) diff;
                    int c1 = map[i].getOrDefault(d, 0);
                    int c2 = map[j].getOrDefault(d, 0);
                    res += c2;
                    map[i].put(d, c1 + c2 + 1);
                }
            }

            return res;
        }
    }
}

// ============================================================
// FILE 424: _447.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 447. Number of Boomerangs
 *
 * Given n points in the plane that are all pairwise distinct,
 * a "boomerang" is a tuple of points (i, j, k) such that the distance
 * between i and j equals the distance between i and k (the order of the tuple matters).
 *
 * Find the number of boomerangs.
 * You may assume that n will be at most 500 and coordinates of points are all in the
range [-10000, 10000] (inclusive).

 Example:
```

```
 Input:
 [[0,0],[1,0],[2,0]]

 Output:
 2

 Explanation:
 The two boomerangs are [[1,0],[0,0],[2,0]] and [[1,0],[2,0],[0,0]]*/
public class _447 {

    public static class Solution1 {
        /**
         * Looked at these two posts: https://discuss.leetcode.com/topic/66587/clean-java-solution-o-n-2-166ms and
         * https://discuss.leetcode.com/topic/66521/share-my-straightforward-solution-with-hashmap-o-n-2, basically,
         * have a HashMap, key is the distance, value is the number of points that are this distance apart to this point.
         * Note: we clear up this map every time after we traverse one point with the rest of the other points.
         * <p>
         * Time complexity: O(n^2)
         * Space complexity: O(n)
         */

        public int numberOfBoomerangs(int[][] points) {
            int result = 0;
            if (points == null || points.length == 0 || points[0].length == 0) {
                return result;
            }
            int totalPts = points.length;
            Map<Long, Integer> map = new HashMap();
            for (int i = 0; i < totalPts; i++) {
                for (int j = 0; j < totalPts; j++) {
                    if (i == j) {
                        continue;
                    }
                    long d = calcDistance(points[i], points[j]);
                    map.put(d, map.getOrDefault(d, 0) + 1);
                }

                for (int val : map.values()) {
                    result += val * (val - 1);
                }
                map.clear();
            }
            return result;
        }

        private long calcDistance(int[] p1, int[] p2) {
            long x = p2[0] - p1[0];
            long y = p2[1] - p1[1];
            return x * x + y * y;
        }
```

```
    }

}

// ============================================================
// FILE 425: _448.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 448. Find All Numbers Disappeared in an Array
 *
 * Given an array of integers where 1 ≤ a[i] ≤ n (n = size of array), some elements
appear twice and others appear once.
 * Find all the elements of [1, n] inclusive that do not appear in this array.
 * Could you do it without extra space and in O(n) runtime? You may assume the returned
list does not count as extra space.

 Example:

 Input:
 [4,3,2,7,8,2,3,1]

 Output:
 [5,6]

 */
public class _448 {

    public static class Solution1 {
        /**
         * O(n) space
         * O(n) time
         */
        public List<Integer> findDisappearedNumbers(int[] nums) {

            int max = Integer.MIN_VALUE;
            for (int i : nums) {
                max = Math.max(max, i);
            }
            max = Math.max(max, nums.length);

            Map<Integer, Integer> map = new HashMap();
            for (int i = 1; i <= max; i++) {
                map.put(i, 0);
            }

            for (int i : nums) {
```

```
                if (map.get(i) == 0) {
                    map.put(i, 1);
                } else {
                    map.put(i, map.get(i) + 1);
                }
            }

            List<Integer> result = new ArrayList();
            for (int i : map.keySet()) {
                if (map.get(i) == 0) {
                    result.add(i);
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * O(1) space
         * O(n) time
         */
        public List<Integer> findDisappearedNumbers(int[] nums) {

            for (int i = 0; i < nums.length; i++) {
                int val = Math.abs(nums[i]) - 1;
                if (nums[val] > 0) {
                    nums[val] = -nums[val];
                }
            }

            List<Integer> result = new ArrayList();
            for (int i = 0; i < nums.length; i++) {
                if (nums[i] > 0) {
                    result.add(i + 1);
                }
            }
            return result;
        }
    }

}

// ============================================================
// FILE 426: _449.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;
```

```
/**
 * 449. Serialize and Deserialize BST
 *
 * Serialization is the process of converting a data structure or
 * object into a sequence of bits so that it can be stored in a file or memory buffer,
 * or transmitted across a network connection link to be reconstructed later in the same
or another computer environment.
 *
 * Design an algorithm to serialize and deserialize a binary search tree.
 * There is no restriction on how your serialization/deserialization algorithm should
work.
 * You just need to ensure that a binary search tree can be serialized to a string and
this
 * string can be deserialized to the original tree structure.
 * The encoded string should be as compact as possible.

 Note: Do not use class member/global/static variables to store states. Your serialize
and deserialize algorithms should be stateless.
 */

public class _449 {

    public static class Solution1 {
        /**
         * Preorder
         * Reference: https://discuss.leetcode.com/topic/97922/pre-or-post-order-with-
only-keeping-one-bound-beat-98-and-95
         */

        // Encodes a tree to a single string.
        public String serialize(TreeNode root) {
            if (root == null) {
                return null;
            }
            StringBuilder stringBuilder = new StringBuilder();
            return serialize(root, stringBuilder);
        }

        private String serialize(TreeNode root, StringBuilder stringBuilder) {
            if (root == null) {
                return null;
            }
            stringBuilder.append(root.val).append(" ");
            serialize(root.left, stringBuilder);
            serialize(root.right, stringBuilder);
            return stringBuilder.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.length() == 0) {
                return null;
            }
            String[] values = data.split(" ");
```

```
            int[] index = new int[]{0};/**TODO: Why must use an int array, instead of
just an int?*/
            return deserialize(values, index, Integer.MAX_VALUE);
        }

        private TreeNode deserialize(String[] values, int[] index, int maxValue) {
            if (index[0] >= values.length || Integer.valueOf(values[index[0]]) >=
maxValue) {
                return null;
            }
            TreeNode root = new TreeNode(Integer.valueOf(values[index[0]++]));
            root.left = deserialize(values, index, root.val);
            root.right = deserialize(values, index, maxValue);
            return root;
        }
    }

    public static class Solution2 {
        /**
         * Postorder
         * Reference: https://discuss.leetcode.com/topic/97922/pre-or-post-order-with-
only-keeping-one-bound-beat-98-and-95
         */

        public String serialize(TreeNode root) {
            if (root == null) {
                return null;
            }
            StringBuilder sb = new StringBuilder();
            return serialize(root, sb);
        }

        private String serialize(TreeNode root, StringBuilder sb) {
            if (root == null) {
                return null;
            }
            serialize(root.left, sb);
            serialize(root.right, sb);
            sb.append(root.val).append(" ");
            return sb.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.length() == 0) {
                return null;
            }
            String[] values = data.split(" ");
            int[] index = new int[]{values.length - 1};/**TODO: This is not just one
element any more like in the preorder solution above*/
            return deserialize(values, index, Integer.MIN_VALUE);
        }

        private TreeNode deserialize(String[] values, int[] index, int minValue) {
```

```
            if (index[0] < 0 || Integer.valueOf(values[index[0]]) < minValue) {
                return null;
            }
            TreeNode root = new TreeNode(Integer.valueOf(values[index[0]--]));
            root.right = deserialize(values, index, root.val);
            root.left = deserialize(values, index, minValue);
            return root;
        }
    }

    public static class Solution3 {
        /**
         * This is a generic solution that applies to both BT and BST. And also the
easiest to follow.
         */

        // Encodes a tree to a single string.
        public String serialize(TreeNode root) {
            Queue<TreeNode> queue = new LinkedList<>();
            StringBuilder stringBuilder = new StringBuilder();
            if (root == null) {
                return stringBuilder.toString();
            }
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (curr == null) {
                        stringBuilder.append("# ");
                    } else {
                        stringBuilder.append(curr.val + " ");
                        queue.offer(curr.left);
                        queue.offer(curr.right);
                    }
                }
            }
            return stringBuilder.toString();
        }

        // Decodes your encoded data to tree.
        public TreeNode deserialize(String data) {
            if (data == null || data.length() == 0) {
                return null;
            }
            String[] nodes = data.split(" ");
            TreeNode root = new TreeNode(Integer.valueOf(nodes[0]));
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            for (int i = 1; i < nodes.length; i++) {
                TreeNode curr = queue.poll();
                if (!nodes[i].equals("#")) {
                    curr.left = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.left);
```

```
                }
                if (!nodes[++i].equals("#")) {
                    curr.right = new TreeNode(Integer.valueOf(nodes[i]));
                    queue.offer(curr.right);
                }
            }
            return root;
        }
    }
}

// ============================================================
// FILE 427: _450.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 450. Delete Node in a BST
 *
 * Given a root node reference of a BST and a key, delete the node with the given key in
the BST.
 * Return the root node reference (possibly updated) of the BST.

 Basically, the deletion can be divided into two stages:

 Search for a node to remove.
 If the node is found, delete the node.
 Note: Time complexity should be O(height of tree).

 Example:

 root = [5,3,6,2,4,null,7]
 key = 3

     5
    / \
   3   6
  / \   \
 2  4   7

 Given key to delete is 3. So we find the node with value 3 and delete it.

 One valid answer is [5,4,6,2,null,null,7], shown in the following BST.

     5
    / \
   4   6
  /     \
 2       7

 Another valid answer is [5,2,6,null,4,null,7].
```

```
     5
    / \
   2   6
   \   \
   4   7
 */
public class _450 {
    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/65792/recursive-easy-to-understand-java-solution
         * Steps:
         * 1. Recursively find the node that has the same value as the key, while setting the left/right nodes equal to the returned subtree
         * 2. Once the node is found, have to handle the below 4 cases
         * a. node doesn't have left or right - return null
         * b. node only has left subtree- return the left subtree
         * c. node only has right subtree- return the right subtree
         * d. node has both left and right - find the minimum value in the right subtree, set that value to the currently found node, then recursively delete the minimum value in the right subtree
         */
        public TreeNode deleteNode(TreeNode root, int key) {
            if (root == null) {
                return root;
            }
            if (root.val > key) {
                root.left = deleteNode(root.left, key);
            } else if (root.val < key) {
                root.right = deleteNode(root.right, key);
            } else {
                if (root.left == null) {
                    return root.right;
                } else if (root.right == null) {
                    return root.left;
                }

                TreeNode minNode = getMin(root.right);
                root.val = minNode.val;
                root.right = deleteNode(root.right, root.val);
            }
            return root;
        }

        private TreeNode getMin(TreeNode node) {
            while (node.left != null) {
                node = node.left;
            }
            return node;
        }
    }
```

```
}

// ============================================================
// FILE 428: _451.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 451. Sort Characters By Frequency
 *
 * Given a string, sort it in decreasing order based on the frequency of characters.

 Example 1:

 Input:
 "tree"

 Output:
 "eert"

 Explanation:
 'e' appears twice while 'r' and 't' both appear once.
 So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

 Example 2:

 Input:
 "cccaaa"

 Output:
 "cccaaa"

 Explanation:
 Both 'c' and 'a' appear three times, so "aaaccc" is also a valid answer.
 Note that "cacaca" is incorrect, as the same characters must be together.

 Example 3:

 Input:
 "Aabb"

 Output:
 "bbAa"

 Explanation:
 "bbaA" is also a valid answer, but "Aabb" is incorrect.
 Note that 'A' and 'a' are treated as two different characters.
```

```
 */
public class _451 {

    public static class Solution1 {
        public String frequencySort(String s) {
            Map<Character, Integer> map = new HashMap();
            for (char c : s.toCharArray()) {
                map.put(c, map.getOrDefault(c, 0) + 1);
            }
            List<Map.Entry<Character, Integer>> list = new ArrayList<>(map.entrySet());
            Collections.sort(list, (o1, o2) -> (o2.getValue() - o1.getValue()));
            StringBuilder stringBuilder = new StringBuilder();
            for (Map.Entry<Character, Integer> entry : list) {
                for (int i = 0; i < entry.getValue(); i++) {
                    stringBuilder.append(entry.getKey());
                }
            }
            return stringBuilder.toString();
        }
    }
}

// ============================================================
// FILE 429: _452.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 *452. Minimum Number of Arrows to Burst Balloons
 *
 * There are a number of spherical balloons spread in two-dimensional space.
 * For each balloon, provided input is the start and end coordinates of the horizontal
diameter.
 * Since it's horizontal, y-coordinates don't matter and hence the x-coordinates of start
and end of the diameter suffice.
 * Start is always smaller than end. There will be at most 104 balloons.
 *
 * An arrow can be shot up exactly vertically from different points along the x-axis.
 * A balloon with xstart and xend bursts by an arrow shot at x if xstart ≤ x ≤ xend.
 * There is no limit to the number of arrows that can be shot.
 * An arrow once shot keeps travelling up infinitely.
 * The problem is to find the minimum number of arrows that must be shot to burst all
balloons.

 Example:

 Input:
 [[10,16], [2,8], [1,6], [7,12]]

 Output:
```

```
 2

 Explanation:
 One way is to shoot one arrow for example at x = 6 (bursting the balloons [2,8] and
[1,6]) and another arrow at x = 11 (bursting the other two balloons).
 */
public class _452 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/66579/java-greedy-soution/6
         */
        public int findMinArrowShots(int[][] points) {
            if (points == null || points.length == 0) {
                return 0;
            }
            // sort points based on their end point.
            Arrays.sort(points, (p1, p2) -> Integer.compare(p1[1], p2[1]));
            int currentEnd = points[0][1];
            int count = 1;
            for (int[] p : points) {
                // if the point starts after currentEnd, it means this balloons not been
bursted. Then we shot the balloon in its end point. Otherwise, means this balloon has
been bursted, then ignore it.
                if (p[0] > currentEnd) {
                    count++;
                    currentEnd = p[1];
                } else {
                    continue;
                }
            }
            return count;
        }
    }
}

// ============================================================
// FILE 430: _453.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 453. Minimum Moves to Equal Array Elements
 *
 * Given a non-empty integer array of size n,
 * find the minimum number of moves required to make all array elements equal, where a
move is incrementing n - 1 elements by 1.

 Example:

 Input:
 [1,2,3]
```

```
 Output:
 3

 Explanation:
 Only three moves are needed (remember each move increments two elements):

 [1,2,3]  =>  [2,3,3]  =>  [3,4,3]  =>  [4,4,4]*/

public class _453 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/66557/java-o-n-solution-short
         * i.e. Add 1 to n-1 elements basically equals to subtracting 1 from one element.
So the easiest way
         * to make all elements in this array equal is to make all of them equal to the
minimum element.
         */
        public int minMoves(int[] nums) {
            if (nums.length == 0) {
                return 0;
            }
            int min = nums[0];
            for (int n : nums) {
                min = Math.min(min, n);
            }
            int res = 0;
            for (int n : nums) {
                res += n - min;
            }
            return res;
        }
    }

}

// ============================================================
// FILE 431: _454.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 454. 4Sum II
 *
 * Given four lists A, B, C, D of integer values,
 * compute how many tuples (i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is
zero.
 * To make problem a bit easier, all A, B, C, D have same length of N where 0 ≤ N ≤ 500.
 * All integers are in the range of -228 to 228 - 1 and the result is guaranteed to be at
most 231 - 1.
```

```
 Example:

 Input:
 A = [ 1, 2]
 B = [-2,-1]
 C = [-1, 2]
 D = [ 0, 2]

 Output:
 2

 Explanation:
 The two tuples are:
 1. (0, 0, 0, 1) -> A[0] + B[0] + C[0] + D[1] = 1 + (-2) + (-1) + 2 = 0
 2. (1, 1, 0, 0) -> A[1] + B[1] + C[0] + D[0] = 2 + (-1) + (-1) + 0 = 0*/

public class _454 {

    public static class Solution1 {
        public int fourSumCount(int[] A, int[] B, int[] C, int[] D) {
            Map<Integer, Integer> map = new HashMap();
            int result = 0;
            int len = A.length;
            for (int i = 0; i < len; i++) {
                for (int j = 0; j < len; j++) {
                    int sum = A[i] + B[j];
                    if (map.containsKey(sum)) {
                        map.put(sum, map.get(sum) + 1);
                    } else {
                        map.put(sum, 1);
                    }
                }
            }

            for (int i = 0; i < len; i++) {
                for (int j = 0; j < len; j++) {
                    int sum = -(C[i] + D[j]);
                    if (map.containsKey(sum)) {
                        result += map.get(sum);
                    }
                }
            }

            return result;
        }
    }
}

// ============================================================
// FILE 432: _455.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.Arrays;

/**
 * 455. Assign Cookies
 *
 * Assume you are an awesome parent and want to give your children some cookies.
 * But, you should give each child at most one cookie.
 * Each child i has a greed factor gi, which is the minimum size of a cookie
 * that the child will be content with;
 * and each cookie j has a size sj. If sj >= gi, we can assign the cookie j to the child
i,
 * and the child i will be content. Your goal is to maximize the number of your content
children and output the maximum number.

 Note:
 You may assume the greed factor is always positive.
 You cannot assign more than one cookie to one child.

 Example 1:
 Input: [1,2,3], [1,1]
 Output: 1
 Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1,
2, 3.
 And even though you have 2 cookies, since their size is both 1, you could only make the
child whose greed factor is 1 content.
 You need to output 1.

 Example 2:
 Input: [1,2], [1,2,3]
 Output: 2
 Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1,
2.
 You have 3 cookies and their sizes are big enough to gratify all of the children,
 You need to output 2.*/

public class _455 {

    public static class Solution1 {
        public int findContentChildren(int[] g, int[] s) {
            Arrays.sort(g);
            Arrays.sort(s);

            int result = 0;
            for (int i = 0, j = 0; i < g.length && j < s.length; ) {
                if (s[j] >= g[i]) {
                    result++;
                    i++;
                }
                j++;
            }
            return result;
        }
    }
```

```
}

// ============================================================
// FILE 433: _456.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;
/**
 * 456. 132 Pattern
 *
 * Given a sequence of n integers a1, a2, ..., an,
 * a 132 pattern is a subsequence ai, aj, ak such that i < j < k and ai < ak < aj.
 * Design an algorithm that takes a list of n numbers as input and checks whether there
is a 132 pattern in the list.

 Note: n will be less than 15,000.

 Example 1:
 Input: [1, 2, 3, 4]

 Output: False

 Explanation: There is no 132 pattern in the sequence.
 Example 2:
 Input: [3, 1, 4, 2]

 Output: True

 Explanation: There is a 132 pattern in the sequence: [1, 4, 2].
 Example 3:
 Input: [-1, 3, 2, 0]

 Output: True

 Explanation: There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and
[-1, 2, 0].
 */

public class _456 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/67881/single-pass-c-o-n-space-and-
time-solution-8-lines-with-detailed-explanation
         * It scans only once, this is the power of using correct data structure.
         * It goes from the right to the left.
         * It keeps pushing elements into the stack,
         * but it also keeps poping elements out of the stack as long as the current
element is bigger than this number.
         */
        public boolean find132pattern(int[] nums) {
```

```
            Deque<Integer> stack = new LinkedList<>();

            int s3 = Integer.MIN_VALUE;
            for (int i = nums.length - 1; i >= 0; i--) {
                if (nums[i] < s3) {
                    return true;
                } else {
                    while (!stack.isEmpty() && nums[i] > stack.peek()) {
                        s3 = Math.max(s3, stack.pop());
                    }
                }
                stack.push(nums[i]);
            }

            return false;
        }
    }

}

// ============================================================
// FILE 434: _457.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 457. Circular Array Loop
 *
 * You are given an array of positive and negative integers.
 * If a number n at an index is positive, then move forward n steps.
 * Conversely, if it's negative (-n), move backward n steps.
 *
 * Assume the first element of the array is forward next to the last element,
 * and the last element is backward next to the first element.
 * Determine if there is a loop in this array.
 * A loop starts and ends at a particular index with more than 1 element along the loop.
The loop must be "forward" or "backward'.
 *
 * Example 1: Given the array [2, -1, 1, 2, 2], there is a loop, from index 0 -> 2 -> 3 -
> 0.
 * Example 2: Given the array [-1, 2], there is no loop.
 *
 * Note: The given array is guaranteed to contain no element "0".
 *
 * Can you do it in O(n) time complexity and O(1) space complexity?
 */
public class _457 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/66894/java-slow-fast-pointer-
solution
         */
        public boolean circularArrayLoop(int[] nums) {
```

```
            int n = nums.length;
            for (int i = 0; i < n; i++) {
                if (nums[i] == 0) {
                    continue;
                }
                // slow/fast pointer
                int j = i;
                int k = getIndex(i, nums);
                while (nums[k] * nums[i] > 0 && nums[getIndex(k, nums)] * nums[i] > 0) {
                    if (j == k) {
                        // check for loop with only one element
                        if (j == getIndex(j, nums)) {
                            break;
                        }
                        return true;
                    }
                    j = getIndex(j, nums);
                    k = getIndex(getIndex(k, nums), nums);
                }
                // loop not found, set all element along the way to 0
                j = i;
                int val = nums[i];
                while (nums[j] * val > 0) {
                    int next = getIndex(j, nums);
                    nums[j] = 0;
                    j = next;
                }
            }
            return false;
        }

        public int getIndex(int i, int[] nums) {
            int n = nums.length;
            return i + nums[i] >= 0 ? (i + nums[i]) % n : n + ((i + nums[i]) % n);
        }
    }
}

// ============================================================
// FILE 435: _458.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 458. Poor Pigs
 *
 * There are 1000 buckets, one and only one of them contains poison,
 * the rest are filled with water.
 * They all look the same.
 * If a pig drinks that poison it will die within 15 minutes.
 * What is the minimum amount of pigs you need to figure out which bucket contains the
poison within one hour.
 * Answer this question, and write an algorithm for the follow-up general case.
```

```
 Follow-up:
 If there are n buckets and a pig drinking poison will die within m minutes,
 how many pigs (x) you need to figure out the "poison" bucket within p minutes? There is
exact one bucket with poison.

 */
public class _458 {

    public static class Solution1 {
        public int poorPigs(int buckets, int minutesToDie, int minutesToTest) {
            if (buckets-- == 1) {
                return 0;
            }
            int base = minutesToTest / minutesToDie + 1;
            int count = 0;
            while (buckets > 0) {
                buckets /= base;
                count++;
            }
            return count;
        }
    }

}

// ============================================================
// FILE 436: _459.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 459. Repeated Substring Pattern
 * Given a non-empty string check if it can be constructed by taking a substring of it
and appending multiple copies of the substring together.
 * You may assume the given string consists of lowercase English letters only and its
length will not exceed 10000.

 Example 1:
 Input: "abab"

 Output: True

 Explanation: It's the substring "ab" twice.
 Example 2:
 Input: "aba"

 Output: False
 Example 3:
 Input: "abcabcabcabc"

 Output: True
```

```
 Explanation: It's the substring "abc" four times. (And the substring "abcabc" twice.)
 */

public class _459 {

    public static class Solution1 {
        public boolean repeatedSubstringPattern(String s) {
            int len = s.length();
            for (int i = len / 2; i >= 1; i--) {
                String pattern = s.substring(0, i);
                if (len % i == 0) {
                    String formedString = formStringByCopying(pattern, len / i);
                    if (formedString.equals(s)) {
                        return true;
                    }
                }
            }
            return false;
        }

        private String formStringByCopying(String pattern, int k) {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < k; i++) {
                sb.append(pattern);
            }
            return sb.toString();
        }
    }

    public static class Solution2 {
        /**
         * credit: https://discuss.leetcode.com/topic/68089/repeated-substring-pattern-
simple-java-solution-using-kmp
         */
        public static boolean repeatedSubstringPattern(String str) {
            //build the KMP pattern.
            int n = str.length();
            int cur = 0;
            int j = 1;
            int[] pattern = new int[n];
            pattern[0] = 0;

            while (j < n) {
                if (str.charAt(cur) == str.charAt(j)) {
                    pattern[j++] = ++cur;
                } else {
                    if (cur == 0) {
                        pattern[j++] = 0;
                    } else {
                        cur = pattern[cur - 1];//start from beginning of current matching
pattern.
                    }
                }
            }
```

```
            return (pattern[n - 1] > 0 && n % (n - pattern[n - 1]) == 0);
        }
    }

    public static class Solution3 {
        public static boolean repeatedSubstringPattern(String str) {
            String s = str + str;
            return s.substring(1, s.length() - 1).contains(str);
        }
    }
}

// ============================================================
// FILE 437: _460.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.LinkedHashSet;

/**460. LFU Cache

 * Design and implement a data structure for Least Frequently Used (LFU) cache.
 * It should support the following operations: get and put.

 get(key) - Get the value (will always be positive) of the key if the key exists in the
cache, otherwise return -1.
 put(key, value) - Set or insert the value if the key is not already present. When the
cache reaches its capacity,
 it should invalidate the least frequently used item before inserting a new item.
 For the purpose of this problem, when there is a tie (i.e., two or more keys that have
the same frequency),
 the least recently used key would be evicted.

 Follow up:
 Could you do both operations in O(1) time complexity?

 Example:

 LFUCache cache = new LFUCache( 2 /* capacity  );

        cache.put(1, 1);
        cache.put(2, 2);
        cache.get(1);       // returns 1
        cache.put(3, 3);    // evicts key 2
        cache.get(2);       // returns -1 (not found)
        cache.get(3);       // returns 3.
        cache.put(4, 4);    // evicts key 1.
        cache.get(1);       // returns -1 (not found)
        cache.get(3);       // returns 3
        cache.get(4);       // returns 4
```

```
 */

public class _460 {
    public static class Solution1 {
        /**
         * Wikipedia: The simplest method to employ an LFU algorithm is to assign a
counter to every
         * block that is loaded into the cache. Each time a reference is made to that
block the counter
         * is increased by one. When the cache reaches capacity and has a new block
waiting to be
         * inserted the system will search for the block with the lowest counter and
remove it from the
         * cache.
         * Policy to handle frequency ties: based on timestamp, the entries that get set
into cache earlier will be evicted first.
         */

        public static class LFUCache {
            /**
             * credit: https://discuss.leetcode.com/topic/69737/java-o-1-very-easy-
solution-using-3-hashmaps-and-linkedhashset/2
             */

            HashMap<Integer, Integer> keyToValue;
            /**
             * key is the key, value is the value
             */
            HashMap<Integer, Integer> keyToCount;
            /**
             * key is the key, value if the count of the key/value pair
             */
            HashMap<Integer, LinkedHashSet<Integer>> countToLRUKeys;
            /**
             * key is count, value is a set of keys that have the same key, but keeps
insertion order
             */
            int cap;
            int minimumCount;

            public LFUCache(int capacity) {
                this.minimumCount = -1;
                this.cap = capacity;
                this.keyToValue = new HashMap<>();
                this.keyToCount = new HashMap<>();
                this.countToLRUKeys = new HashMap<>();
                this.countToLRUKeys.put(1, new LinkedHashSet<>());
            }

            public int get(int key) {
                if (!keyToValue.containsKey(key)) {
                    return -1;
                }
                int count = keyToCount.get(key);
```

```
                keyToCount.put(key, count + 1);
                countToLRUKeys.get(count).remove(key);

                if (count == minimumCount && countToLRUKeys.get(count).size() == 0) {
                    /**This means this key's count equals to current minimumCount
                     * AND
                     * this count doesn't have any entries in the cache.
                     * So, we'll increment minimumCount by 1 to get the next LFU cache
entry
                     * when we need to evict.*/
                    minimumCount++;
                }

                if (!countToLRUKeys.containsKey(count + 1)) {
                    countToLRUKeys.put(count + 1, new LinkedHashSet<>());
                }
                countToLRUKeys.get(count + 1).add(key);

                return keyToValue.get(key);
            }

            public void put(int key, int value) {
                if (cap <= 0) {
                    return;
                }

                if (keyToValue.containsKey(key)) {
                    /**If the key is already in the cache, we can simply overwrite this
entry;
                     * then call get(key) which will do the update work.*/
                    keyToValue.put(key, value);
                    get(key);
                    return;
                }

                /**If the key is not in the cache, we'll check the size first, evict the
LFU entry first,
                 * then insert this one into cache.*/
                if (keyToValue.size() >= cap) {
                    int evit = countToLRUKeys.get(minimumCount).iterator().next();
                    countToLRUKeys.get(minimumCount).remove(evit);
                    keyToValue.remove(evit);
                }
                keyToValue.put(key, value);
                keyToCount.put(key, 1);
                countToLRUKeys.get(1).add(key);/**Because we put this key/value into
cache for the first time, so its count is 1*/
                minimumCount = 1;/**For the same above reason, minimumCount is of course
1*/
            }
        }
    }

}
```

```
// ============================================================
// FILE 438: _461.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 461. Hamming Distance
 *
 * The Hamming distance between two integers is the number of positions at which the
corresponding bits are different.

 Given two integers x and y, calculate the Hamming distance.

 Note:
 0 ≤ x, y < 231.

 Example:

 Input: x = 1, y = 4

 Output: 2

 Explanation:
 1   (0 0 0 1)
 4   (0 1 0 0)
 ↑   ↑

 The above arrows point to positions where the corresponding bits are different.
 */
public class _461 {
        public static class Solution1 {
                public int hammingDistance(int x, int y) {
                        int n = x ^ y;
                        int count = 0;
                        while (n != 0) {
                                count++;
                                n &= (n - 1);
                        }
                        return count;
                }
        }
}

// ============================================================
// FILE 439: _462.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
```

```
 * 462. Minimum Moves to Equal Array Elements II
 *
 * Given a non-empty integer array,
 * find the minimum number of moves required to make all array elements equal,
 * where a move is incrementing a selected element by 1 or decrementing a selected
element by 1.
 *
 * You may assume the array's length is at most 10,000.

 Example:

 Input:
 [1,2,3]

 Output:
 2

 Explanation:
 Only two moves are needed (remember each move increments or decrements one element):

 [1,2,3]  =>  [2,2,3]  =>  [2,2,2]
 */

public class _462 {

    public static class Solution1 {
        public int minMoves2(int[] nums) {
            /**sort this array, find the median of this array as the pivot*/
            Arrays.sort(nums);
            int result = 0;
            int result1 = 0;
            if (nums.length % 2 != 0) {
                int median = nums[nums.length / 2];
                for (int n : nums) {
                    result += Math.abs(n - median);
                }
                return result;
            } else {
                int median1 = nums[nums.length / 2];
                for (int n : nums) {
                    result1 += Math.abs(n - median1);
                }
                int result2 = 0;
                int median2 = nums[nums.length / 2 - 1];
                for (int n : nums) {
                    result2 += Math.abs(n - median2);
                }
                result1 = Math.min(result1, result2);
                return result1;
            }
        }
    }

}
```

```
// ============================================================
// FILE 440: _463.java
// ============================================================

package com.fishercoder.solutions;
/**
 * 463. Island Perimeter
 *
 * You are given a map in form of a two-dimensional integer grid where 1 represents land
and 0 represents water.
 * Grid cells are connected horizontally/vertically (not diagonally).
 * The grid is completely surrounded by water, and there is exactly one island (i.e., one
or more connected land cells).
 * The island doesn't have "lakes" (water inside that isn't connected to the water around
the island).
 * One cell is a square with side length 1.
 * The grid is rectangular, width and height don't exceed 100. Determine the perimeter of
the island.

 Example:

 [[0,1,0,0],
 [1,1,1,0],
 [0,1,0,0],
 [1,1,0,0]]

 Answer: 16
 Explanation: The perimeter is the 16 yellow stripes in the image below:
 */
public class _463 {

    public static class Solution1 {
        /**
         * Inspired by this post: https://discuss.leetcode.com/topic/68983/java-9-line-
solution-add-4-for-each-land-and-remove-2-for-each-internal-edge
         * 1. we increment the count by 4 whenever we encounter an island
         * 2. also, we check in two directions: island's left and island's top, we only
check these two directions,
         * see if this island has any island neighbors, if so, we'll deduct two from it.
         */
        public int islandPerimeter(int[][] grid) {
            int count = 0;
            for (int i = 0; i < grid.length; i++) {
                for (int j = 0; j < grid[0].length; j++) {
                    if (grid[i][j] == 1) {
                        count += 4;
                        if (i > 0 && grid[i - 1][j] == 1) {
                            count -= 2;
                        }
                        if (j > 0 && grid[i][j - 1] == 1) {
                            count -= 2;
                        }
                    }
```

```
                }
            }
            return count;
        }
    }
}

// ============================================================
// FILE 441: _464.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 464. Can I Win
 *
 * In the "100 game," two players take turns adding, to a running total, any integer from
1..10.
 * The player who first causes the running total to reach or exceed 100 wins.
 * What if we change the game so that players cannot re-use integers?
 * For example, two players might take turns drawing from a common pool of numbers of
1..15 without replacement until they reach a total >= 100.
 * Given an integer maxChoosableInteger and another integer desiredTotal, determine if
the first player to move can force a win, assuming both players play optimally.
 *You can always assume that maxChoosableInteger will not be larger than 20 and
desiredTotal will not be larger than 300.

 Example

 Input:
 maxChoosableInteger = 10
 desiredTotal = 11

 Output:
 false

 Explanation:
 No matter which integer the first player choose, the first player will lose.
 The first player can choose an integer from 1 up to 10.
 If the first player choose 1, the second player can only choose integers from 2 up to
10.
 The second player will win by choosing 10 and get a total = 11, which is >=
desiredTotal.
 Same with other integers chosen by the first player, the second player will always win.
 */
public class _464 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/68896/java-solution-using-hashmap-
with-detailed-explanation
         */
```

```
    Map<Integer, Boolean> map;
    boolean[] used;

    public boolean canIWin(int maxChoosableInteger, int desiredTotal) {
        int sum = (1 + maxChoosableInteger) * maxChoosableInteger / 2;
        if (sum < desiredTotal) {
            return false;
        }
        if (desiredTotal <= 0) {
            return true;
        }

        map = new HashMap();
        used = new boolean[maxChoosableInteger + 1];
        return helper(desiredTotal);
    }

    public boolean helper(int desiredTotal) {
        if (desiredTotal <= 0) {
            return false;
        }
        int key = format(used);
        if (!map.containsKey(key)) {
            // try every unchosen number as next step
            for (int i = 1; i < used.length; i++) {
                if (!used[i]) {
                    used[i] = true;
                    // check whether this lead to a win (i.e. the other player lose)
                    if (!helper(desiredTotal - i)) {
                        map.put(key, true);
                        used[i] = false;
                        return true;
                    }
                    used[i] = false;
                }
            }
            map.put(key, false);
        }
        return map.get(key);
    }

    // transfer boolean[] to an Integer
    public int format(boolean[] used) {
        int num = 0;
        for (boolean b : used) {
            num <<= 1;
            if (b) {
                num |= 1;
            }
        }
        return num;
    }
}
```

```
}

// ============================================================
// FILE 442: _465.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Stack;

/**
 * 465. Optimal Account Balancing
 *
 * A group of friends went on holiday and sometimes lent each other money.
 * For example, Alice paid for Bill's lunch for $10.
 * Then later Chris gave Alice $5 for a taxi ride.
 * We can model each transaction as a tuple (x, y, z) which means person x gave person y
$z.
 * Assuming Alice, Bill, and Chris are person 0, 1, and 2 respectively (0, 1, 2 are the
person's ID),
 * the transactions can be represented as [[0, 1, 10], [2, 0, 5]].

 Given a list of transactions between a group of people, return the minimum number of
transactions required to settle the debt.

 Note:

 A transaction will be given as a tuple (x, y, z). Note that x ? y and z > 0.
 Person's IDs may not be linear, e.g. we could have the persons 0, 1, 2 or we could also
have the persons 0, 2, 6.
 Example 1:

 Input:
 [[0,1,10], [2,0,5]]

 Output:
 2

 Explanation:
 Person #0 gave person #1 $10.
 Person #2 gave person #0 $5.

 Two transactions are needed. One way to settle the debt is person #1 pays person #0 and
#2 $5 each.
 Example 2:

 Input:
 [[0,1,10], [1,0,1], [1,2,5], [2,0,5]]
```

```
 Output:
 1

 Explanation:
 Person #0 gave person #1 $10.
 Person #1 gave person #0 $1.
 Person #1 gave person #2 $5.
 Person #2 gave person #0 $5.

 Therefore, person #1 only need to give person #0 $4, and all debt is settled.
 */
public class _465 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/68948/easy-java-solution-with-
explanation
         */
        public int minTransfers(int[][] transactions) {
            if (transactions == null || transactions.length == 0) {
                return 0;
            }
            Map<Integer, Integer> acc = new HashMap<>();
            for (int i = 0; i < transactions.length; i++) {
                int id1 = transactions[i][0];
                int id2 = transactions[i][1];
                int m = transactions[i][2];
                acc.put(id1, acc.getOrDefault(id1, 0) - m);
                acc.put(id2, acc.getOrDefault(id2, 0) + m);
            }
            List<Integer> negs = new ArrayList<>();
            List<Integer> poss = new ArrayList<>();
            for (Integer key : acc.keySet()) {
                int m = acc.get(key);
                if (m == 0) {
                    continue;
                }
                if (m < 0) {
                    negs.add(-m);
                } else {
                    poss.add(m);
                }
            }
            int ans = Integer.MAX_VALUE;
            Stack<Integer> stNeg = new Stack<>();
            Stack<Integer> stPos = new Stack<>();
            for (int i = 0; i < 1000; i++) {
                for (Integer num : negs) {
                    stNeg.push(num);
                }
                for (Integer num : poss) {
                    stPos.push(num);
                }
                int cur = 0;
                while (!stNeg.isEmpty()) {
```

```
                    int n = stNeg.pop();
                    int p = stPos.pop();
                    cur++;
                    if (n == p) {
                        continue;
                    }
                    if (n > p) {
                        stNeg.push(n - p);
                    } else {
                        stPos.push(p - n);
                    }
                }
                ans = Math.min(ans, cur);
                Collections.shuffle(negs);
                Collections.shuffle(poss);
            }
            return ans;
        }
    }
}

// ============================================================
// FILE 443: _466.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 466. Count The Repetitions
 *
 * Define S = [s,n] as the string S which consists of n connected strings s. For example,
["abc", 3] ="abcabcabc".

 On the other hand, we define that string s1 can be obtained from string s2 if we can
remove some characters from s2 such that it becomes s1.
 For example, “abc” can be obtained from “abdbec” based on our definition, but it can not
be obtained from “acbbe”.
 You are given two non-empty strings s1 and s2 (each at most 100 characters long) and two
integers 0 ≤ n1 ≤ 106 and 1 ≤ n2 ≤ 106.
 Now consider the strings S1 and S2, where S1=[s1,n1] and S2=[s2,n2]. Find the maximum
integer m such that [S2,m] can be obtained from S1.

 Example:

 Input:
 s1="acb", n1=4
 s2="ab", n2=2

 Return:
 2

 */
public class _466 {
```

```
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/70707/ugly-java-brute-force-
solution-but-accepted-1088ms
         */
        public int getMaxRepetitions(String s1, int n1, String s2, int n2) {
            char[] s1chars = s1.toCharArray();
            char[] s2chars = s2.toCharArray();
            int i = 0;
            int j = 0;
            int count1 = 0;
            int count2 = 0;
            while (count1 < n1) {
                if (s1chars[i] == s2chars[j]) {
                    j++;
                    if (j == s2.length()) {
                        j = 0;
                        count2++;
                    }
                }
                i++;
                if (i == s1.length()) {
                    i = 0;
                    count1++;
                }
            }
            return count2 / n2;
        }
    }

}

// ============================================================
// FILE 444: _467.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 467. Unique Substrings in Wraparound String
 *
 * Consider the string s to be the infinite wraparound string of
"abcdefghijklmnopqrstuvwxyz",
 * so s will look like this:
"...zabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcd....".
 * Now we have another string p.
 * Your job is to find out how many unique non-empty substrings of p are present in s.
 * In particular, your input is the string p and you need to output the number of
different non-empty substrings of p in the string s.
 * Note: p consists of only lowercase English letters and the size of p might be over
10000.

 Example 1:
 Input: "a"
```

```
 Output: 1
 Explanation: Only the substring "a" of string "a" is in the string □s.

 Example 2:
 Input: "cac"
 Output: 2
 Explanation: There are two substrings "a", "c" of string "cac" in the string s.

 Example 3:
 Input: "zab"
 Output: 6
 Explanation: There are six substrings "z", "a", "b", "za", "ab", "zab" of string "zab"
in the string s.
 */
public class _467 {
    public static class Solution1 {
        /**
         * A naive solution would definitely result in TLE.
         * Since the pattern keeps repeating, DP is the way to go.
         * Because the input consists merely of lowercase English letters, we could
maintain an array of size 26,
         * keep updating this array, counting the substrings that end with this letter,
keep updating it with the largest one possible.
         * <p>
         * Inspired by this: https://discuss.leetcode.com/topic/70658/concise-java-
solution-using-dp
         */
        public int findSubstringInWraproundString(String p) {
            if (p == null || p.isEmpty()) {
                return 0;
            }
            if (p.length() < 2) {
                return p.length();
            }
            int count = 0;
            int[] dp = new int[26];
            dp[p.charAt(0) - 'a'] = 1;
            int len = 1;
            for (int i = 1; i < p.length(); i++) {
                if (p.charAt(i) - 1 == p.charAt(i - 1) || (p.charAt(i) == 'a' &&
p.charAt(i - 1) == 'z')) {
                    len++;
                } else {
                    len = 1;
                }
                dp[p.charAt(i) - 'a'] = Math.max(len, dp[p.charAt(i) - 'a']);
            }

            for (int i : dp) {
                count += i;
            }
            return count;
        }
    }
```

```
}

// ============================================================
// FILE 445: _468.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 468. Validate IP Address
 *
 *  Write a function to check whether an input string is a valid IPv4 address or IPv6
address or neither.
 *  IPv4 addresses are canonically represented in dot-decimal notation, which consists of
four decimal numbers,
 *  each ranging from 0 to 255, separated by dots ("."), e.g.,172.16.254.1;
 *  Besides, leading zeros in the IPv4 is invalid. For example, the address 172.16.254.01
is invalid.

 IPv6 addresses are represented as eight groups of four hexadecimal digits,
 each group representing 16 bits.
 The groups are separated by colons (":").
 For example, the address 2001:0db8:85a3:0000:0000:8a2e:0370:7334 is a valid one.
 Also, we could omit some leading zeros among four hexadecimal digits and
 some low-case characters in the address to upper-case ones,
 so 2001:db8:85a3:0:0:8A2E:0370:7334 is also a valid IPv6 address(Omit leading zeros and
using upper cases).

 However, we don't replace a consecutive group of zero value with a single empty
 group using two consecutive colons (::) to pursue simplicity.
 For example, 2001:0db8:85a3::8A2E:0370:7334 is an invalid IPv6 address.

 Besides, extra leading zeros in the IPv6 is also invalid.
 For example, the address 02001:0db8:85a3:0000:0000:8a2e:0370:7334 is invalid.

 Note: You may assume there is no extra space or special characters in the input string.

 Example 1:
 Input: "172.16.254.1"
 Output: "IPv4"
 Explanation: This is a valid IPv4 address, return "IPv4".

 Example 2:
 Input: "2001:0db8:85a3:0:0:8A2E:0370:7334"
 Output: "IPv6"
 Explanation: This is a valid IPv6 address, return "IPv6".

 Example 3:
 Input: "256.256.256.256"
```

```
 Output: "Neither"
 Explanation: This is neither a IPv4 address nor a IPv6 address.
 validIPAddress */
public class _468 {

    public static class Solution1 {

        static final String NEITHER = "Neither";

        public String validIPAddress(String IP) {
            if (IP.contains(".")) {
                return isValidIPv4(IP);
            } else if (IP.contains(":")) {
                return isValidIPv6(IP);
            } else {
                return NEITHER;
            }
        }

        private String isValidIPv6(String IP) {
            if (getDelimiterCount(IP, ':') != 7) {
                return NEITHER;
            }
            String[] bytes = IP.split("\\:");
            if (bytes.length != 8) {
                return NEITHER;
            }
            for (int i = 0; i < 8; i++) {
                if (hasInvalidIPV6Char(bytes[i])) {
                    return NEITHER;
                }
                try {
                    if (bytes[i].length() > 4) {
                        return NEITHER;
                    }
                    int intNum = Integer.parseInt(bytes[i], 16);
                    if (intNum < 0) {
                        return NEITHER;
                    }
                    if (i == 0 && intNum != 0 && bytes[i].charAt(0) == '0') {
                        return NEITHER;
                    }
                } catch (Exception e) {
                    return NEITHER;
                }

            }
            return "IPv6";
        }

        private String isValidIPv4(String IP) {
            if (getDelimiterCount(IP, '.') != 3) {
                return NEITHER;
            }
```

```
            String[] bytes = IP.split("\\.");
            if (bytes.length != 4) {
                return NEITHER;
            }
            for (String num : bytes) {
                try {
                    int intNum = Integer.parseInt(num);
                    if (intNum > 255 || intNum < 0) {
                        return NEITHER;
                    }
                    if (intNum != 0) {
                        for (int i = 0; i < num.length(); i++) {
                            if (num.charAt(i) == '0') {
                                return NEITHER;
                            } else {
                                break;
                            }
                        }
                    } else if (intNum == 0) {
                        if (num.length() != 1) {
                            return NEITHER;
                        }
                    }
                } catch (Exception e) {
                    return NEITHER;
                }
            }
            return "IPv4";
        }

        private boolean hasInvalidIPV6Char(String str) {
            Set<Character> set = new HashSet<>(Arrays.asList('0', '1', '2', '3', '4',
'5', '6', '7', '8', '9',
                    'a', 'b', 'c', 'd', 'e', 'f', 'A', 'B', 'C', 'D', 'E', 'F'));
            for (char c : str.toCharArray()) {
                if (!set.contains(c)) {
                    return true;
                }
            }
            return false;
        }

        private int getDelimiterCount(String ip, char delimiter) {
            int count = 0;
            for (char c : ip.toCharArray()) {
                if (c == delimiter) {
                    count++;
                }
            }
            return count;
        }
    }
}
```

```
// ============================================================
// FILE 446: _469.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;

/**
 * 469. Convex Polygon
 *
 * Given a list of points that form a polygon when joined sequentially, find if this
polygon is convex (Convex polygon definition).

 Note:

 There are at least 3 and at most 10,000 points.
 Coordinates are in the range -10,000 to 10,000.
 You may assume the polygon formed by given points is always a simple polygon (Simple
polygon definition).
 In other words, we ensure that exactly two edges intersect at each vertex, and that
edges otherwise don't intersect each other.

 Example 1:

 [[0,0],[0,1],[1,1],[1,0]]

 Answer: True

 Example 2:

 [[0,0],[0,10],[10,10],[10,0],[5,5]]

 Answer: False

 */
public class _469 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/70706/beyond-my-knowledge-java-
solution-with-in-line-explanation
         */
        public boolean isConvex(List<List<Integer>> points) {
            // For each set of three adjacent points A, B, C, find the cross product AB ·
BC. If the sign of
            // all the cross products is the same, the angles are all positive or
negative (depending on the
            // order in which we visit them) so the polygon is convex.
            boolean gotNegative = false;
            boolean gotPositive = false;
            int numPoints = points.size();
            int B;
            int C;
```

```
            for (int A = 0; A < numPoints; A++) {
                // Trick to calc the last 3 points: n - 1, 0 and 1.
                B = (A + 1) % numPoints;
                C = (B + 1) % numPoints;

                int crossProduct =
                        crossProductLength(
                                points.get(A).get(0), points.get(A).get(1),
                                points.get(B).get(0), points.get(B).get(1),
                                points.get(C).get(0), points.get(C).get(1));
                if (crossProduct < 0) {
                    gotNegative = true;
                } else if (crossProduct > 0) {
                    gotPositive = true;
                }
                if (gotNegative && gotPositive) {
                    return false;
                }
            }

            // If we got this far, the polygon is convex.
            return true;
        }

        // Return the cross product AB x BC.
        // The cross product is a vector perpendicular to AB and BC having length |AB| *
|BC| * Sin(theta) and
        // with direction given by the right-hand rule. For two vectors in the X-Y plane,
the result is a
        // vector with X and Y components 0 so the Z component gives the vector's length
and direction.
        private int crossProductLength(int Ax, int Ay, int Bx, int By, int Cx, int Cy) {
            // Get the vectors' coordinates.
            int BAx = Ax - Bx;
            int BAy = Ay - By;
            int BCx = Cx - Bx;
            int BCy = Cy - By;

            // Calculate the Z coordinate of the cross product.
            return (BAx * BCy - BAy * BCx);
        }
    }

}

// ============================================================
// FILE 447: _471.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 471. Encode String with Shortest Length
 *
```

```
 Given a non-empty string, encode the string such that its encoded length is the
shortest.

 The encoding rule is: k[encoded_string], where the encoded_string inside the square
 brackets is being repeated exactly k times.

 Note:
 k will be a positive integer and encoded string will not be empty or have extra space.
 You may assume that the input string contains only lowercase English letters. The
string's length is at most 160.
 If an encoding process does not make the string shorter,
 then do not encode it. If there are several solutions, return any of them is fine.

 Example 1:

 Input: "aaa"
 Output: "aaa"
 Explanation: There is no way to encode it such that it is shorter than the input string,
so we do not encode it.
 Example 2:

 Input: "aaaaa"
 Output: "5[a]"
 Explanation: "5[a]" is shorter than "aaaaa" by 1 character.
 Example 3:

 Input: "aaaaaaaaaa"
 Output: "10[a]"
 Explanation: "a9[a]" or "9[a]a" are also valid solutions, both of them have the same
length = 5, which is the same as "10[a]".
 Example 4:

 Input: "aabcaabcd"
 Output: "2[aabc]d"
 Explanation: "aabc" occurs twice, so one answer can be "2[aabc]d".
 Example 5:

 Input: "abbbabbbcabbbabbbc"
 Output: "2[2[abbb]c]"
 Explanation: "abbbabbbc" occurs twice, but "abbbabbbc" can also be encoded to
"2[abbb]c", so one answer can be "2[2[abbb]c]".
 */
public class _471 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/71963/accepted-solution-in-java
         */
        public String encode(String s) {
            String[][] dp = new String[s.length()][s.length()];

            for (int l = 0; l < s.length(); l++) {
                for (int i = 0; i < s.length() - l; i++) {
                    int j = i + l;
```

```
                    String substr = s.substring(i, j + 1);
                    // Checking if string length < 5. In that case, we know that encoding
will not help.
                    if (j - i < 4) {
                        dp[i][j] = substr;
                    } else {
                        dp[i][j] = substr;
                        // Loop for trying all results that we get after dividing the
strings into 2 and combine the   results of 2 substrings
                        for (int k = i; k < j; k++) {
                            if ((dp[i][k] + dp[k + 1][j]).length() < dp[i][j].length()) {
                                dp[i][j] = dp[i][k] + dp[k + 1][j];
                            }
                        }

                        // Loop for checking if string can itself found some pattern in
it which could be repeated.
                        for (int k = 0; k < substr.length(); k++) {
                            String repeatStr = substr.substring(0, k + 1);
                            if (repeatStr != null
                                    && substr.length() % repeatStr.length() == 0
                                    && substr.replaceAll(repeatStr, "").length() == 0) {
                                String ss = substr.length() / repeatStr.length() + "[" +
dp[i][i + k] + "]";
                                if (ss.length() < dp[i][j].length()) {
                                    dp[i][j] = ss;
                                }
                            }
                        }
                    }
                }
            }
            return dp[0][s.length() - 1];
        }
    }

}

// ============================================================
// FILE 448: _472.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 472. Concatenated Words
 *
 * Given a list of words, please write a program that returns all concatenated words in
```

```
the given list of words.

 A concatenated word is defined as a string that is comprised entirely of at least two
shorter words in the given array.

 Example:
 Input:
["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]

 Output: ["catsdogcats","dogcatsdog","ratcatdogcat"]

 Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";
 "dogcatsdog" can be concatenated by "dog", "cats" and "dog";
 "ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".
 Note:
 The number of elements of the given array will not exceed 10,000
 The length sum of elements in the given array will not exceed 600,000.
 All the input string will only include lower case letters.
 The returned elements order does not matter.

 */

public class _472 {

        public static class Solution1 {
                private TrieNode root;
                private int maxWordLen;

                public List<String> findAllConcatenatedWordsInADict(String[] words) {
                        ResultType result = buildTrie(words);
                        root = result.root;
                        maxWordLen = result.maxWordLen;

                        List<String> validConcatenatedWords = new ArrayList();
                        for (String word : words) {
                                if (word == null || word.length() == 0) {
                                        continue;
                                }
                                remove(word, root);/** every word is comprised of every
word itself, thus this word itself needs to be removed first for checking it*/
                                int n = word.length();
                                boolean[] dp = new boolean[n + 1];
                                dp[0] = true;

                                for (int i = 1; i <= n; i++) {
                                        for (int j = 1; j <= i && j <= maxWordLen; j++) {
                                                if (!dp[i - j]) {
                                                        continue;
                                                }

                                                String subWord = word.substring(i - j,
i);
                                                if (contains(subWord, root)) {
                                                        dp[i] = true;
```

```
                                        break;
                                }
                        }
                }

                if (dp[n]) {
                        validConcatenatedWords.add(word);
                }
                undoRemove(word, root);
        }
        return validConcatenatedWords;
}

public ResultType buildTrie(String[] words) {
        ResultType result = new ResultType();

        TrieNode root = new TrieNode();
        int maxWordLen = 0;

        for (String word : words) {
                maxWordLen = Math.max(maxWordLen, word.length());
                char[] chars = word.toCharArray();
                TrieNode node = root;
                for (int i = 0; i < chars.length; i++) {
                        char c = chars[i];
                        if (node.children[c - 'a'] == null) {
                                node.children[c - 'a'] = new TrieNode();
                        }
                        node = node.children[c - 'a'];
                }
                node.isWord = true;
        }

        result.root = root;
        result.maxWordLen = maxWordLen;
        return result;
}

public class ResultType {
        int maxWordLen;
        TrieNode root;
}

// Returns true if the word is in the trie.
public boolean contains(String word, TrieNode root) {
        TrieNode node = root;
        for (int i = 0; i < word.length(); i++) {
                if (node.children[word.charAt(i) - 'a'] == null) {
                        return false;
                }
                node = node.children[word.charAt(i) - 'a'];
        }
        return node.isWord;
}
```

```
                // mark that word on
                public void undoRemove(String word, TrieNode root) {
                        TrieNode node = root;
                        for (int i = 0; i < word.length(); i++) {
                                node = node.children[word.charAt(i) - 'a'];
                        }
                        node.isWord = true;
                }

                // mark that word off, we are not really deleting that word
                public void remove(String word, TrieNode root) {
                        TrieNode node = root;
                        for (int i = 0; i < word.length(); i++) {
                                node = node.children[word.charAt(i) - 'a'];
                        }
                        node.isWord = false;
                }

                class TrieNode {
                        boolean isWord;
                        TrieNode[] children = new TrieNode[26];

                        public TrieNode() {
                        }
                }
        }

        public static class Solution2 {
                public List<String> findAllConcatenatedWordsInADict(String[] words) {
                        List<String> result = new ArrayList<>();
                        Set<String> preWords = new HashSet<>();
                        /**Words could only be formed by other words that are shorter
than itself, so we sort them based on their lengths first.*/
                        Arrays.sort(words, (s1, s2) -> s1.length() - s2.length());

                        for (int i = 0; i < words.length; i++) {
                                if (canForm(words[i], preWords)) {
                                        result.add(words[i]);
                                }
                                preWords.add(words[i]);
                        }

                        return result;
                }

                boolean canForm(String word, Set<String> dict) {
                        if (dict.isEmpty()) {
                                return false;
                        }
                        boolean[] dp = new boolean[word.length() + 1];
                        dp[0] = true;
                        for (int i = 1; i <= word.length(); i++) {
                                for (int j = 0; j < i; j++) {
```

```
                                    if (dp[j] && dict.contains(word.substring(j, i)))
{
                                        dp[i] = true;
                                        break;
                                    }
                                }
                            }
                            return dp[word.length()];
                        }
                    }
}

// ============================================================
// FILE 449: _473.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 473. Matchsticks to Square
 *
 * Remember the story of Little Match Girl? By now, you know exactly what matchsticks the
little match girl has,
 * please find out a way you can make one square by using up all those matchsticks.
 * You should not break any stick, but you can link them up, and each matchstick must be
used exactly one time.
 * Your input will be several matchsticks the girl has, represented with their stick
length.
 * Your output will either be true or false, to represent whether you could make one
square using all the matchsticks the little match girl has.

 Example 1:
 Input: [1,1,2,2,2]
 Output: true
 Explanation: You can form a square with length 2, one side of the square came two sticks
with length 1.

 Example 2:
 Input: [3,3,3,3,4]
 Output: false
 Explanation: You cannot find a way to form a square with all the matchsticks.

 Note:
 The length sum of the given matchsticks is in the range of 0 to 10^9.
 The length of the given matchstick array will not exceed 15.
 */
public class _473 {

    public static class Solution1 {
    /**
     * Partially inspired by: https://discuss.leetcode.com/topic/72107/java-dfs-solution-
with-explanation/2
```

```
     * One hidden requirement: you'll have to use up all of the given matchsticks,
nothing could be left behind.
     */
    public boolean makesquare(int[] nums) {
        if (nums == null || nums.length < 4) {
            return false;
        }
        Arrays.sort(nums);
        reverse(nums);
        int sum = 0;
        for (int i : nums) {
            sum += i;
        }
        if (sum % 4 != 0) {
            return false;
        }

        return dfs(nums, new int[4], 0, sum / 4);
    }

        private boolean dfs(int[] nums, int[] sums, int index, int target) {
            if (index == nums.length) {
                if (sums[0] == target && sums[1] == target && sums[2] == target) {
                    return true;
                }
                return false;
            }
            for (int i = 0; i < 4; i++) {
                if (sums[i] + nums[index] > target) {
                    continue;
                }
                sums[i] += nums[index];
                if (dfs(nums, sums, index + 1, target)) {
                    return true;
                }
                sums[i] -= nums[index];
            }
            return false;
        }

        private void reverse(int[] nums) {
            int left = 0;
            int right = nums.length - 1;
            while (left < right) {
                int tmp = nums[left];
                nums[left] = nums[right];
                nums[right] = tmp;
                left++;
                right--;
            }
        }
    }

}
```

```
// ============================================================
// FILE 450: _474.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 474. Ones and Zeroes
 *
 * In the computer world, use restricted resource you have to generate maximum benefit is
what we always want to pursue.
 * For now, suppose you are a dominator of m 0s and n 1s respectively.
 * On the other hand, there is an array with strings consisting of only 0s and 1s.
 * Now your task is to find the maximum number of strings that you can form with given m
0s and n 1s.
 * Each 0 and 1 can be used at most once.

 Note:

 The given numbers of 0s and 1s will both not exceed 100. The size of given string array
won't exceed 600.

 Example 1:
 Input: Array = {"10", "0001", "111001", "1", "0"}, m = 5, n = 3
 Output: 4
 Explanation: This are totally 4 strings can be formed by the using of 5 0s and 3 1s,
which are “10,”0001”,”1”,”0”

 Example 2:
 Input: Array = {"10", "0", "1"}, m = 1, n = 1
 Output: 2
 Explanation: You could form "10", but then you'd have nothing left. Better form "0" and
"1".

 */
public class _474 {
    public static class Solution1 {
        public int findMaxForm(String[] strs, int m, int n) {
            int[][] dp = new int[m + 1][n + 1];
            for (String str : strs) {
                int[] count = count(str);
                for (int i = m; i >= count[0]; i--) {
                    for (int j = n; j >= count[1]; j--) {
                        dp[i][j] = Math.max(dp[i][j], dp[i - count[0]][j - count[1]] +
1);
                    }
                }
            }
            return dp[m][n];
        }

        private int[] count(String str) {
            int[] res = new int[2];
```

```
            for (int i = 0; i < str.length(); i++) {
                res[str.charAt(i) - '0']++;
            }
            return res;
        }
    }

}

// ============================================================
// FILE 451: _475.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 475.
 *
 Winter is coming! Your first job during the contest is to design a standard heater with
fixed warm radius to warm all the houses.

 Now, you are given positions of houses and heaters on a horizontal line, find out
minimum radius of heaters so that all houses could be covered by those heaters.

 So, your input will be the positions of houses and heaters seperately, and your expected
output will be the minimum radius standard of heaters.

 Note:
 Numbers of houses and heaters you are given are non-negative and will not exceed 25000.
 Positions of houses and heaters you are given are non-negative and will not exceed 10^9.
 As long as a house is in the heaters' warm radius range, it can be warmed.
 All the heaters follow your radius standard and the warm radius will the same.

 Example 1:
 Input: [1,2,3],[2]
 Output: 1
 Explanation: The only heater was placed in the position 2, and if we use the radius 1
standard, then all the houses can be warmed.

 Example 2:
 Input: [1,2,3,4],[1,4]
 Output: 1
 Explanation: The two heater was placed in the position 1 and 4. We need to use radius 1
standard, then all the houses can be warmed.
 */
public class _475 {

        public static class Solution1 {
                //credit: https://discuss.leetcode.com/topic/71460/short-and-clean-java-
binary-search-solution
                public int findRadius(int[] houses, int[] heaters) {
                        Arrays.sort(heaters);
```

```
                        int radius = Integer.MIN_VALUE;
                        for (int house : houses) {
                                int index = Arrays.binarySearch(heaters, house);
                                if (index < 0) {
                                        index = ~index;
                                }
                                int distance1 = index - 1 >= 0 ? house - heaters[index -
1] : Integer.MAX_VALUE;
                                int distance2 = index < heaters.length ? heaters[index] -
house : Integer.MAX_VALUE;

                                radius = Math.max(radius, Math.min(distance1,
distance2));
                        }
                        return radius;
                }
        }
}

// ============================================================
// FILE 452: _476.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 476. Number Complement
 *
 * Given a positive integer, output its complement number. The complement strategy is to
flip the bits of its binary representation.

 Note:
 The given integer is guaranteed to fit within the range of a 32-bit signed integer.
 You could assume no leading zero bit in the integer’s binary representation.
 Example 1:
 Input: 5
 Output: 2
 Explanation: The binary representation of 5 is 101 (no leading zero bits), and its
complement is 010. So you need to output 2.
 Example 2:
 Input: 1
 Output: 0
 Explanation: The binary representation of 1 is 1 (no leading zero bits), and its
complement is 0. So you need to output 0.
 */
public class _476 {

    public static class Solution1 {
        public int findComplement(int num) {
            return ~num & ((Integer.highestOneBit(num) << 1) - 1);
        }
    }

    public static class Solution2 {
```

```
        public int findComplement(int num) {
            String str = Integer.toBinaryString(num);
            StringBuilder sb = new StringBuilder();
            char[] chars = str.toCharArray();
            for (int i = 0; i < chars.length; i++) {
                if (chars[i] == '0') {
                    sb.append("1");
                } else {
                    sb.append("0");
                }
            }
            return Integer.parseInt(sb.toString(), 2);
        }
    }
}

// ============================================================
// FILE 453: _477.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 477. Total Hamming Distance
 *
 * The Hamming distance between two integers is the number of positions at which the
corresponding bits are different.

 Now your job is to find the total Hamming distance between all pairs of the given
numbers.

 Example:
 Input: 4, 14, 2
 Output: 6
 Explanation: In binary representation, the 4 is 0100, 14 is 1110, and 2 is 0010 (just
 showing the four bits relevant in this case). So the answer will be:
 HammingDistance(4, 14) + HammingDistance(4, 2) + HammingDistance(14, 2) = 2 + 2 + 2 = 6.

 Note:
 Elements of the given array are in the range of 0 to 10^9
 Length of the array will not exceed 10^4.
 */
public class _477 {
        public static class Solution1 {
                public int totalHammingDistance(int[] nums) {
                        int r = 0;
                        for (int i = 0; i < 32; i++) {
                                int one = 0;
                                int zero = 0;
                                int bit = 1 << i;
                                for (int n : nums) {
                                        if ((n & bit) != 0) {
                                                one++;
                                        } else {
```

```
                                            zero++;
                                        }
                                    }
                                    r += one * zero;
                                }
                                return r;
                            }
                        }

}

// ============================================================
// FILE 454: _479.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 479. Largest Palindrome Product
 *
 * Find the largest palindrome made from the product of two n-digit numbers.
 Since the result could be very large, you should return the largest palindrome mod 1337.

 Example:
 Input: 2
 Output: 987

 Explanation: 99 x 91 = 9009, 9009 % 1337 = 987

 Note:
 The range of n is [1,8].
 */
public class _479 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/74125/java-solution-using-assumed-
max-palindrom
         */
        public int largestPalindrome(int n) {
            // if input is 1 then max is 9
            if (n == 1) {
                return 9;
            }

            // if n = 3 then upperBound = 999 and lowerBound = 99
            int upperBound = (int) Math.pow(10, n) - 1;
            int lowerBound = upperBound / 10;
            long maxNumber = (long) upperBound * (long) upperBound;

            // represents the first half of the maximum assumed palindrom.
            // e.g. if n = 3 then maxNumber = 999 x 999 = 998001 so firstHalf = 998
            int firstHalf = (int) (maxNumber / (long) Math.pow(10, n));

            boolean palindromFound = false;
```

```
            long palindrom = 0;

            while (!palindromFound) {
                // creates maximum assumed palindrom
                // e.g. if n = 3 first time the maximum assumed palindrom will be 998 899
                palindrom = createPalindrom(firstHalf);

                // here i and palindrom/i forms the two factor of assumed palindrom
                for (long i = upperBound; upperBound > lowerBound; i--) {
                    // if n= 3 none of the factor of palindrom  can be more than 999 or
less than square root of assumed palindrom
                    if (palindrom / i > maxNumber || i * i < palindrom) {
                        break;
                    }

                    // if two factors found, where both of them are n-digits,
                    if (palindrom % i == 0) {
                        palindromFound = true;
                        break;
                    }
                }

                firstHalf--;
            }
            return (int) (palindrom % 1337);
        }

        private long createPalindrom(long num) {
            String str = num + new StringBuilder().append(num).reverse().toString();
            return Long.parseLong(str);
        }
    }
}

// ============================================================
// FILE 455: _480.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Collections;
import java.util.PriorityQueue;

/**
 * 480. Sliding Window Median
 *
 * Median is the middle value in an ordered integer list. If the size of the list is
even, there is no middle value. So the median is the mean of the two middle value.

 Examples:
 [2,3,4] , the median is 3

 [2,3], the median is (2 + 3) / 2 = 2.5
```

```
 Given an array nums, there is a sliding window of size k which is moving from the very left of the array to the very right.
 You can only see the k numbers in the window. Each time the sliding window moves right by one position.
 Your job is to output the median array for each window in the original array.

 For example,
 Given nums = [1,3,-1,-3,5,3,6,7], and k = 3.

 Window position                Median
 ---------------               -----
 [1  3  -1] -3  5  3  6  7       1
 1 [3  -1  -3] 5  3  6  7       -1
 1  3 [-1  -3  5] 3  6  7       -1
 1  3  -1 [-3  5  3] 6  7       3
 1  3  -1  -3 [5  3  6] 7       5
 1  3  -1  -3  5 [3  6  7]      6

 Therefore, return the median sliding window as [1,-1,-1,3,5,6].

 Note:
 You may assume k is always valid, ie: 1 ≤ k ≤ input array's size for non-empty array.
 */
public class _480 {
    public static class Solution1 {

        /**
         * You cannot simply use minus sign '-' to denote the descending order, because e.g. 3 and -3 might both exist in this array,
         * so we'll have to use the original numbers themselves to store in the heaps.
         */
        private PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        private PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        public double[] medianSlidingWindow(int[] nums, int k) {
            int n = nums.length - k + 1;
            if (n <= 0) {
                return new double[0];
            }
            double[] result = new double[n];

            for (int i = 0; i <= nums.length; i++) {
                if (i >= k) {
                    result[i - k] = getMedian();
                    remove(nums[i - k]);
                }
                if (i < nums.length) {
                    add(nums[i]);
                }
            }

            return result;
        }
```

```
        private double getMedian() {
            if (maxHeap.isEmpty() && minHeap.isEmpty()) {
                return 0;
            }

            if (maxHeap.size() == minHeap.size()) {
                return ((double) maxHeap.peek() + (double) minHeap.peek()) / 2.0;
            } else {
                return (double) minHeap.peek();
            }
        }

        private void remove(int num) {
            if (num < getMedian()) {
                maxHeap.remove(num);
            } else {
                minHeap.remove(num);
            }
            if (maxHeap.size() > minHeap.size()) {
                minHeap.add(maxHeap.poll());
            }
            if (minHeap.size() - maxHeap.size() > 1) {
                maxHeap.add(minHeap.poll());
            }
        }

        private void add(int num) {
            if (num < getMedian()) {
                maxHeap.add(num);
            } else {
                minHeap.add(num);
            }
            if (maxHeap.size() > minHeap.size()) {
                minHeap.add(maxHeap.poll());
            }
            if (minHeap.size() - maxHeap.size() > 1) {
                maxHeap.add(minHeap.poll());
            }
        }
    }

}

// ============================================================
// FILE 456: _481.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 481. Magical String
 *
 * A magical string S consists of only '1' and '2' and obeys the following rules:
```

```
 The string S is magical because concatenating the number of contiguous occurrences of
characters '1' and '2' generates the string S itself.

 The first few elements of string S is the following: S = "1221121221221121122……"

 If we group the consecutive '1's and '2's in S, it will be:

 1 22 11 2 1 22 1 22 11 2 11 22 ......

 and the occurrences of '1's or '2's in each group are:

 1 2    2 1 1 2 1 2 2 1 2 2 ......

 You can see that the occurrence sequence above is the S itself.

 Given an integer N as input, return the number of '1's in the first N number in the
magical string S.

 Note: N will not exceed 100,000.

 Example 1:
 Input: 6
 Output: 3
 Explanation: The first 6 elements of magical string S is "12211" and it contains three
1's, so return 3.
 */
public class _481 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/74917/simple-java-solution-using-
one-array-and-two-pointers
         * Algorithm:
         *
         * 1. Create an int array a and initialize the first 3 elements with 1, 2, 2.
         * 2. Create two pointers head and tail. head points to the number which will be
used to generate new numbers.
         * tail points to the next empty position to put the new number. Then keep
generating new numbers until tail >= n.
         * 3. Need to create the array 1 element more than n to avoid overflow because
the last round head might points to a number 2.
         * 4. A trick to flip number back and forth between 1 and 2: num = num ^ 3
         */
        public int magicalString(int n) {
            if (n <= 0) {
                return 0;
            }
            if (n <= 3) {
                return 1;
            }

            int[] a = new int[n + 1];
            a[0] = 1;
```

```
            a[1] = 2;
            a[2] = 2;

            int head = 2;
            int tail = 3;
            int num = 1;
            int result = 1;

            while (tail < n) {
                for (int i = 0; i < a[head]; i++) {
                    a[tail] = num;
                    if (num == 1 && tail < n) {
                        result++;
                    }
                    tail++;
                }
                num = num ^ 3;
                head++;
            }

            return result;
        }
    }

}

// ============================================================
// FILE 457: _482.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 482. License Key Formatting
 *
 * Now you are given a string S, which represents a software license key which we would
like to format.
 * The string S is composed of alphanumerical characters and dashes.
 * The dashes split the alphanumerical characters within the string into groups.
 * (i.e. if there are m dashes, the string is split into m+1 groups).
 * The dashes in the given string are possibly misplaced.
 * We want each group of characters to be of length K
 * (except for possibly the first group, which could be shorter, but still must contain
at least one character).
 * To satisfy this requirement, we will reinsert dashes. Additionally, all the lower case
letters in the string must be converted to upper case.
 * So, you are given a non-empty string S, representing a license key to format, and an
integer K.
 * And you need to return the license key formatted according to the description above.

 Example 1:
 Input: S = "2-4A0r7-4k", K = 4
 Output: "24A0-R74K"
 Explanation: The string S has been split into two parts, each part has 4 characters.
```

```
 Example 2:
 Input: S = "2-4A0r7-4k", K = 3
 Output: "24-A0R-74K"
 Explanation: The string S has been split into three parts, each part has 3 characters
except the first part as it could be shorter as said above.

 Note:
 The length of string S will not exceed 12,000, and K is a positive integer.
 String S consists only of alphanumerical characters (a-z and/or A-Z and/or 0-9) and
dashes(-).
 String S is non-empty.
 */
public class _482 {
    public static class Solution1 {
        public String licenseKeyFormatting(String S, int K) {
            StringBuilder stringBuilder = new StringBuilder();
            char[] SChars = S.toCharArray();
            for (int i = S.length() - 1, j = 0; i >= 0; ) {
                if (j < K) {
                    if (SChars[i] != '-') {
                        if (SChars[i] >= 'a' && SChars[i] <= 'z') {
                            stringBuilder.append(Character.toUpperCase(SChars[i]));
                        } else {
                            stringBuilder.append(SChars[i]);
                        }
                        j++;
                    }
                    i--;
                } else if (j == K) {
                    j = 0;
                    stringBuilder.append('-');
                }
            }
            if (stringBuilder.length() > 1 &&
stringBuilder.substring(stringBuilder.length() - 1).equals("-")) {
                return stringBuilder.reverse().substring(1);
            }
            return stringBuilder.reverse().toString();
        }
    }
}

// ============================================================
// FILE 458: _483.java
// ============================================================

package com.fishercoder.solutions;

import java.math.BigInteger;

/**
 * 483. Smallest Good Base
 *
```

```
 * For an integer n, we call k>=2 a good base of n, if all digits of n base k are 1.

 Now given a string representing n, you should return the smallest good base of n in
string format.

 Example 1:
 Input: "13"
 Output: "3"
 Explanation: 13 base 3 is 111.

 Example 2:
 Input: "4681"
 Output: "8"
 Explanation: 4681 base 8 is 11111.

 Example 3:
 Input: "1000000000000000000"
 Output: "999999999999999999"
 Explanation: 1000000000000000000 base 999999999999999999 is 11.

 Note:
 The range of n is [3, 10^18].
 The string representing n is always valid and will not have leading zeros.

 */
public class _483 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/82130/java-solution-with-hand-
writing-explain
         */
        public String smallestGoodBase(String n) {
            long nn = Long.parseLong(n);
            long res = 0;
            for (int k = 60; k >= 2; k--) {
                long start = 2;
                long end = nn;
                while (start < end) {
                    long m = start + (end - start) / 2;

                    BigInteger left = BigInteger.valueOf(m);
                    left = left.pow(k).subtract(BigInteger.ONE);
                    BigInteger right =
BigInteger.valueOf(nn).multiply(BigInteger.valueOf(m).subtract(BigInteger.ONE));
                    int cmr = left.compareTo(right);
                    if (cmr == 0) {
                        res = m;
                        break;
                    } else if (cmr < 0) {
                        start = m + 1;
                    } else {
                        end = m;
                    }
```

```
                }

                if (res != 0) {
                    break;
                }
            }
            return "" + res;
        }
    }

}

// ============================================================
// FILE 459: _484.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 484. Find Permutation
 *
 * By now, you are given a secret signature consisting of character 'D' and 'I'.
 * 'D' represents a decreasing relationship between two numbers, 'I' represents an
increasing relationship between two numbers.
 * And our secret signature was constructed by a special integer array, which contains
uniquely all the different number from 1 to n (n is the length of the secret signature
plus 1).
 * For example, the secret signature "DI" can be constructed by array [2,1,3] or [3,1,2],
 * but won't be constructed by array [3,2,4] or [2,1,3,4], which are both illegal
constructing special string that can't represent the "DI" secret signature.

 On the other hand, now your job is to find the lexicographically smallest permutation of
[1, 2, ... n] could refer to the given secret signature in the input.

 Example 1:
 Input: "I"
 Output: [1,2]
 Explanation: [1,2] is the only legal initial spectial string can construct secret
signature "I", where the number 1 and 2 construct an increasing relationship.

 Example 2:
 Input: "DI"
 Output: [2,1,3]
 Explanation: Both [2,1,3] and [3,1,2] can construct the secret signature "DI",
 but since we want to find the one with the smallest lexicographical permutation, you
need to output [2,1,3]

 Note:
 The input string will only contain the character 'D' and 'I'.
 The length of input string is a positive integer and will not exceed 10,000
 */
public class _484 {
    public static class Solution1 {
```

```
        /**
         * credit:https://discuss.leetcode.com/topic/76221/java-o-n-clean-solution-easy-
to-understand
         *
         * For example, given IDIIDD we start with sorted sequence 1234567
         * Then for each k continuous D starting at index i we need to reverse [i, i+k]
portion of the sorted sequence.
         *
         * e.g.
         * IDIIDD
         *
         * 1234567 // sorted
         * 1324765 // answer
         */
        public int[] findPermutation(String s) {
            int[] result = new int[s.length() + 1];
            for (int i = 0; i <= s.length(); i++) {
                result[i] = i + 1;
            }
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == 'D') {
                    int left = i;
                    while (i < s.length() && s.charAt(i) == 'D') {
                        i++;
                    }
                    reverse(result, left, i);
                }
            }
            return result;
        }

        private void reverse(int[] result, int left, int i) {
            while (left < i) {
                int temp = result[left];
                result[left] = result[i];
                result[i] = temp;
                left++;
                i--;
            }
        }
    }

}

// ============================================================
// FILE 460: _485.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 485. Max Consecutive Ones
 *
 * Given a binary array, find the maximum number of consecutive 1s in this array.
```

```
 *
 * Example 1:
 *
 * Input: [1,1,0,1,1,1]
 * Output: 3
 * Explanation: The first two digits or the last three digits are consecutive 1s. The
maximum number of consecutive 1s is 3.
 *
 * Note:
 * The input array will only contain 0 and 1.
 * The length of input array is a positive integer and will not exceed 10,000
 * */
public class _485 {
    public static class Solution1 {
        public int findMaxConsecutiveOnes(int[] nums) {
            int maxLen = 0;
            for (int i = 0; i < nums.length; i++) {
                int currentLen = 0;
                while (i < nums.length && nums[i] == 1) {
                    i++;
                    currentLen++;
                }
                maxLen = Math.max(maxLen, currentLen);
            }
            return maxLen;
        }
    }
}

// ============================================================
// FILE 461: _486.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 486. Predict the Winner
 *
 * Given an array of scores that are non-negative integers.
 * Player 1 picks one of the numbers from either end of the array followed by the player
2 and then player 1 and so on.
 * Each time a player picks a number, that number will not be available for the next
player.
 * This continues until all the scores have been chosen.
 * The player with the maximum score wins.
 * Given an array of scores, predict whether player 1 is the winner.
 * You can assume each player plays to maximize his score.

 Example 1:
 Input: [1, 5, 2]
 Output: False
 Explanation: Initially, player 1 can choose between 1 and 2.
 If he chooses 2 (or 1), then player 2 can choose from 1 (or 2) and 5. If player 2
chooses 5, then player 1 will be left with 1 (or 2).
```

```
 So, final score of player 1 is 1 + 2 = 3, and player 2 is 5.
 Hence, player 1 will never be the winner and you need to return False.

 Example 2:
 Input: [1, 5, 233, 7]
 Output: True
 Explanation: Player 1 first chooses 1. Then player 2 have to choose between 5 and 7. No
matter which number player 2 choose, player 1 can choose 233.
 Finally, player 1 has more score (234) than player 2 (12), so you need to return True
representing player1 can win.

 Note:
 1 <= length of the array <= 20.
 Any scores in the given array are non-negative integers and will not exceed 10,000,000.
 If the scores of both players are equal, then player 1 is still the winner.
 */
public class _486 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/76312/java-1-line-recursion-
solution
         * Explanation
         * So assuming the sum of the array it SUM, so eventually player1 and player2
will split the SUM between themselves.
         * For player1 to win, he/she has to get more than what player2 gets.
         * If we think from the prospective of one player, then what he/she gains each
time is a plus,
         * while, what the other player gains each time is a minus. Eventually if player1
can have a >0 total, player1 can win.
         *
         * Helper function simulate this process. In each round:
         * if e==s, there is no choice but have to select nums[s]
         * otherwise, this current player has 2 options:
         * --> nums[s]-helper(nums,s+1,e): this player select the front item, leaving the
other player a choice from s+1 to e
         * --> nums[e]-helper(nums,s,e-1): this player select the tail item, leaving the
other player a choice from s to e-1
         * Then take the max of these two options as this player's selection, return it.
         */
        public boolean predictTheWinner(int[] nums) {
            return helper(nums, 0, nums.length - 1, new Integer[nums.length]
[nums.length]) >= 0;
        }

        private int helper(int[] nums, int start, int end, Integer[][] mem) {
            if (mem[start][end] == null) {
                mem[start][end] = start == end ? nums[end] :
                        Math.max(nums[end] - helper(nums, start, end - 1, mem),
                                nums[start] - helper(nums, start + 1, end, mem));
            }
            return mem[start][end];
        }
    }
```

```
}

// ============================================================
// FILE 462: _487.java
// ============================================================

package com.fishercoder.solutions;

/**
 * Given a binary array, find the maximum number of consecutive 1s in this array.

 Example 1:
 Input: [1,1,0,1,1,1]
 Output: 3
 Explanation: The first two digits or the last three digits are consecutive 1s.
 The maximum number of consecutive 1s is 3.
 Note:

 The input array will only contain 0 and 1.
 The length of input array is a positive integer and will not exceed 10,000
 */
public class _487 {

    public static int findMaxConsecutiveOnes(int[] nums) {
        int maxOnes = 0;
        for (int i = 0; i < nums.length; i++) {
            int newOnes = 0;
            while (i < nums.length && nums[i] == 1) {
                newOnes++;
                i++;
            }
            maxOnes = Math.max(maxOnes, newOnes);
        }
        return maxOnes;
    }

}

// ============================================================
// FILE 463: _488.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 488. Zuma Game
 *
 * Think about Zuma Game. You have a row of balls on the table, colored red(R),
yellow(Y), blue(B), green(G), and white(W).
 * You also have several balls in your hand.

 Each time, you may choose a ball in your hand, and insert it into the row (including the
leftmost place and rightmost place).
```

```
 Then, if there is a group of 3 or more balls in the same color touching, remove these
balls. Keep doing this until no more balls can be removed.

 Find the minimal balls you have to insert to remove all the balls on the table. If you
cannot remove all the balls, output -1.

 Examples:

 Input: "WRRBBW", "RB"
 Output: -1
 Explanation: WRRBBW -> WRR[R]BBW -> WBBW -> WBB[B]W -> WW

 Input: "WWRRBBWW", "WRBRW"
 Output: 2
 Explanation: WWRRBBWW -> WWRR[R]BBWW -> WWBBWW -> WWBB[B]WW -> WWWW -> empty

 Input:"G", "GGGGG"
 Output: 2
 Explanation: G -> G[G] -> GG[G] -> empty

 Input: "RBYYBBRRB", "YRBGB"
 Output: 3
 Explanation: RBYYBBRRB -> RBYY[Y]BBRRB -> RBBBRRB -> RRRB -> B -> B[B] -> BB[B] -> empty

 Note:

 You may assume that the initial row of balls on the table won’t have any 3 or more
consecutive balls with the same color.
 The number of balls on the table won't exceed 20, and the string represents these balls
is called "board" in the input.
 The number of balls in your hand won't exceed 5, and the string represents these balls
is called "hand" in the input.
 Both input strings will be non-empty and only contain characters 'R','Y','B','G','W'.
 */
public class _488 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/79820/short-java-solution-beats-98
         * Two layer of recursion, pretty cool!
         */

        int maxcount = 6;   // the max balls you need will not exceed 6 since "The number
of balls in your hand won't exceed 5"

        public int findMinStep(String board, String hand) {
            int[] handCount = new int[26];
            for (int i = 0; i < hand.length(); ++i) {
                ++handCount[hand.charAt(i) - 'A'];
            }
            int result = dfs(board + "#", handCount);  // append a "#" to avoid special
process while j==board.length, make the code shorter.
            return result == maxcount ? -1 : result;
```

```
        }

        private int dfs(String s, int[] handCount) {
            s = removeConsecutive(s);
            if (s.equals("#")) {
                return 0;
            }
            int result = maxcount;
            int need = 0;
            for (int i = 0, j = 0; j < s.length(); ++j) {
                if (s.charAt(j) == s.charAt(i)) {
                    continue;
                }
                need = 3 - (j - i);     //balls need to remove current consecutive balls.
                if (handCount[s.charAt(i) - 'A'] >= need) {
                    handCount[s.charAt(i) - 'A'] -= need;
                    result = Math.min(result, need + dfs(s.substring(0, i) +
s.substring(j), handCount));
                    handCount[s.charAt(i) - 'A'] += need;
                }
                i = j;
            }
            return result;
        }

        //remove consecutive balls longer than 3
        private String removeConsecutive(String board) {
            for (int i = 0, j = 0; j < board.length(); ++j) {
                if (board.charAt(j) == board.charAt(i)) {
                    continue;
                }
                if (j - i >= 3) {
                    return removeConsecutive(board.substring(0, i) + board.substring(j));
                } else {
                    i = j;
                }
            }
            return board;
        }
    }
}

// ============================================================
// FILE 464: _490.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * There is a ball in a maze with empty spaces and walls. The ball can go through empty
spaces by rolling up, down, left or right, but it won't stop rolling until hitting a
```

```
wall. When the ball stops, it could choose the next direction.

 Given the ball's start position, the destination and the maze, determine whether the
ball could stop at the destination.

 The maze is represented by a binary 2D array. 1 means the wall and 0 means the empty
space. You may assume that the borders of the maze are all walls. The start and
destination coordinates are represented by row and column indexes.

 Example 1

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (4, 4)

 Output: true
 Explanation: One possible way is : left -> down -> left -> down -> right -> down ->
right.

 Example 2

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (3, 2)

 Output: false
 Explanation: There is no way for the ball to stop at the destination.

 Note:
 There is only one ball and one destination in the maze.
 Both the ball and the destination exist on an empty space, and they will not be at the
same position initially.
 The given maze does not contain border (like the red rectangle in the example pictures),
but you could assume the border of the maze are all walls.
 The maze contains at least 2 empty spaces, and both the width and height of the maze
won't exceed 100.
 */
public class _490 {
    /**
     * BFS: the key part of this algorithm is that: this is a ball that won't stop
```

```
rolling until it hits a wall.
     * This means we'll have to store all the empty spaces where the ball was forced to
stop into the queue, these are
     * the only places where the next starting points could be.
     * Then we use BFS to traverse through all four directions of each starting point.
     * <p>
     * Also, another point to note is: it must be a stop point for the ball.
     */
    public boolean hasPath(int[][] maze, int[] start, int[] destination) {
        if (start[0] == destination[0] && destination[0] == destination[1]) {
            return true;
        }
        final int[] directions = new int[]{-1, 0, 1, 0, -1};
        Queue<Point> queue = new LinkedList<>();
        queue.offer(new Point(start[0], start[1]));
        int m = maze.length;
        int n = maze[0].length;
        boolean[][] visited = new boolean[m][n];
        visited[start[0]][start[1]] = true;

        while (!queue.isEmpty()) {
            Point curr = queue.poll();
            int x = curr.x;
            int y = curr.y;//keep the original value
            for (int i = 0; i < directions.length - 1; i++) {
                int xx = x;
                int yy = y;//use temp variables to move
                while (xx >= 0 && yy >= 0 && xx < m && yy < n && maze[xx][yy] == 0) {
                    xx += directions[i];
                    yy += directions[i + 1];
                }
                xx -= directions[i];
                yy -= directions[i + 1];
                if (visited[xx][yy]) {
                    continue;
                }
                visited[xx][yy] = true;
                if (destination[0] == xx && destination[1] == yy) {
                    return true;
                }
                queue.offer(new Point(xx, yy));
            }
        }
        return false;
    }

    class Point {
        int x;
        int y;

        public Point(int x, int y) {
            this.x = x;
            this.y = y;
        }
```

```
    }
}

// ============================================================
// FILE 465: _491.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 491. Increasing Subsequences
 *
 * Given an integer array, your task is to find all the different possible increasing
subsequences of the given array,
 * and the length of an increasing subsequence should be at least 2 .

 Example:
 Input: [4, 6, 7, 7]
 Output: [[4, 6], [4, 7], [4, 6, 7], [4, 6, 7, 7], [6, 7], [6, 7, 7], [7,7], [4,7,7]]

 Note:
 The length of the given array will not exceed 15.
 The range of integer in the given array is [-100,100].
 The given array may contain duplicates, and two equal integers should also be considered
as a special case of increasing sequence.
 */
public class _491 {

    public static class Solution1 {
        public List<List<Integer>> findSubsequences(int[] nums) {
            if (nums == null || nums.length == 1) {
                return new ArrayList<>();
            }
            Set<List<Integer>> answer = new HashSet<>();
            List<Integer> list = new ArrayList<>();
            return new ArrayList<>(backtracking(nums, 0, list, answer));
        }

        private Set<List<Integer>> backtracking(int[] nums, int start, List<Integer>
currList,
            Set<List<Integer>> answer) {
            if (currList.size() >= 2) {
                answer.add(new ArrayList<>(currList));
            }
            for (int i = start; i < nums.length; i++) {
                if (currList.size() == 0 || currList.get(currList.size() - 1) <= nums[i])
{
                    currList.add(nums[i]);
                    backtracking(nums, i + 1, currList, answer);
```

```
                    currList.remove(currList.size() - 1);
                }
            }
            return answer;
        }
    }
}

// ============================================================
// FILE 466: _492.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 492. Construct the Rectangle
 *
 * For a web developer, it is very important to know how to design a web page's size.
 * So, given a specific rectangular web page’s area, your job by now is to design a
rectangular web page,
 * whose length L and width W satisfy the following requirements:

 1. The area of the rectangular web page you designed must equal to the given target
area.
 2. The width W should not be larger than the length L, which means L >= W.
 3. The difference between length L and width W should be as small as possible.

 You need to output the length L and the width W of the web page you designed in
sequence.
 Example:
 Input: 4
 Output: [2, 2]
 Explanation: The target area is 4, and all the possible ways to construct it are [1,4],
[2,2], [4,1].
 But according to requirement 2, [1,4] is illegal; according to requirement 3,  [4,1] is
not optimal compared to [2,2]. So the length L is 2, and the width W is 2.

 Note:
 The given area won't exceed 10,000,000 and is a positive integer
 The web page's width and length you designed must be positive integers.

 */
public class _492 {

    public static class Solution1 {
        public int[] constructRectangle(int area) {
            int i = 0;
            int j = area;
            int[] result = new int[2];
            while (i <= j) {
                long product = i * j;
                if (product == area) {
                    result[0] = j--;
                    result[1] = i++;
```

```
                } else if (product > area) {
                    j--;
                } else {
                    i++;
                }
            }
            return result;
        }
    }

}

// ============================================================
// FILE 467: _493.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 493. Reverse Pairs
 *
 * Given an array nums, we call (i, j) an important reverse pair if i < j and nums[i] >
2*nums[j].

 You need to return the number of important reverse pairs in the given array.

 Example1:

 Input: [1,3,2,3,1]
 Output: 2
 Example2:

 Input: [2,4,3,5,1]
 Output: 3

 Note:
 The length of the given array will not exceed 50,000.
 All the numbers in the input array are in the range of 32-bit integer.
 */
public class _493 {

    public static class Solution1 {

        /**
         * reference: https://discuss.leetcode.com/topic/78933/very-short-and-clear-
mergesort-bst-java-solutions
         */
        public int reversePairs(int[] nums) {
            return mergeSort(nums, 0, nums.length - 1);
        }

        private int mergeSort(int[] nums, int start, int end) {
```

```
            if (start >= end) {
                return 0;
            }
            int mid = start + (end - start) / 2;
            int cnt = mergeSort(nums, start, mid) + mergeSort(nums, mid + 1, end);
            for (int i = start, j = mid + 1; i <= mid; i++) {
                /**it has to be 2.0 instead of 2, otherwise it's going to stack overflow,
i.e. test3 is going to fail*/
                while (j <= end && nums[i] > nums[j] * 2.0) {
                    j++;
                }
                cnt += j - (mid + 1);
            }
            Arrays.sort(nums, start, end + 1);
            return cnt;
        }
    }
}

// ============================================================
// FILE 468: _494.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 494. Target Sum
 *
 *  You are given a list of non-negative integers, a1, a2, ..., an, and a target, S.
 *  Now you have 2 symbols + and -. For each integer, you should choose one from + and -
as its new symbol.
 *  Find out how many ways to assign symbols to make sum of integers equal to target S.

 Example 1:

 Input: nums is [1, 1, 1, 1, 1], S is 3.
 Output: 5
 Explanation:

 -1+1+1+1+1 = 3
 +1-1+1+1+1 = 3
 +1+1-1+1+1 = 3
 +1+1+1-1+1 = 3
 +1+1+1+1-1 = 3

 There are 5 ways to assign symbols to make the sum of nums be target 3.

 Note:

 The length of the given array is positive and will not exceed 20.
 The sum of elements in the given array will not exceed 1000.
 Your output answer is guaranteed to be fitted in a 32-bit integer.

 */
```

```
public class _494 {

    public static class Solution1 {
        public int findTargetSumWays(int[] nums, int S) {
            return find(0, nums, S);
        }

        private int find(int p, int[] nums, int sum) {
            if (p == nums.length) {
                if (sum == 0) {
                    return 1;
                } else {
                    return 0;
                }
            }
            return find(p + 1, nums, sum + nums[p]) + find(p + 1, nums, sum - nums[p]);
        }
    }

}

// ============================================================
// FILE 469: _495.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 495. Teemo Attacking
 *
 * In LLP world, there is a hero called Teemo and his attacking can make his enemy Ashe be in poisoned condition.
 * Now, given the Teemo's attacking ascending time series towards Ashe and the poisoning time duration per Teemo's attacking,
 * you need to output the total time that Ashe is in poisoned condition.

 You may assume that Teemo attacks at the very beginning of a specific time point, and makes Ashe be in poisoned condition immediately.

 Example 1:
 Input: [1,4], 2
 Output: 4
 Explanation: At time point 1, Teemo starts attacking Ashe and makes Ashe be poisoned immediately.
 This poisoned status will last 2 seconds until the end of time point 2.
 And at time point 4, Teemo attacks Ashe again, and causes Ashe to be in poisoned status for another 2 seconds.
 So you finally need to output 4.

 Example 2:
 Input: [1,2], 2
 Output: 3
 Explanation: At time point 1, Teemo starts attacking Ashe and makes Ashe be poisoned.
 This poisoned status will last 2 seconds until the end of time point 2.
```

```
 However, at the beginning of time point 2, Teemo attacks Ashe again who is already in
poisoned status.
 Since the poisoned status won't add up together, though the second poisoning attack will
still work at time point 2, it will stop at the end of time point 3.
 So you finally need to output 3.

 Note:
 You may assume the length of given time series array won't exceed 10000.
 You may assume the numbers in the Teemo's attacking time series and his poisoning time
duration per attacking are non-negative integers, which won't exceed 10,000,000.
 */
public class _495 {

    public static class Solution1 {
        public int findPoisonedDuration(int[] timeSeries, int duration) {
            if (timeSeries == null || timeSeries.length == 0 || duration == 0) {
                return 0;
            }
            int totalDuration = 0;
            for (int i = 0; i < timeSeries.length - 1; i++) {
                if ((timeSeries[i + 1] - timeSeries[i]) >= duration) {
                    totalDuration += duration;
                } else {
                    totalDuration += (timeSeries[i + 1] - timeSeries[i]);
                }
            }
            totalDuration += duration;//plus the last one duration
            return totalDuration;
        }
    }

}

// ============================================================
// FILE 470: _496.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.Stack;

/**
 * 496. Next Greater Element I
 *
 * You are given two arrays (without duplicates) nums1 and nums2 where nums1’s elements
are subset of nums2.
 * Find all the next greater numbers for nums1's elements in the corresponding places of
nums2.
 * The Next Greater Number of a number x in nums1 is the first greater number to its
right in nums2. If it does not exist, output -1 for this number.

 Example 1:
```

```
 Input: nums1 = [4,1,2], nums2 = [1,3,4,2].
 Output: [-1,3,-1]
 Explanation:
 For number 4 in the first array, you cannot find the next greater number for it in the
second array, so output -1.
 For number 1 in the first array, the next greater number for it in the second array is
3.
 For number 2 in the first array, there is no next greater number for it in the second
array, so output -1.

 Example 2:
 Input: nums1 = [2,4], nums2 = [1,2,3,4].
 Output: [3,-1]
 Explanation:
 For number 2 in the first array, the next greater number for it in the second array is
3.
 For number 4 in the first array, there is no next greater number for it in the second
array, so output -1.

 Note:
 All elements in nums1 and nums2 are unique.
 The length of both nums1 and nums2 would not exceed 1000.
 */
public class _496 {

    public static class Solution1 {
        public int[] nextGreaterElement(int[] findNums, int[] nums) {
            Stack<Integer> stack = new Stack();
            Map<Integer, Integer> map = new HashMap();
            for (int i = 0; i < nums.length; i++) {
                while (!stack.isEmpty() && nums[i] > stack.peek()) {
                    map.put(stack.pop(), nums[i]);
                }
                stack.push(nums[i]);
            }

            while (!stack.isEmpty()) {
                map.put(stack.pop(), -1);
            }

            int[] result = new int[findNums.length];
            for (int i = 0; i < findNums.length; i++) {
                result[i] = map.get(findNums[i]);
            }
            return result;
        }
    }

}

// ============================================================
// FILE 471: _498.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 498. Diagonal Traverse
 *
 *  Given a matrix of m x N elements (m rows, N columns), return all elements of the
matrix in diagonal order
 *  as shown in the below image.

 Example:

 Input:
 [
 [ 1, 2, 3 ],
 [ 4, 5, 6 ],
 [ 7, 8, 9 ]
 ]
 Output:  [1,2,4,7,5,3,6,8,9]

 Note:

 The total number of elements of the given matrix will not exceed 10,000.

 */
public class _498 {

public static class Solutoin1 {
    /**
     * Reference: https://discuss.leetcode.com/topic/77865/concise-java-solution/2
     * Just keep walking the matrix, when hitting the four borders (top, bottom, left or
right),
     * just directions and keep walking.
     */
    public int[] findDiagonalOrder(int[][] matrix) {

        if (matrix == null || matrix.length == 0) {
            return new int[0];
        }
        int m = matrix.length;
        int n = matrix[0].length;
        int[] result = new int[m * n];
        int d = 1;
        int i = 0;
        int j = 0;
        for (int k = 0; k < m * n; ) {
            result[k++] = matrix[i][j];
            i -= d;
            j += d;

            if (i >= m) {
                i = m - 1;
                j += 2;
                d = -d;
            }
```

```
            if (j >= n) {
                j = n - 1;
                i += 2;
                d = -d;
            }
            if (i < 0) {
                i = 0;
                d = -d;
            }
            if (j < 0) {
                j = 0;
                d = -d;
            }
        }
        return result;
    }
}

}

// ============================================================
// FILE 472: _499.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 499. The Maze III
 *
 * There is a ball in a maze with empty spaces and walls.
 * The ball can go through empty spaces by rolling up (u), down (d), left (l) or right
(r),
 * but it won't stop rolling until hitting a wall.
 *
 * When the ball stops, it could choose the next direction.
 * There is also a hole in this maze. The ball will drop into the hole if it rolls on to
the hole.
 *
 * Given the ball position, the hole position and the maze,
 * find out how the ball could drop into the hole by moving the shortest distance.
 *
 * The distance is defined by the number of empty spaces traveled by the ball from the
start
 * position (excluded) to the hole (included).
 *
 * Output the moving directions by using 'u', 'd', 'l' and 'r'.
 * Since there could be several different shortest ways, you should output the
lexicographically smallest way.
 *
 * If the ball cannot reach the hole, output "impossible".
 * The maze is represented by a binary 2D array.
```

```
 * 1 means the wall and 0 means the empty space.
 * You may assume that the borders of the maze are all walls.
 * The ball and the hole coordinates are represented by row and column indexes.

 Example 1

 Input 1: a maze represented by a 2D array

 0 0 0 0 0
 1 1 0 0 1
 0 0 0 0 0
 0 1 0 0 1
 0 1 0 0 0

 Input 2: ball coordinate (rowBall, colBall) = (4, 3)
 Input 3: hole coordinate (rowHole, colHole) = (0, 1)

 Output: "lul"
 Explanation: There are two shortest ways for the ball to drop into the hole.
 The first way is left -> up -> left, represented by "lul".
 The second way is up -> left, represented by 'ul'.
 Both ways have shortest distance 6, but the first way is lexicographically smaller
because 'l' < 'u'. So the output is "lul".

 Example 2

 Input 1: a maze represented by a 2D array

 0 0 0 0 0
 1 1 0 0 1
 0 0 0 0 0
 0 1 0 0 1
 0 1 0 0 0

 Input 2: ball coordinate (rowBall, colBall) = (4, 3)
 Input 3: hole coordinate (rowHole, colHole) = (3, 0)
 Output: "impossible"
 Explanation: The ball cannot reach the hole.

 Note:
 There is only one ball and one hole in the maze.
 Both the ball and hole exist on an empty space, and they will not be at the same
position initially.
 The given maze does not contain border (like the red rectangle in the example pictures),
but you could assume the border of the maze are all walls.
 The maze contains at least 2 empty spaces, and the width and the height of the maze
won't exceed 30.*/
public class _499 {
    public static class Solutoin1 {
        /**credit: https://discuss.leetcode.com/topic/77474/similar-to-the-maze-ii-easy-
understanding-java-bfs-solution*/

        public String findShortestWay(int[][] maze, int[] ball, int[] hole) {
```

```
            final int[] directions = new int[]{-1, 0, 1, 0, -1};
            Queue<Point> heap = new PriorityQueue<>();
            heap.offer(new Point(ball[0], ball[1], 0, ""));
            int m = maze.length;
            int n = maze[0].length;
            Point[][] points = new Point[m][n];
            for (int i = 0; i < m * n; i++) {
                points[i / n][i % n] = new Point(i / n, i % n);//initialize the length
array
            }
            String[] ds = new String[]{"u", "r", "d", "l"};
            while (!heap.isEmpty()) {
                Point curr = heap.poll();
                if (points[curr.x][curr.y].compareTo(curr) <= 0) {
                    continue;//if we have already found a shorter route
                }
                points[curr.x][curr.y] = curr;
                for (int i = 0; i < directions.length - 1; i++) {
                    int x = curr.x;
                    int y = curr.y;
                    int distance = curr.distance;//use temp variables to move
                    //we need below while loop to find only "stop" points that could be
put into the queue
                    while (x >= 0 && y >= 0 && x < m && y < n && maze[x][y] == 0 && (x !=
hole[0] || y != hole[1])) {
                        x += directions[i];
                        y += directions[i + 1];
                        distance++;
                    }
                    if (x != hole[0] || y != hole[1]) {
                        x -= directions[i];
                        y -= directions[i + 1];
                        distance--;
                    }
                    heap.offer(new Point(x, y, distance, curr.path + ds[i]));
                }
            }
            return points[hole[0]][hole[1]].distance == Integer.MAX_VALUE ? "impossible"
: points[hole[0]][hole[1]].path;
        }

        class Point implements Comparable<Point> {
            int x;
            int y;
            int distance;
            String path;

            public Point(int x, int y) {
                this.x = x;
                this.y = y;
                this.distance = Integer.MAX_VALUE;
                this.path = "";
            }
```

```
            public Point(int x, int y, int distance, String path) {
                this.x = x;
                this.y = y;
                this.distance = distance;
                this.path = path;
            }

            @Override
            public int compareTo(Point o) {
                return (this.distance == o.distance) ? this.path.compareTo(o.path) :
this.distance - o.distance;
            }
        }
    }
}

// ============================================================
// FILE 473: _500.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 500. Keyboard Row
 *
 * Given a List of words, return the words that can be typed using letters of alphabet on
only one row's of American keyboard like the image below.
 *
 * American keyboard
 *
 * Example 1:
 * Input: ["Hello", "Alaska", "Dad", "Peace"]
 * Output: ["Alaska", "Dad"]
 *
 * Note:
 * You may use one character in the keyboard more than once.
 * You may assume the input string will only contain letters of alphabet.
 */
public class _500 {

    public String[] findWords(String[] words) {
        final Set<Character> row1 = new HashSet<>(Arrays.asList('q', 'w', 'e', 'r', 't',
'y', 'u', 'i', 'o', 'p'));
        final Set<Character> row2 = new HashSet<>(Arrays.asList('a', 's', 'd', 'f', 'g',
'h', 'j', 'k', 'l'));
        final Set<Character> row3 = new HashSet<>(Arrays.asList('z', 'x', 'c', 'v', 'b',
'n', 'm'));
        final List<Set<Character>> setList = Arrays.asList(row1, row2, row3);
```

```
        List<String> wordList = new ArrayList<>();
        for (String word : words) {
            String lowerCaseWord = word.toLowerCase();
            boolean can = true;
            for (Set<Character> row : setList) {
                for (char c : lowerCaseWord.toCharArray()) {
                    if (!row.contains(c)) {
                        can = false;
                        break;
                    }
                }
                if (can) {
                    wordList.add(word);
                    break;
                }
                can = true;
            }
        }
        String[] result = new String[wordList.size()];
        for (int i = 0; i < wordList.size(); i++) {
            result[i] = wordList.get(i);
        }
        return result;
    }

}

// ============================================================
// FILE 474: _501.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;

/**
 * 501. Find Mode in Binary Search Tree
 * Given a binary search tree with duplicates. You have to find all the mode(s) in given
binary tree.

 For example:
 Given binary tree [1,null,2,2],
  1
   \
   2
  /
 2
```

```
 return [2].

 Note: If a tree has more than one mode, you can return them in any order.

 */
public class _501 {

    public static class Solution1 {
        public int[] findMode(TreeNode root) {
            int[] result = new int[]{};
            Map<Integer, Integer> map = new HashMap();
            if (root == null) {
                return result;
            }
            List<Integer> list = bfs(root, map);
            result = new int[list.size()];
            for (int i = 0; i < list.size(); i++) {
                result[i] = list.get(i);
            }
            return result;
        }

        private List<Integer> bfs(TreeNode root, Map<Integer, Integer> map) {
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode treeNode = queue.poll();
                    if (treeNode.left != null) {
                        queue.offer(treeNode.left);
                    }
                    if (treeNode.right != null) {
                        queue.offer(treeNode.right);
                    }
                    map.put(treeNode.val, map.getOrDefault(treeNode.val, 0) + 1);
                }
            }

            int highestFrequency = 0;
            List<Integer> list = new ArrayList<>();
            for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
                if (entry.getValue() > highestFrequency) {
                    highestFrequency = entry.getValue();
                    list.clear();
                    list.add(entry.getKey());
                } else if (entry.getValue() == highestFrequency) {
                    list.add(entry.getKey());
                }
            }

            return list;
        }
    }
```

```
    public static class Solution2 {
        public int[] findMode(TreeNode root) {
            Map<Integer, Integer> map = new HashMap<>();
            dfs(root, map);
            int modeCount = 0;
            for (int key : map.keySet()) {
                modeCount = Math.max(modeCount, map.get(key));
            }
            List<Integer> mode = new ArrayList<>();
            for (int key : map.keySet()) {
                if (map.get(key) == modeCount) {
                    mode.add(key);
                }
            }
            int[] result = new int[mode.size()];
            for (int i = 0; i < mode.size(); i++) {
                result[i] = mode.get(i);
            }
            return result;
        }

        private void dfs(TreeNode root, Map<Integer, Integer> map) {
            if (root == null) {
                return;
            }
            dfs(root.left, map);
            map.put(root.val, map.getOrDefault(root.val, 0) + 1);
            dfs(root.right, map);
        }
    }

}

// ============================================================
// FILE 475: _502.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 502. IPO
 *
 * Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares
to Venture Capital, LeetCode would like to work on some projects to increase its capital
before the IPO. Since it has limited resources, it can only finish at most k distinct
projects before the IPO. Help LeetCode design the best way to maximize its total capital
after finishing at most k distinct projects.

 You are given several projects. For each project i, it has a pure profit Pi and a
minimum capital of Ci is needed to start the corresponding project. Initially, you have W
capital. When you finish a project, you will obtain its pure profit and the profit will
```

```
be added to your total capital.

 To sum up, pick a list of at most k distinct projects from given projects to maximize
your final capital, and output your final maximized capital.

 Example 1:
 Input: k=2, W=0, Profits=[1,2,3], Capital=[0,1,1].

 Output: 4

 Explanation: Since your initial capital is 0, you can only start the project indexed 0.
 After finishing it you will obtain profit 1 and your capital becomes 1.
 With capital 1, you can either start the project indexed 1 or the project indexed 2.
 Since you can choose at most 2 projects, you need to finish the project indexed 2 to get
the maximum capital.
 Therefore, output the final maximized capital, which is 0 + 1 + 3 = 4.
 Note:
 You may assume all numbers in the input are non-negative integers.
 The length of Profits array and Capital array will not exceed 50,000.
 The answer is guaranteed to fit in a 32-bit signed integer.

 */
public class _502 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/77768/very-simple-greedy-java-
solution-using-two-priorityqueues
         */
        public int findMaximizedCapital(int k, int W, int[] Profits, int[] Capital) {
            PriorityQueue<int[]> capitalHeap = new PriorityQueue<>((a, b) -> a[0] -
b[0]);
            PriorityQueue<int[]> profitHeap = new PriorityQueue<>((a, b) -> b[1] - a[1]);
            for (int i = 0; i < Profits.length; i++) {
                capitalHeap.add(new int[]{Capital[i], Profits[i]});
            }
            while (k-- > 0) {
                while (!capitalHeap.isEmpty() && capitalHeap.peek()[0] <= W) {
                    profitHeap.add(capitalHeap.poll());
                }
                if (profitHeap.isEmpty()) {
                    break;
                }
                W += profitHeap.poll()[1];
            }
            return W;
        }
    }

}

// ============================================================
// FILE 476: _503.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 503. Next Greater Element II
 *
 * Given a circular array (the next element of the last element is the first element of
the array),
 * print the Next Greater Number for every element.
 * The Next Greater Number of a number x is the first greater number to its traversing-
order next in the array,
 * which means you could search circularly to find its next greater number. If it doesn't
exist, output -1 for this number.

 Example 1:
 Input: [1,2,1]
 Output: [2,-1,2]
 Explanation: The first 1's next greater number is 2;
 The number 2 can't find next greater number;
 The second 1's next greater number needs to search circularly, which is also 2.
 Note: The length of given array won't exceed 10000.
 */
public class _503 {

    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/77881/typical-ways-to-solve-
circular-array-problems-java-solution
         * Note: we store INDEX into the stack, reversely, the larger index put at the
bottom of the stack, the smaller index at the top
         */
        public int[] nextGreaterElements(int[] nums) {
            if (nums == null || nums.length == 0) {
                return nums;
            }
            int len = nums.length;
            Stack<Integer> stack = new Stack<>();
            for (int i = len - 1; i >= 0; i--) {
                stack.push(i);
                //push all indexes into the stack reversely
            }
            int[] result = new int[len];
            for (int i = len - 1; i >= 0; i--) {
                result[i] = -1;
                //initialize it to be -1 in case we cannot find its next greater element
in the array
                while (!stack.isEmpty() && (nums[stack.peek()] <= nums[i])) {
                    stack.pop();
                }
                if (!stack.isEmpty()) {
                    result[i] = nums[stack.peek()];
                }
```

```
                stack.push(i);
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * credit: https://leetcode.com/articles/next-greater-element-ii/
         */
        public int[] nextGreaterElements(int[] nums) {
            int[] result = new int[nums.length];
            Stack<Integer> stack = new Stack<>();
            for (int i = nums.length * 2 - 1; i >= 0; i--) {
                while (!stack.isEmpty() && nums[stack.peek()] <= nums[i % nums.length]) {
                    stack.pop();
                }
                result[i % nums.length] = stack.isEmpty() ? -1 : nums[stack.peek()];
                stack.push(i % nums.length);
            }
            return result;
        }
    }

}

// ============================================================
// FILE 477: _504.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 504. Base 7
 *
 * Given an integer, return its base 7 string representation.
 Example 1:
 Input: 100
 Output: "202"

 Example 2:
 Input: -7
 Output: "-10"

 Note: The input will be in range of [-1e7, 1e7].
 */
public class _504 {

    public static class Solution1 {
        public String convertToBase7(int num) {
            return String.valueOf(Integer.toString(num, 7));
        }
    }
}
```

```
// ============================================================
// FILE 478: _505.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 505. The Maze II
 *
 * There is a ball in a maze with empty spaces and walls. The ball can go through empty spaces by rolling up, down, left or right,
 * but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.
 * Given the ball's start position, the destination and the maze, find the shortest distance for the ball to stop at the destination. The distance is defined by the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included). If the ball cannot stop at the destination, return -1.
 * The maze is represented by a binary 2D array. 1 means the wall and 0 means the empty space. You may assume that the borders of the maze are all walls. The start and destination coordinates are represented by row and column indexes.

 Example 1

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
 Input 3: destination coordinate (rowDest, colDest) = (4, 4)

 Output: 12
 Explanation: One shortest way is : left -> down -> left -> down -> right -> down -> right.
 The total distance is 1 + 1 + 3 + 1 + 2 + 2 + 2 = 12.

 Example 2

 Input 1: a maze represented by a 2D array

 0 0 1 0 0
 0 0 0 0 0
 0 0 0 1 0
 1 1 0 1 1
 0 0 0 0 0

 Input 2: start coordinate (rowStart, colStart) = (0, 4)
```

```
 Input 3: destination coordinate (rowDest, colDest) = (3, 2)

 Output: -1
 Explanation: There is no way for the ball to stop at the destination.

 Note:
 There is only one ball and one destination in the maze.
 Both the ball and the destination exist on an empty space, and they will not be at the
same position initially.
 The given maze does not contain border (like the red rectangle in the example pictures),
but you could assume the border of the maze are all walls.
 The maze contains at least 2 empty spaces, and both the width and height of the maze
won't exceed 100.
 */
public class _505 {

    public static class Solution1 {
        /**
         * The difference between II and I of this problem:
         * the extra array is not boolean type any more, but int type, and it's recording
the length of each point to start point.
         */
        public int shortestDistance(int[][] maze, int[] start, int[] destination) {

            final int[] directions = new int[]{-1, 0, 1, 0, -1};
            Queue<Point> queue = new LinkedList<>();
            queue.offer(new Point(start[0], start[1], 0));
            int m = maze.length;
            int n = maze[0].length;
            int[][] length = new int[m][n];
            for (int i = 0; i < m * n; i++) {
                length[i / n][i % n] = Integer.MAX_VALUE;//initialize the length array
            }

            while (!queue.isEmpty()) {
                Point curr = queue.poll();
                if (length[curr.x][curr.y] <= curr.distance) {
                    continue;
                }
                length[curr.x][curr.y] = curr.distance;
                for (int i = 0; i < directions.length - 1; i++) {
                    int x = curr.x;
                    int y = curr.y;
                    int distance = curr.distance;//use temp variables to move
                    //we need below while loop to find only "stop" points that could be
put into the queue
                    while (x >= 0 && y >= 0 && x < m && y < n && maze[x][y] == 0) {
                        x += directions[i];
                        y += directions[i + 1];
                        distance++;
                    }
                    x -= directions[i];
                    y -= directions[i + 1];
                    distance--;
```

```
                    queue.offer(new Point(x, y, distance));
                }
            }
            return length[destination[0]][destination[1]] == Integer.MAX_VALUE ? -1 :
length[destination[0]][destination[1]];

        }

        class Point {
            int x;
            int y;
            int distance;

            public Point(int x, int y, int distance) {
                this.x = x;
                this.y = y;
                this.distance = distance;
            }
        }
    }
}

// ============================================================
// FILE 479: _506.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 506. Relative Ranks
 *
 * Given scores of N athletes, find their relative ranks and the people with the top
three highest scores,
 * who will be awarded medals: "Gold Medal", "Silver Medal" and "Bronze Medal".

 Example 1:
 Input: [5, 4, 3, 2, 1]
 Output: ["Gold Medal", "Silver Medal", "Bronze Medal", "4", "5"]
 Explanation: The first three athletes got the top three highest scores, so they got
"Gold Medal", "Silver Medal" and "Bronze Medal".
 For the left two athletes, you just need to output their relative ranks according to
their scores.

 Note:
 N is a positive integer and won't exceed 10,000.
 All the scores of athletes are guaranteed to be unique.
 */
public class _506 {

    public static class Solution1 {
```

```
        public String[] findRelativeRanks(int[] nums) {
            int[] tmp = new int[nums.length];
            for (int i = 0; i < nums.length; i++) {
                tmp[i] = nums[i];
            }
            Arrays.sort(tmp);
            Map<Integer, String> rankMap = new HashMap();
            int len = nums.length;
            for (int i = len - 1; i >= 0; i--) {
                if (i == len - 1) {
                    rankMap.put(tmp[i], "Gold Medal");
                } else if (i == len - 2) {
                    rankMap.put(tmp[i], "Silver Medal");
                } else if (i == len - 3) {
                    rankMap.put(tmp[i], "Bronze Medal");
                } else {
                    rankMap.put(tmp[i], String.valueOf(len - i));
                }
            }
            String[] result = new String[len];
            for (int i = 0; i < len; i++) {
                result[i] = rankMap.get(nums[i]);
            }
            return result;
        }
    }

}

// ============================================================
// FILE 480: _507.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 507. Perfect Number
 *
 * We define the Perfect Number is a positive integer that is equal to the sum of all its
positive divisors except itself.
 * Now, given an integer n, write a function that returns true when it is a perfect
number and false when it is not.

 Example:
 Input: 28
 Output: True
 Explanation: 28 = 1 + 2 + 4 + 7 + 14

 Note: The input number n will not exceed 100,000,000. (1e8)
 */
public class _507 {

    public static class Solution1 {
        public boolean checkPerfectNumber(int num) {
```

```
            if (num == 1) {
                return false;
            }
            int sum = 0;
            for (int i = 2; i <= Math.sqrt(num); i++) {
                if (num % i == 0) {
                    sum += i;
                    if (i != num / i) {
                        sum += num / i;
                    }
                }
            }
            sum++;
            return sum == num;
        }
    }
}

// ============================================================
// FILE 481: _508.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;

/**
 * 508. Most Frequent Subtree Sum
 *
 * Given the root of a tree, you are asked to find the most frequent subtree sum.
 * The subtree sum of a node is defined as the sum of all the node values formed by the
subtree rooted at that node (including the node itself).
 * So what is the most frequent subtree sum value? If there is a tie, return all the
values with the highest frequency in any order.

 Examples 1
 Input:
    5
  /  \
 2   -3
 return [2, -3, 4], since all the values happen only once, return all of them in any
order.

 Examples 2
 Input:

    5
```

```
  /  \
 2   -5
 return [2], since 2 happens twice, however -5 only occur once.

 Note: You may assume the sum of values in any subtree is in the range of 32-bit signed
integer.
 */
public class _508 {

    public static class Solution1 {
        //my purely original but verbose solution
        public int[] findFrequentTreeSum(TreeNode root) {
            if (root == null) {
                return new int[]{};
            }

            Map<TreeNode, Integer> map = new HashMap();
            postOrder(root, map);

            Map<Integer, Integer> frequencyMap = new HashMap<>();
            for (Map.Entry entry : map.entrySet()) {
                frequencyMap.put((Integer) entry.getValue(),
frequencyMap.getOrDefault(entry.getValue(), 0) + 1);
            }

            List<Map.Entry<Integer, Integer>> list = new LinkedList<>
(frequencyMap.entrySet());
            Collections.sort(list, (o1, o2) -> (o2.getValue()).compareTo(o1.getValue()));

            int mostFrequency = list.get(0).getValue();
            List<Integer> topFrequencyList = new ArrayList<>();
            topFrequencyList.add(list.get(0).getKey());
            int i = 1;
            while (i < list.size() && list.get(i).getValue() == mostFrequency) {
                topFrequencyList.add(list.get(i).getKey());
                i++;
            }

            int[] result = new int[topFrequencyList.size()];
            for (int j = 0; j < topFrequencyList.size(); j++) {
                result[j] = topFrequencyList.get(j);
            }

            return result;
        }

        private int postOrder(TreeNode root, Map<TreeNode, Integer> map) {
            int left = 0;
            int right = 0;
            if (root.left != null) {
                left = postOrder(root.left, map);
            }
            if (root.right != null) {
                right = postOrder(root.right, map);
```

```
            }
            if (root.left == null && root.right == null) {
                map.put(root, root.val);
                return root.val;
            }
            int sum = left + right + root.val;
            map.put(root, sum);
            return sum;
        }
    }

    public static class Solution2 {
        //my 2nd purely original but verbose solution
        public int[] findFrequentTreeSum(TreeNode root) {
            Map<Integer, Integer> map = new HashMap<>();
            dfs(root, map);
            List<Map.Entry<Integer, Integer>> entryList = new ArrayList<>
(map.entrySet());
            Collections.sort(entryList, (a, b) -> b.getValue() - a.getValue());
            List<Integer> list = new ArrayList<>();
            for (int i = 0; i < entryList.size(); i++) {
                if (list.size() == 0) {
                    list.add(entryList.get(i).getKey());
                } else {
                    if (map.get(list.get(0)) == entryList.get(i).getValue()) {
                        list.add(entryList.get(i).getKey());
                    } else {
                        break;
                    }
                }
            }
            int[] result = new int[list.size()];
            for (int i = 0; i < list.size(); i++) {
                result[i] = list.get(i);
            }
            return result;
        }

        private int dfs(TreeNode root, Map<Integer, Integer> map) {
            if (root == null) {
                return 0;
            }
            if (root.left == null && root.right == null) {
                map.put(root.val, map.getOrDefault(root.val, 0) + 1);
                return root.val;
            }
            int leftVal = 0;
            if (root.left != null) {
                leftVal = dfs(root.left, map);
            }
            int rightVal = 0;
            if (root.right != null) {
                rightVal = dfs(root.right, map);
            }
```

```
            int val = leftVal + rightVal + root.val;
            map.put(val, map.getOrDefault(val, 0) + 1);
            return val;
        }
    }

    //a more concise and space-efficient solution:
https://discuss.leetcode.com/topic/77775/verbose-java-solution-postorder-traverse-
hashmap-18ms
    //the key difference between the above post and my original solution is that it's
using Frequency as the key of the HashMap
}

// ============================================================
// FILE 482: _509.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 509. Fibonacci Number
 *
 * The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci
sequence,
 * such that each number is the sum of the two preceding ones, starting from 0 and 1.
That is,
 *
 * F(0) = 0,   F(1) = 1
 * F(N) = F(N - 1) + F(N - 2), for N > 1.
 * Given N, calculate F(N).
 *
 * Example 1:
 * Input: 2
 * Output: 1
 * Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.
 *
 * Example 2:
 * Input: 3
 * Output: 2
 * Explanation: F(3) = F(2) + F(1) = 1 + 1 = 2.
 *
 * Example 3:
 * Input: 4
 * Output: 3
 * Explanation: F(4) = F(3) + F(2) = 2 + 1 = 3.
 *
 * Note:
 * 0 ≤ N ≤ 30.
 */
public class _509 {
  public static class Solution1 {
```

```
    /**
     * Time: O(n)
     * Space: O(n)
     */
    public int fib(int N) {
      List<Integer> list = new ArrayList<>();
      list.add(0);
      list.add(1);
      for (int i = 2; i <= N; i++) {
        list.add(list.get(i - 1) + list.get(i - 2));
      }
      return list.get(N);
    }
  }

  public static class Solution2 {
    /**
     * Time: O(n)
     * Space: O(n)
     */
    public int fib(int N) {
      if (N < 2) {
        return N;
      }
      int[] fibNums = new int[N + 1];
      fibNums[0] = 0;
      fibNums[1] = 1;
      for (int i = 2; i <= N; i++) {
        fibNums[i] = fibNums[i - 1] + fibNums[i - 2];
      }
      return fibNums[N];
    }
  }

  public static class Solution3 {
    /**
     * Time: O(n)
     * Space: O(1)
     */
    public int fib(int N) {
      if (N < 2) {
        return N;
      }
      int a = 0;
      int b = 1;
      while (N-- > 1) {
        int sum = a + b;
        a = b;
        b = sum;
      }
      return b;
    }
  }
}
```

```
// ============================================================
// FILE 483: _513.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 513. Find Bottom Left Tree Value
 *
 * Given a binary tree, find the leftmost value in the last row of the tree.

 Example 1:
 Input:

   2
  / \
 1   3

 Output:
 1
 Example 2:
 Input:


      1
     / \
    2   3
   /   / \
  4   5   6
     /
    7

 Output:
 7
 Note: You may assume the tree (i.e., the given root node) is not NULL.
 */
public class _513 {

    public static class Solution1 {
        public int findBottomLeftValue(TreeNode root) {
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            TreeNode leftMost = root;
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (i == 0) {
```

```
                        leftMost = curr;
                    }
                    if (curr.left != null) {
                        queue.offer(curr.left);
                    }
                    if (curr.right != null) {
                        queue.offer(curr.right);
                    }
                }
            }
            return leftMost.val;
        }
    }
}

// ============================================================
// FILE 484: _514.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 514. Freedom Trail
 *
 * In the video game Fallout 4, the quest "Road to Freedom" requires players to reach a
metal dial called
 * the "Freedom Trail Ring", and use the dial to spell a specific keyword in order to
open the door.

 Given a string ring, which represents the code engraved on the outer ring and another
string key,
 which represents the keyword needs to be spelled.
 You need to find the minimum number of steps in order to spell all the characters in the
keyword.

 Initially, the first character of the ring is aligned at 12:00 direction.
 You need to spell all the characters in the string key one by one by
 rotating the ring clockwise or anticlockwise to make each character of the string key
 aligned at 12:00 direction and then by pressing the center button.

 At the stage of rotating the ring to spell the key character key[i]:
 You can rotate the ring clockwise or anticlockwise one place, which counts as 1 step.
 The final purpose of the rotation is to align one of the string ring's characters at the
12:00 direction,
 where this character must equal to the character key[i].
 If the character key[i] has been aligned at the 12:00 direction,
 you need to press the center button to spell, which also counts as 1 step.
 After the pressing, you could begin to spell the next character in the key (next stage),
otherwise, you've finished all the spelling.

 Example:
 Input: ring = "godding", key = "gd"
 Output: 4
 Explanation:
```

```
 For the first key character 'g', since it is already in place, we just need 1 step to
spell this character.
 For the second key character 'd', we need to rotate the ring "godding" anticlockwise by
two steps to make it become "ddinggo".
 Also, we need 1 more step for spelling.
 So the final output is 4.

 Note:
 Length of both ring and key will be in range 1 to 100.
 There are only lowercase letters in both strings and might be some duplcate characters
in both strings.
 It's guaranteed that string key could always be spelled by rotating the string ring.
 */
public class _514 {

    public static class Solution1 {
        public int findRotateSteps(String ring, String key) {
            int n = ring.length();
            int m = key.length();
            int[][] dp = new int[m + 1][n];
            for (int i = m - 1; i >= 0; i--) {
                for (int j = 0; j < n; j++) {
                    dp[i][j] = Integer.MAX_VALUE;
                    for (int k = 0; k < n; k++) {
                        if (ring.charAt(k) == key.charAt(i)) {
                            int diff = Math.abs(j - k);
                            int step = Math.min(diff, n - diff);
                            dp[i][j] = Math.min(dp[i][j], step + dp[i + 1][k]);
                        }
                    }
                }
            }
            return dp[0][0] + m;
        }
    }
}

// ============================================================
// FILE 485: _515.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 515. Find Largest Value in Each Tree Row
 *
 * You need to find the largest value in each row of a binary tree.
```

```
 Example:
 Input:

      1
     / \
    3   2
   / \   \
  5  3   9

 Output: [1, 3, 9]
 */
public class _515 {

    public static class Solution1 {
        public List<Integer> largestValues(TreeNode root) {
            List<Integer> list = new ArrayList<>();
            Queue<TreeNode> queue = new LinkedList<>();
            if (root != null) {
                queue.offer(root);
                while (!queue.isEmpty()) {
                    int max = Integer.MIN_VALUE;
                    int size = queue.size();
                    for (int i = 0; i < size; i++) {
                        TreeNode curr = queue.poll();
                        max = Math.max(max, curr.val);
                        if (curr.left != null) {
                            queue.offer(curr.left);
                        }
                        if (curr.right != null) {
                            queue.offer(curr.right);
                        }
                    }
                    list.add(max);
                }
            }
            return list;
        }
    }

    public static class Solution2 {
        public List<Integer> largestValues(TreeNode root) {
            List<Integer> res = new ArrayList<>();
            if (root == null) {
                return res;
            }
            dfs(root, res, 0);
            return res;

        }

        public void dfs(TreeNode root, List<Integer> res, int level) {
            if (root == null) {
                return;
```

```
            }
            if (level == res.size()) {
                res.add(root.val);
            }
            res.set(level, Math.max(res.get(level), root.val));
            dfs(root.left, res, level + 1);
            dfs(root.right, res, level + 1);
        }
    }

}

// ============================================================
// FILE 486: _516.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 516. Longest Palindromic Subsequence
 *
 * Given a string s, find the longest palindromic subsequence's length in s.
 * You may assume that the maximum length of s is 1000.

 Example 1:
 Input:
 "bbbab"
 Output:
 4
 One possible longest palindromic subsequence is "bbbb".

 Example 2:
 Input:
 "cbbd"
 Output:
 2
 One possible longest palindromic subsequence is "bb".
 */
public class _516 {

    public static class Solution1 {
        /**
         * Inspired by https://discuss.leetcode.com/topic/78603/straight-forward-java-dp-
solution
         * dp[i][j] means the longest palindromic subsequence's length of substring(i, j)
         * so, in the end, we return dp[0][s.length() - 1] which means the longest
palindromic subsequence
         * of this whole string.
         */
        public int longestPalindromeSubseq(String s) {
            int[][] dp = new int[s.length()][s.length()];
            for (int i = s.length() - 1; i >= 0; i--) {
                dp[i][i] = 1;//initialization
                for (int j = i + 1; j < s.length(); j++) {
```

```
                    if (s.charAt(i) == s.charAt(j)) {
                        dp[i][j] = dp[i + 1][j - 1] + 2;
                    } else {
                        dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                    }
                }
            }
            return dp[0][s.length() - 1];
        }
    }
}

// ============================================================
// FILE 487: _517.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 517. Super Washing Machines
 *
 * You have n super washing machines on a line. Initially, each washing machine has some dresses or is empty.

 For each move, you could choose any m (1 ? m ? n) washing machines, and pass one dress of each washing machine to one of its adjacent washing machines at the same time .

 Given an integer array representing the number of dresses in each washing machine from left to right on the line, you should find the minimum number of moves to make all the washing machines have the same number of dresses. If it is not possible to do it, return -1.

 Example1

 Input: [1,0,5]

 Output: 3

 Explanation:
 1st move:    1     0 <-- 5    =>    1     1     4
 2nd move:    1 <-- 1 <-- 4    =>    2     1     3
 3rd move:    2     1 <-- 3    =>    2     2     2
 Example2

 Input: [0,3,0]

 Output: 2

 Explanation:
 1st move:    0 <-- 3     0    =>    1     2     0
 2nd move:    1     2 --> 0    =>    1     1     1
 Example3

 Input: [0,2,0]
```

```
 Output: -1

 Explanation:
 It's impossible to make all the three washing machines have the same number of dresses.
 Note:
 The range of n is [1, 10000].
 The range of dresses number in a super washing machine is [0, 1e5].

 */
public class _517 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/79938/super-short-easy-java-o-n-
solution
         */
        public int findMinMoves(int[] machines) {
            int total = 0;
            for (int i : machines) {
                total += i;
            }
            if (total % machines.length != 0) {
                return -1;
            }
            int avg = total / machines.length;
            int cnt = 0;
            int max = 0;
            for (int load : machines) {
                cnt += load - avg; //load-avg is "gain/lose"
                max = Math.max(Math.max(max, Math.abs(cnt)), load - avg);
            }
            return max;
        }
    }
}

// ============================================================
// FILE 488: _520.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 520. Detect Capital
 *
 * Given a word, you need to judge whether the usage of capitals in it is right or not.
 * We define the usage of capitals in a word to be right when one of the following cases
holds:

 All letters in this word are capitals, like "USA".
 All letters in this word are not capitals, like "leetcode".
 Only the first letter in this word is capital if it has more than one letter, like
"Google".
 Otherwise, we define that this word doesn't use capitals in a right way.
```

```
 Example 1:
 Input: "USA"
 Output: True

 Example 2:
 Input: "FlaG"
 Output: False

 Note: The input will be a non-empty word consisting of uppercase and lowercase latin
letters.
 */
public class _520 {
    public static class Solution1 {
        public boolean detectCapitalUse(String word) {
            char[] words = word.toCharArray();
            boolean firstLetterCap = false;
            if (Character.isUpperCase(words[0])) {
                firstLetterCap = true;
            }

            if (firstLetterCap) {
                if (words.length >= 2) {
                    int i = 2;
                    if (Character.isUpperCase(words[1])) {
                        //then all following must be all uppercase
                        while (i < words.length) {
                            if (!Character.isUpperCase(words[i])) {
                                return false;
                            }
                            i++;
                        }
                        return true;
                    } else {
                        //then all following must be all lowercase
                        while (i < words.length) {
                            if (!Character.isLowerCase(words[i])) {
                                return false;
                            }
                            i++;
                        }
                        return true;
                    }
                }
                return true;
            } else {
                //then all following must be all lowercase
                int i = 1;
                while (i < words.length) {
                    if (!Character.isLowerCase(words[i])) {
                        return false;
                    }
                    i++;
                }
```

```
                return true;
            }
        }
    }
}

// ============================================================
// FILE 489: _521.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 521. Longest Uncommon Subsequence I
 *
 * Given a group of two strings, you need to find the longest uncommon subsequence of
this group of two strings.
 * The longest uncommon subsequence is defined as the longest subsequence of one of these
strings and this subsequence should not be any subsequence of the other strings.
 * A subsequence is a sequence that can be derived from one sequence by deleting some
characters without changing the order of the remaining elements.
 * Trivially, any string is a subsequence of itself and an empty string is a subsequence
of any string.
 * The input will be two strings, and the output needs to be the length of the longest
uncommon subsequence.
 * If the longest uncommon subsequence doesn't exist, return -1.

 Example 1:
 Input: "aba", "cdc"
 Output: 3
 Explanation: The longest uncommon subsequence is "aba" (or "cdc"),
 because "aba" is a subsequence of "aba",
 but not a subsequence of any other strings in the group of two strings.

 Note:
 Both strings' lengths will not exceed 100.
 Only letters from a ~ z will appear in input strings.
 */
public class _521 {
    public static class Solution1 {
        /**
         * The gotcha point of this question is:
         * 1. if a and b are identical, then there will be no common subsequence, return
-1
         * 2. else if a and b are of equal length, then any one of them will be a
subsequence of the other string
         * 3. else if a and b are of different length, then the longer one is a required
subsequence because the longer string cannot be a subsequence of the shorter one
         * Or in other words, when a.length() != b.length(), no subsequence of b will be
equal to a, so return Math.max(a.length(), b.length())
         */
        public int findLUSlength(String a, String b) {
            if (a.equals(b)) {
                return -1;
```

```
            }
            return Math.max(a.length(), b.length());
        }
    }
}

// ============================================================
// FILE 490: _522.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.Comparator;

/**
 * 522. Longest Uncommon Subsequence II
 *
 * Given a list of strings, you need to find the longest uncommon subsequence among them.
 * The longest uncommon subsequence is defined as the longest subsequence of one of these
strings and
 * this subsequence should not be any subsequence of the other strings.
 * A subsequence is a sequence that can be derived from one sequence by deleting some
characters
 * without changing the order of the remaining elements.
 * Trivially, any string is a subsequence of itself and an empty string is a subsequence
of any string.
 * The input will be a list of strings, and the output needs to be the length of the
longest uncommon subsequence.
 * If the longest uncommon subsequence doesn't exist, return -1.

 Example 1:
 Input: "aba", "cdc", "eae"
 Output: 3
 Note:

 All the given strings' lengths will not exceed 10.
 The length of the given list will be in the range of [2, 50].
 */
public class _522 {

    public static class Solution1 {
        /**Idea: if there's such a LUS there in the list, it must be one of the strings
in the given list,
        so we'll just go through the list and check if one string is NOT subsequence of
any others, if so, return it, otherwise, return -1*/
        public int findLUSlength(String[] strs) {
            Arrays.sort(strs, new Comparator<String>() {
                @Override
                public int compare(String o1, String o2) {
                    return o2.length() - o1.length();
                }
            });
```

```
            for (int i = 0; i < strs.length; i++) {
                boolean found = true;
                for (int j = 0; j < strs.length; j++) {
                    if (i == j) {
                        continue;
                    } else if (isSubsequence(strs[i], strs[j])) {
                        found = false;
                        break;
                    }
                }
                if (found) {
                    return strs[i].length();
                }
            }
            return -1;
        }

        private boolean isSubsequence(String a, String b) {
            int i = 0;
            for (int j = 0; i < a.length() && j < b.length(); j++) {
                if (a.charAt(i) == b.charAt(j)) {
                    i++;
                }
            }
            return i == a.length();
        }
    }
}

// ============================================================
// FILE 491: _523.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 523. Continuous Subarray Sum
 *
 * Given a list of non-negative numbers and a target integer k,
 * write a function to check if the array has a continuous subarray of size at least 2
 * that sums up to the multiple of k, that is, sums up to n*k where n is also an integer.

 Example 1:
 Input: [23, 2, 4, 6, 7],  k=6
 Output: True
 Explanation: Because [2, 4] is a continuous subarray of size 2 and sums up to 6.

 Example 2:
 Input: [23, 2, 6, 4, 7],  k=6
 Output: True
 Explanation: Because [23, 2, 6, 4, 7] is an continuous subarray of size 5 and sums up to
```

```
42.

 Note:
 The length of the array won't exceed 10,000.
 You may assume the sum of all the numbers is in the range of a signed 32-bit integer.

 */
public class _523 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/80793/java-o-n-time-o-k-space/20
         * "The reason we use modulus is:
         * (a+(n*x))%x is same as (a%x)
         * e.g. in case of the array [23,2,6,4,7] the running sum is [23,25,31,35,42]
         * and the remainders are [5,1,1,5,0].
         * We got reminder 5 at index 0 and at index 3.
         * That means, in between these two indexes we must have added a number which is
multiple of the k.
         * Hope this clarifies your doubt :)"
         */
        public boolean checkSubarraySum(int[] nums, int k) {
            Map<Integer, Integer> map = new HashMap<>();
            map.put(0, -1);
            int sum = 0;
            for (int i = 0; i < nums.length; i++) {
                sum += nums[i];
                if (k != 0) {
                    /**Because if k == 0, sum %= k will throw ArithmeticException.*/
                    sum %= k;
                }
                Integer prev = map.get(sum);
                if (prev != null) {
                    if (i - prev > 1) {
                        /**This makes sure that it has length at least 2*/
                        return true;
                    }
                } else {
                    map.put(sum, i);
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        public boolean checkSubarraySum(int[] nums, int k) {
            if (nums == null || nums.length == 0) {
                return false;
            }

            //Two continuous zeroes will form a subarray of length 2 with sum 0, 0*k = 0
will always be true
            for (int i = 0; i < nums.length - 1; i++) {
```

```
                if (nums[i] == 0 && nums[i + 1] == 0) {
                    return true;
                }
            }

            //then k cannot be zero any more
            if (k == 0 || nums.length < 2) {
                return false;
            }

            int[] preSums = new int[nums.length + 1];
            for (int i = 1; i <= nums.length; i++) {
                preSums[i] = preSums[i - 1] + nums[i - 1];
            }

            for (int i = 1; i <= nums.length; i++) {
                for (int j = 0; j < i - 1; j++) {
                    if ((preSums[i] - preSums[j]) % k == 0) {
                        return true;
                    }
                }
            }
            return false;
        }
    }

}

// ============================================================
// FILE 492: _524.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Collections;
import java.util.List;

/**
 * 524. Longest Word in Dictionary through Deleting
 *
 * Given a string and a string dictionary, find the longest string in the dictionary that
 * can be formed by deleting some characters of the given string.
 * If there are more than one possible results,
 * return the longest word with the smallest lexicographical order.
 * If there is no possible result, return the empty string.

 Example 1:
 Input:
 s = "abpcplea", d = ["ale","apple","monkey","plea"]

 Output:
 "apple"
 Example 2:
 Input:
```

```
 s = "abpcplea", d = ["a","b","c"]

 Output:
 "a"

 Note:
 All the strings in the input will only contain lower-case letters.
 The size of the dictionary won't exceed 1,000.
 The length of all the strings in the input won't exceed 1,000.
 */
public class _524 {

    public static class Solution1 {
        public String findLongestWord(String s, List<String> d) {
            Collections.sort(d, (a, b) -> a.length() == b.length() ? a.compareTo(b) :
b.length() - a.length());
            for (String dictWord : d) {
                int i = 0;
                for (char c : s.toCharArray()) {
                    if (i < dictWord.length() && dictWord.charAt(i) == c) {
                        i++;
                    }
                }
                if (i == dictWord.length()) {
                    return dictWord;
                }
            }
            return "";
        }
    }

}

// ============================================================
// FILE 493: _525.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 525. Contiguous Array
 *
 * Given a binary array, find the maximum length of a contiguous subarray with equal
number of 0 and 1.

 Example 1:
 Input: [0,1]
 Output: 2
 Explanation: [0, 1] is the longest contiguous subarray with equal number of 0 and 1.

 Example 2:
```

```
 Input: [0,1,0]
 Output: 2
 Explanation: [0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0
and 1.

 Note: The length of the given binary array will not exceed 50,000.
 */
public class _525 {

    public static class Solution1 {
        //credit: https://leetcode.com/articles/contiguous-array/#approach-3-using-
hashmap-accepted
        public int findMaxLength(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            int count = 0;
            int max = 0;
            Map<Integer, Integer> map = new HashMap();
            map.put(0, -1);//initialize the map, which means at index zero, the length of
contiguous subarray is -1
            for (int i = 0; i < nums.length; i++) {
                count += nums[i] == 1 ? 1 : -1;
                if (map.containsKey(count)) {
                    max = Math.max(i - map.get(count), max);
                } else {
                    map.put(count, i);//only when the map does not have this key, we put
it in the map, this avoids overwriting the map
                    //also, it helps us keep the most leftside value in the map to help
us compute the longest contigous array length
                }
            }
            return max;
        }
    }
}

// ================================================================
// FILE 494: _526.java
// ================================================================

package com.fishercoder.solutions;

/**
 * 526. Beautiful Arrangement
 *
 * Suppose you have N integers from 1 to N.
 * We define a beautiful arrangement as an array that is constructed by these N numbers
successfully
 * if one of the following is true for the ith position (1 ≤ i ≤ N) in this array:
 * The number at the ith position is divisible by i.
 * i is divisible by the number at the ith position.
 * Now given N, how many beautiful arrangements can you construct?
```

```
 Example 1:

 Input: 2
 Output: 2

 Explanation:

 The first beautiful arrangement is [1, 2]:
 Number at the 1st position (i=1) is 1, and 1 is divisible by i (i=1).
 Number at the 2nd position (i=2) is 2, and 2 is divisible by i (i=2).

 The second beautiful arrangement is [2, 1]:
 Number at the 1st position (i=1) is 2, and 2 is divisible by i (i=1).
 Number at the 2nd position (i=2) is 1, and i (i=2) is divisible by 1.

 Note:
 N is a positive integer and will not exceed 15.
 */
public class _526 {
        public static class Solution1 {
                /**
                 * A good post to look at: https://discuss.leetcode.com/topic/79916/java-
solution-backtracking
                 * and there's a generic template afterwards for backtracking problems
                 */

                int count = 0;

                public int countArrangement(int N) {
                        backtracking(N, new int[N + 1], 1);
                        return count;
                }

                private void backtracking(int N, int[] used, int pos) {
                        if (pos > N) {
                                count++;
                                return;
                        }

                        for (int i = 1; i <= N; i++) {
                                if (used[i] == 0 && (i % pos == 0 || pos % i == 0)) {
                                        used[i] = 1;
                                        backtracking(N, used, pos + 1);
                                        used[i] = 0;
                                }
                        }
                }
        }

}

// ============================================================
// FILE 495: _527.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.List;

/**
 * 527. Word Abbreviation
 *
 * Given an array of n distinct non-empty strings, you need to generate minimal possible
abbreviations for every word following rules below.

 Begin with the first character and then the number of characters abbreviated, which
followed by the last character.
 If there are any conflict, that is more than one words share the same abbreviation,
 a longer prefix is used instead of only the first character until making the map
 from word to abbreviation become unique.
 In other words, a final abbreviation cannot map to more than one original words.
 If the abbreviation doesn't make the word shorter, then keep it as original.

 Example:
 Input: ["like", "god", "internal", "me", "internet", "interval", "intension", "face",
"intrusion"]
 Output: ["l2e","god","internal","me","i6t","interval","inte4n","f2e","intr4n"]

 Note:
 Both n and the length of each word will not exceed 400.
 The length of each word is greater than 1.
 The words consist of lowercase English letters only.
 The return answers should be in the same order as the original array.
 */
public class _527 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/82613/really-simple-and-
straightforward-java-solution
         */
        public List<String> wordsAbbreviation(List<String> dict) {
            int len = dict.size();
            String[] ans = new String[len];
            int[] prefix = new int[len];
            for (int i = 0; i < len; i++) {
                prefix[i] = 1;
                ans[i] = abbreviate(dict.get(i), 1); // make abbreviation for each string
            }
            for (int i = 0; i < len; i++) {
                while (true) {
                    HashSet<Integer> set = new HashSet<>();
                    for (int j = i + 1; j < len; j++) {
                        if (ans[j].equals(ans[i])) {
                            set.add(j); // check all strings with the same abbreviation
                        }
```

```
                    }
                    if (set.isEmpty()) {
                        break;
                    }
                    set.add(i);
                    for (int k : set) {
                        ans[k] = abbreviate(dict.get(k), ++prefix[k]); // increase the
prefix
                    }
                }
            }
            return Arrays.asList(ans);
        }

        private String abbreviate(String word, int k) {
            if (k + 2 >= word.length()) {
                return word;
            }
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.append(word.substring(0, k));
            stringBuilder.append(word.length() - 1 - k);
            stringBuilder.append(word.substring(word.length() - 1));
            return stringBuilder.toString();
        }
    }

}

// ============================================================
// FILE 496: _529.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 529. Minesweeper
 *
 * Let's play the minesweeper game (Wikipedia, online game)!

 You are given a 2D char matrix representing the game board. 'm' represents an unrevealed
mine, 'E' represents an unrevealed empty square, 'B' represents a revealed blank square
that has no adjacent (above, below, left, right, and all 4 diagonals) mines, digit ('1'
to '8') represents how many mines are adjacent to this revealed square, and finally 'X'
represents a revealed mine.

 Now given the next click position (row and column indices) among all the unrevealed
squares ('m' or 'E'), return the board after revealing this position according to the
following rules:

 If a mine ('m') is revealed, then the game is over - change it to 'X'.
 If an empty square ('E') with no adjacent mines is revealed, then change it to revealed
```

```
blank ('B') and all of its adjacent unrevealed squares should be revealed recursively.
 If an empty square ('E') with at least one adjacent mine is revealed, then change it to
a digit ('1' to '8') representing the number of adjacent mines.
 Return the board when no more squares will be revealed.

 Example 1:
 Input:

 [['E', 'E', 'E', 'E', 'E'],
 ['E', 'E', 'm', 'E', 'E'],
 ['E', 'E', 'E', 'E', 'E'],
 ['E', 'E', 'E', 'E', 'E']]

 Click : [3,0]

 Output:

 [['B', '1', 'E', '1', 'B'],
 ['B', '1', 'm', '1', 'B'],
 ['B', '1', '1', '1', 'B'],
 ['B', 'B', 'B', 'B', 'B']]

 Explanation:


 Example 2:
 Input:

 [['B', '1', 'E', '1', 'B'],
 ['B', '1', 'm', '1', 'B'],
 ['B', '1', '1', '1', 'B'],
 ['B', 'B', 'B', 'B', 'B']]

 Click : [1,2]

 Output:

 [['B', '1', 'E', '1', 'B'],
 ['B', '1', 'X', '1', 'B'],
 ['B', '1', '1', '1', 'B'],
 ['B', 'B', 'B', 'B', 'B']]

 Explanation:

 Note:
 The range of the input matrix's height and width is [1,50].
 The click position will only be an unrevealed square ('m' or 'E'), which also means the
input board contains at least one clickable square.
 The input board won't be a stage when game is over (some mines have been revealed).
 For simplicity, not mentioned rules should be ignored in this problem. For example, you
don't need to reveal all the unrevealed mines when the game is over, consider any cases
that you will win the game or flag any squares.
 */
public class _529 {
```

```
    public static class Solution1 {
        public char[][] updateBoard(char[][] board, int[] click) {
            int m = board.length;
            int n = board[0].length;
            Queue<int[]> queue = new LinkedList();
            queue.offer(click);
            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                int currRow = curr[0];
                int currCol = curr[1];
                if (board[currRow][currCol] == 'M') {
                    /**This also covers the corner case: when click[] happens to be on a
mine, then it'll exit directly.
                     * Otherwise, we'll just continue and mark this cell to be 'M' and
keep processing all 'E' cells in the queue.*/
                    board[currRow][currCol] = 'X';
                } else {
                    /**scan all four directions of this curr cell, count all mines, this
includes 'X' and 'M' */
                    int count = 0;
                    for (int i = -1; i < 2; i++) {
                        for (int j = -1; j < 2; j++) {
                            if (i == 0 && j == 0) {
                                continue;
                            }
                            int nextRow = currRow + i;
                            int nextCol = currCol + j;
                            if (nextRow >= m || nextRow < 0 || nextCol >= n || nextCol <
0) {
                                continue;
                            }
                            if (board[nextRow][nextCol] == 'M' || board[nextRow][nextCol]
== 'X') {
                                count++;
                            }
                        }
                    }

                    if (count > 0) {
                        /**There are mines around this cell, so update it with the number
of mines*/
                        board[currRow][currCol] = (char) (count + '0');
                    } else {
                        /**There is no mines around this cell, so update it to be 'B'*/
                        board[currRow][currCol] = 'B';

                        /**then we'll also check all of its four surrounding cells, if
it's 'E'. we'll also update it to be 'B' and offer it into the queue*/
                        for (int i = -1; i < 2; i++) {
                            for (int j = -1; j < 2; j++) {
                                if (i == 0 && j == 0) {
                                    continue;
                                }
```

```
                                int nextRow = currRow + i;
                                int nextCol = currCol + j;
                                if (nextRow >= m || nextRow < 0 || nextCol >= n ||
nextCol < 0) {
                                    continue;
                                }
                                if (board[nextRow][nextCol] == 'E') {
                                    /**we offer 'E' cells into the queue*/
                                    queue.offer(new int[]{nextRow, nextCol});
                                    /**then update this cell to be 'B' */
                                    board[nextRow][nextCol] = 'B';
                                }
                            }
                        }
                    }
                }
            }
            return board;
        }
    }
}

// ============================================================
// FILE 497: _530.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.Iterator;
import java.util.TreeSet;

/**
 * 530. Minimum Absolute Difference in BST
 *
 * Given a binary search tree with non-negative values, find the minimum absolute
difference between values of any two nodes.

 Example:

 Input:

   1
    \
     3
    /
   2

 Output:
 1

 Explanation:
 The minimum absolute difference is 1, which is the difference between 2 and 1 (or
```

```
between 2 and 3).
 Note: There are at least two nodes in this BST.
 */
public class _530 {

    public static class Solution1 {
        public int getMinimumDifference(TreeNode root) {
            TreeSet<Integer> treeset = new TreeSet<>();
            treeset.add(root.val);
            dfs(root, treeset);
            int diff = Integer.MAX_VALUE;
            Iterator<Integer> iterator = treeset.iterator();
            int prev = iterator.next();
            while (iterator.hasNext()) {
                int current = iterator.next();
                diff = Math.min(diff, Math.abs(current - prev));
                prev = current;
            }
            return diff;
        }

        private void dfs(TreeNode root, TreeSet<Integer> treeset) {
            if (root.left != null) {
                treeset.add(root.left.val);
                dfs(root.left, treeset);
            }
            if (root.right != null) {
                treeset.add(root.right.val);
                dfs(root.right, treeset);
            }
        }
    }
}

// ============================================================
// FILE 498: _531.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 531. Lonely Pixel I
 *
 * Given a picture consisting of black and white pixels, find the number of black lonely
pixels.
 * The picture is represented by a 2D char array consisting of 'B' and 'W', which means
black and white pixels respectively.
 * A black lonely pixel is character 'B' that located at a specific position where the
same row and same column don't have any other black pixels.

 Example:
 Input:
 [['W', 'W', 'B'],
 ['W', 'B', 'W'],
```

```
 ['B', 'W', 'W']]

 Output: 3
 Explanation: All the three 'B's are black lonely pixels.

 Note:
 The range of width and height of the input 2D array is [1,500].
 */
public class _531 {

    public static class Solution1 {
        public int findLonelyPixel(char[][] picture) {
            int m = picture.length;
            int n = picture[0].length;
            int count = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (picture[i][j] == 'B' && isLonely(i, j, picture, m, n)) {
                        count++;
                    }
                }
            }
            return count;
        }

        private boolean isLonely(int row, int col, char[][] picture, int m, int n) {
            for (int i = 0; i < m; i++) {
                if (i != row) {
                    if (picture[i][col] == 'B') {
                        return false;
                    }
                }
            }

            for (int j = 0; j < n; j++) {
                if (j != col) {
                    if (picture[row][j] == 'B') {
                        return false;
                    }
                }
            }
            return true;
        }
    }
}

// ==========================================================
// FILE 499: _532.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
```

```
/**
 * 532. K-diff Pairs in an Array
 *
 * Given an array of integers and an integer k, you need to find the number of unique k-
diff pairs in the array.
 * Here a k-diff pair is defined as an integer pair (i, j), where i and j are both
numbers in the array and their absolute difference is k.

 Example 1:
 Input: [3, 1, 4, 1, 5], k = 2
 Output: 2
 Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5).
 Although we have two 1s in the input, we should only return the number of unique pairs.

 Example 2:
 Input:[1, 2, 3, 4, 5], k = 1
 Output: 4
 Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4,
5).

 Example 3:
 Input: [1, 3, 1, 5, 4], k = 0
 Output: 1
 Explanation: There is one 0-diff pair in the array, (1, 1).

 Note:
 The pairs (i, j) and (j, i) count as the same pair.
 The length of the array won't exceed 10,000.
 All the integers in the given input belong to the range: [-1e7, 1e7].
 */
public class _532 {

    public static class Solution1 {
        public int findPairs(int[] nums, int k) {
            if (nums == null || nums.length == 0 || k < 0) {
                return 0;
            }

            Map<Integer, Integer> map = new HashMap();
            for (int num : nums) {
                map.put(num, map.getOrDefault(num, 0) + 1);
            }

            int answer = 0;
            for (int key : map.keySet()) {
                if (k == 0) {
                    if (map.get(key) >= 2) {
                        answer++;
                    }
                } else {
                    if (map.containsKey(key + k)) {
                        answer++;
                    }
```

```
                }
            }
            return answer;
        }
    }

}

// ============================================================
// FILE 500: _533.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 533. Lonely Pixel II
 *
 * Given a picture consisting of black and white pixels, and a positive integer N,
 * find the number of black pixels located at some specific row R and column C that align
with all the following rules:

 Row R and column C both contain exactly N black pixels.
 For all rows that have a black pixel at column C, they should be exactly the same as row
R
 The picture is represented by a 2D char array consisting of 'B' and 'W', which means
black and white pixels respectively.

 Example:
 Input:
 [['W', 'B', 'W', 'B', 'B', 'W'],
 ['W', 'B', 'W', 'B', 'B', 'W'],
 ['W', 'B', 'W', 'B', 'B', 'W'],
 ['W', 'W', 'B', 'W', 'B', 'W']]

 N = 3
 Output: 6
 Explanation: All the bold 'B' are the black pixels we need (all 'B's at column 1 and 3).
         0    1    2    3    4    5         column index
 0    [['W', 'B', 'W', 'B', 'B', 'W'],
 1     ['W', 'B', 'W', 'B', 'B', 'W'],
 2     ['W', 'B', 'W', 'B', 'B', 'W'],
 3     ['W', 'W', 'B', 'W', 'B', 'W']]
 row index

 Take 'B' at row R = 0 and column C = 1 as an example:
 Rule 1, row R = 0 and column C = 1 both have exactly N = 3 black pixels.
 Rule 2, the rows have black pixel at column C = 1 are row 0, row 1 and row 2. They are
exactly the same as row R = 0.

 Note:
 The range of width and height of the input 2D array is [1,200].
```

```
 */
public class _533 {
    public static class Solution1 {
        /**
         * Credit: https://discuss.leetcode.com/topic/81686/verbose-java-o-m-n-solution-
hashmap/5
         * This program is very well designed to do things:
         * 1. it scans the entire matrix once, but does two things in this scan:
         * first it records an array of cols that keeps count of how many 'B' are there
in each column;
         * second, at the end of traversing each column, it puts this entire row as the
key into a HashMap
         * when there N number of 'B's in this row.
         * 2. then we could go through the HashMap keyset:
         * if one row has N number of 'B's, we go through this row's each column to see
if any element in this row is 'B' and also that element's column has N 'B's
         */
        public int findBlackPixel(char[][] picture, int N) {
            if (picture == null || picture.length == 0 || picture[0].length == 0) {
                return 0;
            }
            int m = picture.length;
            int n = picture[0].length;
            int[] cols = new int[n];
            Map<String, Integer> map = new HashMap<>();
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < m; i++) {
                int count = 0;
                for (int j = 0; j < n; j++) {
                    if (picture[i][j] == 'B') {
                        count++;
                        cols[j]++;
                    }
                    stringBuilder.append(picture[i][j]);
                }
                if (count == N) {
                    /**we use this entire row string as key for the map*/
                    map.put(stringBuilder.toString(),
map.getOrDefault(stringBuilder.toString(), 0) + 1);
                }
                stringBuilder.setLength(0);
            }

            int answer = 0;
            for (String key : map.keySet()) {
                if (map.get(key) != N) {
                    continue;
                }
                for (int i = 0; i < n; i++) {
                    if (key.charAt(i) == 'B' && cols[i] == N) {
                        answer += N;
                    }
                }
            }
```

```
            return answer;
        }
    }

}

// ============================================================
// FILE 501: _535.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.Random;

/**
 * 535. Encode and Decode TinyURL
 *
 * TinyURL is a URL shortening service where you enter a URL such as
https://leetcode.com/problems/design-tinyurl
 * and it returns a short URL such as http://tinyurl.com/4e9iAk.
 * Design the encode and decode methods for the TinyURL service.
 * There is no restriction on how your encode/decode algorithm should work.
 * You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can
be decoded to the original URL.

 Note: Do not use class member/global/static variables to store states. Your encode and
decode algorithms should be stateless.
 */

public class _535 {

    public static class Solution1 {
        /**
         * Simple counter approach
         * Analysis:
         *      The range of URLs that can be decoded is limited by the range of Integer.
         *      If excessively large number of URLs have to be encoded, after the range
of Integer is exceeded,
         *          integer overflow could lead to overwriting previous URL's encodings.
         *      The length of the URL isn't necessary shorter than incoming URL.
         *      One potential security issue with this problem is that it's very easy to
predict the next code generated,
         *          since this pattern is very easy to be detected.
         */
        public class Codec {
            int i = 0;
            Map<Integer, String> map = new HashMap();
            public static final String PREFIX = "http://tinyurl/";

            public String encode(String longUrl) {
                map.put(i, longUrl);
                return PREFIX + i++;
```

```
            }

            public String decode(String shortUrl) {
                return map.get(Integer.parseInt(shortUrl.substring(PREFIX.length())));
            }
        }
    }

    public static class Solution2 {
        /**
         * Use Java built-in HashCode
         * Analysis:
         *      hashCode() does NOT generate unique codes for different strings,
collision might happen.
         *          As the number of URLs increase, the probability of getting collision
increases which leads to failure.
         */
        public class Codec {
            Map<Integer, String> map = new HashMap<>();
            public static final String PREFIX = "http://tinyurl/";

            // Encodes a URL to a shortened URL.
            public String encode(String longUrl) {
                /**I don't need to create a local variable to cache longUrl.hashCode()
                 * since Java's String cache it already. :) Look at its source code.*/
                map.put(longUrl.hashCode(), longUrl);
                return PREFIX + longUrl.hashCode();
            }

            // Decodes a shortened URL to its original URL.
            public String decode(String shortUrl) {
                return map.get(Integer.parseInt(shortUrl.substring(PREFIX.length())));
            }
        }
    }

    public static class Solution3 {
        /**Use a random number*/
        Map<Integer, String> map = new HashMap<>();
        Random random = new Random();
        public static final String PREFIX = "http://tinyurl/";

        // Encodes a URL to a shortened URL.
        public String encode(String longUrl) {
            int num = random.nextInt(Integer.MAX_VALUE);
            while (map.containsKey(num)) {
                num = random.nextInt(Integer.MAX_VALUE);
            }
            map.put(num, longUrl);
            return PREFIX + num;
        }

        // Decodes a shortened URL to its original URL.
        public String decode(String shortUrl) {
```

```
            return map.get(Integer.parseInt(shortUrl.substring(PREFIX.length())));
        }
    }

    public static class Solution4 {
        /**
         * Use a random but fixed length encoding
         * Analysis:
         *      1. This is the most optimal solution so far.
         *      2. The number of URLs that can be encoded can be as big as Math.pow((10 +
26*2), FIXED_LENGTH)
         *      3. The length of the shortened URL is fixed at a certain length, which
could be a significant reduce for large URLs
         *      4. The performance of this scheme is pretty good, due to much smaller
probability of encountering collision
         *      5. Predicting pattern/encoding isn't possible in this case since random
numbers are used.
         */
        Map<String, String> map = new HashMap<>();
        public static final String PREFIX = "http://tinyurl/";
        public static final int FIXED_LENGTH = 7;
        Random random = new Random();
        String alphabet =
"0123456789abcdefghijklmnopgrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

        // Encodes a URL to a shortened URL.
        public String encode(String longUrl) {
            String shortKey = getRandomFixedLengthKey();
            while (map.containsKey(shortKey)) {
                shortKey = getRandomFixedLengthKey();
            }
            map.put(shortKey, longUrl);
            return PREFIX + shortKey;
        }

        private String getRandomFixedLengthKey() {
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < FIXED_LENGTH; i++) {
                stringBuilder.append(alphabet.charAt(random.nextInt(alphabet.length())));
            }
            return stringBuilder.toString();
        }

        // Decodes a shortened URL to its original URL.
        public String decode(String shortUrl) {
            return map.get(shortUrl.substring(PREFIX.length()));
        }
    }
}

// ============================================================
// FILE 502: _536.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 536. Construct Binary Tree from String
 *
 * You need to construct a binary tree from a string consisting of parenthesis and
integers.
 The whole input represents a binary tree. It contains an integer followed by zero, one
or two pairs of parenthesis.
 The integer represents the root's value and a pair of parenthesis contains a child
binary tree with the same structure.
 You always start to construct the left child node of the parent first if it exists.

 Example:
 Input: "4(2(3)(1))(6(5))"
 Output: return the tree root node representing the following tree:

       4
     /   \
    2     6
   / \   /
  3   1 5

 Note:
 There will only be '(', ')', '-' and '0' ~ '9' in the input string.
 An empty tree is represented by "" instead of "()".
 */

public class _536 {

    public static class Solution1 {
        public TreeNode str2tree(String s) {
            if (s.equals("")) {
                return null;
            }
            int firstParen = s.indexOf("(");
            int val = firstParen == -1 ? Integer.parseInt(s) :
Integer.parseInt(s.substring(0, firstParen));
            TreeNode cur = new TreeNode(val);
            if (firstParen == -1) {
                return cur;
            }
            int start = firstParen;
            int leftParenCount = 0;
            for (int i = start; i < s.length(); i++) {
                if (s.charAt(i) == '(') {
                    leftParenCount++;
                } else if (s.charAt(i) == ')') {
                    leftParenCount--;
```

```
                }
                if (leftParenCount == 0 && start == firstParen) {
                    cur.left = str2tree(s.substring(start + 1, i));
                    start = i + 1;
                } else if (leftParenCount == 0) {
                    cur.right = str2tree(s.substring(start + 1, i));
                }
            }
            return cur;
        }
    }

    public static class Solution2 {
        public TreeNode str2tree(String s) {
            Deque<TreeNode> stack = new ArrayDeque<>();
            for (int i = 0, j = i; i < s.length(); i++, j = i) {
                char c = s.charAt(i);
                if (c == ')') {
                    stack.pop();
                } else if (c >= '0' && c <= '9' || c == '-') {
                    while (i + 1 < s.length() && s.charAt(i + 1) >= '0' && s.charAt(i +
1) <= '9') {
                        i++;
                    }
                    TreeNode curr = new TreeNode(Integer.valueOf(s.substring(j, i + 1)));
                    if (!stack.isEmpty()) {
                        TreeNode parent = stack.peek();
                        if (parent.left != null) {
                            parent.right = curr;
                        } else {
                            parent.left = curr;
                        }
                    }
                    stack.push(curr);
                }
            }
            return stack.isEmpty() ? null : stack.peek();
        }
    }

}

// ============================================================
// FILE 503: _537.java
// ============================================================

package com.fishercoder.solutions;

import java.util.stream.Stream;

/**
 * 537. Complex Number Multiplication
 *
 * Given two strings representing two complex numbers.
```

```
 You need to return a string representing their multiplication. Note i2 = -1 according to
the definition.

 Example 1:
 Input: "1+1i", "1+1i"
 Output: "0+2i"
 Explanation: (1 + i) * (1 + i) = 1 + i2 + 2 * i = 2i, and you need convert it to the
form of 0+2i.

 Example 2:
 Input: "1+-1i", "1+-1i"
 Output: "0+-2i"
 Explanation: (1 - i) * (1 - i) = 1 + i2 - 2 * i = -2i, and you need convert it to the
form of 0+-2i.

 Note:
 The input strings will not have extra blank.
 The input strings will be given in the form of a+bi, where the integer a and b will both
belong to the range of [-100, 100]. And the output should be also in this form.
 */
public class _537 {

    public static class Solution1 {
        public String complexNumberMultiply(String a, String b) {
            String[] part1And2 = a.split("\\+");
            String[] part3And4 = b.split("\\+");
            String product1 = String.valueOf(Integer.parseInt(part1And2[0]) *
Integer.parseInt(part3And4[0]));//this is real number multiplication
            String product2 = multiply(part1And2[0], part3And4[1]);
            String product3 = multiply(part3And4[0], part1And2[1]);
            String product4 = multiplyTwoIs(part3And4[1], part1And2[1]);
            String twoISum = sumTwoI(product2, product3);
            String numberValue = String.valueOf(Integer.valueOf(product1) +
Integer.valueOf(product4));
            return numberValue + "+" + twoISum;
        }

        private String sumTwoI(String product2, String product3) {
            int number2 = Integer.parseInt(product2.substring(0, product2.length() - 1));
            int number3 = Integer.parseInt(product3.substring(0, product3.length() - 1));
            return String.valueOf(number2 + number3) + "i";
        }

        private String multiplyTwoIs(String p, String q) {
            int number1 = Integer.parseInt(p.substring(0, p.length() - 1));
            int number2 = Integer.parseInt(q.substring(0, q.length() - 1));
            int numberProduct = number1 * number2;
            return String.valueOf(-numberProduct);
        }

        private String multiply(String p, String withI) {
            int numberPart = Integer.parseInt(withI.substring(0, withI.length() - 1));
            return String.valueOf(numberPart * Integer.valueOf(p)) + "i";
```

```
        }
    }

    public static class Solution2 {
        /**
         * (a + bi) * (c + di) could become (ac - bd) + (ad + bc)*i
         * Thus, we have the following function
         */
        public String complexNumberMultiply(String a, String b) {
            int[] coefficients1 =
Stream.of(a.split("\\+|i")).mapToInt(Integer::parseInt).toArray();
            int[] coefficients2 =
Stream.of(b.split("\\+|i")).mapToInt(Integer::parseInt).toArray();
            return (coefficients1[0] * coefficients2[0] - coefficients1[1] *
coefficients2[1]) + "+"
                    + (coefficients1[0] * coefficients2[1] + coefficients1[1] *
coefficients2[0] + "i");
        }
    }

}

// ============================================================
// FILE 504: _538.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.TreeMap;

/**
 * 538. Convert BST to Greater Tree
 *
 * Given a Binary Search Tree (BST), convert it to a Greater Tree such that every key of
the original BST is changed
 * to the original key plus sum of all keys greater than the original key in BST.

 Example:

 Input: The root of a Binary Search Tree like this:
     5
   /   \
  2    13

 Output: The root of a Greater Tree like this:
     18
   /   \
  20   13
 */
```

```
public class _538 {

    public static class Solution1 {
        /**
         * Traverse in this order: right -> root -> left
         */
        public TreeNode convertBST(TreeNode root) {
            dfs(root, 0);
            return root;
        }

        private int dfs(TreeNode root, int val) {
            if (root == null) {
                return val;
            }
            root.val += dfs(root.right, val);
            return dfs(root.left, root.val);
        }
    }

    public static class Solution2 {
        //This solution is generic for both BST and regular binary trees
        public TreeNode convertBST(TreeNode root) {
            if (root == null) {
                return root;
            }
            List<Integer> list = new ArrayList<>();
            putNodeToList(list, root);
            Collections.sort(list);
            int[] sums = new int[list.size()];
            sums[list.size() - 1] = 0;
            for (int i = list.size() - 2; i >= 0; i--) {
                sums[i] = sums[i + 1] + list.get(i + 1);
            }
            TreeMap<Integer, Integer> treeMap = new TreeMap<>();
            for (int i = 0; i < list.size(); i++) {
                treeMap.put(list.get(i), sums[i]);
            }
            TreeNode result = new TreeNode(treeMap.get(list.get(0)));
            return generateResultRoot(root, treeMap, result);
        }

        private TreeNode generateResultRoot(TreeNode root, TreeMap<Integer, Integer>
treeMap, TreeNode result) {
            if (root != null) {
                result.val = treeMap.get(root.val) + root.val;
            }
            if (root.left != null) {
                result.left = new TreeNode(0);
                generateResultRoot(root.left, treeMap, result.left);
            }
            if (root.right != null) {
                result.right = new TreeNode(0);
```

```
                generateResultRoot(root.right, treeMap, result.right);
            }
            return result;
        }

        private void putNodeToList(List<Integer> list, TreeNode root) {
            if (root != null) {
                list.add(root.val);
            }
            if (root.left != null) {
                putNodeToList(list, root.left);
            }
            if (root.right != null) {
                putNodeToList(list, root.right);
            }
        }
    }

}

// ============================================================
// FILE 505: _539.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;

/**
 * 539. Minimum Time Difference
 *
 * Given a list of 24-hour clock time points in "Hour:Minutes" format,
 * find the minimum minutes difference between any two time points in the list.

 Example 1:
 Input: ["23:59","00:00"]
 Output: 1

 Note:
 The number of time points in the given list is at least 2 and won't exceed 20000.
 The input time is legal and ranges from 00:00 to 23:59.
 */
public class _539 {

    public static class Soluiton1 {

        public int findMinDifference(List<String> timePoints) {
            /**there are in total 24*60 = 1440 possible time points*/
            final int ALL_POSSIBLE_TIMEPOINTS = 1440;
            boolean[] allTimePoints = new boolean[ALL_POSSIBLE_TIMEPOINTS];
            for (String eachTime : timePoints) {
                String[] timeParts = eachTime.split(":");
                int hour = Integer.valueOf(timeParts[0]);
                int minute = Integer.valueOf(timeParts[1]);
```

```
                int value = hour * 60 + minute;
                if (allTimePoints[value]) {
                    return 0;
                }
                allTimePoints[value] = true;
            }

            int min = Integer.MAX_VALUE;
            int prev = 0;
            int first = Integer.MAX_VALUE;
            int last = Integer.MIN_VALUE;
            for (int i = 0; i < ALL_POSSIBLE_TIMEPOINTS; i++) {
                if (allTimePoints[i]) {
                    if (first != Integer.MAX_VALUE) {
                        min = Math.min(min, i - prev);
                    }
                    first = Math.min(first, i);
                    last = Math.max(last, i);
                    prev = i;
                }
            }
            min = Math.min(min, (ALL_POSSIBLE_TIMEPOINTS - last + first));
            return min;
        }
    }

}

// ============================================================
// FILE 506: _540.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 540. Single Element in a Sorted Array
 *
 * Given a sorted array consisting of only integers where every
 * element appears twice except for one element which appears once.
 * Find this single element that appears only once.

 Example 1:
 Input: [1,1,2,3,3,4,4,8,8]
 Output: 2
 Example 2:
 Input: [3,3,7,7,10,11,11]
 Output: 10
 Note: Your solution should run in O(log n) time and O(1) space.
 */
public class _540 {
    public static class Solution1 {
        public int singleNonDuplicate(int[] nums) {
            int result = 0;
            for (int i = 0; i < nums.length; i++) {
```

```
                result ^= nums[i];
            }
            return result;
        }
    }

    public static class Solution2 {
        public int singleNonDuplicate(int[] nums) {
            int start = 0;
            int end = nums.length - 1;
            while (start < end) {
                int mid = start + (end - start) / 2;
                if (nums[mid] != nums[mid + 1] && nums[mid] != nums[mid - 1]) {
                    return nums[mid];
                } else if (nums[mid] == nums[mid + 1] && mid % 2 == 0) {
                    start = mid + 1;
                } else if (nums[mid] == nums[mid - 1] && mid % 2 == 1) {
                    start = mid + 1;
                } else {
                    end = mid - 1;
                }
            }
            return nums[start];
        }
    }
}

// ============================================================
// FILE 507: _541.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 541. Reverse String II
 *
 * Given a string and an integer k, you need to reverse the first k characters for every
2k characters counting from the start of the string.
 * If there are less than k characters left, reverse all of them.
 * If there are less than 2k but greater than or equal to k characters, then reverse the
first k characters and left the other as original.

 Example:
 Input: s = "abcdefg", k = 2
 Output: "bacdfeg"

 Restrictions:
 The string consists of lower English letters only.
 Length of the given string and k will in the range [1, 10000]
 */
public class _541 {

    public static class Solution1 {
        public String reverseStr(String s, int k) {
```

```
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < s.length(); i = i + 2 * k) {
                if (s.length() >= (i + k)) {
                    stringBuilder.append(new StringBuilder(s.substring(i, i +
k)).reverse());
                } else {
                    stringBuilder.append(new StringBuilder(s.substring(i)).reverse());
                    break;
                }
                if ((i + 2 * k) <= s.length()) {
                    stringBuilder.append(s.substring(i + k, i + 2 * k));
                } else {
                    stringBuilder.append(s.substring(i + k));
                }
            }
            return stringBuilder.toString();
        }
    }
}

// ============================================================
// FILE 508: _542.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;
import java.util.List;

/**
 * 542. 01 Matrix
 *
 * Given a matrix consists of 0 and 1, find the distance of the nearest 0 for each cell.

 The distance between two adjacent cells is 1.
 Example 1:
 Input:

 0 0 0
 0 1 0
 0 0 0
 Output:
 0 0 0
 0 1 0
 0 0 0
 Example 2:
 Input:

 0 0 0
 0 1 0
 1 1 1
 Output:
 0 0 0
```

```
 0 1 0
 1 2 1

 Note:
 The number of elements of the given matrix will not exceed 10,000.
 There are at least one 0 in the given matrix.
 The cells are adjacent in only four directions: up, down, left and right.
 */
public class _542 {

    public static class Solution1 {

        public List<List<Integer>> updateMatrix(List<List<Integer>> matrix) {
            int m = matrix.size();
            int n = matrix.get(0).size();
            Deque<int[]> deque = new LinkedList<>();
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (matrix.get(i).get(j) == 0) {
                        deque.offer(new int[]{i, j});
                    } else {
                        matrix.get(i).set(j, Integer.MAX_VALUE);
                    }
                }
            }

            final int[] dirs = new int[]{0, 1, 0, -1, 0};
            while (!deque.isEmpty()) {
                int[] currentCell = deque.poll();
                for (int i = 0; i < dirs.length - 1; i++) {
                    int nextRow = currentCell[0] + dirs[i];
                    int nextCol = currentCell[1] + dirs[i + 1];
                    if (nextRow < 0 || nextCol < 0 || nextRow >= m || nextCol >= n ||
matrix.get(nextRow).get(nextCol) <= matrix.get(currentCell[0]).get(currentCell[1]) + 1) {
                        continue;
                    }
                    deque.offer(new int[]{nextRow, nextCol});
                    matrix.get(nextRow).set(nextCol,
matrix.get(currentCell[0]).get(currentCell[1]) + 1);
                }
            }
            return matrix;
        }
    }

}

// ============================================================
// FILE 509: _543.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
```

```
/**
 * 543. Diameter of Binary Tree
 *
 * Given a binary tree, you need to compute the length of the diameter of the tree.
 * The diameter of a binary tree is the length of the longest path between any two nodes
in a tree.
 * This path may or may not pass through the root.

 Example:
 Given a binary tree
     1
    / \
   2   3
  / \
 4   5
 Return 3, which is the length of the path [4,2,1,3] or [5,2,1,3].

 Note: The length of path between two nodes is represented by the number of edges between
them.
 */
public class _543 {

    public static class Solution1 {
        /**This is a very great problem for practicing recursion:
         * 1. What dfs() returns is the max height it should pick from either its left or
right subtree, that's
         * what the int return type stands for;
         * 2. And during the recursion, we can keep updating the global variable:
"diameter";
         * 3. When computing length/height of a subtree, we should take the max of its
left and right, then plus one
         * and left height should be like this
         * int left = dfs(root.left);
         * instead of dfs(root.left) + 1;
         * we'll only plus one at the end
         * */
        int diameter = 0;

        public int diameterOfBinaryTree(TreeNode root) {
            dfs(root);
            return diameter;
        }

        private int dfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int left = dfs(root.left);
            int right = dfs(root.right);
            diameter = Math.max(diameter, left + right);
            return Math.max(left, right) + 1;
        }
    }
```

```
}

// ============================================================
// FILE 510: _544.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * During the NBA playoffs, we always arrange the rather strong team to play with the
rather weak team,
 * like make the rank 1 team play with the rank nth team, which is a good strategy to
make the contest more interesting.
 * Now, you're given n teams, you need to output their final contest matches in the form
of a string.
 * The n teams are given in the form of positive integers from 1 to n, which represents
their initial rank.
 * (Rank 1 is the strongest team and Rank n is the weakest team.)
 * We'll use parentheses('(', ')') and commas(',') to represent the contest team pairing
- parentheses('(' , ')')
 * for pairing and commas(',') for partition. During the pairing process in each round,
 * you always need to follow the strategy of making the rather strong one pair with the
rather weak one.

 Example 1:
 Input: 2
 Output: (1,2)

 Explanation:
 Initially, we have the team 1 and the team 2, placed like: 1,2.
 Then we pair the team (1,2) together with '(', ')' and ',', which is the final answer.


 Example 2:
 Input: 4
 Output: ((1,4),(2,3))

 Explanation:
 In the first round, we pair the team 1 and 4, the team 2 and 3 together, as we need to
make the strong team and weak team together.
 And we got (1,4),(2,3).
 In the second round, the winners of (1,4) and (2,3) need to play again to generate the
final winner, so you need to add the paratheses outside them.
 And we got the final answer ((1,4),(2,3)).


 Example 3:
 Input: 8
 Output: (((1,8),(4,5)),((2,7),(3,6)))

 Explanation:
```

```
 First round: (1,8),(2,7),(3,6),(4,5)
 Second round: ((1,8),(4,5)),((2,7),(3,6))
 Third round: (((1,8),(4,5)),((2,7),(3,6)))
 Since the third round will generate the final winner, you need to output the answer
(((1,8),(4,5)),((2,7),(3,6))).

 Note:
 The n is in range [2, 212].
 We ensure that the input n can be converted into the form 2k, where k is a positive
integer.
 */
public class _544 {

    public String findContestMatch(int n) {
        List<String> pairs = new ArrayList<>();
        int left = 1;
        int right = n;
        while (left < right) {
            pairs.add("(" + left + "," + right + ")");
            left++;
            right--;
        }
        if (n == 2) {
            return pairs.get(0);
        }
        return generateFinal(pairs, n / 2);
    }

    private String generateFinal(List<String> pairs, int n) {
        if (n > 2) {
            int size = pairs.size();
            int left = 0;
            int right = size - 1;
            List<String> newPairs = new ArrayList<>();
            while (left < right) {
                String newPair = "(" + pairs.get(left) + "," + pairs.get(right) + ")";
                newPairs.add(newPair);
                left++;
                right--;
            }
            return generateFinal(newPairs, n / 2);
        }
        return "(" + pairs.get(0) + "," + pairs.get(1) + ")";
    }

}

// ============================================================
// FILE 511: _545.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
```

```
import java.util.ArrayList;
import java.util.List;

/**
 * 545. Boundary of Binary Tree
 *
 * Given a binary tree, return the values of its boundary in anti-clockwise direction
starting from root.
 * Boundary includes left boundary, addLeaves, and right boundary in order without
duplicate nodes.
 * Left boundary is defined as the path from root to the left-most node.
 * Right boundary is defined as the path from root to the right-most node.
 * If the root doesn't have left subtree or right subtree,
 * then the root itself is left boundary or right boundary.
 * Note this definition only applies to the input binary tree, and not applies to any
subtrees.
 * The left-most node is defined as a leaf node you could reach when you always firstly
travel
 * to the left subtree if exists. If not, travel to the right subtree.
 * Repeat until you reach a leaf node.
 * The right-most node is also defined by the same way with left and right exchanged.

 Example 1
 Input:
  1
   \
   2
  / \
 3   4

 Ouput:
 [1, 3, 4, 2]

 Explanation:
 The root doesn't have left subtree, so the root itself is left boundary.
 The addLeaves are node 3 and 4.
 The right boundary are node 1,2,4. Note the anti-clockwise direction means you should
output reversed right boundary.
 So order them in anti-clockwise without duplicates and we have [1,3,4,2].


 Example 2
 Input:
     ____1_____
    /          \
   2           3
  / \          /
 4   5        6
    / \      / \
   7  8     9  10

 Ouput:
 [1,2,4,7,8,9,10,6,3]
```

```
 Explanation:
 The left boundary are node 1,2,4. (4 is the left-most node according to definition)
 The addLeaves are node 4,7,8,9,10.
 The right boundary are node 1,3,6,10. (10 is the right-most node).
 So order them in anti-clockwise without duplicate nodes we have [1,2,4,7,8,9,10,6,3].

 */
public class _545 {
    public static class Solution1 {
        public List<Integer> boundaryOfBinaryTree(TreeNode root) {
            List<Integer> nodes = new ArrayList<>();
            if (root == null) {
                return nodes;
            }

            nodes.add(root.val);
            leftBoundary(root.left, nodes);
            addLeaves(root.left, nodes);
            addLeaves(root.right, nodes);
            rightBoundary(root.right, nodes);
            return nodes;
        }

        public void leftBoundary(TreeNode root, List<Integer> nodes) {
            if (root == null || (root.left == null && root.right == null)) {
                /**we don't want to add any LEAVES in leftBoundary and rightBoundary
functions either,
                 * that's why we have the later condition in the if branch.*/
                return;
            }
            nodes.add(root.val);// add BEFORE child visit
            if (root.left == null) {
                leftBoundary(root.right, nodes);
            } else {
                leftBoundary(root.left, nodes);
            }
        }

        public void rightBoundary(TreeNode root, List<Integer> nodes) {
            if (root == null || (root.right == null && root.left == null)) {
                return;
            }
            if (root.right == null) {
                rightBoundary(root.left, nodes);
            } else {
                rightBoundary(root.right, nodes);
            }
            nodes.add(root.val); // add AFTER child visit(reverse)
        }

        public void addLeaves(TreeNode root, List<Integer> nodes) {
            if (root == null) {
                return;
```

```
            }
            if (root.left == null && root.right == null) {
                nodes.add(root.val);
                return;
            }
            addLeaves(root.left, nodes);
            addLeaves(root.right, nodes);
        }
    }

}

// ============================================================
// FILE 512: _546.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 546. Remove Boxes
 *
 * Given several boxes with different colors represented by different positive numbers.
 * You may experience several rounds to remove boxes until there is no box left.
 * Each time you can choose some continuous boxes with the same color (composed of k
boxes, k >= 1), remove them and get k*k points.
 * Find the maximum points you can get.

 Example 1:
 Input:

 [1, 3, 2, 2, 2, 3, 4, 3, 1]
 Output:
 23

 Explanation:
 [1, 3, 2, 2, 2, 3, 4, 3, 1]
 ----> [1, 3, 3, 4, 3, 1] (3*3=9 points)
 ----> [1, 3, 3, 3, 1] (1*1=1 points)
 ----> [1, 1] (3*3=9 points)
 ----> [] (2*2=4 points)
 Note: The number of boxes n would not exceed 100.

 */

public class _546 {
    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/remove-boxes/#approach-2-using-dp-with-
memorizationaccepted
         *
         * For an entry in dp[l][r][k], l represents the starting index of the subarray,
         * r represents the ending index of the subarray
         * and k represents the number of elements similar to the rth element
         * following it which can be combined to obtain the point information to be
```

```
stored in dp[l][r][k].
         */
        public int removeBoxes(int[] boxes) {
            int[][][] dp = new int[100][100][100];
            return calculatePoints(boxes, dp, 0, boxes.length - 1, 0);
        }

        public int calculatePoints(int[] boxes, int[][][] dp, int l, int r, int k) {
            if (l > r) {
                return 0;
            }
            if (dp[l][r][k] != 0) {
                return dp[l][r][k];
            }
            while (r > l && boxes[r] == boxes[r - 1]) {
                r--;
                k++;
            }
            dp[l][r][k] = calculatePoints(boxes, dp, l, r - 1, 0) + (k + 1) * (k + 1);
            for (int i = l; i < r; i++) {
                if (boxes[i] == boxes[r]) {
                    dp[l][r][k] = Math.max(dp[l][r][k],
                            calculatePoints(boxes, dp, l, i, k + 1) +
calculatePoints(boxes, dp, i + 1, r - 1, 0));
                }
            }
            return dp[l][r][k];
        }
    }
}

// ============================================================
// FILE 513: _547.java
// ============================================================

package com.fishercoder.solutions;

/**
 *547. Friend Circles
 *
 *There are N students in a class.
 * Some of them are friends, while some are not. Their friendship is transitive in
nature.
 * For example, if A is a direct friend of B, and B is a direct friend of C, then A is an
indirect friend of C.
 * And we defined a friend circle is a group of students who are direct or indirect
friends.

 Given a N*N matrix m representing the friend relationship between students in the class.
 If m[i][j] = 1, then the ith and jth students are direct friends with each other,
otherwise not.
 And you have to output the total number of friend circles among all the students.

 Example 1:
```

```
 Input:
 [[1,1,0],
 [1,1,0],
 [0,0,1]]
 Output: 2
 Explanation:The 0th and 1st students are direct friends, so they are in a friend circle.
 The 2nd student himself is in a friend circle. So return 2.

 Example 2:
 Input:
 [[1,1,0],
 [1,1,1],
 [0,1,1]]
 Output: 1
 Explanation:The 0th and 1st students are direct friends, the 1st and 2nd students are
direct friends,
 so the 0th and 2nd students are indirect friends. All of them are in the same friend
circle, so return 1.

 Note:
 N is in range [1,200].
 m[i][i] = 1 for all students.
 If m[i][j] = 1, then m[j][i] = 1.
 */
public class _547 {

    public static class Solution1 {
        public int findCircleNum(int[][] M) {
            if (M == null || M.length == 0 || M[0].length == 0) {
                return 0;
            }
            int m = M.length;//number of rows in this matrix
            UnionFind unionFind = new UnionFind(m);
            for (int i = 0; i < m; i++) {
                for (int j = i + 1; j < m; j++) {
                    if (M[i][j] == 1) {
                        unionFind.union(i, j);
                    }
                }
            }
            return unionFind.count;
        }

        class UnionFind {
            int count;
            int[] root;

            public UnionFind(int m) {
                root = new int[m];
                for (int i = 0; i < m; i++) {
                    root[i] = i;
                }
                count = m;
            }
```

```
            public void union(int i, int j) {
                int x = find(root, i);
                int y = find(root, j);
                if (x != y) {
                    count--;
                    root[x] = y;//path compression
                }
            }

            public int find(int[] ids, int i) {
                if (ids[i] == i) {
                    return i;
                }
                return find(ids, ids[i]);
            }
        }
    }

}

// ============================================================
// FILE 514: _548.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 548. Split Array with Equal Sum
 *
 * Given an array with n integers, you need to find if there are triplets (i, j, k) which satisfies following conditions:
 * 0 < i, i + 1 < j, j + 1 < k < n - 1
 * Sum of subarrays (0, i - 1), (i + 1, j - 1), (j + 1, k - 1) and (k + 1, n - 1) should be equal.
 * where we define that subarray (L, R) represents a slice of the original array starting from the element indexed L to the element indexed R.

 Example:
 Input: [1,2,1,2,1,2,1]
 Output: True
 Explanation:
 i = 1, j = 3, k = 5.
 sum(0, i - 1) = sum(0, 0) = 1
 sum(i + 1, j - 1) = sum(2, 2) = 1
 sum(j + 1, k - 1) = sum(4, 4) = 1
 sum(k + 1, n - 1) = sum(6, 6) = 1

 Note:
 1 <= n <= 2000.
 Elements in the given array will be in range [-1,000,000, 1,000,000].
```

```
 */
public class _548 {

    public static class Solution1 {
        public boolean splitArray(int[] nums) {
            int len = nums.length;
            if (len < 7) {
                return false;
            }
            int[] sum = new int[len];
            sum[0] = nums[0];
            for (int i = 1; i < len; i++) {
                sum[i] = sum[i - 1] + nums[i];
            }
            for (int j = 3; j < len - 3; j++) {
                Set<Integer> set = new HashSet<>();
                for (int i = 1; i < j - 1; i++) {
                    if (sum[i - 1] == sum[j - 1] - sum[i]) {
                        /**this is sum(0, i-1) and sum(i+1, j-1)*/
                        set.add(sum[i - 1]);
                    }
                }
                for (int k = j + 2; k < len - 1; k++) {
                    if (sum[k - 1] - sum[j] == sum[len - 1] - sum[k] &&
set.contains(sum[k - 1] - sum[j])) {
                        /**this is sum(j+1, k-1) and sum(k+1, len-1)*/
                        return true;
                    }
                }
            }
            return false;
        }
    }

}

// ============================================================
// FILE 515: _549.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 549. Binary Tree Longest Consecutive Sequence II
 *
 * Given a binary tree, you need to find the length of Longest Consecutive Path in Binary
Tree.
 Especially, this path can be either increasing or decreasing. For example, [1,2,3,4] and
[4,3,2,1] are both considered valid,
 but the path [1,2,4,3] is not valid. On the other hand, the path can be in the child-
Parent-child order, where not necessarily be parent-child order.
```

```
 Example 1:
 Input:
   1
  / \
 2   3
 Output: 2

 Explanation: The longest consecutive path is [1, 2] or [2, 1].
 Example 2:
 Input:
   2
  / \
 1   3
 Output: 3
 Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].
 Note: All the values of tree nodes are in the range of [-1e7, 1e7].
 */
public class _549 {

    public static class Solution1 {
        int max = 0;

        public int longestConsecutive(TreeNode root) {
            longestPath(root);
            return max;
        }

        private int[] longestPath(TreeNode root) {
            if (root == null) {
                return new int[]{0, 0};
            }
            int increasing = 1;
            int decreasing = 1;
            if (root.left != null) {
                int[] left = longestPath(root.left);
                if (root.val == root.left.val + 1) {
                    decreasing = left[1] + 1;
                } else if (root.val == root.left.val - 1) {
                    increasing = left[0] + 1;
                }
            }

            if (root.right != null) {
                int[] right = longestPath(root.right);
                if (root.val == root.right.val + 1) {
                    decreasing = Math.max(right[1] + 1, decreasing);
                } else if (root.val == root.right.val - 1) {
                    increasing = Math.max(right[0] + 1, increasing);
                }
            }

            max = Math.max(max, decreasing + increasing - 1);
            return new int[]{increasing, decreasing};
        }
```

```
    }
}

// ============================================================
// FILE 516: _551.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 551. Student Attendance Record I
 *
 * You are given a string representing an attendance record for a student. The record
only contains the following three characters:

 'A' : Absent.
 'L' : Late.
 'P' : Present.
 A student could be rewarded if his attendance record doesn't contain more than one 'A'
(absent) or more than two continuous 'L' (late).

 You need to return whether the student could be rewarded according to his attendance
record.

 Example 1:
 Input: "PPALLP"
 Output: True

 Example 2:
 Input: "PPALLL"
 Output: False

 */
public class _551 {

    public static class Solution1 {
        public boolean checkRecord(String s) {
            int aCount = 0;
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == 'A') {
                    aCount++;
                    if (aCount > 1) {
                        return false;
                    }
                } else if (s.charAt(i) == 'L') {
                    int continuousLCount = 0;
                    while (i < s.length() && s.charAt(i) == 'L') {
                        i++;
                        continuousLCount++;
                        if (continuousLCount > 2) {
                            return false;
                        }
                    }
                    i--;
```

```
                }
            }
            return true;
        }
    }

}

// ============================================================
// FILE 517: _552.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 552. Student Attendance Record II
 *
 * Given a positive integer n, return the number of all possible attendance records with length n,
 * which will be regarded as rewardable. The answer may be very large, return it after mod 109 + 7.

 A student attendance record is a string that only contains the following three characters:

 'A' : Absent.
 'L' : Late.
 'P' : Present.

 A record is regarded as rewardable if it doesn't contain more than one 'A' (absent) or more than two continuous 'L' (late).

 Example 1:

 Input: n = 2
 Output: 8

 Explanation:
 There are 8 records with length 2 will be regarded as rewardable:
 "PP" , "AP", "PA", "LP", "PL", "AL", "LA", "LL"
 Only "AA" won't be regarded as rewardable owing to more than one absent times.

 Note: The value of n won't exceed 100,000.
 */
public class _552 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/86526/improving-the-runtime-from-o-n-to-o-log-n
         */
        public int checkRecord(int n) {
            final int MOD = 1000000007;
            int[][][] f = new int[n + 1][2][3];
```

```
            f[0] = new int[][]{{1, 1, 1}, {1, 1, 1}};
            for (int i = 1; i <= n; i++) {
                for (int j = 0; j < 2; j++) {
                    for (int k = 0; k < 3; k++) {
                        int val = f[i - 1][j][2]; // ...P
                        if (j > 0) {
                            val = (val + f[i - 1][j - 1][2]) % MOD; // ...A
                        }
                        if (k > 0) {
                            val = (val + f[i - 1][j][k - 1]) % MOD; // ...L
                        }
                        f[i][j][k] = val;
                    }
                }
            }
            return f[n][1][2];
        }
    }

}

// ============================================================
// FILE 518: _553.java
// ============================================================

package com.fishercoder.solutions;

import java.util.StringJoiner;

/**
 * 553. Optimal Division
 *
 * Given a list of positive integers, the adjacent integers will perform the float
division. For example, [2,3,4] -> 2 / 3 / 4.
 However, you can add any number of parenthesis at any position to change the priority of
operations.
 You should find out how to add parenthesis to get the maximum result,
 and return the corresponding expression in string format.
 Your expression should NOT contain redundant parenthesis.

 Example:
 Input: [1000,100,10,2]
 Output: "1000/(100/10/2)"
 Explanation:
 1000/(100/10/2) = 1000/((100/10)/2) = 200
 However, the bold parenthesis in "1000/((100/10)/2)" are redundant,
 since they don't influence the operation priority. So you should return
"1000/(100/10/2)".

 Other cases:
 1000/(100/10)/2 = 50
 1000/(100/(10/2)) = 50
 1000/100/10/2 = 0.5
```

```
 1000/100/(10/2) = 2
 Note:

 The length of the input array is [1, 10].
 Elements in the given array will be in range [2, 1000].
 There is only one optimal division for each test case.
 */
public class _553 {
    public static class Solution1 {
        /**
         * Credit: https://github.com/lydxlx1/LeetCode/blob/master/src/_553.java
         */
        public String optimalDivision(int[] nums) {
            /**https://docs.oracle.com/javase/8/docs/api/java/util/StringJoiner.html:
             * StringJoiner is used to construct a sequence of characters separated by a
delimiter and optionally starting with a supplied prefix and ending with a supplied
suffix.
             * The String "[George:Sally:Fred]" may be constructed as follows:
             StringJoiner sj = new StringJoiner(":", "[", "]");
             sj.add("George").add("Sally").add("Fred");
             String desiredString = sj.toString();*/

            if (nums.length == 1) {
                return "" + nums[0];
            }
            if (nums.length == 2) {
                return nums[0] + "/" + nums[1];
            }

            /**Tricky one: the solution is fixed: always wrap the one from the second
until the last.
             * Another important thing to note that such way could work is that:
             * the prerequisite is: Elements will be in range [2,1000], so no elements
are smaller than 1.*/
            StringJoiner stringJoiner = new StringJoiner("/");
            for (int i = 1; i < nums.length; i++) {
                stringJoiner.add("" + nums[i]);
            }
            return String.format("%d/(%s)", nums[0], stringJoiner.toString());
        }
    }

}

// ============================================================
// FILE 519: _554.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

```
/**
 * 554. Brick Wall
 *
 * There is a brick wall in front of you. The wall is rectangular and has several rows of bricks.
 * The bricks have the same height but different width.
 * You want to draw a vertical line from the top to the bottom and cross the least bricks.
 * The brick wall is represented by a list of rows.
 * Each row is a list of integers representing the width of each brick in this row from left to right.
 * If your line go through the edge of a brick,
 * then the brick is not considered as crossed.
 * You need to find out how to draw the line to cross the least bricks and return the number of crossed bricks.
 * You cannot draw a line just along one of the two vertical edges of the wall, in which case the line will obviously cross no bricks.

 Example:
 Input:
 [[1,2,2,1],
 [3,1,2],
 [1,3,2],
 [2,4],
 [3,1,2],
 [1,3,1,1]]

 Output: 2
 Explanation:

 Note:
 The width sum of bricks in different rows are the same and won't exceed INT_MAX.
 The number of bricks in each row is in range [1,10,000]. The height of wall is in range [1,10,000]. Total number of bricks of the wall won't exceed 20,000.
 */
public class _554 {
    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/brick-wall/
         *
         * we make use of a HashMap
         * map which is used to store entries in the form:
         * (sum,count). Here,
         * sum refers to the cumulative sum of the bricks' widths encountered in the current row, and
         * count refers to the number of times the corresponding sum is obtained. Thus,
         * sum in a way, represents the positions of the bricks's boundaries relative to the leftmost boundary.
         *
         * This is done based on the following observation:
         * We will never obtain the same value of sum twice while traversing over a particular row.
         * Thus, if the sum value is repeated while traversing over the rows, it means some row's brick boundary coincides with some previous row's brick boundary.
```

```
         * This fact is accounted for by incrementing the corresponding count value.
         * But, for every row, we consider the sum only upto the second last brick, since
the last boundary isn't a valid boundary for the solution.
         */

        public int leastBricks(List<List<Integer>> wall) {
            Map<Integer, Integer> map = new HashMap();
            for (List<Integer> row : wall) {
                int sum = 0;
                for (int i = 0; i < row.size() - 1; i++) {
                    //NOTE: i < row.size()-1
                    sum += row.get(i);
                    if (map.containsKey(sum)) {
                        map.put(sum, map.get(sum) + 1);
                    } else {
                        map.put(sum, 1);
                    }
                }
            }
            int result = wall.size();
            for (int key : map.keySet()) {
                result = Math.min(result, wall.size() - map.get(key));
            }
            return result;
        }
    }
}

// ============================================================
// FILE 520: _555.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 555. Split Concatenated Strings
 *
 * Given a list of strings, you could concatenate these strings together into a loop,
 * where for each string you could choose to reverse it or not.
 *
 * Among all the possible loops, you need to find the lexicographically biggest string
after cutting the loop,
 * which will make the looped string into a regular one.
 *
 * Specifically, to find the lexicographically biggest string, you need to experience two
phases:
 *
 * 1. Concatenate all the strings into a loop, where you can reverse some strings or not
and connect them in the same order as given.
 * 2. Cut and make one breakpoint in any place of the loop, which will make the looped
string into a regular one starting from
 * the character at the cutpoint.
 *
 * And your job is to find the lexicographically biggest one among all the possible
```

```
regular strings.

 Example:
 Input: "abc", "xyz"
 Output: "zyxcba"

 Explanation: You can get the looped string "-abcxyz-", "-abczyx-", "-cbaxyz-", "-cbazyx-
",
 where '-' represents the looped status.
 The answer string came from the fourth looped one,
 where you could cut from the middle character 'a' and get "zyxcba".

 Note:
 The input strings will only contain lowercase letters.
 The total length of all the strings will not over 1,000.

 */
public class _555 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/86477/neat-java-solution
         * and article: https://leetcode.com/articles/split-assembled-strings/#approach-
3-optimized-solution-accepted
         */
        public String splitLoopedString(String[] strs) {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < strs.length; i++) {
                sb.setLength(0);
                String reverse = sb.append(strs[i]).reverse().toString();
                if (strs[i].compareTo(reverse) < 0) {
                    strs[i] = reverse;
                }
            }
            String result = "";
            for (int i = 0; i < strs.length; i++) {
                sb.setLength(0);
                String reverse = sb.append(strs[i]).reverse().toString();
                for (String str : new String[]{strs[i], reverse}) {
                    for (int k = 0; k < str.length(); k++) {
                        sb.setLength(0);
                        sb.append(str.substring(k));
                        for (int j = i + 1; j < strs.length; j++) {
                            sb.append(strs[j]);
                        }
                        for (int j = 0; j < i; j++) {
                            sb.append(strs[j]);
                        }
                        sb.append(str.substring(0, k));
                        if (sb.toString().compareTo(result) > 0) {
                            result = sb.toString();
                        }
                    }
```

```
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 521: _556.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 556. Next Greater Element III
 *
 * Given a positive 32-bit integer n, you need to find the smallest 32-bit integer which
has exactly the same digits existing
 * in the integer n and is greater in value than n. If no such positive 32-bit integer
exists, you need to return -1.
 *
 * Example 1:
 * Input: 12
 * Output: 21
 *
 * Example 2:
 * Input: 21
 * Output: -1
 */
public class _556 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/85759/this-problem-is-the-same-to-
next-permutation-algorithm-only and https://discuss.leetcode.com/topic/85755/java-
solution-like-next-permutation-problem-o-n
         */

        public int nextGreaterElement(int n) {
            char[] digits = String.valueOf(n).toCharArray();
            int i = digits.length - 2;
            while (i >= 0 && digits[i + 1] <= digits[i]) {
                i--;
            }
            if (i < 0) {
                return -1;
            }
            int j = digits.length - 1;
            while (j >= 0 && digits[j] <= digits[i]) {
                j--;
            }
            swap(digits, i, j);
            reverse(digits, i + 1);
            try {
                return Integer.parseInt(new String(digits));
```

```
            } catch (Exception e) {
                return -1;
            }
        }

        private void reverse(char[] a, int start) {
            int i = start;
            int j = a.length - 1;
            while (i < j) {
                swap(a, i, j);
                i++;
                j--;
            }
        }

        private void swap(char[] a, int i, int j) {
            char temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

}

// ============================================================
// FILE 522: _557.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 557. Reverse Words in a String III
 *
 * Given a string, you need to reverse the order of characters in each word within a
sentence while still preserving whitespace and initial word order.

 Example 1:
 Input: "Let's take LeetCode contest"
 Output: "s'teL ekat edoCteeL tsetnoc"
 Note: In the string, each word is separated by single space and there will not be any
extra space in the string.
 */
public class _557 {

    public static class Solution1 {
        public String reverseWords(String s) {
            StringBuilder stringBuilder = new StringBuilder();
            StringBuilder sbUtils = new StringBuilder();
            for (String word : s.split(" ")) {
                sbUtils.setLength(0);
                sbUtils.append(word);
                stringBuilder.append(sbUtils.reverse().toString());
                stringBuilder.append(" ");
            }
```

```
            stringBuilder.setLength(stringBuilder.length() - 1);
            return stringBuilder.toString();
        }
    }
}

// ============================================================
// FILE 523: _559.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.List;

/**
 * 559. Maximum Depth of N-ary Tree
 *
 * Given a n-ary tree, find its maximum depth.
 *
 * The maximum depth is the number of nodes along the longest path from the root node
down to the farthest leaf node.
 *
 * For example, given a 3-ary tree:
 *                            1
 *                          / | \
 *                         3  2  4
 *                        / \
 *                       5  6
 *
 * We should return its max depth, which is 3.
 *
 * Note:
 *
 * The depth of the tree is at most 1000.
 * The total number of nodes is at most 5000.
 */
public class _559 {
  public static class Solution1 {
    public int maxDepth(Node root) {
      int maxDepth = 0;
      if (root == null) {
        return maxDepth;
      }
      List<List<Integer>> allPaths = new ArrayList<>();
      List<Integer> currentPath = new ArrayList<>();
      dfs(root, currentPath, allPaths);
      for (List<Integer> path : allPaths) {
        maxDepth = Math.max(path.size(), maxDepth);
      }
      return maxDepth;
    }
```

```
    private void dfs(Node root, List<Integer> currentPath, List<List<Integer>> allPaths)
{
      if (root == null) {
        allPaths.add(new ArrayList<>(currentPath));
      }
      if (root.children != null && !root.children.isEmpty()) {
        currentPath.add(root.val);
        for (Node child : root.children) {
          dfs(child, new ArrayList<>(currentPath), allPaths);
        }
      }
      if (root.children == null || root.children.isEmpty()) {
        currentPath.add(root.val);
        allPaths.add(new ArrayList<>(currentPath));
      }
    }
  }
}

// ============================================================
// FILE 524: _560.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 560. Subarray Sum Equals K
 *
 * Given an array of integers and an integer k, you need to find the total number of
continuous subarrays whose sum equals to k.

 Example 1:
 Input:nums = [1,1,1], k = 2
 Output: 2
 Note:
 The length of the array is in range [1, 20,000].
 The range of numbers in the array is [-1000, 1000] and the range of the integer k is
[-1e7, 1e7].
 */
public class _560 {

    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/87850/java-solution-presum-hashmap
         * We know the key to solve this problem is SUM[i, j].
         * So if we know SUM[0, i - 1] and SUM[0, j],
         * then we can easily get SUM[i, j] via (SUM[0, j] - SUM[0, i-1]).
         * To achieve this, we just need to go through the array,
         * calculate the current sum and save number of all seen PreSum to a HashMap.
         * <p>
```

```
         * Time complexity O(n), Space complexity O(n).
         */
        public int subarraySum(int[] nums, int k) {
            Map<Integer, Integer> preSum = new HashMap();
            int sum = 0;
            int result = 0;
            preSum.put(0, 1);
            for (int i = 0; i < nums.length; i++) {
                sum += nums[i];
                if (preSum.containsKey(sum - k)) {
                    result += preSum.get(sum - k);
                }
                preSum.put(sum, preSum.getOrDefault(sum, 0) + 1);
            }
            return result;
        }
    }

}

// ============================================================
// FILE 525: _561.java
// ============================================================

package com.fishercoder.solutions;

/**
 * Created by fishercoder on 4/23/17.
 */

import java.util.Arrays;

/**
 * Array Partition I:
 *
 *
 * Given an array of 2n integers, your task is to group these integers into n pairs of
integer,
 * say (a1, b1), (a2, b2), ..., (an, bn) which makes sum of min(ai, bi) for all i from 1
to n as large as possible.

 Example 1:
 Input: [1,4,3,2]

 Output: 4
 Explanation: n is 2, and the maximum sum of pairs is 4.

 Note:
 n is a positive integer, which is in the range of [1, 10000].
 All the integers in the array will be in the range of [-10000, 10000].*/
public class _561 {

    public static class Solution1 {
        public int arrayPairSum(int[] nums) {
```

```
            Arrays.sort(nums);
            int sum = 0;
            for (int i = 0; i < nums.length; i += 2) {
                sum += nums[i];
            }
            return sum;
        }
    }

}

// ============================================================
// FILE 526: _562.java
// ============================================================

package com.fishercoder.solutions;

/**
 * Created by fishercoder on 4/23/17.
 *
 * 562. Longest Line of Consecutive One in Matrix
 *
 * Given a 01 matrix m, find the longest line of consecutive one in the matrix. The line
could be horizontal, vertical, diagonal or anti-diagonal.

 Example:

 Input:
 [[0,1,1,0],
  [0,1,1,0],
  [0,0,0,1]]

 Output: 3
 Hint: The number of elements in the given matrix will not exceed 10,000.
 */
public class _562 {

    public static class Solution1 {
        public int longestLine(int[][] M) {
            if (M == null || M.length == 0) {
                return 0;
            }
            int[][] directions = new int[][]{
                    {-1, 0},
                    {-1, 1},
                    {0, 1},
                    {1, 1},
                    {1, 0},
                    {1, -1},
                    {0, -1},
                    {-1, -1},
            };
            int longestLine = 0;
            int m = M.length;
```

```
            int n = M[0].length;
            int[][][] cache = new int[m][n][8];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (M[i][j] == 1) {
                        for (int k = 0; k < directions.length; k++) {
                            int nextI = i + directions[k][0];
                            int nextJ = j + directions[k][1];
                            int thisLine = 1;
                            if (nextI >= 0 && nextI < m && nextJ >= 0 && nextJ < n &&
cache[nextI][nextJ][k] != 0) {
                                thisLine += cache[nextI][nextJ][k];
                                cache[i][j][k] = thisLine;
                            } else {
                                while (nextI >= 0 && nextI < m && nextJ >= 0 && nextJ < n
&& M[nextI][nextJ] == 1) {
                                    thisLine++;
                                    cache[i][j][k] = thisLine;
                                    nextI += directions[k][0];
                                    nextJ += directions[k][1];
                                }
                            }
                            longestLine = Math.max(longestLine, thisLine);
                        }
                    }
                }
            }
            return longestLine;
        }
    }

}

// ============================================================
// FILE 527: _563.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 563. Binary Tree Tilt
 *
 * Given a binary tree, return the tilt of the whole tree.
 * The tilt of a tree node is defined as the absolute difference between the sum of all
left subtree node values
 * and the sum of all right subtree node values.
 * Null node has tilt 0.
 * The tilt of the whole tree is defined as the sum of all nodes' tilt.

 Example:
 Input:
    1
```

```
  /   \
 2     3

 Output: 1

 Explanation:
 Tilt of node 2 : 0
 Tilt of node 3 : 0
 Tilt of node 1 : |2-3| = 1
 Tilt of binary tree : 0 + 0 + 1 = 1

 Note:
 The sum of node values in any subtree won't exceed the range of 32-bit integer.
 All the tilt values won't exceed the range of 32-bit integer.*/
public class _563 {
    public static class Solution1 {

        int tilt = 0;

        public int findTilt(TreeNode root) {
            findTiltDfs(root);
            return tilt;
        }

        public int findTiltDfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int leftTilt = 0;
            if (root.left != null) {
                leftTilt = findTiltDfs(root.left);
            }
            int rightTilt = 0;
            if (root.right != null) {
                rightTilt = findTiltDfs(root.right);
            }
            if (root.left == null && root.right == null) {
                return root.val;
            }
            tilt += Math.abs(leftTilt - rightTilt);
            return leftTilt + rightTilt + root.val;
        }
    }

}

// ============================================================
// FILE 528: _564.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 564. Find the Closest Palindrome
```

```
 *
 * Given an integer n, find the closest integer (not including itself), which is a
palindrome.

 The 'closest' is defined as absolute difference minimized between two integers.

 Example 1:

 Input: "123"
 Output: "121"

 Note:

 The input n is a positive integer represented by string, whose length will not exceed
18.
 If there is a tie, return the smaller one as answer.

 */
public class _564 {
    public static class Solution1 {

        public String nearestPalindromic(String n) {
            if (n.length() >= 2 && allNine(n)) {
                String s = "1";
                for (int i = 0; i < n.length() - 1; i++) {
                    s += "0";
                }
                s += "1";
                return s;
            }
            boolean isOdd = (n.length() % 2 != 0);
            String left = n.substring(0, (n.length() + 1) / 2);
            long[] increment = {-1, 0, +1};
            String ret = n;
            long minDiff = Long.MAX_VALUE;
            for (long i : increment) {
                String s = getPalindrom(Long.toString(Long.parseLong(left) + i), isOdd);
                if (n.length() >= 2 && (s.length() != n.length() || Long.parseLong(s) ==
0)) {
                    s = "";
                    for (int j = 0; j < n.length() - 1; j++) {
                        s += "9";
                    }
                }
                long diff = s.equals(n) ? Long.MAX_VALUE : Math.abs(Long.parseLong(s) -
Long.parseLong(n));
                if (diff < minDiff) {
                    minDiff = diff;
                    ret = s;
                }
            }
            return ret;
        }
```

```
        private String getPalindrom(String s, boolean isOdd) {
            String right = new StringBuilder(s).reverse().toString();
            return isOdd ? s.substring(0, s.length() - 1) + right : s + right;
        }

        private boolean allNine(String s) {
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) != '9') {
                    return false;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 529: _565.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 565. Array Nesting
 *
 A zero-indexed array A consisting of N different integers is given. The array contains
all integers in the range [0, N - 1].

 Sets S[K] for 0 <= K < N are defined as follows:

 S[K] = { A[K], A[A[K]], A[A[A[K]]], ... }.

 Sets S[K] are finite for each K and should NOT contain duplicates.

 Write a function that given an array A consisting of N integers, return the size of the
largest set S[K] for this array.

 Example 1:
 Input: A = [5,4,0,3,1,6,2]
 Output: 4
 Explanation:
 A[0] = 5, A[1] = 4, A[2] = 0, A[3] = 3, A[4] = 1, A[5] = 6, A[6] = 2.

 One of the longest S[K]:
 S[0] = {A[0], A[5], A[6], A[2]} = {5, 6, 2, 0}
 Note:
 N is an integer within the range [1, 20,000].
 The elements of A are all distinct.
 Each element of array A is an integer within the range [0, N-1].
 */
public class _565 {

    public static class Solution1 {
        public int arrayNesting(int[] nums) {
```

```
            if (nums == null || nums.length == 0) {
                return 0;
            }
            boolean[] visited = new boolean[nums.length];
            int answer = 0;
            for (int i : nums) {
                int count = 0;
                int j = i;
                while (j >= 0 && j < nums.length && !visited[j]) {
                    count++;
                    visited[j] = true;
                    j = nums[j];
                }
                answer = Math.max(answer, count);
            }
            return answer;
        }
    }

}

// ============================================================
// FILE 530: _566.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 566. Reshape the Matrix
 *
 In MATLAB, there is a very useful function called 'reshape', which can reshape a matrix into a new one with different size but keep its original data.
 You're given a matrix represented by a two-dimensional array, and two positive integers r and c representing the row number and column number of the wanted reshaped matrix, respectively.
 The reshaped matrix need to be filled with all the elements of the original matrix in the same row-traversing order as they were.
 If the 'reshape' operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

 Example 1:
 Input:
 nums =
 [[1,2],
 [3,4]]
 r = 1, c = 4
 Output:
 [[1,2,3,4]]
 Explanation:
 The row-traversing of nums is [1,2,3,4]. The new reshaped matrix is a 1 * 4 matrix, fill it row by row by using the previous list.

 Example 2:
 Input:
```

```
 nums =
 [[1,2],
 [3,4]]
 r = 2, c = 4
 Output:
 [[1,2],
 [3,4]]
 Explanation:
 There is no way to reshape a 2 * 2 matrix to a 2 * 4 matrix. So output the original
matrix.

 Note:
 The height and width of the given matrix is in range [1, 100].
 The given r and c are all positive.
 */
public class _566 {

    public static class Solution1 {
        public int[][] matrixReshape(int[][] nums, int r, int c) {
            if (nums == null || nums.length == 0) {
                return nums;
            }
            int m = nums.length;
            int n = nums[0].length;
            if (r * c > m * n) {
                return nums;
            }
            int k = 0;
            int[][] result = new int[r][c];
            for (int i = 0; i < r; i++) {
                for (int j = 0; j < c; j++, k++) {
                    result[i][j] = nums[k / n][k % n];
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 531: _567.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 567. Permutation in String
 *
 * Given two strings s1 and s2, write a function to return true if s2 contains the
permutation of s1.
 * In other words, one of the first string's permutations is the substring of the second
string.

 Example 1:
```

```
 Input:s1 = "ab" s2 = "eidbaooo"
 Output:True
 Explanation: s2 contains one permutation of s1 ("ba").

 Example 2:
 Input:s1= "ab" s2 = "eidboaoo"
 Output: False

 Note:
 The input strings only contain lower case letters.
 The length of both given strings is in range [1, 10,000].
 */
public class _567 {

    public static class Solution1 {
        /**
         * credit: sliding window: https://discuss.leetcode.com/topic/87845/java-
solution-sliding-window
         */
        public boolean checkInclusion(String s1, String s2) {
            int len1 = s1.length();
            int len2 = s2.length();
            if (len1 > len2) {
                return false;
            }

            int[] count = new int[26];
            for (int i = 0; i < len1; i++) {
                count[s1.charAt(i) - 'a']++;
                count[s2.charAt(i) - 'a']--;
            }

            if (allZeroes(count)) {
                return true;
            }

            for (int i = len1; i < len2; i++) {
                count[s2.charAt(i) - 'a']--;
                count[s2.charAt(i - len1) - 'a']++;
                if (allZeroes(count)) {
                    return true;
                }
            }

            return false;
        }

        private boolean allZeroes(int[] count) {
            for (int i : count) {
                if (i != 0) {
                    return false;
                }
            }
            return true;
```

```
        }
    }
}

// ============================================================
// FILE 532: _568.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 568. Maximum Vacation Days
 *
 * LeetCode wants to give one of its best employees the option to travel among N cities
to collect algorithm problems.
 * But all work and no play makes Jack a dull boy,
 * you could take vacations in some particular cities and weeks.
 * Your job is to schedule the traveling to maximize the number of
 * vacation days you could take, but there are certain rules and restrictions you need to
follow.

 Rules and restrictions:
 You can only travel among N cities, represented by indexes from 0 to N-1. Initially, you
are in the city indexed 0 on Monday.
 The cities are connected by flights.
 The flights are represented as a N*N matrix (not necessary symmetrical),
 called flights representing the airline status from the city i to the city j.
 If there is no flight from the city i to the city j, flights[i][j] = 0;
 Otherwise, flights[i][j] = 1. Also, flights[i][i] = 0 for all i.

 You totally have K weeks (each week has 7 days) to travel.
 You can only take flights at most once per day and can only take flights on each week's
Monday morning.
 Since flight time is so short, we don't consider the impact of flight time.
 For each city, you can only have restricted vacation days in different weeks,
 given an N*K matrix called days representing this relationship.
 For the value of days[i][j], it represents the maximum days you could take vacation in
the city i in the week j.
 You're given the flights matrix and days matrix, and you need to output the maximum
vacation days you could take during K weeks.

 Example 1:
 Input:flights = [[0,1,1],[1,0,1],[1,1,0]], days = [[1,3,1],[6,0,3],[3,3,3]]
 Output: 12
 Explanation:
 Ans = 6 + 3 + 3 = 12.

 One of the best strategies is:
 1st week : fly from city 0 to city 1 on Monday, and play 6 days and work 1 day.
 (Although you start at city 0, we could also fly to and start at other cities since it
is Monday.)
 2nd week : fly from city 1 to city 2 on Monday, and play 3 days and work 4 days.
```

```
 3rd week : stay at city 2, and play 3 days and work 4 days.


 Example 2:
 Input:flights = [[0,0,0],[0,0,0],[0,0,0]], days = [[1,1,1],[7,7,7],[7,7,7]]
 Output: 3
 Explanation:
 Ans = 1 + 1 + 1 = 3.

 Since there is no flights enable you to move to another city, you have to stay at city 0
for the whole 3 weeks.
 For each week, you only have one day to play and six days to work.
 So the maximum number of vacation days is 3.


 Example 3:
 Input:flights = [[0,1,1],[1,0,1],[1,1,0]], days = [[7,0,0],[0,7,0],[0,0,7]]
 Output: 21
 Explanation:
 Ans = 7 + 7 + 7 = 21

 One of the best strategies is:
 1st week : stay at city 0, and play 7 days.
 2nd week : fly from city 0 to city 1 on Monday, and play 7 days.
 3rd week : fly from city 1 to city 2 on Monday, and play 7 days.

 Note:
 N and K are positive integers, which are in the range of [1, 100].
 In the matrix flights, all the values are integers in the range of [0, 1].
 In the matrix days, all the values are integers in the range [0, 7].
 You could stay at a city beyond the number of vacation days,
 but you should work on the extra days, which won't be counted as vacation days.
 If you fly from the city A to the city B and take the vacation on that day,
 the deduction towards vacation days will count towards the vacation days of city B in
that week.
 We don't consider the impact of flight hours towards the calculation of vacation days.
 */
public class _568 {

    public static class Solution1 {
        /**
         * credit: https://leetcode.com/articles/maximum-vacation-days/#approach-2-using-
dfs-with-memoization-accepted
         */
        public int maxVacationDays(int[][] flights, int[][] days) {
            int[][] memo = new int[flights.length][days[0].length];
            for (int[] l : memo) {
                Arrays.fill(l, Integer.MIN_VALUE);
            }
            return dfs(flights, days, 0, 0, memo);
        }

        public int dfs(int[][] flights, int[][] days, int curCity, int weekno, int[][]
memo) {
```

```
            if (weekno == days[0].length) {
                return 0;
            }
            if (memo[curCity][weekno] != Integer.MIN_VALUE) {
                return memo[curCity][weekno];
            }
            int maxvac = 0;
            for (int i = 0; i < flights.length; i++) {
                if (flights[curCity][i] == 1 || i == curCity) {
                    int vac = days[i][weekno] + dfs(flights, days, i, weekno + 1, memo);
                    maxvac = Math.max(maxvac, vac);
                }
            }
            memo[curCity][weekno] = maxvac;
            return maxvac;
        }
    }

}

// ============================================================
// FILE 533: _572.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 572. Subtree of Another Tree
 *
 Given two non-empty binary trees s and t,
 check whether tree t has exactly the same structure and node values with a subtree of s.
 A subtree of s is a tree consists of a node in s and all of this node's descendants. The
tree s could also be considered as a subtree of itself.

 Example 1:
 Given tree s:

     3
    / \
   4   5
  / \
 1   2
 Given tree t:
   4
  / \
 1   2
 Return true, because t has the same structure and node values with a subtree of s.

 Example 2:
 Given tree s:

     3
```

```
    / \
   4   5
  / \
 1   2
    /
   0

 Given tree t:
   4
  / \
 1   2
 Return false.
 */
public class _572 {

    public static class Solution1 {
        public boolean isSubtree(TreeNode s, TreeNode t) {
            if (s == null && t == null) {
                return true;
            }
            boolean isSubTree = false;
            if (s != null && t != null && s.val == t.val) {
                isSubTree = isSameTree(s, t);
            }
            if (isSubTree) {
                return true;
            }
            boolean isSubTreeLeft = false;
            if (s.left != null) {
                isSubTreeLeft = isSubtree(s.left, t);
            }
            if (isSubTreeLeft) {
                return true;
            }
            boolean isSubTreeRight = false;
            if (s.right != null) {
                isSubTreeRight = isSubtree(s.right, t);
            }
            if (isSubTreeRight) {
                return true;
            }
            return false;
        }

        private boolean isSameTree(TreeNode p, TreeNode q) {
            if (p == null || q == null) {
                return p == q;
            }
            return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right,
q.right);
        }
    }

    public static class Solution2 {
```

```
        public boolean isSubtree(TreeNode s, TreeNode t) {
            if (s == null) {
                return false;
            }
            if (same(s, t)) {
                return true;
            }
            return isSubtree(s.left, t) || isSubtree(s.right, t);
        }

        private boolean same(TreeNode s, TreeNode t) {
            if (s == null || t == null) {
                return s == t;
            }
            if (s.val != t.val) {
                return false;
            }
            return same(s.left, t.left) && same(s.right, t.right);
        }

    }
}

// ============================================================
// FILE 534: _573.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 573. Squirrel Simulation
 *
 * There's a tree, a squirrel, and several nuts.
 * Positions are represented by the cells in a 2D grid.
 * Your goal is to find the minimal distance for the squirrel to collect all the nuts and
 * put them under the tree one by one.
 * The squirrel can only take at most one nut at one time and can move in four directions
-
 * up, down, left and right, to the adjacent cell.
 * The distance is represented by the number of moves.

 Example 1:

 Input:
 Height : 5
 Width : 7
 Tree position : [2,2]
 Squirrel : [4,4]
 Nuts : [[3,0], [2,5]]
 Output: 12

 Explanation:

 Note:
```

```
 All given positions won't overlap.
 The squirrel can take at most one nut at one time.
 The given positions of nuts have no order.
 Height and width are positive integers. 3 <= height * width <= 10,000.
 The given positions contain at least one nut, only one tree and one squirrel.
 */
public class _573 {

    public static class Solution1 {

        /**
         * reference: https://leetcode.com/articles/squirrel-simulation
         *
         * 1. The order in which to pick the nuts does not matter except the first one
         * because for all the other nuts, the squirrel needs to travel back and forth.
         *
         * 2. For the first nut to be picked, there's some distance we might be able to
save, what is this distance?
         * It is the difference between the squirrel's original starting point to the
first nut and that the distance from this
         * first nut to the tree.
         * This is because, only for the first nut, the squirrel does NOT need to travel
back and forth, it only needs to travel from
         * its starting position to the nut position and then return to the tree.
         *
         * 3. For the rest of all other nuts, the squirrel always needs to go back and
forth.
         *
         * 4. So how can we find the first nut to go to so that we could have the maximum
saved distance?
         * We iterate through all of the nuts and keep updating the savedDist as below:
         */
        public int minDistance(int height, int width, int[] tree, int[] squirrel, int[][]
nuts) {
            int totalDist = 0;
            int savedDist = Integer.MIN_VALUE;
            for (int[] nut : nuts) {
                totalDist += (getDist(nut, tree) * 2);//it needs to travel back and
forth, so times two
                savedDist = Math.max(savedDist, getDist(nut, tree) - getDist(nut,
squirrel));
            }
            return totalDist - savedDist;
        }

        private int getDist(int[] pointA, int[] pointB) {
            return Math.abs(pointA[0] - pointB[0]) + Math.abs(pointA[1] - pointB[1]);
        }
    }
}

// ============================================================
// FILE 535: _575.java
```

```
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 575. Distribute Candies
 *
 Given an integer array with even length, where different numbers in this array represent
different kinds of candies. Each number means one candy of the corresponding kind. You
need to distribute these candies equally in number to brother and sister. Return the
maximum number of kinds of candies the sister could gain.

 Example 1:
 Input: candies = [1,1,2,2,3,3]
 Output: 3
 Explanation:
 There are three different kinds of candies (1, 2 and 3), and two candies for each kind.
 Optimal distribution: The sister has candies [1,2,3] and the brother has candies
[1,2,3], too.
 The sister has three different kinds of candies.
 Example 2:
 Input: candies = [1,1,2,3]
 Output: 2
 Explanation: For example, the sister has candies [2,3] and the brother has candies
[1,1].
 The sister has two different kinds of candies, the brother has only one kind of candies.
 Note:

 The length of the given array is in range [2, 10,000], and will be even.
 The number in given array is in range [-100,000, 100,000].
 */
public class _575 {
    public static class Solution1 {
        public int distributeCandies(int[] candies) {
            Arrays.sort(candies);
            int sisCount = 0;
            for (int i = 0; i < candies.length; i++) {
                int val = candies[i];
                sisCount++;
                if (sisCount >= candies.length / 2) {
                    return candies.length / 2;
                }
                while (i < candies.length && candies[i] == val) {
                    i++;
                }
                i--;
            }
            return sisCount;
        }
    }
}
```

```
// ============================================================
// FILE 536: _576.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 576. Out of Boundary Paths
 *
 * There is an m by n grid with a ball.
 * Given the start coordinate (i,j) of the ball,
 * you can move the ball to adjacent cell or cross the grid boundary in four directions
(up, down, left, right).
 * However, you can at most move N times. Find out the number of paths to move the ball
out of grid boundary.
 * The answer may be very large, return it after mod 109 + 7.

 Example 1:

 Input:m = 2, n = 2, N = 2, i = 0, j = 0
 Output: 6
 Explanation:

 Example 2:

 Input:m = 1, n = 3, N = 3, i = 0, j = 1
 Output: 12
 Explanation:

 Note:

 Once you move the ball out of boundary, you cannot move it back.
 The length and height of the grid is in range [1,50].
 N is in range [0,50].

 */
public class _576 {
    public static class Solution1 {
        /**
         * reference: https://leetcode.com/articles/out-of-boundary-paths/#approach-2-
recursion-with-memoization-accepted
         */
        public int findPaths(int m, int n, int N, int x, int y) {
            int M = 1000000000 + 7;
            int[][] dp = new int[m][n];
            dp[x][y] = 1;
            int count = 0;
            for (int moves = 1; moves <= N; moves++) {
                int[][] temp = new int[m][n];
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        if (i == m - 1) {
                            count = (count + dp[i][j]) % M;
                        }
```

```
                        if (j == n - 1) {
                            count = (count + dp[i][j]) % M;
                        }
                        if (i == 0) {
                            count = (count + dp[i][j]) % M;
                        }
                        if (j == 0) {
                            count = (count + dp[i][j]) % M;
                        }
                        temp[i][j] = (((i > 0 ? dp[i - 1][j] : 0) + (i < m - 1 ? dp[i +
1][j] : 0)) % M
                                + ((j > 0 ? dp[i][j - 1] : 0) + (j < n - 1 ? dp[i][j + 1]
: 0)) % M) % M;
                    }
                }
                dp = temp;
            }
            return count;
        }
    }

}

// ============================================================
// FILE 537: _581.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 581. Shortest Unsorted Continuous Subarray
 *
 * Given an integer array, you need to find one continuous subarray that if you only sort
this subarray in ascending order, then the whole array will be sorted in ascending order,
too.

 You need to find the shortest such subarray and output its length.

 Example 1:
 Input: [2, 6, 4, 8, 10, 9, 15]
 Output: 5
 Explanation: You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole
array sorted in ascending order.

 Note:
 Then length of the input array is in range [1, 10,000].
 The input array may contain duplicates, so ascending order here means <=.

 */
public class _581 {

    public static class Solution1 {
```

```
        /**
         * credit: https://discuss.leetcode.com/topic/89282/java-o-n-time-o-1-space
         * Use start and end to keep track of the minimum subarray nums[start...end]
which must be sorted for the entire array nums.
         * If start < end < 0 at the end of the for loop, then the array is already fully
sorted.
         *
         * Time: O(n)
         * Space: O(1)
         */
        public int findUnsortedSubarray(int[] nums) {
            int n = nums.length;
            int start = -1;
            int end = -2;
            int min = nums[n - 1];
            int max = nums[0];
            for (int i = 1; i < n; i++) {
                max = Math.max(max, nums[i]);
                min = Math.min(min, nums[n - 1 - i]);
                if (nums[i] < max) {
                    end = i;
                }
                if (nums[n - 1 - i] > min) {
                    start = n - 1 - i;
                }
            }
            return end - start + 1;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(nlogn)
         * Space: O(n)
         */
        public int findUnsortedSubarray(int[] nums) {
            int[] clones = nums.clone();
            Arrays.sort(clones);
            int start = nums.length;
            int end = 0;
            for (int i = 0; i < nums.length; i++) {
                if (clones[i] != nums[i]) {
                    start = Math.min(start, i);
                    end = Math.max(end, i);
                }
            }
            return (end - start > 0) ? end - start + 1 : 0;
        }
    }

}

// ==========================================================
// FILE 538: _582.java
```

```
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;

/**
 * 582. Kill Process
 *
 * Given n processes, each process has a unique PID (process id) and its PPID (parent
process id).

 Each process only has one parent process, but may have one or more children processes.
This is just like a tree structure. Only one process has PPID that is 0, which means this
process has no parent process. All the PIDs will be distinct positive integers.

 We use two list of integers to represent a list of processes, where the first list
contains PID for each process and the second list contains the corresponding PPID.

 Now given the two lists, and a PID representing a process you want to kill, return a
list of PIDs of processes that will be killed in the end. You should assume that when a
process is killed, all its children processes will be killed. No order is required for
the final answer.

 Example 1:
 Input:
 pid =  [1, 3, 10, 5]
 ppid = [3, 0, 5, 3]
 kill = 5
 Output: [5,10]

 Explanation:
     3
   /   \
  1     5
       /
      10

 Kill 5 will also kill 10.

 Note:
 The given kill id is guaranteed to be one of the given PIDs.
 n >= 1.
 */
public class _582 {

    public static class Solution1 {
        public List<Integer> killProcess(List<Integer> pid, List<Integer> ppid, int kill)
{
```

```
            Map<Integer, List<Integer>> map = new HashMap<>();
            for (int i = 0; i < pid.size(); i++) {
                map.putIfAbsent(ppid.get(i), new LinkedList<>());
                map.get(ppid.get(i)).add(pid.get(i));
            }
            List<Integer> result = new LinkedList<>();
            Deque<Integer> stack = new ArrayDeque<>();
            stack.offer(kill);
            while (!stack.isEmpty()) {
                int curr = stack.poll();
                result.add(curr);
                List<Integer> list = map.get(curr);
                if (list != null) {
                    stack.addAll(list);
                }
            }
            return result;
        }
    }

}

// ============================================================
// FILE 539: _583.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 583. Delete Operation for Two Strings
 *
 * Given two words word1 and word2, find the minimum number of steps required to make
word1 and word2 the same, where in each step you can delete one character in either
string.

 Example 1:
 Input: "sea", "eat"
 Output: 2

 Explanation: You need one step to make "sea" to "ea" and another step to make "eat" to
"ea".

 Note:
 The length of given words won't exceed 500.
 Characters in given words can only be lower-case letters.
 */
public class _583 {

    public static class Solution1 {

        public int minDistance(String word1, String word2) {
            int m = word1.length();
            int n = word2.length();
            int[][] dp = new int[m + 1][n + 1];
```

```
            for (int i = 1; i <= m; i++) {
                for (int j = 1; j <= n; j++) {
                    dp[i][j] = word1.charAt(i - 1) == word2.charAt(j - 1) ? dp[i - 1][j - 
1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
            return m + n - 2 * dp[m][n];
        }
    }

}

// ============================================================
// FILE 540: _587.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Point;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 587. Erect the Fence
 *
 * There are some trees, where each tree is represented by (x,y) coordinate in a two-
dimensional garden.
 * Your job is to fence the entire garden using the minimum length of rope as it is 
expensive.
 * The garden is well fenced only if all the trees are enclosed.
 * Your task is to help find the coordinates of trees which are exactly located on the 
fence perimeter.

 Example 1:
 Input: [[1,1],[2,2],[2,0],[2,4],[3,3],[4,2]]
 Output: [[1,1],[2,0],[4,2],[3,3],[2,4]]
 Explanation:

 Example 2:
 Input: [[1,2],[2,2],[4,2]]
 Output: [[1,2],[2,2],[4,2]]
 Explanation:

 Even you only have trees in a line, you need to use rope to enclose them.

 Note:
 All trees should be enclosed together. You cannot cut the rope to enclose trees that 
will separate them in more than one group.
 All input integers will range from 0 to 100.
 The garden has at least one tree.
 All coordinates are distinct.
```

```
 Input points have NO order. No order required for output.
 */
public class _587 {
    public static class Solution1 {

        /**
         * credit: https://discuss.leetcode.com/topic/89323/java-solution-convex-hull-
algorithm-gift-wrapping-aka-jarvis-march
         * There are couple of ways to solve Convex Hull problem.
https://en.wikipedia.org/wiki/Convex_hull_algorithms
         * The following code implements Gift wrapping aka Jarvis march algorithm
         * https://en.wikipedia.org/wiki/Gift_wrapping_algorithm and
         * also added logic to handle case of multiple Points in a line
         * because original Jarvis march algorithm assumes no three points are collinear.
         * It also uses knowledge in this problem https://leetcode.com/problems/convex-
polygon.
         * Disscussion: https://discuss.leetcode.com/topic/70706/beyond-my-knowledge-
java-solution-with-in-line-explanation
         */
        public List<Point> outerTrees(Point[] points) {
            Set<Point> result = new HashSet<>();

            // Find the leftmost point
            Point first = points[0];
            int firstIndex = 0;
            for (int i = 1; i < points.length; i++) {
                if (points[i].x < first.x) {
                    first = points[i];
                    firstIndex = i;
                }
            }
            result.add(first);

            Point cur = first;
            int curIndex = firstIndex;
            do {
                Point next = points[0];
                int nextIndex = 0;
                for (int i = 1; i < points.length; i++) {
                    if (i == curIndex) {
                        continue;
                    }
                    int cross = crossProductLength(cur, points[i], next);
                    if (nextIndex == curIndex || cross > 0
                            // Handle collinear points
                            || (cross == 0 && distance(points[i], cur) > distance(next,
cur))) {
                        next = points[i];
                        nextIndex = i;
                    }
                }
                // Handle collinear points
                for (int i = 0; i < points.length; i++) {
                    if (i == curIndex) {
```

```
                        continue;
                    }
                    int cross = crossProductLength(cur, points[i], next);
                    if (cross == 0) {
                        result.add(points[i]);
                    }
                }

                cur = next;
                curIndex = nextIndex;

            } while (curIndex != firstIndex);

            return new ArrayList<>(result);
        }

        private int crossProductLength(Point A, Point B, Point C) {
            // Get the vectors' coordinates.
            int BAx = A.x - B.x;
            int BAy = A.y - B.y;
            int BCx = C.x - B.x;
            int BCy = C.y - B.y;

            // Calculate the Z coordinate of the cross product.
            return (BAx * BCy - BAy * BCx);
        }

        private int distance(Point p1, Point p2) {
            return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
        }
    }

}

// ============================================================
// FILE 541: _588.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 588. Design In-Memory File System
 *
 * Design an in-memory file system to simulate the following functions:

 ls: Given a path in string format. If it is a file path, return a list that only
contains this file's name.
```

```
 If it is a directory path, return the list of file and directory names in this
directory.
 Your output (file and directory names together) should in lexicographic order.

 mkdir: Given a directory path that does not exist,
 you should make a new directory according to the path.
 If the middle directories in the path don't exist either,
 you should create them as well. This function has void return type.

 addContentToFile: Given a file path and file content in string format.
 If the file doesn't exist, you need to create that file containing given content.
 If the file already exists, you need to append given content to original content.
 This function has void return type.

 readContentFromFile: Given a file path, return its content in string format.

 Example:
 Input:
 ["FileSystem","ls","mkdir","addContentToFile","ls","readContentFromFile"]
 [[],["/"],["/a/b/c"],["/a/b/c/d","hello"],["/"],["/a/b/c/d"]]
 Output:
 [null,[],null,null,["a"],"hello"]

 Note:

 You can assume all file or directory paths are absolute paths which begin with / and do
not end with / except that the path is just "/".
 You can assume that all operations will be passed valid parameters and users will not
attempt to retrieve file content or list a directory or file that does not exist.
 You can assume that all directory names and file names only contain lower-case letters,
and same names won't exist in the same directory.

 */
public class _588 {
    public static class Solution1 {
        /**
         * Credit: https://github.com/lydxlx1/LeetCode/blob/master/src/_588.java
         */
        public static class FileSystem {

            class TrieNode {
                boolean isFile = false;
                Map<String, TrieNode> map;
                String name;
                StringBuilder stringBuilder;

                TrieNode(String name) {
                    this.name = name;
                    map = new HashMap<>();
                    stringBuilder = new StringBuilder();
                }
            }

            TrieNode root = new TrieNode("");
```

```
            TrieNode dfs(String path) {
                TrieNode node = root;
                for (String eachPath : path.split("/")) {
                    if (eachPath.isEmpty()) {
                        continue;
                    }
                    if (!node.map.containsKey(eachPath)) {
                        node.map.put(eachPath, new TrieNode(eachPath));
                    }
                    node = node.map.get(eachPath);
                }
                return node;
            }

            public FileSystem() {
            }

            public List<String> ls(String path) {
                TrieNode node = dfs(path);
                if (node.isFile) {
                    return Arrays.asList(node.name);
                } else {
                    List<String> files = new ArrayList(node.map.keySet());
                    Collections.sort(files);
                    return files;
                }
            }

            public void mkdir(String path) {
                dfs(path);
            }

            public void addContentToFile(String filePath, String content) {
                TrieNode node = dfs(filePath);
                node.isFile = true;
                node.stringBuilder.append(content);
            }

            public String readContentFromFile(String filePath) {
                return dfs(filePath).stringBuilder.toString();
            }
        }
    }

/**
 * Your FileSystem object will be instantiated and called as such:
 * FileSystem obj = new FileSystem();
 * List<String> param_1 = obj.ls(path);
 * obj.mkdir(path);
 * obj.addContentToFile(filePath,content);
 * String param_4 = obj.readContentFromFile(filePath);
 */
}
```

```
// ============================================================
// FILE 542: _589.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.List;

/**
 * 589. N-ary Tree Preorder Traversal
 *
 * Given an n-ary tree, return the preorder traversal of its nodes' values.
 *
 * For example, given a 3-ary tree:
 *
 *            1
 *          / | \
 *         3  2  4
 *        / \
 *       5   6
 *
 * Return its preorder traversal as: [1,3,5,6,2,4].
 *
 * Note:
 *
 * Recursive solution is trivial, could you do it iteratively?
 */
public class _589 {
  public static class Solution1 {
    public List<Integer> preorder(Node root) {
      List<Integer> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      dfs(root, result);
      return result;
    }

    private void dfs(Node root, List<Integer> result) {
      if (root == null) {
        return;
      }
      result.add(root.val);
      if (root.children.size() > 0) {
        for (Node child : root.children) {
          dfs(child, result);
        }
      }
    }
  }
}
```

```
// ============================================================
// FILE 543: _590.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Node;
import java.util.ArrayList;
import java.util.List;

/**
 * 590. N-ary Tree Postorder Traversal
 *
 * Given an n-ary tree, return the postorder traversal of its nodes' values.
 *
 * For example, given a 3-ary tree:
 *
 *              1
 *            / | \
 *           3  2  4
 *          / \
 *         5   6
 *
 * Return its postorder traversal as: [5,6,3,2,4,1].
 *
 * Note:
 *
 * Recursive solution is trivial, could you do it iteratively?
 */
public class _590 {
  public static class Solution1 {
    public List<Integer> postorder(Node root) {
      List<Integer> result = new ArrayList<>();
      if (root == null) {
        return result;
      }
      dfs(root, result);
      result.add(root.val);
      return result;
    }

    private void dfs(Node root, List<Integer> result) {
      if (root == null) {
        return;
      }
      if (root.children.size() > 0) {
        for (Node child : root.children) {
          dfs(child, result);
          result.add(child.val);
        }
      }
    }
  }
```

```
}

// ============================================================
// FILE 544: _591.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 591. Tag Validator
 *
 * Given a string representing a code snippet,
 * you need to implement a tag validator to parse the code and return whether it is
valid.
 * A code snippet is valid if all the following rules hold:

 1. The code must be wrapped in a valid closed tag. Otherwise, the code is invalid.

 2. A closed tag (not necessarily valid) has exactly the following format :
<TAG_NAME>TAG_CONTENT</TAG_NAME>.
 Among them, <TAG_NAME> is the start tag, and </TAG_NAME> is the end tag.
 The TAG_NAME in start and end tags should be the same.
 A closed tag is valid if and only if the TAG_NAME and TAG_CONTENT are valid.

 3. A valid TAG_NAME only contain upper-case letters, and has length in range [1,9].
Otherwise, the TAG_NAME is invalid.

 4. A valid TAG_CONTENT may contain other valid closed tags, cdata and any characters
(see note1) EXCEPT unmatched <,
 unmatched start and end tag, and unmatched or closed tags with invalid TAG_NAME.
Otherwise, the TAG_CONTENT is invalid.

 5. A start tag is unmatched if no end tag exists with the same TAG_NAME, and vice versa.
 However, you also need to consider the issue of unbalanced when tags are nested.


6. A < is unmatched if you cannot find a subsequent >. And when you find a < or </,
 all the subsequent characters until the next > should be parsed as TAG_NAME (not
necessarily valid).

7. The cdata has the following format : <![CDATA[CDATA_CONTENT]]>.
 The range of CDATA_CONTENT is defined as the characters between <![CDATA[ and the first
subsequent ]]>.

 8. CDATA_CONTENT may contain any characters.
 The function of cdata is to forbid the validator to parse CDATA_CONTENT,
 so even it has some characters that can be parsed as tag (no matter valid or invalid),
you should treat it as regular characters.

 Valid Code Examples:
 Input: "<DIV>This is the first line <![CDATA[<div>]]></DIV>"
```

```
 Output: True

 Explanation:

 The code is wrapped in a closed tag : <DIV> and </DIV>.

 The TAG_NAME is valid, the TAG_CONTENT consists of some characters and cdata.

 Although CDATA_CONTENT has unmatched start tag with invalid TAG_NAME, it should be
considered as plain text, not parsed as tag.

 So TAG_CONTENT is valid, and then the code is valid. Thus return true.


 Input: "<DIV>>>  ![cdata[]] <![CDATA[<div>]>]]>]]>>]</DIV>"

 Output: True

 Explanation:

 We first separate the code into : start_tag|tag_content|end_tag.

 start_tag -> "<DIV>"

 end_tag -> "</DIV>"

 tag_content could also be separated into : text1|cdata|text2.

 text1 -> ">>  ![cdata[]] "

 cdata -> "<![CDATA[<div>]>]]>", where the CDATA_CONTENT is "<div>]>"

 text2 -> "]]>>]"


 The reason why start_tag is NOT "<DIV>>>" is because of the rule 6.
 The reason why cdata is NOT "<![CDATA[<div>]>]]>]]>" is because of the rule 7.
 Invalid Code Examples:
 Input: "<A>  <B> </A>   </B>"
 Output: False
 Explanation: Unbalanced. If "<A>" is closed, then "<B>" must be unmatched, and vice
versa.

 Input: "<DIV>  div tag is not closed  <DIV>"
 Output: False

 Input: "<DIV>  unmatched <  </DIV>"
 Output: False

 Input: "<DIV> closed tags with invalid tag name  <b>123</b> </DIV>"
 Output: False

 Input: "<DIV> unmatched tags with invalid tag name  </1234567890> and <CDATA[[]]>
```

```
</DIV>"
 Output: False

 Input: "<DIV>  unmatched start tag <B>  and unmatched end tag </C>  </DIV>"
 Output: False

 Note:
 For simplicity, you could assume the input code (including the any characters mentioned
above)
 only contain letters, digits, '<','>','/','!','[',']' and ' '.
 */
public class _591 {

    public static class Solution1 {

        /**
         * Credit: https://discuss.leetcode.com/topic/91300/java-solution-use-startswith-
and-indexof
         */
        public boolean isValid(String code) {
            Deque<String> stack = new ArrayDeque<>();
            for (int i = 0; i < code.length(); ) {
                if (i > 0 && stack.isEmpty()) {
                    return false;
                }
                if (code.startsWith("<![CDATA[", i)) {
                    int j = i + 9;//"<![CDATA[" length is 9
                    i = code.indexOf("]]>", j);
                    if (i < 0) {
                        return false;
                    }
                    i += 3;//"]]>" length is 3
                } else if (code.startsWith("</", i)) {
                    int j = i + 2;
                    i = code.indexOf(">", j);
                    if (i < 0 || i == j || i - j > 9) {
                        return false;
                    }
                    for (int k = j; k < i; k++) {
                        if (!Character.isUpperCase(code.charAt(k))) {
                            return false;
                        }
                    }
                    String s = code.substring(j, i++);
                    if (stack.isEmpty() || !stack.pop().equals(s)) {
                        return false;
                    }
                } else if (code.startsWith("<", i)) {
                    int j = i + 1;
                    i = code.indexOf(">", j);
                    if (i < 0 || i == j || i - j > 9) {
                        return false;
                    }
                    for (int k = j; k < i; k++) {
```

```
                        if (!Character.isUpperCase(code.charAt(k))) {
                            return false;
                        }
                    }
                    String s = code.substring(j, i++);
                    stack.push(s);
                } else {
                    i++;
                }
            }
            return stack.isEmpty();
        }
    }

}

// ============================================================
// FILE 545: _592.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 592. Fraction Addition and Subtraction
 *
 * Given a string representing an expression of fraction addition and subtraction,
 * you need to return the calculation result in string format.
 * The final result should be irreducible fraction.
 * If your final result is an integer,
 * say 2, you need to change it to the format of fraction that has denominator 1.
 * So in this case, 2 should be converted to 2/1.

 Example 1:
 Input:"-1/2+1/2"
 Output: "0/1"

 Example 2:
 Input:"-1/2+1/2+1/3"
 Output: "1/3"

 Example 3:
 Input:"1/3-1/2"
 Output: "-1/6"

 Example 4:
 Input:"5/3+1/3"
 Output: "2/1"

 Note:
 The input string only contains '0' to '9', '/', '+' and '-'. So does the output.
 Each fraction (input and output) has format ±numerator/denominator.
```

```
 If the first input fraction or the output is positive, then '+' will be omitted.
 The input only contains valid irreducible fractions,
 where the numerator and denominator of each fraction will always be in the range [1,10].
 If the denominator is 1, it means this fraction is actually an integer in a fraction
format defined above.
 The number of given fractions will be in the range [1,10].
 The numerator and denominator of the final result are guaranteed to be valid and in the
range of 32-bit int.
 */
public class _592 {

    public static class Solution1 {

        /**
         * Credit: https://discuss.leetcode.com/topic/89993/java-solution-fraction-
addition-and-gcd
         */
        public String fractionAddition(String expression) {
            List<String> nums = new ArrayList<>();
            int i = 0;
            int j = 0;
            while (j <= expression.length()) {
                if (j == expression.length() || j != i && (expression.charAt(j) == '-' ||
expression.charAt(j) == '+')) {
                    if (expression.charAt(i) == '+') {
                        nums.add(expression.substring(i + 1, j));
                    } else {
                        nums.add(expression.substring(i, j));
                    }
                    i = j;
                }
                j++;
            }

            String result = "0/1";
            for (String frac : nums) {
                result = add(result, frac);
            }
            return result;
        }

        private String add(String result, String frac) {
            String[] frac1 = frac.split("/");
            String[] frac2 = result.split("/");
            int n1 = Integer.parseInt(frac1[0]);
            int d1 = Integer.parseInt(frac1[1]);
            int n2 = Integer.parseInt(frac2[0]);
            int d2 = Integer.parseInt(frac2[1]);
            int numerator = n1 * d2 + n2 * d1;
            int denominator = d1 * d2;
            if (numerator == 0) {
                return "0/1";
            }
```

```
            boolean negative = numerator * denominator < 0;
            numerator = Math.abs(numerator);
            denominator = Math.abs(denominator);
            int gcd = getGCD(numerator, denominator);

            return (negative ? "-" : "") + (numerator / gcd) + "/" + (denominator / gcd);
        }

        private int getGCD(int a, int b) {
            if (a == 0 || b == 0) {
                return a + b;
            }
            return getGCD(b, a % b);
        }
    }

}

// ============================================================
// FILE 546: _593.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 593. Valid Square
 *
 * Given the coordinates of four points in 2D space, return whether the four points could
construct a square.

 The coordinate (x,y) of a point is represented by an integer array with two integers.

 Example:
 Input: p1 = [0,0], p2 = [1,1], p3 = [1,0], p4 = [0,1]
 Output: True
 Note:
 All the input integers are in the range [-10000, 10000].
 A valid square has four equal sides with positive length and four equal angles (90-
degree angles).
 Input points have no order.

 */
public class _593 {
    public static class Solution1 {
        /**
         * Note: I don't need to use backtracking to find all permutations, this is an
overkill.
         * This is the most easy one: https://leetcode.com/articles/kill-process-
2/#approach-3-checking-every-case-accepted
         */
```

```
        public boolean validSquare(int[] p1, int[] p2, int[] p3, int[] p4) {
            List<int[]> input = new ArrayList<>(Arrays.asList(p1, p2, p3, p4));
            List<List<int[]>> allPermuations = getAllPermutations(input);
            for (List<int[]> eachPermutation : allPermuations) {
                if (isValid(eachPermutation)) {
                    return true;
                }
            }
            return false;
        }

        private List<List<int[]>> getAllPermutations(List<int[]> input) {
            List<List<int[]>> result = new ArrayList();
            List<int[]> init = new ArrayList<>();
            result.add(init);
            return backTracking(result, input, 0);
        }

        private List<List<int[]>> backTracking(List<List<int[]>> result, List<int[]>
input, int pos) {
            if (pos == input.size()) {
                return result;
            }
            List<List<int[]>> newResult = new ArrayList<>();
            for (List<int[]> eachList : result) {
                for (int i = 0; i <= eachList.size(); i++) {
                    List<int[]> newList = new ArrayList<>(eachList);
                    newList.add(i, input.get(pos));
                    newResult.add(newList);
                }
            }
            result = newResult;
            return backTracking(result, input, pos + 1);
        }

        private boolean isValid(List<int[]> points) {
            int[] p1 = points.get(0);
            int[] p2 = points.get(1);
            int[] p3 = points.get(2);
            int[] p4 = points.get(3);
            double distance = (Math.pow(p1[0] - p2[0], 2) + Math.pow(p1[1] - p2[1], 2));
            return distance == (Math.pow(p2[0] - p3[0], 2) + Math.pow(p2[1] - p3[1], 2))
                    && distance == (Math.pow(p3[0] - p4[0], 2) + Math.pow(p3[1] - p4[1],
2))
                    && distance == (Math.pow(p4[0] - p1[0], 2) + Math.pow(p4[1] - p1[1],
2))
                    && isRightAngle(p1, p2, p3)
                    && noDuplicate(p1, p2, p3, p4);
        }

        public boolean noDuplicate(int[] p1, int[] p2, int[] p3, int[] p4) {
            return !Arrays.equals(p1, p2)
                    && !Arrays.equals(p1, p3)
```

```
                    && !Arrays.equals(p1, p4)
                    && !Arrays.equals(p2, p3)
                    && !Arrays.equals(p2, p4)
                    && !Arrays.equals(p3, p4);
        }

        public boolean isRightAngle(int[] p1, int[] p2, int[] p3) {
            double angle1 = Math.atan2(p2[1] - p1[1], p2[0] - p1[0]);
            double angle2 = Math.atan2(p3[1] - p1[1], p3[0] - p1[0]);
            double degree = Math.toDegrees(angle1 - angle2);
            return degree % 45 == 0;
        }
    }

}

// ============================================================
// FILE 547: _594.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 594. Longest Harmonious Subsequence
 *
 * We define a harmonious array is an array where the difference between its maximum
value and its minimum value is exactly 1.

 Now, given an integer array, you need to find the length of its longest harmonious
subsequence among all its possible subsequences.

 Example 1:
 Input: [1,3,2,2,5,2,3,7]
 Output: 5

 Explanation: The longest harmonious subsequence is [3,2,2,2,3].
 Note: The length of the input array will not exceed 20,000.

 */
public class _594 {
    public static class Solution1 {
        public int findLHS(int[] nums) {
            Map<Integer, Integer> map = new HashMap<>();
            for (int i : nums) {
                map.put(i, map.getOrDefault(i, 0) + 1);
            }
            int max = 0;
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(nums[i] + 1)) {
                    max = Math.max(max, map.get(nums[i]) + map.get(nums[i] + 1));
                }
```

```
            }
            return max;
        }
    }
}

// ============================================================
// FILE 548: _598.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 598. Range Addition II

 Given an m * n matrix m initialized with all 0's and several update operations.
 Operations are represented by a 2D array,
 and each operation is represented by an array with two positive integers a and b,
 which means m[i][j] should be added by one for all 0 <= i < a and 0 <= j < b.

 You need to count and return the number of maximum integers in the matrix after
performing all the operations.

 Example 1:
 Input:
 m = 3, n = 3
 operations = [[2,2],[3,3]]
 Output: 4

 Explanation:
 Initially, m =
 [[0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]

 After performing [2,2], m =
 [[1, 1, 0],
 [1, 1, 0],
 [0, 0, 0]]

 After performing [3,3], m =
 [[2, 2, 1],
 [2, 2, 1],
 [1, 1, 1]]

 So the maximum integer in m is 2, and there are four of it in m. So return 4.
 Note:
 The range of m and n is [1,40000].
 The range of a is [1,m], and the range of b is [1,n].
 The range of operations size won't exceed 10,000.
 */

public class _598 {
```

```
    public static class Solution1 {
        public int maxCount(int m, int n, int[][] ops) {
            int x = m;
            int y = n;
            for (int[] op : ops) {
                x = Math.min(x, op[0]);
                y = Math.min(y, op[1]);
            }
            return x * y;
        }
    }
}

// ============================================================
// FILE 549: _599.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 599. Minimum Index Sum of Two Lists
 *
 Suppose Andy and Doris want to choose a restaurant for dinner,
 and they both have a list of favorite restaurants represented by strings.
 You need to help them find out their common interest with the least list index sum.
 If there is a choice tie between answers, output all of them with no order requirement.
You could assume there always exists an answer.

 Example 1:
 Input:
 ["Shogun", "Tapioca Express", "Burger King", "KFC"]
 ["Piatti", "The Grill at Torrey Pines", "Hungry Hunter Steakhouse", "Shogun"]
 Output: ["Shogun"]
 Explanation: The only restaurant they both like is "Shogun".

 Example 2:
 Input:
 ["Shogun", "Tapioca Express", "Burger King", "KFC"]
 ["KFC", "Shogun", "Burger King"]
 Output: ["Shogun"]
 Explanation: The restaurant they both like and have the least index sum is "Shogun" with
index sum 1 (0+1).

 Note:
 The length of both lists will be in the range of [1, 1000].
 The length of strings in both lists will be in the range of [1, 30].
 The index is starting from 0 to the list length minus 1.
 No duplicates in both lists.
 */
```

```
public class _599 {
    public static class Solution1 {
        public String[] findRestaurant(String[] list1, String[] list2) {
            if (list1 == null || list2 == null) {
                return new String[0];
            }
            Map<String, Integer> map1 = putIntoMap(list1);
            Map<String, Integer> map2 = putIntoMap(list2);
            int leastIndexSum = Integer.MAX_VALUE;
            List<String> resultList = new ArrayList<>();
            for (String key1 : map1.keySet()) {
                if (map2.containsKey(key1)) {
                    int indexSum = map1.get(key1) + map2.get(key1);
                    if (indexSum < leastIndexSum) {
                        resultList.clear();
                        resultList.add(key1);
                        leastIndexSum = indexSum;
                    } else if (indexSum == leastIndexSum) {
                        resultList.add(key1);
                    }
                }
            }
            String[] result = new String[resultList.size()];
            for (int i = 0; i < resultList.size(); i++) {
                result[i] = resultList.get(i);
            }
            return result;
        }

        private Map<String, Integer> putIntoMap(String[] list) {
            Map<String, Integer> map = new HashMap<>();
            for (int i = 0; i < list.length; i++) {
                map.put(list[i], i);
            }
            return map;
        }
    }
}

// ============================================================
// FILE 550: _600.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 600. Non-negative Integers without Consecutive Ones
 *
 * Given a positive integer n, find the number of non-negative integers less than or
equal to n, whose binary representations do NOT contain consecutive ones.

 Example 1:
 Input: 5
 Output: 5
```

```
 Explanation:
 Here are the non-negative integers <= 5 with their corresponding binary representations:
 0 : 0
 1 : 1
 2 : 10
 3 : 11
 4 : 100
 5 : 101
 Among them, only integer 3 disobeys the rule (two consecutive ones) and the other 5
satisfy the rule.

 Note: 1 <= n <= 109
 */
public class _600 {

    public static class Solution1 {
        /**
         * Credit: https://leetcode.com/articles/non-negative-integers-without-
consecutive-ones/#approach-3-using-bit-manipulation-accepted
         */
        public int findIntegers(int num) {
            int[] f = new int[32];
            f[0] = 1;
            f[1] = 2;
            for (int i = 2; i < f.length; i++) {
                f[i] = f[i - 1] + f[i - 2];
            }
            int i = 30;
            int sum = 0;
            int prevBit = 0;
            while (i >= 0) {
                if ((num & (1 << i)) != 0) {
                    sum += f[i];
                    if (prevBit == 1) {
                        sum--;
                        break;
                    }
                    prevBit = 1;
                } else {
                    prevBit = 0;
                }
                i--;
            }
            return sum + 1;
        }
    }

}

// ============================================================
// FILE 551: _604.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 604. Design Compressed String Iterator
 *
 * Design and implement a data structure for a compressed string iterator. It should
support the following operations: next and hasNext.
 The given compressed string will be in the form of each letter followed by a positive
integer representing
 the number of this letter existing in the original uncompressed string.

 next() - if the original string still has uncompressed characters, return the next
letter; Otherwise return a white space.
 hasNext() - Judge whether there is any letter needs to be uncompressed.

 Note:
 Please remember to RESET your class variables declared in StringIterator,
 as static/class variables are persisted across multiple test cases. Please see here for
more details.

 Example:

 StringIterator iterator = new StringIterator("L1e2t1C1o1d1e1");

 iterator.next(); // return 'L'
 iterator.next(); // return 'e'
 iterator.next(); // return 'e'
 iterator.next(); // return 't'
 iterator.next(); // return 'C'
 iterator.next(); // return 'o'
 iterator.next(); // return 'd'
 iterator.hasNext(); // return true
 iterator.next(); // return 'e'
 iterator.hasNext(); // return false
 iterator.next(); // return ' '

 */
public class _604 {
    public static class Solution1 {
        public static class StringIterator {

            Deque<int[]> deque;

            public StringIterator(String compressedString) {
                deque = new ArrayDeque<>();
                int len = compressedString.length();
                int i = 0;
                while (i < len) {
                    int j = i + 1;
                    while (j < len && Character.isDigit(compressedString.charAt(j))) {
                        j++;
                    }
```

```
                    deque.addLast(new int[]{compressedString.charAt(i) - 'A',
Integer.parseInt(compressedString.substring(i + 1, j))});
                    i = j;
                }
            }

            public char next() {
                if (deque.isEmpty()) {
                    return ' ';
                }
                int[] top = deque.peek();
                top[1]--;
                if (top[1] == 0) {
                    deque.pollFirst();
                }
                return (char) ('A' + top[0]);
            }

            public boolean hasNext() {
                return !deque.isEmpty();
            }
        }
    }
}

// ============================================================
// FILE 552: _605.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 605. Can Place Flowers
 *
 * Suppose you have a long flowerbed in which some of the plots are planted and some are
not.
 * However, flowers cannot be planted in adjacent plots - they would compete for water
and both would die.

 Given a flowerbed (represented as an array containing 0 and 1, where 0 means empty and 1
means not empty),
 and a number n, return if n new flowers can be planted in it without violating the no-
adjacent-flowers rule.

 Example 1:
 Input: flowerbed = [1,0,0,0,1], n = 1
 Output: True

 Example 2:
 Input: flowerbed = [1,0,0,0,1], n = 2
 Output: False

 Note:
 The input array won't violate no-adjacent-flowers rule.
```

```
 The input array size is in the range of [1, 20000].
 n is a non-negative integer which won't exceed the input array size.

 */
public class _605 {

    public static class Solution1 {
        public boolean canPlaceFlowers(int[] flowerbed, int n) {
            int count = 0;
            int i = 0;
            while (i < flowerbed.length) {
                if (flowerbed[i] == 0 && (i == 0 || flowerbed[i - 1] == 0) && (i ==
flowerbed.length - 1 || flowerbed[i + 1] == 0)) {
                    count++;
                    flowerbed[i] = 1;
                }
                if (count >= n) {
                    return true;
                }
                i++;
            }
            if (count >= n) {
                return true;
            }
            return false;
        }
    }

    public static class Solution2 {
        public boolean canPlaceFlowers(int[] flowerbed, int n) {
            int len = flowerbed.length;
            if (len == 1) {
                if ((flowerbed[0] == 0 && n <= 1) || n == 0) {
                    return true;
                }
                return false;
            }
            if (flowerbed[0] == 0 && flowerbed[1] == 0) {
                flowerbed[0] = 1;
                n--;
            }
            for (int i = 1; i < len - 1; i++) {
                if (flowerbed[i] == 0 && flowerbed[i - 1] == 0 && flowerbed[i + 1] == 0)
{
                    n--;
                    flowerbed[i] = 1;//modify the input, discuss this with interviwer, if
not allowed, then have a copy of this input and modify copy
                }
                if (n <= 0) {
                    return true;
                }
            }
            if (len >= 2 && flowerbed[len - 2] == 0 && flowerbed[len - 1] == 0) {
                n--;
```

```
            }
            if (n <= 0) {
                return true;
            }
            return false;
        }
    }

}

// ============================================================
// FILE 553: _606.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 606. Construct String from Binary Tree
 *
 * You need to construct a string consists of parenthesis and integers from a binary tree
with the preorder traversing way.
 The null node needs to be represented by empty parenthesis pair "()".
 And you need to omit all the empty parenthesis pairs that don't affect the one-to-one
mapping relationship between the string
 and the original binary tree.

 Example 1:
 Input: Binary tree: [1,2,3,4]
     1
   /   \
  2     3
 /
4

 Output: "1(2(4))(3)"

 Explanation: Originallay it needs to be "1(2(4())())(3()())",
 but you need to omit all the unnecessary empty parenthesis pairs.
 And it will be "1(2(4))(3)".

 Example 2:
 Input: Binary tree: [1,2,3,null,4]
    1
  /   \
 2     3
  \
  4

 Output: "1(2()(4))(3)"

 Explanation: Almost the same as the first example,
 except we can't omit the first parenthesis pair to break the one-to-one mapping
```

```
relationship between the input and the output.
 */
public class _606 {

    public static class Solution1 {
        public String tree2str(TreeNode t) {
            if (t == null) {
                return "";
            }
            String result = "" + t.val;
            String left = tree2str(t.left);
            String right = tree2str(t.right);
            if (left.equals("") && right.equals("")) {
                return result;
            }
            if (left.equals("")) {
                return result + "()(" + right + ")";
            }
            if (right.equals("")) {
                return result + "(" + left + ")";
            }
            return result + "(" + left + ")(" + right + ")";
        }
    }

    public static class Solution2 {
        public String tree2str(TreeNode t) {
            if (t == null) {
                return "";
            }
            StringBuilder sb = new StringBuilder();
            preorder(t, sb);
            return sb.toString();
        }

        private void preorder(TreeNode root, StringBuilder sb) {
            if (root == null) {
                return;
            }
            sb.append(root.val);
            if (root.left != null) {
                sb.append("(");
                preorder(root.left, sb);
                sb.append(")");
            }
            if (root.right != null) {
                if (root.left == null) {
                    sb.append("()");
                }
                sb.append("(");
                preorder(root.right, sb);
                sb.append(")");
            }
        }
```

```
    }

}

// ============================================================
// FILE 554: _609.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 609. Find Duplicate File in System
 *
 * Given a list of directory info including directory path, and all the files with
contents in this directory,
 * you need to find out all the groups of duplicate files in the file system in terms of
their paths.
 * A group of duplicate files consists of at least two files that have exactly the same
content.

 A single directory info string in the input list has the following format:

 "root/d1/d2/.../dm f1.txt(f1_content) f2.txt(f2_content) ... fn.txt(fn_content)"

 It means there are n files (f1.txt, f2.txt ... fn.txt with content f1_content,
f2_content ... fn_content, respectively)
 in directory root/d1/d2/.../dm. Note that n >= 1 and m >= 0. If m = 0, it means the
directory is just the root directory.

 The output is a list of group of duplicate file paths.
 For each group, it contains all the file paths of the files that have the same content.
 A file path is a string that has the following format:

 "directory_path/file_name.txt"

 Example 1:

 Input:
 ["root/a 1.txt(abcd) 2.txt(efgh)", "root/c 3.txt(abcd)", "root/c/d 4.txt(efgh)", "root
4.txt(efgh)"]

 Output:
 [["root/a/2.txt","root/c/d/4.txt","root/4.txt"],["root/a/1.txt","root/c/3.txt"]]

 Note:
 No order is required for the final output.
 You may assume the directory name,
 file name and file content only has letters and digits, and the length of file content
is in the range of [1,50].
```

```
 The number of files given is in the range of [1,20000].
 You may assume no files or directories share the same name in a same directory.

 You may assume each given directory info represents a unique directory. Directory path
and file infos are separated by a single blank space.

 Follow up beyond contest:
 Imagine you are given a real file system, how will you search files? DFS or BFS ?
 If the file content is very large (GB level), how will you modify your solution?
 If you can only read the file by 1kb each time, how will you modify your solution?
 What is the time complexity of your modified solution? What is the most time consuming
part and memory consuming part of it? How to optimize?
 How to make sure the duplicated files you find are not false positive?
 */
public class _609 {

    public static class Solution1 {
        public List<List<String>> findDuplicate(String[] paths) {
            Map<String, List<String>> map = new HashMap<>();//key is the file content,
value is the list of directories that has this directory/file
            for (String path : paths) {
                String[] dirAndFiles = path.split(" ");
                for (int i = 1; i < dirAndFiles.length; i++) {
                    String content = dirAndFiles[i].substring(dirAndFiles[i].indexOf("(")
+ 1, dirAndFiles[i].indexOf(")"));
                    if (!map.containsKey(content)) {
                        map.put(content, new ArrayList<>());
                    }
                    List<String> dirs = map.get(content);
                    dirs.add(dirAndFiles[0] + "/" + dirAndFiles[i].substring(0,
dirAndFiles[i].indexOf("(")));
                    map.put(content, dirs);
                }
            }

            List<List<String>> result = new ArrayList<>();
            for (String content : map.keySet()) {
                if (map.get(content).size() > 1) {
                    List<String> dupFile = new ArrayList<>();
                    for (String dir : map.get(content)) {
                        dupFile.add(dir);
                    }
                    result.add(dupFile);
                }
            }
            return result;
        }
    }

    /**Answers to follow-up questions:
     * 1. Imagine you are given a real file system, how will you search files? DFS or BFS
?
     * A: Both BFS and DFS could do the work, but BFS will use extra memory, however, BFS
```

```
takes advantage of memory locality, so BFS could be faster.
     *
     * 2. If the file content is very large (GB level), how will you modify your
solution?
     * A: We'll fist map all files according to their sizes, since files with different
sizes are guaranteed to be different, then
     * we can hash a small part of the files using MD5, SHA256, etc. Only when their md5
or sha256 is the same, we'll compare the contents byte by byte.
     *
     * 3. If you can only read the file by 1kb each time, how will you modify your
solution?
     * A: This is not going to change the solution, we can hash this 1kb chunk, and then
also only compare byte by byte when it's necessary.
     *
     * 4. What is the time complexity of your modified solution? What is the most time
consuming part and memory consuming part of it? How to optimize?
     * A: O(n^2*k), in the worst time, we'll have to compare the file with every other
file, k is the length of the file.
     * Comparing the file (by size, by hash and eventually byte by byte) is the most time
consuming part.
     * Generating hash for every file will be the most memory consuming part.
     *
     * 5. How to make sure the duplicated files you find are not false positive?
     * A: Size comparision, hash detection, byte by byte check, etc. will pretty sure to
rule out false positive.
     * */

}

// ============================================================
// FILE 555: _611.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 611. Valid Triangle Number
 * Given an array consists of non-negative integers,
 * your task is to count the number of triplets chosen
 * from the array that can make triangles if we take them as side lengths of a triangle.

 Example 1:
 Input: [2,2,3,4]
 Output: 3

 Explanation:
 Valid combinations are:
 2,3,4 (using the first 2)
 2,3,4 (using the second 2)
 2,2,3

 Note:
```

```
 The length of the given array won't exceed 1000.
 The integers in the given array are in the range of [0, 1000].
 */
public class _611 {
    public static class Solution1 {
        /**
         * Rule: among three sides, we need to find whether the longest of the three is
smaller than the sum of the two shorter one.
         * If so, then these three could form a valid triangle.
         */

        public int triangleNumber(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            Arrays.sort(nums);
            int triplets = 0;
            for (int i = 2; i < nums.length; i++) {
                int left = 0;
                int right = i - 1;
                while (left < right) {
                    if (nums[i] < nums[left] + nums[right]) {
                        triplets += (right - left);
                        right--;
                    } else {
                        left++;
                    }
                }
            }
            return triplets;
        }
    }

}

// ============================================================
// FILE 556: _616.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 616. Add Bold Tag in String
 *
 * Given a string s and a list of strings dict,
 * you need to add a closed pair of bold tag <b> and </b>
 * to wrap the substrings in s that exist in dict.
 * If two such substrings overlap, you need to wrap them together by only one pair of
closed bold tag.
 * Also, if two substrings wrapped by bold tags are consecutive, you need to combine
them.

 Example 1:
 Input:
```

```
 s = "abcxyz123"
 dict = ["abc","123"]
 Output:
 "<b>abc</b>xyz<b>123</b>"

 Example 2:
 Input:
 s = "aaabbcc"
 dict = ["aaa","aab","bc"]
 Output:
 "<b>aaabbc</b>c"

 Note:
 The given dict won't contain duplicates, and its length won't exceed 100.
 All the strings in input have length in range [1, 1000].
 */
public class _616 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/92112/java-solution-boolean-array
         */
        public String addBoldTag(String s, String[] dict) {
            boolean[] shouldBold = new boolean[s.length()];
            for (int i = 0, end = 0; i < s.length(); i++) {
                for (String word : dict) {
                    if (s.startsWith(word, i)) {
                        end = Math.max(end, i + word.length());
                    }
                }
                shouldBold[i] = end > i;
            }
            StringBuilder stringBuilder = new StringBuilder();
            for (int i = 0; i < s.length(); i++) {
                if (!shouldBold[i]) {
                    stringBuilder.append(s.charAt(i));
                    continue;
                }
                int j = i;
                while (j < s.length() && shouldBold[j]) {
                    j++;
                }
                stringBuilder.append("<b>" + s.substring(i, j) + "</b>");
                i = j - 1;
            }
            return stringBuilder.toString();
        }
    }

}

// ============================================================
// FILE 557: _617.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 617. Merge Two Binary Trees
 *
 * Given two binary trees and imagine that when you put one of them to cover the other,
 * some nodes of the two trees are overlapped while the others are not.
 You need to merge them into a new binary tree.
 The merge rule is that if two nodes overlap,
 then sum node values up as the new value of the merged node.
 Otherwise, the NOT null node will be used as the node of new tree.

 Example 1:
 Input:
 Tree 1                     Tree 2
    1                         2
   / \                       / \
  3   2                     1   3
 /                           \   \
5                             4   7
 Output:
 Merged tree:
     3
    / \
   4   5
  / \   \
 5   4   7
 Note: The merging process must start from the root nodes of both trees.
 */
public class _617 {

    public static class Solution1 {
        public TreeNode mergeTrees(TreeNode t1, TreeNode t2) {
            if (t1 == null) {
                return t2;
            }
            if (t2 == null) {
                return t1;
            }
            TreeNode mergedNode = new TreeNode(t1.val + t2.val);
            mergedNode.left = mergeTrees(t1.left, t2.left);
            mergedNode.right = mergeTrees(t1.right, t2.right);
            return mergedNode;
        }
    }

}

// ============================================================
// FILE 558: _621.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * 621. Task Scheduler
 *
 * Given a char array representing tasks CPU need to do.
 * It contains capital letters A to Z where different letters represent different tasks.
 * Tasks could be done without original order.
 * Each task could be done in one interval. For each interval, CPU could finish one task
or just be idle.
 * However, there is a non-negative cooling interval n that means between two same tasks,
 * there must be at least n intervals that CPU are doing different tasks or just be idle.
 * You need to return the least number of intervals the CPU will take to finish all the
given tasks.

 Example 1:
 Input: tasks = ['A','A','A','B','B','B'], n = 2
 Output: 8
 Explanation: A -> B -> idle -> A -> B -> idle -> A -> B.

 Note:
 The number of tasks is in the range [1, 10000].
 The integer n is in the range [0, 100].
 */
public class _621 {

    public static class Solution1 {

        public int leastInterval(char[] tasks, int n) {
            Map<Character, Integer> map = new HashMap<>();
            for (char c : tasks) {
                map.put(c, map.getOrDefault(c, 0) + 1);
            }
            PriorityQueue<Task> maxHeap = new PriorityQueue<>((a, b) -> b.total -
a.total);
            for (Map.Entry<Character, Integer> entry : map.entrySet()) {
                maxHeap.offer(new Task(entry.getValue(), entry.getKey()));
            }
            int times = 0;
            while (!maxHeap.isEmpty()) {
                int i = 0;
                List<Task> temp = new ArrayList<>();
                while (i <= n) {
                    if (!maxHeap.isEmpty()) {
                        if (maxHeap.peek().total > 1) {
                            Task curr = maxHeap.poll();
                            temp.add(new Task(curr.total - 1, curr.character));
```

```
                        } else {
                            maxHeap.poll();
                        }
                    }
                    times++;
                    if (maxHeap.isEmpty() && temp.size() == 0) {
                        break;
                    }
                    i++;
                }
                for (Task task : temp) {
                    maxHeap.offer(task);
                }
            }
            return times;
        }

        class Task {
            int total;
            char character;

            public Task(int total, char character) {
                this.total = total;
                this.character = character;
            }
        }
    }

}

// ============================================================
// FILE 559: _623.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 623. Add One Row to Tree
 *
 * Given the root of a binary tree,
 * then value v and depth d,
 * you need to add a row of nodes with value v at the given depth d.
 * The root node is at depth 1.

 The adding rule is: given a positive integer depth d,
 for each NOT null tree nodes N in depth d-1,
 create two tree nodes with value v as N's left subtree root and right subtree root.
 And N's original left subtree should be the left subtree of the new left subtree root,
 its original right subtree should be the right subtree of the new right subtree root.
 If depth d is 1 that means there is no depth d-1 at all,
 then create a tree node with value v as the new root of the whole
 original tree, and the original tree is the new root's left subtree.
```

```
 Example 1:
 Input:
 A binary tree as following:
      4
    /   \
   2     6
  / \   /
 3   1 5

 v = 1

 d = 2

 Output:
       4
      / \
     1   1
    /     \
   2       6
  / \     /
 3   1   5


 Example 2:
 Input:
 A binary tree as following:
    4
   /
  2
 / \
 3   1

 v = 1

 d = 3

 Output:
        4
       /
      2
     / \
    1   1
   /     \
  3       1

 Note:
 The given d is in range [1, maximum depth of the given tree + 1].
 The given binary tree has at least one tree node.
 */
public class _623 {

    public static class Solution1 {
```

```
        public TreeNode addOneRow(TreeNode root, int v, int d) {
            if (d == 1) {
                TreeNode newRoot = new TreeNode(v);
                newRoot.left = root;
                return newRoot;
            } else {
                dfs(root, v, d);
                return root;
            }
        }

        private void dfs(TreeNode root, int v, int d) {
            if (root == null) {
                return;
            }
            if (d == 2) {
                TreeNode newLeft = new TreeNode(v);
                TreeNode newRight = new TreeNode(v);
                newLeft.left = root.left;
                newRight.right = root.right;
                root.left = newLeft;
                root.right = newRight;
            } else {
                dfs(root.left, v, d - 1);
                dfs(root.right, v, d - 1);
            }
        }
    }

}

// ============================================================
// FILE 560: _624.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 624. Maximum Distance in Arrays
 *
 * Given m arrays, and each array is sorted in ascending order.
 * Now you can pick up two integers from two different arrays (each array picks one)
 * and calculate the distance. We define the distance between two
 * integers a and b to be their absolute difference |a-b|. Your task is to find the
maximum distance.

 Example 1:
 Input:
 [[1,2,3],
 [4,5],
```

```
 [1,2,3]]

 Output: 4

 Explanation:

 One way to reach the maximum distance 4 is to pick 1 in the first or third array and
pick 5 in the second array.

 Note:
 Each given array will have at least 1 number. There will be at least two non-empty
arrays.
 The total number of the integers in all the m arrays will be in the range of [2, 10000].
 The integers in the m arrays will be in the range of [-10000, 10000].
 */
public class _624 {

    public static class Solution1 {
        public int maxDistance(int[][] arrays) {
            List<Integer> max = new ArrayList<>();
            for (int[] array : arrays) {
                max.add(array[array.length - 1]);
            }
            Collections.sort(max);
            int ans = Integer.MIN_VALUE;
            for (int[] array : arrays) {
                int big = array[array.length - 1] == max.get(max.size() - 1) ?
max.get(max.size() - 2) : max.get(max.size() - 1);
                ans = Math.max(ans, big - array[0]);
            }
            return ans;
        }
    }
}

// ============================================================
// FILE 561: _625.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 625. Minimum Factorization
 *
 * Given a positive integer a, find the smallest positive integer b whose multiplication
of each digit equals to a.

 If there is no answer or the answer is not fit in 32-bit signed integer, then return 0.

 Example 1
 Input:
```

```
 48
 Output:
 68

 Example 2
 Input:
 15
 Output:
 35
 */
public class _625 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/92854/java-solution-result-array
         * and https://leetcode.com/articles/minimum-factorization/#approach-3-using-
factorizationaccepted
         */
        public int smallestFactorization(int a) {
            //case 1: a < 10
            if (a < 10) {
                return a;
            }

            //case 2: start with 9 and try every possible digit
            List<Integer> resultArray = new ArrayList<>();
            for (int i = 9; i > 1; i--) {
                //if current digit divides a, then store all occurences of current digit
in res
                while (a % i == 0) {
                    a = a / i;
                    resultArray.add(i);
                }
            }

            //if a could not be broken in form of digits, return 0
            if (a != 0) {
                return 0;
            }

            //get the result from the result array in reverse order
            long result = 0;
            for (int i = resultArray.size() - 1; i >= 0; i--) {
                result = result * 10 + resultArray.get(i);
                if (result > Integer.MAX_VALUE) {
                    return 0;
                }
            }
            return (int) result;
        }
    }

}
```

```
// ============================================================
// FILE 562: _628.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 628. Maximum Product of Three Numbers
 *
 * Given an integer array, find three numbers whose product is maximum and output the
maximum product.
 Example 1:
 Input: [1,2,3]
 Output: 6

 Example 2:
 Input: [1,2,3,4]
 Output: 24

 Note:
 The length of the given array will be in range [3,104] and all elements are in the range
[-1000, 1000].
 Multiplication of any three numbers in the input won't exceed the range of 32-bit signed
integer.
 */
public class _628 {

    public static class Solution1 {
        public int maximumProduct(int[] nums) {
            Arrays.sort(nums);
            int product = 1;
            if (nums.length >= 3) {
                for (int i = nums.length - 1; i >= nums.length - 3; i--) {
                    product *= nums[i];
                }
                int anotherProduct = nums[0] * nums
                        [1] * nums[nums.length - 1];
                product = Math.max(product, anotherProduct);
            } else {
                for (int i = 0; i < nums.length; i++) {
                    product *= nums[i];
                }
            }
            return product;
        }
    }
}

// ============================================================
// FILE 563: _629.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 629. K Inverse Pairs Array
 *
 * Given two integers n and k, find how many different arrays consist of numbers from 1
to n such that there are exactly k inverse pairs.
 We define an inverse pair as following: For ith and jth element in the array,
 if i < j and a[i] > a[j] then it's an inverse pair; Otherwise, it's not.
 Since the answer may very large, the answer should be modulo 109 + 7.

 Example 1:
 Input: n = 3, k = 0
 Output: 1
 Explanation:
 Only the array [1,2,3] which consists of numbers from 1 to 3 has exactly 0 inverse pair.

 Example 2:
 Input: n = 3, k = 1
 Output: 2
 Explanation:
 The array [1,3,2] and [2,1,3] have exactly 1 inverse pair.

 Note:
 The integer n is in the range [1, 1000] and k is in the range [0, 1000].
 */
public class _629 {

    public static class Solution1 {

        /**
         * reference: https://leetcode.com/articles/k-inverse-pairs-array/#approach-5-
another-optimized-dynamic-programming-approachaccepted
         * and
         * https://discuss.leetcode.com/topic/93815/java-dp-o-nk-solution
         */
        public int kInversePairs(int n, int k) {
            int mod = 1000000007;
            if (k > n * (n - 1) / 2 || k < 0) {
                return 0;
            }
            if (k == 0 || k == n * (n - 1) / 2) {
                return 1;
            }
            long[][] dp = new long[n + 1][k + 1];
            dp[2][0] = 1;
            dp[2][1] = 1;
            for (int i = 3; i <= n; i++) {
                dp[i][0] = 1;
                for (int j = 1; j <= Math.min(k, i * (i - 1) / 2); j++) {
                    dp[i][j] = dp[i][j - 1] + dp[i - 1][j];
                    if (j >= i) {
                        dp[i][j] -= dp[i - 1][j - i];
                    }
```

```
                    dp[i][j] = (dp[i][j] + mod) % mod;
                }
            }
            return (int) dp[n][k];
        }
    }
}

// ============================================================
// FILE 564: _630.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * 630. Course Schedule III
 *
 * There are n different online courses numbered from 1 to n.
 * Each course has some duration(course length) t and closed on dth day.
 * A course should be taken continuously for t days and must be finished before or on the
dth day. You will start at the 1st day.

 Given n online courses represented by pairs (t,d), your task is to find the maximal
number of courses that can be taken.

 Example:
 Input: [[100, 200], [200, 1300], [1000, 1250], [2000, 3200]]
 Output: 3

 Explanation:
 There're totally 4 courses, but you can take 3 courses at most:
 First, take the 1st course, it costs 100 days so you will finish it on the 100th day,
and ready to take the next course on the 101st day.
 Second, take the 3rd course, it costs 1000 days so you will finish it on the 1100th day,
and ready to take the next course on the 1101st day.
 Third, take the 2nd course, it costs 200 days so you will finish it on the 1300th day.
 The 4th course cannot be taken now, since you will finish it on the 3300th day, which
exceeds the closed date.

 Note:
 The integer 1 <= d, t, n <= 10,000.
 You can't take two courses simultaneously.
 */
public class _630 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/93790/short-java-code-using-
priorityqueue
         * Sort by finish date!!! This is greedy! We should take those classes that
finish early first.
         */
```

```
        public int scheduleCourse(int[][] courses) {
            Arrays.sort(courses, (a, b) -> a[1] - b[1]);
            int day = 0;
            PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
            for (int[] course : courses) {
                day += course[0];
                maxHeap.offer(course[0]);
                if (day > course[1]) {
                    day -= maxHeap.poll();//drop the previous courses that took the most
time
                }
            }
            return maxHeap.size();
        }
    }
}

// ============================================================
// FILE 565: _631.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Stack;

/**
 * 631. Design Excel Sum Formula
 *
 * Your task is to design the basic function of Excel and implement the function of sum
formula.
 * Specifically, you need to implement the following functions:

 Excel(int H, char W):
 This is the constructor.
 The inputs represents the height and width of the Excel form.
 H is a positive integer, range from 1 to 26.
 It represents the height.
 W is a character range from 'A' to 'Z'.
 It represents that the width is the number of characters from 'A' to W.
 The Excel form content is represented by a height * width 2D integer array C, it should
be initialized to zero.
 You should assume that the first row of C starts from 1, and the first column of C
starts from 'A'.

 void Set(int row, char column, int val):
 Change the value at C(row, column) to be val.

 int Get(int row, char column):
 Return the value at C(row, column).

 int Sum(int row, char column, List of Strings : numbers):
 This function calculate and set the value at C(row, column),
 where the value should be the sum of cells represented by numbers.
```

```
 This function return the sum result at C(row, column).
 This sum formula should exist until this cell is overlapped by another value or another
sum formula.

 numbers is a list of strings that each string represent a cell or a range of cells.
 If the string represent a single cell,
 then it has the following format : ColRow. For example, "F7" represents the cell at (7,
F).
 If the string represent a range of cells,
 then it has the following format : ColRow1:ColRow2.
 The range will always be a rectangle,
 and ColRow1 represent the position of the top-left cell,
 and ColRow2 represents the position of the bottom-right cell.


 Example 1:
 Excel(3,"C");
 // construct a 3*3 2D array with all zero.
 //   A B C
 // 1 0 0 0
 // 2 0 0 0
 // 3 0 0 0

 Set(1, "A", 2);
 // set C(1,"A") to be 2.
 //   A B C
 // 1 2 0 0
 // 2 0 0 0
 // 3 0 0 0

 Sum(3, "C", ["A1", "A1:B2"]);
 // set C(3,"C") to be the sum of value at C(1,"A")
 //and the values sum of the rectangle range whose top-left cell is C(1,"A") and bottom-
right cell is C(2,"B"). Return 4.
 //   A B C
 // 1 2 0 0
 // 2 0 0 0
 // 3 0 0 4

 Set(2, "B", 2);
 // set C(2,"B") to be 2. Note C(3, "C") should also be changed.
 //   A B C
 // 1 2 0 0
 // 2 0 2 0
 // 3 0 0 6

 Note:
 You could assume that there won't be any circular sum reference.
 For example, A1 = sum(B1) and B1 = sum(A1).
 The test cases are using double-quotes to represent a character.
 Please remember to RESET your class variables declared in class Excel,
 as static/class variables are persisted across multiple test cases. Please see here for
more details.
 */
```

```
public class _631 {

    public static class Solution1 {
        /**
         * Credit: https://leetcode.com/articles/design-excel-sum-formula/#approach-1-
using-topological-sortaccepted
         */
        public static class Excel {

            Formula[][] formulas;

            class Formula {
                Formula(HashMap<String, Integer> c, int v) {
                    val = v;
                    cells = c;
                }

                HashMap<String, Integer> cells;
                int val;
            }

            Stack<int[]> stack = new Stack<>();

            public Excel(int H, char W) {
                formulas = new Formula[H][(W - 'A') + 1];
            }

            public int get(int r, char c) {
                if (formulas[r - 1][c - 'A'] == null) {
                    return 0;
                }
                return formulas[r - 1][c - 'A'].val;
            }

            public void set(int r, char c, int v) {
                formulas[r - 1][c - 'A'] = new Formula(new HashMap<String, Integer>(),
v);
                topologicalSort(r - 1, c - 'A');
                execute_stack();
            }

            public int sum(int r, char c, String[] strs) {
                HashMap<String, Integer> cells = convert(strs);
                int summ = calculate_sum(r - 1, c - 'A', cells);
                set(r, c, summ);
                formulas[r - 1][c - 'A'] = new Formula(cells, summ);
                return summ;
            }

            public void topologicalSort(int r, int c) {
                for (int i = 0; i < formulas.length; i++) {
                    for (int j = 0; j < formulas[0].length; j++) {
                        if (formulas[i][j] != null && formulas[i][j].cells.containsKey(""
+ (char) ('A' + c) + (r + 1))) {
```

```
                            topologicalSort(i, j);
                        }
                    }
                }
                stack.push(new int[]{r, c});
            }

            public void execute_stack() {
                while (!stack.isEmpty()) {
                    int[] top = stack.pop();
                    if (formulas[top[0]][top[1]].cells.size() > 0) {
                        calculate_sum(top[0], top[1], formulas[top[0]][top[1]].cells);
                    }
                }
            }

            public HashMap<String, Integer> convert(String[] strs) {
                HashMap<String, Integer> res = new HashMap<>();
                for (String st : strs) {
                    if (st.indexOf(":") < 0) {
                        res.put(st, res.getOrDefault(st, 0) + 1);
                    } else {
                        String[] cells = st.split(":");
                        int si = Integer.parseInt(cells[0].substring(1));
                        int ei = Integer.parseInt(cells[1].substring(1));
                        char sj = cells[0].charAt(0);
                        char ej = cells[1].charAt(0);
                        for (int i = si; i <= ei; i++) {
                            for (char j = sj; j <= ej; j++) {
                                res.put("" + j + i, res.getOrDefault("" + j + i, 0) + 1);
                            }
                        }
                    }
                }
                return res;
            }

            public int calculate_sum(int r, int c, HashMap<String, Integer> cells) {
                int sum = 0;
                for (String s : cells.keySet()) {
                    int x = Integer.parseInt(s.substring(1)) - 1;
                    int y = s.charAt(0) - 'A';
                    sum += (formulas[x][y] != null ? formulas[x][y].val : 0) *
cells.get(s);
                }
                formulas[r][c] = new Formula(cells, sum);
                return sum;
            }
        }
    }

    /**
     * Your Excel object will be instantiated and called as such:
     * Excel obj = new Excel(H, W);
```

```
     * obj.set(r,c,v);
     * int param_2 = obj.get(r,c);
     * int param_3 = obj.sum(r,c,strs);
     */

}

// ============================================================
// FILE 566: _632.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;
import java.util.PriorityQueue;

/**
 * 632. Smallest Range
 *
 * You have k lists of sorted integers in ascending order.
 * Find the smallest range that includes at least one number from each of the k lists.

 We define the range [a,b] is smaller than range [c,d] if b-a < d-c or a < c if b-a == d-
c.

 Example 1:
 Input:[[4,10,15,24,26], [0,9,12,20], [5,18,22,30]]
 Output: [20,24]
 Explanation:
 List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
 List 2: [0, 9, 12, 20], 20 is in range [20,24].
 List 3: [5, 18, 22, 30], 22 is in range [20,24].

 Note:
 The given list may contain duplicates, so ascending order means >= here.
 1 <= k <= 3500
 -105 <= value of elements <= 105.
 For Java users, please note that the input type has been changed to List<List<Integer>>.
 And after you reset the code template, you'll see this point.

 */
public class _632 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/94445/java-code-using-
priorityqueue-similar-to-merge-k-array/2
         */
        public int[] smallestRange(List<List<Integer>> nums) {
            PriorityQueue<int[]> minHeap = new PriorityQueue<>(nums.size(), (a, b) ->
a[0] - b[0]);
            /**int[] array consists of three numbers: value; which list in nums; index of
value in this list*/

            int max = nums.get(0).get(0);
```

```
            for (int i = 0; i < nums.size(); i++) {
                minHeap.offer(new int[]{nums.get(i).get(0), i, 0});
                max = Math.max(max, nums.get(i).get(0));
            }
            int minRange = Integer.MAX_VALUE;
            int start = -1;
            while (minHeap.size() == nums.size()) {
                int[] curr = minHeap.poll();
                if (max - curr[0] < minRange) {
                    minRange = max - curr[0];
                    start = curr[0];
                }
                if (curr[2] + 1 < nums.get(curr[1]).size()) {
                    curr[0] = nums.get(curr[1]).get(curr[2] + 1);
                    curr[2]++;
                    minHeap.offer(curr);
                    max = Math.max(max, curr[0]);
                }
            }
            return new int[]{start, start + minRange};
        }
    }
}

// ============================================================
// FILE 567: _633.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 633. Sum of Square Numbers
 *
 * Given a non-negative integer c, your task is to decide whether there're two integers a
and b such that a2 + b2 = c.

 Example 1:
 Input: 5
 Output: True
 Explanation: 1 * 1 + 2 * 2 = 5

 Example 2:
 Input: 3
 Output: False
 */
public class _633 {
    public static class Solution1 {
        public boolean judgeSquareSum(int c) {
            if (c < 0) {
                return false;
            }
            int left = 0;
            int right = (int) (Math.sqrt(c));
            while (left <= right) {
```

```
                int curr = left * left + right * right;
                if (curr > c) {
                    right--;
                } else if (curr < c) {
                    left++;
                } else {
                    return true;
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 568: _634.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 634. Find the Derangement of An Array
 *
 * In combinatorial mathematics, a derangement is a permutation of the elements of a set,
such that no element appears in its original position.

 There's originally an array consisting of n integers from 1 to n in ascending order, you
need to find the number of derangement it can generate.

 Also, since the answer may be very large, you should return the output mod 109 + 7.

 Example 1:
 Input: 3
 Output: 2
 Explanation: The original array is [1,2,3]. The two derangements are [2,3,1] and
[3,1,2].

 Note:
 n is in the range of [1, 106].
 */
public class _634 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/94442/java-5-lines-o-1-space-
solution
         * and https://leetcode.com/articles/find-derangements/#approach-5-using-formula-
accepted
         */
        private static final int M = 1000000007;

        public int findDerangement(int n) {
            long ans = 1;
            for (int i = 1; i <= n; i++) {
                ans = (i * ans % M + (i % 2 == 0 ? 1 : -1)) % M;
```

```
            }
            return (int) ans;
        }
    }
}

// ============================================================
// FILE 569: _635.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.LinkedList;
import java.util.List;

/**
 * 635. Design Log Storage System
 *
 * You are given several logs that each log contains a unique id and timestamp.
 * Timestamp is a string that has the following format:
Year:Month:Day:Hour:Minute:Second,
 * for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.
 * Design a log storage system to implement the following functions:

 void Put(int id, string timestamp): Given a log's unique id and timestamp, store the log
in your storage system.

 int[] Retrieve(String start, String end, String granularity):
 Return the id of logs whose timestamps are within the range from start to end.
 Start and end all have the same format as timestamp.
 However, granularity means the time level for consideration.
 For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", granularity =
"Day",
 it means that we need to find the logs within the range from Jan. 1st 2017 to Jan. 2nd
2017.

 Example 1:
 put(1, "2017:01:01:23:59:59");
 put(2, "2017:01:01:22:59:59");
 put(3, "2016:01:01:00:00:00");
 retrieve("2016:01:01:01:01:01","2017:01:01:23:00:00","Year"); // return [1,2,3], because
you need to return all logs within 2016 and 2017.
 retrieve("2016:01:01:01:01:01","2017:01:01:23:00:00","Hour"); // return [1,2], because
you need to return all logs start from 2016:01:01:01 to 2017:01:01:23, where log 3 is
left outside the range.

 Note:
 There will be at most 300 operations of Put or Retrieve.
 Year ranges from [2000,2017]. Hour ranges from [00,23].
 Output for Retrieve has no order required.
 */
public class _635 {
```

```
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/94449/concise-java-solution
         */
        public static class LogSystem {

            /**
             * These indices denote and string endings of timestamps of different
granularity, i.e.
             * timestamp[1] in timestamps: "2017:01:01:22:59:59"
             * -> 2017: 4, 01: 7, 01: 10, 22: 13, 59: 16, 59: 19
             */

            List<String[]> timestamps;
            List<String> units;
            int[] indices;

            public LogSystem() {
                timestamps = new LinkedList<>();
                units = Arrays.asList("Year", "Month", "Day", "Hour", "Minute",
"Second");
                indices = new int[]{4, 7, 10, 13, 16, 19};
            }

            public void put(int id, String timestamp) {
                timestamps.add(new String[]{Integer.toString(id), timestamp});
            }

            public List<Integer> retrieve(String s, String e, String gra) {
                List<Integer> res = new LinkedList<>();
                int index = units.indexOf(gra);
                int stringEnd = indices[index];
                for (String[] timestamp : timestamps) {
                    if (timestamp[1].substring(0, stringEnd).compareTo(s.substring(0,
stringEnd)) >= 0
                            && timestamp[1].substring(0,
stringEnd).compareTo(e.substring(0, stringEnd)) <= 0) {
                        res.add(Integer.parseInt(timestamp[0]));
                    }
                }
                return res;
            }
        }
    }

}

// ==========================================================
// FILE 570: _636.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.Deque;
```

```
import java.util.LinkedList;
import java.util.List;

/**
 * 636. Exclusive Time of Functions
 *
 * Given the running logs of n functions that are executed in a nonpreemptive single
threaded CPU,
 * find the exclusive time of these functions.
 * Each function has a unique id, start from 0 to n-1.
 * A function may be called recursively or by another function.
 * A log is a string has this format : function_id:start_or_end:timestamp.
 * For example, "0:start:0" means function 0 starts from the very beginning of time 0.
 * "0:end:0" means function 0 ends to the very end of time 0.
 * Exclusive time of a function is defined as the time spent within this function,
 * the time spent by calling other functions should not be considered as this function's
exclusive time.
 * You should return the exclusive time of each function sorted by their function id.

 Example 1:
 Input:
 n = 2
 logs =
 ["0:start:0",
 "1:start:2",
 "1:end:5",
 "0:end:6"]
 Output:[3, 4]
 Explanation:
 Function 0 starts at time 0, then it executes 2 units of time and reaches the end of
time 1.
 Now function 0 calls function 1, function 1 starts at time 2, executes 4 units of time
and end at time 5.
 Function 0 is running again at time 6, and also end at the time 6, thus executes 1 unit
of time.
 So function 0 totally execute 2 + 1 = 3 units of time, and function 1 totally execute 4
units of time.

 Note:
 Input logs will be sorted by timestamp, NOT log id.
 Your output should be sorted by function id, which means the 0th element of your output
corresponds to the exclusive time of function 0.
 Two functions won't start or end at the same time.
 Functions could be called recursively, and will always end.
 1 <= n <= 100
 */
public class _636 {
    public static class Solution1 {
        /**
         * Based on the example, it's difficult to see how function 2 executes 4 units of
time, actually
         * we can add 1 to all end times to make it easier to understand and AC'ed.
         */
        public int[] exclusiveTime(int n, List<String> logs) {
```

```
            /**Stack is the way to go:
             * we keep pushing the logId onto the stack whenever we just encounter this
logId's start timestamp,
             * we'll pop this logId only when we encounter this logId's end timestamp.
             * Meanwhile, we keep a counter called prevTime,
             * whenever the stack is not empty, we'll always deduct prevTime from the
last logId on the stack.*/
            Deque<Integer> stack = new LinkedList<>();
            int[] result = new int[n];
            int prevTime = 0;
            for (String log : logs) {
                String[] parts = log.split(":");
                if (!stack.isEmpty()) {
                    result[stack.peek()] += Integer.parseInt(parts[2]) - prevTime;
                }
                prevTime = Integer.parseInt(parts[2]);
                if (parts[1].equals("start")) {
                    stack.addFirst(Integer.parseInt(parts[0]));//i.e. stack.push()
                } else {
                    prevTime++;
                    //remember to have result pluse 1 to match the problem AC criteria
                    result[stack.pollFirst()]++;//i.e. stack.pop()
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 571: _637.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 637. Average of Levels in Binary Tree
 *
 Given a non-empty binary tree, return the average value of the nodes on each level in
the form of an array.

 Example 1:
 Input:
  3
 / \
9  20
  /  \
```

```
 15   7

 Output: [3, 14.5, 11]
 Explanation:

 The average value of nodes on level 0 is 3,  on level 1 is 14.5, and on level 2 is 11.
Hence return [3, 14.5, 11].
 Note:
 The range of node's value is in the range of 32-bit signed integer.
 */
public class _637 {

    public static class Solution1 {
        public List<Double> averageOfLevels(TreeNode root) {
            List<Double> result = new ArrayList<>();
            if (root == null) {
                return result;
            }
            Queue<TreeNode> queue = new LinkedList<>();
            queue.offer(root);
            while (!queue.isEmpty()) {
                int size = queue.size();
                double sum = 0.0;
                for (int i = 0; i < size; i++) {
                    TreeNode curr = queue.poll();
                    if (curr != null) {
                        sum += curr.val;
                    }
                    if (curr.left != null) {
                        queue.offer(curr.left);
                    }
                    if (curr.right != null) {
                        queue.offer(curr.right);
                    }
                }
                result.add(sum / size);
            }
            return result;
        }
    }

}

// ============================================================
// FILE 572: _638.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 638. Shopping Offers
```

```
 *
 * In LeetCode Store, there are some kinds of items to sell. Each item has a price.
 * However, there are some special offers, and a special offer consists of one or more
different kinds of items with a sale price.
 * You are given the each item's price, a set of special offers,
 * and the number we need to buy for each item.
 * The job is to output the lowest price you have to pay for exactly certain items as
given,
 * where you could make optimal use of the special offers.
 * Each special offer is represented in the form of an array,
 * the last number represents the price you need to pay for this special offer,
 * other numbers represents how many specific items you could get if you buy this offer.
 * You could use any of special offers as many times as you want.

 Example 1:
 Input: [2,5], [[3,0,5],[1,2,10]], [3,2]
 Output: 14
 Explanation:
 There are two kinds of items, A and B. Their prices are $2 and $5 respectively.
 In special offer 1, you can pay $5 for 3A and 0B
 In special offer 2, you can pay $10 for 1A and 2B.
 You need to buy 3A and 2B, so you may pay $10 for 1A and 2B (special offer #2), and $4
for 2A.

 Example 2:
 Input: [2,3,4], [[1,1,0,4],[2,2,1,9]], [1,2,1]
 Output: 11
 Explanation:
 The price of A is $2, and $3 for B, $4 for C.
 You may pay $4 for 1A and 1B, and $9 for 2A ,2B and 1C.
 You need to buy 1A ,2B and 1C, so you may pay $4 for 1A and 1B (special offer #1), and
$3 for 1B, $4 for 1C.
 You cannot add more items, though only $9 for 2A ,2B and 1C.

 Note:
 There are at most 6 kinds of items, 100 special offers.
 For each item, you need to buy at most 6 of them.
 You are not allowed to buy more items than you want, even if that would lower the
overall price.
 */
public class _638 {
    public static class Solution1 {
        /**
         * reference: https://leetcode.com/articles/shopping-offers/#approach-1-using-
recursion-accepted
         */
        public int shoppingOffers(List<Integer> price, List<List<Integer>> special,
List<Integer> needs) {
            return shopping(price, special, needs, 0);
        }

        public int shopping(List<Integer> price, List<List<Integer>> special,
List<Integer> needs, int i) {
            if (i == special.size()) {
```

```
                return dot(needs, price);
            }
            ArrayList<Integer> clone = new ArrayList(needs);
            int j = 0;
            for (j = 0; j < special.get(i).size() - 1; j++) {
                int diff = clone.get(j) - special.get(i).get(j);
                if (diff < 0) {
                    break;
                }
                clone.set(j, diff);
            }
            if (j == special.get(i).size() - 1) {
                return Math.min(special.get(i).get(j) + shopping(price, special, clone,
i), shopping(price, special, needs, i + 1));
            } else {
                return shopping(price, special, needs, i + 1);
            }
        }

        public int dot(List<Integer> a, List<Integer> b) {
            int sum = 0;
            for (int i = 0; i < a.size(); i++) {
                sum += a.get(i) * b.get(i);
            }
            return sum;
        }
    }
}

// ============================================================
// FILE 573: _639.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 639. Decode Ways II
 *
 * A message containing letters from A-Z is being encoded to numbers using the following
mapping way:

 'A' -> 1
 'B' -> 2
 ...
 'Z' -> 26
 Beyond that, now the encoded string can also contain the character '*',
 which can be treated as one of the numbers from 1 to 9.

 Given the encoded message containing digits and the character '*',
 return the total number of ways to decode it.

 Also, since the answer may be very large, you should return the output mod 109 + 7.

 Example 1:
```

```
 Input: "*"
 Output: 9
 Explanation: The encoded message can be decoded to the string: "A", "B", "C", "D", "E",
"F", "G", "H", "I".
 Example 2:
 Input: "1*"
 Output: 9 + 9 = 18

 Note:
 The length of the input string will fit in range [1, 105].
 The input string will only contain the character '*' and digits '0' - '9'.

 */
public class _639 {
    public static class Solution1 {
        /**
         * reference: https://leetcode.com/articles/decode-ways-ii/#approach-2-dynamic-
programming-accepted
         */
        int m = 1000000007;

        public int numDecodings(String s) {
            long[] dp = new long[s.length() + 1];
            dp[0] = 1;
            dp[1] = s.charAt(0) == '*' ? 9 : s.charAt(0) == '0' ? 0 : 1;
            for (int i = 1; i < s.length(); i++) {
                if (s.charAt(i) == '*') {
                    dp[i + 1] = 9 * dp[i];
                    if (s.charAt(i - 1) == '1') {
                        dp[i + 1] = (dp[i + 1] + 9 * dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '2') {
                        dp[i + 1] = (dp[i + 1] + 6 * dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '*') {
                        dp[i + 1] = (dp[i + 1] + 15 * dp[i - 1]) % m;
                    }
                } else {
                    dp[i + 1] = s.charAt(i) != '0' ? dp[i] : 0;
                    if (s.charAt(i - 1) == '1') {
                        dp[i + 1] = (dp[i + 1] + dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '2' && s.charAt(i) <= '6') {
                        dp[i + 1] = (dp[i + 1] + dp[i - 1]) % m;
                    } else if (s.charAt(i - 1) == '*') {
                        dp[i + 1] = (dp[i + 1] + (s.charAt(i) <= '6' ? 2 : 1) * dp[i -
1]) % m;
                    }
                }
            }
            return (int) dp[s.length()];
        }
    }
}

// ============================================================
// FILE 574: _640.java
```

```
// ============================================================

package com.fishercoder.solutions;

/**
 * 640. Solve the Equation
 *
 * Solve a given equation and return the value of x in the form of string "x=#value".
 * The equation contains only '+', '-' operation, the variable x and its coefficient.

 If there is no solution for the equation, return "No solution".

 If there are infinite solutions for the equation, return "Infinite solutions".

 If there is exactly one solution for the equation, we ensure that the value of x is an
integer.

 Example 1:
 Input: "x+5-3+x=6+x-2"
 Output: "x=2"

 Example 2:
 Input: "x=x"
 Output: "Infinite solutions"

 Example 3:
 Input: "2x=x"
 Output: "x=0"

 Example 4:
 Input: "2x+3x-6x=x+2"
 Output: "x=-1"

 Example 5:
 Input: "x=x+2"
 Output: "No solution"

 */
public class _640 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/95203/concise-java-solution/7
         */
        public String solveEquation(String equation) {
            String[] parts = equation.split("=");
            int[] left = evaluate(parts[0]);
            int[] right = evaluate(parts[1]);
            if (left[0] == right[0] && left[1] == right[1]) {
                return "Infinite solutions";
            } else if (left[0] == right[0]) {
                return "No solution";
            }
            return "x=" + (right[1] - left[1]) / (left[0] - right[0]);
        }
```

```
        private int[] evaluate(String part) {
            int[] result = new int[2];//result[0] is the coefficient for x, result[1] is
the coefficient for constants
            String[] tokens = part.split("(?=[+-])");  // ()for match group; ?= for match
and include in res; [+-] means + or -;
            for (String token : tokens) {
                if (token.equals("+x") || token.equals("x")) {
                    result[0]++;
                } else if (token.equals("-x")) {
                    result[0]--;
                } else if (token.contains("x")) {
                    result[0] += Integer.parseInt(token.substring(0, token.length() -
1));
                } else {
                    result[1] += Integer.parseInt(token);
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 575: _642.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/**
 * 642. Design Search Autocomplete System
 *
 * Design a search autocomplete system for a search engine. Users may input a sentence
(at least one word and end with a special character '#'). For each character they type
except '#', you need to return the top 3 historical hot sentences that have prefix the
same as the part of sentence already typed. Here are the specific rules:

 The hot degree for a sentence is defined as the number of times a user typed the exactly
same sentence before.
 The returned top 3 hot sentences should be sorted by hot degree (The first is the
hottest one). If several sentences have the same degree of hot, you need to use ASCII-
code order (smaller one appears first).
 If less than 3 hot sentences exist, then just return as many as you can.
 When the input is a special character, it means the sentence ends, and in this case, you
need to return an empty list.
 Your job is to implement the following functions:
```

The constructor function:

AutocompleteSystem(String[] sentences, int[] times): This is the constructor. The input is historical data. Sentences is a string array consists of previously typed sentences. Times is the corresponding times a sentence has been typed. Your system should record these historical data.

Now, the user wants to input a new sentence. The following function will provide the next character the user types:

List<String> input(char c): The input c is the next character typed by the user. The character will only be lower-case letters ('a' to 'z'), blank space (' ') or a special character ('#'). Also, the previously typed sentence should be recorded in your system. The output will be the top 3 historical hot sentences that have prefix the same as the part of sentence already typed.

Example:
Operation: AutocompleteSystem(["i love you", "island","ironman", "i love leetcode"], [5,3,2,2])
The system have already tracked down the following sentences and their corresponding times:
"i love you" : 5 times
"island" : 3 times
"ironman" : 2 times
"i love leetcode" : 2 times
Now, the user begins another search:

Operation: input('i')
Output: ["i love you", "island","i love leetcode"]
Explanation:
There are four sentences that have prefix "i". Among them, "ironman" and "i love leetcode" have same hot degree. Since ' ' has ASCII code 32 and 'r' has ASCII code 114, "i love leetcode" should be in front of "ironman". Also we only need to output top 3 hot sentences, so "ironman" will be ignored.

Operation: input(' ')
Output: ["i love you","i love leetcode"]
Explanation:
There are only two sentences that have prefix "i ".

Operation: input('a')
Output: []
Explanation:
There are no sentences that have prefix "i a".

Operation: input('#')
Output: []
Explanation:
The user finished the input, the sentence "i a" should be saved as a historical sentence in system. And the following input will be counted as a new search.

Note:
The input sentence will always start with a letter and end with '#', and only one blank

```
space will exist between two words.
 The number of complete sentences that to be searched won't exceed 100. The length of
each sentence including those in the historical data won't exceed 100.
 Please use double-quote instead of single-quote when you write test cases even for a
character input.
 Please remember to RESET your class variables declared in class AutocompleteSystem, as
static/class variables are persisted across multiple test cases. Please see here for more
details.
 */
public class _642 {
    public static class Solution1 {

        /**
         * reference: https://discuss.leetcode.com/topic/96150/java-solution-trie-and-
priorityqueue/3
         */
        public class AutocompleteSystem {

            Map<String, Integer> map;
            List<Map.Entry<String, Integer>> answers;
            StringBuilder stringBuilder;

            public AutocompleteSystem(String[] sentences, int[] times) {
                map = new HashMap<>();
                answers = new ArrayList<>();
                stringBuilder = new StringBuilder();

                for (int i = 0; i < sentences.length; i++) {
                    map.put(sentences[i], map.getOrDefault(sentences[i], 0) + times[i]);
                }
            }

            public List<String> input(char c) {
                List<String> result = new ArrayList<>();
                if (c == '#') {
                    map.put(stringBuilder.toString(),
map.getOrDefault(stringBuilder.toString(), 0) + 1);
                    stringBuilder.setLength(0);
                    answers.clear();/**The user has finished typing, so we'll clean
answers to get ready for next search*/
                } else {
                    stringBuilder.append(c);
                    /**when its length is 1, we find all the prefix that is a match and
put them into answers,
                     * then for the rest, we'll just remove those that are not match with
the prefix any more, we do this logic in else branch*/
                    if (stringBuilder.length() == 1) {
                        for (Map.Entry<String, Integer> entry : map.entrySet()) {
                            if (entry.getKey().startsWith(stringBuilder.toString())) {
                                answers.add(entry);
                            }
                        }
                        Collections.sort(answers, (a, b) -> a.getValue() == b.getValue()
? a.getKey().compareTo(b.getKey()) : b.getValue() - a.getValue());
```

```
                    } else {
                        for (Iterator<Map.Entry<String, Integer>> iterator =
answers.iterator(); iterator.hasNext(); ) {
                            if
(!iterator.next().getKey().startsWith(stringBuilder.toString())) {
                                iterator.remove();
                            }
                        }
                    }
                    for (int i = 0; i < 3 && i < answers.size(); i++) {
                        result.add(answers.get(i).getKey());
                    }
                }
                return result;
            }
        }

    }
}

// ============================================================
// FILE 576: _643.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 643. Maximum Average Subarray I
 * Given an array consisting of n integers, find the contiguous subarray of given length
k that has the maximum average value.
 * And you need to output the maximum average value.
 * Example 1:
 * Input: [1,12,-5,-6,50,3], k = 4
 * Output: 12.75
 * Explanation: Maximum average is (12-5-6+50)/4 = 51/4 = 12.75
 * Note:
 * 1 <= k <= n <= 30,000.
 * Elements of the given array will be in the range [-10,000, 10,000].
 */
public class _643 {

    public static class Solution1 {
        public double findMaxAverage(int[] nums, int k) {
            double sum = 0;
            double maxAve = Integer.MIN_VALUE;
            for (int i = 0; i < nums.length; i++) {
                if (k <= i) {
                    sum -= nums[i - k];
                }
                sum += nums[i];
                if ((i + 1) >= k) {
                    maxAve = Math.max(maxAve, sum / k);
                }
            }
```

```
            return maxAve;
        }
    }
}

// ============================================================
// FILE 577: _644.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 644. Maximum Average Subarray II
 * <p>
 * Given an array consisting of n integers, find the contiguous subarray whose length is
greater than
 * or equal to k that has the maximum average value. And you need to output the maximum
average value.
 * <p>
 * Example 1:
 * Input: [1,12,-5,-6,50,3], k = 4
 * Output: 12.75
 * Explanation:
 * when length is 5, maximum average value is 10.8,
 * when length is 6, maximum average value is 9.16667.
 * Thus return 12.75.
 * <p>
 * Note:
 * 1 <= k <= n <= 10,000.
 * Elements of the given array will be in range [-10,000, 10,000].
 * The answer with the calculation error less than 10-5 will be accepted.
 */
public class _644 {
    /**reference: https://leetcode.com/articles/maximum-average-subarray-ii/#approach-2-
using-binary-search-accepted
     * https://discuss.leetcode.com/topic/96123/java-solution-o-nlogm-binary-search-the-
answer/13*/

    /**
     * To understand the idea behind this method, let's look at the following points.
     * Firstly, we know that the value of the average could lie between the range (min,
max)(min,max).
     * Here, minmin and maxmax refer to the minimum and the maximum values out of the
given numsnums array.
     * This is because, the average can't be lesser than the minimum value and can't be
larger than the maximum value.
     * But, in this case, we need to find the maximum average of a subarray with atleast
kk elements.
     * The idea in this method is to try to approximate(guess) the solution and to try to
find if this solution really exists.
     * If it exists, we can continue trying to approximate the solution even to a further
precise value,
     * but choosing a larger number as the next approximation.
     * But, if the initial guess is wrong, and the initial maximum average value(guessed)
```

```
isn't possible,
     * we need to try with a smaller number as the next approximate.
     * <p>
     * Now, instead of doing the guesses randomly, we can make use of Binary Search.
     * With minmin and maxmax as the initial numbers to begin with,
     * we can find out the midmid of these two numbers given by (min+max)/2(min+max)/2.
     * Now, we need to find if a subarray with length greater than or equal to kk is
possible with an average sum greater than this midmid value.
     */
    public double findMaxAverage(int[] nums, int k) {
        double l = -10000;
        double r = 10000;
        while (r - l > 10e-7) {
            double mid = (l + r) / 2;
            if (getMaxSubbaraySumOfSizeK(nums, k, mid) >= 0) {
                l = mid;
            } else {
                r = mid;
            }
        }
        return (l + r) / 2;
    }

    private double getMaxSubbaraySumOfSizeK(int[] nums, int k, double mid) {
        double sum = 0.0;
        for (int i = 0; i <= k - 1; i++) {
            sum += nums[i] - mid;
        }
        double maxSum = sum;
        double prev = nums[0] - mid;
        for (int i = k; i < nums.length; i++) {
            sum = sum - nums[i - k] + nums[i];
            maxSum = Math.max(maxSum, Math.max(sum, sum + prev));
            prev = Math.max(nums[i - k + 1], nums[i - k + 1] + prev) - mid;
        }
        return maxSum;
    }
}

// ============================================================
// FILE 578: _645.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 645. Set Mismatch
 *
 * The set S originally contains numbers from 1 to n.
 * But unfortunately, due to the data error, one of the numbers in
 * the set got duplicated to another number in the set,
 * which results in repetition of one number and loss of another number.
```

```
 Given an array nums representing the data status of this set after the error.
 Your task is to firstly find the number occurs twice and then find the number
 that is missing. Return them in the form of an array.

 Example 1:
 Input: nums = [1,2,2,4]
 Output: [2,3]

 Note:
 The given array size will in the range [2, 10000].
 The given array's numbers won't have any order.
 */
public class _645 {
    public static class Solution1 {
        public int[] findErrorNums(int[] nums) {
            int[] result = new int[2];
            Arrays.sort(nums);
            for (int i = 0; i < nums.length - 1; i++) {
                if (nums[i + 1] == nums[i]) {
                    result[0] = nums[i];
                }
            }
            long temp = 0;
            for (int i = 0; i < nums.length; i++) {
                temp += i + 1;
                temp -= nums[i];
            }
            temp += result[0];
            result[1] = (int) temp;
            return result;
        }
    }
}

// ============================================================
// FILE 579: _646.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 646. Maximum Length of Pair Chain
 *
 * You are given n pairs of numbers. In every pair, the first number is always smaller
than the second number.

 Now, we define a pair (c, d) can follow another pair (a, b) if and only if b < c. Chain
of pairs can be formed in this fashion.

 Given a set of pairs, find the length longest chain which can be formed. You needn't use
up all the given pairs. You can select pairs in any order.
```

```
 Example 1:
 Input: [[1,2], [2,3], [3,4]]
 Output: 2
 Explanation: The longest chain is [1,2] -> [3,4]

 Note:
 The number of given pairs will be in the range [1, 1000].
 */
public class _646 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/96804/java-o-nlog-n-time-o-1-space
         */
        public int findLongestChain(int[][] pairs) {
            Arrays.sort(pairs, (o1, o2) -> o1[1] - o2[1]);
            int result = 0;
            int n = pairs.length;
            int i = -1;
            while (++i < n) {
                result++;
                int curEnd = pairs[i][1];
                while (i + 1 < n && pairs[i + 1][0] <= curEnd) {
                    /**This means, we'll keep incrementing i until pairs[i+1][0] is
                     * exactly greater than curEnd.*/
                    i++;
                }
            }
            return result;
        }
    }

}

// ============================================================
// FILE 580: _647.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 647. Palindromic Substrings
 *
 *  Given a string, your task is to count how many palindromic substrings in this string.
 *  The substrings with different start indexes or end indexes are counted
 *  as different substrings even they consist of same characters.

 Example 1:
 Input: "abc"
 Output: 3
 Explanation: Three palindromic strings: "a", "b", "c".

 Example 2:
```

```
 Input: "aaa"
 Output: 6
 Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

 Note:
 The input string length won't exceed 1000.

 */

public class _647 {

    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/96819/java-solution-8-lines-
extendpalindrome
         */
        public int countSubstrings(String s) {
            int count = 0;
            for (int i = 0; i < s.length(); i++) {
                count += extendPalindrome(s, i, i);//odd length
                count += extendPalindrome(s, i, i + 1);//even length
            }
            return count;
        }

        private int extendPalindrome(String s, int left, int right) {
            int count = 0;
            while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right))
{
                count++;
                left--;
                right++;
            }
            return count;
        }
    }
}

// ============================================================
// FILE 581: _648.java
// ============================================================

package com.fishercoder.solutions;

import java.util.List;

/**
 * 648. Replace Words

 * In English, we have a concept called root,
 * which can be followed by some other words to form another longer word
 * - let's call this word successor.
 * For example, the root an, followed by other, which can form another word another.
 * Now, given a dictionary consisting of many roots and a sentence.
```

```
 * You need to replace all the successor in the sentence with the root forming it.
 * If a successor has many roots can form it, replace it with the root with the shortest
length.

 You need to output the sentence after the replacement.

 Example 1:
 Input: dict = ["cat", "bat", "rat"]
 sentence = "the cattle was rattled by the battery"
 Output: "the cat was rat by the bat"

 Note:
 The input will only have lower-case letters.
 1 <= dict words number <= 1000
 1 <= sentence words number <= 1000
 1 <= root length <= 100
 1 <= sentence words length <= 1000

 */
public class _648 {
    public static class Solution1 {
        public String replaceWords(List<String> dict, String sentence) {
            String[] tokens = sentence.split(" ");
            TrieNode trie = buildTrie(dict);
            return replaceWords(tokens, trie);
        }

        private String replaceWords(String[] tokens, TrieNode root) {
            StringBuilder stringBuilder = new StringBuilder();
            for (String token : tokens) {
                stringBuilder.append(getShortestReplacement(token, root));
                stringBuilder.append(" ");
            }
            return stringBuilder.substring(0, stringBuilder.length() - 1);
        }

        private String getShortestReplacement(String token, final TrieNode root) {
            TrieNode temp = root;
            StringBuilder stringBuilder = new StringBuilder();
            for (char c : token.toCharArray()) {
                stringBuilder.append(c);
                if (temp.children[c - 'a'] != null) {
                    if (temp.children[c - 'a'].isWord) {
                        return stringBuilder.toString();
                    }
                    temp = temp.children[c - 'a'];
                } else {
                    return token;
                }
            }
            return token;
        }

        private TrieNode buildTrie(List<String> dict) {
```

```
            TrieNode root = new TrieNode(' ');
            for (String word : dict) {
                TrieNode temp = root;
                for (char c : word.toCharArray()) {
                    if (temp.children[c - 'a'] == null) {
                        temp.children[c - 'a'] = new TrieNode(c);
                    }
                    temp = temp.children[c - 'a'];
                }
                temp.isWord = true;
            }
            return root;
        }

        public class TrieNode {
            char val;
            TrieNode[] children;
            boolean isWord;

            public TrieNode(char val) {
                this.val = val;
                this.children = new TrieNode[26];
                this.isWord = false;
            }
        }
    }
}

// ============================================================
// FILE 582: _649.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 649. Dota2 Senate
 *
 * In the world of Dota2, there are two parties: the Radiant and the Dire.

 The Dota2 senate consists of senators coming from two parties.
 Now the senate wants to make a decision about a change in the Dota2 game.
 The voting for this change is a round-based procedure.
 In each round, each senator can exercise one of the two rights:

 1. Ban one senator's right:
 A senator can make another senator lose all his rights in this and all the following
rounds.
 2. Announce the victory:
 If this senator found the senators who still have rights to vote are all from the same
party,
 he can announce the victory and make the decision about the change in the game.
```

```
 Given a string representing each senator's party belonging.
 The character 'R' and 'D' represent the Radiant party and the Dire party respectively.
 Then if there are n senators, the size of the given string will be n.

 The round-based procedure starts from the first senator to the last senator in the given
order.
 This procedure will last until the end of voting.
 All the senators who have lost their rights will be skipped during the procedure.

 Suppose every senator is smart enough and will play the best strategy for his own party,
 you need to predict which party will finally announce the victory and make the change in
the Dota2 game.
 The output should be Radiant or Dire.

 Example 1:
 Input: "RD"
 Output: "Radiant"
 Explanation: The first senator comes from Radiant and he can just ban the next senator's
right in the round 1.
 And the second senator can't exercise any rights any more since his right has been
banned.
 And in the round 2, the first senator can just announce the victory since he is the only
guy in the senate who can vote.

 Example 2:
 Input: "RDD"
 Output: "Dire"
 Explanation:
 The first senator comes from Radiant and he can just ban the next senator's right in the
round 1.
 And the second senator can't exercise any rights anymore since his right has been
banned.
 And the third senator comes from Dire and he can ban the first senator's right in the
round 1.
 And in the round 2, the third senator can just announce the victory since he is the only
guy in the senate who can vote.
 Note:
 The length of the given string will in the range [1, 10,000].

 */
public class _649 {

    public static class Solution1 {
        public String predictPartyVictory(String senate) {
            Queue<Integer> radiantQ = new LinkedList<>();
            Queue<Integer> direQ = new LinkedList<>();
            int len = senate.length();
            for (int i = 0; i < len; i++) {
                if (senate.charAt(i) == 'R') {
                    radiantQ.offer(i);
                } else {
                    direQ.offer(i);
                }
            }
```

```
            while (!radiantQ.isEmpty() && !direQ.isEmpty()) {
                int radiantIndex = radiantQ.poll();
                int direIndex = direQ.poll();
                if (radiantIndex < direIndex) {
                    /**Radiant will ban Dire in this case, so we'll add radiant index
back to the queue plus n*/
                    radiantQ.offer(radiantIndex + len);
                } else {
                    direQ.offer(direIndex + len);
                }
            }
            return radiantQ.isEmpty() ? "Dire" : "Radiant";
        }
    }
}

// ============================================================
// FILE 583: _650.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 650. 2 Keys Keyboard
 *
 * Initially on a notepad only one character 'A' is present. You can perform two
operations on this notepad for each step:

 Copy All: You can copy all the characters present on the notepad (partial copy is not
allowed).
 Paste: You can paste the characters which are copied last time.
 Given a number n. You have to get exactly n 'A' on the notepad by performing the minimum
number of steps permitted. Output the minimum number of steps to get n 'A'.

 Example 1:
 Input: 3
 Output: 3

 Explanation:
 Intitally, we have one character 'A'.
 In step 1, we use Copy All operation.
 In step 2, we use Paste operation to get 'AA'.
 In step 3, we use Paste operation to get 'AAA'.

 Note:
 The n will be in the range [1, 1000].
 */
public class _650 {

    public static class Solution1 {
        public int minSteps(int n) {
            int[] dp = new int[n + 1];
            for (int i = 2; i <= n; i++) {
                dp[i] = i;//we assign i to dp[i] first, because for a lot of cases, e.g.
```

```
for most cases when i is odd, its min steps is i itself, if it's not, we can overwrite it
later
                for (int j = i - 1; j > 1; j--) {
                    //traverse backwards, whenever it's divisible by j, we'll update
dp[i] because it's guaranteed to be smaller when j is smaller.
                    if (i % j == 0) {
                        dp[i] = dp[j] + (i / j);
                        break;
                    }
                }
            }
            return dp[n];
        }
    }
}

// ============================================================
// FILE 584: _651.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 651. 4 Keys Keyboard
 *
 * Imagine you have a special keyboard with the following keys:

 Key 1: (A): Prints one 'A' on screen.

 Key 2: (Ctrl-A): Select the whole screen.

 Key 3: (Ctrl-C): Copy selection to buffer.

 Key 4: (Ctrl-V): Print buffer on screen appending it after what has already been
printed.

 Now, you can only press the keyboard for N times (with the above four keys), find out
the maximum numbers of 'A' you can print on screen.

 Example 1:
 Input: N = 3
 Output: 3
 Explanation:
 We can at most get 3 A's on screen by pressing following key sequence:
 A, A, A

 Example 2:
 Input: N = 7
 Output: 9
 Explanation:
 We can at most get 9 A's on screen by pressing following key sequence:
 A, A, A, Ctrl A, Ctrl C, Ctrl V, Ctrl V

 Note:
```

```
 1 <= N <= 50
 Answers will be in the range of 32-bit signed integer.
 */
public class _651 {

    public static class Solution1 {
        /**
         * Minimum needs to be more than 3 A's in a row, otherwise "Ctrl A, Ctrl C, Ctrl
V" will make fewer A's than directly
         * copying A's with the equal number of steps.
         * E.g. when n == 5,
         * if we do 5 this: A, A, Ctrl A, Ctrl C, Ctrl V, => this will result in only
AAAA (4 A's)
         * while if we do A, A, A, A, A, => this will result in AAAAA (5 A's)
         * So, at a minimum, we need to have 3 A's, then it's worth to do "Ctrl A, Ctrl
C, Ctrl V"..
         * That's why we have j = 3 in the inner for loop below.
         */
        public int maxA(int N) {
            int[] dp = new int[N + 1];
            for (int i = 1; i <= N; i++) {
                dp[i] = i;
                for (int j = 3; j < i; j++) {
                    dp[i] = Math.max(dp[i], dp[i - j] * (j - 1));
                }
            }
            return dp[N];
        }
    }

}

// ============================================================
// FILE 585: _652.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

/**
 * 652. Find Duplicate Subtrees
 *
 * Given a binary tree, return all duplicate subtrees. For each kind of duplicate
subtrees, you only need to return the root node of any one of them.
```

```
 Two trees are duplicate if they have the same structure with same node values.

 Example 1:
     1
    / \
   2   3
  /   / \
 4   2   4
    /
   4
 The following are two duplicate subtrees:
     2
    /
   4
 and
 4

 Therefore, you need to return above trees' root in the form of a list.
 */
public class _652 {
    public static class Solution1 {

        /**credit: https://discuss.leetcode.com/topic/97584/java-concise-postorder-
traversal-solution*/

        /**
         * You don't actually need to check if every other tree is a duplicate of current
node,
         * just when you go through each node, you'll see whether there's already one in
the map,
         * since map.containsKey() checks this TreeNode.
         */
        public List<TreeNode> findDuplicateSubtrees(TreeNode root) {
            List<TreeNode> res = new LinkedList<>();
            postorder(root, new HashMap<>(), res);
            return res;
        }

        private String postorder(TreeNode curr, HashMap<String, Integer> map,
List<TreeNode> res) {
            if (curr == null) {
                return "#";
            }
            String serial = curr.val + "," + postorder(curr.left, map, res) + "," +
postorder(curr.right, map, res);
            if (map.getOrDefault(serial, 0) == 1) {
                res.add(curr);
            }
            map.put(serial, map.getOrDefault(serial, 0) + 1);
            return serial;
        }
    }
}
```

```
// ============================================================
// FILE 586: _653.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * 653. Two Sum IV - Input is a BST
 *
 * Given a Binary Search Tree and a target number,
 * return true if there exist two elements in the BST such that their sum is equal to the
given target.

 Example 1:
 Input:
    5
   / \
  3   6
 / \   \
2   4   7

Target = 9
Output: True

Example 2:
Input:
    5
   / \
  3   6
 / \   \
2   4   7

Target = 28
Output: False

 */
public class _653 {

    public static class Solution1 {
        public boolean findTarget(TreeNode root, int k) {
            if (root == null) {
                return false;
            }
            List<Integer> list = new ArrayList<>();
            dfs(root, list);
            for (int i = 0; i < list.size() - 1; i++) {
                for (int j = i + 1; j < list.size(); j++) {
                    if (list.get(i) + list.get(j) == k) {
```

```
                        return true;
                    }
                }
            }
            return false;
        }

        private void dfs(TreeNode root, List<Integer> list) {
            list.add(root.val);
            if (root.left != null) {
                dfs(root.left, list);
            }
            if (root.right != null) {
                dfs(root.right, list);
            }
        }
    }
}

// ============================================================
// FILE 587: _654.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 654. Maximum Binary Tree
 *
 * Given an integer array with no duplicates. A maximum tree building on this array is
defined as follow:
 *
 * The root is the maximum number in the array.
 * The left subtree is the maximum tree constructed from left part subarray divided by
the maximum number.
 * The right subtree is the maximum tree constructed from right part subarray divided by
the maximum number.
 * Construct the maximum tree by the given array and output the root node of this tree.

 Example 1:
 Input: [3,2,1,6,0,5]
 Output: return the tree root node representing the following tree:

    6
  /   \
 3    5
  \   /
  2  0
   \
   1

 Note:
 The size of the given array will be in the range [1,1000].
```

```
 */
public class _654 {

    public static class Solution1 {
        /**
         * Completely my original solution:
         *
         * As the problem states, I always broke the array into two halves and make notes
         * of current max node, then in the recursive call, we can recursively search
         * from its left part to construct its left subtree and its right part to
construct its
         * right subtree.*/
        public TreeNode constructMaximumBinaryTree(int[] nums) {
            int max = Integer.MIN_VALUE;
            int maxIndex = -1;
            for (int i = 0; i < nums.length; i++) {
                if (nums[i] > max) {
                    max = nums[i];
                    maxIndex = i;
                }
            }
            TreeNode root = new TreeNode(max);
            return constructMaxTree(root, maxIndex, nums, 0, nums.length - 1);
        }

        private TreeNode constructMaxTree(TreeNode root, int rootIndex, int[] nums, int
start, int end) {
            if (rootIndex > start) {
                int max = Integer.MIN_VALUE;
                int maxIndex = -1;
                for (int i = start; i < rootIndex; i++) {
                    if (max < nums[i]) {
                        max = nums[i];
                        maxIndex = i;
                    }
                }
                root.left = constructMaxTree(new TreeNode(max), maxIndex, nums, start,
rootIndex - 1);
            }
            if (rootIndex < end) {
                int max = Integer.MIN_VALUE;
                int maxIndex = -1;
                for (int i = rootIndex + 1; i <= end; i++) {
                    if (max < nums[i]) {
                        max = nums[i];
                        maxIndex = i;
                    }
                }
                root.right = constructMaxTree(new TreeNode(max), maxIndex, nums,
rootIndex + 1, end);
            }
            return root;
        }
    }
```

```
    public static class Solution2 {
        /**
         * Completely my original solution as well, but more concise.
         */
        public TreeNode constructMaximumBinaryTree(int[] nums) {
            if (nums == null || nums.length == 0) {
                return null;
            }
            return construct(nums, 0, nums.length - 1);
        }

        TreeNode construct(int[] nums, int start, int end) {
            if (start > end) {
                return null;
            }
            int[] maxArray = findMax(nums, start, end);
            TreeNode root = new TreeNode(maxArray[0]);
            root.left = construct(nums, start, maxArray[1] - 1);
            root.right = construct(nums, maxArray[1] + 1, end);
            return root;
        }

        int[] findMax(int[] nums, int start, int end) {
            int max = nums[start];
            int maxIndex = start;
            for (int i = start + 1; i <= end; i++) {
                if (max < nums[i]) {
                    maxIndex = i;
                    max = nums[i];
                }
            }
            return new int[]{max, maxIndex};
        }
    }
}

// ============================================================
// FILE 588: _655.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

/**
 * 655. Print Binary Tree
 *
 * Print a binary tree in an m*n 2D string array following these rules:
 * 1. The row number m should be equal to the height of the given binary tree.
```

```
 * 2. The column number n should always be an odd number.
 * 3. The root node's value (in string format) should be put in the exactly middle of the
first row it can be put.
 *    The column and the row where the root node belongs will separate the rest space
into two parts (left-bottom part and right-bottom part).
 *    You should print the left subtree in the left-bottom part and print the right
subtree in the right-bottom part.
 *    The left-bottom part and the right-bottom part should have the same size.
 *    Even if one subtree is none while the other is not,
 *    you don't need to print anything for the none subtree but
 *    still need to leave the space as large as that for the other subtree.
 *    However, if two subtrees are none, then you don't need to leave space for both of
them.
 * 4. Each unused space should contain an empty string "".
 * 5. Print the subtrees following the same rules.

 Example 1:
 Input:
   1
  /
 2
 Output:
 [["", "1", ""],
 ["2", "", ""]]

 Example 2:
 Input:
   1
  / \
 2   3
  \
  4
 Output:
 [["", "", "", "1", "", "", ""],
 ["", "2", "", "", "", "3", ""],
 ["", "", "4", "", "", "", ""]]

 Example 3:
 Input:
       1
      / \
     2   5
    /
   3
  /
 4
 Output:

 [["",  "",  "",  "",  "",  "",  "",  "1",  "",  "",  "",  "",  "",  "",  ""]
 ["",  "",  "",  "2",  "",  "",  "",  "",  "",  "",  "",  "5",  "",  "",  ""]
 ["",  "3",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  ""]
 ["4",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  "",  ""]]

 Note: The height of binary tree is in the range of [1, 10]
```

```
 */
public class _655 {

    public static class Solution1 {

        /**
         * Reference: https://discuss.leetcode.com/topic/98381/java-recursive-solution
         * and https://leetcode.com/articles/print-binary-tree/
         */
        public List<List<String>> printTree(TreeNode root) {
            List<List<String>> result = new LinkedList<>();
            int height = root == null ? 1 : getHeight(root);
            int rows = height;
            int columns = (int) (Math.pow(2, height) - 1);
            List<String> row = new ArrayList<>();
            for (int i = 0; i < columns; i++) {
                row.add("");
            }
            for (int i = 0; i < rows; i++) {
                result.add(new ArrayList<>(row));
            }
            populateResult(root, result, 0, rows, 0, columns - 1);
            return result;
        }

        private void populateResult(TreeNode root, List<List<String>> result, int row,
int totalRows, int i, int j) {
            if (row == totalRows || root == null) {
                return;
            }
            result.get(row).set((i + j) / 2, Integer.toString(root.val));
            populateResult(root.left, result, row + 1, totalRows, i, (i + j) / 2 - 1);
            populateResult(root.right, result, row + 1, totalRows, (i + j) / 2 + 1, j);
        }

        private int getHeight(TreeNode root) {
            if (root == null) {
                return 0;
            }
            return 1 + Math.max(getHeight(root.left), getHeight(root.right));
        }
    }
}

// ============================================================
// FILE 589: _656.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

```
/**
 * 656. Coin Path
 *
 * Given an array A (index starts at 1) consisting of N integers: A1, A2, ..., AN and an
integer B.
 * The integer B denotes that from any place (suppose the index is i) in the array A,
 * you can jump to any one of the place in the array A indexed i+1, i+2, …, i+B if this
place can be jumped to.
 * Also, if you step on the index i, you have to pay Ai coins. If Ai is -1, it means you
can’t jump to the place indexed i in the array.
 * Now, you start from the place indexed 1 in the array A,
 * and your aim is to reach the place indexed N using the minimum coins.
 * You need to return the path of indexes (starting from 1 to N) in the array you should
take to get to the place indexed N using minimum coins.
 * If there are multiple paths with the same cost, return the lexicographically smallest
such path.
 * If it's not possible to reach the place indexed N then you need to return an empty
array.

 Example 1:
 Input: [1,2,4,-1,2], 2
 Output: [1,3,5]

 Example 2:
 Input: [1,2,4,-1,2], 1
 Output: []

 Note:
 Path Pa1, Pa2, ..., Pan is lexicographically smaller than Pb1, Pb2, ..., Pbm,
 if and only if at the first i where Pai and Pbi differ, Pai < Pbi; when no such i
exists, then n < m.
 A1 >= 0. A2, ..., AN (if exist) will in the range of [-1, 100].
 Length of A is in the range of [1, 1000].
 B is in the range of [1, 100].
 */

public class _656 {
    public static class Solution1 {

        /**
         * Time: O(n*B)
         * Reference: https://leetcode.com/articles/coin-path/#approach-3-using-dynamic-
programming-accepted
         */
        public List<Integer> cheapestJump(int[] A, int B) {
            int[] next = new int[A.length];
            long[] dp = new long[A.length];
            Arrays.fill(next, -1);
            List<Integer> res = new ArrayList();
            for (int i = A.length - 2; i >= 0; i--) {
                long minCost = Integer.MAX_VALUE;
                for (int j = i + 1; j <= i + B && j < A.length; j++) {
                    if (A[j] >= 0) {
                        long cost = A[i] + dp[j];
```

```
                        if (cost < minCost) {
                            minCost = cost;
                            next[i] = j;
                        }
                    }
                }
                dp[i] = minCost;
            }
            int i;
            for (i = 0; i < A.length && next[i] > 0; i = next[i]) {
                res.add(i + 1);
            }
            if (i == A.length - 1 && A[i] >= 0) {
                res.add(A.length);
            } else {
                return new ArrayList<>();
            }
            return res;
        }
    }
}

// ============================================================
// FILE 590: _657.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 657. Judge Route Circle
 *
 * Initially, there is a Robot at position (0, 0).
 * Given a sequence of its moves, judge if this robot makes a circle,
 * which means it moves back to the original place.

 The move sequence is represented by a string.
 And each move is represent by a character.
 The valid robot moves are R (Right), L (Left), U (Up) and D (down).
 The output should be true or false representing whether the robot makes a circle.

 Example 1:
 Input: "UD"
 Output: true

 Example 2:
 Input: "LL"
 Output: false
 */

public class _657 {
    public static class Solution1 {
        public boolean judgeCircle(String moves) {
            int hori = 0;
            int verti = 0;
```

```
            for (char c : moves.toCharArray()) {
                if (c == 'U') {
                    verti++;
                } else if (c == 'D') {
                    verti--;
                } else if (c == 'L') {
                    hori--;
                } else if (c == 'R') {
                    hori++;
                }
            }
            return verti == 0 && hori == 0;
        }
    }
}

// ============================================================
// FILE 591: _658.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 658. Find K Closest Elements
 *
 * Given a sorted array, two integers k and x,
 * find the k closest elements to x in the array.
 * The result should also be sorted in ascending order.
 * If there is a tie, the smaller elements are always preferred.

 Example 1:
 Input: [1,2,3,4,5], k=4, x=3
 Output: [1,2,3,4]

 Example 2:
 Input: [1,2,3,4,5], k=4, x=-1
 Output: [1,2,3,4]

 Note:
 The value k is positive and will always be smaller than the length of the sorted array.
 Length of the given array is positive and will not exceed 104
 Absolute value of elements in the array and x will not exceed 104
 */
public class _658 {
    public static class Solution1 {
        public List<Integer> findClosestElements(List<Integer> arr, int k, int x) {
            List<Integer> result = new ArrayList<>();
            int index = findInsertPosition(arr, x);
            int right = 0;
            if (arr.get(index) == x) {
                result.add(arr.get(index));
```

```
                right = index + 1;
                k--;
            } else {
                right = index;
            }
            int left = index - 1;
            while (k-- > 0) {
                if (left >= 0 && right < arr.size()) {
                    if (x - arr.get(left) <= arr.get(right) - x) {
                        result.add(0, arr.get(left--));
                    } else {
                        result.add(result.size(), arr.get(right++));
                    }
                } else if (left >= 0) {
                    result.add(0, arr.get(left--));
                } else if (right < arr.size()) {
                    result.add(result.size(), arr.get(right++));
                }
            }
            return result;
        }

        private int findInsertPosition(List<Integer> arr, int x) {
            if (arr == null || arr.size() == 0) {
                return 0;
            }
            int len = arr.size();
            for (int i = 0; i < len; i++) {
                if (arr.get(0) > x) {
                    return 0;
                } else if (arr.get(len - 1) < x) {
                    return len;
                } else if (arr.get(i) < x && arr.get(i + 1) > x) {
                    return i + 1;
                } else if (arr.get(i) == x) {
                    return i;
                }
            }
            return len;
        }
    }
}

// ============================================================
// FILE 592: _659.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 659. Split Array into Consecutive Subsequences
```

```
 *
 * You are given an integer array sorted in ascending order (may contain duplicates),
 * you need to split them into several subsequences,
 * where each subsequences consist of at least 3 consecutive integers. Return whether you
can make such a split.

 Example 1:
 Input: [1,2,3,3,4,5]
 Output: True
 Explanation:
 You can split them into two consecutive subsequences :
 1, 2, 3
 3, 4, 5

 Example 2:
 Input: [1,2,3,3,4,4,5,5]
 Output: True
 Explanation:
 You can split them into two consecutive subsequences :
 1, 2, 3, 4, 5
 3, 4, 5

 Example 3:
 Input: [1,2,3,4,4,5]
 Output: False

 Note:
 The length of the input is in range of [1, 10000]
 */

public class _659 {

    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/99187/java-o-n-time-o-n-space
         */
        public boolean isPossible(int[] nums) {
            Map<Integer, Integer> freqMap = new HashMap<>();
            for (int i : nums) {
                freqMap.put(i, freqMap.getOrDefault(i, 0) + 1);
            }
            Map<Integer, Integer> appendFreqMap = new HashMap<>();
            for (int i : nums) {
                if (freqMap.get(i) == 0) {
                    continue;
                } else if (appendFreqMap.getOrDefault(i, 0) > 0) {
                    appendFreqMap.put(i, appendFreqMap.get(i) - 1);
                    appendFreqMap.put(i + 1, appendFreqMap.getOrDefault(i + 1, 0) + 1);
                } else if (freqMap.getOrDefault(i + 1, 0) > 0 && freqMap.getOrDefault(i +
2, 0) > 0) {
                    freqMap.put(i + 1, freqMap.get(i + 1) - 1);
                    freqMap.put(i + 2, freqMap.get(i + 2) - 1);
                    appendFreqMap.put(i + 3, appendFreqMap.getOrDefault(i + 3, 0) + 1);
                } else {
```

```
                    return false;
                }
                freqMap.put(i, freqMap.get(i) - 1);
            }
            return true;
        }
    }
}

// ============================================================
// FILE 593: _660.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 660. Remove 9
 *
 * Start from integer 1, remove any integer that contains 9 such as 9, 19, 29...

 So now, you will have a new integer sequence: 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, ...

 Given a positive integer n, you need to return the n-th integer after removing. Note
that 1 will be the first integer.

 Example 1:
 Input: 9
 Output: 10

 Hint: n will not exceed 9 x 10^8.
 */
public class _660 {

    public static class Solution1 {
        public int newInteger(int n) {
            return Integer.parseInt(Integer.toString(n, 9));
        }
    }

    public static class Solution2 {
        public int newInteger(int n) {
            int result = 0;
            int base = 1;
            while (n > 0) {
                result += n % 9 * base;
                n /= 9;
                base *= 10;
            }
            return result;
        }
    }
}

// ============================================================
```

```java
// FILE 594: _661.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 661. Image Smoother
 *
 * Given a 2D integer matrix m representing the gray scale of an image,
 * you need to design a smoother to make the gray scale of each cell becomes the average
gray scale (rounding down) of
 * all the 8 surrounding cells and itself. If a cell has less than 8 surrounding cells,
then use as many as you can.

 Example 1:
 Input:
 [[1,1,1],
 [1,0,1],
 [1,1,1]]

 Output:
 [[0, 0, 0],
 [0, 0, 0],
 [0, 0, 0]]

 Explanation:
 For the point (0,0), (0,2), (2,0), (2,2): floor(3/4) = floor(0.75) = 0
 For the point (0,1), (1,0), (1,2), (2,1): floor(5/6) = floor(0.83333333) = 0
 For the point (1,1): floor(8/9) = floor(0.88888889) = 0

 Note:
 The value in the given matrix is in the range of [0, 255].
 The length and width of the given matrix are in the range of [1, 150].

 */
public class _661 {
    public static class Solution1 {
        public int[][] imageSmoother(int[][] M) {
            if (M == null || M.length == 0) {
                return M;
            }
            int m = M.length;
            int n = M[0].length;
            int[][] result = new int[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    bfs(M, i, j, result, m, n);
                }
            }
            return result;
        }

        private void bfs(int[][] M, int i, int j, int[][] result, int m, int n) {
            int sum = M[i][j];
```

```
            int denominator = 1;
            if (j + 1 < n) {
                sum += M[i][j + 1];
                denominator++;
            }
            if (i + 1 < m && j + 1 < n) {
                sum += M[i + 1][j + 1];
                denominator++;
            }
            if (i + 1 < m) {
                sum += M[i + 1][j];
                denominator++;
            }
            if (i + 1 < m && j - 1 >= 0) {
                sum += M[i + 1][j - 1];
                denominator++;
            }
            if (j - 1 >= 0) {
                sum += M[i][j - 1];
                denominator++;
            }
            if (i - 1 >= 0 && j - 1 >= 0) {
                sum += M[i - 1][j - 1];
                denominator++;
            }
            if (i - 1 >= 0) {
                sum += M[i - 1][j];
                denominator++;
            }
            if (i - 1 >= 0 && j + 1 < n) {
                sum += M[i - 1][j + 1];
                denominator++;
            }
            result[i][j] = sum / denominator;
        }
    }
}

// ============================================================
// FILE 595: _662.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.AbstractMap;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;

/**
```

```
 * 662. Maximum Width of Binary Tree
 *
 * Given a binary tree, write a function to get the maximum width of the given tree.
 * The width of a tree is the maximum width among all levels.
 * The binary tree has the same structure as a full binary tree, but some nodes are null.
 * The width of one level is defined as the length between the end-nodes
 * (the leftmost and right most non-null nodes in the level,
 * where the null nodes between the end-nodes are also counted into the length
calculation.

 Example 1:
 Input:

      1
    /   \
   3     2
  / \     \
 5   3     9

 Output: 4
 Explanation: The maximum width existing in the third level with the length 4
(5,3,null,9).

 Example 2:
 Input:

     1
    /
   3
  / \
 5   3

 Output: 2
 Explanation: The maximum width existing in the third level with the length 2 (5,3).

 Example 3:
 Input:

     1
    / \
   3   2
  /
 5

 Output: 2
 Explanation: The maximum width existing in the second level with the length 2 (3,2).
 Example 4:
 Input:

       1
      / \
     3   2
    /     \
   5       9
```

```
  /         \
 6           7

 Output: 8
 Explanation:The maximum width existing in the fourth level with the length 8
(6,null,null,null,null,null,null,7).

 Note: Answer will in the range of 32-bit signed integer.
 */
public class _662 {
    public static class Solution1 {
        /**
         * Use a map to store the node to value map,
         * we use root as index 1, then its left child is 2*i-1 and right child is 2*i
         */
        public int widthOfBinaryTree(TreeNode root) {
            if (root == null) {
                return 0;
            }

            Queue<Map.Entry<TreeNode, Integer>> queue = new LinkedList<>();
            queue.offer(new AbstractMap.SimpleEntry<>(root, 1));
            int max = 1;
            while (!queue.isEmpty()) {
                int size = queue.size();
                List<Map.Entry<TreeNode, Integer>> list = new ArrayList<>();
                for (int i = 0; i < size; i++) {
                    Map.Entry<TreeNode, Integer> curr = queue.poll();
                    if (curr.getKey().left != null) {
                        Map.Entry<TreeNode, Integer> newEntry = new
AbstractMap.SimpleEntry<>(curr.getKey().left, curr.getValue() * 2 - 1);
                        queue.offer(newEntry);
                        list.add(newEntry);
                    }
                    if (curr.getKey().right != null) {
                        Map.Entry<TreeNode, Integer> newEntry = new
AbstractMap.SimpleEntry<>(curr.getKey().right, curr.getValue() * 2);
                        queue.offer(newEntry);
                        list.add(newEntry);
                    }
                }
                if (list.size() > 1) {
                    max = Math.max(list.get(list.size() - 1).getValue() -
list.get(0).getValue() + 1, max);
                }
            }
            return max;
        }
    }
}

// ============================================================
// FILE 596: _663.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 663. Equal Tree Partition
 *
 * Given a binary tree with n nodes,
 * your task is to check if it's possible to partition the tree to two trees which have
the equal sum of values
 * after removing exactly one edge on the original tree.

 Example 1:
 Input:
    5
   / \
  10 10
    /  \
   2   3

 Output: True
 Explanation:
  5
 /
10

 Sum: 15

  10
 /  \
2   3

 Sum: 15


 Example 2:
 Input:
   1
  / \
 2  10
   /  \
  2   20

 Output: False
 Explanation: You can't split the tree into two trees with equal sum after removing
exactly one edge on the tree.

 Note:
 The range of tree node value is in the range of [-100000, 100000].
 1 <= n <= 10000
```

```
 */
public class _663 {
    public static class Solution1 {
        /**
         * The idea is that we use a map to store the sum of each node, then in the end,
         * we check if any node has a sum that is exactly half of total sum.
         */
        public boolean checkEqualTree(TreeNode root) {
            Map<TreeNode, Integer> map = new HashMap<>();
            int totalSum = sumForEachNode(root, map);
            if (totalSum % 2 != 0 || map.size() < 2) {
                return false;
            }
            for (TreeNode key : map.keySet()) {
                if (map.get(key) == totalSum / 2) {
                    return true;
                }
            }
            return false;
        }

        private int sumForEachNode(TreeNode root, Map<TreeNode, Integer> map) {
            if (root == null) {
                return 0;
            }
            if (root.left == null && root.right == null) {
                map.put(root, root.val);
                return root.val;
            }
            int leftVal = 0;
            if (root.left != null) {
                leftVal = sumForEachNode(root.left, map);
            }
            int rightVal = 0;
            if (root.right != null) {
                rightVal = sumForEachNode(root.right, map);
            }
            int val = root.val + leftVal + rightVal;
            map.put(root, val);
            return val;
        }
    }
}

// ============================================================
// FILE 597: _664.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 664. Strange Printer
 *
```

```
 * There is a strange printer with the following two special requirements:
 * The printer can only print a sequence of the same character each time.
 * At each turn, the printer can print new characters starting from and ending at any
places, and will cover the original existing characters.
 * Given a string consists of lower English letters only, your job is to count the
minimum number of turns the printer needed in order to print it.

 Example 1:
 Input: "aaabbb"
 Output: 2
 Explanation: Print "aaa" first and then print "bbb".

 Example 2:
 Input: "aba"
 Output: 2
 Explanation: Print "aaa" first and then print "b" from the second place of the string,
which will cover the existing character 'a'.

 Hint: Length of the given string will not exceed 100.
 */
public class _664 {
    public static class Solution1 {
        /**
         * reference: https://discuss.leetcode.com/topic/100137/java-solution-dp
         */
        public int strangePrinter(String s) {
            int n = s.length();
            if (n == 0) {
                return 0;
            }

            int[][] dp = new int[101][101];
            for (int i = 0; i < n; i++) {
                dp[i][i] = 1;
            }

            for (int i = 1; i < n; i++) {
                for (int j = 0; j < n - i; j++) {
                    dp[j][j + i] = i + 1;
                    for (int k = j + 1; k <= j + i; k++) {
                        int temp = dp[j][k - 1] + dp[k][j + i];
                        if (s.charAt(k - 1) == s.charAt(j + i)) {
                            temp--;
                        }
                        dp[j][j + i] = Math.min(dp[j][j + i], temp);
                    }
                }
            }
            return dp[0][n - 1];
        }
    }
}

// ====================================================================
```

```
// FILE 598: _665.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 665. Non-decreasing Array
 *
 *  Given an array with n integers, your task is to check if it could become non-
decreasing by modifying at most 1 element.

 We define an array is non-decreasing if array[i] <= array[i + 1] holds for every i (1 <=
i < n).

 Example 1:

 Input: [4,2,3]
 Output: True
 Explanation: You could modify the first 4 to 1 to get a non-decreasing array.

 Example 2:

 Input: [4,2,1]
 Output: False
 Explanation: You can't get a non-decreasing array by modify at most one element.

 Note: The n belongs to [1, 10,000].
 */
public class _665 {

    public static class Solution1 {
        public boolean checkPossibility(int[] nums) {
            return modifyIMinusOne(nums) || modifyI(nums);
        }

        private boolean modifyI(int[] nums) {
            int[] tmp = nums.clone();
            int len = tmp.length;
            int modifyTimes = 0;
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
                }
                if (tmp[i] < tmp[i - 1]) {
                    modifyTimes++;
                    if (modifyTimes > 1) {
                        return false;
                    }
                    tmp[i] = tmp[i - 1];
                }
            }
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
```

```
                }
                if (tmp[i] < tmp[i - 1]) {
                    return false;
                }
            }
            return true;
        }

        private boolean modifyIMinusOne(int[] nums) {
            int[] tmp = nums.clone();
            int len = tmp.length;
            int modifyTimes = 0;
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
                }
                if (tmp[i] < tmp[i - 1]) {
                    modifyTimes++;
                    if (modifyTimes > 1) {
                        return false;
                    }
                    tmp[i - 1] = tmp[i];
                }
            }
            for (int i = 1; i < len; i++) {
                if (tmp[i] == tmp[i - 1]) {
                    continue;
                }
                if (tmp[i] < tmp[i - 1]) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ==========================================================
// FILE 599: _666.java
// ==========================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.HashMap;
import java.util.Map;

/**
 * 666. Path Sum IV
 * If the depth of a tree is smaller than 5, then this tree can be represented by a list
of three-digits integers.

 For each integer in this list:
```

```
 The hundreds digit represents the depth D of this node, 1 <= D <= 4.

 The tens digit represents the position P of this node in the level it belongs to, 1 <= P
<= 8.
 The position is the same as that in a full binary tree.

 The units digit represents the value V of this node, 0 <= V <= 9.

 Given a list of ascending three-digits integers representing a binary with the depth
smaller than 5.
 You need to return the totalSum of all paths from the root towards the leaves.

 Example 1:

 Input: [113, 215, 221]
 Output: 12
 Explanation:
 The tree that the list represents is:
   3
  / \
 5   1

 The path totalSum is (3 + 5) + (3 + 1) = 12.

 Example 2:

 Input: [113, 221]
 Output: 4
 Explanation:
 The tree that the list represents is:
 3
  \
  1

 The path totalSum is (3 + 1) = 4.

 */
public class _666 {
    public static class Solution1 {
        /**OMG, since it's no larger than depth 5, I've got a hardcoded solution here....
         * By "harcoded", I mean the constructTree() method.*/
        public int totalSum = 0;

        public int pathSum(int[] nums) {
            TreeNode root = constructTree(nums);
            if (root == null) {
                return 0;
            }
            computePathSum(root, 0);
            return totalSum;
        }

        private void computePathSum(TreeNode root, int pathSum) {
```

```
            pathSum += root.val;
            if (root.left != null) {
                computePathSum(root.left, pathSum);
            }
            if (root.right != null) {
                computePathSum(root.right, pathSum);
            }
            if (root.left == null && root.right == null) {
                totalSum += pathSum;
            }
//            pathSum -= root.val;
            /**this line is not necessary as I'm passing pathSum as a local variable
around, so it's always updated
             it's AC'ed with or without this line*/
        }

        private TreeNode constructTree(int[] nums) {
            if (nums == null || nums.length == 0) {
                return null;
            }
            TreeNode root = new
TreeNode(Integer.parseInt(Integer.toString(nums[0]).substring(2, 3)));
            //depth 2
            for (int i = 1; i < nums.length; i++) {
                if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) == 2 &&
Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 1) {
                    root.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
2 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 2) {
                    root.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                }
            }

            //depth 3
            for (int i = 2; i < nums.length; i++) {
                if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) == 3 &&
Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 1) {
                    root.left.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
3 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 2) {
                    root.left.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
3 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 3) {
                    root.right.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
3 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 4) {
                    root.right.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                }
```

```
            }

            //depth 4
            for (int i = 3; i < nums.length; i++) {
                if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) == 4 &&
Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 1) {
                    root.left.left.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 2) {
                    root.left.left.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 3) {
                    root.left.right.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 4) {
                    root.left.right.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 5) {
                    root.right.left.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 6) {
                    root.right.left.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 7) {
                    root.right.right.left = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                } else if (Integer.parseInt(Integer.toString(nums[i]).substring(0, 1)) ==
4 && Integer.parseInt(Integer.toString(nums[i]).substring(1, 2)) == 8) {
                    root.right.right.right = new
TreeNode(Integer.parseInt(Integer.toString(nums[i]).substring(2, 3)));
                }
            }

            return root;
        }
    }

    public static class Solution2 {
        public int totalSum = 0;

        public int pathSum(int[] nums) {
            Map<Integer, Integer> map = new HashMap<>();
            for (int i = 0; i < nums.length; i++) {
                int key = nums[i] / 10;
                int value = nums[i] % 10;
                map.put(key, value);
            }
            dfs(nums[0] / 10, 0, map);
```

```
            return totalSum;
        }

        private void dfs(int node, int preSum, Map<Integer, Integer> map) {
            int level = node / 10;
            int pos = node % 10;
            int leftChild = (level + 1) * 10 + pos * 2 - 1;
            int rightChild = (level + 1) * 10 + pos * 2;
            int currSum = preSum + map.get(node);
            if (!map.containsKey(leftChild) && !map.containsKey(rightChild)) {
                totalSum += currSum;
                return;
            }

            if (map.containsKey(leftChild)) {
                dfs(leftChild, currSum, map);
            }
            if (map.containsKey(rightChild)) {
                dfs(rightChild, currSum, map);
            }
        }
    }
}

// ============================================================
// FILE 600: _667.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 667. Beautiful Arrangement II
 *
 *  Given two integers n and k, you need to construct a list which contains n different
positive integers ranging from 1 to n
 *  and obeys the following requirement:
 *  Suppose this list is [a1, a2, a3, ... , an],
 *  then the list [|a1 - a2|, |a2 - a3|, |a3 - a4|, ... , |an-1 - an|] has exactly k
distinct integers.
 *  If there are multiple answers, print any of them.

 Example 1:

 Input: n = 3, k = 1
 Output: [1, 2, 3]
 Explanation: The [1, 2, 3] has three different positive integers ranging from 1 to 3,
and the [1, 1] has exactly 1 distinct integer: 1.

 Example 2:
```

```
 Input: n = 3, k = 2
 Output: [1, 3, 2]
 Explanation: The [1, 3, 2] has three different positive integers ranging from 1 to 3,
and the [2, 1] has exactly 2 distinct integers: 1 and 2.

 Note:

 The n and k are in the range 1 <= k < n <= 104.
 */

public class _667 {

        public static class Solutoin1 {
                /**This brute force solution will result in TLE as soon as n = 10 and k =
4.*/
                public int[] constructArray(int n, int k) {
                        List<List<Integer>> allPermutaions = findAllPermutations(n);
                        int[] result = new int[n];
                        for (List<Integer> perm : allPermutaions) {
                                if (isBeautifulArrangement(perm, k)) {
                                        convertListToArray(result, perm);
                                        break;
                                }
                        }
                        return result;
                }

                private void convertListToArray(int[] result, List<Integer> perm) {
                        for (int i = 0; i < perm.size(); i++) {
                                result[i] = perm.get(i);
                        }
                }

                private boolean isBeautifulArrangement(List<Integer> perm, int k) {
                        Set<Integer> diff = new HashSet<>();
                        for (int i = 0; i < perm.size() - 1; i++) {
                                diff.add(Math.abs(perm.get(i) - perm.get(i + 1)));
                        }
                        return diff.size() == k;
                }

                private List<List<Integer>> findAllPermutations(int n) {
                        List<List<Integer>> result = new ArrayList<>();
                        backtracking(new ArrayList<>(), result, n);
                        return result;
                }

                private void backtracking(List<Integer> list, List<List<Integer>> result,
int n) {
                        if (list.size() == n) {
                                result.add(new ArrayList<>(list));
                                return;
                        }
```

```
                        for (int i = 1; i <= n; i++) {
                                if (list.contains(i)) {
                                        continue;
                                }
                                list.add(i);
                                backtracking(list, result, n);
                                list.remove(list.size() - 1);
                        }
                }
        }

        public static class Solutoin2 {
                /**This is a very smart solution:
                 * First, we can see that the max value k could reach is n-1 which
                 * comes from a sequence like this:
                 * when n = 8, k = 5, one possible sequence is:
                 * 1, 8, 2, 7, 3, 4, 5, 6
                 * absolute diffs are:
                 *  7, 6, 5, 4, 1, 1, 1
                 *  so, there are total 5 distinct integers.
                 *
                 *  So, we can just form such a sequence by putting the first part first
and
                 *  decrement k along the way, when k becomes 1, we just put the rest
numbers in order.*/
                public int[] constructArray(int n, int k) {
                        int[] result = new int[n];
                        int left = 1;
                        int right = n;
                        for (int i = 0; i < n && left <= right; i++) {
                                if (k > 1) {
                                        result[i] = k-- % 2 != 0 ? left++ : right--;
                                } else {
                                        result[i] = k % 2 != 0 ? left++ : right--;
                                }
                        }
                        return result;
                }
        }
}

// ============================================================
// FILE 601: _668.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 668. Kth Smallest Number in Multiplication Table
 *
 *  Nearly every one have used the Multiplication Table.
 *  But could you find out the k-th smallest number quickly from the multiplication
```

```
table?
 *  Given the height m and the length n of a m * n Multiplication Table,
 *  and a positive integer k, you need to return the k-th smallest number in this table.

 Example 1:
 Input: m = 3, n = 3, k = 5
 Output:
 Explanation:
 The Multiplication Table:
 1      2       3
 2      4       6
 3      6       9

 The 5-th smallest number is 3 (1, 2, 2, 3, 3).


 Example 2:
 Input: m = 2, n = 3, k = 6
 Output:
 Explanation:
 The Multiplication Table:
 1      2       3
 2      4       6

 The 6-th smallest number is 6 (1, 2, 2, 3, 4, 6).

 Note:

 The m and n will be in the range [1, 30000].
 The k will be in the range [1, m * n]
 */

public class _668 {
    public static class Solution1 {
        /**
         * This brute force approach resulted in
         * TLE on Leetcode and
         * OOM error by _668test.test3() when running in my localhost:
         * java.lang.OutOfMemoryError: Java heap space
         * at java.util.Arrays.copyOf(Arrays.java:3210)
         * at java.util.Arrays.copyOf(Arrays.java:3181)
         * at java.util.PriorityQueue.grow(PriorityQueue.java:300)
         * at java.util.PriorityQueue.offer(PriorityQueue.java:339)
         */
        public int findKthNumber(int m, int n, int k) {
            PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> a - b);
            for (int i = 1; i <= m; i++) {
                for (int j = 1; j <= n; j++) {
                    minHeap.offer(i * j);
                }
            }
            while (k-- > 1) {
                minHeap.poll();
            }
```

```
            return minHeap.peek();
        }
    }

    public static class Solution2 {
        /**reference: https://discuss.leetcode.com/topic/101132/java-solution-binary-search*/
        public int findKthNumber(int m, int n, int k) {
            int low = 1;
            int high = m * n + 1;
            while (low < high) {
                int mid = low + (high - low) / 2;
                int c = count(mid, m, n);
                if (c >= k) {
                    high = mid;
                } else {
                    low = mid + 1;
                }
            }
            return high;
        }

        int count(int v, int m, int n) {
            int count = 0;
            for (int i = 1; i <= m; i++) {
                int temp = Math.min(v / i, n);
                count += temp;
            }
            return count;
        }
    }
}

// ============================================================
// FILE 602: _669.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 669. Trim a Binary Search Tree
 *
 * Given a binary search tree and the lowest and highest boundaries as L and R,
 * trim the tree so that all its elements lies in [L, R] (R >= L).
 * You might need to change the root of the tree, so the result should return the new root of the trimmed binary search tree.

 Example 1:

 Input:
   1
  / \
```

```
 0   2

 L = 1
 R = 2

 Output:
 1
  \
  2

 Example 2:

 Input:
  3
 / \
0   4
 \
 2
 /
1

 L = 1
 R = 3

 Output:
    3
   /
  2
 /
1

 */
public class _669 {
    public static class Solution1 {
        public TreeNode trimBST(TreeNode root, int L, int R) {
            if (root == null) {
                return root;
            }

            if (root.val > R) {
                return trimBST(root.left, L, R);
            }

            if (root.val < L) {
                return trimBST(root.right, L, R);
            }

            root.left = trimBST(root.left, L, R);
            root.right = trimBST(root.right, L, R);
            return root;
        }
    }
}
```

```
// ============================================================
// FILE 603: _670.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 670. Maximum Swap
 *
 * Given a non-negative integer, you could swap two digits at most once to get the
maximum valued number.
 * Return the maximum valued number you could get.

 Example 1:
 Input: 2736
 Output: 7236
 Explanation: Swap the number 2 and the number 7.

 Example 2:
 Input: 9973
 Output: 9973
 Explanation: No swap.

 Note:
 The given number is in the range [0, 108]

 */
public class _670 {
    public static class Solution1 {
        public int maximumSwap(int num) {
            String numStr = String.valueOf(num);
            int max = num;
            for (int i = 0; i < numStr.length() - 1; i++) {
                for (int j = i + 1; j < numStr.length(); j++) {
                    if (numStr.charAt(i) < numStr.charAt(j)) {
                        StringBuilder sb = new StringBuilder(numStr);
                        sb.replace(i, i + 1, String.valueOf(numStr.charAt(j)));
                        sb.replace(j, j + 1, String.valueOf(numStr.charAt(i)));
                        max = Math.max(max, Integer.parseInt(sb.toString()));
                    }
                }
            }
            return max;
        }
    }
}

// ============================================================
// FILE 604: _671.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
```

```
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;

/**
 * 671. Second Minimum Node In a Binary Tree
 *
 *  Given a non-empty special binary tree consisting of nodes with the non-negative
value,
 *  where each node in this tree has exactly two or zero sub-node.
 *  If the node has two sub-nodes, then this node's value is the smaller value among its
two sub-nodes.
 *  Given such a binary tree, you need to output the second minimum value in the set made
of all the nodes' value in the whole tree.
 *  If no such second minimum value exists, output -1 instead.

 Example 1:

 Input:
   2
  / \
 2   5
    / \
   5   7

 Output: 5
 Explanation: The smallest value is 2, the second smallest value is 5.

 Example 2:

 Input:
  2
 / \
2  2

 Output: -1
 Explanation: The smallest value is 2, but there isn't any second smallest value.

 */
public class _671 {
    public static class Solution1 {
        public int findSecondMinimumValue(TreeNode root) {
            if (root == null) {
                return -1;
            }
            Set<Integer> set = new TreeSet<>();
            dfs(root, set);
            Iterator<Integer> iterator = set.iterator();
            int count = 0;
            while (iterator.hasNext()) {
                count++;
                int result = iterator.next();
                if (count == 2) {
```

```
                    return result;
                }
            }
            return -1;
        }

        private void dfs(TreeNode root, Set<Integer> set) {
            if (root == null) {
                return;
            }
            set.add(root.val);
            dfs(root.left, set);
            dfs(root.right, set);
            return;
        }
    }
}

// ============================================================
// FILE 605: _672.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 672. Bulb Switcher II
 * There is a room with n lights which are turned on initially and 4 buttons on the wall.
 * After performing exactly m unknown operations towards buttons,
 * you need to return how many different kinds of status of the n lights could be.
 * Suppose n lights are labeled as number [1, 2, 3 ..., n], function of these 4 buttons
are given below:

 Flip all the lights.
 Flip lights with even numbers.
 Flip lights with odd numbers.
 Flip lights with (3k + 1) numbers, k = 0, 1, 2, ...

 Example 1:

 Input: n = 1, m = 1.
 Output: 2
 Explanation: Status can be: [on], [off]

 Example 2:

 Input: n = 2, m = 1.
 Output: 3
 Explanation: Status can be: [on, off], [off, on], [off, off]

 Example 3:

 Input: n = 3, m = 1.
```

```
 Output: 4
 Explanation: Status can be: [off, on, off], [on, off, on], [off, off, off], [off, on,
on].

 Note: n and m both fit in range [0, 1000].
 */
public class _672 {
    public static class Solution1 {
        public int flipLights(int n, int m) {
            if (m < 1) {
                return 1;
            }
            int[][] dp = new int[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (j == 0) {
                        dp[i][j] = 2;
                    } else if (i == 0 && j == 1) {
                        dp[i][j] = 3;
                    } else if (i == 0) {
                        dp[i][j] = 4;
                    } else if (i == 1 && j == 1) {
                        dp[i][j] = 4;
                    } else if (i == 1 && j > 1) {
                        dp[i][j] = 7;
                    } else if (j == 1) {
                        dp[i][j] = 4;
                    } else if (i == 1) {
                        dp[i][j] = 7;
                    } else {
                        dp[i][j] = 8;
                    }
                }
            }
            CommonUtils.print2DIntArray(dp);
            return dp[m - 1][n - 1];
        }
    }

    public static class Solution2 {
        public int flipLights(int n, int m) {
            if (n == 1 && m > 0) {
                return 2;
            } else if (n == 2 && m == 1) {
                return 3;
            } else if ((n > 2 && m == 1) || (n == 2 && m > 1)) {
                return 4;
            } else if (n > 2 && m == 2) {
                return 7;
            } else if (n > 2 && m > 2) {
                return 8;
            } else {
                return 1;
            }
```

```
        }
    }
}

// ============================================================
// FILE 606: _673.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 673. Number of Longest Increasing Subsequence
 * Given an unsorted array of integers, find the number of longest increasing
subsequence.

 Example 1:

 Input: [1,3,5,4,7]
 Output: 2
 Explanation: The two longest increasing subsequence are [1, 3, 4, 7] and [1, 3, 5, 7].

 Example 2:

 Input: [2,2,2,2,2]
 Output: 5
 Explanation: The length of longest continuous increasing subsequence is 1,
 and there are 5 subsequences' length is 1, so output 5.

 Note: Length of the given array will be not exceed 2000 and the answer is guaranteed to
be fit in 32-bit signed int.
 */
public class _673 {
    public static class Solution1 {
        /**
         * Reference: https://discuss.leetcode.com/topic/103020/java-c-simple-dp-
solution-with-explanation
         */
        public int findNumberOfLIS(int[] nums) {
            int n = nums.length;
            int[] cnt = new int[n];
            int[] len = new int[n];
            int max = 0;
            int count = 0;
            for (int i = 0; i < n; i++) {
                len[i] = cnt[i] = 1;
                for (int j = 0; j < i; j++) {
                    if (nums[i] > nums[j]) {
                        if (len[i] == len[j] + 1) {
                            cnt[i] += cnt[j];
                        }
                        if (len[i] < len[j] + 1) {
                            len[i] = len[j] + 1;
                            cnt[i] = cnt[j];
                        }
```

```
                    }
                }

                if (max == len[i]) {
                    count += cnt[i];
                }

                if (len[i] > max) {
                    max = len[i];
                    count = cnt[i];
                }
            }
            return count;
        }
    }
}

// ============================================================
// FILE 607: _674.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 674. Longest Continuous Increasing Subsequence
 * Given an unsorted array of integers, find the length of longest continuous increasing
subsequence.

 Example 1:
 Input: [1,3,5,4,7]
 Output: 3
 Explanation: The longest continuous increasing subsequence is [1,3,5],
 its length is 3. Even though [1,3,5,7] is also an increasing subsequence,
 it's not a continuous one where 5 and 7 are separated by 4.

 Example 2:
 Input: [2,2,2,2,2]
 Output: 1
 Explanation: The longest continuous increasing subsequence is [2], its length is 1.

 Note: Length of the array will not exceed 10,000.
 */
public class _674 {
    public static class Solution1 {
        public int findLengthOfLCIS(int[] nums) {
            int longest = 0;
            for (int i = 0; i < nums.length; i++) {
                int len = 1;
                for (int j = i + 1; j < nums.length; j++) {
                    if (nums[j - 1] < nums[j]) {
                        len++;
                        continue;
                    } else {
                        break;
```

```
                    }
                }
                longest = Math.max(longest, len);
            }
            return longest;
        }
    }
}

// ============================================================
// FILE 608: _675.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;

/**
 * 675. Cut Off Trees for Golf Event
 *
 * You are asked to cut off trees in a forest for a golf event.
 * The forest is represented as a non-negative 2D map, in this map:
 * 0 represents the obstacle can't be reached.
 * 1 represents the ground can be walked through.
 *
 * The place with number bigger than 1 represents a tree can be walked through,
 * and this positive number represents the tree's height.
 *
 * You are asked to cut off all the trees in this forest in the order of tree's height -
 * always cut off the tree with lowest height first.
 * And after cutting, the original place has the tree will become a grass (value 1).
 *
 * You will start from the point (0, 0) and you should output the minimum steps you need
to walk to cut off all the trees.
 *
 * If you can't cut off all the trees, output -1 in that situation.
 * You are guaranteed that no two trees have the same height and there is at least one
tree needs to be cut off.
 *
 * Example 1:
 * Input:
 * [
 * [1,2,3],
 * [0,0,4],
 * [7,6,5]
 * ]
 * Output: 6
 *
 * Example 2:
 * Input:
 * [
```

```
 * [1,2,3],
 * [0,0,0],
 * [7,6,5]
 * ]
 * Output: -1
 *
 * Example 3:
 * Input:
 * [
 * [2,3,4],
 * [0,0,5],
 * [8,7,6]
 * ]
 * Output: 6
 *
 * Explanation: You started from the point (0,0) and you can cut off the tree in (0,0)
directly without walking.
 * Hint: size of the given matrix will not exceed 50x50.
 */

public class _675 {
    public static class Solution1 {
        public int cutOffTree(List<List<Integer>> forest) {
            if (forest == null || forest.isEmpty() || forest.size() == 0 ||
forest.get(0).get(0) == 0) {
                return -1;
            }
            int m = forest.size();
            int n = forest.get(0).size();
            /**cut trees in ascending order*/
            PriorityQueue<Tree> heap = new PriorityQueue<>((a, b) -> a.height -
b.height);
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (forest.get(i).get(j) > 1) {
                        /**This is important: we'll add trees that are only taller than
1!!!*/
                        heap.offer(new Tree(i, j, forest.get(i).get(j)));
                    }
                }
            }

            int sum = 0;
            Tree start = new Tree();
            while (!heap.isEmpty()) {
                Tree curr = heap.poll();
                int step = bfs(forest, curr, start, m, n);
                if (step == -1) {
                    return -1;
                }
                sum += step;
                start = curr;
            }
            return sum;
```

```
        }

        private int bfs(List<List<Integer>> forest, Tree target, Tree start, int m, int
n) {
            int[] dirs = new int[]{0, 1, 0, -1, 0};
            boolean[][] visited = new boolean[m][n];
            Queue<Tree> queue = new LinkedList<>();
            queue.offer(start);
            visited[start.x][start.y] = true;
            int step = 0;
            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int k = 0; k < size; k++) {
                    Tree tree = queue.poll();
                    if (tree.x == target.x && tree.y == target.y) {
                        return step;
                    }

                    for (int i = 0; i < 4; i++) {
                        int nextX = tree.x + dirs[i];
                        int nextY = tree.y + dirs[i + 1];
                        if (nextX < 0 || nextY < 0 || nextX >= m || nextY >= n ||
visited[nextX][nextY] || forest.get(nextX).get(nextY) == 0) {
                            continue;
                        }
                        queue.offer(new Tree(nextX, nextY,
forest.get(nextX).get(nextY)));
                        visited[nextX][nextY] = true;
                    }
                }
                step++;
            }
            return -1;
        }

        class Tree {
            int x;
            int y;
            int height;

            public Tree(int x, int y, int height) {
                this.x = x;
                this.y = y;
                this.height = height;
            }

            public Tree() {
                this.x = 0;
                this.y = 0;
                this.height = 0;
            }
        }
    }
}
```

```
// ============================================================
// FILE 609: _676.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 676. Implement Magic Dictionary
 * Implement a magic directory with buildDict, and search methods.
 * For the method buildDict, you'll be given a list of non-repetitive words to build a
dictionary.
 * For the method search, you'll be given a word,
 * and judge whether if you modify exactly one character into another character in this
word,
 * the modified word is in the dictionary you just built.

 Example 1:

 Input: buildDict(["hello", "leetcode"]), Output: Null
 Input: search("hello"), Output: False
 Input: search("hhllo"), Output: True
 Input: search("hell"), Output: False
 Input: search("leetcoded"), Output: False

 Note:

 You may assume that all the inputs are consist of lowercase letters a-z.
 For contest purpose, the test data is rather small by now.
 You could think about highly efficient algorithm after the contest.
 Please remember to RESET your class variables declared in class MagicDictionary,
 as static/class variables are persisted across multiple test cases. Please see here for
more details.

 */
public class _676 {

    public static class Solution1 {
        public static class MagicDictionary {

            Set<String> wordSet;

            /**
             * Initialize your data structure here.
             */
            public MagicDictionary() {
                wordSet = new HashSet<>();
            }

            /**
             * Build a dictionary through a list of words
```

```
             */
            public void buildDict(String[] dict) {
                for (String word : dict) {
                    wordSet.add(word);
                }
            }

            /**
             * Returns if there is any word in the trie that equals to the given word
after modifying exactly one character
             */
            public boolean search(String word) {
                for (String candidate : wordSet) {
                    if (modifyOneChar(word, candidate)) {
                        return true;
                    }
                }
                return false;
            }

            private boolean modifyOneChar(String word, String candidate) {
                if (word.length() != candidate.length()) {
                    return false;
                }
                int diff = 0;
                for (int i = 0; i < word.length(); i++) {
                    if (word.charAt(i) != candidate.charAt(i)) {
                        diff++;
                    }
                    if (diff > 1) {
                        return false;
                    }
                }
                return diff == 1;
            }
        }
    }
}

// ==========================================================
// FILE 610: _677.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 677. Map Sum Pairs
 *
 *  Implement a MapSum class with insert, and sum methods.

 For the method insert, you'll be given a pair of (string, integer).
```

```
 The string represents the key and the integer represents the value.
 If the key already existed, then the original key-value pair will be overridden to the
new one.

 For the method sum, you'll be given a string representing the prefix,
 and you need to return the sum of all the pairs' value whose key starts with the prefix.

 Example 1:

 Input: insert("apple", 3), Output: Null
 Input: sum("ap"), Output: 3
 Input: insert("app", 2), Output: Null
 Input: sum("ap"), Output: 5

 */
public class _677 {
    public static class Solution1 {
        public static class MapSum {

            Map<String, Integer> map;

            /**
             * Initialize your data structure here.
             */
            public MapSum() {
                map = new HashMap<>();
            }

            public void insert(String key, int val) {
                map.put(key, val);
            }

            public int sum(String prefix) {
                int sum = 0;
                for (String key : map.keySet()) {
                    if (key.startsWith(prefix)) {
                        sum += map.get(key);
                    }
                }
                return sum;
            }
        }

    }
}

// ============================================================
// FILE 611: _678.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;
```

```
/**
 * 678. Valid Parenthesis String
 *
 *  Given a string containing only three types of characters: '(', ')' and '*',
 *  write a function to check whether this string is valid. We define the validity of a
string by these rules:

 1. Any left parenthesis '(' must have a corresponding right parenthesis ')'.
 2. Any right parenthesis ')' must have a corresponding left parenthesis '('.
 3. Left parenthesis '(' must go before the corresponding right parenthesis ')'.
 4. '*' could be treated as a single right parenthesis ')' or a single left parenthesis
'(' or an empty string.
 5. An empty string is also valid.

 Example 1:
 Input: "()"
 Output: True

 Example 2:
 Input: "(*)"
 Output: True

 Example 3:
 Input: "(*))"
 Output: True

 */
public class _678 {
    public static class Solution1 {
        /**
         * This solution is correct, but will result in TLE by test4
         */
        public boolean checkValidString(String s) {
            if (!s.contains("*")) {
                Stack<Character> stack = new Stack();
                int i = 0;
                while (i < s.length()) {
                    if (s.charAt(i) == '(') {
                        stack.push(s.charAt(i));
                    } else {
                        if (stack.isEmpty()) {
                            return false;
                        } else {
                            stack.pop();
                        }
                    }
                    i++;
                }
                return stack.isEmpty();
            } else {
                int index = s.indexOf("*");
                String transformedS = s.substring(0, index) + s.substring(index + 1);
                if (checkValidString(transformedS)) {
                    return true;
```

```
                }
                transformedS = s.substring(0, index) + "(" + s.substring(index + 1);
                if (checkValidString(transformedS)) {
                    return true;
                }
                transformedS = s.substring(0, index) + ")" + s.substring(index + 1);
                if (checkValidString(transformedS)) {
                    return true;
                }
            }
            return false;
        }
    }

    public static class Solution2 {
        public boolean checkValidString(String s) {
            return isValid(s, 0, 0);
        }

        private boolean isValid(String s, int start, int cnt) {
            if (cnt < 0) {
                return false;
            }
            for (int i = start; i < s.length(); i++) {
                char c = s.charAt(i);
                if (c == '(') {
                    cnt++;
                } else if (c == ')') {
                    if (cnt <= 0) {
                        return false;
                    }
                    cnt--;
                } else if (c == '*') {
                    /**Extra caution: start should be i+1, not start+1 !*/
                    return isValid(s, i + 1, cnt + 1)
                            || isValid(s, i + 1, cnt - 1)
                            || isValid(s, i + 1, cnt);
                }
            }
            return cnt == 0;
        }
    }

    public static class Solution3 {
        /**
         * Greedy solution:
         * 1. Let lo mean the lowest possible open left paren
         * 2. Let hi mean the possibilities of highest possible open left paren, so as
long as s.charAt(i) != ')', it's possible to be a '(', so we'll increment hi by 1
         * 2. If at any time, hi becomes negative, that means this string will never be
valid
         * 3. Each time, we'll let lo be at a minimum of zero
         */
        public boolean checkValidString(String s) {
```

```
            int lo = 0;
            int hi = 0;
            for (int i = 0; i < s.length(); i++) {
                lo += s.charAt(i) == '(' ? 1 : -1;
                hi += s.charAt(i) != ')' ? 1 : -1;
                if (hi < 0) {
                    break;
                }
                lo = Math.max(0, lo);
            }
            return lo == 0;
        }
    }
}

// ============================================================
// FILE 612: _679.java
// ============================================================

package com.fishercoder.solutions;

import java.util.stream.IntStream;

/**
 * 679. 24 Game
 *
 *  You have 4 cards each containing a number from 1 to 9.
 *  You need to judge whether they could operated through *, /, +, -, (, ) to get the
value of 24.

 Example 1:
 Input: [4, 1, 8, 7]
 Output: True
 Explanation: (8-4) * (7-1) = 24

 Example 2:
 Input: [1, 2, 1, 2]
 Output: False

 Note:
 The division operator / represents real division, not integer division. For example, 4 /
(1 - 2/3) = 12.
 Every operation done is between two numbers.
 In particular, we cannot use - as a unary operator. For example, with [1, 1, 1, 1] as
input, the expression -1 - 1 - 1 - 1 is not allowed.
 You cannot concatenate numbers together. For example, if the input is [1, 2, 1, 2], we
cannot write this as 12 + 12.

 */

public class _679 {
    public static class Solution1 {
        /**Since there are only 4 cards and only 4 operations, we can iterate through all
possible combinations, there's a total of 9216 possibilities:
```

```
         * 1. we pick two out of four cards, with order (since order matters for
division), 4 * 3 = 12, then pick one of four operations: 12 * 4 = 48;
         * 2. then we pick two from these three numbers: 12 * 4 * 3 * 4 * 2 = 1152
         * 3. then we pick the remaining two: 1152 * 2 * 4 = 9216 (with order and out of
4 operations)*/
        public boolean judgePoint24(int[] nums) {
            return dfs(IntStream.of(nums).mapToDouble(num -> num).toArray());
        }

        private boolean dfs(double[] nums) {
            if (nums.length == 1) {
                return Math.abs(nums[0] - 24) < 1e-8;//1e-8 means 0.000000001, i.e.
10^(-8)
            }

            for (int i = 0; i < nums.length; i++) {
                for (int j = 0; j < nums.length; j++) {
                    if (i != j) {
                        int len = 0;
                        double[] a = new double[nums.length - 1];
                        for (int k = 0; k < nums.length; k++) {
                            if (k != i && k != j) {
                                a[len++] = nums[k];
                            }
                        }

                        a[len] = nums[i] + nums[j];
                        if (dfs(a)) {
                            return true;
                        }

                        a[len] = nums[i] - nums[j];
                        if (dfs(a)) {
                            return true;
                        }

                        a[len] = nums[i] * nums[j];
                        if (dfs(a)) {
                            return true;
                        }

                        if (nums[j] > 1e-8) {
                            a[len] = nums[i] / nums[j];
                            if (dfs(a)) {
                                return true;
                            }
                        }
                    }
                }
            }
            return false;
        }
    }
}
```

```
// ============================================================
// FILE 613: _680.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 680. Valid Palindrome II
 *
 *  Given a non-empty string s, you may delete at most one character. Judge whether you
can make it a palindrome.

 Example 1:
 Input: "aba"
 Output: True

 Example 2:
 Input: "abca"
 Output: True
 Explanation: You could delete the character 'c'.

 Note:
 The string will only contain lowercase characters a-z. The maximum length of the string
is 50000.

 */
public class _680 {
    public static class Solution1 {
        public boolean validPalindrome(String s) {
            int left = 0;
            int right = s.length() - 1;
            int diff = 0;
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) {
                    left++;
                    diff++;
                    if (diff > 1) {
                        break;
                    }
                } else {
                    left++;
                    right--;
                }
            }
            if (diff < 2) {
                return true;
            }
            diff = 0;
            left = 0;
            right = s.length() - 1;
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) {
                    right--;
```

```
                    diff++;
                    if (diff > 1) {
                        break;
                    }
                } else {
                    left++;
                    right--;
                }
            }
            return diff < 2;
        }
    }

    public static class Solution2 {
        public boolean validPalindrome(String s) {
            int left = 0;
            int right = s.length() - 1;
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) {
                    return isValid(s, left + 1, right) || isValid(s, left, right - 1);
                }
                left++;
                right--;
            }
            return true;
        }

        private boolean isValid(String s, int left, int right) {
            while (left < right) {
                if (s.charAt(left++) != s.charAt(right--)) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 614: _681.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 681. Next Closest Time
 *
 * Given a time represented in the format "HH:MM",
 * form the next closest time by reusing the current digits.
 * There is no limit on how many times a digit can be reused.
 * You may assume the given input string is always valid.
```

```
 * For example, "01:34", "12:09" are all valid. "1:34", "12:9" are all invalid.

 Example 1:

 Input: "19:34"
 Output: "19:39"
 Explanation: The next closest time choosing from digits 1, 9, 3, 4, is 19:39,
 which occurs 5 minutes later.  It is not 19:33, because this occurs 23 hours and 59
minutes later.

 Example 2:

 Input: "23:59"
 Output: "22:22"
 Explanation: The next closest time choosing from digits 2, 3, 5, 9, is 22:22.
 It may be assumed that the returned time is next day's time since it is smaller than the
input time numerically.

 */
public class _681 {
    public static class Solution1 {
        public String nextClosestTime(String time) {
            int cur = 60 * Integer.parseInt(time.substring(0, 2));
            cur += Integer.parseInt(time.substring(3));
            Set<Integer> allowed = new HashSet();
            for (char c : time.toCharArray()) {
                if (c != ':') {
                    allowed.add(c - '0');
                }
            }

            while (true) {
                cur = (cur + 1) % (24 * 60);
                int[] digits = new int[]{cur / 60 / 10, cur / 60 % 10, cur % 60 / 10, cur
% 60 % 10};
                search:
                {
                    for (int d : digits) {
                        if (!allowed.contains(d)) {
                            break search;
                        }
                    }
                    return String.format("%02d:%02d", cur / 60, cur % 60);
                }
            }
        }
    }
}

// ============================================================
// FILE 615: _682.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.Stack;

/**
 * 682. Baseball Game
 *
 *  You're now a baseball game point recorder.

 Given a list of strings, each string can be one of the 4 following types:

 Integer (one round's score): Directly represents the number of points you get in this
round.
 "+" (one round's score): Represents that the points you get in this round are the sum of
the last two valid round's points.
 "D" (one round's score): Represents that the points you get in this round are the
doubled data of the last valid round's points.
 "C" (an operation, which isn't a round's score): Represents the last valid round's
points you get were invalid and should be removed.

 Each round's operation is permanent and could have an impact on the round before and the
round after.

 You need to return the sum of the points you could get in all the rounds.

 Example 1:

 Input: ["5","2","C","D","+"]
 Output: 15
 Explanation:
 Round 1: You could get 5 points. The sum is: 5.
 Round 2: You could get 2 points. The sum is: 7.
 Operation 1: The round 2's data was invalid. The sum is: 5.
 Round 3: You could get 10 points (the round 2's data has been removed). The sum is: 15.
 Round 4: You could get 5 + 10 = 15 points. The sum is: 30.

 Example 2:

 Input: ["5","-2","4","C","D","9","+","+"]
 Output: 27
 Explanation:
 Round 1: You could get 5 points. The sum is: 5.
 Round 2: You could get -2 points. The sum is: 3.
 Round 3: You could get 4 points. The sum is: 7.
 Operation 1: The round 3's data is invalid. The sum is: 3.
 Round 4: You could get -4 points (the round 3's data has been removed). The sum is: -1.
 Round 5: You could get 9 points. The sum is: 8.
 Round 6: You could get -4 + 9 = 5 points. The sum is 13.
 Round 7: You could get 9 + 5 = 14 points. The sum is 27.

 Note:
 The size of the input list will be between 1 and 1000.
 Every integer represented in the list will be between -30000 and 30000.
 */
public class _682 {
```

```
    public static class Solution1 {
        public int calPoints(String[] ops) {
            Stack<Integer> stack = new Stack<>();
            int sum = 0;
            int firstLast = Integer.MIN_VALUE;
            int secondLast = Integer.MIN_VALUE;
            for (String op : ops) {
                if (op.equals("+")) {
                    if (!stack.isEmpty()) {
                        firstLast = stack.pop();
                    }
                    if (!stack.isEmpty()) {
                        secondLast = stack.pop();
                    }
                    int thisRoundPoints = firstLast + secondLast;

                    if (secondLast != Integer.MIN_VALUE) {
                        stack.push(secondLast);
                    }
                    if (firstLast != Integer.MIN_VALUE) {
                        stack.push(firstLast);
                    }
                    stack.push(thisRoundPoints);
                    sum += thisRoundPoints;

                    firstLast = Integer.MIN_VALUE;
                    secondLast = Integer.MIN_VALUE;
                } else if (op.equals("D")) {
                    if (!stack.isEmpty()) {
                        int thisRoundPoints = stack.peek() * 2;
                        stack.push(thisRoundPoints);
                        sum += thisRoundPoints;
                    }
                } else if (op.equals("C")) {
                    if (!stack.isEmpty()) {
                        int removedData = stack.pop();
                        sum -= removedData;
                    }
                } else {
                    Integer val = Integer.parseInt(op);
                    sum += val;
                    stack.push(val);
                }
            }
            return sum;
        }
    }
}

// ============================================================
// FILE 616: _683.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 683. K Empty Slots
 *
 *  There is a garden with N slots. In each slot, there is a flower.
 *
 *  The N flowers will bloom one by one in N days.
 *
 *  In each day, there will be exactly one flower blooming and it will be in the status
of blooming since then.
 *
 *  Given an array flowers consists of number from 1 to N. Each number in the array
represents the place where the flower will open in that day.
 *  For example, flowers[i] = x means that the unique flower that blooms at day i will be
at position x,
 *  where i and x will be in the range from 1 to N.
 *
 *  Also given an integer k, you need to output in which day there exists two flowers in
the status of blooming,
 *  and also the number of flowers between them is k and these flowers are not blooming.
 *  If there isn't such day, output -1.

 Example 1:
 Input:
 flowers: [1,3,2]
 k: 1
 Output: 2
 Explanation: In the second day, the first and the third flower have become blooming.

 Example 2:
 Input:
 flowers: [1,2,3]
 k: 1
 Output: -1

 Note:
 The given array will be in the range [1, 20000].

 */
public class _683 {

    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/104771/java-c-simple-o-n-solution*/
        public int kEmptySlots(int[] flowers, int k) {
            int[] days = new int[flowers.length];
            for (int i = 0; i < flowers.length; i++) {
                days[flowers[i] - 1] = i + 1;
            }
            int left = 0;
            int right = k + 1;
            int result = Integer.MAX_VALUE;
            for (int i = 0; right < flowers.length; i++) {
                if (days[i] < days[left] || days[i] <= days[right]) {
```

```
                    if (i == right) {
                        result = Math.min(result, Math.max(days[left], days[right]));
                    }
                    left = i;
                    right = k + 1 + i;
                }
            }
            return result == Integer.MAX_VALUE ? -1 : result;
        }
    }
}

// ============================================================
// FILE 617: _684.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 684. Redundant Connection
 *
 *  We are given a "tree" in the form of a 2D-array, with distinct values for each node.
 *  In the given 2D-array, each element pair [u, v] represents that v is a child of u in
the tree.
 *  We can remove exactly one redundant pair in this "tree" to make the result a (rooted)
tree.
 *  You need to find and output such a pair. If there are multiple answers for this
question, output the one appearing last in the 2D-array. There is always at least one
answer.

 Example 1:
 Input: [[1,2], [1,3], [2,3]]
 Output: [2,3]
 Explanation: Original tree will be like this:
   1
  / \
 2 - 3


 Example 2:
 Input: [[1,2], [1,3], [3,1]]
 Output: [3,1]
 Explanation: Original tree will be like this:
   1
  / \\
 2   3

 Note:
 The size of the input 2D-array will be between 1 and 1000.
 Every integer represented in the 2D-array will be between 1 and 2000.
 */
```

```
public class _684 {

    public static class Solution1 {
        /**
         * This is my original solution. A little verbose.
         */
        class UnionFind {
            int[] ids;
            Set<Integer> nodes;
            Set<Integer> visitedNodes;
            int[] redundantConn;
            int m;
            int n;

            public UnionFind(int[][] edges) {
                m = edges.length;
                n = edges[0].length;
                nodes = new HashSet<>();
                visitedNodes = new HashSet<>();
                redundantConn = new int[]{};
                for (int i = 0; i < m; i++) {
                    for (int j = 0; j < n; j++) {
                        nodes.add(edges[i][j]);
                    }
                }
                ids = new int[nodes.size()];
                for (int i = 0; i < ids.length; i++) {
                    ids[i] = i + 1;
                }
            }

            public int[] union(int[] edge) {
                int x = find(edge[0] - 1);
                int y = find(edge[1] - 1);

                if (x == y && visitedNodes.contains(edge[0]) &&
visitedNodes.contains(edge[1])) {
                    redundantConn = edge;
                }

                if (x != y) {
                    if (x < y) {
                        ids[y] = x + 1;
                    } else {
                        ids[x] = y + 1;
                    }
                }

                visitedNodes.add(edge[0]);
                visitedNodes.add(edge[1]);
                return redundantConn;
            }

            private int find(int id) {
```

```
                if (isTree()) {
                    return ids[id];
                }
                if ((id + 1) != ids[id]) {
                    return find(ids[id] - 1);
                }
                return id;
            }

            private boolean isTree() {
                Set<Integer> set = new HashSet<>();
                for (int i : ids) {
                    set.add(i);
                }
                return set.size() == 1;
            }
        }

        public int[] findRedundantConnection(int[][] edges) {
            UnionFind unionFind = new UnionFind(edges);
            int[] result = new int[]{};
            for (int[] edge : edges) {
                result = unionFind.union(edge);
            }
            return result;
        }
    }
}

// ============================================================
// FILE 618: _685.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 685. Redundant Connection II
 *
 * In this problem, a rooted tree is a directed graph such that, there is exactly one
node (the root) for which all other nodes are
 * descendants of this node, plus every node has exactly one parents, except for the root
node which has no parents.
 * The given input is a directed graph that started as a rooted tree with N nodes (with
distinct values 1, 2, ..., N),
 * with one additional directed edge added. The added edge has two different vertices
chosen from 1 to N, and was not an edge that already existed.
 * The resulting graph is given as a 2D-array of edges. Each element of edges is a pair
[u, v] that
 * represents a directed edge connecting nodes u and v, where u is a parents of child v.
```

```
 * Return an edge that can be removed so that the resulting graph is a rooted tree of N nodes.
 * If there are multiple answers, return the answer that occurs last in the given 2D-array.

 Example 1:
 Input: [[1,2], [1,3], [2,3]]
 Output: [2,3]
 Explanation: The given directed graph will be like this:
   1
  / \
 v   v
 2-->3

 Example 2:
 Input: [[1,2], [2,3], [3,4], [4,1], [1,5]]
 Output: [4,1]
 Explanation: The given directed graph will be like this:
 5 <- 1 -> 2
      ^    |
      |    v
      4 <- 3

 Note:
 The size of the input 2D-array will be between 3 and 1000.
 Every integer represented in the 2D-array will be between 1 and N, where N is the size of the input array.
 */
public class _685 {
    public static class Solution1 {
        /**
         * My original solution, failed by _685Test.test3
         */
        class UnionFind {
            int[] ids;
            Set<Integer> nodes;
            int[][] edges;
            List<LinkedNode> visitedLinkedNodes;
            Set<Integer> visitedValues;
            int[] redundantConn;
            int m;
            int n;

            class LinkedNode {
                List<LinkedNode> parents;//at most, there's one node that has two parents
                int val;

                public LinkedNode(int val) {
                    this.val = val;
                }

                public LinkedNode(int val, LinkedNode parent) {
                    if (parents == null) {
                        parents = new ArrayList<>();
```

```
                }
                this.parents.add(parent);
                this.val = val;
            }

            public void addParent(LinkedNode parent) {
                if (parents == null) {
                    parents = new ArrayList<>();
                }
                this.parents.add(parent);
            }
        }

        public UnionFind(int[][] edges) {
            this.edges = edges;
            m = edges.length;
            n = edges[0].length;
            nodes = new HashSet<>();
            visitedLinkedNodes = new ArrayList<>();
            visitedValues = new HashSet<>();
            redundantConn = new int[]{};
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    nodes.add(edges[i][j]);
                }
            }
            ids = new int[nodes.size()];
            for (int i = 0; i < ids.length; i++) {
                ids[i] = i + 1;
            }
        }

        public void union(int[] edge) {
            if (!visitedValues.contains(edge[1])) {
                LinkedNode parent = new LinkedNode(edge[0]);
                LinkedNode node = new LinkedNode(edge[1], parent);
                visitedLinkedNodes.add(node);
                visitedValues.add(edge[1]);
            } else {
                for (int i = 0; i < visitedLinkedNodes.size(); i++) {
                    LinkedNode node = visitedLinkedNodes.get(i);
                    if (node.val == edge[1]) {
                        node.addParent(new LinkedNode(edge[0]));
                    }
                }
            }
        }

        public int[] findRedundantDirectedConnection() {
            int index = hasTwoParents();
            if (index != -1) {
                List<LinkedNode> parents =
visitedLinkedNodes.get(index).parents;//parents size is fixed, only 2
                int[] result = new int[2];
```

```
                    for (int i = 0; i < parents.size(); i++) {
                        if (hasCycle(visitedLinkedNodes.get(index), parents.get(i))) {
                            result = new int[]{parents.get(i).val,
visitedLinkedNodes.get(index).val};
                            break;
                        }
                    }
                    return result;
                } else {
                    return edges[m - 1];
                }
            }

            private boolean hasCycle(LinkedNode linkedNode, LinkedNode parent) {
                Set<Integer> visited = new HashSet<>();
                visited.add(linkedNode.val);
                while (parent != null) {
                    if (visited.contains(parent.val)) {
                        return true;
                    }
                    visited.add(parent.val);
                    parent = findParent(parent.val);
                }
                return false;
            }

            private LinkedNode findParent(int val) {
                for (int i = 0; i < visitedLinkedNodes.size(); i++) {
                    if (visitedLinkedNodes.get(i).val == val) {
                        return visitedLinkedNodes.get(i).parents.get(0);
                    }
                }
                return null;
            }

            private int hasTwoParents() {
                for (int i = 0; i < visitedLinkedNodes.size(); i++) {
                    if (visitedLinkedNodes.get(i).parents.size() > 1) {
                        return i;
                    }
                }
                return -1;
            }
        }

        public int[] findRedundantDirectedConnection(int[][] edges) {
            UnionFind unionFind = new UnionFind(edges);
            /**two cases:
             * 1. the entire edges are just one directed loop, in this case, just return
the last edge, see test2 in _685Test.java
             * 2. there's one directed loop, but one node of the loop has two parents, in
this case, what we'll need to do
             * is just to return the edge in this loop that points to the child that has
two parents, see test1 in _685Test.java
```

```
             * also, in case 2, use the id of the node that has two parents as the id for
all nodes in this loop, this way, I could know which of its
             * two parents is in the loop and should be the redundant one to return.
             * */
            for (int[] edge : edges) {
                unionFind.union(edge);
            }
            return unionFind.findRedundantDirectedConnection();
        }
    }

    public static class Solution2 {
        /**credit: https://discuss.leetcode.com/topic/105108/c-java-union-find-with-
explanation-o-n*/
        public int[] findRedundantDirectedConnection(int[][] edges) {
            int[] can1 = {-1, -1};
            int[] can2 = {-1, -1};
            int[] parent = new int[edges.length + 1];
            for (int i = 0; i < edges.length; i++) {
                if (parent[edges[i][1]] == 0) {
                    parent[edges[i][1]] = edges[i][0];
                } else {
                    can2 = new int[]{edges[i][0], edges[i][1]};
                    can1 = new int[]{parent[edges[i][1]], edges[i][1]};
                    edges[i][1] = 0;
                }
            }
            for (int i = 0; i < edges.length; i++) {
                parent[i] = i;
            }
            for (int i = 0; i < edges.length; i++) {
                if (edges[i][1] == 0) {
                    continue;
                }
                int child = edges[i][1];
                int father = edges[i][0];
                if (root(parent, father) == child) {
                    if (can1[0] == -1) {
                        return edges[i];
                    }
                    return can1;
                }
                parent[child] = father;
            }
            return can2;
        }

        int root(int[] parent, int i) {
            while (i != parent[i]) {
                parent[i] = parent[parent[i]];
                i = parent[i];
            }
            return i;
        }
```

```
    }
}

// ============================================================
// FILE 619: _686.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 686. Repeated String Match
 *
 * Given two strings A and B, find the minimum number of times A has to be repeated such
that B is a substring of it.
 * If no such solution, return -1.
 * For example, with A = "abcd" and B = "cdabcdab".
 * Return 3, because by repeating A three times (“abcdabcdabcd”),
 * B is a substring of it; and B is not a substring of A repeated two times ("abcdabcd").

 Note:
 The length of A and B will be between 1 and 10000.
 */

public class _686 {
    public static class Solution1 {
        public int repeatedStringMatch(String A, String B) {
            Set<Character> set = new HashSet<>();
            for (char c : A.toCharArray()) {
                set.add(c);
            }
            for (char c : B.toCharArray()) {
                if (!set.contains(c)) {
                    return -1;
                }
            }
            StringBuilder stringBuilder = new StringBuilder(A);
            for (int i = 0; i < B.length(); i++) {
                if (stringBuilder.toString().contains(B)) {
                    return i + 1;
                }
                stringBuilder.append(A);
            }
            return -1;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(N(N+M))
         * Space: O(N + M)
         * */
```

```
        public int repeatedStringMatch(String A, String B) {
            int count = 1;
            StringBuilder sb = new StringBuilder(A);
            for (; sb.length() < B.length(); count++) {
                sb.append(A);
            }
            if (sb.indexOf(B) >= 0) {
                return count;
            }
            sb.append(A);
            if (sb.indexOf(B) >= 0) {
                return count + 1;
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 620: _687.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 687. Longest Univalue Path
 *
 * Given a binary tree, find the length of the longest path where each node in the path
has the same value.
 * This path may or may not pass through the root.

 Note: The length of path between two nodes is represented by the number of edges between
them.

 Example 1:
 Input:
     5
    / \
   4   5
  / \   \
 1  1   5

 Output:
 2

 Example 2:
 Input:

     1
    / \
   4   5
  / \   \
```

```
 4  4   5
 Output:
 2

 Note: The given binary tree has not more than 10000 nodes. The height of the tree is not
more than 1000.
 */

public class _687 {
    public static class Solution1 {
        /**
         * Use a one element array to pass in and out is a common technique for handling
tree questions.
         */
        public int longestUnivaluePath(TreeNode root) {
            int[] result = new int[1];
            if (root != null) {
                dfs(root, result);
            }
            return result[0];
        }

        // calculate longest univalue path from root to leaves
        // In addition, the maximum univalue path cross the root node is calculated and
then global maximum is udpated.
        private int dfs(TreeNode root, int[] result) {
            int leftPath = root.left == null ? 0 : dfs(root.left, result);
            int rightPath = root.right == null ? 0 : dfs(root.right, result);
            int leftResult = (root.left != null && root.left.val == root.val) ? leftPath
+ 1 : 0;
            int rightResult = (root.right != null && root.right.val == root.val) ?
rightPath + 1 : 0;
            result[0] = Math.max(result[0], leftResult + rightResult);
            return Math.max(leftResult, rightResult);
        }
    }
}

// ============================================================
// FILE 621: _688.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.LinkedList;
import java.util.Queue;

/**
 * 688. Knight Probability in Chessboard
 *
 *  On an NxN chessboard, a knight starts at the r-th row and c-th column and attempts to
make exactly K moves.
 *  The rows and columns are 0 indexed, so the top-left square is (0, 0), and the bottom-
```

```
right square is (N-1, N-1).
 *  A chess knight has 8 possible moves it can make, as illustrated below.
 *  Each move is two squares in a cardinal direction, then one square in an orthogonal
direction.
 *  Each time the knight is to move, it chooses one of eight possible moves uniformly at
random
 *  (even if the piece would go off the chessboard) and moves there.
 *  The knight continues moving until it has made exactly K moves or has moved off the
chessboard.
 *  Return the probability that the knight remains on the board after it has stopped
moving.

 Example:

 Input: 3, 2, 0, 0
 Output: 0.0625
 Explanation: There are two moves (to (1,2), (2,1)) that will keep the knight on the
board.
 From each of those positions, there are also two moves that will keep the knight on the
board.
 The total probability the knight stays on the board is 0.0625.

 Note:
 N will be between 1 and 25.
 K will be between 0 and 100.
 The knight always initially starts on the board.
 */
public class _688 {

    public static class Solution1 {
        /**
         * This BFS solution results in TLE on Leetcode.
         */
        public double knightProbability(int N, int K, int r, int c) {
            int[][] directions = {{-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2},
{-1, -2}, {-2, -1}};
            Queue<int[]> queue = new LinkedList<>();
            queue.offer(new int[]{r, c});
            int level = K;
            while (level-- > 0) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    int[] curr = queue.poll();
                    for (int[] direction : directions) {
                        int x = curr[0] + direction[0];
                        int y = curr[1] + direction[1];
                        if (x >= 0 && x < N && y >= 0 && y < N) {
                            queue.offer(new int[]{x, y});
                        }
                    }
                }
            }
            double prob = queue.size();
            for (int i = 0; i < K; i++) {
```

```
                prob /= 8;
            }
            return prob;
        }
    }

    public static class Solution2 {
        /**
         * Let f[r][c][k] mean the probability that the knight is still on board after k
steps,
         * we can deduce a recursion from its k-1 steps
         * In addition, instead of using a 3-d array, we can only keep the most recent
two layers,
         * i.e. using only two 2-d arrays.
         */
        public double knightProbability(int N, int K, int r, int c) {
            int[][] directions = {{-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2},
{-1, -2}, {-2, -1}};
            double[][] dp0 = new double[N][N];
            for (double[] row : dp0) {
                Arrays.fill(row, 1);
            }
            for (int k = 0; k < K; k++) {
                double[][] dp1 = new double[N][N];
                for (int i = 0; i < N; i++) {
                    for (int j = 0; j < N; j++) {
                        for (int l = 0; l < directions.length; l++) {
                            int[] direction = directions[l];
                            int x = i + direction[0];
                            int y = j + direction[1];
                            if (x >= 0 && y >= 0 && x < N && y < N) {
                                dp1[i][j] += dp0[x][y];
                            }
                        }
                    }
                }
                dp0 = dp1;
            }
            return dp0[r][c] / Math.pow(8, K);
        }
    }

    public static void main(String... args) {
        System.out.println((double) 2 / 8);
    }
}

// ============================================================
// FILE 622: _689.java
// ============================================================

package com.fishercoder.solutions;

/**
```

```
 * 689. Maximum Sum of 3 Non-Overlapping Subarrays
 *
 * In a given array nums of positive integers, find three non-overlapping subarrays with
maximum sum.
 * Each subarray will be of size k, and we want to maximize the sum of all 3*k entries.
 * Return the result as a list of indices representing the starting position of each
interval (0-indexed).
 * If there are multiple answers, return the lexicographically smallest one.

 Example:
 Input: [1,2,1,2,6,7,5,1], 2
 Output: [0, 3, 5]
 Explanation: Subarrays [1, 2], [2, 6], [7, 5] correspond to the starting indices [0, 3,
5].
 We could have also taken [2, 1], but an answer of [1, 3, 5] would be lexicographically
larger.

 Note:
 nums.length will be between 1 and 20000.
 nums[i] will be between 1 and 65535.
 k will be between 1 and floor(nums.length / 3).
 */
public class _689 {
    public static class Solution1 {
        /**we basically need to find the interval (i, i+k-1) as the middle interval,
where k <= i <= n-2k
         * then this interval (0, i-1) will be the left interval
         * the interval (i+k, n-1) will be the right interval.
         *
         * Please pay special attention to the variable name I use here: this `k` is not
a random one, it's the `k`
         * from the passed in parameter.
         *
         * Credit: https://discuss.leetcode.com/topic/105577/c-java-dp-with-explanation-
o-n/*/
        public int[] maxSumOfThreeSubarrays(int[] nums, int k) {
            if (nums == null || nums.length == 0) {
                return new int[]{};
            }
            int n = nums.length;
            int[] sums = new int[n + 1];
            for (int i = 0; i < n; i++) {
                sums[i + 1] = sums[i] + nums[i];
            }

            int[] leftMax = new int[n];
            for (int i = k, total = sums[k] - sums[0]; i < n; i++) {
                if (sums[i + 1] - sums[i + 1 - k] > total) {
                    leftMax[i] = i + 1 - k;
                    total = sums[i + 1] - sums[i + 1 - k];
                } else {
                    leftMax[i] = leftMax[i - 1];
                }
            }
```

```
            int[] rightMax = new int[n];
            rightMax[n - k] = n - k;
            for (int i = n - k - 1, total = sums[n] - sums[n - k]; i >= 0; i--) {
                if (sums[i + k] - sums[i] >= total) {
                    rightMax[i] = i;
                    total = sums[i + k] - sums[i];
                } else {
                    rightMax[i] = rightMax[i + 1];
                }
            }

            //try to find all possible middle intervals
            int[] result = new int[3];
            int max = 0;
            for (int i = k; i <= n - 2 * k; i++) {
                int left = leftMax[i - 1];
                int right = rightMax[i + k];
                int total = (sums[i + k] - sums[i]) + (sums[left + k] - sums[left]) +
(sums[right + k] - sums[right]);
                if (total > max) {
                    max = total;
                    result[0] = left;
                    result[1] = i;
                    result[2] = right;
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        /**reference: https://leetcode.com/articles/maximum-sum-of-3-non-overlapping-
intervals*/
    }
}

// ============================================================
// FILE 623: _690.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.Employee;

import java.util.List;
import java.util.stream.Collectors;

/**
 * 690. Employee Importance
 *
 * You are given a data structure of employee information, which includes the employee's
unique id,
 * his importance value and his direct subordinates' id.
```

```
 * For example, employee 1 is the leader of employee 2, and employee 2 is the leader of
employee 3.
 * They have importance value 15, 10 and 5, respectively.
 * Then employee 1 has a data structure like [1, 15, [2]], and
 * employee 2 has [2, 10, [3]], and employee 3 has [3, 5, []].
 * Note that although employee 3 is also a subordinate of employee 1, the relationship is
not direct.
 * Now given the employee information of a company, and an employee id,
 * you need to return the total importance value of this employee and all his
subordinates.

 Example 1:

 Input: [[1, 5, [2, 3]], [2, 3, []], [3, 3, []]], 1
 Output: 11
 Explanation:
 Employee 1 has importance value 5, and he has two direct subordinates: employee 2 and
employee 3.
 They both have importance value 3. So the total importance value of employee 1 is 5 + 3
+ 3 = 11.

 Note:

 One employee has at most one direct leader and may have several subordinates.
 The maximum number of employees won't exceed 2000.

 */
public class _690 {

    public static class Solution1 {

        int total = 0;

        public int getImportance(List<Employee> employees, int id) {
            Employee manager = employees.stream().filter(e -> e.id ==
id).collect(Collectors.toList()).get(0);
            total += manager.importance;
            manager.subordinates.forEach(subId -> getImportance(employees, subId));

            /**The above line is equivalent to below for loop*/
//            for (int subId : manager.subordinates) {
//                getImportance(employees, subId);
//            }

            return total;
        }

    }
}

// ============================================================
// FILE 624: _691.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 691. Stickers to Spell Word
 *
 * We are given N different types of stickers. Each sticker has a lowercase English word on it.
 * You would like to spell out the given target string by cutting individual letters from your collection of stickers and rearranging them.
 * You can use each sticker more than once if you want, and you have infinite quantities of each sticker.
 * What is the minimum number of stickers that you need to spell out the target? If the task is impossible, return -1.

 Example 1:
 Input:
 ["with", "example", "science"], "thehat"
 Output:
 3

 Explanation:
 We can use 2 "with" stickers, and 1 "example" sticker.
 After cutting and rearrange the letters of those stickers, we can form the target "thehat".
 Also, this is the minimum number of stickers necessary to form the target string.

 Example 2:
 Input:
 ["notice", "possible"], "basicbasic"
 Output:
 -1

 Explanation:
 We can't form the target "basicbasic" from cutting letters from the given stickers.

 Note:
 stickers has length in the range [1, 50].
 stickers consists of lowercase English words (without apostrophes).
 target has length in the range [1, 15], and consists of lowercase English letters.
 In all test cases, all words were chosen randomly from the 1000 most common US English words, and the target was chosen as a concatenation of two random words.
 The time limit may be more challenging than usual. It is expected that a 50 sticker test case can be solved within 35ms on average.
 */
public class _691 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/106273/c-java-python-dp-memoization-with-optimization-29-ms-c/2
         */
        public int minStickers(String[] stickers, String target) {
```

```
            int m = stickers.length;
            int[][] mp = new int[m][26];
            Map<String, Integer> dp = new HashMap<>();
            for (int i = 0; i < m; i++) {
                for (char c : stickers[i].toCharArray()) {
                    mp[i][c - 'a']++;
                }
            }
            dp.put("", 0);
            return helper(dp, mp, target);
        }

        private int helper(Map<String, Integer> dp, int[][] mp, String target) {
            if (dp.containsKey(target)) {
                return dp.get(target);
            }
            int ans = Integer.MAX_VALUE;
            int n = mp.length;
            int[] tar = new int[26];
            for (char c : target.toCharArray()) {
                tar[c - 'a']++;
            }
            // try every sticker
            for (int i = 0; i < n; i++) {
                // optimization
                if (mp[i][target.charAt(0) - 'a'] == 0) {
                    continue;
                }
                StringBuilder sb = new StringBuilder();
                // apply a sticker on every character a-z
                for (int j = 0; j < 26; j++) {
                    if (tar[j] > 0) {
                        for (int k = 0; k < Math.max(0, tar[j] - mp[i][j]); k++) {
                            sb.append((char) ('a' + j));
                        }
                    }
                }
                String s = sb.toString();
                int tmp = helper(dp, mp, s);
                if (tmp != -1) {
                    ans = Math.min(ans, 1 + tmp);
                }
            }
            dp.put(target, ans == Integer.MAX_VALUE ? -1 : ans);
            return dp.get(target);
        }
    }
}

// ============================================================
// FILE 625: _692.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.SortedSet;
import java.util.TreeSet;

/**
 * 692. Top K Frequent Words
 *
 * Given a non-empty list of words, return the k most frequent elements.
 * Your answer should be sorted by frequency from highest to lowest.
 * If two words have the same frequency, then the word with the lower alphabetical order
comes first.

 Example 1:
 Input: ["i", "love", "leetcode", "i", "love", "coding"], k = 2
 Output: ["i", "love"]
 Explanation: "i" and "love" are the two most frequent words.
 Note that "i" comes before "love" due to a lower alphabetical order.

 Example 2:
 Input: ["the", "day", "is", "sunny", "the", "the", "the", "sunny", "is", "is"], k = 4
 Output: ["the", "is", "sunny", "day"]
 Explanation: "the", "is", "sunny" and "day" are the four most frequent words,
 with the number of occurrence being 4, 3, 2 and 1 respectively.

 Note:
 You may assume k is always valid, 1 ≤ k ≤ number of unique elements.
 Input words contain only lowercase letters.

 Follow up:
 Try to solve it in O(n log k) time and O(n) extra space.
 Can you solve it in O(n) time with only O(k) extra space?

 */
public class _692 {

    public static class Solution1 {
        /**
         * O(n) extra space
         * O(nlogk) time
         * */
        public List<String> topKFrequent(String[] words, int k) {
            Map<String, Integer> map = new HashMap<>();
            for (String word : words) {
                map.put(word, map.getOrDefault(word, 0) + 1);
            }

            SortedSet<Map.Entry<String, Integer>> sortedset = new TreeSet<>(
                    (e1, e2) -> {
                        if (e1.getValue() != e2.getValue()) {
```

```
                            return e2.getValue() - e1.getValue();
                        } else {
                            return e1.getKey().compareToIgnoreCase(e2.getKey());
                        }
                    });
            sortedset.addAll(map.entrySet());

            List<String> result = new ArrayList<>();
            Iterator<Map.Entry<String, Integer>> iterator = sortedset.iterator();
            while (iterator.hasNext() && k-- > 0) {
                result.add(iterator.next().getKey());
            }
            return result;
        }
    }
}

// ==========================================================
// FILE 626: _693.java
// ==========================================================

package com.fishercoder.solutions;

/**
 * 693. Binary Number with Alternating Bits
 *
 * Given a positive integer, check whether it has alternating bits: namely, if two
adjacent bits will always have different values.

 Example 1:
 Input: 5
 Output: True
 Explanation:
 The binary representation of 5 is: 101

 Example 2:
 Input: 7
 Output: False
 Explanation:
 The binary representation of 7 is: 111.

 Example 3:
 Input: 11
 Output: False
 Explanation:
 The binary representation of 11 is: 1011.

 Example 4:
 Input: 10
 Output: True
 Explanation:
 The binary representation of 10 is: 1010.
 */
```

```
public class _693 {
    public static class Solution1 {
        public boolean hasAlternatingBits(int n) {
            String binaryStr = Integer.toBinaryString(n);
            for (int i = 1; i < binaryStr.length(); i++) {
                if (binaryStr.charAt(i - 1) == binaryStr.charAt(i)) {
                    return false;
                }
            }
            return true;
        }
    }

    public static class Solution2 {
        public boolean hasAlternatingBits(int n) {
            return Integer.bitCount(((n >> 1) ^ n) + 1) == 1;
        }
    }
}

// ============================================================
// FILE 627: _694.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 694. Number of Distinct Islands
 *
 * Given a non-empty 2D array grid of 0's and 1's, an island is a group of 1's
(representing land)
 * connected 4-directionally (horizontal or vertical.) You may assume all four edges of
the grid are surrounded by water.
 * Count the number of distinct islands.
 * An island is considered to be the same as another if and only if
 * one island can be translated (and not rotated or reflected) to equal the other.

 Example 1:

 11000
 11000
 00011
 00011

 Given the above grid map, return 1.

 Example 2:
```

```
 11011
 10000
 00001
 11011

 Given the above grid map, return 3.

 Notice that:

 11
 1

 and

 1
 11

 are considered different island shapes, because we do not consider reflection /
rotation.

 Note: The length of each dimension in the given grid does not exceed 50.
 */
public class _694 {
    public static class Solution1 {
        /**
         * My original idea:
         * my not fully working yet: the equals() and hashcode() methods need to be
refined
         * because HashSet is not really filtering the islands wiht the same shape.
         */
        class Quadrilateral {
            int[] topLeft;
            int[] bottomLeft;
            int[] topRight;
            int[] bottomRight;
            int area;

            public Quadrilateral(int i, int j) {
                this.area = 0;
                this.topLeft = new int[]{i, j};
                this.topRight = new int[]{i, j};
                this.bottomLeft = new int[]{i, j};
                this.bottomRight = new int[]{i, j};
            }

            @Override
            public boolean equals(Object o) {
                if (this == o) {
                    return true;
                }
                if (!(o instanceof Quadrilateral)) {
                    return false;
                }
```

```
                Quadrilateral that = (Quadrilateral) o;
                return this.area == that.area && checkDistance(that);
            }

            private boolean checkDistance(Quadrilateral that) {
                int thisTop = computeDistance(this.topLeft, this.topRight);
                int thatTop = computeDistance(that.topLeft, that.topRight);
                if (thisTop != thatTop) {
                    return false;
                }
                int thisRight = computeDistance(this.topRight, this.bottomRight);
                int thatRight = computeDistance(that.topRight, that.bottomRight);
                if (thisRight != thatRight) {
                    return false;
                }
                int thisBottom = computeDistance(this.bottomRight, this.bottomLeft);
                int thatBottom = computeDistance(that.bottomRight, that.bottomLeft);
                if (thisBottom != thatBottom) {
                    return false;
                }
                int thisLeft = computeDistance(this.bottomLeft, this.topLeft);
                int thatLeft = computeDistance(that.bottomLeft, that.topLeft);
                return thisLeft == thatLeft;
            }

            private int computeDistance(int[] A, int[] B) {
                return (int) (Math.pow(A[0] - B[0], 2) + Math.pow(A[1] - B[1], 2));
            }

            @Override
            public int hashCode() {
                return area + computeDistance(this.topLeft, this.topRight) +
computeDistance(this.topRight, this.bottomRight)
                        + computeDistance(this.bottomRight, this.bottomLeft) +
computeDistance(this.bottomLeft, this.topLeft);
            }

            public void addPoint(int i, int j) {
                //todo: check wether this point (i,j) is in the range, if not, expand the
range
                if (i == topRight[0]) {
                    topRight[1] = Math.max(topRight[1], j);
                }
                if (j == topRight[1]) {
                    topRight[0] = Math.min(topRight[1], i);
                }
                if (i == topLeft[0]) {
                    topLeft[1] = Math.min(topLeft[1], j);
                }
                if (j == topLeft[1]) {
                    topLeft[0] = Math.min(topLeft[0], i);
                }
                if (i == bottomLeft[0]) {
                    bottomLeft[1] = Math.min(bottomLeft[1], j);
```

```
                }
                if (j == bottomLeft[1]) {
                    bottomLeft[0] = Math.max(bottomLeft[0], i);
                }
                if (j == bottomRight[1]) {
                    bottomRight[0] = Math.max(bottomRight[0], i);
                }
                if (i == bottomRight[0]) {
                    bottomRight[1] = Math.max(bottomRight[1], j);
                }
            }

            public void addArea() {
                this.area++;
            }
        }

        public int numDistinctIslands(int[][] grid) {
            Set<Quadrilateral> set = new HashSet<>();
            int m = grid.length;
            int n = grid[0].length;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        Quadrilateral quadrilateral = dfs(grid, i, j, m, n, new
Quadrilateral(i, j));
                        set.add(quadrilateral);
                    }
                }
            }
            return set.size();
        }

        private Quadrilateral dfs(int[][] grid, int i, int j, int m, int n, Quadrilateral
quadrilateral) {
            if (i < 0 || j < 0 || i >= m || j >= n || grid[i][j] == 0) {
                return quadrilateral;
            }
            grid[i][j] = 0;
            quadrilateral.addPoint(i, j);
            quadrilateral.addArea();
            quadrilateral = dfs(grid, i + 1, j, m, n, quadrilateral);
            quadrilateral = dfs(grid, i - 1, j, m, n, quadrilateral);
            quadrilateral = dfs(grid, i, j + 1, m, n, quadrilateral);
            quadrilateral = dfs(grid, i, j - 1, m, n, quadrilateral);
            return quadrilateral;
        }
    }

    public static class Solution2 {
        int[][] directions = new int[][]{
                {0, 1},
                {1, 0},
                {0, -1},
```

```
                {-1, 0}
        };

        public int numDistinctIslands(int[][] grid) {
            int m = grid.length;
            int n = grid[0].length;
            Set<List<List<Integer>>> uniqueShapeIslands = new HashSet<>();
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    List<List<Integer>> island = new ArrayList<>();
                    if (dfs(i, j, i, j, grid, m, n, island)) {
                        uniqueShapeIslands.add(island);
                    }
                }
            }
            return uniqueShapeIslands.size();
        }

        private boolean dfs(int i0, int j0, int i, int j, int[][] grid, int m, int n,
List<List<Integer>> island) {
            if (i < 0 || j < 0 || i >= m || j >= n || grid[i][j] <= 0) {
                return false;
            }
            island.add(Arrays.asList(i - i0, j - j0));
            grid[i][j] *= -1;
            for (int k = 0; k < 4; k++) {
                dfs(i0, j0, i + directions[k][0], j + directions[k][1], grid, m, n,
island);
            }
            return true;
        }
    }
}

// ==========================================================
// FILE 628: _695.java
// ==========================================================

package com.fishercoder.solutions;

/**
 * 695. Max Area of Island
 *
 * Given a non-empty 2D array grid of 0's and 1's, an island is a group of 1's
(representing land)
 * connected 4-directionally (horizontal or vertical.)
 * You may assume all four edges of the grid are surrounded by water.
 * Find the maximum area of an island in the given 2D array. (If there is no island, the
maximum area is 0.)

 Example 1:

 [[0,0,1,0,0,0,0,1,0,0,0,0,0],
 [0,0,0,0,0,0,0,1,1,1,0,0,0],
```

```
 [0,1,1,0,1,0,0,0,0,0,0,0,0],
 [0,1,0,0,1,1,0,0,1,0,1,0,0],
 [0,1,0,0,1,1,0,0,1,1,1,0,0],
 [0,0,0,0,0,0,0,0,0,0,1,0,0],
 [0,0,0,0,0,0,0,1,1,1,0,0,0],
 [0,0,0,0,0,0,0,1,1,0,0,0,0]]

 Given the above grid, return 6. Note the answer is not 11, because the island must be
connected 4-directionally.

 Example 2:

 [[0,0,0,0,0,0,0,0]]

 Given the above grid, return 0.

 Note: The length of each dimension in the given grid does not exceed 50.
 */

public class _695 {

    public static class Solution1 {
        public int maxAreaOfIsland(int[][] grid) {
            if (grid == null || grid.length == 0) {
                return 0;
            }
            int m = grid.length;
            int n = grid[0].length;
            int max = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        int area = dfs(grid, i, j, m, n, 0);
                        max = Math.max(area, max);
                    }
                }
            }
            return max;
        }

        int dfs(int[][] grid, int i, int j, int m, int n, int area) {
            if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == 0) {
                return area;
            }
            grid[i][j] = 0;
            area++;
            area = dfs(grid, i + 1, j, m, n, area);
            area = dfs(grid, i, j + 1, m, n, area);
            area = dfs(grid, i - 1, j, m, n, area);
            area = dfs(grid, i, j - 1, m, n, area);
            return area;
        }
    }
```

```
}

// ============================================================
// FILE 629: _696.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 696. Count Binary Substrings
 *
 * Give a string s, count the number of non-empty (contiguous) substrings that have the
same number of 0's and 1's,
 * and all the 0's and all the 1's in these substrings are grouped consecutively.
 * Substrings that occur multiple times are counted the number of times they occur.

 Example 1:

 Input: "00110011"
 Output: 6
 Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's:
"0011", "01", "1100", "10", "0011", and "01".

 Notice that some of these substrings repeat and are counted the number of times they
occur.

 Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped
together.

 Example 2:

 Input: "10101"
 Output: 4
 Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of
consecutive 1's and 0's.

 Note:
 s.length will be between 1 and 50,000.
 s will only consist of "0" or "1" characters.
 */
public class _696 {
    public static class Solution1 {
        public int countBinarySubstrings(String s) {
            int n = s.length();
            /**a[i][0] denotes from most left up to i (inclusive), how many consecutive
0's
             * a[i][1] denotes from most left up to i (inclusive), how many consecutive
1's*/
            int[][] a = new int[n][2];
            /**a[i][0] denotes from i (inclusive) to the most right, how many consecutive
0's
             * b[i][0] denotes from i (inclusive) to the most right, how many consecutive
1's*/
            int[][] b = new int[n][2];
```

```
            for (int i = 0; i < n; i++) {
                if (s.charAt(i) == '0') {
                    a[i][0] = 1 + (i - 1 >= 0 ? a[i - 1][0] : 0);
                } else {
                    a[i][1] = 1 + (i - 1 >= 0 ? a[i - 1][1] : 0);
                }
            }
            for (int i = n - 1; i >= 0; i--) {
                if (s.charAt(i) == '0') {
                    b[i][0] = 1 + (i + 1 < n ? b[i + 1][0] : 0);
                } else {
                    b[i][1] = 1 + (i + 1 < n ? b[i + 1][1] : 0);
                }

            }
            long ans = 0;
            for (int i = 0; i + 1 < n; i++) {
                ans += Math.min(a[i][0], b[i + 1][1]);
                ans += Math.min(a[i][1], b[i + 1][0]);
            }
            return (int) ans;
        }
    }
}

// ============================================================
// FILE 630: _697.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 697. Degree of an Array
 *
 * Given a non-empty array of non-negative integers nums,
 * the degree of this array is defined as the maximum frequency of any one of its
elements.
 * Your task is to find the smallest possible length of a (contiguous) subarray of nums,
that has the same degree as nums.

 Example 1:

 Input: [1, 2, 2, 3, 1]
 Output: 2
 Explanation:
 The input array has a degree of 2 because both elements 1 and 2 appear twice.
 Of the subarrays that have the same degree:
 [1, 2, 2, 3, 1], [1, 2, 2, 3], [2, 2, 3, 1], [1, 2, 2], [2, 2, 3], [2, 2]
```

```
 The shortest length is 2. So return 2.

 Example 2:
 Input: [1,2,2,3,1,4,2]
 Output: 6

 Note:
 nums.length will be between 1 and 50,000.
 nums[i] will be an integer between 0 and 49,999.
 */
public class _697 {
    public static class Solution1 {
        public int findShortestSubArray(int[] nums) {
            Map<Integer, Integer> map = new HashMap<>();
            for (int i = 0; i < nums.length; i++) {
                if (map.containsKey(nums[i])) {
                    map.put(nums[i], map.get(nums[i]) + 1);
                } else {
                    map.put(nums[i], 1);
                }
            }
            int degree = -1;
            for (int key : map.keySet()) {
                degree = Math.max(degree, map.get(key));
            }
            List<Integer> candidateNums = new ArrayList();
            for (int key : map.keySet()) {
                if (map.get(key) == degree) {
                    candidateNums.add(key);
                }
            }
            int shortest = Integer.MAX_VALUE;
            for (int candidate : candidateNums) {
                shortest = Math.min(shortest, findLength(nums, candidate));
            }
            return shortest;
        }

        int findLength(int[] arr, int candidate) {
            int firstAppearance = Integer.MAX_VALUE;
            for (int i = 0; i < arr.length; i++) {
                if (arr[i] == candidate) {
                    firstAppearance = i;
                    break;
                }
            }
            int lastAppearance = arr.length - 1;
            for (int i = arr.length - 1; i > firstAppearance; i--) {
                if (arr[i] == candidate) {
                    lastAppearance = i;
                    break;
                }
            }
            return (lastAppearance - firstAppearance) + 1;
```

```
        }
    }

    public static class Solution2 {
        public int findShortestSubArray(int[] nums) {
            Map<Integer, Integer> count = new HashMap<>();
            Map<Integer, Integer> left = new HashMap<>();
            Map<Integer, Integer> right = new HashMap<>();

            for (int i = 0; i < nums.length; i++) {
                count.put(nums[i], count.getOrDefault(nums[i], 0) + 1);
                if (!left.containsKey(nums[i])) {
                    left.put(nums[i], i);
                }
                right.put(nums[i], i);
            }

            int result = nums.length;
            int degree = Collections.max(count.values());
            for (int num : count.keySet()) {
                if (count.get(num) == degree) {
                    result = Math.min(result, right.get(num) - left.get(num) + 1);
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 631: _698.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 698. Partition to K Equal Sum Subsets
 *
 * Given an array of integers nums and a positive integer k,
 * find whether it's possible to divide this array into k non-empty subsets whose sums
are all equal.

 Example 1:

 Input: nums = [4, 3, 2, 3, 5, 2, 1], k = 4
 Output: True
 Explanation: It's possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with
equal sums.

 Note:
 1 <= k <= len(nums) <= 16.
 0 < nums[i] < 10000.
 */
public class _698 {
```

```
    public static class Solution1 {
        public boolean canPartitionKSubsets(int[] nums, int k) {
            long sum = 0;
            for (int num : nums) {
                sum += num;
            }
            if (sum % k != 0) {
                return false;
            }
            int equalSum = (int) (sum / k);
            boolean[] visited = new boolean[nums.length];
            return canPartition(nums, visited, 0, k, 0, 0, equalSum);
        }

        private boolean canPartition(int[] nums, boolean[] visited, int startIndex, int
k, int currSum, int currNum, int target) {
            if (k == 1) {
                return true;
            }
            if (currSum == target && currNum > 0) {
                /**Everytime when we get currSum == target, we'll start from index 0 and
look up the numbers that are not used yet
                 * and try to find another sum that could equal to target*/
                return canPartition(nums, visited, 0, k - 1, 0, 0, target);
            }
            for (int i = startIndex; i < nums.length; i++) {
                if (!visited[i]) {
                    visited[i] = true;
                    if (canPartition(nums, visited, i + 1, k, currSum + nums[i],
currNum++, target)) {
                        return true;
                    }
                    visited[i] = false;
                }
            }
            return false;
        }
    }

}

// ==========================================================
// FILE 632: _699.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 699. Falling Squares
 *
```

```
 * On an infinite number line (x-axis), we drop given squares in the order they are
given.
 * The i-th square dropped (positions[i] = (left, side_length)) is a
 * square with the left-most point being positions[i][0] and sidelength positions[i][1].
 * The square is dropped with the bottom edge parallel to the number line, and
 * from a higher height than all currently landed squares. We wait for each square to
stick before dropping the next.
 * The squares are infinitely sticky on their bottom edge, and will
 * remain fixed to any positive length surface they touch (either the number line or
another square).
 * Squares dropped adjacent to each other will not stick together prematurely.
 * Return a list ans of heights.
 * Each height ans[i] represents the current highest height of any square we have
dropped,
 * after dropping squares represented by positions[0], positions[1], ..., positions[i].

 Example 1:

 Input: [[1, 2], [2, 3], [6, 1]]
 Output: [2, 5, 5]
 Explanation:


 After the first drop of positions[0] = [1, 2]:
 _aa
 _aa
 -------
 The maximum height of any square is 2.


 After the second drop of positions[1] = [2, 3]:
 __aaa
 __aaa
 __aaa
 _aa__
 _aa__
 --------------
 The maximum height of any square is 5.
 The larger square stays on top of the smaller square despite where its center
 of gravity is, because squares are infinitely sticky on their bottom edge.


 After the third drop of positions[1] = [6, 1]:
 __aaa
 __aaa
 __aaa
 _aa
 _aa___a
 --------------
 The maximum height of any square is still 5.

 Thus, we return an answer of [2, 5, 5].
```

```
 Example 2:

 Input: [[100, 100], [200, 100]]
 Output: [100, 100]
 Explanation: Adjacent squares don't get stuck prematurely - only their bottom edge can
stick to surfaces.

 Note:
 1 <= positions.length <= 1000.
 1 <= positions[0] <= 10^8.
 1 <= positions[1] <= 10^6.
 */

public class _699 {
    public static class Solution1 {
        /**credit: https://discuss.leetcode.com/topic/107107/easy-understood-o-n-2-
solution-with-explanation*/
        public List<Integer> fallingSquares(int[][] positions) {
            List<Interval> intervals = new ArrayList<>();
            List<Integer> result = new ArrayList<>();
            int height = 0;
            for (int[] position : positions) {
                Interval curr = new Interval(position[0], position[0] + position[1] - 1,
position[1]);
                height = Math.max(height, getHeight(intervals, curr));
                result.add(height);
            }
            return result;
        }

        private int getHeight(List<Interval> intervals, Interval curr) {
            int preMaxHeight = 0;
            for (Interval interval : intervals) {
                if (interval.end < curr.start || interval.start > curr.end) {
                    continue;
                }
                preMaxHeight = Math.max(preMaxHeight, interval.height);
            }
            curr.height += preMaxHeight;
            intervals.add(curr);
            return curr.height;
        }

        class Interval {
            int start;
            int end;
            int height;

            public Interval(int start, int end, int height) {
                this.start = start;
                this.end = end;
                this.height = height;
            }
        }
```

```
    }
}

// ============================================================
// FILE 633: _700.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 700. Search in a Binary Search Tree
 *
 * Given the root node of a binary search tree (BST) and a value.
 * You need to find the node in the BST that the node's value equals the given value.
 * Return the subtree rooted with that node. If such node doesn't exist, you should
return NULL.
 *
 * For example,
 *
 * Given the tree:
 *         4
 *        / \
 *       2   7
 *      / \
 *     1   3
 *
 * And the value to search: 2
 * You should return this subtree:
 *
 *       2
 *      / \
 *     1   3
 * In the example above, if we want to search the value 5, since there is no node with
value 5, we should return NULL.
 *
 * Note that an empty tree is represented by NULL,
 * therefore you would see the expected output (serialized tree format) as [], not null.
 * */
public class _700 {
    public static class Solution1 {
        public TreeNode searchBST(TreeNode root, int val) {
            if (root == null) {
                return null;
            } else if (root.val == val) {
                return root;
            } else if (root.val > val) {
                return searchBST(root.left, val);
            } else {
                return searchBST(root.right, val);
            }
        }
    }
```

```
}

// ============================================================
// FILE 634: _701.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 701. Insert into a Binary Search Tree
 *
 * Given the root node of a binary search tree (BST) and a value to be inserted into the
tree, insert the value into the BST. Return the root node of the BST after the insertion.
It is guaranteed that the new value does not exist in the original BST.
 * Note that there may exist multiple valid ways for the insertion, as long as the tree
remains a BST after insertion. You can return any of them.
 *
 * For example,
 *
 * Given the tree:
 *         4
 *        / \
 *       2   7
 *      / \
 *     1   3
 * And the value to insert: 5
 *
 *  You can return this binary search tree:
 *
 *          4
 *        /   \
 *       2     7
 *      / \   /
 *     1   3 5
 *
 * This tree is also valid:
 *
 *          5
 *        /   \
 *       2     7
 *      / \
 *     1   3
 *          \
 *           4
 */
public class _701 {
  public static class Solution1 {
    public TreeNode insertIntoBST(TreeNode root, int val) {
      if (root == null) {
        return new TreeNode(val);
      }
      if (root.val < val) {
```

```
        root.right = insertIntoBST(root.right, val);
      } else {
        root.left = insertIntoBST(root.left, val);
      }
      return root;
    }
  }
}

// ============================================================
// FILE 635: _703.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.PriorityQueue;

/**
 * 703. Kth Largest Element in a Stream
 *
 * Design a class to find the kth largest element in a stream. Note that it is the kth
largest element in the sorted order, not the kth distinct element.
 * Your KthLargest class will have a constructor which accepts an integer K and an
integer array nums,
 * which contains initial elements from the stream.
 * For each call to the method KthLargest.add, return the element representing the kth
largest element in the stream.
 *
 * Example:
 *
 * int K = 3;
 * int[] arr = [4,5,8,2];
 * KthLargest kthLargest = new KthLargest(3, arr);
 * kthLargest.add(3);   // returns 4
 * kthLargest.add(5);   // returns 5
 * kthLargest.add(10);  // returns 5
 * kthLargest.add(9);   // returns 8
 * kthLargest.add(4);   // returns 8
 * Note:
 * You may assume that nums' length ≥ K-1 and K ≥ 1.
 */
public class _703 {
  public static class Solution1 {
    public static class KthLargest {
      PriorityQueue<Integer> heap;
      int maxK;

      public KthLargest(int k, int[] nums) {
        heap = new PriorityQueue<>(Collections.reverseOrder());
        for (int num : nums) {
          heap.offer(num);
```

```
        }
        maxK = k;
      }

      public int add(int val) {
        List<Integer> tmp = new ArrayList<>();
        int result = 0;
        int tmpK = maxK;
        heap.offer(val);
        while (tmpK-- > 0) {
          result = heap.poll();
          tmp.add(result);
        }
        for (int num : tmp) {
          heap.offer(num);
        }
        return result;
      }
    }
  }
}

// ============================================================
// FILE 636: _704.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 704. Binary Search
 *
 * Given a sorted (in ascending order) integer array nums of n elements and a target
value, write a function to search target in nums.
 * If target exists, then return its index, otherwise return -1.
 *
 * Example 1:
 *
 * Input: nums = [-1,0,3,5,9,12], target = 9
 * Output: 4
 * Explanation: 9 exists in nums and its index is 4
 *
 * Example 2:
 *
 * Input: nums = [-1,0,3,5,9,12], target = 2
 * Output: -1
 * Explanation: 2 does not exist in nums so return -1
 *
 * Note:
 *
 * You may assume that all elements in nums are unique.
 * n will be in the range [1, 10000].
 * The value of each element in nums will be in the range [-9999, 9999].
 */
public class _704 {
```

```
  public static class Solution1 {
    public int search(int[] nums, int target) {
      int left = 0;
      int right = nums.length - 1;
      if (target < nums[left] || target > nums[right]) {
        return -1;
      }
      if (nums[left] == target) {
        return left;
      } else if (nums[right] == target) {
        return right;
      }
      while (left <= right) {
        int mid = left + (right - left) / 2;
        if (target == nums[mid]) {
          return mid;
        } else if (target > nums[mid]) {
          left = mid + 1;
        } else {
          right = mid - 1;
        }
      }
      return -1;
    }
  }
}

// ============================================================
// FILE 637: _705.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 705. Design HashSet
 *
 * Design a HashSet without using any built-in hash table libraries.
 *
 * To be specific, your design should include these functions:
 *
 * add(value): Insert a value into the HashSet.
 * contains(value) : Return whether the value exists in the HashSet or not.
 * remove(value): Remove a value in the HashSet. If the value does not exist in the
HashSet, do nothing.
 *
 * Example:
 *
 * MyHashSet hashSet = new MyHashSet();
 * hashSet.add(1);
 * hashSet.add(2);
 * hashSet.contains(1);    // returns true
```

```
 * hashSet.contains(3);    // returns false (not found)
 * hashSet.add(2);
 * hashSet.contains(2);    // returns true
 * hashSet.remove(2);
 * hashSet.contains(2);    // returns false (already removed)
 *
 * Note:
 *
 * All values will be in the range of [0, 1000000].
 * The number of operations will be in the range of [1, 10000].
 * Please do not use the built-in HashSet library.
 */
public class _705 {
  public static class Solution1 {
    class MyHashSet {
      Map<Integer, Integer> map;

      /** Initialize your data structure here. */
      public MyHashSet() {
        map = new HashMap<>();
      }

      public void add(int key) {
        map.put(key, 0);
      }

      public void remove(int key) {
        if (map.containsKey(key)) {
          map.remove(key);
        }
      }

      /** Returns true if this set contains the specified element */
      public boolean contains(int key) {
        return map.containsKey(key);
      }
    }
  }
}

// ============================================================
// FILE 638: _706.java
// ============================================================

package com.fishercoder.solutions;

/**706. Design HashMap
 *
 * Design a HashMap without using any built-in hash table libraries.
 *
 * To be specific, your design should include these functions:
 *
 * put(key, value) : Insert a (key, value) pair into the HashMap. If the value already
exists in the HashMap, update the value.
```

```
 * get(key): Returns the value to which the specified key is mapped, or -1 if this map
contains no mapping for the key.
 * remove(key) : Remove the mapping for the value key if this map contains the mapping
for the key.
 *
 * Example:
 *
 * MyHashMap hashMap = new MyHashMap();
 * hashMap.put(1, 1);
 * hashMap.put(2, 2);
 * hashMap.get(1);            // returns 1
 * hashMap.get(3);            // returns -1 (not found)
 * hashMap.put(2, 1);          // update the existing value
 * hashMap.get(2);            // returns 1
 * hashMap.remove(2);          // remove the mapping for 2
 * hashMap.get(2);            // returns -1 (not found)
 *
 * Note:
 *
 * All keys and values will be in the range of [0, 1000000].
 * The number of operations will be in the range of [1, 10000].
 * Please do not use the built-in HashMap library.*/
public class _706 {
    public static class Solution1 {
        /**
         * credit: https://leetcode.com/problems/design-hashmap/discuss/152746/Java-
Solution
         */
        class MyHashMap {

            final ListNode[] nodes = new ListNode[10000];

            public void put(int key, int value) {
                int i = idx(key);
                if (nodes[i] == null) {
                    nodes[i] = new ListNode(-1, -1);
                }
                ListNode prev = find(nodes[i], key);
                if (prev.next == null) {
                    prev.next = new ListNode(key, value);
                } else {
                    prev.next.val = value;
                }
            }

            public int get(int key) {
                int i = idx(key);
                if (nodes[i] == null) {
                    return -1;
                }
                ListNode node = find(nodes[i], key);
                return node.next == null ? -1 : node.next.val;
            }
```

```
            public void remove(int key) {
                int i = idx(key);
                if (nodes[i] == null) {
                    return;
                }
                ListNode prev = find(nodes[i], key);
                if (prev.next == null) {
                    return;
                }
                prev.next = prev.next.next;
            }

            int idx(int key) {
                return Integer.hashCode(key) % nodes.length;
            }

            ListNode find(ListNode bucket, int key) {
                ListNode node = bucket;
                ListNode prev = null;
                while (node != null && node.key != key) {
                    prev = node;
                    node = node.next;
                }
                return prev;
            }

            class ListNode {
                int key;
                int val;
                ListNode next;

                ListNode(int key, int val) {
                    this.key = key;
                    this.val = val;
                }
            }
        }

/**
 * Your MyHashMap object will be instantiated and called as such:
 * MyHashMap obj = new MyHashMap();
 * obj.put(key,value);
 * int param_2 = obj.get(key);
 * obj.remove(key);
 */
    }
}

// ============================================================
// FILE 639: _709.java
// ============================================================

package com.fishercoder.solutions;
```

```
/**709. To Lower Case
 *
 * Implement function ToLowerCase() that has a string parameter str, and returns the same
string in lowercase.
 *
 *  Example 1:
 *
 * Input: "Hello"
 * Output: "hello"
 * Example 2:
 *
 * Input: "here"
 * Output: "here"
 * Example 3:
 *
 * Input: "LOVELY"
 * Output: "lovely"*/
public class _709 {
    public static class Solution1 {
        public String toLowerCase(String str) {
            return str.toLowerCase();
        }
    }
}

// ============================================================
// FILE 640: _712.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 712. Minimum ASCII Delete Sum for Two Strings
 *
 * Given two strings s1, s2, find the lowest ASCII sum of deleted characters to make two
strings equal.

 Example 1:
 Input: s1 = "sea", s2 = "eat"
 Output: 231
 Explanation: Deleting "s" from "sea" adds the ASCII value of "s" (115) to the sum.
 Deleting "t" from "eat" adds 116 to the sum.
 At the end, both strings are equal, and 115 + 116 = 231 is the minimum sum possible to
achieve this.

 Example 2:
 Input: s1 = "delete", s2 = "leet"
 Output: 403
 Explanation: Deleting "dee" from "delete" to turn the string into "let",
 adds 100[d]+101[e]+101[e] to the sum.  Deleting "e" from "leet" adds 101[e] to the sum.
 At the end, both strings are equal to "let", and the answer is 100+101+101+101 = 403.
 If instead we turned both strings into "lee" or "eet", we would get answers of 433 or
417, which are higher.
```

```
 Note:
 0 < s1.length, s2.length <= 1000.
 All elements of each string will have an ASCII value in [97, 122].
 */
public class _712 {
    public static class Solution1 {
        //credit: https://leetcode.com/articles/minimum-ascii-delete-sum-for-two-strings/
        public int minimumDeleteSum(String s1, String s2) {
            int[][] dp = new int[s1.length() + 1][s2.length() + 1];

            for (int i = s1.length() - 1; i >= 0; i--) {
                dp[i][s2.length()] = dp[i + 1][s2.length()] + s1.codePointAt(i);
            }

            for (int j = s2.length() - 1; j >= 0; j--) {
                dp[s1.length()][j] = dp[s1.length()][j + 1] + s2.codePointAt(j);
            }

            for (int i = s1.length() - 1; i >= 0; i--) {
                for (int j = s2.length() - 1; j >= 0; j--) {
                    if (s1.charAt(i) == s2.charAt(j)) {
                        dp[i][j] = dp[i + 1][j + 1];
                    } else {
                        dp[i][j] = Math.min(dp[i + 1][j] + s1.codePointAt(i), dp[i][j +
1] + s2.codePointAt(j));
                    }
                }
            }

            return dp[0][0];
        }
    }
}

// ============================================================
// FILE 641: _713.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 713. Subarray Product Less Than K
 *
 * Your are given an array of positive integers nums.
 * Count and print the number of (contiguous) subarrays where the product of all the
elements in the subarray is less than k.

 Example 1:
 Input: nums = [10, 5, 2, 6], k = 100
 Output: 8
 Explanation: The 8 subarrays that have product less than 100 are: [10], [5], [2], [6],
[10, 5], [5, 2], [2, 6], [5, 2, 6].
 Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.
 Note:
```

```
 0 < nums.length <= 50000.
 0 < nums[i] < 1000.
 0 <= k < 10^6.

 */
public class _713 {
    public static class Solution1 {
        /**O(n^2) solution, accepted initially, then Leetcode added one test case to fail
it.*/
        public int numSubarrayProductLessThanK(int[] nums, int k) {
            int result = 0;
            for (int i = 0; i < nums.length; i++) {
                int product = nums[i];
                if (product < k) {
                    result++;
                    for (int j = i + 1; j < nums.length; j++) {
                        product *= nums[j];
                        if (product < k) {
                            result++;
                        } else {
                            break;
                        }
                    }
                }
            }
            return result;
        }
    }

    public static class Solution2 {
        public int numSubarrayProductLessThanK(int[] nums, int k) {
            if (k < 2) {
                return 0;
            }
            int result = 0;
            int product = 1;
            for (int i = 0, right = 0; right < nums.length; right++) {
                product *= nums[right];
                while (i < nums.length && product >= k) {
                    product /= nums[i++];
                }
                result += right - i + 1;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 642: _714.java
// ============================================================

package com.fishercoder.solutions;
```

```
/**
 * 714. Best Time to Buy and Sell Stock with Transaction Fee
 *
 * Your are given an array of integers prices, for which the i-th element is the price of
a given stock on day i;
 * and a non-negative integer fee representing a transaction fee.
 * You may complete as many transactions as you like,
 * but you need to pay the transaction fee for each transaction.
 * You may not buy more than 1 share of a stock at a time (ie. you must sell the stock
share before you buy again.)

 Return the maximum profit you can make.

 Example 1:
 Input: prices = [1, 3, 2, 8, 4, 9], fee = 2
 Output: 8
 Explanation: The maximum profit can be achieved by:
 Buying at prices[0] = 1
 Selling at prices[3] = 8
 Buying at prices[4] = 4
 Selling at prices[5] = 9
 The total profit is ((8 - 1) - 2) + ((9 - 4) - 2) = 8.

 Note:
 0 < prices.length <= 50000.
 0 < prices[i] < 50000.
 0 <= fee < 50000.
 */
public class _714 {
    public static class Solution1 {
        /**O(n) time
         * O(n) space
         * credit: https://discuss.leetcode.com/topic/108009/java-c-clean-code-dp-greedy
         * */
        public int maxProfit(int[] prices, int fee) {
            int n = prices.length;
            if (n < 2) {
                return 0;
            }
            int[] hold = new int[n];
            int[] sell = new int[n];
            hold[0] = -prices[0];
            for (int i = 1; i < prices.length; i++) {
                hold[i] = Math.max(hold[i - 1], sell[i - 1] - prices[i]);
                sell[i] = Math.max(sell[i - 1], hold[i - 1] - fee + prices[i]);
            }
            return sell[n - 1];
        }
    }

    public static class Solution2 {
        /**O(n) time
         * O(1) space
```

```
         * credit: https://leetcode.com/articles/best-time-to-buy-and-sell-stock-with-transaction-fee/
         *
         * cash: the max profit we could have if we did not have a share of stock in hand
         * hold: the max profit we could have if we hold one share of stack in hand
         *
         * to transition from the i-th day to the i+1 th day, we have two options:
         * 1. sell our stock: cash = Math.max(cash, hold + prices[i] - fee)
         * 2. buy a stock: hold = Math.max(hold, cash - prices[i])
         * */
        public int maxProfit(int[] prices, int fee) {
            int cash = 0;
            int hold = -prices[0];
            for (int i = 1; i < prices.length; i++) {
                cash = Math.max(cash, hold + prices[i] - fee);
                hold = Math.max(hold, cash - prices[i]);
            }
            return cash;
        }
    }
}

// ============================================================
// FILE 643: _716.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Stack;
import java.util.TreeMap;

/**
 * 716. Max Stack
 *
 * Design a max stack that supports push, pop, top, peekMax and popMax.
 *
 push(x) -- Push element x onto stack.
 pop() -- Remove the element on top of the stack and return it.
 top() -- Get the element on the top.
 peekMax() -- Retrieve the maximum element in the stack.
 popMax() -- Retrieve the maximum element in the stack, and remove it. If you find more than one maximum elements, only remove the top-most one.

 Example 1:
 MaxStack stack = new MaxStack();
 stack.push(5);
 stack.push(1);
 stack.push(5);
 stack.top(); -> 5
 stack.popMax(); -> 5
 stack.top(); -> 1
```

```
 stack.peekMax(); -> 5
 stack.pop(); -> 1
 stack.top(); -> 5
 Note:
 -1e7 <= x <= 1e7
 Number of operations won't exceed 10000.
 The last four operations won't be called when stack is empty.
 */
public class _716 {
    public static class Solution1 {
      /**This is O(n) for popMax() and pop() while O(1) for the other three operations
which is UN-acceptable during an interview!
       * We need to do better than O(n) time complexity in order to ace the interview!
       * But O(1) is impossible, so let's aim for O(logn).*/
        public static class MaxStack {

            private int max;
            private Stack<Integer> stack;

            /**
             * initialize your data structure here.
             */
            public MaxStack() {
                max = Integer.MIN_VALUE;
                stack = new Stack<>();
            }

            public void push(int x) {
                if (x > max) {
                    max = x;
                }
                stack.push(x);
            }

            public int pop() {
                if (stack.peek() == max) {
                    int result = stack.pop();
                    max = findMax();
                    return result;
                } else {
                    return stack.pop();
                }
            }

            private int findMax() {
                if (!stack.isEmpty()) {
                    Iterator<Integer> iterator = stack.iterator();
                    int max = stack.peek();
                    while (iterator.hasNext()) {
                        max = Math.max(max, iterator.next());
                    }
                    return max;
                } else {
                    max = Integer.MIN_VALUE;
```

```
                return max;
            }
        }

        public int top() {
            return stack.peek();
        }

        public int peekMax() {
            return max;
        }

        public int popMax() {
            Stack<Integer> tmp = new Stack<>();
            int result = 0;
            while (!stack.isEmpty()) {
                if (stack.peek() != max) {
                    tmp.push(stack.pop());
                } else {
                    result = stack.pop();
                    break;
                }
            }
            while (!tmp.isEmpty()) {
                stack.push(tmp.pop());
            }
            max = findMax();
            return result;
        }
    }
}

public static class Solution2 {
  /** Use a treemap and a doubly linked list to achieve O(logn) time complexity. */

  static class Node {
    int val;
    Node prev;
    Node next;

    public Node(int val) {
      this.val = val;
    }
  }

  static class DoublyLinkedList {
    Node head;
    Node tail;

    public DoublyLinkedList() {
      head = new Node(0);
      tail = new Node(0);
      head.next = tail;
      tail.prev = head;
```

```
      }

      public Node add(int val) {
        /**For this doubly linked list, we always add it to the end of the list*/
        Node x = new Node(val);
        x.next = tail;
        x.prev = tail.prev;
        tail.prev.next = x;
        tail.prev = tail.prev.next;
        return x;
      }

      public int pop() {
        /**for pop(), we always pop one from the tail of the doubly linked list*/
        return unlink(tail.prev).val;
      }

      public Node unlink(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
        return node;
      }

      public int peek() {
        return tail.prev.val;
      }
    }

    public static class MaxStack {
      TreeMap<Integer, List<Node>> treeMap;
      /**
       * the reason we have a list of nodes as treemap's value is because one value could
be pushed
       * multiple times into this MaxStack and we want to keep track of all of them.
       */
      DoublyLinkedList doublyLinkedList;

      /** initialize your data structure here. */
      public MaxStack() {
        treeMap = new TreeMap();
        doublyLinkedList = new DoublyLinkedList();
      }

      public void push(int x) {
        Node node = doublyLinkedList.add(x);
        if (!treeMap.containsKey(x)) {
          treeMap.put(x, new ArrayList<>());
        }
        treeMap.get(x).add(node);
      }

      public int pop() {
        int val = doublyLinkedList.pop();
        List<Node> nodes = treeMap.get(val);
```

```
        nodes.remove(nodes.size() - 1);
        if (nodes.isEmpty()) {
          treeMap.remove(val);
        }
        return val;
      }

      public int top() {
        return doublyLinkedList.peek();
      }

      public int peekMax() {
        return treeMap.lastKey();
      }

      public int popMax() {
        int max = treeMap.lastKey();
        List<Node> nodes = treeMap.get(max);
        Node node = nodes.remove(nodes.size() - 1);
        doublyLinkedList.unlink(node);
        if (nodes.isEmpty()) {
          treeMap.remove(max);
        }
        return max;
      }
    }
  }
}

// ============================================================
// FILE 644: _717.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 717. 1-bit and 2-bit Characters
 *
 * We have two special characters. The first character can be represented by one bit 0.
The second character can be represented by two bits (10 or 11).
 * Now given a string represented by several bits. Return whether the last character must
be a one-bit character or not. The given string will always end with a zero.

 Example 1:
 Input:
 bits = [1, 0, 0]
 Output: True
 Explanation:
 The only way to decode it is two-bit character and one-bit character. So the last
character is one-bit character.

 Example 2:
 Input:
 bits = [1, 1, 1, 0]
```

```
 Output: False
 Explanation:
 The only way to decode it is two-bit character and two-bit character. So the last
character is NOT one-bit character.

 Note:
 1 <= len(bits) <= 1000.
 bits[i] is always 0 or 1.
 */
public class _717 {
    public static class Solution1 {
        public boolean isOneBitCharacter(int[] bits) {
            int n = bits.length;
            int i = 0;
            while (i < n - 1) {
                if (bits[i] == 0) {
                    i++;
                } else {
                    i += 2;
                }
            }
            return i == n - 1 ? true : false;
        }
    }
}

// ============================================================
// FILE 645: _718.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 718. Maximum Length of Repeated Subarray
 *
 * Given two integer arrays A and B, return the maximum length of an subarray that
appears in both arrays.

 Example 1:
 Input:
 A: [1,2,3,2,1]
 B: [3,2,1,4,7]
 Output: 3

 Explanation:
 The repeated subarray with maximum length is [3, 2, 1].
 Note:
 1 <= len(A), len(B) <= 1000
 0 <= A[i], B[i] < 100
 */
public class _718 {
    public static class Solution1 {
        public int findLength(int[] A, int[] B) {
            if (A == null || B == null || A.length == 0 || B.length == 0) {
```

```
                return 0;
            }
            int[][] dp = new int[A.length + 1][B.length + 1];
            int result = 0;
            for (int i = A.length - 1; i >= 0; i--) {
                for (int j = B.length - 1; j >= 0; j--) {
                    if (A[i] == B[j]) {
                        dp[i][j] = dp[i + 1][j + 1] + 1;
                    }
                    result = Math.max(result, dp[i][j]);
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 646: _719.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * 719. Find K-th Smallest Pair Distance
 *
 * Given an integer array, return the k-th smallest distance among all the pairs.
 * The distance of a pair (A, B) is defined as the absolute difference between A and B.

 Example 1:
 Input:
 nums = [1,3,1]
 k = 1
 Output: 0
 Explanation:
 Here are all the pairs:
 (1,3) -> 2
 (1,1) -> 0
 (3,1) -> 2
 Then the 1st smallest distance pair is (1,1), and its distance is 0.

 Note:
 2 <= len(nums) <= 10000.
 0 <= nums[i] < 1000000.
 1 <= k <= len(nums) * (len(nums) - 1) / 2.
 */

public class _719 {
    public static class Solution1 {
        /**
         * This brute force solution results in TLE of course.
```

```
         */
        public int smallestDistancePair(int[] nums, int k) {
            if (nums == null || nums.length == 0) {
                return 0;
            }
            PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> a - b);
            for (int i = 0; i < nums.length - 1; i++) {
                for (int j = i + 1; j < nums.length; j++) {
                    minHeap.offer(Math.abs(nums[j] - nums[i]));
                }
            }

            int result = 0;
            while (k-- > 0) {
                result = minHeap.poll();
            }
            return result;
        }
    }

    public static class Solution2 {
        /**
         * credit: https://leetcode.com/articles/find-k-th-smallest-pair-
distance/#approach-3-binary-search-sliding-window-accepted
         */
        public int smallestDistancePair(int[] nums, int k) {
            Arrays.sort(nums);
            int lo = 0;
            int hi = nums[nums.length - 1] - nums[0];
            while (lo < hi) {
                int mi = (lo + hi) / 2;
                int count = 0;
                int left = 0;
                for (int right = 0; right < nums.length; ++right) {
                    while (nums[right] - nums[left] > mi) {
                        left++;
                    }
                    count += right - left;
                }
                //count = number of pairs with distance <= mi
                if (count >= k) {
                    hi = mi;
                } else {
                    lo = mi + 1;
                }
            }
            return lo;
        }
    }
}

// ============================================================
// FILE 647: _720.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 720. Longest Word in Dictionary.
 *
 * Given a list of strings words representing an English Dictionary,
 * find the longest word in words that can be built one character at a time by other
words in words.
 * If there is more than one possible answer, return the longest word with the smallest
lexicographical order.
 * If there is no answer, return the empty string.

 Example 1:
 Input:
 words = ["w","wo","wor","worl", "world"]
 Output: "world"
 Explanation:
 The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

 Example 2:
 Input:
 words = ["a", "banana", "app", "appl", "ap", "apply", "apple"]
 Output: "apple"
 Explanation:
 Both "apply" and "apple" can be built from other words in the dictionary. However,
"apple" is lexicographically smaller than "apply".

 Note:
 All the strings in the input will only contain lowercase letters.
 The length of words will be in the range [1, 1000].
 The length of words[i] will be in the range [1, 30].
 */

public class _720 {
    public static class Solution1 {
        public String longestWord(String[] words) {
            TrieNode root = buildTrie(words);
            return findLongestWord(root, words);
        }

        private String findLongestWord(TrieNode root, String[] words) {
            String longestWord = "";
            for (String word : words) {
                if (longestWord.length() > word.length() || (longestWord.length() ==
word.length() && (longestWord.compareToIgnoreCase(word) < 0))) {
                    continue;
                }
                TrieNode tmp = root;
                boolean validWord = true;
                for (char c : word.toCharArray()) {
                    if (tmp.children[c - 'a'] != null) {
                        tmp = tmp.children[c - 'a'];
                        if (!tmp.isWord) {
```

```
                            validWord = false;
                            break;
                        }
                    }
                }
                if (validWord) {
                    longestWord = word;
                }
            }
            return longestWord;
        }

        private TrieNode buildTrie(String[] words) {
            TrieNode root = new TrieNode(' ');
            for (String word : words) {
                TrieNode tmp = root;
                for (char c : word.toCharArray()) {
                    if (tmp.children[c - 'a'] == null) {
                        tmp.children[c - 'a'] = new TrieNode(c);
                    }
                    tmp = tmp.children[c - 'a'];
                }
                tmp.isWord = true;
            }
            return root;
        }

        class TrieNode {
            char val;
            boolean isWord;
            TrieNode[] children;

            public TrieNode(char val) {
                this.val = val;
                this.isWord = false;
                this.children = new TrieNode[26];
            }
        }
    }
}

// ============================================================
// FILE 648: _721.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
```

```
import java.util.Stack;

/**
 * 721. Accounts Merge
 *
 * Given a list accounts, each element accounts[i] is a list of strings, where the first
element accounts[i][0] is a name, and the rest of the elements are emails representing
emails of the account.
 * Now, we would like to merge these accounts.
 * Two accounts definitely belong to the same person if there is some email that is
common to both accounts.
 * Note that even if two accounts have the same name, they may belong to different people
as people could have the same name.
 * A person can have any number of accounts initially, but all of their accounts
definitely have the same name.
 * After merging the accounts, return the accounts in the following format:
 * the first element of each account is the name, and the rest of the elements are emails
in sorted order.
 * The accounts themselves can be returned in any order.

 Example 1:
 Input:
 accounts = [["John", "johnsmith@mail.com", "john00@mail.com"], ["John",
"johnnybravo@mail.com"], ["John", "johnsmith@mail.com", "john_newyork@mail.com"],
["Mary", "mary@mail.com"]]
 Output: [["John", 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com'],
["John", "johnnybravo@mail.com"], ["Mary", "mary@mail.com"]]

 Explanation:
 The first and third John's are the same person as they have the common email
"johnsmith@mail.com".
 The second John and Mary are different people as none of their email addresses are used
by other accounts.
 We could return these lists in any order, for example the answer [['Mary',
'mary@mail.com'], ['John', 'johnnybravo@mail.com'],
 ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still
be accepted.

 Note:
 The length of accounts will be in the range [1, 1000].
 The length of accounts[i] will be in the range [1, 10].
 The length of accounts[i][j] will be in the range [1, 30].
 */
public class _721 {

    public static class Solution1 {
        /**credit: https://leetcode.com/articles/accounts-merge/#approach-1-depth-first-
search-accepted
         *
         * Time Complexity: O(∑ai*logai) where ai is the length of accounts[i].
         * Without the log factor, this is the complexity to build the graph and search
for each component. The log factor is for sorting each component at the end.
         * Space Complexity: O(∑ai) the space used by the graph and search.
         * .*/
```

```
        public List<List<String>> accountsMerge(List<List<String>> accounts) {
            Map<String, String> emailToName = new HashMap();
            Map<String, ArrayList<String>> graph = new HashMap();
            for (List<String> account : accounts) {
                String name = "";
                for (String email : account) {
                    if (name == "") {
                        name = email;
                        continue;
                    }
                    graph.computeIfAbsent(email, x -> new ArrayList<>
()).add(account.get(1));
                    graph.computeIfAbsent(account.get(1), x -> new ArrayList<>
()).add(email);
                    emailToName.put(email, name);
                }
            }

            Set<String> seen = new HashSet();
            List<List<String>> ans = new ArrayList();
            for (String email : graph.keySet()) {
                if (!seen.contains(email)) {
                    seen.add(email);
                    Stack<String> stack = new Stack();
                    stack.push(email);
                    List<String> component = new ArrayList();
                    while (!stack.empty()) {
                        String node = stack.pop();
                        component.add(node);
                        for (String nei : graph.get(node)) {
                            if (!seen.contains(nei)) {
                                seen.add(nei);
                                stack.push(nei);
                            }
                        }
                    }
                    Collections.sort(component);
                    component.add(0, emailToName.get(email));
                    ans.add(component);
                }
            }
            return ans;
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/articles/accounts-merge/#approach-2-union-find-
accepted
         * DSU stands for Disjoint Set Union: https://en.wikipedia.org/wiki/Disjoint-
set_data_structure
         *
         * Time complexity: O(AlogA)
         * Space complexity: O(A)
         * */
```

```
    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        DSU dsu = new DSU();
        Map<String, String> emailToName = new HashMap<>();
        Map<String, Integer> emailToId = new HashMap<>();
        int id = 0;
        for (List<String> account : accounts) {
            String name = "";
            for (String email : account) {
                if (name.equals("")) {
                    name = email;
                    continue;
                }
                emailToName.put(email, name);
                if (!emailToId.containsKey(email)) {
                    emailToId.put(email, id++);
                }
                dsu.union(emailToId.get(account.get(1)), emailToId.get(email));
            }
        }

        Map<Integer, List<String>> ans = new HashMap<>();
        for (String email : emailToName.keySet()) {
            int index = dsu.find(emailToId.get(email));
            ans.computeIfAbsent(index, x -> new ArrayList()).add(email);
        }
        for (List<String> component : ans.values()) {
            Collections.sort(component);
            component.add(0, emailToName.get(component.get(0)));
        }
        return new ArrayList<>(ans.values());
    }

    class DSU {
        int[] parent;

        public DSU() {
            parent = new int[10001];
            for (int i = 0; i <= 10000; i++) {
                parent[i] = i;
            }
        }

        public int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }

        public void union(int x, int y) {
            parent[find(x)] = find(y);
        }
    }
}
```

```
}

// ============================================================
// FILE 649: _723.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 723. Candy Crush
 *
 * This question is about implementing a basic elimination algorithm for Candy Crush.
 * Given a 2D integer array board representing the grid of candy, different positive
integers board[i][j]
 * represent different types of candies.
 * A value of board[i][j] = 0 represents that the cell at position (i, j) is empty.
 * The given board represents the state of the game following the player's move.
 * Now, you need to restore the board to a stable state by crushing candies according to
the following rules:
 *
 * If three or more candies of the same type are adjacent vertically or horizontally,
 * "crush" them all at the same time - these positions become empty.
 * After crushing all candies simultaneously, if an empty space on the board has candies
on top of itself,
 * then these candies will drop until they hit a candy or bottom at the same time.
 * (No new candies will drop outside the top boundary.)
 * After the above steps, there may exist more candies that can be crushed.
 * If so, you need to repeat the above steps.
 * If there does not exist more candies that can be crushed (ie. the board is stable),
then return the current board.
 * You need to perform the above rules until the board becomes stable, then return the
current board.

 Example 1:

 Input:
 board =
 [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],
[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],
[4,1,4,4,1014]]

 Output:
 [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],
[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]
 Explanation:

 Note:
 The length of board will be in the range [3, 50].
 The length of board[i] will be in the range [3, 50].
 Each board[i][j] will initially start as an integer in the range [1, 2000].

 */
public class _723 {
    public static class Solution1 {
```

```
        /**credit: https://leetcode.com/articles/candy-crush/*/
        public int[][] candyCrush(int[][] board) {
            int row = board.length;
            int col = board[0].length;
            boolean todo = false;
            for (int i = 0; i < row; i++) {
                for (int j = 0; j < col - 2; j++) {
                    int v = Math.abs(board[i][j]);
                    if (v != 0 && v == Math.abs(board[i][j + 1]) && v ==
Math.abs(board[i][j + 2])) {
                        //mark all of them to be negative
                        board[i][j] = board[i][j + 1] = board[i][j + 2] = -v;
                        todo = true;
                    }
                }
            }

            for (int i = 0; i < row - 2; i++) {
                for (int j = 0; j < col; j++) {
                    int v = Math.abs(board[i][j]);
                    if (v != 0 && v == Math.abs(board[i + 1][j]) && v == Math.abs(board[i
+ 2][j])) {
                        board[i + 1][j] = board[i + 2][j] = board[i][j] = -v;
                        todo = true;
                    }
                }
            }

            for (int j = 0; j < col; j++) {
                int wr = row - 1;
                for (int i = row - 1; i >= 0; i--) {
                    if (board[i][j] > 0) {
                        board[wr--][j] = board[i][j];
                    }
                }
                while (wr >= 0) {
                    board[wr--][j] = 0;
                }
            }

            return todo ? candyCrush(board) : board;
        }
    }
}

// ============================================================
// FILE 650: _724.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 724. Find Pivot Index
 *
```

```
 * Given an array of integers nums, write a method that returns the "pivot" index of this
array.
 * We define the pivot index as the index where the sum of the numbers to the left of the
index is equal
 * to the sum of the numbers to the right of the index.
 * If no such index exists, we should return -1. If there are multiple pivot indexes, you
should return the left-most pivot index.

 Example 1:
 Input:
 nums = [1, 7, 3, 6, 5, 6]
 Output: 3
 Explanation:
 The sum of the numbers to the left of index 3 (nums[3] = 6) is equal to the sum of
numbers to the right of index 3.
 Also, 3 is the first index where this occurs.

 Example 2:
 Input:
 nums = [1, 2, 3]
 Output: -1
 Explanation:
 There is no index that satisfies the conditions in the problem statement.

 Note:
 The length of nums will be in the range [0, 10000].
 Each element nums[i] will be an integer in the range [-1000, 1000].
 */
public class _724 {
    public static class Solution1 {
        /**Space: O(n)
         * Time: O(n)*/
        public int pivotIndex(int[] nums) {
            if (nums == null || nums.length == 0) {
                return -1;
            }
            int[] sums = new int[nums.length];
            sums[0] = nums[0];
            for (int i = 1; i < nums.length; i++) {
                sums[i] = sums[i - 1] + nums[i];
            }
            for (int i = 0; i < nums.length; i++) {
                if (i == 0 && 0 == sums[nums.length - 1] - sums[i] || (i > 0 && sums[i -
1] == sums[nums.length - 1] - sums[i])) {
                    return i;
                }
            }
            return -1;
        }
    }

    public static class Solution2 {
        /**Space: O(1)
         * Time: O(n)*/
```

```
        public int pivotIndex(int[] nums) {
            int total = 0;
            for (int num : nums) {
                total += num;
            }
            int sum = 0;
            for (int i = 0; i < nums.length; sum += nums[i++]) {
                if (sum * 2 == total - nums[i]) {
                    return i;
                }
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 651: _725.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 725. Split Linked List in Parts
 *
 * Given a (singly) linked list with head node root,
 * write a function to split the linked list into k consecutive linked list "parts".
 * The length of each part should be as equal as possible:
 * no two parts should have a size differing by more than 1. This may lead to some parts
being null.
 * The parts should be in order of occurrence in the input list,
 * and parts occurring earlier should always have a size greater than or equal parts
occurring later.
 * Return a List of ListNode's representing the linked list parts that are formed.

 Examples 1->2->3->4, k = 5 // 5 equal parts [ [1], [2], [3], [4], null ]

 Example 1:
 Input:
 root = [1, 2, 3], k = 5
 Output: [[1],[2],[3],[],[]]
 Explanation:
 The input and each element of the output are ListNodes, not arrays.
 For example, the input root has root.val = 1, root.next.val = 2, \root.next.next.val =
3, and root.next.next.next = null.
 The first element output[0] has output[0].val = 1, output[0].next = null.
 The last element output[4] is null, but it's string representation as a ListNode is [].

 Example 2:
 Input:
 root = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], k = 3
 Output: [[1, 2, 3, 4], [5, 6, 7], [8, 9, 10]]
```

```
 Explanation:
 The input has been split into consecutive parts with size difference at most 1, and
earlier parts are a larger size than the later parts.

 Note:
 The length of root will be in the range [0, 1000].
 Each value of a node in the input will be an integer in the range [0, 999].
 k will be an integer in the range [1, 50].

 */
public class _725 {
    public static class Solution1 {
        /**My very original solution, but verbose.*/
        public ListNode[] splitListToParts(ListNode root, int k) {
            int len = getLength(root);
            int aveSize = len / k;
            int extra = len % k;
            ListNode[] result = new ListNode[k];
            for (int i = 0; i < k; i++) {
                result[i] = root;
                int aveSizeTmp = aveSize;
                aveSizeTmp += extra-- > 0 ? 1 : 0;
                int aveSizeTmp2 = aveSizeTmp;
                while (aveSizeTmp-- > 0) {
                    root = root.next;
                }
                if (result[i] != null) {
                    ListNode tmp = result[i];
                    while (tmp.next != null && aveSizeTmp2-- > 1) {
                        tmp = tmp.next;
                    }
                    tmp.next = null;
                }
            }
            return result;
        }

        private int getLength(ListNode root) {
            int len = 0;
            ListNode tmp = root;
            while (tmp != null) {
                len++;
                tmp = tmp.next;
            }
            return len;
        }
    }

    public static class Solution2 {
        /**More concise version*/
        public ListNode[] splitListToParts(ListNode root, int k) {
            int len = getLength(root);
            int aveSize = len / k;
            int extra = len % k;
```

```
            ListNode[] result = new ListNode[k];
            ListNode prev = null;
            for (int i = 0; i < k; i++, extra--) {
                result[i] = root;
                for (int j = 0; j < aveSize + (extra > 0 ? 1 : 0); j++) {
                    prev = root;
                    root = root.next;
                }
                if (prev != null) {
                    prev.next = null;
                }
            }
            return result;
        }

        private int getLength(ListNode root) {
            int len = 0;
            ListNode tmp = root;
            while (tmp != null) {
                len++;
                tmp = tmp.next;
            }
            return len;
        }
    }
}

// ============================================================
// FILE 652: _727.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 727. Minimum Window Subsequence
 *
 * Given strings S and T, find the minimum (contiguous) substring W of S, so that T is a
subsequence of W.
 * If there is no such window in S that covers all characters in T,
 * return the empty string "". If there are multiple such minimum-length windows, return
the one with the left-most starting index.

 Example 1:
 Input:
 S = "abcdebdde", T = "bde"
 Output: "bcde"

 Explanation:
 "bcde" is the answer because it occurs before "bdde" which has the same length.
 "deb" is not a smaller window because the elements of T in the window must occur in
order.
```

```
 Note:
 All the strings in the input will only contain lowercase letters.
 The length of S will be in the range [1, 20000].
 The length of T will be in the range [1, 100].
 */
public class _727 {
    public static class Solution1 {
        /**
         * This naive brute force results in TLE.
         */
        public String minWindow(String S, String T) {
            String result = S;
            for (int i = 0; i < S.length(); i++) {
                for (int j = i + T.length(); j <= S.length(); j++) {
                    String sub = S.substring(i, j);
                    if (sub.length() < result.length() && isSubsequence(T, sub)) {
                        result = sub;
                    }
                }
            }
            return result.equals(S) ? "" : result;
        }

        private boolean isSubsequence(String T, String sub) {
            int i = 0;
            for (int j = 0; i < T.length() && j < sub.length(); j++) {
                if (T.charAt(i) == sub.charAt(j)) {
                    i++;
                }
            }
            return i == T.length();
        }
    }

    public static class Solution2 {
        /**credit: https://github.com/lydxlx1/LeetCode/blob/master/src/_727.java*/
        public String minWindow(String S, String T) {
            int[][] dp = new int[S.length() + 1][T.length() + 1];
            int INFINITY = 1 << 29;
            Arrays.fill(dp[0], INFINITY);
            dp[0][0] = 0;
            for (int i = 1; i <= S.length(); i++) {
                for (int j = 1; j <= T.length(); j++) {
                    dp[i][j] = dp[i - 1][j] + 1;
                    if (S.charAt(i - 1) == T.charAt(j - 1)) {
                        dp[i][j] = dp[i - 1][j - 1] + 1;
                    }
                }
            }
            int ans = INFINITY;
            int tail = -1;
            for (int i = 1; i <= S.length(); i++) {
                if (dp[i][T.length()] < ans) {
                    ans = dp[i][T.length()];
```

```
                    tail = i;
                }
            }
            return ans == INFINITY ? "" : S.substring(tail - ans, tail);
        }

    }
}

// ============================================================
// FILE 653: _728.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 728. Self Dividing Numbers
 *
 * A self-dividing number is a number that is divisible by every digit it contains.
 * For example, 128 is a self-dividing number because 128 % 1 == 0, 128 % 2 == 0, and 128
% 8 == 0.
 * Also, a self-dividing number is not allowed to contain the digit zero.
 * Given a lower and upper number bound, output a list of every possible self dividing
number, including the bounds if possible.

 Example 1:
 Input:
 left = 1, right = 22
 Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 22]
 Note:

 The boundaries of each input argument are 1 <= left <= right <= 10000.
 */
public class _728 {
    public static class Solution1 {
        public List<Integer> selfDividingNumbers(int left, int right) {
            List<Integer> result = new ArrayList<>();
            for (int num = left; num <= right; num++) {
                if (isSelfDividing(num)) {
                    result.add(num);
                }
            }
            return result;
        }

        private boolean isSelfDividing(int num) {
            int tmp = num;
            while (tmp != 0) {
                int digit = tmp % 10;
                if (digit == 0 || num % digit != 0) {
                    return false;
```

```
                }
                tmp /= 10;
            }
            return true;
        }
    }
}

// ============================================================
// FILE 654: _729.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

/**
 * 729. My Calendar I
 *
 * Implement a MyCalendar class to store your events. A new event can be added if adding
the event will not cause a double booking.
 * Your class will have the method, book(int start, int end).
 * Formally, this represents a booking on the half open interval [start, end),
 * the range of real numbers x such that start <= x < end.
 * A double booking happens when two events have some non-empty intersection (ie., there
is some time that is common to both events.)
 * For each call to the method MyCalendar.book,
 * return true if the event can be added to the calendar successfully without causing a
double booking.
 * Otherwise, return false and do not add the event to the calendar.

 Your class will be called like this: MyCalendar cal = new MyCalendar();
MyCalendar.book(start, end)

 Example 1:
 MyCalendar();
 MyCalendar.book(10, 20); // returns true
 MyCalendar.book(15, 25); // returns false
 MyCalendar.book(20, 30); // returns true

 Explanation:

 The first event can be booked.  The second can't because time 15 is already booked by
another event.
 The third event can be booked, as the first event takes every time less than 20, but not
including 20.
 Note:

 The number of calls to MyCalendar.book per test case will be at most 1000.
 In calls to MyCalendar.book(start, end), start and end are integers in the range [0,
10^9].
```

```
 */
public class _729 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/111205/java-8-liner-treemap
         */
        public static class MyCalendar {
            TreeMap<Integer, Integer> calendar;

            public MyCalendar() {
                calendar = new TreeMap<>();
            }

            public boolean book(int start, int end) {
                Integer floorKey = calendar.floorKey(start);
                if (floorKey != null && calendar.get(floorKey) > start) {
                    return false;
                }
                Integer ceilingKey = calendar.ceilingKey(start);
                if (ceilingKey != null && ceilingKey < end) {
                    return false;
                }
                calendar.put(start, end);
                return true;
            }
        }
    }

    public static class Solution2 {
        public class MyCalendar {
            List<int[]> calendar;

            MyCalendar() {
                calendar = new ArrayList();
            }

            public boolean book(int start, int end) {
                for (int i = 0; i < calendar.size(); i++) {
                    if (calendar.get(i)[0] < end && start < calendar.get(i)[1]) {
                        return false;
                    }
                }
                calendar.add(new int[]{start, end});
                return true;
            }
        }
    }
}

// ============================================================
// FILE 655: _733.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.LinkedList;
import java.util.Queue;

/**
 * 733. Flood Fill
 *
 * An image is represented by a 2-D array of integers, each integer representing the
pixel value of the image (from 0 to 65535).
 * Given a coordinate (sr, sc) representing the starting pixel (row and column) of the
flood fill, and a pixel value newColor, "flood fill" the image.
 * To perform a "flood fill", consider the starting pixel, plus any pixels connected 4-
directionally to the starting
 * pixel of the same color as the starting pixel,
 * plus any pixels connected 4-directionally to those pixels (also with the same color as
the starting pixel), and so on.
 * Replace the color of all of the aforementioned pixels with the newColor.
 * At the end, return the modified image.

 Example 1:
 Input:
 image = [[1,1,1],[1,1,0],[1,0,1]]
 sr = 1, sc = 1, newColor = 2
 Output: [[2,2,2],[2,2,0],[2,0,1]]

 Explanation:
 From the center of the image (with position (sr, sc) = (1, 1)), all pixels connected
 by a path of the same color as the starting pixel are colored with the new color.
 Note the bottom corner is not colored 2, because it is not 4-directionally connected
 to the starting pixel.

 Note:
 The length of image and image[0] will be in the range [1, 50].
 The given starting pixel will satisfy 0 <= sr < image.length and 0 <= sc <
image[0].length.
 The value of each color in image[i][j] and newColor will be an integer in [0, 65535].
 */
public class _733 {
    public static class Solution1 {
        public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
            int[] directions = new int[]{0, 1, 0, -1, 0};
            int m = image.length;
            int n = image[0].length;
            int originalValue = image[sr][sc];
            image[sr][sc] = newColor;

            boolean[][] visited = new boolean[m][n];

            Queue<int[]> queue = new LinkedList<>();
            queue.offer(new int[]{sr, sc});
            while (!queue.isEmpty()) {
                int[] curr = queue.poll();
                visited[curr[0]][curr[1]] = true;
                for (int i = 0; i < directions.length - 1; i++) {
```

```
                    int nextR = curr[0] + directions[i];
                    int nextC = curr[1] + directions[i + 1];
                    if (nextR < 0 || nextC < 0 || nextR >= m || nextC >= n ||
image[nextR][nextC] != originalValue || visited[nextR][nextC]) {
                        continue;
                    }
                    image[nextR][nextC] = newColor;
                    queue.offer(new int[]{nextR, nextC});
                }
            }
            return image;
        }
    }
}

// ============================================================
// FILE 656: _734.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 734. Sentence Similarity
 *
 * Given two sentences words1, words2 (each represented as an array of strings), and a
list of similar word pairs pairs, determine if two sentences are similar.
 * For example, "great acting skills" and "fine drama talent" are similar, if the similar
word pairs are pairs = [["great", "fine"], ["acting","drama"], ["skills","talent"]].
 * Note that the similarity relation is not transitive. For example, if "great" and
"fine" are similar, and "fine" and "good" are similar, "great" and "good" are not
necessarily similar.
 * Also, a word is always similar with itself. For example, the sentences words1 =
["great"], words2 = ["great"], pairs = [] are similar, even though there are no specified
similar word pairs.

 Note:

 The length of words1 and words2 will not exceed 1000.
 The length of pairs will not exceed 2000.
 The length of each pairs[i] will be 2.
 The length of each words[i] and pairs[i][j] will be in the range [1, 20].
 */
public class _734 {
    public static class Solution1 {
        public boolean areSentencesSimilar(String[] words1, String[] words2, String[][]
pairs) {
            if (words1.length != words2.length) {
                return false;
            }
            for (int i = 0; i < words1.length; i++) {
                if (!isSimilar(words1[i], words2[i], pairs)) {
                    return false;
                }
            }
```

```
            return true;
        }

        private boolean isSimilar(String word1, String word2, String[][] pairs) {
            if (word1.equals(word2)) {
                return true;
            }
            for (int i = 0; i < pairs.length; i++) {
                String[] pair = pairs[i];
                if (pair[0].equals(word1) && pair[1].equals(word2)) {
                    return true;
                }
                if (pair[0].equals(word2) && pair[1].equals(word1)) {
                    return true;
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 657: _735.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Stack;

/**
 * 735. Asteroid Collision
 *
 * We are given an array asteroids of integers representing asteroids in a row.
 * For each asteroid, the absolute value represents its size, and the sign represents its
direction
 * (positive meaning right, negative meaning left). Each asteroid moves at the same
speed.
 * Find out the state of the asteroids after all collisions.
 * If two asteroids meet, the smaller one will explode. If both are the same size, both
will explode. Two asteroids moving in the same direction will never meet.

 Example 1:
 Input:
 asteroids = [5, 10, -5]
 Output: [5, 10]
 Explanation:
 The 10 and -5 collide resulting in 10.  The 5 and 10 never collide.

 Example 2:
 Input:
 asteroids = [8, -8]
 Output: []
 Explanation:
 The 8 and -8 collide exploding each other.
```

```
 Example 3:
 Input:
 asteroids = [10, 2, -5]
 Output: [10]
 Explanation:
 The 2 and -5 collide resulting in -5.  The 10 and -5 collide resulting in 10.

 Example 4:
 Input:
 asteroids = [-2, -1, 1, 2]
 Output: [-2, -1, 1, 2]
 Explanation:
 The -2 and -1 are moving left, while the 1 and 2 are moving right.
 Asteroids moving the same direction never meet, so no asteroids will meet each other.

 Note:
 The length of asteroids will be at most 10000.
 Each asteroid will be a non-zero integer in the range [-1000, 1000]..

 */
public class _735 {
    public static class Solution1 {
        public int[] asteroidCollision(int[] asteroids) {
            Stack<Integer> stack = new Stack();
            for (int i = 0; i < asteroids.length; i++) {
                if (!stack.isEmpty() && stack.peek() > 0 && asteroids[i] < 0) {
                    if (Math.abs(stack.peek()) < Math.abs(asteroids[i])) {
                        stack.pop();
                        stack.push(asteroids[i]);
                        collide(stack);
                    } else if (Math.abs(stack.peek()) == Math.abs(asteroids[i])) {
                        stack.pop();
                    }
                } else {
                    stack.push(asteroids[i]);
                }
            }
            int[] result = new int[stack.size()];
            int i = stack.size();
            while (!stack.isEmpty()) {
                result[--i] = stack.pop();
            }
            return result;
        }

        private void collide(Stack<Integer> stack) {
            do {
                Integer top = stack.pop();
                if (!stack.isEmpty() && stack.peek() * top < 0) {
                    if (stack.peek() < Math.abs(top)) {
                        stack.pop();
                        stack.push(top);
                    } else if (stack.peek() == Math.abs(top)) {
```

```
                        stack.pop();
                        break;
                    } else {
                        break;
                    }
                } else if (stack.isEmpty() || stack.peek() * top > 0) {
                    stack.push(top);
                    break;
                }
            } while (!stack.isEmpty());
        }
    }
}

// ============================================================
// FILE 658: _737.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 737. Sentence Similarity II
 *
 * Given two sentences words1, words2 (each represented as an array of strings), and a
list of similar word pairs pairs, determine if two sentences are similar.
 * For example, words1 = ["great", "acting", "skills"] and words2 = ["fine", "drama",
"talent"] are similar,
 * if the similar word pairs are pairs = [["great", "good"], ["fine", "good"],
["acting","drama"], ["skills","talent"]].
 * Note that the similarity relation is transitive.
 *
 * For example, if "great" and "good" are similar, and "fine" and "good" are similar,
then "great" and "fine" are similar.
 * Similarity is also symmetric. For example, "great" and "fine" being similar is the
same as "fine" and "great" being similar.
 * Also, a word is always similar with itself. For example, the sentences words1 =
["great"], words2 = ["great"], pairs = [] are similar, even though there are no specified
similar word pairs.
 * Finally, sentences can only be similar if they have the same number of words. So a
sentence like words1 = ["great"] can never be similar to words2 = ["doubleplus","good"].

 Note:

 The length of words1 and words2 will not exceed 1000.
 The length of pairs will not exceed 2000.
 The length of each pairs[i] will be 2.
 The length of each words[i] and pairs[i][j] will be in the range [1, 20].

 */
public class _737 {
    public static class Solution1 {
```

```
        public boolean areSentencesSimilarTwo(String[] words1, String[] words2, String[]
[] pairs) {
            if (words1.length != words2.length) {
                return false;
            }

            Map<String, String> map = new HashMap<>();
            for (String[] pair : pairs) {
                if (!map.containsKey(pair[0])) {
                    map.put(pair[0], pair[0]);
                }
                if (!map.containsKey(pair[1])) {
                    map.put(pair[1], pair[1]);
                }

                setParent(map, pair[0], pair[1]);
            }

            for (int i = 0; i < words1.length; i++) {
                String parent1 = getParent(map, words1[i]);
                String parent2 = getParent(map, words2[i]);
                if (!parent1.equals(parent2)) {
                    return false;
                }
            }
            return true;
        }

        private void setParent(Map<String, String> map, String word1, String word2) {
            String parent1 = getParent(map, word1);
            String parent2 = getParent(map, word2);
            map.put(parent1, parent2);
        }

        private String getParent(Map<String, String> map, String word) {
            if (!map.containsKey(word)) {
                return word;
            }
            while (!word.equals(map.get(word))) {
                word = map.get(word);
            }
            return word;
        }
    }
}

// ============================================================
// FILE 659: _738.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 738. Monotone Increasing Digits
```

```
 *
 * Given a non-negative integer N, find the largest number that is less than or equal to
N with monotone increasing digits.
 * (Recall that an integer has monotone increasing digits if and only if each pair of
adjacent digits x and y satisfy x <= y.)

 Example 1:
 Input: N = 10
 Output: 9

 Example 2:
 Input: N = 1234
 Output: 1234

 Example 3:
 Input: N = 332
 Output: 299

 Note: N is an integer in the range [0, 10^9].
 */
public class _738 {
    public static class Solution1 {
        /**
         * credit: https://discuss.leetcode.com/topic/112808/simple-python-solution-w-
explanation/2
         */
        public int monotoneIncreasingDigits(int N) {
            String s = Integer.toString(N);
            int index = -1;
            for (int i = s.length() - 2; i >= 0; i--) {
                if (s.charAt(i) > s.charAt(i + 1) || (index != -1 && s.charAt(index) ==
s.charAt(i))) {
                    index = i;
                }
            }
            return index == -1 ? N : N - Integer.parseInt(s.substring(index + 1,
s.length())) - 1;
        }
    }
}

// ============================================================
// FILE 660: _739.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 739. Daily Temperatures
 *
 * Given a list of daily temperatures, produce a list that,
 * for each day in the input, tells you how many days you would have to wait until a
warmer temperature.
 * If there is no future day for which this is possible, put 0 instead.
```

```
 * For example, given the list temperatures = [73, 74, 75, 71, 69, 72, 76, 73], your
output should be [1, 1, 4, 2, 1, 1, 0, 0].
 *
 * Note: The length of temperatures will be in the range [1, 30000]. Each temperature
will be an integer in the range [30, 100].
 */
public class _739 {

    public static class Solution1 {
        public int[] dailyTemperatures(int[] temperatures) {
            if (temperatures == null || temperatures.length == 0) {
                return temperatures;
            }
            int[] result = new int[temperatures.length];
            for (int i = 0; i < temperatures.length; i++) {
                for (int j = i + 1; j < temperatures.length; j++) {
                    if (temperatures[j] > temperatures[i]) {
                        result[i] = j - i;
                        break;
                    }
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 661: _740.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 740. Delete and Earn
 *
 * Given an array nums of integers, you can perform operations on the array.
 * In each operation, you pick any nums[i] and delete it to earn nums[i] points. After,
you must delete every element equal to nums[i] - 1 or nums[i] + 1.
 * You start with 0 points. Return the maximum number of points you can earn by applying
such operations.

 Example 1:
 Input: nums = [3, 4, 2]
 Output: 6
 Explanation:
 Delete 4 to earn 4 points, consequently 3 is also deleted.
 Then, delete 2 to earn 2 points. 6 total points are earned.

 Example 2:
 Input: nums = [2, 2, 3, 3, 3, 4]
 Output: 9
 Explanation:
 Delete 3 to earn 3 points, deleting both 2's and the 4.
```

```
 Then, delete 3 again to earn 3 points, and 3 again to earn 3 points.
 9 total points are earned.

 Note:
 The length of nums is at most 20000.
 Each element nums[i] is an integer in the range [1, 10000].*/

public class _740 {
  public static class Solution1 {
    /**
     * Since the number is within range [1, 10000], we can build another array:
     * each number in the array denotes the total sum of this number that appears in this
array
     * and
     * use the numbers themselves in the indices of another array
     *
     * credit: https://leetcode.com/problems/delete-and-earn/discuss/109895/JavaC++-
Clean-Code-with-Explanation*/
    public int deleteAndEarn(int[] nums) {
      int n = 10001;
      int[] values = new int[n];
      for (int num : nums) {
        values[num] += num;
      }

      int take = 0;
      int skip = 0;
      for (int i = 0; i < n; i++) {
        int takeI = skip + values[i];
        int skipI = Math.max(skip, take);
        take = takeI;
        skip = skipI;
      }
      return Math.max(take, skip);
    }
  }
}

// ============================================================
// FILE 662: _743.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 743. Network Delay Time
 *
 * There are N network nodes, labelled 1 to N.
 *
 *  Given times, a list of travel times as directed edges times[i] = (u, v, w), where u
is the source node, v is the target node, and w is the time it takes for a signal to
travel from source to target.
 *
 * Now, we send a signal from a certain node K. How long will it take for all nodes to
```

```
receive the signal? If it is impossible, return -1.
 *
 * Note:
 *  * N will be in the range [1, 100].
 *  * K will be in the range [1, N].
 *  * The length of times will be in the range [1, 6000].
 *  * All edges times[i] = (u, v, w) will have 1 <= u, v <= N and 0 <= w <= 100.
 */

public class _743 {
    public static class Solution1 {
        public static final int MAX = 100_00_00;

        private int[][] parseTimes(int[][] times, int N) {
            int[][] weights = new int[N + 1][N + 1];

            for (int i = 0; i < N + 1; ++i) {
                for (int j = 0; j < N + 1; ++j) {
                    weights[i][j] = MAX;
                }
            }

            for (int[] time : times) {
                int u = time[0];
                int v = time[1];
                int w = time[2];
                weights[u][v] = w;
            }

            return weights;
        }

        private int findMinUnvisitedIndex(int[] a, boolean[] visited) {
            int min = Integer.MAX_VALUE;
            int minIdx = 1;

            for (int i = 1; i < a.length; ++i) {
                if (a[i] < min && !visited[i]) {
                    min = a[i];
                    minIdx = i;
                }
            }
            return minIdx;
        }

        public int networkDelayTime(int[][] times, int N, int K) {
            // Time from u to v is weights[u][v]
            int[][] weights = parseTimes(times, N);

            // Time from node K to note i is totalTime[i]
            int[] totalTime = new int[N + 1];
            for (int j = 0; j <= N; ++j) {
                totalTime[j] = MAX;
            }
```

```
            totalTime[K] = 0;

            boolean[] visited = new boolean[N + 1];
            int visitCount = 0;

            while (visitCount != N) {
                int node = findMinUnvisitedIndex(totalTime, visited);

                // The smallest time to a node is more than our designated max
                // so it must be unreachable
                if (totalTime[node] >= MAX) {
                    return -1;
                }

                visited[node] = true;
                visitCount += 1;

                // Update the time to each node if we try to visit it from our current
node
                for (int i = 1; i <= N; ++i) {
                    totalTime[i] = Math.min(weights[node][i] + totalTime[node],
totalTime[i]);
                }
            }

            // What is the maximum time to any particular node
            int maxVal = Integer.MIN_VALUE;
            for (int k = 1; k <= N; ++k) {
                maxVal = Math.max(totalTime[k], maxVal);
            }

            return maxVal;
        }
    }
}

// ============================================================
// FILE 663: _744.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 744. Find Smallest Letter Greater Than Target
 *
 * Given a list of sorted characters letters containing only lowercase letters, and given
a target letter target, find the smallest element in the list that is larger than the
given target.
 * Letters also wrap around. For example, if the target is target = 'z' and letters =
['a', 'b'], the answer is 'a'.

 Examples:
 Input:
 letters = ["c", "f", "j"]
```

```
 target = "a"
 Output: "c"

 Input:
 letters = ["c", "f", "j"]
 target = "c"
 Output: "f"

 Input:
 letters = ["c", "f", "j"]
 target = "d"
 Output: "f"

 Input:
 letters = ["c", "f", "j"]
 target = "g"
 Output: "j"

 Input:
 letters = ["c", "f", "j"]
 target = "j"
 Output: "c"

 Input:
 letters = ["c", "f", "j"]
 target = "k"
 Output: "c"

 Note:
 letters has a length in range [2, 10000].
 letters consists of lowercase letters, and contains at least 2 unique letters.
 target is a lowercase letter.
 */
public class _744 {
    public static class Solution1 {
        public char nextGreatestLetter(char[] letters, char target) {
            if (letters[0] > target) {
                return letters[0];
            }
            int left = 0;
            int right = letters.length - 1;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (letters[mid] > target) {
                    while (letters[mid] > target) {
                        mid--;
                    }
                    return letters[++mid];
                } else {
                    left = mid + 1;
                }
            }
            if (right < letters.length && letters[right] > target) {
                return letters[right];
```

```
            }
            return letters[0];
        }
    }
}

// ============================================================
// FILE 664: _746.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 746. Min Cost Climbing Stairs
 *
 * On a staircase, the i-th step has some non-negative cost cost[i] assigned (0 indexed).
 * Once you pay the cost, you can either climb one or two steps.
 * You need to find minimum cost to reach the top of the floor, and you can either start
from the step with index 0, or the step with index 1.

 Example 1:
 Input: cost = [10, 15, 20]
 Output: 15
 Explanation: Cheapest is start on cost[1], pay that cost and go to the top.

 Example 2:
 Input: cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
 Output: 6
 Explanation: Cheapest is start on cost[0], and only step on 1s, skipping cost[3].

 Note:
 cost will have a length in the range [2, 1000].
 Every cost[i] will be an integer in the range [0, 999].
 */

public class _746 {
  public static class Solution1 {
    public int minCostClimbingStairs(int[] cost) {
      int[] dp = new int[cost.length];
      dp[0] = cost[0];
      dp[1] = cost[1];
      for (int i = 2; i < cost.length; i++) {
        dp[i] = Math.min(dp[i - 1] + cost[i], dp[i - 2] + cost[i]);
      }
      return Math.min(dp[cost.length - 1], dp[cost.length - 2]);
    }
  }
}

// ============================================================
// FILE 665: _747.java
// ============================================================

package com.fishercoder.solutions;
```

```java
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 747. Largest Number Greater Than Twice of Others
 *
 * In a given integer array nums, there is always exactly one largest element.
 * Find whether the largest element in the array is at least twice as much as every other
number in the array.
 * If it is, return the index of the largest element, otherwise return -1.

 Example 1:
 Input: nums = [3, 6, 1, 0]
 Output: 1
 Explanation: 6 is the largest integer, and for every other number in the array x,
 6 is more than twice as big as x.  The index of value 6 is 1, so we return 1.

 Example 2:
 Input: nums = [1, 2, 3, 4]
 Output: -1
 Explanation: 4 isn't at least as big as twice the value of 3, so we return -1.

 Note:
 nums will have a length in the range [1, 50].
 Every nums[i] will be an integer in the range [0, 99].
 */
public class _747 {

  public static class Solution1 {
    public int dominantIndex(int[] nums) {
      Map<Integer, Integer> map = new HashMap<>();
      int max;
      int secondMax;
      for (int i = 0; i < nums.length; i++) {
        map.put(nums[i], i);
      }
      Arrays.sort(nums);
      max = nums[nums.length - 1];
      secondMax = nums[nums.length - 2];
      if (max >= 2 * secondMax) {
        return map.get(max);
      } else {
        return -1;
      }
    }
  }

  public static class Solution2 {
    public int dominantIndex(int[] nums) {
      int max = Integer.MIN_VALUE;
      int maxIndex = -1;
      for (int i = 0; i < nums.length; i++) {
```

```
        if (nums[i] > max) {
          max = nums[i];
          maxIndex = i;
        }
      }
      for (int i = 0; i < nums.length; i++) {
        if (nums[i] * 2 > max && i != maxIndex) {
          return -1;
        }
      }
      return maxIndex;
    }
  }
}

// ============================================================
// FILE 666: _748.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 748. Shortest Completing Word
 *
 * Find the minimum length word from a given dictionary words,
 * which has all the letters from the string licensePlate. Such a word is said to
complete the given string licensePlate
 *
 * Here, for letters we ignore case. For example, "P" on the licensePlate still matches
"p" on the word.
 * It is guaranteed an answer exists. If there are multiple answers, return the one that
occurs first in the array.
 * The license plate might have the same letter occurring multiple times. For example,
 * given a licensePlate of "PP", the word "pair" does not complete the licensePlate, but
the word "supper" does.

 Example 1:
 Input: licensePlate = "1s3 PSt", words = ["step", "steps", "stripe", "stepple"]
 Output: "steps"
 Explanation: The smallest length word that contains the letters "S", "P", "S", and "T".
 Note that the answer is not "step", because the letter "s" must occur in the word twice.
 Also note that we ignored case for the purposes of comparing whether a letter exists in
the word.

 Example 2:
 Input: licensePlate = "1s3 456", words = ["looks", "pest", "stew", "show"]
 Output: "pest"
 Explanation: There are 3 smallest length words that contains the letters "s".
 We return the one that occurred first.

 Note:
 licensePlate will be a string with length in range [1, 7].
 licensePlate will contain digits, spaces, or letters (uppercase or lowercase).
 words will have a length in the range [10, 1000].
```

```
 Every words[i] will consist of lowercase letters, and have length in range [1, 15].

 */
public class _748 {

  public static class Solution1 {
    public String shortestCompletingWord(String licensePlate, String[] words) {
      int[] counts = new int[26];
      for (char c : licensePlate.toCharArray()) {
        if (Character.isAlphabetic(c)) {
          counts[Character.toLowerCase(c) - 'a']++;
        }
      }
      String result = "";
      for (String word : words) {
        if (isComplete(word, counts)) {
          if (result.equals("")) {
            result = word;
          } else if (word.length() < result.length()) {
            result = word;
          }
        }
      }
      return result;
    }

    private boolean isComplete(String word, int[] counts) {
      int[] tmp = counts.clone();
      for (char c : word.toCharArray()) {
        if (tmp[c - 'a'] > 0) {
          tmp[c - 'a']--;
        }
      }
      for (int i : tmp) {
        if (i != 0) {
          return false;
        }
      }
      return true;
    }
  }
}

// ============================================================
// FILE 667: _749.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 749. Contain Virus
 *
 * A virus is spreading rapidly, and your task is to quarantine the infected area by
installing walls.
```

```
 * The world is modeled as a 2-D array of cells, where 0 represents uninfected cells,
 * and 1 represents cells contaminated with the virus.
 * A wall (and only one wall) can be installed between any two 4-directionally adjacent
cells, on the shared boundary.
 * Every night, the virus spreads to all neighboring cells in all four directions unless
blocked by a wall.
 * Resources are limited. Each day, you can install walls around only one region --
 * the affected area (continuous block of infected cells) that threatens the most
uninfected cells the following night. There will never be a tie.
 * Can you save the day? If so, what is the number of walls required? If not, and the
world becomes fully infected, return the number of walls used.

 Example 1:
 Input: grid =
 [[0,1,0,0,0,0,0,1],
 [0,1,0,0,0,0,0,1],
 [0,0,0,0,0,0,0,1],
 [0,0,0,0,0,0,0,0]]

 Output: 10
 Explanation:
 There are 2 contaminated regions.
 On the first day, add 5 walls to quarantine the viral region on the left. The board
after the virus spreads is:

 [[0,1,0,0,0,0,1,1],
 [0,1,0,0,0,0,1,1],
 [0,0,0,0,0,0,1,1],
 [0,0,0,0,0,0,0,1]]

 On the second day, add 5 walls to quarantine the viral region on the right. The virus is
fully contained.


 Example 2:
 Input: grid =
 [[1,1,1],
 [1,0,1],
 [1,1,1]]

 Output: 4
 Explanation: Even though there is only one cell saved, there are 4 walls built.
 Notice that walls are only built on the shared boundary of two different cells.


 Example 3:
 Input: grid =
 [[1,1,1,0,0,0,0,0,0],
 [1,0,1,0,1,1,1,1,1],
 [1,1,1,0,0,0,0,0,0]]
 Output: 13
 Explanation: The region on the left only builds two new walls.
 Note:
 The number of rows and columns of grid will each be in the range [1, 50].
```

```
 Each grid[i][j] will be either 0 or 1.
 Throughout the described process, there is always a contiguous viral region that will
infect strictly more uncontaminated squares in the next round.
 */
public class _749 {
  public static class Solution1 {
    public int containVirus(int[][] grid) {
      return -1;
    }
  }
}

// ============================================================
// FILE 668: _750.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 750. Number Of Corner Rectangles
 *
 * Given a grid where each entry is only 0 or 1, find the number of corner rectangles.
 * A corner rectangle is 4 distinct 1s on the grid that form an axis-aligned rectangle.
 * Note that only the corners need to have the value 1. Also, all four 1s used must be
distinct.

 Example 1:
 Input: grid =
 [[1, 0, 0, 1, 0],
 [0, 0, 1, 0, 1],
 [0, 0, 0, 1, 0],
 [1, 0, 1, 0, 1]]
 Output: 1
 Explanation: There is only one corner rectangle, with corners grid[1][2], grid[1][4],
grid[3][2], grid[3][4].

 Example 2:
 Input: grid =
 [[1, 1, 1],
 [1, 1, 1],
 [1, 1, 1]]
 Output: 9
 Explanation: There are four 2x2 rectangles, four 2x3 and 3x2 rectangles, and one 3x3
rectangle.

 Example 3:
 Input: grid =
 [[1, 1, 1, 1]]
 Output: 0
 Explanation: Rectangles must have four distinct corners.

 Note:
 The number of rows and columns of grid will each be in the range [1, 200].
 Each grid[i][j] will be either 0 or 1.
```

```
 The number of 1s in the grid will be at most 6000.*/
public class _750 {
  public static class Solution1 {
    public int countCornerRectangles(int[][] grid) {
      if (grid == null || grid.length < 2) {
        return 0;
      }
      int m = grid.length;
      int n = grid[0].length;
      int count = 0;
      for (int i = 0; i < m - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
          if (grid[i][j] == 1) {
            for (int jNext = j + 1; jNext < n; jNext++) {
              if (grid[i][jNext] == 1) {
                for (int iNext = i + 1; iNext < m; iNext++) {
                  if (grid[iNext][j] == 1 && grid[iNext][jNext] == 1) {
                    count++;
                  }
                }
              }
            }
          }
        }
      }
      return count;
    }
  }
}

// ============================================================
// FILE 669: _754.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 754. Reach a Number
 *
 * You are standing at position 0 on an infinite number line. There is a goal at position
target.
 * On each move, you can either go left or right. During the n-th move (starting from 1),
you take n steps.
 * Return the minimum number of steps required to reach the destination.

 Example 1:
 Input: target = 3
 Output: 2
 Explanation:
 On the first move we step from 0 to 1.
 On the second step we step from 1 to 3.

 Example 2:
 Input: target = 2
```

```
 Output: 3
 Explanation:
 On the first move we step from 0 to 1.
 On the second move we step  from 1 to -1.
 On the third move we step from -1 to 2.

 Note:
 target will be a non-zero integer in the range [-10^9, 10^9].
 */

public class _754 {
  public static class Solution1 {
    /**Two case:
     * 1. go to the right, and reach the goal exactly.
     * 2. go over the goal by several steps:
     *    by even number, then you can choose one of the steps that went right to go back
to the left (the step is half of what you went over)
     *    by odd number, then you keep going until you are over by an even number.*/
    public int reachNumber(int target) {
      int absTarget = Math.abs(target);
      int steps = 1;
      int sum = 0;
      while (sum < absTarget || (sum - absTarget) % 2 == 1) {
        sum += steps;
        steps++;
      }
      return steps - 1;
    }
  }
}

// ============================================================
// FILE 670: _755.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 755. Pour Water
 *
 * We are given an elevation map, heights[i] representing the height of the terrain at
that index.
 * The width at each index is 1. After V units of water fall at index K, how much water
is at each index?
 * Water first drops at index K and rests on top of the highest terrain or water at that
index.
 * Then, it flows according to the following rules:

 If the droplet would eventually fall by moving left, then move left.
 Otherwise, if the droplet would eventually fall by moving right, then move right.
 Otherwise, rise at it's current position.

 Here, "eventually fall" means that the droplet will eventually be at a lower level if it
moves in that direction. Also, "level" means the height of the terrain plus any water in
```

```
that column.
 We can assume there's infinitely high terrain on the two sides out of bounds of the
array. Also, there could not be partial water being spread out evenly on more than 1 grid
block - each unit of water has to be in exactly one block.

 Example 1:
 Input: heights = [2,1,1,2,1,2,2], V = 4, K = 3
 Output: [2,2,2,3,2,2,2]
 Explanation:
 #       #
 #       #
 ##  # ###
 #########
 0123456    <- index

 The first drop of water lands at index K = 3:

 #       #
 #   w   #
 ##  # ###
 #########
 0123456

 When moving left or right, the water can only move to the same level or a lower level.
 (By level, we mean the total height of the terrain plus any water in that column.)
 Since moving left will eventually make it fall, it moves left.
 (A droplet "made to fall" means go to a lower height than it was at previously.)

 #       #
 #       #
 ## w# ###
 #########
 0123456

 Since moving left will not make it fall, it stays in place.  The next droplet falls:

 #       #
 #   w   #
 ## w# ###
 #########
 0123456

 Since the new droplet moving left will eventually make it fall, it moves left.
 Notice that the droplet still preferred to move left,
 even though it could move right (and moving right makes it fall quicker.)

 #       #
 #  w    #
 ## w# ###
 #########
 0123456

 #       #
 #       #
```

```
 ##ww# ###
 #########
 0123456

 After those steps, the third droplet falls.
 Since moving left would not eventually make it fall, it tries to move right.
 Since moving right would eventually make it fall, it moves right.

 #       #
 #   w   #
 ##ww# ###
 #########
 0123456

 #       #
 #       #
 ##ww#w###
 #########
 0123456

 Finally, the fourth droplet falls.
 Since moving left would not eventually make it fall, it tries to move right.
 Since moving right would not eventually make it fall, it stays in place:

 #       #
 #   w   #
 ##ww#w###
 #########
 0123456

 The final answer is [2,2,2,3,2,2,2]:

 #
 #######
 #######
 0123456


 Example 2:
 Input: heights = [1,2,3,4], V = 2, K = 2
 Output: [2,3,3,4]
 Explanation:
 The last droplet settles at index 1, since moving further left would not cause it to
eventually fall to a lower height.


 Example 3:
 Input: heights = [3,1,3], V = 5, K = 1
 Output: [4,4,4]
 Note:

 heights will have length in [1, 100] and contain integers in [0, 99].
 V will be in range [0, 2000].
 K will be in range [0, heights.length - 1].
```

```
 */

public class _755 {
  public static class Solution1 {
    public int[] pourWater(int[] heights, int V, int K) {
      int index;
      while (V > 0) {
        index = K;
        for (int i = K - 1; i >= 0; i--) {
          if (heights[i] > heights[index]) {
            break;
          } else if (heights[i] < heights[index]) {
            index = i;
          }
        }
        if (index != K) {
          heights[index]++;
          V--;
          continue;
        }

        for (int i = K + 1; i < heights.length; i++) {
          if (heights[i] > heights[index]) {
            break;
          } else if (heights[i] < heights[index]) {
            index = i;
          }
        }
        heights[index]++;
        V--;
      }
      return heights;
    }
  }
}

// ============================================================
// FILE 671: _756.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 756. Pyramid Transition Matrix
 *
 * We are stacking blocks to form a pyramid. Each block has a color which is a one letter
string, like `'Z'`.
 * For every block of color `C` we place not in the bottom row,
 * we are placing it on top of a left block of color `A` and right block of color `B`.
```

```
 * We are allowed to place the block there only if `(A, B, C)` is an allowed triple.
 * We start with a bottom row of bottom,
 * represented as a single string. We also start with a list of allowed triples allowed.
 * Each allowed triple is represented as a string of length 3.
 * Return true if we can build the pyramid all the way to the top, otherwise false.

 Example 1:
 Input: bottom = "XYZ", allowed = ["XYD", "YZE", "DEA", "FFF"]
 Output: true
 Explanation:
 We can stack the pyramid like this:
     A
    / \
   D   E
  / \ / \
 X   Y   Z

 This works because ('X', 'Y', 'D'), ('Y', 'Z', 'E'), and ('D', 'E', 'A') are allowed
triples.

 Example 2:
 Input: bottom = "XXYX", allowed = ["XXX", "XXY", "XYX", "XYY", "YXZ"]
 Output: false
 Explanation:
 We can't stack the pyramid to the top.
 Note that there could be allowed triples (A, B, C) and (A, B, D) with C != D.

 Note:
 bottom will be a string with length in range [2, 8].
 allowed will have length in range [0, 200].
 Letters in all strings will be chosen from the set {'A', 'B', 'C', 'D', 'E', 'F', 'G'}.
 */
public class _756 {
  public static class Solution1 {
    /**credit: https://discuss.leetcode.com/topic/116042/java-solution-map-backtracking*/
    public boolean pyramidTransition(String bottom, List<String> allowed) {
      Map<String, List<String>> map = new HashMap<>();
      for (String s : allowed) {
        String key = s.substring(0, 2);
        if (!map.containsKey(key)) {
          map.put(key, new ArrayList<>());
        }
        map.get(key).add(s.substring(2));
      }

      return helper(bottom, map);
    }

    private boolean helper(String bottom, Map<String, List<String>> map) {
      if (bottom.length() == 1) {
        return true;
      }
      for (int i = 0; i < bottom.length() - 1; i++) {
        if (!map.containsKey(bottom.substring(i, i + 2))) {
```

```
          return false;
        }
      }
      List<String> ls = new ArrayList<>();
      getList(bottom, 0, new StringBuilder(), ls, map);
      for (String s : ls) {
        if (helper(s, map)) {
          return true;
        }
      }
      return false;
    }

    private void getList(String bottom, int idx, StringBuilder sb, List<String> ls,
        Map<String, List<String>> map) {
      if (idx == bottom.length() - 1) {
        ls.add(sb.toString());
        return;
      }
      for (String s : map.get(bottom.substring(idx, idx + 2))) {
        sb.append(s);
        getList(bottom, idx + 1, sb, ls, map);
        sb.deleteCharAt(sb.length() - 1);
      }
    }
  }
}

// ============================================================
// FILE 672: _758.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 758. Bold Words in String
 *
 * Given a set of keywords words and a string S, make all appearances of all keywords in
S bold. Any letters between <b> and </b> tags become bold.
 * The returned string should use the least number of tags possible, and of course the
tags should form a valid combination.
 * For example, given that words = ["ab", "bc"] and S = "aabcd", we should return
"a<b>abc</b>d".
 * Note that returning "a<b>a<b>b</b>c</b>d" would use more tags, so it is incorrect.

 Note:

 words has length in range [0, 50].
 words[i] has length in range [1, 10].
 S has length in range [0, 500].
 All characters in words[i] and S are lowercase letters.

 */
```

```
public class _758 {
  public static class Solution1 {
    /**Interestingly, this problem is exactly the same as 616, using 616's code could get
it AC'ed.*/
    public String boldWords(String[] words, String S) {
      boolean[] shouldBold = new boolean[S.length()];
      for (int i = 0, end = 0; i < S.length(); i++) {
        for (String word : words) {
          if (S.startsWith(word, i)) {
            end = Math.max(end, i + word.length());
          }
        }
        shouldBold[i] = end > i;
      }
      StringBuilder stringBuilder = new StringBuilder();
      for (int i = 0; i < S.length(); i++) {
        if (!shouldBold[i]) {
          stringBuilder.append(S.charAt(i));
          continue;
        }
        int j = i;
        while (j < S.length() && shouldBold[j]) {
          j++;
        }
        stringBuilder.append("<b>" + S.substring(i, j) + "</b>");
        i = j - 1;
      }
      return stringBuilder.toString();
    }
  }
}

// ============================================================
// FILE 673: _760.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 760. Find Anagram Mappings
 *
 * Given two lists Aand B, and B is an anagram of A. B is an anagram of A means B is made
by randomizing the order of the elements in A.
 * We want to find an index mapping P, from A to B. A mapping P[i] = j means the ith
element in A appears in B at index j.
 * These lists A and B may contain duplicates. If there are multiple answers, output any
of them.

 For example, given

 A = [12, 28, 46, 32, 50]
 B = [50, 12, 32, 46, 28]

 We should return
```

```
 [1, 4, 3, 2, 0]
 as P[0] = 1 because the 0th element of A appears at B[1], and P[1] = 4 because the 1st
element of A appears at B[4], and so on.

 Note:

 A, B have equal lengths in range [1, 100].
 A[i], B[i] are integers in range [0, 10^5].
 */
public class _760 {
  public static class Solution1 {
    public int[] anagramMappings(int[] A, int[] B) {
      int[] result = new int[A.length];
      for (int i = 0; i < A.length; i++) {
        for (int j = 0; j < B.length; j++) {
          if (A[i] == B[j]) {
            result[i] = j;
          }
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 674: _762.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 762. Prime Number of Set Bits in Binary Representation
 *
 * Given two integers L and R, find the count of numbers in the range [L, R] (inclusive)
having a prime number of set bits in their binary representation.
 * (Recall that the number of set bits an integer has is the number of 1s present when
written in binary.
 * For example, 21 written in binary is 10101 which has 3 set bits. Also, 1 is not a
prime.)

 Example 1:

 Input: L = 6, R = 10
 Output: 4

 Explanation:
 6 -> 110 (2 set bits, 2 is prime)
 7 -> 111 (3 set bits, 3 is prime)
 9 -> 1001 (2 set bits , 2 is prime)
 10->1010 (2 set bits , 2 is prime)

 Example 2:
```

```
 Input: L = 10, R = 15
 Output: 5

 Explanation:
 10 -> 1010 (2 set bits, 2 is prime)
 11 -> 1011 (3 set bits, 3 is prime)
 12 -> 1100 (2 set bits, 2 is prime)
 13 -> 1101 (3 set bits, 3 is prime)
 14 -> 1110 (3 set bits, 3 is prime)
 15 -> 1111 (4 set bits, 4 is not prime)

 Note:

 L, R will be integers L <= R in the range [1, 10^6].
 R - L will be at most 10000.
 */

public class _762 {
  public static class Solution1 {
    public int countPrimeSetBits(int L, int R) {
      int count = 0;
      for (int i = L; i <= R; i++) {
        if (hasPrimeNumberSetBits(i)) {
          count++;
        }
      }
      return count;
    }

    private boolean hasPrimeNumberSetBits(int num) {
      int k = getSetBits(num);
      if (k <= 1) {
        return false;
      }
      for (int i = 2; i * i <= k; i++) {
        if (k % i == 0) {
          return false;
        }
      }
      return true;
    }

    private int getSetBits(int n) {
      int bits = 0;
      while (n != 0) {
        bits++;
        n &= (n - 1);
      }
      return bits;
    }
  }
}

// ============================================================
```

```
// FILE 675: _763.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 763. Partition Labels
 *
 * A string S of lowercase letters is given.
 * We want to partition this string into as many parts as possible so that each letter
appears
 * in at most one part, and return a list of integers representing the size of these
parts.

 Example 1:
 Input: S = "ababcbacadefegdehijhklij"
 Output: [9,7,8]
 Explanation:
 The partition is "ababcbaca", "defegde", "hijhklij".
 This is a partition so that each letter appears in at most one part.
 A partition like "ababcbacadefegde", "hijhklij" is incorrect, because it splits S into
less parts.

 Note:
 S will have length in range [1, 500].
 S will consist of lowercase letters ('a' to 'z') only.
 */
public class _763 {

  public static class Solution1 {
    public List<Integer> partitionLabels(String S) {
      List<Integer> result = new ArrayList<>();
      int[] last = new int[26];
      /**This is the key step:
       * we find the last occurrence of each letter and record them in last[]*/
      for (int i = 0; i < S.length(); i++) {
        last[S.charAt(i) - 'a'] = i;
      }
      /**record the last end index of the current substring*/
      int end = 0;
      int start = 0;
      for (int i = 0; i < S.length(); i++) {
        end = Math.max(end, last[S.charAt(i) - 'a']);
        if (end == i) {
          result.add(end - start + 1);
          start = end + 1;
        }
      }
      return result;
    }
  }
```

```
}

// ============================================================
// FILE 676: _764.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 764. Largest Plus Sign
 *
 * In a 2D grid from (0, 0) to (N-1, N-1), every cell contains a 1,
 * except those cells in the given list mines which are 0.
 * What is the largest axis-aligned plus sign of 1s contained in the grid? Return the
order of the plus sign.
 * If there is none, return 0.
 *
 * An "axis-aligned plus sign of 1s of order k" has some center grid[x][y] = 1 along with
4 arms of length k-1 going up, down, left, and right, and made of 1s.
 * This is demonstrated in the diagrams below.
 * Note that there could be 0s or 1s beyond the arms of the plus sign,
 * only the relevant area of the plus sign is checked for 1s.

 Examples of Axis-Aligned Plus Signs of Order k:

 Order 1:
 000
 010
 000

 Order 2:
 00000
 00100
 01110
 00100
 00000

 Order 3:
 0000000
 0001000
 0001000
 0111110
 0001000
 0001000
 0000000


 Example 1:
 Input: N = 5, mines = [[4, 2]]
 Output: 2
```

```
 Explanation:
 11111
 11111
 11111
 11111
 11011
 In the above grid, the largest plus sign can only be order 2.  One of them is marked in
bold.

 Example 2:
 Input: N = 2, mines = []
 Output: 1
 Explanation:
 There is no plus sign of order 2, but there is of order 1.

 Example 3:
 Input: N = 1, mines = [[0, 0]]
 Output: 0
 Explanation:
 There is no plus sign, so return 0.

 Note:
 N will be an integer in the range [1, 500].
 mines will have length at most 5000.
 mines[i] will be length 2 and consist of integers in the range [0, N-1].
 (Additionally, programs submitted in C, C++, or C# will be judged with a slightly
smaller time limit.)

 */

public class _764 {
  public static class Solution1 {
    /**Brute force
     *
     * Time: O(N^3)
     * Space: O(mines.length)*/
    public int orderOfLargestPlusSign(int N, int[][] mines) {
      Set<Integer> banned = new HashSet<>();
      for (int[] mine : mines) {
        banned.add(mine[0] * N + mine[1]);
      }
      int result = 0;
      for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++) {
          int k = 0;
          while (k <= row && row < N - k && k <= col && col < N - k
              && !banned.contains((row - k) * N + col)
              && !banned.contains((row + k) * N + col)
              && !banned.contains(row * N + col - k)
              && !banned.contains(row * N + col + k)) {
            k++;
          }
          result = Math.max(result, k);
        }
```

```
      }
      return result;
    }
  }

  public static class Solution2 {
    /**Dp
     *
     * Time: O(N^2)
     * Space: O(N^2)
     * Credit: https://leetcode.com/articles/largest-plus-sign/*/
    public int orderOfLargestPlusSign(int N, int[][] mines) {
      Set<Integer> banned = new HashSet<>();
      for (int[] mine : mines) {
        banned.add(mine[0] * N + mine[1]);
      }

      int[][] dp = new int[N][N];

      for (int row = 0; row < N; row++) {
        int count = 0;
        for (int col = 0; col < N; col++) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = count;
        }

        count = 0;
        for (int col = N - 1; col >= 0; col--) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = Math.min(dp[row][col], count);
        }
      }

      int result = 0;
      for (int col = 0; col < N; col++) {
        int count = 0;
        for (int row = 0; row < N; row++) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = Math.min(dp[row][col], count);
        }

        count = 0;
        for (int row = N - 1; row >= 0; row--) {
          count = banned.contains(row * N + col) ? 0 : count + 1;
          dp[row][col] = Math.min(dp[row][col], count);
          result = Math.max(result, dp[row][col]);
        }
      }
      return result;
    }
  }
}

// ============================================================
```

```java
// FILE 677: _765.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 765. Couples Holding Hands
 *
 * N couples sit in 2N seats arranged in a row and want to hold hands.
 * We want to know the minimum number of swaps so that every couple is sitting side by
side.
 * A swap consists of choosing any two people, then they stand up and switch seats.
 * The people and seats are represented by an integer from 0 to 2N-1, the couples are
numbered in order,
 * the first couple being (0, 1), the second couple being (2, 3), and so on with the last
couple being (2N-2, 2N-1).
 * The couples' initial seating is given by row[i] being the value of the person who is
initially sitting in the i-th seat.

 Example 1:

 Input: row = [0, 2, 1, 3]
 Output: 1
 Explanation: We only need to swap the second (row[1]) and third (row[2]) person.

 Example 2:

 Input: row = [3, 2, 0, 1]
 Output: 0
 Explanation: All couples are already seated side by side.

 Note:
 len(row) is even and in the range of [4, 60].
 row is guaranteed to be a permutation of 0...len(row)-1.
 */

public class _765 {
  public static class Solution1 {
    public int minSwapsCouples(int[] row) {
      int swaps = 0;
      for (int i = 0; i < row.length - 1; i += 2) {
        int coupleValue = row[i] % 2 == 0 ? row[i] + 1 : row[i] - 1;
        if (row[i + 1] != coupleValue) {
          swaps++;
          int coupleIndex = findIndex(row, coupleValue);
          swap(row, coupleIndex, i + 1);
        }
      }
      return swaps;
    }

    private void swap(int[] row, int i, int j) {
      int tmp = row[i];
      row[i] = row[j];
```

```
      row[j] = tmp;
    }

    private int findIndex(int[] row, int value) {
      for (int i = 0; i < row.length; i++) {
        if (row[i] == value) {
          return i;
        }
      }
      return -1;
    }
  }
}

// ============================================================
// FILE 678: _766.java
// ============================================================

package com.fishercoder.solutions;

/**766. Toeplitz Matrix
 *
 * A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same
element.

 Now given an M x N matrix, return True if and only if the matrix is Toeplitz.

 Example 1:

 Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
 Output: True
 Explanation:
 1234
 5123
 9512

 In the above grid, the diagonals are "[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3,
3]", "[4]",
 and in each diagonal all elements are the same, so the answer is True.

 Example 2:

 Input: matrix = [[1,2],[2,2]]
 Output: False
 Explanation:
 The diagonal "[1, 2]" has different elements.

 Note:

 matrix will be a 2D array of integers.
 matrix will have a number of rows and columns in range [1, 20].
 matrix[i][j] will be integers in range [0, 99].

 */
```

```
public class _766 {
  public static class Solution1 {
    public boolean isToeplitzMatrix(int[][] matrix) {
      int m = matrix.length;
      int n = matrix[0].length;
      int i = 0;
      int j = 0;
      int sameVal = matrix[i][j];
      while (++i < m && ++j < n) {
        if (matrix[i][j] != sameVal) {
          return false;
        }
      }

      for (i = 1, j = 0; i < m; i++) {
        int tmpI = i;
        int tmpJ = j;
        sameVal = matrix[i][j];
        while (++tmpI < m && ++tmpJ < n) {
          if (matrix[tmpI][tmpJ] != sameVal) {
            return false;
          }
        }
      }
      for (i = 0, j = 1; j < n; j++) {
        int tmpJ = j;
        int tmpI = i;
        sameVal = matrix[tmpI][tmpJ];
        while (++tmpI < m && ++tmpJ < n) {
          if (matrix[tmpI][tmpJ] != sameVal) {
            return false;
          }
        }
      }
      return true;
    }
  }
}

// ======================================================================
// FILE 679: _767.java
// ======================================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;

/**
 * 767. Reorganize String
 *
 * Given a string S, check if the letters can be rearranged so that two characters that
are adjacent to each other are not the same.
```

```
 If possible, output any possible result.  If not possible, return the empty string.

 Example 1:

 Input: S = "aab"
 Output: "aba"

 Example 2:

 Input: S = "aaab"
 Output: ""

 Note:

 S will consist of lowercase letters and have length in range [1, 500].
 */

public class _767 {
  public static class Solution1 {
    public String reorganizeString(String S) {
      Map<Character, Integer> map = new HashMap<>();
      for (char c : S.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
      }
      int len = S.length();
      for (char c : map.keySet()) {
        if ((len % 2 == 0 && map.get(c) > len / 2) || (len % 2 != 0 && map.get(c) >= len
/ 2 + 2)) {
          return "";
        }
      }
      PriorityQueue<CustChar> queue = new PriorityQueue<>((a, b) -> b.count - a.count);
      for (char c : map.keySet()) {
        queue.offer(new CustChar(c, map.get(c)));
      }

      StringBuilder sb = new StringBuilder();
      while (!queue.isEmpty()) {
        CustChar curr = queue.poll();
        char c = curr.c;
        if (sb.length() > 0 && sb.charAt(sb.length() - 1) != c) {
          sb.append(c);
          if (curr.count > 1) {
            queue.offer(new CustChar(c, curr.count - 1));
          }
        } else if (sb.length() == 0) {
          sb.append(c);
          if (curr.count > 1) {
            queue.offer(new CustChar(c, curr.count - 1));
          }
        } else if (sb.length() > 0 && sb.charAt(sb.length() - 1) == c &&
!queue.isEmpty()) {
          CustChar next = queue.poll();
```

```
          sb.append(next.c);
          if (next.count > 1) {
            queue.offer(new CustChar(next.c, next.count - 1));
          }
          queue.offer(curr);
        }
      }
      return sb.toString();
    }

    class CustChar {
      Character c;
      int count;

      public CustChar(Character c, int count) {
        this.c = c;
        this.count = count;
      }
    }
  }
}

// ============================================================
// FILE 680: _769.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 769. Max Chunks To Make Sorted

 Given an array arr that is a permutation of [0, 1, ..., arr.length - 1], we split the
array into some number of "chunks" (partitions), and individually sort each chunk.
 After concatenating them, the result equals the sorted array.

 What is the most number of chunks we could have made?

 Example 1:

 Input: arr = [4,3,2,1,0]
 Output: 1
 Explanation:
 Splitting into two or more chunks will not return the required result.
 For example, splitting into [4, 3], [2, 1, 0] will result in [3, 4, 0, 1, 2], which
isn't sorted.

 Example 2:

 Input: arr = [1,0,2,3,4]
 Output: 4
 Explanation:
 We can split into two chunks, such as [1, 0], [2, 3, 4].
 However, splitting into [1, 0], [2], [3], [4] is the highest number of chunks possible.
```

```
 Note:

 arr will have length in range [1, 10].
 arr[i] will be a permutation of [0, 1, ..., arr.length - 1].

 */
public class _769 {
  public static class Solution1 {
    /**credit: https://leetcode.com/problems/max-chunks-to-make-
sorted/discuss/113520/Java-solution-left-max-and-right-min.*/
    public int maxChunksToSorted(int[] arr) {
      int len = arr.length;

      int[] maxOfLeft = new int[len];
      maxOfLeft[0] = arr[0];
      for (int i = 1; i < len; i++) {
        maxOfLeft[i] = Math.max(arr[i], maxOfLeft[i - 1]);
      }

      int[] minOfRight = new int[len];
      minOfRight[len - 1] = arr[len - 1];
      for (int i = len - 2; i >= 0; i--) {
        minOfRight[i] = Math.min(minOfRight[i + 1], arr[i]);
      }

      int result = 0;
      for (int i = 0; i < len - 1; i++) {
        if (maxOfLeft[i] <= minOfRight[i + 1]) {
          result++;
        }
      }
      return result + 1;
    }
  }

  public static class Solution2 {
    /**credit: https://leetcode.com/articles/max-chunks-to-make-sorted-i/*/
    public int maxChunksToSorted(int[] arr) {
      int ans = 0;
      int max = 0;
      for (int i = 0; i < arr.length; ++i) {
        max = Math.max(max, arr[i]);
        if (max == i) {
          ans++;
        }
      }
      return ans;
    }
  }
}

// ============================================================
// FILE 681: _771.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 771. Jewels and Stones

 You're given strings J representing the types of stones that are jewels, and S
representing the stones you have.
 Each character in S is a type of stone you have.  You want to know how many of the
stones you have are also jewels.
 The letters in J are guaranteed distinct, and all characters in J and S are letters.
 Letters are case sensitive, so "a" is considered a different type of stone from "A".

 Example 1:
 Input: J = "aA", S = "aAAbbbb"
 Output: 3

 Example 2:
 Input: J = "z", S = "ZZ"
 Output: 0

 Note:
 S and J will consist of letters and have length at most 50.
 The characters in J are distinct.
 */

public class _771 {
  public static class Solution1 {
    public int numJewelsInStones(String J, String S) {
      Set<Character> set = new HashSet<>();
      for (char c : J.toCharArray()) {
        set.add(c);
      }
      int count = 0;
      for (char c : S.toCharArray()) {
        if (set.contains(c)) {
          count++;
        }
      }
      return count;
    }
  }
}

// ============================================================
// FILE 682: _776.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
```

```
/**
 * 776. Split BST
 *
 * Given a Binary Search Tree (BST) with root node root, and a target value V,
 * split the tree into two subtrees where one subtree has nodes that are all smaller or
equal to the target value,
 * while the other subtree has all nodes that are greater than the target value.
 * It's not necessarily the case that the tree contains a node with value V.
 * Additionally, most of the structure of the original tree should remain.
 * Formally, for any child C with parent P in the original tree,
 * if they are both in the same subtree after the split, then node C should still have
the parent P.
 * You should output the root TreeNode of both subtrees after splitting, in any order.

 Example 1:

 Input: root = [4,2,6,1,3,5,7], V = 2
 Output: [[2,1],[4,3,6,null,null,5,7]]

 Explanation:
 Note that root, output[0], and output[1] are TreeNode objects, not arrays.

 The given tree [4,2,6,1,3,5,7] is represented by the following diagram:

       4
    /     \
   2      6
  /  \   / \
 1   3  5   7

 while the diagrams for the outputs are:

   4
 /   \
3     6      and    2
     / \           /
    5   7         1

 Note:

 The size of the BST will not exceed 50.
 The BST is always valid and each node's value is different.
 */

public class _776 {
  public static class Solution1 {
    /** credit: https://discuss.leetcode.com/topic/119481/recursive-java-solution */
    public TreeNode[] splitBST(TreeNode root, int V) {
      TreeNode small = new TreeNode(0);
      TreeNode big = new TreeNode(0);
      split(root, V, small, big);
      return new TreeNode[] {small.right, big.left};
    }
```

```
    private void split(TreeNode root, int v, TreeNode small, TreeNode big) {
      if (root == null) {
        return;
      }
      if (root.val <= v) {
        small.right = root;
        TreeNode right = root.right;
        root.right = null;
        split(right, v, root, big);
      } else {
        big.left = root;
        TreeNode left = root.left;
        root.left = null;
        split(left, v, small, root);
      }
    }
  }

  public static class Solution2 {
    /** credit: https://leetcode.com/articles/split-bst/ */
    public TreeNode[] splitBST(TreeNode root, int V) {
      if (root == null) {
        return new TreeNode[] {null, null};
      } else if (root.val <= V) {
        TreeNode[] result = splitBST(root.right, V);
        root.right = result[0];
        result[0] = root;
        return result;
      } else {
        TreeNode[] result = splitBST(root.left, V);
        root.left = result[1];
        result[1] = root;
        return result;
      }
    }
  }
}

// ============================================================
// FILE 683: _779.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 779. K-th Symbol in Grammar
 *
 * On the first row, we write a 0. Now in every subsequent row,
 * we look at the previous row and replace each occurrence of 0 with 01, and each
```

```
occurrence of 1 with 10.
 * Given row N and index K, return the K-th indexed symbol in row N. (The values of K are
1-indexed.) (1 indexed).

 Examples:
 Input: N = 1, K = 1
 Output: 0

 Input: N = 2, K = 1
 Output: 0

 Input: N = 2, K = 2
 Output: 1

 Input: N = 4, K = 5
 Output: 1

 Explanation:
 row 1: 0
 row 2: 01
 row 3: 0110
 row 4: 01101001

 Note:

 N will be an integer in the range [1, 30].
 K will be an integer in the range [1, 2^(N-1)].
 */

public class _779 {
  public static class Solution1 {
    /**Time: O(2^n)
     * Space: O(2^n)
     * This will result int TLE.*/
    public int kthGrammar(int N, int K) {
      List<List<Integer>> lists = new ArrayList<>();
      lists.add(Arrays.asList(0));
      for (int i = 1; i <= N; i++) {
        List<Integer> curr = new ArrayList<>();
        List<Integer> prev = lists.get(i - 1);
        for (int j = 0; j < prev.size(); j++) {
          if (prev.get(j) == 0) {
            curr.add(0);
            curr.add(1);
          } else {
            curr.add(1);
            curr.add(0);
          }
        }
        lists.add(curr);
      }
      return lists.get(N).get(K - 1);
    }
  }
```

```
  public static class Solution2 {
    /**Time: O(logn)
     * Space: O(1)*/
    public int kthGrammar(int N, int K) {
      return Integer.bitCount(K - 1) % 2;
    }

  }

}

// ============================================================
// FILE 684: _783.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 783. Minimum Distance Between BST Nodes
 *
 * Given a Binary Search Tree (BST) with the root node root, return the minimum
difference between the values of any two different nodes in the tree.

 Example :

 Input: root = [4,2,6,1,3,null,null]
 Output: 1
 Explanation:
 Note that root is a TreeNode object, not an array.

 The given tree [4,2,6,1,3,null,null] is represented by the following diagram:

      4
    /   \
   2     6
  / \
 1  3

 while the minimum difference in this tree is 1, it occurs between node 1 and node 2,
also between node 3 and node 2.

 Note:

 The size of the BST will be between 2 and 100.
 The BST is always valid, each node's value is an integer, and each node's value is
different.
 */

public class _783 {
```

```
  public static class Solution1 {
    public int minDiffInBST(TreeNode root) {
      List<Integer> inorder = new ArrayList<>();
      inorder(root, inorder);
      return findMinDiff(inorder);
    }

    private int findMinDiff(List<Integer> inorder) {
      int minDiff = Integer.MAX_VALUE;
      for (int i = 1; i < inorder.size(); i++) {
        minDiff = Math.min(minDiff, inorder.get(i) - inorder.get(i - 1));
      }
      return minDiff;
    }

    private void inorder(TreeNode root, List<Integer> inorder) {
      if (root == null) {
        return;
      }
      inorder(root.left, inorder);
      inorder.add(root.val);
      inorder(root.right, inorder);
    }
  }
}

// ============================================================
// FILE 685: _784.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * 784. Letter Case Permutation
 *
 * Given a string S, we can transform every letter individually to be lowercase or
uppercase to create another string.
 * Return a list of all possible strings we could create.

 Examples:
 Input: S = "a1b2"
 Output: ["a1b2", "a1B2", "A1b2", "A1B2"]

 Input: S = "3z4"
 Output: ["3z4", "3Z4"]

 Input: S = "12345"
 Output: ["12345"]
```

```
 Note:

 S will be a string with length at most 12.
 S will consist only of letters or digits.

 */

public class _784 {
  public static class Solution1 {
    public List<String> letterCasePermutation(String S) {
      Set<String> result = new HashSet<>();
      result.add(S);
      for (int i = 0; i < S.length(); i++) {
        if (Character.isAlphabetic(S.charAt(i))) {
          Set<String> newResult = new HashSet<>();
          for (String word : result) {
            if (Character.isUpperCase(word.charAt(i))) {
              StringBuilder sb = new StringBuilder();
              for (int j = 0; j < i; j++) {
                sb.append(word.charAt(j));
              }
              sb.append(Character.toLowerCase(word.charAt(i)));
              for (int j = i + 1; j < word.length(); j++) {
                sb.append(word.charAt(j));
              }
              newResult.add(sb.toString());
            } else {
              StringBuilder sb = new StringBuilder();
              for (int j = 0; j < i; j++) {
                sb.append(word.charAt(j));
              }
              sb.append(Character.toUpperCase(word.charAt(i)));
              for (int j = i + 1; j < word.length(); j++) {
                sb.append(word.charAt(j));
              }
              newResult.add(sb.toString());
            }
          }
          result.addAll(newResult);
        }
      }
      return new ArrayList<>(result);
    }
  }
}

// ============================================================
// FILE 686: _788.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;
```

```
/**
 * 788. Rotated Digits
 *
 * X is a good number if after rotating each digit individually by 180 degrees,
 * we get a valid number that is different from X.
 * A number is valid if each digit remains a digit after rotation.
 * 0, 1, and 8 rotate to themselves;
 * 2 and 5 rotate to each other;
 * 6 and 9 rotate to each other,
 * and the rest of the numbers do not rotate to any other number.

 Now given a positive number N, how many numbers X from 1 to N are good?

 Example:
 Input: 10
 Output: 4

 Explanation:
 There are four good numbers in the range [1, 10] : 2, 5, 6, 9.
 Note that 1 and 10 are not good numbers, since they remain unchanged after rotating.

 Note: N  will be in range [1, 10000].
 */
public class _788 {
  public static class Solution1 {
    public int rotatedDigits(int N) {
      int count = 0;
      Map<Character, String> map = new HashMap<>();
      map.put('0', "0");
      map.put('1', "1");
      map.put('8', "8");
      map.put('2', "5");
      map.put('5', "2");
      map.put('6', "9");
      map.put('9', "6");
      for (int i = 1; i <= N; i++) {
        if (isRotatedNumber(i, map)) {
          count++;
        }
      }
      return count;
    }

    private boolean isRotatedNumber(int num, Map<Character, String> map) {
      String originalNum = String.valueOf(num);
      StringBuilder sb = new StringBuilder();
      for (char c : String.valueOf(num).toCharArray()) {
        if (!map.containsKey(c)) {
          return false;
        } else {
          sb.append(map.get(c));
        }
      }
```

```
      return !originalNum.equals(sb.toString());
    }
  }
}

// ============================================================
// FILE 687: _789.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 789. Escape The Ghosts
 *
 * You are playing a simplified Pacman game. You start at the point (0, 0), and your
destination is
 * (target[0], target[1]). There are several ghosts on the map, the i-th ghost starts at
(ghosts[i][0], ghosts[i][1]).
 *
 * Each turn, you and all ghosts simultaneously *may* move in one of 4 cardinal
directions: north, east, west, or
 * south, going from the previous point to a new point 1 unit of distance away.
 *
 * You escape if and only if you can reach the target before any ghost reaches you (for
any given moves the ghosts
 * may take.)  If you reach any square (including the target) at the same time as a
ghost, it doesn't count as an
 * escape.
 *
 * Return True if and only if it is possible to escape.
 */

public class _789 {

    public static class Solution {
        public boolean escapeGhosts(int[][] ghosts, int[] target) {
            int[] currPos = {0, 0};
            int selfDist = getDist(currPos, target);

            for (int[] ghost : ghosts) {
                int ghostDist = getDist(ghost, target);
                if (ghostDist <= selfDist) {
                    return false;
                }
            }
            return true;
        }

        private int getDist(int[] p1, int[] p2) {
            return Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);
        }
    }
}
```

```
// ============================================================
// FILE 688: _791.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 791. Custom Sort String

 S and T are strings composed of lowercase letters. In S, no letter occurs more than
once.

 S was sorted in some custom order previously. We want to permute the characters of T so
that they match the order that S was sorted. More specifically, if x occurs before y in
S, then x should occur before y in the returned string.

 Return any permutation of T (as a string) that satisfies this property.

 Example :
 Input:
 S = "cba"
 T = "abcd"
 Output: "cbad"
 Explanation:
 "a", "b", "c" appear in S, so the order of "a", "b", "c" should be "c", "b", and "a".
 Since "d" does not appear in S, it can be at any position in T. "dcba", "cdba", "cbda"
are also valid outputs.

 Note:

 S has length at most 26, and no character is repeated in S.
 T has length at most 200.
 S and T consist of lowercase letters only.

 */
public class _791 {
  public static class Solution1 {
    public String customSortString(String S, String T) {
      Map<Character, Integer> map = new HashMap<>();
      for (char c : T.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
      }
      StringBuilder sb = new StringBuilder();
      for (char c : S.toCharArray()) {
        if (map.containsKey(c)) {
          int count = map.get(c);
          while (count-- > 0) {
            sb.append(c);
          }
          map.remove(c);
        }
```

```
      }
      for (char c : map.keySet()) {
        int count = map.get(c);
        while (count-- > 0) {
          sb.append(c);
        }
      }
      return sb.toString();
    }
  }
}

// ============================================================
// FILE 689: _796.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 796. Rotate String

 We are given two strings, A and B.

 A shift on A consists of taking string A and moving the leftmost character to the
rightmost position. For example, if A = 'abcde', then it will be 'bcdea' after one shift
on A. Return True if and only if A can become B after some number of shifts on A.

 Example 1:
 Input: A = 'abcde', B = 'cdeab'
 Output: true

 Example 2:
 Input: A = 'abcde', B = 'abced'
 Output: false

 Note:

 A and B will have length at most 100.
 */
public class _796 {
  public static class Solution1 {
    public boolean rotateString(String A, String B) {
      if (A.length() != B.length()) {
        return false;
      }
      for (int i = 0; i < A.length(); i++) {
        if ((A.substring(i) + A.substring(0, i)).equals(B)) {
          return true;
        }
      }
      return false;
    }
  }
}
```

```
// ============================================================
// FILE 690: _799.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 799. Champagne Tower

 We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row.  Each glass holds one cup (250ml) of champagne.
 Then, some champagne is poured in the first glass at the top.
 When the top most glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it.
 When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on.
 (A glass at the bottom row has it's excess champagne fall on the floor.)
 For example, after one cup of champagne is poured, the top most glass is full.
 After two cups of champagne are poured, the two glasses on the second row are half full.
 After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now.
 After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.
 Now after pouring some non-negative integer cups of champagne, return how full the j-th glass in the i-th row is (both i and j are 0 indexed.)

 Example 1:
 Input: poured = 1, query_glass = 1, query_row = 1
 Output: 0.0
 Explanation: We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

 Example 2:
 Input: poured = 2, query_glass = 1, query_row = 1
 Output: 0.5
 Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

 Note:
 poured will be in the range of [0, 10 ^ 9].
 query_glass and query_row will be in the range of [0, 99].

 *
 */
public class _799 {
  public static class Solution1 {
    public double champagneTower(int poured, int queryRow, int queryGlass) {
      double[][] dp = new double[101][101];
      dp[0][0] = poured;
      for (int row = 0; row <= queryRow; row++) {
```

```
        for (int col = 0; col <= row; col++) {
          double quantity = (dp[row][col] - 1.0) / 2.0;
          if (quantity > 0) {
            dp[row + 1][col] += quantity;
            dp[row + 1][col + 1] += quantity;
          }
        }
      }
      return Math.min(dp[queryRow][queryGlass], 1.0);
    }
  }
}

// ============================================================
// FILE 691: _800.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 800. Similar RGB Color

 In the following, every capital letter represents some hexadecimal digit from 0 to f.

 The red-green-blue color "#AABBCC" can be written as "#ABC" in shorthand.  For example,
"#15c" is shorthand for the color "#1155cc".

 Now, say the similarity between two colors "#ABCDEF" and "#UVWXYZ" is -(AB - UV)^2 - (CD
- WX)^2 - (EF - YZ)^2.

 Given the color "#ABCDEF", return a 7 character color that is most similar to #ABCDEF,
and has a shorthand (that is, it can be represented as some "#XYZ"

 Example 1:
 Input: color = "#09f166"
 Output: "#11ee66"
 Explanation:
 The similarity is -(0x09 - 0x11)^2 -(0xf1 - 0xee)^2 - (0x66 - 0x66)^2 = -64 -9 -0 = -73.
 This is the highest among any shorthand color.

 Note:

 color is a string of length 7.
 color is a valid RGB color: for i > 0, color[i] is a hexadecimal digit from 0 to f
 Any answer which has the same (highest) similarity as the best answer will be accepted.
 All inputs and outputs should use lowercase letters, and the output is 7 characters.

 */
public class _800 {
  public static class Solution1 {
```

```
    public String similarRGB(String color) {
      List<String> allShortHandCombinations = computeAllShorthandCombinations();
      int minSimilarity = Integer.MIN_VALUE;
      String result = "";
      for (String candidate : allShortHandCombinations) {
        int similarity = computeSimilarity(candidate, color);
        if (similarity > minSimilarity) {
          result = candidate;
          minSimilarity = similarity;
        }
      }
      return result;
    }

    private int computeSimilarity(String candidate, String color) {
      return -(Integer.parseInt(candidate.substring(1, 3), 16) - Integer.parseInt(
          color.substring(1, 3), 16)) * (Integer.parseInt(candidate.substring(1, 3), 16)
          - Integer.parseInt(color.substring(1, 3), 16))
          - (Integer.parseInt(candidate.substring(3, 5), 16) - Integer.parseInt(
          color.substring(3, 5), 16)) * (Integer.parseInt(candidate.substring(3, 5), 16)
          - Integer.parseInt(color.substring(3, 5), 16))
          - (Integer.parseInt(candidate.substring(5, 7), 16) - Integer.parseInt(
          color.substring(5, 7), 16)) * (Integer.parseInt(candidate.substring(5, 7), 16)
          - Integer.parseInt(color.substring(5, 7), 16));
    }

    private List<String> computeAllShorthandCombinations() {
      List<String> result = new ArrayList<>();
      List<Character> hexNumber = new ArrayList<>(
          Arrays.asList('0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c',
'd', 'e',
              'f'));
      for (int i = 0; i < hexNumber.size(); i++) {
        for (int j = 0; j < hexNumber.size(); j++) {
          for (int k = 0; k < hexNumber.size(); k++) {
            StringBuilder sb = new StringBuilder();
            sb.append("#");
            sb.append(hexNumber.get(i));
            sb.append(hexNumber.get(i));
            sb.append(hexNumber.get(j));
            sb.append(hexNumber.get(j));
            sb.append(hexNumber.get(k));
            sb.append(hexNumber.get(k));
            result.add(sb.toString());
          }
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 692: _804.java
```

```
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 804. Unique Morse Code Words

 International Morse Code defines a standard encoding where each letter is mapped to a
series of dots and dashes, as follows: "a" maps to ".-", "b" maps to "-...", "c" maps to
"-.-.", and so on.

 For convenience, the full table for the 26 letters of the English alphabet is given
below:

 [".-","-...","-.-.","-..",".","..-.","--.","....","..",".---","-.-",".-..","--","-.","--
-",".--.","--.-",".-.","...","-","..-","...-",".--","-..-","-.--","--.."]

 Now, given a list of words, each word can be written as a concatenation of the Morse
code of each letter. For example, "cab" can be written as "-.-.-....-", (which is the
concatenation "-.-." + "-..." + ".-"). We'll call such a concatenation, the
transformation of a word.

 Return the number of different transformations among all words we have.

 Example:
 Input: words = ["gin", "zen", "gig", "msg"]
 Output: 2
 Explanation:
 The transformation of each word is:
 "gin" -> "--...-."
 "zen" -> "--...-."
 "gig" -> "--....-."
 "msg" -> "--....-."

 There are 2 different transformations, "--...-." and "--....-.".

 Note:
 The length of words will be at most 100.
 Each words[i] will have length in range [1, 12].
 words[i] will only consist of lowercase letters.
 */
public class _804 {
  public static class Solution1 {
    public int uniqueMorseRepresentations(String[] words) {
      String[] morseCodes =
          new String[] {".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..",
".---",
              "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.", "...", "-", "..-",
"...-",
              ".--", "-..-", "-.--", "--.."};
      Set<String> concatenation = new HashSet<>();
```

```
      StringBuilder sb = new StringBuilder();
      for (String word : words) {
        sb.setLength(0);
        for (char c : word.toCharArray()) {
          sb.append(morseCodes[c - 'a']);
        }
        concatenation.add(sb.toString());
      }
      return concatenation.size();
    }
  }
}

// ============================================================
// FILE 693: _806.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 806. Number of Lines To Write String

 We are to write the letters of a given string S, from left to right into lines.
 Each line has maximum width 100 units, and if writing a letter would cause the width of
the line to exceed 100 units, it is written on the next line.
 We are given an array widths, an array where widths[0] is the width of 'a', widths[1] is
the width of 'b', ..., and widths[25] is the width of 'z'.

 Now answer two questions:
 how many lines have at least one character from S,
 and what is the width used by the last such line?
 Return your answer as an integer list of length 2.

 Example :
 Input:
 widths = [10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10]
 S = "abcdefghijklmnopqrstuvwxyz"
 Output: [3, 60]
 Explanation:
 All letters have the same length of 10. To write all 26 letters,
 we need two full lines and one line with 60 units.

 Example :
 Input:
 widths = [4,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10,10]
 S = "bbbcccdddaaa"
 Output: [2, 4]
 Explanation:
 All letters except 'a' have the same length of 10, and
 "bbbcccdddaa" will cover 9 * 10 + 2 * 4 = 98 units.
 For the last 'a', it is written on the second line because
 there is only 2 units left in the first line.
 So the answer is 2 lines, plus 4 units in the second line.
```

```
 Note:
 The length of S will be in the range [1, 1000].
 S will only contain lowercase letters.
 widths is an array of length 26.
 widths[i] will be in the range of [2, 10].

 */
public class _806 {
  public static class Solution1 {
    public int[] numberOfLines(int[] widths, String S) {
      int numOfLines = 1;
      int offsetInCurrentLine = 0;
      for (char c : S.toCharArray()) {
        if (offsetInCurrentLine + widths[c - 'a'] < 100) {
          offsetInCurrentLine += widths[c - 'a'];
        } else if (offsetInCurrentLine + widths[c - 'a'] == 100) {
          numOfLines++;
          offsetInCurrentLine = 0;
        } else {
          numOfLines++;
          offsetInCurrentLine = widths[c - 'a'];
        }
      }
      return new int[] {numOfLines, offsetInCurrentLine};
    }
  }
}

// ============================================================
// FILE 694: _811.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 811. Subdomain Visit Count

 A website domain like "discuss.leetcode.com" consists of various subdomains.
 At the top level, we have "com", at the next level, we have "leetcode.com", and at the
lowest level, "discuss.leetcode.com".
 When we visit a domain like "discuss.leetcode.com", we will also visit the parent
domains "leetcode.com" and "com" implicitly.
 Now, call a "count-paired domain" to be a count (representing the number of visits this
domain received), followed by a space,
 followed by the address.
 An example of a count-paired domain might be "9001 discuss.leetcode.com".
 We are given a list cpdomains of count-paired domains. We would like a list of count-
paired domains, (in the same format as the input, and in any order),
 that explicitly counts the number of visits to each subdomain.
```

```
 Example 1:
 Input:
 ["9001 discuss.leetcode.com"]
 Output:
 ["9001 discuss.leetcode.com", "9001 leetcode.com", "9001 com"]
 Explanation:
 We only have one website domain: "discuss.leetcode.com". As discussed above, the
subdomain "leetcode.com" and "com" will also be visited. So they will all be visited 9001
times.

 Example 2:
 Input:
 ["900 google.mail.com", "50 yahoo.com", "1 intel.mail.com", "5 wiki.org"]
 Output:
 ["901 mail.com","50 yahoo.com","900 google.mail.com","5 wiki.org","5 org","1
intel.mail.com","951 com"]
 Explanation:
 We will visit "google.mail.com" 900 times, "yahoo.com" 50 times, "intel.mail.com" once
and "wiki.org" 5 times. For the subdomains, we will visit "mail.com" 900 + 1 = 901 times,
"com" 900 + 50 + 1 = 951 times, and "org" 5 times.

 Notes:

 The length of cpdomains will not exceed 100.
 The length of each domain name will not exceed 100.
 Each address will have either 1 or 2 "." characters.
 The input count in any count-paired domain will not exceed 10000.
 */
public class _811 {
  public static class Solution1 {
    public List<String> subdomainVisits(String[] cpdomains) {
      Map<String, Integer> map = new HashMap<>();
      for (String each : cpdomains) {
        String[] pair = each.split(" ");
        String[] subDomains = pair[1].split("\\.");
        StringBuilder sb = new StringBuilder();
        for (int i = subDomains.length - 1; i >= 0; i--) {
          if (i < subDomains.length - 1) {
            sb.insert(0, ".");
          }
          sb.insert(0, subDomains[i]);
          map.put(sb.toString(), map.getOrDefault(sb.toString(), 0) +
Integer.parseInt(pair[0]));
        }
      }
      List<String> result = new ArrayList<>();
      for (String key : map.keySet()) {
        result.add(map.get(key) + " " + key);
      }
      return result;
    }
  }
}
```

```
// ============================================================
// FILE 695: _814.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 814. Binary Tree Pruning
 *
 * We are given the head node root of a binary tree, where additionally every node's
value is either a 0 or a 1.
 * Return the same tree where every subtree (of the given tree) not containing a 1 has
been removed.
 * (Recall that the subtree of a node X is X, plus every node that is a descendant of X.)
 *
 * Example 1:
 * Input: [1,null,0,0,1]
 * Output: [1,null,0,null,1]
 * Explanation:
 * Only the red nodes satisfy the property "every subtree not containing a 1".
 * The diagram on the right represents the answer.
 * 1                               1
 *  \                               \
 *   0             ---->            0
 *  / \                               \
 * 0   1                               1
 *
 *
 * Example 2:
 * Input: [1,0,1,0,0,0,1]
 * Output: [1,null,1,null,1]
 *              1                 1
 *             / \                 \
 *            0   1    ---->        1
 *           / \  / \                \
 *          0  0 0  1                1
 *
 *
 * Example 3:
 * Input: [1,1,0,1,1,0,1,0]
 * Output: [1,1,0,1,1,null,1]
 *               1                     1
 *              / \                   / \
 *             1   0     ----->      1  0
 *            / \ / \                / \  \
 *           1  1 0 1              1  1   1
 *          /
 *         0
 *
 *
 * Note:
```

```
 *
 * The binary tree will have at most 100 nodes.
 * The value of each node will only be 0 or 1.*/
public class _814 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/binary-tree-
pruning/discuss/122730/C%2B%2BJavaPython-Self-Explaining-Solution-and-2-lines*/
        public TreeNode pruneTree(TreeNode root) {
            if (root == null) {
                return root;
            }
            root.left = pruneTree(root.left);
            root.right = pruneTree(root.right);
            if (root.left == null && root.right == null && root.val == 0) {
                return null;
            }
            return root;
        }
    }
}

// ============================================================
// FILE 696: _819.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 819. Most Common Word

 Given a paragraph and a list of banned words, return the most frequent word that is not
in the list of banned words.
 It is guaranteed there is at least one word that isn't banned, and that the answer is
unique.
 Words in the list of banned words are given in lowercase, and free of punctuation.
 Words in the paragraph are not case sensitive.  The answer is in lowercase.

 Example:
 Input:
 paragraph = "Bob hit a ball, the hit BALL flew far after it was hit."
 banned = ["hit"]
 Output: "ball"
 Explanation:
 "hit" occurs 3 times, but it is a banned word.
 "ball" occurs twice (and no other word does), so it is the most frequent non-banned word
in the paragraph.
 Note that words in the paragraph are not case sensitive,
 that punctuation is ignored (even if adjacent to words, such as "ball,"),
```

```
 and that "hit" isn't the answer even though it occurs more because it is banned.

 Note:

 1 <= paragraph.length <= 1000.
 1 <= banned.length <= 100.
 1 <= banned[i].length <= 10.
 The answer is unique, and written in lowercase (even if its occurrences in paragraph may
have uppercase symbols, and even if it is a proper noun.)
 paragraph only consists of letters, spaces, or the punctuation symbols !?',;.
 Different words in paragraph are always separated by a space.
 There are no hyphens or hyphenated words.
 Words only consist of letters, never apostrophes or other punctuation symbols.
 */
public class _819 {
  public static class Solution1 {
    public String mostCommonWord(String paragraph, String[] banned) {
      Set<String> bannedSet = new HashSet(Arrays.asList(banned));
      String[] words = paragraph.replaceAll("[^a-zA-Z ]",
"").toLowerCase().split("\\s+");
      Map<String, Integer> map = new HashMap<>();
      Arrays.stream(words)
          .filter(word -> !bannedSet.contains(word))
          .forEach(word -> map.put(word, map.getOrDefault(word, 0) + 1));
      String result = "";
      int freq = 0;
      for (String key : map.keySet()) {
        if (map.get(key) > freq) {
          result = key;
          freq = map.get(key);
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 697: _821.java
// ============================================================

package com.fishercoder.solutions;

import java.util.TreeSet;

/**
 * 821. Shortest Distance to a Character

  Given a string S and a character C,
  return an array of integers representing the shortest distance from the character C in
the string.

 Example 1:
```

```
 Input: S = "loveleetcode", C = 'e'
 Output: [3, 2, 1, 0, 1, 0, 0, 1, 2, 2, 1, 0]

 Note:

 S string length is in [1, 10000].
 C is a single character, and guaranteed to be in string S.
 All letters in S and C are lowercase.

 */
public class _821 {

  public static class Solution1 {
    public int[] shortestToChar(String S, char C) {
      int[] result = new int[S.length()];
      TreeSet<Integer> cIndices = new TreeSet();
      for (int i = 0; i < S.length(); i++) {
        if (S.charAt(i) == C) {
          cIndices.add(i);
        }
      }
      for (int i = 0; i < S.length(); i++) {
        int leftDist = Integer.MAX_VALUE;
        if (cIndices.floor(i) != null) {
          leftDist = Math.abs(cIndices.floor(i) - i);
        }
        int rightDist = Integer.MAX_VALUE;
        if (cIndices.ceiling(i) != null) {
          rightDist = Math.abs(cIndices.ceiling(i) - i);
        }
        result[i] = Math.min(leftDist, rightDist);
      }
      return result;
    }
  }
}

// ================================================================
// FILE 698: _824.java
// ================================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 824. Goat Latin
 *
 * A sentence S is given, composed of words separated by spaces. Each word consists of
lowercase and uppercase letters only.
 *
 * We would like to convert the sentence to "Goat Latin" (a made-up language similar to
```

```
Pig Latin.)
 *
 * The rules of Goat Latin are as follows:
 *
 *     If a word begins with a vowel (a, e, i, o, or u), append "ma" to the end of the
word.
 *     For example, the word 'apple' becomes 'applema'.
 *
 *     If a word begins with a consonant (i.e. not a vowel), remove the first letter and
append it to the end, then add "ma".
 *     For example, the word "goat" becomes "oatgma".
 *
 *     Add one letter 'a' to the end of each word per its word index in the sentence,
starting with 1.
 *     For example, the first word gets "a" added to the end, the second word gets "aa"
added to the end and so on.
 *
 * Return the final sentence representing the conversion from S to Goat Latin.
 *
 * Example 1:
 *
 * Input: "I speak Goat Latin"
 * Output: "Imaa peaksmaaa oatGmaaaa atinLmaaaaa"
 *
 * Example 2:
 *
 * Input: "The quick brown fox jumped over the lazy dog"
 * Output: "heTmaa uickqmaaa rownbmaaaa oxfmaaaaa umpedjmaaaaaa overmaaaaaaa hetmaaaaaaaa
azylmaaaaaaaaa ogdmaaaaaaaaaa"
 *
 * Notes:
 *
 *     S contains only uppercase, lowercase and spaces. Exactly one space between each
word.
 *     1 <= S.length <= 100.
 */
public class _824 {

  public static class Solution1 {
    public String toGoatLatin(String S) {
      StringBuilder sb = new StringBuilder();
      Set<Character> vowels =
          new HashSet(Arrays.asList('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'));
      String[] words = S.split(" ");
      for (int i = 0; i < words.length; i++) {
        if (vowels.contains(words[i].charAt(0))) {
          String newWord = words[i] + "ma";
          int j = i + 1;
          while (j-- > 0) {
            newWord += 'a';
          }
          sb.append(newWord);
          sb.append(" ");
        } else {
```

```
          StringBuilder subSb = new StringBuilder(words[i].substring(1));
          subSb.append(words[i].charAt(0));
          subSb.append("ma");
          int j = i + 1;
          while (j-- > 0) {
            subSb.append("a");
          }
          sb.append(subSb.toString());
          sb.append(" ");
        }
      }
      return sb.substring(0, sb.length() - 1);
    }
  }
}

// ============================================================
// FILE 699: _830.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 830. Positions of Large Groups
 *
 * In a string S of lowercase letters, these letters form consecutive groups of the same
character.
 * For example, a string like S = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z" and
"yy".
 * Call a group large if it has 3 or more characters.  We would like the starting and
ending positions of every large group.
 * The final answer should be in lexicographic order.
 *
 * Example 1:
 * Input: "abbxxxxzzy"
 * Output: [[3,6]]
 * Explanation: "xxxx" is the single large group with starting  3 and ending positions 6.
 *
 * Example 2:
 * Input: "abc"
 * Output: []
 * Explanation: We have "a","b" and "c" but no large group.
 *
 * Example 3:
 * Input: "abcdddeeeeaabbbcd"
 * Output: [[3,5],[6,9],[12,14]]
 *
 * Note:  1 <= S.length <= 1000
 */
public class _830 {
```

```
  public static class Solution1 {
    public List<List<Integer>> largeGroupPositions(String S) {
      List<List<Integer>> result = new ArrayList<>();
      char[] chars = S.toCharArray();
      for (int i = 0; i < chars.length; ) {
        char first = chars[i];
        int j = i + 1;
        while (j < chars.length && first == chars[j]) {
          j++;
        }
        if ((j - i) >= 3) {
          result.add(Arrays.asList(i, j - 1));
        }
        i = j;
      }
      return result;
    }
  }
}

// ============================================================
// FILE 700: _832.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 832. Flipping an Image
 *
 * Given a binary matrix A, we want to flip the image horizontally, then invert it, and
return the resulting image.
 * To flip an image horizontally means that each row of the image is reversed.  For
example, flipping [1, 1, 0] horizontally results in [0, 1, 1].
 * To invert an image means that each 0 is replaced by 1, and each 1 is replaced by 0.
For example, inverting [0, 1, 1] results in [1, 0, 0].
 *
 * Example 1:
 * Input: [[1,1,0],[1,0,1],[0,0,0]]
 * Output: [[1,0,0],[0,1,0],[1,1,1]]
 * Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
 * Then, invert the image: [[1,0,0],[0,1,0],[1,1,1]]
 *
 * Example 2:
 * Input: [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]
 * Output: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
 * Explanation: First reverse each row: [[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]].
 * Then invert the image: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
 *
 * Notes:
 * 1 <= A.length = A[0].length <= 20
 * 0 <= A[i][j] <= 1
 */
public class _832 {
  public static class Solution1 {
```

```
    public int[][] flipAndInvertImage(int[][] A) {
      int m = A.length;
      int n = A[0].length;
      int[][] result = new int[m][n];
      for (int i = 0; i < m; i++) {
        int[] flipped = (reverse(A[i]));
        result[i] = invert(flipped);
      }
      return result;
    }

    private int[] invert(int[] flipped) {
      int[] result = new int[flipped.length];
      for (int i = 0; i < flipped.length; i++) {
        if (flipped[i] == 0) {
          result[i] = 1;
        } else {
          result[i] = 0;
        }
      }
      return result;
    }

    private int[] reverse(int[] nums) {
      for (int i = 0, j = nums.length - 1; i < j; i++, j--) {
        int tmp = nums[i];
        nums[i] = nums[j];
        nums[j] = tmp;
      }
      return nums;
    }
  }
}

// ============================================================
// FILE 701: _836.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 836. Rectangle Overlap
 *
 * A rectangle is represented as a list [x1, y1, x2, y2], where (x1, y1) are the
coordinates of
 * its bottom-left corner, and (x2, y2) are the coordinates of its top-right corner.
 * Two rectangles overlap if the area of their intersection is positive.
 * To be clear, two rectangles that only touch at the corner or edges do not overlap.
 * Given two (axis-aligned) rectangles, return whether they overlap.
 *
 * Example 1:
 * Input: rec1 = [0,0,2,2], rec2 = [1,1,3,3]
 * Output: true
 *
```

```
 * Example 2:
 * Input: rec1 = [0,0,1,1], rec2 = [1,0,2,1]
 * Output: false
 *
 * Notes:
 * Both rectangles rec1 and rec2 are lists of 4 integers.
 * All coordinates in rectangles will be between -10^9 and 10^9.
 * */
public class _836 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/rectangle-
overlap/discuss/132340/C%2B%2BJavaPython-1-line-Solution-1D-to-2D*/
        public boolean isRectangleOverlap(int[] rec1, int[] rec2) {
            return rec1[0] < rec2[2] && rec2[0] < rec1[2] && rec1[1] < rec2[3] && rec2[1]
< rec1[3];
        }
    }
}

// ============================================================
// FILE 702: _840.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 840. Magic Squares In Grid
 *
 * A 3 x 3 magic square is a 3 x 3 grid filled with distinct numbers from 1 to 9 such
that each row,
 * column, and both diagonals all have the same sum.
 *
 * Given an grid of integers, how many 3 x 3 "magic square" subgrids are there?  (Each
subgrid is contiguous).
 *
 * Example 1:
 *
 * Input: [[4,3,8,4],
 *         [9,5,1,9],
 *         [2,7,6,2]]
 *
 * Output: 1
 *
 * Explanation:
 * The following subgrid is a 3 x 3 magic square:
 * 438
 * 951
 * 276
 *
 * while this one is not:
 * 384
```

```
 * 519
 * 762
 *
 * In total, there is only one magic square inside the given grid.
 * Note:
 *
 * 1 <= grid.length <= 10
 * 1 <= grid[0].length <= 10
 * 0 <= grid[i][j] <= 15
 */
public class _840 {
  public static class Solution1 {
    public int numMagicSquaresInside(int[][] grid) {
      int m = grid.length;
      int n = grid[0].length;
      int count = 0;
      for (int i = 0; i < m - 2; i++) {
        for (int j = 0; j < n - 2; j++) {
          Set<Integer> set = new HashSet<>();
          int sum = grid[i][j] + grid[i][j + 1] + grid[i][j + 2];
          if (sum == grid[i + 1][j] + grid[i + 1][j + 1] + grid[i + 1][j + 2]
              && sum == grid[i + 2][j] + grid[i + 2][j + 1] + grid[i + 2][j + 2]

              && sum == grid[i][j] + grid[i + 1][j] + grid[i + 2][j]
              && sum == grid[i][j + 1] + grid[i + 1][j + 1] + grid[i + 2][j + 1]
              && sum == grid[i][j + 2] + grid[i + 1][j + 2] + grid[i + 2][j + 2]

              && sum == grid[i][j] + grid[i + 1][j + 1] + grid[i + 2][j + 2]
              && sum == grid[i][j + 2] + grid[i + 1][j + 1] + grid[i + 2][j]

              && set.add(grid[i][j]) && isLegit(grid[i][j])
              && set.add(grid[i][j + 1]) && isLegit(grid[i][j + 1])
              && set.add(grid[i][j + 2]) && isLegit(grid[i][j + 2])
              && set.add(grid[i + 1][j]) && isLegit(grid[i + 1][j])
              && set.add(grid[i + 1][j + 1]) && isLegit(grid[i + 1][j + 1])
              && set.add(grid[i + 1][j + 2]) && isLegit(grid[i + 1][j + 2])
              && set.add(grid[i + 2][j]) && isLegit(grid[i + 2][j])
              && set.add(grid[i + 2][j + 1]) && isLegit(grid[i + 2][j + 1])
              && set.add(grid[i + 2][j + 2]) && isLegit(grid[i + 2][j + 2])
          ) {
            count++;
          }
        }
      }
      return count;
    }

    private boolean isLegit(int num) {
      return num <= 9 && num >= 1;
    }
  }
}

// ============================================================
```

```
// FILE 703: _844.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 844. Backspace String Compare
 *
 * Given two strings S and T, return if they are equal when both are typed into empty
text editors. # means a backspace character.
 *
 * Example 1:
 * Input: S = "ab#c", T = "ad#c"
 * Output: true
 * Explanation: Both S and T become "ac".
 *
 *  Example 2:
 * Input: S = "ab##", T = "c#d#"
 * Output: true
 * Explanation: Both S and T become "".
 *
 *  Example 3:
 * Input: S = "a##c", T = "#a#c"
 * Output: true
 * Explanation: Both S and T become "c".
 *
 * Example 4:
 * Input: S = "a#c", T = "b"
 * Output: false
 * Explanation: S becomes "c" while T becomes "b".
 * Note:
 *
 * 1 <= S.length <= 200
 * 1 <= T.length <= 200
 * S and T only contain lowercase letters and '#' characters.
 * Follow up:
 *
 * Can you solve it in O(N) time and O(1) space?
 */
public class _844 {
  public static class Solution1 {
    public boolean backspaceCompare(String S, String T) {
      String processedS = process(S);
      String processedT = process(T);
      return processedS.equals(processedT);
    }

    private String process(String str) {
      StringBuilder sb = new StringBuilder();
      for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) == '#') {
          if (sb.length() > 0) {
            sb.deleteCharAt(sb.length() - 1);
          }
```

```
        } else {
          sb.append(str.charAt(i));
        }
      }
      return sb.reverse().toString();
    }
  }
}

// ============================================================
// FILE 704: _849.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 849. Maximize Distance to Closest Person
 *
 * In a row of seats, 1 represents a person sitting in that seat, and 0 represents that
the seat is empty.
 * There is at least one empty seat, and at least one person sitting.
 * Alex wants to sit in the seat such that the distance between him and the closest
person to him is maximized.
 * Return that maximum distance to closest person.
 *
 * Example 1:
 * Input: [1,0,0,0,1,0,1]
 * Output: 2
 * Explanation:
 * If Alex sits in the second open seat (seats[2]), then the closest person has distance
2.
 * If Alex sits in any other open seat, the closest person has distance 1.
 * Thus, the maximum distance to the closest person is 2.
 *
 * Example 2:
 * Input: [1,0,0,0]
 * Output: 3
 * Explanation:
 * If Alex sits in the last seat, the closest person is 3 seats away.
 * This is the maximum distance possible, so the answer is 3.
 * Note:
 *
 * 1 <= seats.length <= 20000
 * seats contains only 0s or 1s, at least one 0, and at least one 1.
 * */
public class _849 {
    public static class Solution1 {
        int maxDist = 0;

        public int maxDistToClosest(int[] seats) {
            for (int i = 0; i < seats.length; i++) {
                if (seats[i] == 0) {
                    extend(seats, i);
                }
```

```
            }
            return maxDist;
        }

        private void extend(int[] seats, int position) {
            int left = position - 1;
            int right = position + 1;
            int leftMinDistance = 1;
            while (left >= 0) {
                if (seats[left] == 0) {
                    leftMinDistance++;
                    left--;
                } else {
                    break;
                }
            }
            int rightMinDistance = 1;
            while (right < seats.length) {
                if (seats[right] == 0) {
                    rightMinDistance++;
                    right++;
                } else {
                    break;
                }
            }
            int maxReach = 0;
            if (position == 0) {
                maxReach = rightMinDistance;
            } else if (position == seats.length - 1) {
                maxReach = leftMinDistance;
            } else {
                maxReach = Math.min(leftMinDistance, rightMinDistance);
            }
            maxDist = Math.max(maxDist, maxReach);
        }
    }
}

// ============================================================
// FILE 705: _852.java
// ============================================================

package com.fishercoder.solutions;

/**852. Peak Index in a Mountain Array
 *
 * Let's call an array A a mountain if the following properties hold:
 *
 * A.length >= 3
 * There exists some 0 < i < A.length - 1 such that A[0] < A[1] < ... A[i-1] < A[i] >
A[i+1] > ... > A[A.length - 1]
 * Given an array that is definitely a mountain, return any i such that A[0] < A[1] < ...
A[i-1] < A[i] > A[i+1] > ... > A[A.length - 1].
 *
```

```
 * Example 1:
 *
 * Input: [0,1,0]
 * Output: 1
 * Example 2:
 *
 * Input: [0,2,1,0]
 * Output: 1
 * Note:
 *
 * 3 <= A.length <= 10000
 * 0 <= A[i] <= 10^6
 * A is a mountain, as defined above.*/
public class _852 {
    public static class Solution1 {
        public int peakIndexInMountainArray(int[] A) {
            for (int i = 1; i < A.length - 1; i++) {
                if (A[i] > A[i + 1]) {
                    return i;
                }
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 706: _859.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 859. Buddy Strings
 *
 * Given two strings A and B of lowercase letters, return true if and only if we can swap
two letters in A so that the result equals B.
 *
 *
 *
 * Example 1:
 *
 * Input: A = "ab", B = "ba"
 * Output: true
 * Example 2:
 *
 * Input: A = "ab", B = "ab"
 * Output: false
 * Example 3:
 *
 * Input: A = "aa", B = "aa"
```

```
 * Output: true
 * Example 4:
 *
 * Input: A = "aaaaaaabc", B = "aaaaaaacb"
 * Output: true
 * Example 5:
 *
 * Input: A = "", B = "aa"
 * Output: false
 *
 *
 * Note:
 *
 * 0 <= A.length <= 20000
 * 0 <= B.length <= 20000
 * A and B consist only of lowercase letters.
 */
public class _859 {
  public static class Solution1 {
    public boolean buddyStrings(String A, String B) {
      if (A.length() != B.length()) {
        return false;
      }
      Character c1 = null;
      Character c2 = null;
      Set<Character> set = new HashSet<>();
      int count = 0;
      for (int i = 0; i < A.length(); i++) {
        if (A.charAt(i) != B.charAt(i)) {
          if (count > 2) {
            return false;
          }
          if (c1 == null) {
            c1 = B.charAt(i);
            c2 = A.charAt(i);
            count++;
            continue;
          }
          if (c1 != A.charAt(i) || c2 != B.charAt(i)) {
            return false;
          }
          count++;
        }
        set.add(A.charAt(i));
      }
      return count == 2 || (count == 0 && set.size() < A.length());
    }
  }
}

// ============================================================
// FILE 707: _860.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 860. Lemonade Change
 *
 * At a lemonade stand, each lemonade costs $5.
 *
 * Customers are standing in a queue to buy from you,
 * and order one at a time (in the order specified by bills).
 * Each customer will only buy one lemonade and pay with either a $5, $10, or $20 bill.
 * You must provide the correct change to each customer,
 * so that the net transaction is that the customer pays $5.
 * Note that you don't have any change in hand at first.
 * Return true if and only if you can provide every customer with correct change.
 *
 * Example 1:
 * Input: [5,5,5,10,20]
 * Output: true
 * Explanation:
 * From the first 3 customers, we collect three $5 bills in order.
 * From the fourth customer, we collect a $10 bill and give back a $5.
 * From the fifth customer, we give a $10 bill and a $5 bill.
 * Since all customers got correct change, we output true.
 *
 * Example 2:
 * Input: [5,5,10]
 * Output: true
 *
 * Example 3:
 * Input: [10,10]
 * Output: false
 *
 * Example 4:
 * Input: [5,5,10,10,20]
 * Output: false
 * Explanation:
 * From the first two customers in order, we collect two $5 bills.
 * For the next two customers in order, we collect a $10 bill and give back a $5 bill.
 * For the last customer, we can't give change of $15 back because we only have two $10
bills.
 * Since not every customer received correct change, the answer is false.
 *
 *
 * Note:
 * 0 <= bills.length <= 10000
 * bills[i] will be either 5, 10, or 20.
 */
public class _860 {
  public static class Solution1 {
    public boolean lemonadeChange(int[] bills) {
      Map<Integer, Integer> map = new HashMap<>();
```

```
      for (int bill : bills) {
        if (bill == 5) {
          map.put(5, map.getOrDefault(5, 0) + 1);
        } else if (bill == 10) {
          if (!map.containsKey(5)) {
            return false;
          } else {
            map.put(5, map.get(5) - 1);
            if (map.get(5) == 0) {
              map.remove(5);
            }
            map.put(10, map.getOrDefault(10, 0) + 1);
          }
        } else {
          if (!map.containsKey(5)) {
            return false;
          } else {
            if (!map.containsKey(10)) {
              if (!map.containsKey(5) || map.get(5) < 3) {
                return false;
              } else {
                map.put(5, map.get(5) - 3);
                if (map.get(5) == 0) {
                  map.remove(5);
                }
              }
            } else {
              if (!map.containsKey(5)) {
                return false;
              } else {
                map.put(5, map.get(5) - 1);
                if (map.get(5) == 0) {
                  map.remove(5);
                }
                map.put(10, map.get(10) - 1);
                if (map.get(10) == 0) {
                  map.remove(10);
                }
              }
            }
          }
          map.put(20, map.getOrDefault(20, 0) + 1);
        }
      }
      return true;
    }
  }
}

// ============================================================
// FILE 708: _867.java
// ============================================================

package com.fishercoder.solutions;
```

```
/**
 * 867. Transpose Matrix
 *
 * Given a matrix A, return the transpose of A.
 * The transpose of a matrix is the matrix flipped over it's main diagonal, switching the
row and column indices of the matrix.
 *
 * Example 1:
 *
 * Input: [[1,2,3],[4,5,6],[7,8,9]]
 * Output: [[1,4,7],[2,5,8],[3,6,9]]
 * Example 2:
 *
 * Input: [[1,2,3],[4,5,6]]
 * Output: [[1,4],[2,5],[3,6]]
 *
 * Note:
 *
 * 1 <= A.length <= 1000
 * 1 <= A[0].length <= 1000
 * */
public class _867 {
    public static class Solution1 {
        public int[][] transpose(int[][] A) {
            int[][] result = new int[A[0].length][A.length];
            for (int i = 0; i < A.length; i++) {
                for (int j = 0; j < A[0].length; j++) {
                    result[j][i] = A[i][j];
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 709: _868.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 868. Binary Gap
 *
 * Given a positive integer N, find and return the longest distance between two
consecutive 1's in the binary representation of N.
 *
 * If there aren't two consecutive 1's, return 0.
 *
 * Example 1:
```

```
 * Input: 22
 * Output: 2
 * Explanation:
 * 22 in binary is 0b10110.
 * In the binary representation of 22, there are three ones, and two consecutive pairs of
1's.
 * The first consecutive pair of 1's have distance 2.
 * The second consecutive pair of 1's have distance 1.
 * The answer is the largest of these two distances, which is 2.
 *
 * Example 2:
 * Input: 5
 * Output: 2
 * Explanation:
 * 5 in binary is 0b101.
 *
 * Example 3:
 * Input: 6
 * Output: 1
 * Explanation:
 * 6 in binary is 0b110.
 *
 * Example 4:
 * Input: 8
 * Output: 0
 * Explanation:
 * 8 in binary is 0b1000.
 * There aren't any consecutive pairs of 1's in the binary representation of 8, so we
return 0.
 *
 * Note:
 * 1 <= N <= 10^9
 */
public class _868 {
  public static class Solution1 {
    public int binaryGap(int N) {
      String bin = Integer.toBinaryString(N);
      List<Integer> oneIndexes = new ArrayList<>();
      for (int i = 0; i < bin.length(); i++) {
        if (bin.charAt(i) == '1') {
          oneIndexes.add(i);
        }
      }
      int maxGap = 0;
      for (int i = 0; i < oneIndexes.size() - 1; i++) {
        maxGap = Math.max(oneIndexes.get(i + 1) - oneIndexes.get(i), maxGap);
      }
      return maxGap;
    }
  }
}

// ============================================================
// FILE 710: _872.java
```

```
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 872. Leaf-Similar Trees
 *
 * Consider all the leaves of a binary tree.  From left to right order, the values of
those leaves form a leaf value sequence.
 *
 *          3
 *         / \
 *        5   1
 *       / \ / \
 *      6  2 9 8
 *        / \
 *       7  4
 *
 * For example, in the given tree above, the leaf value sequence is (6, 7, 4, 9, 8).
 *
 * Two binary trees are considered leaf-similar if their leaf value sequence is the same.
 *
 * Return true if and only if the two given trees with head nodes root1 and root2 are
leaf-similar.
 *
 * Note:
 *
 * Both of the given trees will have between 1 and 100 nodes.
 */
public class _872 {
  public static class Solution1 {
    public boolean leafSimilar(TreeNode root1, TreeNode root2) {
      List<Integer> leaves1 = new ArrayList<>();
      List<Integer> leaves2 = new ArrayList<>();
      preorder(root1, leaves1);
      preorder(root2, leaves2);
      return leaves1.equals(leaves2);
    }

    private void preorder(TreeNode root,
        List<Integer> leaves) {
      if (root == null) {
        return;
      }
      if (root.left == null && root.right == null) {
        leaves.add(root.val);
      }
      preorder(root.left, leaves);
      preorder(root.right, leaves);
    }
```

```
  }
}

// ============================================================
// FILE 711: _876.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 876. Middle of the Linked List
 *
 * Given a non-empty, singly linked list with head node head, return a middle node of
linked list.
 *
 * If there are two middle nodes, return the second middle node.
 *
 * Example 1:
 *
 * Input: [1,2,3,4,5]
 * Output: Node 3 from this list (Serialization: [3,4,5])
 * The returned node has value 3.  (The judge's serialization of this node is [3,4,5]).
 * Note that we returned a ListNode object ans, such that:
 * ans.val = 3, ans.next.val = 4, ans.next.next.val = 5, and ans.next.next.next = NULL.
 *
 *  Example 2:
 *
 * Input: [1,2,3,4,5,6]
 * Output: Node 4 from this list (Serialization: [4,5,6])
 * Since the list has two middle nodes with values 3 and 4, we return the second one.
 *
 * Note:
 *
 * The number of nodes in the given list will be between 1 and 100.
 */
public class _876 {
  public static class Solution1 {
    public ListNode middleNode(ListNode head) {
      ListNode fast = head;
      ListNode slow = head;
      while (fast != null && fast.next != null) {
        fast = fast.next.next;
        slow = slow.next;
      }
      return slow;
    }
  }
}

// ============================================================
// FILE 712: _884.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 884. Uncommon Words from Two Sentences
 *
 * We are given two sentences A and B.  (A sentence is a string of space separated words.
Each word consists only of lowercase letters.)
 *
 * A word is uncommon if it appears exactly once in one of the sentences, and does not
appear in the other sentence.
 *
 * Return a list of all uncommon words.
 *
 * You may return the list in any order.
 *
 *
 *
 * Example 1:
 *
 * Input: A = "this apple is sweet", B = "this apple is sour"
 * Output: ["sweet","sour"]
 * Example 2:
 *
 * Input: A = "apple apple", B = "banana"
 * Output: ["banana"]
 *
 *
 * Note:
 *
 * 0 <= A.length <= 200
 * 0 <= B.length <= 200
 * A and B both contain only spaces and lowercase letters.
 */
public class _884 {
  public static class Solution1 {
    public String[] uncommonFromSentences(String A, String B) {
      Map<String, Integer> map = new HashMap<>();
      for (String word : A.split(" ")) {
        map.put(word, map.getOrDefault(word, 0) + 1);
      }

      for (String word : B.split(" ")) {
        map.put(word, map.getOrDefault(word, 0) + 1);
      }
      List<String> result = new ArrayList<>();
      for (String key : map.keySet()) {
        if (map.get(key) == 1) {
          result.add(key);
```

```
        }
      }
      String[] strs = new String[result.size()];
      result.toArray(strs);
      return strs;
    }
  }
}

// ============================================================
// FILE 713: _885.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 885. Spiral Matrix III
 *
 * On a 2 dimensional grid with R rows and C columns, we start at (r0, c0) facing east.
 * Here, the north-west corner of the grid is at the first row and column, and the south-
east corner of the grid is at the last row and column.
 * Now, we walk in a clockwise spiral shape to visit every position in this grid.
 * Whenever we would move outside the boundary of the grid, we continue our walk outside
the grid (but may return to the grid boundary later.)
 * Eventually, we reach all R * C spaces of the grid.
 * Return a list of coordinates representing the positions of the grid in the order they
were visited.
 *
 * Example 1:
 * Input: R = 1, C = 4, r0 = 0, c0 = 0
 * Output: [[0,0],[0,1],[0,2],[0,3]]
 *
 * Example 2:
 * Input: R = 5, C = 6, r0 = 1, c0 = 4
 * Output: [[1,4],[1,5],[2,5],[2,4],[2,3],[1,3],[0,3],[0,4],[0,5],[3,5],[3,4],[3,3],
[3,2],[2,2],[1,2],[0,2],[4,5],[4,4],[4,3],[4,2],[4,1],[3,1],[2,1],[1,1],[0,1],[4,0],
[3,0],[2,0],[1,0],[0,0]]
 *
 * Note:
 * 1 <= R <= 100
 * 1 <= C <= 100
 * 0 <= r0 < R
 * 0 <= c0 < C
 * */
public class _885 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/spiral-matrix-iii/discuss/158977/Java-
15-lines-concise-solution-with-comments*/
        public int[][] spiralMatrixIII(int R, int C, int r0, int c0) {
            int[] directions = new int[]{0, 1, 0, -1, 0};
            int[][] result = new int[R * C][2];
            int i = 0;
            result[i++] = new int[]{r0, c0};
            int len = 0;
```

```
            int d = 0;
            while (i < R * C) {
                if (d == 0 || d == 2) {
                    //plus one when moving east or west
                    len++;
                }
                for (int k = 0; k < len; k++) {
                    r0 += directions[d];
                    c0 += directions[d + 1];
                    if (r0 >= 0 && r0 < R && c0 >= 0 && c0 < C) {
                        result[i++] = new int[]{r0, c0};
                    }
                }
                d = (d + 1) % 4;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 714: _888.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashSet;

/**
 * 888. Fair Candy Swap
 *
 * Alice and Bob have candy bars of different sizes: A[i] is the size of the i-th bar of
candy that Alice has, and B[j]
 * is the size of the j-th bar of candy that Bob has.
 *
 * Since they are friends, they would like to exchange one candy bar each so that after
the exchange, they both have the
 * same total amount of candy.  (The total amount of candy a person has is the sum of the
sizes of candy bars they have.)
 *
 * Return an integer array ans where ans[0] is the size of the candy bar that Alice must
exchange, and ans[1] is the
 * size of the candy bar that Bob must exchange.
 *
 * If there are multiple answers, you may return any one of them.  It is guaranteed an
answer exists.
 *
 *
 *
 * Example 1:
 *
 * Input: A = [1,1], B = [2,2]
 * Output: [1,2]
 * Example 2:
```

```
 *
 * Input: A = [1,2], B = [2,3]
 * Output: [1,2]
 * Example 3:
 *
 * Input: A = [2], B = [1,3]
 * Output: [2,3]
 * Example 4:
 *
 * Input: A = [1,2,5], B = [2,4]
 * Output: [5,4]
 */

public class _888 {
    public static class Solution1 {
        public int[] fairCandySwap(int[] A, int[] B) {
            int aSum = 0;
            int bSum = 0;
            int diff = 0;
            int[] ans = new int[2];
            for (int bar : A) {
                aSum += bar;
            }
            for (int bar : B) {
                bSum += bar;
            }
            diff = aSum - bSum;
            HashSet<Integer> set = new HashSet<>();
            for (int bar : A) {
                set.add(bar);
            }
            for (int bar : B) {
                if (set.contains(bar + diff / 2)) {
                    ans[0] = bar + diff / 2;
                    ans[1] = bar;
                    break;
                }
            }
            return ans;
        }
    }
}

// ============================================================
// FILE 715: _890.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
```

```
import java.util.Set;

/**
 * 890. Find and Replace Pattern
 *
 * You have a list of words and a pattern, and you want to know which words in words
matches the pattern.
 * A word matches the pattern if there exists a permutation of letters p so that after
replacing every letter x in the pattern with p(x), we get the desired word.
 * (Recall that a permutation of letters is a bijection from letters to letters: every
letter maps to another letter, and no two letters map to the same letter.)
 * Return a list of the words in words that match the given pattern.
 * You may return the answer in any order.
 *
 *  Example 1:
 *
 * Input: words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"
 * Output: ["mee","aqq"]
 * Explanation: "mee" matches the pattern because there is a permutation {a -> m, b -> e,
...}.
 * "ccc" does not match the pattern because {a -> c, b -> c, ...} is not a permutation,
 * since a and b map to the same letter.
 *
 * Note:
 * 1 <= words.length <= 50
 * 1 <= pattern.length = words[i].length <= 20
 */
public class _890 {
  public static class Solution1 {
    public List<String> findAndReplacePattern(String[] words, String pattern) {
      List<String> result = new ArrayList<>();
      for (String word : words) {
        Map<Character, Character> map = new HashMap<>();
        Set<Character> set = new HashSet<>();
        boolean match = true;
        for (int i = 0; i < pattern.length(); i++) {
          if (map.containsKey(pattern.charAt(i))) {
            if (word.charAt(i) != map.get(pattern.charAt(i))) {
              match = false;
              break;
            }
          } else {
            map.put(pattern.charAt(i), word.charAt(i));
            if (!set.add(word.charAt(i))) {
              match = false;
            }
          }
        }
        if (match) {
          result.add(word);
        }
      }
      return result;
    }
```

```
  }
}

// ============================================================
// FILE 716: _893.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 893. Groups of Special-Equivalent Strings
 *
 * You are given an array A of strings.
 * A move onto S consists of swapping any two even indexed characters of S, or any two
odd indexed characters of S.
 * Two strings S and T are special-equivalent if after any number of moves onto S, S ==
T.
 * For example, S = "zzxy" and T = "xyzz" are special-equivalent because we may make the
moves "zzxy" -> "xzzy" -> "xyzz"
 * that swap S[0] and S[2], then S[1] and S[3].
 *
 * Now, a group of special-equivalent strings from A is a non-empty subset of A such
that:
 * Every pair of strings in the group are special equivalent, and;
 * The group is the largest size possible (ie., there isn't a string S not in the group
such that S is special equivalent to every string in the group)
 * Return the number of groups of special-equivalent strings from A.
 *
 * Example 1:
 * Input: ["abcd","cdab","cbad","xyzz","zzxy","zzyx"]
 * Output: 3
 * Explanation:
 * One group is ["abcd", "cdab", "cbad"], since they are all pairwise special equivalent,
and none of the other strings are all pairwise special equivalent to these.
 * The other two groups are ["xyzz", "zzxy"] and ["zzyx"].  Note that in particular,
"zzxy" is not special equivalent to "zzyx".
 *
 * Example 2:
 * Input: ["abc","acb","bac","bca","cab","cba"]
 * Output: 3
 *
 * Note:
 * 1 <= A.length <= 1000
 * 1 <= A[i].length <= 20
 * All A[i] have the same length.
 * All A[i] consist of only lowercase letters.
 * */
public class _893 {
    public static class Solution1 {
```

```
        /**my original solution, a bit lengthy:
         * generate a unique signaure as key for each equivelant group and sum them up*/
        public int numSpecialEquivGroups(String[] A) {
            return
Arrays.stream(A).map(this::getCommonKey).collect(Collectors.toSet()).size();
        }

        private String getCommonKey(String word) {
            char[] oddIndexed = new char[word.length() / 2];
            char[] evenIndexed = new char[word.length() / 2 + (word.length() % 2 == 0 ? 0
: 1)];
            char[] array = word.toCharArray();
            for (int i = 0; i < array.length - 1; i += 2) {
                evenIndexed[i / 2] = array[i];
                oddIndexed[i / 2] = array[i + 1];
            }
            if (word.length() % 2 != 0) {
                evenIndexed[evenIndexed.length - 1] = array[array.length - 1];
            }
            Arrays.sort(oddIndexed);
            Arrays.sort(evenIndexed);
            return new StringBuffer().append(new String(evenIndexed)).append(new
String(oddIndexed)).toString();
        }
    }

    public static class Solution2 {
        /**
         * more concise solution: https://leetcode.com/problems/groups-of-special-
equivalent-strings/discuss/163413/Java-Concise-Set-Solution
         * but somehow a bit slower than mine: 12 ms vs 7ms
         * I guess due to the problem constraint and this: "1 <= A[i].length <= 20" to
have made this problem simpler
         */
        public int numSpecialEquivGroups(String[] A) {
            Set<String> set = new HashSet<>();
            for (String str : A) {
                int[] odd = new int[26];
                int[] even = new int[26];
                for (int i = 0; i < str.length(); i++) {
                    if (i % 2 == 0) {
                        even[str.charAt(i) - 'a']++;
                    } else {
                        odd[str.charAt(i) - 'a']++;
                    }
                }
                String key = Arrays.toString(even) + Arrays.toString(odd);
                set.add(key);
            }
            return set.size();
        }
    }
}
```

```java
// ============================================================
// FILE 717: _896.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 896. Monotonic Array
 *
 * An array is monotonic if it is either monotone increasing or monotone decreasing.
 *
 * An array A is monotone increasing if for all i <= j, A[i] <= A[j].
 * An array A is monotone decreasing if for all i <= j, A[i] >= A[j].
 *
 * Return true if and only if the given array A is monotonic.
 *
 * Example 1:
 * Input: [1,2,2,3]
 * Output: true
 *
 * Example 2:
 * Input: [6,5,4,4]
 * Output: true
 *
 * Example 3:
 * Input: [1,3,2]
 * Output: false
 *
 * Example 4:
 * Input: [1,2,4,5]
 * Output: true
 *
 * Example 5:
 * Input: [1,1,1]
 * Output: true
 *
 * Note:
 *
 * 1 <= A.length <= 50000
 * -100000 <= A[i] <= 100000
 */
public class _896 {
    public static class Solution1 {
        public boolean isMonotonic(int[] A) {
            int i = 0;
            for (; i < A.length - 1; i++) {
                if (A[i] <= A[i + 1]) {
                    continue;
                } else {
                    break;
                }
            }
            if (i == A.length - 1) {
                return true;
```

```
            }
            i = 0;
            for (; i < A.length - 1; i++) {
                if (A[i] >= A[i + 1]) {
                    continue;
                } else {
                    break;
                }
            }
            return i == A.length - 1;
        }
    }
}

// ============================================================
// FILE 718: _897.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 897. Increasing Order Search Tree
 *
 * Given a tree, rearrange the tree in in-order so that the leftmost node in the tree is
now the root of the tree, and every node has no left child and only 1 right child.
 *
 * Example 1:
 * Input: [5,3,6,2,4,null,8,1,null,null,null,7,9]
 *
 *        5
 *       / \
 *     3    6
 *    / \    \
 *   2   4    8
 *  /        / \
 * 1        7   9
 *
 * Output: [1,null,2,null,3,null,4,null,5,null,6,null,7,null,8,null,9]
 *
 *  1
 *   \
 *    2
 *     \
 *      3
 *       \
 *        4
 *         \
 *          5
 *           \
 *            6
```

```
 *             \
 *              7
 *               \
 *                8
 *                 \
 *                  9
 * Note:
 *
 * The number of nodes in the given tree will be between 1 and 100.
 * Each node will have a unique integer value from 0 to 1000.
 *
 */
public class _897 {
  public static class Solution1 {
    public TreeNode increasingBST(TreeNode root) {
      List<Integer> inorderList = new ArrayList<>();
      inorderTraversal(root, inorderList);
      return constructTree(inorderList);
    }

    private TreeNode constructTree(List<Integer> inorderList) {
      if (inorderList.isEmpty() || inorderList.size() == 0) {
        return null;
      }
      TreeNode root = new TreeNode(inorderList.get(0));
      TreeNode tmp = root;
      for (int i = 1; i < inorderList.size(); i++) {
        tmp.right = new TreeNode(inorderList.get(i));
        tmp = tmp.right;
      }
      return root;
    }

    private void inorderTraversal(TreeNode root, List<Integer> inorderList) {
      if (root == null) {
        return;
      }
      if (root.left != null) {
        inorderTraversal(root.left, inorderList);
      }
      inorderList.add(root.val);
      if (root.right != null) {
        inorderTraversal(root.right, inorderList);
      }
    }
  }
}

// ============================================================
// FILE 719: _900.java
// ============================================================

package com.fishercoder.solutions;
```

```
/**
 * 900. RLE Iterator
 *
 * Write an iterator that iterates through a run-length encoded sequence.
 *
 * The iterator is initialized by RLEIterator(int[] A), where A is a run-length encoding
of some sequence.
 * More specifically, for all even i, A[i] tells us the number of times that the non-
negative integer value A[i+1] is repeated in the sequence.
 *
 * The iterator supports one function: next(int n),
 * which exhausts the next n elements (n >= 1) and returns the last element exhausted in
this way.
 * If there is no element left to exhaust, next returns -1 instead.
 *
 * For example, we start with A = [3,8,0,9,2,5], which is a run-length encoding of the
 * sequence [8,8,8,5,5].
 * This is because the sequence can be read as "three eights, zero nines, two fives".
 *
 * Example 1:
 *
 * Input: ["RLEIterator","next","next","next","next"], [[[3,8,0,9,2,5]],[2],[1],[1],[2]]
 * Output: [null,8,8,5,-1]
 * Explanation:
 * RLEIterator is initialized with RLEIterator([3,8,0,9,2,5]).
 * This maps to the sequence [8,8,8,5,5].
 * RLEIterator.next is then called 4 times:
 * .next(2) exhausts 2 terms of the sequence, returning 8.  The remaining sequence is now
[8, 5, 5].
 * .next(1) exhausts 1 term of the sequence, returning 8.  The remaining sequence is now
[5, 5].
 * .next(1) exhausts 1 term of the sequence, returning 5.  The remaining sequence is now
[5].
 * .next(2) exhausts 2 terms, returning -1.  This is because the first term exhausted was
5,
 *  but the second term did not exist.  Since the last term exhausted does not exist, we
return -1.
 *
 * Note:
 *
 * 0 <= A.length <= 1000
 * A.length is an even integer.
 * 0 <= A[i] <= 10^9
 * There are at most 1000 calls to RLEIterator.next(int n) per test case.
 * Each call to RLEIterator.next(int n) will have 1 <= n <= 10^9.
 */
public class _900 {
  public static class Solution1 {
    public static class RLEIterator {

      int index;
      int[] array;

      public RLEIterator(int[] A) {
```

```
        index = 0;
        array = A;
      }

      public int next(int n) {
        int lastElement = -1;
        while (n > 0 && index < array.length) {
          if (array[index] > n) {
            array[index] -= n;
            lastElement = array[index + 1];
            break;
          } else if (array[index] == n) {
            array[index] = 0;
            lastElement = array[index + 1];
            index += 2;
            break;
          } else {
            n -= array[index];
            index += 2;
          }
        }
        return lastElement;
      }

    }
  }
}

// ============================================================
// FILE 720: _905.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 905. Sort Array By Parity
 *
 * Given an array A of non-negative integers, return an array consisting of all the even
elements of A, followed by all the odd elements of A.
 *
 * You may return any answer array that satisfies this condition.
 *
 * Example 1:
 *
 * Input: [3,1,2,4]
 * Output: [2,4,3,1]
 * The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.
 *
 * Note:
 *
 * 1 <= A.length <= 5000
 * 0 <= A[i] <= 5000
 * */
public class _905 {
```

```
    public static class Solution1 {
        public int[] sortArrayByParity(int[] A) {
            for (int i = 0, j = A.length - 1; i < j; i++) {
                if (A[i] % 2 == 0) {
                    continue;
                } else {
                    while (j > i && A[j] % 2 != 0) {
                        j--;
                    }
                    swap(A, i, j);
                }
            }
            return A;
        }

        private void swap(int[] A, int i, int j) {
            int tmp = A[i];
            A[i] = A[j];
            A[j] = tmp;
        }
    }
}

// ============================================================
// FILE 721: _908.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 908. Smallest Range I
 *
 * Given an array A of integers, for each integer A[i] we may choose any x with -K <= x
<= K, and add x to A[i].
 *
 * After this process, we have some array B.
 *
 * Return the smallest possible difference between the maximum value of B and the minimum
value of B.
 *
 *
 *
 * Example 1:
 *
 * Input: A = [1], K = 0
 * Output: 0
 * Explanation: B = [1]
 * Example 2:
 *
 * Input: A = [0,10], K = 2
 * Output: 6
 * Explanation: B = [2,8]
```

```
 * Example 3:
 *
 * Input: A = [1,3,6], K = 3
 * Output: 0
 * Explanation: B = [3,3,3] or B = [4,4,4]
 *
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * 0 <= A[i] <= 10000
 * 0 <= K <= 10000
 */
public class _908 {
    public static class Solution1 {
        public int smallestRangeI(int[] A, int K) {
          Arrays.sort(A);
          int smallestPlus = A[0] + K;
          int biggestMinus = A[A.length - 1] - K;
          int diff = biggestMinus - smallestPlus;
          if (diff > 0) {
              return diff;
          } else {
              return 0;
          }
      }
  }

  public static class Solution2 {
      public int smallestRangeI(int[] A, int K) {
          int min = A[0];
          int max = A[0];

          for (int k : A) {
            min = Math.min(min, k);
            max = Math.max(max, k);
          }

          return Math.max(max - min - 2 * K, 0);
      }
  }
}

// ============================================================
// FILE 722: _912.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 912. Sort an Array
 *
```

```
 * Given an array of integers nums, sort the array in ascending order.
 *
 * Example 1:
 *
 * Input: [5,2,3,1]
 * Output: [1,2,3,5]
 * Example 2:
 *
 * Input: [5,1,1,2,0,0]
 * Output: [0,0,1,1,2,5]
 *
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * -50000 <= A[i] <= 50000
 * */
public class _912 {
    public static class Solution1 {
        public int[] sortArray(int[] nums) {
            Arrays.sort(nums);
            return nums;
        }
    }
}

// ============================================================
// FILE 723: _914.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 914. X of a Kind in a Deck of Cards
 *
 * In a deck of cards, each card has an integer written on it.
 * Return true if and only if you can choose X >= 2 such that it is possible to split the
entire deck into 1 or more groups of cards, where:
 * Each group has exactly X cards.
 * All the cards in each group have the same integer.
 *
 * Example 1:
 * Input: [1,2,3,4,4,3,2,1]
 * Output: true
 * Explanation: Possible partition [1,1],[2,2],[3,3],[4,4]
 *
 * Example 2:
 * Input: [1,1,1,2,2,2,3,3]
 * Output: false
 * Explanation: No possible partition.
 *
```

```
 * Example 3:
 * Input: [1]
 * Output: false
 * Explanation: No possible partition.
 *
 * Example 4:
 * Input: [1,1]
 * Output: true
 * Explanation: Possible partition [1,1]
 *
 * Example 5:
 * Input: [1,1,2,2,2,2]
 * Output: true
 * Explanation: Possible partition [1,1],[2,2],[2,2]
 */
public class _914 {
    public static class Solution1 {
        public boolean hasGroupsSizeX(int[] deck) {
            //Size too small for partitions
            if (deck.length < 2) {
                return false;
            }

            //Track repetitions of values in deck array
            Map<Integer, Integer> mapReps = new HashMap<>();
            for (int card : deck) {
                if (!mapReps.containsKey(card)) {
                    mapReps.put(card, 1);
                } else {
                    mapReps.put(card, mapReps.get(card) + 1);
                }
            }

            //Create array of map values
            int num = 0;
            int[] arrReps = new int[mapReps.size()];
            for (Map.Entry<Integer, Integer> e : mapReps.entrySet()) {
                arrReps[num++] = e.getValue();
            }

            //Find greatest common denominator
            num = arrGCD(arrReps, arrReps.length);

            //If gcd of all repetitions is greater than 1, it's partitionable.
            return num > 1;
        }

        private int gcd(int a, int b) {
            return b == 0 ? a : gcd(b, a % b);
        }

        private int arrGCD(int[] arr, int n) {
            int result = arr[0];
            for (int i = 1; i < n; i++) {
```

```
                result = gcd(arr[i], result);
            }

            return result;
        }
    }
}

// ============================================================
// FILE 724: _917.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 917. Reverse Only Letters
 *
 * Given a string S, return the "reversed" string where all characters
 * that are not a letter stay in the same place, and all letters reverse their positions.
 *
 * Example 1:
 *
 * Input: "ab-cd"
 * Output: "dc-ba"
 * Example 2:
 *
 * Input: "a-bC-dEf-ghIj"
 * Output: "j-Ih-gfE-dCba"
 * Example 3:
 *
 * Input: "Test1ng-Leet=code-Q!"
 * Output: "Qedo1ct-eeLg=ntse-T!"
 *
 *
 * Note:
 *
 * S.length <= 100
 * 33 <= S[i].ASCIIcode <= 122
 * S doesn't contain \ or "
 * */
public class _917 {
    public static class Solution1 {
        public String reverseOnlyLetters(String S) {
            char[] array = S.toCharArray();
            for (int i = 0, j = array.length - 1; i < j;) {
                if (Character.isLetter(array[i]) && Character.isLetter(array[j])) {
                    char temp = array[i];
                    array[i++] = array[j];
                    array[j--] = temp;
                } else if (Character.isLetter(array[i])) {
                    j--;
                } else if (Character.isLetter(array[j])) {
                    i++;
                } else {
```

```
                    i++;
                    j--;
                }
            }
            return new String(array);
        }
    }
}

// ============================================================
// FILE 725: _922.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 922. Sort Array By Parity II
 *
 * Given an array A of non-negative integers, half of the integers in A are odd, and half
of the integers are even.
 * Sort the array so that whenever A[i] is odd, i is odd; and whenever A[i] is even, i is
even.
 * You may return any answer array that satisfies this condition.
 *
 * Example 1:
 *
 * Input: [4,2,5,7]
 * Output: [4,5,2,7]
 * Explanation: [4,7,2,5], [2,5,4,7], [2,7,4,5] would also have been accepted.
 *
 *
 * Note:
 * 2 <= A.length <= 20000
 * A.length % 2 == 0
 * 0 <= A[i] <= 1000
 * */
public class _922 {
    public static class Solution1 {
        public int[] sortArrayByParityII(int[] A) {
            for (int i = 0, j = 1; i < A.length - 1 && j < A.length;) {
                if (A[i] % 2 != 0 && A[j] % 2 == 0) {
                    int tmp = A[i];
                    A[i] = A[j];
                    A[j] = tmp;
                    i += 2;
                    j += 2;
                }
                while (i < A.length - 1 && A[i] % 2 == 0) {
                    i += 2;
                }
                while (j < A.length && A[j] % 2 != 0) {
                    j += 2;
                }
            }
```

```
            return A;
        }
    }
}

// ============================================================
// FILE 726: _925.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 925. Long Pressed Name
 *
 * Your friend is typing his name into a keyboard.
 * Sometimes, when typing a character c, the key might get long pressed,
 * and the character will be typed 1 or more times.
 *
 * You examine the typed characters of the keyboard.
 * Return True if it is possible that it was your friends name,
 * with some characters (possibly none) being long pressed.
 *
 * Example 1:
 *
 * Input: name = "alex", typed = "aaleex"
 * Output: true
 * Explanation: 'a' and 'e' in 'alex' were long pressed.
 * Example 2:
 *
 * Input: name = "saeed", typed = "ssaaedd"
 * Output: false
 * Explanation: 'e' must have been pressed twice, but it wasn't in the typed output.
 * Example 3:
 *
 * Input: name = "leelee", typed = "lleeelee"
 * Output: true
 * Example 4:
 *
 * Input: name = "laiden", typed = "laiden"
 * Output: true
 * Explanation: It's not necessary to long press any character.
 *
 * Note:
 * name.length <= 1000
 * typed.length <= 1000
 * The characters of name and typed are lowercase letters.
 * */
public class _925 {
    public static class Solution1 {
        public boolean isLongPressedName(String name, String typed) {
            int i = 0;
            for (int j = 0; i < name.length() && j < typed.length(); i++) {
                if (name.charAt(i) != typed.charAt(j)) {
                    return false;
```

```
                } else if (i < name.length() - 1 && name.charAt(i) != name.charAt(i + 1))
{
                    j++;
                    while (j < typed.length() && name.charAt(i) == typed.charAt(j)) {
                        j++;
                    }
                } else {
                    j++;
                }

            }
            return i == name.length();
        }
    }
}

// ============================================================
// FILE 727: _929.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 929. Unique Email Addresses
 *
 * Every email consists of a local name and a domain name, separated by the @ sign.
 * For example, in alice@leetcode.com, alice is the local name, and leetcode.com is the
domain name.
 * Besides lowercase letters, these emails may contain '.'s or '+'s.
 * If you add periods ('.') between some characters in the local name part of an email
address, mail sent there will be forwarded to the same address without dots in the local
name.
 * For example, "alice.z@leetcode.com" and "alicez@leetcode.com" forward to the same
email address.  (Note that this rule does not apply for domain names.)
 * If you add a plus ('+') in the local name, everything after the first plus sign will
be ignored.
 * This allows certain emails to be filtered, for example m.y+name@email.com will be
forwarded to my@email.com.  (Again, this rule does not apply for domain names.)
 * It is possible to use both of these rules at the same time.
 * Given a list of emails, we send one email to each address in the list.  How many
different addresses actually receive mails?
 *
 * Example 1:
 *
 * Input:
["test.email+alex@leetcode.com","test.e.mail+bob.cathy@leetcode.com","testemail+david@lee
.tcode.com"]
 * Output: 2
 *
```

```
 * Explanation: "testemail@leetcode.com" and "testemail@lee.tcode.com" actually receive
mails
 *
 * Note:
 *
 * 1 <= emails[i].length <= 100
 * 1 <= emails.length <= 100
 * Each emails[i] contains exactly one '@' character.
 * All local and domain names are non-empty.
 * Local names do not start with a '+' character.
 * */
public class _929 {
    public static class Solution1 {
        public int numUniqueEmails(String[] emails) {
            Map<String, Set<String>> map = new HashMap<>();
            for (String email : emails) {
                String[] parts = email.split("@");
                if (!map.containsKey(parts[1])) {
                    map.put(parts[1], new HashSet<>());
                }
                String filteredLocalName = parts[0].substring(0, parts[0].indexOf('+'));
                filteredLocalName = filteredLocalName.replace(".", "");
                map.get(parts[1]).add(filteredLocalName);
            }
            return map.keySet().stream().mapToInt(key -> map.get(key).size()).sum();
        }
    }
}

// ============================================================
// FILE 728: _933.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Deque;
import java.util.LinkedList;

/**
 * 933. Number of Recent Calls
 *
 * Write a class RecentCounter to count recent requests.
 * It has only one method: ping(int t), where t represents some time in milliseconds.
 * Return the number of pings that have been made from 3000 milliseconds ago until now.
 * Any ping with time in [t - 3000, t] will count, including the current ping.
 * It is guaranteed that every call to ping uses a strictly larger value of t than
before.
 *
 * Example 1:
 * Input: inputs = ["RecentCounter","ping","ping","ping","ping"], inputs = [[],[1],[100],
[3001],[3002]]
 * Output: [null,1,2,3,3]
 *
 * Note:
```

```
 * Each test case will have at most 10000 calls to ping.
 * Each test case will call ping with strictly increasing values of t.
 * Each call to ping will have 1 <= t <= 10^9.
 * */
public class _933 {
    public static class Solution1 {
        public static class RecentCounter {

            Deque<Integer> deque;

            public RecentCounter() {
                deque = new LinkedList<>();
            }

            public int ping(int t) {
                while (!deque.isEmpty() && t - deque.getFirst() > 3000) {
                    deque.removeFirst();
                }
                deque.addLast(t);
                return deque.size();
            }
        }
    }
}

// ============================================================
// FILE 729: _935.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 935. Knight Dialer
 *
 * A chess knight can move as indicated in the chess diagram below:
 *
 * https://assets.leetcode.com/uploads/2018/10/12/knight.png
 *
 * |---|---|---|
 * | 1 | 2 | 3 |
 * |---|---|---|
 * | 4 | 5 | 6 |
 * |---|---|---|
 * | 7 | 8 | 9 |
 * |---|---|---|
 * | x | 0 | x |
 * |---|---|---|
 *
 * This time, we place our chess knight on any numbered key of a phone pad (indicated
above), and the knight makes N-1 hops.
 * Each hop must be from one key to another numbered key.
 *
 * Each time it lands on a key (including the initial placement of the knight), it
presses the number of that key, pressing N digits total.
```

```
 *
 * How many distinct numbers can you dial in this manner?
 *
 * Since the answer may be large, output the answer modulo 10^9 + 7.
 *
 * Note:
 *  * 1 <= N <= 5000
 */

public class _935 {
    /*
     * The intuition is to calculate the number of ways
     * we can reach a key k after i hops, based on the number of ways we can reach keys x
after i-1 hops
     * s.t. the knight can move from x to k in one move
     * For example,
     * We can reach 6 in 3 ways after 1 hop (1 -> 6, 7 -> 6 or 0 -> 6)
     * We can reach 8 in 2 ways after 1 hop (1 -> 8 or 3 -> 8)
     * Thus, we can reach 1 in 5 ways after 2 hops:
     * . 1. 1 -> 6 -> 1
     * . 2. 7 -> 6 -> 1
     * . 3. 0 -> 6 -> 1
     *   4. 1 -> 8 -> 1
     *   5. 3 -> 8 -> 1
     */
    public static class Solution1 {
        private static final int MOD = 1000_000_007;

        // whereFromHere[i] is an array of keys that can be reached from the ith digit
        private static final int[][] whereFromHere = {
            {4, 6},    {6, 8},  {7, 9},    {4, 8},  // 0, 1, 2, 3
            {3, 9, 0}, {},      {1, 7, 0},          // 4, 5, 6
            {2, 6},    {1, 3},  {2, 4}              // 7, 8, 9
        };

        public int knightDialer(int N) {
            // a[i] is the number of ways we can end up on the ith digit
            // The initial array is for N = 1, i.e. for 0 hops.
            long[] a = new long[]{1, 1, 1, 1, 1, 1, 1, 1, 1, 1};

            // Simulate N - 1 hops
            for (int i = 0; i < N - 1; ++i) {
                long[] tmp = new long[10];

                // For each digit
                for (int j = 0; j < 10; j++) {
                    // Which other digits can we reach?
                    for (int k : whereFromHere[j]) {
                        tmp[j] = (tmp[j] + a[k]) % MOD;
                    }
                }

                // Sanity checks based on symmetry of the keypad
                assert tmp[1] == tmp[3];
```

```
                assert tmp[4] == tmp[6];
                assert tmp[7] == tmp[9];

                a = tmp;
            }

            long ans = 0;
            for (long k : a) {
                ans = (ans + k) % MOD;
            }

            return (int) ans;
        }
    }
}

// ============================================================
// FILE 730: _937.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

/**
 * 937. Reorder Log Files
 *
 * You have an array of logs.  Each log is a space delimited string of words.
 *
 * For each log, the first word in each log is an alphanumeric identifier.  Then, either:
 *
 * Each word after the identifier will consist only of lowercase letters, or;
 * Each word after the identifier will consist only of digits.
 * We will call these two varieties of logs letter-logs and digit-logs.  It is guaranteed
that each log has at least one word after its identifier.
 *
 * Reorder the logs so that all of the letter-logs come before any digit-log.
 * The letter-logs are ordered lexicographically ignoring identifier, with the identifier
used in case of ties.
 * The digit-logs should be put in their original order.
 *
 * Return the final order of the logs.
 *
 * Example 1:
 *
 * Input: ["a1 9 2 3 1","g1 act car","zo4 4 7","ab1 off key dog","a8 act zoo"]
 * Output: ["g1 act car","a8 act zoo","ab1 off key dog","a1 9 2 3 1","zo4 4 7"]
 *
 * Note:
 * 0 <= logs.length <= 100
 * 3 <= logs[i].length <= 100
 * logs[i] is guaranteed to have an identifier, and a word after the identifier.
```

```
 */
public class _937 {
  public static class Solution1 {
    public String[] reorderLogFiles(String[] logs) {
      TreeMap<String, String> letterLogMap = new TreeMap<>();
      List<String> digitLogList = new ArrayList<>();
      for (String log : logs) {
        int firstSpaceIndex = log.indexOf(' ');
        String id = log.substring(0, firstSpaceIndex);
        if (Character.isAlphabetic(log.charAt(firstSpaceIndex + 1))) {
          String key = log.substring(firstSpaceIndex + 1) + id;
          letterLogMap.put(key, log);
        } else {
          digitLogList.add(log);
        }
      }
      String[] reorderedLogs = new String[logs.length];
      int i = 0;
      for (String key : letterLogMap.keySet()) {
        reorderedLogs[i++] = letterLogMap.get(key);
      }
      for (String log : digitLogList) {
        reorderedLogs[i++] = log;
      }
      return reorderedLogs;
    }
  }
}

// ============================================================
// FILE 731: _938.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.ArrayList;
import java.util.List;

/**
 * 938. Range Sum of BST
 *
 * Given the root node of a binary search tree, return the sum of values of all nodes
with value between L and R (inclusive).
 *
 * The binary search tree is guaranteed to have unique values.
 *
 * Example 1:
 *
 * Input: root = [10,5,15,3,7,null,18], L = 7, R = 15
 * Output: 32
 *
 *
 * Example 2:
```

```
 *
 * Input: root = [10,5,15,3,7,13,18,1,null,6], L = 6, R = 10
 * Output: 23
 *
 * Note:
 *
 * The number of nodes in the tree is at most 10000.
 * The final answer is guaranteed to be less than 2^31.
 */
public class _938 {
  public static class Solution1 {
    public int rangeSumBST(TreeNode root, int L, int R) {
      if (root == null) {
        return 0;
      }
      List<Integer> list = new ArrayList<>();
      dfs(root, L, R, list);
      return list.stream().mapToInt(num -> num).sum();
    }

    private void dfs(TreeNode root, int l, int r, List<Integer> list) {
      if (root == null) {
        return;
      }
      if (root.val <= r && root.val >= l) {
        list.add(root.val);
      }
      dfs(root.left, l, r, list);
      dfs(root.right, l, r, list);
    }
  }
}

// ============================================================
// FILE 732: _941.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 941. Valid Mountain Array
 *
 * Given an array A of integers, return true if and only if it is a valid mountain array.
 *
 * Recall that A is a mountain array if and only if:
 *
 * A.length >= 3
 * There exists some i with 0 < i < A.length - 1 such that:
 * A[0] < A[1] < ... A[i-1] < A[i]
 * A[i] > A[i+1] > ... > A[B.length - 1]
 *
 *
 * Example 1:
 *
```

```
 * Input: [2,1]
 * Output: false
 * Example 2:
 *
 * Input: [3,5,5]
 * Output: false
 * Example 3:
 *
 * Input: [0,3,2,1]
 * Output: true
 *
 *
 * Note:
 *
 * 0 <= A.length <= 10000
 * 0 <= A[i] <= 10000
 * */
public class _941 {
    public static class Solution1 {
        public boolean validMountainArray(int[] A) {
            int i = 0;
            for (; i < A.length - 1; i++) {
                if (A[i] < A[i + 1]) {
                    continue;
                } else if (A[i] == A[i + 1]) {
                    return false;
                } else {
                    break;
                }
            }
            if (i == 0 || i >= A.length - 1) {
                return false;
            }
            for (; i < A.length - 1; i++) {
                if (A[i] > A[i + 1]) {
                    continue;
                } else {
                    return false;
                }
            }
            return i == A.length - 1;
        }
    }
}

// ==================================================================
// FILE 733: _942.java
// ==================================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Queue;
```

```
/**
 * 942. DI String Match
 *
 * Given a string S that only contains "I" (increase) or "D" (decrease), let N =
S.length.
 *
 * Return any permutation A of [0, 1, ..., N] such that for all i = 0, ..., N-1:
 *
 * If S[i] == "I", then A[i] < A[i+1]
 * If S[i] == "D", then A[i] > A[i+1]
 *
 * Example 1:
 * Input: "IDID"
 * Output: [0,4,1,3,2]
 *
 * Example 2:
 * Input: "III"
 * Output: [0,1,2,3]
 *
 * Example 3:
 * Input: "DDI"
 * Output: [3,2,0,1]
 *
 * Note:
 * 1 <= S.length <= 10000
 * S only contains characters "I" or "D".
 */
public class _942 {
  public static class Solution1 {
    public int[] diStringMatch(String S) {
      Queue<Integer> deque = new ArrayDeque<>();
      for (int i = 0; i <= S.length(); i++) {
        deque.add(i);
      }
      int[] result = new int[S.length() + 1];
      for (int i = 0; i <= S.length(); i++) {
        if (i == S.length()) {
          result[i] = ((ArrayDeque<Integer>) deque).pollLast();
        } else if (S.charAt(i) == 'I') {
          result[i] = ((ArrayDeque<Integer>) deque).pollFirst();
        } else {
          result[i] = ((ArrayDeque<Integer>) deque).pollLast();
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 734: _944.java
// ============================================================

package com.fishercoder.solutions;
```

```
/**
 * 944. Delete Columns to Make Sorted
 *
 * We are given an array A of N lowercase letter strings, all of the same length.
 * Now, we may choose any set of deletion indices, and for each string, we delete all the characters in those indices.
 * For example, if we have an array A = ["abcdef","uvwxyz"] and deletion indices {0, 2, 3},
 * then the final array after deletions is ["bef", "vyz"], and the remaining columns of A are ["b","v"], ["e","y"],
 * and ["f","z"].  (Formally, the c-th column is [A[0][c], A[1][c], ..., A[A.length-1][c]].)
 *
 * Suppose we chose a set of deletion indices D such that after deletions, each remaining column in A is in non-decreasing sorted order.
 *
 * Return the minimum possible value of D.length.
 *
 * Example 1:
 *
 * Input: ["cba","daf","ghi"]
 * Output: 1
 * Explanation:
 * After choosing D = {1}, each column ["c","d","g"] and ["a","f","i"] are in non-decreasing sorted order.
 * If we chose D = {}, then a column ["b","a","h"] would not be in non-decreasing sorted order.
 * Example 2:
 *
 * Input: ["a","b"]
 * Output: 0
 * Explanation: D = {}
 * Example 3:
 *
 * Input: ["zyx","wvu","tsr"]
 * Output: 3
 * Explanation: D = {0, 1, 2}
 *
 *
 * Note:
 *
 * 1 <= A.length <= 100
 * 1 <= A[i].length <= 1000
 * */
public class _944 {
  public static class Solution1 {
    public int minDeletionSize(String[] A) {
      if (A == null || A.length == 0) {
        return 0;
      }
      int deletion = 0;
      for (int i = 0; i < A[0].length(); i++) {
        for (int j = 0; j < A.length - 1; j++) {
```

```
          if (A[j].charAt(i) > A[j + 1].charAt(i)) {
            deletion++;
            break;
          }
        }
      }
      return deletion;
    }
  }
}

// ============================================================
// FILE 735: _950.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;

/**
 * 950. Reveal Cards In Increasing Order
 *
 * In a deck of cards, every card has a unique integer.  You can order the deck in any
order you want.
 * Initially, all the cards start face down (unrevealed) in one deck.
 * Now, you do the following steps repeatedly, until all cards are revealed:
 * Take the top card of the deck, reveal it, and take it out of the deck.
 * If there are still cards in the deck, put the next top card of the deck at the bottom
of the deck.
 * If there are still unrevealed cards, go back to step 1.  Otherwise, stop.
 * Return an ordering of the deck that would reveal the cards in increasing order.
 *
 * The first entry in the answer is considered to be the top of the deck.
 *
 * Example 1:
 * Input: [17,13,11,2,3,5,7]
 * Output: [2,13,3,11,5,17,7]
 *
 * Explanation:
 * We get the deck in the order [17,13,11,2,3,5,7] (this order doesn't matter), and
reorder it.
 * After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the
deck.
 * We reveal 2, and move 13 to the bottom.  The deck is now [3,11,5,17,7,13].
 * We reveal 3, and move 11 to the bottom.  The deck is now [5,17,7,13,11].
 * We reveal 5, and move 17 to the bottom.  The deck is now [7,13,11,17].
 * We reveal 7, and move 13 to the bottom.  The deck is now [11,17,13].
 * We reveal 11, and move 17 to the bottom.  The deck is now [13,17].
 * We reveal 13, and move 17 to the bottom.  The deck is now [17].
 * We reveal 17.
 * Since all the cards revealed are in increasing order, the answer is correct.
 *
```

```
 * Note:
 * 1 <= A.length <= 1000
 * 1 <= A[i] <= 10^6
 * A[i] != A[j] for all i != j
 */
public class _950 {
  public static class Solution1 {
    public int[] deckRevealedIncreasing(int[] deck) {
      Arrays.sort(deck);
      Deque<Integer> deque = new ArrayDeque<>();
      for (int i = deck.length - 1; i >= 0; i--) {
        if (i != deck.length - 1) {
          deque.addFirst(deque.pollLast());
        }
        deque.addFirst(deck[i]);
      }
      int[] result = new int[deck.length];
      int i = 0;
      while (!deque.isEmpty()) {
        result[i++] = deque.pollFirst();
      }
      return result;
    }
  }
}

// ============================================================
// FILE 736: _951.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 951. Flip Equivalent Binary Trees
 *
 * For a binary tree T, we can define a flip operation as follows: choose any node, and
swap the left and right child subtrees.
 *
 * A binary tree X is flip equivalent to a binary tree Y if and only if we can make X
equal to Y after some number of flip operations.
 *
 * Write a function that determines whether two binary trees are flip equivalent.  The
trees are given by root nodes root1 and root2.
 *
 * Note:
 *  * Each tree will have at most 100 nodes.
 *  * Each value in each tree will be a unique integer in the range [0, 99].
 */

public class _951 {
    public static class Solution1 {
        public boolean flipEquiv(TreeNode root1, TreeNode root2) {
```

```
            if (root1 == null && root2 == null) {
                return true;
            }
            if (root1 == null || root2 == null) {
                return false;
            }

            if (root1.val != root2.val) {
                return false;
            }

            return (
                (flipEquiv(root1.left, root2.left) && flipEquiv(root1.right,
root2.right))
                || (flipEquiv(root1.left, root2.right) && flipEquiv(root1.right,
root2.left))
            );
        }
    }
}

// ============================================================
// FILE 737: _953.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 953. Verifying an Alien Dictionary
 *
 * In an alien language, surprisingly they also use english lowercase letters, but
possibly in a different order. The order of the alphabet is some permutation of lowercase
letters.
 *
 * Given a sequence of words written in the alien language, and the order of the
alphabet, return true if and only if the given words are sorted lexicographicaly in this
alien language.
 *
 * Example 1:
 *
 * Input: words = ["hello","leetcode"], order = "hlabcdefgijkmnopqrstuvwxyz"
 * Output: true
 * Explanation: As 'h' comes before 'l' in this language, then the sequence is sorted.
 *
 * Example 2:
 *
 * Input: words = ["word","world","row"], order = "worldabcefghijkmnpqstuvxyz"
 * Output: false
 * Explanation: As 'd' comes after 'l' in this language, then words[0] > words[1], hence
the sequence is unsorted.
 *
```

```
 * Example 3:
 *
 * Input: words = ["apple","app"], order = "abcdefghijklmnopqrstuvwxyz"
 * Output: false
 * Explanation: The first three characters "app" match, and the second string is shorter
(in size.) According to lexicographical rules "apple" > "app", because 'l' > 'ø', where
'ø' is defined as the blank character which is less than any other character (More info).
 *
 *
 * Note:
 * 1 <= words.length <= 100
 * 1 <= words[i].length <= 20
 * order.length == 26
 * All characters in words[i] and order are english lowercase letters.
 */
public class _953 {
  public static class Solution1 {
    public boolean isAlienSorted(String[] words, String order) {
      if (words.length == 1) {
        return true;
      }

      Map<Character, Integer> map = new HashMap<>();
      for (int i = 0; i < order.length(); i++) {
        map.put(order.charAt(i), i);
      }

      for (int i = 0; i < words.length - 1; i++) {
        String firstWord = words[i];
        String secondWord = words[i + 1];
        if (!sorted(firstWord, secondWord, map)) {
          return false;
        }
      }
      return true;
    }

    private boolean sorted(String firstWord, String secondWord, Map<Character, Integer>
map) {
      for (int i = 0; i < Math.min(firstWord.length(), secondWord.length()); i++) {
        if (firstWord.charAt(i) == secondWord.charAt(i)) {
          continue;
        } else {
          if (map.get(firstWord.charAt(i)) > map.get(secondWord.charAt(i))) {
            return false;
          } else {
            return true;
          }
        }
      }
      return firstWord.length() <= secondWord.length();
    }
  }
}
```

```
// ============================================================
// FILE 738: _954.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/**
 * 954. Array of Doubled Pairs
 *
 * Given an array of integers A with even length, return true if and only if it is
possible to reorder it
 * such that A[2 * i + 1] = 2 * A[2 * i] for every 0 <= i < len(A) / 2.
 *
 * Example 1:
 * Input: [3,1,3,6]
 * Output: false
 *
 * Example 2:
 * Input: [2,1,2,6]
 * Output: false
 *
 * Example 3:
 * Input: [4,-2,2,-4]
 * Output: true
 * Explanation: We can take two groups, [-2,-4] and [2,4] to form [-2,-4,2,4] or
[2,4,-2,-4].
 *
 * Example 4:
 * Input: [1,2,4,16,8,4]
 * Output: false
 *
 * Note:
 * 0 <= A.length <= 30000
 * A.length is even
 * -100000 <= A[i] <= 100000
 * */
public class _954 {
    public static class Solution1 {
        public boolean canReorderDoubled(int[] A) {
            for (int i = 0; i < A.length; i++) {
                if (A[i] < 0) {
                    A[i] = -A[i];
                }
            }
            Arrays.sort(A);
            Map<Integer, Integer> map = new HashMap<>();
            for (int num : A) {
                map.put(num, map.getOrDefault(num, 0) + 1);
            }
```

```
            for (int num : A) {
                if (num != 0) {
                    if (map.get(num) < 0) {
                        return false;
                    } else if (map.get(num) == 0) {
                        continue;
                    } else {
                        int count = map.get(num);
                        map.put(num, 0);
                        int doubleNum = num * 2;
                        if (!map.containsKey(doubleNum)) {
                            return false;
                        } else {
                            map.put(doubleNum, map.get(doubleNum) - count);
                        }
                    }
                } else if (map.get(num) % 2 != 0) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ======================================================
// FILE 739: _961.java
// ======================================================

package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 961. N-Repeated Element in Size 2N Array
 *
 * In a array A of size 2N, there are N+1 unique elements, and exactly one of these
elements is repeated N times.
 *
 * Return the element repeated N times.
 *
 * Example 1:
 *
 * Input: [1,2,3,3]
 * Output: 3
 * Example 2:
 *
 * Input: [2,1,2,5,3,2]
 * Output: 2
 * Example 3:
 *
 * Input: [5,1,5,2,5,3,5,4]
 * Output: 5
```

```
 *
 *
 * Note:
 *
 * 4 <= A.length <= 10000
 * 0 <= A[i] < 10000
 * A.length is even
 * */
public class _961 {
    public static class Solution1 {
        public int repeatedNTimes(int[] A) {
            Set<Integer> set = new HashSet<>();
            for (int num : A) {
                if (!set.add(num)) {
                    return num;
                }
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 740: _965.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 965. Univalued Binary Tree
 *
 * A binary tree is univalued if every node in the tree has the same value.
 *
 * Return true if and only if the given tree is univalued.
 *
 * Example 1:
 *
 *         1
 *        / \
 *       1   1
 *      / \   \
 *     1  1   1
 *
 * Input: [1,1,1,1,1,null,1]
 * Output: true
 *
 *
 * Example 2:
 *        2
 *       / \
 *      2   2
 *     / \
```

```
 *    5  2
 *
 * Input: [2,2,2,5,2]
 * Output: false
 *
 *
 * Note:
 *
 * The number of nodes in the given tree will be in the range [1, 100].
 * Each node's value will be an integer in the range [0, 99].
 */
public class _965 {
  public static class Solution1 {
    public boolean isUnivalTree(TreeNode root) {
      if (root == null) {
        return true;
      }
      return dfs(root, root.val);
    }

    private boolean dfs(TreeNode root, int value) {
      if (root == null) {
        return true;
      }
      if (root.val != value) {
        return false;
      }
      return dfs(root.left, value) && dfs(root.right, value);
    }
  }
}

// ============================================================
// FILE 741: _966.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 966. Vowel Spellchecker
 *
 * Given a wordlist, we want to implement a spellchecker that converts a query word into
a correct word.
 *
 * For a given query word, the spell checker handles two categories of spelling mistakes:
 *
 * Capitalization: If the query matches a word in the wordlist (case-insensitive), then
the query word is returned with
 * the same case as the case in the wordlist.
```

```
 * Example: wordlist = ["yellow"], query = "YellOw": correct = "yellow"
 * Example: wordlist = ["Yellow"], query = "yellow": correct = "Yellow"
 * Example: wordlist = ["yellow"], query = "yellow": correct = "yellow"
 *
 * Vowel Errors: If after replacing the vowels ('a', 'e', 'i', 'o', 'u') of the query
word with any vowel individually,
 * it matches a word in the wordlist (case-insensitive), then the query word is returned
with the same case as the
 * match in the wordlist.
 * Example: wordlist = ["YellOw"], query = "yollow": correct = "YellOw"
 * Example: wordlist = ["YellOw"], query = "yeellow": correct = "" (no match)
 * Example: wordlist = ["YellOw"], query = "yllw": correct = "" (no match)
 *
 * In addition, the spell checker operates under the following precedence rules:
 *
 * When the query exactly matches a word in the wordlist (case-sensitive), you should
return the same word back.
 * When the query matches a word up to capitlization, you should return the first such
match in the wordlist.
 * When the query matches a word up to vowel errors, you should return the first such
match in the wordlist.
 * If the query has no matches in the wordlist, you should return the empty string.
 *
 * Given some queries, return a list of words answer, where answer[i] is the correct word
for query = queries[i].
 *
 * Example 1:
 *
 * Input: wordlist = ["KiTe","kite","hare","Hare"], queries =
["kite","Kite","KiTe","Hare","HARE","Hear","hear","keti","keet","keto"]
 * Output: ["kite","KiTe","KiTe","Hare","hare","","","KiTe","","KiTe"]
 *
 * Note:
 *
 * 1 <= wordlist.length <= 5000
 * 1 <= queries.length <= 5000
 * 1 <= wordlist[i].length <= 7
 * 1 <= queries[i].length <= 7
 * All strings in wordlist and queries consist only of english letters.
 *
 * */

public class _966 {

    public static class Solution1 {
        public String[] spellchecker(String[] wordlist, String[] queries) {
            Map<String, String> caseMap = new HashMap<>();
            Set<String> set = new HashSet<>();

            // Case Part Mapping
            for (String word : wordlist) {
                if (!caseMap.containsKey(word.toLowerCase())) {
                    caseMap.put(word.toLowerCase(), word);
                }
```

```
                set.add(word);
            }

            // Vowel Part Mapping
            Map<String, String> vowelMap = new HashMap<>();
            for (String word : wordlist) {
                String genericVal = makeGenericVowel(word);
                if (!vowelMap.containsKey(genericVal)) {
                    vowelMap.put(genericVal, word);
                }
            }

            String[] ans = new String[queries.length];

            for (int i = 0; i < queries.length; i++) {
                if (set.contains(queries[i])) {
                    ans[i] = queries[i];
                } else if (caseMap.containsKey(queries[i].toLowerCase())) {
                    ans[i] = caseMap.get(queries[i].toLowerCase());
                } else if (vowelMap.containsKey(makeGenericVowel(queries[i]))) {
                    ans[i] = vowelMap.get(makeGenericVowel(queries[i]));
                } else {
                    ans[i] = "";
                }
            }

            return ans;
        }

        private String makeGenericVowel(String s) {
            String vowel = "aeiou";
            char[] ch = s.toLowerCase().toCharArray();
            for (int i = 0; i < ch.length; i++) {
                if (vowel.indexOf(ch[i]) != -1) {
                    ch[i] = '#';
                }
            }

            return String.valueOf(ch);
        }
    }
}

// ============================================================
// FILE 742: _970.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
```

```
import java.util.Set;

/**
 * 970. Powerful Integers
 *
 * Given two non-negative integers x and y, an integer is powerful if it is equal to x^i
+ y^j for some integers i >= 0 and j >= 0.
 *
 * Return a list of all powerful integers that have value less than or equal to bound.
 *
 * You may return the answer in any order.  In your answer, each value should occur at
most once.
 *
 * Example 1:
 *
 * Input: x = 2, y = 3, bound = 10
 * Output: [2,3,4,5,7,9,10]
 * Explanation:
 * 2 = 2^0 + 3^0
 * 3 = 2^1 + 3^0
 * 4 = 2^0 + 3^1
 * 5 = 2^1 + 3^1
 * 7 = 2^2 + 3^1
 * 9 = 2^3 + 3^0
 * 10 = 2^0 + 3^2
 *
 * Example 2:
 *
 * Input: x = 3, y = 5, bound = 15
 * Output: [2,4,6,8,10,14]
 *
 *
 * Note:
 * 1 <= x <= 100
 * 1 <= y <= 100
 * 0 <= bound <= 10^6
 */
public class _970 {
  public static class Solution1 {
    /**This approach results in Time Limit Exceeded since it's apparently doing
     * redundant checks.*/
    public List<Integer> powerfulIntegers(int x, int y, int bound) {
      Set<Integer> result = new HashSet<>();
      int small = x;
      int big = y;
      if (x > y) {
        small = y;
        big = x;
      }
      int maxPower = bound / small;
      for (int i = 0; i <= maxPower + 1; i++) {
        for (int j = 0; j <= maxPower + 1; j++) {
          int sum = (int) (Math.pow(small, i) + Math.pow(big, j));
          if (sum <= bound) {
```

```
            result.add(sum);
          }
        }
      }
      List<Integer> list = new ArrayList<>(result);
      Collections.sort(list);
      return list;
    }
  }

  public static class Solution2 {
    /** credit: https://leetcode.com/problems/powerful-
integers/discuss/214212/JavaC%2B%2BPython-Brute-Force */
    public List<Integer> powerfulIntegers(int x, int y, int bound) {
      Set<Integer> result = new HashSet<>();
      for (int i = 1; i < bound; i *= x > 1 ? x : bound + 1) {
        for (int j = 1; i + j <= bound; j *= y > 1 ? y : bound + 1) {
          result.add(i + j);
        }
      }
      return new ArrayList<>(result);
    }
  }
}

// ============================================================
// FILE 743: _973.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 973. K Closest Points to Origin
 *
 * We have a list of points on the plane.  Find the K closest points to the origin (0,
0).
 *
 * (Here, the distance between two points on a plane is the Euclidean distance.)
 *
 * You may return the answer in any order.  The answer is guaranteed to be unique (except
for the order that it is in.)
 *
 *
 *
 * Example 1:
 *
 * Input: points = [[1,3],[-2,2]], K = 1
 * Output: [[-2,2]]
 * Explanation:
 * The distance between (1, 3) and the origin is sqrt(10).
 * The distance between (-2, 2) and the origin is sqrt(8).
 * Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
```

```
 * We only want the closest K = 1 points from the origin, so the answer is just [[-2,2]].
 * Example 2:
 *
 * Input: points = [[3,3],[5,-1],[-2,4]], K = 2
 * Output: [[3,3],[-2,4]]
 * (The answer [[-2,4],[3,3]] would also be accepted.)
 *
 *
 * Note:
 *
 * 1 <= K <= points.length <= 10000
 * -10000 < points[i][0] < 10000
 * -10000 < points[i][1] < 10000
 * */
public class _973 {

    public static class Solution1 {
        public int[][] kClosest(int[][] points, int K) {
            int[][] ans = new int[K][2];

            PriorityQueue<int[]> pq = new PriorityQueue<>((o1, o2) -> {
                double dist1 = getDistance(o1);
                double dist2 = getDistance(o2);

                if (dist1 > dist2) {
                    return 1;
                } else if (dist1 < dist2) {
                    return -1;
                } else {
                    return 0;
                }
            });

            for (int[] point : points) {
                pq.add(point);
            }

            for (int i = 0; i < K; i++) {
                ans[i] = pq.poll();
            }

            return ans;
        }

        private double getDistance(int[] point) {
            return Math.sqrt(Math.pow(point[0], 2) + Math.pow(point[1], 2));
        }
    }
}

// ============================================================
// FILE 744: _974.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 974. Subarray Sums Divisible by K
 *
 * Given an array A of integers, return the number of (contiguous, non-empty) subarrays
that have a sum
 * divisible by K.
 * */

public class _974 {
    public static class Solution1 {
        public int subarraysDivByK(int[] A, int K) {
            int count = 0;
            int sum = 0;
            Map<Integer, Integer> map = new HashMap<>();
            map.put(0, 1);

            for (int i = 0; i < A.length; i++) {
                sum = (sum + A[i]) % K;
                if (sum < 0) {
                    sum += K;
                }

                count += map.getOrDefault(sum, 0);
                map.put(sum, map.getOrDefault(sum, 0) + 1);
            }

            return count;
        }
    }
}

// ============================================================
// FILE 745: _976.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 976. Largest Perimeter Triangle
 *
 * Given an array A of positive lengths, return the largest perimeter of a triangle with
non-zero area, formed from 3
 * of these lengths.
 *
 * If it is impossible to form any triangle of non-zero area, return 0.
 * */
```

```
public class _976 {

    public static class Solution1 {
        public int largestPerimeter(int[] A) {
            Arrays.sort(A);
            int n = A.length;

            for (int i = n - 1; i > 1; i--) {
                if (A[i] < A[i - 1] + A[i - 2]) {
                    return A[i] + A[i - 1] + A[i - 2];
                }
            }

            return 0;
        }
    }
}

// ============================================================
// FILE 746: _977.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 977. Squares of a Sorted Array
 *
 * Given an array of integers A sorted in non-decreasing order, return an array of the
squares of each number, also in sorted non-decreasing order.
 *
 * Example 1:
 * Input: [-4,-1,0,3,10]
 * Output: [0,1,9,16,100]
 *
 * Example 2:
 * Input: [-7,-3,2,3,11]
 * Output: [4,9,9,49,121]
 *
 *
 * Note:
 * 1 <= A.length <= 10000
 * -10000 <= A[i] <= 10000
 * A is sorted in non-decreasing order.
 */
public class _977 {
  public static class Solution1 {
    public int[] sortedSquares(int[] A) {
      int[] result = new int[A.length];
      for (int i = 0; i < A.length; i++) {
        result[i] = (int) Math.pow(A[i], 2);
      }
      Arrays.sort(result);
```

```
      return result;
    }
  }
}

// ============================================================
// FILE 747: _979.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 979. Distribute Coins in Binary Tree
 *
 * Given the root of a binary tree with N nodes, each node in the tree has node.val
coins, and there are N coins total.
 * In one move, we may choose two adjacent nodes and move one coin from one node to
another.  (The move may be from parent to child, or from child to parent.)
 * Return the number of moves required to make every node have exactly one coin.
 *
 * Example 1:
 *
 *      3
 *     / \
 *    0  0
 *
 * Input: [3,0,0]
 * Output: 2
 * Explanation: From the root of the tree, we move one coin to its left child, and one
coin to its right child.
 *
 * Example 2:
 *
 *    0
 *   / \
 *  3  0
 *
 * Input: [0,3,0]
 * Output: 3
 * Explanation: From the left child of the root, we move two coins to the root [taking
two moves].  Then, we move one coin from the root of the tree to the right child.
 *
 * Example 3:
 *
 *    0
 *   / \
 *  1  2
 *
 * Input: [1,0,2]
 * Output: 2
 *
 * Example 4:
```

```
 *
 *     1
 *    / \
 *   0  0
 *    \
 *    3
 *
 * Input: [1,0,0,null,3]
 * Output: 4
 *
 *
 * Note:
 * 1<= N <= 100
 * 0 <= node.val <= N
 * */
public class _979 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/distribute-coins-in-binary-
tree/discuss/221930/JavaC%2B%2BPython-Recursive-Solution*/
        int moves = 0;

        public int distributeCoins(TreeNode root) {
            dfs(root);
            return moves;
        }

        int dfs(TreeNode root) {
            if (root == null) {
                return 0;
            }
            int left = dfs(root.left);
            int right = dfs(root.right);
            moves += Math.abs(left) + Math.abs(right);
            return root.val + left + right - 1;
        }
    }
}

// ============================================================
// FILE 748: _985.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 985. Sum of Even Numbers After Queries
 *
 * We have an array A of integers, and an array queries of queries.
 * For the i-th query val = queries[i][0], index = queries[i][1], we add val to A[index].
Then, the answer to the i-th query is the sum of the even values of A.
 * (Here, the given index = queries[i][1] is a 0-based index, and each query permanently
modifies the array A.)
 * Return the answer to all queries.  Your answer array should have answer[i] as the
answer to the i-th query.
```

```
 *
 * Example 1:
 *
 * Input: A = [1,2,3,4], queries = [[1,0],[-3,1],[-4,0],[2,3]]
 * Output: [8,6,2,4]
 * Explanation:
 * At the beginning, the array is [1,2,3,4].
 * After adding 1 to A[0], the array is [2,2,3,4], and the sum of even values is 2 + 2 +
4 = 8.
 * After adding -3 to A[1], the array is [2,-1,3,4], and the sum of even values is 2 + 4
= 6.
 * After adding -4 to A[0], the array is [-2,-1,3,4], and the sum of even values is -2 +
4 = 2.
 * After adding 2 to A[3], the array is [-2,-1,3,6], and the sum of even values is -2 + 6
= 4.
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * -10000 <= A[i] <= 10000
 * 1 <= queries.length <= 10000
 * -10000 <= queries[i][0] <= 10000
 * 0 <= queries[i][1] < A.length
 */
public class _985 {
  public static class Solution1 {
    public int[] sumEvenAfterQueries(int[] A, int[][] queries) {
      int[] result = new int[A.length];
      for (int i = 0; i < A.length; i++) {
        int col = queries[i][1];
        A[col] = A[col] + queries[i][0];
        result[i] = computeEvenSum(A);
      }
      return result;
    }

    private int computeEvenSum(int[] A) {
      int sum = 0;
      for (int num : A) {
        if (num % 2 == 0) {
          sum += num;
        }
      }
      return sum;
    }
  }
}

// ============================================================
// FILE 749: _987.java
// ============================================================

package com.fishercoder.solutions;
```

```
import com.fishercoder.common.classes.TreeNode;

import java.util.List;
import java.util.ArrayList;
import java.util.PriorityQueue;
import java.util.TreeMap;

/**
 * 987. Vertical Order Traversal of a Binary Tree
 *
 * Given a binary tree, return the vertical order traversal of its nodes values.
 * For each node at position (X, Y), its left and right children respectively will be at
positions (X-1, Y-1) and (X+1, Y-1).
 * Running a vertical line from X = -infinity to X = +infinity, whenever the vertical
line touches some nodes, we report the values of the nodes in order from top to bottom
(decreasing Y coordinates).
 * If two nodes have the same position, then the value of the node that is reported first
is the value that is smaller.
 * Return an list of non-empty reports in order of X coordinate.  Every report will have
a list of values of nodes.
 *
 * Example 1:
 *
 *        3
 *       / \
 *      9  20
 *        /  \
 *       15  7
 *
 * Input: [3,9,20,null,null,15,7]
 * Output: [[9],[3,15],[20],[7]]
 * Explanation:
 * Without loss of generality, we can assume the root node is at position (0, 0):
 * Then, the node with value 9 occurs at position (-1, -1);
 * The nodes with values 3 and 15 occur at positions (0, 0) and (0, -2);
 * The node with value 20 occurs at position (1, -1);
 * The node with value 7 occurs at position (2, -2).
 *
 *
 * Example 2:
 *
 *       1
 *      / \
 *     2   3
 *    / \ / \
 *   4  5 6 7
 *
 * Input: [1,2,3,4,5,6,7]
 * Output: [[4],[2],[1,5,6],[3],[7]]
 * Explanation:
 * The node with value 5 and the node with value 6 have the same position according to
the given scheme.
 * However, in the report "[1,5,6]", the node value of 5 comes first since 5 is smaller
than 6.
```

```
 *
 * Note:
 *
 * The tree will have between 1 and 1000 nodes.
 * Each node's value will be between 0 and 1000.
 * */
public class _987 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/vertical-order-traversal-of-a-binary-tree/discuss/231148/Java-TreeMap-Solution*/
        public List<List<Integer>> verticalTraversal(TreeNode root) {
            TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>> map = new TreeMap<>();
            dfs(root, 0, 0, map);
            List<List<Integer>> list = new ArrayList<>();
            for (TreeMap<Integer, PriorityQueue<Integer>> ys : map.values()) {
                list.add(new ArrayList<>());
                for (PriorityQueue<Integer> nodes : ys.values()) {
                    while (!nodes.isEmpty()) {
                        list.get(list.size() - 1).add(nodes.poll());
                    }
                }
            }
            return list;
        }

        private void dfs(TreeNode root, int x, int y, TreeMap<Integer, TreeMap<Integer, PriorityQueue<Integer>>> map) {
            if (root == null) {
                return;
            }
            if (!map.containsKey(x)) {
                map.put(x, new TreeMap<>());
            }
            if (!map.get(x).containsKey(y)) {
                map.get(x).put(y, new PriorityQueue<>());
            }
            map.get(x).get(y).offer(root.val);
            dfs(root.left, x - 1, y + 1, map);
            dfs(root.right, x + 1, y + 1, map);
        }
    }
}

// ============================================================
// FILE 750: _989.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
```

```
/**
 * 989. Add to Array-Form of Integer
 *
 * For a non-negative integer X, the array-form of X is an array of its digits in left to
right order.  For example, if X = 1231, then the array form is [1,2,3,1].
 *
 * Given the array-form A of a non-negative integer X, return the array-form of the
integer X+K.
 *
 * Example 1:
 *
 * Input: A = [1,2,0,0], K = 34
 * Output: [1,2,3,4]
 * Explanation: 1200 + 34 = 1234
 * Example 2:
 *
 * Input: A = [2,7,4], K = 181
 * Output: [4,5,5]
 * Explanation: 274 + 181 = 455
 * Example 3:
 *
 * Input: A = [2,1,5], K = 806
 * Output: [1,0,2,1]
 * Explanation: 215 + 806 = 1021
 * Example 4:
 *
 * Input: A = [9,9,9,9,9,9,9,9,9,9], K = 1
 * Output: [1,0,0,0,0,0,0,0,0,0,0]
 * Explanation: 9999999999 + 1 = 10000000000
 *
 * Note：
 *
 * 1 <= A.length <= 10000
 * 0 <= A[i] <= 9
 * 0 <= K <= 10000
 * If A.length > 1, then A[0] != 0
 */
public class _989 {
  public static class Solution1 {
    public List<Integer> addToArrayForm(int[] A, int K) {
      List<Integer> kDigitsReversed = new ArrayList<>();
      int divisor = 10;
      while (K != 0) {
        kDigitsReversed.add(K % divisor);
        K /= 10;
      }
      List<Integer> result = new ArrayList<>();
      int prevFlow = 0;
      for (int i = A.length - 1, j = 0; i >= 0 || j < kDigitsReversed.size(); i --, j++)
{
        int sum;
        if (i >= 0 && j < kDigitsReversed.size()) {
          sum = A[i] + kDigitsReversed.get(j);
        } else if (i >= 0) {
```

```
          sum = A[i];
        } else {
          sum = kDigitsReversed.get(j);
        }
        int flow = 0;
        if (prevFlow != 0) {
          sum += prevFlow;
        }
        if (sum > 9) {
          flow = 1;
        }
        sum %= 10;
        prevFlow = flow;
        result.add(sum);
      }
      if (prevFlow != 0) {
        result.add(prevFlow);
      }
      Collections.reverse(result);
      return result;
    }
  }
}

// ============================================================
// FILE 751: _993.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Set;

/**
 * 993. Cousins in Binary Tree
 *
 * In a binary tree, the root node is at depth 0, and children of each depth k node are
at depth k+1.
 * Two nodes of a binary tree are cousins if they have the same depth, but have different
parents.
 * We are given the root of a binary tree with unique values, and the values x and y of
two different nodes in the tree.
 * Return true if and only if the nodes corresponding to the values x and y are cousins.
 *
 * Example 1:
 *        1
 *       / \
 *      2   3
 *     /
 *    4
 *
```

```
 * Input: root = [1,2,3,4], x = 4, y = 3
 * Output: false
 *
 * Example 2:
 *         1
 *        / \
 *       2   3
 *       \    \
 *       4    5
 *
 * Input: root = [1,2,3,null,4,null,5], x = 5, y = 4
 * Output: true
 *
 * Example 3:
 *          1
 *         / \
 *        2  3
 *        \
 *        4
 *
 * Input: root = [1,2,3,null,4], x = 2, y = 3
 * Output: false
 *
 *
 * Note:
 *
 * The number of nodes in the tree will be between 2 and 100.
 * Each node has a unique integer value from 1 to 100.
 */
public class _993 {
  public static class Solution1 {
    public boolean isCousins(TreeNode root, int x, int y) {
      Queue<TreeNode> queue = new LinkedList<>();
      queue.offer(root);
      while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
          TreeNode current = queue.poll();
          if (current.left != null) {
            queue.offer(current.left);
          }
          if (current.right != null) {
            queue.offer(current.right);
          }
          if (current.left != null && current.right != null) {
            if (current.left.val == x && current.right.val == y
                || current.left.val == y && current.right.val == x) {
              return false;
            }
          }
        }
        if (checkQueue(queue, x, y)) {
          return true;
        }
```

```
      }
      return false;
    }

    private boolean checkQueue(Queue<TreeNode> queue, int x, int y) {
      Set<Integer> set = new HashSet<>();
      Queue<TreeNode> tmp = new LinkedList<>(queue);
      while (!tmp.isEmpty()) {
        set.add(tmp.poll().val);
      }
      return set.contains(x) && set.contains(y);
    }
  }
}

// ============================================================
// FILE 752: _994.java
// ============================================================

package com.fishercoder.solutions;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 994. Rotting Oranges
 *
 * In a given grid, each cell can have one of three values:
 *
 * the value 0 representing an empty cell;
 * the value 1 representing a fresh orange;
 * the value 2 representing a rotten orange.
 * Every minute, any fresh orange that is adjacent (4-directionally) to a
 * rotten orange becomes rotten.
 * Return the minimum number of minutes that must elapse until no cell has a fresh
orange.
 * If this is impossible, return -1 instead.
 *
 * Example 1:
 * Input: [[2,1,1],[1,1,0],[0,1,1]]
 * Output: 4
 *
 * Example 2:
 * Input: [[2,1,1],[0,1,1],[1,0,1]]
 * Output: -1
 * Explanation:  The orange in the bottom left corner (row 2, column 0) is never rotten,
because rotting only happens 4-directionally.
 *
 * Example 3:
 * Input: [[0,2]]
 * Output: 0
 * Explanation:  Since there are already no fresh oranges at minute 0, the answer is just
0.
 *
```

```
 * Note:
 * 1 <= grid.length <= 10
 * 1 <= grid[0].length <= 10
 * grid[i][j] is only 0, 1, or 2.
 */
public class _994 {
    public static class Solution1 {
        int[] directions = new int[]{0, 1, 0, -1, 0};

        public int orangesRotting(int[][] grid) {
            Queue<int[]> rottens = new LinkedList<>();
            for (int i = 0; i < grid.length; i++) {
                for (int j = 0; j < grid[0].length; j++) {
                    if (grid[i][j] == 2) {
                        rottens.add(new int[]{i, j});
                    }
                }
            }
            int times = 0;
            while (!rottens.isEmpty()) {
                int size = rottens.size();
                boolean counted = false;
                for (int k = 0; k < size; k++) {
                    int[] rotten = rottens.poll();
                    for (int i = 0; i < 4; i++) {
                        int x = rotten[0] + directions[i];
                        int y = rotten[1] + directions[i + 1];
                        if (x >= 0 && x < grid.length && y >= 0 && y < grid[0].length &&
grid[x][y] == 1) {
                            grid[x][y] = 2;
                            if (!counted) {
                                times++;
                            }
                            counted = true;
                            rottens.add(new int[]{x, y});
                        }
                    }
                }
            }
            for (int i = 0; i < grid.length; i++) {
                for (int j = 0; j < grid[0].length; j++) {
                    if (grid[i][j] == 1) {
                        return -1;
                    }
                }
            }
            return times;
        }
    }
}

// ============================================================
// FILE 753: _997.java
// ============================================================
```

```
package com.fishercoder.solutions;

import java.util.HashSet;
import java.util.Set;

/**
 * 997. Find the Town Judge
 *
 * In a town, there are N people labelled from 1 to N.
 * There is a rumor that one of these people is secretly the town judge.
 *
 * If the town judge exists, then:
 *
 * The town judge trusts nobody.
 * Everybody (except for the town judge) trusts the town judge.
 * There is exactly one person that satisfies properties 1 and 2.
 * You are given trust, an array of pairs trust[i] = [a, b] representing that the person
labelled a trusts the person labelled b.
 *
 * If the town judge exists and can be identified, return the label of the town judge.
 * Otherwise, return -1.
 *
 * Example 1:
 * Input: N = 2, trust = [[1,2]]
 * Output: 2
 *
 * Example 2:
 * Input: N = 3, trust = [[1,3],[2,3]]
 * Output: 3
 *
 * Example 3:
 * Input: N = 3, trust = [[1,3],[2,3],[3,1]]
 * Output: -1
 *
 * Example 4:
 * Input: N = 3, trust = [[1,2],[2,3]]
 * Output: -1
 *
 * Example 5:
 * Input: N = 4, trust = [[1,3],[1,4],[2,3],[2,4],[4,3]]
 * Output: 3
 *
 * Note:
 * 1 <= N <= 1000
 * trust.length <= 10000
 * trust[i] are all different
 * trust[i][0] != trust[i][1]
 * 1 <= trust[i][0], trust[i][1] <= N
 */
public class _997 {
  public static class Solution1 {
    public int findJudge(int N, int[][] trust) {
      int[] trustPoints = new int[N];
```

```
      Set<Integer> trustOthers = new HashSet<>();
      for (int[] eachTrust : trust) {
        trustPoints[eachTrust[1] - 1]++;
        trustOthers.add(eachTrust[0]);
      }
      int judge = -1;
      for (int i = 0; i < trustPoints.length; i++) {
        if (trustPoints[i] == N - 1 && !trustOthers.contains(i + 1)) {
            judge = i + 1;
        }
      }
      return judge;
    }
  }
}

// ============================================================
// FILE 754: _999.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 999. Available Captures for Rook
 *
 * On an 8 x 8 chessboard, there is one white rook.
 * There also may be empty squares, white bishops, and black pawns.
 * These are given as characters 'R', '.', 'B', and 'p' respectively.
 * Uppercase characters represent white pieces, and lowercase characters represent black
pieces.
 *
 * The rook moves as in the rules of Chess:
 * it chooses one of four cardinal directions (north, east, west, and south),
 * then moves in that direction until it chooses to stop, reaches the edge of the board,
 * or captures an opposite colored pawn by moving to the same square it occupies.
 * Also, rooks cannot move into the same square as other friendly bishops.
 *
 * Return the number of pawns the rook can capture in one move.
 *
 * Example 1:
 *
 * Input:[
 * 8 [".",".",".",".",".",".",".","."],
 * 7 [".",".",".","p",".",".",".","."],
 * 6 [".",".",".","R",".",".",".","p"],
 * 5 [".",".",".",".",".",".",".","."],
 * 4 [".",".",".",".",".",".",".","."],
 * 3 [".",".",".","p",".",".",".","."],
 * 2 [".",".",".",".",".",".",".","."],
 * 1 [".",".",".",".",".",".",".","."]]
 *     a   b   c   d   e   f   g   h
 *
 * Output: 3
 * Explanation:
```

```
 * In this example the rook is able to capture all the pawns.
 *
 * Example 2:
 *
 * Input:[
 * 8 [".",".",".",".",".",".",".","."],
 * 7 [".","p","p","p","p","p",".","."],
 * 6 [".","p","p","B","p","p",".","."],
 * 5 [".","p","B","R","B","p",".","."],
 * 4 [".","p","p","B","p","p",".","."],
 * 3 [".","p","p","p","p","p",".","."],
 * 2 [".",".",".",".",".",".",".","."],
 * 1 [".",".",".",".",".",".",".","."]]
 *     a   b   c   d   e   f   g   h
 *
 * Output: 0
 * Explanation:
 * Bishops are blocking the rook to capture any pawn.
 *
 * Example 3:
 *
 * Input: [
 * 8 [".",".",".",".",".",".",".","."],
 * 7 [".",".",".","p",".",".",".","."],
 * 6 [".",".",".","p",".",".",".","."],
 * 5 ["p","p",".","R",".","p","B","."],
 * 4 [".",".",".",".",".",".",".","."],
 * 3 [".",".",".","B",".",".",".","."],
 * 2 [".",".",".","p",".",".",".","."],
 * 1 [".",".",".",".",".",".",".","."]]
 *     a   b   c   d   e   f   g   h
 *
 * Output: 3
 * Explanation:
 * The rook can capture the pawns at positions b5, d6 and f5.
 *
 * Note:
 * board.length == board[i].length == 8
 * board[i][j] is either 'R', '.', 'B', or 'p'
 * There is exactly one cell with board[i][j] == 'R'
 */
public class _999 {
  public static class Solution1 {
    int[] directions = new int[] {0, 1, 0, -1, 0};

    public int numRookCaptures(char[][] board) {
      int m = board.length;
      int n = board[0].length;
      int rowR = -1;
      int colR = -1;
      for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
          if (board[i][j] == 'R') {
            rowR = i;
```

```
          colR = j;
          break;
        }
      }
    }
    int count = 0;
    for (int i = 0; i < 4; i++) {
      int neighborRow = rowR + directions[i];
      int neighborCol = colR + directions[i + 1];
      if (neighborRow >= 0 && neighborRow < m
          && neighborCol >= 0 && neighborCol < n
          && board[neighborRow][neighborCol] != 'B') {
        if (directions[i] == 0 && directions[i + 1] == 1) {
          while (neighborCol < n) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborCol++;
            }
          }
        } else if (directions[i] == 1 && directions[i + 1] == 0) {
          while (neighborRow < m) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborRow++;
            }
          }
        } else if (directions[i] == 0 && directions[i + 1] == -1) {
          while (neighborCol >= 0) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborCol--;
            }
          }
        } else {
          while (neighborRow >= 0) {
            if (board[neighborRow][neighborCol] == 'p') {
              count++;
              break;
            } else if (board[neighborRow][neighborCol] == 'B') {
              break;
            } else {
              neighborRow--;
```

```
              }
            }
          }
        }
      }

      return count;
    }
  }
}

// ============================================================
// FILE 755: _1002.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1002. Find Common Characters
 *
 * Given an array A of strings made only from lowercase letters,
 * return a list of all characters that show up in all strings within the list (including
duplicates).
 * For example, if a character occurs 3 times in all strings but not 4 times, you need to
include that character three times in the final answer.
 *
 * You may return the answer in any order.
 *
 * Example 1:
 * Input: ["bella","label","roller"]
 * Output: ["e","l","l"]
 *
 * Example 2:
 * Input: ["cool","lock","cook"]
 * Output: ["c","o"]
 *
 * Note:
 *
 * 1 <= A.length <= 100
 * 1 <= A[i].length <= 100
 * A[i][j] is a lowercase letter
 */
public class _1002 {
  public static class Solution1 {
    public List<String> commonChars(String[] A) {
      int[][] charCount = new int[A.length][26];
      for (int i = 0; i < A.length; i++) {
        for (char c : A[i].toCharArray()) {
          charCount[i][c - 'a']++;
        }
      }
```

```
      List<String> result = new ArrayList<>();
      for (int i = 0; i < 26; i++) {
        while (charCount[0][i] != 0) {
          char c = (char) (i + 'a');
          boolean valid = true;
          charCount[0][i]--;
          for (int j = 1; j < A.length; j++) {
            if (charCount[j][i] == 0) {
              valid = false;
              break;
            } else {
              charCount[j][i]--;
            }
          }
          if (!valid) {
            break;
          } else {
            result.add("" + c);
          }
        }
      }
      return result;
    }
  }
}

// ============================================================
// FILE 756: _1003.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayDeque;
import java.util.Deque;

/**
 * 1003. Check If Word Is Valid After Substitutions
 *
 * We are given that the string "abc" is valid.
 *
 * From any valid string V, we may split V into two pieces X and Y such that X + Y (X
concatenated with Y) is equal to V.  (X or Y may be empty.)  Then, X + "abc" + Y is also
valid.
 *
 * If for example S = "abc", then examples of valid strings are: "abc", "aabcbc",
"abcabc", "abcabcababcc".  Examples of invalid strings are: "abccba", "ab", "cababc",
"bac".
 *
 * Return true if and only if the given string S is valid.
 *
 * Note:
 *  * 1 <= S.length <= 20000
 *  * S[i] is 'a', 'b', or 'c'
 */
```

```
public class _1003 {
    public static class Solution1 {
        public boolean isValid(String S) {
            Deque<Character> stack = new ArrayDeque<>();

            for (char c : S.toCharArray()) {
                if (c == 'c') {
                    if (stack.isEmpty() || stack.pop() != 'b') {
                        return false;
                    }
                    if (stack.isEmpty() || stack.pop() != 'a') {
                        return false;
                    }
                } else {
                    stack.push(c);
                }
            }

            return stack.isEmpty();
        }
    }
}

// ============================================================
// FILE 757: _1005.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.PriorityQueue;

/**
 * 1005. Maximize Sum Of Array After K Negations
 *
 * Given an array A of integers, we must modify the array in the following way:
 * we choose an i and replace A[i] with -A[i], and we repeat this process K times in
total.
 * (We may choose the same index i multiple times.)
 *
 * Return the largest possible sum of the array after modifying it in this way.
 *
 * Example 1:
 * Input: A = [4,2,3], K = 1
 * Output: 5
 * Explanation: Choose indices (1,) and A becomes [4,-2,3].
 *
 * Example 2:
 * Input: A = [3,-1,0,2], K = 3
 * Output: 6
 * Explanation: Choose indices (1, 2, 2) and A becomes [3,1,0,2].
 * Example 3:
 *
```

```
 * Input: A = [2,-3,-1,5,-4], K = 2
 * Output: 13
 * Explanation: Choose indices (1, 4) and A becomes [2,3,-1,5,4].
 *
 *
 * Note:
 *
 * 1 <= A.length <= 10000
 * 1 <= K <= 10000
 * -100 <= A[i] <= 100
 * */
public class _1005 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/maximize-sum-of-array-after-k-
negations/discuss/252228/A-very-simple-java-solution*/
        public int largestSumAfterKNegations(int[] A, int K) {
            PriorityQueue<Integer> minHeap = new PriorityQueue<>();
            for (int num : A) {
                minHeap.offer(num);
            }
            while (K-- > 0) {
                minHeap.offer(-minHeap.poll());
            }
            int sum = 0;
            while (!minHeap.isEmpty()) {
                sum += minHeap.poll();
            }
            return sum;
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/problems/maximize-sum-of-array-after-k-
negations/discuss/252254/JavaC%2B%2BPython-Sort*/
        public int largestSumAfterKNegations(int[] A, int K) {
            Arrays.sort(A);
            for (int i = 0; i < A.length && K > 0 && A[i] < 0; i++, K--) {
                A[i] = -A[i];
            }
            int sum = 0;
            int min = Integer.MAX_VALUE;
            for (int i = 0; i < A.length; i++) {
                min = Math.min(min, A[i]);
                sum += A[i];
            }
            return K % 2 == 0 ? sum : sum - min * 2;
        }
    }
}

// ============================================================
// FILE 758: _1008.java
// ============================================================
```

```
package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 1008. Construct Binary Search Tree from Preorder Traversal
 *
 * Return the root node of a binary search tree that matches the given preorder
traversal.
 * (Recall that a binary search tree is a binary tree where for every node, any
descendant of node.left has a value < node.val, and any descendant of node.right has a
value > node.val.  Also recall that a preorder traversal displays the value of the node
first, then traverses node.left, then traverses node.right.)
 *
 *
 * Example 1:
 *
 * Input: [8,5,1,7,10,12]
 * Output: [8,5,10,1,7,null,12]
 *
 *              8
 *             / \
 *            5  10
 *           / \   \
 *          1   7  12
 *
 * Note:
 *
 * 1 <= preorder.length <= 100
 * The values of preorder are distinct.
 * */
public class _1008 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/construct-binary-search-tree-from-
preorder-traversal/discuss/252232/JavaC%2B%2BPython-O(N)-Solution*/
        int i = 0;

        public TreeNode bstFromPreorder(int[] preorder) {
            return bstFromPreorder(preorder, Integer.MAX_VALUE);
        }

        private TreeNode bstFromPreorder(int[] preorder, int bound) {
            if (i == preorder.length || preorder[i] > bound) {
                return null;
            }
            TreeNode root = new TreeNode(preorder[i++]);
            root.left = bstFromPreorder(preorder, root.val);
            root.right = bstFromPreorder(preorder, bound);
            return root;
        }
    }
}

// ============================================================
```

```java
// FILE 759: _1009.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1009. Complement of Base 10 Integer
 *
 * Every non-negative integer N has a binary representation.
 * For example, 5 can be represented as "101" in binary,
 * 11 as "1011" in binary, and so on.
 *
 * Note that except for N = 0, there are no leading zeroes in any binary representation.
 *
 * The complement of a binary representation is the number in binary you get when
 * changing every 1 to a 0 and 0 to a 1.  For example, the complement of "101" in binary
is "010" in binary.
 *
 * For a given number N in base-10, return the complement of it's binary representation
as a base-10 integer.
 *
 * Example 1:
 * Input: 5
 * Output: 2
 * Explanation: 5 is "101" in binary, with complement "010" in binary, which is 2 in
base-10.
 *
 * Example 2:
 * Input: 7
 * Output: 0
 * Explanation: 7 is "111" in binary, with complement "000" in binary, which is 0 in
base-10.
 *
 * Example 3:
 * Input: 10
 * Output: 5
 * Explanation: 10 is "1010" in binary, with complement "0101" in binary, which is 5 in
base-10.
 *
 * Note:
 * 0 <= N < 10^9
 * */
public class _1009 {
    public static class Solution1 {
        public int bitwiseComplement(int N) {
            if (N == 0) {
                return 1;
            }
            List<Integer> list = new ArrayList<>();
            while (N != 0) {
                list.add(N & 1);
```

```
                N >>= 1;
            }
            int result = 0;
            int exp = list.size() - 1;
            for (int i = list.size() - 1; i >= 0; i--) {
                if (list.get(i) == 0) {
                    result += Math.pow(2, exp);
                }
                exp--;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 760: _1010.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1010. Pairs of Songs With Total Durations Divisible by 60
 *
 * In a list of songs, the i-th song has a duration of time[i] seconds.
 *
 * Return the number of pairs of songs for which their total duration in seconds is
divisible by 60.
 * Formally, we want the number of indices i < j with (time[i] + time[j]) % 60 == 0.
 *
 * Example 1:
 * Input: [30,20,150,100,40]
 * Output: 3
 * Explanation: Three pairs have a total duration divisible by 60:
 * (time[0] = 30, time[2] = 150): total duration 180
 * (time[1] = 20, time[3] = 100): total duration 120
 * (time[1] = 20, time[4] = 40): total duration 60
 *
 * Example 2:
 * Input: [60,60,60]
 * Output: 3
 * Explanation: All three pairs have a total duration of 120, which is divisible by 60.
 *
 * Note:
 *
 * 1 <= time.length <= 60000
 * 1 <= time[i] <= 500
 * */
public class _1010 {
    public static class Solution1 {
        /**Credit: https://leetcode.com/problems/pairs-of-songs-with-total-durations-
```

```
divisible-by-60/discuss/256726/Java-O(n)-code-w-comment-similar-to-Two-Sum
         *
         * Think of Problem 1: Two Sum
         * Assume target is 60, each item in time % 60.
         * Then this problem becomes very similar to Problem 1.
         * */
        public int numPairsDivisibleBy60(int[] time) {
            int result = 0;
            Map<Integer, Integer> map = new HashMap<>();
            for (int t : time) {
                int d = (60 - t % 60) % 60;
                if (map.containsKey(d)) {
                    result += map.get(d);
                }
                map.put(t % 60, map.getOrDefault(t % 60, 0) + 1);
            }
            return result;
        }
    }
}

// ============================================================
// FILE 761: _1011.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1011. Capacity To Ship Packages Within D Days
 *
 * A conveyor belt has packages that must be shipped from one port to another within D
days.
 *
 * The i-th package on the conveyor belt has a weight of weights[i].
 * Each day, we load the ship with packages on the conveyor belt (in the order given by
weights).
 * We may not load more weight than the maximum weight capacity of the ship.
 *
 * Return the least weight capacity of the ship that will result in all the packages on
the conveyor belt being shipped within D days.
 *
 * Note:
 *  * 1 <= D <= weights.length <= 50000
 *  * 1 <= weights[i] <= 500
 */

public class _1011 {
    public static class Solution1 {
        public int daysToShip(int[] weights, int capacity) {
            int days = 0;
            int currentShip = 0;

            for (int k : weights) {
                if (currentShip + k > capacity) {
```

```
                    currentShip = 0;
                    days += 1;
                }
                currentShip += k;
            }

            return days + 1;
        }

        public int shipWithinDays(int[] weights, int D) {

            int sum = 0;
            int max = 0;

            for (int k : weights) {
                sum += k;
                max = Math.max(max, k);
            }

            // Minimum possible capacity needs to be as much as the heaviest package
            int lower = max;
            // Maximum possible capacity is the total weight of all packages
            int upper = sum;

            if (daysToShip(weights, lower) <= D) {
                return lower;
            }

            // Guess is for capacity
            int currentGuess;
            int bestGuess = -1;

            // Binary search
            while (lower <= upper) {
                currentGuess = (upper + lower) / 2;
                if (daysToShip(weights, currentGuess) <= D) {
                    bestGuess = currentGuess;
                    upper = currentGuess - 1;
                } else {
                    lower = currentGuess + 1;
                }
            }

            return bestGuess;
        }
    }
}

// ==========================================================
// FILE 762: _1013.java
// ==========================================================

package com.fishercoder.solutions;
```

```
/**
 * 1013. Partition Array Into Three Parts With Equal Sum
 *
 * Given an array A of integers, return true if and only if we can partition the array
into three non-empty parts with equal sums.
 * Formally, we can partition the array if we can find indexes i+1 < j with
 * (A[0] + A[1] + ... + A[i] == A[i+1] + A[i+2] + ... + A[j-1] == A[j] + A[j-1] + ... +
A[A.length - 1])
 *
 * Example 1:
 * Input: [0,2,1,-6,6,-7,9,1,2,0,1]
 * Output: true
 * Explanation: 0 + 2 + 1 = -6 + 6 - 7 + 9 + 1 = 2 + 0 + 1
 *
 * Example 2:
 * Input: [0,2,1,-6,6,7,9,-1,2,0,1]
 * Output: false
 *
 * Example 3:
 * Input: [3,3,6,5,-2,2,5,1,-9,4]
 * Output: true
 * Explanation: 3 + 3 = 6 = 5 - 2 + 2 + 5 + 1 - 9 + 4
 *
 * Note:
 *
 * 3 <= A.length <= 50000
 * -10000 <= A[i] <= 10000*/
public class _1013 {
    public static class Solution1 {
        public boolean canThreePartsEqualSum(int[] A) {
            int sum = 0;
            for (int i = 0; i < A.length; i++) {
                sum += A[i];
            }
            if (sum % 3 != 0) {
                return false;
            }
            int equalSum = sum / 3;
            int left = 0;
            int leftSum = 0;
            while (left < A.length - 2 && leftSum != equalSum) {
                leftSum += A[left++];
            }
            if (left > A.length - 2 || leftSum != equalSum) {
                return false;
            }

            int right = A.length - 1;
            int rightSum = 0;
            while (right > left && rightSum != equalSum) {
                rightSum += A[right--];
            }
            if (right < left || rightSum != equalSum) {
                return false;
```

```
            }
            int middleSum = 0;
            for (int i = left; i <= right; i++) {
                middleSum += A[i];
            }
            return middleSum == equalSum;
        }
    }
}

// ============================================================
// FILE 763: _1014.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1014. Best Sightseeing Pair
 *
 * Given an array A of positive integers, A[i] represents the value of the i-th
sightseeing spot,
 * and two sightseeing spots i and j have distance j - i between them.
 *
 * The score of a pair (i < j) of sightseeing spots is (A[i] + A[j] + i - j) :
 * the sum of the values of the sightseeing spots, minus the distance between them.
 *
 * Return the maximum score of a pair of sightseeing spots.
 *
 * Note:
 *  * 2 <= A.length <= 50000
 *  * 1 <= A[i] <= 1000
 */

public class _1014 {
    public static class Solution1 {
        public int maxScoreSightseeingPair(int[] A) {
            int bestPrevious = A[0];
            int maxSum = 0;

            for (int i = 1; i < A.length; ++i) {
                maxSum = Math.max(maxSum, bestPrevious + A[i] - i);
                bestPrevious = Math.max(bestPrevious, A[i] + i);
            }

            return maxSum;
        }
    }
}

// ============================================================
// FILE 764: _1018.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.ArrayList;
import java.util.List;

/**
 * 1018. Binary Prefix Divisible By 5
 *
 * Given an array A of 0s and 1s, consider N_i: the i-th subarray from A[0] to A[i]
interpreted as a binary number (from most-significant-bit to least-significant-bit.)
 *
 * Return a list of booleans answer, where answer[i] is true if and only if N_i is
divisible by 5.
 *
 * Example 1:
 *
 * Input: [0,1,1]
 * Output: [true,false,false]
 * Explanation:
 * The input numbers in binary are 0, 01, 011; which are 0, 1, and 3 in base-10.  Only
the first number is divisible by 5, so answer[0] is true.
 * Example 2:
 *
 * Input: [1,1,1]
 * Output: [false,false,false]
 * Example 3:
 *
 * Input: [0,1,1,1,1,1]
 * Output: [true,false,false,false,true,false]
 * Example 4:
 *
 * Input: [1,1,1,0,1]
 * Output: [false,false,false,false,false]
 *
 *
 * Note:
 *
 * 1 <= A.length <= 30000
 * A[i] is 0 or 1
 * */
public class _1018 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/binary-prefix-divisible-by-
5/discuss/266051/Java-beats-100*/
        public List<Boolean> prefixesDivBy5(int[] A) {
            List<Boolean> result = new ArrayList<>(A.length);
            int remainder = 0;
            for (int a : A) {
                remainder = ((remainder << 1) + a) % 5;
                result.add(remainder == 0);
            }
            return result;
        }
    }
}
```

```
// ============================================================
// FILE 765: _1020.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1020. Number of Enclaves
 *
 * Given a 2D array A, each cell is 0 (representing sea) or 1 (representing land)
 *
 * A move consists of walking from one land square 4-directionally to another land
square, or off the boundary of the grid.
 *
 * Return the number of land squares in the grid for which we cannot walk off the
boundary of the grid in any number of moves.
 */

public class _1020 {
    public static class Solution1 {
        public void walk(int[][] A, boolean[][] visited, int x, int y) {
            if (x >= A.length || x < 0 || y >= A[0].length || y < 0) {
                return;
            }
            if (visited[x][y]) {
                return;
            }
            if (A[x][y] == 0) {
                return;
            }

            visited[x][y] = true;

            walk(A, visited, x - 1, y);
            walk(A, visited, x, y - 1);
            walk(A, visited, x, y + 1);
            walk(A, visited, x + 1, y);
        }

        public int numEnclaves(int[][] A) {
            int n = A.length;
            int m = A[0].length;
            boolean[][] visited = new boolean[n][m];

            for (int i = 0; i < n; ++i) {
                walk(A, visited, i, 0);
                walk(A, visited, i, m - 1);
            }

            for (int j = 0; j < m; ++j) {
                walk(A, visited, 0, j);
                walk(A, visited, n - 1, j);
            }
```

```
            int unreachables = 0;
            for (int i = 0; i < n; ++i) {
                for (int j = 0; j < m; ++j) {
                    if (A[i][j] == 1 && !visited[i][j]) {
                        ++unreachables;
                    }
                }
            }

            return unreachables;
        }
    }
}

// ======================================================
// FILE 766: _1021.java
// ======================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**1021. Remove Outermost Parentheses
 *
 * A valid parentheses string is either empty (""), "(" + A + ")", or A + B, where A and
B are valid parentheses strings, and + represents string concatenation.  For example, "",
"()", "(())()", and "(()(()))" are all valid parentheses strings.
 *
 * A valid parentheses string S is primitive if it is nonempty, and there does not exist
a way to split it into S = A+B, with A and B nonempty valid parentheses strings.
 *
 * Given a valid parentheses string S, consider its primitive decomposition: S = P_1 +
P_2 + ... + P_k, where P_i are primitive valid parentheses strings.
 *
 * Return S after removing the outermost parentheses of every primitive string in the
primitive decomposition of S.
 *
 *
 * Example 1:
 * Input: "((()))(())"
 * Output: "()()()"
 * Explanation:
 * The input string is "((()))(())", with primitive decomposition "((()))" + "(())".
 * After removing outer parentheses of each part, this is "()()" + "()" = "()()()".
 *
 * Example 2:
 * Input: "((())())(()(()))"
 * Output: "()())()(())"
 * Explanation:
 * The input string is "((())())(()(()))", with primitive decomposition "((()))" + "
(())" + "((()))".
 * After removing outer parentheses of each part, this is "()()" + "()" + "()(())" = "()
```

```
()()()(())".
 *
 * Example 3:
 * Input: "()()"
 * Output: ""
 * Explanation:
 * The input string is "()()", with primitive decomposition "()" + "()".
 * After removing outer parentheses of each part, this is "" + "" = "".
 *
 * Note:
 *
 * S.length <= 10000
 * S[i] is "(" or ")"
 * S is a valid parentheses string
 * */
public class _1021 {
    public static class Solution1 {
        public String removeOuterParentheses(String S) {
            List<String> primitives = new ArrayList<>();
            for (int i = 1; i < S.length(); i++) {
                int initialI = i - 1;
                int left = 1;
                while (i < S.length() && left > 0) {
                    if (S.charAt(i) == '(') {
                        left++;
                    } else {
                        left--;
                    }
                    i++;
                }
                primitives.add(S.substring(initialI, i));
            }
            StringBuilder sb = new StringBuilder();
            for (String primitive : primitives) {
                sb.append(primitive.substring(1, primitive.length() - 1));
            }
            return sb.toString();
        }
    }
}

// ============================================================
// FILE 767: _1022.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.List;

/**
 * Source: https://leetcode.com/problems/sum-of-root-to-leaf-binary-numbers/
```

```
 *
 * 1022. Sum of Root To Leaf Binary Numbers
 *
 * Given a binary tree, each node has value 0 or 1.  Each root-to-leaf path represents a
binary number starting with the most significant bit.  For example, if the path is 0 -> 1
-> 1 -> 0 -> 1, then this could represent 01101 in binary, which is 13.
 *
 * For all leaves in the tree, consider the numbers represented by the path from the root
to that leaf.
 *
 * Return the sum of these numbers.
 *
 * Example 1:
 *
 *        1
 *       / \
 *      0   1
 *     / \ / \
 *    0  1 0 1
 *
 * Input: [1,0,1,0,1,0,1]
 * Output: 22
 * Explanation: (100) + (101) + (110) + (111) = 4 + 5 + 6 + 7 = 22
 *
 * Note:
 *
 * The number of nodes in the tree is between 1 and 1000.
 * node.val is 0 or 1.
 * The answer will not exceed 2^31 - 1.
 * */
public class _1022 {
    public static class Solution1 {
        public int sumRootToLeaf(TreeNode root) {
            List<List<Integer>> paths = new ArrayList<>();
            dfs(root, paths, new ArrayList<>());
            int sum = 0;
            for (List<Integer> list : paths) {
                int num = 0;
                for (int i : list) {
                    num = (num << 1) + i;
                }
                sum += num;
            }
            return sum;
        }

        private void dfs(TreeNode root, List<List<Integer>> paths, List<Integer> path) {
            path.add(root.val);
            if (root.left != null) {
                dfs(root.left, paths, path);
                path.remove(path.size() - 1);
            }
            if (root.right != null) {
                dfs(root.right, paths, path);
```

```
                path.remove(path.size() - 1);
            }
            if (root.left == null && root.right == null) {
                paths.add(new ArrayList<>(path));
            }
        }
    }
}

// ============================================================
// FILE 768: _1029.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1029. Two City Scheduling
 *
 * There are 2N people a company is planning to interview.
 * The cost of flying the i-th person to city A is costs[i][0], and the cost of flying
the i-th person to city B is costs[i][1].
 * Return the minimum cost to fly every person to a city such that exactly N people
arrive in each city.
 *
 * Example 1:
 *
 * Input: [[10,20],[30,200],[400,50],[30,20]]
 * Output: 110
 * Explanation:
 * The first person goes to city A for a cost of 10.
 * The second person goes to city A for a cost of 30.
 * The third person goes to city B for a cost of 50.
 * The fourth person goes to city B for a cost of 20.
 *
 * The total minimum cost is 10 + 30 + 50 + 20 = 110 to have half the people interviewing
in each city.
 *
 * Note:
 *
 * 1 <= costs.length <= 100
 * It is guaranteed that costs.length is even.
 * 1 <= costs[i][0], costs[i][1] <= 1000
 * */
public class _1029 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/two-city-scheduling/discuss/280173/Java-
4-lines-intuitive-solution
         * and
         * https://leetcode.com/problems/two-city-scheduling/discuss/278771/Java-sort-
solution*/
        public int twoCitySchedCost(int[][] costs) {
            Arrays.sort(costs, (a, b) -> (a[0] - a[1] - (b[0] - b[1])));
```

```
            int cost = 0;
            for (int i = 0; i < costs.length; i++) {
                if (i < costs.length / 2) {
                    cost += costs[i][0];
                } else {
                    cost += costs[i][1];
                }
            }
            return cost;
        }
    }
}

// ============================================================
// FILE 769: _1030.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 1030. Matrix Cells in Distance Order
 *
 * We are given a matrix with R rows and C columns has cells with integer coordinates (r,
c), where 0 <= r < R and 0 <= c < C.
 *
 * Additionally, we are given a cell in that matrix with coordinates (r0, c0).
 *
 * Return the coordinates of all cells in the matrix, sorted by their distance from (r0,
c0) from smallest distance to largest distance.  Here, the distance between two cells
(r1, c1) and (r2, c2) is the Manhattan distance, |r1 - r2| + |c1 - c2|.  (You may return
the answer in any order that satisfies this condition.)
 *
 * Example 1:
 * Input: R = 1, C = 2, r0 = 0, c0 = 0
 * Output: [[0,0],[0,1]]
 * Explanation: The distances from (r0, c0) to other cells are: [0,1]
 *
 * Example 2:
 * Input: R = 2, C = 2, r0 = 0, c0 = 1
 * Output: [[0,1],[0,0],[1,1],[1,0]]
 * Explanation: The distances from (r0, c0) to other cells are: [0,1,1,2]
 * The answer [[0,1],[1,1],[0,0],[1,0]] would also be accepted as correct.
 *
 * Example 3:
 * Input: R = 2, C = 3, r0 = 1, c0 = 2
 * Output: [[1,2],[0,2],[1,1],[0,1],[1,0],[0,0]]
 * Explanation: The distances from (r0, c0) to other cells are: [0,1,1,2,2,3]
 * There are other answers that would also be accepted as correct, such as [[1,2],[1,1],
[0,2],[1,0],[0,1],[0,0]].
```

```
 *
 * Note:
 *
 * 1 <= R <= 100
 * 1 <= C <= 100
 * 0 <= r0 < R
 * 0 <= c0 < C
 * */
public class _1030 {
    public static class Solution1 {
        public int[][] allCellsDistOrder(int R, int C, int r0, int c0) {
            int[][] result = new int[R * C][2];
            Queue<int[]> queue = new LinkedList<>();
            queue.offer(new int[]{r0, c0});
            boolean[][] visited = new boolean[R][C];
            int i = 0;
            while (!queue.isEmpty()) {
                int[] cell = queue.poll();
                int row = cell[0];
                int col = cell[1];

                if (row < 0 || row >= R || col < 0 || col >= C || visited[row][col]) {
                    continue;
                }

                visited[row][col] = true;
                result[i++] = new int[]{row, col};
                queue.offer(new int[]{row, col + 1});
                queue.offer(new int[]{row + 1, col});
                queue.offer(new int[]{row - 1, col});
                queue.offer(new int[]{row, col - 1});

            }
            return result;
        }
    }
}

// ============================================================
// FILE 770: _1033.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1033. Moving Stones Until Consecutive
 *
 * Three stones are on a number line at positions a, b, and c.
 *
 * Each turn, you pick up a stone at an endpoint (ie., either the lowest or highest
position stone),
 * and move it to an unoccupied position between those endpoints.
```

```
 * Formally, let's say the stones are currently at positions x, y, z with x < y < z.
 * You pick up the stone at either position x or position z, and move that stone to an
integer position k, with x < k < z and k != y.
 *
 * The game ends when you cannot make any more moves, ie. the stones are in consecutive
positions.
 *
 * When the game ends, what is the minimum and maximum number of moves that you could
have made?
 * Return the answer as an length 2 array: answer = [minimum_moves, maximum_moves]
 *
 * Note:
 *  * 1 <= a <= 100
 *  *  1 <= b <= 100
 *  *  1 <= c <= 100
 *  *  a != b, b != c, c != a
 */

public class _1033 {
    public static class Solution1 {
        private int minMoves(int x, int y, int z) {
            // already consecutive integers, nothing to be done
            if (x + 1 == y && y + 1 == z) {
                return 0;
            }
            // one of the following (sample) cases:
            // 1, 2, 8 (8 -> 3)
            // 1, 7, 8 (1 -> 6)
            // 1, 3, 8 (8 -> 2)
            // 1, 6, 8 (1 -> 7)
            if (y - x <= 2 || z - y <= 2) {
                return 1;
            }

            // move z to y + 1, x to y - 1
            return 2;
        }

        private int maxMoves(int x, int y, int z) {
            return z - x - 2;
        }

        public int[] numMovesStones(int a, int b, int c) {
            int[] t = {a, b, c};
            Arrays.sort(t);

            int min = minMoves(t[0], t[1], t[2]);
            int max = maxMoves(t[0], t[1], t[2]);

            return new int[]{min, max};
        }
    }
}
```

```
// ============================================================
// FILE 771: _1037.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1037. Valid Boomerang
 *
 * A boomerang is a set of 3 points that are all distinct and not in a straight line.
 *
 * Given a list of three points in the plane, return whether these points are a
boomerang.
 *
 * Example 1:
 * Input: [[1,1],[2,3],[3,2]]
 * Output: true
 *
 * Example 2:
 * Input: [[1,1],[2,2],[3,3]]
 * Output: false
 *
 * Note:
 * points.length == 3
 * points[i].length == 2
 * 0 <= points[i][j] <= 100
 * */
public class _1037 {
    public static class Solution1 {
        public boolean isBoomerang(int[][] points) {
            return (points[1][1] - points[0][1]) * (points[2][0] - points[0][0]) !=
(points[2][1] - points[0][1]) * (points[1][0] - points[0][0]);
        }
    }
}

// ============================================================
// FILE 772: _1038.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

/**
 * 1038. Binary Search Tree to Greater Sum Tree
 *
 * Given the root of a binary search tree with distinct values,
 * modify it so that every node has a new value equal to the sum of the values of the
original tree that are greater than or equal to node.val.
 * As a reminder, a binary search tree is a tree that satisfies these constraints:
 *
 * The left subtree of a node contains only nodes with keys less than the node's key.
 * The right subtree of a node contains only nodes with keys greater than the node's key.
```

```
 * Both the left and right subtrees must also be binary search trees.
 *
 *
 * Example 1:
 *                            4(30)
 *                           /    \
 *                       1(36)   6(21)
 *                      /   \     /    \
 *                 0(36)  2(35) 5(26) 7(15)
 *                          \          \
 *                         3(33)        8(8)
 *
 * Input: [4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]
 * Output: [30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]
 *
 *
 * Note:
 *
 * The number of nodes in the tree is between 1 and 100.
 * Each node will have value between 0 and 100.
 * The given tree is a binary search tree.*/
public class _1038 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/binary-search-tree-to-greater-sum-
tree/discuss/286725/JavaC%2B%2BPython-Revered-Inorder-Traversal*/
        int greaterSum = 0;

        public TreeNode bstToGst(TreeNode root) {
            if (root.right != null) {
                bstToGst(root.right);
            }
            greaterSum = root.val = greaterSum + root.val;
            if (root.left != null) {
                bstToGst(root.left);
            }
            return root;
        }
    }
}

// ============================================================
// FILE 773: _1043.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1043. Partition Array for Maximum Sum
 *
 * Given an integer array A, you partition the array into (contiguous) subarrays of
length at most K.  After partitioning, each subarray has their values changed to become
the maximum value of that subarray.
 * Return the largest sum of the given array after partitioning.
 *
```

```
 * Example 1:
 * Input: A = [1,15,7,9,2,5,10], K = 3
 * Output: 84
 * Explanation: A becomes [15,15,15,9,10,10,10]
 *
 * Note:
 * 1 <= K <= A.length <= 500
 * 0 <= A[i] <= 10^6
 * */
public class _1043 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/partition-array-for-maximum-
sum/discuss/290863/JavaC%2B%2BPython-DP*/
        public int maxSumAfterPartitioning(int[] A, int K) {
            int N = A.length;
            int[] dp = new int[N];
            for (int i = 0; i < N; i++) {
                int curMax = 0;
                for (int k = 1; k <= K && i - k + 1 >= 0; k++) {
                    curMax = Math.max(curMax, A[i - k + 1]);
                    dp[i] = Math.max(dp[i], (i >= k ? dp[i - k] : 0) + curMax * k);
                }
            }
            return dp[N - 1];
        }
    }
}

// ============================================================
// FILE 774: _1046.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 1046. Last Stone Weight
 *
 * We have a collection of rocks, each rock has a positive integer weight.
 *
 * Each turn, we choose the two heaviest rocks and smash them together.
 * Suppose the stones have weights x and y with x <= y.  The result of this smash is:
 *
 * If x == y, both stones are totally destroyed;
 * If x != y, the stone of weight x is totally destroyed, and the stone of weight y has
new weight y-x.
 * At the end, there is at most 1 stone left.  Return the weight of this stone (or 0 if
there are no stones left.)
 *
 * Example 1:
 * Input: [2,7,4,1,8,1]
 * Output: 1
 * Explanation:
```

```
 * We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then,
 * we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then,
 * we combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
 * we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of last
stone.
 *
 * Note:
 * 1 <= stones.length <= 30
 * 1 <= stones[i] <= 1000
 * */
public class _1046 {
    public static class Solution1 {
        public int lastStoneWeight(int[] stones) {
            PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> b - a);
            for (int stone : stones) {
                heap.offer(stone);
            }
            while (!heap.isEmpty()) {
                if (heap.size() >= 2) {
                    int one = heap.poll();
                    int two = heap.poll();
                    int diff = one - two;
                    heap.offer(diff);
                } else {
                    return heap.poll();
                }
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 775: _1047.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1047. Remove All Adjacent Duplicates In String
 *
 * Given a string S of lowercase letters, a duplicate removal consists of choosing two
adjacent and equal letters, and removing them.
 *
 * We repeatedly make duplicate removals on S until we no longer can.
 *
 * Return the final string after all such duplicate removals have been made.  It is
guaranteed the answer is unique.
 *
 *
 *
 * Example 1:
 *
 * Input: "abbaca"
```

```
 * Output: "ca"
 * Explanation:
 * For example, in "abbaca" we could remove "bb" since the letters are adjacent and
equal, and this is the only possible move.
 * The result of this move is that the string is "aaca", of which only "aa" is possible,
so the final string is "ca".
 *
 *
 * Note:
 *
 * 1 <= S.length <= 20000
 * S consists only of English lowercase letters.*/
public class _1047 {
    public static class Solution1 {
        public String removeDuplicates(String S) {
            StringBuilder sb = new StringBuilder(S);
            for (int i = 0; i < S.length() - 1; i++) {
                if (S.charAt(i) == S.charAt(i + 1)) {
                    return removeDuplicates(S.substring(0, i) + S.substring(i + 2));
                }
            }
            return sb.toString();
        }
    }
}

// ============================================================
// FILE 776: _1049.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1049. Last Stone Weight II
 *
 * We have a collection of rocks, each rock has a positive integer weight.
 * Each turn, we choose any two rocks and smash them together.
 * Suppose the stones have weights x and y with x <= y.  The result of this smash is:
 * If x == y, both stones are totally destroyed;
 * If x != y, the stone of weight x is totally destroyed, and the stone of weight y has
new weight y-x.
 * At the end, there is at most 1 stone left.  Return the smallest possible weight of
this stone (the weight is 0 if there are no stones left.)
 *
 * Example 1:
 * Input: [2,7,4,1,8,1]
 * Output: 1
 *
 * Explanation:
 * We can combine 2 and 4 to get 2 so the array converts to [2,7,1,8,1] then,
 * we can combine 7 and 8 to get 1 so the array converts to [2,1,1,1] then,
 * we can combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
 * we can combine 1 and 1 to get 0 so the array converts to [1] then that's the optimal
value.
```

```
 *
 * Note:
 * 1 <= stones.length <= 30
 * 1 <= stones[i] <= 100
 * */
public class _1049 {
    public static class Solution1 {
        public int lastStoneWeightII(int[] stones) {
            //TODO: implement it
            return 1;
        }
    }
}

// ============================================================
// FILE 777: _1051.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1051. Height Checker
 *
 * Students are asked to stand in non-decreasing order of heights for an annual photo.
 * Return the minimum number of students not standing in the right positions.
 * (This is the number of students that must move in order for all students to be
standing in non-decreasing order of height.)
 *
 * Example 1:
 * Input: [1,1,4,2,1,3]
 * Output: 3
 * Explanation:
 * Students with heights 4, 3 and the last 1 are not standing in the right positions.
 *
 * Note:
 *
 * 1 <= heights.length <= 100
 * 1 <= heights[i] <= 100
 * */
public class _1051 {
    public static class Solution1 {
        public int heightChecker(int[] heights) {
            int[] originals = Arrays.copyOf(heights, heights.length);
            Arrays.sort(heights);
            int count = 0;
            for (int i = 0; i < originals.length; i++) {
                if (heights[i] != originals[i]) {
                    count++;
                }
            }
            return count;
        }
```

```
    }
}

// ============================================================
// FILE 778: _1055.java
// ============================================================

package com.fishercoder.solutions;

import java.util.stream.IntStream;

/**
 * 1055. Fixed Point
 *
 * Given an array A of distinct integers sorted in ascending order,
 * return the smallest index i that satisfies A[i] == i.  Return -1 if no such i exists.
 *
 * Example 1:
 * Input: [-10,-5,0,3,7]
 * Output: 3
 * Explanation:
 * For the given array, A[0] = -10, A[1] = -5, A[2] = 0, A[3] = 3, thus the output is 3.
 *
 * Example 2:
 * Input: [0,2,5,8,17]
 * Output: 0
 * Explanation:
 * A[0] = 0, thus the output is 0.
 *
 * Example 3:
 * Input: [-10,-5,3,4,7,9]
 * Output: -1
 * Explanation:
 * There is no such i that A[i] = i, thus the output is -1.
 *
 * Note:
 *
 * 1 <= A.length < 10^4
 * -10^9 <= A[i] <= 10^9
 * */
public class _1055 {
    public static class Solution1 {
        public int fixedPoint(int[] A) {
            return IntStream.range(0, A.length).filter(i -> A[i] ==
i).findFirst().orElse(-1);
        }
    }
}

// ============================================================
// FILE 779: _1056.java
// ============================================================

package com.fishercoder.solutions;
```

```
import java.util.HashMap;
import java.util.Map;

/**
 * 1056. Confusing Number
 *
 * Given a number N, return true if and only if it is a confusing number, which satisfies
the following condition:
 * We can rotate digits by 180 degrees to form new digits. When 0, 1, 6, 8, 9 are rotated
180 degrees,
 * they become 0, 1, 9, 8, 6 respectively. When 2, 3, 4, 5 and 7 are rotated 180 degrees,
 * they become invalid. A confusing number is a number that when rotated 180 degrees
becomes a different number with each digit valid.
 *
 * Example 1:
 * Input: 6
 * Output: true
 * Explanation:
 * We get 9 after rotating 6, 9 is a valid number and 9!=6.
 *
 *
 * Example 2:
 * Input: 89
 * Output: true
 * Explanation:
 * We get 68 after rotating 89, 86 is a valid number and 86!=89.
 *
 *
 * Example 3:
 * Input: 11
 * Output: false
 * Explanation:
 * We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus
11 is not a confusing number.
 *
 * Example 4:
 * Input: 25
 * Output: false
 * Explanation:
 * We get an invalid number after rotating 25.
 *
 *  Note:
 *
 * 0 <= N <= 10^9
 * After the rotation we can ignore leading zeros, for example if after rotation we have
0008 then this number is considered as just 8.*/
public class _1056 {
    public static class Solution1 {
        Map<Integer, Integer> map = new HashMap<Integer, Integer>() {{
            put(0, 0);
            put(1, 1);
            put(8, 8);
            put(6, 9);
```

```
            put(9, 6);
        }
        };

        public boolean confusingNumber(int N) {
            if (N == 0) {
                return false;
            }
            int newNumber = 0;
            int originalN = N;
            while (N != 0) {
                newNumber *= 10;
                int digit = N % 10;
                if (!map.containsKey(digit)) {
                    return false;
                }
                digit = map.get(digit);
                newNumber += digit;
                N /= 10;
            }
            return newNumber != originalN;
        }
    }
}

// ============================================================
// FILE 780: _1065.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

/**
 * 1065. Index Pairs of a String
 *
 * Given a text string and words (a list of strings),
 * return all index pairs [i, j] so that the substring text[i]...text[j] is in the list
of words.
 *
 * Example 1:
 * Input: text = "thestoryofleetcodeandme", words = ["story","fleet","leetcode"]
 * Output: [[3,7],[9,13],[10,17]]
 *
 * Example 2:
 * Input: text = "ababa", words = ["aba","ab"]
 * Output: [[0,1],[0,2],[2,3],[2,4]]
 * Explanation:
 * Notice that matches can overlap, see "aba" is found in [0,2] and [2,4].
 *
 * Note:
```

```
 *
 * All strings contains only lowercase English letters.
 * It's guaranteed that all strings in words are different.
 * 1 <= text.length <= 100
 * 1 <= words.length <= 20
 * 1 <= words[i].length <= 50
 * Return the pairs [i,j] in sorted order (i.e. sort them by their first coordinate in
case of ties sort them by their second coordinate).
 * */
public class _1065 {
    public static class Solution1 {
        public int[][] indexPairs(String text, String[] words) {
            List<List<Integer>> lists = new ArrayList<>();
            for (String word : words) {
                lists.addAll(findAllMatchsForThisWord(word, text));
            }
            if (lists.isEmpty()) {
                return new int[][]{};
            }
            Collections.sort(lists, (o1, o2) -> {
                if (o1.get(0) > o2.get(0)) {
                    return 1;
                } else if (o1.get(0) < o2.get(0)) {
                    return -1;
                } else {
                    if (o1.get(1) > o2.get(1)) {
                        return 1;
                    } else {
                        return -1;
                    }
                }
            });
            int[][] result = new int[lists.size()][lists.get(0).size()];
            for (int i = 0; i < lists.size(); i++) {
                result[i][0] = lists.get(i).get(0);
                result[i][1] = lists.get(i).get(1);
            }
            return result;
        }

        private List<List<Integer>> findAllMatchsForThisWord(String word, String text) {
            List<List<Integer>> lists = new ArrayList<>();
            for (int i = 0; i <= text.length() - word.length(); i++) {
                if (text.substring(i, i + word.length()).equals(word)) {
                    lists.add(Arrays.asList(i, i + word.length() - 1));
                }
            }
            return lists;
        }
    }
}

// ==========================================================
// FILE 781: _1071.java
```

```java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1071. Greatest Common Divisor of Strings
 *
 * For strings S and T, we say "T divides S" if and only if S = T + ... + T  (T
concatenated with itself 1 or more times)
 * Return the largest string X such that X divides str1 and X divides str2.
 *
 * Example 1:
 * Input: str1 = "ABCABC", str2 = "ABC"
 * Output: "ABC"
 *
 * Example 2:
 * Input: str1 = "ABABAB", str2 = "ABAB"
 * Output: "AB"
 *
 * Example 3:
 * Input: str1 = "LEET", str2 = "CODE"
 * Output: ""
 *
 * Note:
 *
 * 1 <= str1.length <= 1000
 * 1 <= str2.length <= 1000
 * str1[i] and str2[i] are English uppercase letters.
 * */
public class _1071 {
    public static class Solution1 {
        public String gcdOfStrings(String str1, String str2) {
            if (str1.isEmpty() || str2.isEmpty()) {
                return "";
            }
            String commomDivisor = str2;
            while (commomDivisor != null && !commomDivisor.isEmpty()) {
                if (isDivisor(str1, commomDivisor)) {
                    return commomDivisor;
                } else {
                    commomDivisor = findNextShorterCommonDivisor(str2, commomDivisor);
                }
            }
            return "";
        }

        private String findNextShorterCommonDivisor(String str2, String commomDivisor) {
            int length = nextPossibleLength(str2, commomDivisor.length() - 1);
            while (length > 1) {
                if (isDivisor(str2, length)) {
                    return str2.substring(0, length);
                }
                length = nextPossibleLength(str2, length - 1);
            }
```

```
            return null;
        }

        private boolean isDivisor(String str2, int length) {
            String commonDivisorCandidate = str2.substring(0, length);
            for (int i = 0; i < str2.length() - length; i += length) {
                if (!str2.substring(i, i + length).equals(commonDivisorCandidate)) {
                    return false;
                }
            }
            return true;
        }

        private boolean isDivisor(String str1, String commomDivisor) {
            if (str1.length() == commomDivisor.length()) {
                return str1.equals(commomDivisor);
            }
            int i = 0;
            for (; i < str1.length() - commomDivisor.length(); i +=
commomDivisor.length()) {
                if (!str1.substring(i, i + commomDivisor.length()).equals(commomDivisor))
{
                    return false;
                }
            }
            return i == (str1.length() - commomDivisor.length());
        }

        private int nextPossibleLength(String str2, int bound) {
            if (bound <= 0) {
                return -1;
            }
            int len = bound;
            while (str2.length() % len != 0) {
                len--;
            }
            return len;
        }
    }
}

// ==========================================================
// FILE 782: _1078.java
// ==========================================================

package com.fishercoder.solutions;

import java.util.stream.Collectors;
import java.util.stream.IntStream;

/**
 * 5083. Occurrences After Bigram
 * <p>
 * Given words first and second, consider occurrences in some text of the form "first
```

```
second third",
 * where second comes immediately after first, and third comes immediately after second.
 * For each such occurrence, add "third" to the answer, and return the answer.
 * <p>
 * Example 1:
 * Input: text = "alice is a good girl she is a good student", first = "a", second =
"good"
 * Output: ["girl","student"]
 * <p>
 * Example 2:
 * Input: text = "we will we will rock you", first = "we", second = "will"
 * Output: ["we","rock"]
 * <p>
 * Note:
 * 1 <= text.length <= 1000
 * text consists of space separated words, where each word consists of lowercase English
letters.
 * 1 <= first.length, second.length <= 10
 * first and second consist of lowercase English letters.
 */
public class _1078 {
    public static class Solution1 {
        public String[] findOcurrences(String text, String first, String second) {
            String[] words = text.split(" ");
            return IntStream
                    .range(0, words.length - 2)
                    .filter(i -> words[i].equals(first) && words[i + 1].equals(second))
                    .mapToObj(i -> words[i + 2])
                    .collect(Collectors.toList())
                    .stream()
                    .toArray(String[]::new);
        }
    }
}

// ============================================================
// FILE 783: _1079.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.stream.IntStream;

/**
 * 5087. Letter Tile Possibilities
 *
 * You have a set of tiles, where each tile has one letter tiles[i] printed on it.
 * Return the number of possible non-empty sequences of letters you can make.
 *
 * Example 1:
 * Input: "AAB"
```

```
 * Output: 8
 * Explanation: The possible sequences are "A", "B", "AA", "AB", "BA", "AAB", "ABA",
"BAA".
 *
 * Example 2:
 * Input: "AAABBC"
 * Output: 188
 *
 * Note:
 * 1. 1 <= tiles.length <= 7
 * 2. tiles consists of uppercase English letters.
 * */
public class _1079 {
    public static class Solution1 {
        public int numTilePossibilities(String tiles) {
            char[] chars = tiles.toCharArray();
            Arrays.sort(chars);
            boolean[] used = new boolean[chars.length];
            StringBuilder sb = new StringBuilder();
            List<String> result = new ArrayList<>();
            dfs(chars, used, sb, result);
            return result.size();
        }

        private void dfs(char[] chars, boolean[] used, StringBuilder sb, List<String>
result) {
            if (sb.length() != 0) {
                result.add(sb.toString());
            }
            IntStream.range(0, chars.length)
                    .filter(i -> !used[i])
                    .filter(i -> i <= 0 || chars[i - 1] != chars[i] || used[i - 1])
                    .forEach(i -> {
                        used[i] = true;
                        sb.append(chars[i]);
                        dfs(chars, used, sb, result);
                        used[i] = false;
                        sb.deleteCharAt(sb.length() - 1);
                    });
        }
    }
}

// ============================================================
// FILE 784: _1085.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1085. Sum of Digits in the Minimum Number
 *
 * Given an array A of positive integers, let S be the sum of the digits of the minimal
element of A.
```

```
 * Return 0 if S is odd, otherwise return 1.
 *
 * Example 1:
 * Input: [34,23,1,24,75,33,54,8]
 * Output: 0
 * Explanation:
 * The minimal element is 1, and the sum of those digits is S = 1 which is odd, so the
answer is 0.
 *
 * Example 2:
 * Input: [99,77,33,66,55]
 * Output: 1
 * Explanation:
 * The minimal element is 33, and the sum of those digits is S = 3 + 3 = 6 which is even,
so the answer is 1.
 *
 * Note:
 * 1 <= A.length <= 100
 * 1 <= A[i].length <= 100
 * */
public class _1085 {
    public static class Solution1 {
        public int sumOfDigits(int[] A) {
            int smallestNumber = A[0];
            for (int i = 1; i < A.length; i++) {
                smallestNumber = Math.min(smallestNumber, A[i]);
            }
            int sum = 0;
            while (smallestNumber > 0) {
                sum += smallestNumber % 10;
                smallestNumber /= 10;
            }
            return sum % 2 == 0 ? 1 : 0;
        }
    }
}

// ============================================================
// FILE 785: _1086.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.PriorityQueue;
import java.util.TreeMap;

/**
 * 1086. High Five
 *
 * Given a list of scores of different students, return the average score of each
student's top five scores in the order of each student's id.
```

```
 * Each entry items[i] has items[i][0] the student's id, and items[i][1] the student's
score.
 * The average score is calculated using integer division.
 *
 * Example 1:
 * Input: [[1,91],[1,92],[2,93],[2,97],[1,60],[2,77],[1,65],[1,87],[1,100],[2,100],
[2,76]]
 * Output: [[1,87],[2,88]]
 * Explanation:
 * The average of the student with id = 1 is 87.
 * The average of the student with id = 2 is 88.6. But with integer division their
average converts to 88.
 *
 * Note:
 * 1 <= items.length <= 1000
 * items[i].length == 2
 * The IDs of the students is between 1 to 1000
 * The score of the students is between 1 to 100
 * For each student, there are at least 5 scores
 * */
public class _1086 {
    public static class Solution1 {
        public int[][] highFive(int[][] items) {
            TreeMap<Integer, List<Integer>> map = new TreeMap<>();
            for (int[] studentToScore : items) {
                if (map.containsKey(studentToScore[0])) {
                    map.get(studentToScore[0]).add(studentToScore[1]);
                } else {
                    List<Integer> list = new ArrayList<>();
                    list.add(studentToScore[1]);
                    map.put(studentToScore[0], list);
                }
            }
            int[][] result = new int[map.size()][2];
            for (int id : map.keySet()) {
                List<Integer> scores = map.get(id);
                Collections.sort(scores);
                int sum = 0;
                for (int i = scores.size() - 1; i >= scores.size() - 5 && i >= 0; i--) {
                    sum += scores.get(i);
                }
                result[id - 1][0] = id;
                result[id - 1][1] = sum / 5;
            }
            return result;
        }
    }

    public static class Solution2 {
        public int[][] highFive(int[][] items) {
            TreeMap<Integer, PriorityQueue<Integer>> treeMap = new TreeMap<>();
            for (int[] studentToScores : items) {
                if (treeMap.containsKey(studentToScores[0])) {
                    PriorityQueue<Integer> maxHeap = treeMap.get(studentToScores[0]);
```

```
                    maxHeap.offer(studentToScores[1]);
                    if (maxHeap.size() > 5) {
                        maxHeap.poll();
                    }
                    treeMap.put(studentToScores[0], maxHeap);
                } else {
                    PriorityQueue<Integer> maxHeap = new PriorityQueue<>();
                    maxHeap.offer(studentToScores[1]);
                    treeMap.put(studentToScores[0], maxHeap);
                }
            }
            int[][] result = new int[treeMap.size()][2];
            for (int id : treeMap.keySet()) {
                result[id - 1][0] = id;
                int sum = 0;
                PriorityQueue<Integer> maxHeap = treeMap.get(id);
                while (!maxHeap.isEmpty()) {
                    sum += maxHeap.poll();
                }
                result[id - 1][1] = sum / 5;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 786: _1089.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1089. Duplicate Zeros
 *
 * Given a fixed length array arr of integers, duplicate each occurrence of zero,
shifting the remaining elements to the right.
 *
 * Note that elements beyond the length of the original array are not written.
 *
 * Do the above modifications to the input array in place, do not return anything from
your function.
 *
 * Example 1:
 * Input: [1,0,2,3,0,4,5,0]
 * Output: null
 * Explanation: After calling your function, the input array is modified to:
[1,0,0,2,3,0,0,4]
 *
 * Example 2:
 * Input: [1,2,3]
 * Output: null
 * Explanation: After calling your function, the input array is modified to: [1,2,3]
 *
```

```
 * Note:
 *
 * 1 <= arr.length <= 10000
 * 0 <= arr[i] <= 9
 * */
public class _1089 {
    public static class Solution1 {
        public void duplicateZeros(int[] arr) {
            for (int i = 0; i < arr.length - 1; i++) {
                if (arr[i] == 0) {
                    duplicateZeros(arr, i + 1);
                    i++;
                }
            }
        }

        private void duplicateZeros(int[] arr, int zeroIndex) {
            int tmp = arr[zeroIndex];
            arr[zeroIndex] = 0;
            for (int i = zeroIndex + 1; i < arr.length; i++) {
                int next = arr[i];
                arr[i] = tmp;
                tmp = next;
            }
        }
    }
}

// ============================================================
// FILE 787: _1099.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1099. Two Sum Less Than K
 *
 * Given an array A of integers and integer K,
 * return the maximum S such that there exists i < j with A[i] + A[j] = S and S < K.
 * If no i, j exist satisfying this equation, return -1.
 *
 * Example 1:
 * Input: A = [34,23,1,24,75,33,54,8], K = 60
 * Output: 58
 * Explanation:
 * We can use 34 and 24 to sum 58 which is less than 60.
 *
 * Example 2:
 * Input: A = [10,20,30], K = 15
 * Output: -1
 * Explanation:
 * In this case it's not possible to get a pair sum less that 15.
```

```
 *
 * Note:
 * 1 <= A.length <= 100
 * 1 <= A[i] <= 1000
 * 1 <= K <= 2000
 * */
public class _1099 {
    public static class Solution1 {
        /**
         * Time: O(n^2)
         * Space: O(1)
         * */
        public int twoSumLessThanK(int[] A, int K) {
            int maxSum = Integer.MIN_VALUE;
            for (int i = 0; i < A.length - 1; i++) {
                for (int j = i + 1; j < A.length; j++) {
                    if (A[i] + A[j] < K) {
                        maxSum = Math.max(maxSum, A[i] + A[j]);
                    }
                }
            }
            return maxSum == Integer.MIN_VALUE ? -1 : maxSum;
        }
    }

    public static class Solution2 {
        /**
         * Time: O(nlogn)
         * Space: O(1)
         * */
        public int twoSumLessThanK(int[] A, int K) {
            Arrays.sort(A);
            int left = 0;
            int right = A.length - 1;
            int sum = Integer.MIN_VALUE;
            while (left < right) {
                int newSum = A[left] + A[right];
                if (newSum < K && newSum > sum) {
                    sum = newSum;
                } else if (newSum >= K) {
                    right--;
                } else {
                    left++;
                }
            }
            return sum == Integer.MIN_VALUE ? -1 : sum;
        }
    }
}

// ============================================================
// FILE 788: _1108.java
// ============================================================
```

```
package com.fishercoder.solutions;

/**
 * 1108. Defanging an IP Address
 *
 * Given a valid (IPv4) IP address, return a defanged version of that IP address.
 * A defanged IP address replaces every period "." with "[.]".
 *
 * Example 1:
 * Input: address = "1.1.1.1"
 * Output: "1[.]1[.]1[.]1"
 *
 * Example 2:
 * Input: address = "255.100.50.0"
 * Output: "255[.]100[.]50[.]0"
 *
 * Constraints:
 * The given address is a valid IPv4 address.
 * */
public class _1108 {
    public static class Solution1 {
        public String defangIPaddr(String address) {
            //String.replaceAll() takes in a regex which needs to be escaped
            return address.replaceAll("\\.", "\$$
\\.\
$$");
        }
    }

    public static class Solution2 {
        public String defangIPaddr(String address) {
            //String.replace() takes in a string which does NOT need to be escaped
            return address.replace(".", "[.]");
        }
    }
}

// ============================================================
// FILE 789: _1119.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 1119. Remove Vowels from a String
 *
 * Given a string S, remove the vowels 'a', 'e', 'i', 'o', and 'u' from it, and return
the new string.
 *
 * Example 1:
```

```
 * Input: "leetcodeisacommunityforcoders"
 * Output: "ltcdscmmntyfrcdrs"
 *
 * Example 2:
 * Input: "aeiou"
 * Output: ""
 *
 * Note:
 * S consists of lowercase English letters only.
 * 1 <= S.length <= 1000
 * */
public class _1119 {
    public static class Solution1 {
        public String removeVowels(String S) {
            Set<Character> vowels = new HashSet<>(Arrays.asList('a', 'e', 'i', 'o',
'u'));
            StringBuilder stringBuilder = new StringBuilder();
            for (char c : S.toCharArray()) {
                if (!vowels.contains(c)) {
                    stringBuilder.append(c);
                }
            }
            return stringBuilder.toString();
        }
    }

    public static class Solution2 {
        public String removeVowels(String S) {
            return S.replaceAll("[aeiou]", "");
        }
    }
}

// ============================================================
// FILE 790: _1122.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.TreeMap;

/**
 * 1122. Relative Sort Array
 *
 * Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all elements in
arr2 are also in arr1.
 * Sort the elements of arr1 such that the relative ordering of items in arr1 are the
same as in arr2.
 * Elements that don't appear in arr2 should be placed at the end of arr1 in ascending
order.
 *
 * Example 1:
 * Input: arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
```

```
 * Output: [2,2,2,1,4,3,3,9,6,7,19]
 *
 * Constraints:
 * arr1.length, arr2.length <= 1000
 * 0 <= arr1[i], arr2[i] <= 1000
 * Each arr2[i] is distinct.
 * Each arr2[i] is in arr1.
 * */
public class _1122 {
    public static class Solution1 {
        public int[] relativeSortArray(int[] arr1, int[] arr2) {
            TreeMap<Integer, Integer> map = new TreeMap<>();
            Arrays.stream(arr1).forEach(num -> map.put(num, map.getOrDefault(num, 0) +
1));
            int[] result = new int[arr1.length];
            int i = 0;
            for (int num : arr2) {
                int count = map.get(num);
                while (count-- > 0) {
                    result[i++] = num;
                }
                map.remove(num);
            }
            for (int key : map.keySet()) {
                int count = map.get(key);
                while (count-- > 0) {
                    result[i++] = key;
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 791: _1128.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1128. Number of Equivalent Domino Pairs
 *
 * Given a list of dominoes, dominoes[i] = [a, b] is equivalent to dominoes[j] = [c, d]
if and only if either (a==c and b==d), or (a==d and b==c) - that is, one domino can be
rotated to be equal to another domino.
 * Return the number of pairs (i, j) for which 0 <= i < j < dominoes.length, and
dominoes[i] is equivalent to dominoes[j].
 *
 * Example 1:
 *
```

```
 * Input: dominoes = [[1,2],[2,1],[3,4],[5,6]]
 * Output: 1
 *
 * Constraints:
 * 1 <= dominoes.length <= 40000
 * 1 <= dominoes[i][j] <= 9
 * */
public class _1128 {
    public static class Solution1 {
        public int numEquivDominoPairs(int[][] dominoes) {
            int m = dominoes.length;
            Map<Integer, Integer> map = new HashMap<>();
            int count = 0;
            for (int i = 0; i < m; i++) {
                int smaller = Math.min(dominoes[i][0], dominoes[i][1]);
                int bigger = Math.max(dominoes[i][0], dominoes[i][1]);
                int key = smaller * 10 + bigger;
                count += map.getOrDefault(key, 0);
                map.put(key, map.getOrDefault(key, 0) + 1);
            }
            return count;
        }

    }
}

// ============================================================
// FILE 792: _1133.java
// ============================================================

package com.fishercoder.solutions;

import java.util.TreeMap;

/**
 * 1133. Largest Unique Number
 *
 * Given an array of integers A, return the largest integer that only occurs once.
 * If no integer occurs once, return -1.
 *
 * Example 1:
 * Input: [5,7,3,9,4,9,8,3,1]
 * Output: 8
 * Explanation:
 * The maximum integer in the array is 9 but it is repeated. The number 8 occurs only
once, so it's the answer.
 *
 * Example 2:
 * Input: [9,9,8,8]
 * Output: -1
 * Explanation:
 * There is no number that occurs only once.
 *
 * Note:
```

```
 * 1 <= A.length <= 2000
 * 0 <= A[i] <= 1000
 * */
public class _1133 {
    public static class Solution1 {
        public int largestUniqueNumber(int[] A) {
            TreeMap<Integer, Integer> treeMap = new TreeMap<>((a, b) -> b - a);
            for (int num : A) {
                treeMap.put(num, treeMap.getOrDefault(num, 0) + 1);
            }
            for (int key : treeMap.keySet()) {
                if (treeMap.get(key) == 1) {
                    return key;
                }
            }
            return -1;
        }
    }

    public static class Solution2 {
        public int largestUniqueNumber(int[] A) {
            int[] count = new int[1001];
            for (int num : A) {
                count[num]++;
            }
            for (int i = 1000; i >= 0; i--) {
                if (count[i] == 1) {
                    return i;
                }
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 793: _1134.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1134. Armstrong Number
 *
 * The k-digit number N is an Armstrong number if and only if the k-th power of each
digit sums to N.
 * Given a positive integer N, return true if and only if it is an Armstrong number.
 *
 * Example 1:
 * Input: 153
 * Output: true
 * Explanation:
 * 153 is a 3-digit number, and 153 = 1^3 + 5^3 + 3^3.
 *
```

```
 * Example 2:
 * Input: 123
 * Output: false
 * Explanation:
 * 123 is a 3-digit number, and 123 != 1^3 + 2^3 + 3^3 = 36.
 *
 * Note:
 * 1 <= N <= 10^8
 * */
public class _1134 {
    public static class Solution1 {
        public boolean isArmstrong(int N) {
            int numOfDigits = 0;
            int copyN = N;
            while (copyN > 0) {
                copyN /= 10;
                numOfDigits++;
            }
            int sum = 0;
            copyN = N;
            while (N > 0) {
                int digit = N % 10;
                sum += Math.pow(digit, numOfDigits);
                N /= 10;
            }
            return sum == copyN;
        }
    }
}

// ============================================================
// FILE 794: _1137.java
// ============================================================

package com.fishercoder.solutions;

import java.util.stream.IntStream;

/**
 * 1137. N-th Tribonacci Number
 *
 * The Tribonacci sequence Tn is defined as follows:
 *
 * T0 = 0, T1 = 1, T2 = 1, and Tn+3 = Tn + Tn+1 + Tn+2 for n >= 0.
 *
 * Given n, return the value of Tn.
 *
 * Example 1:
 * Input: n = 4
 * Output: 4
 *
 * Explanation:
 * T_3 = 0 + 1 + 1 = 2
 * T_4 = 1 + 1 + 2 = 4
```

```
 *
 * Example 2:
 * Input: n = 25
 * Output: 1389537
 *
 * Constraints:
 * 0 <= n <= 37
 * The answer is guaranteed to fit within a 32-bit integer, ie. answer <= 2^31 - 1.
 * */
public class _1137 {
    public static class Solution1 {
        public int tribonacci(int n) {
            if (n <= 1) {
                return n;
            }
            int[] numbers = new int[n + 1];
            numbers[0] = 0;
            numbers[1] = 1;
            numbers[2] = 1;
            IntStream.rangeClosed(3, n).forEach(i -> numbers[i] = numbers[i - 1] +
numbers[i - 2] + numbers[i - 3]);
            return numbers[n];
        }
    }
}

// ============================================================
// FILE 795: _1150.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1150. Check If a Number Is Majority Element in a Sorted Array
 *
 * Given an array nums sorted in non-decreasing order, and a number target, return True
if and only if target is a majority element.
 * A majority element is an element that appears more than N/2 times in an array of
length N.
 *
 * Example 1:
 * Input: nums = [2,4,5,5,5,5,5,6,6], target = 5
 * Output: true
 * Explanation:
 * The value 5 appears 5 times and the length of the array is 9.
 * Thus, 5 is a majority element because 5 > 9/2 is true.
 *
 * Example 2:
 * Input: nums = [10,100,101,101], target = 101
 * Output: false
 * Explanation:
 * The value 101 appears 2 times and the length of the array is 4.
 * Thus, 101 is not a majority element because 2 > 4/2 is false.
 *
```

```
 * Note:
 * 1 <= nums.length <= 1000
 * 1 <= nums[i] <= 10^9
 * 1 <= target <= 10^9
 **/
public class _1150 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/check-if-a-number-is-majority-element-in-a-sorted-array/discuss/358130/Java-just-one-binary-search-O(logN))-0ms-beats-100*/
        public boolean isMajorityElement(int[] nums, int target) {
            int firstIndex = findFirstOccur(nums, target);
            int plusHalfIndex = firstIndex + nums.length / 2;
            return plusHalfIndex < nums.length && nums[plusHalfIndex] == target;
        }

        private int findFirstOccur(int[] nums, int target) {
            int left = 0;
            int right = nums.length;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (nums[mid] < target) {
                    left = mid + 1;
                } else if (nums[mid] >= target) {
                    right = mid;
                }
            }
            return left;
        }
    }
}

// ============================================================
// FILE 796: _1154.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Calendar;

/**
 * 1154. Day of the Year
 *
 * Given a string date representing a Gregorian calendar date formatted as YYYY-MM-DD, return the day number of the year.
 *
 * Example 1:
 *
 * Input: date = "2019-01-09"
 * Output: 9
 * Explanation: Given date is the 9th day of the year in 2019.
 * Example 2:
 *
 * Input: date = "2019-02-10"
 * Output: 41
```

```
 * Example 3:
 *
 * Input: date = "2003-03-01"
 * Output: 60
 * Example 4:
 *
 * Input: date = "2004-03-01"
 * Output: 61
 *
 * Constraints:
 *
 * date.length == 10
 * date[4] == date[7] == '-', and all other date[i]'s are digits
 * date represents a calendar date between Jan 1st, 1900 and Dec 31, 2019.
 * */
public class _1154 {
    public static class Solution1 {
        Calendar cal = Calendar.getInstance();

        public int dayOfYear(String date) {
            int year = Integer.parseInt(date.substring(0, 4));
            int month = Integer.parseInt(date.substring(5, 7));
            int day = Integer.parseInt(date.substring(8, 10));
            int thirtyDays = 30;
            int thirtyOneDays = 31;
            if (month == 1) {
                return day;
            } else if (month == 2) {
                return day + thirtyOneDays;
            } else {
                int daysInFeb = isLeapYear(year) ? 29 : 28;
                if (month == 3) {
                    return thirtyOneDays + daysInFeb + day;
                } else if (month == 4) {
                    return 2 * thirtyOneDays + daysInFeb + day;
                } else if (month == 5) {
                    return 2 * thirtyOneDays + daysInFeb + day + thirtyDays;
                } else if (month == 6) {
                    return 3 * thirtyOneDays + daysInFeb + day + thirtyDays;
                } else if (month == 7) {
                    return 3 * thirtyOneDays + daysInFeb + day + 2 * thirtyDays;
                } else if (month == 8) {
                    return 4 * thirtyOneDays + daysInFeb + day + 2 * thirtyDays;
                } else if (month == 9) {
                    return 5 * thirtyOneDays + daysInFeb + day + 2 * thirtyDays;
                } else if (month == 10) {
                    return 5 * thirtyOneDays + daysInFeb + day + 3 * thirtyDays;
                } else if (month == 11) {
                    return 6 * thirtyOneDays + daysInFeb + day + 3 * thirtyDays;
                } else {
                    return 6 * thirtyOneDays + daysInFeb + day + 4 * thirtyDays;
                }
            }
        }
```

```
        private boolean isLeapYear(int year) {
            cal.set(Calendar.YEAR, year);
            return cal.getActualMaximum(Calendar.DAY_OF_YEAR) > 365;
        }
    }
}

// ============================================================
// FILE 797: _1160.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1160. Find Words That Can Be Formed by Characters
 *
 * You are given an array of strings words and a string chars.
 * A string is good if it can be formed by characters from chars (each character can only
be used once).
 * Return the sum of lengths of all good strings in words.
 *
 * Example 1:
 *
 * Input: words = ["cat","bt","hat","tree"], chars = "atach"
 * Output: 6
 * Explanation:
 * The strings that can be formed are "cat" and "hat" so the answer is 3 + 3 = 6.
 *
 * Example 2:
 *
 * Input: words = ["hello","world","leetcode"], chars = "welldonehoneyr"
 * Output: 10
 * Explanation:
 * The strings that can be formed are "hello" and "world" so the answer is 5 + 5 = 10.
 *
 * Note:
 *
 * 1 <= words.length <= 1000
 * 1 <= words[i].length, chars.length <= 100
 * All strings contain lowercase English letters only.
 * */
public class _1160 {
    public static class Solution1 {
        public int countCharacters(String[] words, String chars) {
            int length = 0;
            Map<Character, Integer> map = new HashMap<>();
            for (char c : chars.toCharArray()) {
                int count = map.getOrDefault(c, 0);
                map.put(c, count + 1);
```

```
            }
            for (String word : words) {
                if (canForm(word, map)) {
                    length += word.length();
                }
            }
            return length;
        }

        private boolean canForm(String word, final Map<Character, Integer> map) {
            Map<Character, Integer> tmp = new HashMap<>(map);
            for (Character c : word.toCharArray()) {
                if (tmp.containsKey(c) && tmp.get(c) > 0) {
                    int count = tmp.get(c);
                    tmp.put(c, count - 1);
                } else {
                    return false;
                }
            }
            return true;
        }

    }
}

// ============================================================
// FILE 798: _1165.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1165. Single-Row Keyboard
 *
 * There is a special keyboard with all keys in a single row.
 * Given a string keyboard of length 26 indicating the layout of the keyboard (indexed
from 0 to 25),
 * initially your finger is at index 0.
 * To type a character, you have to move your finger to the index of the desired
character. The time taken to move your finger from index i to index j is |i - j|.
 *
 * You want to type a string word. Write a function to calculate how much time it takes
to type it with one finger.
 *
 * Example 1:
 * Input: keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"
 * Output: 4
 * Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0
again to write 'a'.
 * Total time = 2 + 1 + 1 = 4.
 *
 * Example 2:
 * Input: keyboard = "pqrstuvwxyzabcdefghijklmno", word = "leetcode"
 * Output: 73
```

```
 *
 * Constraints:
 * keyboard.length == 26
 * keyboard contains each English lowercase letter exactly once in some order.
 * 1 <= word.length <= 10^4
 * word[i] is an English lowercase letter.
 * */
public class _1165 {
    public static class Solution1 {
        public int calculateTime(String keyboard, String word) {
            int time = 0;
            int fromIndex = 0;
            for (char c : word.toCharArray()) {
                int fingerMoves = Math.abs(fromIndex - keyboard.indexOf(c));
                fromIndex = keyboard.indexOf(c);
                time += fingerMoves;
            }
            return time;
        }
    }
}

// ============================================================
// FILE 799: _1170.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1170. Compare Strings by Frequency of the Smallest Character
 *
 * Let's define a function f(s) over a non-empty string s,
 * which calculates the frequency of the smallest character in s.
 * For example, if s = "dcce" then f(s) = 2 because the smallest character is "c" and its
frequency is 2.
 * Now, given string arrays queries and words,
 * return an integer array answer,
 * where each answer[i] is the number of words such that f(queries[i]) < f(W), where W is
a word in words.
 *
 * Example 1:
 * Input: queries = ["cbd"], words = ["zaaaz"]
 * Output: [1]
 * Explanation: On the first query we have f("cbd") = 1, f("zaaaz") = 3 so f("cbd") <
f("zaaaz").
 *
 * Example 2:
 * Input: queries = ["bbb","cc"], words = ["a","aa","aaa","aaaa"]
 * Output: [1,2]
 * Explanation: On the first query only f("bbb") < f("aaaa"). On the second query both
f("aaa") and f("aaaa") are both > f("cc").
 *
```

```
 * Constraints:
 * 1 <= queries.length <= 2000
 * 1 <= words.length <= 2000
 * 1 <= queries[i].length, words[i].length <= 10
 * queries[i][j], words[i][j] are English lowercase letters.
 * */
public class _1170 {
    public static class Solution1 {
        /**
         * Use simple iteration when finding counts
         * Time: O(n^m) where m is the size of queries and n is the size of words
         * Space: O(max(m, n) where m is the size of queries and n is the size of words)
         * */
        public int[] numSmallerByFrequency(String[] queries, String[] words) {
            int[] queriesMinFrequecies = new int[queries.length];
            for (int i = 0; i < queries.length; i++) {
                queriesMinFrequecies[i] = computeLowestFrequency(queries[i]);
            }

            int[] wordsMinFrequecies = new int[words.length];
            for (int i = 0; i < words.length; i++) {
                wordsMinFrequecies[i] = computeLowestFrequency(words[i]);
            }
            Arrays.sort(wordsMinFrequecies);

            int[] result = new int[queries.length];
            for (int i = 0; i < result.length; i++) {
                result[i] = search(wordsMinFrequecies, queriesMinFrequecies[i]);
            }
            return result;
        }

        private int search(int[] nums, int target) {
            int count = 0;
            for (int i = nums.length - 1; i >= 0; i--) {
                if (nums[i] > target) {
                    count++;
                } else {
                    break;
                }
            }
            return count;
        }

        private int computeLowestFrequency(String string) {
            char[] str = string.toCharArray();
            Arrays.sort(str);
            String sortedString = new String(str);
            int frequency = 1;
            for (int i = 1; i < sortedString.length(); i++) {
                if (sortedString.charAt(i) == sortedString.charAt(0)) {
                    frequency++;
                } else {
                    break;
```

```
                }
            }
            return frequency;
        }
    }

    public static class Solution2 {
        /**
         * Use binary search when finding counts
         * Time: O(n^logn) where m is the size of queries and n is the size of words
         * Space: O(max(m, n) where m is the size of queries and n is the size of words)
         * */
        public int[] numSmallerByFrequency(String[] queries, String[] words) {
            int[] queriesMinFrequecies = new int[queries.length];
            for (int i = 0; i < queries.length; i++) {
                queriesMinFrequecies[i] = computeLowestFrequency(queries[i]);
            }

            int[] wordsMinFrequecies = new int[words.length];
            for (int i = 0; i < words.length; i++) {
                wordsMinFrequecies[i] = computeLowestFrequency(words[i]);
            }
            Arrays.sort(wordsMinFrequecies);

            int[] result = new int[queries.length];
            for (int i = 0; i < result.length; i++) {
                result[i] = binarySearch(wordsMinFrequecies, queriesMinFrequecies[i]);
            }
            return result;
        }

        private int binarySearch(int[] nums, int target) {
            int left = 0;
            int right = nums.length - 1;
            while (left <= right) {
                int mid = (left + right) / 2;
                if (nums[mid] <= target) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
            return nums.length - left;
        }

        private int computeLowestFrequency(String string) {
            char[] str = string.toCharArray();
            Arrays.sort(str);
            String sortedString = new String(str);
            int frequency = 1;
            for (int i = 1; i < sortedString.length(); i++) {
                if (sortedString.charAt(i) == sortedString.charAt(0)) {
                    frequency++;
                } else {
```

```
                    break;
                }
            }
            return frequency;
        }
    }
}

// ============================================================
// FILE 800: _1184.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1184. Distance Between Bus Stops
 *
 * A bus has n stops numbered from 0 to n - 1 that form a circle. We know the distance
between all pairs of neighboring stops where distance[i] is the distance between the
stops number i and (i + 1) % n.
 * The bus goes along both directions i.e. clockwise and counterclockwise.
 * Return the shortest distance between the given start and destination stops.
 *
 * Example 1:
 * Input: distance = [1,2,3,4], start = 0, destination = 1
 * Output: 1
 * Explanation: Distance between 0 and 1 is 1 or 9, minimum is 1.
 *
 * Example 2:
 * Input: distance = [1,2,3,4], start = 0, destination = 2
 * Output: 3
 * Explanation: Distance between 0 and 2 is 3 or 7, minimum is 3.
 *
 * Example 3:
 * Input: distance = [1,2,3,4], start = 0, destination = 3
 * Output: 4
 * Explanation: Distance between 0 and 3 is 6 or 4, minimum is 4.
 *
 * Constraints:
 * 1 <= n <= 10^4
 * distance.length == n
 * 0 <= start, destination < n
 * 0 <= distance[i] <= 10^4
 * */
public class _1184 {
    public static class Solution1 {
        public int distanceBetweenBusStops(int[] distance, int start, int destination) {
            if (start > destination) {
                int tmp = start;
                start = destination;
                destination = tmp;
            }
            int clockwise = 0;
            for (int i = start; i < destination; i++) {
```

```
                clockwise += distance[i];
            }
            int counterClockwise = 0;
            for (int i = destination; i < distance.length; i++) {
                counterClockwise += distance[i];
            }
            for (int i = 0; i < start; i++) {
                counterClockwise += distance[i];
            }

            return Math.min(clockwise, counterClockwise);
        }
    }
}

// ============================================================
// FILE 801: _1185.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1185. Day of the Week
 *
 * Given a date, return the corresponding day of the week for that date.
 * The input is given as three integers representing the day, month and year
respectively.
 * Return the answer as one of the following values {"Sunday", "Monday", "Tuesday",
"Wednesday", "Thursday", "Friday", "Saturday"}.
 *
 * Example 1:
 * Input: day = 31, month = 8, year = 2019
 * Output: "Saturday"
 *
 * Example 2:
 * Input: day = 18, month = 7, year = 1999
 * Output: "Sunday"
 *
 * Example 3:
 * Input: day = 15, month = 8, year = 1993
 * Output: "Sunday"
 *
 * Constraints:
 * The given dates are valid dates between the years 1971 and 2100.
 * */
public class _1185 {
    public static class Solution1 {
        /**
         * Time: O(1)
         * Space: O(1)
         *
         * Plain and simple algorithm:
         * based on the fact that 1/1/1971 is a Friday and calculate the given day.
         * */
```

```
        public String dayOfTheWeek(int day, int month, int year) {
            String[] daysInTheWeek = new String[]{"Sunday", "Monday", "Tuesday",
"Wednesday", "Thursday", "Friday", "Saturday"};
            int[] daysInTheMonth = new int[]{31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31};
            int numberOfDays = 0;
            for (int i = 1971; i < year; i++) {
                numberOfDays += i % 4 == 0 ? 366 : 365;
            }
            if (year % 4 == 0) {
                daysInTheMonth[1] = 29;
            }
            for (int i = 0; i < month - 1; i++) {
                numberOfDays += daysInTheMonth[i];
            }
            numberOfDays += day - 1;
            return daysInTheWeek[(numberOfDays + 5) % 7];
        }
    }
}

// ==========================================================
// FILE 802: _1189.java
// ==========================================================

package com.fishercoder.solutions;

/**
 * 1189. Maximum Number of Balloons
 *
 * Given a string text, you want to use the characters of text to form as many instances
of the word "balloon" as possible.
 * You can use each character in text at most once. Return the maximum number of
instances that can be formed.
 *
 * Example 1:
 * Input: text = "nlaebolko"
 * Output: 1
 *
 * Example 2:
 * Input: text = "loonbalxballpoon"
 * Output: 2
 *
 * Example 3:
 * Input: text = "leetcode"
 * Output: 0
 *
 * Constraints:
 * 1 <= text.length <= 10^4
 * text consists of lower case English letters only.
 * */
public class _1189 {
    public static class Solution1 {
        public int maxNumberOfBalloons(String text) {
```

```
            int[] counts = new int[26];
            for (char c : text.toCharArray()) {
                counts[c - 'a']++;
            }
            return Math.min(counts[0], Math.min(counts[1], Math.min(counts[11] / 2,
Math.min(counts[14] / 2, counts[13])));
        }
    }
}

// ============================================================
// FILE 803: _1196.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1196. How Many Apples Can You Put into the Basket
 *
 * You have some apples, where arr[i] is the weight of the i-th apple.  You also have a
basket that can carry up to 5000 units of weight.
 * Return the maximum number of apples you can put in the basket.
 *
 * Example 1:
 * Input: arr = [100,200,150,1000]
 * Output: 4
 * Explanation: All 4 apples can be carried by the basket since their sum of weights is
1450.
 *
 * Example 2:
 * Input: arr = [900,950,800,1000,700,800]
 * Output: 5
 * Explanation: The sum of weights of the 6 apples exceeds 5000 so we choose any 5 of
them.
 *
 * Constraints:
 * 1 <= arr.length <= 10^3
 * 1 <= arr[i] <= 10^3
 * */
public class _1196 {
    public static class Solution1 {
        public int maxNumberOfApples(int[] arr) {
            Arrays.sort(arr);
            int sum = 0;
            int i = 0;
            for (; i < arr.length; i++) {
                sum += arr[i];
                if (sum > 5000) {
                    break;
                }
            }
            return i;
```

```
        }
    }
}

// ============================================================
// FILE 804: _1198.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1198. Find Smallest Common Element in All Rows
 *
 * Given a matrix mat where every row is sorted in increasing order, return the smallest
common element in all rows.
 * If there is no common element, return -1.
 *
 * Example 1:
 * Input: mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]
 * Output: 5
 *
 * Constraints:
 * 1 <= mat.length, mat[i].length <= 500
 * 1 <= mat[i][j] <= 10^4
 * mat[i] is sorted in increasing order.
 * */
public class _1198 {
    public static class Solution1 {
        public int smallestCommonElement(int[][] mat) {
            int m = mat.length;
            int n = mat[0].length;
            for (int j = 0; j < n; j++) {
                int minCommon = mat[0][j];
                //we'll start from the second row
                int i = 1;
                for (; i < m; i++) {
                    if (thisRowHasThisNumber(mat[i], minCommon)) {
                        continue;
                    } else {
                        break;
                    }
                }
                if (i == m) {
                    return minCommon;
                }
            }
            return -1;
        }

        private boolean thisRowHasThisNumber(int[] nums, int target) {
            int left = 0;
            int right = nums.length - 1;
            while (left <= right) {
                int mid = left + (right - left) / 2;
```

```
                if (target == nums[mid]) {
                    return true;
                } else if (target > nums[mid]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
            return false;
        }
    }
}

// ============================================================
// FILE 805: _1200.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 1200. Minimum Absolute Difference
 *
 * Given an array of distinct integers arr, find all pairs of elements with the minimum
absolute difference of any two elements.
 * Return a list of pairs in ascending order(with respect to pairs), each pair [a, b]
follows
 * a, b are from arr
 * a < b
 * b - a equals to the minimum absolute difference of any two elements in arr
 *
 * Example 1:
 * Input: arr = [4,2,1,3]
 * Output: [[1,2],[2,3],[3,4]]
 * Explanation: The minimum absolute difference is 1. List all pairs with difference
equal to 1 in ascending order.
 *
 * Example 2:
 * Input: arr = [1,3,6,10,15]
 * Output: [[1,3]]
 *
 * Example 3:
 * Input: arr = [3,8,-10,23,19,-4,-14,27]
 * Output: [[-14,-10],[19,23],[23,27]]
 *
 * Constraints:
 * 2 <= arr.length <= 10^5
 * -10^6 <= arr[i] <= 10^6
 * */
public class _1200 {
    public static class Solution1 {
```

```
        /**
         * Time: O(nlogn) due to sorting
         * Space: O(k) where k is the distinct number of differences between two numbers
in the given array
         * */
        public List<List<Integer>> minimumAbsDifference(int[] arr) {
            Arrays.sort(arr);
            int minimumDiff = arr[1] - arr[0];
            List result = new ArrayList();
            result.add(Arrays.asList(arr[0], arr[1]));
            for (int i = 1; i < arr.length - 1; i++) {
                int diff = arr[i + 1] - arr[i];
                if (minimumDiff == diff) {
                    result.add(Arrays.asList(arr[i], arr[i + 1]));
                } else if (minimumDiff > diff) {
                    minimumDiff = diff;
                    result = new ArrayList();
                    result.add(Arrays.asList(arr[i], arr[i + 1]));
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 806: _1207.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 1207. Unique Number of Occurrences
 *
 * Given an array of integers arr,
 * write a function that returns true if and only if the number of occurrences of each
value in the array is unique.
 *
 * Example 1:
 * Input: arr = [1,2,2,1,1,3]
 * Output: true
 * Explanation: The value 1 has 3 occurrences, 2 has 2 and 3 has 1. No two values have
the same number of occurrences.
 *
 * Example 2:
 * Input: arr = [1,2]
 * Output: false
 *
```

```
 * Example 3:
 * Input: arr = [-3,0,1,-3,1,1,1,-3,10,0]
 * Output: true
 *
 * Constraints:
 * 1 <= arr.length <= 1000
 * -1000 <= arr[i] <= 1000
 * */
public class _1207 {
    public static class Solution1 {
        public boolean uniqueOccurrences(int[] arr) {
            Map<Integer, Integer> map = new HashMap<>();
            Arrays.stream(arr).forEach(num -> {
                map.put(num, map.containsKey(num) ? map.get(num) + 1 : 1);
            });
            Set<Integer> set = new HashSet<>();
            return map.keySet().stream().mapToInt(key -> key).allMatch(key ->
set.add(map.get(key)));
        }
    }
}

// ============================================================
// FILE 807: _1213.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.List;

/**
 * 1213. Intersection of Three Sorted Arrays
 *
 * Given three integer arrays arr1, arr2 and arr3 sorted in strictly increasing order,
 * return a sorted array of only the integers that appeared in all three arrays.
 *
 * Example 1:
 * Input: arr1 = [1,2,3,4,5], arr2 = [1,2,5,7,9], arr3 = [1,3,4,5,8]
 * Output: [1,5]
 * Explanation: Only 1 and 5 appeared in the three arrays.
 *
 * Constraints:
 *
 * 1 <= arr1.length, arr2.length, arr3.length <= 1000
 * 1 <= arr1[i], arr2[i], arr3[i] <= 2000
 * */
public class _1213 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/intersection-of-three-sorted-
arrays/discuss/397603/Simple-Java-solution-beats-100*/
        public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
            List<Integer> result = new ArrayList();
            int i = 0;
```

```
            int j = 0;
            int k = 0;
            while (i < arr1.length && j < arr2.length && k < arr3.length) {
                if (arr1[i] == arr2[j] && arr1[i] == arr3[k]) {
                    result.add(arr1[i]);
                    i++;
                    j++;
                    k++;
                } else if (arr1[i] < arr2[j]) {
                    i++;
                } else if (arr2[j] < arr3[k]) {
                    j++;
                } else {
                    k++;
                }
            }
            return result;
        }
    }
}

// ============================================================
// FILE 808: _1217.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1217. Play with Chips
 *
 * There are some chips, and the i-th chip is at position chips[i].
 * You can perform any of the two following types of moves any number of times (possibly
zero) on any chip:
 *
 * Move the i-th chip by 2 units to the left or to the right with a cost of 0.
 * Move the i-th chip by 1 unit to the left or to the right with a cost of 1.
 *
 * There can be two or more chips at the same position initially.
 *
 * Return the minimum cost needed to move all the chips to the same position (any
position).
 *
 * Example 1:
 * Input: chips = [1,2,3]
 * Output: 1
 * Explanation: Second chip will be moved to positon 3 with cost 1. First chip will be
moved to position 3 with cost 0. Total cost is 1.
 *
 * Example 2:
 * Input: chips = [2,2,2,3,3]
 * Output: 2
 * Explanation: Both fourth and fifth chip will be moved to position two with cost 1.
Total minimum cost will be 2.
 *
```

```
 * Constraints:
 * 1 <= chips.length <= 100
 * 1 <= chips[i] <= 10^9
 * */
public class _1217 {
    public static class Solution1 {
        public int minCostToMoveChips(int[] chips) {
            //TODO: implement it
            return -1;
        }
    }
}

// ============================================================
// FILE 809: _1228.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 1228. Missing Number In Arithmetic Progression
 *
 * In some array arr, the values were in arithmetic progression: the values arr[i+1] -
arr[i] are all equal for every 0 <= i < arr.length - 1.
 * Then, a value from arr was removed that was not the first or last value in the array.
 * Return the removed value.
 *
 * Example 1:
 * Input: arr = [5,7,11,13]
 * Output: 9
 * Explanation: The previous array was [5,7,9,11,13].
 *
 * Example 2:
 * Input: arr = [15,13,12]
 * Output: 14
 * Explanation: The previous array was [15,14,13,12].
 *
 * Constraints:
 * 3 <= arr.length <= 1000
 * 0 <= arr[i] <= 10^5
 * */
public class _1228 {
    public static class Solution1 {
        /**A super verbose and inefficient but working way...*/
        public int missingNumber(int[] arr) {
            Arrays.sort(arr);
            Map<Integer, List<Integer>> map = new HashMap<>();
            for (int i = 0; i < arr.length - 1; i++) {
```

```
                int diff = arr[i + 1] - arr[i];
                List<Integer> list = map.getOrDefault(diff, new ArrayList<>());
                list.add(i);
                map.put(diff, list);
            }
            int smallDiff = arr[arr.length - 1];
            int bigDiff = 0;
            for (int key : map.keySet()) {
                smallDiff = Math.min(smallDiff, key);
                bigDiff = Math.max(bigDiff, key);
            }
            return arr[map.get(bigDiff).get(0)] + smallDiff;
        }
    }

    public static class Solution2 {
        /**credit: https://leetcode.com/problems/missing-number-in-arithmetic-
progression/discuss/408474/JavaC%2B%2BPython-Arithmetic-Sum-and-Binary-Search*/
        public int missingNumber(int[] arr) {
            int min = arr[0];
            int max = arr[0];
            int sum = 0;
            for (int num : arr) {
                max = Math.max(max, num);
                min = Math.min(min, num);
                sum += num;
            }
            return (max + min) * (arr.length + 1) / 2 - sum;
        }
    }
}

// ============================================================
// FILE 810: _1232.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1232. Check If It Is a Straight Line
 *
 * You are given an array coordinates, coordinates[i] = [x, y],
 * where [x, y] represents the coordinate of a point.
 * Check if these points make a straight line in the XY plane.
 *
 * Example 1:
 * Input: coordinates = [[1,2],[2,3],[3,4],[4,5],[5,6],[6,7]]
 * Output: true
 *
 * Example 2:
 * Input: coordinates = [[1,1],[2,2],[3,4],[4,5],[5,6],[7,7]]
 * Output: false
 *
 * Constraints:
```

```
 * 2 <= coordinates.length <= 1000
 * coordinates[i].length == 2
 * -10^4 <= coordinates[i][0], coordinates[i][1] <= 10^4
 * coordinates contains no duplicate point.
 * */
public class _1232 {
    public static class Solution1 {
        /**
         * To check if they share the same slope, we use this formula:
         *
         * check whether (y4 - y3)/(x4- x3) equals to (y2 - y1)/(x2 - x1)
         * considering denominator could be zero, we'll change it to use multiplication
instead of division,
         * thus it becomes
         * check whether (y4 - y3)*(x2 - x1) equals (x4 - x3)*(y2 - y1)
         * */
        public boolean checkStraightLine(int[][] coordinates) {
            for (int i = 2; i < coordinates.length - 1; i++) {
                if ((coordinates[1][0] - coordinates[0][0]) * (coordinates[i + 1][1] -
coordinates[i][1])
                        != (coordinates[1][1] - coordinates[0][1]) * (coordinates[i + 1]
[0] - coordinates[i][0])) {
                    return false;
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 811: _1237.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 1237. Find Positive Integer Solution for a Given Equation
 *
 * Given a function  f(x, y) and a value z, return all positive integer pairs x and y
where f(x,y) == z.
 * The function is constantly increasing, i.e.:
 *
 * f(x, y) < f(x + 1, y)
 * f(x, y) < f(x, y + 1)
 * The function interface is defined like this:
 *
 * interface CustomFunction {
 * public:
 *   // Returns positive integer f(x, y) for any given positive integer x and y.
```

```
 *   int f(int x, int y);
 * };
 * For custom testing purposes you're given an integer function_id and a target z as
input,
 * where function_id represent one function from an secret internal list, on the examples
you'll know only two functions from the list.
 * You may return the solutions in any order.
 *
 * Example 1:
 * Input: function_id = 1, z = 5
 * Output: [[1,4],[2,3],[3,2],[4,1]]
 * Explanation: function_id = 1 means that f(x, y) = x + y
 *
 * Example 2:
 * Input: function_id = 2, z = 5
 * Output: [[1,5],[5,1]]
 * Explanation: function_id = 2 means that f(x, y) = x * y
 *
 * Constraints:
 * 1 <= function_id <= 9
 * 1 <= z <= 100
 * It's guaranteed that the solutions of f(x, y) == z will be on the range 1 <= x, y <=
1000
 * It's also guaranteed that f(x, y) will fit in 32 bit signed integer if 1 <= x, y <=
1000
 */
public class _1237 {

    // This is the custom function interface.
    // You should not implement it, or speculate about its implementation
    abstract class CustomFunction {
        // Returns f(x, y) for any given positive integers x and y.
        // Note that f(x, y) is increasing with respect to both x and y.
        // i.e. f(x, y) < f(x + 1, y), f(x, y) < f(x, y + 1)
        public abstract int f(int x, int y);
    }

    public static class Solution1 {
        /**Time: O(x*y)
         * Space: O(1)*/
        public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
            List<List<Integer>> result = new ArrayList<>();
            for (int x = 1; x <= 1000; x++) {
                for (int y = 1; y <= 1000; y++) {
                    if (customfunction.f(x, y) == z) {
                        result.add(Arrays.asList(x, y));
                    }
                }
            }
            return result;
        }
    }

    public static class Solution2 {
```

```
        /**
         * linear search
         *
         * Time: O(x + y)
         * Space: O(1)
         */
        public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
            List<List<Integer>> result = new ArrayList<>();
            int x = 1;
            int y = 1000;
            while (x < 1001 && y > 0) {
                int functionResult = customfunction.f(x, y);
                if (functionResult < z) {
                    x++;
                } else if (functionResult > z) {
                    y--;
                } else {
                    result.add(Arrays.asList(x++, y--));
                }
            }
            return result;
        }
    }

    public static class Solution3 {
        /**
         * binary search
         *
         * Time: O(xlogy)
         * Space: O(1)
         * */
        public List<List<Integer>> findSolution(CustomFunction customfunction, int z) {
            List<List<Integer>> result = new ArrayList<>();
            for (int x = 1; x <= 1000; x++) {
                int left = 1;
                int right = 1001;
                while (left < right) {
                    int y = (left + right) / 2;
                    if (customfunction.f(x, y) < z) {
                        left = y + 1;
                    } else {
                        right = y;
                    }
                }
                if (customfunction.f(x, left) == z) {
                    result.add(Arrays.asList(x, left));
                }
            }
            return result;
        }
    }

}
```

```java
// ============================================================
// FILE 812: _1243.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 1243. Array Transformation
 *
 * Given an initial array arr, every day you produce a new array using the array of the
previous day.
 *
 * On the i-th day, you do the following operations on the array of day i-1 to produce
the array of day i:
 *
 * If an element is smaller than both its left neighbor and its right neighbor, then this
element is incremented.
 * If an element is bigger than both its left neighbor and its right neighbor, then this
element is decremented.
 * The first and last elements never change.
 * After some days, the array does not change. Return that final array.
 *
 * Example 1:
 * Input: arr = [6,2,3,4]
 * Output: [6,3,3,4]
 * Explanation:
 * On the first day, the array is changed from [6,2,3,4] to [6,3,3,4].
 * No more operations can be done to this array.
 *
 * Example 2:
 * Input: arr = [1,6,3,4,3,5]
 * Output: [1,4,4,4,4,5]
 * Explanation:
 * On the first day, the array is changed from [1,6,3,4,3,5] to [1,5,4,3,4,5].
 * On the second day, the array is changed from [1,5,4,3,4,5] to [1,4,4,4,4,5].
 * No more operations can be done to this array.
 *
 * Constraints:
 * 1 <= arr.length <= 100
 * 1 <= arr[i] <= 100
 * */
public class _1243 {
    public static class Solution1 {
        public List<Integer> transformArray(int[] arr) {
            int[] copy;
            do {
                copy = Arrays.copyOf(arr, arr.length);
                for (int i = 1; i < arr.length - 1; i++) {
                    if (copy[i] < copy[i - 1] && copy[i] < copy[i + 1]) {
                        arr[i]++;
```

```
                    } else if (copy[i] > copy[i - 1] && copy[i] > copy[i + 1]) {
                        arr[i]--;
                    }
                }
            } while (!Arrays.equals(copy, arr));
            return Arrays.stream(arr)
                    .boxed()
                    .collect(Collectors.toList());
        }
    }
}

// ============================================================
// FILE 813: _1252.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1252. Cells with Odd Values in a Matrix
 *
 * Given n and m which are the dimensions of a matrix initialized by zeros and given an
array indices where indices[i] = [ri, ci]. For each pair of [ri, ci] you have to
increment all cells in row ri and column ci by 1.
 * Return the number of cells with odd values in the matrix after applying the increment
to all indices.
 *
 * Example 1:
 * 0, 0, 0     1, 2, 1    1, 3, 1
 * 0, 0, 0     0, 1, 0    1, 3 ,1
 *
 * Input: n = 2, m = 3, indices = [[0,1],[1,1]]
 * Output: 6
 * Explanation: Initial matrix = [[0,0,0],[0,0,0]].
 * After applying first increment it becomes [[1,2,1],[0,1,0]].
 * The final matrix will be [[1,3,1],[1,3,1]] which contains 6 odd numbers.
 *
 * Example 2:
 * 0, 0    0, 1    2, 2
 * 0, 0    1, 2    2, 2
 *
 * Input: n = 2, m = 2, indices = [[1,1],[0,0]]
 * Output: 0
 * Explanation: Final matrix = [[2,2],[2,2]]. There is no odd number in the final matrix.
 *
 *
 * Constraints:
 * 1 <= n <= 50
 * 1 <= m <= 50
 * 1 <= indices.length <= 100
 * 0 <= indices[i][0] < n
 * 0 <= indices[i][1] < m
 * */
public class _1252 {
```

```
    public static class Solution1 {
        /**
         * Time: O(m*n + k) where k is the length of indices
         * Space: O(m*n)
         * */
        public int oddCells(int n, int m, int[][] indices) {
            int[][] matrix = new int[n][m];
            for (int i = 0; i < indices.length; i++) {
                addOneToRow(matrix, indices[i][0]);
                addOneToColumn(matrix, indices[i][1]);
            }
            int oddNumberCount = 0;
            for (int i = 0; i < matrix.length; i++) {
                for (int j = 0; j < matrix[0].length; j++) {
                    if (matrix[i][j] % 2 != 0) {
                        oddNumberCount++;
                    }
                }
            }
            return oddNumberCount;
        }

        private void addOneToColumn(int[][] matrix, int columnIndex) {
            for (int i = 0; i < matrix.length; i++) {
                matrix[i][columnIndex] += 1;
            }
        }

        private void addOneToRow(int[][] matrix, int rowIndex) {
            for (int j = 0; j < matrix[0].length; j++) {
                matrix[rowIndex][j] += 1;
            }
        }
    }

    public static class Solution2 {
        /**
         * Time: O(m*n + k) where k is the length of indices
         * Space: O(m + n)
         */
        public int oddCells(int n, int m, int[][] indices) {
            boolean[] row = new boolean[n];
            boolean[] column = new boolean[m];
            for (int[] index : indices) {
                row[index[0]] ^= true;
                column[index[1]] ^= true;
            }
            int oddNumberCount = 0;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < m; j++) {
                    oddNumberCount += row[i] ^ column[j] ? 1 : 0;
                }
            }
            return oddNumberCount;
```

```
        }
    }
}

// ============================================================
// FILE 814: _1260.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

/**
 * 1260. Shift 2D Grid
 *
 * Given a 2D grid of size m x n and an integer k. You need to shift the grid k times.
 *
 * In one shift operation:
 * Element at grid[i][j] becomes at grid[i][j + 1].
 * Element at grid[i][n - 1] becomes at grid[i + 1][0].
 * Element at grid[n - 1][n - 1] becomes at grid[0][0].
 * Return the 2D grid after applying shift operation k times.
 *
 * Example 1:
 * Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 1
 * Output: [[9,1,2],[3,4,5],[6,7,8]]
 *
 * Example 2:
 * Input: grid = [[3,8,1,9],[19,7,2,5],[4,6,11,10],[12,0,21,13]], k = 4
 * Output: [[12,0,21,13],[3,8,1,9],[19,7,2,5],[4,6,11,10]]
 *
 * Example 3:
 * Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 9
 * Output: [[1,2,3],[4,5,6],[7,8,9]]
 *
 * Constraints:
 * m == grid.length
 * n == grid[i].length
 * 1 <= m <= 50
 * 1 <= n <= 50
 * -1000 <= grid[i][j] <= 1000
 * 0 <= k <= 100
 * */
public class _1260 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/shift-2d-grid/discuss/431102/JavaPython-
3-simple-code-using-mod*/
        public List<List<Integer>> shiftGrid(int[][] grid, int k) {
            int m = grid.length;
            int n = grid[0].length;
            int totalNumbers = m * n;
            int start = totalNumbers - k % totalNumbers;
```

```
            LinkedList<List<Integer>> result = new LinkedList<>();
            for (int i = start; i < totalNumbers + start; i++) {
                int moveIndex = i % totalNumbers;
                int moveRow = moveIndex / n;
                int moveColumn = moveIndex % n;
                if ((i - start) % n == 0) {
                    result.add(new ArrayList<>());
                }
                result.peekLast().add(grid[moveRow][moveColumn]);
            }
            return result;
        }
    }
}

// ============================================================
// FILE 815: _1266.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1266. Minimum Time Visiting All Points
 *
 * On a plane there are n points with integer coordinates points[i] = [xi, yi].
 * Your task is to find the minimum time in seconds to visit all points.
 *
 * You can move according to the next rules:
 * In one second always you can either move vertically, horizontally by one unit or
diagonally
 * (it means to move one unit vertically and one unit horizontally in one second).
 * You have to visit the points in the same order as they appear in the array.
 *
 * Example 1:
 * Input: points = [[1,1],[3,4],[-1,0]]
 * Output: 7
 * Explanation: One optimal path is [1,1] -> [2,2] -> [3,3] -> [3,4] -> [2,3] -> [1,2] ->
[0,1] -> [-1,0]
 * Time from [1,1] to [3,4] = 3 seconds
 * Time from [3,4] to [-1,0] = 4 seconds
 * Total time = 7 seconds
 *
 * Example 2:
 * Input: points = [[3,2],[-2,2]]
 * Output: 5
 *
 * Constraints:
 * points.length == n
 * 1 <= n <= 100
 * points[i].length == 2
 * -1000 <= points[i][0], points[i][1] <= 1000
 * */
public class _1266 {
    public static class Solution1 {
```

```
        /**
         * Time: O(n)
         * Space: O(1)
         *
         * credit: https://leetcode.com/problems/minimum-time-visiting-all-
points/discuss/436142/Sum-of-Chebyshev-distance-between-two-consecutive-points
         * */
        public int minTimeToVisitAllPoints(int[][] points) {
            int minTime = 0;
            for (int i = 0; i < points.length - 1; i++) {
                minTime += chebyshevDistance(points[i], points[i + 1]);
            }
            return minTime;
        }

        private int chebyshevDistance(int[] pointA, int[] pointB) {
            return Math.max(Math.abs(pointA[0] - pointB[0]), Math.abs(pointA[1] -
pointB[1]));
        }
    }
}

// ============================================================
// FILE 816: _1267.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.utils.CommonUtils;

/**
 * 1267. Count Servers that Communicate
 *
 * You are given a map of a server center, represented as a m * n integer matrix grid,
where 1 means that on that cell there is a server and 0 means that it is no server.
 * Two servers are said to communicate if they are on the same row or on the same column.
 *
 * Return the number of servers that communicate with any other server.
 *
 * Example 1:
 * Input: grid = [[1,0],[0,1]]
 * Output: 0
 * Explanation: No servers can communicate with others.
 *
 * Example 2:
 * Input: grid = [[1,0],[1,1]]
 * Output: 3
 * Explanation: All three servers can communicate with at least one other server.
 *
 * Example 3:
 * Input: grid = [[1,1,0,0],[0,0,1,0],[0,0,1,0],[0,0,0,1]]
 * Output: 4
 * Explanation: The two servers in the first row can communicate with each other.
 * The two servers in the third column can communicate with each other. The server at
```

```
right bottom corner can't communicate with any other server.
 *
 * Constraints:
 * m == grid.length
 * n == grid[i].length
 * 1 <= m <= 250
 * 1 <= n <= 250
 * grid[i][j] == 0 or 1
 * */
public class _1267 {
    public static class Solution1 {
        /**credit: https://leetcode.com/problems/count-servers-that-
communicate/discuss/436188/Java-or-Clean-And-Simple-or-Beats-100*/
        public int countServers(int[][] grid) {
            int m = grid.length;
            int n = grid[0].length;
            int[] rowCount = new int[m];
            int[] columnCount = new int[n];
            int total = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1) {
                        rowCount[i]++;
                        columnCount[j]++;
                        total++;
                    }
                }
            }
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1 && rowCount[i] == 1 && columnCount[j] == 1) {
                        total--;
                    }
                }
            }
            return total;
        }
    }
}

// ============================================================
// FILE 817: _1271.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * 1271. Hexspeak
 *
 * A decimal number can be converted to its Hexspeak representation by first converting
```

```
it to an uppercase hexadecimal string,
 * then replacing all occurrences of the digit 0 with the letter O, and the digit 1 with
the letter I.
 * Such a representation is valid if and only if it consists only of the letters in the
set {"A", "B", "C", "D", "E", "F", "I", "O"}.
 * Given a string num representing a decimal integer N, return the Hexspeak
representation of N if it is valid, otherwise return "ERROR".
 *
 * Example 1:
 * Input: num = "257"
 * Output: "IOI"
 * Explanation:  257 is 101 in hexadecimal.
 *
 * Example 2:
 * Input: num = "3"
 * Output: "ERROR"
 *
 * Constraints:
 * 1 <= N <= 10^12
 * There are no leading zeros in the given string.
 * All answers must be in uppercase letters.
 * */
public class _1271 {
    public static class Solution1 {
        public String toHexspeak(String num) {
            long numInt = Long.parseLong(num);
            String hexString = Long.toHexString(numInt);
            StringBuilder sb = new StringBuilder();
            Set<Character> set = new HashSet<>(Arrays.asList('A', 'B', 'C', 'D', 'E',
'F', '1', '0', 'a', 'b', 'c', 'd', 'e', 'f'));
            for (char c : hexString.toCharArray()) {
                if (!set.contains(c)) {
                    return "ERROR";
                } else if (c == '1') {
                    sb.append("I");
                } else if (c == '0') {
                    sb.append("O");
                } else {
                    sb.append(Character.toUpperCase(c));
                }
            }
            return sb.toString();
        }
    }
}

// ============================================================
// FILE 818: _1277.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1277. Count Square Submatrices with All Ones
```

```
 *
 * Given a m * n matrix of ones and zeros, return how many square submatrices have all
ones.
 *
 * Example 1:
 * Input: matrix =
 * [
 *   [0,1,1,1],
 *   [1,1,1,1],
 *   [0,1,1,1]
 * ]
 * Output: 15
 * Explanation:
 * There are 10 squares of side 1.
 * There are 4 squares of side 2.
 * There is  1 square of side 3.
 * Total number of squares = 10 + 4 + 1 = 15.
 *
 * Example 2:
 * Input: matrix =
 * [
 *   [1,0,1],
 *   [1,1,0],
 *   [1,1,0]
 * ]
 * Output: 7
 * Explanation:
 * There are 6 squares of side 1.
 * There is 1 square of side 2.
 * Total number of squares = 6 + 1 = 7.
 *
 * Constraints:
 * 1 <= arr.length <= 300
 * 1 <= arr[0].length <= 300
 * 0 <= arr[i][j] <= 1
 * */
public class _1277 {
    public static class Solution1 {
        /**
         * In-place solution.
         *
         * credit: https://leetcode.com/problems/count-square-submatrices-with-all-
ones/discuss/441306/Python-DP-solution
         * */
        public int countSquares(int[][] matrix) {
            int m = matrix.length;
            int n = matrix[0].length;
            int count = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (matrix[i][j] > 0 && i > 0 && j > 0) {
                        matrix[i][j] = Math.min(matrix[i - 1][j - 1], Math.min(matrix[i -
1][j], matrix[i][j - 1])) + 1;
                    }
```

```
                    count += matrix[i][j];
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        /**
         * Use m*n extra space solution.
         *
         * credit: https://leetcode.com/problems/count-square-submatrices-with-all-
ones/discuss/441312/Java-Simple-DP-solution
         */
        public int countSquares(int[][] matrix) {
            int m = matrix.length;
            int n = matrix[0].length;
            int[][] dp = new int[m][n];
            for (int i = 0; i < m; i++) {
                if (matrix[i][0] == 1) {
                    dp[i][0] = 1;
                }
            }
            for (int j = 0; j < n; j++) {
                if (matrix[0][j] == 1) {
                    dp[0][j] = 1;
                }
            }
            for (int i = 1; i < m; i++) {
                for (int j = 1; j < n; j++) {
                    if (matrix[i][j] == 1) {
                        dp[i][j] = Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j],
dp[i][j - 1])) + 1;
                    }
                }
            }
            int count = 0;
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    count += dp[i][j];
                }
            }
            return count;
        }
    }
}

// ============================================================
// FILE 819: _1281.java
// ============================================================

package com.fishercoder.solutions;

/**
```

```
 * 1281. Subtract the Product and Sum of Digits of an Integer
 *
 * Given an integer number n, return the difference between the product of its digits and
the sum of its digits.
 *
 * Example 1:
 * Input: n = 234
 * Output: 15
 * Explanation:
 * Product of digits = 2 * 3 * 4 = 24
 * Sum of digits = 2 + 3 + 4 = 9
 * Result = 24 - 9 = 15
 *
 * Example 2:
 * Input: n = 4421
 * Output: 21
 * Explanation:
 * Product of digits = 4 * 4 * 2 * 1 = 32
 * Sum of digits = 4 + 4 + 2 + 1 = 11
 * Result = 32 - 11 = 21
 *
 * Constraints:
 * 1 <= n <= 10^5
 * */
public class _1281 {
    public static class Solution1 {
        public int subtractProductAndSum(int n) {
            int sum = 0;
            int product = 1;
            while (n > 0) {
                int digit = n % 10;
                n /= 10;
                sum += digit;
                product *= digit;
            }
            return product - sum;
        }
    }
}

// ============================================================
// FILE 820: _1282.java
// ============================================================

package com.fishercoder.solutions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * 1282. Group the People Given the Group Size They Belong To
 *
```

```
 * There are n people whose IDs go from 0 to n - 1 and each person belongs exactly to one group.
 * Given the array groupSizes of length n telling the group size each person belongs to, return the groups there are and the people's IDs each group includes.
 * You can return any solution in any order and the same applies for IDs.
 * Also, it is guaranteed that there exists at least one solution.
 *
 * Example 1:
 * Input: groupSizes = [3,3,3,3,3,1,3]
 * Output: [[5],[0,1,2],[3,4,6]]
 * Explanation:
 * Other possible solutions are [[2,1,6],[5],[0,4,3]] and [[5],[0,6,2],[4,3,1]].
 *
 * Example 2:
 * Input: groupSizes = [2,1,3,3,3,2]
 * Output: [[1],[0,5],[2,3,4]]
 *
 * Constraints:
 * groupSizes.length == n
 * 1 <= n <= 500
 * 1 <= groupSizes[i] <= n
 * */
public class _1282 {
    public static class Solution1 {
        public List<List<Integer>> groupThePeople(int[] groupSizes) {
            Map<Integer, List<Integer>> map = new HashMap<>();
            for (int i = 0; i < groupSizes.length; i++) {
                List<Integer> list = map.getOrDefault(groupSizes[i], new ArrayList<>());
                list.add(i);
                map.put(groupSizes[i], list);
            }
            List<List<Integer>> result = new ArrayList<>();
            for (int key : map.keySet()) {
                List<Integer> list = map.get(key);
                int i = 0;
                do {
                    result.add(list.subList(i, i + key));
                    i += key;
                } while (i + key <= list.size());
            }
            return result;
        }
    }
}

// ============================================================
// FILE 821: _1287.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1287. Element Appearing More Than 25% In Sorted Array
 *
```

```
 * Given an integer array sorted in non-decreasing order, there is exactly one integer in
the array that occurs more than 25% of the time.
 * Return that integer.
 *
 * Example 1:
 * Input: arr = [1,2,2,6,6,6,6,7,10]
 * Output: 6
 *
 * Constraints:
 *
 * 1 <= arr.length <= 10^4
 * 0 <= arr[i] <= 10^5
 * */
public class _1287 {
    public static class Solution1 {
        public int findSpecialInteger(int[] arr) {
            int quarter = arr.length / 4;
            for (int i = 0; i < arr.length - quarter; i++) {
                if (arr[i] == arr[i + quarter]) {
                    return arr[i];
                }
            }
            return -1;
        }
    }
}

// ============================================================
// FILE 822: _1290.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.ListNode;

/**
 * 1290. Convert Binary Number in a Linked List to Integer
 *
 * Given head which is a reference node to a singly-linked list.
 * The value of each node in the linked list is either 0 or 1.
 * The linked list holds the binary representation of a number.
 *
 * Return the decimal value of the number in the linked list.
 *
 * Example 1:
 * Input: head = [1,0,1]
 * Output: 5
 * Explanation: (101) in base 2 = (5) in base 10
 *
 * Example 2:
 * Input: head = [0]
 * Output: 0
 *
 * Example 3:
```

```
 * Input: head = [1]
 * Output: 1
 *
 * Example 4:
 * Input: head = [1,0,0,1,0,0,1,1,1,0,0,0,0,0,0]
 * Output: 18880
 *
 * Example 5:
 * Input: head = [0,0]
 * Output: 0
 *
 * Constraints:
 *
 * The Linked List is not empty.
 * Number of nodes will not exceed 30.
 * Each node's value is either 0 or 1.
 * */
public class _1290 {
    public static class Solution1 {
        public int getDecimalValue(ListNode head) {
            StringBuilder sb = new StringBuilder();
            while (head != null) {
                sb.append(head.val);
                head = head.next;
            }
            return Integer.parseInt(sb.toString(), 2);
        }
    }
}

// ============================================================
// FILE 823: _1295.java
// ============================================================

package com.fishercoder.solutions;

import java.util.Arrays;

/**
 * 1295. Find Numbers with Even Number of Digits
 *
 * Given an array nums of integers, return how many of them contain an even number of
digits.
 *
 * Example 1:
 * Input: nums = [12,345,2,6,7896]
 * Output: 2
 * Explanation:
 * 12 contains 2 digits (even number of digits).
 * 345 contains 3 digits (odd number of digits).
 * 2 contains 1 digit (odd number of digits).
 * 6 contains 1 digit (odd number of digits).
 * 7896 contains 4 digits (even number of digits).
 * Therefore only 12 and 7896 contain an even number of digits.
```

```
 *
 * Example 2:
 * Input: nums = [555,901,482,1771]
 * Output: 1
 * Explanation:
 * Only 1771 contains an even number of digits.
 *
 * Constraints:
 * 1 <= nums.length <= 500
 * 1 <= nums[i] <= 10^5
 * */
public class _1295 {
    public static class Solution1 {
        public int findNumbers(int[] nums) {
            int count = 0;
            for (int num : nums) {
                if (String.valueOf(num).length() % 2 == 0) {
                    count++;
                }
            }
            return count;
        }
    }

    public static class Solution2 {
        public int findNumbers(int[] nums) {
            return (int) Arrays.stream(nums).filter(num -> String.valueOf(num).length() %
2 == 0).count();
        }
    }
}

// ============================================================
// FILE 824: _1296.java
// ============================================================

package com.fishercoder.solutions;

import java.util.TreeMap;

/**
 * 1296. Divide Array in Sets of K Consecutive Numbers
 *
 * Given an array of integers nums and a positive integer k,
 * find whether it's possible to divide this array into sets of k consecutive numbers
 * Return True if its possible otherwise return False.
 *
 * Example 1:
 * Input: nums = [1,2,3,3,4,4,5,6], k = 4
 * Output: true
 * Explanation: Array can be divided into [1,2,3,4] and [3,4,5,6].
 *
 * Example 2:
 * Input: nums = [3,2,1,2,3,4,3,4,5,9,10,11], k = 3
```

```
 * Output: true
 * Explanation: Array can be divided into [1,2,3] , [2,3,4] , [3,4,5] and [9,10,11].
 *
 * Example 3:
 * Input: nums = [3,3,2,2,1,1], k = 3
 * Output: true
 *
 * Example 4:
 * Input: nums = [1,2,3,4], k = 3
 * Output: false
 * Explanation: Each array should be divided in subarrays of size 3.
 *
 * Constraints:
 * 1 <= nums.length <= 10^5
 * 1 <= nums[i] <= 10^9
 * 1 <= k <= nums.length
 * */
public class _1296 {
    public static class Solution1 {
        public boolean isPossibleDivide(int[] nums, int k) {
            if (nums.length % k != 0) {
                return false;
            }
            TreeMap<Integer, Integer> treeMap = new TreeMap();
            int min = nums[0];
            for (int num : nums) {
                treeMap.put(num, treeMap.getOrDefault(num, 0) + 1);
                min = Math.min(min, num);
            }
            while (!treeMap.isEmpty()) {
                if (!isConsecutiveK(treeMap, min, k)) {
                    return false;
                }
                min = findNextMin(treeMap);
                if (min == Integer.MIN_VALUE) {
                    break;
                }
            }
            return true;
        }

        private int findNextMin(TreeMap<Integer, Integer> treeMap) {
            return treeMap.isEmpty() ? Integer.MIN_VALUE :
treeMap.entrySet().iterator().next().getKey();
        }

        private boolean isConsecutiveK(TreeMap<Integer, Integer> treeMap, int min, int k)
{
            int count = treeMap.get(min);
            treeMap.remove(min);
            for (int i = 1; i < k; i++) {
                int key = min + i;
                if (!treeMap.containsKey(key) || treeMap.get(key) < count) {
                    return false;
```

```
                } else if (treeMap.get(key) > count) {
                    treeMap.put(key, treeMap.get(key) - count);
                } else {
                    treeMap.remove(key);
                }
            }
            return true;
        }
    }
}

// ============================================================
// FILE 825: _1297.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * 1297. Maximum Number of Occurrences of a Substring
 *
 * Given a string s, return the maximum number of ocurrences of any substring under the
following rules:
 *
 * The number of unique characters in the substring must be less than or equal to
maxLetters.
 * The substring size must be between minSize and maxSize inclusive.
 *
 * Example 1:
 * Input: s = "aababcaab", maxLetters = 2, minSize = 3, maxSize = 4
 * Output: 2
 * Explanation: Substring "aab" has 2 ocurrences in the original string.
 * It satisfies the conditions, 2 unique letters and size 3 (between minSize and
maxSize).
 *
 * Example 2:
 * Input: s = "aaaa", maxLetters = 1, minSize = 3, maxSize = 3
 * Output: 2
 * Explanation: Substring "aaa" occur 2 times in the string. It can overlap.
 *
 * Example 3:
 * Input: s = "aabcabcab", maxLetters = 2, minSize = 2, maxSize = 3
 * Output: 3
 *
 * Example 4:
 * Input: s = "abcde", maxLetters = 2, minSize = 3, maxSize = 3
 * Output: 0
 *
 * Constraints:
 * 1 <= s.length <= 10^5
```

```
 * 1 <= maxLetters <= 26
 * 1 <= minSize <= maxSize <= min(26, s.length)
 * s only contains lowercase English letters.
 * */
public class _1297 {
    public static class Solution1 {
        public int maxFreq(String s, int maxLetters, int minSize, int maxSize) {
            int maxFreq = 0;
            for (int size = minSize; size <= maxSize; size++) {
                maxFreq = Math.max(maxFreq, getMaxFreqWithThisSize(s, maxLetters, size));
            }
            return maxFreq;
        }

        private int getMaxFreqWithThisSize(String str, int maxLetters, int size) {
            Map<String, Integer> map = new HashMap<>();
            for (int i = 0; i <= str.length() - size; i++) {
                String substring = str.substring(i, i + size);
                Set<Character> set = new HashSet<>();
                for (int j = 0; j < substring.length(); j++) {
                    set.add(substring.charAt(j));
                    if (set.size() > maxLetters) {
                        break;
                    }
                }
                if (set.size() <= maxLetters) {
                    map.put(substring, map.getOrDefault(substring, 0) + 1);
                }
            }
            int max = 0;
            for (String key : map.keySet()) {
                max = Math.max(max, map.get(key));
            }
            return max;
        }
    }
}

// ============================================================
// FILE 826: _1299.java
// ============================================================

package com.fishercoder.solutions;

import java.util.PriorityQueue;

/**
 * 1299. Replace Elements with Greatest Element on Right Side
 *
 * Given an array arr, replace every element in that array with the greatest element
among the elements to its right, and replace the last element with -1.
 * After doing so, return the array.
 *
 * Example 1:
```

```
 * Input: arr = [17,18,5,4,6,1]
 * Output: [18,6,6,6,1,-1]
 * */
public class _1299 {
    public static class Solution1 {
        public int[] replaceElements(int[] arr) {
            PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> b - a);
            for (int num : arr) {
                heap.offer(num);
            }
            int[] result = new int[arr.length];
            for (int i = 0; i < arr.length - 1; i++) {
                heap.remove(arr[i]);
                result[i] = heap.peek();
            }
            result[arr.length - 1] = -1;
            return result;
        }
    }
}

// ============================================================
// FILE 827: _1300.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1300. Sum of Mutated Array Closest to Target
 *
 * Given an integer array arr and a target value target, return the integer value such
that when we change all the integers larger than value in the given array to be equal to
value,
 * the sum of the array gets as close as possible (in absolute difference) to target.
 * In case of a tie, return the minimum such integer.
 * Notice that the answer is not neccesarilly a number from arr.
 *
 * Example 1:
 * Input: arr = [4,9,3], target = 10
 * Output: 3
 * Explanation: When using 3 arr converts to [3, 3, 3] which sums 9 and that's the
optimal answer.
 *
 * Example 2:
 * Input: arr = [2,3,5], target = 10
 * Output: 5
 *
 * Example 3:
 * Input: arr = [60864,25176,27249,21296,20204], target = 56803
 * Output: 11361
 * */
public class _1300 {
    public static class Solution1 {
        public int findBestValue(int[] arr, int target) {
```

```
            int ave = target / arr.length;
            int min = findMin(arr);
            int max = findMax(arr);
            //if ave is the best value, what's the difference to target?
            int closetDiff = findClosestDiffIfReplaceWithVal(arr, ave, target);
            int bestValue = ave;
            //extend candidate towards the right to see how close the sum could be to
target
            int candidateOnTheRight = ave;
            while (candidateOnTheRight <= max) {
                int thisOne = findClosestDiffIfReplaceWithVal(arr, ++candidateOnTheRight,
target);
                if (thisOne >= closetDiff) {
                    break;
                } else {
                    closetDiff = thisOne;
                    bestValue = candidateOnTheRight;
                }
            }

            //extend candidate towards the left to see how close the sum could be to
target
            int candidateOnTheLeft = ave;
            while (candidateOnTheLeft >= min) {
                int thisOne = findClosestDiffIfReplaceWithVal(arr, --candidateOnTheLeft,
target);
                if (thisOne >= closetDiff) {
                    break;
                } else {
                    closetDiff = thisOne;
                    bestValue = candidateOnTheLeft;
                }
            }
            return bestValue;
        }

        private int findClosestDiffIfReplaceWithVal(int[] arr, int replaceValue, int
target) {
            int sum = 0;
            for (int i = 0; i < arr.length; i++) {
                if (arr[i] > replaceValue) {
                    sum += replaceValue;
                } else {
                    sum += arr[i];
                }
            }
            return Math.abs(sum - target);
        }

        private int findMax(int[] arr) {
            int max = arr[0];
            for (int i = 1; i < arr.length; i++) {
                max = Math.max(max, arr[i]);
            }
```

```
            return max;
        }

        private int findMin(int[] arr) {
            int min = arr[0];
            for (int i = 1; i < arr.length; i++) {
                min = Math.min(min, arr[i]);
            }
            return min;
        }
    }
}

// ============================================================
// FILE 828: _1302.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 1302. Deepest Leaves Sum
 *
 * Given a binary tree, return the sum of values of its deepest leaves.
 *
 * Example 1:
 *                1
 *               / \
 *              2   3
 *             / \   \
 *            4  5   6
 *           /        \
 *          7         8
 *
 * Input: root = [1,2,3,4,5,null,6,7,null,null,null,null,8]
 * Output: 15
 *
 * Constraints:
 * The number of nodes in the tree is between 1 and 10^4.
 * The value of nodes is between 1 and 100.
 * */
public class _1302 {
    public static class Solution1 {
        public int deepestLeavesSum(TreeNode root) {
            int depth = maxDepth(root);
            return bfs(root, depth);
        }

        private int bfs(TreeNode root, int depth) {
            Queue<TreeNode> queue = new LinkedList<>();
```

```
            queue.offer(root);
            int currentLevel = 0;
            int sum = 0;
            while (!queue.isEmpty()) {
                int size = queue.size();
                currentLevel++;
                for (int i = 0; i < size; i++) {
                    TreeNode currNode = queue.poll();
                    if (currentLevel == depth) {
                        sum += currNode.val;
                    }
                    if (currNode.left != null) {
                        queue.offer(currNode.left);
                    }
                    if (currNode.right != null) {
                        queue.offer(currNode.right);
                    }
                }
            }
            return sum;
        }

        private int maxDepth(TreeNode root) {
            if (root == null) {
                return 0;
            }
            return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
        }
    }
}

// ============================================================
// FILE 829: _1304.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1304. Find N Unique Integers Sum up to Zero
 *
 * Given an integer n, return any array containing n unique integers such that they add
up to 0.
 *
 * Example 1:
 * Input: n = 5
 * Output: [-7,-1,1,3,4]
 * Explanation: These arrays also are accepted [-5,-1,1,2,3] , [-3,-1,2,-2,4].
 *
 * Example 2:
 * Input: n = 3
 * Output: [-1,0,1]
 *
 * Example 3:
 * Input: n = 1
```

```
 * Output: [0]
 * */
public class _1304 {
    public static class Solution1 {
        public int[] sumZero(int n) {
            int[] result = new int[n];
            int start = -n / 2;
            for (int i = 0; i < n / 2; i++) {
                result[i] = start++;
            }
            if (n % 2 == 0) {
                start++;
            }
            for (int i = n / 2; i < n; i++) {
                result[i] = start++;
            }
            return result;
        }
    }
}

// ============================================================
// FILE 830: _1305.java
// ============================================================

package com.fishercoder.solutions;

import com.fishercoder.common.classes.TreeNode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * 1305. All Elements in Two Binary Search Trees
 *
 * Given two binary search trees root1 and root2.
 * Return a list containing all the integers from both trees sorted in ascending order.
 *
 * Example 1:
 * Input: root1 = [2,1,4], root2 = [1,0,3]
 * Output: [0,1,1,2,3,4]
 *
 * Example 2:
 * Input: root1 = [0,-10,10], root2 = [5,1,7,0,2]
 * Output: [-10,0,0,1,2,5,7,10]
 *
 * Example 3:
 * Input: root1 = [], root2 = [5,1,7,0,2]
 * Output: [0,1,2,5,7]
 *
 * Example 4:
 * Input: root1 = [0,-10,10], root2 = []
 * Output: [-10,0,10]
```

```
 *
 * Example 5:
 * Input: root1 = [1,null,8], root2 = [8,1]
 * Output: [1,1,8,8]
 *
 * Constraints:
 * Each tree has at most 5000 nodes.
 * Each node's value is between [-10^5, 10^5].*/
public class _1305 {
    public static class Solution1 {
        public List<Integer> getAllElements(TreeNode root1, TreeNode root2) {
            List<Integer> list1 = getAllNodes(root1);
            List<Integer> list2 = getAllNodes(root2);
            List<Integer> merged = new ArrayList<>();
            merged.addAll(list1);
            merged.addAll(list2);
            Collections.sort(merged);
            return merged;
        }

        private List<Integer> getAllNodes(TreeNode root) {
            List<Integer> list = new ArrayList<>();
            return inorder(root, list);
        }

        List<Integer> inorder(TreeNode root, List<Integer> result) {
            if (root == null) {
                return result;
            }
            inorder(root.left, result);
            result.add(root.val);
            return inorder(root.right, result);
        }
    }
}

// ============================================================
// FILE 831: _1309.java
// ============================================================

package com.fishercoder.solutions;

import java.util.HashMap;
import java.util.Map;

/**
 * 1309. Decrypt String from Alphabet to Integer Mapping
 *
 * Given a string s formed by digits ('0' - '9') and '#' . We want to map s to English
lowercase characters as follows:
 * Characters ('a' to 'i') are represented by ('1' to '9') respectively.
 * Characters ('j' to 'z') are represented by ('10#' to '26#') respectively.
 * Return the string formed after mapping.
 * It's guaranteed that a unique mapping will always exist.
```

```
 *
 * Example 1:
 * Input: s = "10#11#12"
 * Output: "jkab"
 * Explanation: "j" -> "10#" , "k" -> "11#" , "a" -> "1" , "b" -> "2".
 *
 * Example 2:
 * Input: s = "1326#"
 * Output: "acz"
 *
 * Example 3:
 * Input: s = "25#"
 * Output: "y"
 *
 * Example 4:
 * Input: s = "12345678910#11#12#13#14#15#16#17#18#19#20#21#22#23#24#25#26#"
 * Output: "abcdefghijklmnopqrstuvwxyz"
 *
 * Constraints:
 * 1 <= s.length <= 1000
 * s[i] only contains digits letters ('0'-'9') and '#' letter.
 * s will be valid string such that mapping is always possible.
 * */
public class _1309 {
    public static class Solution1 {
        //TODO: very silly solution, optimze it
        public String freqAlphabets(String s) {
            Map<String, String> map = new HashMap<>();
            map.put("1", "a");
            map.put("2", "b");
            map.put("3", "c");
            map.put("4", "d");
            map.put("5", "e");
            map.put("6", "f");
            map.put("7", "g");
            map.put("8", "h");
            map.put("9", "i");
            map.put("10#", "j");
            map.put("11#", "k");
            map.put("12#", "l");
            map.put("13#", "m");
            map.put("14#", "n");
            map.put("15#", "o");
            map.put("16#", "p");
            map.put("17#", "q");
            map.put("18#", "r");
            map.put("19#", "s");
            map.put("20#", "t");
            map.put("21#", "u");
            map.put("22#", "v");
            map.put("23#", "w");
            map.put("24#", "x");
            map.put("25#", "y");
            map.put("26#", "z");
```

```
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < s.length(); ) {
                if (Integer.parseInt("" + s.charAt(i)) == 1 && i + 1 < s.length() && i +
2 < s.length() && s.charAt(i + 2) == '#') {
                    sb.append(map.get(s.substring(i, i + 3)));
                    i += 3;
                } else if (Integer.parseInt("" + s.charAt(i)) == 2 && i + 1 < s.length()
&& i + 2 < s.length() && s.charAt(i + 2) == '#') {
                    sb.append(map.get(s.substring(i, i + 3)));
                    i += 3;
                } else {
                    sb.append(map.get("" + s.charAt(i)));
                    i++;
                }
            }
            return sb.toString();
        }
    }
}

// ============================================================
// FILE 832: _1317.java
// ============================================================

package com.fishercoder.solutions;

/**
 * 1317. Convert Integer to the Sum of Two No-Zero Integers
 *
 * Given an integer n. No-Zero integer is a positive integer which doesn't contain any 0
in its decimal representation.
 *
 * Return a list of two integers [A, B] where:
 * A and B are No-Zero integers.
 * A + B = n
 * It's guarateed that there is at least one valid solution. If there are many valid
solutions you can return any of them.
 *
 * Example 1:
 * Input: n = 2
 * Output: [1,1]
 * Explanation: A = 1, B = 1. A + B = n and both A and B don't contain any 0 in their
decimal representation.
 *
 * Example 2:
 * Input: n = 11
 * Output: [2,9]
 *
 * Example 3:
 * Input: n = 10000
 * Output: [1,9999]
 *
 * Example 4:
 * Input: n = 69
```

```
 * Output: [1,68]
 *
 * Example 5:
 * Input: n = 1010
 * Output: [11,999]
 *
 * Constraints:
 * 2 <= n <= 10^4
 * */
public class _1317 {
    public static class Solution1 {
        public int[] getNoZeroIntegers(int n) {
            int left = 1;
            int right = n - 1;
            while (left <= right) {
                if (noZero(left) && noZero(right)) {
                    return new int[]{left, right};
                } else {
                    left++;
                    right--;
                }
            }
            return null;
        }

        private boolean noZero(int num) {
            while (num != 0) {
                if (num % 10 == 0) {
                    return false;
                } else {
                    num /= 10;
                }
            }
            return true;
        }
    }
}
```